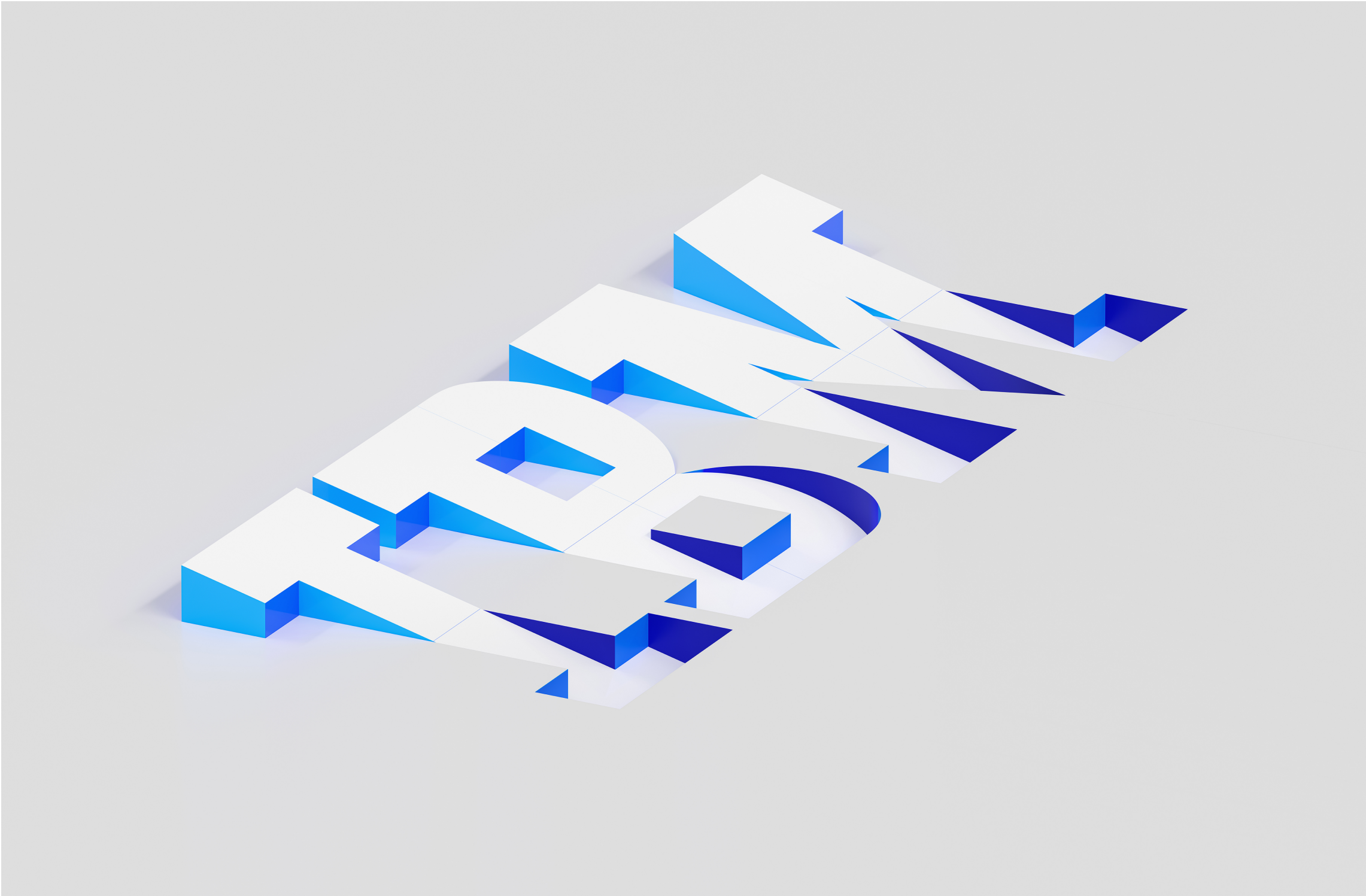


Cloud Internet Services (CIS)

Product guide



Edition notices

This PDF was created on 2026-10-01 as a supplement to *Cloud Internet Services (CIS)* in the IBM Cloud docs. It might not be a complete set of information or the latest version. For the latest information, see the IBM Cloud documentation at <https://cloud.ibm.com/docs/cis>.

Getting started with IBM Cloud Internet Services

IBM Cloud® Internet Services (CIS) is a cloud-native solution that helps protect and optimize your internet-facing applications. Powered by Cloudflare’s global network, CIS acts as a reverse proxy to help secure traffic, improve performance, and maintain availability.

CIS delivers three essential benefits:

- Security – Safeguards your applications from DDoS attacks, malicious bots, and other web threats using features like a Web Application Firewall (WAF), rate limiting, and SSL/TLS encryption.
- Reliability – Helps ensure uptime and resilience through global load balancing, automatic failover, and robust DNS services.
- Performance – Accelerates content delivery with a global Content Delivery Network (CDN), intelligent caching, and optimized protocols.

After you provision a CIS instance on IBM Cloud, you can access and configure these features.

Before you begin

Before you begin using CIS, make sure that you have met the following requirements:

- You need an IBMid to sign in to IBM Cloud and manage CIS. If you don’t have one, you can [create an account](#) to access trials, demos, and services.
- You must have a domain that is registered with a domain registrar. CIS protects and accelerates traffic for this domain.
- Decide whether you want to protect your entire domain ([full setup](#)) or specific subdomains ([partial setup](#)). This decision affects how you’ll configure DNS and what level of protection and optimization you get from CIS.
- [Compare and select a CIS plan](#) that best meets your needs (Lite, Standard, or Enterprise). Each plan offers different levels of security, performance, and feature access.
- Familiarize yourself with [CIS core concepts](#) and terminology to better understand the configuration steps.



Tip: Use Google Chrome when you work with CIS.

Getting started process

This section outlines high-level steps to onboard your domain and activate CIS features.

Updates to your DNS delegation, name servers, or CNAME records can take up to 24–48 hours to propagate globally.

1. [Create a CIS instance](#) in your IBM Cloud account. The CIS instance acts as the central place to manage your domain's security, DNS, and performance settings.
2. Add your domain to your CIS instance. CIS supports multiple ways to onboard a domain, depending on your DNS setup needs. Choose the method that best fits your setup:
 - [Configure a full setup \(authoritative DNS\)](#): CIS becomes your domain’s DNS provider. You update your domain registrar’s name servers to point to CIS. This gives you full control and access to all CIS features.
 - [Configure a partial setup \(CNAME-based\)](#): You keep your current DNS provider and only route specific subdomains (for example, `www.techcorp.com`) through CIS by configuring CNAME records. Ideal if you can’t or prefer not to change your domain’s name servers.
3. [Import or add your DNS records](#). Before updating DNS delegation or CNAME records, make sure that all required DNS records exist in CIS. This step ensures that CIS can immediately resolve traffic when delegation is complete.

You can import existing DNS records or add them manually. Make sure that required records such as A, AAAA, MX, and CNAME records are present.

4. Configure DNS delegation based on the setup type that you selected. After DNS records are in place, update your registrar or DNS provider to route traffic through CIS.

For a full setup:

- a. Sign in to your domain registrar or DNS host and locate the **Name Server (NS) settings**.
- b. Update the NS records to point to CIS name servers (for full setup) or your existing DNS provider (for partial setup with CNAMEs).
- c. Save your changes.

For a partial setup:

- a. Sign in to your DNS provider and locate the DNS record settings.
- b. Configure the required CNAME records to proxy specific subdomains through CIS.
- c. Save your changes.

5. Verify DNS delegation and domain resolution. After propagation begins, confirm that your domain is delegated correctly and that CIS is resolving traffic as expected.

Use a DNS lookup tool, such as `dig` or `nslookup` to verify authoritative name servers and record resolution. For example:

```
$ dig NS example.com
```

6. [Configure your name servers with the registrar or existing DNS provider.](#)

- If you're using a full setup, update your domain registrar's name servers to the ones provided by CIS. This change routes all DNS queries through CIS so that its security and performance features can take effect.
- If you're using a partial setup, configure the necessary CNAME records at your DNS provider to proxy specific subdomains through CIS.

7. Confirm that CIS is resolving the domain information for your application, hostname, or website.

To proceed, select **Reliability > DNS**. Be sure to add the appropriate DNS records. Add the A Record and any AAAA or MX entries that are populated. If you forget to add these records before the registrar's delegation is complete, CIS can't resolve the domain information for your internet-facing applications.

8. [Review CIS best practices](#) and [explore the CIS console](#) to familiarize yourself with domain management, DNS configuration, traffic analytics, and security settings.
9. Enable and configure additional features. After your domain is active, explore the full range of CIS capabilities, such as DDoS protection, rate limiting and bot management, SSL/TLS settings, page rules, and real-time traffic analytics.

Concepts

Caching

Caching is the process of storing files on edge servers to improve the response time when serving those files to customers. By storing files closer to users, caching reduces the time it takes for data to travel across the network, commonly known as latency.

By default, CIS caches static files, which include many types of image and text files (non-HTML). The cache includes files only from your websites and doesn't cover third-party resources, such as those from social networking sites. Currently, CIS does not cache based on MIME type.



Note: CIS doesn't cache HTML files by default because they are generally considered dynamic. However, if static HTML can be clearly distinguished from dynamic HTML, it is possible to cache HTML files by using page rules.

Cached files have a specified expiration time, called **Time-to-Live (TTL)**, after which they are purged from the cache. You can also manually purge files from the cache at any time. After files are purged, CIS goes back to your origin server to reload the files and update the cache with the most recent versions.

For detailed information on cache settings and options, see [Using page rules with caching](#).

Default caching behavior

CIS caches static content based on your visitors' locations, the CIS data center they connect to, and how frequently the resource is requested at that data center.

Caching occurs only within the CIS data center that serves the request. The following resources aren't cached by CIS:

- Off-site or third-party resources
- Content hosted on DNS records that aren't proxied

By default, CIS respects the origin web server's cache headers unless an edge cache TTL page rule overrides them:

- CIS doesn't cache resources if the `cache-control` header is set to `private`, `no-store`, `no-cache`, or `max-age=0`, or if a cookie is present in the response.
- CIS caches resources if `cache-control` is `public` with a `max-age` greater than 0, or if the expires header specifies a future date.
- When both `max-age` and `expires` headers are set, `max-age` takes precedence.

When both the `cache-control` directive and the `expires` headers aren't present, CIS caches certain HTTP response codes with the following default edge cache TTL:

Response code	TTL
200, 206, 301	120 m
302, 303	20 m
404, 410	3 m
403	1 m

Default cache response codes

CIS provides several options to customize caching:

- Set caching behavior for individual URLs by using CIS page rules.
- Use CIS [edge functions](#) or the [API](#) to customize caching.
- Adjust caching levels, TTL, and more through the [CIS CLI](#).



Tip: The maximum file size that CIS caches is 512 MB for Trial and Standard Next customers, and 5 GB for Enterprise customers. Enterprise customers can request caching of larger files by [opening a support case](#).

File extensions cached by default

CIS uses file extensions to cache content. The following file extensions are cached automatically:

File extensions

bmp	gif	pict	tif
class	ico	pls	tiff
css	jar	png	ttf
csv	jpg	ppt	webp
doc	js	pptx	woff
docx	mid	ps	woff2
ejs	midi	svg	xls
eot	otf	svgz	xlsx
eps	pdf	swf	

File extensions cached by default

CIS doesn't cache content based on MIME type, and by default, it doesn't cache HTML pages. However, it does automatically cache your website's `robots.txt` file. To cache extra content, such as HTML pages, you can set up custom page rules to control what gets cached.

Serving stale content

Serve stale content keeps a limited version of the site online if the server goes down. Even if the content expires, CIS continues serving cached content to users when origin servers are offline.

Understanding CIS cache responses

The output of the `CF-Cache-Status` header shows whether a resource is cached.

Response code	Definition
HIT	The resource was found in the CIS cache.
MISS	The resource wasn't found in the CIS cache and was served from the origin web server.
EXPIRED	The resource was found in the cache but has since expired and was served from the origin web server.
STALE	The resource was served from cache, but is expired. CIS wasn't able to contact the origin to retrieve the updated resource.
BYPASS	The origin server instructed CIS to bypass the cache by using a <code>cache-control</code> header set to <code>no-cache</code> , <code>private</code> , or <code>max-age=0</code> . BYPASS is returned when you enable origin <code>cache-control</code> . CIS also sets BYPASS when your origin web server sends cookies in the response header.
REVALIDATED	The resource is served from cache, but is stale. The resource was revalidated by either an <code>If-Modified-Since</code> header or an <code>If-None-Match</code> header.
UPDATING	The resource was served from the cache, but is expired. The resource is being updated by the origin web server. UPDATING is typically seen only for popular cached resources.
DYNAMIC	The resource wasn't cached by default and your current CIS caching configuration doesn't instruct CIS to cache the resource. Instead, the resource was requested from the origin web server. Use page rules to implement custom caching options.

Cache response codes and definitions

Using query string sorting

Enterprise Only: CIS treats URLs with query strings in different orders as separate files in the cache. For example, if one user requests:

`/video/123456?title=0&byline=0&portrait=0&color=987654`

and another requests:

`/video/123456?byline=0&color=987654&portrait=0&title=0`

CIS fetches the file from the origin again, even if it's already cached.

The Query String Sort feature sorts query strings "before" caching, increasing cache hit rates. You can enable Query String Sort by using the toggle on the

Caching page.

Compression and optimization

IBM Cloud® Internet Services uses GZIP and Brotli compression for various content types to improve page load times. Compression is applied based on the browser’s `User-Agent` .

If you already use GZIP, CIS accepts your settings as long as the appropriate headers are passed in the server response.

CIS supports GZIP only when it communicates with your origin server, and can deliver content that is:

- Compressed by using GZIP or Brotli
- Uncompressed

CIS’s reverse proxy can convert between compressed and uncompressed formats. For example, it can fetch GZIP-compressed content from your origin and serve it uncompressed to clients. This process occurs independently of caching.



Note: CIS removes the `Accept-Encoding` header and ignores its value.

Compressed content types

In addition to serving stale content and minifying CSS, JS, and HTML, CIS compresses responses with GZIP or Brotli for compatible browsers. CIS returns GZIP or Brotli-encoded responses to compatible clients and browsers for the following content types:

```
text/html
text/richtext
text/plain
text/css
text/x-script
text/x-component
text/x-java-source
text/x-markdown
application/javascript
application/x-javascript
text/javascript
text/js
image/x-icon
image/vnd.microsoft.icon
application/x-perl
application/x-httpd-cgi
text/xml
application/xml
application/xml+rss
application/vnd.api+json
application/x-protobuf
application/json
multipart/bag
multipart/mixed
application/xhtml+xml
font/ttf
font/otf
font/x-woff
image/svg+xml
application/vnd.ms-fontobject
application/ttf
application/x-ttf
application/otf
application/x-otf
application/truetype
application/opentype
application/x-opentype
application/font-woff
application/eot
application/font
application/font-sfnt
application/wasm
application/javascript-binast
application/manifest+json
```

 **Note:** If you don't want a particular response from your origin to be encoded, you can set **cache-control: no-transform** to disable encoding at your origin web server.

Enabling Brotli compression

To enable Brotli compression, follow these steps:

1. Log in to your CIS account.
2. Choose the appropriate domain.
3. Click the **Performance** tab.
4. Click the **Advanced** tab.
5. Toggle the Brotli switch to **On**.

Minification for HTML, JavaScript, and CSS

CIS uses **Minify web content** to remove all unnecessary characters from HTML, JavaScript and CSS files, without affecting their functionality.

- CSS and JavaScript minification operates on cached CSS and JS files only.
- After CIS returns a cache HIT for the file, it is returned to browsers in minified form, which allows CIS to deliver a complete minification result.
- To apply minification changes, purge your CIS cache.

Image size optimization compression

CIS optimizes images that are cached in its edge network by stripping metadata and applying either lossy or lossless compression to speed up delivery. Off-site images are not optimized.

 **Note:** Image size optimization compression is only available as a page rule in the console. The domain level setting is available from the CLI.

Image size optimization compression options

The following sections detail the image size optimization compression options.

Lossless

- Strips most metadata (for example, EXIF data).
- Retains all visual detail.
- Decompressed output matches the original image.

Lossy

- Strips metadata and reduces image size by approximately 15%.
- Some original data is permanently lost.
- For PNGs, the results are effectively the same as lossless.

Lossless and Lossy modes attempt to remove as much metadata as possible, but complete removal might not be possible due to factors like caching.

WebP

- WebP provides superior compression:
 - Approximately 26% smaller than PNG (lossless)
 - 25–34% smaller than JPEG (lossy)
- CIS creates and caches WebP versions.
- WebP is served only if:
 - The browser's **Accept** header includes WebP.
 - The compressed image is significantly smaller than the lossy or lossless version.

Supported **Accept** header example:

```
$ Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
```

Currently, WebP is only supported in Firefox, Google Chrome, and Opera browsers.



Important: Disable WebP at your origin to avoid caching issues with browsers that don't support WebP.

Verifying that image size optimization is active

Activate image size optimization by using a [page rule](#) or [CLI domain settings](#).

CIS adds the following headers to optimized images:

```
cf-bgj: imgq:85

cf-polished: qual=85, origFmt=jpeg, origSize=95005

cf-cache-status: HIT
```

The `cf-bgj` header confirms that CIS applied image size optimization and returns the image quality setting (`imgq`). The `cf-polished` header represents the image size optimization status and returns the image quality setting (`qual`) and original size (`origSize`). The `cf-cache-status` confirms that the image was cached and image size optimization can be applied.

WebP conversion doesn't change the image URL. The `Content-Type HTTP` header tells the browser the original format of an image.

Common Cf-Polished statuses

The following table lists common Cf-Polished statuses and how to troubleshoot them. If no Cf-Polished header is returned, try to use single-file cache purge to purge the image.

Status	Definition	Recommendation
<code>input_too_large</code>	Image is too large or complex to process and needs a lower resolution.	Use PNG or JPEG images that are less than 1,000 px and 10 MB.
<code>not_compressed</code> or <code>not_needed</code>	Image was fully optimized at the origin server and no compression was applied.	
<code>webp_bigger</code>	Polish attempted to convert to WebP, but the image was either optimized at the origin server, created with a low-quality setting, or both.	Because the WebP version doesn't exist, the status is set on the PNG or JPEG version of the response.
<code>cannot_optimize</code> or <code>internal_error</code>	Image is corrupted or incomplete at the origin server.	Upload a new version of the image to the origin server.
<code>format_not_supported</code>	Image format isn't supported (for example, BMP, TIFF), the origin server is using extra optimization software that is not compatible with Polish, or both.	Try converting the input image to a web-compatible format (for example, PNG, JPEG), disabling any extra optimization software at the origin server, or both.
<code>vary_header_present</code>	The origin web server sent a <code>Vary</code> header with a value other than <code>accept-encoding</code> .	If the origin web server is attempting to support WebP, disable WebP at the origin web server and let Polish perform the WebP conversion.

Common Cf-Polished statuses



Note: Optimization still works when `accept-encoding` isn't the only value in the `Vary` header.

DDoS attack concepts

Distributed Denial of Service (DDoS) attacks are among the most common types of internet attacks that your website or host can encounter.

A distributed denial of service (DDoS) attack is a malicious attempt to disrupt normal traffic of a server, service, or network by overwhelming the target or its surrounding infrastructure with a flood of internet traffic. DDoS attacks achieve effectiveness by utilizing many compromised computer systems as sources of attack traffic. Exploited machines can include computers and other networked resources such as IoT devices. From a high level, a DDoS attack is like a traffic jam that is clogging up a highway, preventing regular traffic from arriving at its destination.

How attackers generate DDoS traffic

An attacker gains control of a network of online machines to carry out a DDoS attack. Computers and other machines (such as IoT devices) are infected with malware, turning each one into a bot (or zombie). The attacker controls the group of bots, which is called a *botnet*.

After establishing a botnet, the attacker directs the machines by sending updated instructions to each bot using remote control. A targeted IP address can receive requests from a multitude of bots, causing the targeted server or network to overflow capacity. This overflow creates a denial-of-service to normal traffic. Because each bot is a legitimate internet device, separating the attack traffic from normal traffic can be difficult.

Common types of DDoS attacks

DDoS attack vectors target varying components of a network connection. While nearly all DDoS attacks involve overwhelming a target device or network with traffic, attacks can be divided into three categories. An attacker can use one or multiple attack vectors, and might even cycle through these attack vectors based on the countermeasures that are taken by the target.

Common types are:

- Application layer attacks (Layer 7)
- Protocol attacks (Layer 3 and Layer 4)
- Volumetric attacks (amplification attacks)

Application layer attacks

An application layer attack is sometimes referred to as a *Layer-7 DDoS attack* (in reference to the 7th layer of the OSI model). The goal of these attacks is to exhaust the resources of the victim, by targeting the layer where web pages are generated on the server and delivered to the visitors in response to HTTP requests (the application layer). Layer-7 attacks are challenging, because the traffic can be difficult to identify as malicious.

Protocol attacks

Protocol attacks use weaknesses in Layer 3 and Layer 4 of the ISO protocol stack to render the target inaccessible. These attacks, also known as a state-exhaustion attacks, cause a service disruption by consuming all the available *state table* capacity of web application servers, or of intermediate resources such as firewalls and load balancers.

Volumetric attacks

This category of attack attempts to create congestion by consuming all available bandwidth between the target and the wider internet. Large amounts of data are sent to a target by using a form of amplification, or by other means of creating massive traffic, such as requests from a botnet.

How CIS mitigates network-level and application-level attacks

Generally, attacks fall into two categories:

Layer-3 or Layer-4 attacks	Layer-7 attacks
These attacks consist of a flood of traffic at ISO Layer 3 (the network layer), such as ICMP floods, or at Layer 4 (the transport layer), such as TCP SYN floods or reflected UDP floods.	These attacks send malicious ISO Layer-7 requests (the application layer), such as GET floods.
Automatically blocked at CIS edge	CIS handles these attacks with Defense mode, WAF, and security-level settings.

Types of network attacks

What do I do if I’m under a DDoS attack?

1. Turn on "Defense mode" from the **Overview** page.
2. Set your DNS records for maximum security.
3. Do not rate-limit or throttle requests from IBM CIS. The bandwidth is required to assist with your situation.
4. Block specific countries and visitors if necessary.

Related links

- [About DDoS protection in CIS](#)
- [Preventing DDoS attacks](#)

- [Responding to DDoS attacks](#)
- [Using Defense mode for DDoS attacks](#)
- [Third-party services and DDoS protection](#)
- [HTTP DDoS Attack Protection managed ruleset](#)

Domain lifecycle and state transitions

Domains that are configured in CIS begin in **Pending** state, and become **Active** after the domain's authoritative name servers are configured correctly at the DNS provider or registrar.

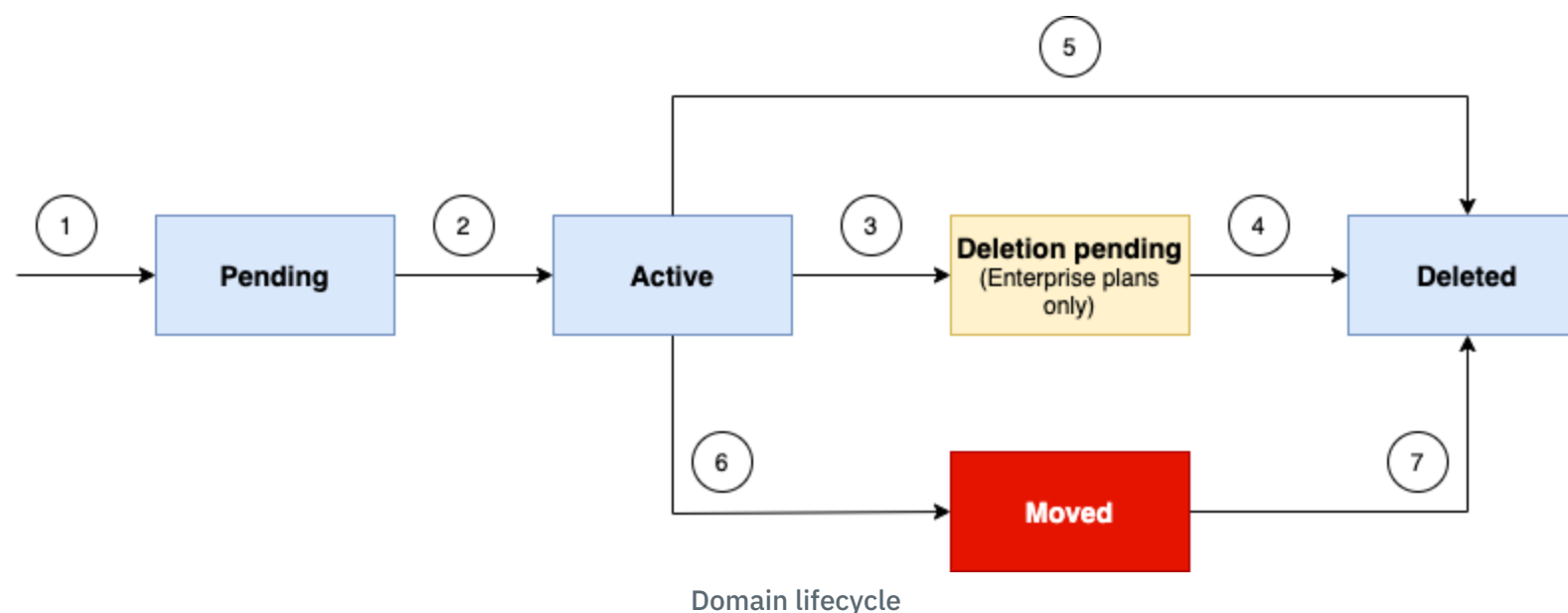
The following diagram shows a sample domain lifecycle, with labels that correspond to the list of domain state transitions:

1. Adding a domain to a CIS instance puts the domain in **Pending** state. Authoritative name servers are assigned but not yet configured at the DNS provider or registrar.
2. After the authoritative name server records are configured correctly at the DNS provider or registrar, the domain becomes **Active**.
3. (Enterprise only): Deleting the domain sets its status as **Deletion pending**. For Enterprise plan domains, deletion is not immediate and can take up to 24 hours.
4. (Enterprise only): After the deletion period, the domain is fully deleted.
5. (Standard): A Standard plan level instance can be deleted immediately.
6. The domain's name server configuration is regularly monitored. If the authoritative name server's settings change at the DNS provider or registrar, the domain enters the **Moved** state. This condition might occur when moving the domain to a different CIS instance, or when the name server configuration is incorrect.



Important: A domain in the **Moved** state might stop serving traffic and can be deleted. If you believe that your domain is incorrectly marked as **Moved**, [contact IBM Support](#).

7. Domains in the **Moved** state might be deleted automatically or can be deleted by the user.



Domain lifecycle states

You might encounter the following domain lifecycle states:

- **Pending** - The zone was added to a CIS instance, and the DNS configuration is being verified at the DNS provider or registrar to activate the domain. After the correct name servers (or the CNAME validation records) are confirmed, the domain changes to **Active**. Otherwise, it remains **Pending**.
- **Active** - The DNS configuration was successfully verified and the domain is actively serving traffic.
- **Moved** - The zone was previously **Active**, but DNS validation detected that the domain is no longer configured for the assigned authoritative name servers only, for a number of repeated checks. This state indicates the start of zone deprovisioning. The domain stays in **Moved** state for 7 days, or indefinitely if an active subscription on the zone exists. If the same domain becomes **Active** or **Pending** in a different account, this domain no longer serves traffic because **Pending** or **Active** zones take precedence.
- **Deletion pending** - (Enterprise only) The customer initiated the deletion of this domain from the CIS instance. Enterprise domains can take up to 24 hours to be removed.
- **Deleted** - This state indicates that the domain was removed from a customer's dashboard and its services are deprovisioned, but not removed from the Cloudflare database.

Domain settings

Domain settings define how a domain is secured, delivered, and managed at the edge. When you onboard a domain and delegate traffic to CIS name servers, inbound requests are processed at the CIS edge before being forwarded to the origin.

Domain settings establish the policies that protect applications from threats, ensure high availability through resilient traffic routing, and improve performance through edge optimization. Each domain operates as an independent configuration boundary, enabling tailored security, reliability, and performance controls for different applications or workloads.

What domain settings control

Domain settings determine how your domain operates across DNS, security, performance, and traffic management.

- **DNS behavior:** Controls how DNS records resolve and whether traffic is routed directly to the origin or proxied through the edge network.
- **TLS and HTTPS configuration:** Defines how certificates are managed, which TLS versions are allowed, and whether HTTPS is enforced for incoming requests.
- **Security features:** Governs how traffic is inspected and mitigated by security capabilities such as the Web Application Firewall (WAF), DDoS protection, bot management, and configurable security levels.
- **Performance optimization:** Specifies caching, compressed, and protocol support (for example, `HTTP/2` or `HTTP/3`).
- **Traffic handling:** Determines how requests are redirected, rewritten, or processed before reaching the origin server.

Why configure domain settings

Default settings provide baseline security and performance. However, most applications require customization to meet specific technical, security, or compliance requirements.

Configure domain settings to:

- Enforce security policies.
- Optimize performance for static and dynamic content.
- Control how traffic is cached and routed.
- Meet TLS and encryption standards.
- Reduce load on origin infrastructure.

Proper configuration ensures that your domain operates securely, efficiently, and in alignment with operational requirements.

Advanced security

Domain settings provide the configuration framework through which advanced security features are enabled, managed, and enforced for each domain. These settings include the following features, which you can change, enable, or disable.

- **Browser integrity check** - The browser integrity check looks for HTTP headers that are commonly abused by spammers. It denies traffic with those headers access to your page. It also blocks or challenges visitors that do not have a user agent, or who add a nonstandard user agent. This tactic is commonly used by abuse bots, crawlers, or APIs.
- **Challenge passage** - Controls how long a visitor that passed a challenge (or JavaScript challenge) gains access to your site before they are challenged again. This challenge is based on the visitor's IP, and therefore does not apply to challenges presented by WAF rules because they are based on an action that the user performs on your site.
- **Security level** - Sets the security level of your website to determine which visitors receive a challenge page.
- **Always use HTTPS** - Redirects all visitors to the HTTPS version.
- **Email obfuscation** - Prevents spam from harvesters and bots that try to access email addresses on your pages.
- **Automatic HTTPS rewrites** - Helps fix mixed content by changing `http` to `https` for all resources (or links) on your website that can be served with HTTPS.
- **Opportunistic encryption** - Allows browsers to benefit from the improved performance of HTTP/2 by informing them that your site is available over an encrypted connection.
- **Universal SSL** - Activates universal SSL certificates from your zone to the edge.
- **True client IP header** - Sends the user's IP address in the True-Client-IP header.

DNS behavior and resolution logic

The Domain Name System (DNS) underpins the web, working transparently in the background to convert human-readable website names into computer-readable numerical IP addresses. These addresses follow the [internet's RFC 1918 guidelines for IPv4 and RFC 4193 for IPv6](#). In short, DNS servers

match domain names, such as `ibm.com`, to their corresponding IP addresses, which most people never need to know.

To perform this translation, the DNS system queries a network of interconnected DNS servers across the internet. This process is similar to using a phone book or a map to find a specific location.

Name servers

A name server provides services that respond to queries against a directory, translating meaningful, text-based web or hostnames into IP addresses.

Name server delegation occurs when a domain's name server gets a request for a subdomain's records and responds by referring the requester to the delegated name server that manages the subdomain. This process allows decentralized management of large domains, such as `ibm.com`.

With a custom domain name server, you can use the DNS provider's servers with the customized reference name of your own domain. For example, you can define your name server as `ns1.cloud.ibm.com` instead of the provider's default like `ns1.acme.com`.

Proxying DNS records and global load balancers

CIS supports proxying for global load balancers and DNS records. When a record or load balancer is proxied, its traffic runs directly through CIS.



Note: Currently, DNS records of type **A**, **AAAA**, or **CNAME** can be proxied. For more information, see [DNS record types](#).

Setting proxy modes

Load balancers and DNS records support both DNS-only and HTTP proxy modes. You can have HTTP proxy and DNS-only domains in the same CIS instance, but the traffic routing behavior differs: traffic for records that are proxied flows through CIS, while traffic for records that are nonproxied (DNS-only mode) flows directly from the client to the origin.

HTTP proxy mode

In HTTP proxy mode, CIS announces IBM IP addresses externally, but protects (masks) your origin server IP addresses. The announced IP address records have an automatic TTL. Traffic flows through CIS, where all the security, performance, and reliability features, such as firewall rules and caching, are applied. The "automatic" TTL (five minutes) also reduces the number of authoritative queries that are made against CIS.

DNS-only mode

In DNS-only mode, the records are resolved to the origin IP, and you can customize the TTL for your records. For global load balancers, CIS serves the addresses of the healthy origin servers directly, but relies on DNS resolvers, which respect the short TTL, to requery CIS DNS for an updated list of healthy addresses.



Important: In DNS-only mode, none of the CIS security, reliability, and performance features are applied.

Root record CNAME flattening

The "CNAME flattening" feature in CIS makes it possible for the root records to bypass the IETF RFC restriction. This restriction states that if a root record is a CNAME, it can't have any other records for that domain. CIS authoritative servers bypass this restriction by returning the `A records` corresponding to the CNAME target instead of returning the CNAME itself, effectively hiding the CNAME. This technique allows other records, such as MX records, to be added to the domain even though the root record is a CNAME.

Secure DNS

DNSSEC is a technology that digitally "signs" DNS data so you can be confident that it is valid. To eliminate vulnerability from the internet, DNSSEC must be deployed at each step in the lookup process, from the root zone to the final domain name (for example, `www.icann.org`).

Batch DNS record changes

CIS supports batch DNS record changes, allowing you to update multiple zone records in a single action. This approach reduces manual effort and simplifies domain management tasks, such as migrations, environment setups, or automation workflows. While the CIS console supports individual changes, batch operations are best performed by using API.

The [Record type](#). This operation works as an overwrite, so all fields in a record are always affected.

- `Posts`: Used to create new records. The `id` field is not required. For field definitions, see the [Create DNS Record](#) endpoint and select the appropriate record type from the request body specification.

Example request

In this example, the proxied field for the first record that is listed in the `puts` assumes the default value `false`.

```
{
  "deletes": [
    {
      "id": "023e105f4ecef8ad9ca31a8372d0c353"
    }
  ],
  "patches": [
    {
      "id": "023e105f4ecef8ad9ca31a8372d0c353",
      "comment": "Domain verification record",
      "name": "example.com",
      "proxied": true,
      "settings": {},
      "tags": [],
      "ttl": 3600,
      "content": "198.51.100.4",
      "type": "A"
    }
  ],
  "posts": [
    {
      "comment": "Domain verification record",
      "name": "example.com",
      "proxied": true,
      "settings": {},
      "tags": [],
      "ttl": 3600,
      "content": "198.51.100.4",
      "type": "A"
    }
  ],
  "puts": [
    {
      "id": "023e105f4ecef8ad9ca31a8372d0c353",
      "comment": "Domain verification record",
      "name": "example.com",
      "proxied": true,
      "settings": {},
      "tags": [],
      "ttl": 3600,
      "content": "198.51.100.4",
      "type": "A"
    }
  ]
}
```

Events

Security Events provide real-time visibility into malicious and suspicious traffic that is detected and mitigated at the CIS edge before it reaches your origin server. These events enable continuous monitoring of threat activity, analysis of traffic patterns, and response to risks that are targeting web applications, APIs, and supporting infrastructure. By capturing enforcement actions and request details, CIS Security Events help validate security controls and support ongoing threat management. For more information, see [Using the CIS Security Events capability](#).

The following elements are available on the console:

- Events summary: Provides a high-level overview of security activity for the selected time range. Summary metrics help you to quickly assess traffic patterns, measure enforcement actions, and evaluate the effectiveness of applied security controls before reviewing detailed event information.
- Events by action: Provides a time-based visualization of security events that are grouped by the action that is taken for each request during the selected time range.
- Events by service: Lists the security-related activity per security feature. For example, custom rules and WAF.
- Top events by source: Provides a breakdown of traffic that is flagged or mitigated by a CIS security feature during the selected time range.
- Sampled event logs: A list of sampled security events by date to show the action that is taken and applied by various CIS security features.

Global load balancing

A global load balancer manages traffic across server resources that are located in multiple regions. While a single origin server can serve all website content if traffic volumes and latency are manageable, a global load balancer uses a pool approach to distribute traffic across multiple origins. This pooling provides several benefits, including:

- Minimizing response time
- Increasing availability through redundancy
- Maximizing traffic throughput

A global load balancer routes traffic to the pool with the highest priority, distributing the load among the origin servers within that pool. If the primary pool becomes unavailable, traffic automatically routes to the next highest-priority pool.

When pools are assigned to specific regions, traffic that originates from those regions is sent to the corresponding regional pools first. Traffic falls back only to default pools (typically the lowest-priority pool) if all regional pools for that area are unavailable.

How it works

When you create a global load balancer in CIS, a DNS record is automatically added with the name of the load balancer. When a client makes a DNS request, the load balancer returns one of the origin IP addresses from the associated origin pool. For example, consider an origin pool that contains two origin IP addresses `169.61.244.18` and `169.61.244.19`. If a global load balancer named `glbcust.ibmmo.com` is created by using this pool, a client on the internet can run the following command:

```
$ $ ping glbcust.ibmmo.com
PING glbcust.ibmmo.com (169.61.244.18): 56 data bytes
```

In this example, CIS:

- Created a DNS record named `glbcust.ibmmo.com`.
- Used the global load balancer to resolve the DNS name to one of the IP addresses in the origin pool.



Note: In this configuration, the global load balancer doesn't end the TCP connection.

However, if you enable proxy mode for the DNS record or the global load balancer and set **Security > TLS > Mode** to anything other than **Off**, the behavior changes. CIS ends the client connection and establishes a second connection to the origin server.

In this proxied example, CIS:

- Still creates a DNS record named `glbcust.ibmmo.com`.
- Resolves the DNS name to a CIS-provided IP address.

Now, connections to `glbcust.ibmmo.com` are ended by CIS, and HTTPS certificates are hosted by CIS (which is required for TCP termination).

After the client connects to the application, the connection path looks like this:

```
[client]<--tls-->[cis]<-->[origin server]
```

This setup improves security and allows CIS to apply extra features like caching, firewall rules, and traffic inspection.

Related links

- [About global load balancers](#)
- [Creating global load balancers](#)

gRPC protocol support

gRPC (gRPC Remote Procedure Call) is an open source framework that allows applications to communicate efficiently and reliably. It enables the creation of compact, high-performance APIs that reduce bandwidth usage, lower latency, and support scalable service-to-service communication.

gRPC features

- Client-Server Model: gRPC uses a client-server architecture in which the client sends a request to the server, and the server processes the request and returns a response. This model supports efficient communication between services and simplifies integration across distributed systems.
- Protocol Buffers (Protobuf): gRPC uses Protocol Buffers (protobuf) as its Interface Definition Language (IDL). Protobuf provides a language-neutral

and platform-neutral mechanism for serializing structured data, enabling strongly typed contracts between client and server and improving data transmission efficiency.

- HTTP/2: gRPC runs over HTTP/2, which provides multiplexing (multiple requests over a single connection), binary data transfer, header compression, and built-in support for streaming. Streaming enables clients and servers to send or receive data continuously without opening multiple connections, supporting both large datasets and real-time communication.
- Cross-platform and multi-language: gRPC supports multiple programming languages, such as Java, C++, Python, Go, Ruby, Node.js, and many more. This versatility makes it great for building systems where services are written in different languages.

CIS supports gRPC protocol for any proxied gRPC endpoints. To enable or disable gRPC support, navigate to the **Reliability** section, select the **Advanced** tab, and toggle the gRPC switch.

Before enabling gRPC support, ensure that your endpoint meets the following requirements:

- The gRPC endpoint must listen on port `443` and support TLS and HTTP/2.
- HTTP/2 must be advertised over Application-Layer Protocol Negotiation (ALPN).
- gRPC requests must use the `application/grpc` or `application/grpc+<message type>` content-type header.

HTTP protocols

HTTP/2 and HTTP/3 accelerate page load times are included in all IBM Cloud® Internet Services plans. HTTP/2 is enabled by default and requires an SSL certificate at the CIS edge network. Domains on Trial plans can't disable HTTP/2.

Browsers and web servers automatically negotiate the highest protocol available, so HTTP/3 takes precedence over HTTP/2. CIS uses HTTP/1.x only for communication between the origin web server and CIS edge.

To determine the protocol used for your connection, enter `example.com/cdn-cgi/trace` in a web browser or HTTP client, replacing `example.com` with your domain. The `http=` value in the results indicates the protocol:

- `http=h2` means that the connection used HTTP/2
- `http=http2+quic/99` indicates HTTP/3
- `http=http/1.x` indicates HTTP/1.x

HTTP/2

HTTP/2 improves page load times through:

Connection multiplexing

Retrieves multiple resources over a single network connection. Responses are sent as resources become available, helping to avoid delays in page rendering.

HTTP header compression

Compresses headers and simplifies HTTP requests to avoid resending headers.

HTTP/2 server push

To improve page load speed, CIS provides additional resources for a client to cache without waiting for additional requests.



Note: Not all browsers support HTTP/2 and use HTTP 1.x instead. Connection multiplexing is handled on a per-domain basis.

HTTP/3

HTTP/3 enables fast, reliable, and secure connections. By default, HTTP/3 encrypts internet transport by using QUIC, a protocol developed by Google. You can enable HTTP/3 through the Cloudflare Network application. The following methods show you how to experiment with HTTP/3.

Using Google Chrome as your HTTP/3 client

To connect to your website over HTTP/3 using Chrome, first download and install the [latest Chrome Canary build](#). Then, enable HTTP/3 support in Chrome Canary using the `--enable-quic` and `--quic-version=h3-23` [command-line arguments](#).

After Chrome starts, enter your domain in the address bar. To verify the protocol version, open Chrome's Developer Tools and go to the **Network** tab. If `http2+quic/99` doesn't appear in the **Protocol** column, try reloading the page.

Using cURL

The cURL command-line tool supports HTTP/3. [Download the most recent version](#) and follow [the instructions to enable HTTP/3 support](#).

For macOS, use Homebrew to install cURL with HTTP/3 support:

```
$ brew install --HEAD -s https://raw.githubusercontent.com/cloudflare/homebrew-cloudflare/master/curl.rb
```

Then, run an HTTP/3 cURL with the `--http3` command-line flag:

```
$ ./curl -I https://blog.cloudflare.com/ --http3
```

Confirm that HTTP/3 appears in the response and that there are no error messages.

IP firewalls

Legacy IP firewall rules are still supported, but custom rules provide greater flexibility and control. You can accomplish 99.99% or more of IP rule use cases using custom rules, along with advanced features like rule sequencing, custom responses, and detailed logging. For new configurations, use [custom rules](#) whenever possible.

Warning: Since these legacy rules are evaluated earlier in the traffic sequence, allowing an IP, ASN, or country will bypass any configured WAF, custom, or rate limiting rules.

IBM Cloud Internet Services provides multiple mechanisms to control traffic, helping protect your domains, URLs, and directories from excessive requests, specific requesters, and targeted IP addresses.

IP rules

With IP rules you can control access for specific IP addresses, IP ranges, specific countries, specific ASNs, and certain CIDR blocks. Available actions on incoming requests are:

- Allow
- Block
- Challenge (Captcha)
- JavaScript challenge (Defense mode)

For example, if you notice that a particular IP is causing malicious requests, you can block that user by IP address.



Note: IP rules operate at OSI Layer 3 and Layer 4, allowing you to control access for TCP, HTTP, and HTTPS traffic across individual IP addresses, ranges, countries, ASNs, or CIDR blocks.

Allowed IP addresses take precedence over country-based IP rules. However, an allow rule in IP rules does not override a country block that is configured through custom rules.

When you allow a country by using IP rules, requests from that country bypass custom rules, rate limiting rules, and other ruleset-based configurations. Managed Rules and legacy WAF rules still apply.

Domain lockdown

Domain lockdown allows you to restrict access to specific subdomains or URL paths by allowlisting certain IP addresses or IP ranges. Any requests from IPs not on the allowlist are automatically blocked and receive an **Access Denied** response.

This capability is useful when you need more granular control than standard IP firewall rules, which apply to all subdomains of a domain or across all domains in your account. Domain lockdown lets you define restrictions at the subdomain or even URL path level.

Domain lockdown supports the following use cases:

- Specific subdomains - Restrict access to individual subdomains for specific IP addresses. For example, you can allow IP `1.2.3.4` access to the domain `foo.example.com` and allow IP `5.6.7.8` access to domain `bar.example.com`, without allowing access in the reverse direction.
- Specific URLs - Restrict access to specific URL paths. For example, you can allow IP `1.2.3.4` access to directory `example.com/foo/*` and allow IP `5.6.7.8` access to directory `example.com/bar/*`, but not allow the reverse.



Note: All CIS plans have access to Domain Lockdown. The number of rules varies depending on Standard versus Enterprise plans.

You can configure multiple allowlisted IP addresses (both IPv4 and IPv6) and apply them to multiple targets. For example:

```
$ staging.example.com/*  
example.com/wiki/*
```

```
$ 192.0.2.0/24  
2001:db8::/64  
203.0.113.5
```

Only requests from the specified IPs are allowed to access the listed URLs. All others are blocked.

You can also assign a priority to each domain lockdown rule to determine how overlapping rules are handled. Lower numbers indicate higher priority.

User-agent blocking rules

With user-agent blocking rules, you can act on any `user-agent` string you select. This capability works like domain lockdown, except that the rule examines the incoming `user-agent` header instead of the IP address. You can choose how to handle a matching request with the same list of actions that you established in the IP rules (`block`, `challenge`, and `JS challenge`).

User-agent blocking applies to your entire zone. You can't specify subdomains or URL paths in the same manner as you can with a domain lockdown.

This tool is useful for blocking any user-agent strings that you deem suspicious.



Note: Matching is performed by using substring matching. You can match a full `user-agent` string or any portion of it. For example, entering `curl` or `Python-urllib` block those user agents.

Range applications

Range applications allow you to extend DDoS protection, TLS encryption, and custom rule policies to TCP and UDP services beyond traditional HTTP/HTTPS traffic. This secures your TCP-based services beyond just web traffic and provides comprehensive security and performance benefits within IBM Cloud CIS.

Unlike standard web applications protected by CIS, Range applications enable your non-web services, such as VPN endpoints, NAT gateways, database ports, and custom TCP services, to leverage CIS global network and threat intelligence. Traffic destined for these services passes through the CIS infrastructure, where it benefits from multiple layers of security and performance enhancements, including:

- DDoS protection: Mitigate Layer 3/4 attacks against TCP/UDP protocols, including volumetric floods and protocol abuse, to protect non-web services from disruption.
- TLS encryption: Add TLS encryption to services that lack native support or have complex backend setups. This helps secure data in transit and protect against interception or theft.
- Custom rule integration: Enforce granular access controls by allowing, blocking, or challenging specific IP addresses, IP ranges, or other criteria. This reduces unauthorized access and minimizes attack surfaces.
- Secure HTTP(S) Range applications: Apply Layer 7 custom rules to protect HTTP(S)-based Range applications, enabling fine-tuned filtering and policy enforcement. For more information, see [About WAF custom rules](#).
- Configure load balancers: Support TCP health checks, automatic failover, and traffic steering to optimize performance and maintain high availability across Range-enabled services.
- Proxy protocol support: Preserve client IP visibility across proxy and load balancer setups, which is critical for accurate logging, security policy enforcement, and auditing.



Note: Range applications are only available to Enterprise customers for an additional cost, and is priced per bandwidth usage.

Related links

- [Working with Range applications](#)
- [About Range applications](#)

Smart routing and tiered caching

Reduce excess latency by optimizing routing decisions across the global internet by using real-time network data. On average, this technique can reduce internet latency by 30% and connection errors by 27%.

Smart Routing

Smart Routing efficiently directs connections across the internet by avoiding packet loss, congestion, and outages. It optimizes the paths that your data takes by analyzing latency and packet loss from real-time network connections. Using this data, Smart Routing identifies the best transit providers between any two points worldwide to ensure that your traffic follows the most reliable route.



Note: The optimized route differs from the regular request path. After you enable Smart Routing for your domain, it can take up to 24 hours to switch from the regular path to the optimized route.

Tiered Caching

Tiered Caching improves cache hit ratios by using a hierarchical approach that reduces requests to your origin servers and decreases their load. When a cache miss occurs, instead of immediately querying the origin, Tiered Caching asks other Points of Presence (PoPs) if they have the requested data. Since PoPs are typically closer to each other than to your origin, this reduces latency and speeds up content delivery.

Tiered Caching also concentrates connections to your origin servers so they come from fewer PoPs. This feature reduces the number of open connections to the origin, easing server load and improving performance.

TCP connections and limits

Transmission Control Protocol (TCP) connections are established between clients and servers when accessing CIS-protected resources. When users request a website using CIS, their requests pass through the CIS infrastructure, enabling DDoS protection and enhanced security.

How CIS connects users to the origin server

CIS acts as an intermediary between the user and the origin by routing user requests through CIS/Cloudflare to the application's origin server. User traffic is routed to the nearest point of presence (PoP); where the request is then processed. In case a request is not served from one of Cloudflare's data centers, a connection will be opened to the origin server to forward the request.

Connection limits

When HTTP/HTTPS traffic is proxied through CIS, there are often two established TCP connections: the first is between the requesting client to CIS, and the second is between CIS and the origin server. Each connection has their own set of TCP and HTTP limits.

Connection limits between the requesting client and CIS

Table 1 lists connection limits between the requesting client and CIS.

Type	Limit (seconds)	HTTP status code at limit	Configurable
Connection Keep-Alive HTTP/1.1	400	TCP connection closed	No
Connection Idle HTTP/2	400	TCP connection closed	No

Connection limits between the requesting client and CIS

Connection limits between CIS and the origin server

Table 2 lists connection limits between CIS and the origin server.

Type	Limit (seconds)	HTTP status code at limit	Configurable
Complete TCP Connection [1]	19	522	No
TCP ACK Timeout [2]	90	522	No
TCP Keep-Alive Interval [3]	30	520	No
Proxy Idle Timeout [4]	900	520	No
Proxy Read Timeout [5]	125	524	524 No

HTTP/2 Pings to Origin	Off	N/A	Yes
HTTP/2 Connection Idle [7]	900	No	No

Connection limits between CIS and the origin server. Some TCP connections can be customized for Enterprise customers.

TCP connections and keep-alives

CIS uses keep-alive connections to improve performance and reduce costs by minimizing the need for repeated TCP connections when proxying customer traffic to origin servers. To optimize this process, ensure that HTTP keep-alive is enabled on your origin server, allowing reuse of open TCP connections up to the Proxy Idle Timeout after the last request. This helps prevent connection resets and enhances efficiency.

Typically, HTTP opens a new TCP connection for each request-response cycle, which can add overhead. Keep-alives allow a single TCP connection to remain open for multiple requests, reducing latency and network traffic, ultimately enhancing user experience.

While TCP connections can remain active, idle connections are usually terminated after a certain time. CIS has a default idle timeout of 400 seconds for user connections and sends keep-alive probes every 75 seconds. If nine consecutive probes go unanswered, CIS ends the connection with a TCP Reset (RST) packet.

It's important to note that keep-alives do not guarantee connection stability due to factors like capacity balancing or maintenance, so applications should be designed to handle disconnections gracefully. Enterprise customers can customize TCP connection settings between users and CIS, as well as between CIS and the origin server.

1. TCP uses a three-way handshake to establish a reliable connection (SYN, SYN-ACK, ACK) over an IP based connection. SYN is short for synchronize, and ACK is short for acknowledgement. [↪](#)
2. The final step in the TCP three-way handshake, confirming the establishment of a connection. [↪](#)
3. A TCP keep-alive is used to maintain a connection between two endpoints by sending packets to check if the connection is still active. This helps prevent idle connections from being prematurely closed. If a response is not received after a defined period, the connection is terminated. [↪](#)
4. When a TCP connection is in an idle state, it means that the connection has been established, but neither endpoint is sending any data. In the context of HTTP, an idle connection is when an established connection between a client and a server is not currently transmitting any HTTP requests or responses. [↪](#)
5. A proxy read timeout is the maximum amount of time a proxy server waits for a response from the origin server before terminating the connection. This setting can only be modified with Enterprise plans. [↪](#)
6. A proxy write timeout is the maximum amount of time a proxy server allows for sending data to the client before terminating the connection. [↪](#)
7. When a TCP connection is in an idle state, it means that the connection has been established, but neither endpoint is sending any data. In the context of HTTP, an idle connection is when an established connection between a client and a server is not currently transmitting any HTTP requests or responses. [↪](#)

Traffic sequencing

Traffic sequencing refers to the order in which IBM Cloud® Internet Services processes incoming web traffic. Understanding this sequence helps clarify how CIS applies different security and performance features, such as DDoS protection, firewall rules, and edge functions.

When you work with CIS, it can sometimes be confusing to know the order in which different operations are applied to your traffic. For example, whether firewall rules run before edge functions or page rules. Traffic typically flows through CIS in the following order:

1. [Distributed Denial of Service \(DDoS\) protection](#) - Initial defense to block large-scale attacks.
2. [URL rewrites](#) - Adjustments to incoming URLs before further processing.
3. [Page rules](#) - Custom rules that modify how requests are handled.
4. [IP firewall rules](#) - Filtering based on IP addresses.
5. [Bot management](#) - Identifies and manages automated traffic.
6. WAF and custom rules, which generally process in this order:
 - a. [Custom rules](#)
 - b. [WAF rules](#)
 - c. [Rate limiting](#)



Note: Based on the actions and priority settings of the rules, steps in the firewall sequence can be bypassed. For example, a firewall rule with an early priority that allows certain traffic is processed first, and then skips over the rest of the rules in the sequence.

7. [Edge functions](#) - Custom scripts that run on the edge to modify requests or responses.
8. [Global load balancer](#) - Distributes traffic across your origin servers.

Knowing this sequence helps you understand how different security and performance features interact and which rules or functions take precedence.

URL normalization

URL normalization modifies the URLs of incoming requests so that they conform to a consistent formatting standard.

When you enable URL normalization, all incoming URLs are normalized before they pass to subsequent global network features that accept a URL input. Rule expressions that filter traffic based on URLs will then trigger correctly, regardless of the format of the incoming URL. If you turn off URL normalization, CIS forwards the URL to the origin in its original form.

URL normalization does not perform redirects, and doesn't change the address shown in the visitor's browser. The normalization operation happens on the global network and affects CIS features that are executed later and (optionally) the URL received at the origin server.



Note: URL normalization requires that you proxy the DNS records of your domain (or subdomain) through CIS.

URL normalization settings

The following settings manage URL normalization.

Normalization type (default: RFC-3986)

Selects the type of normalization to perform:

- **RFC-3986** – Applies URL normalization strictly according to [RFC 3986](#).
- **CIS** – In addition to what is defined in RFC 3986, applies extra URL normalization techniques.

Normalize incoming URLs (default: On)

Configures the URLs of all incoming traffic to CIS:

- When enabled, all incoming URLs are normalized before they pass to subsequent CIS features that can receive a URL as input, such as Page Rules, custom rules, Workers, and Access.
- When disabled, incoming URLs are not normalized before passing to subsequent CIS features.

Normalize URLs to origin (default: Off)

Configures URLs sent to the origin:

- When enabled, requests sent to the origin are normalized.
- When disabled, requests sent to the origin are not modified.

You can only view and enable this option when Normalize incoming URLs is enabled.

Web Application Firewall protection

A Web Application Firewall (WAF) protects web applications by filtering and monitoring HTTP traffic between the application and the internet. As a Layer-7 defense in the [OSI model](#), it targets threats at the application layer, such as cross-site forgery, cross-site scripting (XSS), file inclusion, and SQL injection.

Unlike a traditional proxy that handles outgoing client traffic, a WAF functions as a reverse proxy, inspecting incoming traffic before it reaches the server. This setup blocks malicious requests and shields the application from direct exposure to the internet.

WAFs operate through configurable rule sets or policies that are designed to filter out malicious traffic and protect application vulnerabilities. One major benefit of a WAF is the ability to quickly modify these policies, enabling rapid responses to emerging threats, such as applying rate limiting during a [DDoS attack](#).

CIS WAF provides a way to configure and manage security rules. After you redirect your DNS to CIS, you can begin applying custom rules. You also get

detailed reporting that provides insights into blocked threats and the specific rules or rule groups involved.

Best practices

Best practices for CIS setup

Because IBM Cloud® Internet Services is positioned at the edge of your network, you'll need to take a few steps to guarantee a smooth integration with your CIS services. Consider these recommended best practices for integrating CIS with your origin servers.

You can do these steps either before or after you change your DNS and activate our proxy service. These recommendations allow CIS to connect to your origin servers properly. They'll help you prevent any issues with API or HTTPS traffic, and help your logs capture the correct IP addresses of your customers, rather than the protective CIS IP addresses.

Here's what you need to set up:

- Restore the originating IPs of your customers
- Incorporate CIS IP addresses
- Make sure that your security settings don't interfere with API traffic
- Configure your security settings as strictly as possible

Best practice 1: Know how to restore the originating IPs of your customers

As a reverse proxy, CIS provides the origination IP in these headers:

- `CF-Connecting-IP`
- `X-Forwarded-For`
- `True-Client-IP` (optional)

You can restore user IP addresses by using various tools for infrastructures such as Apache, Windows IIS, and NGINX.

Best practice 2: Incorporate CIS IP addresses to make integration smoother

Take the following two steps:

- Remove any rate limiting of CIS IP addresses
- Set up your ACLs to allow only CIS IP addresses and other trusted parties

You can find the updated list of IP ranges for IBM CIS [at this location](#).

Best practice 3: Configure your security settings as strictly as possible

CIS provides some options for encrypting your traffic. As a reverse proxy, the TLS connection is terminated at Cloudflare and a new TLS connection is opened to your origin servers. For your termination with CIS, you can upload a custom certificate from your account, use a wildcard certificate that is provisioned for you by CIS, or both.

Upload a custom certificate

You can upload your public and private keys when you create an Enterprise domain. If you upload your own certificate, you gain immediate compatibility with encrypted traffic, and you maintain control over your certificate (for example, an Extended Validation (EV) certificate). Remember that you are responsible for managing your certificate if you upload a custom certificate. For example, IBM CIS doesn't track the certificate expiration dates.

Alternatively, use a certificate provisioned by CIS

IBM CIS works with several Certificate Authorities (CAs) to provide domain wildcard certificates by default. Manual verification might be required for setting up these certificates, and your support team can help you perform these additional steps.

Change your TLS setting to End-to-End CA Signed

Most of our Enterprise customers use the End-to-End CA Signed security setting. An **End-to-End CA Signed** setting requires a valid, CA-signed certificate installed on your web server. The certificate's expiration date must be in the future, and it must have a matching hostname or Subject Alternative Name (SAN).

Managing CIS for optimal security

The IBM Cloud® Internet Services (CIS) security settings include safe defaults that are designed to avoid false positives and negative influence on your traffic. However, these safe default settings do not provide the best security posture for every customer. Follow these steps to make sure that your CIS

account is configured in a safe and secure way.

Best practice 1: Secure your origin IP addresses

When a subdomain uses CIS for proxying, all traffic is protected because CIS returns IP addresses that are associated with its proxy network. This ensures that connections are routed through CIS proxies first, which hides the origin IP addresses.

Use CIS proxies for all DNS records for HTTP traffic from your origin

To improve the security of your origin IP address, you must proxy all HTTP and HTTPS traffic.

See the difference by querying a non-proxied and a proxied record:

```
$ dig nonproxied.theburritobot.com +short
1.2.3.4 (The origin IP address)

$ dig proxied.theburritobot.com +short
104.16.22.6 , 104.16.23.6 (CIS IP addresses)
```

Obscure non-proxied origin records with non-standard names

Any records that can't be proxied through CIS, and that still use your origin IP, such as FTP, can be secured by creating additional obfuscation.

If you require a record for your origin that can't be proxied by CIS, use a non-standard name. For example, instead of `ftp.example.com` use `[random word or-random characters].example.com`. This obfuscation makes dictionary scans of your DNS records less likely to expose your origin IP addresses.

Use separate IP ranges for HTTP and non-HTTP traffic if possible

Some customers use separate IP ranges for HTTP and non-HTTP traffic. This approach helps them proxy all records for their HTTP IP range, and hide all non-HTTP traffic with a different IP subnet.

Best practice 2: Activate your Web Application Firewall safely

Your WAF is available in the **Security** section. These steps walk through the settings in reverse order to ensure that your WAF is configured as safely as possible before you enable it for your entire domain.

These initial settings can reduce false positives by populating **Security Events** for further tuning. Your WAF is updated automatically to handle new vulnerabilities as they are identified. For more information, see [Using Security events capability](#).

The following are some examples of the types of attacks that WAF can protect against:

- SQL injection attack
- Cross-site scripting
- Cross-site forgery

CIS provides several preconfigured rulesets within WAF for Layer 7 protection. Each ruleset includes protections to mitigate the most common web application attacks. You can enable or disable these rulesets and override individual rules to meet your security requirements. For optimal protection, enable the CIS Managed Ruleset at the highest priority. You can use the OWASP ruleset for additional coverage, but place it after the CIS Managed Ruleset in your rule execution order.



Note: The OWASP ruleset is prone to false positives and offers only marginal benefits when placed before the CIS Managed Ruleset and WAF attack score. If you enable the OWASP ruleset, monitor traffic and adjust rule settings as needed to prevent false positives. For more information, see [WAF actions](#) for details about the ruleset and the behavior of each rule.

For more information, see [Web Application Firewall \(WAF\) concepts](#).

Best practice 3: Configure WAF Attack Score safely

WAF Attack Score strengthens your security posture by detecting variations of known attacks by using machine learning. It complements WAF managed rules by identifying malicious requests that do not match existing rule signatures.

To configure WAF Attack Score safely, begin with monitoring traffic before you enforce blocking actions. This approach allows you to review scored traffic in Security Events and adjust thresholds to minimize false positives.

For more information, see [using WAF Attack Score](#) to configure WAF Attack Score in a custom rule.

CIS recommends using WAF managed rules along with WAF Attack Score. Managed rules protect against known attack patterns, while WAF Attack Score

detects modified or obfuscated attacks, providing layered and adaptive protection for your domain. For more information, see [About WAF Attack Score](#).

Best practice 4: Configure your TLS settings

Protect your site and control your Transport Layer Security (TLS) settings. Manage the certificates used to secure traffic to your site. IBM CIS functions as a reverse proxy and provides multiple options for encrypting your traffic. As a reverse proxy, TLS connections terminate at CIS data centers, and a new TLS connection is opened to your origin server.

TLS offers six modes of operations that are listed from most secure to least secure:

1. [Authenticated origin pull](#): (Enterprise only)
2. [HTTPS only origin pull](#) (Enterprise only)
3. [End-to-end CA signed](#) (default and recommended)
4. [End-to-end flexible](#) (edge-to-origin certificates can be self-signed)
5. [Client-to-edge](#) (edge-to-origin not encrypted, self-signed certificates are not supported)
6. [Off](#) (not recommended)

See [TLS options](#) for details on the different modes.

With IBM CIS, you can use custom certificates or a wildcard certificate that is provisioned for you by CIS.

Upload custom certificates

You can upload your custom certificate by clicking **Add Certificate** and entering your certificate, private key, and bundle method. After you upload a custom certificate, you gain immediate compatibility with encrypted traffic and maintain control over your certificate (for example, an Extended Validation (EV) certificate).

CIS does not support certificate pinning with ordered or Universal certificates. If you want to use certificate pinning, upload and maintain your own custom certificate.



Important: You are responsible for managing your certificate if you upload a custom certificate. For example, CIS doesn't track the certificate expiration date.

Order dedicated certificates

CIS simplifies certificate management by offering dedicated certificates. You no longer need to generate private keys, create certificate signing requests (CSR), or remember to renew certificates. You can order a dedicated certificate by clicking **Add Certificate** and ordering a wildcard certificate or entering hostnames to order a dedicated custom certificate. Available certificate types include:

- SHA-2/ECDSA signed certificate that uses a P-256 key,
- SHA-2/RSA signed certificate that uses an RSA 2048-bit key, and
- SHA-1/RSA signed certificate that uses an RSA 2048-bit key.

CIS can issue certificates for all top-level domains (TLDs) except the following:

- **.cu** (Cuba)
- **.iq** (Iraq)
- **.ir** (Iran)
- **.kp** (North Korea)
- **.sd** (Sudan)
- **.ss** (South Sudan)
- **.ye** (Yemen)

CIS manages the certificate expiration date for dedicated certificates.

To edit the hostnames on a dedicated custom certificate, you must reorder the certificate and then delete the original one. For example, suppose that you order a dedicated custom certificate with the hostname **alpha.yourdomain.com**. To add the hostname **beta.yourdomain.com** to your dedicated custom certificate, order another dedicated custom certificate with the hostnames **alpha.yourdomain.com** and **beta.yourdomain.com**. Afterward, you must delete the original dedicated custom certificate.



Note: The first time that you order a dedicated certificate, the Domain Control Validation (DCV) process runs and generates a corresponding TXT record. If you delete the TXT record, the DCV process runs again when you order another dedicated certificate. If you delete a dedicated certificate, the TXT record corresponding to the DCV process is not deleted.

The following are common errors that might occur when you order dedicated certificates:

- Error connecting to certificate service.
- Error while requesting from certificate service.

If you receive an error when you order certificates, refresh the page and try again.

Use a provisioned certificate

IBM has partnered with several certificate authorities (CAs) to provide wildcard certificates for customer domains. Manual verification might be required during the setup process. Your support team can help you complete these additional steps.

Certificate priority at our edge

Certificates are served at the edge in the following priority order:

1. Uploaded custom
2. Dedicated custom
3. Dedicated wildcard
4. Universal

Minimum TLS version

Higher TLS versions provide stronger security but might prevent some customers from connecting to your site. For more information, see [Minimum TLS version](#).

Best practice 5: Configure rate limiting

Use rate-limiting rules to protect your site or API from malicious traffic by blocking client IP addresses that match a URL pattern or exceed a defined threshold.

Common use cases for rate limiting include:

- Enforcing granular access control to resources. This process includes access control based on criteria, such as user agent, IP address, referrer, host, country, and world region.
 - Limit by user agent: A common use case is to limit the rate of requests performed by individual user agents.
 - Allow specific IP addresses or ASNs: Another use case when you control access to resources is to exclude or include IP addresses or Autonomous System Numbers (ASNs) from a rate-limiting rule.
 - Limit by referrer: Some applications receive requests that are originated by other sources (for example, used by advertisements that link to third-party pages). You can limit the number of requests that are generated by individual referrer pages to manage quotas or avoid indirect DDoS attacks.
 - Limit by destination host: Create a rate-limiting rule that uses the host as a counting characteristic.
- Protecting against credential stuffing and account takeover attacks.
- Limiting the number of operations performed by individual clients. This action includes preventing scraping by bots, accessing sensitive data, bulk creation of new accounts, and programmatic buying in e-commerce platforms.
 - Prevent content scraping (through query string)
 - Prevent content scraping (through body)
 - Limit requests from bots
- Protecting REST APIs from resource exhaustion (targeted DDoS attacks) and resources from abuse in general.
 - Prevent volumetric attacks
 - Protect resources
- Protecting GraphQL APIs by preventing server overload and limiting the number of operations.
 - Limit the number of operations
 - Prevent server overload

Managing your CIS deployment for optimal reliability

To achieve optimal reliability for your IBM Cloud® Internet Services deployment, you can set up a helpful DNS configuration and global load balancers. For additional reliability, you can use page rules to ensure that your web content is delivered, even if your origin server or the cache has a problem.

Generally, the recommended best practices are these:

- Set up your DNS to take advantage of CIS proxy servers and other features.
- Use one or more global load balancers to distribute your site traffic evenly.
- Set up appropriate page rules to manage your caching and other options.

Each of these items provides certain functionality you can use to create a more reliable CIS deployment.

Notice that the CIS interface is organized into sections for *security*, *reliability*, and *performance*.

Setting up DNS

To get started setting up your DNS configuration, select **DNS** from the navigation menu.

For information about setting up and managing your DNS for reliability, see [Setting up your Domain Name System for CIS](#).

Setting up global load balancers

To get started setting up your global load balancers, select **Global Load Balancers** from the navigation menu.

For information about setting up and managing your global load balancers, see [Global load balancer concepts](#).

Using page rules to increase reliability

The following list shows some recommended page rule settings to give your site maximum reliability.

- Origin Cache Control
- Forwarding URL

Serve Stale Content

You can use the **Serve Stale Content** setting to keep a limited version of your site online if your server goes down.

With **Serve Stale Content**, when your server goes down, CIS serves pages from the cache, so your visitors still see some of the pages they are trying to visit. Your visitors see a message at the top of the page telling them that they are in offline browsing mode. Serve Stale Content returns an HTTP status 503, however, 503 is also used by many other web applications. When your server comes back online, CIS bumps users back to regular browsing seamlessly.

If CIS does not have the requested page in its cache, your visitor sees an error page letting them know that the website page they are requesting is offline.

Setting up Serve Stale Content

To enable **Serve Stale Content**, take the following steps.

1. Use the navigation menu to go to **Performance > Caching**.
2. Toggle the **Serve Stale Content**.

Serve Stale Content limitations

- **Serve Stale Content** integrates with the Internet Archive. CIS crawls the most popular URLs that have a **200** HTTP status from the past 5 hours. This means that only some pages on your site are viewable when your origin server goes down.
- Sites that were added recently don't have a large cache of their site available. This means that **Serve Stale Content** might not work if you added the site only a few days ago.
- CIS does not show private content or handle form submission (POSTs) if your server is down. Visitors are shown an **error on checkout** page or **items require a login to view**.
- To trigger **Serve Stale Content**, your web server must be returning a standard HTTP Error code of 502 or 504 timeout. Serve Stale Content also works when CIS encounters issues contacting your origin (Errors 521 and 523), timeouts (522 and 524), SSL errors (525 and 526) or an unknown error (520). **Serve Stale Content** is not triggered for other HTTP response codes, such as 404s, 500, 503, database connection errors, internal server error, or empty replies from server.
- **Serve Stale Content** does not work if a **Cache Everything** page rule is enabled with the **Edge Cache Expire TTL** set lower than the caching frequency, because the **Edge Cache Expire TTL** causes the **Serve Stale Content** cache to be purged in the corresponding interval.

Origin Cache Control

You can use the **Origin Cache Control** Page Rule setting to determine what content is cached from your origin and how often the content is updated, which has an effect on reliability and on performance. By default, if no settings are changed and no headers that prevent caching are sent from your origin server,

CIS caches all static content with certain extensions. These types of content include images, CSS, and JavaScript. This caching is primarily for performance reasons.

To set up **Origin Cache Control** use page rules to turn on specific headers that give the wanted behavior with respect to each resource of your content. To understand how to use **Origin Cache Control**, some more general explanation of page rules and overall caching behavior for CIS is required to provide context, which is covered in the next several sections. Three methods exist that you can use to control caching in general, and **Origin Cache Control** is the second one.

Setting **Origin Cache Control** invokes caching rules that seek to adhere closely to internet best practices and RFCs, primarily with respect to revalidation. For example, the CIS default behavior with `max-age=0` is not to cache at all, whereas setting **Origin Cache Control** caches, but it always revalidates.

Setting up Origin Cache Control

Follow these steps to enable **Origin Cache Control**.

1. Use the navigation menu to select **Page Rules** under **Performance**.
2. Create a page rule with the URL pattern that references your domain.
3. Add the **Origin Cache Control** setting with the toggle on.
4. Select **Provision Resource**.

Page Rule precedence

These specific page rules take precedence for caching overall.

- If a page rule has **Cache Level** set to `Bypass`, the resources that match that page rule are not cached. CIS still acts as a proxy, and our other performance features remain active. However, your content is fetched from your origin server directly, instead of served from our cache.
- If a page rule has **Cache Level** set to `Cache everything`, resources that match the page rule are cached. **Using this page rule setting is the only way to tell CIS to cache resources beyond what CIS considers static, including HTML.**

If no page rule is set, CIS uses the `Standard` caching mode, which is based the extension of the resource. CIS caches static resources only.

Origin cache-control headers

The second way to alter what CIS caches is through caching headers sent from the origin. CIS respects these settings, but you can override them by specifying an **Edge Cache TTL** page rule setting. Here are the headers CIS considers when deciding what resources to cache from your origin:

- If the **Cache-Control** header is set to `private`, `no-store`, `no-cache`, or `max-age=0`, or if there is a cookie in the response, then CIS does not cache the resource. Note that sensitive material should not be cached, so you might consider using one of these headers in that case.
- If the **Cache-Control** header is set to `public` and the `max-age` is greater than 0, or if the `Expires` headers are set any time in the future, the resource is cached.



Note: According to RFC rules, `Cache-Control: max-age` trumps `Expires` headers. If both are seen and they do not agree, `max-age` wins.

Using the `s-maxage` header

The third way to control caching behavior and browser caching behavior together is by using the `s-maxage` Cache-Control header.

Normally CIS respects the `max-age` directive.

```
Cache-Control: max-age=1000
```

But if you want to specify a cache timeout that's different from the browser, CIS can use `s-maxage`. The following example tells CIS to cache the object for 200 seconds and the browser to cache the object for 60 seconds.

```
Cache-Control: s-maxage=200, max-age=60
```

Basically, `s-maxage` is intended to be followed ONLY by reverse proxies (so the browser should ignore it) while (CIS) gives priority to `s-maxage` if it is present. CIS respects whichever value is higher: the browser cache setting or the `max-age` header.

Summary on cache control headers and page rules for reliability

The following main areas are important for reliability in caching:

- Check your origin's caching headers to make sure there are no overriding headers for cacheable resources (`Cache-Control` and `Expires`).
- CIS always caches static content by default, with the following TTL depending on the return code:


```
$ 200 301 120m;  
302 303 20m;  
403 5m; for reliability  
404 5m;  
any 0s;
```

- To cache more, create a Page Rule with **Cache Level** set to `Cache everything` on the URL (if your web server returns a 404 when requesting this URL, this result is cached for 5m only).
- To avoid caching on a URL, create a page rule with **Cache Level** set to `Bypass`.

Forwarding URL

To ensure that your content is always available, create page rule with the **Forwarding URL** setting used, in case your site is unavailable.



Note: When you enable a **Forwarding URL**, all of your other settings are disabled because you are sending all of your traffic to another URL.

Setting up a forwarding URL

To enable **Forwarding URL**, follow these steps:

1. Use the navigation menu to select **Page Rules** under **Performance**.
2. Create a page rule with the URL pattern that references your domain.
3. Add the **Forwarding URL** setting.
4. Select the forwarding type and enter the destination URL.
5. Select **Provision Resource**.

Forwarding URL examples

Imagine that you want to make it easy for anyone coming to reach a URL such as:

```
*www.example.com/+
```

```
*example.com/+
```

This pattern matches:

```
$ http://example.com/+  
http://www.example.com/+  
https://www.example.com/+  
https://blog.example.com/+  
https://www.blog.example.com/+
```

It does not match:

```
$ http://www.example.com/blog/+ [extra directory before the +]  
http://www.example.com+ [no trailing slash]
```

After you've created the pattern that matches what you want, add the **Forwarding URL** setting and select the forwarding type and enter the destination URL. For example:

```
https://plus.google.com/yourid
```

Select **Provision Resource**. Within a few seconds any requests that match the pattern are forwarded to the new URL with the specified redirect.

Advanced forwarding options

If you use a basic redirect, such as forwarding the root domain to `www.yourdomain.com`, you lose anything else in the URL. For example, you could set up the pattern:

```
example.com
```

And have it forward to:

```
http://www.example.com
```

But then if someone entered:

example.com/some-particular-page.html

Then they are redirected to:

www.example.com

instead of:

www.example.com/some-particular-page.html

The solution is to use variables. Each wildcard corresponds to a variable that can be referenced in the forwarding address. The variables are represented by a `$` followed by a number. To refer to the first wildcard you'd use `$1`, to refer to the second wildcard you'd use `$2`, and so on. To fix the forwarding from the root to `www` in the previous example, use the same pattern:

example.com/*

Then set up the following URL for traffic to forward to:

http://www.example.com/\$1

In this case, if someone went to:

example.com/some-particular-page.html

They are redirected to:

http://www.example.com/some-particular-page.html

Managing your CIS deployment for best performance

IBM Cloud® Internet Services (CIS) can provide the fastest experience for your customers because it optimizes your images, and it stores your web content as near as possible to your end-users. Your content is loaded from proxied edge servers (which reduces latency).

With CIS, you can enhance your site's performance further by using best practices to speed up the loading of your web content. Here are some specific best practices for enhancing the performance of your web content within CIS.

Recommended and best practices:

- Cache as much of your static and semi-static web content as possible
- For event-driven content, purge your cache with the API

Best practice 1: Cache as much static and semi-static content as possible

- Enable **Cache Everything** for static HTML web pages
- Use conservative **Time-to-live (TTL)** for your content that changes occasionally

Utilize conservative TTLs (Time-to-Lives) for content that changes occasionally

If content rarely changes, you can set a conservative TTL to utilize our cache as much as possible. If you have a high percentage of re-validation requests, you could increase the TTLs of your content without negatively affecting your customers. By using the cache more effectively, you'll increase performance because you'll revalidate less often.

How do I tell if items are being cached?

CIS adds the response header `CF-Cache-Status` when it attempts to cache an object. If caching is successful, the value of this header indicates its status with one of these keywords:

- **MISS:** The asset was not yet in the cache or the TTL had expired (that is, it had reached the cache-control maximum age of 0).
- **HIT:** The asset was delivered from the cache.
- **EXPIRED:** This asset was delivered from cache, but the next request requires revalidation.
- **REVALIDATED:** The asset was delivered from cache. The TTL was expired, but an `If-Modified-Since` request to the origin indicated that the asset had not changed. Therefore, the version in cache is considered valid again.

Best practice 2: For event-driven content, purge your cache

For example, every time a new post is added to your blog, you can purge the CIS cache. It is common to see event-driven content, and CIS can guarantee that no stale content is reaching your users. The commands to purge the cache immediately across the entire global network are:

- Purge all files

- Purge by prefixes (Enterprise only)
- Purge by hostnames (Enterprise only)
- Purge by tags (Enterprise only)
- Purge by URLs

Purge all files

You have the option to force the entire cache to revalidate. You can reset all of the objects stored in the cache so that every request is routed to the origin server.

Purge by prefixes (Enterprise only)

Enterprise plan users can purge the cache by URL prefix or path separators in the URL. For example, valid purge requests for a URL such as

`https://www.example.com/foo/bar/baz/qux.jpg` include:

- `www.example.com/`
- `www.example.com/foo/`
- `www.example.com/foo/bar/`
- `www.example.com/foo/bar/baz/`
- `www.example.com/foo/bar/baz/qux.jpg`

Purging by prefix is useful when you want to purge everything within a directory, or increase control over cached objects in a particular path. It can also simplify the number of purge calls made.

Purge by hostnames (Enterprise only)

Purging by hostname is similar to purging by prefixes. Use a list of hostnames to purge the cache of any assets associated with those hostnames.

Purge by tags (Enterprise only)

Tags let you define buckets of content that you want to purge. It is an excellent way to combine objects that are commonly changed together. An HTML blog post, for example, and all of its image content could be tagged together. Mobile-only content also could be bundled using cache-tags, so that you can purge everything when you push a new update to your mobile domain.

Purge by URLs

With purge by URL, cached resources are immediately removed from the stored assets in your Content Delivery Network (CDN) across all data centers. New requests for the purged asset receive the latest version from your origin web server and add it back to your CDN cache within the specific data center that served the request.

Use advanced caching features

Bypass Cache on Cookie: Configured in a page rule, this feature allows you to serve a cached object unless a cookie of a specific name exists. For example, you can serve a cached version of the homepage unless you find a `SessionID` cookie indicating that the customer is logged in, and therefore should be presented with personalized content.

Rate limiting best practices

The following sections cover typical rate limiting configurations for common use cases. You can combine the provided example rules and adjust them to your own scenario.

The main use cases for rate limiting are the following:

- Enforce granular access control to resources which includes the access control based on criteria such as user agent, IP address, referrer, host, country, and world region.
- Protect against credential stuffing and account takeover attacks.
- Limit the number of operations performed by individual clients. Includes preventing scraping by bots, accessing sensitive data, bulk creation of new accounts, and programmatic buying in e-commerce platforms.
- Protect REST APIs from resource exhaustion (targeted DDoS attacks) and resources from abuse in general.
- Protect GraphQL APIs by preventing server overload and limiting the number of operations.

Enforcing granular access control

You can use rate limiting to control how users and applications access your resources. Rate limiting helps protect your application from abuse by restricting

traffic based on attributes such as user agent, IP address, referrer, or host.

Each of the following examples demonstrates how to configure a rate-limiting rule for a specific access control scenario.

Limiting request by user agent

You can restrict the number of requests that are allowed for a specific user agent. The following rule example allows mobile app users to make up to 100 requests every 10 minutes. You might also create a separate rule limiting the rate for desktop browsers.

Setting	Value
Matching criteria	User Agent equals <code>MobileApp</code>
Expression	<code>http.user_agent eq "MobileApp"</code>
Counting characteristics	IP
Rate (Requests/Period)	100 requests / 10 minutes
Action	Managed Challenge

Example—Limit requests by user agent

Allowing specific IP addresses or ASNs

Control access by including or excluding certain IP addresses or Autonomous System Numbers (ASNs) from a rate limiting rule.

The following rate limiting rule example allows up to 10 requests per minute from the same IP address and make a `GET` request to the `/status` path, provided the IP address is not included in the [IP list](#) entitled `partner_ips`.

Setting	Value
Matching criteria	URI Path equals <code>/status</code> and Request Method equals <code>GET</code> and IP Source Address is not in list <code>partner_ips</code>
Expression	<code>http.request.uri.path eq "/status" and http.request.method eq "GET" and not ip.src in \$partner_ips</code>
Counting characteristics	IP
Rate (Requests/Period)	10 requests / 1 minute
Action	Managed Challenge

Example—Allow specific IPs or ASNs

Limiting requests by referrer

You can limit requests that originate from referrer pages, such as third-party advertisements or external websites. This use case helps to reduce the risk of indirect denial-of-service (DDoS) attacks and helps you to manage request quotas.

Setting	Value
Matching criteria	URI Path equals <code>/status</code> and Request Method equals <code>GET</code>
Expression	<code>http.request.uri.path eq "/status" and http.request.method eq "GET"</code>
Counting characteristics	Header (<code>Referrer</code>). The HTTP header name uses a misspelling of <code>referrer</code> .
Rate (Requests/Period)	100 requests / 10 minutes
Action	Block

Example—Limit requests by referrer



Note: This example rule requires Advanced Rate Limiting.

Protecting against credential stuffing

You can use rate limiting to protect login endpoints from credential stuffing attacks. Credential stuffing occurs when attackers use automated scripts to try multiple username and password combinations on a login form. Rate limiting helps mitigate these attacks by restricting repeated failed login attempts from the same IP address.

The following examples show three rate-limiting rules that increase restrictions and penalties based on the number of failed login attempts.

Rule 1: Initial protection threshold

Rule 1 allows up to four failed login attempts per minute. When the limit is exceeded, the system triggers a Managed Challenge. This configuration helps legitimate users recover from occasional login errors while discouraging automated bots.

Setting	Value
Matching criteria	Hostname equals <code>example.com</code> and URI Path equals <code>/login</code> and Request Method equals <code>POST</code>
Expression	<code>http.host eq "example.com" and http.request.uri.path eq "/login" and http.request.method eq "POST"</code>
Counting characteristics	IP
Increment counter when	URI Path equals <code>/login</code> and Method equals <code>POST</code> and Response code is in (401, 403)
Counting expression	<code>http.request.uri.path eq "/login" and http.request.method eq "POST" and http.response.code in {401 403}</code>
Rate (Requests/Period)	4 requests / 1 minute
Action	Managed Challenge

Rule 1 configuration

Rule 2: Intermediate protection threshold

If legitimate users pass the challenge when reaching rule 1's rate limit, Rule 2 applies additional protection for clients that continue to make failed login attempts. It allows up to 10 failed attempts over 10 minutes before triggering another Managed Challenge.

Setting	Value
Matching criteria	Hostname equals <code>example.com</code> and URI Path equals <code>/login</code> and Request Method equals <code>POST</code>
Expression	<code>http.host eq "example.com" and http.request.uri.path eq "/login" and http.request.method eq "POST"</code>
Counting characteristics	IP
Increment counter when	URI Path equals <code>/login</code> and Request Method equals <code>POST</code> and Response Status Code is in (401, 403)
Counting expression	<code>http.request.uri.path eq "/login" and http.request.method eq "POST" and http.response.code in {401 403}</code>
Rate (Requests/Period)	10 requests / 10 minutes
Action	Managed Challenge

Rule 2 configuration

Rule 3: Strict protection threshold

Rule 3 enforces a stronger penalty to clients exceeding the rule 2 threshold by blocking an IP address for one day after 20 failed login attempts within an hour. This rule provides a final defense against persistent attack attempts.

Setting	Value
Matching criteria	Host equals <code>example.com</code>
Expression	<code>http.host eq "example.com"</code>
Counting characteristics	IP
Increment counter when	URI Path equals <code>/login</code> and Request Method equals <code>POST</code> and Response Status Code is in (401, 403)
Counting expression	<code>http.request.uri.path eq "/login" and http.request.method eq "POST" and http.response.code in {401 403}</code>
Rate (Requests/Period)	20 requests / 1 hour
Action	Block for 1 day

Rule 3 configuration



Important: All these example rules require a Business plan or higher.

These three rules have a counting expression separate from the rule expression (also known as mitigation expression). When you configure a separate counting expression, the matching criteria is used when an action is triggered. In the counting expression, you can include conditions based on the HTTP response status code and HTTP response headers to integrate the rate limiting with your backend logic.

You can also decide to have two different expressions: a counting expression and a rule/mitigation expression — to define:

1. The requests used to compute the rate.
2. The requests actually acted upon.

For example, Rule 3 example computes the rate considering `POST` requests to `/login` that returned a `401` or `403` HTTP status code. However, when the rate limit is exceeded, CIS blocks every request to the `example.com` host generated by the same IP.

Limiting the number of operations

You can use rate limiting to control how many operations a client performs within a specific time period. The rules that you configure depend on your application’s behavior and risk profile.

The following examples show how to prevent content scraping and automated activity that can overload your system or misuse your data. Examples include limiting requests by query string, JSON body parameters, or bot characteristics.

Preventing content scraping by using query string

In this example, clients perform operations (such as price lookups or adding items to a basket) on an e-commerce website through query string parameters. For example, a typical request that is sent by a client might be similar to the following:

```
$ GET https://store.com/merchant?action=lookup_price&product_id=215
Cookie: session_id=12345
```

Your security team might want to consider setting up a limit on the number of times a client can lookup prices to prevent bots — which might have eluded CIS Bot Management — from scraping the store's entire catalog.

Setting	Value
Matching criteria	URI Path equals <code>/merchant</code> and URI Query String contains <code>action=lookup_price</code>
Expression	<code>http.request.uri.path eq "/merchant" and http.request.uri.query contains "action=lookup_price"</code>
Counting characteristics	IP
Rate (Requests/Period)	10 requests / 2 minutes

Action	Managed Challenge
Rule 1 configuration	
Setting	Value
Matching criteria	URI Path equals <code>/merchant</code> and URI Query String contains <code>action=lookup_price</code>
Expression	<code>http.request.uri.path eq "/merchant" and http.request.uri.query contains "action=lookup_price"</code>
Counting characteristics	IP
Rate (Requests/Period)	20 requests / 5 minutes
Action	Block

Rule 2 configuration	
----------------------	--

These two rate limiting rules match requests performing a selected action (look up price, in this example) and use `IP` as the counting characteristic. Similarly, to the [/login example](#), the two rules help reduce false positives in case of persistent (but legitimate) visitors.

You can limit the lookup of a specific `product_id` by using a query string parameter. By adding a query parameter as a counting characteristic, the rate is calculated across all requests, regardless of the client.

The following example limits the number of lookups for each `product_id` to 50 requests in 10 seconds.

Setting	Value
Matching criteria	URI Path equals <code>/merchant</code>
Expression	<code>http.request.uri.path eq "/merchant"</code>
Counting characteristics	Query (<code>product_id</code>)
Rate (Requests/Period)	20 requests / 10 seconds
Action	Block

Rule limit via query string	
-----------------------------	--

 **Note:** This example rule requires Advanced Rate Limiting.

You might follow the same pattern of rate limiting rules to protect applications handling reservations and bookings.

Preventing content scraping by using request body

Consider an application that handles the operation and its parameters through the request body in JSON format. For example, the `lookup_price` operation might look like the following:

```
$ POST https://api.store.com/merchant
Cookie: session_id=12345

Body:
{
  "action": "lookup_price",
  "product_id": 215
}
```

In this scenario, you can create a following rule to limit the number of actions from individual sessions:

Setting	Value
---------	-------

Matching criteria	URI Path equals <code>/merchant</code> and JSON String <code>action</code> equals <code>lookup_price</code>
Expression	<code>http.request.uri.path eq "/merchant" and lookup_json_string(http.request.body.raw, "action") eq "lookup_price"</code>
Counting characteristics	Cookie (<code>session_id</code>)
Rate (Requests/Period)	10 requests / 2 minutes
Action	Managed Challenge

Rule—Limit lookup actions per session



Note: This example rule requires Advanced Rate Limiting and payload inspection.

You can also limit the number of lookups of each `product_id` regardless of the client making the requests by deploying a rule like the following:

Setting	Value
Matching criteria	URI Path equals <code>/merchant</code> and JSON field <code>action</code> equals <code>lookup_price</code>
Expression	<code>http.request.uri.path eq "/merchant" and lookup_json_string(http.request.body.raw, "action") eq "lookup_price"</code>
Counting characteristics	JSON field (<code>product_id</code>)
Rate (Requests/Period)	50 requests / 10 seconds
Action	Block

Rule—Limit lookup requests per product ID

This example rule requires Advanced Rate Limiting and payload inspection.



Important: If the request body is not JSON format, you can use the [http.request.body.raw](#) field and regular expressions (along with the [matches operator](#)) to achieve the same goal.

Limiting requests from bots

You can use rate limiting to control automated traffic from bots. A common approach is to monitor requests that return a high number of `403` or `404` response status codes, which often indicate automated scraping activity.

In this situation, you might configure a rule similar to the following:

Setting	Value
Matching criteria	Hostname equals <code>example.com</code>
Expression	<code>http.host eq "example.com"</code>
Counting characteristics	IP
Increment counter when	Response Status Code is in (401, 403)
Counting expression	<code>http.response.code in {401 403}</code>
Rate (Requests/Period)	5 requests / 3 minutes
Action	Managed Challenge

Rule—Limit requests based on response status codes



Note: This example rule requires a Business plan or higher.

To control the rate of actions performed by automated sources, consider use rate limiting rules together with [Bot Management](#). With Bot Management, you can use the [bot score](#) as part of the matching criteria to apply the rule only to automated or likely automated traffic. For example, you can use a maximum score (or threshold) of `30` for likely automated traffic and `10` for automated traffic.

You can enhance protection by combining rate limiting with [Bot Management](#). With Bot Management, you can use the [bot score](#) as part of the matching criteria to apply the rule only to automated or likely automated traffic.

For example:

- A bot score lesser than `30` represents likely automated traffic.
- A bot score lesser than `10` represents confirmed automated traffic.

Limiting requests per session

If your application uses session cookies, use the cookie as the counting characteristic. This method groups requests from different IPs within the same session—useful for detecting distributed bot attacks.

Rule 1

Setting	Value
Matching criteria	Bot Score less than 30 and URI Query String contains <code>action=delete</code>
Expression	<code>cis.bot_management.score lt 30 and http.request.uri.query contains "action=delete"</code>
Counting characteristics	Cookie (<code>session_id</code>)
Rate (Requests/Period)	10 requests / 1 minute
Action	Managed Challenge
Limit request for bot score less than 30	

Rule 2

Setting	Value
Matching criteria	Bot Score less than 10 and URI Query String contains <code>action=delete</code>
Expression	<code>cis.bot_management.score lt 10 and http.request.uri.query contains "action=delete"</code>
Counting characteristics	Cookie (<code>session_id</code>)
Rate (Requests/Period)	20 requests / 5 minutes
Action	Block
Limit request for bot score less than 10	



Note: These example rules require Advanced Rate Limiting and Bot Management.

Using JA3 fingerprints

If the application does not use a session cookie, you can use JA3 fingerprints to identify individual clients. A [JA3 fingerprint](#) is a unique identifier, available to customers with [Bot Management](#), that allows CIS to identify requests coming from the same client. All clients have an associated fingerprint, whether they are automated or not.

Setting	Value
---------	-------

Matching criteria	URI Path equals <code>/merchant</code> and Bot Score less than 10
Expression	<code>http.request.uri.path eq "/merchant" and cf.bot_management.score lt 10</code>
Counting characteristics	JA3 Fingerprint
Rate (Requests/Period)	10 requests / 1 minute
Action	Managed Challenge

Limit request by using JA3 Fingerprint

 **Note:** This example rule requires Advanced Rate Limiting and Bot Management.

Protecting REST APIs

REST APIs can create high load on backend systems because API requests often require intensive processing or large data lookups. Uncontrolled API access can cause performance degradation or even downtime. Use Advanced Rate Limiting to prevent abuse, mitigate volumetric attacks, and protect critical resources.

Protecting resources

Even `GET` requests can strain the application or consume bandwidth when used for large data downloads, such as files or images.

For instance, consider the following endpoint:

```
$ GET https://api.store.com/files/<FILE_ID>
Header: x-api-key=9375
```

To prevent abuse while allowing legitimate downloads, you can define a rule that limits file requests without writing separate rules for each file.

Setting	Value
Matching criteria	Hostname equals <code>api.example.com</code> and Request Method equals <code>GET</code>
Expression	<code>http.host eq "api.example.com" and http.request.method eq "GET"</code>
Counting characteristics	Path
Rate (Requests/Period)	As suggested by API Discovery or assessed by analyzing past traffic.
Action	Block

Rule — Limit file downloads by path

 **Note:** This example rule requires Advanced Rate Limiting.

This rule limits downloads to 10 requests per 10 minutes per file under `https://api.store.com/files/*`. By using Path as the counting characteristic, you can avoid creating new rules for every new `<FILE_ID>`. The rate is calculated per file, regardless of client IP or session ID.

You can further strengthen protection by combining `Path` with a client identifier such as `x-api-key` or `IP`. This approach allows you to restrict the number of downloads a specific client can make for a given file.

Setting	Value
Matching criteria	Hostname equals <code>api.store.com</code> and Request Method equals <code>GET</code>
Expression	<code>http.host eq "api.example.com" and http.request.method eq "GET"</code>

Counting characteristics	Path and Header (x-api-key)
Rate (Requests/Period)	As suggested by API Discovery or assessed by analyzing past traffic.
Action	Block

Rule — Limit file downloads by path and header



Note: This example rule requires Advanced Rate Limiting.

Protecting GraphQL APIs

Preventing server overload for GraphQL APIs can be different from preventing overload for RESTful APIs. One of the biggest challenges that are posed by applications that are built on GraphQL is that a single path manages all queries to the server, and every request is usually a **POST** operation. This prevents creating different rate limits for different API based on the HTTP method and URI path.

However, instead of using the method and path like a RESTful API, the purpose of the request is usually embedded in the body, which has information on what data the client wants to fetch or mutate (according to [GraphQL's terminology](#) for server-side data modification), along with any additional data that is required to carry out the action.

To prevent server overload, consider the following approaches:

- Limit the number of times a particular user can call the same GraphQL operation name.
- Limit the total amount of query complexity any given user is allowed to request.
- Limit any individual request's query complexity.

The following examples are based on an application that accepts reviews for movies.

```
$ POST https://moviereviews.example.com/graphql
Cookie: session_id=12345

Body:
{
  "data": {
    "createReview": {
      "stars": 5,
      "commentary": "This is a great movie!"
    }
  }
}
```

Limiting the number of operations

To limit the rate of actions, create the following rule:

Setting	Value
Matching criteria	URI Path equals /graphql and Body contains createReview
Expression	http.request.uri.path eq "/graphql" and http.request.body.raw contains "createReview"
Counting characteristics	Cookie (session_id)
Rate (Requests/Period)	5 requests / 1 hour
Action	Block

Limit the number of operations



Note: This example rule requires Advanced Rate Limiting and payload inspection.

Limiting the total amount of query complexity

The complexity of handling a GraphQL request can vary significantly. Because the API uses a single endpoint, it can be difficult to determine the complexity of each request before it is processed.

To prevent resource exhaustion on the origin server, limit the total request complexity per client over time, rather than limiting the number of requests. CIS Rate Limiting enables you to create rules that track complexity over time and block requests that exceed a defined complexity budget.

This method requires the origin server to assign a complexity score to each request and include that score in the HTTP response header. The rate-limiting mechanism then uses the score information to update the complexity budget for that specific client.

The following example defines a total complexity budget of 1,000 per hour:

Setting	Value
Matching criteria	URI Path contains <code>/graphql</code>
Expression	<code>http.request.uri.path eq "/graphql"</code>
Counting characteristics	Cookie (<code>session_id</code>)
Score per period	1,000
Period	1 hour
Response header name	<code>score</code>
Action	Block

Limit the total amount of query complexity

Note: This example rule requires Advanced Rate Limiting and payload inspection.

When the origin server processes a request, it adds a `score` HTTP header to the response with a value which indicates how much work the origin has performed to handle it. For example, `100`. In the next hour, the same client can perform requests up to an additional budget of `900`. As soon as this budget is exceeded, later requests are blocked until the timeout expires.

Limiting any individual query’s complexity

API Shield customers can use GraphQL malicious query protection to protect their GraphQL APIs. This feature scans incoming GraphQL traffic for queries that might overload the origin server and cause a denial-of-service condition.

You can create rules to limit the depth and size of incoming GraphQL queries. These rules help block suspicious or excessively complex queries before they impact performance.

Using the CIS Security Events capability

Reviewing Security Events gives you insight into your web traffic and into any potentially malicious activity against your website. Reviewing Security Events also helps you optimize your WAF configuration.

The CIS Security Events table

The Security Events table shows you information about web requests that are blocked by the WAF. Each entry shows one blocked request.

- **Triggered rule** indicates which rule blocked the request. Any of the following actions is available:
 - **Block**: A hard block.
 - **Challenge**: A CAPTCHA page that humans can bypass.
 - **Simulate**: A request that is allowed through normally, but is logged.

Sometimes the triggered rule is not identified. In this case, the UI shows a  instead of the rule ID.

- **IP Address**: Shows the source IP address of the web request.
- **Location**: Shows the country that is associated with the source IP of the web request.
- **Host**: Shows the hostname of the server that the web request reaches.
- **Date**: Shows the day that the event occurred.

CIS Security Events details

When viewing Security Events, you can click the arrow on an event to expand the details for that event. One section of your page shows the event details, along with the Ray-Id. The other section shows request details such as Header, URI, Protocol, the type of firewall that blocked the request, and User Agent.

Suppose, for example, you see that the triggered rule for an event has an ID of `981176`. This means that the block was caused by OWASP. When any rules in the OWASP ruleset is matched, the “threat score” of the request increases. The **Paranoia level** setting (`P1` to `P4`) for your zone translates to a threshold. If the cumulative score of all the matched OWASP rules exceeds that threshold, rule `981176` is triggered and blocks the request.

This means that all requests blocked by OWASP show on your Security Events as blocked by `981176`. Expand the event details and view the **Event Triggers** section to see the individual OWASP rules that matched to increase the request’s threat score.

What do I do if valid traffic is blocked?

Expand each event to see event details. The **Event Triggers** section displays all the individual OWASP rules that matched for OWASP rule-triggered events. Decide whether this traffic looks normal for your website, or if it was appropriately blocked. If you decide that this block is a false positive, you can go back to your WAF configuration and disable individual OWASP rules until this request no longer exceeds your sensitivity threshold.

Security event retention policy

Enterprise plans retain security events for 90 days. Standard plans retain security events for 30 days.

Known limitations

CIS Security Events currently has the following limitations:

- Security Events uses [sampled data](#). As a result, the dashboard might not display all events.
- Security Events are pulled using GraphQL. As a result, it is subject to the [GraphQL query limits](#).
- The IBM Cloud console displays up to 10,000 events per query. Beyond this limit, additional events are not displayed.

Release notes for CIS

Use these release notes to learn about the latest updates to IBM Cloud® Internet Services that are grouped by date.

25 August 2026

Table improvements

The follow tables have been updated:

- **DNS records** - The **Proxy** is now available within the Create/Edit record flow. The table will continue to show the Proxy status of records and to display the Proxy status of each record. Optionally, you can add a **Last modified** column using the table settings (⚙️).
- **Logpush jobs** - A new **Delivery health** column was added to the table to show the health status of each job. The **Last successful delivery** and **Last failed delivery** columns remain available and can be added to the table using the table settings (⚙️).

28 July 2026

Updates to the Plan and Getting started pages

Updated the Plan and Getting started pages with improved page logic and refreshed visuals to enhance usability and provide a more streamlined onboarding and planning experience.

Table improvements

Updated the DNS records, Alert policies, and Security events tables to improve how users view, search, filter, and manage information. These updates provide a more streamlined table experience with improved navigation and filtering capabilities, helping users find and manage data more efficiently.

16 April 2026

UI support for Logpush Cloud Object Storage job authentication update

Added UI support for creating Logpush jobs with IBM Cloud Object Storage destinations using IBM Cloud API key authentication. Cloud Object Storage destinations no longer require Logpush ownership validation and now use an API key with the Object Writer IAM role, consistent with the model used for IBM Cloud Logs jobs. The API key can be updated in the UI when editing the Logpush job. For more information, see [Managing Logpush jobs](#).

10 March 2026

API and CLI support for Logpush Cloud Object Storage job authentication update

Added API and CLI support for creating Logpush jobs with IBM Cloud Object Storage destinations using IBM Cloud API key authentication. Cloud Object Storage destinations no longer require Logpush ownership validation and now use an API key with the Object Writer IAM role, consistent with the model used for IBM Cloud Logs jobs. For more information, see [Managing Logpush jobs](#).

03 December 2025

Custom and managed lists

Released UI and API support for custom lists and managed lists to enhance control over security and traffic management policies.

This release introduces a new Lists tab on the Account page, enabling users to create and manage custom lists or use CIS-managed lists directly

from the CIS console. CLI and Terraform support for lists were previously released in July, and full API support is also now available (see the [API change log](#) for details).

- [Creating custom lists and adding items.](#)
- [Managed lists](#) provide preconfigured threat intelligence and reputation data that is maintained by IBM to simplify policy enforcement. The specific managed lists available depend on your subscription plan.

These features enable more accurate policy targeting and improve operational efficiency within CIS. For more information, see [Working with lists](#).

31 July 2025

X-Request-ID Header for improved CLI request tracing

The CIS CLI now includes an automatically generated `X-Request-ID` header with every API request. This unique UUID enhances traceability and debugging by enabling better correlation of CLI calls across back-end systems. No user action is needed, and this update is fully transparent, with no impact on CLI functionality or output.

For more information, see [How can I help IBM Support trace my CLI API request?](#).

28 February 2025

Ruleset Engine migration

Two existing features are now part of the Ruleset Engine rules language, and they are updated to use the [Migrating to WAF custom rules](#).

- Rate-limiting rules: The new version of these rules allows you to apply rate-limiting to more specific kinds of traffic. For more information, see [Migrating to new rate-limiting rules](#).

13 January 2025

QRadar and Splunk Logpush destinations

Added support for configuring CIS Logpush jobs to push logs directly to QRadar and Splunk instances. For more information, see [Managing Logpush jobs](#).

05 December 2024

IBM Cloud Logs and generic Logpush destinations

Added support for configuring CIS Logpush jobs to push logs to IBM Cloud Log (ICL) instances and generic destinations. For more information, see [Managing Logpush jobs](#).

21 October 2024

Enhanced security

Released support for [SSL.com as a Certificate Authority](#), offering you more trusted SSL certificate options.

12 June 2024

Managed Rules

Released the new [managed rules](#) feature, which replaces the current WAF. Enterprise plans have a [Migration to managed rules](#) that allows an intermediary step where you can compare security events. Standard Next plans migrate directly to Managed rules without the intermediary review step.

21 June 2023

Enterprise Tier plans released

Released the new [Enterprise Tier plans](#), which replaces the current Enterprise Package, Enterprise GLB, and Enterprise Security plans.

16 March 2023

Standard Next plan released

Released the new [Standard Next plan](#), which replaces the current Standard plan.

Solution tutorials

Resilient and secure multi-region Kubernetes clusters with IBM Cloud Internet Services

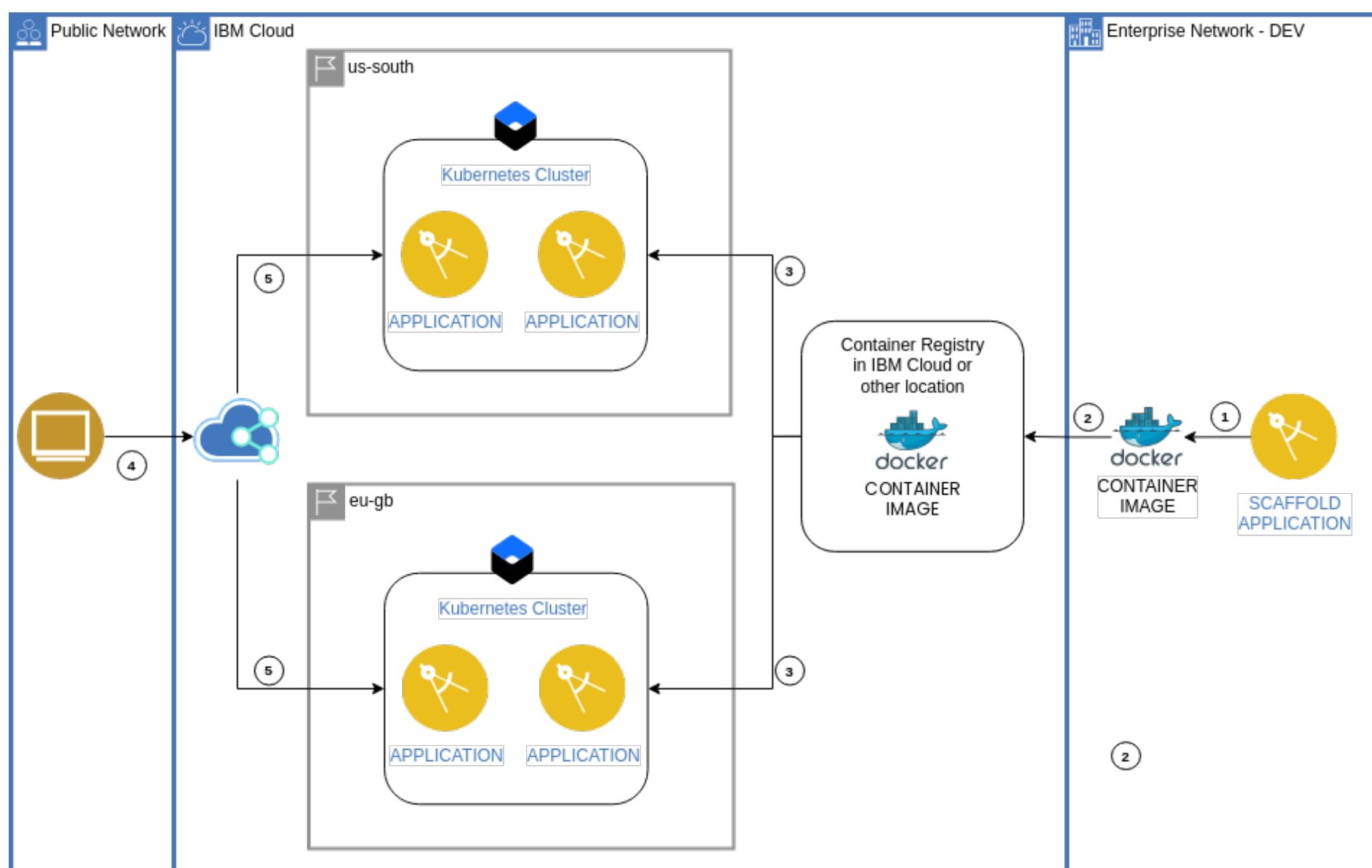
 **Tip:** This tutorial may incur costs. Use the [Cost Estimator](#) to generate a cost estimate based on your projected usage.

Users are less likely to experience downtime when an application is designed with resiliency in mind. When implementing a solution with Kubernetes Service, you benefit from built-in capabilities, like load balancing and isolation, increased resiliency against potential failures with hosts, networks, or apps. By creating many clusters and if an outage occurs with one cluster, users can still access an app that is also deployed in another cluster. With many clusters in different locations, users can also access the closest cluster and reduce network latency. For additional resiliency, you have the option to also select the multi-zone clusters, meaning your nodes are deployed across many zones within a location.

This tutorial highlights how Cloud Internet Services (CIS), a uniform platform to configure and manage the Domain Name System (DNS), Global Load Balancing (GLB), Web Application Firewall (WAF), and protection against Distributed Denial of Service (DDoS) for internet applications, can be integrated with Kubernetes clusters to support this scenario and to deliver a secure and resilient solution across many locations.

Objectives

- Deploy an application on many Kubernetes clusters in different locations.
- Distribute traffic across many clusters with a Global Load Balancer.
- Route users to the closest cluster.
- Protect your application from security threats.
- Increase application performance with caching.



Architecture diagram of the tutorial

1. The developer builds a Docker image for the application.
2. The image is pushed to a Container Registry.
3. The application is deployed to Kubernetes clusters in Dallas and London.
4. End-users access the application.
5. IBM Cloud Internet Services is configured to intercept requests to the application and to distribute the load across the clusters. In addition, DDoS Protection and Web Application Firewall are enabled to protect the application from common threats. Optionally assets like images, CSS files are cached.

Before you begin

This tutorial requires:

- IBM Cloud CLI,
 - IBM Cloud Kubernetes Service plugin (`kubernetes-service`),
- `kubectl` to interact with Kubernetes clusters,

You will find instructions to download and install these tools for your operating environment in the [Getting started with solution tutorials](#) guide.

In addition, make sure you:

- own a custom domain so you can configure the DNS for this domain to point to IBM Cloud Internet Services name servers.
- and [understand the basics of Kubernetes](#).

Step 1: Deploy an application to one location

This tutorial deploys a Kubernetes application to clusters in many locations. You will start with one location, Dallas, and then repeat these steps for London.

Create a Kubernetes cluster

A minimal cluster with one (1) zone, one (1) worker node is sufficient for this tutorial.

When creating the following Kubernetes cluster:

1. Set **Cluster name** to **my-us-cluster**.
2. Locate in **North America** and **Dallas**
3. Open the [Kubernetes clusters](#) and click **Create cluster**.
4. Create a cluster on your choice of **Infrastructure**.
 - The following steps are if you select **VPC** for Kubernetes on VPC infrastructure. You are required to create a VPC and subnet(s) before creating the Kubernetes cluster. Reference the [Creating VPC clusters](#) documentation for more details.
 1. Click **Create VPC**.
 2. Under the **Location** section, select a **Geography** and **Region**, for example `North America` and `Dallas`.
 3. Enter a **Name** of your VPC, select a **Resource group** and optionally, add **Tags** to organize your resources.
 4. Uncheck **Allow SSH** and **Allow ping** from the **Default security group**.
 5. Uncheck **Create subnet in every zone**.
 6. Click on **Create**.
 7. Under **Worker zones and subnets**, uncheck the two zones for which the subnet wasn't created.
 8. Set the **Worker nodes per zone** to `1` and click on **Change flavor** to explore and change to the worker node flavor of your choice.
 9. Under **Ingress**, enable **Ingress secrets management** and select your existing Secrets Manager instance.
 10. Enter a **Cluster name** and select the same **Resource group** that you used for the VPC.
 11. Logging or Monitoring aren't required in this tutorial, disable those options and click on **Create**.
 12. While you waiting for the cluster to become active, attach a public gateway to the VPC. Navigate to the [Virtual private clouds](#).
 13. Click on the name for the VPC used by the cluster and scroll down to subnets section.
 14. Click on the name of the subnet created earlier and in the **Public Gateway** section, click on **Detached** to change the state to **Attached**.
 - The following steps are if you select **Classic** for Kubernetes on Classic infrastructure. Reference the [Creating a standard classic cluster](#) documentation for more details.
 1. Under the **Location** section, select a **Geography**, multizone **Availability**, and **Metro** for example `North America` and `Dallas`.
 2. Under **Worker zones and VLANs**, uncheck all zones except for one.
 3. Set the **Worker nodes per zone** to `1` and click on **Change flavor** to explore and change to the worker node flavor of your choice.
 4. Under **Master service endpoint**, select **Both private & public endpoints**.
 5. Under **Ingress**, enable **Ingress secrets management** and select your existing Secrets Manager instance.
 6. Enter a **Cluster name** and select the **Resource group** to create these resources under.
 7. Logging or Monitoring aren't required in this tutorial, disable those options and click on **Create**.

While the cluster is getting ready, you are going to prepare the application.

Deploy the application to the Kubernetes cluster

The cluster should be ready. You can check its status in the [Kubernetes Service](#) console.

1. Gain access to your cluster as described on the Access tab of your cluster. Something like:

```
$ MYCLUSTER=my-us-cluster
ibmcloud ks cluster config --cluster $MYCLUSTER
```

2. Create the deployment using a pre-built image of the application. The application source code can be found in this [GitHub repository](#).

```
$ kubectl create deploy hello-world-deployment --image=icr.io/solution-tutorials/tutorial-scalable-webapp-kubernetes
```

Example output: `deployment "hello-world-deployment" created` .

3. Make the application accessible within the cluster by creating a service:

```
$ kubectl expose deployment/hello-world-deployment --type=ClusterIP --port=80 --name=hello-world-service --target-port=3000
```

It returns message like `service "hello-world-service" exposed` . To see the services:

```
$ kubectl get services
```

4. Run the application in the cluster with two replicas:

```
$ kubectl scale deployment hello-world-deployment --replicas=2
```

5. You can check the status of the deployment with the following command:

```
$ kubectl get pods
```

Get the Ingress Subdomain assigned to the cluster

When a Kubernetes cluster is created, it gets assigned an Ingress subdomain (for example, *my-us-cluster.us-south.containers.appdomain.cloud*) and a public Application Load Balancer IP address.

1. Retrieve the Ingress subdomain of the cluster:

```
$ ibmcloud ks cluster get --cluster $MYCLUSTER
```

Look for the `Ingress Subdomain` value.

2. Make note of this information for a later step.

This tutorial uses the Ingress Subdomain to configure the Global Load Balancer. You could also replace the Ingress Subdomain with the public Application Load Balancer, ALB of the cluster. An `<IngressSubdomain>` looks something like `my-us-cluster-e7f2ca73139645ddf61a8702003a483a-0000.us-south.containers.appdomain.cloud`

Configure the Ingress for your DNS subdomain

It will be required to have your own DNS domain name and a global load balancer subdomain will be created in the following task: `<glb_name>`. `<your_domain_name>` . Something like `hello-world-service.example.com` `<glb_name> = hello-world-service` and `<your_domain_name> = example.com`

1. Create the file `glb-ingress.yaml` and replace the placeholders with their respective values:

```
$ apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: <glb-name>
  annotations:
    spec.ingressClassName: "public-iks-k8s-nginx"
spec:
  rules:
  - host: <glb-name>.<your_domain_name>
    http:
      paths:
      - path: /
        pathType: Prefix
      backend:
```



```
service:
  name: hello-world-service
  port:
    number: 80
```

2. Add the ingress instance:

```
$ kubectl apply -f glb-ingress.yaml
```

It can take a few minutes before ingress becomes available as indicated by a value in the ADDRESS column in the command:

```
$ kubectl get ingress
```

3. Now test by configuring the curl **Host** http header with your DNS subdomain name to override the default of `<IngressSubdomain>`:

```
$ curl --header 'Host: <glb_name>.<your_domain_name>' <IngressSubdomain>/hostname
```

The curl command would look something like this: `curl --header 'Host: hello-world-service.ibmom.com' my-us-cluster-e7f2ca73139645ddf61a8702003a483a-0000.us-south.containers.appdomain.cloud/hostname`

Step 2: And then to another location

Repeat the previous steps for the London location with the following replacements:

- In [Create a Kubernetes cluster](#) replace:
 - the cluster name **my-us-cluster** with **my-uk-cluster**;
 - the location from **North America** and **Dallas** with **Europe** and **London**.
- In the [Deploy the application to the Kubernetes cluster](#) replace:
 - Replace the MYCLUSTER= **my-us-cluster** with **my-uk-cluster**
- [Configure the Ingress for the DNS subdomain](#)

Step 3: Configure multi-location load-balancing

Your application is now running in two clusters but it is missing one component for the users to access either clusters transparently from a single entry point.

In this section, you will configure IBM Cloud Internet Services (CIS) to distribute the load between the two clusters. CIS is a one-stop shopping service providing *Global Load Balancer (GLB)*, *Caching*, *Web Application Firewall (WAF)* and *Page rule* to secure your applications while ensuring the reliability and performance for your Cloud applications.

To configure a global load balancer, you will need:

- to point a custom domain to CIS name servers,
- to retrieve the Ingress Subdomain of the Kubernetes clusters,
- to configure health checks to validate the availability of your application,
- and to define origin pools pointing to the clusters.

Register a custom domain with IBM Cloud Internet Services

The first step is to create an instance of CIS and to point your custom domain to CIS name servers.

1. If you don't own a domain, you can buy one from a registrar.
2. Navigate to [IBM Cloud Internet Services](#) in the IBM Cloud catalog.
3. Pick a plan, set the service name and resource group, and click **Create** to create an instance of the service.
4. When the service instance is provisioned, click on **Add domain**.
5. Enter your domain name and click **Next**.
6. Setup your DNS records is an optional step and can be skipped for this tutorial. click on **Next**.
7. When the name servers are assigned, configure your registrar or domain name provider to use the name servers listed.
8. At this point you can click on **Cancel** to get back to the main page, after you've configured your registrar or the DNS provider, it may require up to 24

hours for the changes to take effect.



Tip: When the domain's status on the Overview page changes from *Pending* to *Active*, you can use the `dig <your_domain_name> ns` command to verify that the new name servers have taken effect.

Configure Health Check for the Global Load Balancer

Health Checks monitor responses to HTTP/HTTPS requests from origin pools on a set interval. They are used with origin pools to determine if the pools are still running properly.

1. In the IBM Cloud Internet Services dashboard, use the navigation menu to select **Reliability > Global Load Balancers**.
2. Select the **Health checks** tab and click **Create**.
 - a. Set **Name** to **hello-world-service**
 - b. Set **Monitor Type** to **HTTP**.
 - c. Set **Port** to **80**.
 - d. Set **Path** to **/**.
 - e. In the **Configure request headers (optional)** add Header name: `Host` and Value: `<glb_name>.<your_domain_name>`
 - f. Click **Create**.



Tip: When building your own applications, you could define a dedicated health endpoint such as `/healthz` where you would report the application state.

Define Origin Pools

A pool is a group of origin servers that traffic is intelligently routed to when attached to a GLB. With clusters in the United Kingdom and United States, you can define location-based pools and configure CIS to redirect users to the closest clusters based on the geographical location of the user requests.

One pool for the cluster in Dallas

1. Select the **Origin pools** tab and click **Create**.
2. Set **Name** to `US`.
3. Set **Origin Name** to `us-cluster`.
4. Set **Origin Address** to the kubernetes service `<IngressSubdomain>` printed by `ibmcloud ks cluster get --cluster $MYCLUSTER` for the US cluster.
5. Set **Health check** to the one created in the previous section.
6. Set **Health Check Region** to `Western North America`.
7. Click **Save**.

One pool for the cluster in London

1. Select the **Origin pools** tab and click **Create**.
2. Set **Name** to `UK`.
3. Set **Origin Name** to `uk-cluster`.
4. Set **Origin Address** to the kubernetes service `<IngressSubdomain>` printed by `ibmcloud ks cluster get --cluster $MYCLUSTER` for the UK cluster.
5. Set **Health check** to the one created in the previous section.
6. Set **Health Check Region** to `Western Europe`.
7. Click **Save**.

Create the Global Load Balancer

With the origin pools defined, you can complete the configuration of the load balancer.

1. Select the **Load balancers** tab and click **Create**.
2. Enter a name, `<glb_name>`, under **Name** for the Global Load Balancer. This name will also be part of your universal application URL (`http://<glb_name>.<your_domain_name>`), regardless of the location.
3. Under **Geo routes**, click **Add route**.
 - a. Select **Default** from the **Region** drop down.
 - b. Select the pool **US**.

c. Click **Add**.

Repeat the process to create the following:

Region	Origin Pool
Default	US
Western Europe	UK
Eastern Europe	UK
Northeast Asia	UK
Southeast Asia	UK
Western North America	US
Eastern North America	US

List of Geo routes to create

With this configuration, users in Europe and in Asia will be redirected to the cluster in London, users in US to the Dallas cluster. When a request does not match any of the defined route, it will be redirected to the pool(s) in the **Default** region.

4. Click **Create**

At this stage, you have successfully configured a Global Load Balancer with Kubernetes clusters across many locations. You can access the GLB URL `http://<glb_name>.<your_domain_name>/hostname` to view your application. Based on your location, you are redirected to the closest cluster or a cluster from the default pool if CIS was not able to map your IP address to a specific location.

Step 4: Secure the application

Turn the Web Application Firewall On

The Web Application Firewall(WAF) protects your web application against ISO Layer 7 attacks. Usually, it is combined with grouped rule-sets, these rule-sets aim to protect against vulnerabilities in the application by filtering out malicious traffic.

- 1. In the IBM Cloud Internet Services dashboard, navigate to **Security**, then on the **WAF**.
- 2. Ensure the WAF is **On**.
- 3. Click **OWASP Rule Set**. From this page, you can review the **OWASP Core Rule Set** and individually enable or disable rules. When a rule is enabled, if an incoming request triggers the rule, the global threat score will be increased. The **Sensitivity** setting will decide whether an **Action** is triggered for the request.
 - a. Leave default OWASP rule sets as it is.
 - b. Set **Sensitivity** to **Low**.
 - c. Set **Action** to **Simulate** to log all the events.
- 4. Click **CIS Rule Set**. This page shows additional rules based on common technology stacks for hosting websites.

☒ **Tip:** For a secured connection with HTTPS, you can either obtain a certificate from [Let's Encrypt](#) or through [IBM Cloud Secrets Manager](#).

Increase performance and protect from Denial of Service attacks

A distributed denial of service ([DDoS](#)) attack is a malicious attempt to disrupt normal traffic of a server, service, or network by overwhelming the target or its surrounding infrastructure with a flood of internet traffic. CIS is equipped to protect your domain from DDoS.

- 1. In the CIS dashboard, select **Reliability > Global Load Balancer**.
- 2. Locate the GLB you created in the **Load Balancers** table.
- 3. Enable the Security and Performance features in the **Proxy** column:

Load balancers

Origin pools

Health checks

Events

Load balancers

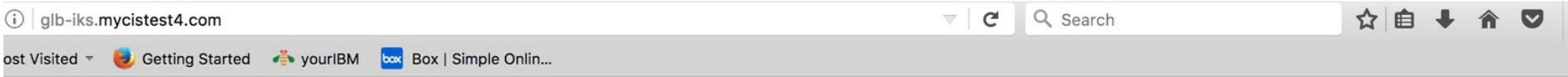
Global load balancers help distribute your proxied traffic across servers within an origin pool, and based on the pool's health, fail over to a different pool if needed.

Create +

Health	Hostname	Available pools	TTL	Proxy ⓘ	Enabled	
▼ ● Healthy	hello-world-service	1 of 1	Automatic	● ✓	● ✓	⋮

CIS Proxy Toggle ON

Your GLB is now protected. An immediate benefit is that the origin IP addresses of your clusters will be hidden from the clients. If CIS detects a threat for an upcoming request, the user may see a screen like this one before being redirected to your application:



● ● ●

Checking your browser before accessing mycistest4.com.

This process is automatic. Your browser will redirect to your requested content shortly.
Please allow up to 5 seconds...

[DDoS protection by Cloudflare](#)
Ray ID: 42395a1f812ea512

Verifying - DDoS protection

In addition, you can now control what content gets cached by CIS and how long it stays cached. Go to **Performance > Caching** to define the global caching level and the browser expiration. You can customize the global security and caching rules with **Page Rules**. Page Rules enable fine-grained configuration using specific domain paths. As example with Page Rules, you could decide to cache all contents under **/assets** for **3 days**:

Page Rule

URL match

.mydomain.com/assets/

Page rule will be placed at lowest priority.

Enabled

Off ● On

Rule Behavior

Setting

Set as

Cache Level	Cache everything	⊗
Edge Cache TTL	3 days	⊗

Add setting +

Page Rules

Step 5: Remove resources

Remove Kubernetes Cluster resources

1. Remove the Ingress, you can do so by running the following command:

```
$ kubectl delete -f glb-ingress.yaml
```

2. Remove the service, you can do so by running the following command:

```
$ kubectl delete service hello-world-service
```

3. Remove the deployment, you can do so by running the following command:

```
$ kubectl delete deployment hello-world-deployment
```

4. Delete the clusters if you created them specifically for this tutorial.

Remove CIS resources

1. Remove the GLB.
2. Remove the origin pools.
3. Remove the health checks.
4. Update the DNS for your custom domain.
5. Delete the CIS instance if you created it specifically for this tutorial.

Related content

- [IBM Cloud Internet Services](#)
- [Manage your IBM CIS for optimal security](#)
- [Kubernetes Service](#)
- [Building containers from images](#)
- [Managing CIS for optimal security](#)

Deploy isolated workloads across multiple locations and zones



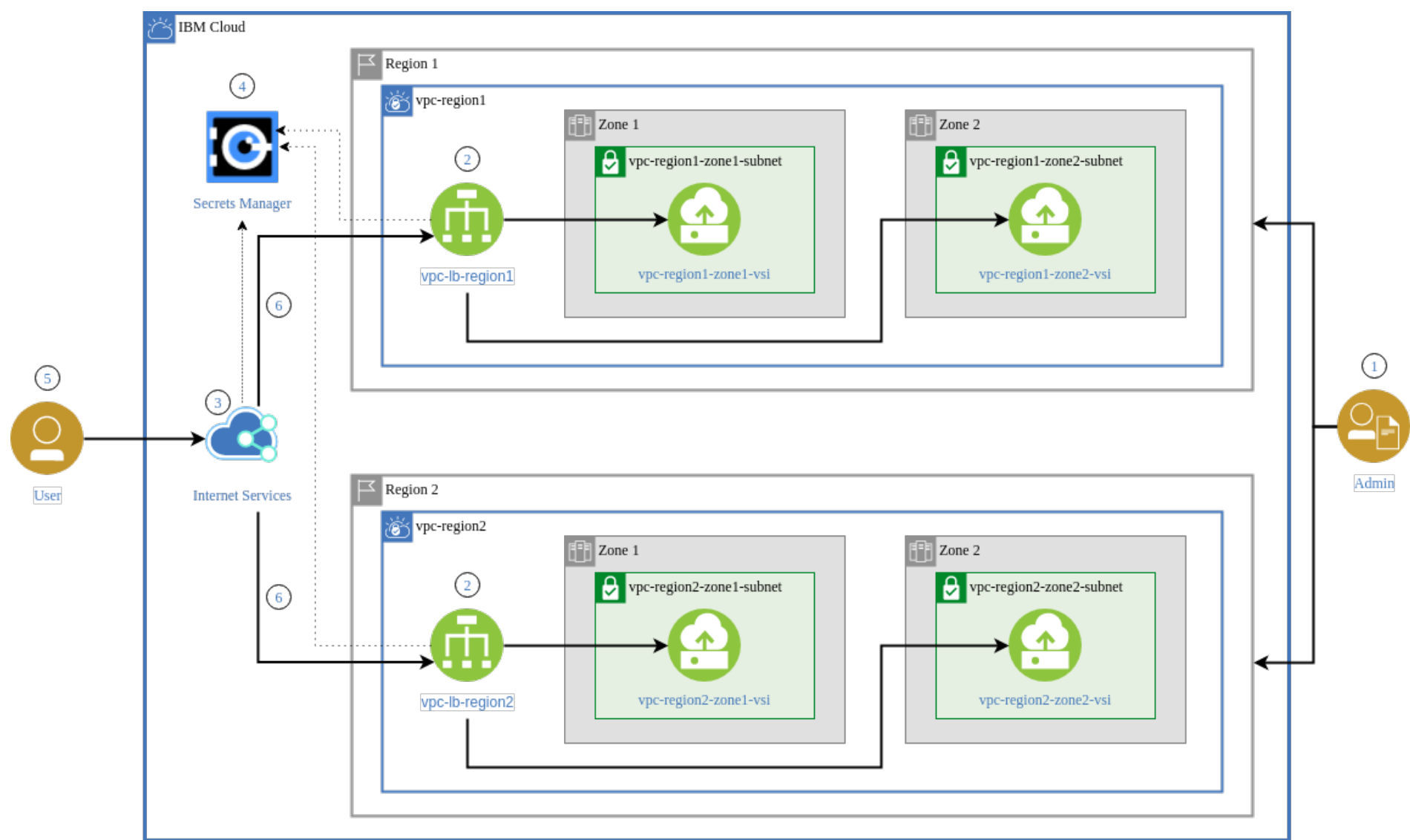
Tip: This tutorial may incur costs. Use the [Cost Estimator](#) to generate a cost estimate based on your projected usage.

This tutorial walks you through steps for setting up highly available and isolated workloads by provisioning IBM Cloud® Virtual Private Clouds (VPCs). You will create virtual server instances (VSIs) in multiple zones within one region to ensure the high availability of the application. You will create additional VSIs in a second region and configure a global load balancer (GLB) to offer high availability between regions and reduce network latency for users in different geographies.

You will provision an IBM Cloud Internet Services (CIS) service as the GLB from the catalog, and an IBM Cloud Secrets Manager service from the catalog to manage the Transport Layer Security (TLS) certificate for all incoming HTTPS requests.

Objectives

- Understand the isolation of workloads through infrastructure objects available for virtual private clouds.
- Use a load balancer between zones within a region to distribute traffic among virtual servers.
- Use a global load balancer between regions to implement high availability, increase resiliency and reduce latency.



Architecture diagram of the tutorial

1. The admin (DevOps) provisions VSIs in subnets under two different zones in a VPC in region 1 and repeats the same in a VPC created in region 2.
2. The admin creates a load balancer with a backend pool of servers in different zones of region 1 and a frontend listener. Repeats the same in region 2.
3. The admin provisions an IBM Cloud Internet Services instance with an associated custom domain and creates a global load balancer pointing to the load balancers created in two different VPCs.
4. The admin enables HTTPS encryption by adding the domain SSL certificate to the Secrets Manager service.
5. The user makes an HTTP/HTTPS request and the global load balancer handles the request.
6. The request is routed to the load balancers both on the global and local level. The request is then fulfilled by the available server instance.

Before you begin

- Check for user permissions. Be sure that your user account has sufficient permissions to create and manage VPC resources. See the list of [required permissions](#) for VPC.
- You need an SSH key to connect to the virtual servers. If you don't have an SSH key, see [the instructions](#) for creating a key for VPC.

Step 1: Create virtual private clouds, subnets and virtual server instances

In this section, you will create your own VPC in region 1 with subnets created in two different zones of region 1 followed by the provisioning of VSIs.

Create a virtual private cloud

1. Navigate to the [Virtual private clouds](#) page and click on **Create**.
2. Under the **Location** section, select a **Geography** and **Region**, for example **Europe** and **London**.
3. Enter `vpc-region1` for the name of your VPC, select a **Resource group** and optionally, add **Tags** to organize your resources.
4. Uncheck **Allow SSH** and **Allow ping** from the **Default security group**. SSH access will later be added to a maintenance security group. The maintenance security group is added to an instance to allow SSH access from a bastion server. Ping access is not required for this tutorial.
5. Leave **Enable access to classic resources** unchecked and **Create a default prefix for each zone** checked.
6. Under **Subnets** change the name of the Zone 1 subnet. Click the pencil icon:
 - Enter `vpc-region1-zone1-subnet` as your subnet's unique name.
 - Select the same **Resource group** as the VPC resource group.
 - Leave the defaults in the other values.
 - Click **Save**
7. Under **Subnets** change the name of the Zone 2 subnet. Click the pencil icon:
 - Enter `vpc-region1-zone2-subnet` as your subnet's unique name.
 - Select the same **Resource group** as the VPC resource group.

- Leave the defaults in the other values.
 - Click **Save**
- Under **Subnets** delete the subnet in Zone 3. Click the minus icon.
 - Click **Create virtual private cloud** to provision the instance.

Create a security group to allow inbound traffic to your application

Enable inbound rules for HTTP (80) and HTTPS (443) ports to the application by defining rules in a security group. In later steps, you will add VSIs to the security group.

- Navigate to the [Security groups](#) page and click on **Create**.
- Enter `vpc-region1-sg` for the name, select the same **Resource group** as the VPC resource group.
- Select the previously created **vpc-region1** virtual private cloud.
- Add the same **Inbound rules** as found in the table below and then click **Create security group**.

Protocol	Source type	Source	Value
TCP	Any	0.0.0.0/0	Ports 80-80
TCP	Any	0.0.0.0/0	Ports 443-443

Inbound rules

Provision virtual server instances

- Navigate to the [Subnets](#) page.
- Verify the status for all subnets is **Available**.
- Click on **vpc-region1-zone1-subnet** followed by **Attached resources**, then under **Attached instances** click on **Create**.
 - Enter `vpc-region1-zone1-vsi` as your virtual server's unique name.
 - Verify or set the **Virtual private cloud**, **Resource group**, **Location** and **Zone** fields.
- Under the **Image and profile** section, click on **Change image**.
- In the **Search items** field, type `Ubuntu` and pick any version of the image and click on **Save**.
- Under the **Profile** section, click on **Change profile**.
- Pick **Compute** with `2 vCPUs` and `4 GB RAM` as your profile and click on **Save**.
- Set **SSH keys** to the SSH key you created earlier.
- Under **Network attachments with Virtual network interface**, click on the pencil icon on the row for the **eth0** interface.
 - Click on **Next**.
 - Check `vpc-region1-sg` and uncheck the **VPC default** security group.
 - Click on **Next** a few times and then on **Save**.
- Click **Create virtual server**.
- Repeat the above steps to provision a **vpc-region1-zone2-vsi** virtual server in the **vpc-region1-zone2-subnet** subnet.

Step 2: Create resources in another location

Repeat the above steps from Step 1 to provision a new VPC with subnets and virtual server instances in another region, for example, **Frankfurt**. Follow the same naming conventions as above while substituting `region2` for `region1`.

Step 3: Install and configure web server on the virtual server instances



Tip: Follow the steps mentioned in [securely access remote instances with a bastion host](#) for secured maintenance of the servers. Use a bastion host which acts as a `jump` server and a maintenance security group to the VSIs previously provisioned. One bastion host in each VPC is required.

Once you successfully SSHed into the server provisioned in subnet of **zone 1** of **region 1**,

- At the prompt, run the following commands to install Nginx as your web server

```
sudo apt-get update
sudo apt-get install nginx
```


2. Check the status of the Nginx service with the following command:

```
sudo systemctl status nginx
```

The output should show you that the Nginx service is **active** and running.

3. Optionally verify that Nginx works as expected. `curl localhost`. You should see the default Nginx welcome page.

4. To update the html page with the region and zone details, run the below command

```
nano /var/www/html/index.nginx-debian.html
```

Append the region and zone to the `h1` tag quoting `Welcome to nginx!` to now read `Welcome to nginx! server running in **zone 1 of region 1**` and save the changes.

5. Verify the changes by running a `curl localhost` command.

6. Repeat the above steps to install and configure the web server on the VSIs in subnets of all the zones and don't forget to update the html to include the respective region and zone information.

Step 4: Distribute traffic between zones with load balancers

In this section, you will create two load balancers. One in each region to distribute traffic among multiple server instances under respective subnets within different zones.

Create a security group to allow inbound and outbound traffic through the load balancers

To allow traffic to the application, you need to enable inbound and outbound rules for HTTP (80) and HTTPS (443) ports. In later steps, when creating load balancers, you will add them to the security group defining those rules.

1. Navigate to the [Security groups](#) page and click on **Create**.
2. Select the previously created **vpc-region1** virtual private cloud.
3. Enter `vpc-lb-sg` for the name, select the same **Resource group** as the VPC resource group.
4. Add the same **Inbound rules** as found in the table below.

Protocol	Source type	Source	Value
TCP	Any	0.0.0.0/0	Ports 80-80
TCP	Any	0.0.0.0/0	Ports 443-443

Inbound rules

5. Add the same **Outbound rules** as found in the table below and then click **Create security group**.

Protocol	Source type	Source	Value
TCP	Any	0.0.0.0/0	Ports 80-80
TCP	Any	0.0.0.0/0	Ports 443-443

Outbound rules

6. Repeat the steps above in **region 2**.

Configure load balancers

1. Navigate to the [Load balancers](#) page and click **Create**.
2. Select **Application Load Balancer (ALB)** as the Load balancer type.
3. Under the **Location** section, select the same **Geography** and **Region** you used for the previously created **vpc-region1** virtual private cloud.
4. Enter `vpc-lb-region1` for the name, select the same **Resource group** as the VPC resource group.
5. Select the previously created **vpc-region1** virtual private cloud.
6. Select the **Subnets** of **vpc-region1-zone1-subnet** and **vpc-region1-zone2-subnet**.

7. Click **Create pool** to create a new back-end pool of VSIs that acts as equal peers to share the traffic routed to the pool. Set the parameters with the values below and click **Create** when done.
 - **Name:** region1-pool
 - **Protocol:** HTTP
 - **Session stickiness:** None
 - **Proxy protocol:** Disabled
 - **Method:** Round robin
 - **Health check path:** /
 - **Health protocol:** HTTP
 - **Health port:** Leave blank
 - **Interval(sec):** 15
 - **Timeout(sec):** 5
 - **Max retries:** 2
 - Click on **Create**.
8. Click **Attach server** to add server instances to the pool.
 - From the **Subnets** dropdown, select **vpc-region1-zone1-subnet** and **vpc-region1-zone2-subnet**.
 - Select the instances your created and set 80 as the **port**.
 - Click **Attach** to complete the creation of a back-end pool.
9. Click **Create listener** to create a new front-end listener; A listener is a process that checks for connection requests.
 - **Back-end pool:** region1-pool
 - **Protocol:** HTTP
 - **Proxy protocol:** not checked
 - **Port:** 80
 - **Maximum connections:** Leave it empty and click **Create**.
10. Under **Security Groups** check **vpc-lb-sg** and uncheck the default security group.
11. Click **Create load balancer** to provision the load balancer.
12. Repeat the steps above in **region 2**, this time naming the load balancer **vpc-lb-region2** and the back-end pool **region2-pool** .

Test the load balancers

1. Wait until the status of the load balancer changes to **Active**.
2. Open the **Hostname** in a web browser.
3. Refresh the page several times and notice the load balancer returning results from different zones or virtual server instances with each refresh.
4. Save the address for future reference.

You may have noticed that the requests are not encrypted and supports only HTTP. You will configure an SSL certificate and enable HTTPS in the next section.

Step 5: Configure multi-location load-balancing

Your application is now running in two regions, but it's missing one component for the users to access it transparently from a single entry point.

In this section, you will configure IBM Cloud Internet Services (CIS) to distribute the load between the two regions. CIS provides *Global Load Balancer (GLB)*, *Caching*, *Web Application Firewall (WAF)* and *Page rule* to secure your applications while ensuring the reliability and performance for your cloud applications.

To configure a global load balancer, you will need:

- to point a custom domain to CIS name servers,
- to retrieve the IP addresses or hostnames of the VPC load balancers,
- to configure health checks to validate the availability of your application,
- and to define origin pools pointing to the VPC load balancers.

Add a custom domain to IBM Cloud Internet Services

The first step is to create an instance of CIS and to point your custom domain to CIS name servers.

1. If you don't own a domain, you can buy one from a registrar.

2. Navigate to [IBM Cloud Internet Services](#) in the IBM Cloud catalog.
3. Pick a plan, set the service name and resource group, and click **Create** to create an instance of the service.
4. When the service instance is provisioned, click on **Add domain**.
5. Enter your domain name and click **Next**.
6. Setup your DNS records is an optional step and can be skipped for this tutorial. click on **Next**.
7. When the name servers are assigned, configure your registrar or domain name provider to use the name servers listed.
8. At this point you can click on **Cancel** to get back to the main page, after you've configured your registrar or the DNS provider, it may require up to 24 hours for the changes to take effect.



Tip: When the domain's status on the Overview page changes from *Pending* to *Active*, you can use the `dig <your_domain_name> ns` command to verify that the new name servers have taken effect.

Configure Health Check for the Global Load Balancer

A health check helps gain insight into the availability of pools so that traffic can be routed to the healthy ones. These checks periodically send HTTP/HTTPS requests and monitor the responses.

1. In the IBM Cloud Internet Services dashboard, navigate to **Reliability > Global Load Balancers**.
2. Select **Health checks** and click **Create**.
3. Set **Name** to `nginx`.
4. Select **HTTP** for **Monitor Type**.
5. Set **Port** to `80`.
6. Set **Path** to `/`.
7. Click **Create**.



Tip: When building your own applications, you could define a dedicated health endpoint such as `/health` where you would report the application state.

Define Origin Pools

A pool is a group of origin VSIs or load balancers that traffic is intelligently routed to when attached to a global load balancer. With VPC load balancers in two regions, you can define location-based pools and configure CIS to redirect users to the closest VPC load balancer based on the geographical location of the user requests.

Origin pool for the VPC load balancer

1. Select **Origin pools** and click **Create**.
2. Enter a name for the pool: `region-1-pool`.
3. Set **Origin Name** to `region-1`.
4. Set **Origin Address** to the hostname of region1 VPC load balancer, see the **overview page** of the VPC load balancer.
5. Select a **Existing health check** and select the health check created earlier.
6. Select a **Health check region** close to the location region 1.
7. Click **Save**.
8. Repeat the above steps for region 2.

Create the global load balancer

With the origin pools defined, you can complete the configuration of the load balancer.

1. Select **Load balancers** and click **Create**.
2. Keep the defaults of **Enable:** `On` and **Proxy:** `Off`.
3. Enter the name for the global load balancer, `lb`, this name will be the initial characters in the subdomain to access the application. (http://`lb`.`<your_domain_name>`).

4. Click **Add route**.
5. Select the **Region**: **Default**.
6. Select the origin pools that you just created, that is **region-1-pool** and **region-2-pool**.
7. Click **Add**.
8. Expand the section of **Geo routes**, you can distribute traffic based on the origin region.



Note: You can add more routes if desired based on geographies and direct traffic to the closest pool. Click **Add route**, select a global load balancer region for example, **Western Europe** and select the pool desired for example **region-2-pool** and click **Add**. A request does not match any of the defined route, it will be redirected to the **Default origin pools**, users in the global load balancer region you have define will be directed to the closest Load Balancers/VSIs.

9. Click **Create**.

Step 6: Secure with HTTPS

HTTPS encryption requires signed certificates to be accessible from both the CIS global load balancer and the VPC load balancers. The IBM Cloud Secrets Manager will be used to order or import and then manage the certificate for your domain. Identity and Access Management (IAM) service authorization is then configured to allow read access to the certificate from the desired service.

Create and authorize a Secrets Manager instance

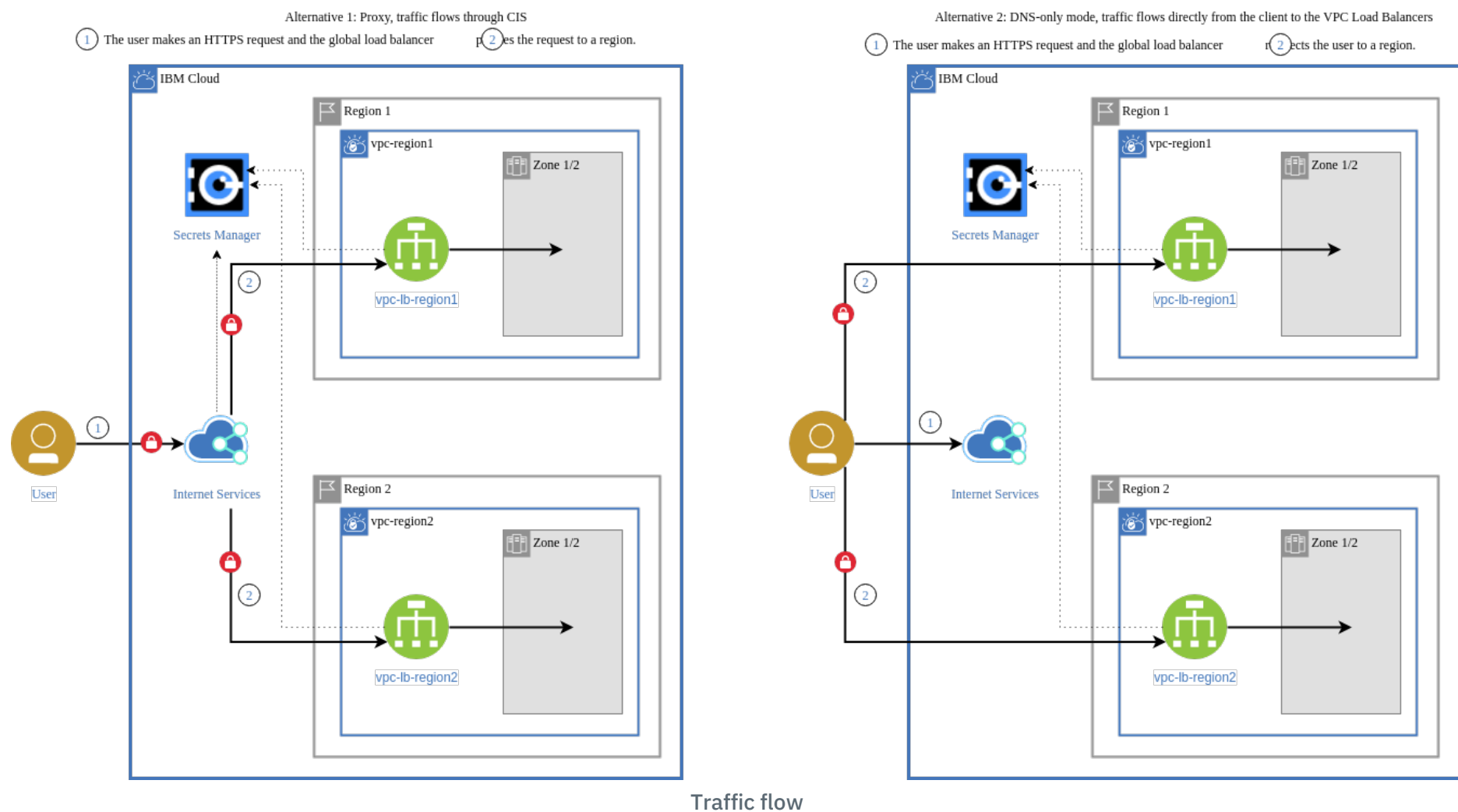
1. If you have an existing [Secrets Manager](#) instance, you can use it for this tutorial or create a new one if needed by following the steps outlined in [Creating a Secrets Manager service instance](#).
2. Create an authorization that gives the VPC load balancer service access to the Secrets Manager instance that contains the SSL certificate.
 - Navigate to [Identity and Access Authorizations](#).
 - Click **Create** and select **VPC Infrastructure Services** as the source service.
 - Select **Specific resources, Resource type** and then **Load Balancer for VPC**.
 - **Secrets Manager** as the **Target service**.
 - Assign the **Writer** service access role.
 - The target service instance may be **All resources**, or it may be your specific Secrets Manager instance if desired. Leaver the **All resources** selected for now.
 - Click on **Authorize**.
3. Continuing in the **Manage authorizations** page, create an authorization that gives the Secrets Manager access to CIS:
 - Click **Create** and choose **Secrets Manager** as the source service.
 - Choose **All resources** or just the Secrets Manager created earlier.
 - **Internet Services** as the target service.
 - Choose **All resources** or just the CIS created earlier.
 - Assign the **Manager** service access role.
 - Click on **Authorize**.



Tip: If your CIS instance supports multiple domains, you can also assign **Reader** role to the CIS instance and **Manager** to the specific domain that is you are using for your solution. See [granting service access to specific domains](#) topic.

IBM CIS supports proxying for global load balancers. When a load balancer is proxied, it means that its traffic runs directly through CIS. Load balancers support both DNS-only and HTTP proxy modes, consider which of the two alternatives below best match your use case before proceeding as the traffic routing behavior differs as follows:

- Alternative 1: Traffic that is proxied flows through CIS.
- Alternative 2: Traffic that is non-proxied (DNS-only mode) flows directly from the client to the origin. In DNS-only mode, none of the CIS security, performance, and reliability features are applied.



For more information read through the [Proxying DNS records and global load balancers](#) topic.

Alternative 1: Proxy, traffic flows through CIS

This first alternative creates a wildcard certificate for custom domain, and then proxies it in the IBM Cloud Internet Services (CIS) allowing you to take advantage of industry leading security, protection and performance capabilities. Replace `example.com` with your custom domain name in the steps below.

Tip: Currently ordering certificates is by using **Let's Encrypt**, you may follow the topic [Supported certificate authorities](#) for updates. Using **Let's Encrypt** requires an ACME account. Follow the steps outlined in the [Connecting third-party certificate authorities](#) topic to create or register your account. In addition, you are required to add a DNS provider following the steps in the [Connecting DNS providers](#) topic. For this tutorial, you must add CIS as your DNS provider.

Initially HTTPS is configured from the user to Secrets Manager only.

- Order a certificate in Secrets Manager
 - Open the Secrets Manager service and select **Secrets** on the left.
 - Click **Add**.
 - If you are using a new Secrets Manager instance you will need to configure it prior to ordering your certificate. Follow the steps outlined under [Preparing to order public certificates](#).
 - Click on **Public certificate** and then click on **Next**.
 - Complete the form:
 - Name** - type a name you can remember.
 - Description** - enter a description of your choice.
 - Click on **Next**.
 - Under **Certificate authority** select your configured **Let's Encrypt** certificate authority engine.
 - Under **Key algorithm**, pick your preferred algorithm,
 - Bundle certificates** - leave off
 - Automatic certificate rotation** - leave off
 - Under **DNS provider** select your configured DNS provider instance
 - Click on **Select domains** check the **Select with wildcard** and leave the domain itself unchecked and click on **Done**.
 - Click **Next**.
 - Review your selections and click on **Add**.
 - You don't need to wait for the activation to complete to execute the next step, but you do need to wait for it to complete prior to verifying success.
- Configure HTTPS from client web browsers to the CIS endpoint. In CIS configure TLS Security:
 - Open the **Security** panel and choose **TLS**.

- For the **Mode** choose **Client-to-edge**. This will terminate HTTPS connections at the global load balancer and will switch to HTTP connections to the VPC load balancer.
3. In the CIS configure the global load balancer to use TLS:
 - Open **Reliability** panel and choose **Global load balancers**.
 - Locate the global load balancer created earlier and turn on Proxy.
 4. In a browser open <https://lb.example.com> to verify success.

Next configure HTTPS from CIS to the VPC load balancer.

Add an HTTPS listener to the VPC load balancers:

1. Navigate to **VPC** then **Load balancers** and click **vpc-lb-region1**.
2. Choose **Front-end listeners**.
3. Click **Create listener**.
4. Select the **Default back-end pool**: `region1-pool` or `region2-pool`.
5. Select **HTTPS** and enter for **Port** a value of `443`.
6. Select the **Secrets Manager** instance you created earlier, the SSL Certificate drop down should show the certificate **name** that you ordered using your Secrets Manager instance earlier from Let's Encrypt. Click on **Create**.



Tip: If the SSL Certificate drop down does not have **example.com** you may have missed the authorization step above that gives the VPC load balancer access to the Secrets Manager service. Verify that the Secrets Manager service has a certificate for **example.com**.

7. Repeat the above steps for the **vpc-lb-region2** load balancer.

The wildcard certificate created will allow access to domain name like vpc-lb-region1. **example.com**. Open the the **Overview** tab of the VPC load balancer **vpc-lb-region1** and notice that the **Hostname** is xxxxxx-REGION.lb.appdomain.cloud. The wildcard certificate is not going to work. Fix that problem by creating an alias and then update the configuration.

1. A DNS CNAME record can be created to allow clients to look up vpc-lb-region1. **example.com** and resolve xxxxxx-REGION.lb.appdomain.cloud.
 - In the CIS, open **Reliability** panel and choose **DNS**.
 - Scroll down to DNS Records and create a record of Type: **CNAME**, Name: **vpc-lb-region1**, TTL: **Automatic** and Alias Domain Name: **VPC load balancer Hostname**.
 - Add a DNS CNAME record for **vpc-lb-region2**.
2. Now adjust the global load balancer to use the new CNAME records.
 - Open **Reliability** panel and choose **Global Load Balancers**.
 - Find and edit the **Origin Pools** to change the **Origins Origin Address** to **vpc-lb-region1.example.com**.
 - Repeat the above steps for **vpc-lb-region2.example.com**.
3. Turn on end-to-end security.
 - Open the **Security** panel and choose **TLS**.
 - For the **Mode** choose **End-to-end CA signed**. This will use HTTPS connections at the global load balancer and HTTPS connections to the VPC load balancer.

In a browser open <https://lb.example.com> to verify success

Alternative 2: DNS-only mode, traffic flows directly from the client to the VPC Load Balancers

In this alternative you will order an SSL certificate for `lb.example.com` from [Let's Encrypt](#) through Secrets Manager and configure the global load balancer.

It is not currently possible to order a certificate directly for a CIS global load balancer, but it is possible to order one for a CNAME record. So we will create a CNAME to order the certificate.

1. Open the CIS service you created by earlier, you can find it in the [Resource list](#)
2. Navigate to **Global Load Balancers** under **Reliability** and click **DNS**.
3. Scroll down to DNS Records section and create a new record:
 - Type: `CNAME`

- Name: **lb**
- TTL: **default (Automatic)**
- Alias Domain Name: **zzz.example.com**
- Click **Add Record**

4. Order a certificate in Secrets Manager

- Open the Secrets Manager service and select **Secrets** on the left.
- Click **Add** and then **Public certificate**. Click on **Next**.
- Complete the form:
 - **Name** - **lb-alias**.
 - **Description** - enter a description of your choice.
 - Click on **Next**.
 - Under **Certificate authority** select your configured **Let's Encrypt** certificate authority engine.
 - Under **Key algorithm** select **RSA4096**
 - **Bundle certificates** - leave off
 - **Automatic certificate rotation** - leave off
 - Under **DNS provider** select your configured DNS provider instance
 - Click on **Select domains**
 - Expand the domain listed to view the list of subdomains and select the check box next to the lb.example.com and click on **Done**.
- Click **Next**.
- Review your selections and click on **Add**.

Create a HTTPS listener:

1. Navigate to the VPC **Load balancers** page.
2. Select **vpc-lb-region1**
3. Under **Front-end listeners**, Click **Create**
 - **Protocol**: HTTPS
 - **Port**: 443
 - **Back-end pool**: POOL in the same region
 - Choose the current region as your SSL region
 - Choose the SSL certificate order name you just created for **lb.example.com**
4. Click **Save** to configure an HTTPS listener

Repeat the above steps in the load balancer of **region 2**.

In a browser open <https://lb.example.com> to verify success

Failover test

By now, you should have seen that most of the time you are accessing the servers in **region 1** as it's assigned higher weight compared to the servers in **region 2**. Let's introduce a health check failure in the **region 1** origin pool,

1. Navigate to the list of **virtual server instances**.
2. Click **three dots(...)** next to the server(s) running in **zone 1** of **region 1** and click **Stop**.
3. Repeat the same for server(s) running in **zone 2** of **region 1**.
4. Return to GLB under CIS service and wait until the health status changes to **Critical**.
5. Now, when you refresh your domain url, you should always be hitting the servers in **region 2**.

 **Tip:** Don't forget to **start** the servers in zone 1 and zone 2 of region 1.

Step 7: Remove resources

- Remove the global load balancer, origin pools and health checks under the CIS service

- Remove the certificates in the Secrets Manager service.
- Remove the load balancers, VSIs, subnets and VPCs.
- Under [Resource list](#), delete the services used in this tutorial.

Related content

- [Getting started with Virtual Private Cloud \(VPC\)](#)
- [Using Load Balancers in IBM Cloud VPC](#)
- [Getting started with IBM Cloud Internet Services](#)
- [Getting started with Secrets Manager](#)

Configuring a highly available application

You can deploy your IBM Cloud® Code Engine application across multiple regions to make it resilient to regional failures. Note that this example uses a global content delivery network (CDN) called [IBM Cloud Internet Services](#), but you can use alternate providers. This example also uses a custom domain.

Prerequisites

- You must have a custom domain name for your application, such as `example.com`. This domain name is used by your Code Engine application.
- Set up an instance of [Cloud Internet Services \(CIS\)](#).
- [Add your domain name to Cloud Internet Services \(CIS\)](#). When you register your domain name with Cloud Internet Services (CIS), you are delegating control of your domain name to Cloud Internet Services (CIS). Note that this step can take a while to complete.

Step 1: Create projects in different regions

Create a Code Engine project in three different regions. You can use a common naming pattern and a shared tag.

For example, create a project called `global-app-project` in the `au-syd`, `eu-de`, and `br-sao` regions with either the CLI or from the console.

Name	Status	Tag	Location	Resource group	Created
global-app-project	Ready	global-app	Sydney (au-syd)	default	
global-app-project	Ready	global-app	Frankfurt (eu-de)	default	2 min
global-app-project	Ready	global-app	Sao Paulo (br-sao)	default	3 min

Projects in multiple regions

For more information, see [Managing projects](#).

Step 2: Deploy your apps in multiple regions

Now that your projects are created in multiple regions, deploy your application in each project.

For example, deploy the `codeengine/helloworld` app.

1. From the [Code Engine projects](#) page, click the name of one of the projects that you created.
2. Click **Applications** and then click **Create** to create a new application.
3. Configure your app with the following settings.
 - a. Name your application `global-app`.
 - b. Select **Use an existing container image** to reference a container image for your app.
 - c. Enter `icr.io/codeengine/helloworld` for your image reference.
 - d. In the **Resources & scaling** section, set your minimum number of instances to 1. By setting your minimum number of instances to 1, you can enable health checks from your CIS instance to monitor the availability of pools so that traffic can be routed to the healthy ones.
 - e. Leave the rest of the options at the default settings and click **Create**.
4. Repeat these steps to create the application in each project.

For more information about deploying your application, see [Deploying applications](#).

Step 3: Generate a certificate for your custom domain

In an enterprise environment, work with your corporate domain administrator to obtain the necessary certificates. However, if the custom domain is within your control and you want quickly create a certificate that is not self-certified, then you can optionally use the [Let's Encrypt](#) service and [Certbot](#) to obtain a certificate.

1. Install [Certbot](#). Certbot is a client for the [Automatic Certificate Management Environment \(ACME\)](#) protocol for automating interactions between a CA and a server. The Let's Encrypt service uses this client to verify domain ownership and issue certificates. From the [Certbot Instructions page](#), select **Other** as the software and select the operating system for your workstation to obtain the applicable information to install the Certbot command line.
2. Run the following command to create your certificate. This example command creates a certificate for the `example.com` and `www.example.com` custom domains. Be sure to update the command for your own custom domain.

```
$ certbot certonly --manual --preferred-challenges dns --email webmaster@example.com --server https://acme-v02.api.letsencrypt.org/directory --agree-tos --domain example.com --domain www.example.com
```

3. To verify that you own the domain, set a **TXT** record with your domain registrar for the domains that you requested in the previous step with values that were provided with the Certbot tool output; for example, `_acme_challenge.example.com` and `_acme_challenge.www.example.com`. After you set the **TXT** record, continue with the Certbot command.
4. Certbot retrieves the certificate that is signed by Let's Encrypt. The location where the certificate is stored is provided by the Certbot output. Find the `fullchain.pem` and `privkey.pem` files.

Step 4: Create a TLS secret

Create a TLS secret to store your certificate in Code Engine.

1. From the [Code Engine projects](#) page, click the name of one of the projects that you created.
2. Select **Secrets and configmaps**.
3. Click **Create**.
4. Click **TLS secret**.
5. Enter `global-tls` as the name.
6. Copy the content of the `fullchain.pem` file into the **Certificate chain** field.
7. Copy the content of the `privkey.pem` file into the **Private key** field.
8. Click **Create**.
9. Repeat these steps to create a TLS secret in each project that you created earlier.

For more information, see [Working with secrets](#).

Step 5: Configure the custom domain mappings

After your apps are deployed, configure a custom domain mapping for them.

1. From the [Code Engine projects](#) page, click the name of one of the projects that you created.
2. Select **Applications**.
3. Select your `global-app` application.
4. Select **Domain mappings**.
5. Select **Public** for your visibility.
6. Click **Create** to create a custom domain mapping.
7. Click **Select** to choose an existing TLS secret and select `global-tls`.
8. Enter your fully qualified domain name; for example, `www.example.com`.
9. Note the **CNAME** target value. You need this value to set up routing for your domain in CIS.
10. Verify that the app name is `global-app`.
11. Click **Create**.
12. Repeat these steps to create a custom domain mapping for each application that you created.

Step 6: Configure a health check

When you created your applications, you set the **Minimum number of instances** to 1. Because there is always an instance of your app running in each region, you can set up a health check from your CIS instance to monitor the availability of pools. By setting up a health check, traffic is always routed to a running instance, making your app highly available.

1. From your CIS instance, navigate to **Reliability > Global load balancers > Health checks** .
2. Click **Create**.
3. Name your health check the same as your application name: `global-app` .
4. Set the **Monitor type** to `HTTPS` and the **Port** to `443` .
5. Accept the defaults for the rest of the options. Note that if you are using an app other than `codeengine/helloworld` app, adjust any options that your app requires.
6. Click **Create**.

For more information, see [Setting up health checks](#).

Step 7: Configure the Cloud Internet Services (CIS) load-balancer

After your custom domain mappings are in a `Ready` state, configure the Cloud Internet Services (CIS) load-balancer for your application global endpoint. For more information, see [Configuring a global load balancer](#).

1. Go to the Reliability page in the Cloud Internet Services (CIS) console.
2. Select **Origin pools** and click **Create**.
 - a. Name your pool `global-app-au-syd` .
 - b. Set the **Origin address** to the CNAME target of your domain name mapping.
 - c. Set the Host header to your domain name.
 - d. From the **Health check**, select **Existing health check** and then select `global-app` .
 - e. Click **Save**.
 - f. Repeat these steps for each region that contains your deployed app. Change the name to reflect the region that you are targeting. For example, `global-app-de-eu` and `global-app-br-sao` .
3. Select **Load balancers** and click **Create**.
 - a. Name your load balancer. Note that this name appears in your custom domain URL. For example, if your custom domain is `global-app.example.com` and you name your load balancer `global-app` , your URL is `global-app.example.com` .
 - b. Set **Traffic steering** to `Geo` .
 - c. Add your **Geo** routes. You can choose to create a route for all CIS regions or only some regions.
 - If you create a route for all CIS regions, then in each route that you create, add all the origin pools that you created earlier. Sort them so that a region that contains your running app and is closest to the region route that you are configuring. For example, if you created apps in `au-syd` , `eu-de` , and `br-sao` , then for `Oceania` , put `au-syd` first. For Eastern and Western Europe, put `de-eu` first. And for North and South America, put `br-sao` first.
 - If you create a route for only some CIS regions, add a route for the **Default** region. This route is the fallback to use when a specified region is not available.
 - d. Click **Create** to create the load balancer.

Step 8: Verify that your app is available

Open a browser and enter your load balancer name plus your custom domain name; for example, `www.global-app.example.com`

Now your applications are highly available.

Step 9: Cleaning up your tutorial

1. Delete the global load balancers and origin pools from CIS.
2. Delete your DNS records from CIS. For more information, see [Deleting DNS records](#).
3. Delete each project that you created. When you delete a project, all the components contained in that project are also deleted. For more information, see [Delete a project](#).

Note that your custom domain is not deleted, but is no longer associated with the application that you created.

Creating a CIS instance

To create a CIS instance, follow these steps:

1. Log in to the [IBM Cloud console](#) and click **Catalog** in the top navigation bar.
2. Search for **Internet Services** and click the tile from the search results to open the provisioning page.
3. Review and select a pricing plan. For more information, see [Comparing CIS plans](#).
4. In the Configure your resources section:
 - a. Enter a service name to identify your CIS instance.
 - b. Choose a resource group to organize your services. The resource group can't be changed after the instance is provisioned.
 - c. Optionally, add user or access management tags.
5. Select the checkbox to confirm that you have read and agree to the licensing terms.
6. Click **Create** on the Overview page to begin provisioning your instance.



Note: The No-cost Trial plan is limited to one instance per account.

7. From the first page of the CIS application, click **Add Domain** and continue to step 2 in the [Getting started](#) topic to add and configure your domain.

Navigating to your CIS instance

To open to your CIS instance:

1. Go to **Navigation Menu** icon  and click **Resource list**, then expand **Security**.
2. Expand **Security** and click the name of your CIS instance.

Configuring a full (authoritative) zone

A "full setup" delegates DNS authority to CIS. You update your domain's name servers at your registrar so CIS becomes the authoritative DNS provider for your domain. This can be applied to a root domain (`example.com`) or a subdomain (`sub.example.com`) if needed.

To configure a full (authoritative) zone, follow these steps:

1. Prepare your domain:
 - a. Ensure you own the domain or subdomain.
 - b. Identify your current DNS provider or registrar.
 - c. Gather all existing DNS records (A, AAAA, MX, TXT, CNAME, DKIM, SPF, CAA, etc.) that must be preserved.
 - d. Disable DNSSEC at your registrar if it is currently enabled.
 - e. Remove or resolve any conflicting records at the registrar that could prevent delegation.
2. Add your domain to CIS:
 - a. Log in to the **IBM Cloud Console** and open your CIS instance.
 - b. Navigate to **Domains > Add domain**.
 - c. Enter your domain or subdomain name and select **Full setup (authoritative DNS)**.
 - d. Proceed to add the domain.
3. Import or recreate DNS records in CIS:
 - a. Go to **DNS > Records** in the CIS dashboard.
 - b. Import existing DNS records from your previous provider if available.
 - c. If import is not available, manually add all necessary records, including:
 - A and AAAA records
 - MX and TXT records (including SPF, DKIM)
 - CAA and other service-specific records
 - d. Verify that all records are correct before updating name servers.



Note: Missing or incorrect records cause service interruptions after CIS becomes authoritative.

4. Update name servers at your registrar:
 - a. In the CIS console, note the **assigned name servers** (for example, `ns004.name.cloud.ibm.com` , `ns005.name.cloud.ibm.com`).
 - b. Log in to your domain registrar.
 - c. Locate your domain's **DNS / Nameserver settings**.
 - d. Replace the existing name servers with the CIS name servers exactly as provided.
 - e. Save the changes.



Note: Ensure name servers are entered accurately. Typos or omissions break DNS resolution.

5. Wait for propagation and verify:
 - DNS changes can take **24–48 hours**, or in rare cases up to **72 hours**, to propagate globally.
 - Use `dig` or `nslookup` to confirm that the domain now resolves with CIS name servers.
 - The CIS console automatically detects when delegation is complete, and the domain status changes from **Pending** to **Active**.
6. Optionally, after propagation, re-enable DNSSEC at your registrar if required. Use the DS records provided by CIS to complete DNSSEC setup.
7. Confirm DNS functionality:
 - a. Test your domain's website, email, and any other services.
 - b. Ensure all DNS records (for example, A, MX, TXT, CAA) are resolving correctly.
 - c. Troubleshoot any issues by checking records in CIS or verifying name server settings at your registrar.

CNAME (partial) configuration

Configuring a partial (CNAME) zone

A partial or CNAME setup allows you to use reverse proxy while you maintain your primary and authoritative DNS provider.

This configuration is useful when you can't change your authoritative DNS provider and want to proxy only individual subdomains through the global network.

After you complete a partial setup, the actual resolution of your records to CIS depends on the CNAME records added at your authoritative DNS provider. Check your authoritative DNS provider to learn which records are pointing to `{your-hostname}.cdn.cloudflare.net` .

The following table shows the recommended setup configurations for a child zone (subdomain).

Parent domain setup	Recommended child subdomain setup
Parent domain on CIS through a Full setup	Full setup only
Parent domain on CIS through a CNAME setup	CNAME setup only
The parent domain is not on CIS	Can choose Full or CNAME setup

Parent and child domain setup

Setting up a CNAME zone

To set up a CNAME, take the following steps.

1. Create the `partial` type zone by using the CIS API or CLI.
 - To create the `partial` type zone with CIS API:

POST `https://{api}/v1/{crn}/zones`

```
data: {
  "name": "ibmnetworkdemo.com",
  "jump_start": false,
  "type": "partial"
}
```

- To create the `partial` type zone with CIS CLI:

```
$ ibmcloud cis domain-add ibmnetworkdemo.com --type partial --output JSON
```

If you encounter the error message `Partial zone signup not allowed` , contact Support.

2. Get the txt record `verification_key` and `cname_suffix` from the response:

```
{
  "result": {
    "id": "1df93abfb59849abd3e34fde156a4c21",
    "name": "ibmnetworkdemo.com",
    "status": "active",
    "paused": false,
    "verification_key": "476754457-428595283",
    "cname_suffix": "cdn.cloudflare.net",
    "original_name_servers": [
      "ns1.softlayer.com",
      "ns2.softlayer.com"
    ],
    "original_registrar": "everyones internet, ltd. dba s (id: 925)",
    "original_dnshost": null,
    "modified_on": "2021-05-07T06:46:19.326826Z",
    "created_on": "2021-05-07T01:57:53.163247Z",
    "account": {
      "id": "b0c53e3f037b8cdc62b5cb373b8c55e6",
      "name": "57aea3aa-a38e-4760-ada5-a698bca56171"
```

```
    }
  },
  "success": true,
  "errors": [],
  "messages": []
}
```

3. Add the record `cloudflare-verify` to the parent DNS zone indicated by the `verification-key` (in this example, `ibmnetworkdemo.com`):

```
$ txt cloudflare-verify.ibmnetworkdemo.com 476754457-428595283
```

4. After CIS verifies the record, the zone is activated. This process might take several hours.

Verifying the CNAME

To verify your CNAME setup, follow these steps:

1. Add an A record in CIS and enable proxy:

```
$ www.ibmnetworkdemo.com A 169.48.151.44 true 1
```

2. Add the CNAME record in the authoritative DNS:

```
$ www.ibmnetworkdemo.com www.ibmnetworkdemo.com.cdn.cloudflare.net
```

The response appears similar to the following example:

```
check::
dig www.ibmnetworkdemo.com a

; <<>> DiG 9.10.6 <<>> www.ibmnetworkdemo.com a
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 13528
;; flags: qr rd ra; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 512
;; QUESTION SECTION:
;www.ibmnetworkdemo.com.      IN      A

;; ANSWER SECTION:
www.ibmnetworkdemo.com.      899     IN      CNAME   www.ibmnetworkdemo.com.cdn.cloudflare.net.
www.ibmnetworkdemo.com.cdn.cloudflare.net. 299 IN A 104.18.8.216
www.ibmnetworkdemo.com.cdn.cloudflare.net. 299 IN A 104.18.9.216
```

Getting CIS running with a new subdomain

Follow these steps to practice using CIS with a subdomain or CNAME. This example uses the GoDaddy domain registrar. Your case might be different, depending on which registrar you use.

1. Deploy a CIS instance by using the IBM Cloud console or API.



Note: Mirror the name of your CIS instance with the domain or subdomain.

2. Click **Let’s get started**.

3. Enter the subdomain in the **Domain name** field and click **Connect and continue**.

4. For new subdomains click **Next Step**, for existing subdomains click **Import records**.

5. Make note the **New NS records** for entry into the GoDaddy DNS Management system.

6. Log in to GoDaddy domain Registrar.

7. Navigate to the DNS Management page for the candidate CIS domain. IBM Cloud® must be updated through the API.

8. Click **ADD** to create an NS Host record for each name server entry provided from CIS.
9. Set up a DNS record with the supplied CIS NS Records:
 - Type **Nameserver**
 - Host **subdomain** (the name of the subdomain, for example, `subdomain.example.com`)
 - Points to `ns004.name.cloud.ibm.com`
 - TTL ½ hour
10. The DNS Management page now reflects the two new NS records for the subdomain.
11. Go back to the IBM Cloud Internet Services instance.
12. After the NS Records are DNS propagated, it is reflected in the DNS tools, and you are ready to proceed.
13. With completed NS propagation, click **Check name servers**.
14. CIS starts checking NS configuration for the subdomain and might be in a **Pending State**.



Note: Any typographical errors in can produce a "silent" failure. If your subdomain is in the pending state for more than 30 minutes, you might have an error.

Upon successful subdomain name server and NS record confirmation, the CIS instance reports **Active**. Customers are now ready to start configuring the service and routing traffic to their services.

DNS resolution in partial (CNAME) configuration

In a partial (CNAME) configuration, CIS handles DNS records a bit differently from full zones to internally resolve the origin server where proxied HTTP requests are sent to.

Records within the same zone

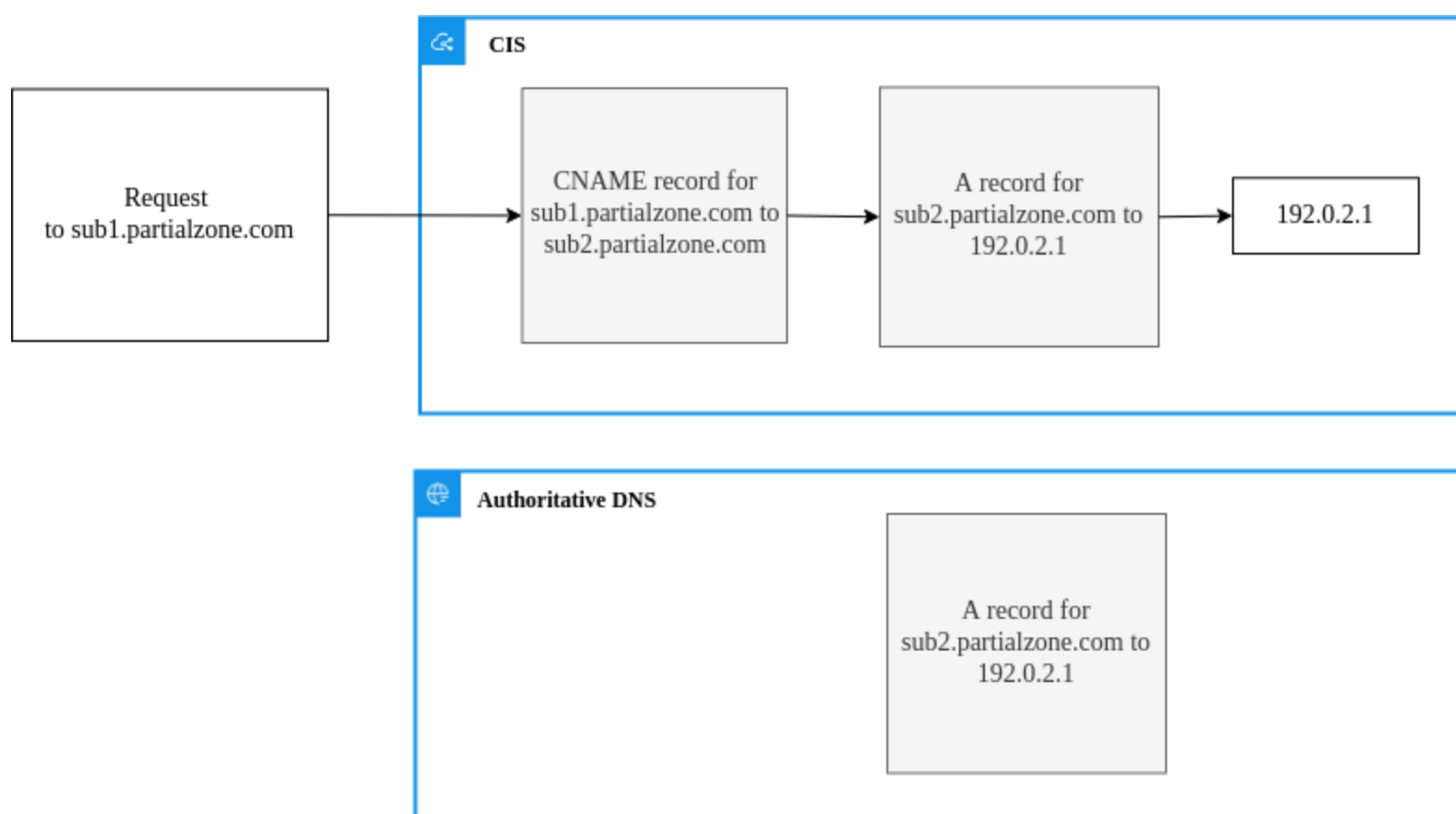
When you create a new DNS record in a partial zone, CIS automatically checks whether any of your CNAME records point to existing A, AAAA, or CNAME records within the same zone.

For example, CIS shows a warning if you have the following records in your partial zone:

```
sub1.partialzone.com CNAME sub2.partialzone.com
sub2.partialzone.com A 192.0.2.1
```

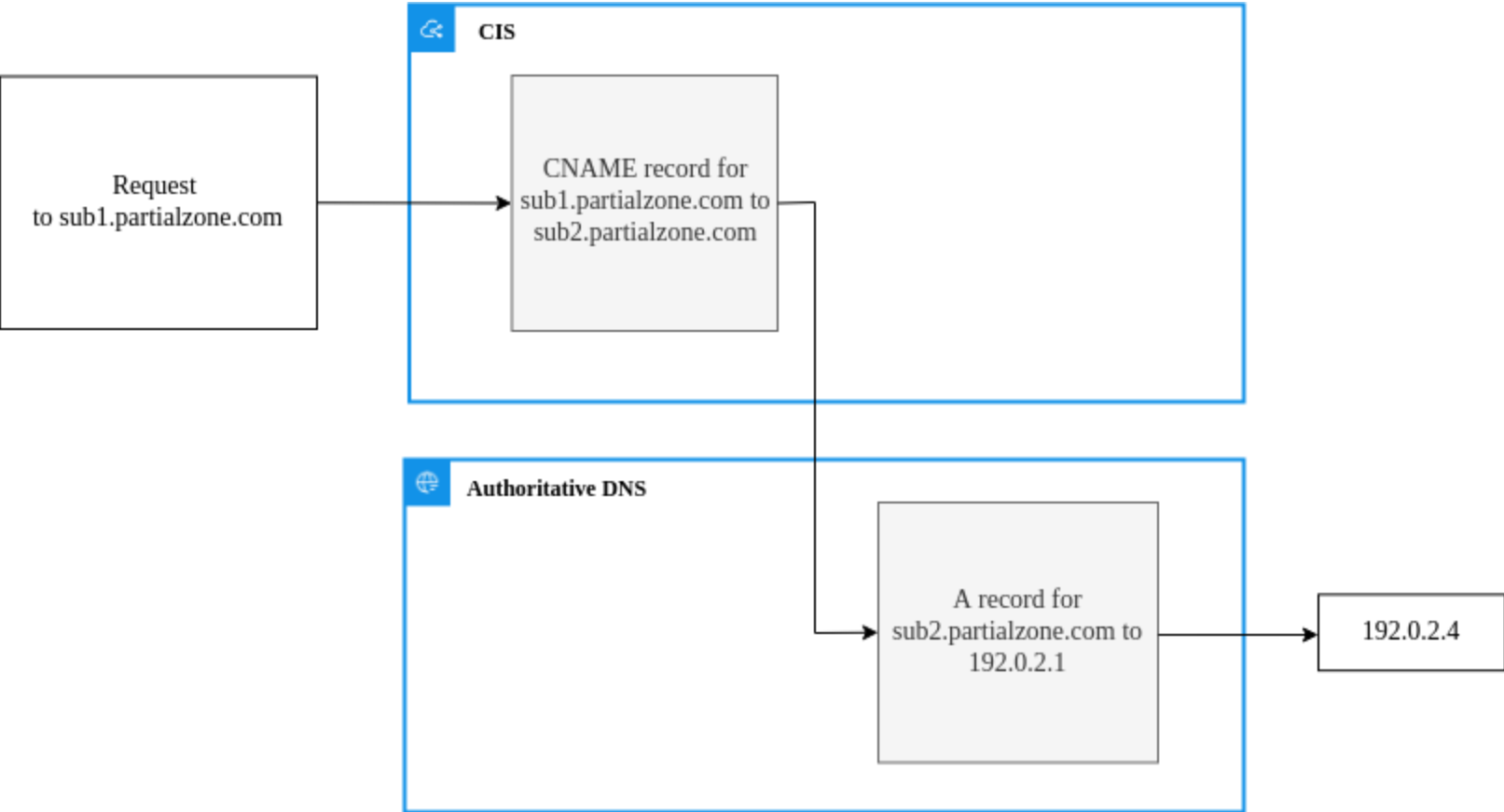
Because CIS contains both the CNAME and its target, the DNS resolution sends incoming HTTP requests to `sub1.partialzone.com` to the origin `192.0.2.1`.

This can cause issues if you already have DNS records for `sub2.partialzone.com` at your authoritative DNS provider. These records might point to `192.0.2.4`, another IP address, or another domain. However, because CIS contains the initial record and the target, it never queries your authoritative DNS provider for the record for `sub2.partialzone.com`.



Path a request takes without passing through an authoritative DNS

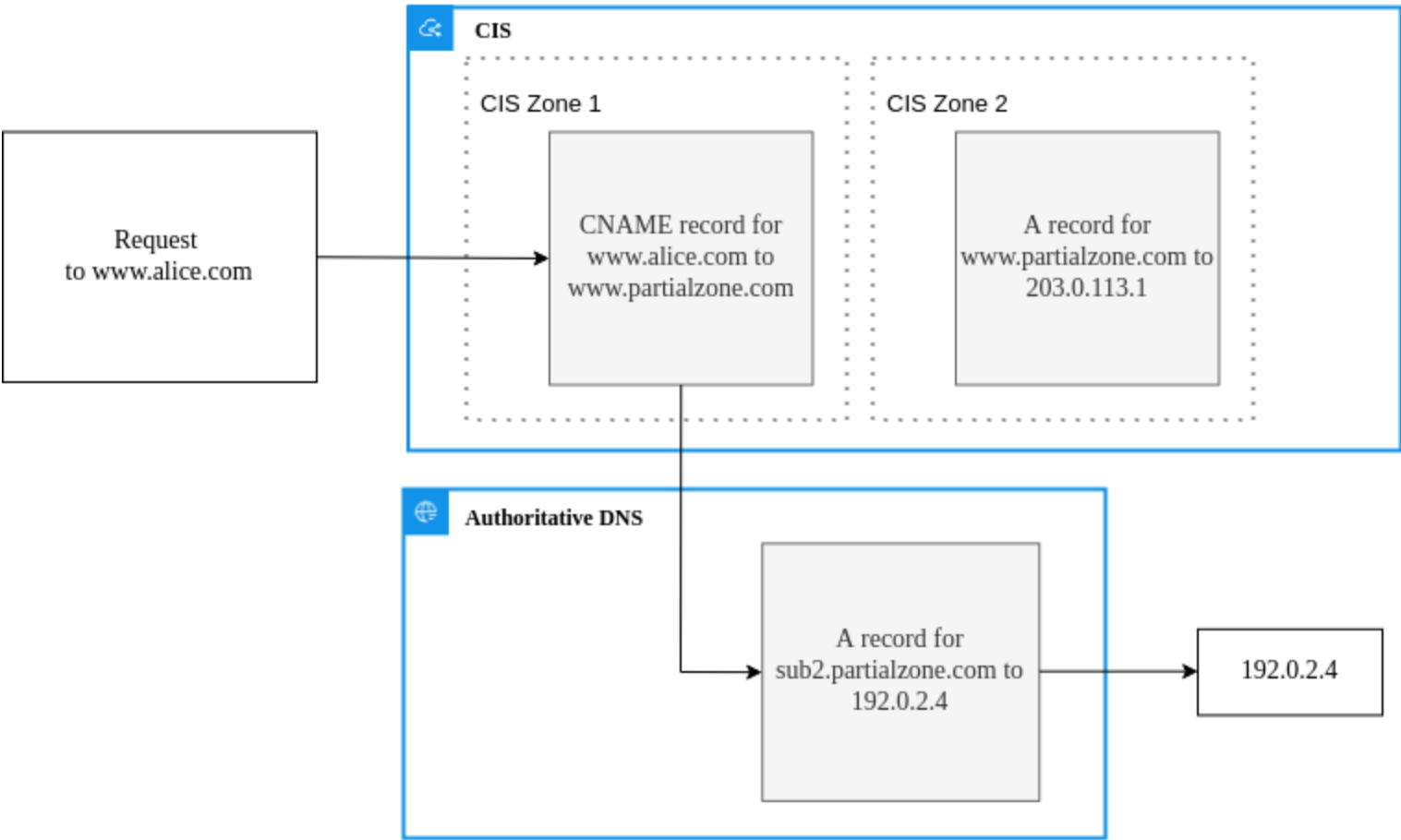
If you don't have the target of the CNAME record within your partial zone this DNS resolution happens differently.



Path a request takes passing through an authoritative DNS

Records pointing to a partial zone within the same account

You can also create a CNAME record in a zone (partial or full setup) that points to a record in another partial zone within your account. In this case, CIS always resolves the CNAME target based on the value at your authoritative DNS provider of the partial target zone.



Path a request takes within the same account

Configuring DNS records and settings

Before directing traffic to your domain through CIS, import or manually recreate your existing DNS records within CIS. This action helps ensure that all domain resources, such as subdomains and dependent services, continue to resolve correctly after traffic is routed through CIS. Although this step is optional, failing to configure DNS records in CIS can result in partial service disruption or inaccessible website components.

You can import your DNS records by uploading a file you’ve exported from your current DNS provider, or you can manually enter each record in CIS. To import, click **Import records** and upload the appropriate file.

After you’ve finished importing or creating your records, or if you choose to skip this step, click **Next step** to continue the setup process.

Configuring DNS records is essential to ensure your domain operates correctly, securely, and efficiently. DNS records store information about your domain and determine how traffic is routed to your website or application. By configuring your DNS settings, you can:

- Update or add services to launch a new website.
- Connect a mail server.
- Create a subdomain.

Keeping your DNS records up to date allows you to maintain full control over your domain, optimize performance, and ensure your site remains reliably accessible online.

Adding DNS records

You can use the **Type** list menu to select the type of record you want to create. Each DNS record type has a Name and Time-To-Live (TTL) associated with it.

Anything that is entered into the Name field has the domain name appended, unless the domain name is manually appended in the field already. For example, if `www` or `www.example.com` is typed into the field, the API handles both as `www.example.com`. If the exact domain name is typed into the name field, then it isn't appended on itself (for example, `example.com` is handled as `example.com`). However, the list of DNS records shows only the names without the domain name added, so `www.example.com` is shown as `www` and `example.com` is shown as `example.com`. The TTL has a default value of `Automatic`, but can be changed by the user. A proxied DNS record always has a TTL of `Automatic`, so a newly proxied record adopts this configuration during this change.

 **Tip:** For records such as A record and CNAME, the automatic TTL is 300s.

A Type record

To add this record type, valid values must exist in the **Name** and **IPv4 Address** fields. A **TTL** can also be specified from the list menu, with a default value of `Automatic`.

Required Fields: Name, IPv4 Address Optional Field: TTL (default value is `Automatic`)

AAAA Type record

To add this record type, valid values must exist in the **Name** and **IPv6 Address** fields. A **TTL** can also be specified from the list menu, with the default value of `Automatic`.

Required Fields: Name, IPv6 Address Optional Field: TTL (default value is `Automatic`)

CNAME Type record

To add this record type, a valid value must exist in the **Name** field and a fully qualified domain name must be in the **Domain Name** (FQDN) field. A **TTL** can also be specified from the list menu, with the default value of `Automatic`.

Required Fields: Name, Domain Name (for CNAME) Optional Field: TTL (default value is `Automatic`)

 **Tip:** Enterprise plans are able to CNAME another domain provided that the domain is configured within CIS.

\$ Ex.
Configured CIS Domains:
- example.com
- different.com

test.example.com -CNAME-> test.different.com



Note: The CNAME flattening feature is enabled by default, and can't be turned off.



Important: CIS does not support Cloudflare's CNAME setup. The only way to activate your domain in CIS is to delegate your NS Records management to CIS.

MX Type record

To add this record type, a valid value must exist in the **Name** field and a valid address must exist in the **Mail Server** field. A **TTL** can also be specified from the list menu, with the default value of **Automatic**.

- Required Fields: Name, Mail Server
- Optional Fields: TTL (default value is **Automatic**), Priority (default value is 1)

LOC Type record

To add this record type, a valid value must exist in the **Name** field. If you need more specific information, select **Configure LOC options**. A **TTL** can also be specified from the list menu, with the default value of **Automatic**.

- Required Fields: Name
- Optional Fields: LOC options (click **Configure LOC options** to configure)

CAA Type record

To add this record type, valid values must exist in the **Name** and **Value** fields. The Value field correlates to the value of the **Tag** list field, which defaults to **Send violation reports to URL**. A **TTL** can also be specified from the list, with the default value of **Automatic**.

- Required Fields: Name, Value (associated to tag)
- Optional Fields: TTL (default value is **Automatic**), Tag (default is to send violation reports to URL)

SRV Type record

To add this record type, valid values must exist in the **Name**, **Service Name** and **Target** fields. Use the list menu to select a **protocol**, which defaults to the UDP protocol. Additionally, you can specify **Priority**, **Weight** and **Port**. These three fields default to a value of 1. A **TTL** can also be specified from the list menu, with the default value of **Automatic**.

- Required Fields: Name, Service Name, Target
- Optional Fields: TTL (default value is **Automatic**), Protocol (defaulted to UDP), Priority (defaulted to 1), Weight (defaulted to 1), Port (defaulted to 1)

TXT Type record

To add this record type, valid values must exist in the **Name** and **Content** fields. A **TTL** can also be specified from the list menu, with the default value of **Automatic**.

- Required Fields: Name, Content
- Optional Field: TTL (default value is **Automatic**)



Note: The first time that you order a dedicated certificate Domain Control Validation (DCV) process occurs, which generates a corresponding TXT record. If you delete the TXT record, the DCV process happens again when you order another dedicated certificate. If you delete a dedicated certificate, the TXT record corresponding to the DCV process is not deleted.

NS Type record

To add this record type, valid values must exist in the **Name** and **Name Server** fields. A **TTL** can also be specified from the list menu, with the default value of **Automatic**.

- Required Fields: Name, Name Server
- Optional Field: TTL (default value is **Automatic**)

PTR type record



Note: The PTR record option that is shown in the DNS Records list menu is not for adding PTR records for Reverse DNS resolution. The purpose is

to add a PTR Record to the Forward DNS resolution for the domain. PTR in Forward DNS is allowed under the [DNS specification](#).

PTR records primarily prevent emails from going to spam folders. Because CIS doesn't support email traffic by default, you must set the PTR record to the location of your email server. Contact your email provider for assistance.

SOA type record

A Start of Authority (SOA) record stores administrative information about a domain, such as the primary name server, administrator email address, and zone timing parameters.

In CIS, the SOA record is automatically created and managed when you onboard a domain. You don't have to manually create or configure an SOA record. CIS manages SOA records internally, and they are not available for viewing or modification.

Updating DNS records

In each record row, you can click the **Edit record** option from the menu, which opens a dialog box that you can use to update the record.

After you are finished making your changes, select **Update record** to save them, or **Cancel** to abort the changes.

Deleting DNS records

In each record row, you can select the **Delete record** option from the menu, which opens a dialog box to confirm the deletion process.

You can select the **Delete** button to confirm your delete action. Select **Cancel** if you don't want to delete.

Importing and exporting DNS records

DNS records can be imported into and exported from CIS. All files are imported and exported as .txt files in BIND format. Learn more about [BIND format](#). Click the Actions menu and select to import or export records.

Import records - By default, a total of 3500 DNS records are allowed (imported and created on CIS). You can import multiple files, one at a time, as long as the total number of records is under the max limit. After you import, you are shown a summary with the number of records that were successfully added and the number that failed, along with the reason why each record failed.

Export records - Use [Export records](#) to create a backup of your zone file, or export it to use with another DNS provider. When this menu option is clicked, the records are downloaded to the location specified by your browser settings (typically the Downloads folder). To select another folder location, change your browser's settings to prompt you for a location with each download.

Configuring and managing your secure DNS

DNSSEC is a technology to digitally sign DNS data so you can know it is valid. To eliminate vulnerability from the internet, DNSSEC must be deployed at each step in the lookup, from root zone to final domain name (for example, [www.icann.org](#)).

DNSSEC adds a layer of authentication to the internet's DNS infrastructure, which otherwise is not secure. Secure DNS guarantees that visitors are directed to **your** web server when they type your domain name into a web browser. All that you need to do is enable DNSSEC in your DNS page from your IBM CIS account and add the DS record to your registrar.

You can select **View DS records** to display the information that is needed to add the DS record to your registrar. You must copy parts of the DS record and paste them into your registrar's dashboard. Every registrar is different, and your registrar might only require you to enter information for some of the available fields.

Configuring your name servers with the registrar or existing DNS provider

To begin receiving the benefits of CIS, you must delegate your domain to CIS. To delegate a domain, create an NS record with the name servers provided by CIS at your domain's registrar or existing DNS provider. If you are unsure of who the registrar is for your domain, you can look it up at lookup.icann.org.

If you delegate a subdomain (for instance, `subdomain.example.com`) from another DNS provider, you must replace the existing name server (NS) records and replace them with a name server record for each of the name servers that are provided by CIS. See [Managing DNS records in Cloudflare](#) for instructions by provider.

After you configure your registrar or DNS provider, it can take up to 24-48 hours for the changes to take effect. When we verify that the specified name servers were configured correctly for your domain or subdomain, the domain's status changes from `Pending` to `Active`.

**Important:** Your domain must move to `Active` state within 60 days or your domain and any configuration data is removed.

Adding domains to your CIS instance

IBM Cloud® Internet Services supports multiple domains with the Standard Next and Enterprise Tier plans.

To add a domain to your CIS instance, navigate to your CIS instance and go to the **Overview** page. Under the **Domain** section, select the plus (+) icon next to your domain. This reveals a section to connect an additional domain. Follow the steps in this section to complete adding a new domain:

- Connect domain
- Set up DNS records
- Delegate domain management

After adding a new domain, use the list menu in the **Domain** section to switch between your domains. The **Domain** section is located on the **Overview** page and at the beginning of every page. Switching between domains allows you to configure settings for each domain.

Exploring the CIS console

You'll begin by using the Overview page as your working base of operations. It shows all of the current parameters for your deployment.

After you've set up your DNS and configured it, you are ready to go!

Using the Overview page

Using the Overview page, you can see the status of all your selections. Each setting links to the section of the user interface where the setting is configured. To modify any selection, you can navigate by clicking the link for the setting. For example, to change the load balancer configuration or add a new load balancer, click the `Load Balancer` field.

On the Overview page, you might see that your domain name configuration is in **Pending** status, or in **Active** status. **Pending** status indicates that your domain is not fully set up, yet. You have to update your DNS provider or registrar with the name servers that are provided as part of the setup process.

Enterprise only: The **Service Details** section of the Overview also allows you to add additional domains to your instance of CIS, and to switch between multiple domains.

Changing the Service mode

In the Service Mode section of the Overview page is a list menu to select one of two modes:

- **Defense Mode** helps protect against existing or predicted DNS attacks. This mode prevents all traffic from reaching your origin servers through your domain.
- **Pause Service** disables all security and performance benefits to your domain. DNS functions still resolve for your website, but traffic is sent directly to configured origins.

Setting up Service mode

1. Select the mode you want from the list menu.
2. Click **Activate mode**.
3. Confirm or cancel the selection in the confirmation window.

A notification appears on all pages to show that either Pause Service or Defense Mode is active. To return to normal operation, click **Deactivate mode** in the notification banner.

Configuring and managing your DNS

Go to the **Reliability** section, click the **DNS** tab and add a record. Type in the information about your DNS record and then click **Add record** to implement your changes.

After creating your records, consider turning on the `Proxy` setting. Most of the features of CIS require that the internet traffic to your site flow through CIS infrastructure. In other words, it only applies to proxied records and load balancers. To really reap the benefit of CIS, make sure that your DNS records and load balancers have the proxy setting enabled.

Setting up and manage your caching

Next, you can set up caching. You have the option of three types of caching, available from the caching list menu.

- **No query string:** Only delivers resources from cache when there is no query string.
- **Query string independent:** Delivers the same resource to everyone independent of the query string.



Note: The **Ignore Query String** setting applies only to static file extensions. This setting removes the query string when generating the cache key, so that a request for `style.css?something` is normalized to `style.css` when serving from the cache.

- **Query string dependent:** Delivers a different resource each time the query string changes.

Purge cache

You can purge your cache to prepare for updates at any time by entering the URL into the purge cache field. You can purge a single file or multiple files (up to 30 at a time).

Browser expiration

You can use the list menu to select the time of browser expiration that you require; for example, 8 hours, or 1 day.

Enterprise only: You can also instruct CIS not to override browser cache control by setting this to **Respect Existing Headers**.

Using Development mode

Development mode is intended for use when major updates or new file uploads are required, or any time you do not want the end users to work from the cache at all, but to retrieve files directly from the origin servers. To begin using **Development Mode**, toggle the switch to **Enabled** position. To stop using **Development Mode**, toggle the switch to **Disabled** position. **Development mode** expires automatically after 3 hours.

Managing page rules

You can use page rules to specify particular settings that only apply to certain URLs, for example to have different cache control settings for certain URL paths. Use the list menus to configure the page rule. The rule settings are divided into three categories: **Security**, **Performance**, and **Reliability**.

Notice that when certain rules are enabled, other options become grayed out, if those options are in conflict with the other rules you've just selected. After you've selected the page rules that you want, click **Provision** to enable them. The new rules take effect immediately, and they can be viewed immediately on the Page Rules page.

You also can enable or disable your page rules from the table displayed in the Page Rules page. See [Using page rules](#) for details.

Security settings

By default, DDoS protection is enabled for any DNS records or load balancers with proxy on.



Note: Only proxied records result in traffic being routed through CIS cloud edge for edge processing.

Security settings are:

- Turn on WAF using the toggle on the **Web Application Firewall** page. When you toggle the rules on or off, the changes are applied immediately.
- **Enterprise only** - The **Rate limiting** page allows you to configure rate-limiting rules to avoid noisy-neighbor problems and ward off DDoS.
- On the **IP Firewall** page, you can configure access rules based on IP, country code, or ASN. You can also configure rules to block user agents. The domain lockdown section of this page allows you to limit access to your domain to certain IP addresses.
- You can review firewall-related events on the **Events** page.

Certificates

When you configure a domain, IBM CIS automatically deploys a universal certificate for that domain. Thus, you don't need to do anything to have certificate-based protection in that domain. If you want, you can upload your own self-managed certificate. You'll need a separate certificate for each domain, and you'll see an error message if the certificate you are uploading does not match your domain. You can also order custom certificates on this page.



Important: If you want to upload your own certificate, you must ensure that you replace expiring certificates before they expire, otherwise your visitors might not be able to connect. Enterprise plan users can set up an expiration notice using the [Alerts feature](#).

Using service endpoints to privately connect to CIS

To help ensure that you have enhanced control and security over your configuration when you use IBM Cloud® Internet Services, you can use private routes to IBM Cloud® service endpoints. Private routes are not accessible or reachable over the internet. By using the IBM Cloud private service endpoints feature, you can protect your configuration from threats from the public network and logically extend your private network.



Note: When you use the CIS private endpoint, any DNS records or global load balancers can be reached on the public network.

Before you begin

To get started, enable [virtual routing and forwarding \(VRF\) and service endpoints](#) for your infrastructure account. After you enable VRF for your account, you can connect to CIS by using a private IP that is accessible only through the IBM Cloud private network.

To learn more about private connections on IBM Cloud, see [Service endpoints for private connections](#).

Setting up service endpoints for CIS



Note: These service endpoints are only for managing the configuration of an instance with the API or CLI, the control plane. Using the IBM Cloud private service endpoints feature does not affect the data plane, and the public endpoints remain available. For information on the control plane and data plane, see [Learning about CIS architecture and workload isolation](#).

Targeting private endpoints

Before you target a private endpoint for CIS:

1. Make sure that your IBM Cloud infrastructure account is enabled for [virtual routing and forwarding \(VRF\)](#).

When you enable VRF, a separate routing table is created for your account, and connections to and from your account's resources are routed separately on the IBM Cloud network. To learn more about VRF technology, see [Virtual routing and forwarding on IBM Cloud](#).



Important: Enabling VRF permanently alters the networking for your account. Be sure that you understand the impact to your account and resources. After you enable VRF, it can't be disabled.

2. Make sure that your IBM Cloud infrastructure account is enabled for [service endpoints](#).



Note: After you enable VRF and service endpoints for your account, all existing and future CIS resources and instances become available from both the public and private endpoints.

From the CLI

Step 1. Configure the IBM Cloud private network on your virtual server

Prepare your virtual server instance or test server by configuring your routing table for the IBM Cloud private network.

1. To route traffic to the IBM Cloud private network, run the following command on your virtual server instance:

```
$ route add -net 166.9.0.0/16 gw <gateway> dev <gateway_interface>
```

Replace `<gateway>` (for example, `10.x.x.x`) and `<gateway_interface>` (for example, `eth10`) with the appropriate values.

2. Optional: Verify that the route was added successfully by displaying your new routing table.

```
$ route -n
```

Step 2. Target the CIS private endpoint

After you configure your virtual server instance to accept IBM Cloud private network traffic, you can target the private endpoint for CIS by using the CIS API or CIS CLI plug-in.

1. Log in to the IBM Cloud console.

```
$ ibmcloud login
```

 **Note:** If the login fails, run the `ibmcloud login --sso` command to try again. The `--sso` parameter is required when you log in with a federated ID. If you use this option, go to the link listed in the CLI output to generate a one-time passcode.

2. Optional: Make sure that your account is enabled for VRF and service endpoints.

```
$ ibmcloud account show
```

The following CLI output shows the account details of a VRF and service endpoint-enabled account.

```
Retrieving account John Doe's Account of john.doe@email.com...
OK

Account ID:          d154dfbd0bc2edefthyuffc9b5ca318
Currently Targeted Account:  true
Linked Softlayer Account:  1008967
VRF Enabled:         true
Service Endpoint Enabled:  true
```

 **Tip:** To learn how to set up your account for connecting to a private network, see [Enabling VRF and service endpoints](#).

Step 3. Create a CIS resource on the private network

Test your private network connection by using the [CIS CLI plug-in](#).

1. Log in to a CIS private endpoint instance.

```
$ ibmcloud login -a private.cloud.ibm.com
```

With the API

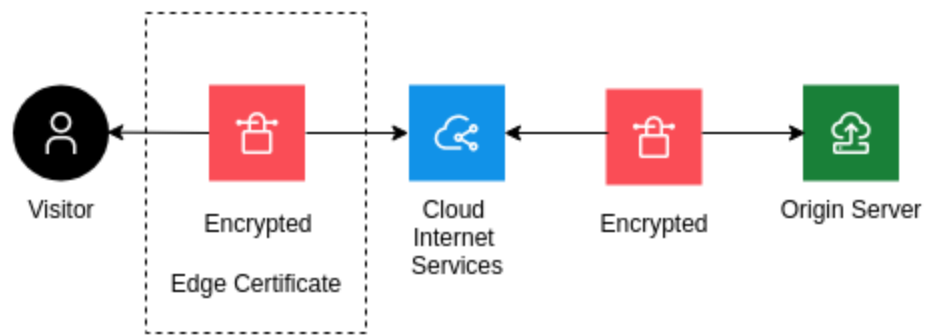
Use the service endpoint's FQDN `api.private.cis.cloud.ibm.com` in the URL to access the service, as shown in the following example.

```
$ curl https://api.private.cis.cloud.ibm.com/v1/:crn/zones -H 'content-type: application/json' -H 'accept: application/json' -H 'x-auth-user-token: Bearer xxxxxx'
```

Managing certificates

Managing edge certificates

Edge certificates are TLS certificates that are presented to users when they access your domain over HTTPS. These certificates secure the connection between the browser and the IBM Cloud® Internet Services edge network. CIS offers three types of edge certificates: Universal, Advanced, and Custom.



Universal certificates

By default, CIS issues free, unshared, publicly trusted SSL certificates to all domains added on CIS. For these Universal certificates, CIS controls the validity periods and certificate authorities (CAs), making sure that renewals always occur. Universal certificates that are issued by Let's Encrypt or Google Trust Services have a 90-day validity period.

CIS can change the CA of Universal certificates without prior notice, and notifies you of these changes. If you prefer to select your own issuing certificate authority, order an advanced certificate.

To change the universal certificate settings from the API, see [Comparing CIS plans](#) for more information. If you need more, submit a support case. For more information, see [Creating support cases](#).

Selecting certificates for a hostname

CIS applies TLS certificates to a hostname by evaluating several criteria in a defined order. This process ensures that the most specific and appropriate certificate is used for each request.

Certificate selection order

For a specific hostname, CIS determines which certificate to present by using the following precedence rules:

- Hostname specificity:** A specific subdomain certificate (`www.example.com`) takes precedence over a wildcard certificate (`*.example.com`) for requests to `www.example.com`.
- Zone specificity:** A specific subdomain certificate (`www.example.com`) takes precedence over a custom hostname certificate when the domain is active as a zone in CIS.
- Certificate priority:** When multiple certificates match the same hostname, CIS selects the certificate based on its priority level.

Priority	Certificate type
1	Custom certificates
2	Advanced certificates
3	Universal certificates

certificate type priority

- Certificate expiration:** If multiple certificates still qualify, CIS presents the certificate that was issued most recently. If a certificate is removed, CIS selects the remaining certificate with the latest expiration date.



Note: When CIS renews a certificate, the renewed certificate receives a new expiration date. As a result, CIS presents the renewed certificate because it now has the latest expiration date.

Certificate renewal failure and replacement

For certificates managed by CIS, renewal attempts begin at the auto renewal period and continue until 24 hours before the expiration. If a certificate fails to renew and another valid certificate exists for the hostname, CIS deploys the valid certificate within these last 24 hours.

CAA records

A [Certificate Authority Authorization \(CAA\) DNS record](#) specifies which certificate authorities (CAs) are allowed to issue certificates for a domain. This record reduces the chance of unauthorized certificate issuance and promotes standardization across your organization.

The following table lists the CAA record content for each CA:

Certificate authority	CAA record content
Let's Encrypt	letsencrypt.org
Google Trust Services	pki.goog; cansignhttpexchanges=yes
SSL.com	ssl.com
DigiCert	digicert.com; cansignhttpexchanges=yes
Sectigo	sectigo.com
CAA record content for each CA	

Certificate statuses

Each certificate status describes your current position in the issuance process, and can vary depending on the type of certificate.

New certificate statuses

When you order a new certificate, whether it's an edge certificate or one for a custom hostname, its status moves through the following stages as it progresses to the global network.

1. **Initializing**
2. **Pending Validation**
3. **Pending Issuance**
4. **Pending Deployment**
5. **Active**

After you issue a certificate, it moves to **Pending Validation**, and changes to **Active** after the validation is completed. If you see any errors, you might need to take more actions to validate the certificate.

If you deactivate a certificate, it moves to **Deactivating** and then the **Inactive** status.

Custom certificate statuses

When you use a custom certificate and your zone status is **Pending** or **Moved**, your certificate might have a status of **Holding Deployment**.

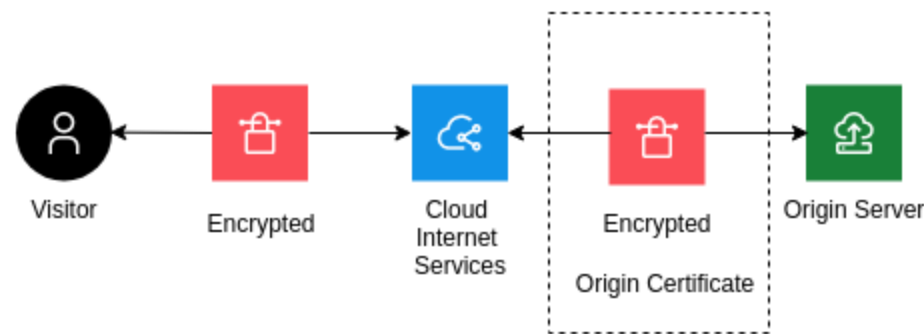
When your zone becomes active, your custom certificate deploys automatically and changes to an **Active** status. However, if your zone is already active when you upload a custom certificate, you don't see this status.

Related links

- [Known limitations for certificates](#)
- [Introducing: Backup Certificates](#)

Managing origin certificates

Origin certificates are TLS certificates that are issued by IBM Cloud® Internet Services to secure traffic between the CIS edge and your origin server. Order free TLS certificates to install on your origin server.



A diagram of an origin certificate

Note: CIS origin certificates are valid for use only in CIS.

Ordering an origin certificate

To order an origin certificate, provide a Certificate Signing Request (CSR) or select a private key type for CIS to generate a key and CSR.

Specify up to 100 hostnames (including wildcards) on your origin that the certificate protects. Wildcards provide only one level of coverage. Use multiple wildcards on the same certificate for broader coverage (for example, `*.yourdomain.com` and `*.yoursubdomain.yourdomain.com`). CIS Origin Certificates do not permit IP addresses.

Specify the expiration. The default certificate expiration is 15 years; the shortest expiration is seven days.

Important: The private key is available only after you order a certificate if the private key and CSR were generated by CIS.

Tip: To get an origin certificate with the API, see [Get an existing origin certificate](#).

Installing an origin certificate on your server

Apache HTTPD

1. Order an origin certificate.
2. Copy the private key and origin certificate in PEM format into separate files to the directory on your server where you keep your key and certificate files.
3. Locate your Apache configuration file. Typically, the file names are `httpd.conf` or `apache2.conf` and the locations are `/etc/httpd/` or `/etc/apache2/`. However, your configuration file might vary, especially if you use a special interface to manage your server. See Apache's [DistrosDefaultLayout](#) for a complete list of default installation layouts. The following command is one way to search for the SSL configuration file on Linux.

```
$ grep -i -r "SSLCertificateFile" /etc/httpd/
```

4. Locate the `<VirtualHost>` block to configure. Optionally, copy the existing non-secure virtual host for your site to be available through HTTP and HTTPS because each type of connection requires a virtual host.
5. Configure the `<VirtualHost>` block for SSL. The following example represents a simple configuration for SSL. Use the filenames for your certificate and private key. `SSLCertificateFile` is your origin CA certificate filename and `SSLCertificateKeyFile` is your origin CA private key filename.

```
<VirtualHost 192.168.0.1:443>
  DocumentRoot      /var/www/html2
  ServerName        www.mydomain.com
  SSLEngine          on
  SSLCertificateFile /path/to/your_domain_name.crt
  SSLCertificateKeyFile /path/to/your_private.key
</VirtualHost>
```

6. Test your configuration. Before you restart Apache, verify that your configuration files have no errors. Run the following command to test your configuration.

```
$ apachectl configtest
```

7. Restart Apache. Run the following commands to restart Apache with SSL support.


```
$ apachectl stop
apachectl start
```



Note: If SSL support does not load with `apache start`, run the command `apachectl startssl`. If Apache starts with only SSL support by using `apachectl startssl`, adjust the Apache startup configuration to include SSL support in the command `apachectl start`. Otherwise, if a server restarts, you might be required to manually restart Apache with `apachectl startssl`. This restart typically involves removing the `<IfDefine SSL>` and `</IfDefine SSL>` tags that enclose your configuration.

NGINX

1. Order an origin certificate.
2. Copy the private key and origin certificate in PEM format into separate files to the directory on your server where you keep the key and certificate files.
3. Update your NGINX virtual hosts file. Edit the NGINX virtual host file for your website. The following example represents a server block for SSL support. Enable the `ssl` parameter on listening sockets in the server block for your site to be available through HTTP and HTTPS.

```
server {
    listen 80;
    listen 443;

    ssl on;
    ssl_certificate /path/to/your_certificate.pem;
    ssl_certificate_key /path/to/your_private.key;
    location / {
        root /home/www/public_html/yourdomain.com/public;
        index index.html;
    }
}
```

4. Restart NGINX. Run one of the following commands to restart NGINX.

```
$ sudo /etc/init.d/nginx restart
sudo systemctl restart nginx
```

Apache Tomcat

1. Order an origin certificate.
2. Copy the private key and origin certificate in PKCS #7 format (`cert.p7b`) to separate files in the directory on your server where you keep the key and certificate files.



Note: You must install the SSL Certificate file to the same keystore and under the same alias name (or "server") that you used to generate your CSR. The installation in the next step does not work if the SSL Certificate file is installed to a different keystore.

3. Install the certificate. Run the following command to install the SSL Certificate file to your keystore.

```
$ keytool -import -trustcacerts -alias server -file cert.p7b -keystore your_site_name.jks
```

A confirmation message appears: **"Certificate reply was installed in the keystore."** Enter **y** or **yes** if asked to trust the certificate. Your keystore file (your_site_name.jks) is now ready to use on your Tomcat Server.

4. Configure your SSL connector. Configure an SSL connector for Tomcat to be able to accept secure connections.
 - a. Open the Tomcat server.xml file in a text editor. The server.xml file is typically located in the conf folder of your Tomcat's home directory.
 - b. Identify the connector to use to secure the new keystore. A connector with port 443 or 8443 is typically used.
 - c. Remove any comment tags that might be surrounding the connector.
 - d. Update the correct keystore filename and password in your connector configuration.

The following example represents a configured SSL Connector block.

```
<Connector port="443" maxHttpHeaderSize="8192" maxThreads="150"
minSpareThreads="25" maxSpareThreads="75" enableLookups="false"
```

```
disableUploadTimeout="true" acceptCount="100" scheme="https"
secure="true" SSLEnabled="true" clientAuth="false" sslProtocol="TLS"
keyAlias="server" keystoreFile="/home/user_name/your_site_name.jks"
keystorePass="your_keystore_password" />
```

 **Note:** If your Tomcat version is earlier than Tomcat 7, change `keystorePass` to `keypass` .

- 5. Save your server.xml file.
- 6. Restart Tomcat.

Microsoft Internet Information Services (IIS) 7.0

- 1. Create a CSR in IIS Manager and export it as `.pem` . The IIS Manager is located under Administrative Tools.
- 2. Order a CIS origin certificate by using your CSR.
- 3. Copy the origin certificate to the desktop of your server.
- 4. Open IIS Manager and select your server's hostname under **Connections**.
- 5. Select **Server Certificates** from the IIS section in the center menu.
- 6. Select the action **Complete Certificate Request** from the Actions menu. On the **Specify Certificate Authority Response** page under **File name containing the certification authority's response**, click `...` to browse to the `.cer` certificate file that was copied to the desktop, select the file, and click **Open**.
- 7. Enter a friendly name for the certificate. The friendly name identifies the certificate.
- 8. Select **OK** to finish the certificate installation to your server.
- 9. Bind the certificate to your website. Select your website by expanding **Sites** under your server's name in the menu under **Connections** in the IIS Manager. Select **Bindings** under **Edit Site** from the Actions menu. Select **Add** from the Site Bindings window and submit the following information.

Type	https
IP Address	All Unassigned
Port	443
SSL Certificate	your_cert_friendly_name

- 10. Your website is now configured to accept secure connections.

Microsoft Internet Information Services (IIS) 8.0 and 8.5

- 1. Create a CSR in IIS Manager and export it as `.pem` . The IIS Manager is located under Administrative Tools.
- 2. Order a CIS origin certificate by using your CSR.
- 3. Copy the origin certificate to the desktop of your server.
- 4. Open IIS Manager and select your server's hostname under **Connections**.
- 5. Select **Server Certificates** from the IIS section in the center menu.
- 6. Select the action **Complete Certificate Request** from the Actions menu. On the **Specify Certificate Authority Response** page under **File name containing the certification authority's response**, click `...` to browse to the `.cer` certificate file that was copied to the desktop, select the file, and click **Open**.
- 7. Enter a friendly name for the certificate, which identifies the certificate.
- 8. Select **OK** to finish the certificate installation to your server.
- 9. Bind the certificate to your website. Select your website by expanding **Sites** under your server's name in the menu under **Connections** in the IIS Manager. Select **Bindings** under **Edit Site** from the Actions menu. Select **Add** from the Site Bindings window and submit the following information.

Type	https
IP Address	All Unassigned
Port	443
SSL Certificate	your_cert_friendly_name

- 10. Optionally, configure your SSL certificate to use Server Name Indication (SNI) if you have multiple sites that use SSL bound to the same IP address.

Select the **Require Server Name Indication** box.

11. Your website is now configured to accept secure connections.

Certificate chains

In some cases, the origin web servers require you to upload the certificate chain. Use these links to download either an [ECC](#) or an [RSA](#) version, and then upload the certificate chain to your origin web server.

Revoking an origin certificate

Delete your CIS origin certificate. This process can't be undone.

To revoke an origin certificate with the API, see [Revoke a created origin certificate](#).

Renewal and expiration of certificates

Understand how custom certificates are renewed, how expiration is handled, and what happens to your domains when certificates expire in CIS.

Renewing custom certificates

Replace or update custom certificates before they expire, as CIS cannot renew certificates that you upload. If certificates are not replaced in time, users might not be able to connect to your application.

CIS provides certificate-related alert policy types for TLS certificates such as certificate validation, renewal, and expiration. To receive these alerts, customers must explicitly configure the corresponding CIS alert policies, as certificate alerts are not enabled by default. For more information, see [Alert types](#).

Expired certificates

If a valid replacement certificate that covers some or all of the SANs in the expiring custom certificate is already available, CIS removes the expiring certificate within 24 hours before expiration. No downtime is expected during this transition.

If no valid replacement certificate is available, CIS removes the custom certificate after it expires.

Affected domains and subdomains fall back to any other active certificate that covers the hostnames that are associated with the expiring certificate (for example, Universal or Advanced certificates).



Note: Certificates in a certificate pack are treated as a single unit. The pack expires when the first certificate reaches its **Not After** date. For example, if a pack includes both ECDSA and RSA certificates, the entire pack is removed when either certificate expires.

Working with Range applications

About Range applications

Range applications extend DDoS protection, TLS encryption, and custom rule policies to TCP and UDP services, not just traditional HTTP/HTTPS traffic. These apps secure your TCP-based services beyond just web traffic and provide comprehensive security and performance benefits within IBM Cloud CIS.

Range application use cases

Use cases for Range applications include:

Protecting a corporate VPN gateway

A company exposes a VPN for remote employees. A Range application ensures the VPN remains available during DDoS attacks while masking the origin IP.

Safeguarding NAT and network devices

Network Address Translations (NAT) devices manage inbound and outbound traffic. Range applications prevent abuse while maintaining reliable connectivity.

Securing a partner-accessible database

A MySQL or PostgreSQL database is accessed by trusted partners. Range applications allow traffic to pass securely without exposing the database server to the open internet.

Protecting collaboration and third-party integrations

A company integrates with collaboration platforms such as Zoom or Microsoft Teams. Because traffic originates from a known and limited set of IP ranges, a Range application allows this traffic to pass uninterrupted while preventing it from being misclassified as DDoS activity.

Range application WAF support use cases

Range applications can be configured to operate at either Layer 7 (HTTPS) or Layer 4 (TCP), which determines whether WAF inspection is applied. The following use cases describe how WAF support varies based on the application type and traffic handling model.

Enabling WAF inspection for HTTPS-based Range applications

Use a Range application with type **HTTPS** (for example, port **9443**) when your application requires Layer 7 protection.

In this configuration, the TLS handshake is ended at the CIS edge. After TLS is terminated at the CIS edge, traffic is decrypted and inspected by the WAF. CIS can then re-encrypt the traffic before forwarding it to the origin, ensuring secure communication between CIS and the origin.

As a result, traffic is inspected and protected before it reaches the back-end service, while secure communication between CIS and the origin is maintained.

Example of traffic flow: **Client → HTTPS:443 → CIS (WAF inspection) → HTTPS:9443 → Origin**

Using TCP-based Range applications without WAF support

Use Range applications with **TCP** for services that require **layer 4** proxying.

In this configuration, CIS operates at the transport layer and does not stop TLS or inspect **layer 7** traffic. Because the traffic is forwarded as TCP, the payload is not decrypted and is not evaluated by the WAF.

As a result, WAF protections are not applied. Instead, CIS provides network level capabilities such as DDoS protection, IP masking, and secure TCP proxying while it forwards traffic directly to the origin.

Example of traffic flow: **Client → TCP → CIS (L4 proxy/DDoS protection) → TCP → Origin**

Supported protocols and use cases

The application type determines how traffic is routed from the CIS edge to your origin:

- TCP or UDP: Use when you want CIS to proxy traffic directly to your origin.
- HTTP or HTTPS: Required for Edge functions or Bot Management; traffic is routed through the CIS pipeline.
- RDP: Securely connect by using Remote Desktop Protocol (RDP), with built-in DDoS protection and access control.

- SSH: Securely access remote servers over SSH without exposing your origin IP.
- Minecraft: Protect and accelerate Minecraft servers with DDoS mitigation and improved network performance.



Note: For protocol-specific limitations, see [Known issues and limitations](#).

Security and port configuration

By default, the Range application listens on all ports, which might raise concerns during security audits. However, the Range application only proxies connections from edge ports explicitly that are configured in CIS.

When a TCP handshake is initiated on any port of a Range application IP, CIS may complete the handshake at the edge. However, only ports explicitly configured for a Range application are proxied to the origin. Connections to other ports are immediately terminated at the edge without reaching the origin.

The Range application proxies traffic to the origin only when a Range application is configured for the requested port.

Creating Range applications

Creating a Range application involves configuring how traffic is routed to your origin servers, with support for specifying protocols, origin settings, and advanced features like TLS termination and custom rules. You can add a Range application by using the CIS console or API.



Note: Range is only available to Enterprise customers for an additional cost, and is priced per bandwidth usage.

Before you begin

Review the following planning considerations:

- You can create up to 10 Range applications, each with a unique origin. Every Range application with a unique origin requires a unique IP address, which are a limited resource. If you need more than 10 Range applications, submit a support case to IBM. Requests to increase the limit require a use case review and may take several days



Tip: You can create more than 10 applications if they share an existing origin, but use different ports.

- For TCP Range applications, only custom IP rules apply because these rules operate at OSI Layer 3 and 4. HTTP(S) Range applications support both firewall rules and IP rules. Generally, firewall rules are designed for Layer 7 (HTTP) properties, such as request headers and body content.



Note: For more information, see [Range applications](#).

Creating a Range application in the console

To add a Range application in the console, follow these steps:

1. In the CIS console, navigate to the **Security** section.
2. Select the **Range** tab, then click **Create**.
3. Select an application type from the list: TCP, UDP, HTTP, HTTPS, RDP, SSH, or Minecraft. For more information, see [Protocols and use cases](#).
4. Enter the application name. This name associates your application with a DNS name on your CIS domain.
5. Enter the edge port. CIS listens for incoming connections on these ports and proxies them to your origin. Connections to these addresses are proxied to your origin.



Note: You can enter a port range for example, (), but the origin must have an equal number of consecutive ports matching the range.

6. Select the edge IP connectivity.
7. In the Origin section, enter the origin IP and port of your TCP application, or select an existing load balancer and port.
8. Optionally, enable custom rules. When enabled, rules with a "block" or "allowlist" action are enforced for the application.
9. Optionally, enable edge TLS termination. When enabled, select the type of TLS termination you want to use from the list.

10. Optionally, select a [PROXY Protocol](#) if you have a proxy inline that supports it. This is useful if you are running a service that requires knowledge of the true client IP. Usually, this setting remains `off`.
11. Click **Create**.

Provisioning a Range application incurs additional costs, based on the amount of bandwidth used per application.

Viewing metrics

Your application is now ready to proxy TCP traffic through CIS (Cloudflare).

Navigate to **Metrics > Range** to view your number of connections to applications and throughput traffic.

 **Note:** The graphs show metrics for up to 10 applications.

To toggle application metrics, use the Chart key or click **Select applications**. To change the Metrics data timeframe, use the list menu.

API usage examples

Use the following examples to create and list the Range applications.

Creating Range applications

You can designate an origin in two ways in Range applications:

- Origin IP - Use the `origin_direct` parameter.
- Load balancer - Use the `origin_dns` and `origin_port` parameters.

For the origin IP request:

```
curl --request POST \
  --url https://api-int.cis.dev.cloud.ibm.com/v1/crn:v1:staging:public:internet-svcs-ci:global:a/ce12845bf2914ca18db35bedcd9aefa2:e04e7ccd-9197-4b6d-bf98-933de5074fe6::/zones/2566bebb17f5c0b559a9873a692ccab5/range/apps \
  --header 'Content-Type: application/json' \
  --header 'X-AUTH-USER-TOKEN: Bearer XXXX' \
  --data '{"protocol":"tcp/22","dns":{"type":"CNAME","name":"origin.entdemo5.batflare.com"},"origin_direct":["tcp://8.8.8.8:22"],"proxy_protocol":"off","ip_firewall":true}'
```

Example response:

```
{
  "result": {
    "id": "de4d336aea6f4f56b998ffdec9a0f00c",
    "protocol": "tcp/22",
    "dns": {
      "type": "CNAME",
      "name": "origin.entdemo5.batflare.com"
    },
    "origin_direct": [
      "tcp://8.8.8.8:22"
    ],
    "ip_firewall": true,
    "proxy_protocol": "off",
    "tls": "off",
    "traffic_type": "direct",
    "edge_ips": {
      "type": "dynamic",
      "connectivity": "all"
    },
    "created_on": "2025-08-13T07:39:54.175727Z",
    "modified_on": "2025-08-13T07:39:54.175727Z"
  },
  "success": true,
  "errors": [],
  "messages": []
}
```

Example request using a load balancer:

```
curl -X POST \
https://api.cis.cloud.ibm.com/v1/<url-encoded-crn>/zones/<zone-d>/range/apps \
-H 'X-Auth-User-Token: <token>' \
-d '{"protocol":"tcp/22","dns":{"type":"CNAME","name":"ssh.example.com"},"origin_dns":{"name":"test"},"proxy_protocol":true,"ip_firewall":true, "origin_port": 43}'
```

Example response:

```
{
  "result": {
    "id": "4f70c3d4f20576b79135b898295e8093",
    "protocol": "tcp/22",
    "dns": {
      "type": "CNAME",
      "name": "ssh.example.com"
    },
    "origin_dns": {
      "name": "test"
    },
    "origin_port": 43,
    "ip_firewall": true,
    "proxy_protocol": true,
    "created_on": "2019-01-09T17:39:09.190606Z",
    "modified_on": "2019-01-09T17:39:09.190606Z"
  },
  "success": true,
  "errors": [],
  "messages": []
}
```

DNS Name

The DNS name associated with your application on your domain.

Protocol/Edge port

The port your application listens on; incoming connections are proxied to your origin.

Origin direct

The IP and port where your application runs, specifying traffic flow from edge to origin.

IP Firewall

If enabled, CIS enforces custom rule criteria, such as allowing, blocking, or challenging specific IPs or ranges, replacing legacy IP firewall behavior to provide more granular access control.

PROXY Protocol

Enable only if you have an inline proxy supporting PROXY Protocol v1; usually disabled.

Origin DNS

The load balancer name used as your origin.

Origin Port

The port of your origin service.

For more information, see [Creating Range applications](#).

Listing all Range applications

Use the following request to list all Range applications:

```
curl --request GET \
--url https://api-int.cis.dev.cloud.ibm.com/v1/crn:v1:staging:public:internet-svcs-ci:global:a/ce12845bf2914ca18db35bedcd9aefa2:e04e7ccd-9197-4b6d-bf98-933de5074fe6::/zones/2566bebb17f5c0b559a9873a692ccab5/range/apps \
--header'content-type: application/json'
--header 'X-AUTH-USER-TOKEN: Bearer XXXX' \
```


Example response:

```
{
  "result": [
    {
      "id": "de4d336aea6f4f56b998ffdec9a0f00c",
      "protocol": "tcp/22",
      "dns": {
        "type": "CNAME",
        "name": "origin.entdemo5.batflare.com"
      },
      "origin_direct": [
        "tcp://8.8.8.8:22"
      ],
      "ip_firewall": true,
      "proxy_protocol": "off",
      "tls": "off",
      "traffic_type": "direct",
      "edge_ips": {
        "type": "dynamic",
        "connectivity": "all"
      },
      "created_on": "2025-08-13T07:39:54.175727Z",
      "modified_on": "2025-08-13T07:39:54.175727Z"
    }
  ],
  "success": true,
  "errors": [],
  "messages": []
}
```

For more information, see [Listing Range applications](#).

Listing specific Range applications

Use the following request to list specific Range applications:

```
$ curl -X GET \
https://api.cis.cloud.ibm.com/v1/<url-encoded-crn>/zones/<zone-d>/range/apps/4f70c3d4f20546b79135b898295e8093
```

Example responses:

- Applications by Origin IP:

```
{
  "result": {
    "id": "4f70c3d4f20546b79135b898295e8093",
    "protocol": "tcp/22",
    "dns": {
      "type": "CNAME",
      "name": "ssh.example.com"
    },
    "origin_direct": [
      "tcp://172.0.2.1:22"
    ],
    "ip_firewall": true,
    "proxy_protocol": true,
    "created_on": "2019-01-09T17:33:09.190606Z",
    "modified_on": "2019-01-09T17:33:09.190606Z"
  },
  "success": true,
  "errors": [],
  "messages": []
}
```

- Applications by load balancer:

```
{
  "result": {
```

```
{
  "id": "555359036e7f4acc82d69b916f62caba",
  "protocol": "tcp/22",
  "dns": {
    "type": "CNAME",
    "name": "ssh.example.com"
  },
  "origin_dns": {
    "name": "test_update"
  },
  "origin_port": 76,
  "ip_firewall": true,
  "proxy_protocol": true,
  "created_on": "2019-01-10T22:26:47.167008Z",
  "modified_on": "2019-01-10T22:26:47.167008Z"
},
"success": true,
"errors": [],
"messages": []
}
```

Enabling Proxy protocol

IBM Cloud Internet Services intercepts packets before forwarding them to your server, so that if you look up a client IP, you see CIS's IP rather than the true client IP. However, some services you run might require you to know the true client IP. In those cases, you can use a proxy protocol for CIS to pass the client IP to your service.

Sending proxy information along is dependent on whether TCP or UDP is used. For TCP, Range supports adding Proxy Protocol v1, which is the human readable-version supported by Amazon ELB and NGINX. For UDP applications, CIS has developed a custom proxy protocol called Simple Proxy Protocol.



Note: This feature requires an Enterprise plan. If you would like to upgrade, contact your account team. For more information, see [Comparing CIS plans](#).

Enabling proxy protocol v1 for TCP

1. In the CIS console, navigate to the **Security** section.
2. Select the **Range** tab.
3. Locate the application that will use the PROXY protocol and click **Configure**.
4. From the menu, select **PROXY Protocol v1**.

When TCP applications are configured to use proxy protocol v1, CIS prepends each inbound TCP connection with the proxy protocol plain-text header.

The proxy protocol v1 header

The proxy protocol prepends every connection with a header reporting the client IP address and port. A proxy protocol plain-text header has the following format:

```
PROXY_STRING + single space + INET_PROTOCOL + single space + CLIENT_IP + single space + PROXY_IP + single space + CLIENT_PORT + single space + PROXY_PORT + "\r\n"
```

The following is an example proxy protocol line for an IPv4 address:

```
PROXY TCP4 192.0.2.0 192.0.2.255 42300 443\r\n
```

The following is an example proxy protocol line for an IPv6 address:

```
PROXY TCP6 2001:db8:: 2001:db8:ffff:ffff:ffff:ffff:ffff:ffff 42300 443\r\n
```

Enabling Proxy Protocol v2 for TCP/UDP

1. In the CIS console, navigate to the **Security** section.
2. Select the **Range** tab.
3. Locate the application that will use the proxy protocol and click **Configure**.
4. From the menu, select **PROXY Protocol v2**.

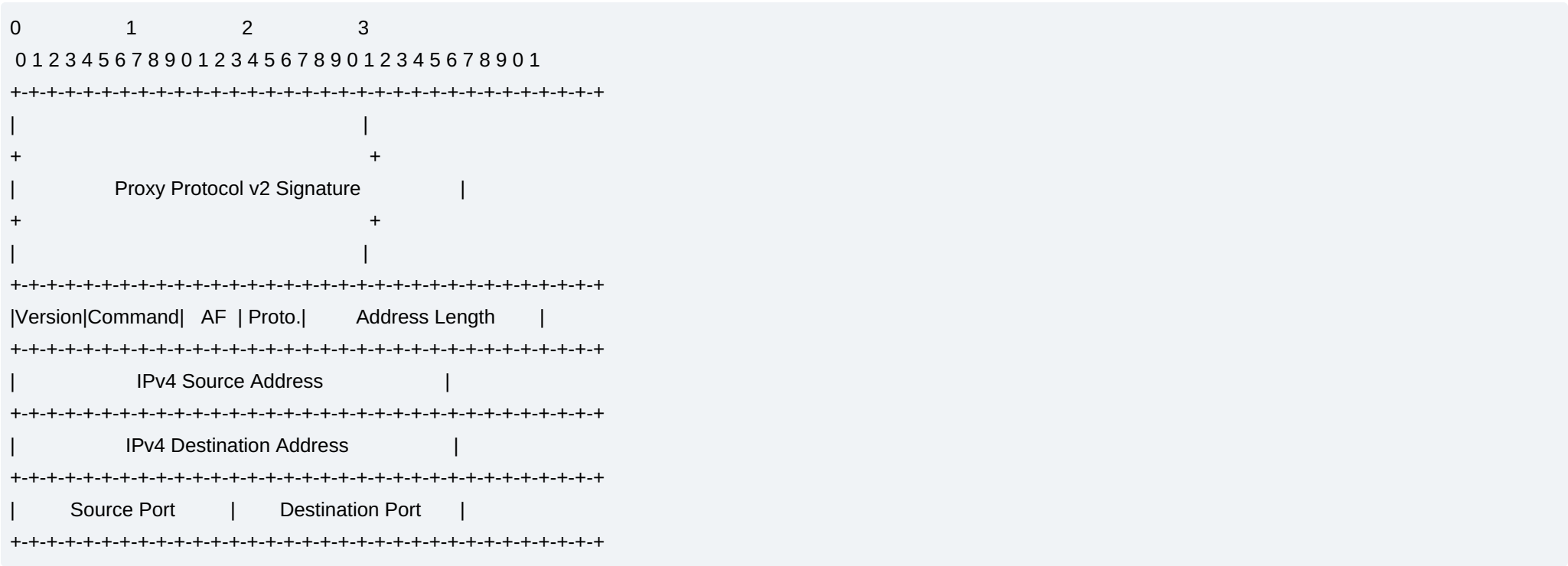
When TCP applications are configured to use proxy protocol v2, CIS prepends each inbound TCP connection with the proxy protocol binary header.

When UDP applications are configured to use PROXY Protocol v2, CIS prepends the first UDP datagram on a stream with a proxy protocol binary header.

The proxy protocol v2 header

The proxy protocol prepends every connection with a header reporting the client IP address and port.

A proxy protocol binary header for a IPv4 incoming address has the following format:



A proxy protocol binary header for a IPv6 incoming address has the following format:



Enable Simple Proxy Protocol for UDP

When using UDP, the client source IP and port information can be obtained by using Simple Proxy Protocol, a lightweight protocol developed specifically for UDP.

To enable it, click **Configure** on a Range application and toggle the setting for **Simple Proxy Protocol** to **On**.

Simple Proxy Protocol dictates that your origin must also prepend packets meant for the client with the same header, including original client source information. This is done to validate that packets coming in are in fact intended for the client.

Working with TLS

About TLS

Transport Layer Security (TLS) is a standard security protocol for establishing encrypted links between a web server and a browser in an online communication. A TLS certificate is necessary to create a TLS connection with a website and comprises the domain name, the name of the company, and additional data, such as company address, city, state, and country. The certificate also shows the expiration date and details of the issuing certificate authority (CA).

How Does TLS Work?

When a browser initiates a connection with a TLS-secured website, it first retrieves the site's TLS Certificate to check whether the certificate is still valid. It verifies that the CA is one that the browser trusts, and that the certificate is used by the website for which it is issued. If any of these checks fail, you get a warning, which indicates that the website is not secured by a valid certificate.

When a TLS certificate is installed on a web server, it enables a secure connection between the web server and the browser that connects to it. The website's URL is prefixed with "HTTPS" instead of "HTTP" and a padlock is shown on the address bar. If the website uses an extended validation (EV) certificate, the browser might also show a green address bar.

Setting Transport Layer Security (TLS) options

The Transport Layer Security (TLS) options let you control whether visitors can browse your website over a secure connection, and when they do, how IBM Cloud® Internet Services connects to your origin server.



Tip: Use the latest version of the TLS protocol (TLS 1.3) for improved security and performance by switching from **Off** to **On**.

TLS encryption modes

Set the TLS mode by selecting one of the following options from the Mode list.

These options are listed in the order from the most secure to the least secure (Off):

1. [Authenticated origin pull](#): (Enterprise only)
2. [HTTPS only origin pull](#) (Enterprise only)
3. [End-to-end CA signed](#) (default and recommended)
4. [End-to-end flexible](#) (edge to origin certificates can be self-signed)
5. [Client-to-edge](#) (edge to origin not encrypted, self-signed certificates are not supported)
6. [Off](#) (not recommended)

Authenticated origin pull

Enterprise only. In this mode, a secure connection is set up between the visitor and CIS and a secure connection between CIS and your web server. When CIS connects to your origin server, it presents a client certificate for authentication. This process helps ensure that only authorized requests from CIS can access your origin server. Use this mode if you want only trusted connections to reach your server. For more information, see [Authenticated origin pull](#).

HTTPS only origin pull

Enterprise only. In this mode, a secure connection is set up between the visitor and CIS, and a secure and authenticated connection between CIS and your server. This mode has the same certificate requirements as end-to-end CA signed. However, it upgrades all connections between CIS and your origin webserver from HTTP to HTTPS, even if the visitor requests the original content over HTTP.

End-to-end CA signed

This mode is the default and recommended setup. A secure connection is set up between the visitor and CIS, and a secure and authenticated connection between CIS and your web server. Your server must be set up to handle HTTPS connections, with a valid TLS certificate. This certificate must be signed by a certificate authority (CA), have an expiration date in the future, and respond for the request domain name (hostname). Use this TLS mode for best security practices, unless you understand the potential security threats of changing to one of the less strict modes.



A diagram of end to end CA signed TLS

End-to-end flexible

In this mode, a secure connection is set up between your visitor and CIS, while a secure but non-authenticated connection between CIS and your web server. Your server must be set up to handle HTTPS connections, at a minimum with a self-signed certificate. However, the authenticity of this certificate is not verified. CIS connects to your origin web server when the address of your origin web server is correctly configured in your DNS settings. Use this mode if you can't obtain a certificate that is signed from a CA. Keep in mind that it is less secure compared to the end-to-end CA signed mode.



A diagram of end to end flexible TLS

Client-to-edge

In this mode, a secure encrypted connection is set up between your visitor and CIS, and a non-secure unencrypted connection between CIS and your web server. You don't need to have a TLS certificate on your web server, but your visitors still see the site as being HTTPS-enabled. This option is not recommended if you have any sensitive information on your website. This mode works only for redirecting traffic from port `443` (HTTPS) to port `80` (HTTP) and must be used only as a last resort if you are not able to set up TLS on your own web server. It is *less secure* than any other option (even "Off"), and might cause trouble when you decide to switch away from it.



A diagram of client to edge TLS

Off

In this mode, no secure connection exists between your visitors and CIS, and between CIS and your web server. Visitors can view your website only over HTTP, and any visitor who attempts to connect with HTTPS receives an `HTTP 301 Redirect` to the plain HTTP version of your website. This mode is not recommended because it exposes all data and is least secure compared to the previous modes.



Traffic encryption - Minimum TLS version

Set the minimum TLS version for traffic that tries to connect to your site by selecting one of the versions from the list.

By default, this version is set to **1.2**. Higher TLS versions provide additional security, but might not be supported by all browsers, which might prevent some customers from connecting to your site.

 **Tip:** To update the minimum TLS version with the API, see [Update minimum TLS version setting](#).

Using mutual TLS

Mutual Transport Layer Security (mTLS) authentication ensures that traffic is both secure and trusted in both directions between a client and server. It is only available to customers at any Enterprise plan level.

When mTLS is configured, access is granted only to requests with a corresponding client certificate. When a request reaches your application, CIS responds with a request for the client certificate. If the client fails to present the certificate, the request is not allowed to proceed. Otherwise, the key exchange proceeds.

Client-authenticated TLS handshake

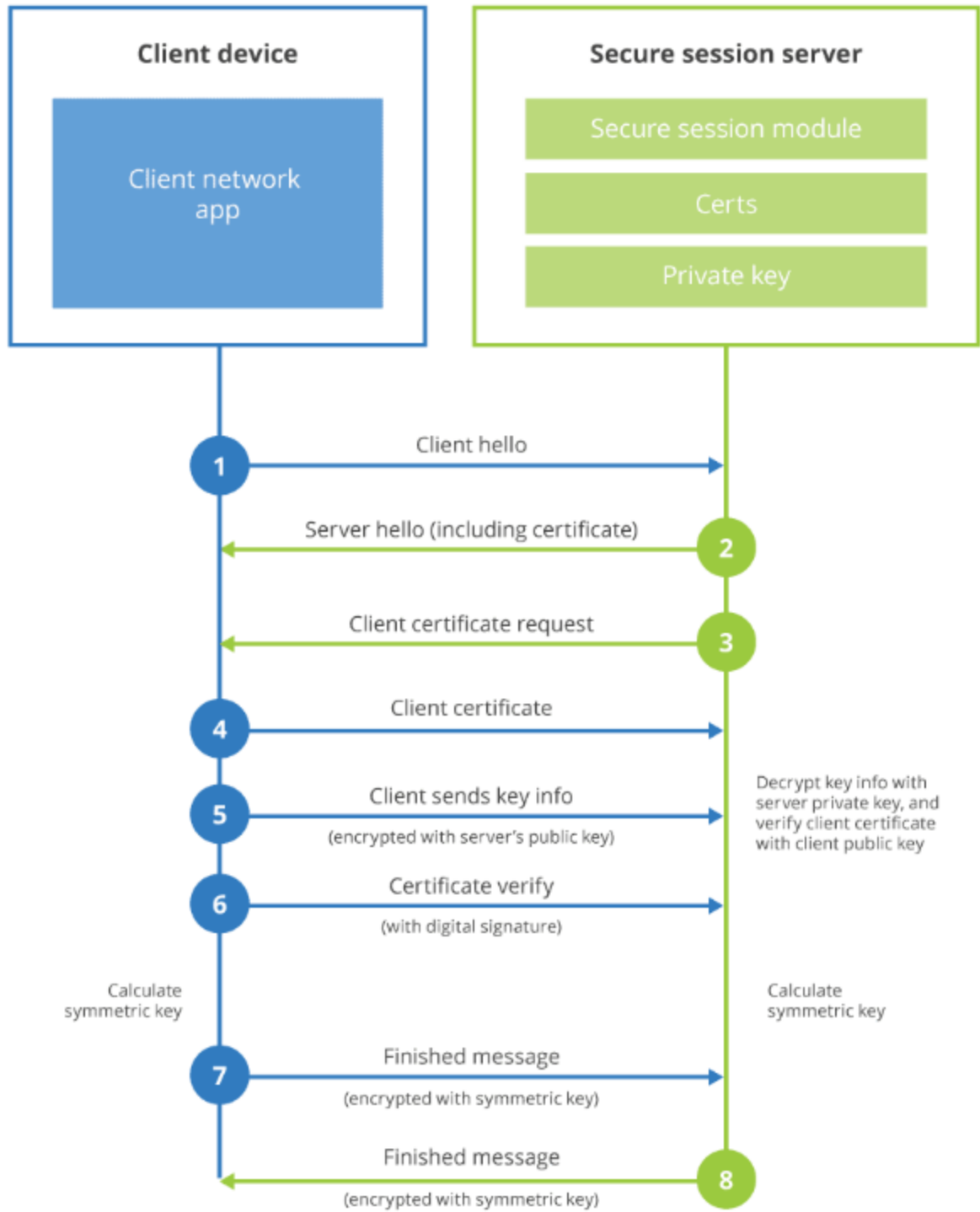


Diagram of an mTLS handshake

Configuring mutual TLS

Mutual TLS (mTLS) provides certificate-based client authentication for enhanced security. mTLS is not enabled by default and requires prior authorization

on a per-domain basis. To configure mTLS, follow these steps:

1. Request authorization. [Create a support case](#) to request mTLS enablement for your IBM Cloud account.



Note: After mTLS is enabled, it can't be disabled.

2. Enable mTLS:

- a. In the CIS console, click **Security**, then select the **Mutual TLS** tab.
- b. Click **Enable**.

3. Upload Root certificates:

- a. In the **Root certificates** table on the **Mutual TLS** page, click **Add** to define a new root certificate.
- b. Paste the certificate content into the Certificate content field. Provide a name for the Root CA certificate, and add one or more fully qualified domain names (FQDNs) of the endpoints that will use this certificate.

These FQDNs are the hostnames that are used by the resources protected by the mTLS application policy. You must associate the Root CA with the FQDN used by the protected application.

- c. Click **Save**.



Note: If using an intermediate certificate, upload the entire certificate chain.

4. Create an mTLS access policy:

- a. In the **MTLS access policies** table on the **Mutual TLS** page, click **Create** to create an access application.
- b. Select or enter a hostname that matches one of the FQDNs associated with the uploaded root certificate and click **Create**.

The application policy is pre-set to use a decision of `non_identity`, and include a rule that matches any valid client certificate.

5. Enable client certificate forwarding with the API:

To forward validated client certificates to your origin server (useful for logging, auditing, or back-end authentication), you must enable forwarding.



Note: The client certificate is forwarded only with the first request in each mTLS connection.

For an API **curl** example to enable client certificate forwarding, see the [Update access certificates settings](#) API.

Client certificate headers sent to your origin server:

- `Cf-Client-Cert-Der-Base64`: Base64-encoded DER version of the client certificate
- `Cf-Client-Cert-Sha256`: SHA-256 fingerprint of the client certificate

To verify forwarding status with the API:

```
$ curl -X GET https://api.cis.cloud.ibm.com/v1/crn:v1:bluemix:public:internet-svcs:global:a/<account-id>:<instance-id>:/zones/<zone-id>/access/certificates/settings \
-H 'X-Auth-User-Token: Bearer <IAM-TOKEN>'
```

6. Create a WAF custom rule to block unauthenticated requests:

To improve security, especially for sensitive endpoints such as login paths or APIs, create a WAF rule to block requests that do not present a valid client certificate.

- a. In the CIS console, navigate back to the **Security** section, then select the **Custom rules** tab.
- b. Click **Create**.
- c. In the Create WAF custom rule side panel, click **Use Expression Editor**.
- d. Enter the following condition (update `hostname` and `path` as needed) in the **Use Expression Builder** field.

```
$ (http.host eq "example.com" and http.request.uri.path eq "/authenticate" and not cf.tls_client_auth.cert_verified)
```

- e. Set the **Action** to **Block**.

f. Click **Create**.



Note: This rule allows access to authenticate only if the client certificate is successfully verified. Requests without mTLS or with invalid certificates are blocked.

Testing mTLS access

The following example uses **curl** to test mTLS authentication by making requests with and without a client certificate.

1. Attempt to access the site without a client certificate.

This example demonstrates using **curl** to test access to a site that enforces mTLS. The target URL in this example is `https://auth.example.com`.

```
$ curl -sv https://auth.example.com
```

Because no client certificate is provided, the request is expected to fail with a `403 Forbidden` response.

2. Add your client certificate and private key to the request:

```
$ curl -sv https://auth.example.com --cert example.pem --key key.pem
```

If the client is properly authenticated, the response includes a `CF_Authorization Set-Cookie` header, indicating successful mTLS authentication.

Validating mutual TLS

When you enable this access policy, use the following workflow to validate client certificates:



Note: All requests to the origin are evaluated for a valid client certificate.

1. The client initiates a connection by sending a `Hello` message.
2. The access application responds with a `Hello` and requests the client certificate.
3. The client sends its certificate for validation.
4. The client certificate is authenticated against the configured root certificate authority.
5. If a certificate chain is used, the system also checks for expired certificates and validates the entire chain.
6. If the client certificate is trusted, a signed JSON Web Token (JWT) is generated for the client that allows the request and subsequent requests to proceed.
7. If the client doesn't present a valid certificate, the server returns a `403 Forbidden` response.

To retrieve access certificates with the API, see [List access certificates](#).

Authenticated origin pull

Authenticated Origin Pull is a security feature that helps ensure secure communication between the IBM Cloud® Internet Services and your origin server. It uses mutual TLS (mTLS) to authenticate both the CIS and the origin server, which allows only requests from the trusted IBM Cloud® Internet Services to access your server. This setup protects your origin server from unauthorized access and enhances the overall security of your web applications.

Origin web servers validate that a web request came from IBM Cloud® Internet Services through authenticated origin pulls. CIS uses TLS client certificate authentication, a feature that is supported by most web servers to present a CIS certificate when it establishes a connection between CIS and the origin web server. By validating this certificate at your origin web server, access is limited to CIS connections.

An authenticated origin pull is important when you take advantage of the Web Application Firewall (WAF). After your origin web server enforces authenticated origin pulls, all HTTPS requests outside of CIS are blocked from reaching your origin.

You can configure authenticated origin pull by using one of the following options:

- Zone-level authenticated origin pull by using CIS certificates
- Zone-level authenticated origin pull by using customer certificates
- Per-Hostname authenticated origin pull by using customer certificates

Client certificates are not deleted from CIS upon expiration unless a delete or replace request is sent to the CIS API. However, requests are dropped at your origin if your origin accepts only a valid client certificate.



Note: Authenticated Origin Pull does not work in **SSL mode Off (not secure)** or **Client-to-Edge**.

Zone-level authenticated origin pull with CIS certificates

This option allows secure content access from the origin server by authenticating requests with CIS provided certificates at the zone level.

CIS uses the following CA to sign certificates for the authenticated origin pull service:

- [Authenticated origin pull certificate](#)

Download the certificate and store the file on your origin web server, for example in `/path/to/origin-pull-ca.pem`.

To enable authenticated origin, pull globally on a zone:

1. Install the certificate at the origin web server to authenticate all connections.
2. Configure CIS with [End-to-End flexible](#).
3. Configure the origin web server to accept client certificates.
4. Enable authenticated origin pull by using the [CIS CLI](#).

Zone-level authenticated origin pull with custom certificates

This option enables secure content access from the origin server by using customer-uploaded certificates for authentication at the zone level.

1. If you use an ECC key that is generated by OpenSSL, first remove `-----BEGIN EC PARAMETERS-----...-----END EC PARAMETERS-----` from the certificate file.
2. Ensure the certificate and key are in the following format before you upload to CIS:

```
$ cat app_example_com.pem
-----BEGIN CERTIFICATE-----
MIIFJDCCBAygAwIBAgIQD0ifmj/Yi5Nz2gdUySbfzANBgkqhkiG9w0BAQsFADBN
MQswCQYDVQQGEwJVUzEVMBMGA1UEChMMRGlnaUNlcnQgSW5jMScwJQYDVQQDEx5E
...
SzSHfXp5Inu/3V08I72q1QNzOCgY1XeL4GKVcj4or6cT6tX6oJH7ePPmfrBfqI/O
OeH8gMJ+FuwtxYEPa4hBf38M5eU5xWG7
-----END CERTIFICATE-----
```

3. Replace line endings with the string `\n`:

```
MYCERT="$(cat app_example_com.pem|perl -pe 's/\r?\n/\n/'|sed -e 's/..$/')" $ MYKEY="$(cat app_example_com.key|perl -pe 's/\r?\n/\n/'| sed -e 's/..$/')"

echo $MYCERT -----BEGIN CERTIFICATE-----\nMIIFJDCCBAygAwIBAgIQD0ifmj/Yi5NP/
2gdUySbfzANBgkqhkiG9w0BAQsFADBN\nMQswCQYDVQQGEwJVUzEVMBMGA1UEChMMRGlnaUNlcnQgSW5jMScwJQYDVQQDEx5E...SzSHfXp5Inu/
3V08I72q1QNzOCgY1XeL4GKVcj4or6cT6tX6oJH7ePPmfrBfqI/O\nOeH8gMJ+FuwtxYEPa4hBf38M5eU5xWG7\n-----END CERTIFICATE-----\n
```

4. Build the payload:

```
$ request_body=${< <(cat <<EOF { "certificate": "$MYCERT", "private_key": "$MYKEY" } } EOF )}
```

5. Upload the client certificate and private key from the [CIS CLI](#).
6. Enable authenticated origin pull from the [CIS CLI](#).

Per-hostname authenticated origin pull by using customer certificates

This option allows secure content access from the origin server by authenticating requests with customer certificates on a per-hostname basis.

When you enable authenticated origin pull per hostname, all proxied traffic to the specified hostname is authenticated at the origin web server. You can use client certificates from your own public key infrastructure to authenticate connections from CIS.

To upload a client certificate in CIS:

1. If you use an ECC key that is generated by OpenSSL, first remove `-----BEGIN EC PARAMETERS-----...-----END EC PARAMETERS-----` from the certificate file.
2. Make sure that the certificate is in the following format before you upload it to CIS.

```
$ cat app_example_com.pem
-----BEGIN CERTIFICATE-----
MIIFJDCCBAygAwIBAgIQD0ifmj/Yi5NP/2gdUySbfzANBgkqhkiG9w0BAQsFADBN
MQswCQYDVQQGEwJVUzEVMBMGA1UEChMMRGlnaUNlcnQgSW5jMScwJQYDVQQDEx5E
...
SzSHfXp5Inu/3V08I72q1QNzOCgY1XeL4GKVcj4or6cT6tX6oJH7ePPmfrBfqI/O
OeH8gMJ+FuwtXYEPa4hBf38M5eU5xWG7
-----END CERTIFICATE-----
```

3. Replace line endings with the string `\n`:

```
MYCERT="$(cat app_example_com.pem|perl -pe 's/\r?\n/\\n/'|sed -e 's/..$//')" $ MYKEY="$(cat app_example_com.key|perl -pe 's/\r?\n/\\n/'|sed -e 's/..$//')"
```

```
echo $MYCERT -----BEGIN CERTIFICATE-----
\nMIIFJDCCBAygAwIBAgIQD0ifmj/Yi5NP/2gdUySbfzANBgkqhkiG9w0BAQsFADBN\nMQswCQYDVQQGEwJVUzEVMBMGA1UEChMMRGlnaUNlcnQgSW5jMScwJQY
DVQQDEx5E...SzSHfXp5Inu/3V08I72q1QNzOCgY1XeL4GKVcj4or6cT6tX6oJH7ePPmfrBfqI/O\nOeH8gMJ+FuwtXYEPa4hBf38M5eU5xWG7\n-----END CERTIFICATE---
--\n
```

4. Build the payload:

```
$ request_body=$(cat <<EOF { "certificate": "$MYCERT", "private_key": "$MYKEY" } } EOF ))
```

5. Upload the client certificate and private key from the [CIS CLI](#).
6. Enable authenticated origin pull on the specified hostname through the [CIS CLI](#). Link the client certificate to the specific hostname.

Replacing a client certificate without downtime

For hostname:

1. Upload the new certificate.
2. Link the new certificate ID and hostname and enabled values.

For global:

1. Upload the new certificate.
2. Check that the new certificate is in Active state.
3. After the certificate is active, delete the old certificate.

Apply a different client certificate (at zone and hostname level) simultaneously

1. Upload a certificate by following the steps in the [Zone-level authenticated origin pull](#) section.
2. Upload multiple certificates by following the steps in the [Per-hostname authenticated origin pull by using customer certificates](#) section.

Installing on Apache and NGINX

Use the following instructions for configuring TLS authenticated origin pulls for either NGINX or Apache origin web servers.

Setting up Apache

Use **End-to-End flexible** and update the origin web server SSL configuration by using the following steps.

1. Download the authenticated origin pull certificate ([origin-pull-ca.pem](#)).
2. Store the certificate in a file on your origin web server, for example in `/path/to/origin-pull-ca.pem`.
3. Add the following lines to the SSL configuration for your origin web server:

```
SSLVerifyClient require
SSLVerifyDepth 1
SSLCACertificateFile /path/to/origin-pull-ca.pem
```

Setting up NGINX

Use **End-to-End flexible** and update the origin web server SSL configuration by using the following steps.

1. Download the authenticated origin pull certificate ([origin-pull-ca.pem](#)).

2. Store the certificate in a file on your origin web server, for example in `/etc/nginx/certs/cloudflare.crt` .
3. Add the following lines to the SSL configuration for your origin web server:

```
ssl_client_certificate /etc/nginx/certs/cloudflare.crt;
ssl_verify_client on;
```

Setting up cipher suites

Cipher suites are a combination of algorithms and protocols that help to secure network connections during the TLS handshake. IBM Cloud® Internet Services secures network connections by using edge and origin cipher suites.

Edge cipher suites

The following ciphers are supported at the cloud edge. You can restrict the ciphers that are used for your domain through the CIS CLI plugin to the IBM Cloud CLI. See the `ciphers` option on the [domain settings command](#).

OpenSSL Name	TLS 1.0	TLS 1.1	TLS 1.2	TLS 1.3	IANA name
ECDHE-ECDSA-AES128-GCM-SHA256			✓		TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256
ECDHE-ECDSA-CHACHA20-POLY1305			✓		TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256
ECDHE-RSA-AES128-GCM-SHA256			✓		TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
ECDHE-RSA-CHACHA20-POLY1305			✓		TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256
ECDHE-ECDSA-AES128-SHA256			✓		TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256
ECDHE-ECDSA-AES128-SHA	✓	✓	✓		TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA
ECDHE-RSA-AES128-SHA256			✓		TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256
ECDHE-RSA-AES128-SHA	✓	✓	✓		TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA
AES128-GCM-SHA256			✓		TLS_RSA_WITH_AES_128_GCM_SHA256
AES128-SHA256			✓		TLS_RSA_WITH_AES_128_CBC_SHA256
AES128-SHA	✓	✓	✓		TLS_RSA_WITH_AES_128_CBC_SHA
ECDHE-ECDSA-AES256-GCM-SHA384			✓		TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384
ECDHE-ECDSA-AES256-SHA384			✓		TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384
ECDHE-RSA-AES256-GCM-SHA384			✓		TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
ECDHE-RSA-AES256-SHA384			✓		TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384
ECDHE-RSA-AES256-SHA	✓	✓	✓		TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA
AES256-GCM-SHA384			✓		TLS_RSA_WITH_AES_256_GCM_SHA384
AES256-SHA256			✓		TLS_RSA_WITH_AES_256_CBC_SHA256

AES256-SHA	✓	✓	✓		TLS_RSA_WITH_AES_256_CBC_SHA
DES-CBC3-SHA	✓				TLS_RSA_WITH_3DES_EDE_CBC_SHA
AEAD-AES128-GCM-SHA256			✓		TLS_AES_128_GCM_SHA256
AEAD-AES256-GCM-SHA384			✓		TLS_AES_256_GCM_SHA384
AEAD-CHACHA20-POLY1305-SHA256			✓		TLS_CHACHA20_POLY1305_SHA256
Edge cipher suites					

Origin cipher suites

The following ciphers are supported at the origin. You can restrict the ciphers that are used for your domain through the CIS CLI plugin to the IBM Cloud CLI. See the `ciphers` option on the [domain settings command](#).

OpenSSL Name	TLS 1.0	TLS 1.1	TLS 1.2	TLS 1.3	IANA name
AEAD-AES128-GCM-SHA256 [1]				✓	TLS_AES_128_GCM_SHA256
AEAD-AES256-GCM-SHA384 [2]				✓	TLS_AES_256_GCM_SHA384
AEAD-CHACHA20-POLY1305-SHA256 [3]				✓	TLS_CHACHA20_POLY1305_SHA256
ECDHE-ECDSA-AES128-GCM-SHA256			✓		TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256
ECDHE-RSA-AES128-GCM-SHA256			✓		TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
ECDHE-RSA-AES128-SHA	✓	✓	✓		TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA
AES128-GCM-SHA256			✓		TLS_RSA_WITH_AES_128_GCM_SHA256
AES128-SHA	✓	✓	✓		TLS_RSA_WITH_AES_128_CBC_SHA
ECDHE-ECDSA-AES256-GCM-SHA384			✓		TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384
ECDHE-RSA-AES256-SHA384			✓		TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384
AES256-SHA	✓	✓	✓		TLS_RSA_WITH_AES_256_CBC_SHA
DES-CBC3-SHA	✓				TLS_RSA_WITH_3DES_EDE_CBC_SHA
Origin cipher suites					

Managing cipher suites from the CLI

You can manage cipher suites from the CLI.

Getting the cipher suites from the CLI

To list the `cipher` variable, run the following command:

```
$ ibmcloud cis domain-settings DNS_DOMAIN_ID [-g, --group GROUP | -f, --feature FEATURE] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. This option is required.

-f, --feature

Specifies the domain setting feature to check. This option is mutually exclusive with -g, --group. Set this option to the following variable:

- **ciphers** : Defines an allowlist of TLS cipher suites for edge TLS termination, using the **BoringSSL** cipher format. This command lists only cipher suites that customers have explicitly allowlisted. If no cipher suites are allowlisted, the command returns an empty list and uses the default cipher suites. For the list of default cipher suites, see [Edge cipher suites](#).

Example

Get **ciphers** settings for domain **31984fea73a15b45779fa0df4ef62f9b** .

```
$ ibmcloud cis domain-settings 31984fea73a15b45779fa0df4ef62f9b --feature "ciphers" --instance "cis-demo"
```

For more information, see [ibmcloud cis domain-settings](#).

Updating the cipher suites from the CLI

To update the **cipher** variable, run the following command:

```
$ ibmcloud cis domain-settings-update DNS_DOMAIN_ID (-f, --feature FEATURE) (-v, --value VALUE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. This option is required.

-f, --feature

Feature of domain settings to update. This option is required. Valid **cipher** value is:

- **ciphers** : An allowlist of ciphers for TLS termination. These ciphers must be in the **BoringSSL** format.

-v, --value

Specifies the value to set for the feature on the domain. This option is required.

- Valid values for **tls_client_auth** are **on** and **off** .
- Valid values for ciphers are:
 - **ECDHE-ECDSA-AES128-GCM-SHA256**
 - **ECDHE-ECDSA-CHACHA20-POLY1305**
 - **ECDHE-RSA-AES128-GCM-SHA256**
 - **ECDHE-RSA-CHACHA20-POLY1305**
 - **ECDHE-ECDSA-AES128-SHA256**
 - **ECDHE-ECDSA-AES128-SHA**
 - **ECDHE-RSA-AES128-SHA256**
 - **ECDHE-RSA-AES128-SHA**
 - **AES128-GCM-SHA256**
 - **AES128-SHA256**
 - **AES128-SHA**
 - **ECDHE-ECDSA-AES256-GCM-SHA384**
 - **ECDHE-ECDSA-AES256-SHA384**
 - **ECDHE-RSA-AES256-GCM-SHA384**
 - **ECDHE-RSA-AES256-SHA384**
 - **ECDHE-RSA-AES256-SHA**

- AES256-GCM-SHA384
- AES256-SHA256
- AES256-SHA
- DES-CBC3-SHA
- default

For example, use `-v default` to reset the specific configured cipher suites like `-v AES256-SHA256,AES256-SHA` , to the default value.

Example

Enable `tls_client_auth` for domain 31984fea73a15b45779fa0df4ef62f9b.

```
$ ibmcloud cis domain-settings-update 31984fea73a15b45779fa0df4ef62f9b --feature tls_client_auth --value on --instance "cis-demo"
```

For more information, see [ibmcloud cis domain-settings-update](#).



Note: To revert to the default cipher list, specify `-v default` after `-f` ciphers.

Managing cipher suites with the API

You can manage cipher suites with the API.

Getting ciphers with the API

To get ciphers with the API, take the following steps.

1. Set up your environment with the right variables.
2. Store any variables to be used in the API commands. For example,
 - `crn` (string): The full URL-encoded cloud resource name (CRN) of the resource instance.
 - `zone_identifier` (string): The zone identifier.
3. When all variables are initiated, get the ciphers:

```
$ curl -X GET https://api.cis.cloud.ibm.com/v1/:crn/zones/:zone_id/settings/ciphers -H 'content-type: application/json' -H 'accept: application/json' -H 'x-auth-user-token: Bearer xxxxxx'
```

Updating ciphers with the API

To update ciphers with the API, take the following steps.

1. Set up your environment with the right variables.
2. Store any variables to be used in the API commands. For example,
 - `crn` (string): The full URL-encoded cloud resource name (CRN) of the resource instance.
 - `zone_identifier` (string): The zone identifier.
 - `value` (string): The cipher suites that you want to include.
3. When all variables are initiated, get the ciphers:

```
$ curl -X PATCH https://api.cis.cloud.ibm.com/v1/:crn/zones/:zone_id/settings/ciphers -H 'content-type: application/json' -H 'x-auth-user-token: Bearer xxxxxx' -d '{"value": ["AES256-GCM-SHA384", "AES256-SHA256"]}'
```

1. Although TLS 1.3 uses the same cipher suite space as previous versions of TLS, TLS 1.3 cipher suites are defined differently, specifying only the symmetric ciphers, and can't be used for TLS 1.2. Similarly, TLS 1.2 and lower cipher suites can't be used with TLS 1.3 (IETF TLS 1.3 draft 21). BoringSSL also hardcodes cipher preferences in this order for TLS 1.3. [↩](#)
2. Although TLS 1.3 uses the same cipher suite space as previous versions of TLS, TLS 1.3 cipher suites are defined differently, specifying only the symmetric ciphers, and can't be used for TLS 1.2. Similarly, TLS 1.2 and lower cipher suites can't be used with TLS 1.3 (IETF TLS 1.3 draft 21). BoringSSL also hardcodes cipher preferences in this order for TLS 1.3. [↩](#)
3. Although TLS 1.3 uses the same cipher suite space as previous versions of TLS, TLS 1.3 cipher suites are defined differently, specifying only the

symmetric ciphers, and can't be used for TLS 1.2. Similarly, TLS 1.2 and lower cipher suites can't be used with TLS 1.3 (IETF TLS 1.3 draft 21). BoringSSL also hardcodes cipher preferences in this order for TLS 1.3. [↩](#)

Creating Total TLS

Total TLS allows CIS to issue individual certificates for your proxied hostnames. These certificates protect the proxied hostnames not covered by [Universal certificates](#).



Important: Total TLS certificates follow the Common Name (CN) restriction of 64 characters ([RFC 5280](#)). If you have a hostname that exceeds this length, you can create an [Advanced Certificate](#) through API to cover it.

The issued certificates have a type of **Advanced - Total TLS**, and their default validity period is 90 days.

Getting the Total TLS for CLI

To get the Total TLS, run the following command:

```
$ cis total-tls cert-get DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of DNS domain.

-i, --instance

Instance name or ID. If not set, the context instance specified by `cis instance-set INSTANCE` is used.

--output

Specify output format, only JSON is supported.

Updating the Total TLS with CLI

To update the Total TLS, run the following command:

```
$ cis total-tls cert-update DNS_DOMAIN_ID [-c, --certificate_authority CERTIFICATE_AUTHORITY] (--enabled true|false) [-i, --instance INSTANCE] [--output FORMAT]
```

You can also update a custom rule by providing a JSON file or a JSON string directly:

```
$ cis total-tls cert-update DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain.

--enabled

Enable or disable Total TLS setting for a zone. Valid values are `true` and `false`.

-c, --certificate_authority

The certificate authority. Valid values are `google`, `lets_encrypt`, and `ssl_com`.

--json value

The JSON file or JSON string that is used to set Total TLS Settings or disable the feature for a zone.

- The required fields in JSON data are `enabled`.

`enabled`: Enable or disable Total TLS setting for a zone. Valid values are `true` and `false`.

- The optional fields are `certificate_authority` .

`certificate_authority` : The certificate authority. Valid values are `google` , `lets_encrypt` , and `ssl_com` .

Sample JSON data:

```
{
  "certificate_authority": "lets_encrypt",
  "enabled": true
}
```

`-i, --instance value`

Instance name or ID. If not set, the context instance specified by `cis instance-set INSTANCE` is used.

`--output value`

Specify output format, only `JSON` is supported.

Getting the Total TLS with the API

Follow these steps to get the Total TLS:

1. Set up your API environment with the correct variables.
2. Store the following values in variables to be used in the API command:

`CRN` : The full URL-encoded Cloud Resource Name (CRN) of the service instance.

`ZONE_ID` : The domain ID.

3. When all variables are initiated, get the Total TLS:

```
$ curl --request GET \
  --url https://api.cis.cloud.ibm.com/v1/{CRN}/zones/{ZONE_ID}/acm/total_tls \
  --header 'Content-Type: application/json' \
  --header 'X-AUTH-USER-TOKEN: REDACTED'
```

Changing the Total TLS with API

Follow these steps to create the Total TLS:

1. Set up your API environment with the correct variables.
2. Store the following values in variables to be used in the API command:

`CRN` : The full URL-encoded Cloud Resource Name (CRN) of the service instance.

`ZONE_ID` : The domain ID.

3. When all variables are initiated, create the Total TLS:

```
$ curl --request POST \
  --url https://api.cis.cloud.ibm.com/v1/{CRN}/zones/{ZONE_ID}/acm/total_tls \
  --header 'Content-Type: application/json' \
  --header 'X-AUTH-USER-TOKEN: REDACTED' \
  --data '{
    "enabled": true,
    "certificate_authority": "google"
  }'
```

To enable Total TLS with the API, send a `POST` request with the `enabled` parameter set to either `true` or `false` . You can also specify a certificate authority by providing a value for the `certificate_authority` parameter.

Limitations

Total TLS has the following limitations:

- Total TLS doesn't issue certificates for any hostnames that are used with:
 - [Global load balancers](#)
 - [Range applications](#)
- Total TLS is not supported for [partial CNAME setup](#).

You can use other types of certificates or manually [order advanced certificates](#) for these hostnames.

Deleting certificates

After [Total TLS is enabled](#), be cautious when deleting Total TLS-managed certificates associated with proxied hostnames. Doing so signals that the hostname should be excluded from future Total TLS issuance. The system will not automatically provision new certificates for that hostname, even if its DNS record is deleted and re-created later.

About post-quantum cryptography

Post-quantum cryptography (PQC) is cryptographic mechanisms that are designed to remain secure against adversaries equipped with large-scale quantum computers. These algorithms aim to protect encrypted data against future attacks that might break the widely used public-key cryptography.

One of the primary drivers for PQC adoption is the [harvest-now, decrypt-later](#) threat model, in which attackers collect encrypted traffic to decrypt when quantum computing becomes practical.

Edge security and content delivery platforms, including CIS, play a key role in mitigating this risk by securing data in transit through modern TLS protocols and cryptographic best practices.

PQC and TLS in CIS

CIS secures customer traffic with TLS 1.3, which provides the cryptographic foundation necessary for post-quantum adoption. TLS 1.3 improves both security and performance compared to earlier TLS versions and enables more flexible cryptographic negotiation during the handshake.

Cryptographic building blocks of TLS

A secure TLS session relies on three distinct classes of cryptographic algorithms:

1. **Symmetric encryption algorithms:** Used to encrypt and decrypt data, ensuring confidentiality and integrity.
2. **Key agreement mechanisms:** Allows the client and server to establish shared session keys securely. Classical key exchange methods (such as elliptic-curve-based algorithms) are vulnerable to quantum attacks, making it a primary focus of PQC adoption efforts.
3. **Digital signature algorithms:** Signature algorithms authenticate servers through TLS certificates. These algorithms also need to evolve toward post-quantum alternatives to ensure long-term trust in a quantum-capable future.

As a result, PQC transition efforts primarily impact key agreement and signature components, rather than data encryption itself.

Hybrid key agreement: a practical transition model

To balance security, compatibility, and operational stability, the industry widely adopts a hybrid key agreement approach. In this model, a TLS session establishes keys using:

- A well-understood classical key agreement algorithm, and
- An additional post-quantum key encapsulation mechanism (KEM)

Both mechanisms contribute cryptographic material to the final session key. This ensures that the connection remains secure even if one class of algorithm becomes vulnerable in the future.

CIS uses this standards-aligned approach for connections from CIS to customer origin servers, enabling post-quantum protection without requiring immediate, disruptive changes from clients.

Post-quantum connectivity from CIS to origin servers

CIS supports post-quantum hybrid key agreement for securing connections between the CIS edge and customer origin servers. This protects backend traffic from long-term cryptographic risks and aligns CIS with industry efforts to adopt quantum-resistant primitives.

Post-quantum key material is typically larger than classical cryptographic parameters. As a result:

- TLS handshake messages might increase in size
- Initial handshake messages might span multiple network packets.

Although this behavior is defined in the TLS 1.3 specification, it can reveal implementation issues in legacy infrastructure such as middle boxes, firewalls, and load balancers that incorrectly assume fixed message sizes. These considerations are relevant for origin connectivity, where customer-controlled infrastructure plays a role, and factored into architecture and testing strategies.

Working with DDoS protection

About DDoS protection in CIS

CIS provides DDoS protection through DNS ingestion, traffic inspection, unlimited mitigation, and integrated Layer-7 security features.

How CIS ingests and protects traffic

IBM Cloud Internet Services ingests traffic by returning a CIS IP address on the DNS lookup for a domain, instead of the actual record for the origin server's IP address. This allows CIS to ingest, single-pass inspect, and re-encrypt data before sending it to the origin server destination.

CIS can also act in DNS-only mode, returning the actual DNS record without obfuscating the IP, which disables DDoS and the other functions of CIS. To enable CIS protections, switch the "proxy" slider next to each DNS record to **on**; to disable protections, switch to **off**.

Unlimited DDoS mitigation

DDoS mitigation is typically an expensive service that can grow in cost when under attack. Unlimited DDoS mitigation is included with CIS at no additional cost.

Layer-7 mitigation options available in CIS

Though DDoS is enabled by default in CIS, you can further configure Layer 7 security by:

- Configuring WAF ruleset sensitivity and response behavior
- Adding rate limiting
- Adding firewall rules

Use these features to customize Layer 7 mitigation of both volumetric and non-volumetric attacks.

Mitigating non-volumetric attacks

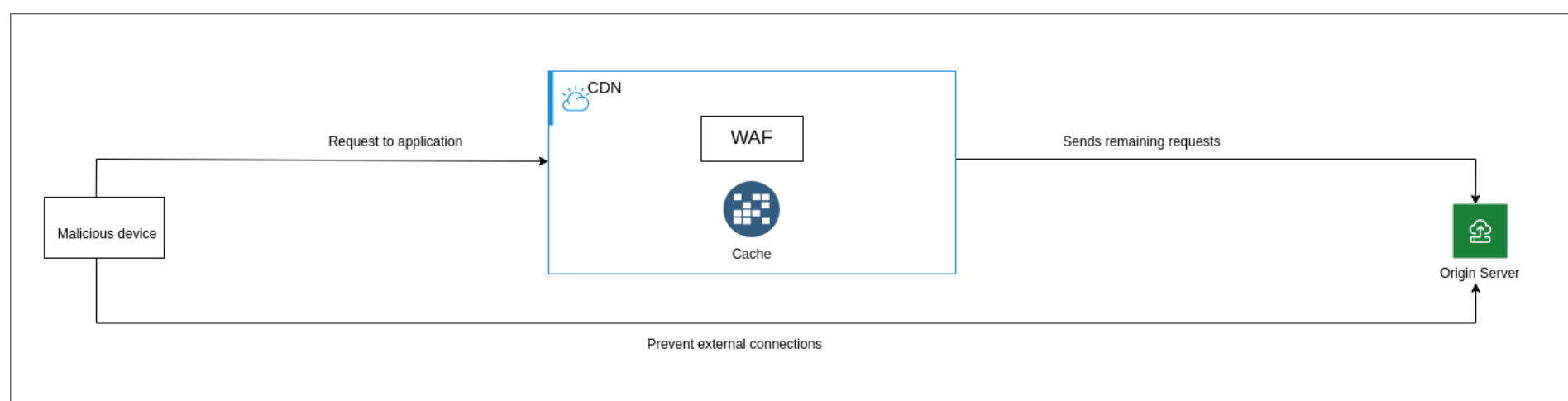
CIS WAF contains rulesets to mitigate non-volumetric attacks, including cross-site forgery, cross-site-scripting (XSS), file inclusion, and SQL injection. For additional information about WAF, see [Web Application Firewall concepts](#).

Related links

- [DDoS attack concepts](#)
- [Preventing DDoS attacks](#)
- [Responding to DDoS attacks](#)
- [Using Defense mode for DDoS attacks](#)
- [Third-party services and DDoS protection](#)
- [HTTP DDoS Attack Protection managed ruleset](#)

Preventing DDoS attacks

An effective way to prevent DDoS attacks targeting your web servers is to reduce the requests that reach those servers. Requests can come to your origin server from your web application and from direct connections to the server itself.



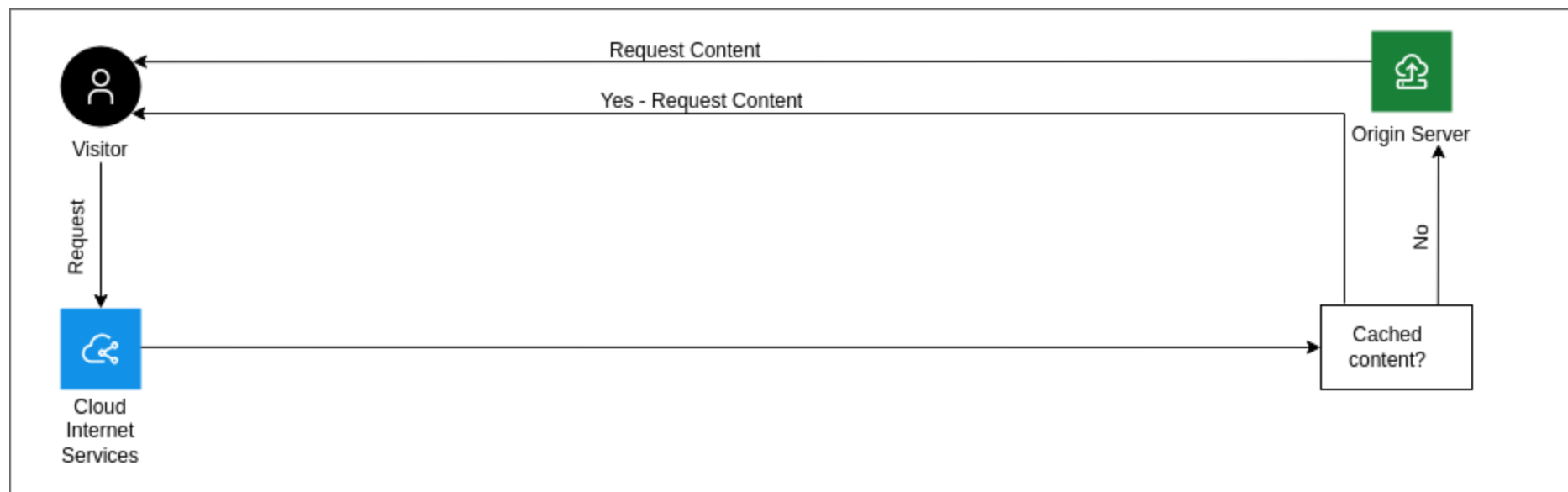
The paths requests can take to your servers

Reducing application requests to your origin server

Using caching to offload traffic

A cache stores copies of frequently accessed resources such as images and CSS files. When a resource is cached—whether on a user’s browser or a Content Delivery Network (CDN) server—requests for that resource do not have to go to your origin server. These resources are instead served directly by the cache.

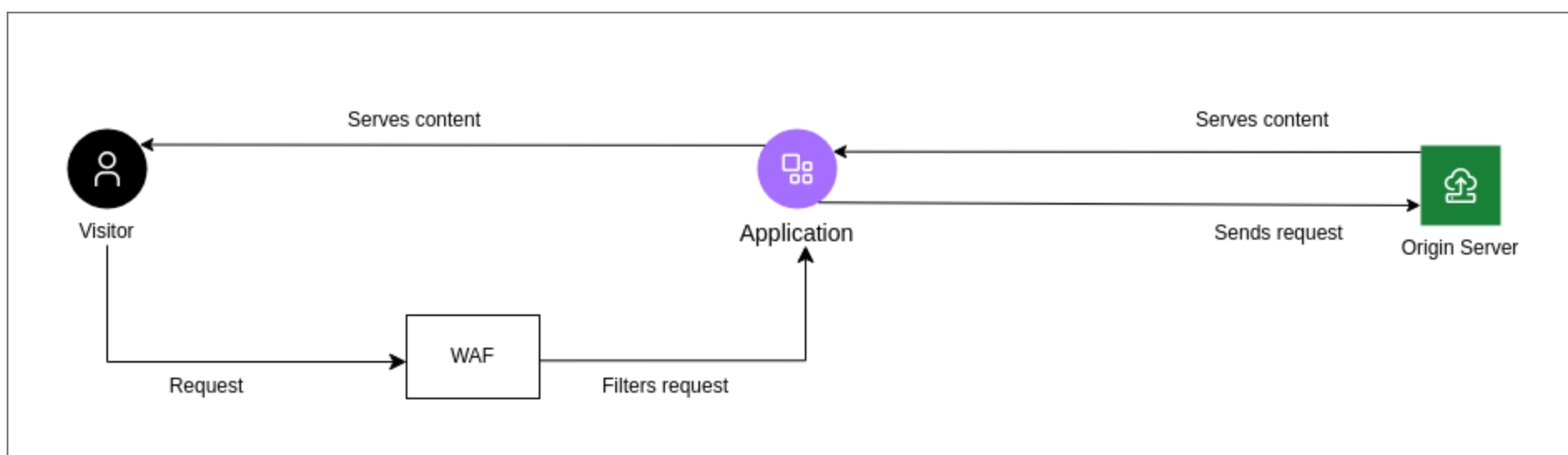
During a DDoS attack, caching reduces the number of requests that go to your origin server, which makes it harder for your server to get overwhelmed by traffic.



Reduce requests to the origin by using caching

Using a Web Application Firewall (WAF) to filter malicious traffic

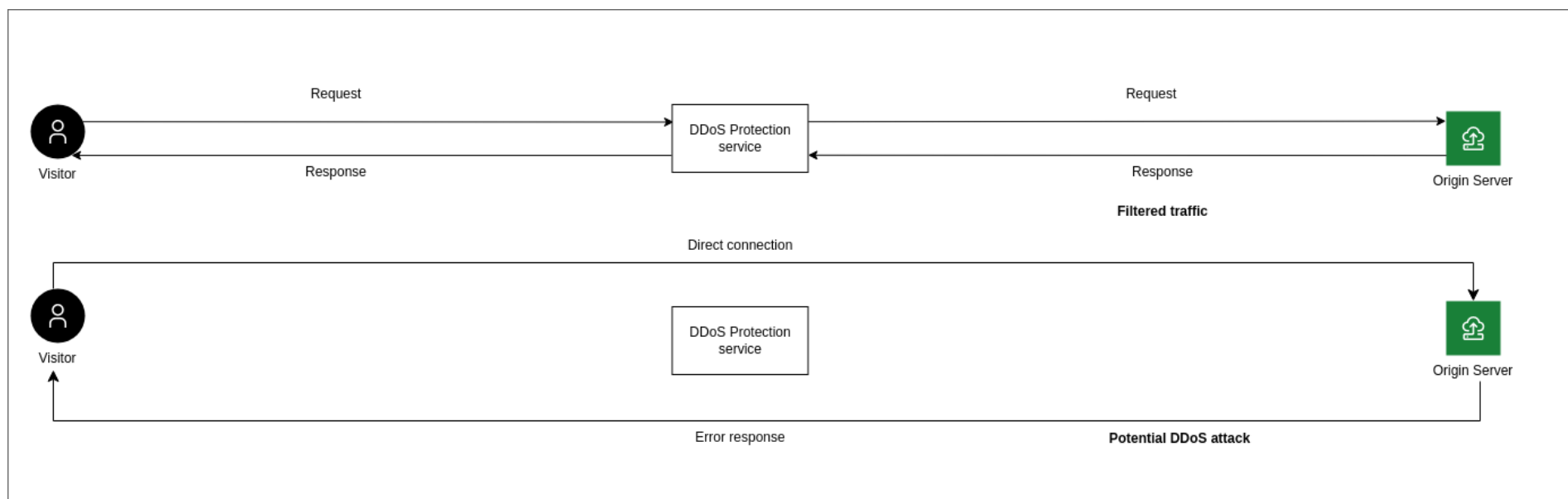
A WAF creates a shield between a web application and the internet. The WAF checks incoming web requests and filters potentially malicious traffic to mitigate common attacks.



Reduce requests to the origin by using WAF

Prevent external connections

Generally, your origin server should accept only requests that come from your web application, especially in the context of DDoS attacks. Traffic that bypasses your web application also bypasses any WAF or caching you have and has a stronger chance of overwhelming your origin.



Prevent external connection requests to the origin

Related links

- [DDoS attack concepts](#)

- [About DDoS protection in CIS](#)
- [Responding to DDoS attacks](#)
- [Using Defense mode during a DDoS attack](#)
- [Third-party services and DDoS protection](#)
- [HTTP DDoS Attack Protection managed ruleset](#)

Responding to DDoS attacks

The CIS network automatically mitigates large DDoS attacks, but these attacks can still affect your application. All users should perform the following steps to better secure their application.

1. Make sure the `ruleset_phase` parameter (with `ddos_I7`) in the [DDoS attack concepts](#)
2. [About DDoS protection in CIS](#)
3. [Preventing DDoS attacks](#)
4. [Using Defense mode for DDoS attacks](#)
5. [Third-party services and DDoS protection](#)
6. [HTTP DDoS Attack Protection managed ruleset](#)

Using Defense mode for DDoS attacks

When your site is under a DDoS attack, you can switch on defense mode to quickly mitigate the attack. Defense mode uses additional security checks to help mitigate Layer 7 DDoS attacks, allowing validated users through while blocking suspicious traffic. The following guidance explains how to apply defense mode at the domain or page level and what to expect when it is enabled.

To set your entire domain in defense mode when you are under attack, turn on “Defense mode” from the CIS **Overview** page. When enabled:

- Do not rate-limit or throttle requests from IBM CIS, because CIS needs the bandwidth to assist you with your situation.
- You can block specific countries and visitors if necessary.
- Visitors see an interstitial page, and access to your site may be temporarily paused, which can impact analytics.



Note: Defense mode is disabled by default and intended as a last resort when a zone is under attack.

Applying defense mode selectively

Instead of enabling defense mode for the entire domain, you can apply it to specific pages or site sections by adjusting the security level with a configuration rule.

For example, consider the following incoming request matching scenario:

- Field: `URI Path`
- Operator: `starts with`
- Value: `/admin`

1. If you are using the Expression Editor, enter the following expression.

```
$ (starts_with(http.request.uri.path, "/admin"))
```

2. For defense mode, select **Add**.
3. Move the switch to **On**.

To target specific ASNs (hosts/ISPs that own IP addresses), countries, or IP ranges, use **IP Access Rules**.

Potential issues

Defense mode requires the browser to support JavaScript to display and pass the interstitial page. This requirement can affect third-party analytics tools, which might not record traffic accurately while defense mode is active.

Related links

- [DDoS attack concepts](#)

- [About DDoS protection in CIS](#)
- [Preventing DDoS attacks](#)
- [Responding to DDoS attacks](#)
- [Third-party services and DDoS protection](#)
- [HTTP DDoS Attack Protection managed ruleset](#)

Protecting against low and slow DDoS attacks

A [low and slow DDoS attack](#) is typically a non-volumetric attack in which an attacker sends a small number of HTTP requests at a very slow rate. These attacks are designed to avoid detection while gradually exhausting server resources.

[Slowloris](#) is a common type of low and slow attack. In a Slowloris attack, the attacker establishes multiple [TCP connections](#) to the target server by using HTTP or HTTPS. Instead of completing the HTTP request, the attacker sends incomplete HTTP header lines. The server waits for the request to complete and keeps the connection open.

To maintain the connection, the attacker periodically sends additional partial headers or header fields. This behavior can include sending partial HTTP headers or using the content-length header to declare a request body size that is larger than the data actually sent.

The recommended best practice to defend against low and slow attacks is to use an HTTP reverse proxy, such as CIS [CDN](#) or [WAF](#) service. The reverse proxy acts as a protective layer in front of the origin server. It waits for a complete HTTP request before forwarding traffic to the origin, serving content from cache, or applying other configured actions. By enabling request body buffering, CIS absorbs slow and incomplete requests at the edge. Incomplete requests are not forwarded to the origin server.

If possible, CIS serves requests from [Cache](#) or Edge functions. Otherwise, CIS forwards the request to the origin only after the request is fully received and passes all WAF checks. As a result, the attack traffic never reaches the origin server, similar to CIS protection against TCP-based Slowloris attacks.

CIS also enforces timeouts on incomplete HTTP requests after a series of [keepalive probes](#). There is no minimum traffic threshold required to activate this protection. In addition, custom firewall rules validate payload size and perform basic sanity checks to ensure that request content matches expected behavior.

The [RUDY \(R-U-Dead-Yet?\)](#) attack is a low-rate denial-of-service (DoS) technique that targets web servers by holding connections open for long periods.

Unlike traditional DDoS attacks that overwhelm servers with a large volume of requests in a short time, RUDY creates a small number of long-running HTTP requests. It submits form data extremely slowly, which keeps server resources occupied and prevents the server from responding to legitimate users. Because the traffic volume is low and the requests appear valid, RUDY attacks are difficult to detect and often bypass volume-based DDoS protections.

RUDY targets the application layer (Layer 7) by exploiting how web forms process HTTP POST requests. The attacker sends form data one byte at a time and intentionally delays completion of the request. As a result, application threads remain open while waiting for the request to finish, eventually exhausting server resources and disrupting normal application operation.

Third-party services and DDoS protection

When you integrate third-party services, such as Content Delivery Networks (CDNs), VPNs, or NAT devices with CIS, it is important to understand how these components interact with Distributed Denial of Service (DDoS) protection systems. The following sections describe recommended configuration settings and adjustments to DDoS rules to make sure that these services work seamlessly together. The goal is to maintain strong traffic filtering, reduce potential risks, and avoid any negative impact on performance.

Using a third-party CDN in front of CIS

Some CIS customers choose to place a third-party CDN in front of CIS to cache and serve their resources.

However, it is "not" recommended to place a third-party CDN in front of CIS. Some CDN providers might introduce discrepancies into HTTP requests that deviate from protocol standards and protocol best practices. Also, because traffic to CIS then originates from a limited set of the CDN's IP addresses, this can (in rare cases) cause the traffic to be misidentified as a DDoS attack. For example, when you use the Akamai CDN in front of CIS, the high volume of traffic from a limited set of IPs can resemble attack behavior to CIS.

Instead, use CIS and the [underlying Cloudflare network](#) and its CDN capabilities, which provide the following benefits:

- Reduced latency by removing an extra network hop between vendor data centers.
- Immediate DDoS protection at the first point of contact with the internet (an industry best practice).



Note: If you use a third-party CDN in front of CIS and CIS mitigates a DDoS attack, you must still pay your CDN provider for the attack traffic

the provider that is processed before mitigation.

Recommended DDoS configuration adjustments for CDN or proxy

If you use a CDN or proxy in front of CIS, change the action and/or sensitivity level of the following DDoS rules:

- HTTP requests with unusual HTTP headers or URI path (signature #1) with the rule ID ...3486aee1
- HTTP requests with unusual HTTP headers or URI path (signature #56) with the rule ID ...e269dfd6
- HTTP requests with unusual HTTP headers or URI path (signature #57) with the rule ID ...f35a42a0
- Requests coming from known bad sources with the rule ID ...3a679c52

Recommended DDoS configuration adjustments for VPN and NAT

**Note:** Without an Advanced DDoS Protection subscription, you can't customize DDoS rule actions, such as setting them to `Log`. However, you can still adjust rule sensitivity to reduce the likelihood of false positives if unintended blocking occurs.

If your organization uses VPNs, NATs, or third-party services that generate high traffic volumes (over 100 Mbps), you might need to adjust your DDoS configuration to avoid disruptions. Both HTTP-based (Layer 7) and network-layer (Layer 4) traffic can trigger managed DDoS rules. The following recommendations help you use expression filters and tune rule sensitivity to better accommodate legitimate high-volume traffic:

- Use expression filters to exclude specific traffic from the rules. You can exclude a combination of source ports, source IP addresses, destination ports, destination IP addresses, and protocol.
- Reduce the sensitivity level of relevant managed rules. Setting a rule to "Essentially Off" prevents the rule from being triggered. For more information on rule adjustments, see the [Adjusting the sensitivity of DDoS L7 ruleset with the API](#).

For Enterprise users, DDoS ruleset actions can be configured by using the Ruleset API to log matching traffic. While no dedicated UI exists for the DDoS ruleset, setting a rule’s action to `Log` allows visibility into flagged traffic through API-accessible logs. This action can help you gather insights before you apply exclusions or change the rule sensitivity.

Example: Adjusting the sensitivity of DDoS Layer 7 ruleset with the API

To adjust the sensitivity of the DDoS Layer 7 ruleset:

```
curl -X PUT https://api.cis.cloud.ibm.com/v1/$CRN/rulesets/phases/ddos_l7/entrypoint
--header "Content-Type: application/json"
--header "X-Auth-User-Token: Bearer $IAM_TOKEN"
--data '{
  "action": "execute",
  "description": "ddos override",
  "enabled": true,
  "expression": "true",
  "action_parameters": {
    "id": "4d21379b4f9f4bb088e0729962c8b3cf",
    "overrides": {
      "rules": [
        {
          "id": "dd42da7baabe4e518eaf11c393596a9d",
          "sensitivity_level": "eoff"
        }
      ]
    }
  },
  {
    "id": "78da7447eca94fabbc405fc923e2920",
    "sensitivity_level": "medium"
  }
]
```

**Note:** The DDoS Layer 7 ruleset always uses the ID `4d21379b4f9f4bb088e0729962c8b3cf`. Also, the `expression` field must always be set to `true`.

In this example, two commonly triggered rules in high-traffic scenarios are being overridden:

"HTTP requests causing a high number of origin errors." - dd42da7baabe4e518eaf11c393596a9d

"HTTP requests causing a high request rate to origin." - 78da7447eca94fabbc405fc923e2920

To retrieve rule IDs and descriptions, use this API call:

```
$ curl -X GET https://api.cis.cloud.ibm.com/v1/$CRN/rulesets/4d21379b4f9f4bb088e0729962c8b3cf
--header "Content-Type: application/json"
--header "X-Auth-User-Token: Bearer $IAM_TOKEN"
```

HTTP DDoS Attack Protection managed ruleset

The CIS HTTP DDoS Attack Protection managed ruleset is a set of pre-configured rules that are designed to detect and mitigate known Layer 7 (application-layer) DDoS attack vectors across the CIS global network.



Note: Currently, the HTTP DDoS Attack Protection ruleset is only available using the CIS API.

The managed ruleset helps protect against:

- Known DDoS attack patterns and tools
- Suspicious request behavior
- Protocol violations
- Requests that generate excessive origin errors
- Traffic floods targeting the origin or cache
- Other application-layer attack vectors

The HTTP DDoS Attack Protection managed ruleset is always enabled for all CIS customers and cannot be disabled. However, you can customize how the ruleset responds to detected attacks.

The ruleset also provides visibility into Layer 7 DDoS attacks mitigated by CIS, allowing you to review both active and historical attack activity.

Ruleset configuration

If you expect large volumes of legitimate traffic, you can customize the HTTP DDoS Attack Protection settings to reduce false positives, where legitimate requests are incorrectly detected as attack traffic and blocked or challenged.

You can adjust the behavior of the rules in the managed ruleset by modifying the following parameters:

- The performed **action** when an attack is detected.
- The **sensitivity level** of attack detection mechanisms.

Certain actions and sensitivity levels are available only on specific CIS plans.



Note: Currently, you can define account-level overrides for the HTTP DDoS Attack Protection managed ruleset only through the API.

To adjust the rule behavior, use [expressions](#) to tailor DDoS protection for different categories of incoming requests.

Customers on Standard plans can create one override only and cannot customize the rule expression. In this case, the single override (containing one or more configuration settings) applies to all incoming requests.

Parameters

Configure the HTTP DDoS Attack Protection managed ruleset to change the action that is applied to a given attack or modify the sensitivity level of the detection mechanism.

You can [define overrides using the Rulesets API](#).

The following parameters are available in the HTTP DDoS Attack Protection managed ruleset:

- Action
- Sensitivity level

Action

API property name: `action`.

The action determines how CIS handles HTTP requests that match rules in the HTTP DDoS Attack Protection managed ruleset.

Action	API value	Description
Block	<code>block</code>	Blocks HTTP requests that match the rule expression.
Managed Challenge	<code>managed_challenge</code>	Managed Challenges reduce the amount of time legitimate users spend solving CAPTCHAs by dynamically selecting an appropriate challenge based on request characteristics.
Interactive Challenge	<code>challenge</code>	Presents an interactive challenge to clients making HTTP requests that match the rule expression.
Log	<code>log</code>	Available only on Enterprise plans with Advanced DDoS Protection. Logs requests that match the expression of a rule detecting HTTP DDoS attacks. Recommended for validating a rule before committing to a more severe action.
Connection Close	Not applicable (Internal rule action that you cannot use in overrides)	Instructs the client to establish a new connection by disabling keep-alive , instead of reusing the existing connection. Existing requests are not affected.
Force Connection Close	Not applicable (Internal rule action that you cannot use in overrides).	- Closes active HTTP connections, forcing the client to reconnect. This action does not block individual requests. - For HTTP/2 and HTTP/3, the connection is closed even if it affects other requests on the same connection. - The performed action depends on the HTTP version: - HTTP/1: set the Connection header to close. - HTTP/2: send a GOAWAY frame to the client.
DDoS Dynamic	Not applicable (Internal rule action that you cannot use in overrides).	Performs a specific action according to a set of internal guidelines defined by CIS. The executed action may be one of the listed actions or another undisclosed mitigation.

List of available actions

Sensitivity level

API property name: `sensitivity_level` .

The sensitivity level defines how aggressively the rule detects attacks by adjusting the mitigation thresholds:

- Higher sensitivity uses lower thresholds and detects attacks more aggressively.
- Lower sensitivity uses higher thresholds and is less aggressive.

The available sensitivity levels are:

UI value	API value
High	<code>default</code>
Medium	<code>medium</code>
Low	<code>low</code>
Essentially Off	<code>eoff</code>

Available sensitivity levels

The default sensitivity level is **High**.

When you select **Essentially Off**, the rule typically does not trigger mitigation actions, including Log. However, if an attack reaches an exceptional scale, CIS protection systems will still apply mitigation to protect the CIS network.

Selecting **Essentially Off** sets an extremely low sensitivity level. In most cases, traffic is not mitigated, but exceptionally large attacks are still mitigated to maintain network stability.

Sensitivity controls whether traffic is detected as an attack. Action controls what happens when an attack is detected.

When you select **Log**, requests are logged and displayed in the dashboard without mitigation. As with Essentially Off, exceptionally large attacks are still mitigated to protect the CIS network.

Rule categories

The HTTP DDoS Attack Protection managed rules are grouped into the following categories (tags) based on the type of traffic and mitigation behavior.

Name	Description
botnets	Rules that detect requests originating from known botnets. These rules have very high detection accuracy and a low risk of false positives. Keep these rules enabled.
unusual-requests	Rules that identify requests with suspicious characteristics that are not commonly observed in legitimate traffic.
advanced	Rules associated with features available to Enterprise customers.
generic	Rules that detect and mitigate floods of requests. These rules are effective against attacks without known signatures, but they may also trigger during unusually high volumes of legitimate traffic. To reduce the risk of false positives, these rules use a higher requests-per-second (rps) activation threshold. By default, these rules rate limit or challenge traffic, but you can override them to block traffic if required.
read-only	Highly targeted rules designed to mitigate DDoS attacks with a high confidence rate. These rules are read-only . You cannot override their sensitivity level or action.
test	Rules used to test the detection, mitigation, and alerting capabilities of CIS DDoS protection products.

Available rule categories

Related links

- [DDoS attack concepts](#)
- [Preventing DDoS attacks](#)
- [Responding to DDoS attacks](#)
- [Using Defense mode for DDoS attacks](#)
- [Third-party services and DDoS protection](#)
- [About DDoS protection in CIS](#)

Working with rulesets

About rulesets

You can use the CIS Ruleset Engine feature suite to create and deploy rules and rulesets in CIS with the same basic syntax.

Main features

The following features apply to rulesets:

- **Powerful syntax:** Rule expressions use a powerful rules language similar to the [wirefilter](#) syntax that allows you to create complex rules.
- **High-performance rule evaluation:** Allows you to have many rules in CIS with minimal impact on performance.
- **Engine powering CIS:** CIS continue to build products on top of the Ruleset Engine, which means that you can use the same API methods for configuring different products with the same customization possibilities. The Ruleset Engine also supports the different phases of the request lifecycle.

Phases

A phase defines a stage in the life of a request where you can run rulesets. Phases are defined by CIS and can't be modified.

Phases exist at the instance level and at the zone level. For the same phase, rules that are defined at the instance level are evaluated before the rules defined at the zone level.

Each phase has, at most, one entrypoint ruleset at the instance and zone level.

Currently, only phases at the zone level are available. This page is updated as instance and zone level phases become available in subsequent releases.

Phase list

The following table lists the phases that are available within the Ruleset Engine APIs and phases execute in the order they appear in the table.

Phase name	Description	Supported interfaces
<code>http_config_settings</code>	Configuration rules	API
<code>http_request_origin</code>	Origin rules	API
<code>ddos_l7</code>	HTTP DDoS Attack Protection rules	API
<code>http_request_firewall_custom</code>	Custom rules	API, CLI, UI
<code>http_ratelimit</code>	Rate-limiting rules	API, CLI, UI
<code>http_request_firewall_managed</code>	Web Application Firewall (WAF)	API, CLI, UI

Available phases

Entrypoint ruleset

**Note:** Entrypoint rulesets are abstracted away when you use the CIS UI. They are required to deploy and override rulesets when you use the API, CLI, SDK, or Terraform.

An entrypoint ruleset contains a list of ordered rules that run in a phase at the instance or zone level. This ruleset is an entrypoint for all rules that run in a phase. Some of these rules might run other rulesets.

Each phase has, at most, one entrypoint ruleset at the instance level and at the zone level.

The `kind` field of a phase entrypoint ruleset has one of the following values:

- `root`: Used for a phase entrypoint ruleset at the instance level
- `zone`: Used for a phase entrypoint ruleset at the zone level

If a 404 status code is returned when you fetch an endpoint ruleset, the particular ruleset phase likely doesn't exist. Create it by using the updated API for the [Deploying managed rulesets](#). To learn how to override rulesets with endpoint rulesets, see [Overriding managed rulesets](#).

Ruleset Engine rules language

The CIS Ruleset Engine rules language is a flexible and intuitive specification for building rule expressions. Based on the [Wireshark display filters](#), the rules language allows you to precisely target HTTP requests with a syntax and semantics familiar to security engineers.

Ruleset Engine rules actions

The action of a rule determines how CIS handles matches for the rule expression.

The following table lists the actions available in the Ruleset Engine language:

Action	API value	Description	Stops rule evaluation?
Interactive challenge	challenge	Useful for ensuring that the visitor accessing the site is human, not automated. The client that made the request must pass an interactive challenge. If successful, CIS accepts the matched request; otherwise, it is blocked.	Yes
JS Challenge	js_challenge	Useful for ensuring that bots and spam can't access the requested resource; browsers, however, are free to satisfy the challenge automatically. The client that made the request must pass a JavaScript challenge before proceeding. If successful, CIS accepts the matched request; otherwise, it is blocked.	Yes
Managed challenge (recommended)	managed_challenge	Helps reduce the time spent solving CAPTCHAs across the Internet. Depending on the characteristics of a request, CIS will dynamically choose the appropriate type of challenge from the following actions based on specific criteria: - Show a non-interactive challenge page (similar to the current JS challenge). - Show a custom interactive challenge (for example, clicking a button).	Yes
Block	block	Matching requests are denied access to the site.	Yes
Skip	skip	Allows user to dynamically skip one or more security features or products for a request. Depending on the rule configuration, matching requests will skip the evaluation of one or more security features or products: - Skip all remaining rules in the current ruleset - Skip rulesets - Skip rules of a ruleset - Skip phases - Skip specific security products that are not based on the Ruleset Engine The available skip options depend on the phase where you configure the rule.	No (However, some rules might be skipped)
Log	log	Records matching requests in the CIS logs. Only available on Enterprise plans. Recommended for validating rules before committing to a more severe action.	No
Execute	execute	Executes the rules in the ruleset specified in the rule configuration. You can specify a managed ruleset or a custom ruleset to execute. In the CIS UI, this action is not listed in action selection dropdowns.	No
Rewrite	rewrite	Adjusts the URI path, query string, and/or HTTP headers of requests and responses, according to the rule configuration. Only available in WAF custom rules checking for exposed credentials, in the <code>http_request_firewall_custom</code> phase at the instance level. In the CIS UI, this action is called Exposed-Credential-Check Header.	No

Integrating lists in rules

CIS allows you to use lists in firewall expressions for more efficient rule management. Lists allow you to create a group of items and refer to them collectively, by name, in your expressions. Each list type supports items of a specific data type. All items in a list must have the same data type. Learn how to use these lists within the CIS dashboard and API to enhance security and streamline rule creation.

Expression Builder

The Expression Editor allows users to create custom firewall rules by defining conditions based on attributes like IP addresses, HTTP headers, and URL paths. It helps filter traffic with greater precision using logical operators (AND, OR, NOT) for more granular security control.

To use a list in the Expression Builder, follow these steps:

- 1. Select **is in list** or **is not in list** from the **Operator** menu.
- 2. Select a list from the **Value** menu. Depending on your plan, you might be able to select a [managed IP list](#).
- 3. To commit your changes and enable the rule, select **Deploy**. If you are not ready to enable the rule, select **Save as Draft**.

Expression Editor

The Expression Editor allows you to write and modify logical expressions used in security rules. It enables the combination of various conditions, such as IP address, request headers, and URL paths, to create tailored rules for traffic filtering and security enforcement.

To refer to a list in a rule expression, use `$<list_name>` and specify the `in` operator. Only one value in the list has to match the left-hand side of the expression (before the `in` operator) for the simple expression to evaluate to `true`. If there is no match, the expression evaluates to `false`.

Examples:

- Expression matching requests from IP addresses that are in an IP list named `office_network`:

```
$ ip.src in $office_network
```

- Expression matching requests with a source IP address different from IP addresses in the `office_network` IP list:

```
$ not ip.src in $office_network
```

- Expression matching requests from IP addresses in the CIS Open Proxies [managed IP list](#).

```
$ ip.src in $cf.open_proxies
```

Setting up and tuning CIS WAF security

Cloud Internet Services (CIS) inspects incoming web and API traffic and blocks unwanted requests using predefined rulesets. This topic walks you through the initial steps to configure the Web Application Firewall (WAF) and quickly enable protection against the most common web attacks.

Attention: When performing a *penetration test (pentest)*, start with the most restrictive WAF configuration to evaluate the full defensive capabilities of CIS. To achieve this, configure the following settings:

- Enable all managed rules.
 - [CIS OWASP Core Ruleset](#): Set paranoia level to 4 (`PL4`) and score threshold to **High** (`High-25 and higher`).
 - [CIS managed ruleset](#): Set the default ruleset (for all signatures) to **Block**.
- Create a custom rule to block any request with an Attack Score less than or equal to `20`.

Expression preview: `(cf.waf.score le 20)`

After testing, you can adjust this configuration to accommodate your application's expected behavior and reduce false positives.

Before you begin

Before testing WAF capabilities, make sure that you have a CIS instance set up in your IBM Cloud account and your domain added to CIS.

Configuring WAF

Follow these high-level steps to configure WAF for your zones:

1. Deploy the CIS managed ruleset.

The [CIS managed ruleset](#) protects against Common Vulnerabilities and Exposures (CVEs) and attack vectors using signature-based detection to identify common attacks, with a focus on minimizing false positives.



Note: CIS updates these rulesets during emergency releases to respond to high-profile, zero-day threats.

- a. In the IBM Cloud console, open the **Navigation Menu** icon , then click **Resource list** and expand **Security**.
- b. Click your CIS instance name.
- c. Navigate to **Security > WAF** and make sure that the **CIS Managed Ruleset** Status toggle is enabled.

By default, a subset of rules is enabled to balance protection with reduced false positives. You can review and enable additional rules based on your application's stack.

In particular situations, enabling the managed ruleset can cause some false positives. False positives are legitimate requests inadvertently mitigated by the WAF. For information on addressing false positives, see [Handling false positives in managed rules](#).

For pentesting, enable all rules with the following settings:

- Ruleset action: **Block**
- Ruleset status: Enabled (enables all rules in the ruleset)

For more information, see [Working with WAF managed rules](#).

2. Create a custom rule based on WAF Attack Score.



Note: WAF Attack Score is only available to Standard plan customers.

The WAF Attack Score is a machine-learning layer that complements CIS's managed rulesets, providing additional protection against SQL injection (SQLi), cross-site scripting (XSS), and many remote code execution (RCE) attacks. It helps identify rule bypasses and potentially new, undiscovered attacks.

[Create a custom rule](#) using the Attack Score field:



Note: The Attack Score field is a number from (likely malicious) to (likely clean) classifying how likely an incoming request is malicious or not. Allows you to detect new attack techniques before they are publicly known.

- When incoming requests match:

Field	Operator	Value
WAF Attack Score	less than or equal to	20

Rule conditions for detecting unverified low-score bot requests

Expression Preview:

- Choose action: **Block**

If you're on the CIS Standard, Standard Next, or Free Trial plans, you can create a custom rule using the WAF Attack Score Class field instead. For example, use the following rule expression: .

3. Create a custom rule based on Bot Score.

The Bot Score feature is available only to CIS Enterprise Premier customers with [Bot Management](#). Standard Next and Enterprise Advanced/Usage plans can use Super Bot Fight Mode, but note that no UI is currently available for configuration.

Create a custom rule using the Bot Score and Verified Bot fields:



Note: Bot scores range from . A score below typically indicates automation. The Verified Bot field identifies bots that are transparent about their identity and purpose.

- When incoming requests match:

Field	Operator	Value	Logic
Bot Score	less than	20	And
Verified Bot	equals	Off	

Match conditions for applying a managed challenge to suspected bot traffic

- Choose action: **Managed Challenge**

For more information about the bot-related fields you can use in expressions, see [Bot Management fields](#).

With these protections in place, your WAF setup provides robust mitigation against known and unknown threats while minimizing false positives.

4. Optional: Deploy the [CIS OWASP Core Ruleset](#).

After configuring the CIS Managed Ruleset and attack score, you can also deploy the CIS OWASP Core Ruleset. This managed ruleset is CIS's implementation of the OWASP ModSecurity Core Ruleset. Its attack coverage significantly overlaps with CIS managed ruleset by detecting common attack vectors, such as SQLi and XSS.



Important: This ruleset is prone to false positives and provides limited additional benefit when combined with CIS managed rules and WAF Attack Score. If you decide to deploy this managed ruleset, you must monitor and adjust its settings based on your traffic to prevent false positives.

From the CIS dashboard, go to **Security > WAF** and ensure that the **CIS OWASP Core Ruleset** Status toggle is enabled.

This deploys the ruleset with the default config: **paranoia level** = **PL1** and **score threshold** = **Medium (40+)** .

Ruleset configuration: Unlike the signature-based CIS managed ruleset, the OWASP Core Ruleset is score-based. You select a certain paranoia level (levels vary from PL1 to PL4, where PL1 is the lowest level), which enables an increasing larger group of rules. You also select a score threshold, which decides when to perform the configured action. Low paranoia with a high score threshold usually leads to fewer false positives. For an example of how the OWASP Core Ruleset is evaluated, see the [OWASP evaluation example](#).

Two common configuration strategies:

- Strict first: Start with **paranoia level** = **PL4** and **score threshold** = **High - 25 and higher** . Reduce the score threshold and paranoia level until you achieve a good false positives/true positives rate for your incoming traffic.
- Permissive first: Start from a more permissive configuration (**paranoia level** = **PL1** , **score threshold** = **Low - 60 and higher**) and increase both parameters to adjust your protection, trying to keep a low number of false positives.

For more information on configuring the OWASP Core Ruleset, see [Using fields, functions, and expressions](#).

5. Review traffic in security dashboards.

After configuring the WAF, monitor how your settings affect incoming traffic:

- Use [CIS metrics](#) to explore traffic patterns, including traffic not blocked by WAF rules. All data provided by traffic detections is available (for example, WAF Attack Score and Bot Score).
- Use [Security Events](#) to review mitigated traffic.
- Enterprise plans can access HTTP requests and security events using [CIS Logpush](#).

Rule recommendations

CIS doesn't recommend blocking traffic solely based on the WAF Attack Score for all values below **50** , since the *Likely attack* range (scores between **21** and **50**) tends to have false positives. However, if you choose to block traffic based on this score, you might consider one of the following:

- Use a stricter WAF Attack Score value in your expression. For example, block traffic with a WAF Attack Score below **20** or below **15** (you might need to adjust the exact threshold).
- Combine a higher WAF Attack Score threshold with additional filters when blocking incoming traffic. For example, include a check for a specific URI path in your expression or use bot score as part of your criteria.

Next steps

After your WAF is configured, you'll have a solid baseline of protection against common and advanced web threats.

To further enhance protection, consider:

- [Allowlist certain IP addresses](#)
- [Create custom rules that block traffic from embargoed countries](#)
- [Define rate limits](#)

About WAF Attack Score

WAF Attack Score helps detect variations of known attacks by analyzing request behavior and payload characteristics. This capability complements [WAF managed rules](#) by identifying malicious traffic that does not exactly match existing rule signatures.

WAF managed rules use continuously updated rulesets to detect well-known attack patterns with a lower false-positive rate. These rules are effective for established attack vectors but are not optimized to detect modified or obfuscated attacks, such as those generated through fuzzing techniques.

WAF Attack Score addresses this gap by using a machine learning model to evaluate each request and assign an attack score from 1 to 99, based on the likelihood that the request is malicious. Similar to [Bot Management](#), you can use this score to identify and act on suspicious traffic that does not directly trigger a managed rule.

For maximum protection, CIS recommends using WAF managed rules and WAF Attack Score together to detect both known attacks and their variations.

Available scores

The CIS WAF provides the following attack score fields:

Field	Type	Description	Required plan
WAF Attack Score <code>cf.waf.score</code>	Number	A global score from 1–99 that combines the score of each WAF attack vector into a single score.	Enterprise
WAF SQLi Attack Score <code>cf.waf.score.sqli</code>	Number	A score from 1–99 classifying the SQL injection (SQLi) attack vector.	Enterprise
WAF XSS Attack Score <code>cf.waf.score.xss</code>	Number	A score from 1–99 classifying the cross-site scripting (XSS) attack vector.	Enterprise
WAF RCE Attack Score <code>cf.waf.score.rce</code>	Number	A score from 1–99 classifying the command injection or remote code execution (RCE) attack vector.	Enterprise
WAF Attack Score Class <code>cf.waf.score.class</code>	String	The attack score class of the current request, based on the WAF attack score. Possible values: <code>attack</code> , <code>likely_attack</code> , <code>likely_clean</code> , and <code>clean</code> .	All plans

Available WAF scores

You can use these fields in expressions of [custom rules](#) and [rate limiting rules](#). WAF Attack Score fields use the `Number` data type and range from `1` to `99`, with the following meanings:

- A score of 1 indicates that the request is very likely malicious.
- A score of 99 indicates that the request is likely clean.

The special score `100` indicates that the request reached the WAF Attack Score classification system, but the system did not assign a score.

In [Logpush](#), you might also see a score of `0`. This value means that the request did not reach the stage where attack scores are calculated. For example, another rule or protection mechanism might have already mitigated the request. The value `0` does not appear in the CIS console.

The global WAF Attack Score is mathematically derived from individual attack vector scores, such as SQL injection (SQLi) and cross-site scripting (XSS) scores, using a weighted system. These scores are interdependent, and the global score is not a simple sum of the individual values. Lower global scores indicate a higher likelihood that the request is malicious, while higher scores indicate that the request is more likely clean.

The WAF Attack Score class field can have one of the following values, depending on the calculated request attack score:

Console label	Field value	Description
Attack	attack	Attack score between 1 and 20.
Likely attack	likely_attack	Attack score between 21 and 50.
Likely clean	likely_clean	Attack score between 51 and 80.
Clean	clean	Attack score between 81 and 99.

WAF Attack Score classes

Requests with the special attack score of 100 appear in the CIS console with a WAF Attack Score Class value of `Unscored`. You cannot use this class value in rule expressions.

WAF Attack Score automatically detects and decodes Base64, JavaScript (Unicode escape sequences), and URL-encoded content. This decoding applies to all parts of the request, including the URL, headers, and request body.

Rule recommendations

Avoid blocking traffic based solely on WAF Attack Score values less than 50. Scores in the Likely attack range (21 – 50) might include false positives and often require additional filtering criteria.

If you choose to block traffic based on the attack score, use one of the following approaches:

- Use a more strict WAF Attack Score value in your expression. For example, block traffic with a WAF Attack Score less than 20 or 15 (you might need to adjust the exact threshold).
- When you block incoming traffic based on the WAF Attack Score, combine a higher WAF Attack Score threshold with additional filters. For example, include a check for a specific URI path in your expression or use bot score as part of your criteria.

Using WAF Attack Score

To configure WAF Attack Score in a custom rule, follow these steps:

1. To create a custom rule, select one of the following:
 - If you are an Enterprise plan customer, [create a custom rule](#) that blocks requests with a WAF Attack Score less than or equal to 20. This value is the recommended starting threshold.
 - Equivalent rule expression: `cf.waf.score le 20`
 - Action: **Block**
 - If you are a Standard plan customer, create a custom rule by using the **WAF Attack Score Class** field instead. For example, block requests with a score class of `Attack`.
 - Equivalent rule expression: `cf.waf.score.class eq "attack"`
 - Action: **Block**
2. Monitor the rule closely during the first few days. Verify that the selected threshold or class value aligns with your traffic patterns. Adjust the rule if you observe false positives or missed threats.
3. Update the rule action. If you are an Enterprise plan customer and you created a rule with a **Log** action, change the rule action to a stronger action, like **Managed Challenge** or **Block**.

The WAF Attack Score and Bot Score measure different types of risk. WAF Attack Score identifies requests that resemble known attack patterns, including variations that WAF Managed Rules might not detect. Bot Score identifies automated traffic and evaluates the likelihood that a request originates from a bot. Use WAF Attack Score to detect malicious payloads and attack behavior. Use Bot Score to identify and manage automated traffic.

Working with lists

About lists

Use lists to refer to a group of items (such as IP addresses) collectively, by name, in rule expressions. You can create your own custom lists or use lists managed by CIS, such as [managed IP lists](#).

Lists have the following advantages:

- When creating a rule, using a list is easier and less error-prone than adding a long list of items, such as IP addresses to a rule expression.
- When updating a set of rules that target the same group of IP addresses (or hostnames), using an IP list (or a hostname list) is easier and less error prone than editing multiple rules.
- Lists are easier to read and more informative, particularly when you use descriptive names for your lists.

When you update the content of a list, any rules that use the list are automatically updated, so you can make a single change to your list rather than modify rules individually.

CIS stores your lists at the account level. You can use the same list in rules of different zones in your CIS account.

Supported lists

CIS supports the following lists:

- [Custom lists](#): Includes custom IP lists, hostname lists, and ASN lists.
- [Managed lists](#): Lists managed and updated by CIS.

List availability

List availability varies according to the list type and your CIS plan and subscriptions.

	Trial (no cost)	Standard Next	Enterprise
Availability	Yes	Yes	Yes
Number of custom lists	1	10	10
Custom lists with IPs	Yes	Yes	Yes
Custom lists with ASNs	No	No	Yes
Custom lists with hostnames	No	No	Yes
Open Proxies managed list	No	No	Yes
Anonymizers managed list	No	No	Yes [1]
VPNs managed list	No	No	Yes [2]
Malware managed list	No	No	Yes [3]
Botnets, Command and Control Servers managed list	No	No	Yes [4]
Available list			

Additional details and exceptions apply by plan:

- Enterprise users can request a higher quota of custom lists or a larger maximum number of items across lists.
- The maximum number of custom lists per data type is 40 lists.

List requirements

All user-defined lists must follow these formatting and usage rules:

- You can have a maximum number of 10,000 list items across all custom lists.
- For a list name, use only lowercase letters, numbers, and the underscore `_` character in the name. A valid name satisfies this regular expression: `^[a-z0-9_]+$`.
- The maximum length of a list name is 50 characters.
- You can only delete a list when there are no rules (enabled or disabled) that reference that list.

 **Note:** To learn how to use lists in Ruleset Engine rules, see [Integrating lists in rules](#).

Using lists in rules

After you’ve created a custom list, you can use it in your rule expressions to match incoming traffic based on the list’s contents. Custom lists are used with the `in` operator as shown:

```
$ <FIELD> in $<LIST_NAME>
```

This rule checks whether the specified field matches any item in the custom list.

The fields that you can use vary according to the list type:

List type	Available fields
IP address	Fields with type <code>IP address</code> listed in the Fields reference
Hostname	<code>http.host</code>
ASN	<code>ip.src.asnum</code>

Available custom list type

You can apply lists in a variety of use cases, such as:

- Blocking or allowing traffic from specific IP ranges
- Filtering requests by known domains
- Prioritizing routing for traffic from specific ISPs or regions

For details on how to build rule expressions with lists, see [Integrating lists in rules](#).

-
1. Enterprise Usage plan includes Open Proxies only; custom lists are supported. [↩](#)
 2. Enterprise Usage plan includes Open Proxies only; custom lists are supported. [↩](#)
 3. Enterprise Usage plan includes Open Proxies only; custom lists are supported. [↩](#)
 4. Enterprise Usage plan includes Open Proxies only; custom lists are supported. [↩](#)

Using custom lists

About custom lists

Custom lists in CIS allow you to group related data—such as IP addresses, hostnames, or Autonomous System Numbers (ASNs) into named collections that can be reused across multiple rule expressions. These lists simplify management, improve rule clarity, and provide a more scalable way to control access and filtering.

What are custom lists?

A custom list is a reusable set of similar data entries. Instead of hardcoding multiple values into every rule, you can define a custom list once and reference it by name in your custom security rules. This approach is ideal for managing dynamic threat intelligence feeds, trusted traffic sources, or internal network assets.

CIS supports the following types of custom lists:

- [IP address lists](#) for grouping IPv4 or IPv6 addresses.

- [Hostname lists](#) for specifying domain names or subdomains.
- [ASN lists](#) for identifying network ownership by Autonomous System Numbers.

 **Note:** Lists with hostnames and ASNs are available only to Enterprise customers.

Each list type has its own properties and CSV file format as shown in [Custom list types](#).

Managed versus custom lists

In addition to custom lists that you define, CIS also provides managed lists, which are predefined and maintained by IBM. These lists include curated sets of known threat sources or commonly targeted IP ranges, and can be used similar to custom lists. For information on managed lists, see [Using managed lists](#).

Custom list types

CIS supports the following custom list types:

- Lists with IP addresses (also known as IP lists)
- Lists with hostnames
- Lists with ASNs (autonomous system numbers)

 **Note:** Lists with hostnames and ASNs are only available to Enterprise customers. For more information, see [List availability](#).

For more information about lists, see [About lists](#).

Lists with IP addresses (IP lists)

List items in custom lists with IP addresses must be in one of the following formats:

- Individual IPv4 addresses
- IPv4 CIDR ranges with a prefix from `/8` to `/32`
- IPv6 CIDR ranges with a prefix from `/12` to `/64`

 **Note:** You can combine individual addresses and CIDR ranges in the same list.

To specify an IPv6 address, enter it as a CIDR range with a `/64` prefix, the largest supported prefix for IPv6 CIDR ranges.

For example, instead of `2001:db8:6a0b:1a01:d423:43b9:13c5:2e8f`, enter one of the following:

- `2001:db8:6a0b:1a01:0000:0000:0000/64`
- `2001:db8:6a0b:1a01::/64` (using the [double colon notation](#))

The IPv6 address topology describes the last 64 bits as the host identifier. Matching on a `/128` prefix would identify a specific IPv6 address, but not the host in general. It would be possible for an attacker to change their specific IPv6 address from a single machine.

You can use uppercase or lowercase characters for IPv6 addresses in lists. However, when you save the list, uppercase characters are converted to lowercase.

CSV file format

When uploading items to a custom list with IP addresses using a CSV file, use the following file format (enter one item per line):

```
$ <IP_ADDRESS_1>,<DESCRIPTION>
<IP_ADDRESS_2>
```

The `<DESCRIPTION>` field is optional.

Lists with hostnames

 **Note:** Available to Enterprise customers.

List items in custom lists with hostnames must be Fully Qualified Domain Names (FQDNs). An item might contain a `*` prefix/subdomain wildcard, which must be followed by a `.` (period). An item cannot include a scheme (for example, `https://`) or a URL path.

For example, the following entries would be valid for a custom list with hostnames:

- `example.com`
- `api.example.com`
- `*.example.com`

However, `example.com/path/subfolder` would not be a valid entry.

You can add any valid hostname (a valid FQDN) to a custom list with hostnames. The hostnames do not need to belong to the current CIS account.

CSV file format

When uploading items to a custom list with hostnames using a CSV file, use the following file format:

```
$ <HOSTNAME_1>,<DESCRIPTION>
<HOSTNAME_2>
```

The `<DESCRIPTION>` field is optional.

Lists with ASNs



Note: Available to Enterprise customers.

List items in custom lists with ASNs must be integer values.

For example, the following entries would be valid for a list with ASNs:

- `1`
- `13335`
- `64512`

CSV file format

When uploading items to a custom list with ASNs using a CSV file, use the following file format:

```
$ <ASN_1>,<DESCRIPTION>
<ASN_2>
```

The `<DESCRIPTION>` field is optional.

Creating custom lists and adding items

Custom lists are a practical way to group related items, such as IP addresses, domains, or user IDs, for easier management and reuse. Custom lists help simplify rule configuration, reduce repetition, and make updates more efficient by centralizing control in one place.

You can create and add items to a custom list using the console, CLI, API, and Terraform.

Creating a custom list and adding items in the console

Follow these steps to create a custom list in the console:

1. In the CIS console, navigate to the **Account** page.
2. Select the **Lists** tab and then click **Create**.
3. Enter a name for your list and then select the [Type](#) of list.
4. Optionally, enter a description to clarify the purpose of the list.
5. Click **Create custom list**.
6. Enter the values, depending on the list type.
7. Optionally, enter a description and then click **Add item**.
8. Click **Create list items**.

Creating a custom list from the CLI

To create a custom list for your instance, enter the following command:

```
$ ibmcloud cis custom-lists list-create (--kind KIND) (--name NAME) [--description DESCRIPTION] [-i, --instance INSTANCE]
```

You can also accept JSON input (from a file or directly as a string):

```
$ ibmcloud cis custom-lists list-create (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE]
```

Command options

--kind

Custom list kind. Valid values: `ip`, `asn`, and `hostname`.

--name

The list name.

--description

Description of the list.

-i, --instance

Instance name or ID. If not set, the context instance specified by `cis instance-set INSTANCE` is used.

--json

The JSON file or JSON string used to describe a custom list.

kind: Kind of custom list. Valid values are `ip`, `asn`, and `hostname`. Required.

name: The list name. Required.

description: Description of the list. Optional.

Sample JSON data:

```
{
  "kind": "ip",
  "name": "string",
  "description": "string"
}
```

Command examples

```
$ ibmcloud cis custom-lists list-create --kind ip --name iplistone --instance crn:v1:staging:public:internet-svcs-
ci:global:a/c987fg3e4h278745690dp435683568rp:eg7kb437-4893-56yl-4wn9-c595j8t78gr9::
```

```
$ ibmcloud cis custom-lists list-create --json @example.json --instance crn:v1:staging:public:internet-svcs-ci:global:a/c987fg3e4h278745690dp435683568rp:eg7kb437-
4893-56yl-4wn9-c595j8t78gr9::
```

Adding items to a custom list from the CLI

To create a new item in a custom list from the CLI, run the following command:

```
$ ibmcloud cis custom-lists item-create LIST_ID (--asn ASN | --ip IP | --hostname HOSTNAME) [--comment COMMENT] [-i, --instance INSTANCE]
```

You can also accept JSON input (from a file or directly as a string):

```
$ ibmcloud cis custom-lists item-create LIST_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE]
```

Command options

LIST_ID

The ID of the custom list.

`--asn`

The ASN value.

`--ip`

The IPv4 address.

`--hostname`

The hostname.

`--comment`

To provide a brief comment on the item.

`-i, --instance`

Instance name or ID. If instance or ID is not set, the context instance specified by `cis instance-set INSTANCE` will be used.

`--json`

The JSON file or JSON string used to describe a custom list.

`items`: List of custom list items to create. Valid values are `asn`, `hostname`, and `ip`. Required.

`comment`: Comment on the item. Optional.

Sample JSON data:

```
[
  {
    "asn": 19604,
    "comment": "My list of developer IPs.",
    "hostname": "cloud.ibm.com",
    "ip": "172.64.0.0/13"
  }
]
```

Examples

```
$ ibmcloud cis custom-lists item-create f93d11a87c4945a0a6bd12820776a66d --ip 192.0.0.3 --instance crn:v1:staging:public:internet-svcs-ci:global:a/c987fg3e4h278745690dp435683568rp:eg7kb437-4893-56yl-4wn9-c595j8t78gr9::
```

```
$ ibmcloud cis custom-lists item-create f93d11a87c4945a0a6bd12820776a66d --json @example.json --instance crn:v1:staging:public:internet-svcs-ci:global:a/c987fg3e4h278745690dp435683568rp:eg7kb437-4893-56yl-4wn9-c595j8t78gr9::
```

Creating a custom list with the API

Follow these steps to create a custom list with the API:

1. Set up your API environment with the correct variables.
2. Store the following values in variables to be used in the API command:

`CRN`: The full URL-encoded CRN of the service instance.

3. When all variables are initiated, create the custom list:

```
$ curl -X POST "https://api.cis.cloud.ibm.com/v1/$CRN/rules/lists" \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json" \
--data '{
  "kind": "ip",
  "name": "malicious_ips",
  "description": "IPs associated with known bad actors"
}'
```

Adding items to a custom list with the API

Follow these steps to add items to a custom list with the API:

1. Set up your API environment with the correct variables.
2. Store the following values in variables to be used in the API command:

`CRN` : The full URL-encoded CRN of the service instance.

`LIST_ID` : The custom list ID.

3. When all variables are initiated, create the custom list items:

```
$ curl -X POST "https://api.cis.cloud.ibm.com/v1/$CRN/rules/lists/$LIST_ID/items" \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json" \
--data '[
  {"ip": "23.87.13.199"},
  {"ip": "45.20.84.39"},
  {"ip": "72.48.188.49"},
  {"ip": "76.35.97.136"}
]
```

Getting the list operation status with the API

Operations to manage list items are asynchronous. Follow these steps to get the list operation status with the API:

1. Set up your API environment with the correct variables.
2. Store the following values in variables to be used in the API command:

`CRN` : The full URL-encoded CRN of the service instance.

`OPERATION_ID` : The list operation ID returned during the list item operation.

3. When all variables are initiated, get the list operation status:

```
$ curl -X POST "https://api.cis.cloud.ibm.com/v1/$CRN/rules/lists/bulk_operations/$OPERATION_ID" \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json"
```

For more information, see [Lists API](#).

Creating a custom list with Terraform

The following example creates a custom list:

```
resource ibm_cis_custom_list custom_list {
  cis_id = "crn:v1:staging:public:internet-svcs-ci:global:a/01652b251c3ae2787110a995d8db0135:1a9174b6-0106-417a-844b-c8eb43a72f63::"
  kind = "ip"
  name = "testip"
  description = "custom list"
}
```

Adding items to a custom list with Terraform

The following example adds items to a custom list:

```
resource ibm_cis_custom_list_items items1 {
  cis_id = "crn:v1:staging:public:internet-svcs-ci:global:a/01652b251c3ae2787110a995d8db0135:1a9174b6-0106-417a-844b-c8eb43a72f63::"
  list_id = "ibm_cis_custom_list.custom_list.list_id"
  items {
    ip = "1.2.3.6"
  }
  items {
    ip = "2.3.4.5"
  }
  items {
    ip = "192.168.0.1"
  }
}
```



```
}
```

Using managed lists

CIS provides managed lists you can use in rule expressions. These lists are regularly updated.

Managed IP lists

You can access CIS IP threat intelligence through managed IP lists. CIS offers several managed IP lists, but the specific lists available to you depend on your subscription plan.

Display name	Name in expressions	Description
Cloudflare Open Proxies	<code>cf.open_proxies</code>	IP addresses of known open HTTP and SOCKS proxy endpoints, which are frequently used to launch attacks and hide attackers identity.
Cloudflare Anonymizers	<code>cf.anonymizer</code>	IP addresses of known anonymizers (open SOCKS proxies, VPNs, and TOR nodes).
Cloudflare VPNs	<code>cf.vpn</code>	IP addresses of known VPN servers.
Cloudflare Malware	<code>cf.malware</code>	IP addresses of known sources of malware.
Cloudflare Botnets, Command and Control Servers	<code>cf.botnetcc</code>	IP addresses of known botnet command-and-control servers.

Available managed IP lists

Listing managed lists in an instance from the CLI

To list managed lists in an instance from the CLI, run the following command:

```
$ cis managed-lists [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance specified by `cis instance-set INSTANCE` is used.

`--output`

Specify output format, only JSON is supported now.

Command example

```
$ ibmcloud cis managed-lists --instance crn:v1:staging:public:internet-svcs-ci:global:a/c987fg3e4h278745690dp435683568rp:eg7kb437-4893-56yl-4wn9-c595j8t78gr9:: --output json
```

Listing managed lists in an instance with the API

The following example lists managed lists in an instance:

```
$ curl -X GET https://api.cis.cloud.ibm.com/v1/$CRN/rules/managed_lists \
-H 'Content-Type: application/json' \
-H 'Accept: application/json' \
-H 'X-Auth-User-Token: Bearer xxxxxx'
```

Listing managed lists in an instance with Terraform

The following example lists managed lists in an instance:

```
data ibm_cis_managed_lists managed_lists {
  cis_id = "crn:v1:staging:public:internet-svcs-ci:global:a/01652b251c3ae2787110a995d8db0135:1a9174b6-0106-417a-844b-c8eb43a72f63::"
}
```

Working with WAF custom rules

WAF custom rules offer power and flexibility by targeting HTTP traffic and applying custom criteria to block, challenge, log, or allow certain requests.

You can create many types of WAF custom rules. However, the number of active rules on your site is limited by your customer plan. For more information on entitlements, see [Comparing CIS plans](#).

The number of active rules per plan is fixed. Currently, you can't purchase more active rules.



Important: Before getting started, it's a good idea to review [Using fields, functions, and expressions](#).

You can create, update, and delete a custom rule by using the console, CLI, API, or Terraform.

Creating a custom rule in the console

Follow these steps to create a custom rule in the console:



Note: WAF custom rules are configured by using the existing Firewall rules page. Any legacy firewall rules that were previously created on your domain are automatically converted into WAF custom rules.

1. In the CIS console, navigate to the **Security**.
2. Select the **Custom rules** tab and then click **Create**.
3. Enter an optional description.
4. Optionally, input a priority, if necessary. A priority of zero is a null priority and is evaluated last.
5. Use the UI builder in the **Incoming requests** section to add a condition. To build an expression with multiple conditions, click either:
 - **And** - to evaluate conditions that use *and* logic
 - **Or** - to evaluate conditions or groups of previously *and*'ed conditions that use *or* logic

You can see that as you build a condition, the Expression Preview shows the expression in plain text.



Note: In the Expression Preview, you can click to edit your expression manually instead of using the Visual Expression Builder, or switch between the two. However, depending on the complexity of a manually constructed expression, the Visual Expression Builder might be unable to render it.

6. Pick an action from the **Response** list menu.
7. To save your rule, choose the most appropriate option by clicking either:
 - **Save as draft** to save your rule, but keep it disabled.
 - **Save and deploy** to save your rule and activate it.

Updating a custom rule in the console

Follow these steps to update an existing custom rule in the console:

1. In the CIS console, navigate to the **Security**.
2. Select the **Custom rules** tab.
3. In the Custom rules table, locate the rule that you want to modify, then click the Actions menu on the right of the row.
4. Select **Edit**.
5. Make your changes to the rule.
6. To save your rule, choose the most appropriate option by clicking either:
 - **Save as draft** to save your rule, but keep it disabled.
 - **Save and deploy** to save your rule and activate it.



Note: To pause or activate any rule in the list of existing rules, click the **Enabled** toggle.

Deleting a custom rule in the console

Follow these steps to delete an existing custom rule in the console:

1. In the CIS console, navigate to the **Security**.
2. In the Custom rules table, locate the rule to delete and click the Actions menu on the right of the row.
3. Select **Delete** from the action menu.
4. Click **Delete WAF custom rule** to delete the rule.

Creating a custom rule from the CLI

To create a custom rule from the CLI, follow these steps:

1. [Set up your CLI environment](#).
2. Log in to your account from the CLI. After you enter the password, the system prompts for the account and region that you want to use:

```
$ ibmcloud login --sso
```

3. Run the following command to create a custom rule:

```
$ ibmcloud cis custom-waf rule-create DNS_DOMAIN_ID --match EXPRESSION --action ACTION [--description DESCRIPTION] [--enabled true|false] [-i, --instance INSTANCE] [--output FORMAT]
```

You can also create a custom rule by providing a JSON file or a JSON string directly:

```
$ ibmcloud cis custom-waf rule-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain.

--match

Specifies the conditions that must be matched for the rule to run. For match value, see [Using fields, functions, and expressions](#).

--action

The rule action to perform. Valid values: "block", "challenge", "js_challenge", "managed_challenge", "log", "skip". For "block" and "skip" actions, use JSON file or JSON string instead.

--enabled

Indicates if the rule is active. The default is "false".

--description

A brief description of the rule.

--json

The JSON file or JSON string that is used to describe a custom rule.

- The required fields in JSON data are **expression** and **action**.
 - **expression** : Specifies the conditions that must be matched for the rule to run.
 - **action** : The rule action to perform. Valid values are **block**, **challenge**, **js_challenge**, **managed_challenge**, **log**, and **skip**.
- The optional fields are **description**, **enabled**, **logging**, and **action_parameters**.
 - **action_parameters** : The rule action parameters.
 - **ruleset** : Skip all remaining rules or one or more WAF-managed rulesets. Valid value is **current**.

- `phases` : Skips WAF components for matching requests. Valid values are `http_ratelimit` , `http_request_firewall_managed` , and `http_request_sbfm` .
- `products` : Skips specific security products for matching requests. Valid values are `waf` , `rateLimit` , `securityLevel` , `hot` , `bic` , `uaBlock` , and `zoneLockdown` .
- `response` : Define a custom response for `block` action.
 - `status_code` : Choose an HTTP status code for the response in the range `400-499` .
 - `content_type` : The content type of a custom response. Valid response types are : `text/html` , `text/plain` , `application/json` , `text/xml` .
 - `content` : The response body.
- `description` : Briefly describes the rule.
- `enabled` : Indicates whether the rule is active or not. When this field is disabled, matched requests don't appear in firewall requests.
- `logging` : Log requests matching the skip rule. This field is only available for 'skip' action.

Sample JSON data:

```
{
  "description": "test-custom-rule",
  "expression": "(http.cookie contains \"test\")",
  "action": "skip",
  "logging": {
    "enabled": true
  },
  "action_parameters": {
    "ruleset": "current",
    "phases": [
      "http_ratelimit",
      "http_request_firewall_managed",
      "http_request_sbfm"
    ],
    "products": [
      "waf",
      "rateLimit",
      "securityLevel",
      "hot",
      "bic",
      "uaBlock",
      "zoneLockdown"
    ]
  },
  "enabled": true
}
```

`-i, --instance`

Instance name or ID. If not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

Specify output format, only `JSON` is supported.

Updating a custom rule from the CLI

Run the following command to update a custom rule from the CLI:

```
$ ibmcloud cis custom-waf rule-update DNS_DOMAIN_ID [--match EXPRESSION] [--action ACTION] [--description DESCRIPTION] [--enabled true|false] [-i, --instance INSTANCE] [--output FORMAT]
```

You can also update a custom rule by providing a JSON file or a JSON string directly:

```
$ ibmcloud cis custom-waf rule-update DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain.

`RULE_ID`

The ID of the rule.

`--match`

Specifies the conditions that must be matched for the rule to run. For match value, see [Using fields, functions, and expressions](#).

`--action`

The rule action to perform. Valid values: "block", "challenge", "js_challenge", "managed_challenge", "log", "skip".For "block" and "skip" actions, use JSON file or JSON string instead.

`--enabled`

Indicates if the rule is active. The default value is "false".

`--description`

A brief description of the rule.

`--json`

The JSON file or JSON string that is used to describe a custom rule.

- The required fields in JSON data are `expression` and `action` .
 - `expression` : Specifies the conditions that must be matched for the rule to run.
 - `action` : The rule action to perform. Valid values are `block` , `challenge` , `js_challenge` , `managed_challenge` , `log` , and `skip` .
- The optional fields are `description` , `enabled` , `logging` , and `action_parameters` .
 - `action_parameters` : The rule action parameters.
 - `ruleset` : Skip all remaining rules or one or more WAF-managed rulesets. Valid value is `current`
 - `phases` : Skips WAF components for matching requests. Valid values are `http_ratelimit` , `http_request_firewall_managed` , and `http_request_sbfm` .
 - `products` : Skips specific security products for matching requests. Valid values are `waf` , `rateLimit` , `securityLevel` , `hot` , `bic` , `uaBlock` , and `zoneLockdown` .
 - `response` : Define a custom response for 'block' action.
 - `status_code` : Choose an HTTP status code for the response in the range `400-499` .
 - `content_type` : The content type of a custom response. Valid response types are `text/html` , `text/plain` , `application/json` , and `text/xml` .
 - `content` : The response body.
 - `description` : Briefly describes the rule.
 - `enabled` : Indicates whether the rule is active or not. When this field is disabled, matched requests don't appear in firewall requests.
 - `logging` : Log requests matching the skip rule. This field is only available for 'skip' action.

Sample JSON data:

```
{
  "description": "test-custom-rule",
  "expression": "(http.cookie contains \"test\")",
  "action": "block",
  "action_parameters": {
    "response": {
      "status_code": 429,
      "content_type": "text/xml",
      "content": "reject"
    }
  }
}
```

```
}
},
"enabled": true
}
```

`-i, --instance`

Instance name or ID. If not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

Specify output format, only `JSON` is supported.

Deleting a custom rule from the CLI

Run the following command to delete a custom rule from the CLI:

```
$ ibmcloud cis custom-waf rule-delete DNS_DOMAIN_ID RULE_ID [-f, --force] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain.

`RULE_ID`

The ID of the custom rule.

`-i, --instance`

Instance name or ID. If not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`-f, --force`

Attempt to delete a custom rule without prompting for confirmation.

`--output`

Specify output format, only `JSON` is supported.

Command examples

- To create a custom rule:

```
ibmcloud cis custom-waf rule-create 601b728b86e630c744c81740f72570c3 --action challenge --description "rule 1" --enabled true --match "(http.host eq 'www.example.com')"
```

- To update a custom rule:

```
ibmcloud cis custom-waf rule-update 601b728b86e630c744c81740f72570c3 4d37cb6f87654e96a18bc531628a4d27 --enabled true
```

- To delete a custom rule:

```
ibmcloud cis custom-waf rule-delete 601b728b86e630c744c81740f72570c3 4d37cb6f87654e96a18bc531628a4d27
```

Getting the custom rule entry point for the API

All custom rule API operations require a `RULESET_ID` of the entry point ruleset for the custom rules phase.

Follow these steps to get the custom rule entry point ruleset:

1. Set up your API environment with the correct variables.
2. Store the following values in variables to be used in the API command:

`CRN`: The full URL-encoded Cloud Resource Name (CRN) of the service instance.

`ZONE_ID`: The domain ID.

- When all variables are initiated, get the entry point ruleset:

```
$ curl -X GET "https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets/phases/http_request_firewall_custom/entrypoint" \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json"
```

The ruleset ID is in the response of the successful request. If the preceding call returns a 404 Not Found response, use the following API to create the entry point ruleset for the custom rule phase:

```
$ curl -X POST https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json" \
--data '{
  "name": "Zone-level phase entry point",
  "kind": "zone",
  "description": "Custom rule entry point ruleset.",
  "phase": "http_request_firewall_custom"
}'
```

Creating a custom rule with the API

Follow these steps to create a custom rule with the API:

- Set up your API environment with the correct variables.
- Store the following values in variables to be used in the API command:

CRN : The full URL-encoded Cloud Resource Name (CRN) of the service instance.

ZONE_ID : The domain ID.

RULESET_ID : The ID of the custom rule endpoint ruleset.

- When all variables are initiated, create the custom rule:

```
$ curl -X POST "https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets/$RULESET_ID/rules" \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json" \
--data '{
  "description": "My custom rule with plain text response",
  "expression": "(ip.src.country eq \"GB\") or ip.src.country eq \"FR\") and cf.waf.score lt 20",
  "action": "block",
  "action_parameters": {
    "response": {
      "status_code": 403,
      "content": "Your request was blocked.",
      "content_type": "text/plain"
    }
  }
}'
```

Updating a custom rule with the API

Follow these steps to update an existing custom rule with the API:

- Set up your API environment with the correct variables.
- Store the following values in variables to be used in the API command:

CRN : The full URL-encoded Cloud Resource Name (CRN) of the service instance.

ZONE_ID : The domain ID.

RULESET_ID : The ID of the custom rule endpoint ruleset.

RULE_ID : The ID of the custom rule to be modified.

- When all variables are initiated, update the custom rule:

```
$ curl -X PATCH "https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets/$RULESET_ID/rules/$RULE_ID" \
```



```
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json" \
--data '{
  "enabled": false,
  "description": "block GB and FR or based on IP Reputation (temporarily disabled)"
}'
```

Deleting a custom rule with the API

Follow these steps to delete an existing custom rule with the API:

1. Set up your API environment with the correct variables.
2. Store the following values in variables to be used in the API command:

CRN: The full URL-encoded Cloud Resource Name (CRN) of the service instance.

ZONE_ID: The domain ID.

RULESET_ID: The ID of the custom rule endpoint ruleset.

RULE_ID: The ID of the custom rule to be modified.

3. When all variables are initiated, delete the custom rule:

```
$ curl -X DELETE "https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets/$RULESET_ID/rules/$RULE_ID" \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json"
```

Creating a custom rule with Terraform

To create a custom rule, you must create an entry point first, then create the custom rule. To do so, follow these steps:

1. To create an entry point ruleset, run the following command:

```
$ resource "ibm_cis_ruleset_entrypoint_version" "test" {
  cis_id   = "<cis-id>"
  domain_id = "<domain-id>"
  phase = "http_request_firewall_custom"
  rulesets {
    description = "Entry point ruleset for custom ruleset"
  }
  lifecycle {
    ignore_changes = [
      rulesets
    ]
  }
}
```

2. To create a custom rule, run the following command:

```
resource ibm_cis_ruleset_rule "config" {
  cis_id   = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  ruleset_id = resource.ibm_cis_ruleset_entrypoint_version.rulesets[0].ruleset_id
  rule {
    action = "block"
    description = "var.description"
    expression = "true"
    enabled = "false"
    action_parameters {
      response {
        status_code = var.status_code
        content = var.content
        content_type = "text/plain"
      }
    }
  }
  position {
    index = var.index
    after = <id of any existing rule>
  }
}
```

```
    before = <id of any existing rule>
  }
}
}
```

The following example shows how to create an entry point and WAF custom rule:

```
resource ibm_cis_ruleset_entrypoint_version test {
  cis_id   = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  phase = "http_request_firewall_custom"
  rulesets {
    description = "Entrypoint ruleset for custom ruleset"
  }
  lifecycle {
    ignore_changes = [
      rulesets
    ]
  }
}

data ibm_cis_ruleset_entrypoint_versions custom_rule_ep {
  cis_id   = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  phase = "http_request_firewall_custom"
  depends_on = [
    ibm_cis_ruleset_entrypoint_version.test
  ]
}

resource ibm_cis_ruleset_rule "custom_rule1" {
  cis_id   = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  ruleset_id = data.ibm_cis_ruleset_entrypoint_versions.custom_rule_ep.rulesets[0].ruleset_id
  rule {
    action = "skip"
    description = "adding custom rule via terraform"
    expression = "true"
    enabled = "true"
    action_parameters{
      ruleset = "current"
      phases = ["http_ratelimit", "http_request_sbfm","http_request_firewall_managed"]
      products = ["zoneLockdown","uaBlock", "bic", "hot", "securityLevel", "rateLimit","waf"]
    }
  }
}

resource ibm_cis_ruleset_rule "custom_rule2" {
  cis_id   = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  ruleset_id = data.ibm_cis_ruleset_entrypoint_versions.custom_rule_ep.rulesets[0].ruleset_id
  rule {
    action = "block"
    description = "adding custom rule via terraform2"
    expression = "(http.request.uri.path eq \"/path1\")"
    enabled = "false"
  }
}
```

For more information, see [ibm_cis_ruleset_rule](#) in the Terraform registry.



Note: You can update a custom rule with Terraform by modifying the preceding example and running the `terraform apply` command. To delete the rule, simply remove the configuration and run `terraform apply`.

About WAF custom rules

Use the WAF custom rules feature to control incoming traffic by filtering requests to a domain. You can perform actions such as **Block** or **Managed challenge** on incoming requests according to the rules you define.

Like other rules evaluated by the Ruleset Engine, WAF custom rules require the following basic parameters:

- An expression that specifies the criteria you are matching traffic on using the rules language.
- An action that specifies what to do when there is a match for the rule.



Important: Custom rules are evaluated in the order that they are set, from first to last position, in the custom rules table. You can view the table by navigating to **Security > Firewall rules**. Some actions like **Block** stop the evaluation of other rules. For more details on actions and their behavior, see [Ruleset Engine rule actions](#).

Common use cases

The following sections detail examples of common use cases for WAF custom rules.

Only allow traffic from specified countries

Using the **Block** action, this example blocks requests based on their country code by using the `ip.src.country` field, allowing requests from only two countries: the United States and Mexico:

```
$ (not ip.src.country in {"US" "MX"})
```

Block requests by IP reputation

IP reputation is a score from 0 (zero risk) to 100 (high risk), classifying the IP reputation of a visitor.



Note: IP reputation is calculated based on Project Honeypot, external public IP information, and internal threat intelligence from managed rules and DDoS.

Using the **Block** action, this example blocks requests based on country codes (ISO 3166-1 Alpha 2 ↗ format), from IP addresses that score greater than 0:

```
$ (ip.src.country in {"CN" "TW" "US" "GB"} and cf.threat_score gt 0)
```

Only allow traffic from specified IP addresses

The **Block** action restricts access to your domain, allowing only requests from specified IP addresses and denying all others.

For example, to allow traffic only from the IPs `192.0.2.10` and `203.0.113.25`, run the following expression:

```
$ (not ip.src in {"192.0.2.10" "203.0.113.25"})
```

Use one of the following expressions depending on the protocol you want to restrict:

- If you want to specify subnets instead of single IP addresses, IBM Cloud CIS supports only:

IPv4: `/16` and `/24`

IPv6: `/32`, `/48`, and `/64`

- Always include both IPv4 and IPv6 for full coverage.
- Blocking only IPv4 (0.0.0.0/0) still allows IPv6 traffic. To block both:

```
$ (ip.src in {0.0.0.0/0} or ip.src in {::/0})
```

Configuring a rule to skip other CIS features

The skip action supports different skip options, according to the security features or products that you want to skip.

This section contains examples of different skip rule scenarios for WAF custom rules. Take the following considerations into account:

- The `{zone_id}` value is the ID of the zone where you want to add the rule.
- The `{ruleset_id}` value is the ID of the entry point ruleset of the `http_request_firewall_custom` phase. For details on obtaining this ruleset ID, see [Getting the custom rule entry point for the API](#). The rule skips all remaining rules in the current ruleset for requests that match the rule expression:

```
$ curl -X POST "https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets/$RULESET_ID/rules" \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json" \
--data '{
  "action": "skip",
  "action_parameters": {
    "ruleset": "current"
  },
  "expression": "http.request.uri.path contains \"/skip-current-ruleset/\"",
  "description": ""
}'
```

Skip a phase

This example uses the [Create a zone ruleset](#) to add a rule to the existing `http_request_firewall_custom` phase entry point ruleset with ID `RULESET_ID`. The rule skips the `http_ratelimit` phase for requests that match the rule expression:

```
$ curl -X POST "https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets/$RULESET_ID/rules" \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json" \
--data '{
  "action": "skip",
  "action_parameters": {
    "phases": [
      "http_ratelimit"
    ]
  },
  "expression": "http.request.uri.path contains \"/skip-phase/\"",
  "description": ""
}'
```

Skip a phase and do not log matching requests

This example uses the [Create a zone ruleset](#) rule operation to add a rule that both skips the `http_ratelimit` phase and disables event logging for the current rule:

```
$ curl -X POST "https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets/$RULESET_ID/rules" \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json" \
--data '{
  "action": "skip",
  "action_parameters": {
    "phases": [
      "http_ratelimit"
    ]
  },
  "logging": {
    "enabled": false
  },
  "expression": "http.request.uri.path contains \"/disable-logging/\"",
  "description": ""
}'
```

Skip security products

This example uses the [Create a zone ruleset](#) rule operation to add a rule that skips the Zone Lockdown and User Agent Blocking products for requests that match the rule expression:

```
$ curl -X POST "https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets/$RULESET_ID/rules" \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json" \
--data '{
  "action": "skip",
  "action_parameters": {
    "products": [
      "zoneLockdown",
      "uaBlock"
    ]
  }
```

```
}  
"expression": "http.request.uri.path contains \"/skip-products/";  
"description": ""  
}
```

Working with WAF custom rules

WAF custom rules offer power and flexibility by targeting HTTP traffic and applying custom criteria to block, challenge, log, or allow certain requests.

You can create many types of WAF custom rules. However, the number of active rules on your site is limited by your customer plan. For more information on entitlements, see [Comparing CIS plans](#).

The number of active rules per plan is fixed. Currently, you can't purchase more active rules.



Important: Before getting started, it's a good idea to review [Using fields, functions, and expressions](#).

You can create, update, and delete a custom rule by using the console, CLI, API, or Terraform.

Creating a custom rule in the console

Follow these steps to create a custom rule in the console:



Note: WAF custom rules are configured by using the existing Firewall rules page. Any legacy firewall rules that were previously created on your domain are automatically converted into WAF custom rules.

1. In the CIS console, navigate to the **Security**.
2. Select the **Custom rules** tab and then click **Create**.
3. Enter an optional description.
4. Optionally, input a priority, if necessary. A priority of zero is a null priority and is evaluated last.
5. Use the UI builder in the **Incoming requests** section to add a condition. To build an expression with multiple conditions, click either:
 - **And** - to evaluate conditions that use *and* logic
 - **Or** - to evaluate conditions or groups of previously *and*'ed conditions that use *or* logic

You can see that as you build a condition, the Expression Preview shows the expression in plain text.



Note: In the Expression Preview, you can click to edit your expression manually instead of using the Visual Expression Builder, or switch between the two. However, depending on the complexity of a manually constructed expression, the Visual Expression Builder might be unable to render it.

6. Pick an action from the **Response** list menu.
7. To save your rule, choose the most appropriate option by clicking either:
 - **Save as draft** to save your rule, but keep it disabled.
 - **Save and deploy** to save your rule and activate it.

Updating a custom rule in the console

Follow these steps to update an existing custom rule in the console:

1. In the CIS console, navigate to the **Security**.
2. Select the **Custom rules** tab.
3. In the Custom rules table, locate the rule that you want to modify, then click the Actions menu on the right of the row.
4. Select **Edit**.
5. Make your changes to the rule.
6. To save your rule, choose the most appropriate option by clicking either:
 - **Save as draft** to save your rule, but keep it disabled.

- **Save and deploy** to save your rule and activate it.



Note: To pause or activate any rule in the list of existing rules, click the **Enabled** toggle.

Deleting a custom rule in the console

Follow these steps to delete an existing custom rule in the console:

1. In the CIS console, navigate to the **Security**.
2. In the Custom rules table, locate the rule to delete and click the Actions menu on the right of the row.
3. Select **Delete** from the action menu.
4. Click **Delete WAF custom rule** to delete the rule.

Creating a custom rule from the CLI

To create a custom rule from the CLI, follow these steps:

1. [Set up your CLI environment](#).
2. Log in to your account from the CLI. After you enter the password, the system prompts for the account and region that you want to use:

```
$ ibmcloud login --sso
```

3. Run the following command to create a custom rule:

```
$ ibmcloud cis custom-waf rule-create DNS_DOMAIN_ID --match EXPRESSION --action ACTION [--description DESCRIPTION] [--enabled true|false] [-i, --instance INSTANCE] [--output FORMAT]
```

You can also create a custom rule by providing a JSON file or a JSON string directly:

```
$ ibmcloud cis custom-waf rule-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain.

--match

Specifies the conditions that must be matched for the rule to run. For match value, see [Using fields, functions, and expressions](#).

--action

The rule action to perform. Valid values: "block", "challenge", "js_challenge", "managed_challenge", "log", "skip". For "block" and "skip" actions, use JSON file or JSON string instead.

--enabled

Indicates if the rule is active. The default is "false".

--description

A brief description of the rule.

--json

The JSON file or JSON string that is used to describe a custom rule.

- The required fields in JSON data are **expression** and **action**.
 - **expression** : Specifies the conditions that must be matched for the rule to run.
 - **action** : The rule action to perform. Valid values are **block**, **challenge**, **js_challenge**, **managed_challenge**, **log**, and **skip**.
- The optional fields are **description**, **enabled**, **logging**, and **action_parameters**.

- `action_parameters` : The rule action parameters.
- `ruleset` : Skip all remaining rules or one or more WAF-managed rulesets. Valid value is `current` .
- `phases` : Skips WAF components for matching requests. Valid values are `http_ratelimit` , `http_request_firewall_managed` , and `http_request_sbfm` .
- `products` : Skips specific security products for matching requests. Valid values are `waf` , `rateLimit` , `securityLevel` , `hot` , `bic` , `uaBlock` , and `zoneLockdown` .
- `response` : Define a custom response for `block` action.
 - `status_code` : Choose an HTTP status code for the response in the range `400-499` .
 - `content_type` : The content type of a custom response. Valid response types are : `text/html` , `text/plain` , `application/json` , `text/xml` .
 - `content` : The response body.
- `description` : Briefly describes the rule.
- `enabled` : Indicates whether the rule is active or not. When this field is disabled, matched requests don't appear in firewall requests.
- `logging` : Log requests matching the skip rule. This field is only available for 'skip' action.

Sample JSON data:

```
{
  "description": "test-custom-rule",
  "expression": "(http.cookie contains \"test\")",
  "action": "skip",
  "logging": {
    "enabled": true
  },
  "action_parameters": {
    "ruleset": "current",
    "phases": [
      "http_ratelimit",
      "http_request_firewall_managed",
      "http_request_sbfm"
    ],
    "products": [
      "waf",
      "rateLimit",
      "securityLevel",
      "hot",
      "bic",
      "uaBlock",
      "zoneLockdown"
    ]
  },
  "enabled": true
}
```

`-i, --instance`

Instance name or ID. If not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

Specify output format, only `JSON` is supported.

Updating a custom rule from the CLI

Run the following command to update a custom rule from the CLI:

```
$ ibmcloud cis custom-waf rule-update DNS_DOMAIN_ID [--match EXPRESSION] [--action ACTION] [--description DESCRIPTION] [--enabled true|false] [-i, --instance INSTANCE] [--output FORMAT]
```


You can also update a custom rule by providing a JSON file or a JSON string directly:

```
$ ibmcloud cis custom-waf rule-update DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain.

RULE_ID

The ID of the rule.

--match

Specifies the conditions that must be matched for the rule to run. For match value, see [Using fields, functions, and expressions](#).

--action

The rule action to perform. Valid values: "block", "challenge", "js_challenge", "managed_challenge", "log", "skip".For "block" and "skip" actions, use JSON file or JSON string instead.

--enabled

Indicates if the rule is active. The default value is "false".

--description

A brief description of the rule.

--json

The JSON file or JSON string that is used to describe a custom rule.

- The required fields in JSON data are **expression** and **action**.
 - **expression** : Specifies the conditions that must be matched for the rule to run.
 - **action** : The rule action to perform. Valid values are **block** , **challenge** , **js_challenge** , **managed_challenge** , **log** , and **skip** .
- The optional fields are **description** , **enabled** , **logging** , and **action_parameters** .
 - **action_parameters** : The rule action parameters.
 - **ruleset** : Skip all remaining rules or one or more WAF-managed rulesets. Valid value is **current**
 - **phases** : Skips WAF components for matching requests. Valid values are **http_ratelimit** , **http_request_firewall_managed** , and **http_request_sbfm** .
 - **products** : Skips specific security products for matching requests. Valid values are **waf** , **rateLimit** , **securityLevel** , **hot** , **bic** , **uaBlock** , and **zoneLockdown** .
 - **response** : Define a custom response for 'block' action.
 - **status_code** : Choose an HTTP status code for the response in the range **400-499** .
 - **content_type** : The content type of a custom response. Valid response types are **text/html** , **text/plain** , **application/json** , and **text/xml** .
 - **content** : The response body.
 - **description** : Briefly describes the rule.
 - **enabled** : Indicates whether the rule is active or not. When this field is disabled, matched requests don't appear in firewall requests.
 - **logging** : Log requests matching the skip rule. This field is only available for 'skip' action.

Sample JSON data:

```
{
  "description": "test-custom-rule",
  "expression": "(http.cookie contains \"test\")",
  "action": "block",
  "action_parameters": {
```

```
    "response": {
      "status_code": 429,
      "content_type": "text/xml",
      "content": "reject"
    }
  },
  "enabled": true
}
```

`-i, --instance`

Instance name or ID. If not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

Specify output format, only `JSON` is supported.

Deleting a custom rule from the CLI

Run the following command to delete a custom rule from the CLI:

```
$ ibmcloud cis custom-waf rule-delete DNS_DOMAIN_ID RULE_ID [-f, --force] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain.

`RULE_ID`

The ID of the custom rule.

`-i, --instance`

Instance name or ID. If not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`-f, --force`

Attempt to delete a custom rule without prompting for confirmation.

`--output`

Specify output format, only `JSON` is supported.

Command examples

- To create a custom rule:

```
ibmcloud cis custom-waf rule-create 601b728b86e630c744c81740f72570c3 --action challenge --description "rule 1" --enabled true --match "(http.host eq \"www.example.com\")"
```

- To update a custom rule:

```
ibmcloud cis custom-waf rule-update 601b728b86e630c744c81740f72570c3 4d37cb6f87654e96a18bc531628a4d27 --enabled true
```

- To delete a custom rule:

```
ibmcloud cis custom-waf rule-delete 601b728b86e630c744c81740f72570c3 4d37cb6f87654e96a18bc531628a4d27
```

Getting the custom rule entry point for the API

All custom rule API operations require a `RULESET_ID` of the entry point ruleset for the custom rules phase.

Follow these steps to get the custom rule entry point ruleset:

1. Set up your API environment with the correct variables.

2. Store the following values in variables to be used in the API command:

CRN : The full URL-encoded Cloud Resource Name (CRN) of the service instance.

ZONE_ID : The domain ID.

3. When all variables are initiated, get the entry point ruleset:

```
$ curl -X GET "https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets/phases/http_request_firewall_custom/entrypoint" \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json"
```

The ruleset ID is in the response of the successful request. If the preceding call returns a 404 Not Found response, use the following API to create the entry point ruleset for the custom rule phase:

```
$ curl -X POST https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json" \
--data '{
  "name": "Zone-level phase entry point",
  "kind": "zone",
  "description": "Custom rule entry point ruleset.",
  "phase": "http_request_firewall_custom"
}'
```

Creating a custom rule with the API

Follow these steps to create a custom rule with the API:

1. Set up your API environment with the correct variables.
2. Store the following values in variables to be used in the API command:

CRN : The full URL-encoded Cloud Resource Name (CRN) of the service instance.

ZONE_ID : The domain ID.

RULESET_ID : The ID of the custom rule endpoint ruleset.

3. When all variables are initiated, create the custom rule:

```
$ curl -X POST "https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets/$RULESET_ID/rules" \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json" \
--data '{
  "description": "My custom rule with plain text response",
  "expression": "(ip.src.country eq \"GB\") or ip.src.country eq \"FR\") and cf.waf.score lt 20",
  "action": "block",
  "action_parameters": {
    "response": {
      "status_code": 403,
      "content": "Your request was blocked.",
      "content_type": "text/plain"
    }
  }
}'
```

Updating a custom rule with the API

Follow these steps to update an existing custom rule with the API:

1. Set up your API environment with the correct variables.
2. Store the following values in variables to be used in the API command:

CRN : The full URL-encoded Cloud Resource Name (CRN) of the service instance.

ZONE_ID : The domain ID.

RULESET_ID : The ID of the custom rule endpoint ruleset.

RULE_ID : The ID of the custom rule to be modified.

3. When all variables are initiated, update the custom rule:

```
$ curl -X PATCH "https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets/$RULESET_ID/rules/$RULE_ID" \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json" \
--data '{
  "enabled": false,
  "description": "block GB and FR or based on IP Reputation (temporarily disabled)"
}'
```

Deleting a custom rule with the API

Follow these steps to delete an existing custom rule with the API:

1. Set up your API environment with the correct variables.
2. Store the following values in variables to be used in the API command:

CRN : The full URL-encoded Cloud Resource Name (CRN) of the service instance.

ZONE_ID : The domain ID.

RULESET_ID : The ID of the custom rule endpoint ruleset.

RULE_ID : The ID of the custom rule to be modified.

3. When all variables are initiated, delete the custom rule:

```
$ curl -X DELETE "https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets/$RULESET_ID/rules/$RULE_ID" \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json"
```

Creating a custom rule with Terraform

To create a custom rule, you must create an entry point first, then create the custom rule. To do so, follow these steps:

1. To create an entry point ruleset, run the following command:

```
$ resource "ibm_cis_ruleset_entrypoint_version" "test" {
  cis_id   = "<cis-id>"
  domain_id = "<domain-id>"
  phase = "http_request_firewall_custom"
  rulesets {
    description = "Entry point ruleset for custom ruleset"
  }
  lifecycle {
    ignore_changes = [
      rulesets
    ]
  }
}
```

2. To create a custom rule, run the following command:

```
resource ibm_cis_ruleset_rule "config" {
  cis_id   = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  ruleset_id = resource.ibm_cis_ruleset_entrypoint_version.rulesets[0].ruleset_id
  rule {
    action = "block"
    description = "var.description"
    expression = "true"
    enabled = "false"
    action_parameters {
      response {
```

```

        status_code = var.status_code
        content = var.content
        content_type = "text/plain"
    }
}
position {
    index = var.index
    after = <id of any existing rule>
    before = <id of any existing rule>
}
}
}

```

The following example shows how to create an entry point and WAF custom rule:

```

resource ibm_cis_ruleset_entrypoint_version test {
  cis_id   = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  phase = "http_request_firewall_custom"
  rulesets {
    description = "Entrypoint ruleset for custom ruleset"
  }
  lifecycle {
    ignore_changes = [
      rulesets
    ]
  }
}
data ibm_cis_ruleset_entrypoint_versions custom_rule_ep {
  cis_id   = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  phase = "http_request_firewall_custom"
  depends_on = [
    ibm_cis_ruleset_entrypoint_version.test
  ]
}

resource ibm_cis_ruleset_rule "custom_rule1" {
  cis_id   = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  ruleset_id = data.ibm_cis_ruleset_entrypoint_versions.custom_rule_ep.rulesets[0].ruleset_id
  rule {
    action = "skip"
    description = "adding custom rule via terraform"
    expression = "true"
    enabled = "true"
    action_parameters{
      ruleset = "current"
      phases = ["http_ratelimit", "http_request_sbfm", "http_request_firewall_managed"]
      products = ["zoneLockdown", "uaBlock", "bic", "hot", "securityLevel", "rateLimit", "waf"]
    }
  }
}

resource ibm_cis_ruleset_rule "custom_rule2" {
  cis_id   = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  ruleset_id = data.ibm_cis_ruleset_entrypoint_versions.custom_rule_ep.rulesets[0].ruleset_id
  rule {
    action = "block"
    description = "adding custom rule via terraform2"
    expression = "(http.request.uri.path eq \"/path1\")"
    enabled = "false"
  }
}

```

For more information, see [ibm_cis_ruleset_rule](#) in the Terraform registry.



Note: You can update a custom rule with Terraform by modifying the preceding example and running the `terraform apply` command. To

delete the rule, simply remove the configuration and run `terraform apply` .

Using fields, functions, and expressions



Deprecated: Firewall rules are deprecated. CIS moved existing firewall rules to WAF custom rules. For more information on this change, see [Migrating to custom rules](#).

Along with actions, fields and expressions are the building blocks of firewall rules. These two elements work together when defining the criteria to use when a firewall rule is matched.

Fields

When CIS receives an HTTP request, it is examined and a table of fields is produced to match against. This field table exists while the current request is being processed. Think of it as a table that holds the request properties to be matched against expressions.

Each field value can be sourced from different places, such as:

- Primitive properties are , obtained directly from the traffic – for example, `http.request.uri.path` .
- Derived values, resulting from a transformation, composition, or basic operation – for example, making the value of `http.request.uri.path` all lowercase and available as a field of another field.
- Computer values, resulting from a lookup, computation, or other intelligence – for example, a `cf.threat_score` calculated dynamically by a machine learning process that inspects related primitive and derived values.

Available fields

Field name	Type	Example value	Notes
http.cookie	String	session=A12345;-background=light	Entire cookie as a string
http.host	String	www.example.com	The hostname used in the full request URI
http.referer	String	<i>HTTP referer header</i>	
http.request.full_uri	String	https://www.example.com/articles/index?section=539061&expand=comments	The full URI as received by the web server (does not include <i>#fragment</i> , which is not sent to web servers)
http.request.method	String	POST	The HTTP method, in uppercase
http.request.uri	String	/articles/index?section=539061&expand=comments	The absolute URI of the request
http.request.uri.path	String	/articles/index	The path of the request
http.request.uri.query	String	section=539061&expand=comments	The whole query string, minus the delimiting prefix "?"
http.user_agent	String	Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/65.0.3325.181 Safari/537.36	The whole HTTP user agent
http.x_forwarded_for	String	<i>The full X-Forwarded-For HTTP header</i>	
ip.src	IP address	93.155.208.22	The client TCP IP address, which can be adjusted to reflect the real client IP of the original client as applicable (for example, by using HTTP headers like X-Forwarded-For or X-Real-IP)
ip.src.asnum	Number	222	The Autonomous System (AS) number

ip.src.country	String	GB	The 2-letter country code
ssl	Boolean	true	Whether the HTTP connection to the client is encrypted

Available fields



Note: These standard fields follow the naming convention of the Wireshark display field reference. However, some subtle variations might exist in the preceding example values.

In addition to the standard fields, the following Cloudflare-defined fields are also available:

Field name	Type	Example value	Notes
cf.client.bot	Boolean	true	This field indicates whether the request is coming from a known bot or crawler, regardless of good or bad intent.
cf.threat_score	Number	A 0-100 value	This field represents a risk score, 0 indicates low risk as determined by Cloudflare. Values greater than 10 can represent spammers or bots, and values greater than 40 point to bad actors on the internet. It is rare to see values higher than 60, so tune your firewall rules to challenge those greater than 10, and to block those greater than 50.
cf.waf.score	Number	1-99	Machine learning–based score that estimates the likelihood of a request being malicious. Scores range from 1 (most likely malicious) to 99 (most likely safe). Lower scores indicate a higher risk. Useful for creating threshold-based firewall rules.

Available Cloudflare fields

Functions

The firewall rules language has several functions to convert fields.



Note: These functions are not currently supported in the Expression Builder.

Function name	Argument types	Return type	Usage example	Notes
lower	String	String	lower(http.host) == "www.example.com"	Converts a string field to lowercase. Only uppercase ASCII bytes are being converted. Every other bytes are left as-is.
upper	String	String	upper(http.host) == "www.example.com"	Converts a string field to uppercase. Only lowercase ASCII bytes are being converted. Every other bytes are left as-is.

Firewall rules functions

Expressions

An expression returns true or false based on a match against incoming traffic. For example:

```
http.host eq "www.example.com" and ip.src in 92.182.212.0/24
```

In this example, two single expressions comprise a compound expression. Think of each single expression as a condition. Each condition is evaluated individually before applying and logic to determine the final result of the compound expression.

Looking at the first single expression, you can see that it contains:

- a field - http.host
- a comparison operator - eq
- a value - "www.example.com"

Not all conditions have the same structure. Other examples that use different structures are discussed in the next section.

Comparison operators

The following comparison operators are available for use in expressions:

English	C-like	Description
eq	==	Equal
ne	!=	Not equal
lt	<	Less than
le	<=	Less than or equal to
gt	>	Greater than
ge	>=	Greater than or equal to
contains		Exactly contains
matches	~	Re2 inspired regular expression
in		Value appears in a set of values. Supports ranges that use the ".." notation.
not	!	See Boolean comparison
bitwise_and	&	Compare bit field value

Comparison operators for expressions



Note: Currently the Expression Builder supports only English operators.

An expression might contain a mix of English and C-like operators. For example, `ip.src eq 93.184.216.34` is equivalent to `ip.src == 93.184.216.34`.

Certain comparison operators apply to specific fields based on type. The following matrix provides examples of which operators are available for various field types:

English	C-like	String	IP address	Number
eq	==	http.request.uri.path eq "/articles/2008/"	ip.src eq 93.184.216.0	cf.threat_score eq 10
ne	!=	http.request.uri.path ne "/articles/2010/"	ip.src ne 93.184.216.0	cf.threat_score ne 60
lt	<	http.request.uri.path lt "/articles/2009/"		cf.threat_score lt 10
le	<=	http.request.uri.path le "/articles/2008/"		cf.threat_score le 20
gt	>	http.request.uri.path gt "/articles/2006/"		cf.threat_score gt 25
ge	>=	Greater than or equal to		cf.threat_score ge 60
contains		http.request.uri.path contains "/articles/"		
matches	~	http.request.uri.path ~ "^/articles/200[7-8]/\$"		

in	http.request.method in { "HEAD" "GET" }	ip.src in { 93.184.216.0 93.184.216.1 }	cf.threat_score in {0 2 10}
----	---	--	--------------------------------

Comparison operators for fields

**Important:** The evaluation of expressions that use string values is case-sensitive. As such, a firewall rule might require you to define more than one test condition. Enterprise customers can use a regular expression with the matches operator to capture multiple variations with a single expression.

Boolean comparison

For fields of Boolean type (for example, `ssl`) the field appears by itself in the expression when evaluating for a true condition. For a false condition, the **not** operator applies.

True	False
ssl	not ssl

Boolean comparison

Compound expressions

You can create compound expressions by grouping two or more single expressions by using logical operators.

English	C-like	Description	Example	Precedence
not	!	Logical NOT	not (http.host eq "www.example.com" and ip.src in 93.184.216.0/24)	1
and	&&	Logical AND	http.host eq "www.example.com" and ip.src in 93.184.216.0/24	2
xor	^^	Logical XOR	http.host eq "www.example.com" xor ip.src in 93.184.216.0/24	3
or		Logical OR	http.host eq "www.example.com" or ip.src in 93.184.216.0/24	4

Compound expressions

To alter the order of precedence, you can group expressions with parentheses. Using no parentheses, expressions are implicitly grouped based on standard precedence:

```
ssl and http.request.uri.path eq /login or http.request.uri.path eq /oauth
```

Applying explicit grouping:

```
(ssl and http.request.uri.path eq /login) or http.request.uri.path eq /oauth
```

Giving precedence to or with parentheses:

```
ssl and (http.request.uri.path eq /login or http.request.uri.path eq /oauth)
```

While `not` is used for grouping, it can be used to negate a single comparison. For example, `not ip.src eq 93.184.216.0` is equivalent to `not (ip.src eq 93.184.216.0)`.

Finally, you can also negate grouped expressions:

```
not (http.request.method eq "POST" and http.request.uri.path eq "/login")
```

Deviations from Wireshark display filters

Firewall rules expressions are inspired by Wireshark display filters. However, the implementation deviates in the following ways:

- For CIDR IP equality tests, Wireshark allows ranges in the format `ip.src == 1.2.3.0/24`, while CIS supports equality tests using only a single

IP address. To compare a CIDR, use the `in` operator; for example, `ip.src in {1.2.3.0/24}`.

- In Wireshark, `ssl` is a protocol field that contains hundreds of other fields of various types that are available for comparison in multiple ways. However, in Firewall Rules, `ssl` is a single Boolean field that is used to determine whether the connection from the client to CIS is encrypted.
- The `slice` operator is not supported.
- Not all functions are supported. Currently, `len()`, and `count()` are not supported.

Working with WAF managed rules

About WAF managed rules

You can use managed rules to deploy pre-configured managed rulesets that provide immediate protection against the following threats:

- Zero-day vulnerabilities
- Top-10 attack techniques
- Use of stolen and exposed credentials
- Extraction of sensitive data

Benefits

Managed rules provide the following benefits over the previous WAF version:

New matching engine

Managed rules use the Ruleset Engine feature suite, which allows faster managed rule deployments so that you can check more traffic without scaling issues, while improving performance and security. The rules follow the same syntax that is used in other CIS security features, such as WAF custom rules.

Updated managed rulesets

The CIS OWASP Core Ruleset is an implementation of version 3.3.0 of the [OWASP ModSecurity Core Rule Set \(CRS\)](#). This version introduces paranoia levels and reduces false positives rates compared to the version used in WAF managed rules (2.x). It also provides greater control over sensitivity scoring, with clear visibility into how individual rules contribute to the score and the total score for a triggered request. In addition, the updated ruleset separates rule status from rule action, which makes it easier to configure both independently.

Better rule browsing and configuration

The default configuration of managed rules has a low false positive rate while providing a food security baseline for your web applications. For the best possible security, enable as many rules as possible. You might need to customize the ruleset behavior based on the underlying application. Bulk editing controls in addition to inline single rule controls contribute to faster configuration changes based on specific use cases.

Rulesets

These managed rulesets are regularly updated. You can adjust the behavior of specific rules in these rulesets, choosing from several possible actions.

Ruleset name	Description	Ruleset ID
CIS Managed Ruleset	This ruleset provides fast and effective protection for all of your applications. The ruleset is updated frequently to cover new vulnerabilities and reduce false positives.	efb7b8c949ac4650a09736fc376e9aee
CIS OWASP Core Ruleset	Based on the Open Web Application Security Project (OWASP) ModSecurity Core Rule Set (v3.3.0).	4814384a9e5d4991b9815dcfc25d2f1f
Exposed credentials check	Deploy an automated credentials check on your user authentication endpoints. For any credential pair, the CIS WAF performs a lookup against a public database of stolen credentials.	c2e184081120413c86c3ab7e14069605

Available rulesets and ruleset IDs

Request body inspection limit

The WAF inspects HTTP request bodies up to 128KB by default. This limit applies to all managed rulesets, including the CIS Managed Ruleset

and OWASP Core Ruleset, and determines how much of each request the WAF can evaluate. Customers can request an increase of the inspection limit to 1 MB by [creating a support case](#).



Note: Increasing the inspection limit can affect how managed rules evaluate traffic and could impact rule matches. Review ruleset behavior if higher limits are required.

Deploying managed rulesets

You can deploy managed rulesets at the zone or instance level by using the CLI, API, or Terraform.



Tip: You can deploy a managed ruleset with customized behavior (changes that are executed in addition to the default configuration) by overriding it. For more information, see [Overriding managed rulesets](#).

Deployment workflow

Use the following workflow to deploy a managed ruleset to a phase at the zone level:

1. Get your zone ID.
2. Run the **List zone** rulesets operation to obtain the available rulesets.
3. Find the ruleset ID of the managed ruleset that you want to deploy.
4. Identify the phase where you want to deploy the managed ruleset. Make sure that the managed ruleset belongs to the same phase where you want to deploy it.
5. Add a rule to the zone-level phase entry point ruleset that runs the managed ruleset.

Deploying managed rulesets in the console

To deploy managed rulesets in the CIS console, follow these steps:

1. In the CIS console, navigate to the **Security** section.
2. Select the **WAF** tab.
3. Click **Add exception** to open the Add exception side panel.
4. Enter a name for your managed rules exception.
5. Define the match condition by building an expression:
 - Choose the request field, select an operator, and provide a value.
 - You can also use the Expression Builder to manually write expressions
 - Combine multiple conditions using **And** and **Or** operators to create complex logic.
6. Choose whether to log requests that match the expression.
7. Decide how the ruleset should behave for matching requests:
 - Skip all remaining rules, or
 - Skip specific rules from a managed ruleset

If you select to skip specific rules, click **Browse rules**, select a ruleset, and then select the specific rules to skip.
8. Click **Deploy** to activate the configuration.

After deployment, you can return at any time to edit your managed ruleset configuration.

Deploying managed rulesets from the CLI

You can deploy managed rulesets from the CLI.

Listing managed rulesets from the CLI

To list all zone-managed rulesets from the CLI, follow these steps:

1. [Set up your CLI environment.](#)
2. Log in to your account from the CLI. After you enter the password, the system prompts for the account and region that you want to use:

```
$ ibmcloud login --sso
```

3. Run the following command to create a custom rule:

```
$ ibmcloud cis managed-waf rulesets DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the domain.

-i, --instance

Instance name or ID.

--output

Specifies the output format; only JSON is supported.

Updating deployed ruleset from the CLI

To update a deployed ruleset from the CLI, follow these steps:

1. [Set up your CLI environment.](#)
2. Log in to your account from the CLI. After you enter the password, the system prompts for the account and region that you want to use:

```
$ ibmcloud login --sso
```

3. Run the following command to create a custom rule:

```
$ ibmcloud cis managed-waf deployment-add-ruleset DNS_DOMAIN_ID RULESET_ID [--match EXPRESSION] [--enabled true|false] [--override-action ACTION] [--override-status STATUS] [--paranoia-level LEVEL] [--override-rules RULE] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the domain.

--match

The conditions that must be matched for the rule to run. See [Using fields, functions, and expressions](#) for a list of values to match.

--enabled

Indicates whether the rule is active. The default is **true**.

--override-action

The ruleset action of any overrides. Valid values are **managed_challenge**, **block**, **js_challenge**, **log**, **challenge**.

--paranoia-level

The OWASP paranoia level. Valid values are **PL1**, **PL2**, **PL3**, **PL4**. This is only available for the **CIS OWASP Core Ruleset**.

override-rules

The rules options of the overrides. For example **--override-rules rule=RULE_ID,action=ACTION,enabled=STATUS**.

-i, --instance

The instance name or ID.

--output

specifies the output format; only JSON is supported.

Command example

This example shows how to deploy the ruleset `efb7b8c949ac4650a09736fc376e9aee` and the overriding the rule present inside the ruleset. It is also overriding rules with category wordpress.

```
ibmcloud cis managed-waf deployment-add-ruleset $domain efb7b8c949ac4650a09736fc376e9aee --match "(http.cookie eq \"example.com/contact?page=1234\")"
--action execute --enabled false --override-action block --override-status false --override-rules rule=5de7edfa648c4d6891dc3e7f84534ffa,action=log,enabled=true --
override-rules action=managed_challenge,rule=e3a567afc347477d9702d9047e97d760
```

Deploying managed rulesets with the API

You can deploy managed rulesets with the API.

Getting the custom rule entry point for the API

All custom rule API operations require a `RULESET_ID` of the entry point ruleset for the custom rules phase. This entry point ruleset may already exist or needs to be created if it does not exist.

Follow these steps to get the custom rule entry point ruleset:

1. Set up your API environment with the correct variables.
2. Store the following values in variables to be used in the API command:

`CRN`: The full URL-encoded Cloud Resource Name (CRN) of the service instance.

`ZONE_ID`: The domain ID.

3. When all variables are initiated, get the entry point ruleset:

```
$ curl -X GET "https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets/phases/http_request_firewall_managed/entrypoint" \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json"
```

The ruleset ID will be in the response of the successful request. If the preceding call returns a 404 Not Found response, use the following API to create the endpoint ruleset for the custom rule phase:

```
$ curl -x POST https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json" \
--data '{
  "name": "Zone-level phase entry point",
  "kind": "zone",
  "description": "WAF managed rule endpoint ruleset.",
  "phase": "http_request_firewall_managed"
}'
```

Listing managed rulesets with the API

Follow these steps to list managed rulesets with the API:

1. Set up your API environment with the correct variables.
2. Store the following values in variables to be used in the API command:

`CRN`: The full URL-encoded Cloud Resource Name (CRN) of the service instance.

`ZONE_ID`: The domain ID.

`RULESET_ID`: The ID of the custom rule endpoint ruleset.

3. When all variables are initiated, list all zone-managed rulesets:

```
curl -X GET \
  https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets \
  -H 'content-type: application/json' \
  -H 'accept: application/json' \
```



```
-H 'x-auth-user-token: Bearer xxxxxx'
```

Updating entry point rulesets with the API

Follow these steps to update an entry point ruleset with the API:

1. Set up your API environment with the correct variables.
2. Store the following values in variables to be used in the API command:

CRN : The full URL-encoded Cloud Resource Name (CRN) of the service instance.

ZONE_ID : The domain ID.

RULESET_ID : The ID of the custom rule endpoint ruleset.

3. When all variables are initiated, update the entry point ruleset:

```
curl -X PUT \
  https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets/phases/$RULESET_PHASE/endpoint \
  -H 'content-type: application/json' \
  -H 'accept: application/json' \
  -H 'x-auth-user-token: Bearer xxxxxx' \
  -d '{"rules":[{"action":"execute","action_parameters":{"id":"4814384a9e5d4991b9815dcfc25d2f1f"},"expression":"true","description":"Execute WAF OWASP ruleset"}]}'
```

Where:

- **\$RULESET_PHASE** is the ruleset phase that will be deployed. Use `http_request_firewall_managed` to deploy managed WAF rulesets.
- **-d** is the object of attributes that are required to create the ruleset.
 - **rules** is the array of rules to deploy with the ruleset. For example:
 - **action** is the action for the rule to take. See [Rules actions](#) for a description of actions that can be used.
 - **action_parameters** is the object for defining what the action should operate on.
 - **id** is the ID of the ruleset to run. This ID is retrieved from the `list zone rulesets` operation.
 - **expression** is the condition under which the rule runs. Using `true` means that this rule always run.
 - **description** defines the summary of what your rule is accomplishing.

Deploying managed rulesets with Terraform

Listing managed rulesets with Terraform

The following example lists managed rulesets by using Terraform:

```
data "ibm_cis_rulesets" "tests" {
  cis_id = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
}
```

For more information about the arguments and attributes, see [ibm_cis_rulesets](#) in the Terraform registry.

Listing all rules of a managed ruleset

The following example lists all rules of a managed ruleset by using Terraform:

```
resource "ibm_cis_ruleset" "config" {
  cis_id = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  ruleset_id = "943c5da120114ea5831dc1edf8b6f769"
}
```

For more information about the arguments and attributes, see [ibm_cis_ruleset](#) in the Terraform registry.

Overriding rule

The following example overrides a rule by using Terraform:

```
resource "ibm_cis_ruleset_entrypoint_version" "test" {
  cis_id   = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  phase = "http_request_firewall_managed"
  rulesets {
    description = "Entrypoint ruleset for managed ruleset"
    rules {
      action = "execute"
      description = "Deploy CIS managed ruleset"
      enabled = true
      expression = "true"
      action_parameters {
        id = "efb7b8c949ac4650a09736fc376e9aee"
      }
      overrides {
        action = "block"
        enabled = true
        override_rules {
          rule_id = "var.overriden_rule.id"
          enabled = true
          action = "block"
        }
      }
      categories {
        category = "wordpress"
        enabled = true
        action = "block"
      }
    }
  }
}
```

For more information about the arguments and attributes, see [ibm_cis_ruleset_entrypoint_version](#) in the Terraform registry.

Overriding managed rulesets

To customize the behavior of a managed ruleset, override it at deployment. When you override a ruleset, you specify changes to be executed in addition to the default configuration.

You can override a managed ruleset at the following levels:

- Ruleset overrides apply to all rules in the executed ruleset.
- Tag overrides apply to all rules with a specific tag. For example, use a tag override to customize the CIS Managed Ruleset so all rules with the wordpress tag are set to Block. If multiple tags have overrides and if a rule has more than one of these tags, the tag overrides order determines the behavior. For rules tagged with multiple overridden tags, the last tag’s overrides apply.
- Rule overrides apply to specific rules in a managed ruleset, referenced by their Rule ID.



Important: Specific overrides take precedence over more general ones, and rule overrides take precedence over tag overrides, which take precedence over ruleset overrides.

Ruleset overrides and tag overrides apply to both existing and future rules in the managed ruleset. If you want to override existing rules only, you must use rule overrides.

Overriding workflow (API)

To apply an override for a managed ruleset, take the following steps.

1. List the rulesets available by using the list rulesets operation.
2. List the rules for that ruleset by using the get ruleset operation.
3. Call the update ruleset operation on your phase entry point.
4. Specify the overrides in the `action_parameters` of the rule that executes your managed ruleset.


```

"action_parameters": {
  "id": "<RULESET_ID>",
  "overrides": {
    // ruleset overrides
    "property-to-modify": "value",
    "property-to-modify": "value",
    // tag overrides
    "categories": [
      {
        "category": "<TAG_NAME>",
        "property-to-modify": "value",
        "property-to-modify": "value"
      }
    ],
    // rule overrides
    "rules": [
      {
        "id": "<RULE_ID>",
        "property-to-modify": "value",
        "property-to-modify": "value"
      }
    ]
  }
}

```

You can override the following rule properties.

- "action"
- "enabled"

Some managed rulesets can have extra override requirements, or they might override other rule properties.

It is not effective to enable all the rules in a managed ruleset at the instance level by using an override. This change can affect all the zones in your instance. Some rules are disabled by default because they eventually affect legitimate traffic. Do not enable these rules across zones without previous consideration.

Overriding managed rulesets in the console

To override a managed ruleset in the console, follow these steps:

1. In the CIS console, navigate to the **Security** section.
2. Select the **WAF** tab.
3. Click the name of the ruleset that you want override.
4. From the Configure deployment side panel, click **Browse rules**.
5. In the Override CIS Managed Ruleset rules side panel, you can:
 - Select an action for any rule that you want to override.
 - Select the **Status** toggle to enable or disable a rule.
6. After you have reconfigured the ruleset rules, click **Continue**.
7. Verify the list of rules to override, then click **Save**.



Tip: Click **Reset to default** to restore the original settings.

Listing managed rulesets from the CLI

To list all zone rulesets from the CLI, run the following command:

```
$ ibmcloud cis managed-waf rulesets DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the domain.

-i, --instance value

The instance name or ID.

--output value

Specifies the output format; only JSON is supported.

Listing rules under a zone ruleset from the CLI

To list the rules under a zone ruleset from the CLI, run the following command:

```
$ ibmcloud cis managed-waf ruleset DNS_DOMAIN_ID RULESET_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the domain.

RULESET_ID

The ID of the ruleset for the rules to be listed.

-i, --instance value

The instance name or ID.

--output value

Specifies the output format; only JSON is supported.

Overriding managed rulesets from the CLI

To override a managed WAF ruleset from the CLI, run the following command:

```
$ ibmcloud cis managed-waf deployment-add-ruleset DNS_DOMAIN_ID RULESET_ID [--match EXPRESSION] [--enabled true|false] [--override-action ACTION] [--override-status STATUS] [--paranoia-level LEVEL] [--override-rules RULE] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

the ID of the domain.

--match value

The conditions that must be matched for the rule to run. See [Using fields, functions, and expressions](#) for a list of values to match.

--enabled value

Indicates whether the rule is active. Defaults to "true".

--override-action value

The ruleset action of any overrides. Valid values are "managed_challenge", "block", "js_challenge", "log", "challenge".

--paranoia-level value

The OWASP paranoia level. Valid values are "PL1", "PL2", "PL3", "PL4" and it's only available for **CIS OWASP Core Ruleset**.

--override-rules value

The rules options of the overrides. For example, **--override-rules rule=RULE_ID,action=ACTION,enabled=STATUS**.

-i, --instance value

The instance name or ID.

`--output value`

Specifies the output format; only JSON is supported.

Listing zone rulesets from the API

To list all zone rulesets from the API, run the following command:

```
curl -X GET \
  https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets \
  -H 'content-type: application/json' \
  -H 'accept: application/json' \
  -H 'x-auth-user-token: Bearer xxxxxx'
```

Listing all rules for a ruleset from the API

To list all rules for a specific ruleset, run the following command:

```
curl -X GET \
  https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets/$RULESET_ID \
  -H 'content-type: application/json' \
  -H 'accept: application/json' \
  -H 'x-auth-user-token: Bearer xxxxxx'
```

Where:

- `$RULESET_ID` is the ID of the managed ruleset which the rules are listed for.

Overriding an entry point ruleset from the API

To update the entry point ruleset from the API with an override, run the following command:

```
curl -X PUT \
  https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets/phases/$RULESET_PHASE/entrypoint \
  -H 'content-type: application/json' \
  -H 'accept: application/json' \
  -H 'x-auth-user-token: Bearer xxxxxx' \
  -d '{"description":"Deploy managed ruleset, enabling a specific rule with log action","rules":[{"action":"execute","expression":"true","action_parameters":{"id":"<MANAGED_RULESET_ID>"},"overrides":{"rules":[{"id":"<RULE_ID>","enabled":true,"action":"log"}]}]}'
```

Where:

- `$RULESET_PHASE` is the ruleset phase that is deployed to. Use `http_request_firewall_managed` to override managed WAF rulesets.
- `-d` is the object of attributes that are required to create the ruleset.
 - `description` defines your own summary of what a ruleset is accomplishing.
 - `rules` is the array of rules to deploy with the ruleset.
 - `action` is the action for the rule to take. See [WAF ruleset actions](#) for a description of available actions.
 - `action_parameters` is the object for defining what the action operates on.
 - `id` is the ID of the ruleset to execute. This ID is retrieved from the list zone rulesets operation.
 - `overrides` is the object of overrides to set upon the selected ruleset.
 - `rules` is the list of rules that are overridden with the selected properties.
 - `id` is the ID of the rule to override. This ID is retrieved from the list zone ruleset rules operation.
 - `enabled` overwrites even when the rule is enabled.
 - `action` specifies the overridden action that the rule takes.
 - `expression` is the condition under which the rule runs. Using "true" means that this rule always runs.
 - `description` defines your own summary of what the rule is accomplishing.

Listing managed rulesets with Terraform

The following example lists all managed rulesets using Terraform:

```
$ data "ibm_cis_rulesets" "tests" {
  cis_id   = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
}
```

For more information about the arguments and attributes, see [ibm_cis_rulesets](#) in the Terraform registry.

Listing all rules of a managed ruleset with Terraform

The following example lists all rules of a managed ruleset using Terraform:

```
$ resource "ibm_cis_ruleset" "config" {
  cis_id   = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  ruleset_id = "943c5da120114ea5831dc1edf8b6f769"
}
```

For more information about the arguments and attributes, see [ibm_cis_ruleset](#) in the Terraform registry.

Overriding a rule with Terraform

This example shows how to deploy the CIS managed ruleset with various overrides. First, it enables and blocks traffic for all rules. Then, it enables and blocks traffic for a specific rule. Finally, it enables and blocks traffic for all rules in the `wordpress` category. Essentially, this example illustrates the different methods for overriding deployed rules (global, specific, by category).

```
$ resource "ibm_cis_ruleset_entrypoint_version" "test" {
  cis_id   = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  phase = "http_request_firewall_managed"
  rulesets {
    description = "Entrypoint ruleset for managed ruleset"
    rules {
      action = "execute"
      description = "Deploy CIS managed ruleset"
      enabled = true
      expression = "true"
      action_parameters {
        id = "efb7b8c949ac4650a09736fc376e9aee"
      }
      overrides {
        action = "block"
        enabled = true
        override_rules {
          rule_id = "var.overriden_rule.id"
          enabled = true
          action = "block"
        }
      }
      categories {
        category = "wordpress"
        enabled = true
        action = "block"
      }
    }
  }
}
```

The following example shows how to create a WAF entry point and override a rule with Terraform:

```
resource "ibm_cis_ruleset_entrypoint_version" "waf_config" {
  cis_id   = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  phase = "http_request_firewall_managed"
  rulesets {
    description = "Entry Point rulesets"
  }
}
```

```

}
lifecycle {
  ignore_changes = [
    rulesets
  ]
}
}

data "ibm_cis_ruleset_entrypoint_versions" "test"{
  cis_id   = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  phase = "http_request_firewall_managed"
  depends_on = [
    ibm_cis_ruleset_entrypoint_version.waf_config
  ]
}

resource ibm_cis_ruleset_rule "rule1" {

  cis_id   = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  ruleset_id = data.ibm_cis_ruleset_entrypoint_versions.test.rulesets[0].ruleset_id

  rule {
    action = "execute"
    description = "OWASP Core RuleSet"
    enabled = true
    expression = "true"
    action_parameters {
      id = "4814384a9e5d4991b9815dcfc25d2f1f"
      overrides {
        categories {
          category = "paranoia-level-2"
          enabled = false
        }
        categories {
          category = "paranoia-level-3"
          enabled = false
        }
        categories {
          category = "paranoia-level-4"
          enabled = false
        }
      }
      override_rules {
        rule_id   = "6179ae15870a4bb7b2d480d4843b323c"
        action     = "block"
        score_threshold = 60
        enabled = true
      }
      override_rules {
        rule_id   = "8ac8bc2a661e475d940980f9317f28e1"
        enabled   = true
      }
    }
  }
}

resource ibm_cis_ruleset_rule "rule2" {

  cis_id   = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  ruleset_id = data.ibm_cis_ruleset_entrypoint_versions.test.rulesets[0].ruleset_id

  rule {
    action = "execute"
    description = "CIS exposed credential ruleset"
    enabled = true
    expression = "true"
    action_parameters {
      id = "c2e184081120413c86c3ab7e14069605"

```



```

      overrides {
        override_rules {
          rule_id = "53f38cd8974a4cd3bc9a8a64fc731fb0"
          enabled = true
          action = "log"
        }
      }
    }
  }
}

resource ibm_cis_ruleset_rule "rule3" {

  cis_id    = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  ruleset_id = data.ibm_cis_ruleset_entrypoint_versions.test.rulesets[0].ruleset_id

  rule {
    action = "execute"
    description = "CIS Managed ruleset"
    enabled = true
    expression = "true"
    action_parameters {
      id = "efb7b8c949ac4650a09736fc376e9aee"
      overrides {
        override_rules {
          rule_id = "5de7edfa648c4d6891dc3e7f84534ffa"
          enabled = true
          action = "block"
        }
      }
    }
  }
}

```

For more information about the arguments and attributes, see [ibm_cis_ruleset_entrypoint_version](#) in the Terraform registry.

Handling false positives in managed rules

If you encounter a false positive caused by a managed rule, take one of the following steps.

- **Add an exception**: Exceptions allow you to skip the execution of WAF managed rulesets or some of their rules for certain requests.
- **Adjust the OWASP managed ruleset**: A request blocked by rule ID `6179ae15870a4bb7b2d480d4843b323c` and description `949110: Inbound Anomaly Score Exceeded` refers to the OWASP Core Ruleset. To resolve the issue, [configure the OWASP managed ruleset](#).
- **Disable the corresponding managed rules**: Create an override to disable specific rules. This can avoid false positives, but also reduces the overall site security. For more information, see [Overriding managed rulesets](#).

If you contact Support to verify whether a WAF managed rule triggers as expected, [provide a HAR file](#) captured while sending the specific request that you want to verify.

Recommendations

- If one specific rule causes false positives, disable that specific rule and not the entire ruleset.
- For false positives within the administrator area of your website, add an exception that disables a managed rule for the admin section of your site resources by using an expression similar to the following example.

```
$ http.host eq "example.com" and starts_with(http.request.uri.path, "/admin")
```

Available managed rulesets

CIS rulesets

Select **View CIS rules** to reveal the rulesets of this package. rulesets are as follows:

- **Drupal** - Enable this ruleset only if the Drupal CMS is used for this domain.
- **Flash** [\[1\]](#) - Enable this ruleset only if Adobe Flash content is used for this domain.

- **Joomla** [\[2\]](#) - Enable this ruleset only if the Joomla CMS is used for this domain.
- **Magento** [\[3\]](#) - Enable this ruleset only if the Magento CMS is used for this domain.
- **Miscellaneous** - Contains rules to deal with known malicious traffic, or fix flaws in specific web applications.
- **PHP** [\[4\]](#) - Enable this ruleset if PHP is used for this domain.
- **Plone** [\[5\]](#) - Enable this ruleset only if the Plone CMS is used for this domain.
- **Specials** - Contains a number of rules that were created to deal with specific attack types.
- **WHMCS** [\[6\]](#) - Enable this ruleset only if WHMCS is used for this domain.
- **Wordpress** - Enable this ruleset only if the WordPress CMS is used for this domain.

 **Tip:** The **Specials** ruleset contains rules appropriate for virtually all applications and websites on the internet. This ruleset is the core of the WAF security offering, with rules that target common attacks like SQLi, XSS, and LFI. Always enable Specials.

Enable only the rulesets that correspond to your technology stack. For instance, if you use Wordpress, but no other technologies, enable only the Specials and Wordpress rulesets. Avoid enabling rulesets that are not relevant to your tech stack.

Select any of the specific rulesets to see further details about each of the rules included.

Use the CIS ruleset to perform the following actions on each rule:

- **Disable** turns off the rule.
- **Log** logs the event and does not block or challenge the visitor. You can still decide to set to **Block** or **Challenge** after you review the logs.
- **Block** blocks the request entirely, with no option to bypass it for that request.
- **Challenge** displays a challenge (CAPTCHA) page that must be completed before the request in question is allowed access.

You might notice that the names of the rules don't reveal exactly how they work and that they are mostly a general summary of their function. This is deliberate. For security purposes, CIS does not reveal the code (or other exact information) used to filter traffic. Doing so prevents malicious actors from reverse-engineering it to bypass our defenses.

-
1. This ruleset contains more rules that complement the technology-specific protections provided by similar rules in the OWASP ruleset. [↪](#)
 2. This ruleset contains more rules that complement the technology-specific protections provided by similar rules in the OWASP ruleset. [↪](#)
 3. This ruleset contains more rules that complement the technology-specific protections provided by similar rules in the OWASP ruleset. [↪](#)
 4. This ruleset contains more rules that complement the technology-specific protections provided by similar rules in the OWASP ruleset. [↪](#)
 5. This ruleset contains more rules that complement the technology-specific protections provided by similar rules in the OWASP ruleset. [↪](#)
 6. This ruleset contains more rules that complement the technology-specific protections provided by similar rules in the OWASP ruleset. [↪](#)

OWASP ruleset

The OWASP Core Ruleset for WAF contains generic attack detection rules. The OWASP rules protect against many common attack categories, including SQL injection, cross-site scripting, and local file inclusion. CIS provides, but does not curate these rules.

OWASP is an industry standard that provides a good security baseline. For more information, see:

- [OWASP on GitHub](#)
- [OWASP Core Ruleset](#)
- [OWASP Core Ruleset Documentation](#)
- [OWASP Core Ruleset Changelog](#)

Managing OWASP

Depending on your WAF handling in CIS, you might be on a different version of OWASP. The OWASP v2.x uses sensitivity thresholds, and OWASP v3.x uses paranoia levels. Zones created on or after 8 May 2024 will use OWASP v3.x. Zones created before that date use OWASP v2.x and will be migrated by 14 February 2025, unless you manually migrate earlier.

Understanding the OWASP package

The OWASP ModSecurity Core Ruleset assigns a score to each request based on how many OWASP rules trigger. The ruleset in CIS is based off

Paranoia level

The paranoia level settings are part of the core ruleset. The paranoia level (PL) helps to define how aggressive the core ruleset is.

Paranoia level	Description
PL 1	Provides a set of rules that rarely trigger a false alarm, though false alarms can happen depending on the local setup.
PL 2	Provides additional rules that detect more attacks, but the additional rules might also trigger new false alarms over legitimate HTTP requests.
PL 3	Provides more rules for certain specialized attacks. The risk of false alarms increases.
PL 4	Provides rules that are so aggressive they detect almost every possible attack. This paranoia level flags a lot of legitimate traffic as malicious.

Paranoia levels

Sensitivity threshold

A request can trigger a set of OWASP rules that have a high to low severity score associated with them. The final score is calculated based on all the rules triggered. After calculating the final score, CIS compares it to the sensitivity threshold selected in the beginning, and then either blocks, challenges, or logs the request based on the option selected.

Set OWASP sensitivity to `low` initially, then review for false positives before increasing the sensitivity. If you set it to `high`, check the logs on CIS, and fine-tune the OWASP ruleset to work for your application.

Keep in mind that you can only toggle OWASP rules on or off, unlike rules in the CIS rulesets, which can be set to **Disable**, **Simulate**, **Challenge**, or **Block**.

The sensitivity score required to trigger the WAF for a specific sensitivity is as follows:

Sensitivity score	Trigger threshold
Low	60 and higher
Medium	40 and higher
High	25 and higher

Sensitivity and triggers

 **Tip:** With a high sensitivity, large file uploads trigger the WAF. (2.x only)

For Ajax requests, the following scores are applied instead:

Sensitivity score	Trigger threshold
Low	120 and higher
Medium	80 and higher
High	65 and higher

Ajax request sensitivity and triggers

Review the (security) events log to see the final score, as well as the individual triggered rules.

Managing OWASP packages

The OWASP ModSecurity Core Ruleset contains several rules from the OWASP project. CIS does not write or curate OWASP rules. Click a ruleset name under **Group** to reveal the rule descriptions. Unlike the CIS managed ruleset, specific OWASP rules are either turned on or off.

To manage OWASP thresholds, set the paranoia level in the **Package: OWASP ModSecurity Core Ruleset** section. Setting the paranoia level to **P1** disables the entire OWASP package, including all its rules. Determining the appropriate paranoia level depends on your business industry and operations. For instance, a P1 setting is appropriate for large file uploads.

 **Important:** The rule ID 981176 is displayed when a request is blocked by OWASP. Also, some OWASP rules listed in the activity log do not appear in the list of rules under Package: OWASP ModSecurity Core Ruleset because disabling those rules is not recommended.

In sampled logs and security events, the rule associated with requests mitigated by the OWASP Core Ruleset is the last rule in this managed ruleset: **949110: Inbound Anomaly Score Exceeded**. In the CIS dashboard, users can see the scores of individual rules contributing to the final request threat score by expanding the **OWASP score contributions** subsection. Similarly, a list of rule IDs can be seen in the **score_rules** field of the GraphQL API response.

Exposed Credentials Check ruleset

The Exposed Credentials Check ruleset (part of Managed Rules) is a set of pre-configured rules for content management system applications that check against a public database of stolen credentials. When enabled in a rule, exposed credentials checking occurs when the rule expression evaluates to **true**.

The WAF checks the username and password pair in the request against a public database of known stolen credentials. When both the rule expression and the exposed credentials check are **true**, the rule match triggers the action that is configured in the rule.

The WAF can perform one of the following actions when it detects exposed credentials.

- **Exposed-Credential-Check Header:** Adds a HTTP header to HTTP requests with exposed credentials. Your application at the origin can then force a password reset, start a two-factor authentication process, or any other action. The **Exposed-Credential-Check** HTTP header is added with a value of **1**.
- **Managed Challenge:** Helps reduce the lifetimes of human time spent solving CAPTCHAs across the internet. Depending on the characteristics of a request, CIS dynamically chooses the appropriate type of challenge.
- **Block:** Blocks HTTP requests containing exposed credentials.
- **JS Challenge:** Presents a noninteractive challenge to the clients who are making HTTP requests with exposed credentials.
- **Log:** (Enterprise only) Logs requests with exposed credentials in the logs. Logging is recommended for validating a rule before you commit to a more severe action.
- **Interactive Challenge:** Presents an interactive challenge to the clients who are making HTTP requests with exposed credentials.

The default action for the rules in the Exposed Credentials Check managed ruleset is **Exposed-Credential-Check Header** (named **rewrite** in the API).

The best practice is to use only the **Exposed-Credential-Check Header** (**rewrite** in the API) and **Log** (**log**).

Checking for leaked credentials

Enterprise accounts can configure IBM Cloud Internet Services to check incoming authentication requests for username and password pairs that are known to have been compromised. You can define expressions that identify the username and password fields in your application's login requests.

Unlike the [Exposed Credentials Check ruleset](#), leaked credential checks allow you to define which fields in your own login requests are checked.

Before you begin

Install the CIS CLI plug-in, set the context instance, and identify the DNS domain that receives your application's login requests. For more information, see the [CIS CLI reference](#).

The examples use ruleset expressions to extract values from a JSON request body. Adjust the expressions to match the structure of your application's authentication requests.

Checking and changing the status

Checking the status

Check whether leaked credential checks are enabled for a DNS domain:

```
$ ibmcloud cis leaked-credential-checks status-get DNS_DOMAIN_ID
```

To return the result as JSON, specify `--output json`.

Enabling or disabling checks

Enable leaked credential checks:

```
$ ibmcloud cis leaked-credential-checks status-update DNS_DOMAIN_ID --enabled true
```

To disable leaked credential checks, set `--enabled false`.

You can also provide the configuration as a JSON file:

```
{
  "enabled": true
}
```

```
$ ibmcloud cis leaked-credential-checks status-update DNS_DOMAIN_ID --json @status.json
```

Managing custom detections

A custom detection contains ruleset expressions that locate the username and password in an authentication request.

Creating a detection

Create a custom detection by specifying the username and password expressions:

```
$ ibmcloud cis leaked-credential-checks detection-create DNS_DOMAIN_ID --username 'lookup_json_string(http.request.body.raw, "username")' --password 'lookup_json_string(http.request.body.raw, "password")'
```

Both `--username` and `--password` are required when you create a detection by using command options.

You can also provide the detection as a JSON file:

```
{
  "username": "lookup_json_string(http.request.body.raw, \"username\")",
  "password": "lookup_json_string(http.request.body.raw, \"password\")"
}
```

```
$ ibmcloud cis leaked-credential-checks detection-create DNS_DOMAIN_ID --json @detection.json --output json
```

Save the returned detection ID. You need it to retrieve, update, or delete the detection.

Listing and retrieving detections

List all custom detections for a DNS domain:

```
$ ibmcloud cis leaked-credential-checks detection-list DNS_DOMAIN_ID
```

Retrieve a specific detection:

```
$ ibmcloud cis leaked-credential-checks detection-get DNS_DOMAIN_ID DETECTION_ID
```

Updating a detection

Update one or both expressions in an existing detection. The following example changes the username expression:

```
$ ibmcloud cis leaked-credential-checks detection-update DNS_DOMAIN_ID DETECTION_ID --username 'lookup_json_string(http.request.body.raw, "email")'
```

You can also provide the updated fields as a JSON file:

```
$ ibmcloud cis leaked-credential-checks detection-update DNS_DOMAIN_ID DETECTION_ID --json @detection.json
```

Deleting a detection

Delete a custom detection:

```
$ ibmcloud cis leaked-credential-checks detection-delete DNS_DOMAIN_ID DETECTION_ID
```

The command prompts you to confirm the deletion. To delete the detection without a confirmation prompt, specify `--force`.

OWASP evaluation example

The following example calculates the OWASP request threat score for an incoming request.

The OWASP managed ruleset configuration is as follows:

- OWASP Anomaly Score Threshold: `High - 25 and higher`
- OWASP Paranoia Level: `PL3`
- OWASP Action: **Managed Challenge**

This table shows the progress of the OWASP ruleset evaluation:

Rule ID	Paranoia level	Rule matched?	Rule score	Cumulative threat score
–	–	–	–	0
...1813a269	PL3	Yes	+5	5
...ccc02be6	PL3	No	–	5
...96bfe867	PL2	Yes	+5	10
...48b74690	PL1	Yes	+5	15
...3297003f	PL2	Yes	+3	18
...317f28e1	PL1	No	–	18
...682bb405	PL2	Yes	+5	23
...56bb8946	PL2	No	–	23
...e5f94216	PL3	Yes	+3	26
(...)	(...)	(...)	(...)	(...)
...f3b37cb1	PL4	(not evaluated)	–	26

Progress of the OWASP ruleset evaluation

Final request threat score: `26`

Since `26` $\geq$ `25` — that is, the threat score is greater than the configured score threshold — CIS applies the configured action (**Managed Challenge**). If you configured a score threshold of `Medium - 40 and higher`, CIS wouldn't apply the action, since the request threat score would be lower than the score threshold (`26` $<$ `40`).

[Sampled logs in Security Events](#) would display the following details for the example incoming request handled by the OWASP Core Ruleset.

In sampled logs, the rule associated with requests mitigated by the CIS OWASP Core Ruleset is the last rule in this managed ruleset: `949110: Inbound Anomaly Score Exceeded`, with rule ID `6179ae15870a4bb7b2d480d4843b323c`. To get the scores of individual rules contributing to the final request threat score, expand **Additional logs** in the event details.

Working with rate-limiting rules

Migrating to new rate-limiting rules

The CIS WAF provides both automatic protection from vulnerabilities and the flexibility to create WAF custom rules.

CIS recommends that you create new ruleset-based, rate-limiting rules to replace any existing rate-limiting rules that you might have configured in the previous rate-limiting rule handling.

The old and new implementation of rate-limiting rules have separate rules lists because the two implementations do not offer the same set of features. Recreate your rate limiting configuration (in the previous version) by using the new rate-limiting rules with the [firewall rules](#) or [WAF custom rules](#), combining multiple parameters of the HTTP request.

Separate counting and mitigation expressions

In the new version of Rate Limiting, counting and mitigation expressions are separate. The counting expression defines which requests are used to compute the rate. The mitigation expression defines which requests are mitigated after the threshold is reached. Using these separate expressions, you can track the rate of requests on a specific path such as `/login` and, when an IP exceeds the threshold, block every request from the same IP addressed at your domain.

Relevant changes for API users

The new rate-limiting rules are based on the [Ruleset Engine rules language](#). To configure rate-limiting rules with the API, you must use the [Rulesets API](#). The Rulesets API is used on all recent CIS security products to provide a uniform user experience when interacting with the CIS API.

⊖ The [previous Rate Limiting API](#) is being deprecated. Migrate your API calls to the new [Rulesets API](#).

Manually migrate your rate-limiting rules by recreating them with the Rulesets API using the `http_ratelimit` phase. This lets you take full advantage of the new features and refine your rate-limiting logic during the transition.

After 30 July 2025, you can no longer create or modify rules using the legacy rate-limiting APIs. Any existing legacy rate-limiting rules are automatically migrated to the new ruleset-based framework after this date.

Configuring rate limiting

⊖ **Deprecated:** The *previous* version of rate-limiting rules is deprecated. Active rules from the previous version will no longer function.

Rate limiting (Enterprise plan only) protects against denial-of-service attacks, brute-force login attempts, and other types of abusive behavior targeting the application layer.

Creating a custom rate limiting rule in the console

To create a custom rate limiting rule, complete the following steps:

1. In the Cloud Internet Services (CIS) console, navigate to the **Security > Rate limiting**, and then click **Create rule**.
2. Enter a name for the rule.
3. Define the match condition for the rule:
 - a. Select a request field (for example, URI path, HTTP method, or header).
 - b. Select an operator (for example, equals, contains, or matches).
 - c. Enter or select a value.
 - d. Optional: Use the Expression Builder to create a custom expression for more complex matching logic.
 - e. Optional: Combine multiple conditions by using **And** and **Or** operators to create compound rules.
 - f. Optional: Disable **Cache status** if you want the rate-limiting rule to consider only requests that reach the origin server. By default, cached requests are included in the rate calculation.
4. In **With the same characteristics**, select the characteristic that Cloud Internet Services (CIS) uses to identify matching requests. Common options include IP address, IP with NAT support, session, headers, cookie, query string, or JA3 fingerprint.
5. Optional: To define a custom counting expression:
 - a. Enable **Use custom counting expression**.

- b. Enter the counting expression. By default, the counting expression matches the rule expression. A custom counting expression allows you to count requests differently from how they are matched.
6. In **When rate exceeds**, configure the request threshold:
 - a. Enter the maximum number of requests allowed.
 - b. Select the time period that Cloud Internet Services (CIS) uses to evaluate the request rate (for example, 10 seconds, 1 minute, or 1 hour).
7. In **Then take action**, select the action to apply when the threshold is exceeded. Options include **Block** (reject requests), **Challenge** (present a CAPTCHA), **JS Challenge** (present a JavaScript challenge), **Log** (log without action), or **Managed Challenge** (present an appropriate challenge based on client characteristics). For example, selecting **Block** causes Cloud Internet Services (CIS) to reject requests that exceed the configured rate limit.
8. In **With the following behavior**, select the mitigation behavior:
 - a. Select a mitigation timeout to block requests for a specified duration after the threshold is exceeded (for example, 1 minute, 10 minutes, or 1 hour).
 - b. Enterprise customers with the rate limiting add-on can select **Throttle requests over the maximum configured rate** to limit requests instead of applying the configured action.
9. In **Place at**, select the rule order. Rules are evaluated in order, so place more specific rules before general rules.
10. Click **Deploy**.

The rate limiting rule is created and deployed. The rule takes effect immediately and begins monitoring traffic according to your configuration.

Updating a rate limiting rule in the console

To update a rate-limiting rule, complete the following steps:

1. In the Cloud Internet Services (CIS) console, navigate to **Security > Rate limiting**.
2. In the rate limiting ruleset table, locate the rule that you want to modify.
3. Click Actions menu on the right of the row and then select **Edit**.
4. Modify the rule settings as needed.
5. Click **Save** to update the rule.

Deleting a rate limiting rule in the console

To delete a rate limiting rule in the console, follow these steps:

1. In the Cloud Internet Services (CIS) console, navigate to **Security > Rate limiting**.
2. In the rate limiting rules table, locate the rule that you want to delete.
3. Click the Actions menu for the rule, and then select **Delete**.
4. Review the confirmation message and click **Delete** to confirm.

Configuring the response

Select from the actions listed, and specify the timeout period. In this case, the timeout refers to the ban period that the action takes place. A 60-second timeout means that the action is applied for 60 seconds.

Action	Description
Block	Issues a 429 error when the threshold is exceeded
Challenge	User must pass a Google re-Captcha Challenge before proceeding. If successful, the request is accepted. Otherwise, the request gets blocked.
JS Challenge	The user must pass a JavaScript Challenge before proceeding. If successful, the request is accepted. Otherwise, the request gets blocked.
Simulate	You can use this option to test your rule before applying any of the other options in your live environment.

Actions for rate limiting

Getting the rate-limiting rule endpoint for the API

All rate-limiting rule API operations require a `RULESET_ID` of the endpoint ruleset for the rate-limiting rules phase. This endpoint ruleset might exist or might need to be created if it does not exist.

Follow these steps to get the rate-limiting rule endpoint ruleset:

1. Set up your API environment with the correct variables.
2. Store the following values in variables to be used in the API command:

`CRN` : The full URL-encoded Cloud Resource Name (CRN) of the service instance.

`ZONE_ID` : The domain ID.

3. When all variables are initiated, get the endpoint ruleset:

```
$ curl -X GET "https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets/phases/http_ratelimit/endpoint" \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json"
```

The ruleset ID is in the response of the successful request. If the preceding call returns a 404 Not Found response, use the following API to create the endpoint ruleset for the rate-limiting rule phase:

```
$ curl -x POST https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json" \
--data '{
  "name": "Zone-level phase endpoint",
  "kind": "zone",
  "description": "Rate-limiting rule endpoint ruleset.",
  "phase": "http_ratelimit"
}'
```

Creating a rate-limiting rule with the API

Follow these steps to create a rate-limiting rule with the API:

1. Set up your API environment with the correct variables.
2. Store the following values in variables to be used in the API command:

`CRN` : The full URL-encoded Cloud Resource Name (CRN) of the service instance.

`ZONE_ID` : The domain ID.

`RULESET_ID` : The ID of the rate-limiting rule endpoint ruleset.

3. When all variables are initiated, create the rate-limiting rule:

```
$ curl -X POST "https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets/$RULESET_ID/rules" \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json" \
--data '{
  "description": "My rate limiting rule",
  "expression": "(http.request.uri.path matches \"/api/")",
  "action": "block",
  "ratelimit": {
    "characteristics": [
      "cf.colo.id",
      "ip.src"
    ],
    "period": 60,
    "requests_per_period": 100,
    "mitigation_timeout": 600
  }
}'
```

Updating a rate-limiting rule with the API

Follow these steps to update an existing rate-limiting rule with the API:

1. Set up your API environment with the correct variables.
2. Store the following values in variables to be used in the API command:

CRN : The full URL-encoded Cloud Resource Name (CRN) of the service instance.

ZONE_ID : The domain ID.

RULESET_ID : The ID of the rate-limiting rule endpoint ruleset.

RULE_ID : The ID of the rate-limiting rule to be modified.

3. When all variables are initiated, update the rate-limiting rule:

```
$ curl -X PATCH "https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets/$RULESET_ID/rules/$RULE_ID" \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json" \
--data '{
  "enabled": true,
  "description": "rate limit IPs for API"
}'
```

Deleting a rate-limiting rule with the API

Follow these steps to delete an existing rate-limiting rule with the API:

1. Set up your API environment with the correct variables.
2. Store the following values in variables to be used in the API command:

CRN : The full URL-encoded Cloud Resource Name (CRN) of the service instance.

ZONE_ID : The domain ID.

RULESET_ID : The ID of the rate-limiting rule endpoint ruleset.

RULE_ID : The ID of the rate-limiting rule to be modified.

3. When all variables are initiated, delete the rate-limiting rule:

```
$ curl -X DELETE "https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/rulesets/$RULESET_ID/rules/$RULE_ID" \
--header "X-Auth-User-Token: Bearer <API_TOKEN>" \
--header "Content-Type: application/json"
```

Verifying the rate-limiting rules and response consistency by using HTTP status

When you apply rate-limiting rules to a web address or service, it's important to confirm that the rules are enforced correctly. A simple test confirms that the system responds with the following appropriate HTTP status codes:

- **200 OK** or **404 Not Found** for allowed requests
- **429 Too Many Requests** when rate limits are exceeded.

To verify the rate-limiting rules and response consistency, run the following command:

```
$ for i in {1..N}; do curl -s -o /dev/null -w "%{http_code}\n" <your-target-url>; done
```

Command options

N

Number of requests you want to send.

your-target-url

URL of the service or endpoint you want to test.

This command provides output to the HTTP status code for each request and allows you to observe when the rate limit threshold is reached.

Creating a rate limiting rule from the CLI

To create a rate limiting rule from the CLI, follow these steps:

1. [Set up your CLI environment.](#)
2. Log in to your account from the CLI. After you enter the password, the system prompts for the account and region that you want to use:

```
$ ibmcloud login --sso
```

3. Run the following command to create a rate-limiting rule:

```
$ ibmcloud cis ratelimit-rule-create DNS_DOMAIN_ID --url URL [--description DESCRIPTION] [--threshold NUM] [--period SECONDS] [...]
```

You can also create a rate-limiting rule by providing a JSON file or a JSON string directly:

```
$ ibmcloud cis ratelimit-rule-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--json

The JSON file or JSON string that is used to describe a rate limiting rule.

- The required fields in JSON data are **match**, **threshold**, **period** and **action** :
 - **match** : Determines which traffic the rate limiting rule counts toward the threshold.
 - **request** : Matches HTTP requests.
 - **methods** : HTTP Methods, can be a subset **[POST,PUT]** or all **[_ALL_]**. This field is not required to create a rate limit rule. Valid values are **GET**, **POST**, **PUT**, **DELETE**, **PATCH**, **HEAD**, **_ALL_**.
 - **schemes** : HTTP Schemes, can be one **[HTTPS]**, both **[HTTP,HTTPS]** or all **[_ALL_]**. This field is not required.
 - **url** : The URL pattern to match composed of the host and path, for instance, **example.org/path**. Wildcards are expanded to match applicable traffic, query strings are not matched. Use ***** for all traffic to your zone. The max length is **1024**.
 - **response** : Matches HTTP responses before they are returned to the client. If this is defined, then the entire counting of traffic occurs at this stage.
 - **status** : HTTP Status codes, can be one **[403]**, many **[401,403]** or indicate all by not providing this value. This field is not required. The min value is **100** and the max value is **999**.
 - **headers** : Array of response headers to match. If a response does not meet the header criteria, then the request is not counted towards the rate limiting rule. The header matching criteria includes the following properties.
 - **name** : The name of the response header to match.
 - **op** : The operator when matching, eq means equals, ne means not equals. Valid values are **eq** and **ne**.
 - **value** : The value of the header, which is exactly matched.
 - **threshold** : The threshold that triggers the rate limit mitigations, which are combined with a period. For example, the threshold per period. The min value is **2** and the max value is **1000000**.
 - **period** : The time, in seconds, to count matching traffic. If the count exceeds the threshold within this period, the action is performed. The min value is **10** and the max value is **86400**.
 - **action** : The action performed when the threshold of matched traffic within the period defined is exceeded.
 - **mode** : The type of action performed. Valid values are: **simulate**, **ban**, **challenge**, **js_challenge**.
 - **timeout** : The time in seconds, as an integer to perform the mitigation action. The timeout can be the same or greater than the period. This field is valid only when the mode is **simulate** or **ban**. The min value is **10** and the max value is **86400**.
 - **response** : Custom content-type and body to return. This overrides the custom error for the zone. This field is not required. Omission results in the default HTML error page. This field is valid only when mode is **simulate** or **ban**.
 - **content_type** : The content-type of the body, which must be one of the following: **text/plain**, **text/xml**, **application/json**.
 - **body** : The body to return. The content here must conform to the **content_type**. The max length is **10240**.

- The optional fields are `id`, `disabled`, `description`, `correlate` and `bypass` :
 - `id` : Identifier of the rate limiting rule.
 - `disabled` : Whether this rate limiting rule is currently disabled.
 - `description` : A note that you can use to describe the reason for a rate limiting rule.
 - `correlate` : Whether to enable NAT-based rate limiting.
 - `by` : Valid value is `nat`.
 - `bypass` : Criteria that allow the rate limit to be bypassed. For example, to express that you shouldn't apply a rate limit to a set of URLs.
 - `name` : Valid value is `url`.
 - `value` : The url to bypass.

Sample JSON data:

```
{
  "id": "92f17202ed8bd63d69a66b86a49a8f6b",
  "disabled": false,
  "description": "Prevent multiple login failures to mitigate brute force attacks",
  "bypass": [
    {
      "name": "url",
      "value": "api.example.com/*"
    }
  ],
  "threshold": 60,
  "period": 900,
  "correlate": {
    "by": "nat"
  },
  "action": [
    {
      "mode": "simulate",
      "timeout": 86400,
      "response": {
        "content_type": "text/plain",
        "body": "<error>This request has been rate-limited.</error>"
      }
    }
  ],
  "match": {
    "request": {
      "methods": [
        "GET"
      ],
      "schemes": [
        "HTTP",
        "HTTPS"
      ],
      "url": "/*.example.org/path*"
    },
    "response": {
      "status": [
        403, 401
      ],
      "headers": [
        {
          "name": "Cf-Cache-Status",
          "op": "eq",
          "value": "HIT"
        }
      ]
    }
  }
}
```

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Updating a rate limiting rule from the CLI

Run the following command to update a rate limiting rule from the CLI:

```
$ ibmcloud cis ratelimit-rule-update DNS_DOMAIN_ID RATELIMIT_RULE_ID [--url URL] [--description DESCRIPTION] [--threshold NUM] [--period SECONDS] [...]
```

You can also update a rate-limiting rule by providing a JSON file or a JSON string directly:

```
$ ibmcloud cis ratelimit-rule-update DNS_DOMAIN_ID RATELIMIT_RULE_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`RATELIMIT_RULE_ID`

The ID of rate limiting rule. Required.

`--json`

The JSON file or JSON strinthat is used to describe a rate limiting rule.

- The required fields in JSON data are `match`, `threshold`, `period` and `action` :
 - `match` : Determines which traffic the rate limiting rule counts towards the threshold.
 - `request` : Matches HTTP requests.
 - `methods` : HTTP Methods, can be a subset `[POST,PUT]` or all `[ALL]`. This field is not required to create a rate limit rule. Valid values are `GET`, `POST`, `PUT`, `DELETE`, `PATCH`, `HEAD`, `ALL`.
 - `schemes` : HTTP Schemes, can be one `[HTTPS]`, both `[HTTP,HTTPS]` or all `[_ALL_]`. This field is not required.
 - `url` : The URL pattern to match consisted of the host and path, for instance, `example.org/path`. Wildcards are expanded to match applicable traffic, query strings are not matched. Use `*` for all traffic to your zone. The max length is `1024`.
 - `response` : Matches HTTP responses before they are returned to the client. If this field is defined, then the entire counting of traffic occurs at this stage.
 - `status` : HTTP Status codes, can be one `[403]`, many `[401,403]` or indicate all by not providing this value. This field is not required. The min value is `100` and the max value is `999`.
 - `headers` : Array of response headers to match. If a response does not meet the header criteria, then the request is not counted towards the rate limiting rule. An array of header matching criteria includes the following properties.
 - `name` : The name of the response header to match.
 - `op` : The operator when matching, eq means equals, ne means not equals. Valid values are `eq` and `ne`.
 - `value` : The value of the header, which is exactly matched.
 - `threshold` : The threshold that triggers the rate limit mitigations, which are combined with period. For example, the threshold per period. The min value is `2` and the max value is `1000000`.
 - `period` : The time, in seconds, to count matching traffic. If the count exceeds the threshold within this period the action is performed. The min value is `1` and the max value is `3600`.
 - `action` : The action is performed when the threshold of matched traffic within the defined period is exceeded.
 - `mode` : The type of action performed. Valid values are `simulate`, `ban`, `challenge`, `js_challenge`.
 - `timeout` : The time, in seconds, as an integer to perform the mitigation action. Timeout is the same or greater than the

period. This field is valid only when mode is `simulate` or `ban`. The min value is `10` and the max value is `86400`.

- `response` : Custom content-type and body to return. This overrides the custom error for the zone. This field is not required. Omission results in the default HTML error page. This field is valid only when mode is `simulate` or `ban`.
 - `content_type` : The content-type of the body, which must be one of the following: `text/plain`, `text/xml`, `application/json`.
 - `body` : The body to return. The content here must conform to the `content_type`. The max length is `10240`.
- The optional fields are `disabled`, `description`, `correlate` and `bypass` :
 - `disabled` : Whether this rate limiting rule is currently disabled.
 - `description` : A note that you can use to describe the reason for a rate limiting rule.
 - `correlate` : Whether to enable NAT-based rate limiting.
 - `by` : Valid value is `nat`.
 - `bypass` : Criteria that allow the rate limit to be bypassed. For example, to express that you shouldn't apply a rate limit to a set of URLs.
 - `name` : Valid value is `url`.
 - `value` : The url to bypass.

Sample JSON data:

```
{
  "disabled": false,
  "description": "Prevent multiple login failures to mitigate brute force attacks",
  "bypass": [
    {
      "name": "url",
      "value": "api.example.com/*"
    }
  ],
  "threshold": 60,
  "period": 900,
  "correlate": {
    "by": "nat"
  },
  "action": [
    {
      "mode": "simulate",
      "timeout": 86400,
      "response": {
        "content_type": "text/plain",
        "body": "<error>This request has been rate-limited.</error>"
      }
    }
  ],
  "match": {
    "request": {
      "methods": [
        "GET"
      ],
      "schemes": [
        "HTTP",
        "HTTPS"
      ],
      "url": "*example.org/path*"
    },
    "response": {
      "status": [
        403, 401
      ],
      "headers": [
        {
          "name": "Cf-Cache-Status",
          "op": "eq",
          "value": "HIT"
        }
      ]
    }
  ]
}
```



```
}  
}  
}
```

`-i, --instance`

Instance name or ID. If not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

Specify output format, only `JSON` is supported.

Deleting a rate limiting rule from the CLI

Run the following command to delete a rate limiting rule from the CLI:

```
$ ibmcloud cis ratelimit-rule-delete DNS_DOMAIN_ID RATELIMIT_RULE_ID [--instance INSTANCE]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`RATELIMIT_RULE_ID`

The ID of rate limit rule. Required.

`-i, --instance`

Instance name or ID. If not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

For more information, see [CLI rate-limiting](#).

Creating a custom rate-limiting rule with Terraform

To create a rate-limiting ruleset, you must create an endpoint first, then create the rate-limiting ruleset. To do so, follow these steps:

1. To create an endpoint ruleset, run the following command:

```
$ resource "ibm_cis_ruleset_entrypoint_version" "config" {  
  cis_id = data.ibm_cis.cis_instance.id  
  domain_id = data.ibm_cis_domain.cis_domain.domain_id  
  phase = "http_ratelimit"  
  
  rulesets {  
    description = "Zone rate limit endpoint"  
  }  
  lifecycle {  
    ignore_changes = [  
      rulesets  
    ]  
  }  
}
```



Important: Use a `lifecycle` block to prevent Terraform from updating the endpoint ruleset. Some endpoint parameters are updated during every `terraform apply`, which can introduce unintended configuration changes. The `lifecycle` block helps you ignore these updates and maintain resource stability.

2. To create a rate-limiting ruleset, run the following command:

```
resource "ibm_cis_ruleset_rule" "config" {  
  cis_id = data.ibm_cis.cis_instance.id
```

```

domain_id = data.ibm_cis_domain.cis_domain.domain_id
ruleset_id = "data.ibm_cis_ruleset_entrypoint_versions.ruleset_id"
rule {
  action    = "block"
  enabled   = true
  description = "Block IPs making over 100 requests/minute to /api/"
  expression = "(http.request.uri.path matches \"^/api/\")"
  ratelimit {
    characteristics = ["cf.colo.id", "ip.src"]
    period          = 60
    requests_per_period = 100
    mitigation_timeout = 300
  }
}
}
}

```

The following example shows how to create an endpoint and rate-limiting rule with Terraform:

```

resource ibm_cis_ruleset_entrypoint_version test {
  cis_id   = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  phase = "http_ratelimit"
  rulesets {
    description = "Entrypoint ruleset for ratelimit ruleset"
  }
  lifecycle {
    ignore_changes = [
      rulesets
    ]
  }
}

data ibm_cis_ruleset_entrypoint_versions test {
  cis_id   = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  phase = "http_ratelimit"
  depends_on = [
    ibm_cis_ruleset_entrypoint_version.ratelimit_ep
  ]
}

resource "ibm_cis_ruleset_rule" "ratelimit_rule_1" {
  cis_id   = ibm_cis.instance.id
  domain_id = data.ibm_cis_domain.cis_domain.domain_id
  ruleset_id = data.ibm_cis_ruleset_entrypoint_versions.ratelimit_data.rulesets[0].ruleset_id

  rule {
    action    = "block"
    description = "Block IPs making over 100 requests/minute to /api/"
    enabled   = true
    expression = "(http.request.uri.path matches \"^/api/\")"

    rate_limit {
      characteristics = ["cf.colo.id", "ip.src"]
      mitigation_timeout = 300
      period          = 120
      requests_per_period = 100
    }
  }
}

```

Rate-limiting rulesets

Use rate-limiting rules to define rate limits for requests matching an expression, as well as the action to perform when those rate limits are reached.

Rule parameters

As with other rules evaluated by the Ruleset Engine language, rate-limiting rules have the following basic parameters:

- An expression that specifies the criteria you are matching traffic on using the rules language.
- An action that specifies what to perform when there is a match for the rule and any additional conditions are met. With rate-limiting rules, the action occurs when the rate reaches the specified limit.

Rate-limiting rules also require the following additional parameters:

- **Characteristics:** The set of parameters that define how the rate is tracked for this rule.
- **Period:** The period of time to consider (in seconds) when evaluating the rate.
- **Requests per period:** The number of requests over the period of time that will trigger the rate-limiting rule.
- **Duration (or mitigation timeout):** After the rate is reached, the rate-limiting rule blocks further requests for the period of time that is defined in this field.
- **Action behavior:** By default, the rule action is applied for the configured duration (or mitigation timeout), regardless of the request rate during this period.

Important considerations

Rate-limiting rules are evaluated in order, and some actions (like blocking) stop the evaluation of other rules. For more information on actions and their behavior, see [Ruleset Engine rules actions](#).



Note: Refer to [Migrating to WAF custom rules](#) to learn more about the differences between firewall rules and WAF custom rules.



Important: Rate-limiting rules are not designed to allow a precise number of requests to reach the origin server. In some situations, there might be a delay (up to a few seconds) between detecting a request and updating internal counters. Due to this delay, excess requests might still reach the origin server before the mitigation action (such as blocking or challenging) is enforced.

Availability

The rate-limiting ruleset feature is available for CIS users on the Enterprise Advanced and Enterprise Premier plans. For more information, see [Comparing CIS plans](#).

Working with legacy WAF

Migrating to WAF managed rules

The CIS web application firewall (WAF) capabilities are moving under the Ruleset Engine suite. This change requires a manual migration. Use the CIS UI migration wizard and review the configuration as well as [security events](#) before finishing the update.



Important: After you complete the migration, any automation that uses APIs, CLIs, or Terraform that are related to WAF will stop working and must be updated to use the new managed rulesets.



Tip: Instances created after June 2024 use the new Ruleset Engine language and don't need to be migrated.

To migrate your instance to managed rules, take the following steps:

1. Navigate to the **Security** section.
2. Select the **WAF** tab. If the CIS instance has not been upgraded, you receive the message to **Update to the new WAF**.
3. Click **Review configuration**.
4. In the **Review configuration** panel, the rulesets are listed in the order in which they are applied.
5. Enable or disable the rulesets by using the switches in the **Status** column.
6. From the **Actions** menu beside each ruleset, you can choose to edit or reorder the ruleset, or delete it completely. These actions are considered overrides from the default.
7. In the **Managed rulesets** section, you can add or configure rulesets that are not yet added.
 - Click **Add** on the ruleset you want to add, then toggle the switch from **Disabled** to **Enabled** on the new rule.
 - Click **Configure** on the ruleset you want to configure before migrating. In the **Configure deployment** side panel, you can accept all incoming requests or update the scope of execution with the customized filters you make in the expression builder. Then, click **Save**.
8. Click **Deploy** in the side panel to continue.
9. (Enterprise only) Review security events in the **Security > Events** tab.

When you select **Deploy**, both WAF implementations are enabled simultaneously, allowing you to validate your new configuration and finish the upgrade.

After deploying the new WAF configuration, both the old and new WAF Managed Rules run in parallel (called validation mode). During this phase, the WAF tab in the CIS console displays both rulesets, with the new WAF Managed Rules evaluated before the previous version.

- a. Use the current validation mode to review the behavior of the new WAF configuration in **Security > Events**.
 - b. When you are done reviewing your configuration with both WAFs enabled, click **Ready** to complete the upgrade.
10. Select **Turn off previous version** to finalize the migration (this step can't be undone), or cancel to continue editing. This transition does not incur any downtime.

Editing rulesets

From the Action menu, you can select the following actions:

- **Edit:** Opens a panel where you can change ruleset actions and status, as well as perform a batch edit of rules within the ruleset. Edits are considered overrides of the default rules.
- **Delete:** Removes the ruleset from the list.
- **Move up:** Changes the order in which the rulesets are executed by moving one row higher in priority.
- **Move to...:** Changes the priority order in which the rulesets are executed.

Adding an exception

To add your own exceptions, take the following steps.

1. Click **Add exception**.
2. In the **Add exception** side panel, enter a name for your exception.
3. Select options in the **When incoming requests match...** section, or use the expression builder to fine-tune the exception.
4. If you want to log matching requests, move the switch to the **On** position.
5. Select if you want the exception to skip all remaining rules, or skip specific rules from a managed ruleset.
6. Click **Save**.

Managed rules FAQs

What if I don't migrate?

Users who don't manually complete the migration to Managed Rules by 12 June 2025, may be subjected to being automatically migrated, with no expected impact to their current WAF policies or security. From this date forward, you must use the [Ruleset Engine API](#) to make WAF and Managed Rules configurations.

Rules and configuration might be slightly different than before, because the new Managed Rules added more robust OWASP security coverage. This ruleset is updated from OWASP v2.x to OWASP v3.x; which adds paranoia levels and improves false positives rates.

Will there be any WAF downtime caused by the migration?

There is no outage in WAF protection during the migration, but users might observe security events from both the legacy WAF and new WAF managed rules for up to one hour after the migration is complete.

What will happen to the previous WAF APIs?

After deprecation, the previous WAF APIs will not be available and will generate an error, returning a message that indicates to switch to the Managed Rules feature.

How can I confirm that the migration is complete?

Run the Migration Status API check. You can also check to see whether the UI shows the wizard on the WAF page. If the wizard appears, you must migrate.

Can I revert to the previous WAF?

No. Migration to managed rulesets is final and can't be undone.

Why can I no longer see the migration wizard?

The wizard appears only when the previous WAF is enabled. If you don't see the wizard, you've likely already migrated.

How do I migrate without using the IBM UI or migration wizard?

While APIs are available, the wizard is the recommended method.

Expected changes when migrating

When migrating to the new version of the WAF managed rules, the update process will partially carry over settings from the OWASP ModSecurity Core Ruleset used in the previous version.

The following OWASP settings will be migrated:

- **Sensitivity:** Sensitivity levels from the [previous ruleset](#) will be migrated to the Paranoia level (PL) and Score threshold combinations in the new OWASP ruleset:

Old Sensitivity	New PL	New Score Threshold
High	PL2	Medium – 40 or higher
Medium	PL1	High – 25 or higher
Low	PL1	Medium – 40 or higher

Default	PL2	Medium – 40 or higher
---------	-----	-----------------------

Sensitivity values in the OWASP ruleset

- **Action:** Most actions from the previous OWASP ruleset map directly to the new one. However, the **Simulate** action is now mapped to **Log**.

The following OWASP settings will not be migrated, as there are no direct equivalence between the rules in the two versions:

- OWASP group overrides
- OWASP rule overrides

To replace these settings, you must reconfigure the new OWASP Core Ruleset in the WAF managed rules. This includes any customizations, such as tag or rule overrides. For more information, see the [CIS OWASP Core Ruleset](#).

Known issues with migration

You can contact IBM Cloud support for help with the following errors:

- If the number of firewall rules you want to migrate exceeds 200.
- If the length of a firewall rule expression is longer than 4 KB.

Related links

To learn more about the available rulesets, see [About rulesets](#).

WAF ruleset actions

**Note:** CIS WAF capabilities are moving to the [Ruleset Engine rules language](#). For more information on this change, see [Migrating to managed rules](#).

The following table shows the actions that Web Application Firewalls (WAFs) can take.

Action	rulesets	Definition
Block	All	Blocks an attack stops any action before it is posted to your website.
Log	All	To test for false positives, set the WAF to Log mode, which records the response to possible attacks without challenging or blocking.
Challenge	All	■ Manged challenge: Dynamically chooses the appropriate type of challenge based on the characteristics of the request. ■ Interactive challenge: Solve a puzzle to proceed. ■ JavaScript challenge: A challenge page asks visitors to submit a CAPTCHA to continue to your website.

WAF actions

**Note:** In Enterprise plans, you have the flexibility to turn on or off individual WAF rules for a particular URI in a domain, instead of the whole domain or subdomain. For more information, see the [waf-override-create](#) command.

Working with legacy firewall rules

Migrating to WAF custom rules

CIS upgraded existing [firewall rules](#) to use the [Ruleset Engine](#); where they are now known as [custom rules](#). With custom rules, users get as much protection with a few extra features. In the CIS dashboard, these rules can still be found in the **Security page > Firewall rules tab**.



Important: If you haven't migrated to WAF custom rules yet, you might have some invalid configuration that prevents the migration from happening. In this case, contact your account team to get help with the migration to WAF custom rules.

Main differences between firewall rules and custom rules

The main differences between firewall rules and WAF custom rules are as follows:

- [Improved response for Block action](#)
- [Different error page for blocked requests](#)
- [New Skip action replacing both Allow and Bypass actions](#)
- [Custom rules are evaluated in order](#)
- [Logs and events](#)
- [New API](#)

Improved response for Block action

In custom rules, you can customize the response of the Block action.

The default block response is a CIS standard HTML page. If you need to send a custom response for Block actions, configure the custom rule to return a fixed response with a custom response code (403, by default) and a custom body (HTML, JSON, XML, or plain text).

To define a custom response for a single rule, go to **Security > WAF > Custom rules**, edit the custom rule, and complete the block-related options.



Note: Custom block response configurations are not returned by the Firewall Rules API. Use the [1020 error code](#) response. CIS users might customize this error page in **Custom Pages > 1000 Class Errors**.

Requests blocked by a WAF custom rule get a different response: the WAF block response. To customize the default block response, you can either:

- Define a custom WAF block response for your entire zone in **Custom Pages > WAF Block**. This custom page will always have an HTML content type.
- [Define a custom response](#) for requests blocked by a specific WAF custom rule. This custom response supports other content types besides HTML.

If you customized your 1xxx error page in Custom Pages for requests blocked by firewall rules, you must create a new response page for blocked requests by using one of the preceding methods.

New Skip action replacing both Allow and Bypass actions

Firewall Rules supported the Allow and Bypass actions, often used together. These actions were commonly used for handling known legitimate requests — for example, requests coming from trusted IP addresses.

When a request triggered Allow, all remaining firewall rules are not evaluated, effectively allowing the request to continue to the next security product. The Bypass action was designed to specify which security products (such as WAF managed rules, rate-limiting rules, and User Agent Blocking) should not run on the request triggering the action.

With Firewall Rules, if you wanted to stop running all security products for a given request, you would create two rules:

- One rule with Bypass action (selecting all security products).
- One rule with Allow action (to stop executing other firewall rules).

The requirement of having two rules to address this common scenario no longer applies to WAF custom rules. Now use the Skip action, which combines the Allow and Bypass actions. The Skip action fully replaces the Allow and Bypass actions, which are not supported in WAF custom rules.

With the Skip action you can do the following:

- Stop running all the remaining WAF custom rules (equivalent to the Allow action)
- Avoid running other security products (equivalent to the Bypass action)
- A combination of both.

You can also select whether you want to log events matching the custom rule with the Skip action or not. This is especially useful when creating a positive security model to avoid logging large amounts of legitimate traffic.



Note: The Firewall Rules API does not support the Skip action. When you create a custom rule with Skip action, it is translated to Allow and Bypass in the Firewall Rules API. Use the [priority ordering](#). In contrast, WAF custom rules actions do not have such an order. Custom rules are always evaluated in order, and some actions, such as Block, stop the evaluation of other rules.

For example, if you use priority ordering and had the following firewall rules with the same priority both matching an incoming request:

- Firewall rule #1 — Priority: 2 / Action: Block
- Firewall rule #2 — Priority: 2 / Action: Allow

The request is allowed because the Allow action in Firewall Rules has precedence over the Block action.

In contrast, if you create two WAF custom rules where both rules match an incoming request:

- Custom rule #1 — Action: Block
- Custom rule #2 — Action: Skip (configured to skip all remaining custom rules)

The request is blocked because WAF custom rules are evaluated in order and the Block action stops the evaluation of other rules.



Note: For the WAF custom rules converted from your existing firewall rules, CIS preserves your current order of execution.

Logs and events

Events that are logged by WAF custom rules are available at **Security > Events**, with **Custom rules** as their source. For more information, see [Using the CIS Security Events capability](#).

You might still find events that are generated by firewall rules in the Security Events page when you select a time frame including the days when the transition to WAF custom rules occurred. Similarly, you might still find events with both Skip and Allow actions in the same view during the transition period.

New API

The preferred API for managing WAF custom rules is the [Relevant changes for API users](#).



Note: The Firewall Rules API and Filters API will still work until 30 July 2025. There will be a single list of rules for both firewall rules and custom rules, and this list contains WAF custom rules. Thanks to an internal conversion process, the Firewall Rules API, and Filters API return firewall rules/filters converted from these WAF custom rules.

Relevant changes for dashboard users

The **Firewall rules tab** will continue to exist and function as expected. The main difference is the APIs used in the background.

Relevant changes for API users



Deprecated: The [About WAF custom rules](#) for examples of managing WAF custom rules that use the Rulesets API.

Creating, editing, and deleting firewall rules



Deprecated: Firewall rules are deprecated. CIS moved existing firewall rules to WAF custom rules. For more information on this change, see [Migrating to custom rules](#).

IBM Cloud® Internet Services firewall rules offer power and flexibility by targeting HTTP traffic and applying custom criteria to block, challenge,

log, or allow certain requests.

You can create many types of firewall rules. However, the number of active rules on your site is limited by your customer plan. For more information on entitlements, see [Comparing CIS plans](#).

The number of active rules per plan is fixed. Currently, you can't purchase more active rules.



Important: Before getting started, it's a good idea for you to become familiar with [Using fields, functions, and expressions](#).

Creating a firewall rule

Follow these steps to configure a basic firewall rule:

1. Navigate to **Security > Firewall rules**.
2. Click **Create**.
3. Enter an optional description.
4. Optionally, input a priority, if necessary. A priority of zero is a null priority and is evaluated last.
5. Use the UI builder in the **Incoming requests** section to add a condition. To build an expression with multiple conditions, click either:
 - **And** - to evaluate conditions that use *and* logic
 - **Or** - to evaluate conditions or groups of previously *and*'ed conditions that use *or* logic

You can see that as you build a condition, the Expression Preview shows the expression in plain text.



Note: In the Expression preview, you can click to edit your expression manually instead of using the Visual Expression Builder, or switch between the two. However, depending on the complexity of a manually constructed expression, the Visual Expression Builder might be unable to render it.

6. Pick an action from the **Response** list menu.
7. To save your rule, choose the most appropriate option by clicking either:
 - **Save as draft** to save your rule, but leave it disabled.
 - **Save and deploy** to save your rule and activate it.

Editing a firewall rule

Follow these steps to edit an existing rule:

1. Navigate to **Security > Firewall rules**.
2. In the firewall rules table, locate the rule that you want to modify, then click the Actions menu on the right of the row.
3. Select **Edit**.
4. Make your changes to the rule.
5. To save your rule, choose the most appropriate option by clicking either:
 - **Save as draft** to save your rule, but leave it disabled.
 - **Save and deploy** to save your rule and activate it.



Note: To pause or activate any rule in the list of existing rules, click the **Enabled** toggle.

Deleting a firewall rule

Follow these steps to delete an existing rule:

1. Navigate to **Security > Firewall rules**.
2. In the firewall rules table, locate the rule to modify and click the Actions menu on the right of the row.
3. Select **Delete**.
4. Confirm the rule deletion.

Assigning firewall rule actions

 **Deprecated:** Firewall rules are deprecated. CIS moved existing firewall rules to WAF custom rules. For more information on this change, see [Migrating to custom rules](#).

Firewall rule actions tell CIS how to respond to requests that match the criteria you define.

For lightweight firewall rules, go to **Security > IP firewall**, which contains IP rules, User Agent rules, and Domain Lockdown rules. Firewall rules are based on IP address, IP address range, Autonomous System Number (ASN), or country/region.

 **Tip:** Domain lockdown rules specify a list of IP addresses, CIDR ranges, or networks that can access a domain, subdomain, or URL. Anything not on the list is blocked.

For more robust firewall rules, go to **Security > Firewall rules**, where you can create rules that examine incoming HTTP traffic against a set of filters to block, challenge, log, or allow matching requests.

The following table describes the actions that you can assign to your rules. The priority column shows the precedence that the action receives. If a request matches two different rules that have the same priority, precedence determines the action to take.

Action	Available in	Description	Priority
Log	Firewall rules	Logs matching requests on the CIS edge for access with Enterprise Logpush and Logpull. Recommended for testing rule effectiveness you commit to a more severe action. Available to Enterprise customers only.	1
Bypass	Firewall rules	Allows dynamic disabling of security features for a request. Exempts matching requests from evaluation, based on a user-defined list that contains one or more of the following features: Browser Integrity Check , Domain Lockdown , Hotlink Protection , Rate Limiting , Security Level , User Agent Block , WAF Managed Rules . Matching requests are still subject to evaluation within Firewall Rules, based on order of execution.	2
Allow	- Firewall rules - IP firewall	Allows matching requests to access the site, on condition that no other CIS firewall features block the request, such as IP firewall or access rules.	3
Challenge (Captcha)	- Firewall rules - IP firewall - User agent rules	Requires a user to pass a Google reCaptcha Challenge before proceeding. If successful, CIS accepts the matched request; otherwise, it is blocked.	4
JS Challenge	- Firewall rules - IP firewall - User agent rules	Requires a user to pass a CIS JavaScript Challenge before proceeding. If successful, CIS accepts the matched request; otherwise, it is blocked.	5

Block	■ Firewall rules ■ IP firewall ■ User agent rules	Blocks a matching request from accessing the site.	6
-------	---	--	---

Firewall rule actions and priority

Using fields, functions, and expressions


Deprecated: Firewall rules are deprecated. CIS moved existing firewall rules to WAF custom rules. For more information on this change, see [Migrating to custom rules](#).

Along with actions, fields and expressions are the building blocks of firewall rules. These two elements work together when defining the criteria to use when a firewall rule is matched.

Fields

When CIS receives an HTTP request, it is examined and a table of fields is produced to match against. This field table exists while the current request is being processed. Think of it as a table that holds the request properties to be matched against expressions.

Each field value can be sourced from different places, such as:

- Primitive propertiesare , obtained directly from the traffic – for example, `http.request.uri.path` .
- Derived values, resulting from a transformation, composition, or basic operation – for example, making the value of `http.request.uri.path` all lowercase and available as a field of another field.
- Computer values, resulting from a lookup, computation, or other intelligence – for example, a `cf.threat_score` calculated dynamically by a machine learning process that inspects related primitive and derived values.

Available fields

Field name	Type	Example value	Notes
http.cookie	String	session=A12345;-background=light	Entire cookie as a string
http.host	String	www.example.com	The hostname used in the full request URI
http.referer	String	<i>HTTP referer header</i>	
http.request.full_uri	String	https://www.example.com/articles/index?section=539061&expand=comments	The full URI as received by the web server (does not include <i>#fragment</i> , which is not sent to web servers)
http.request.method	String	POST	The HTTP method, in uppercase
http.request.uri	String	/articles/index?section=539061&expand=comments	The absolute URI of the request
http.request.uri.path	String	/articles/index	The path of the request
http.request.uri.query	String	section=539061&expand=comments	The whole query string, minus the delimiting prefix "?"
http.user_agent	String	Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/65.0.3325.181 Safari/537.36	The whole HTTP user agent

http.x_forwarded_for	String	<i>The full X-Forwarded-For HTTP header</i>		
ip.src	IP address	93.155.208.22	The client TCP IP address, which can be adjusted to reflect the real client IP of the original client as applicable (for example, by using HTTP headers like X-Forwarded-For or X-Real-IP)	
ip.src.asnum	Number	222	The Autonomous System (AS) number	
ip.src.country	String	GB	The 2-letter country code	
ssl	Boolean	true	Whether the HTTP connection to the client is encrypted	

Available fields



Note: These standard fields follow the naming convention of the Wireshark display field reference. However, some subtle variations might exist in the preceding example values.

In addition to the standard fields, the following Cloudflare-defined fields are also available:

Field name	Type	Example value	Notes
cf.client.bot	Boolean	true	This field indicates whether the request is coming from a known bot or crawler, regardless of good or bad intent.
cf.threat_score	Number	A 0-100 value	This field represents a risk score, 0 indicates low risk as determined by Cloudflare. Values greater than 10 can represent spammers or bots, and values greater than 40 point to bad actors on the internet. It is rare to see values higher than 60, so tune your firewall rules to challenge those greater than 10, and to block those greater than 50.
cf.waf.score	Number	1-99	Machine learning–based score that estimates the likelihood of a request being malicious. Scores range from 1 (most likely malicious) to 99 (most likely safe). Lower scores indicate a higher risk. Useful for creating threshold-based firewall rules.

Available Cloudflare fields

Functions

The firewall rules language has several functions to convert fields.



Note: These functions are not currently supported in the Expression Builder.

Function name	Argument types	Return type	Usage example	Notes
lower	String	String	lower(http.host) == "www.example.com"	Converts a string field to lowercase. Only uppercase ASCII bytes are being converted. Every other bytes are left as-is.
upper	String	String	upper(http.host) == "www.example.com"	Converts a string field to uppercase. Only lowercase ASCII bytes are being converted. Every other bytes are left as-is.

Firewall rules functions

Expressions

An expression returns true or false based on a match against incoming traffic. For example:

```
http.host eq "www.example.com" and ip.src in 92.182.212.0/24
```

In this example, two single expressions comprise a compound expression. Think of each single expression as a condition. Each condition is evaluated individually before applying and logic to determine the final result of the compound expression.

Looking at the first single expression, you can see that it contains:

- a field - `http.host`
- a comparison operator - `eq`
- a value - `"www.example.com"`

Not all conditions have the same structure. Other examples that use different structures are discussed in the next section.

Comparison operators

The following comparison operators are available for use in expressions:

English	C-like	Description
eq	==	Equal
ne	!=	Not equal
lt	<	Less than
le	<=	Less than or equal to
gt	>	Greater than
ge	>=	Greater than or equal to
contains		Exactly contains
matches	~	Re2 inspired regular expression
in		Value appears in a set of values. Supports ranges that use the ".." notation.
not	!	See Boolean comparison
bitwise_and	&	Compare bit field value

Comparison operators for expressions



Note: Currently the Expression Builder supports only English operators.

An expression might contain a mix of English and C-like operators. For example, `ip.src eq 93.184.216.34` is equivalent to `ip.src == 93.184.216.34`.

Certain comparison operators apply to specific fields based on type. The following matrix provides examples of which operators are available for various field types:

English	C-like	String	IP address	Number
eq	==	<code>http.request.uri.path eq "/articles/2008/"</code>	<code>ip.src eq 93.184.216.0</code>	<code>cf.threat_score eq 10</code>
ne	!=	<code>http.request.uri.path ne "/articles/2010/"</code>	<code>ip.src ne 93.184.216.0</code>	<code>cf.threat_score ne 60</code>
lt	<	<code>http.request.uri.path lt "/articles/2009/"</code>		<code>cf.threat_score lt 10</code>
le	<=	<code>http.request.uri.path le "/articles/2008/"</code>		<code>cf.threat_score le 20</code>

gt	>	http.request.uri.path gt "/articles/2006/"	cf.threat_score gt 25
ge	>=	Greater than or equal to	cf.threat_score ge 60
contains		http.request.uri.path contains "/articles/"	
matches	~	http.request.uri.path ~ "^/articles/200[7-8]/\$"	
in		http.request.method in { "HEAD" "GET" }	ip.src in { 93.184.216.0 93.184.216.1 } cf.threat_score in {0 2 10}

Comparison operators for fields



Important: The evaluation of expressions that use string values is case-sensitive. As such, a firewall rule might require you to define more than one test condition. Enterprise customers can use a regular expression with the matches operator to capture multiple variations with a single expression.

Boolean comparison

For fields of Boolean type (for example, `ssl`) the field appears by itself in the expression when evaluating for a true condition. For a false condition, the **not** operator applies.

True	False
ssl	not ssl

Boolean comparison

Compound expressions

You can create compound expressions by grouping two or more single expressions by using logical operators.

English	C-like	Description	Example	Precedence
not	!	Logical NOT	not (http.host eq "www.example.com" and ip.src in 93.184.216.0/24)	1
and	&&	Logical AND	http.host eq "www.example.com" and ip.src in 93.184.216.0/24	2
xor	^^	Logical XOR	http.host eq "www.example.com" xor ip.src in 93.184.216.0/24	3
or		Logical OR	http.host eq "www.example.com" or ip.src in 93.184.216.0/24	4

Compound expressions

To alter the order of precedence, you can group expressions with parentheses. Using no parentheses, expressions are implicitly grouped based on standard precedence:

```
ssl and http.request.uri.path eq /login or http.request.uri.path eq /oauth
```

Applying explicit grouping:

```
(ssl and http.request.uri.path eq /login) or http.request.uri.path eq /oauth
```

Giving precedence to or with parentheses:

```
ssl and (http.request.uri.path eq /login or http.request.uri.path eq /oauth)
```

While `not` is used for grouping, it can be used to negate a single comparison. For example, `not ip.src eq 93.184.216.0` is equivalent to `not (ip.src eq 93.184.216.0)`.

Finally, you can also negate grouped expressions:

```
not (http.request.method eq "POST" and http.request.uri.path eq "/login")
```

Deviations from Wireshark display filters

Firewall rules expressions are inspired by Wireshark display filters. However, the implementation deviates in the following ways:

- For CIDR IP equality tests, Wireshark allows ranges in the format `ip.src == 1.2.3.0/24`, while CIS supports equality tests using only a single IP address. To compare a CIDR, use the `in` operator; for example, `ip.src in {1.2.3.0/24}`.
- In Wireshark, `ssl` is a protocol field that contains hundreds of other fields of various types that are available for comparison in multiple ways. However, in Firewall Rules, `ssl` is a single Boolean field that is used to determine whether the connection from the client to CIS is encrypted.
- The `slice` operator is not supported.
- Not all functions are supported. Currently, `len()`, and `count()` are not supported.

Prioritizing options

 **Deprecated:** Firewall rules are deprecated. CIS moved existing firewall rules to WAF custom rules. For more information on this change, see [Migrating to custom rules](#).

By default, the action you specify in a rule determines the evaluation sequence for your entire ruleset. However, you can set a **priority** option that allows you to force the evaluation sequence to occur before triggering a rule.

You can manage a priority by using the firewall rules UI in the CIS dashboard. Priority management is also available from the firewall rules API.

 **Note:** The firewall rules engine processes rules in parallel. If you do not specify priority in a large ruleset, a matching conflict is possible. To avoid any conflicts, explicitly control the evaluation sequence of your ruleset by using a priority.

Firewall rules don't have default priorities and do not force you to have a priority for every rule. This gives you the freedom to organize your ruleset the way you want. The numbering can be relatively arbitrary, as long as it makes sense for your particular situation. However, keep in mind the following issues:

- The evaluation sequence starts from the lowest priority number to the highest.
- The valid priority range is 1 to 2147483647.
- Avoid using the number 1 as a priority. Because no other rule can go above it, it makes it harder to renumber your rules.
- Rules with no priorities are evaluated last.

Group ranges of priority numbers into meaningful categories. For example:

- 5000-9999 - Trusted IP addresses
- 10000-19999 - Blocking rules for bad crawlers
- 20000-29999 - Blocking rules for abusive/spam users

 **Note:** These example priority numbers are valid since you can have many more rules than the limit associated with your domain plan. That limit applies solely to active rules.

Working with rules

About rules

Rules in CIS allow you to inspect incoming HTTP requests and apply actions based on matching conditions. Rules are evaluated by the Ruleset Engine, which processes requests through a series of execution phases before traffic reaches the origin server.

A rule consists of the following components:

- Expression: Defines the conditions that determine whether a request matches the rule.
- Action: Specifies what action to take when the request matches the expression.
- Phase: Defines when the rule is evaluated during request processing.

CIS supports several rule-based capabilities through the Rulesets API, including:

- Configuration rules (`http_config_settings`)
- Origin rules (`http_request_origin`)
- Custom rules
- Rate-limiting rules
- Other Ruleset Engine features that are supported by the CIS Rulesets API implementation.

Rules are organized into rulesets. Each ruleset belongs to a specific phase and contains one or more ordered rules that are evaluated when requests reach that phase. Rulesets are versioned, and updates to a ruleset create a new version of the configuration.

The Rulesets API enables you to create, manage, and update rulesets at the zone level. By using rules, you can customize traffic handling, modify origin behavior, apply configuration settings, and implement security controls without changing your applications or origin infrastructure.

Working with configuration rules

About configuration rules

Configuration rules allow you to customize how CIS processes matching HTTP requests. Configuration rules are evaluated in the `http_config_settings` phase of the Ruleset Engine and can modify request-processing settings based on request attributes such as hostnames, URL paths, or request headers.

Each configuration rule consists of an expression that identifies matching requests and a `set_config` action that applies the specified configuration settings. Configuration rules are managed through the Rulesets API and are evaluated before origin rules in the request-processing workflow.

Rule execution order

Configuration rules are evaluated in the Ruleset Engine during the `http_config_settings` phase. Ruleset Engine rules take precedence over Page Rules. If a request matches both a Page Rule and a Ruleset Engine rule, the Ruleset Engine rule is applied.

Creating a configuration rules

Create a configuration rule by adding it to the `http_config_settings` phase ruleset for a zone.

Before you begin

When creating a configuration rule via API, make sure you:

- Set the rule action to `set_config`.
- Define the parameters in the `action_parameters` field according to the settings you wish to override for matching requests.
- Deploy the rule to the `http_config_settings` phase at the zone level.

Creatin a configuration rule via API

Complete the following steps to create a configuration rule for a given zone via API:

1. Check whether a ruleset exists for the `http_config_settings` phase by using the [List zone rulesets](#) API.

2. If a ruleset for the `http_config_settings` phase doesn't exist, create one by using the Create a zone ruleset API as shown in the following example.

Set the following values in the request body:

- kind: `zone`
- phase: `http_config_settings`

```
$ curl -X POST \
  "https://api.cis.cloud.ibm.com/v1/crn:v1:bluemix:public:internet-svcs:global:a/361583ba4e52947c3e111ba9d29702e3:78345201-a3a7-4de3-abe4-7115e6b0e2f1::/zones/2ce4a192bf5978e6b65a09ed2d36d4c2/rulesets" \
  --header "X-Auth-User-Token: Bearer xxxx" \
  --header "Content-Type: application/json" \
  --data '{
    "name": "Zone-level phase entrypoint",
    "kind": "zone",
    "description": "Config settings ruleset",
    "phase": "http_config_settings"
  }'

{"result": {"description": "Config settings ruleset", "id": "b19bf5e7c3bd40b890a0302390e86e0c", "kind": "zone", "last_updated": "2026-06-13T14:52:37.636034Z", "name": "Zone-level phase entrypoint", "phase": "http_config_settings", "version": "1"}, "success": true, "errors": [], "messages": []}%
```

3. Add a configuration rule to the ruleset by using the [Update a zone ruleset](#) API. Alternatively, include the rule in the Create a zone ruleset request mentioned in the step 2.

```
$ curl -X PUT \
  "https://api.cis.cloud.ibm.com/v1/crn:v1:bluemix:public:internet-svcs:global:a/361583ba4e52947c3e111ba9d29702e3:78345201-a3a7-4de3-abe4-7115e6b0e2f1::/zones/2ce4a192bf5978e6b65a09ed2d36d4c2/rulesets/b19bf5e7c3bd40b890a0302390e86e0c" \
  "Content-Type: application/json" \
  -H "Accept: application/json" \
  -H "X-Auth-User-Token: Bearer xxxxx" \
  -d '{
    "description": "My http_config_settings ruleset to execute managed rulesets",
    "kind": "root",
    "phase": "http_config_settings",
    "rules": [
      {
        "action": "set_config",
        "expression": "starts_with(http.request.uri.path, \"/upload/)",
        "description": "Disable request buffering for file uploads",
        "action_parameters": {
          "request_body_buffering": "none"
        }
      }
    ]
  }'

{"result": {"description": "My http_config_settings ruleset to execute managed rulesets", "id": "b19bf5e7c3bd40b890a0302390e86e0c", "kind": "zone", "last_updated": "2026-06-13T15:12:56.987507Z", "name": "Zone-level phase entrypoint", "phase": "http_config_settings", "rules": [{"action": "set_config", "action_parameters": {"request_body_buffering": "none"}, "description": "Disable request buffering for file uploads", "enabled": true, "expression": "starts_with(http.request.uri.path, \"/upload/)", "id": "f2a88e88081c4a7e9ead5134de0a61c9", "last_updated": "2026-06-13T15:12:56.987507Z", "ref": "f2a88e88081c4a7e9ead5134de0a61c9", "version": "1"}], "version": "2"}, "success": true, "errors": [], "messages": []}%
```

Configuration rule examples

The following examples show how to use configuration rules to control request and response body buffering for incoming traffic.

Adding a rule that disables request body buffering for file uploads

Use this rule to stream file upload requests directly to the origin server instead of buffering the request body.

```
$ curl -X PUT \
  "https://api.cis.cloud.ibm.com/v1/{crn}/zones/{zone_id}/rulesets/{ruleset_id}" \
  -H "Content-Type: application/json" \
  -H "Accept: application/json" \
  -H "X-Auth-User-Token: Bearer {IAM_TOKEN}" \
  -d '{
```

```
"phase": "http_config_settings",
"rules": [
  {
    "action": "set_config",
    "expression": "starts_with(http.request.uri.path, \"/upload/\"),
    "description": "Disable request buffering for file uploads",
    "action_parameters": {
      "request_body_buffering": "none"
    }
  }
]
}'
```



Note: This rule applies only to requests whose URL path starts with `/upload/`.

Adding a rule that disables request body buffering for a specific hostname

Use this rule to disable request body buffering only for requests sent to a specific hostname.

```
$ curl -X PUT \
  "https://api.cis.cloud.ibm.com/v1/{crn}/zones/{zone_id}/rulesets/{ruleset_id}" \
  -H "Content-Type: application/json" \
  -H "Accept: application/json" \
  -H "X-Auth-User-Token: Bearer {IAM_TOKEN}" \
  -d '{
    "phase": "http_config_settings",
    "rules": [
      {
        "action": "set_config",
        "expression": "http.host eq \"uploads.example.com\"",
        "description": "Unbuffered uploads for specific hostname",
        "action_parameters": {
          "request_body_buffering": "none"
        }
      }
    ]
  }'
```



Note: Only requests to `uploads.example.com` use unbuffered request handling.

Adding a rule that enables full request body buffering for all requests

Use this rule to buffer the entire request body before forwarding requests to the origin server.

```
$ curl -X PUT \
  "https://api.cis.cloud.ibm.com/v1/{crn}/zones/{zone_id}/rulesets/{ruleset_id}" \
  -H "Content-Type: application/json" \
  -H "Accept: application/json" \
  -H "X-Auth-User-Token: Bearer {IAM_TOKEN}" \
  -d '{
    "phase": "http_config_settings",
    "rules": [
      {
        "action": "set_config",
        "expression": "true",
        "description": "Buffer entire request body",
        "action_parameters": {
          "request_body_buffering": "full"
        }
      }
    ]
  }'
```



Note: Because the rule expression is `true`, the configuration applies to all incoming requests.

Adding a rule that disables request and response body buffering for a specific hostname

Use this rule to stream both request and response bodies for requests sent to a specific hostname.

```
{
  "result": {
    "id": "b19bf5e7c3bd40b890a0302390e86e0c",
    "phase": "http_config_settings",
    "rules": [
      {
        "action": "set_config",
        "action_parameters": {
          "request_body_buffering": "none"
        },
        "description": "Disable request buffering for file uploads",
        "enabled": true,
        "expression": "starts_with(http.request.uri.path, \"/upload/\"",
        "id": "f2a88e88081c4a7e9ead5134de0a61c9"
      }
    ]
  },
  "success": true
}
```

 **Note:** Only requests to `uploads.example.com` are streamed directly between the customer and the origin server without request or response body buffering.

Configuration rules settings

You can configure the following settings in a configuration rule.

Automatic HTTPS Rewrites

It rewrites URLs from `HTTP` to `HTTPS` for resources that can be served over HTTPS.

Use this setting to enable or disable Automatic HTTPS Rewrites for requests that match the rule expression.

API example:

```
{
  "action_parameters": {
    "automatic_https_rewrites": true
  }
}
```

API configuration property name: `automatic_https_rewrites` (boolean).

Browser integrity check

Browser Integrity Check blocks requests based on HTTP headers that are commonly associated with malicious traffic.

Use this setting to enable or disable Browser Integrity Check for requests that match the rule expression.

API example:

```
{
  "action_parameters": {
    "bic": true
  }
}
```

API configuration property name: `bic` (boolean).

Email obfuscation

Email Obfuscation prevents spam by hiding email addresses from bots and harvesters while keeping them visible to human visitors to your site.

Use this setting to enable or disable Email Obfuscation for requests that match the rule expression.

API example:

```
$ "action_parameters": {  
  "email_obfuscation": false  
}
```

API configuration property name: `email_obfuscation` (boolean).

Hotlink protection

Hotlink Protection prevents other websites from embedding images hosted on your domain.

Use this setting to enable or disable Hotlink Protection for requests that match the rule expression.

API example:

```
$ "action_parameters": {  
  "hotlink_protection": true  
}
```

API configuration property name: `hotlink_protection` (boolean).

Under defense mode

Under defense mode performs additional security checks to help mitigate Layer 7 DDoS attacks. Validated users access your website and suspicious traffic is blocked.

Use this setting to enable or disable Under defense mode for requests that match the rule expression.

API example:

```
$ "action_parameters": {  
  "security_level": "under_defense"  
}
```

API configuration property name: `security_level` (string). API values: `off`, `essentially_off`, and `under_defense`.

Request body buffering

Use the Request body buffering setting to configure how CIS buffers HTTP request bodies for requests that match the rule expression.

The following values are supported:

- Standard (default): Allows CIS features, such as the WAF, to inspect a portion of the request body when required.
- Full: Buffers the entire request body before forwarding it to the origin server.
- None: Does not buffer the request body. The request body is streamed directly to the origin server without inspection.



Important: Setting the value to `none` can affect features that inspect request bodies, such as WAF and other security features. Use this option only when request body inspection is not required.

API example:

```
$ "action_parameters": {  
  "request_body_buffering": "full"  
}
```

API configuration property name: `request_body_buffering` (string). API values: `standard`, `full`, and `none`.

Response body buffering

Use the Response body buffering setting to configure how CIS buffers HTTP response bodies for requests that match the rule expression.

The following values are supported:

- Standard (default): Allows CIS features, such as WAF, to inspect a portion of the response body when required.
- None: Does not buffer the response body. The response body is streamed directly to the client without inspection.



Important: Setting the value to `none` can affect features that inspect response bodies, such as the Web Application Firewall (WAF) and other security features. Use this option only when response body inspection is not required.

API example:

```
$ "action_parameters": {  
  "response_body_buffering": "standard"  
}
```

API configuration property name: `response_body_buffering` (string). API values: `standard` and `none` .

SSL mode

Use the SSL setting to configure the SSL/TLS encryption mode for requests that match the rule expression. The SSL/TLS encryption mode determines how CIS connects to your origin server and validates the SSL certificates presented by the origin. For more information, see [SSL mode](#).

API example:

```
$ "action_parameters": {  
  "ssl": "flexible"  
}
```

API configuration property name: `ssl` (string). API values: "off", "flexible", `full` , `strict` , and `origin_pull` .

For complete API examples, see [Creatin a configuration rule via API](#) .

Working with origin rules

About origin rules

Origin rules allow you to modify how requests are sent to your origin server. You can use origin rules to override origin settings such as the destination host header, origin host, or destination port before traffic is forwarded to the origin.

In CIS, origin rules are supported through the Rulesets API by using the `http_request_origin` phase.

Rule execution order

Origin rules are evaluated in the Ruleset Engine during the `http_request_origin` phase. Ruleset Engine rules take precedence over Page Rules. If a request matches both a Page Rule and a Ruleset Engine rule, the Ruleset Engine rule is applied.

Creating a origin rules

Create a origin rule by adding it to the `http_request_origin` phase ruleset for a zone.

Before you begin

When creating a configuration rule with the API, make sure you:

- Set the rule action to `set_config` .
- Define the parameters in the `action_parameters` field according to the settings you wish to override for matching requests.
- Deploy the rule to the `http_request_origin` phase at the zone level.

Creatin a configuration rule with the API

Complete the following steps to create a orgin rule for a given zone via API:

1. Check whether a ruleset exists for the `http_request_origin` phase by using the [List zone rulesets](#) API.
2. If a ruleset for the `http_request_origin` phase doesn't exist, create one by using the **Create a zone ruleset** as shown in the following example.

Set the following values in the request body:

- kind: `zone`
- phase: `http_request_origin`

```
$ curl -X POST \
  "https://api.cis.cloud.ibm.com/v1/crn:v1:bluemix:public:internet-svcs:global:a/361583ba4e52947c3e111ba9d29702e3:78345201-a3a7-4de3-abe4-7115e6b0e2f1::/zones/2ce4a192bf5978e6b65a09ed2d36d4c2/rulesets" \
  --header "X-Auth-User-Token: Bearer xxxx" \
  --header "Content-Type: application/json" \
  --data '{
    "name": "Zone-level phase entryptoint",
    "kind": "zone",
    "description": "Config settings ruleset",
    "phase": "http_request_origin"
  }'

{"result": {"description": "Config settings ruleset", "id": "b19bf5e7c3bd40b890a0302390e86e0c", "kind": "zone", "last_updated": "2026-06-13T14:52:37.636034Z", "name": "Zone-level phase entryptoint", "phase": "http_request_origin", "version": "1"}, "success": true, "errors": [], "messages": []}%
```

3. Add a configuration rule to the ruleset by using the [Update a zone ruleset](#) API. Alternatively, include the rule in the Create a zone ruleset request mentioned in the step 2.

```
$ curl -X PUT \
  "https://api.cis.cloud.ibm.com/v1/crn:v1:bluemix:public:internet-svcs:global:a/361583ba4e52947c3e111ba9d29702e3:78345201-a3a7-4de3-abe4-7115e6b0e2f1::/zones/2ce4a192bf5978e6b65a09ed2d36d4c2/rulesets/b19bf5e7c3bd40b890a0302390e86e0c" \
  "Content-Type: application/json" \
  -H "Accept: application/json" \
  -H "X-Auth-User-Token: Bearer xxxxx" \
  -d '{
    "description": "My http_request_origin ruleset to execute managed rulesets",
    "kind": "root",
    "phase": "http_request_origin",
    "rules": [
      {
        "action": "set_config",
        "expression": "starts_with(http.request.uri.path, \"/upload/)",
        "description": "Disable request buffering for file uploads",
        "action_parameters": {
          "request_body_buffering": "none"
        }
      }
    ]
  }'

{"result": {"description": "My http_request_origin ruleset to execute managed rulesets", "id": "b19bf5e7c3bd40b890a0302390e86e0c", "kind": "zone", "last_updated": "2026-06-13T15:12:56.987507Z", "name": "Zone-level phase entryptoint", "phase": "http_request_origin", "rules": [{"action": "set_config", "action_parameters": {"request_body_buffering": "none"}, "description": "Disable request buffering for file uploads", "enabled": true, "expression": "starts_with(http.request.uri.path, \"/upload/)", "id": "f2a88e88081c4a7e9ead5134de0a61c9", "last_updated": "2026-06-13T15:12:56.987507Z", "ref": "f2a88e88081c4a7e9ead5134de0a61c9", "version": "1"}], "version": "2"}, "success": true, "errors": [], "messages": []}%
```

Origin rules settings

You can configure the following settings in an origin rule.

Host header

The host header setting allows you to rewrite the HTTP `Host` header of incoming requests. The Host header identifies the website or application that the request is intended for on the destination server.

Use this setting when your content is hosted on a third-party server that accepts only host headers that match its own server name. For example, you can update incoming requests from `Host: example.com` to `Host: thirdpartyserver.example.net`.

Considerations

- In most cases, when you rewrite the HTTP `Host` header, you must also configure a DNS record override. The host header override updates only the header value; the DNS record override handles the re-routing of the request.
- When an origin rule performs a host header override, the Server Name Indication (SNI) value of the original request is also updated to the

same value. To set an SNI value that is different from the host header value, add an SNI override in the same origin rule, or create a separate origin rule.

Server Name Indication

The Server Name Indication(SNI) setting allows you to override the SNI value of a request. SNI is an extension to the TLS protocol that enables the server to present the correct certificate for a given hostname during the TLS handshake.

Considerations

- The new SNI value must be a valid hostname on the same CIS account, though it can belong to a different zone.
- Only static values are supported when you override SNI.
- An SNI override takes precedence over SNI rewrites of custom origins when you use CIS for SaaS configurations.

DNS record

The DNS record setting allows you to override the resolved hostname of incoming requests, which determines which origin server receives the request. This is also referred to as a resolve override.

Use this setting when you serve an application from a specific URL path — for example, `mydomain.com/app` and the application is hosted on a different server or by a third party. A DNS record override redirects requests for that endpoint to the appropriate server.

Considerations

- You must specify a valid hostname in the DNS record override. The hostname must exist on the same CIS account, though it can belong to a different zone. You can configure a CNAME, A, or AAAA record that points to a third-party hostname or IP address, either proxied or unproxied.
- In most cases, when you configure a DNS record override, you must also configure a host header override. The DNS record override handles the re-routing of the request; the host header override updates the `Host` HTTP header value. Defining a host header override also updates the SNI value of the original request to the same value. To set a different SNI value, add an SNI override in the same origin rule or create a separate origin rule.

The following examples show DNS records that configure a `resolve.example.com` hostname to point to an external hostname and IP address.

DNS record type	Name	Target	IPv4 address	TTL	Proxy status
CNAME record	resolve.example.com	domain.s3.amazonaws.com	■	Auto	Proxied
A record	resolve.example.com	■	203.0.113.1	Auto	Proxied

DNS record type exmples

Destination port

The Destination port setting allows you to override the destination port of a request, redirecting incoming requests to a port that you specify.

For example, you can override the destination port for requests to `mydomain.com`, so that they are served by the application that runs on port 9000 (`mydomain.com:9000`).

The destination port must be between 1 and 65,535.

Working with page rules

Using page rules

A page rule specifies settings and values that you can apply to a specific URL pattern that references your domain. Page rules help you manage security, performance, and reliability based on each individual URL in your site. The following table describes the page rules that are available to customers, the behaviors they produce, and any special considerations you should keep in mind before you use them.

Security

Setting	Behavior	Considerations
---------	----------	----------------

Always Use HTTPS	Converts any <code>http://</code> URL to an <code>https://</code> URL by creating a 301 redirect.	Using this setting disables all other setting configurations for the rule because CIS forces a redirect to <code>HTTPS</code> for the request, which becomes a new request that is then evaluated against page rules.
Browser Integrity Check	Looks for common HTTP headers that are abused by spammers, and denies access to your page. It also blocks visitors that do not have a user agent, or add a non-standard user agent (also commonly used by abuse bots, crawlers, or APIs).	
Disable Security	Disables the following features: Email Obfuscation , Server Side Excludes and WAF (Web Application Firewall).	If a rule is set to disable security, and another rule is set to enable the WAF, the WAF rule takes precedence regardless of the order in which they appear.
Email Obfuscation	Toggles email obfuscation feature on or off. When the email obfuscation feature is turned on, email addresses on your web page are obfuscated (hidden) from bots, but visible to humans. There are no visible changes to your website for visitors.	
IP Geolocation Header	Includes the country code of the visitor location with all requests to your website. The information can be found in the <code>CF-IPCountry</code> HTTP header.	
Security Level	Controls how high a client threat score must be so that a client encounters a challenge page. Use this setting to present visitors with a Defense Mode challenge when they visit your site. deprecated	
Server Side Excludes	Toggles server-side excludes on or off. Server-side excludes gives you the ability to hide sensitive content from suspicious visitors, but keep it visible to normal visitors.	SSE only works with HTML.
TLS	Controls which TLS mode is used.	
WAF	Toggles WAF rules on or off.	Individual WAF rules can't be turned on or off using page rules.
Automatic HTTPS Rewrites	Toggles automatic HTTPS rewrites on or off. Automatic HTTPS rewrites safely rewrite HTML source links from HTTP to HTTPS.	
Opportunistic Encryption	Toggles opportunistic encryption on or off. Opportunistic encryption allows clients to use traditionally insecure protocols over secure channels.	
Cache Deception Armor	Toggles cache deception armor on or off. Cache deception armor protects from web cache deception attacks while still allowing static assets to be cached. This setting checks that the URL's extension matches the returned content type.	
True Client IP Header	CIS sends the user's IP address in the <code>True-Client-IP</code> header.	Enterprise only

Security rules

Reliability

Setting	Behavior	Considerations
---------	----------	----------------

Serve Stale Content	Keeps a limited version of the site online if the server goes down.	For more information, view Managing your CIS deployment for optimal reliability .
Origin Cache Control	Determines what content is cached from the origin and how often the content is updated.	For more information, view Managing your CIS deployment for optimal reliability .
Forwarding URL	URL to use in case the website is unavailable.	Using this setting disables all other setting configurations because you are forwarding the request somewhere else. For more information, view Managing your CIS deployment for optimal reliability .
Host Header Override	Replaces the host header for the URI matching the page rule to the value specified. This is commonly used for content that is hosted in an S3 bucket.	
Disable Apps	Turns off all CIS apps.	Enterprise only
Origin Error Page Pass-through	Disables CIS error pages that would trigger for issues that are sent from the origin server, and instead displays the error pages set at the origin.	Enterprise only

Reliability rules

Performance

Setting	Behavior	Considerations
Browser Cache TTL	Controls how long resources that are cached by client browsers remain valid.	
Bypass Cache on Cookie	Serves a cached object unless a cookie of a specific name is seen; for example, serve a cached version of the home page unless we see a SessionID cookie indicating that the customer is logged in and should be presented personalized content.	
Cache Level	Bypass - Resources that match that page rule are not cached. ■ No query string - Only delivers resources from cache when there is no query string. ■ Ignore query string - Delivers the same resource to everyone independent of the query string. ■ Standard - Delivers a different resource each time the query string changes. ■ Cache everything - Resources that match the page rule are cached.	By default, HTML content is not cached. A page rule to cache static HTML content must be written.
Edge Cache TTL	Controls how long CIS retains files in the cache.	This setting is optional when specifying cache level.
Resolve Override	Changes the URL or IP that the request matching the page rule resolves to.	
Cache on Cookie	Applies the Cache Everything option (Cache Level setting) based on a regular expression match against a cookie name. If you add both this setting and Bypass Cache on Cookie to the same page rule, Cache On Cookie takes precedence over Bypass Cache on Cookie .	Enterprise only
Disable Performance	Turns off Minify Web Content , Image Load Optimization , Image Size Optimization , and Script Load Optimization .	Enterprise only

Minify Web Content	Minifies HTML, CSS and/or JavaScript files by removing all unnecessary characters from these files.	Enterprise only
Image Load Optimization	Improves load time for pages that include images based on network connection and device type by one of the following: ■ Image Virtualizing - Replaces images with low-resolution placeholder images that have the same dimensions as the original (including third-party images). After the page renders completely, full resolution images are then lazy-loaded (prioritizing images in the browser viewport). This process allows pages to render quickly and minimizes browser reflow. ■ Request Streamlining - Combines multiple individual network requests for images into a single request.	
Image Size Optimization	Reduces image file size by removing metadata (date and time, camera manufacturer, model, and so on), and by compressing images when possible. Smaller file sizes mean faster load times for images and web pages.	
Sort Query String	Treats files with the same query strings as the same file in cache, regardless of the order of the query strings.	Enterprise only
Response Buffering	Enables or disables buffering of responses from the origin server. By default, CIS sends packets to the client as we receive them. Enabling Response Buffering means that CIS waits until it has the entire file before forwarding it to the user.	Enterprise only
Script Load Optimization	Improves paint times by asynchronously loading your JavaScripts, including third-party scripts so that they do not block rendering the content of your pages.	

Performance rules

Page rule URL patterns

A page rule takes effect on a specific URL pattern, matching the following format:

```
<scheme>://<hostname><:port>/<path>
```

An example that uses each component would be:

```
https://www.example.com:80/image.png
```

The *scheme* and *port* components are optional. If the scheme is omitted, it covers both `http://` and `https://` protocols. If the port is not specified, then the rule matches all ports. You can perform basic wildcard matches by using an asterisk (`&asterisk;`) symbol in your rule pattern, allowing it to match a series of similar patterns rather than just one.

Here are three important things to remember with page rules:

- Only one page rule takes effect on any given request.
- Page rules are given priority in order from top to bottom.
- After a URL matches a rule, only that rule is applied; that is, if a page rule was already triggered on a request, any subsequent rules that also match the URL pattern do not take effect. Generally, order your rules from *most specific* to *least specific*.

Page rules can be disabled, in which case they take no action. You can still see them in the list and edit them. Setting the **Enabled** toggle to **Off** creates a page rule that initially is disabled.

For more information, see [Using page rules with caching](#).

Using page rules with caching

Page rules give you the ability to take various actions based on the page's URL, such as creating redirects, fine-tuning caching behavior, or enabling and disabling services.

A page rule takes effect on a URL pattern that matches the following format:

```
<scheme>://<hostname><:port>/<path>
```

For example,

https://www.example.com:80/image.png

The `scheme` and `port` components are optional. If the `scheme` component is omitted, the format accepts `http://` and `https://` protocols. If the `port` is not specified, the rule matches all ports. You can perform basic wildcard matches by using a `*` symbol in your rule pattern, allowing it to match a series of similar patterns.

Page rule considerations

- Only one page rule takes effect on any request.
- Page rules get priority in an order from top to bottom. After a URL matches a rule, only that rule is applied. In other words, if a page rule triggered already on a request, any subsequent rules that also match the URL pattern do not take effect.
- Generally, order from most-specific to least-specific rules.
- Page rules can be disabled, in which case they take no action. However, you can still see the rules in the list and edit them. Setting the **Enabled** toggle to **Off** creates a page rule that is disabled initially.

Forwarding (URL redirection)

Redirect one URL to another using an HTTP 301 or 302 redirect. The contents of any section of a URL that a wildcard matches can be referenced using `$X` syntax. The `X` indicates the index of a glob in the pattern: `$1` is replaced with the first wildcard match, `$2` with the second wildcard match, and so on.

For example, suppose you set the following rule where:

- URL match = `*.example.com/*`
- Enabled = On
- Setting = Forwarding URL
- Set as = 301 Permanent Redirect to `http://example.com/$2/`

Here, a request to `www.example.com/stuff/things` is redirected to `http://example.com/stuff/things`.



Note: Be careful not to create a redirect in which the domain points to itself as a destination. This mistake can cause an infinite redirect error, and the affected URLs do not resolve.

Redirecting to HTTPS

If you want to redirect your visitors to use HTTPS, set the rule behavior to the **Always Use HTTPS** setting instead.

Setting custom caching

Set caching behavior for any URL matching the page rule pattern, using any of the standard cache levels. Setting **Cache Level** to **Cache Everything** caches any content, even if it is not one of the default static file types. Setting **Cache Level** to the **Bypass** setting prevents caching on that URL.

When you specify a cache level by using page rules, you can set an **Edge Cache TTL**, which controls how long CIS retains files in the cache.

Browser Cache TTL controls how long resources that are cached by client browsers remain valid. If a browser requests a resource again and the TTL isn't expired, the browser receives an `HTTP 304 (Not Modified)` response. You can set TTLs ranging from 30 minutes to 1 year.

Not all default caching behaviors are strictly RFC-compliant. Setting **Origin Cache Control** through page rules uses a newer set of caching rules that seeks to adhere more closely to RFCs, primarily regarding revalidation. For example, our default behavior with `max-age=0` is to not cache at all, whereas setting **Origin Cache Control** caches, but it always revalidates.

The following example sets a page rule to cache everything found in the `/images` folder. Cached resources expire in 30 minutes in the user's browser, and they expire after one day in the IBM CIS data centers.

- URL match = `*.example.com/images/*`
- Enabled = On
- Rule behavior settings:

Setting	Set as
Cache level	Cache everything

Browser Cache TTL	30 minutes
Edge Cache TTL	1 day

Rule behavior settings to cache everything in /images

Serve Stale Content serves pages from our cache, even when your server goes down. Visitors see a limited version of your site, with a message that they are in offline browsing mode.

This feature returns an HTTP status 503. When servers are online again, CIS seamlessly takes visitors to regular browsing.



Note: If the requested page is not in the cache, the visitor sees an error page that informs them the page they are requesting is offline. If a **Cache Everything** page rule is enabled with expiration times set lower than the caching frequency, the **Serve Stale Content** is purged in the corresponding interval.

Using resolve override with Cloud Object Storage

The following use case makes the request matching the page rule resolve to a COS bucket resource.

Prerequisites

The following steps assume you have an existing COS instance and bucket with public access. For information on public access see [Allowing public access](#).

Proxy DNS entries are required for domains that are to be matched by the page rule

To perform the actual rewrite and redirection to the COS bucket, the domains that you want to use with this page rule (such as `www.foo.com`) must have DNS entries in CIS with the `proxy` flag set. See [Proxying DNS Records](#) for more information. If all requests to `www.foo.com` are redirected, then a CNAME entry that points to `<bucket-name>` with proxy enabled is sufficient.

Create page rule steps

- Navigate to **Performance > Page Rules**.
- Click **Create rule**.
- Input the value you want for the URL match. For example, `*.foo.com/*`.
 - The URL match must be the same as your COS object name. For example, if you have an object called `reports.txt` under the bucket `my-bucket1`, then both of these URL matches would be valid:
 - `*.foo.com/*`
 - `*.foo.com/reports.txt`
- Use the list menu to select **Resolve Override with COS** under the **Performance** section.
- Use the **Cloud Object Storage Instance** list to select the instance you want.
- Use the **Bucket** list to select the bucket you want.
- Click **Create** to complete the rule.

Editing a page rule

After editing the page rule, the **Resolve Override with COS** is no longer displayed on the page. However, the **Resolve Override** `<bucket>` `<domain>` and **Host Header Override** `<bucket>.<cos-endpoint>` replace **Resolve Override with COS**.

Making changes to the **Resolve Override** does *not* automatically create a new CNAME record (for example, `<updated-bucket>.<domain>`). This is only done upon the initial creation of a page rule using **Resolve Override with COS**. To automatically create the CNAME record for the bucket follow the [Create page rule](#) steps.

Deleting the page rule

If the Resolve Override with COS page rule is no longer needed, the CNAME should be *manually* deleted along with the page rule.

What happens in the background

When creating a **Resolve Override with COS** page rule, CIS automatically creates the other necessary resources for the COS integration. These include:

■ CNAME

- A CNAME DNS record for `<bucket-name>.<cos-endpoint>` as `<bucket-name>` .
- For example, if your CIS domain is `foo.com` , you have a COS bucket called `images` and your public COS endpoint is `s3.us-west.objectstorage.uat.test.net` then CIS creates a CNAME as `images.foo.com` that points to `images.s3.us-west.objectstorage.uat.test.net` .



Note: If the Resolve Override with COS page rule is no longer needed the CNAME should be manually deleted along with the page rule.

■ Host Header Override

- The Host Header Override setting replaces the host header for the URI matching the page rule to `<bucket-name>.<cos-endpoint>` .
- Using the previous example, the Host Header Override value is set to `images.s3.us-west.objectstorage.uat.test.net` .

Working with global load balancers

About global load balancers

Traffic is routed to the highest-priority healthy origin pool and automatically fails over to secondary pools when needed to ensure continuous application availability. When pools are associated with specific regions, traffic is directed to the nearest or most appropriate regional pool. If no regional pools are available, traffic falls back to the default pool.

You can configure global load balancing to:

- Distribute traffic across multiple regions
- Provide failover between application environments
- Route users to regional origin pools
- Improve application availability and throughput
- Optimize traffic routing based on steering policies

CIS global load balancing consists of the following core components:

- **Global load balancers:** Manage DNS-based traffic routing across configured origin pools.
- **Origin pools:** Groups of origin servers that receive application traffic. Pools can contain public IP addresses, hostnames, or load balancer virtual IPs. For more information, see [Origin pools](#).
- **Health checks:** Monitor the health and availability of origins by sending periodic probes to configured endpoints. When an origin or pool becomes unhealthy, traffic is routed to another healthy pool. For more information, see [Health checks](#).
- **Traffic steering policies:** Control how traffic is distributed across origin pools. Supported steering modes include:
 - Off - Failover
 - Geo
 - Random
 - Dynamic latency

For more information, see [Traffic steering](#).

When regional pools are configured, CIS routes traffic according to regional proximity and availability. If no regional pools are available, traffic falls back to the default pool.

CIS global load balancing supports both proxied and non-proxied configurations:

- In non-proxied mode, CIS returns origin IP addresses directly through DNS resolution.
- In proxied mode, CIS terminates client connections and establishes a separate connection to the origin server. This configuration enables additional CIS capabilities such as TLS management, security inspection, caching, and firewall protection.

Use cases: Red Hat OpenShift route failover

You can use global load balancing to provide failover and disaster recovery (DR) between Red Hat OpenShift clusters that are deployed across multiple regions.

In this example, applications are deployed in two Red Hat OpenShift clusters:

- A primary cluster in Washington
- A secondary cluster in Dallas

Both clusters expose the same applications through Red Hat OpenShift routes. IBM Cloud Internet Services global load balancing routes traffic to the primary cluster during normal operations and fails over traffic to the secondary cluster during a DR event.

Creating a global load balancer for Red Hat OpenShift route failover

This workflow configures CIS global load balancing for Red Hat OpenShift routes by using failover traffic steering.

Follow this workflow to configure global load balancing for Red Hat OpenShift route failover:

1. Create two origin pools.
 - Configure the first origin pool for the Washington primary Red Hat OpenShift route.
 - Configure the second origin pool for the Dallas secondary Red Hat OpenShift route.
2. Create a global load balancer and set the traffic steering mode to **Off - Failover**.

3. Configure the Washington origin pool with higher priority.
4. Configure the Dallas origin pool as the fallback pool.

With the **Off - Failover** traffic steering mode:

- Traffic is routed to the primary origin pool during normal operations.
- Traffic fails over to the secondary origin pool when the primary pool becomes unhealthy.
- If all pools become unhealthy, traffic is routed to the configured fallback pool.

Working with load balancers and existing DNS records

A IBM Cloud® Internet Services load balancer is identified by the DNS hostname whose traffic you want to balance (for example, `www.example.com`). The load balancer defines which origin server pools to use, the order in which they are used, and how to geographically distribute traffic among pools.

When you create a load balancer on CIS, you can do one of the following:

- Use a unique hostname with no existing DNS record
- Use a hostname that has an existing DNS record (an A, AAAA, or CNAME record)

If a hostname has an existing DNS record, the load balancer supersedes the DNS record if the DNS record is an A, AAAA, or canonical name (CNAME) record. It does not supersede other record types.

If the load balancer is disabled, preexisting DNS records are served.

If there is no preexisting DNS record with the same name, disabling the load balancer prevents clients from resolving the host, and requests fail.

If a load balancer is manually disabled, traffic is not served to the associated origins or the fallback. If all pools in a load balancer are manually disabled, the fallback origin is used unless the fallback is also disabled. If the fallback is also disabled, the SOA record is served. Otherwise, if there is one pool in the load balancer and the pool is unhealthy, CIS sends traffic to the fallback pool regardless of the fallback pool's health.

HTTP keep-alive (persistent HTTP connection)

CIS maintains keep-alive connections to improve performance and to reduce the cost of recurring TCP connects in the request transaction as CIS proxies customer traffic from its edge network to the site's origin.

Ensure that HTTP keep-alive connections are enabled on your origin. CIS reuses open TCP connections for up to 15 minutes (900 seconds) after the last HTTP request. Origin web servers close TCP connections if too many are open. HTTP keep-alive helps avoid the premature reset of connections for requests that are proxied by CIS.

Session cookies

Configure [Session Affinity](#) to parse HTTP requests by cookie header when you use HTTP cookies to track and bind user sessions to a specific server. Doing so directs each request to the correct application server even when HTTP requests share a TCP connection due to keep-alive.

Setting up origin pools

An origin pool is a group of origin servers that receive traffic from a global load balancer. Origins can be identified by IP address or hostname, and pools can be associated with a specific region or made available globally.

You can configure the minimum number of healthy origins that must be available for a pool to be considered healthy, as well as the health check used to monitor the pool.

If you're familiar with DNS terminology, think of a pool as a record set, except that only healthy origins are returned. You can configure multiple pools and define their failover priority.

- When adding origin servers to a pool, you can identify the origin by hostname or IP address.
- The order of pools in the load balancer determines the standard failover priority. When the number of healthy origins in a pool drops below the configured threshold, the load balancer routes traffic to the next available pool.
- By default, pools are ordered by date created. You can reorder them from the global load balancer dashboard and by using CIS API (use the Update Pools command to set a new `origins` array).
- Dynamic steering uses Round Trip Time (RTT) profiles to determine pool priority. If there is no RTT data for a pool in a region or colocation center, the load balancer uses pool order to determine failover priority.

- Geo steering directs traffic to pools based on the client’s region or point of presence. If there is no geo steering configuration for a region or pool, the load balancer uses pool order to determine failover priority.

Distribution of traffic within a pool

By default, traffic is distributed evenly among the origins in the pool using a round-robin algorithm. This is also true for non-proxied global load balancers.

The origins can be configured with weights, and for proxied global load balancers, the weights determine how much traffic each origin server receives relative to other origins in the pool. Weights are configured as decimal values between 0 and 1, and specify what fraction of traffic goes to the origin.

For each origin:

Percent of traffic to the origin = origin weight / sum of all origin weights

If all origins have weight **1**, traffic is distributed evenly.

Origins with weight **0** do not receive any traffic for this pool. However, session affinity might still override this until all sessions are closed. If the origin is a member in another pool, it might still receive traffic for that pool.

For example, an origin pool is set up with 3 origins that have the following weights: origin-A: 0.4, origin-B: 0.3, and origin-C: 0.3.

- Initially, all origins are healthy. The amount of traffic each origin receives is: origin-A: 40%, origin-B: 30%, and origin-C: 30%.
- Then origin-A turns critical and it no longer receives traffic. The remaining origins have the same weight, and so traffic is distributed evenly, each receiving 50%.
- The administrator changes the weight for origin-C to **0**. Now 100% of new traffic goes to origin-B. With session-affinity turned on, traffic for existing sessions on origin-C continues to go to origin-C until those sessions close (for a maximum of 24 hours).
- Finally, the weight for origin-B is changed to **0**. The pool no longer receives traffic until origin-A returns to a healthy status or the weights for origin-C and origin-B are set to a non-zero value.

Fallback pool

A fallback pool is the lowest-priority pool in a load balancer configuration. It receives traffic when higher-priority pools are unavailable and can also receive traffic during normal operation depending on the selected traffic-steering mode. It is not limited to error handling.

Traffic distribution to a fallback pool depends on the configured traffic-steering mode and the priority that is assigned to each pool. Under certain configurations, the fallback pool can receive traffic even when other pools are healthy.

The fallback pool is defined as the origin pool with the lowest priority, which is represented by the highest numerical value. This designation determines its role when other pools become unavailable. When all primary pools for a region are unavailable, traffic is directed to the fallback pool. This routing occurs regardless of the fallback pool’s health state.



Note: If all pools are disabled, the fallback pool is not available.

Setting up health checks

A health check helps gain insight into the availability of pools so that traffic can be routed to the healthy ones.

These checks periodically send HTTP, HTTPS, or TCP requests and monitor the responses. They can be configured with a customized port, interval, timeout, status code, and more. As soon as a pool is marked unhealthy, traffic is intelligently rerouted to another available pool.



Note: Be aware that your logs have references to Cloudflare because IBM is partnering with Cloudflare to provide CIS.

CIS health checks track the health of pools. They are configured through monitors, which define what type of health check to run and how frequently to run them. CIS monitors your servers from each of the IBM data centers.

Health checks that result in a status change for an origin server are recorded as events in the load balancer event logs. You can create, attach, and configure health checks from either the global load balancer dashboard or the CIS API.

Creating and attaching a health check in the console

To set up a health check, create a monitor and attach it to a load balancer pool.

Before you begin

Make sure that you meet the following prerequisites:

- A CIS instance is provisioned.
- At least one load balancer pool exists.
- Origin servers are added to the pool.

To create and attach a health check in the console, follow these steps:

1. In the CIS console, select your instance.
2. Go to **Load Balancing > Monitors**.
3. Click **Create monitor**.
4. Enter a **Name** for the monitor.
5. Select a **Type**:
 - HTTP
 - HTTPS
 - TCP
 - ICMP
6. Configure the monitor settings based on the selected type:
 - Port (if different from the default)
 - Path (for HTTP or HTTPS)
 - Method (GET or HEAD)
 - Expected status codes
 - Interval
 - Timeout
 - Retry count
7. Click **Create**.
8. Go to **Load Balancing > Pools**.
9. Select the pool that you want to monitor, then click **Edit**.
10. In the **Health monitor** section, select the monitor that you created.
11. Click **Save**.

After the monitor is attached, CIS begins checking the health of each origin server in the pool. If an origin fails the configured number of health checks, it is marked unhealthy and traffic is routed to healthy origins based on your traffic steering configuration.

Health check considerations

- Availability monitoring checks the health of origin servers every 15 seconds. It reports results using email notifications and the CIS API.
- The default retry rate is 5 retries per second, and is configurable. It is not recommended to increase the retry rate significantly. Retries use exponential backoff (1, 2, 4, 8, and 16 seconds, by default).
- You can configure monitoring for specific URLs by sending periodic HTTP requests to the load balancer, taking advantage of customizable intervals, timeouts, and status codes. After an origin server is marked unhealthy, multi-region failover reroutes traffic to the next available server in failover order.
- Load balancer monitors use the following HTTP user-agent: `"Mozilla/5.0 (compatible; Cloudflare-Traffic-Manager/1.0; +https://www.cloudflare.com/traffic-manager/; pool-id: $poolid)"`. The `$poolid` contains the first 16 characters of the load balancer pool that is the target of the health check.
- To prevent health checks from failing, and to secure user infrastructure against spoofed checks from bad actors, do the following:
 - Only accept connections to hosts listed in the CIS IP ranges in your firewall or web-server.
 - Use the CIS user agent to reject HTTP requests that don't come from these ranges.
 - Ensure that your firewall or web server does not block or rate limit CIS health checks.

Health check events

Health Check Events are status changes from pools with connected health checks and their associated origin servers. If an origin's status degrades, a new item appears in a table, with the event's description.

Monitoring health check failures with alert policies

Enterprise plan users can configure alert policies to receive notifications when load balancer health checks fail.

To create an alert policy:

1. From the CIS console, go to **Account > Alerts**.
2. In the Alerting policies view, click **Create** to create an alert policy.
3. Select the appropriate load balancer health metrics.
4. Configure notification channels.

For more information, see [Configuring CIS alert policies](#).

Supported health check types

Type	Description
HTTP	Sends HTTP requests and validates response codes and content.
HTTPS	Sends HTTPS requests and validates TLS-secured responses.
TCP	Verifies that a TCP connection can be established.
ICMP (Ping)	Verifies host reachability using ICMP echo requests.

Supported health check types

Depending on the selected monitor type, additional configuration options might be available, such as:

- HTTP method (GET, HEAD)
- Expected response codes
- Request path
- Host header override
- Timeout
- Interval
- Retry count

Choose the monitor type that best matches your application protocol and availability requirements.

Interaction between health checks and fallback pools

Health check status determines whether a pool is considered healthy or unhealthy. However, health status alone does not determine whether traffic is routed to a pool.

Pool selection also depends on:

- The configured traffic-steering mode
- Pool priority
- Whether region-specific or default pools are defined

A fallback pool can receive traffic even when all other pools are healthy, depending on the traffic-steering configuration.



Note: Enabling a pool as healthy does not guarantee that it is selected over a fallback pool unless the traffic-steering configuration enforces ordered failover.

Configuring traffic steering

Load balancing provides several traffic steering modes, which allow customers to optimize how load balancers route traffic.

You can configure traffic steering from the load balancing dashboard, in the **Edit Load Balancer** panel. You can also configure traffic steering via the CIS API. Available steering options include standard failover (steering disabled), dynamic steering, and geo steering.

Standard failover

Failover (Off) routes traffic to the highest-priority healthy pool. When a pool becomes unhealthy, traffic fails over to the next healthy pool according to pool priority.

If all primary pools become unhealthy, traffic is directed to the configured fallback pool.

In this mode:

- Traffic is sent to the highest-priority healthy pool.
- Lower-priority pools are used only when higher-priority pools become unhealthy.
- The fallback pool receives traffic only when all other pools are unhealthy.

This configuration is useful when you want strict active-passive failover behavior.

Dynamic steering

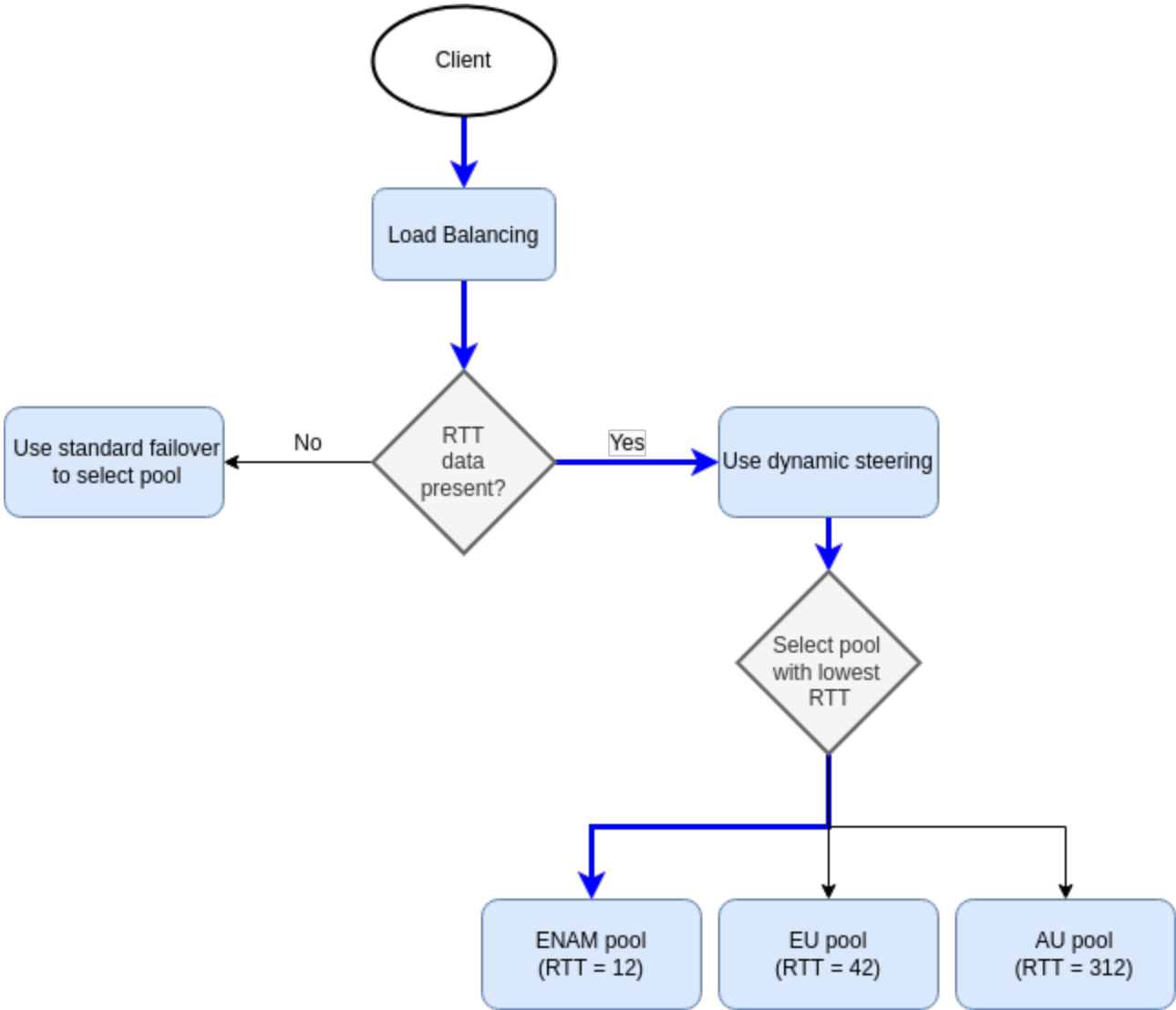
Dynamic steering uses health check data to identify the fastest pool for a CIS region or point of presence (PoP).

Dynamic steering creates Round Trip Time (RTT) profiles based on an exponential weighted moving average (EWMA) of RTT to determine the fastest pool. If there is no current RTT data for your pool in a region or colocation center, CIS directs traffic to the pools in failover order.

Allow 10 minutes for CIS to build an RTT profile the first time you enable dynamic steering for a server pool.

If you are terminating TCP at a cloud provider edge location, the calculated latency might not reflect the true latency to the origin for TCP health checks.

The following diagram shows how CIS routes traffic to the pool with the lowest EWMA among three regions: Eastern North America, Europe, and Australia. In this case, the ENAM pool is selected, because it has the lowest RTT.



Traffic steering in CIS

Geo steering

Geo steering directs traffic to pools based on the client’s region or PoP. Only domains on Enterprise plans can perform geo steering by PoP. Users specify the pools to which the load balancer directs traffic for a geographical region or PoP. You can assign multiple pools to the same region, and the load balancer uses them in failover order. If there is no configuration for a region or pool, the load balancer uses the default

failover order.

Cloudflare defines 13 geographic regions for geo steering. The region of a client is determined by the region of the Cloudflare data center that answers the client’s DNS query. These regions are listed in the following table, along with their region codes.

Region Code	Region
WNAM	Western North America
ENAM	Eastern North America
WEU	Western Europe
EEU	Eastern Europe
NSAM	Northern South America
SSAM	Southern South America
OC	Oceania
ME	Middle East
NAF	Northern Africa
SAF	Southern Africa
SAS	Southern Asia
SEAS	Southeast Asia
NEAS	Northeast Asia

Geographic regions and codes

Traffic steering modes and fallback behavior

Traffic steering determines how incoming traffic is distributed across origin pools. The selected steering mode affects how healthy pools and fallback pools are used during request routing.

The following table describes how traffic is routed for each steering mode and how fallback pools participate in traffic distribution.

Traffic steering mode	Traffic distribution behavior	Fallback pool behavior
Failover (Off)	Traffic is routed to the highest-priority healthy pool. Lower-priority pools are used only if higher-priority pools become unhealthy.	The fallback pool receives traffic only when all higher-priority pools are unhealthy.
Random	Traffic is distributed randomly across healthy pools.	The fallback pool receives traffic even when other pools are healthy.
Dynamic	Traffic is routed to the pool with the best observed latency.	The fallback pool participates in traffic steering when it is healthy.
Geo steering	Traffic is routed according to the configured regional or PoP mapping.	The fallback pool participates in traffic steering when it is healthy.

Traffic steering modes and fallback behavior

Example: fallback pool behavior

You configure the following pools:

Pool	Priority	Purpose
Pool A	1	Primary application
Pool B	2	Maintenance page (fallback pool)

Fallback pool example

Behavior depends on the selected traffic steering mode:

- If traffic steering is set to **Random**, traffic can be routed to **Pool B** even when **Pool A** is healthy.
- If traffic steering is set to **Failover (Off)**, traffic is routed to **Pool B** only when **Pool A** becomes unhealthy.

With Random steering, the maintenance page can receive production traffic even when Pool A is healthy.

This behavior makes sure that the selected steering mode aligns with your availability and traffic distribution requirements.

Setting proxy modes

Load balancers support DNS-only and HTTP proxy modes. You can have HTTP Proxy and DNS-Only domains in the same load balancer region, but the traffic routing behavior differs as follows:

- Traffic for domains using HTTP Proxy mode is routed based on the data center associated with the user making the request.
- Traffic for domains using DNS-Only mode is routed based on the data center associated with the user’s recursive resolver (DNS recursor).

HTTP Proxy mode

In HTTP Proxy mode, load balancers have an automatic TTL. CIS announces IBM IP addresses externally, but protects (masks) your origin server IP addresses. When proxied through CIS, origin IP addresses should be public. Any changes to your load balancer propagate within seconds inside CIS, including any failover events.

Setting the load balancer to HTTP Proxy mode offers the following benefits:

- Failover is faster, because external DNS caches that don't respect short DNS TTLs do not impact failover performance.
- The "automatic" TTL (five minutes) reduces the number of authoritative queries made against CIS without impacting failover performance.

DNS-Only mode

In DNS-Only mode, you can configure load balancers to set a TTL from 30 seconds to 10 minutes. CIS serves the addresses of the healthy origin servers directly, but relies on DNS resolvers respecting the short TTL to re-query the CIS DNS for an updated list of healthy addresses.

Setting session affinity

Loading a website usually requires fetching multiple assets from a web server. CIS session affinity minimizes redundant network requests by automatically directing requests from the same client to the same origin web server.

CIS sets a cookie on the initial response to the client. Using the cookie in subsequent client requests ensures those requests are sent to the same origin, unless the origin is unavailable.

When enabled, CIS session affinity does the following:

- When a client makes its first request, CIS sets a **CFLib** cookie on the client. The cookie encodes the origin to which the request is forwarded.
- Subsequent requests by the same client are forwarded to that origin for the duration of the cookie and as long as the origin server remains healthy.
- If the cookie expires or the origin server is unhealthy, CIS sets a new cookie encoding the appropriate failover origin.

All sessions default to 23 hours unless a custom session TTL is specified (in seconds) between 30 minutes and 7 days. A session affinity cookie

is required to honor the TTL. The session cookie is secure when [Always Use HTTPS](#) is enabled. Additionally, `HttpOnly` is always enabled for the cookie to prevent cross-site scripting attacks.

Setting session affinity from the CLI

When you create a global load balancer from the CLI, take the following steps to set the session affinity:

1. Log in to your IBM Cloud account.
2. Create a global load balancer.
3. Set the following CLI variables:



Note: If you require a specific SameSite configuration in your session affinity cookies, CIS recommends that you provide values for `samesite` and `secure` different from `Auto`, instead of relying on the default behavior. This way, the value of the SameSite cookie attribute does not change due to configuration changes (namely **Always Use HTTPS**).

`session_affinity`: Valid values are `cookie`, `none`.

`ttl`: Time, in seconds, until this load balancer's session affinity cookie expires after being created. Valid values between `1800`, `604800`. Default is `82800`.

`session_affinity_attributes`: Cookie attributes for a session affinity cookie.

samesite

Valid values are:

- `Auto` (default): If **Always Use HTTPS** is enabled, session affinity cookies use `Lax` mode; if disabled, cookies use `None` mode.
- `None`: Cookies are sent with all requests.
- `Lax`: Cookies are sent only to the apex domain (such as `example.com`).
- `Strict`: Cookies are created by the first party (the visited domain).

secure

Valid values are:

- `Auto` (default): If **Always Use HTTPS** is enabled, session affinity cookies use `secure` in the `samesite` attribute; if disabled, cookies don't use `secure`.
- `Always`: `secure` is always set, meaning the cookie is only sent over HTTPS connections.
- `Never`: `secure` is never set, allowing cookies to be sent over both HTTPS and HTTP connections.

drain_duration

Optional. Time, in seconds, where the origin will drain active sessions. After the time elapses, all existing sessions are ended, This field is only used when session affinity is enabled on the load balancer.

zero_downtime_failover

Optional. Automatically sends traffic to endpoints within a pool during transient network issues. Value values are:

- `none` (default): No failover takes place and errors might show to your users.
- `temporary`: Traffic is sent to other endpoints until the originally pinned endpoint is available.
- `sticky`: The session affinity cookie is updated and subsequent requests are sent to the new endpoint moving forward as needed.



Note: Sticky Zero-Downtime Failover is not supported for session affinity by HTTP header.

For example:

```
ibmcloud cis glb-update fc72db47cee8290eaef292cda6e1619a 12b68758126546e0d129c7bbadfa87f0 --json
'{"session_affinity":"cookie","session_affinity_ttl":1800,"session_affinity_attributes":{"samesite":"Auto","secure":"Auto"}}'
```

Setting session affinity with the API

Session affinity is a property of global load balancers, which you can set with the following endpoints:

- [Create a load balancer](#)
- [Edit a load balancer](#)

Customize the behavior of session affinity by using the `session_affinity`, `session_affinity_ttl`, and `session_affinity_attributes` parameters.

To enable session affinity by HTTP header, set the `session_affinity` value to `header` and add your HTTP header names to `session_affinity_attributes.headers`.

When you create a global load balancer using the API, take the following steps to set the session affinity:

1. Set up your API environment with the correct variables.
2. Store the following values in variables to be used in the API command:
 - `session_affinity`, which specifies the type of session affinity the load balancer uses, unless specified as "none" or "default". Valid values are none, `cookie`, `ip_cookie`.
 - `session_affinity_ttl`, which is the time-to-live of the session affinity.
 - `session_affinity_attributes` which include:
 - `samesite` configures the SameSite attribute on the affinity cookie. Valid values are `Auto`, `Lax`, `None`, `Strict`; default `Auto`.
 - `secure` configures the Secure attribute on the session affinity cookie. Valid values are `Auto`, `Always`, `Never`; default `Auto`.
 - `drain_duration` is the value of the drain duration, in seconds.
 - `zero_downtime_failover` (optional) automatically sends traffic to endpoints within a pool during transient network issues. Value values are `temporary`, and `sticky`; default `none`.



Note: Sticky Zero-Downtime Failover is not supported for session affinity by HTTP header.

3. When all variables are initiated, create the global load balancer with session affinity:

```
curl -X POST https://api.cis.cloud.ibm.com/v1/crn/zones/:zone_id/load_balancers -H 'content-type: application/json' -H 'x-auth-user-token: Bearer xxxxxx' -d '{
  "description": "",
  "proxied": true,
  "enabled": true,
  "name": "www.example.com",
  "session_affinity": "none",
  "session_affinity_ttl": 5000,
  "steering_policy": "geo",
  "fallback_pool": "4112ba6c2974ec43886f90736968e838",
  "default_pools": [
    "6563ebae141638f92ebbd4a821bef8c",
    "4112ba6c2974ec43886f90736968e838"
  ],
  "pop_pools": {},
  "region_pools": {
    "EEU": [
      "4112ba6c2974ec43886f90736968e838"
    ],
    "ENAM": [
      "6563ebae141638f92ebbd4a821bef8c"
    ],
    "WEU": [
      "4112ba6c2974ec43886f90736968e838"
    ],
    "WNAM": [
      "6563ebae141638f92ebbd4a821bef8c"
    ]
  }
}
```



Note: If you set `samesite` to `None` in your API request, you can't set `secure` to `Never`.

Creating global load balancers

If you have an e-commerce website or host an application that must remain accessible, you're likely concerned about its availability and performance.

Global load balancer capabilities available with IBM Cloud Internet Services (CIS) help improve application reliability and scalability while delivering the best possible user experience.

A global load balancer distributes user traffic across multiple geographic regions to improve availability and performance.

You can configure a global load balancer by using the console, CLI, API, or Terraform.

Before you begin

- Provision a CIS instance.
- Configure your domain in CIS.
- Identify the IP addresses or FQDNs for your application origins.



Important: CIS can support load balancer pools that use a private IP address. However, private IP pools don't support proxy services or health checks; only DNS-based load balancing is supported. For pools with public IPs, health checks can be configured as described in ([Step 1: Create a health check](#)).

Quick setup

If you already know your pools and health checks, you can create them inline when you create a load balancer:

1. In the CIS console, click **Reliability > Global load balancers > Origin pools** , then click **Create**.
2. Input a pool name, origins, and select a health check.

You can create a new health check here by selecting **Create new** in the Health Check list menu and completing the information.

3. Click **Save**.

The following steps provide the full procedure for creating health checks, pools, and a global load balancer.

Creating a global load balancer in the console

When you open the Global load balancers page (**Reliability > Global load balancers**), the page shows tabs for *Load balancers*, *Origin pools*, *Health checks*, and Events. These tabs display global load balancers and their components as you create or update them.

In the following step-by-step procedure, learn how to configure a setup similar to the following diagram. In this example, application resources are deployed in two data centers (US West and US East), with users accessing the application globally. Users might be accessing this application from all over the world.

To create and configure a global load balancer in the console, follow these steps:

Step 1: Create a health check (optional)

Health checks are optional attachments for origin pools. They run at a configurable interval to check for a specific response body or status code and monitor pool health. After you create a health check, you can add it to a new or existing origin pool.



Note: If you don't define any custom health checks, CIS uses `/` as your default health check path.

1. In the CIS console, click **Reliability > Global load balancers > Health checks** , then click **Create**.
2. Configure the following fields:
 - **Name:** Name of the health check.
 - **Monitor type:** The protocol to use for the health check. (Default: `HTTP`).
 - **Port:** Click the arrows to increase or decrease the port number.
 - **Path:** The endpoint path against which to perform the health check. (Default: `/`)
3. Expand **Advanced options** to configure:

- **Test interval:** The interval (in seconds) between each health check. Shorter intervals can improve failover time, but increase load on the origins as checks come from multiple locations. (Default: `60`).
 - **Method:** The HTTP method to use for the health check. (Default: `GET`)
 - **Timeout (seconds):** The time before marking the health check as failed. (Default: `5`)
 - **Number of retries:** The number of retries to attempt if there is a timeout before marking the origin as unhealthy. CIS attempts retries immediately. (Default: `2`)
 - **Expected response codes:** The expected HTTP response code or code range of the health check. This value must be between 200-299 with wildcards that are denoted with an `x`.
 - **Response body:** A case-insensitive substring to match against in the response body. If CIS does not find this string, it marks the origin as unhealthy.
 - **Healthy threshold:** The number of consecutive successful health check responses that CIS must receive before marking an origin as healthy. This setting prevents CIS from marking an origin healthy after a single successful probe and helps avoid flapping during intermittent recovery. (Default: `1`)
 - **Unhealthy threshold:** The number of consecutive failed health check responses that must occur before CIS marks an origin as unhealthy. This setting helps prevent temporary network or application issues from immediately triggering failover. (Default: `1`)
 - **Follow redirects:** Determines whether the health check automatically follows HTTP 3xx redirect responses. When enabled, the health check continues to the redirected location and evaluates the final response against the configured success criteria (status code and optional response body). When disabled, redirect responses are evaluated directly against the expected response codes.
4. Expand **Configure request headers (optional)** to add and configure HTTP request headers to send in the health check.
- Set a Host header by default. The `User-Agent` header can't be overridden.
5. Click **Create** to complete your health check configuration.

Working with health check events

Use the **Events** tab to monitor and investigate health status changes for origins and pools that are associated with a global load balancer. Health check events represent status changes that configured health checks detect. When an origin status changes, a corresponding event entry appears with details.

The following filtering options are available:

- **Start date:** Filters events by the beginning of a selected date range.
- **End date:** Filters events by the end of a selected date range.
- **Origin health:** Filters events by origin status (for example, Healthy or Critical).
- **Origins:** Filters events by a specific origin.
- **Pool health:** Filters events by pool status (for example, Healthy or Critical).
- **Pools:** Filters events by a specific origin pool.

To see health check events, click **Reliability > Global load balancers > Events**.

Step 2: Create an origin pool

At least one pool is required for each provisioned load balancer. Pools group origins for the load balancer to use.

1. In the CIS console, click **Reliability > Global load balancers > Origin pools**, then click **Create**.
2. Select whether to enable this pool. Disabled pools don't receive traffic and CIS excludes them from health checks. When you disable a pool, any load balancers that use it fail over to the next pool, if any.
3. Configure the required fields:
 - **Name:** A short name (tag) for the pool. Only alphanumeric characters, hyphens, and underscores are allowed.
 - **Origins:** The list of origins within this pool. CIS balances traffic directed at this pool across all currently healthy origins, provided the pool itself is healthy.



Note: If your application servers sit behind a local load balancer, such as an IBM Cloud load balancer, then add your load balancer's FQDN or virtual IP as your origin instead of adding your individual servers.

4. Configure optional fields as needed:
 - **Healthy origin threshold:** The minimum number of origins that must be healthy for this pool to serve traffic. If the number of

healthy origins falls below this number, CIS marks the pool as unhealthy and fails over to the next available pool. (Default: **1**)

- **Pool health check region** : Region from which the health check performs monitoring.
- **Health check**: The health check to use for checking origins within this pool. (Default: **No health check**)

5. Click **Save** to complete origin pool configuration.



Note: The origin pool initially appears as **Unhealthy**. This state changes to **Healthy** after a successful health check by CIS. You might need to refresh your browser to see the state change.

Define as many origin pools as the number of application farms that you have. These farms might be within the same or different geographic regions.

Step 3: Configure the global load balancer

Load balancers help to distribute traffic across multiple origin pools.

1. In the CIS console, click **Reliability > Global load balancers > Load balancers** , then click **Create**.
2. Enable the load balancer.
3. For Proxy, determine whether traffic flows through the security and performance functions on CIS. 'Off' bypasses these systems.
4. Configure the following information:
 - **Name (optional)**: The name to associate with your load balancer.
 - **TTL**: Time-to-live (TTL) of the DNS entry for the IP address returned by this load balancer. This option applies only to unproxied load balancer; otherwise, it defaults to **Automatic**.
 - **Traffic steering**: Controls how incoming user requests are distributed across your configured origin pools by determining the algorithm CIS uses to decide which data center (pool) serves each request. For more information, see [Optimizing traffic steering](#).
 - **Session Affinity**: Always route through the same performance and metrics instance. This option is available only if **Proxy** is enabled.
 - **Geo routes**: A mapping of region or country codes to a list of pools (ordered by their failover priority) for the specified region. Any regions not explicitly defined fall back to using the default pools.



Note: IBM's geographic regions differ from the regions that CIS uses. For details about the geographic regions, see [Geo steering](#).

5. Click **Create** to complete the global load balancer configuration.

Step 4: Verify connectivity

Finally, verify connectivity to your application by accessing the FQDN URL in a browser.



Note: For more information about updating and deleting global load balancers, health checks, and origin pools in the console, see [Managing global load balancers](#).

Configuring the global load balancers from the CLI

To create and configure a global load balancer from the CLI, follow these steps in this order:

1. Create a health check. (optional)
2. Create an origin pool.
3. Create a global load balancer.

Step 1: Creating a health check from the CLI (optional)

Health checks are optional attachments for origin pools. If you don't define any custom health checks, the system uses **/** as your default health check path.

To create a health check from the CLI, run the following command:


```
ibmcloud cis glb-monitor-create (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Where:

`--json`

The JSON file or JSON string that is used to describe a health check. Required.

- The required fields in JSON data are `type`.
 - `type` : The protocol to use for the healthcheck. Valid values are `HTTP`, `HTTPS`, and `TCP`.
- The optional fields are `description`, `timeout`, `retries`, `interval`.
 - `description` : A description of the health check.
 - `timeout` : The timeout (in seconds) before marking the health check as failed.
 - `retries` : The number of retries to attempt if there is a timeout before the origin is marked as unhealthy.
 - `interval` : The interval between each health check.
- For `TCP` type health check. Extra required fields are `port`.
 - `port` : The TCP port to use for the health check.
- For `HTTP/HTTPS` type health check. Extra option fields are `port`, `expected_body`, `expected_codes`, `method`, `path`, `header`, `follow_redirects`, `allow_insecure`.
 - `port` : The TCP port to use for the health check.
 - `expected_body` : A case-insensitive substring to look for in the response body.
 - `expected_codes` : The expected HTTP response code or code range of the health check.
 - `method` : The HTTP method to use for the health check.
 - `path` : The endpoint path to health check against.
 - `header` : The HTTP request headers to send in the health check.
 - `follow_redirects` : Follow redirects if returned by the origin.
 - `allow_insecure` : Don't validate the certificate when monitor use HTTPS.
 - `probe_zone` : Assign this monitor to emulate the specified zone while probing.

Sample JSON data:

For HTTP/HTTPS:

```
{
  "description": "Health monitor of web service",
  "type": "https",
  "method": "GET",
  "path": "/health",
  "header": {
    "Host": [
      "example.com"
    ],
    "X-App-ID": [
      "abc123"
    ]
  },
  "timeout": 5,
  "retries": 2,
  "interval": 90,
  "follow_redirects": true,
  "allow_insecure": false,
  "expected_codes": "2xx",
  "expected_body": "alive",
  "probe_zone": "example.com"
}
```

For TCP:

```
{
  "description": "Health monitor of TCP",
  "type": "tcp",
  "port": 80,
```

```
"timeout": 5,
"retries": 2,
"interval": 90
}
```

`-i, --instance`

Instance name or ID. If you don't set the instance name or ID, the command uses the context instance (`ibmcloud cis instance-set INSTANCE`).

`--output`

The output format. Currently, `json` is the only supported value.

Step 2: Creating an origin pool from the CLI

To create an origin pool from the CLI, run the following command:

```
ibmcloud cis glb-pool-create (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Where:

`--json`

The JSON file or JSON string that is used to describe an origin pool. Required.

- The required fields in JSON data are `name` , `origins` and `check_regions` :
 - `name` : A short name (tag) for the pool.
 - `origins` : A list of origins within this pool.
 - `check_regions` : A list of geographic region code.
- The optional fields are `description` , `minimum_origins` , `enabled` , `monitor` .

Sample JSON data:

```
{
  "name": "us-pool",
  "description": "application server pool in US",
  "origins": [
    {
      "name": "us-app-dal01",
      "address": "1.1.1.1",
      "enabled": true,
      "header": {
        "host": ["test.com"]
      }
    },
    {
      "name": "us-app-dal02",
      "address": "2.2.2.2",
      "enabled": true,
      "header": {
        "host": ["example.com"]
      }
    }
  ],
  "minimum_origins": 1,
  "check_regions": [ "WNAM" ],
  "monitor": "f1aba936b94213e5b8dca0c0dbf1f9cc",
  "enabled": true
}
```

`-i, --instance`

Instance name or ID. If you don't set the instance name or ID, the command uses the context instance (`ibmcloud cis instance-set INSTANCE`).

`--output`

The output format. Currently, `json` is the only supported value.

Step 3: Creating a global load balancer from the CLI

To create a global load balancer from the CLI, run the following command:

```
cis glb-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Where:

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--json`

The JSON file or JSON string that is used to describe a global load balancer. Required.

- The required fields in JSON data are `name`, `fallback_pool`, and `default_pools` :
 - `name` : The DNS hostname to associate with your load balancer.
 - `fallback_pool` : The pool ID to use when CIS detects all other pools as unhealthy.
 - `default_pools` : A list of pool IDs ordered by their failover priority.
- The optional fields are `description`, `ttl`, `region_pools`, `proxied`, `enabled`, `session_affinity`, `session_affinity_ttl`, `steering_policy` :
 - `description` : A description of your load balancer.
 - `ttl` : Time to live (TTL) of the DNS entry for the IP address that this load balancer returns.
 - `region_pools` : A mapping of region and country codes to a list of pool IDs (ordered by their failover priority) for the region.
 - `proxied` : Control whether traffic flows through the security and performance functions on CIS.
 - `enabled` : Whether to enable (the default) this load balancer.
 - `session_affinity` : Helps ensure that CIS consistently directs a user's requests to the same backend server during a session. Valid values are `cookie` and `none`.
 - `session_affinity_ttl` : Time, in seconds, until this load balancer's session affinity cookie expires after CIS creates it. Valid value is between `[1800, 604800]`. The default value is `82800`.
 - `steering_policy` : Valid values for `steering_policy` are `off`, `geo`, `random`, `dynamic_latency`.
 - `off` : Use `default_pools`.
 - `geo` : Use `region_pools/pop_pools`.
 - `random` : Select a pool randomly.
 - `dynamic_latency` : Use round-trip time to select the closest pool in `default_pools` (requires pool health checks).

Sample JSON data:

```
{
  "name": "www.example.com",
  "fallback_pool": "17b5962d775c646f3f9725cbc7a53df4",
  "default_pools": [
    "17b5962d775c646f3f9725cbc7a53df4",
    "9290f38c5d07c2e2f4df57b1f61d4196"
  ],
  "description": "Example global load balancer.",
  "ttl": 60,
  "region_pools": {
    "WNAM": [
      "de90f38ced07c2e2f4df50b1f61d4194",
      "9290f38c5d07c2e2f4df57b1f61d4196"
    ],
    "ENAM": [
      "00920f38ce07c2e2f4df50b1f61d4194"
    ]
  }
}
```

```
}  
}  
}
```

`-i, --instance`

Instance name or ID. If you don't set the instance name or ID, the command uses the context instance (`ibmcloud cis instance-set INSTANCE`).

`--output`

The output format. Currently, `json` is the only supported value.



Note: For more information about creating, updating, and deleting global load balancers, health checks, and origin pools, see [Global load balancer CLI commands](#).

Configuring the global load balancers with the API

To create and configure a global load balancer with the API, follow these steps in this order:

1. Create a health check. (optional)
2. Create an origin pool.
3. Create a global load balancer.

Step 1: Creating a health check with the API (optional)

Health checks are optional attachments for origin pools. If you don't define any custom health checks, the system uses `/` as your default health check path.

Follow these steps to create a health check with the API:

1. Set up your API environment with the correct variables.
2. Store the following value in variables to be used in the API command:

`CRN`: The full URL-encoded CRN of the service instance.

3. When all variables are initiated, create the health check:

```
curl -X POST https://api.cis.cloud.ibm.com/v1/:crn/load_balancers/monitors  
-H 'content-type: application/json'  
-H 'x-auth-user-token: Bearer xxxxxx'  
-d '{  
  "description": "",  
  "type": "http",  
  "interval": 60,  
  "retries": 2,  
  "timeout": 5,  
  "expected_body": "",  
  "expected_codes": "200",  
  "follow_redirects": true,  
  "allow_insecure": false,  
  "path": "/status",  
  "header": {  
    "Host": ["www.example.com"],  
    "X-App-ID": ["abc123"]  
  },  
  "method": "GET"  
}'
```

Step 2: Creating an origin pool with the API

Follow these steps to create an origin pool with the API:

1. Set up your API environment with the correct variables.

2. Store the following value in variables to be used in the API command:

`CRN` : The full URL-encoded CRN of the service instance.

3. When all variables are initiated, create the origin pool:

```
curl -X POST https://api.cis.cloud.ibm.com/v1/:crn/load_balancers/pools
-H 'content-type: application/json'
-H 'x-auth-user-token: Bearer xxxxxx'
-d '{
  "description": "",
  "enabled": true,
  "minimum_origins": 1,
  "monitor": "92859a0f6b4d3e55b953e0e29bb96338",
  "name": "eu-pool",
  "notification_email": "",
  "check_regions": [
    "EEU"
  ],
  "origins": [
    {
      "name": "eu-origin1",
      "address": "150.0.0.1",
      "enabled": true,
      "weight": 1
    },
    {
      "name": "eu-origin2",
      "address": "150.0.0.2",
      "enabled": true,
      "weight": 1
    }
  ]
}
```

Step 3: Creating a global load balancer with the API

Follow these steps to create a global load balancer with the API:

1. Set up your API environment with the correct variables.
2. Store the following values in variables to be used in the API command:

`CRN` : The full URL-encoded CRN of the service instance.

`ZONE_ID` : The domain ID.

3. When all variables are initiated, create the global load balancers:

```
curl -X POST https://api.cis.cloud.ibm.com/v1/:crn/zones/:zone_id/load_balancers
-H 'content-type: application/json'
-H 'x-auth-user-token: Bearer xxxxxx'
-d '{
  "description": "",
  "proxied": true,
  "enabled": true,
  "name": "www.example.com",
  "session_affinity": "none",
  "session_affinity_ttl": 5000,
  "steering_policy": "geo",
  "fallback_pool": "4112ba6c2974ec43886f90736968e838",
  "default_pools": [
    "6563ebae141638f92ebbd4a821bef8c",
    "4112ba6c2974ec43886f90736968e838"
  ],
  "pop_pools": {},
  "region_pools": {
    "EEU": [
      "4112ba6c2974ec43886f90736968e838"
    ],
    "ENAM": [
```

```
        "6563ebae141638f92ebbd4a821bef8c"
      ],
      "WEU": [
        "4112ba6c2974ec43886f90736968e838"
      ],
      "WNAM": [
        "6563ebae141638f92ebbd4a821bef8c"
      ]
    }
  }
}
```



Note: For more information about creating, updating, and deleting global load balancers, health checks, and origin pools, see [set up Terraform for CIS](#).

To create and configure a global load balancer with Terraform, follow these steps in order:

1. Create a health check
2. Create an origin pool
3. Create a global load balancer

Step 1: Creating a health check with Terraform (optional)

Health checks are optional attachments for origin pools. If you don't define any custom health checks, the system uses `/` as your default health check path.

To create a health check with Terraform, use the `ibm_cis_healthcheck` resource:

```
resource "ibm_cis_healthcheck" "health_check" {
  cis_id      = ibm_cis.instance.id
  description = "Health check for web servers"
  expected_body = "alive"
  expected_codes = "200"
  method      = "GET"
  timeout      = 5
  path        = "/health"
  interval     = 60
  retries      = 2
  type        = "http"
  port        = 80
  allow_insecure = false
  follow_redirects = true
}
```

Step 2: Creating an origin pool with Terraform

To create an origin pool with Terraform, use the `ibm_cis_origin_pool` resource:

```
resource "ibm_cis_origin_pool" "origin_pool" {
  cis_id = ibm_cis.instance.id
  name   = "us-pool"

  origins {
    name   = "us-app-dal01"
    address = "1.1.1.1"
    enabled = true
  }

  origins {
    name   = "us-app-dal02"
    address = "2.2.2.2"
    enabled = true
  }

  description = "Application server pool in US"
  enabled     = true
  minimum_origins = 1
  monitor     = ibm_cis_healthcheck.health_check.id
}
```

```
notification_email = ""

check_regions = [
  "WNAM"
]
}
```

Step 3: Creating a global load balancer with Terraform

To create a global load balancer with Terraform, use the `ibm_cis_global_load_balancer` resource:

```
resource "ibm_cis_global_load_balancer" "global_load_balancer" {
  cis_id      = ibm_cis.instance.id
  domain_id   = ibm_cis_domain.domain.id
  name        = "www.example.com"
  fallback_pool_id = ibm_cis_origin_pool.origin_pool.id
  default_pool_ids = [
    ibm_cis_origin_pool.origin_pool.id
  ]
  description  = "Global load balancer for example.com"
  proxied      = true
  enabled      = true
  session_affinity = "cookie"
  steering_policy = "geo"

  pop_pools {
    pop = "LAX"
    pool_ids = [
      ibm_cis_origin_pool.origin_pool.id
    ]
  }

  region_pools {
    region = "WNAM"
    pool_ids = [
      ibm_cis_origin_pool.origin_pool.id
    ]
  }
}
```

For more information about the Terraform resources for global load balancers, health checks, and origin pools, see the following resources in the IBM Cloud Provider Terraform registry:

- [ibm_cis_healthcheck](#)
- [ibm_cis_origin_pool](#)
- [ibm_cis_global_load_balancer](#)

Customizing error pages

IBM Cloud® Internet Services (CIS) has a wide range of error codes that allow us to differentiate specific problems. By default, these error pages mention Cloudflare; however, you can customize and brand these error pages. Custom error pages help you to provide a consistent experience for your users, even if a page load error occurs.

Customizable error pages come in two groups:

Challenges

- Basic security
- Web Application Firewall
- IP Firewall (Country block, Country challenge, IP (range) block)

CIS errors

- 502, 504, and CF 52X errors
- 10XX errors
- Errors that are related to Serve Stale Content



Note: 500, 501, 503, and 505 responses do not trigger custom error pages to avoid breaking specific API endpoints and other web applications. Reserve custom error pages for cases where the origin server can't return a response for the request (520-526 errors).

Custom error template

The following example is a basic custom error template. When you are creating your custom error templates, the maximum page size is 1.5 MB, and the page can't be blank. Additionally, all external resources are inlined with Base64 encoding, making them approximately 50% larger when published.

```
<html>
<head></head>
<body>
::[REPLACE WITH TOKEN NAME]::
</body>
</html>
```

Available custom error tokens

Some types of custom error pages must include one of these tokens anywhere within the HTML of the custom error page. Only one page-specific token can be present per error page, so if you want to customize every error, you must create one custom error page for each error that contains the respective token.

Page Type	Token
All pages	::CLIENT_IP::
All pages	::RAY_ID::
Basic Security (CAPTCHA Challenge)	::CAPTCHA_BOX::
WAF (CAPTCHA Challenge)	::CAPTCHA_BOX::
Country Challenge (CAPTCHA Challenge)	::CAPTCHA_BOX::
Defense Mode (Interstitial Page)	::IM_UNDER_ATTACK_BOX::
5XX Errors	::CLOUDFLARE_ERROR_500S_BOX::
1XXX Errors	::CLOUDFLARE_ERROR_1000S_BOX::

Styling error codes

Each tag has a unique class that you can use to style individual error codes. It is possible to use CSS to stylize the tags in the div/span/section since they all have class IDs.



Tip: Each page (challenge, 5xx errors) has a different ID, so use the preview option to get the proper ID.

Publishing error pages

After you're done customizing your error pages, it's time to publish them to the edge. Publication is done through the [Custom Page CLI](#).

When you publish, the custom error page is requested one time, then cached on the cloud's edge.

Updating error pages

You can update an error page by republishing it.



Note: If CIS can't load your site, or you blocked the US in the CIS firewall, publishing and previewing the error page does not work.

Troubleshooting error pages

- If you encounter errors while attempting to preview or publish your custom error page, run it through a HTML validator and ensure that it is error free.
- Make sure that the minimum page size is greater than 0. You must add content to your page.
- Make sure that you are serving the custom error page with a 200 status code.
- If the site cannot be loaded, or you have blocked the US in the IP firewall, publishing and previewing the error page does not work.

Working with Edge functions

How Edge functions work

IBM Cloud® Internet Services Edge functions allow you to create or modify existing applications, without having to configure or maintain infrastructure, by using a serverless execution environment. Edge functions can be defined and uploaded to the Cloud edge to process requests before they reach the origin. CIS Edge functions can be used to modify HTTP requests and responses, make parallel requests, or generate responses from the Cloud edge.

Edge functions associates actions with URIs based on a defined domain. This association is called a *trigger*. Incoming requests to your site are intercepted at the Cloud edge and matched against the triggers in your account or domain. If the request URL matches the trigger's URI, the action associated with the trigger is run.

Edge functions are modeled on the [Service Worker API](#) available in modern web browsers, and use the same API whenever possible.

The Service Worker API allows you to intercept any request that is made to your site. After your JavaScript is handling the request, you can elect to make any number of subrequests to your site or others, and finally return a response to your visitor.

Unlike standard service workers, Edge functions run on CIS edge servers, not in the user's browser. That means you can trust that your code runs in a trusted environment where it can't be bypassed by malicious clients. It also means that the user does not need to be using a modern browser that supports service workers – you can even intercept requests from API clients that aren't browsers.

Internally, Edge functions use the same V8 JavaScript engine, which is used in the Chrome browser to run workers on our infrastructure. V8 dynamically compiles your JavaScript code into ultra-fast machine code, enhancing performance. This makes it possible for your code to run in microseconds, and for our edge server to run many thousands of scripts per second.

While Edge functions does use V8, it does not use Node.js. The JavaScript APIs available to you inside workers are implemented by us directly. Working with V8 directly allows the code to run more efficiently and with the security controls needed to keep the customers and the infrastructure safe.

Working with Edge functions request properties

Here, we discuss the Cloudflare runtime APIs for CIS Edge functions. The Edge functions runtime provides the following Cloudflare APIs for use by scripts running at the cloud edge.

Constructor syntax

```
$ new Request(input [, init])
```

Constructor parameters

- **input** : Can be either a USVString that contains the URL, or an existing **Request** object. Note that the **url** property is immutable, so when modifying a request and changing the URL, you must pass the new URL in this parameter.
- **init** (optional): An options object that contains custom settings to apply to the request. Valid options are:
 - **method** : The request method, such as **GET** or **POST**
 - **headers** : A **Headers** object
 - **body** : Any text to add to the request.



Note: Requests using the **GET** or **HEAD** methods can't have a body.

- **redirect** : The mode respected when the request is fetched. The default for requests generated from the incoming **fetchEvent** from the event handler is **manual**. Default for newly constructed Requests (in other words, **new Request(url)**) is **follow**. Valid options:
 - **follow** : If a redirect response is returned to the fetch, another fetch is fired based on the **Location** header in the response until a non-redirect code is returned. For example, **await fetch(..)** could never return a **301** redirect.
 - **manual** : redirect responses return from a fetch.

Properties

All properties of an incoming `Request` object (`event.request`) are read-only. To modify a request, you must create a `Request` object and pass the options to modify to its constructor.

- `body` : A simple getter that exposes a `ReadableStream` of the contents.
- `bodyUsed` : A Boolean that declares if the body was used in a response.
- `cf` : An object that contains data provided by our partners at Cloudflare.
- `headers` : Contain the associated `Headers` object for the request.
- `method` : The request method, such as `GET` or `POST` , associated with the request.
- `redirect` : The redirect mode to use (`follow` or `manual`).
- `url` : Contains the URL of the request.

The `cf` object

In addition to the properties on the standard `Request` object, you can use a `request.cf` object to control how features are applied as well as other custom information provided by Cloudflare. For example,

```
if (request.cf.asn == 64512) {
  return new Response('Block the ASN 64512 response')
}
```



Note: If you are using [workers playground](#) to write and test your scripts, the `request.cf` content is not available in preview mode. You must be in production to run that content.

These properties contain special information from an incoming request to help with your app's logic. All plans have access to:

- `asn` : ASN of the incoming request (for example, `395747`).
- `colo` : The three-letter airport code of the data center that the request hit (for example, `"DFW"`).
- `weight` : The browser-requested weight for the HTTP/2 prioritization.
- `exclusive` : The browser-requested HTTP/2 exclusive flag (1 for Chromium-based browsers, 0 for others).
- `group` : HTTP/2 stream ID for the request group (non-zero only for Firefox).
- `group-weight` : HTTP/2 weight for the request group (non-zero only for Firefox).
- `tlsCipher` : The cipher for the connection to CIS (for example, `"AEAD-AES128-GCM-SHA256"`).
- `country` : The two-letter country code of the incoming request. The same value is provided in the `CF-IPCountry` header (for example, `"US"`).
- `tlsClientAuth` : Set only when enabled for mTLS. The object has the following properties: `certIssuerDNLegacy` , `certIssuerDN` , `certIssuerDNRFC2253` , `certSubjectDNLegacy` , `certVerified` , `certNotAfter` , `certSubjectDN` , `certFingerprintSHA1` , `certNotBefore` , `certSerial` , `certPresented` , `certSubjectDNRFC2253`
- `tlsVersion` : The TLS version of the connection to CIS (for example, `TLSv1.3`).

Standard and **Enterprise** plans have access to:

- `requestPriority` : The browser-requested prioritization information in the request object (for example, `"weight=192;exclusive=0;group=3;group-weight=127"`).
- `city` : City of the incoming request (for example, `"Austin"`).
- `continent` : Continent of the incoming request (for example, `"NA"`).
- `httpProtocol` : HTTP Protocol (for example, `"HTTP/2"`).
- `latitude` : Latitude of the incoming request (for example, `"30.27130"`).
- `longitude` : Longitude of the incoming request (for example, `"-97.74260"`).
- `postalCode` : PostalCode of the incoming request (for example, `"78701"`).
- `region` : If known, the [ISO 3166-2](#) name for the first level region associated with the IP address of the incoming request. If it is not known, it is an empty string (for example, `"Texas"`).
- `regionCode` : If known, the [ISO 3166-2](#) code for the first level region associated with the IP address of the incoming request. If it is not known, it is an empty string (for example, `"TX"`).
- `timezone` : Timezone of the incoming request (for example, `"America/Chicago"`).

All plans can set these features on outbound requests.

- `cacheEverything` : This option forces CIS to cache the response for this request, regardless of what headers are seen on the response. This is equivalent to setting the page rule **Cache Level** to **Cache Everything** (for example, `true`).

- `scrapeShield` : Toggles ScrapeShield (for example, `false`).
- `polish` : Set Cloudflare Polish mode. The possible values are "lossy", "lossless" or "off" (for example, `lossless`).
- `minify` : Website optimization to enable/disable Cloudflare Autominify for various file types. The value is an object containing Boolean fields for `javascript` , `css` , and `html` (for example, `{ javascript: true, css: true, html: false }`).
- `mirage` : Image optimization to enable/disable Cloudflare mirage. When you specify this option, the value should always be `false` (for example, `false`).
- `cacheTtl` : This option forces CIS to cache the response for this request, regardless of what headers are seen on the response. This is equivalent to setting two page rules: **Edge Cache TTL** and **Cache Level** (to **Cache Everything**; for example, `300`).
- `resolveOverride` : Redirects the request to an alternate origin server. You can use this to implement load balancing across several origins (for example, `us-east.example.com`).



Note: For security reasons, the hostname set in `resolveOverride` must be proxied on the same CIS zone of the incoming request. Otherwise, the setting is ignored. CNAME hosts are allowed, so to resolve to a host under a different domain or a DNS only domain first declare a CNAME record within your own zone’s DNS mapping to the external hostname, set proxy on CIS, then set `resolveOverride` to point to that CNAME record.

Enterprise only

- `cacheKey` : A request's cache key is what determines if two requests are "the same" for caching purposes. If a request has the same cache key as some previous request, then we can serve the same cached response for both (for example, `'some-key'`).
- `cacheTtlByStatus` : This option is a version of the `cacheTtl` feature which chooses a TTL based on the response's status code. If the response to this request has a status code that matches, CIS caches for the instructed time, and overrides cache instructions sent by the origin (for example, `{ "200-299": 86400, 404: 1, "500-599": 0 }`).



Note: CIS still adheres to standard cache levels, so by default this overrides cache behavior for static files. If you want to cache non-static assets, you must set a Cache Level of Cache Everything using a Page Rule.

An Edge functions script runs after CIS security features, but before everything else. Therefore, an Edge functions script can't affect the operation of security features (since they are already finished), but it can affect other features, like Image Size Optimization, or how the response is cached at the edge.

Updating the `cf` object is similar to modifying a request. You can add the `cf` object to a `Request` by passing a custom object to [fetch](#).

```
// Disable ScrapeShield for this request.
fetch(event.request, { cf: { scrapeShield: false } })
```



Note: Invalid or incorrectly-named settings in the `cf` object are silently ignored. Be careful to test that you are getting the behavior you want.

Edge function use cases



Important: These examples are for demonstration purposes only, and not intended for use in production.

- [A/B testing](#)
- [Adding a response header](#)
- [Aggregating multiple requests](#)
- [Conditional routing](#)
- [Hot-link protection](#)
- [Originless responses](#)
- [Post requests](#)
- [Setting a cookie](#)
- [Signed requests](#)
- [Streaming responses](#)
- [Custom load balancer with Edge functions](#)

- [Caching using fetch](#)
- [Cache API](#)

Using Edge function actions and triggers

Edge functions consist of two main components: actions and triggers. Actions are JavaScript files that define logic to be run at the edge. Triggers are routing rules based on URLs that define when and under what circumstances those actions should run. Triggers connect specific request patterns (like domains and paths) to the appropriate actions. Without an associated trigger, an action won't impact traffic.

Edge function actions

Actions are written in JavaScript and require an event listener to respond to a trigger event. Actions don't affect your traffic unless used by a trigger.

Plan differences

Differences when working with actions depend on your plan:

Standard plans

- Allow only one action per account, automatically named after your domain (name can't be changed).
- To update an action, upload a new JavaScript file or edit the code directly in the editor. Uploading a new file replaces the existing action.
- When you upload a JavaScript file, its name is always set to your domain name.

Enterprise plans

- Can upload an unlimited number of scripts, each with a unique names if wanted.
- When creating an action, you can specify a custom name.
- When uploading a JavaScript file, the action name is taken from the file name.

Working with actions

You can create, upload, edit, and manage actions to define the logic your Edge function runs in response to trigger events. The console provides tools for each of the core operations:

- **Creating an action** - Click **Create** to add an action by using the built-in code editor. After you add your JavaScript code, click **Save** to create your action.
- **Reading an action** - Click an existing action from the actions table to open it in the editor. This allows you to view its code without making changes.
- **Updating an action** - To modify an action, open it in the editor by clicking on it in the list. Make your changes, then click **Save**. When you save your changes, the updated action is uploaded. If the action is in use, the updates take effect immediately.
- **Uploading an action** - Click **Upload** to upload a JavaScript file. Uploading or creating an action with the same name as an existing action causes the existing action to be overwritten. Rename the action file before uploading, or enter a unique name during creation to avoid this behavior.

Uploading can also be used as part of creating or updating an action, depending on whether the name already exists.

- **Deleting an action** - To delete an action, click the **Delete** icon  in the table row for the selected action. An action can't be deleted while it's associated with a trigger. To delete the action, you must first remove all triggers that use it. The **Uses** column shows how many triggers are linked to each action.
- **Associating a trigger** - Triggers are required for an action to run. Add a trigger from the **Triggers** tab and select the action from the list of existing actions. The **Uses** column in the actions table displays the number of triggers currently linked to each action.

Edge function triggers

Triggers (routes) determine how domain traffic is routed to actions. They associate URL patterns, based on a domain on the account, with a predefined action. The URL pattern must include the domain and may contain wildcards either as a prefix to the domain or at the end of the path. If no path is specified, a  is added implicitly. URL patterns can't contain infix wildcards or query parameters.

Plan differences

There are no differences in how triggers work across plans. All plans support adding, editing, and deleting triggers.

Working with triggers

You must add a domain before adding triggers. Triggers define how traffic is routed to your Edge function actions. A trigger connects a URL pattern with an existing action.

 **Note:** Triggers can be created even if no actions exist yet, but they will not invoke any logic until an action is selected.

- **Adding a trigger** - Go to the **Triggers** tab and click **Add trigger**. Enter a URL pattern, then select an action from the list of existing actions to associate with this pattern. This establishes the link between a route and an action.

To keep a trigger path active without running an Edge function, enable **Avoid edge functions**. For example, if `my-function` is assigned to `gamma.cistest-load.com/*`, but you want `gamma.cistest-load.com/data` to bypass it, create a separate trigger for `/data` and enable **Avoid edge functions**. This ensures the `/data` path remains active without invoking `my-function`.

- **Editing a trigger** - Click the **Edit** icon  in the table row of the trigger that you want to modify, then make changes and click **Save**.
- **Deleting a trigger** - Click the **Delete** icon  in the table row of the trigger that you want to modify. This action can't be undone. Deleting a trigger removes the association between the route and the action.

Related links

- [How Edge functions work](#)
- [Edge function examples](#)

Edge function examples

Altering headers

This example illustrates how to add, change, or delete headers sent in a request or returned in a response.

 **Note:** You can't modify the value of certain headers, such as `server`, `eh-cache-tag`, or `eh-cdn-cache-control`.

```
addEventListener('fetch', event => {
  event.respondWith(handleRequest(event.request))
})

async function handleRequest(request) {
  const response = await fetch(request)

  // Clone the response so that it's no longer immutable
  const newResponse = new Response(response.body, response)

  // Add a custom header with a value
  newResponse.headers.append("X-CIS-Functions-Hello", "Hello from CIS Workers")

  // Delete headers
  newResponse.headers.delete("x-header-to-delete")
  newResponse.headers.delete("x-header2-to-delete")

  // Adjust the value for an existing header
  newResponse.headers.set("x-header-to-change", "NewValue")

  return newResponse
}
```

Redirecting country code response

This example illustrates how to redirect a response based on the country code in the header of a visitor.

```
export default {
  async fetch(request) {
    /**
```



```

* A map of the URLs to redirect to
* @param {Object} countryMap
*/
const countryMap = {
  US: "https://example.com/us",
  EU: "https://example.com/eu",
};

// Use the cf object to obtain the country of the request
// more on the cf object: https://developers.cloudflare.com/workers/runtime-apis/request#incomingrequestcfproperties
const country = request.cf.country;

if (country !== null && country in countryMap) {
  const url = countryMap[country];
  // Remove this logging statement from your final output.
  console.log(
    `Based on ${country}-based request, your user would go to ${url}.`,
  );
  return Response.redirect(url);
} else {
  return fetch("https://example.com", request);
}
},
};

```

Modifying responses

This examples shows how to fetch and modify response properties which are immutable by creating a copy first.

```

export default {
  async fetch(request) {
    /**
     * @param {string} headerNameSrc Header to get the new value from
     * @param {string} headerNameDst Header to set based off of value in src
     */
    const headerNameSrc = "foo"; // "Orig-Header"
    const headerNameDst = "Last-Modified";

    /**
     * Response properties are immutable. To change them, construct a new
     * Response and pass modified status or statusText in the ResponseInit
     * object. Response headers can be modified through the headers `set` method.
     */
    const originalResponse = await fetch(request);

    // Change status and statusText, but preserve body and headers
    let response = new Response(originalResponse.body, {
      status: 500,
      statusText: "some message",
      headers: originalResponse.headers,
    });

    // Change response body by adding the foo prop
    const originalBody = await originalResponse.json();
    const body = JSON.stringify({ foo: "bar", ...originalBody });
    response = new Response(body, response);

    // Add a header using set method
    response.headers.set("foo", "bar");

    // Set destination header to the value of the source header
    const src = response.headers.get(headerNameSrc);

    if (src !== null) {
      response.headers.set(headerNameDst, src);
      console.log(
        `Response header "${headerNameDst}" was set to "${response.headers.get(
          headerNameDst,
        )}"`,
      );
    }
  }
};

```



```
}
return response;
},
};
```

Setting security headers

To help improve web application security, set HTTP security headers at the edge or in your application. The following JavaScript example demonstrates how to implement these headers. By modifying the response headers in-flight, you can add protections like these common security headers without changing your back-end application.

X-XSS-Protection

Prevents a page from loading if an XSS attack is detected. [Learn more](#) For example:

```
"X-XSS-Protection": "1; mode=block",
```

X-Frame-Options

Helps prevent clickjacking attacks. [Learn more](#) For example:

```
"X-Frame-Options": "DENY",
```

X-Content-Type-Options

Stops browsers from MIME-sniffing a response away from the declared content-type. [Learn more](#) For example:

```
"X-Content-Type-Options": "nosniff",
```

Permissions-Policy

Lets you control access to browser features. For example, to opt out of Federated Learning of Cohorts (FLoC):

```
"Permissions-Policy": "interest-cohort=()",
```

Referrer-Policy

Controls how much referrer information is included with requests. For example:

```
"Referrer-Policy": "strict-origin-when-cross-origin",
```

Strict-Transport-Security

Enforces secure (HTTPS) connections to your server. These headers are not automatically set because your website might get added to Chrome's HTTP Strict Transport Security (HSTS) preload list. For example:

```
"Strict-Transport-Security" : "max-age=63072000; includeSubDomains; preload",
```

Content-Security-Policy

Permits content from a trusted domain and all its subdomains. [Learn more](#) For example:

```
"Content-Security-Policy": "default-src 'self' example.com *.example.com",
```

Javascript example

```
export default {
  async fetch(request) {
    const DEFAULT_SECURITY_HEADERS = {
      "X-Content-Type-Options": "nosniff",
      "Referrer-Policy": "strict-origin-when-cross-origin",
      "Cross-Origin-Embedder-Policy": 'require-corp; report-to="default";',
      "Cross-Origin-Opener-Policy": 'same-site; report-to="default";',
      "Cross-Origin-Resource-Policy": "same-site",
    };
    const BLOCKED_HEADERS = [
      "Public-Key-Pins",
      "X-Powered-By",
      "X-AspNet-Version",
    ];
```



```

];

let response = await fetch(request);
let newHeaders = new Headers(response.headers);

const tlsVersion = request.cf.tlsVersion;
console.log(tlsVersion);
// This sets the headers for HTML responses:
if (
  newHeaders.has("Content-Type") &&
  !newHeaders.get("Content-Type").includes("text/html")
) {
  return new Response(response.body, {
    status: response.status,
    statusText: response.statusText,
    headers: newHeaders,
  });
}

Object.keys(DEFAULT_SECURITY_HEADERS).map((name) => {
  newHeaders.set(name, DEFAULT_SECURITY_HEADERS[name]);
});

BLOCKED_HEADERS.forEach((name) => {
  newHeaders.delete(name);
});

if (tlsVersion !== "TLSv1.2" && tlsVersion !== "TLSv1.3") {
  return new Response("You need to use TLS version 1.2 or higher.", {
    status: 400,
  });
} else {
  return new Response(response.body, {
    status: response.status,
    statusText: response.statusText,
    headers: newHeaders,
  });
}
},
};
};

```

Working with logs

Using the Logpull service

IBM customers can access the Logpull service on Enterprise accounts. This service allows users to consume request logs over HTTP using the [Logpull command](#). These logs contain data related to the connecting client, the request path through the network, and the response from the origin web server. Users can query for logs starting from 1 minute in the past (relative to the actual time that you make the query).

Enabling log retention

Edge logs are not retained by default. Before you can pull logs using the Logpull CLI, you must enable log retention. To do so, you must check the current setting, then turn log retention on or off. When enabled, logs are retained for 7 days. If retention is turned off, previously saved logs remain available until the retention period expires.

1. To check whether log retention is currently turned off, use the `log-retention` CLI:

```
$ ibmcloud cis log-retention DNS_DOMAIN_ID
```

1. If the output shows that the flag is `off` (default), update the setting as follows.

```
$ ibmcloud cis log-retention-update DNS_DOMAIN_ID --flag on
```

Logpull use cases

Getting logs based on RayID

If you receive an error message after running a command, you can use the RayID provided in the response header to get the logs related to the command.

**Note:** If you have a RAY_ID with `-XXX` on the end, be sure to remove it. For example, `12ab34cdef567gh8-XXX` becomes `12ab34cdef567gh8`.

Use the following command for the request:

```
$ ibmcloud cis logpull DNS_DOMAIN_ID --ray-id RAY_ID
```

The response follows:

```
{
  "ClientIP": "68.278.11.89",
  "ClientRequestHost": "testing.logpull.com",
  "ClientRequestMethod": "GET",
  "ClientRequestURI": "/var/www",
  "EdgeEndTimestamp": 1545155129703000000,
  "EdgeResponseBytes": 1935,
  "EdgeResponseStatus": 403,
  "EdgeStartTimestamp": 1545155129696000000,
  "RayID": "48b371889c489b2c"
}
```

Getting logs based on time duration

If you receive an error message after running a command, but do not know the RayID of the response, you can use a time duration to get all logs for the time period when the error occurred.

Use the following command for the request:

```
$ ibmcloud cis logpull DNS_DOMAIN_ID --start 2019-01-02T01:00:00+00:00 --end 2019-01-02T01:00:00+00:00
```

Where `--start` and `--end` is entered as a UNIX timestamp (in seconds or nanoseconds), or as an absolute timestamp that conforms to RFC 3339, with a time duration of a minute or an hour.



Note: You can retrieve data for a maximum time interval of 1 hour.

The response follows:

```
{
  "ClientIP": "2620:1f7:8c5::1f:949e:c04d",
  "ClientRequestHost": "test.logpull.load.com",
  "ClientRequestMethod": "GET",
  "ClientRequestURI": "/assets/bootstrap-collapse.js",
  "EdgeEndTimestamp": 1545067628046000000,
  "EdgeResponseBytes": 2205,
  "EdgeResponseStatus": 200,
  "EdgeStartTimestamp": 1545067628044000000,
  "RayID": "48ab19434891c7a7"
}
{
  "ClientIP": "2620:1f7:8c5::1f:949e:c04d",
  "ClientRequestHost": "test.logpull.load.com",
  "ClientRequestMethod": "GET",
  "ClientRequestURI": "/assets/3d.gif",
  "EdgeEndTimestamp": 1545067627970000000,
  "EdgeResponseBytes": 2538446,
  "EdgeResponseStatus": 200,
  "EdgeStartTimestamp": 1545067627951000000,
  "RayID": "48ab1942bf96c7a7"
}
{
  "ClientIP": "2620:1f7:8c5::1f:949e:c04d",
  "ClientRequestHost": "test.logpull.load.com",
  "ClientRequestMethod": "GET",
  "ClientRequestURI": "/assets/logo.gif",
  "EdgeEndTimestamp": 1545067628051000000,
  "EdgeResponseBytes": 82257,
  "EdgeResponseStatus": 200,
  "EdgeStartTimestamp": 1545067628048000000,
  "RayID": "48ab194348a0c7a7"
}
{
  "ClientIP": "2620:1f7:8c5::1f:949e:c04d",
  "ClientRequestHost": "test.logpull.load.com",
  "ClientRequestMethod": "GET",
  "ClientRequestURI": "/assets/docs.css",
  "EdgeEndTimestamp": 1545067627952000000,
  "EdgeResponseBytes": 540,
  "EdgeResponseStatus": 200,
  "EdgeStartTimestamp": 1545067627948000000,
  "RayID": "48ab1942af8ac7a7"
}
{
  "ClientIP": "2620:1f7:8c5::1f:949e:c04d",
  "ClientRequestHost": "test.logpull.load.com",
  "ClientRequestMethod": "GET",
  "ClientRequestURI": "/assets/bootstrap.css",
  "EdgeEndTimestamp": 1545067627952000000,
  "EdgeResponseBytes": 17311,
  "EdgeResponseStatus": 200,
  "EdgeStartTimestamp": 1545067627948000000,
  "RayID": "48ab1942af85c7a7"
}
{
  "ClientIP": "2620:1f7:8c5::1f:949e:c04d",
  "ClientRequestHost": "test.logpull.load.com",
  "ClientRequestMethod": "GET",
  "ClientRequestURI": "/assets/jquery.js",
  "EdgeEndTimestamp": 1545067628045000000,
  "EdgeResponseBytes": 33555,
  "EdgeResponseStatus": 200,
  "EdgeStartTimestamp": 1545067628042000000,
  "RayID": "48ab19434882c7a7"
}
{
  "ClientIP": "2620:1f7:8c5::1f:949e:c04d",
  "ClientRequestHost": "test.logpull.load.com",
  "ClientRequestMethod": "GET",
  "ClientRequestURI": "/assets/watson.gif",
  "EdgeEndTimestamp": 1545067628052000000,
  "EdgeResponseBytes": 893230,
  "EdgeResponseStatus": 200,
  "EdgeStartTimestamp": 1545067628046000000,
  "RayID": "48ab194348a3c7a7"
}
{
  "ClientIP": "2620:1f7:8c5::1f:949e:c04d",
  "ClientRequestHost": "test.logpull.load.com",
  "ClientRequestMethod": "GET",
  "ClientRequestURI": "/assets/bootstrap-386.js",
  "EdgeEndTimestamp": 1545067628046000000,
  "EdgeResponseBytes": 1663,
  "EdgeResponseStatus": 200,
  "EdgeStartTimestamp": 1545067628043000000,
  "RayID": "48ab19434884c7a7"
}
{
  "ClientIP": "2620:1f7:8c5::1f:949e:c04d",
  "ClientRequestHost": "test.logpull.load.com",
  "ClientRequestMethod": "GET",
  "ClientRequestURI": "/assets/Fixedsys500c.woff",
  "EdgeEndTimestamp": 1545067630272000000,
  "EdgeResponseBytes": 14055,
  "EdgeResponseStatus": 200,
  "EdgeStartTimestamp": 1545067629064000000,
  "RayID": "48ab1949aca2c7a7"
}
{
  "ClientIP": "2620:1f7:8c5::1f:949e:c04d",
  "ClientRequestHost": "test.logpull.load.com",
  "ClientRequestMethod": "GET",
  "ClientRequestURI": "/assets/bios.gif",
  "EdgeEndTimestamp": 1545067628055000000,
  "EdgeResponseBytes": 1121237,
  "EdgeResponseStatus": 200,
  "EdgeStartTimestamp": 1545067628046000000,
  "RayID": "48ab1943489ec7a7"
}
{
  "ClientIP": "2620:1f7:8c5::1f:949e:c04d",
  "ClientRequestHost": "test.logpull.load.com",
  "ClientRequestMethod": "GET",
  "ClientRequestURI": "/assets/bootstrap-modal.js",
  "EdgeEndTimestamp": 1545067628046000000,
  "EdgeResponseBytes": 2569,
  "EdgeResponseStatus": 200,
  "EdgeStartTimestamp": 1545067628043000000,
  "RayID": "48ab1943488ac7a7"
}
{
  "ClientIP": "2620:1f7:8c5::1f:949e:c04d",
  "ClientRequestHost": "test.logpull.load.com",
  "ClientRequestMethod": "GET",
  "ClientRequestURI": "/assets/holder.js",
  "EdgeEndTimestamp": 1545067628053000000,
  "EdgeResponseBytes": 4593,
  "EdgeResponseStatus": 200,
  "EdgeStartTimestamp": 1545067628046000000,
  "RayID": "48ab19434898c7a7"
}
{
  "ClientIP": "2620:1f7:8c5::1f:949e:c04d",
  "ClientRequestHost": "test.logpull.load.com",
  "ClientRequestMethod": "GET",
  "ClientRequestURI": "/assets/old_school.png",
  "EdgeEndTimestamp": 1545067627960000000,
  "EdgeResponseBytes": 1466,
  "EdgeResponseStatus": 200,
  "EdgeStartTimestamp": 1545067627952000000,
  "RayID": "48ab1942bf92c7a7"
}
{
  "ClientIP": "2620:1f7:8c5::1f:949e:c04d",
  "ClientRequestHost": "test.logpull.load.com",
  "ClientRequestMethod": "GET",
  "ClientRequestURI": "/assets/bootstrap-responsive.css",
  "EdgeEndTimestamp": 1545067627951000000,
  "EdgeResponseBytes": 4797,
  "EdgeResponseStatus": 200,
  "EdgeStartTimestamp": 1545067627948000000,
  "RayID": "48ab1942af86c7a7"
}
```

Available fields

If `fields` are not specified in the request, a limited set of default fields are returned. Find the full list of all available fields here:

```
$ ibmcloud cis logpull DNS_DOMAIN_ID --available-fields
```

Fields are passed as a comma-separated list. For example, to have "ZoneName" and "RayID", use:

```
$ ibmcloud cis logpull DNS_DOMAIN_ID --start 2019-01-02T01:00:00+00:00 --end 2019-01-02T01:00:00+00:00 --fields ZoneName,RayID
```

Field list

Available fields:

```
"BotScore": "int; Cloudflare Bot Score. Scores below 30 are commonly associated with automated traffic. Available only for Bot Management customers. To enable this feature, contact your account team.",
"BotScoreSrc": "string; Detection engine responsible for generating the Bot Score. Possible values are Not Computed | Heuristics | Machine Learning | Behavioral Analysis | Verified Bot | JS Fingerprinting | Cloudflare Service. Available only for Bot Management customers. To enable this feature, contact your account team.",
"CacheCacheStatus": "string; Cache status. Possible values are unknown | miss | expired | updating | stale | hit | ignored | bypass | revalidated | dynamic | stream_hit | deferred \"dynamic\" means that a request is not eligible for cache. This can mean, for example that it was blocked by the firewall. Refer to [Cloudflare cache responses](https://developers.cloudflare.com/cache/concepts/cache-responses/) for more details.",
```

"CacheResponseBytes": "int; Number of bytes returned by the cache.",
"CacheResponseStatus": "int; HTTP status code returned by the cache to the edge. All requests (including non-cacheable ones) go through the cache. Refer also to CacheCacheStatus field.",
"CacheTieredFill": "bool; Tiered Cache was used to serve this request.",
"ClientASN": "int; Client AS number.",
"ClientCountry": "string; 2-letter ISO-3166 country code of the client IP address.",
"ClientDeviceType": "string; Client device type.",
"ClientIP": "string; IP address of the client.",
"ClientIPClass": "string; Client IP class. Possible values are unknown | badHost | searchEngine | allowlist | monitoringService | noRecord | scan | tor.",
"ClientRequestBytes": "int; Number of bytes in the client request.",
"ClientRequestHost": "string; Host requested by the client.",
"ClientRequestMethod": "string; HTTP method of client request.",
"ClientRequestPath": "string; URI path requested by the client.",
"ClientRequestProtocol": "string; HTTP protocol of client request.",
"ClientRequestReferer": "string; HTTP request referrer.",
"ClientRequestURI": "string; URI requested by the client.",
"ClientRequestUserAgent": "string; User agent reported by the client.",
"ClientSSLCipher": "string; Client SSL cipher.",
"ClientSSLProtocol": "string; Client SSL (TLS) protocol. The value \"none\" means that SSL was not used.",
"ClientSrcPort": "int; Client source port.",
"ClientXRequestedWith": "string; X-Requested-With HTTP header.",
"Cookies": "object; String key-value pairs for Cookies. This field is populated based on [Logpush Custom fields] (<https://developers.cloudflare.com/logs/reference/custom-fields/>), which need to be configured.",
"EdgeColoCode": "string; IATA airport code of data center that received the request.",
"EdgeColoID": "int; Cloudflare edge colo id.",
"EdgeEndTimestamp": "int or string; Timestamp at which the edge finished sending response to the client.",
"EdgePathingOp": "string; Indicates what type of response was issued for this request (unknown = no specific action).",
"EdgePathingSrc": "string; Details how the request was classified based on security checks (unknown = no specific classification).",
"EdgePathingStatus": "string; Indicates what data was used to determine the handling of this request (unknown = no data).",
"EdgeRequestHost": "string; Host header on the request from the edge to the origin.",
"EdgeResponseBytes": "int; Number of bytes returned by the edge to the client.",
"EdgeResponseCompressionRatio": "float; The edge response compression ratio is calculated as the ratio between the sizes of the original and compressed responses.",
"EdgeResponseContentType": "string; Edge response Content-Type header value.",
"EdgeResponseStatus": "int; HTTP status code returned by Cloudflare to the client.",
"EdgeServerIP": "string; IP of the edge server making a request to the origin. Possible responses are string in IPv4 or IPv6 format, or empty string. Empty string means that there was no request made to the origin server.",
"EdgeStartTimestamp": "int or string; Timestamp at which the edge received request from the client.",
"OriginIP": "string; IP of the origin server.",
"OriginResponseBytes": "int; Number of bytes returned by the origin server.",
"OriginResponseHTTPExpires": "string; Value of the origin 'expires' header in RFC1123 format.",
"OriginResponseHTTPLastModified": "string; Value of the origin 'last-modified' header in RFC1123 format.",
"OriginResponseStatus": "int; Status returned by the upstream server. The value 0 means that there was no request made to the origin server and the response was served by Cloudflare's Edge. However, if the zone has a Worker running on it, the value 0 could be the result of a Workers subrequest made to the origin.",
"OriginResponseTime": "int; Number of nanoseconds it took the origin to return the response to edge.",
"OriginSSLProtocol": "string; SSL (TLS) protocol used to connect to the origin.",
"ParentRayID": "string; Ray ID of the parent request if this request was made using a Worker script.",
"RayID": "string; ID of the request.",
"RequestHeaders": "object; String key-value pairs for RequestHeaders. This field is populated based on [Logpush Custom fields] (<https://developers.cloudflare.com/logs/reference/custom-fields/>), which need to be configured.",
"ResponseHeaders": "object; String key-value pairs for ResponseHeaders. This field is populated based on [Logpush Custom fields] (<https://developers.cloudflare.com/logs/reference/custom-fields/>), which need to be configured.",
"WAFFlags": "string; Additional configuration flags: simulate (0x1) | null.",
"WAFMatchedVar": "string; The full name of the most-recently matched variable.",
"WorkerCPUTime": "int; Amount of time in microseconds spent executing a worker, if any.",
"WorkerStatus": "string; Status returned from worker daemon.",
"WorkerSubrequest": "bool; Whether or not this request was a worker subrequest.",
"WorkerSubrequestCount": "int; Number of subrequests issued by a worker when handling this request."
"ZoneName": "string; the human-readable name of the zone"

Logpull example

The following is an example `logpull` call and examples of specific types of responses.

■ Request

```
ibmcloud cis logpull DNS_DOMAIN_ID --start 2019-01-02T01:00:00+00:00 --end 2019-01-02T01:00:00+00:00 --fields
ClientRequestURI,EdgeResponseBytes,ParentRayID,WorkerStatus,
OriginResponseTime,EdgeResponseStatus,WorkerSubrequest,ClientRequestProtocol,EdgePathingOp,ClientSrcPort,WorkerSubrequestCount,EdgeReque
stHost,
```

ClientSSLCipher,EdgePathingSrc,OriginResponseStatus,ClientIPClass,EdgeColoID,ClientCountry,ClientRequestHost,WAFFlags,ClientASN,EdgeServerIP,CacheCacheStatus,SecurityLevel,ClientRequestUserAgent,CacheResponseBytes,EdgeStartTimestamp,ClientSSLProtocol,EdgeEndTimestamp,EdgeResponseContentType,ClientRequestBytes,CacheResponseStatus,WorkerCPUTime,RayID,ClientRequestMethod,ClientIP,ClientRequestPath,OriginResponseHTTPExpires,CacheTieredFill,WAFRuleMessage,EdgePathingStatus,ClientDeviceType,OriginSSLProtocol,EdgeRateLimitAction,OriginIP,EdgeRateLimitID,ZoneName,EdgeResponseCompressionRatio,ClientRequestReferer, OriginResponseHTTPLastModified,OriginResponseBytes --timestamps=rfc3339'

■ Response with Status Code 200

```
{
  "CacheCacheStatus": "unknown",
  "CacheResponseBytes": 396,
  "CacheResponseStatus": 200,
  "CacheTieredFill": false,
  "ClientASN": 56046,
  "ClientCountry": "cn",
  "ClientDeviceType": "desktop",
  "ClientIP": "1.1.1.1",
  "ClientIPClass": "noRecord",
  "ClientRequestBytes": 400,
  "ClientRequestHost": "foo.com",
  "ClientRequestMethod": "GET",
  "ClientRequestPath": "/",
  "ClientRequestProtocol": "HTTP/1.1",
  "ClientRequestReferer": "",
  "ClientRequestURI": "/",
  "ClientRequestUserAgent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_13_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/71.0.3578.98 Safari/537.36",
  "ClientSSLCipher": "NONE",
  "ClientSSLProtocol": "none",
  "ClientSrcPort": 4532,
  "EdgeColoID": 134,
  "EdgeEndTimestamp": "2019-01-03T01:54:11Z",
  "EdgePathingOp": "wl",
  "EdgePathingSrc": "macro",
  "EdgePathingStatus": "nr",
  "EdgeRateLimitAction": "",
  "EdgeRateLimitID": 0,
  "EdgeRequestHost": "foo.com",
  "EdgeResponseBytes": 808,
  "EdgeResponseCompressionRatio": 1.57,
  "EdgeResponseContentType": "text/html",
  "EdgeResponseStatus": 200,
  "EdgeServerIP": "172.69.98.106",
  "EdgeStartTimestamp": "2019-01-03T01:54:11Z",
  "OriginIP": "2.2.2.2",
  "OriginResponseBytes": 0,
  "OriginResponseHTTPExpires": "",
  "OriginResponseHTTPLastModified": "Tue, 31 Jan 2017 15:01:11 UTC",
  "OriginResponseStatus": 200,
  "OriginResponseTime": 7000000,
  "OriginSSLProtocol": "unknown",
  "ParentRayID": "00",
  "RayID": "4931d60516c0b0b0",
  "SecurityLevel": "med",
  "WAFFlags": "0",
  "WAFMatchedVar": "",
  "WAFProfile": "unknown",
  "WAFRuleID": "",
  "WAFRuleMessage": "",
  "WorkerCPUTime": 0,
  "WorkerStatus": "unknown",
  "WorkerSubrequest": false,
  "WorkerSubrequestCount": 0,
  "ZoneName": "example.com"
}
```

■ Response with Status Code 404


```
{
  "CacheCacheStatus": "miss",
  "CacheResponseBytes": 209,
  "CacheResponseStatus": 404,
  "CacheTieredFill": false,
  "ClientASN": 56046,
  "ClientCountry": "cn",
  "ClientDeviceType": "desktop",
  "ClientIP": "1.1.1.1",
  "ClientIPClass": "noRecord",
  "ClientRequestBytes": 433,
  "ClientRequestHost": "foo.com",
  "ClientRequestMethod": "GET",
  "ClientRequestPath": "/favicon.ico",
  "ClientRequestProtocol": "HTTP/1.1",
  "ClientRequestReferer": "foo.com/",
  "ClientRequestURI": "/favicon.ico",
  "ClientRequestUserAgent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_13_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/71.0.3578.98 Safari/537.36",
  "ClientSSLCipher": "NONE",
  "ClientSSLProtocol": "none",
  "ClientSrcPort": 4532,
  "EdgeColoID": 134,
  "EdgeEndTimestamp": "2019-01-03T01:54:12Z",
  "EdgePathingOp": "wl",
  "EdgePathingSrc": "macro",
  "EdgePathingStatus": "nr",
  "EdgeRateLimitAction": "",
  "EdgeRateLimitID": 0,
  "EdgeRequestHost": "foo.com",
  "EdgeResponseBytes": 556,
  "EdgeResponseCompressionRatio": 2.87,
  "EdgeResponseContentType": "text/html",
  "EdgeResponseStatus": 404,
  "EdgeServerIP": "172.69.98.148",
  "EdgeStartTimestamp": "2019-01-03T01:54:12Z",
  "OriginIP": "2.2.2.2",
  "OriginResponseBytes": 0,
  "OriginResponseHTTPExpires": "",
  "OriginResponseHTTPLastModified": "",
  "OriginResponseStatus": 404,
  "OriginResponseTime": 7000000,
  "OriginSSLProtocol": "unknown",
  "ParentRayID": "00",
  "RayID": "4931d60a16c8b0b0",
  "SecurityLevel": "med",
  "WAFFlags": "0",
  "WAFMatchedVar": "",
  "WAFProfile": "unknown",
  "WAFRuleMessage": "",
  "WorkerCPUTime": 0,
  "WorkerStatus": "unknown",
  "WorkerSubrequest": false,
  "WorkerSubrequestCount": 0,
  "ZoneName": "example.com"
}
```

■ Request matched a WAF rule (SQLj attack)

```
{
  "CacheCacheStatus": "unknown",
  "CacheResponseBytes": 0,
  "CacheResponseStatus": 0,
  "CacheTieredFill": false,
  "ClientASN": 56046,
  "ClientCountry": "cn",
  "ClientDeviceType": "desktop",
  "ClientIP": "1.1.1.1",
  "ClientIPClass": "noRecord",
  "ClientRequestBytes": 501,
```



```
"ClientRequestHost":"foo.com",
"ClientRequestMethod":"GET",
"ClientRequestPath":"/login.php",
"ClientRequestProtocol":"HTTP/1.1",
"ClientRequestReferer": "",
"ClientRequestURI":"/login.php?username=asdf&password=asdf",
"ClientRequestUserAgent":"Mozilla/5.0 (Macintosh; Intel Mac OS X 10.13; rv:60.0) Gecko/20100101 Firefox/60.0",
"ClientSSLCipher":"NONE",
"ClientSSLProtocol":"none",
"ClientSrcPort":48718,
"EdgeColoID":134,
"EdgeEndTimestamp":"2019-01-04T02:22:26Z",
"EdgePathingOp":"wl",
"EdgePathingSrc":"macro",
"EdgePathingStatus":"nr",
"EdgeRateLimitAction": "",
"EdgeRateLimitID":0,
"EdgeRequestHost": "",
"EdgeResponseBytes":1849,
"EdgeResponseCompressionRatio":2.82,
"EdgeResponseContentType":"text/html",
"EdgeResponseStatus":403,
"EdgeServerIP": "",
"EdgeStartTimestamp":"2019-01-04T02:22:26Z",
"OriginIP": "",
"OriginResponseBytes":0,
"OriginResponseHTTPExpires": "",
"OriginResponseHTTPLastModified": "",
"OriginResponseStatus":0,
"OriginResponseTime":0,
"OriginSSLProtocol":"unknown",
"ParentRayID":"00",
"RayID":"493a3cc9463eb0d4",
"SecurityLevel":"med",
"WAFFlags":"0",
"WAFMatchedVar":"ARGS:USERNAME",
"WAFProfile":"off",
"WAFRuleMessage":"SQLi probing",
"WorkerCPUTime":0,
"WorkerStatus":"unknown",
"WorkerSubrequest":false,
"WorkerSubrequestCount":0,
"ZoneName":"example.com"
}
```

■ Request matched a firewall rule

```
{
"CacheCacheStatus":"unknown",
"CacheResponseBytes":0,
"CacheResponseStatus":0,
"CacheTieredFill":false,
"ClientASN":36351,
"ClientCountry":"us",
"ClientDeviceType":"desktop",
"ClientIP":"1.1.1.1",
"ClientIPClass":"noRecord",
"ClientRequestBytes":90,
"ClientRequestHost":"foo.com",
"ClientRequestMethod":"GET",
"ClientRequestPath":"/",
"ClientRequestProtocol":"HTTP/1.1",
"ClientRequestReferer": "",
"ClientRequestURI":"/",
"ClientRequestUserAgent":"curl/7.47.0",
"ClientSSLCipher":"NONE",
"ClientSSLProtocol":"none",
"ClientSrcPort":57260,
"EdgeColoID":26,
"EdgeEndTimestamp":"2019-01-03T08:48:42Z",
```



```
"EdgePathingOp": "ban",
"EdgePathingSrc": "user",
"EdgePathingStatus": "ip",
"EdgeRateLimitAction": "",
"EdgeRateLimitID": 0,
"EdgeRequestHost": "",
"EdgeResponseBytes": 3556,
"EdgeResponseCompressionRatio": 0,
"EdgeResponseContentType": "text/html",
"EdgeResponseStatus": 403,
"EdgeServerIP": "",
"EdgeStartTimestamp": "2019-01-03T08:48:42Z",
"OriginIP": "",
"OriginResponseBytes": 0,
"OriginResponseHTTPExpires": "",
"OriginResponseHTTPLastModified": "",
"OriginResponseStatus": 0,
"OriginResponseTime": 0,
"OriginSSLProtocol": "unknown",
"ParentRayID": "00",
"RayID": "493a6341d02565e7",
"SecurityLevel": "med",
"WAFFlags": "0",
"WAFMatchedVar": "",
"WAFProfile": "unknown",
"WAFRuleMessage": "",
"WorkerCPUTime": 0,
"WorkerStatus": "unknown",
"WorkerSubrequest": false,
"WorkerSubrequestCount": 0,
"ZoneName": "example.com"
}
```

■ Request was rate limited

```
{
"CacheCacheStatus": "unknown",
"CacheResponseBytes": 0,
"CacheResponseStatus": 0,
"CacheTieredFill": false,
"ClientASN": 36351,
"ClientCountry": "us",
"ClientDeviceType": "desktop",
"ClientIP": "1.1.1.1",
"ClientIPClass": "noRecord",
"ClientRequestBytes": 90,
"ClientRequestHost": "foo.com",
"ClientRequestMethod": "GET",
"ClientRequestPath": "/",
"ClientRequestProtocol": "HTTP/1.1",
"ClientRequestReferer": "",
"ClientRequestURI": "/",
"ClientRequestUserAgent": "curl/7.47.0",
"ClientSSLCipher": "NONE",
"ClientSSLProtocol": "none",
"ClientSrcPort": 33186,
"EdgeColoID": 26,
"EdgeEndTimestamp": "2019-01-03T08:59:55Z",
"EdgePathingOp": "ban",
"EdgePathingSrc": "user",
"EdgePathingStatus": "rateLimit",
"EdgeRateLimitAction": "ban",
"EdgeRateLimitID": 1307134,
"EdgeRequestHost": "",
"EdgeResponseBytes": 3559,
"EdgeResponseCompressionRatio": 0,
"EdgeResponseContentType": "text/html",
"EdgeResponseStatus": 429,
"EdgeServerIP": "",
"EdgeStartTimestamp": "2019-01-03T08:59:55Z",
```

```
"OriginIP":"","  
"OriginResponseBytes":0,  
"OriginResponseHTTPExpires":"","  
"OriginResponseHTTPLastModified":"","  
"OriginResponseStatus":0,  
"OriginResponseTime":0,  
"OriginSSLProtocol":"unknown",  
"ParentRayID":"00",  
"RayID":"493a73ad468419b6",  
"SecurityLevel":"med",  
"WAFFlags":"0",  
"WAFMatchedVar":"","  
"WAFProfile":"unknown",  
"WAFRuleMessage":"","  
"WorkerCPUTime":0,  
"WorkerStatus":"unknown",  
"WorkerSubrequest":false,  
"WorkerSubrequestCount":0,  
"ZoneName":"example.com"  
}
```

■ Origin server is down (Error 521, Web Server is down)

```
{  
"CacheCacheStatus":"miss",  
"CacheResponseBytes":177,  
"CacheResponseStatus":521,  
"CacheTieredFill":false,  
"ClientASN":56046,  
"ClientCountry":"cn",  
"ClientDeviceType":"desktop",  
"ClientIP":"1.1.1.1",  
"ClientIPClass":"noRecord",  
"ClientRequestBytes":1082,  
"ClientRequestHost":"foo.com",  
"ClientRequestMethod":"GET",  
"ClientRequestPath":"/favicon.ico",  
"ClientRequestProtocol":"HTTP/2",  
"ClientRequestReferer":"","  
"ClientRequestURI":"/favicon.ico",  
"ClientRequestUserAgent":"Mozilla/5.0 (Macintosh; Intel Mac OS X 10.13; rv:60.0) Gecko/20100101 Firefox/60.0",  
"ClientSSLCipher":"AEAD-AES128-GCM-SHA256",  
"ClientSSLProtocol":"TLSv1.3",  
"ClientSrcPort":3060,  
"EdgeColoID":134,  
"EdgeEndTimestamp":"2019-01-03T06:33:55Z",  
"EdgePathingOp":"wl",  
"EdgePathingSrc":"macro",  
"EdgePathingStatus":"nr",  
"EdgeRateLimitAction":"","  
"EdgeRateLimitID":0,  
"EdgeRequestHost":"foo.com",  
"EdgeResponseBytes":5177,  
"EdgeResponseCompressionRatio":0,  
"EdgeResponseContentType":"text/html",  
"EdgeResponseStatus":521,  
"EdgeServerIP":"172.69.98.148",  
"EdgeStartTimestamp":"2019-01-03T06:33:55Z",  
"OriginIP":"2.2.2.2",  
"OriginResponseBytes":0,  
"OriginResponseHTTPExpires":"","  
"OriginResponseHTTPLastModified":"","  
"OriginResponseStatus":0,  
"OriginResponseTime":3000000,  
"OriginSSLProtocol":"unknown",  
"ParentRayID":"00",  
"RayID":"49336fc9397ab080",  
"SecurityLevel":"med",  
"WAFFlags":"0",  
"WAFMatchedVar":"","
```

```
"WAFProfile":"unknown",
"WAFRuleMessage":"","
"WorkerCPUTime":0,
"WorkerStatus":"unknown",
"WorkerSubrequest":false,
"WorkerSubrequestCount":0,
"ZoneName":"example.com"
}
```

Limitations

The following usage restrictions apply when using the Logpull feature.

- **Rate limits:** Exceeding these limit results in a `429` error response:
 - 15 requests per minute per zone
 - 180 requests per minute per user
- **Time range:** The maximum difference between the start and end parameters can be 1 hour.
- **Response size:** The maximum response size is 10 GiB per request, which is equivalent to around 15 M records when about 55 fields are selected. More records can be retrieved when fewer fields are selected because the per-record size is smaller.
- **Timeout:** The response will fail with a terminated connection after 10 minutes.
- **Stream timeout:** The request will be terminated with a `408` error response if the connection is idle for 30 seconds. This timeout usually means that the request is too exhaustive (frequent timeouts - more than 12 per hour). Stream timeouts will result in subsequent queries getting blocked with a status code `429` for 1 hour. To avoid the timeout, try requesting records by using fewer number of fields, or try with smaller start and end parameters.

Looking up logs by Ray ID

Enterprise accounts can retrieve the log event that is associated with a specific Ray ID. Use the Ray ID from a response to investigate a request that passed through IBM Cloud Internet Services.

Before you begin

1. Install the CIS CLI plug-in and set the context instance. For more information, see the [CIS CLI reference](#).
2. Enable log retention for the DNS domain. Edge logs are not retained by default.

```
$ ibmcloud cis log-retention-update DNS_DOMAIN_ID --flag on
```

When log retention is enabled, logs are retained for 7 days. For more information, see [Enabling log retention](#).

Finding the Ray ID

The Ray ID is returned in the `cf-ray` response header for a request that passes through CIS.



Note: If the Ray ID includes a data center suffix, such as `12ab34cdef567gh8-XXX`, remove the suffix before you look up the log. In this example, use `12ab34cdef567gh8`.

Looking up a log event

Run the following command:

```
$ ibmcloud cis log-lookup-rayid DNS_DOMAIN_ID RAY_ID [--fields FIELDS] [--timestamps FORMAT] [-i, --instance INSTANCE] [--output FORMAT]
```

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`RAY_ID`

The Ray ID of the request. Required.

`--fields`

A comma-separated list of case-sensitive log fields to return. If this option is not specified, the service returns its default fields.

`--timestamps`

The format for timestamp fields. Valid values are `unix`, `unixnano`, and `rfc3339`. The default value is `unix`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

For the complete command reference, see [ibmcloud cis log-lookup-rayid](#).

Examples

Look up a log event and display the default table output:

```
$ ibmcloud cis log-lookup-rayid DNS_DOMAIN_ID RAY_ID
```

Return selected fields with RFC 3339 timestamps as JSON:

```
$ ibmcloud cis log-lookup-rayid DNS_DOMAIN_ID RAY_ID --fields RayID,ClientIP,EdgeResponseStatus --timestamps rfc3339 --output json
```

The JSON response is similar to the following example:

```
{
  "ClientIP": "68.278.11.89",
  "EdgeResponseStatus": 403,
  "RayID": "48b371889c489b2c"
}
```

If the service returns no log event for the Ray ID, the command reports that the server returned an empty response for that Ray ID.

Managing Logpush jobs

IBM Cloud® Internet Services Enterprise-level plans have access to detailed logs of HTTP, DNS, and Range requests, as well as firewall events for their domains. These logs are helpful for debugging and analytics, especially when combined with other data sources, such as ingress or application server logs at the origin.

Before you begin

Before you create a Logpush job by using the console, review the following information and satisfy any prerequisites:

- Currently, the CIS console supports the following destinations:
 - IBM Cloud Logs
 - Cloud Object Storage
 - IBM QRadar
 - Splunk
- Make sure to [enable log retention](#) before you use Logpush.
- If your destination is not explicitly supported by CIS, it might still be accessible by Logpush with a Custom HTTP destination. This includes your own custom HTTP log servers.



Important: To avoid errors, make sure that the destination can accept a gzipped file upload that is named `test.txt.gz`, containing the compressed content `{"content":"tests"}`.

- For Splunk only: When you send logs to QRadar or Splunk, CIS checks the IP address and port for accessibility and validates the certificate of the HTTP Receiver. If all parameters are valid, the Logpush job is created and begins sending events to the HTTP Receiver log source (QRadar) or the HTTP Event Collector (Splunk).

Creating a Logpush job in the console

To create a Logpush job in the console, follow these steps:

1. In the CIS console, navigate to **Account > Logs**, and then click **Create**.
2. Select and configure destination service types from the available options:

IBM Cloud Logs

1. Select **IBM Cloud Logs** from the service type options.
2. Click **Next**.
3. Enter the **Instance ID**.
4. Select the Instance **Region**.
5. Enter the **API key**.

An API key for the account where the IBM Cloud Logs instance is set up is required. You can use either a user API key or a service ID API key. This key is used to generate a bearer token for the Logpush job. The API key can be rotated by using the [ibmcloud cis logpush-job-create](#) CLI to create a Logpush job.

To create a Logpush job for a specific domain and enable the job, run the following command:

```
$ ibmcloud cis logpush-job-create DNS_DOMAIN_ID --destination PATH --name JOB_NAME --fields all --enable true
```

Where:

-destination : Specifies the path to the destination. Paths for supported destinations are as follows:

IBM Cloud Logs
ibmcl://<INSTANCE_ID>.ingress.<REGION>.logs.cloud.ibm.com/logs/v1/singles?ibm_api_key=<IBM_API_KEY> For example: <code>ibmcl://604a309c-585c-4a42-955d-76239ccc1905.ingress.us-south.logs.cloud.ibm.com/logs/v1/singles?ibm_api_key=zxzeNQI22dPwxxxxxxxx9jxdtn1EVK</code> An API key for the account where the IBM Cloud Logs instance is set up is required. You can use either a user API key or a service ID API key. This key is used to generate a bearer token for the Logpush job. The API key can be rotated by using the Update a Logpush job API. Important: The user or the service ID must be granted the Sender IAM role on the IBM Cloud Logs service.
IBM Cloud Logs path
Cloud Object Storage
cos://<BUCKET_OBJECT_PATH>?region=<REGION>&instance-id=<IBM_CLOUD_OBJECT_STORAGE_INSTANCE_ID>&ibm_api_key=<IBM_API_KEY> For example: <code>cos://cis-test-bucket/logs?region=us&instance-id=f75e6d90-4212-4026-851c-d572071146cd</code>
Cloud Object Storage path
IBM QRadar
<QRADAR_URL>:<LOG_SOURCE_PORT> For example: <code>https://qradar.example.com:8088</code>
IBM QRadar path
Splunk


```
splunk://<SPLUNK_ENDPOINT_URL>?channel=<SPLUNK_CHANNEL_ID>&header_Authorization=
<SPLUNK_AUTH_TOKEN>&sourcetype=<SOURCE_TYPE>&insecure-skip-verify=<INSECURE_SKIP_VERIFY>
```

For example:

```
splunk://cis-analytics.ibm.com:8088/services/collector/raw?header_Authorization=Splunk%20e6d94e8c-5792-4ad1-be3c-29bcaee0197d&channel=f1aafa4f-684f-4c29-a040-7fe184b03712&sourcetype=cloudflare:json&insecure-skip-verify=false
```

Splunk path

Custom HTTP

```
https://<HOSTNAME>?header_Authorization=Basic%20REDACTED&tags=host:<DOMAIN_NAME>,dataset:<LOGPUSH_DATASET>
```

For example:

```
https://logs.example.com?header_Authorization=a64Vxxxxx5Aq
```

Custom HTTP path

- `--name` : Specifies the Logpush job name.
- `--fields` : Specifies the list of log fields to be included in log files. Use commas to separate multiple fields. Use the command `ibmcloud cis logpush-available-fields DNS_DOMAIN_ID --dataset DATASET` to get a comprehensive list of available log fields, or use `all` to include all available fields in the log files.
- `--enable` : Is the flag to enable or disable the Logpush job. Valid values are `true` or `false` (default).

Command examples

CLI examples for the supported destinations:

IBM Cloud Logs

Example

```
$ ibmcloud cis logpush-job-create 601b728b86e630c744c81740f72570c3 --destination "ibmcl://604a309c-585c-4a42-955d-76239ccc1905.ingress.us-south.logs.cloud.ibm.com/logs/v1/singles?ibm_api_key=xxxxxxx" --name logpushJobGen --enable true --fields RayID --dataset http_requests --frequency high --instance 1a9174b6-0106-417a-844b-c8eb43a72f63
```

Cloud Object Storage

Example

```
$ ``sh {: pre}
  ibmcloud cis logpush-job-create 31984fea73a15b45779fa0df4ef62f9b --destination cos://cis-test-bucket/logs/{DATE}?region=us&instance-id=f75e6d90-4212-4026-851c-d572071146cd&ibm_api_key=XXX --name logpushcreate --enable true --fields all --timestamps rfc3339 --dataset http_requests --frequency low -i cis-demo --output JSON
  ``
```

IBM QRadar

Example

```
$ ibmcloud cis logpush-job-create 31984fea73a15b45779fa0df4ef62f9b --destination https://qradar.example.com:8088 --name logpushJobQRadar --enable true --fields RayID --dataset http_requests --frequency high --instance 1a9174b6-0106-417a-844b-c8eb43a72f63
```

Splunk

Example

```
$ ibmcloud cis logpush-job-create 601b728b86e630c744c81740f72570c3 --destination splunk://cis-analytics.ibm.com:8088/services/collector/raw?header_Authorization=Splunk%20e6d94e8c-5792-4ad1-be3c-29bcaee0197d&channel=f1aafa4f-684f-4c29-a040-7fe184b03712&sourcetype=cloudflare:json&insecure-skip-verify=false --name logpushJobSplunk --enable true --fields RayID --dataset http_requests --frequency high --instance 1a9174b6-0106-417a-844b-
```

Custom HTTP

Example

```
$ ibmcloud cis logpush-job-create 601b728b86e630c744c81740f72570c3 --destination https://logs.example.com?header_Authorization=a64VuywesDu5Aq" --name logpushJobGen --enable true --fields RayID --dataset http_requests --frequency high --instance 1a9174b6-0106-417a-844b-c8eb43a72f63
```

Creating a Logpush job with the API

You can use the [Create a Logpush job](#) API to create a Logpush job when using IBM Cloud Logs, Cloud Object Storage, or Splunk.

Getting the available log fields for a dataset with the API

Log fields can be specified in the `logpull_options` of a Logpush Job to customize what is sent to the destination. To get the available log fields for a Logpush dataset, follow these steps:

- a. Set up your API environment with the correct variables.
- b. Store the following values in variables to be used in the API command:

`CRN` : The full URL-encoded CRN of the service instance.

`ZONE_ID` : The domain ID.

`DATASET` : The Logpush dataset being inspected. One of `http_requests` , `range_events` , `dns_logs` , `firewall_events` .

- c. When all variables are initiated, create the Logpush job:

```
$ curl -X GET https://api.cis.cloud.ibm.com/v2/$CRN/zones/$ZONE_ID/logpush/datas/$DATASET/fields \
--header "Content-Type: application/json" \
--header "X-Auth-User-Token: Bearer $IAM_TOKEN"
```

Creating a Logpush job to send logs to your destination

To create a Logpush job to your destination (IBM Cloud Logs, Cloud Object Storage, or Splunk), follow these steps:

- a. Set up your API environment with the correct variables.
- b. Store the following values in variables to be used in the API command:

`CRN` : The full URL-encoded Cloud Resource Name (CRN) of the service instance.

`ZONE_ID` : The domain ID.

`--request body` : Information to create the Logpush job body (`logpush_job_DESTINATION_req`) where `DESTINATION` is one of the following values:

IBM Cloud Logs

`ibmcl` : Information to identify the IBM Cloud Logs instance where the data is pushed. Fields within the `ibmcl` object are:

- `instance_id` : ID of the IBM Cloud Logs instance.
- `region` : Region of the IBM Cloud Logs instance (for example, `us-south`).
- `api_key` : An API key for the account where the IBM Cloud Logs instance is set up is required. You can use either a user API key or a service ID API key. This key is used to generate a bearer token for the Logpush job. The API key can be rotated by using the [Update a Logpush job](#) API.

Important: The user or the service ID must be granted the **Sender** IAM role on the IBM Cloud Logs service.

`ibmcl_names` : Optional field to customize the application and subsystem names used in IBM Cloud Logs. Fields within the `ibmcl_names` object are:

- `application_name` : Custom application name for the job.
- `subsystem_name` : Custom subsystem name for the job.

IBM Cloud Logs destination

Cloud Object Storage

`cos`: Information to identify the Object Storage bucket where the data is pushed. Fields within the `cos` object are:

- `bucket_name` : Name of your COS bucket where logs are sent (example: `cos-bucket001`).
- `region` : Region of the Cloud Object Storage instance (for example, `us-south`).
- `id` : ID of the Cloud Object Storage instance.
- `ibm_api_key` : An API key for the account where the IBM Cloud Logs instance set up is required. You can use a user API key or a service ID API key. This key generates the bearer token for the Logpush job. To rotate the API key, use the [Update a Logpush job](#) API.

Important: The customer or the service ID must have the **Object Writer** IAM role on the Object Storage bucket.

Cloud Object Storage destination

IBM QRadar

`qradar`: Information to identify the QRadar HTTP Receiver where the data is pushed.

IBM QRadar destination

Splunk

`splunk` : Information to identify the Splunk HTTP Event Collector (HEC) where the data is pushed. Fields within the `splunk` object are:

- `endpoint_url` : URL of the Splunk HEC.
- `channel_id` : A random GUID to uniquely identify the log push.
- `skip_verify` : Boolean flag to skip validation of the HTTP Event Collector certificate. Only set this to `true` when the HEC is using a self-signed certificate.
- `source_type` : The Splunk source type (for example: `cloudflare:json`).
- `auth_token` : The Splunk authorization token.

Splunk destination

Custom HTTP

`destination_conf`: Information to configure the Custom HTTP destination where the data is pushed. Headers can be specified to be used by Logpush with query parameters prefixed by `header_` (for example: `header_Authorization=XXXX`).

Custom HTTP destination

`name` : The name of the Logpush job.

`enabled` : Whether the job is enabled. Either `true` or `false` .

logpull_options : The configuration string. For example, **fields=RayID,ZoneID×tamps=rfc3339** .

dataset : The dataset that is pulled. One of **http_requests** , **dns_logs** , **range_events** , **firewall_events** .

frequency : The frequency at which CIS sends batches of logs to your destination. Either **high** or **low** .

- c. When all variables are initiated, create the Logpush job:

IBM Cloud Logs

```
$ curl -X POST https://api.cis.cloud.ibm.com/v2/$CRN/zones/$ZONE_ID/logpush/jobs \
--header "Content-Type: application/json" \
--header "X-Auth-User-Token: Bearer $IAM_TOKEN" \
--data '{
  "ibmcl": {
    "instance_id": "f8k3309c-585c-4a42-955d-76239cccf8k3",
    "region": "us-south",
    "api_key": "f8k3NQI22dPwNVCcmS62YFL1tm9vaehY6C9jxdtnf8k3"
  },
  "ibmcl_names": {
    "application_name": "cis-edge-logs",
    "subsystem_name": "stage-firewall-logs"
  }
  "dataset": "http_requests",
  "enabled": true,
  "logpull_options": "fields=RayID,ZoneID&timestamps=rfc3339",
  "name": "CIS-Edge-Requests",
  "frequency": "low"
}'
```

Cloud Object Storage

```
$ curl -X POST https://api.cis.cloud.ibm.com/v2/$CRN/zones/$ZONE_ID/logpush/jobs \
--header "Content-Type: application/json" \
--header "X-Auth-User-Token: Bearer $IAM_TOKEN" \
--data '{
  "cos": {
    "bucket_name": "example_bucket",
    "path": "temp/",
    "id": "cos_instance_id",
    "region": "us-east"
  },
  "dataset": "firewall_events",
  "enabled": false,
  "name": "CIS-Firewall-COS",
  "frequency": "low",
  "logpull_options": "fields=RayID,ZoneID&timestamps=rfc3339",
  "ownership_challenge": "xxxxxxx"
}'
```

IBM QRadar

```
$ curl -X POST https://api.cis.cloud.ibm.com/v2/$CRN/zones/$ZONE_ID/logpush/jobs \
--header "Content-Type: application/json" \
--header "X-Auth-User-Token: Bearer $IAM_TOKEN" \
--data '{
  "qradar": {
    "url": "https://example.qradar.ibmcloud.com",
    "port": 8088
  },
  "dataset": "firewall_events",
  "enabled": false,
  "name": "CIS-Firewall-QRadar",
  "frequency": "low",
  "logpull_options": "fields=RayID,CacheResponseBytes,CacheResponseStatus,CacheCacheStatus&timestamps=rfc3339"
}'
```

Splunk

```
$ curl -X POST https://api.cis.cloud.ibm.com/v2/$CRN/zones/$ZONE_ID/logpush/jobs \
--header "Content-Type: application/json" \
--header "X-Auth-User-Token: Bearer $IAM_TOKEN" \
--data '{
  "splunk": {
    "endpoint_url": "example.splunkcloud.com:8088/services/collector/raw",
    "channel_id": "def3c136-7a01-4655-b17f-8e25a780ef2c",
    "skip_verify": false,
    "source_type": "cloudflare:json",
    "auth_token": "Splunk fake3585-0f38-4d62-8b43-c4b78584fake"
  },
  "dataset": "http_requests",
  "enabled": true,
  "name": "CIS-Splunk-Logpush",
  "frequency": "high",
  "logpull_options": "fields=RayID,CacheResponseBytes,CacheResponseStatus,CacheCacheStatus&timestamps=rfc3339"
}'
```

Custom HTTP

```
$ curl -X POST https://api.cis.cloud.ibm.com/v2/$CRN/zones/$ZONE_ID/logpush/jobs \
--header "Content-Type: application/json" \
--header "X-Auth-User-Token: Bearer $IAM_TOKEN" \
--data '{
  "destination_conf": "https://logs.example.com?header_Authorization=a64VuywesDu5Aq",
  "dataset": "http_requests",
  "enabled": true,
  "name": "CIS-Custom-Logpush",
  "frequency": "high",
  "logpull_options": "fields=RayID,CacheResponseBytes,CacheResponseStatus,CacheCacheStatus&timestamps=rfc3339"
}'
```

Related link

[Managing user API keys](#)

Accessing log fields

If `fields` are not specified in the request, a limited set of default fields are returned. Find the full list of all available fields using the following request.

```
$ ibmcloud cis logpush-available-fields DNS_DOMAIN_ID [--dataset DATASET] [-i, --instance INSTANCE]
```

Datasets

These available datasets describe the fields available by log category:

- HTTP/ HTTPS requests (`http_requests`)
- Firewall events (`firewall_events`)
- Range events (`range_events`)
- DNS logs (`dns_logs`)

Available fields

The following tables indicate the fields available by dataset. Log fields are subject to change. Use the [Available Log Fields API](#) to pull the most up-to-date version of the available fields.

HTTP requests

This table contains the fields available for http_requests.

Field	Description	Type
BotDetectionIDs	List of IDs that correlate to the Bot Management Heuristic detections made on a request. Available only for Bot Management customers. To enable this feature, contact your account team.	array[int]
BotDetectionTags	List of tags that correlate to the Bot Management Heuristic detections made on a request. Available only for Bot Management customers. To enable this feature, contact your account team.	array[string]
BotScore	Cloudflare Bot Score. Scores below 30 are commonly associated with automated traffic. Available only for Bot Management customers. To enable this feature, contact your account team.	int
BotScoreSrc	Heuristics Machine Learning Behavioral Analysis Verified Bot JS Fingerprinting Cloudflare Service. Available only for Bot Management customers. To enable this feature, contact your account team.	string
BotTags	Type of bot traffic (if available). Available only for Bot Management customers. To enable this feature, contact your account team.	array[string]
CacheCacheStatus	miss expired updating stale hit ignored bypass revalidated dynamic stream_hit deferred "dynamic" means that a request is not eligible for cache. This can mean, for example that it was blocked by the firewall.	string
CacheReserveUsed	Cache Reserve was used to serve this request.	bool
CacheResponseBytes	Number of bytes returned by the cache.	int
CacheResponseStatus	HTTP status code returned by the cache to the edge. All requests (including non-cacheable ones) go through the cache. Refer also to CacheCacheStatus field.	int
CacheTieredFill	Tiered Cache was used to serve this request.	bool
ClientASN	Client AS number.	int
ClientCity	Approximate city of the client.	string
ClientCountry	2-letter ISO-3166 country code of the client IP address.	string
ClientDeviceType	Client device type.	string
ClientIP	IP address of the client.	string
ClientIPClass	badHost searchEngine allowlist monitoringService noRecord scan tor .	string
ClientLatitude	Approximate latitude of the client.	string
ClientLongitude	Approximate longitude of the client.	string

ClientMTLSAuthCertFingerprint	The SHA256 fingerprint of the certificate presented by the client during mTLS authentication. Only populated on the first request on an mTLS connection.	string
ClientMTLSAuthStatus	<code>ok</code> <code>absent</code> <code>untrusted</code> <code>notyetvalid</code> <code>expired</code> .	string
ClientRegionCode	The ISO-3166-2 region code of the client IP address.	string
ClientRequestBytes	Number of bytes in the client request.	int
ClientRequestHost	Host requested by the client.	string
ClientRequestMethod	HTTP method of client request.	string
ClientRequestPath	URI path requested by the client, which includes only the path portion of the requested URL, without the query string.	string
ClientRequestProtocol	HTTP protocol of client request.	string
ClientRequestReferer	HTTP request referrer.	string
ClientRequestScheme	The URL scheme requested by the visitor.	string
ClientRequestSource	Identifies requests as coming from an external source or another service within Cloudflare.	string
ClientRequestURI	URI requested by the client, which includes the TLS Mode: End-to-End flexible path and query string of the requested URL.	string
ClientRequestUserAgent	User agent reported by the client.	string
ClientSSLCipher	Client SSL cipher.	string
ClientSSLProtocol	Client SSL (TLS) protocol. The value "none" means that SSL was not used.	string
ClientSrcPort	Client source port.	int
ClientTCPRTTMs	The smoothed average of TCP round-trip time (SRTT). For the initial request on a connection, this is measured only during connection setup. For a subsequent request on the same connection, it is measured over the entire connection lifetime up until the time that request is received.	int
ClientXRequestedWith	X-Requested-With HTTP header.	string
ContentScanObjResults	List of content scan results.	array[string]
ContentScanObjSizes	List of content object sizes.	array[int]
ContentScanObjTypes	List of content types.	array[string]
Cookies	String key-value pairs for Cookies.	object

EdgeCFConnectingO2O	True if the request looped through multiple zones on the Cloudflare edge. This is considered an orange to orange (O2O) request.	bool
EdgeColoCode	IATA airport code of the data center that received the request.	string
EdgeColoID	Cloudflare edge data center ID.	int
EdgeEndTimestamp	Timestamp at which the edge finished sending response to the client.	int or string
EdgePathingOp	Indicates what type of response was issued for this request (unknown = no specific action).	string
EdgePathingSrc	Details how the request was classified based on security checks (unknown = no specific classification).	string
EdgePathingStatus	Indicates what data was used to determine the handling of this request (unknown = no data).	string
EdgeRequestHost	Host header on the request from the edge to the origin.	string
EdgeResponseBodyBytes	Size of the HTTP response body returned to clients.	int
EdgeResponseBytes	Number of bytes returned by the edge to the client.	int
EdgeResponseCompressionRatio	The edge response compression ratio is calculated as the ratio between the sizes of the original and compressed responses.	float
EdgeResponseContentType	Edge response Content-Type header value.	string
EdgeResponseStatus	HTTP status code returned by Cloudflare to the client.	int
EdgeServerIP	IP of the edge server making a request to the origin. Possible responses are string in IPv4 or IPv6 format, or empty string. Empty string means that there was no request made to the origin server.	string
EdgeStartTimestamp	Timestamp at which the edge received request from the client.	int or string
EdgeTimeToFirstByteMs	Total view of Time To First Byte as measured at Cloudflare's edge. Starts after a TCP connection is established and ends when Cloudflare begins returning the first byte of a response to eyeballs. Includes TLS handshake time (for new connections) and origin response time.	int
JA3Hash	The MD5 hash of the JA3 fingerprint used to profile SSL/TLS clients. Available only for Bot Management customers. To enable this feature, contact your account team.	string
JA4	The JA4 fingerprint used to profile SSL/TLS clients. Available only for Bot Management customers. To enable this feature, contact your account team.	string

JA4Signals	Inter-request statistics computed for this JA4 fingerprint. JA4Signals field is organized in key:value pairs, where values are numbers. Available only for Bot Management customers. To enable this feature, contact your account team.	object
JSDetectionPassed	failed missing. Available only for Bot Management customers. To enable this feature, contact your account team.	string
OriginDNSResponseTimeMs	Time taken to receive a DNS response for an origin name. Usually takes a few milliseconds, but may be longer if a CNAME record is used.	int
OriginIP	IP of the origin server.	string
OriginRequestHeaderSendDurationMs	Time taken to send request headers to origin after establishing a connection. Note that this value is usually 0 .	int
OriginResponseBytes	Number of bytes returned by the origin server.	int
OriginResponseDurationMs	Upstream response time, measured from the first datacenter that receives a request. Includes time taken by Smart Routing and Tiered Cache, plus time to connect and receive a response from origin servers. This field replaces OriginResponseTime.	int
OriginResponseHTTPExpires	Value of the origin 'expires' header in RFC1123 format.	string
OriginResponseHTTPLastModified	Value of the origin 'last-modified' header in RFC1123 format.	string
OriginResponseHeaderReceiveDurationMs	Time taken for origin to return response headers after Cloudflare finishes sending request headers.	int
OriginResponseStatus	Status returned by the upstream server. The value 0 means that there was no response received from the origin server and the response was served by Cloudflare's Edge. However, if the zone has a Edge function running on it, the value 0 could be the result of a Edge functions subrequest made to the origin.	int
OriginResponseTime	Number of nanoseconds it took the origin to return the response to edge.	int
OriginSSLProtocol	SSL (TLS) protocol used to connect to the origin.	string
OriginTCPHandshakeDurationMs	Time taken to complete TCP handshake with origin. This will be 0 if an origin connection is reused.	int
OriginTLShandshakeDurationMs	Time taken to complete TLS handshake with origin. This will be 0 if an origin connection is reused.	int
ParentRayID	Ray ID of the parent request if this request was made using a Edge function script.	string
RayID	ID of the request.	string
RequestHeaders	String key-value pairs for RequestHeaders.	object

ResponseHeaders	String key-value pairs for ResponseHeaders.	object
SecurityAction	Action of the security rule that triggered a terminating action, if any.	string
SecurityActions	allow block challenge jschallenge log connectionClose challengeSolved challengeBypassed jschallengeSolved jschallengeBypassed bypass managedChallenge managedChallengeNonInteractiveSolved managedChallengeInteractiveSolved managedChallengeBypassed rewrite forceConnectionClose skip.	array[string]
SecurityRuleDescription	Description of the security rule that triggered a terminating action, if any.	string
SecurityRuleID	Rule ID of the security rule that triggered a terminating action, if any.	string
SecurityRuleIDs	Array of rule IDs of the security product that matched the request. The security product associated with the rule ID can be found in SecuritySources. The length of the array is the same as SecurityActions and SecuritySources.	array[string]
SecuritySources	asn country ip ipRange securityLevel zoneLockdown waf firewallRules uaBlock rateLimit bic hot l7ddos validation botFight apiShield botManagement dlp firewallManaged firewallCustom apiShieldSchemaValidation apiShieldTokenValidation apiShieldSequenceMitigation.	array[string]
SmartRouteColoID	The Cloudflare data center used to connect to the origin server if Smart Routing is used.	int
UpperTierColoID	The "upper tier" data center that was checked for a cached copy if Tiered Cache is used.	int
VerifiedBotCategory	The category of verified bot.	string
WAFAttackScore	Overall request score generated by the WAF detection module.	int
WAFFlags	null.	string
WAFMatchedVar	The TLS Mode: End-to-End flexible name of the most-recently matched variable.	string
WAFRCEAttackScore	WAF score for an RCE attack.	int
WAFSQLiAttackScore	WAF score for an SQLi attack.	int
WAFXSSAttackScore	WAF score for an XSS attack.	int
WorkerCPUTime	Amount of time in microseconds spent executing a Edge function, if any.	int
WorkerScriptName	The Edge function script name that made the request.	string
WorkerStatus	Status returned from Edge function daemon.	string

WorkerSubrequest	Whether or not this request was a Edge function subrequest.	bool
WorkerSubrequestCount	Number of subrequests issued by a Edge function when handling this request.	int
WorkerWallTimeUs	The elapsed time in microseconds between the start of a Edge function invocation, and when the Edge functions Runtime determines that no more JavaScript needs to run. Specifically, this measures the wall-clock time that the JavaScript context remained open. For example, when returning a response with a large body, the Edge functions runtime can, in some cases, determine that no more JavaScript needs to run, and closes the JS context before all the bytes have passed through and been sent. Alternatively, if you use the <code>waitUntil()</code> API to perform work without blocking the return of a response, this work may continue executing after the response has been returned, and will be included in <code>Edge functionWallTimeUs</code> .	int
ZoneName	The human-readable name of the zone (for example, 'cloudflare.com').	string

Available log fields for HTTP requests

DNS logs

This table contains the fields available for dns_logs.

Field	Description	Type
ColoCode	IATA airport code of the data center that received the request.	string
EDNSSubnet	IPv4 or IPv6 address information corresponding to the EDNS Client Subnet (ECS) forwarded by recursive resolvers. Not all resolvers send this information.	string
EDNSSubnetLength	Size of the EDNS Client Subnet (ECS) in bits. For example, if the last octet of an IPv4 address is omitted (192.0.2.x.), the subnet length will be 24.	int
QueryName	Name of the query that was sent.	string
QueryType	Integer value of query type. For more information, see Query type .	int
ResponseCached	Whether the response was cached or not.	bool
ResponseCode	Integer value of response code. For more information, see Response code .	int
SourceIP	IP address of the client (IPv4 or IPv6).	string
Timestamp	Timestamp at which the query occurred.	int or string

Available log fields for DNS logs

Range requests

This table contains the fields available for range_events.

Field	Description	Type
-------	-------------	------

Application	The unique public ID of the application on which the event occurred.	string
ClientAsn	Client AS number.	int
ClientBytes	The number of bytes read from the client by the Range service.	int
ClientCountry	Country of the client IP address.	string
ClientIP	Client IP address.	string
ClientMatchedIpFirewall	ALLOW BLOCK_ERROR BLOCK_IP BLOCK_COUNTRY BLOCK_ASN WHITELIST_IP WHITELIST_COUNTRY WHITELIST_ASN.	string
ClientPort	Client port.	int
ClientProto	udp unix.	string
ClientTcpRtt	The TCP round-trip time in nanoseconds between the client and Range.	int
ClientTlsCipher	The cipher negotiated between the client and Range. An unknown cipher is returned as "UNK."	string
ClientTlsClientHelloServerName	The server name in the Client Hello message from client to Range.	string
ClientTlsProtocol	none SSLv3 TLSv1 TLSv1.1 TLSv1.2 TLSv1.3.	string
ClientTlsStatus	OK INTERNAL_ERROR INVALID_CONFIG INVALID_SNI HANDSHAKE_FAILED KEYLESS_RPC.	string
ColoCode	IATA airport code of the data center that received the request.	string
ConnectTimestamp	Timestamp at which both legs of the connection (client/edge, edge/origin or nexthop) were established.	int or string
DisconnectTimestamp	Timestamp at which the connection was closed.	int or string
Event	connect disconnect clientFiltered tlsError resolveOrigin originError.	string
IpFirewall	Whether IP Firewall was enabled at time of connection.	bool
OriginBytes	The number of bytes read from the origin by Range.	int
OriginIP	Origin IP address.	string
OriginPort	Origin port.	int
OriginProto	udp unix.	string
OriginTcpRtt	The TCP round-trip time in nanoseconds between Range and the origin.	int
OriginTlsCipher	The cipher negotiated between Range and the origin. An unknown cipher is returned as "UNK."	string
OriginTlsFingerprint	SHA256 hash of origin certificate. An unknown SHA256 hash is returned as an empty string.	string

OriginTlsMode	off TLS Mode: Client-to-Edge TLS Mode: End-to-End flexible TLS Mode: End-to-End CA signed.	string
OriginTlsProtocol	none SSLv3 TLSv1 TLSv1.1 TLSv1.2 TLSv1.3.	string
OriginTlsStatus	OK INTERNAL_ERROR INVALID_CONFIG INVALID_SNI HANDSHAKE_FAILED KEYLESS_RPC.	string
ProxyProtocol	v1 v2 simple.	string
Status	A code indicating reason for connection closure.	int
Timestamp	Timestamp at which the event took place.	int or string

Available log fields for Range requests

Firewall events

This table contains the fields available for firewall_events.

Field	Description	Type
Action	allow block challenge jschallenge log connectionclose challengesolved challengebypassed jschallengesolved jschallengebypassed bypass managedchallenge managedchallengenoninteractivesolved managedchallengeinteractivesolved managedchallengebypassed.	string
ClientASN	The ASN number of the visitor.	int
ClientASNDescription	The ASN of the visitor as string.	string
ClientCountry	Country from which request originated.	string
ClientIP	The visitor's IP address (IPv4 or IPv6).	string
ClientIPClass	badHost searchEngine allowlist monitoringService noRecord scan tor.	string
ClientRefererHost	The referer host.	string
ClientRefererPath	The referer path requested by visitor.	string
ClientRefererQuery	The referer query-string was requested by the visitor.	string
ClientRefererScheme	The referer URL scheme requested by the visitor.	string
ClientRequestHost	The HTTP hostname requested by the visitor.	string
ClientRequestMethod	The HTTP method used by the visitor.	string
ClientRequestPath	The path requested by visitor.	string
ClientRequestProtocol	The version of HTTP protocol requested by the visitor.	string
ClientRequestQuery	The query-string was requested by the visitor.	string
ClientRequestScheme	The URL scheme requested by the visitor.	string

ClientRequestUserAgent	Visitor's user-agent string.	string
ContentScanObjResults	List of content scan results.	array[string]
ContentScanObjSizes	List of content object sizes.	array[int]
ContentScanObjTypes	List of content types.	array[string]
Datetime	The date and time the event occurred at the edge.	int or string
Description	The description of the rule triggered by this request.	string
EdgeColoCode	The airport code of the Cloudflare data center that served this request.	string
EdgeResponseStatus	HTTP response status code returned to browser.	int
Kind	The kind of event, currently only possible values are: firewall.	string
MatchIndex	Rules match index in the chain. The last matching rule will have MatchIndex 0. If another rule matched before the last one, it will have MatchIndex 1. The same applies to any other matching rules, which will have a MatchIndex value of 2, 3, and so on.	int
Metadata	Additional product-specific information. Metadata is organized in key:value pairs. Key and Value formats can vary by Cloudflare security product and can change over time.	object
OriginResponseStatus	HTTP origin response status code returned to browser.	int
OriginatorRayID	The RayID of the request that issued the challenge/jschallenge.	string
RayID	The RayID of the request.	string
Ref	The user-defined identifier for the rule triggered by this request. Use refs to label your rules individually alongside the Cloudflare-provided RuleID. You can set refs via the Rulesets API for some security products.	string
RuleID	The Cloudflare security product-specific RuleID triggered by this request.	string
Source	asn country ip iprange securitylevel zonelockdown waf firewallrules uablock ratelimit bic hot l7ddos validation botfight apishield botmanagement dlp firewallmanaged firewallcustom apishieldschemavalidation apishieldtokenvalidation apishieldsequencemitigation.	string

Available log fields for Firewall events

Using the Instant Logs feature

Enterprise accounts can use the Instant Logs feature to access a live stream of the traffic for their domain from the API or CLI. By watching the data in real time, you can investigate an attack, troubleshoot, debug, or test changes that are made to your network. Instant Logs is lightweight and does not require any extra setup.

Creating Instant Logs from the CLI

To create an Instant Logs Job from the CLI, send a POST request to the Instant Logs job endpoint with the following parameters:

- **Fields:** List any fields that are available in the HTTP request dataset.
- **Sample:** The sample rate of the records set by the client, for instance `"sample": 1` returns 100% of the records and

"sample": 10 returns 10% of the records.



Note: Instant Logs has a maximum data rate supported. For high volume domains, we sample server side as indicated in the "sampleInterval" parameter returned in the logs.

■ **Filters:** Use filters to drill down into specific events. Filters consist of three parts: key, operator, and value.

■ **Keys:**

- ClientASN
- CacheCacheStatus
- ClientCountry
- ClientIP
- ClientRequestHost
- ClientRequestMethod
- ClientRequestPath
- EdgeResponseStatus
- SecurityActions
- SecurityRuleIDs

■ **Operators:** The following table shows the name and behavior of each operator.

Name	Operator
Equals	eq
Not Equals	neq
Greater Than	gt
Greater Than or Equal to	geq
Less Than	lt
Less Than or Equal to	leq
Starts with	startsWith
Ends with	endsWith
Contains	contains
Is in	In

Operator names and behaviors

Creating an Instant Logs job

Create a session by sending a POST request to the Instant Logs job endpoint with the following parameters:

- **Fields** - List any field available in the [HTTP requests dataset](#).
- **Sample** - The sample parameter is the sample rate of the records set by the client: "sample": 1 is 100% of records "sample": 10 is 10% and so on.



Note: Instant Logs has a maximum data rate supported. For high volume domains, we sample server side as indicated in the "sampleInterval" parameter returned in the logs.

- **Filters** - Use filters to drill down into specific events. Filters consist of three parts: key, operator and value. For information about supported operators, see [Using fields, functions, and expressions](#).

Here are three examples of filters:

- Filter when client IP country is not Canada:

```
"filter": "{\n  \"where\": {\n    \"and\": [\n      {\n        \"key\": \"ClientCountry\",\n        \"operator\": \"neq\",\n        \"value\": \"ca\"\n      }\n    ]\n  }\n}"
```

- Filter when the status code returned from CIS is either 200 or 201:

```
"filter": "{\n  \"where\": {\n    \"and\": [\n      {\n        \"key\": \"EdgeResponseStatus\",\n        \"operator\": \"in\",\n        \"value\": \"200,201\"\n      }\n    ]\n  }\n}"
```

- Filter when the request path contains "/static" and the request hostname is "example.com":

```
"filter": "{\n  \"where\": {\n    \"and\": [\n      {\n        \"key\": \"ClientRequestPath\",\n        \"operator\": \"contains\",\n        \"value\": \"/static\"\n      },\n      {\n        \"key\": \"ClientRequestHost\",\n        \"operator\": \"eq\",\n        \"value\": \"example.com\"\n      }\n    ]\n  }\n}"
```

Here is an example request using cURL:

```
curl https://api.cloudflare.com/client/v4/zones/{zone_id}/logpush/edge/jobs \
--header "X-Auth-Email: <EMAIL>" \
--header "X-Auth-Key: <API_KEY>" \
--header "Content-Type: application/json" \
--data '{
  "fields":
    "ClientIP,ClientRequestHost,ClientRequestMethod,ClientRequestURI,EdgeEndTimestamp,EdgeResponseBytes,EdgeResponseStatus,Edge
    StartTimestamp,RayID",
  "sample": 100,
  "filter": "",
  "kind": "instant-logs"
}'
```

Response:

The response includes a new field named **destination_conf**. The value of this field is your unique WebSocket address that receives messages from Cloudflare’s global network.

```
{
  "errors": [],
  "messages": [],
  "result": {
    "id": <JOB_ID>,
    "fields":
      "ClientIP,ClientRequestHost,ClientRequestMethod,ClientRequestURI,EdgeEndTimestamp,EdgeResponseBytes,EdgeResponseStatus,Edge
      StartTimestamp,RayID",
    "sample": 100,
    "filter": "",
    "destination_conf": "wss://logs.cloudflare.com/instant-logs/ws/sessions/<SESSION_ID>",
    "kind": "instant-logs"
  },
  "success": true
}
```

Connecting to WebSocket

Using a CLI utility, you can connect to the WebSocket and start immediately receiving logs.

```
websocat wss://logs.cloudflare.com/instant-logs/ws/sessions/<SESSION_ID>
```

Response:

After you are connected to the websocket, you will receive messages of line-delimited JSON.

Datasets

Currently, the only dataset available is HTTP requests. The list of datasets might expand when other datasets become available through Cloudflare.

Limitations

When the following conditions in the Instant Logs feature are met, the logs stream stops automatically.

- When there is only one active Instant Logs session per domain.
- When the maximum session time is 60 minutes.
- When the connection closes or is not listened to for more than five minutes.

Estimating daily log volume

To estimate your daily log volume, you can look at the relevant count of metrics for your logged datasets and fields.

You can configure a Logpush job to analyze a single log to get the average amount of data per log. To do so, make a GraphQL query with an HTTP dataset (such as `httpRequestsAdaptiveGroups`). Then, multiply the average data per log by the average number of requests per day to get the number of requests per day. For example:

```
echo -n '{"ClientIP":"123.45.67.89","ClientRequestHost":"example.cis.com","ClientRequestMethod":"GET","ClientRequestURI":"/shark/sl-12345","EdgeEndTimestamp":1724947703387000000,"EdgeResponseBytes":495,"EdgeResponseStatus":200,"EdgeStartTimestamp":1724947703256000000,"RayID":"ffadcca953ea3908"}' | \
wc -c
```

The output indicates the log file is 286 bytes in size:

```
286
```

If the domain receives 100 KB requests daily, then the daily log volume calculation would be about 28.6 MB (286 bytes multiplied by 10,000 requests is 28.6 MB).

Enabling enhanced OWASP event logging

To gain deeper insight into potential threats, you can enable logging for all OWASP events that contribute to the overall threat score, not just those events that exceed the configured sensitivity threshold. Enabling enhanced OWASP logging gives you a more complete view of security activity, which helps with analysis, tuning, and audits.

By default (when disabled), only OWASP events that cross the configured sensitivity threshold are logged.

To enable additional OWASP event logging in the console, follow these steps:

- In the CIS console, navigate to the **Account** section.
- In the **Logs** view, enable the **Enhanced OWASP logging** toggle.



Note: When you enable this toggle, the volume of logs generated increases, which can lead to higher storage and processing costs depending on your log retention and usage. For help estimating the impact, see [Estimating daily log volume](#).

After you enable this toggle, all OWASP-triggered events appear in both Logpush jobs (Dataset Firewall Events) and Security Events.

Working with metrics

Setting up GraphQL

GraphQL is a query language for APIs. You can run queries on a single `/graphql` endpoint. GraphiQL uses a built-in introspection system to expose information through the schema.

 **Important:** GraphQL analytics API endpoint is only available for Enterprise-level plans.

Prerequisites

You must have the following permissions to proceed:

- You must have Viewer permission either at the instance level or zone level.
- You must have Reader permission at the zone level.

GraphiQL tool

Using GraphiQL, you can explore the schema and test queries for the GraphQL endpoint.

- a. [Download and install GraphiQL](#).
- b. Open the GraphiQL application and enter the HTTP headers that you want to authorize.
- c. Enter the correct endpoint in the **GraphQL Endpoint** field. Use `https://api.cis.cloud.ibm.com/v1/<crn>/zones/<zoneid>/graphql`.

The window is divided into a query pane and a response pane. You can define the query variables, which are interpolated into the query in the query pane.

To build a query, follow these steps.

- a. Select `POST` from the **Method** list menu.
- b. Click **Edit HTTP Headers**.
- c. Enter the `X-Auth-User-Token` or Authorization token, and set the Content-Type: `application/json`. Be sure to save them.

Start building your query in the query pane. Use the **Document Explorer** to explore the schema and documentation. The Document Explorer shows the different available options for datasets, dimensions, operations, and functions. The words "zone" and "domain" are synonymyous in the Document Explorer.

Paste the following test snippet into the query pane and observe the response in the response pane, adjusting the `datetime` to meet your needs.

```
query {
  viewer {
    zones(filter: {zoneTag: "<your domain ID>"}) {
      httpRequests1hGroups(limit: 5, filter: { datetime_gt: "2020-08-30T04:00:00Z", datetime_lt: "2020-08-31T06:00:00Z"}) {
        sum {
          countryMap {
            bytes
            clientCountryName
          }
        }
        dimensions {
          date
          datetime
        }
      }
    }
    firewallEventsAdaptiveGroups(limit: 10, filter: { datetime_gt: "2020-08-30T04:00:00Z", datetime_lt: "2020-08-31T06:00:00Z"}) {
      count
      dimensions {
        clientCountryName
        clientAsn
        datetimeHour
      }
    }
  }
}
```

```
}

```

Querying basics

GraphQL structures data as a graph. To get the data you need, you can explore the edges of the graph. This is an example query format:

```
viewer {
  zones(filter:...) {
    requests(filter:...) {
      date, time, bytes,...
    }
  }
}
```

The initial node of the user running the query is `viewer`. A viewer is able to access one or more domains (zones). Each zone contains different datasets, such as firewall events for a zone.

Nodes that represent aggregated data include the `groups` suffix, for example, `firewallEventsAdaptiveGroups`. Each group follows a specific structure, shown in the following example.

```
type ExampleGroup {
  count # No subfields, it is just the group size.
  sum {
    # fields that support summing (numbers, maps of numbers)
  }
  avg {
    # fields that support averaging (numbers)
  }
  uniq {
    # fields that support uniqueing (numbers, strings, enums, IPs, dates, etc.)
  }
}
```

This example shows a valid group:

```
httpRequests1mGroups {
  sum {
    bytes
  }
  uniq {
    uniques # unique IPs
  }
  dimensions {
    datetimeMinute
  }
}
```

Datasets

The following is a list of commonly used datasets that are available. For more information on datasets, you can use the [GraphQL introspection mechanism](#).

Dataset	Node
Browser Insights	<code>browserInsightsAdaptiveGroups</code>
Firewall Activity Log	<code>firewallEventsAdaptive</code> <code>firewallEventsAdaptiveByTimeGroups</code>
Firewall Analytics	<code>firewallEventsAdaptiveGroups</code>
Health Check Analytics	<code>healthCheckEvents</code> <code>healthCheckEventsGroups</code>

HTTP Requests	httpRequests1mGroups httpRequests1hGroups httpRequests1dGroups httpRequestsAdaptiveGroups
Image Resizing Analytics	imageResizingRequests1mGroups
Load Balancing Analytics	loadBalancingRequests loadBalancingRequestsGroups
SYN Attacks (DoS Analytics)	synAvgPps1mGroups
Edge Functions Metrics	workersInvocationsAdaptive

Commonly used GraphQL datasets

Errors

The GraphQL Analytics API is a RESTful API based on HTTPS requests and JSON responses and returns familiar HTTP status codes (for example, `404` , `500` , `504`). In conformity to the GraphQL specification, a `200` response can contain an error, which is in contrast to the common REST approach.

All responses contain an errors array, which is null if there are no errors. Non-null errors contain `message` , `path` , and `timestamp` .

The following code is an example error response:

```
{
  "data": null,
  "errors": [
    {
      "message": "cannot request data older than 2678400s",
      "path": [
        "viewer",
        "zones",
        "0",
        "firewallEventsAdaptiveGroups"
      ],
      "extensions": {
        "timestamp": "2019-12-09T21:27:19.195060142Z"
      }
    }
  ]
}
```

Limits

The limits for retaining historical data are defined in the following table.

Data node	Enterprise
browserInsightsAdaptiveGroups	30 days
firewallEventsAdaptiveByTimeGroups	30 days
firewallEventsAdaptiveGroups	30 days
healthCheckEventsGroups	90 days
healthCheckEvents	90 days
httpRequestsAdaptiveGroups	30 days

httpRequests1dGroups	365 days
httpRequests1hGroups	90 days
httpRequests1mGroups	7 days
loadBalancingRequestsGroups	30 days
loadBalancingRequests	30 days
synAvgPps1mGroups	7 days

Historical data limits

Query settings for account limits

To obtain specific information regarding limits for a data node, use the `settings` node.

Field	Description
enabled	Returns <code>true</code> if the data set (node) is available for the current plan.
maxDuration	Defines the maximum time period (in seconds) that can be requested in one query (varies by data node).
maxNumberOfFields	Defines the maximum number of fields that can be requested in one query (varies by data node).
maxPageSize	Defines the maximum number of records that can be returned in one query (varies by data node).
notOlderThan	Limits how far back in the record a query can search (in seconds, varies by data node and plan).

Settings node fields

The following is an example query:

```
{
  viewer {
    zones(filter: { zoneTag: $zoneTag }) {
      settings {
        browserInsightsAdaptiveGroups {
          maxDuration
          maxNumberOfFields
          maxPageSize
          enabled
          notOlderThan
        }
      }
    }
  }
}
```

The query response follows:

```
{
  "data": {
    "viewer": {
      "zones": [
        {
          "settings": {
            "browserInsightsAdaptiveGroups": {
              "enabled": true,
              "maxDuration": 2592000,
              "maxNumberOfFields": 30,
              "maxPageSize": 10000,
```



```

        "notOlderThan": 2595600
      }
    }
  }
]
}
},
"errors": null
}

```

Query limits

The volume of data that a query can return is limited, and there are user limits on daily data volume. The following limits apply in addition to general rate limits enforced by the API:

- A zone-scoped query can include up to 10 zones.
- Queries can request up to 30 fields as indicated by `maxNumberOfFields` in `settings` .
- Responses can return up to 10,000 records. This limit is indicated by `maxPageSize` in `settings` .

Queries must explicitly specify the maximum number of records to return using the `limit` argument.

Sorting

You can sort the order of query result elements using the `orderBy` argument. By default, results are sorted by the primary key of the dataset (table). If you specify another field to sort on, the primary key is included in the sorting key to keep consistent results for pagination.



Note: Ordering within nested structures is not supported.

Sorting examples

Raw data sorting:

```

firewallEventsAdaptive (orderBy: [clientCountryName_ASC]) {
  clientCountryName
}

```

Raw data sorting using multiple fields:

```

firewallEventsAdaptive (orderBy: [clientCountryName_ASC, datetime_DESC]) {
  clientCountryName
  datetime
}

```

Group sorting by aggregation function:

```

httpRequests1hGroups (orderBy: [sum_bytes_DESC]){
  sum {
    bytes
    requests
  }
  dimensions {
    datetime
  }
}

```

Pagination

Pagination, breaking up query results into smaller parts, can be done using `limit` , `orderBy` , and filtering parameters. The GraphQL Analytics API does not support cursors for pagination.

- `limit` (integer) defines how many records to return.
- `orderBy` (string) defines the sort order for the data.

Query pages without cursors

The following examples assume that `date` and `clientCountryName` relationships are unique.

Get the first n results of a query.

To limit results, add the `limit` parameter. For example, query the first two records.

```
firewallEventsAdaptive (limit: 2, orderBy: [datetime_ASC, clientCountryName_ASC]) {
  datetime
  clientCountryName
}
```

Specifying a sort order by date returns less specific results than specifying a sort order by both date and country.

The query response follows:

```
{
  "firewallEventsAdaptive" : [
    {
      "datetime": "2018-11-12T00:00:00Z",
      "clientCountryName": "UM"
    },
    {
      "datetime": "2018-11-12T00:00:00Z",
      "clientCountryName": "US"
    }
  ]
}
```

Query for the next page using filter

To get the next n results, specify a filter to exclude the last result from the previous query. Using the previous example, you can append the greater-than operator (`_gt`) to the `clientCountryName` field and the greater-or-equal operator to the `datetime` field. By making a specific order, you can get the most complete results.

```
firewallEventsAdaptive (limit: 2, orderBy: [datetime_ASC, clientCountryName_ASC], filter: {date_geq: "2018-11-12T00:00:00Z", clientCounterName_gt: "US"}) {
  date
  clientCountryName
}
```

The query response follows:

```
{
  "firewallEventsAdaptive" : [
    {
      "datetime": "2018-11-12T00:00:00Z",
      "clientCountryName": "UY"
    },
    {
      "datetime": "2018-11-12T00:00:00Z",
      "clientCountryName": "UZ"
    }
  ]
}
```

Query the previous page

To get the previous n results, reverse the filters and sort order.

```
firewallEventsAdaptive (limit: 2, orderBy: [datetime_DESC, clientCountryName_DESC, filter: {date_leq: "2018-11-12T00:00:00Z", clientCountryName_lt: "UY"}]) {
  datetime
  clientCountryName
}
```


The query response follows:

```
{
  "firewallEventsAdaptive" : [
    {
      "datetime": "2018-11-12T00:00:00Z",
      "clientCountryName": "US"
    },
    {
      "datetime": "2018-11-12T00:00:00Z",
      "clientCountryName": "UM"
    }
  ]
}
```

Filtering

Filters constrain queries to a particular account or set of zones (domains), requests by date, or a specific query agent. Without filters, performance might degrade, and results might include unimportant data.

Structure

The GraphQL filter is represented by the [GraphQL Input Object](#).

Filters can be used as an argument on the following resources:

- Zones (Domains)
- Tables (data sets)
- Accounts

Zone (domain) filter

The zone filter allows querying zone-related data by zone (domain) ID (`zoneTag`).

```
zones(filter: {zoneTag: "your domain (zone) ID"}) {
  ...
}
```

The zone filter must conform to the following grammar:

```
filter
{ zoneTag: t }
{ zoneTag_gt: t }
{ zoneTag_in: [t, ...] }
```

Compound filters (comma-separated, `AND`, `OR`) are not supported. Zones always sort alphanumerically.

Table (data set) filter

Table filters require that you query at least one node. The `AND` operator can be used to create and combine multi-node filters.

Accounts filter

Account level filtering is supported, and there is a required filter parameter. For example:

```
accounts(filter: {accountTag: $accountTag}) {
  ...
}
```

Operators

Operator support varies, depending on the node type and node name. The following operators are supported for all types:

Operator	Comparison
gt	greater than
lt	less than
geq	greater or equal to
leq	less or equal to
neq	not equal
in	in

Supported operators

Examples

General example:

```
{
  myQuery(
    filter: {
      clientCountry: "UK" # all objects having client country equal to "UK"
      datetime_gt: "2020-01-01 10:00:00" # all object having datetime greater than "2020-01-01 10:00:00"
    }
  )
}
```

Filter a specific node:

```
httpRequests1hGroups(filter: {datetime: "2020-01-01 10:00:00"}) {
  ...
}
```

Filter on multiple fields:

```
httpRequests1hGroups(filter: {datetime_gt: "2018-01-01 10:00:00", datetime_lt: "2018-01-01 11:00:00"}) {
  ...
}
```

Filter using `OR` operator:

```
httpRequests1hGroups(filter: {
  datetime: "2020-01-01 10:00:00",
  OR:[{clientCountryName: "US"}, {clientCountryName: "UK"}]) {
  ...
}
```

Querying Edge Functions metrics with GraphQL

This example uses the GraphQL Analytics API to query for Edge Functions Metrics over a specified time period. You can query up to one week of data for dates up to three months ago.

The following API call requests an Edge Functions script's metrics over a one day period, and outputs the requested fields. Adjust the `datetimeStart` , `datetimeEnd` , `accountTag` , and `scriptName` variables as needed.

The API call

```
PAYLOAD={ "query":
"query GetEdgeFunctionsAnalytics($accountTag: string, $datetimeStart: string, $datetimeEnd: string, $scriptName: string) {
  viewer {
    accounts(filter: {accountTag: $accountTag}) {
```

```
workersInvocationsAdaptive(limit: 100, filter: {
    scriptName: $scriptName,
    datetime_geq: $datetimeStart,
    datetime_leq: $datetimeEnd
}) {
    sum {
        subrequests
        requests
        errors
    }
    quantiles {
        cpuTimeP50
        cpuTimeP99
    }
    dimensions{
        datetimeHour
        scriptName
        status
    }
}
}
```

```
"variables": {
    "accountTag": "90f518ca7113dc0a91513972ba243ba5",
    "datetimeStart": "2020-05-04T00:00:00.000Z",
    "datetimeEnd": "2020-05-04T00:00:00.000Z",
    "scriptName": "edge-function-subrequest-test-client"
}
}'
```

```
curl \
-X POST \
-H "Content-Type: application/json" \
-H "Authorization: IBM IAM TOKEN" \
--data "$(echo $PAYLOAD)" \
https://api.cis.cloud.ibm.com/v1/<crn>/zones/<zoneid>/graphql
```

The results are returned in JSON (as requested), so piping the output to `jq` makes them easier to read. For example:

```
curl \
-X POST \
-H "Content-Type: application/json" \
-H "Authorization: IBM IAM TOKEN" \
--data "$(echo $PAYLOAD)" \
https://api.cis.cloud.ibm.com/v1/<crn>/zones/<zonedid>/graphql | jq .
{
  "data": {
    "viewer": {
      "accounts": [
        {
          "workersInvocationsAdaptive": [
            {
              "dimensions": {
                "datetime": "2020-05-04T18:10:35Z",
                "scriptName": "edge-function-subrequest-test-client",
                "status": "success"
              },
              "quantiles": {
                "cpuTimeP50": 206,
                "cpuTimeP99": 206
              },
              "sum": {
                "errors": 0,
                "requests": 1,
                "subrequests": 0
              }
            },
            {
              "dimensions": {
```

```

      "datetime": "2020-05-04T18:10:34Z",
      "scriptName": "edge-function-subrequest-test-client",
      "status": "success"
    },
    "quantiles": {
      "cpuTimeP50": 291,
      "cpuTimeP99": 291
    },
    "sum": {
      "errors": 0,
      "requests": 1,
      "subrequests": 0
    }
  },
  {
    "dimensions": {
      "datetime": "2020-05-04T18:10:49Z",
      "scriptName": "edge-function-subrequest-test-client",
      "status": "success"
    },
    "quantiles": {
      "cpuTimeP50": 212.5,
      "cpuTimeP99": 261.19
    },
    "sum": {
      "errors": 0,
      "requests": 4,
      "subrequests": 0
    }
  }
]
}
]
}
}
},
"errors": null
}

```

Querying firewall events with GraphQL

This example uses the GraphQL Analytics API to query for firewall events over a specified time period.

The following API call requests Firewall Events over a one-hour period, and outputs the requested fields. Be sure to replace `CIS_DOMAIN_ID` with your domain ID and API token, and adjust the `datetime_geg` and `datetime_leq` values to meet your needs.

The API call

```

PAYLOAD='{ "query":
"query ListFirewallEvents($zoneTag: string, $filter: FirewallEventsAdaptiveFilter_InputObject) {
  viewer {
    zones(filter: { zoneTag: $zoneTag }) {
      firewallEventsAdaptive(
        filter: $filter
        limit: 10
        orderBy: [datetime_DESC]
      ) {
        action
        clientAsn
        clientCountryName
        clientIP
        clientRequestPath
        clientRequestQuery
        datetime
        source
        userAgent
      }
    }
  }
}'

```

```

    }",
    "variables": {
      "zoneTag": "CIS_DOMAIN_ID",
      "filter": {
        "datetime_geq": "2020-04-24T11:00:00Z",
        "datetime_leq": "2020-04-24T12:00:00Z"
      }
    }
  }
}'

curl \
-X POST \
-H "Content-Type: application/json" \
-H "Authorization: IBM IAM TOKEN" \
--data "$(echo $PAYLOAD)" \
https://api.cis.cloud.ibm.com/v1/<crn>/zones/<zoneid>/graphql

```

The results are returned in JSON, so piping the output to jq makes them easier to read. For example:

```

curl \
-X POST \
-H "Content-Type: application/json" \
--data "$(echo $PAYLOAD)" \
https://api.cis.cloud.ibm.com/v1/<crn>/zones/<zoneid>/graphql | jq .
{
  "data": {
    "viewer": {
      "zones": [
        {
          "firewallEventsAdaptive": [
            {
              "action": "log",
              "clientAsn": "5089",
              "clientCountryName": "GB",
              "clientIP": "203.0.113.69",
              "clientRequestPath": "/%3Cscript%3Ealert()%3C/script%3E",
              "clientRequestQuery": "",
              "datetime": "2020-04-24T10:11:24Z",
              "source": "waf",
              "userAgent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.163 Safari/537.36"
            },
            {
              "action": "log",
              "clientAsn": "5089",
              "clientCountryName": "GB",
              "clientIP": "203.0.113.69",
              "clientRequestPath": "/%3Cscript%3Ealert()%3C/script%3E",
              "clientRequestQuery": "",
              "datetime": "2020-04-24T10:11:24Z",
              "source": "waf",
              "userAgent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.163 Safari/537.36"
            },
            {
              "action": "log",
              "clientAsn": "5089",
              "clientCountryName": "GB",
              "clientIP": "203.0.113.69",
              "clientRequestPath": "/%3Cscript%3Ealert()%3C/script%3E",
              "clientRequestQuery": "",
              "datetime": "2020-04-24T10:11:24Z",
              "source": "waf",
              "userAgent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.163 Safari/537.36"
            },
            {
              "action": "log",
              "clientAsn": "5089",
              "clientCountryName": "GB",

```



```
    "clientIP": "203.0.113.69",
    "clientRequestPath": "/%3Cscript%3Ealert()%3C/script%3E",
    "clientRequestQuery": "",
    "datetime": "2020-04-24T10:11:24Z",
    "source": "waf",
    "userAgent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.163
Safari/537.36"
  },
  {
    "action": "log",
    "clientAsn": "5089",
    "clientCountryName": "GB",
    "clientIP": "203.0.113.69",
    "clientRequestPath": "/%3Cscript%3Ealert()%3C/script%3E",
    "clientRequestQuery": "",
    "datetime": "2020-04-24T10:11:24Z",
    "source": "waf",
    "userAgent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.163
Safari/537.36"
  },
  {
    "action": "log",
    "clientAsn": "5089",
    "clientCountryName": "GB",
    "clientIP": "203.0.113.69",
    "clientRequestPath": "/%3Cscript%3Ealert()%3C/script%3E",
    "clientRequestQuery": "",
    "datetime": "2020-04-24T10:11:24Z",
    "source": "waf",
    "userAgent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.163
Safari/537.36"
  },
  {
    "action": "log",
    "clientAsn": "5089",
    "clientCountryName": "GB",
    "clientIP": "203.0.113.69",
    "clientRequestPath": "/%3Cscript%3Ealert()%3C/script%3E",
    "clientRequestQuery": "",
    "datetime": "2020-04-24T10:11:24Z",
    "source": "waf",
    "userAgent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.163
Safari/537.36"
  },
  {
    "action": "drop",
    "clientAsn": "5089",
    "clientCountryName": "GB",
    "clientIP": "203.0.113.69",
    "clientRequestPath": "/%3Cscript%3Ealert()%3C/script%3E",
    "clientRequestQuery": "",
    "datetime": "2020-04-24T10:11:24Z",
    "source": "waf",
    "userAgent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.163
Safari/537.36"
  },
  {
    "action": "log",
    "clientAsn": "58224",
    "clientCountryName": "IR",
    "clientIP": "2.183.175.37",
    "clientRequestPath": "/api/v2",
    "clientRequestQuery": "",
    "datetime": "2020-04-24T10:00:54Z",
    "source": "waf",
    "userAgent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/74.0.3729.169
Safari/537.36"
  },
  {
    "action": "log",
    "clientAsn": "58224",
```

```

      "clientCountryName": "IR",
      "clientIP": "2.183.175.37",
      "clientRequestPath": "/api/v2",
      "clientRequestQuery": "",
      "datetime": "2020-04-24T10:00:54Z",
      "source": "waf",
      "userAgent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/74.0.3729.169
Safari/537.36"
    }
  ]
}
}
},
"errors": null
}

```

Querying basics

There are several basic aspects of querying with GraphQL for metrics collection.

Structure of a GraphQL query

GraphQL structures data as a graph. GraphQL uses a schema to define the objects and their hierarchy in your data graph. You can explore the edges of the graph by using queries to get the needed data. These queries must respect the structure of the schema.

A **node**, followed by its **fields**, is at the core of a GraphQL query. A node is an object of a specific **type**; the type specifies the fields that make up the object.

A field can be another node where the appropriate query would contain nested elements. Some nodes look like functions that can take on arguments to limit the scope of what they can act on. You can apply filters at each node.

Cloudflare GraphQL schema

A typical query against the Cloudflare GraphQL schema is made up of four main components:

- **viewer** - The root node.
- **zones** or **accounts** - Indicate the scope of the query (the domain area or account you want to query). The **viewer** can access one **zones** or **accounts**, or both.
- **data node** or **dataset** - Represent the data you want to query. **zones** or **accounts** might contain one or more datasets.
- **fieldset** - A set of fields or nested fields of the **dataset**.

The query to the Cloudflare GraphQL API must be sent by an HTTP POST request with the payload in JSON format that consists of these fields:

```

$ {
  "query": "",
  "variables": {}
}

```

In the preceding structure, the **query** field must contain a GraphQL query formatted as a single line string (which means all newline symbols should be stripped or escaped), when **variables** is an object that contains all values of used placeholders in the query itself.

A single dataset example

In the following example, the GraphQL query fetches a **datetime**, **action**, and client request HTTP host as the **host** field of 2 WAF events from zone-scoped the **firewallEventsAdaptive** dataset.

```

$ {
  viewer {
    zones(filter: { zoneTag: $tag }) {
      firewallEventsAdaptive(
        filter: {

```



```

      datetime_gt: $start
      datetime_lt: $end
    }
    limit: 2
    orderBy: [
      datetime_DESC
    ]
  ){
    action
    datetime
    host: clientRequestHTTPHost
  }
}
}
}

```

In this query, there are variable placeholders: `$tag`, `$start`, and `$end`. It provides values for those placeholders alongside the query by placing them into the `variables` field of the payload.

Define your variables:

```

${
  "tag": "<zone-tag>",
  "start": "2020-08-03T02:07:05Z",
  "end": "2020-08-03T17:07:05Z"
}

```



Tip: There are multiple ways to send your query to Cloudflare GraphQL API. You can use you favourite GraphQL client or CLI to send a request using cURL.

A sample of a response for this query:

```

{
  "data": {
    "viewer": {
      "zones": [
        {
          "firewallEventsAdaptive": [
            {
              "action": "log",
              "host": "cloudflare.guru",
              "datetime": "2020-08-03T17:07:03Z"
            },
            {
              "action": "log",
              "host": "cloudflare.guru",
              "datetime": "2020-08-03T17:07:01Z"
            }
          ]
        }
      ]
    }
  },
  "errors": null
}

```

Query multiple datasets in a single GraphQL API request

As previously mentioned, a query might contain one or multiple nodes (datasets). At the API level, the data extraction is done simultaneously, but the response is delayed until all dataset queries receive their results. If any fails during the execution, the entire query is terminated, and the error is returned.

A sample query for two datasets in one attempt:

```

${
  viewer {
    zones(filter: { zoneTag: $tag }) {

```

```
last10Events: firewallEventsAdaptive(  
  filter: {  
    datetime_gt: $start  
    datetime_lt: $end  
  }  
  limit: 10  
  orderBy: [  
    datetime_DESC  
  ]  
) {  
  action  
  datetime  
  host: clientRequestHTTPHost  
}  
top3DeviceTypes: httpRequestsAdaptiveGroups(  
  filter: {  
    date: $ts  
  }  
  limit: 10  
  orderBy: [  
    count_DESC  
  ]  
) {  
  count  
  dimensions {  
    device: clientDeviceType  
  }  
}  
}  
}
```

A set of variables for the previous query:

```
{  
  "tag": "<zone-tag>",  
  "start": "2022-10-02T00:26:49Z",  
  "end": "2022-10-04T14:26:49Z",  
  "ts": "2022-10-04"  
}
```

A sample response for the query using the previous variables:

```
{  
  "data": {  
    "viewer": {  
      "zones": [  
        {  
          "last10Events": [  
            {  
              "action": "block",  
              "country": "TR",  
              "datetime": "2022-10-04T08:41:09Z"  
            },  
            {  
              "action": "block",  
              "country": "TR",  
              "datetime": "2022-10-04T08:41:09Z"  
            },  
            {  
              "action": "block",  
              "country": "RU",  
              "datetime": "2022-10-04T01:09:36Z"  
            },  
            {  
              "action": "block",  
              "country": "US",  
              "datetime": "2022-10-03T14:26:49Z"  
            },  
            {  

```

```
      "action": "block",
      "country": "US",
      "datetime": "2022-10-03T14:26:46Z"
    },
    {
      "action": "block",
      "country": "CN",
      "datetime": "2022-10-02T23:51:26Z"
    },
    {
      "action": "block",
      "country": "TR",
      "datetime": "2022-10-02T23:39:41Z"
    },
    {
      "action": "block",
      "country": "TR",
      "datetime": "2022-10-02T23:39:41Z"
    }
  ],
  "top3DeviceTypes": [
    {
      "count": 4580,
      "dimensions": {
        "device": "desktop"
      }
    }
  ]
}
"errors": null
}
```

Composing a query in GraphiQL

You can use a GraphiQL client to compose and execute a GraphQL query.

Setting up a query and choosing a dataset

Click the editing pane of GraphiQL and add the following depicted base query, replacing `zone-id` with your Cloudflare zone ID:

`[Query]`

Adding a base query in the GraphiQL pane

Tip: To find the zone's tag, open your CIS instance in the IBM Cloud console and select the domain for which you want to obtain the tag. In the **Overview** page, scroll to the **API** section in the sidebar. This displays your zone and account tags.

To assist query building, the GraphiQL client has word completion. Insert your cursor in the query, in this case on the line after `zones`, and start entering a value to engage the feature. For example, when you type `firewall`, a popup menu displays the datasets that return firewall information:

GraphiQL word completion assistant to query building

`[firewallEventsAdaptive]`

The text at the end of the list displays a short description of the data that the node returns.

Select the dataset you want to query and insert it. Either select the item in the list, or scroll using arrow keys and press **Return**.

Supply required parameters

Hover your mouse over a field to display a tooltip that describes the dataset. In this example, hovering over the `firewallEventsAdaptive` node displays the description depicted here:

Hovering the mouse over a field to display its description

To display information about the dataset, including required parameters, select the dataset name (blue text). The **Documentation Explorer** opens and displays details about the dataset:

Documentation Explorer window displaying dataset details

Tip: The `filter` and `limit` arguments are required, as indicated by the exclamation mark () after their type definitions (gold text). In this example, the `orderBy` argument is not required, though when used it requires a value of type `ZoneFirewallEventsAdaptiveOrderBy` .

To browse a list of supported filter fields, select the filter type definition (gold text) in the Documentation Explorer. In this example, the type is `ZoneFirewallEventsAdaptiveFilter_InputObject` :

Browsing GraphQL filter fields

The following example query shows the required `filter` and `limit` arguments for `firewallEventsAdaptive` (as well as for the rest of GraphQL nodes):

Example of GraphQL query arguments

Define the fields used by your query

To browse the fields you can use with your query, hover your cursor over the dataset name and in the tooltip that displays. Then select the data type definition (gold text):

Hovering the mouse over a dataset to display available fields

The Documentation Explorer opens and displays a list of fields:

Documentation Explorer window displaying list of fields

To add the data fields that you want to read, type an opening brace () after the closing parenthesis for the parameters, then start typing the name of a field that you want to fetch. Use word completion to choose a field.

The following example query returns the `action` , `datetime` , `clientRequestHTTPHost` , and `userAgent` fields:

Example query with return fields

After you enter all the fields you want to query, select the **Play** button to submit the query. The response pane contains the data fetched from the configured GraphQL API endpoint:

GraphQL response pane

Variable substitution

The GraphQL client allows you to use placeholders for value and supply them through the `variables` part of the payload.

Placeholder names should start with the `$` character, and you do not need to wrap placeholders in quotes when you use them in the query.

Values for placeholders should be provided in JSON format, in which placeholders are addressed without the `$` character. As an example, for a placeholder `$zoneTag` , the GraphQL API will read a value from the `zoneTag` field of the supplied variables object.

To supply a value for a placeholder, select the **Query Variables** pane and edit a JSON object that defines your variables.

This example query uses the `zoneTag` query variable to represent the zone ID:

Example of GraphQL query variables

Using sampling to optimize metrics

Use sampling to analyze a subset of data rather than processing every individual data point. It helps maintain performance and scalability while delivering accurate insights. Given the volume of data CIS handles—over 700 million events per second—sampling is essential for delivering fast, cost-effective metrics across large datasets.

In a small number of cases, metrics that are provided in the CIS dashboard and GraphQL API are based on a *sample*—a subset of the dataset. In these cases, CIS metrics return an estimate derived from the sampled value. For example, if during an attack the sampling rate is 10% and 5,000 events are sampled, CIS estimates the total number of events as 50,000 ($5,000 \times 10$) and reports this value.

CIS primarily uses [adaptive sampling](#), including a method called Adaptive Bit Rate (ABR), which adjusts the level of detail in the data returned based on query complexity and volume. When the number of records is small or the query is simple, full-resolution data (100%) is used. As the dataset grows, or the query becomes more complex, progressively lower sample rates (such as 10% or 1%), are applied to ensure efficient query completion.

This approach prevents large queries from consuming excessive computing resources, ensuring fair distribution and consistent performance for all users. Data is stored at multiple resolutions (100%, 10%, and 1%), allowing the system to select the appropriate resolution based on the query’s complexity and size, helping ABR deliver fast, accurate results.



Note: CIS GraphQL API exposes datasets that are powered by adaptive sampling. These nodes have **Adaptive** in the name and can be discovered through [introspection](#).

Why sampling is applied

CIS metrics are designed to deliver data at the appropriate level of detail as quickly as possible. Sampling helps achieve this by reducing the amount of data that is processed, allowing CIS to return metrics within seconds—even during spikes in volume, such as a burst of firewall events during an attack. Without sampling, queries can take minutes or more to complete, which is too long when validating mitigation efforts or troubleshooting issues.

CIS processes over 700 million events per second across its global network. Storing and processing all of this data in real time would take too much time and computing power to be practical. Sampling balances accuracy with performance, making metrics faster, more scalable, and more efficient. Because the datasets are so large, sampled values remain statistically meaningful and provide reliable insights.

This approach is similar to other domains:

- Google Maps: Lower-resolution imagery when zoomed out mirrors how CIS adjusts sampling rates to deliver fast, relevant insights based on query size.
- Opinion Polls: A small, representative sample can reflect system-wide trends.
- Movie Frames: Viewing at 30 frames per second (fps) instead of 60 fps still tells the full story. Similarly, sampling maintains the key patterns in your data.

While ABR sampling resolution isn’t always visible, the number of rows read is a good indicator: the more rows read, the higher the resolution and reliability of the results.

Types of sampling

CIS metrics use two primary types of sampling: adaptive sampling and fixed sampling. The method applied depends on the dataset and how the data is queried.

Adaptive sampling

CIS metrics primarily rely on adaptive sampling, which means the sample rate fluctuates depending on the volume of data that is ingested or queried. If the number of records is relatively small, sampling is typically not used, allowing full data to be returned. However, as the volume of records increases, progressively lower sample rates are applied to maintain performance and responsiveness.

This model is used in several data sources, including Security Events (also known as Firewall Events) and the Security Event Log. Data nodes that use adaptive sampling are easy to identify by the **Adaptive** suffix in the node name, as in `firewallEventsAdaptive`.

Fixed sampling

The following data nodes are based on fixed sampling, where the sample rate does not vary:

Dataset	Rate	Notes
Firewall Rules Preview	1%	Use with caution. A 1% sample rate does not provide accurate estimates for datasets smaller than a certain threshold, a scenario the CIS dashboard calls out explicitly but the API does not.
Nodes: <code>firewallRulePreviewGroups</code>		
Network metrics	0.012%	Sampling rate is in terms of packet count (1 of every 8,192 packets).
Nodes: <code>ipFlows1mGroups</code> <code>ipFlows1hGroups</code> <code>ipFlows1dGroups</code> <code>ipFlows1mAttacksGroups</code>		

Fixed sampling

Other considerations

Considerations to keep in mind:

Access to raw data

Because sampling is primarily adaptive and automatically adjusts to provide an accurate estimate, the sampling rate cannot be directly controlled. Enterprise customers have access to raw data through [CIS logs](#).

When sampling occurs

Sampling is typically applied to high-traffic datasets where full data metrics are impractical. For smaller datasets, full data analysis is often performed without sampling.

Sampling rates

Sampling rates vary depending on the dataset and product. CIS helps ensure that sampling rates are consistent within a single dataset to maintain accuracy across queries.

Impact on metrics

While sampling reduces the volume of processed data, aggregated metrics like totals, averages, and percentiles are extrapolated based on the sample size. This ensures that the reported metrics represent the entire dataset accurately.

Limitations

Sampling might not capture extremely rare events with very low occurrence rates.

Querying HTTP events by hostname with GraphQL

There are two useful ways you can query HTTP events by hostname with GraphQL:

- Aggregated HTTP metrics by hostname over time
- Top 10 consuming URLs in a zone

Aggregated HTTP metrics by hostname over time

This example shows how to use the GraphQL Analytics API to query aggregated metrics about HTTP events by hostname over a specific period of time.

The following API call requests the number of visits and edge response bytes for the custom hostname `hostname.example.com` over a four day period. Be sure to replace `CLOUDFLARE_ZONE_ID` and `API_TOKEN` with your zone ID and API credentials, and adjust the `datetime_geq` and `datetime_leq` values as needed.

API Call

```
$ echo '{ "query":
"query RequestsAndDataTransferByHostname($zoneTag: string, $filter:filter) {
  viewer {
    zones(filter: {zoneTag: $zoneTag}) {
      httpRequestsAdaptiveGroups(limit: 10, filter: $filter) {
        sum {
          visits
          edgeResponseBytes
        }
        dimensions {
          datetimeHour
        }
      }
    }
  }
}',
"variables": {
  "zoneTag": "<CLOUDFLARE_ZONE_ID>",
  "filter": {
    "datetime_geq": "2022-07-20T11:00:00Z",
    "datetime_lt": "2022-07-24T12:00:00Z",
    "clientRequestHTTPHost": "hostname.example.com",
    "requestSource": "eyeball"
  }
}
}' | tr -d '\n' | curl --silent \
https://api.cloudflare.com/client/v4/graphql \
--header "Authorization: Bearer <API_TOKEN>" \
--header "Accept: application/json" \
--header "Content-Type: application/json" \
--data @- | jq .
```

The returned results will be in JSON format (as requested), so piping the output to `jq` will make them easier to read, like in the following example:

```
{
  "data": {
    "viewer": {
      "zones": [
        {
          "httpRequestsAdaptiveGroups": [
            {
              "dimensions": {
                "datetimeHour": "2022-07-21T10:00:00Z"
              },
              "sum": {
                "edgeResponseBytes": 19849385,
                "visits": 4383
              }
            },
            {
              "dimensions": {
                "datetimeHour": "2022-07-21T06:00:00Z"
              },
              "sum": {
                "edgeResponseBytes": 20607204,
                "visits": 4375
              }
            },
            {
              "dimensions": {
                "datetimeHour": "2022-07-26T05:00:00Z"
              },
              "sum": {
                "edgeResponseBytes": 20170839,
                "visits": 4519
              }
            },
            {
              "dimensions": {
```



```
      "datetimeHour": "2022-07-23T08:00:00Z"
    },
    "sum": {
      "edgeResponseBytes": 20141860,
      "visits": 4448
    }
  },
  {
    "dimensions": {
      "datetimeHour": "2022-07-25T15:00:00Z"
    },
    "sum": {
      "edgeResponseBytes": 21070367,
      "visits": 4469
    }
  },
  {
    "dimensions": {
      "datetimeHour": "2022-07-28T08:00:00Z"
    },
    "sum": {
      "edgeResponseBytes": 19200774,
      "visits": 4345
    }
  },
  {
    "dimensions": {
      "datetimeHour": "2022-07-26T02:00:00Z"
    },
    "sum": {
      "edgeResponseBytes": 20758067,
      "visits": 4502
    }
  },
  {
    "dimensions": {
      "datetimeHour": "2022-07-20T19:00:00Z"
    },
    "sum": {
      "edgeResponseBytes": 22127811,
      "visits": 4443
    }
  },
  {
    "dimensions": {
      "datetimeHour": "2022-07-27T15:00:00Z"
    },
    "sum": {
      "edgeResponseBytes": 20480644,
      "visits": 4268
    }
  },
  {
    "dimensions": {
      "datetimeHour": "2022-07-27T17:00:00Z"
    },
    "sum": {
      "edgeResponseBytes": 19885704,
      "visits": 4287
    }
  }
]
}
]
}
}
},
"errors": null
}
```

Top 10 consuming URLs in a zone

You can use the GraphQL Analytics API to query the top 10 consuming URLs from a zone, helping you identify the URLs with the highest resource usage. Here are some configuration instruction for the following API call:

- To filter on a specific hostname, add the line `"clientRequestHTTPHost": "$2"` below `"requestSource"`.
- Replace `API_TOKEN` with your generated API token using the `Read all resources` permissions. The script will only access zones available to the token's creator.
- Pass the zone ID (`zoneTag`) as a parameter `ARG=$1`.
- To calculate the current date and the date from 30 days ago, use `gdate` on Mac:
 - `CURRENTDATE=$(gdate -u +%FT%TZ)`
 - `OLDDATE=$(gdate -d '-30 days' -u +%FT%TZ)`.
- For specific dates within the last 30 days, set `CURRENTDATE` and `OLDDATE` variables in the format `"YYYY-MM-DDTHH:MM:SSZ"`.

API call

```
$ curl --silent \
https://api.cloudflare.com/client/v4/graphql \
--header "Authorization: Bearer <API_TOKEN>" \
--header "Content-Type: application/json" \
--data '{
  "query": "{viewer {zones(filter: {zoneTag: $zoneTag}) {topPaths: httpRequestsAdaptiveGroups(filter: $filter, limit: 10, orderBy: [sum_edgeResponseBytes_DESC]) {count sum {edgeResponseBytes} dimensions {metric: clientRequestPath}}}}",
  "variables": {
    "zoneTag": "$ARG",
    "filter": {
      "AND": [
        {
          "datetime_geq": "$OLDDATE",
          "datetime_leq": "$CURRENTDATE"
        },
        {
          "requestSource": "eyeball"
        }
      ]
    }
  }
}' | jq -r 'try .data.viewer.zones[].topPaths[] | "\(.dimensions.metric)": \(.sum.edgeResponseBytes)" | sort
```

Viewing firewall event groups

If you have a large amount of firewall events, you might want to view the overall count of events based on different groupings. This example shows looking at firewall events per country in 15 minute groupings.

The API call looks like:

```
$ PAYLOAD='{ "query":
"query ListFirewallEvents($zoneTag: string, $filter: FirewallEventsAdaptiveFilter_InputObject) {
  viewer {
    zones(filter: { zoneTag: $zoneTag }) {
      firewallEventsAdaptiveGroups(
        filter: $filter
        limit: 100
        orderBy: [datetimeFifteenMinutes_DESC]
      ){
        dimensions {
          clientCountryName
          datetimeFifteenMinutes
        }
        count
      }
    }
  }
}',
"variables": {
```

```
"zoneTag": "$CIS_DOMAIN_ID",
"filter": {
  "datetime_geq": "2020-04-24T11:00:00Z",
  "datetime_leq": "2020-04-24T12:00:00Z"
}
}
}'

curl -X POST -H "Content-Type: application/json" -H "Authorization: IBM IAM TOKEN" --data "$(echo $PAYLOAD)"
https://api.cis.cloud.ibm.com/v1/$CRN/zones/$ZONE_ID/graphql
```

An example response:

```
{
  "data": {
    "viewer": {
      "zones": [
        {
          "firewallEventsAdaptiveGroups": [
            {
              "count": 3,
              "dimensions": {
                "clientCountryName": "NL",
                "datetimeFifteenMinutes": "2024-10-28T00:15:00Z"
              }
            },
            {
              "count": 86,
              "dimensions": {
                "clientCountryName": "US",
                "datetimeFifteenMinutes": "2024-10-28T00:15:00Z"
              }
            },
            {
              "count": 3,
              "dimensions": {
                "clientCountryName": "DE",
                "datetimeFifteenMinutes": "2024-10-28T00:15:00Z"
              }
            },
            {
              "count": 15,
              "dimensions": {
                "clientCountryName": "CA",
                "datetimeFifteenMinutes": "2024-10-28T00:15:00Z"
              }
            },
            {
              "count": 2,
              "dimensions": {
                "clientCountryName": "IN",
                "datetimeFifteenMinutes": "2024-10-28T00:15:00Z"
              }
            },
            {
              "count": 366,
              "dimensions": {
                "clientCountryName": "PH",
                "datetimeFifteenMinutes": "2024-10-28T00:00:00Z"
              }
            },
            {
              "count": 83357,
              "dimensions": {
                "clientCountryName": "US",
                "datetimeFifteenMinutes": "2024-10-28T00:00:00Z"
              }
            },
            {
              "count": 146,
```

```
"dimensions":{
  "clientCountryName":"BE",
  "datetimeFifteenMinutes":"2024-10-28T00:00:00Z"
},
{
  "count":477,
  "dimensions":{
    "clientCountryName":"NL",
    "datetimeFifteenMinutes":"2024-10-28T00:00:00Z"
  }
},
{
  "count":58,
  "dimensions":{
    "clientCountryName":"KR",
    "datetimeFifteenMinutes":"2024-10-28T00:00:00Z"
  }
},
{
  "count":10,
  "dimensions":{
    "clientCountryName":"NZ",
    "datetimeFifteenMinutes":"2024-10-28T00:00:00Z"
  }
},
{
  "count":15202,
  "dimensions":{
    "clientCountryName":"CA",
    "datetimeFifteenMinutes":"2024-10-28T00:00:00Z"
  }
},
{
  "count":665,
  "dimensions":{
    "clientCountryName":"GB",
    "datetimeFifteenMinutes":"2024-10-28T00:00:00Z"
  }
},
{
  "count":1048,
  "dimensions":{
    "clientCountryName":"DE",
    "datetimeFifteenMinutes":"2024-10-28T00:00:00Z"
  }
},
{
  "count":3509,
  "dimensions":{
    "clientCountryName":"AU",
    "datetimeFifteenMinutes":"2024-10-28T00:00:00Z"
  }
},
{
  "count":114,
  "dimensions":{
    "clientCountryName":"ID",
    "datetimeFifteenMinutes":"2024-10-28T00:00:00Z"
  }
},
{
  "count":806,
  "dimensions":{
    "clientCountryName":"HK",
    "datetimeFifteenMinutes":"2024-10-28T00:00:00Z"
  }
},
{
  "count":66,
  "dimensions":{
```

```

        "clientCountryName": "PT",
        "datetimeFifteenMinutes": "2024-10-28T00:00:00Z"
    },
    {
        "count": 523,
        "dimensions": {
            "clientCountryName": "IN",
            "datetimeFifteenMinutes": "2024-10-28T00:00:00Z"
        }
    },
    {
        "count": 43,
        "dimensions": {
            "clientCountryName": "FR",
            "datetimeFifteenMinutes": "2024-10-28T00:00:00Z"
        }
    },
    {
        "count": 263,
        "dimensions": {
            "clientCountryName": "CN",
            "datetimeFifteenMinutes": "2024-10-28T00:00:00Z"
        }
    },
    {
        "count": 1,
        "dimensions": {
            "clientCountryName": "AR",
            "datetimeFifteenMinutes": "2024-10-28T00:00:00Z"
        }
    },
    {
        "count": 27,
        "dimensions": {
            "clientCountryName": "PE",
            "datetimeFifteenMinutes": "2024-10-28T00:00:00Z"
        }
    },
    {
        "count": 19,
        "dimensions": {
            "clientCountryName": "XX",
            "datetimeFifteenMinutes": "2024-10-28T00:00:00Z"
        }
    },
    {
        "count": 327,
        "dimensions": {
            "clientCountryName": "JP",
            "datetimeFifteenMinutes": "2024-10-28T00:00:00Z"
        }
    },
    {
        "count": 1018,
        "dimensions": {
            "clientCountryName": "MX",
            "datetimeFifteenMinutes": "2024-10-28T00:00:00Z"
        }
    }
]
}
"errors": null
}

```


Working with alerts

Alert types

CIS offers the following alert types:

- DDoS attack alerts include the following two types:
 - Layer 7 alerts notify WAF and CDN customers when an application-layer (L7) attack is detected and mitigated.
 - Layer 3/4 alerts are intended for customers using Range applications who want to receive a notification when an attack is mitigated.

No action is necessary if you receive a DDoS layer 7 alert. Each alert includes a short description, the time the attack was detected and mitigated, the attack type, its maximum rate of attack, and the target.

- Certificate alerts pertain to TLS certificates. The following types relate to certificate validation, issuance, renewal, and expiration:
 - Dedicated/Advanced certificates receive notifications for validation, issuance renewal, and expiration.
 - Universal SSL certificates are automatically refreshed and no user action is required.
 - Mutual TLS alerts notify you when an mTLS certificate is expiring.
 - Logpush job alerts are sent when the job is disabled due to consistent failures, not when a failure occurs for the first time. Any impacted job destinations are included in the alert.
 - The Web Metrics report is a weekly summary of your account's web metrics.
 - Cloudflare reports include the following alert types:
 - Maintenance alerts show scheduled, changed, or canceled Cloudflare maintenance windows.
 - Incident alerts show ongoing Cloudflare incidents.
 - Security Events alerts include WAF alerts and Advanced WAF alerts.
 - WAF alerts look for traffic spikes across all services that generate firewall log entries. The mean time to detection is 2 hours.
 - Advanced WAF alerts allow you to select the services to monitor, and each selected service is monitored separately. The mean time to detection is 5 minutes.
 - Bot detection alerts notify you when traffic patterns indicate a potential bot attack.
 - Passive Origin Monitoring alerts notify you when CIS cannot reach the application's origin server.
 - Origin error rate alerts notify you when high levels of 5XX HTTP errors are received from the origin server.
 - Load balancing health check alerts are sent when the status of a load balancing health check changes.
 - Pool toggle alerts notify you when the pool is manually enabled or disabled.
- No action is necessary if you receive a pool toggle alert. Each alert includes the state that the pool was toggled to, the time it occurred, and which user made the change.

Limitation for security alerts

The following limitations apply to security alerts, including Security Events alerts and Passive Origin Monitoring alerts.

- Security Events (WAF and Advanced WAF) alerts are not triggered for individual events. Instead, they are sent only when traffic spikes exceed the configured alert threshold. These thresholds cannot be configured. A [Z-score](#) is used to determine the threshold.
- Passive Origin Monitoring alerts are not triggered for individual events. Instead, they are sent only when traffic spikes exceed the configured alert threshold. Instances configured with this alert do not receive duplicate notifications within the same four-hour window. Each alert includes the most recent list of origins returning [521 errors](#).

Configuring alert policies

IBM Cloud® Internet Services provides customizable alert policies that notify you when specific events or conditions occur

within your account. You can set up these alerts using the IBM Cloud console, CLI, or API, enabling proactive monitoring and faster response times.

To send alerts to external systems, see [Configuring webhooks](#).



Note: Alerts are available only on Enterprise plans.

Configuring alert policies in the console

Use the IBM Cloud console to create, update, and delete alert policies.

Creating a security alert policy in the console

To create a security alert policy in the console, follow these steps:

- a. In the CIS console, navigate to the **Account** page, then select the **Alerts** tab.
- b. Select the **Alerting policies**, then click **Create**.
- c. Select the type of alert policy that you want to create. For more information, see [Alert types](#) to select alert policies.
- d. Enter a name for your alerting policy, and optionally enter a description.
- e. Choose an alerting method. You can select a webhook, enter an email address to send alerts to, or both. Only one notification is required to complete configuration.
 - Enter an email address to which Cloud Internet Services (CIS) sends alerts. Click the to add the address to the alert. Repeat for all email addresses.
 - Click **Add webhook** and select an available webhook. If no webhooks are present, this option is unavailable.
- f. Click **Create**.



Tip: By default, alert policies are enabled when created. You can disable alerts that you created by switching the toggle in the **Enabled** column.

Creating an advanced security alert policy in the console

To create an advanced security alert policy in the console, follow these steps:

- a. In the CIS console, navigate to the **Account** page, then select the **Alerts** tab.
- b. Select the **Alerting policies**, then click **Create**
- c. Select **Advanced security events alert**, then click **Next**.
- d. Enter a name for your alerting policy, and optionally enter a description.
- e. Choose an alerting method. You can either select a webhook, or enter an email address to send alerts to, or both. Only one notification is required to complete configuration.
 - Enter an email address to which Cloud Internet Services (CIS) sends alerts. Click the to add the address to the alert. Repeat for all email addresses.
 - Click **Add webhook** and select an available webhook. If no webhooks are present, this option is unavailable.
- f. Click **Next**.
- g. Select which domains the alert policy monitors.
- h. Select which services the alert policy monitors.
- i. Click **Create**.

Updating an alert policy in the console

To update an alert policy in the console, follow these steps:

- a. In the CIS console, navigate to the **Account** page, then select the **Alerts** tab.

- b. In the **Alerting policies**, locate the alert policy that you want to update, then click the Actions menu for that row.
- c. Click **Edit**.
- d. Make necessary changes. You can change the name and description, add or delete email addresses, and add, remove, or update webhooks.
- e. Click **Save**.

Deleting an alert policy in the console

To delete an alert policy, follow these steps:

- a. In the CIS console, navigate to the **Account** page, then select the **Alerts** tab.
- b. In the **Alerting policies** section, locate the alert policy that you want to delete, then click the Actions menu for that row.
- c. Click **Delete**, then confirm deletion.



Note: You can delete only alerts that you created.

Configuring alert policies from the CLI

Use the CLI to list, create, update, and delete alerting policies.

Listing all alert policies from the CLI

To list all alert policies from the CLI, run the following command:

```
$ ibmcloud cis alert-policy list [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`-i, --instance`

The instance name or ID.

`--output`

Specifies the output format; only JSON is supported.

Getting the details of an alert policy from the CLI

To get the details of an alert policy from the CLI, run the following command:

```
$ ibmcloud cis alert-policy get POLICY_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`POLICY_ID`

The ID of alert policy.

`-i, --instance`

The instance name or ID.

`--output`

Specifies the output format; only JSON is supported.

Creating a DDoS Attack alert policy from the CLI

To create a DDoS attack alert policy from the CLI, run the following command:

```
$ ibmcloud cis alert-policy ddos-attack-l7-alert-create --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

--name

The name of the alert policy.

--description

The description for the alert policy.

--emails

The email addresses for dispatching an alert notification (for example, **--emails test1@cn.ibm.com,test2@cn.ibm.com**).

--webhooks

The webhook IDs for dispatching an alert notification (for example, **--webhooks webhookID1,webhookID2**).

--enabled

Determines whether the alert policy is enabled.

-i, --instance

The instance name or ID.

--output

Specifies the output format; only JSON is supported.

Creating a pool toggle alert policy from the CLI

To create a pool toggle alert policy from the CLI, run the following command:

```
$ ibmcloud cis alert-policy pool-toggle-alert-create --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) --pools POOLS --trigger-condition (enabled | disabled | either) [--include-future-pools (true | false)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

--name

The name of the alert policy.

--description

The description for the alert policy.

--emails

The email addresses for dispatching an alert notification. For example, **--emails test1@cn.ibm.com,test2@cn.ibm.com** .

--webhooks value

The webhook IDs for dispatching an alert notification. For example, **--webhooks webhookID1,webhookID2** .

--enabled

Determines whether the alert policy is enabled.

--pools

The IDs of origin pool. If set to all, all pool IDs are used.

--trigger-condition

The condition of pool toggle status.

--include-future-pools

Determines whether to include the future pools.

-i, --instance

The instance name or ID.

`--output` Specifies the output format; only JSON is supported.

Creating a security alert policy from the CLI

Create an alert policy about spikes in firewall events. Firewall events alerts use a z-score calculation over the last six hours and five-minute buckets of events. An alert is triggered whenever the z-score exceeds 3.5 (the threshold). You don't receive duplicate alerts within the same two-hour timeframe.

To create a security alert from the CLI, run the following command:

```
$ ibmcloud cis alert-policy firewall-events-alert-create --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) --domains DOMAINS [--services SERVICES] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com` .

`--webhooks`

The webhook IDs for dispatching an alert notification. For example, `--webhooks webhookID1,webhookID2` .

`--domains`

The domain IDs for the alert policy. For example, `--domains domainID1,domainID2` .

`--services`

Specifies which services the alert monitors (for advanced security alerts). Valid services: `country-access-rules` , `waf` , `firewall-rules` , `ratelimit` , `securitylevel` , `ip-access-rules` , `browser-integrity-check` , `ua-rules` , `lockdowns` , `iprange-access-rules` , `asn-access-rules` , `managed-firewall` , `hotlink` , `ssl-validation` . (Enterprise plan only.)

`--enabled`

Determines whether the alert policy is enabled.

`-i, --instance`

The instance name or ID.

`--output`

Specifies the output format; only JSON is supported.

**Important:** Security alerts and advanced security alerts use the same command. When you create an advanced security events alert command from the CLI, specify the services for the alert. If you do not specify the services for the alert, the mean detection time changes from 5 minutes to 2 hours.

Creating a universal SSL alert policy from the CLI

To create an alert policy for certificate events from the CLI, run the following command:

```
$ ibmcloud cis alert-policy certificate-alert-create --type (universal | dedicated | custom) --name NAME --emails EMAIL [--emails EMAIL ...] [--webhooks WEBHOOK_ID ...] [--enabled (true | false)] [--include-future-certificates (true | false)] [--certificates CERT_ID ...] [-i, --instance INSTANCE] [-o, --output FORMAT]
```

Command options

`--type`

	The type of the certificate.
<code>--name</code>	
	The name of the alert policy.
<code>--description</code>	
	The description for the alert policy.
<code>--emails</code>	
	The email addresses for dispatching an alert notification. For example, <code>--emails test1@cn.ibm.com,test2@cn.ibm.com</code>
<code>--webhooks</code>	
	The webhook ID that for dispatching an alert notification. For example, <code>--webhook webhookID1,webhookID2</code>
<code>--enabled</code>	
	Sets whether the alert policy is enabled.
<code>-i, --instance</code>	
	The instance name or ID. If not set, the context instance that is specified by <code>ibmcloud cis instance-set INSTANCE</code> is used.
<code>--output</code>	
	Specifies the output format; only JSON is supported.

Creating a load balancer health check alert policy from the CLI

To create an alert policy for changes in load balancer health check alerts from the CLI, run the following command:

```
$ ibmcloud cis alert-policy glb-healthcheck-alert-create --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) --pools POOLS [--include-future-pools (true | false)] [--health-status-trigger (healthy | unhealthy | either)] [--event-source-trigger (pool | origin | either)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

<code>--name</code>	
	The name of the alert policy.
<code>--description</code>	
	The description for the alert policy.
<code>--emails</code>	
	The email addresses for dispatching an alert notification (for example, <code>--emails test1@cn.ibm.com,test2@cn.ibm.com</code>).
<code>--webhooks</code>	
	The webhook ID that for dispatching an alert notification (for example, <code>--webhook webhookID1,webhookID2</code>).
<code>--enabled</code>	
	Sets whether the alert policy is enabled.
<code>--pools</code>	
	The IDs of origin pool. If set to <code>all</code> , all the pool IDs are used.
<code>--include-future-pools</code>	
	Sets whether include the future pools (default "false").
<code>--health-status-trigger</code>	
	The trigger condition to fire the notification. Valid values: "healthy", "unhealthy", "either" (default "either").
<code>--event-source-trigger</code>	
	The event source of trigger to fire the notification. Valid values: "pool", "origin", "either" (default "either").
<code>-i, --instance</code>	
	The instance name or ID. If not set, the context instance that is specified by <code>ibmcloud cis instance-set INSTANCE</code> is used.
<code>--output</code>	

Specifies the output format; only JSON is supported.

Updating a DDoS attack alert policy from the CLI

To update a DDoS attack alert policy from the CLI, run the following command:

```
$ ibmcloud cis alert-policy ddos-attack-l7-alert-update POLICY_ID [--name NAME] [--emails EMAILS] [--webhooks WEBHOOKS] [--enabled (true | false)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

POLICY_ID

The ID of alert policy.

--name

The name of the alert policy.

--description

The description for the alert policy.

--emails

The email addresses for dispatching an alert notification (for example, **--emails test1@cn.ibm.com,test2@cn.ibm.com**).

--webhooks

The webhook IDs for dispatching an alert notification (for example, **--webhooks webhookID1,webhookID2**).

--enabled

Determines whether the alert policy is enabled.

-i, --instance

The instance name or ID.

--output

Specifies the output format; only JSON is supported.

Updating a pool toggle alert policy from the CLI

To update a pool toggle alert policy from the CLI, run the following command:

```
$ ibmcloud cis alert-policy pool-toggle-alert-update POLICY_ID [--name NAME] [--emails EMAILS] [--webhooks WEBHOOKS] [--enabled (true | false)] [--pools POOLS] [--trigger-condition (enabled | disabled | either)] [--include-future-pools (true | false)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

--name

The name of the alert policy.

--description

The description for the alert policy.

--emails

The email addresses for dispatching an alert notification (for example, **--emails test1@cn.ibm.com,test2@cn.ibm.com**).

--webhooks

The webhook IDs for dispatching an alert notification (for example, **--webhooks webhookID1,webhookID2**).

--enabled

Determines whether the alert policy is enabled.

`--pools`

The IDs of origin pool. If set to all, all pool IDs are used.

`--trigger-condition`

The condition of pool toggle status.

`--include-future-pools`

Determines whether to include the future pools.

`-i, --instance`

The instance name or ID.

`--output`

Specifies the output format; only JSON is supported.

Updating a security alert policy from the CLI

To update a security alert policy from the CLI, run the following command:

```
$ ibmcloud cis alert-policy firewall-events-alert-update POLICY_ID [--name NAME] [--emails EMAILS] [--webhooks WEBHOOKS] [--enabled (true | false)] [--domains DOMAINS] [--services SERVICES] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification (for example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`).

`--webhooks`

The webhook IDs for dispatching an alert notification (for example, `--webhooks webhookID1,webhookID2`).

`--domains`

The domain IDs for the alert policy (for example, `--domains domainID1,domainID2`).

`--services`

Specifies which services the alert monitors (Enterprise plan only). Valid services: `country-access-rules` , `waf` , `firewall-rules` , `ratelimit` , `securitylevel` , `ip-access-rules` , `browser-integrity-check` , `ua-rules` , `lockdowns` , `iprange-access-rules` , `asn-access-rules` , `managed-firewall` , `hotlink` , `ssl-validation` .

`--enabled`

Determines whether the alert policy is enabled.

`-i, --instance`

The instance name or ID.

`--output`

Specifies the output format; only JSON is supported.

Deleting an alert policy from the CLI

To delete an alert policy from the CLI, run the following command:

```
$ ibmcloud cis alert-policy delete POLICY_ID [-i, --instance INSTANCE] [-f, --force]
```

Command options

POLICY_ID

The id of alert policy.

-f, --force

Attempts to delete alert policy without prompting for confirmation.

-i, --instance

The instance name or ID.

Creating an alert policy with the API

To create an email alert with the API, follow these steps:

- a. Log in to your IBM Cloud account.
- b. Get a token.
- c. Using that token, run one of the following commands:
 - DDoS attack layer 7
 - Pool toggle alert
 - Security (WAF) alert
 - Advanced security (WAF) alert

Creating a DDoS attack policy with the API

To create a DDoS attack layer 7 alert policy with the API, run the following command:

```
curl -X POST \
https://api.cis.cloud.ibm.com/v1/:crn/alerting/policies \
-H 'content-type: application/json' \
-H 'x-auth-user-token: Bearer xxxxxx' \
-d '{"name":"Example Policy","enabled":true,"alert_type":"dos_attack_l7","mechanisms":{"email":{"id":"cistestemail@ibm.com"},"webhooks":[]}}'
```

Where:

- **-d** is the array of attributes that are required to create the alert.
 - **name** is the name of the alert.
 - **enabled** is the state of the alert (one of `true` , `false`).
 - **alert_type** is the type of the alert (one of `dos_attack_l7` , `load_balancing_pool_enablement_alert` , `clickhouse_alert_fw_anomaly` , `clickhouse_alert_fw_ent_anomaly` , `dedicated_ssl_certificate_event_type` , `universal_ssl_event_type` , `load_balancing_health_alert` , or `web_analytics_metrics_update`).
 - **mechanisms** are at least one of `email` , `webhooks` .
 - **description** (optional) is the description of the alert.

Creating a pool toggle alert policy with the API

To create the pool toggle alert policy with the API, run the following command:

```
curl -X POST \
https://api.cis.cloud.ibm.com/v1/:crn/alerting/policies \
-H 'content-type: application/json' \
-H 'x-auth-user-token: Bearer xxxxxx' \
-d '{"name":"Example Policy","enabled":true,"alert_type":"load_balancing_pool_enablement_alert","mechanisms":{"email":{"id":"cistestemail@ibm.com"},"webhooks":[]},
  "filters": {
    "enabled": [
      "false",
      "true"
    ],
    "pool_id": [
      "6e67c08e3bae7eb398101d08def8a68a",
      "df2d9d70fcb194ea60d2e58397cb35a6"
    ]
  }
}'
```



```
  ],
},
"conditions": {
}}
```

Where:

- **-d** is the array of attributes that are required to create the alert.
 - **name** is the name of the alert.
 - **enabled** is the state of the alert (one of `true` , `false`).
 - **alert_type** is the type of the alert (one of `dos_attack_l7` , `load_balancing_pool_enablement_alert` , `clickhouse_alert_fw_anomaly` , `clickhouse_alert_fw_ent_anomaly` , `dedicated_ssl_certificate_event_type` , `universal_ssl_event_type` , or `load_balancing_health_alert`).
 - **mechanisms** are at least one of `email` , `webhooks` .
 - **description** (optional) is the description of the alert.
 - **filter** is the list of all enablement statuses and pool IDs for the pool toggle alert.
 - **conditions** describe for all pools whether the pool is being enabled, disabled, or both. Content is generated automatically if the field is empty.

Creating a security (WAF) alert policy with the API

To create a WAF alert policy with the API, run the following command:

```
curl -X POST \
https://api.cis.cloud.ibm.com/v1/:crn/alerting/policies \
-H 'Content-Type: application/json' \
-H 'X-Auth-User-Token: Bearer xxxxxx' \
-d '{
  "name": "WAF Alerter",
  "description": "Send an email on spike in firewall events for any service",
  "enabled": true,
  "alert_type": "clickhouse_alert_fw_anomaly",
  "mechanisms": {
    "email": [
      {
        "id": "sreteam@techcompany.com"
      }
    ]
  },
  "filters": {
    "zones": [
      "123456ab7d8e9f0g12h2j34l5mn6op78"
    ]
  }
}
```

Where:

- **-d** is the array of attributes that are required to create the alert.
 - **name** is the name of the alert.
 - **description** (optional) is the description of the alert.
 - **enabled** is the state of the alert (one of `true` , `false`).
 - **alert_type** is the type of the alert (one of `dos_attack_l7` , `load_balancing_pool_enablement_alert` , `clickhouse_alert_fw_anomaly` , `clickhouse_alert_fw_ent_anomaly` , `dedicated_ssl_certificate_event_type` , `universal_ssl_event_type` , or `load_balancing_health_alert`).
 - **mechanisms** are at least one of `email` , `webhooks` .
 - **filters** are the list of all zones for the WAF alert.

Creating an advanced security (WAF) alert policy with the API

To create an advanced WAF alert policy with the API, run the following command:

```
curl -X POST \
https://api.cis.cloud.ibm.com/v1/:crn/alerting/policies \
```

```
-H 'Content-Type: application/json' \
-H 'X-Auth-User-Token: Bearer xxxxxx' \
-d '{
  "name": "WAF Alerter",
  "description": "Send an email on spike in firewall events for WAF or browser integrity check",
  "enabled": true,
  "alert_type": "clickhouse_alert_fw_ent_anomaly",
  "mechanisms": {
    "email": [
      {
        "id": "sreteam@techcompany.com"
      }
    ]
  },
  "filters": {
    "services": [
      "waf",
      "bic"
    ],
    "zones": [
      "123456ab7d8e9f0g12h2j34l5mn6op78"
    ]
  }
}'
```

Where:

- **-d** is the array of attributes that are required to create the alert.
 - **name** is the name of the alert.
 - **description** (optional) is the description of the alert.
 - **enabled** is the state of the alert (one of `true` , `false`).
 - **alert_type** is the type of the alert (one of `dos_attack_l7` , `load_balancing_pool_enablement_alert` , `clickhouse_alert_fw_anomaly` , `clickhouse_alert_fw_ent_anomaly` , `dedicated_ssl_certificate_event_type` , `universal_ssl_event_type` , or `load_balancing_health_alert`).
 - **mechanisms** are at least one of `email` , `webhooks` .
 - **filters** are the list of all services to monitor for security events and zones for the Advanced WAF alert.

You can monitor the following services:

Services	Log value
Country IP access rules	country
WAF	waf
Firewall rules	firewallrules
Rate limiting	ratelimit
Security level	securitylevel
IP access rules	ip
Validation	validation
Browser integrity check	bic
Hot link protection	hot
User agent block	uablock
Zone lockdown	zonelockdown

IP range access rules	iprange
ASN IP access rules	asn
Custom firewall	firewallCustom
Managed firewall	firewallManaged
Data loss prevention	dlp

Services that can be monitored by Advanced WAF alerts

Creating a universal SSL alert policy with the API

To create a universal SSL alert policy with the API, run the following command:

```
curl -X POST \
  https://api.cis.cloud.ibm.com/v1/:crn/alerting/policies \
  -H 'content-type: application/json' \
  -H 'x-auth-user-token: Bearer xxxxxx' \
  -d '{"name":"Example Policy","enabled":true,"alert_type":"universal_ssl_event_type","mechanisms":{"email":
[{"id":"cistestemail@ibm.com"}],"webhooks":[]}]'
```

Where:

- **-d** is the array of attributes that are required to create the alert.
 - **name** is the name of the alert.
 - **enabled** is the state of the alert (one of `true` , `false`).
 - **alert_type** is the type of the alert (one of `dos_attack_l7` , `load_balancing_pool_enablement_alert` , `clickhouse_alert_fw_anomaly` , `clickhouse_alert_fw_ent_anomaly` , `dedicated_ssl_certificate_event_type` , `universal_ssl_event_type` , or `load_balancing_health_alert`).
 - **mechanisms** is at least one of `email` , `webhooks` .
 - **description** (optional) is the description of the alert.

Creating a load balancer health check alert policy with the API

To create a load balancer health check alert policy with the API, run the following command:

```
curl -X POST \
https://api.cis.cloud.ibm.com/v1/:crn/alerting/policies \
-H 'content-type: application/json' \
-H 'x-auth-user-token: Bearer xxxxxx' \
-d '{"name":"Example Policy","enabled":true,"alert_type":"load_balancing_health_alert","mechanisms":{"email":
[{"id":"cistestemail@ibm.com"}],"webhooks":[]},
  "filters": {
    "event_source": [
      "pool",
      "origin"
    ],
    "new_health": [
      "Healthy",
      "Unhealthy"
    ],
    "pool_id": [
      "6e67c08e3bae7eb398101d08def8a68a",
      "df2d9d70fcb194ea60d2e58397cb35a6"
    ]
  }
}'
```

Where:

- **-d** is the array of attributes that are required to create the alert.
 - **name** is the name of the alert.

- **enabled** is the state of the alert (one of `true` , `false`).
- **alert_type** is the type of the alert (one of `dos_attack_l7` , `load_balancing_pool_enablement_alert` , `clickhouse_alert_fw_anomaly` , `clickhouse_alert_fw_ent_anomaly` , `dedicated_ssl_certificate_event_type` , `universal_ssl_event_type` , or `load_balancing_health_alert`).
- **mechanisms** is at least one of `email` , `webhooks` .
- **description** (optional) is the description of the alert.
- **filter** is the list of all sources, pools, health status for the load balancing health alert.

Updating an alert policy with the API

To update an alert policy with the API, such as an email alert policy, run the following command:

```
curl -X PUT \
https://api.cis.cloud.ibm.com/v1/:crn/alerting/policies/:policy_id \
-H 'content-type: application/json' \
-H 'x-auth-user-token: Bearer xxxxxx' \
-d '{"name":"Example Policy","enabled":true,"alert_type":"dos_attack_l7","conditions":{},"mechanisms":{"email":
[{"id":"cistestemail@ibm.com"}]},"webhooks":[]}]'
```

 **Tip:** The **conditions** field is required, even though it might be empty.

Deleting an alert policy with the API

To delete an alert policy with the API, run the following command:

```
curl -X DELETE \
https://api.cis.cloud.ibm.com/v1/:crn/alerting/policies/:policy_id \
-H 'content-type: application/json' \
-H 'accept: application/json' \
-H 'x-auth-user-token: Bearer xxxxxx'
```

Configuring webhooks

Webhooks allow IBM Cloud® Internet Services to send real-time notifications to external systems when specific events occur in your account. By integrating webhooks with your alert policies, you can automate responses or trigger workflows in third-party services.

You can configure webhooks using the IBM Cloud console, CLI, or API.

 **Note:** Alerts and webhooks are available only on Enterprise plans.

Configuring webhooks in the console

To configure webhooks in the console, navigate to your **Account** page and select the **Alerts** tab. In the Alerts section, select the **webhooks** tab.

Creating a webhook in the console

- Click **Create**.
- Enter a name for your webhook.
- Enter the webhook URL. This URL can't be updated; if you want to change it, you must delete and re-create the webhook.
- Enter the webhook secret. The secret can't be updated; if you want to change it, you must delete and re-create the webhook.

Editing a webhook in the console

To edit a webhook in the console, click the Edit icon next to the name of the webhook that you want to update. You can edit only the name of the webhook. If you need to modify a webhook, delete and re-create the webhook instead of editing it.

Deleting a webhook in the console

To delete a webhook in the console, click the Actions menu  of the webhook that you want to delete, then select **Delete**. Select **Delete** in the confirmation window.

Configuring webhooks from the CLI

Creating a webhook from the CLI

To create a webhook from the CLI, run the following command:

```
$ ibmcloud cis alert-webhook-create --name NAME --url URL [--secret SECRET] [-i, --instance INSTANCE] [--output FORMAT]
```

Where:

- **--name value** is the name of the webhook.
- **--url value** is the POST endpoint to call when an alert is dispatched.
- **--secret value** is the secret that is passed in the webhook auth header when a webhook alert is dispatched.
- **-i, --instance value** is the instance name or ID. If not set, the context instance that is specified by 'cis instance-set INSTANCE' is used.
- **--output value** specifies the output format; only JSON is supported.

Listing all webhooks from the CLI

To list a webhook from the CLI, run the following command.

```
$ ibmcloud cis alert-webhooks [-i, --instance INSTANCE] [--output FORMAT]
```

Where:

- **-i, --instance value** is the instance name or ID. If not set, the context instance that is specified by 'cis instance-set INSTANCE' is used.
- **--output value** specifies the output format; only JSON is supported.

Getting details for a webhook from the CLI

To get details for a webhook from the CLI, run the following command:

```
$ ibmcloud cis alert-webhook WEBHOOK_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Where:

- **WEBHOOK_ID** is the ID of webhook.
- **-i, --instance value** is the instance name or ID. If not set, the context instance that is specified by 'cis instance-set INSTANCE' is used.
- **--output value** specifies the output format; only JSON is supported.

Updating a webhook from the CLI

To update a webhook from the CLI, run the following command:

```
$ ibmcloud cis alert-webhook-update WEBHOOK_ID [--name NAME] [--url URL] [--secret SECRET] [-i, --instance INSTANCE] [--output FORMAT]
```

Where:

- **WEBHOOK_ID** is the ID of webhook.
- **--name value** is the name of the webhook.
- **--url value** is the POST endpoint to call when an alert is dispatched.
- **--secret value** is the secret that is passed in the webhook auth header when a webhook alert is dispatched.

- **-i, --instance value** is the instance name or ID. If not set, the context instance that is specified by 'cis instance-set INSTANCE' is used.
- **--output value** specifies the output format; only JSON is supported.

Deleting a webhook from the CLI

To delete a webhook from the CLI, run the following command:

```
$ ibmcloud cis alert-webhook-delete WEBHOOK_ID [-i, --instance INSTANCE] [-f, --force]
```

Where:

- **WEBHOOK_ID** is the ID of webhook.
- **i, --instance value** is the instance name or ID. If not set, the context instance that is specified by 'cis instance-set INSTANCE' is used.
- **-f, --force** attempts to delete webhook without prompting for confirmation.

Configuring webhooks with the API

To call these methods, you must be assigned one or more IAM access roles.

- `internet-svcs.zones.read`
- `internet-svcs.zones.update`

 **Tip:** You can check your access by going to **Users > name > Access policies**.

Creating a webhook with the API

Creating a webhook alert is a two-step process. First, create the webhook, then use the ID in the response that you receive to create the alert.

To create a webhook by with the API, follow these steps:

- Set up your API environment with the correct variables.
- Store the following variables to be used in the API command:
 - `crn`: the full URL-encoded CRN of the service instance.
 - `name`: the name of the webhook.
 - `url`: the URL of the webhook.
 - `secret`: the optional secret or API key that is needed to use the webhook.
- When all variables are initiated, run the following command to create the webhook:

```
curl -X POST
https://api.cis.cloud.ibm.com/v1/:crn/alerting/destinations/webhooks
-H 'content-type: application/json'
-H 'x-auth-user-token: Bearer xxxxxx'
-d '{"name":"Example
Webhook","url":"https://hooks.slack.com/services/Ds3fdBFbV/456464Gdd"}'
```

The following response is returned:

```
{
  "result": {
    "id": "6d16fcab-3e80-44b3-b59b-a3716237832e"
  },
  "success": true,
  "errors": [],
  "messages": []
}
```

Use the ID in the response that you received to create the alert:

```
curl -X POST \
https://api.cis.cloud.ibm.com/v1/:crn/alerting/policies \
-H 'content-type: application/json' \
-H 'x-auth-user-token: Bearer xxxxxx' \
-d '{"name":"Example Policy","enabled":true,"alert_type":"dos_attack_l7","mechanisms":{"email":[{"id":"cistestemail@ibm.com"}],"webhooks":[{"id": "6d16fcab3e8044b3b59ba3716237832e"}]}'
```

Listing all webhooks with the API

To list all webhooks with the API, follow these steps:

- Set up your API environment with the correct variables.
- Store the following variable to be used in the API command:
 - `crn`: the full URL-encoded CRN of the service instance.
- When all variables are initiated, get the list of webhooks:

```
curl -X GET
https://api.cis.cloud.ibm.com/v1/:crn/alerting/destinations/webhooks
-H 'content-type: application/json'
-H 'accept: application/json'
-H 'x-auth-user-token: Bearer xxxxxx'
```

Getting details for a webhook with the API

To get the details of a webhook with the API, follow these steps:

- Set up your API environment with the correct variables.
- Store the following variables to be used in the API command:
 - `crn`: the full URL-encoded CRN of the service instance.
 - `webhook_id`: alert webhook identifier.
- When all variables are initiated, get the webhook details:

```
curl -X GET
https://api.cis.cloud.ibm.com/v1/:crn/alerting/destinations/webhooks/:webhook_id
-H 'content-type: application/json'
-H 'accept: application/json'
-H 'x-auth-user-token: Bearer xxxxxx'
```

Updating a webhook with the API

To update a webhook with the API, follow these steps:

- Set up your API environment with the correct variables.
- Store the following variables to be used in the API command:
 - `crn`: the full URL-encoded CRN of the service instance.
 - `webhook_id`: alert webhook identifier.
 - `name`: the name of the webhook.
 - `url`: the URL of the webhook.
 - `secret`: the optional secret or API key that is needed to use the webhook.
- When all variables are initiated, update the webhook:

```
curl -X PUT
https://api.cis.cloud.ibm.com/v1/:crn/alerting/destinations/webhooks/:webhook_id
-H 'content-type: application/json'
-H 'x-auth-user-token: Bearer xxxxxx'
-d '{"name":"Example Webhook","url":"https://hooks.slack.com/services/Ds3fdBFbV/456464Gdd"}'
```

Deleting a webhook with the API

To delete a webhook by with the API, follow these steps:

- a. Set up your API environment with the correct variables.
- b. Store the following variables to be used in the API command:
 - `crn`: the full URL-encoded CRN of the service instance.
 - `webhook_id`: alert webhook identifier.
- c. When all variables are initiated, delete the webhook:

```
curl -X DELETE
https://api.cis.cloud.ibm.com/v1/:crn/alerting/destinations/webhooks/:webhook_id
-H 'content-type: application/json'
-H 'accept: application/json'
-H 'x-auth-user-token: Bearer xxxxxx'
```

Working with Bot management

About bot management

Bot management blocks undesired or malicious internet bot traffic while allowing useful bots to access web properties. Bot management detects bot activity, discerns between desirable and undesirable bot behavior, and identifies the sources of the undesirable activity.

Unmanaged bots can cause serious issues for web properties. Excessive bot traffic can put a heavy load on web servers, slowing or denying service to legitimate users (for example, DDoS attacks). Malicious bots can perform cyber attacks such as scraping content from websites, stealing user credentials, and spreading spam content.

What bot managers do

A bot manager is any software product that manages bots. Bot managers should be able to block some bots and allow others through, instead of simply blocking all non-human traffic. If all bots are blocked and Google bots aren't able to index a page, for instance, then that page can't show up in Google search results, resulting in greatly reduced organic traffic to the website.

An effective bot manager accomplishes the following goals:

- Distinguishes bots from human visitors
- Identifies bot reputation
- Identifies bot origin IP addresses and block based on IP reputation
- Analyzes bot behavior
- Adds "good" bots to allowlists
- Challenges potential bots with a CAPTCHA test, JavaScript injection, or other methods
- Rate limits any potential bot that is over-using a service
- Denies access to certain content or resources for "bad" bots
- Serves alternative content to bots

What are bots and what do they do?

A bot is a computer program that automatically does certain actions on a network. The tasks a bot is programmed to do are fairly simple, but the bot can do these tasks repeatedly at a much faster rate than a human can.

Bots don't access the internet using a traditional web browser or a mouse to interact with visual content. Bots are software programs that typically use a "headless browser" to make HTTP requests and other activities.

Bots can do almost any repetitive, non-creative task that can be automated, including filling out forms, reading and downloading content, and even hold basic conversations with humans as chatbots. Like any tool that can be used for good, bots can be used for malicious behavior, too.

Differences between good bots and bad bots

It is estimated that up to half of all internet traffic is bot traffic. Some bots are malicious, and some are "good."

Bots that misuse online products or services can be considered "bad." Bad bots range from malicious to simply annoying; for instance, breaking into user accounts to steal data or buying concert tickets online to assist scalpers.

A bot that performs a helpful service can be considered "good." For example, customer service chatbots, search engine crawlers, and performance monitoring bots are generally good bots. Good bots look for and abide by the rules outlined in a website's `robots.txt` file.

The `robots.txt` file

`Robots.txt` is a file that outlines the rules for bots accessing properties on a web server, though the file itself does not enforce these rules. Anyone programming a bot is expected to follow an honor system and make sure that their bot checks a website's `robots.txt` file before accessing the website. Malicious bots don't follow this system, which generates the need for bot management.

How bot management works

To identify bots, bot managers might use JavaScript challenges (which determine whether a traditional web browser is being used) or CAPTCHA challenges. They can also determine which users are humans and which are bots by behavioral analysis; by comparing a user's behavior to the standard behavior of users in the past.

When a bot is identified as bad, it can be redirected to a different page or blocked from accessing a web resource altogether.

Good bots can be added to an allowlist. A bot manager can also distinguish between good and bad bots by using further behavioral analysis.

Another bot management approach is to use the `robots.txt` file to set up a honeypot. A honeypot is a fake target for bad actors that, when accessed, exposes the bad actor as malicious. In the case of a bot, a honeypot could be a webpage on the site that's forbidden to bots by the `robots.txt` file. Good bots will read the `robots.txt` file and avoid that webpage; some bad bots will crawl the webpage. By tracking the IP address of the bots that access the honeypot, bad bots can be identified and blocked.

Kinds of bot attacks bot management mitigates

A bot management solution can help stop a variety of attacks, including the following:

- DDoS attacks
- DoS attacks
- Credential stuffing
- Credit card stuffing
- Brute force password cracking
- Spam content
- Data scraping and web scraping
- Email address harvesting
- Ad fraud
- Click fraud

These bot activities are not always considered "malicious," but a bot manager should still be able to mitigate them:

- Inventory hoarding
- Automated posting on social forums or platforms
- Shopping cart stuffing

How does CIS manage bots?

CIS collects data from requests flowing through its network every day. With this data, powered by Cloudflare's machine learning and behavior analysis, CIS can identify likely bot activity and can provide you with information on how to allow or disallow specific bot traffic using Firewall Rules.

Cloudflare Bot Management uses the following detection mechanisms, each producing their own scores, which are then combined to form a single score:

- Machine learning: A highly accurate bot identification machine learning model trained on trillions of requests with minimal impact to request processing speed
- Heuristics engine: Detects bots by screening requests through a set of simple rules that capture bots based on certain attributes of the requests made.
- Behavioral analysis: Detects bots that have never been seen, calculating and analyzing normal visitor behavior over an extended period of time.
- Verified bots: A way to avoid accidental blocks of useful bots using several validators and a bots directory of unique good bot identities.
- JS fingerprinting: A challenge-response system with challenge injected into the webpage on Cloudflare's edge and rendered in the background for validation.

Machine learning models

Enterprise customers can enable auto-updates to their machine learning models for the newest bot detection models as

they are released.

To enable auto-updates:

- a. Log in to the CIS and select your account and domain.
- b. Go to **Security > Bots**.
- c. Select **Configure Bot Management**.
- d. Enable auto-updates to the machine learning model.



Note: Enabling auto-updates for the machine learning model upgrades you to the latest version immediately. You can toggle the button off within 24 hours to revert to the previous version. To make changes after 24 hours, contact support.

What has changed?

If you are on an older machine learning model, you'll see a score change to requests scored by the machine learning source instantly. If you are already on the latest model, the changes show only after a new machine learning model becomes the global default.

You will be notified prior to a new machine learning model becoming the global default.

Why you should upgrade

By not updating to the latest version, you will be using a machine learning model that is no longer maintained or monitored by the engineering team. As internet traffic changes and new trends evolve, the scoring accuracy of older versions can degrade.

JavaScript detections

IBM Cloud® Internet Services (CIS) bot features include JavaScript detections. A small amount of JavaScript is injected into client devices using [Google's Picasso fingerprinting technique](#). Picasso results are factored into bot scores and help CIS classify traffic as automated or human. `BotScoreSrc: Not Computed` and a score of `0` are relevant to Picasso JavaScript Fingerprinting requests. These requests are exempt from being blocked by any firewall rules.

This detection technique gathers general data about the machines reaching CIS. For example, if a particular user is accessing CIS via Google Chrome on a MacBook Pro, because there are millions of people using Google Chrome on a MacBook Pro, CIS can't identify specific individuals. CIS also takes steps to anonymize and phase out data for added privacy.

JavaScript is only injected in response to requests for HTML pages or page views, excluding AJAX calls. API and mobile app traffic is unaffected. Additionally, code is not injected again until its 30-minute session life expires. The Picasso script is roughly 70 KB and execution time varies by device, anywhere from 90 ms to around 500 ms.

The snippets of JavaScript will contain a source pointing to the challenge platform with paths that start with `/cdn-cgi/challenge-platform/...`.

Enable JavaScript detections

For no-cost Trial plans (Bot Fight Mode), JavaScript detections are automatically enabled and can't be disabled.

For all other plans (Super Bot Fight Mode and Bot Management for Enterprise), JavaScript detections are optional. To adjust your settings, go to **Security > Bots**.

Enforcing execution of JavaScript detections

To ensure that visitors have previously run and passed JavaScript detections, you can create a firewall rule with the field `cf.bot_management.js_detection.passed` which performs a Managed Challenge action. Enter `(cf.bot_management.js_detection.passed)` into the **Expression preview** section as a singular rule.

Considerations

- If you have a Content Security Policy (CSP):
 - Ensure that anything under `/cdn-cgi/challenge-platform/` is allowed. Your CSP should allow scripts that are served

from your origin domain (`script-src self`).

- If your CSP uses a `nonce` for script tags, CIS adds these nonces to the scripts it injects by parsing your CSP response header.
- If your CSP does not use `nonce` for script tags and JavaScript Detection is enabled, you might see a console error such as

```
Refused to execute inline script because it violates the following Content Security Policy directive: "script-src 'self'". Either the 'unsafe-inline' keyword, a hash ('sha256-b123b8a70+4jEj+d6gWI9U6iilUJlrlnRJbRR/uQl2Jc='), or a nonce ('nonce-...') is required to enable inline execution.
```

- It is not recommended to use `unsafe-inline`. Instead, it is recommend that you use CSP `nonces` in script tags which are parsed and supported in the CDN.

Bot management fields

Bot Management provides access to several new variables within the expression builder of Ruleset Engine-based features, such as [WAF custom rules](#).

Ruleset Engine fields

Bot Management provides access to several new variables within the expression builder of Ruleset Engine-based products, such as WAF custom rules.

- **Bot Score** (`cf.bot_management.score`): An integer between 1-99 that indicates that a request comes from a bot.
- **Verified Bot** (`cf.bot_management.verified_bot`): A boolean value that is true if the request comes from a good bot, like Google or Bing. Most customers choose to allow this traffic.
- **Serves Static Resource** (`cf.bot_management.static_resource`): An identifier that matches file extensions for many types of static resources. Use this variable if you send emails that retrieve static images.
- **ja3Hash** (`cf.bot_management.ja3_hash`) and **ja4** (`cf.bot_management.ja4`): A JA3/JA4 fingerprint helps you profile specific SSL/TLS clients across different destination IPs, Ports, and X509 certificates.
- **Bot Detection IDs** (`cf.bot_management.detection_ids`): List of IDs that correlate to the Bot Management heuristic detections made on a request (you can have multiple heuristic detections on the same request).
- **Verified Bot Categories** (`cf.verified_bot_category`): A string that allows you to segment your verified bot traffic by its type and purpose.

Log fields

After you enable Bot Management, CIS also surfaces bot information in its [HTTP requests log fields](#):

- BotDetectionIDs
- BotScore
- BotScoreSrc
- BotTags

Enhancing security

Managing access for CIS

Access to IBM Cloud® Internet Services service instances for users in your account is controlled by IBM Cloud Identity and Access Management (IAM). Every user that accesses the (CIS) service in your account must be assigned an access policy with an IAM role defined. The policy determines what actions that a user can perform within the context of the service or instance that you select. The allowable actions are customized and defined by the IBM Cloud service as operations that are allowed to be performed on the service. The actions are then mapped to IAM user roles.

Policies enable access to be granted at different levels. Some access includes the following options:

- Access across all instances of the service in your account
- Access to an individual service instance in your account

After you define the scope of the access policy, you assign a role, which determines the user's level of access.

If you are configuring a VPC with CIS, you can prevent DDoS attacks by allowing traffic only through CIS (allowlist) in your VPC. Set your Network ACL and Security groups to allowlist CIS traffic.

Review the following table that outlines what actions each role allows within the (CIS) service. The platform and service roles for CIS are listed under "Internet Services". If you're using the CLI or API to assign access, use `internet-svcs` for the service name.

Platform management roles enable users to perform tasks on service resources at the platform level, for example, assign user access for the service and create or delete instances.

For more information about IAM roles, see [IBM Cloud IAM roles](#).

Platform management role	Description of actions
Manager	Create and delete instances, domains, and configurations.
Reader	View information about instances and domains.
Service Configuration Reader	Read services configuration for Governance management.
Writer	Change existing configurations.

IAM user roles and actions

For information about assigning user roles in the console, see [Managing access to resources](#).

Learning about CIS architecture and workload isolation

Review the following sample architecture for IBM Cloud® Internet Services, and learn about different isolation levels so that you can choose the solution that best meets the requirements of the workloads that you want to run in the cloud.

CIS architecture and workload isolation

Cloud Internet Services (CIS) is a public, global, multitenant service offered in partnership with Cloudflare. It offers DNS name resolution, global load balancing, and security and CDN services, for zones or domains delegated to this service.

In the control plane, you can configure your zone and the services that are applied to traffic to your site through the UI, CLI, or API. All access authorization and authentication to your zone or domain is managed through Identity and Access Management (IAM) access policies.

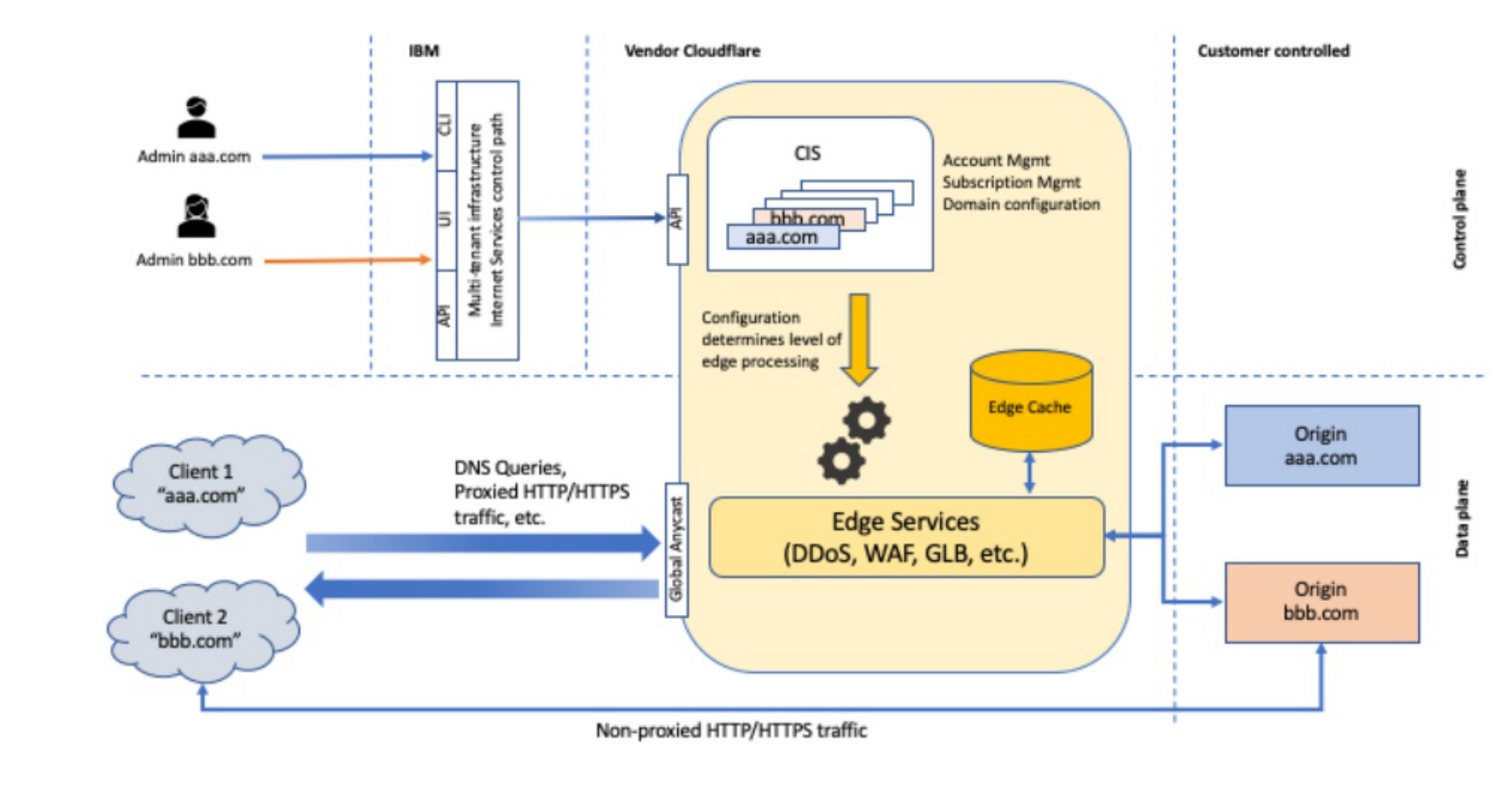
All configuration requests eventually reach the multitenant Cloud Internet Services (CIS) control plane on IBM Cloud® as API calls to an SSL-secured API endpoint. The control plane interacts with the platform IAM service to authenticate the user and authorize the action. The original request targets the customer's Cloud Internet Services (CIS) instance. Each Cloud Internet Services (CIS) instance is uniquely mapped to an anonymized subaccount in Cloudflare's system. The request is converted to a Cloudflare API request that targets the subaccount and is delivered via HTTPS to the Cloudflare API endpoint. Zones and domains from different customers are isolated and maintained in separate subaccounts within the Cloud Internet Services (CIS) account at Cloudflare. Access to the account at Cloudflare is strictly controlled and limited. Access to the Cloud

Internet Services (CIS) control plane infrastructure is also strictly controlled and limited to essential maintenance personnel only.

Data that is stored at Cloudflare is encrypted except when it is required to be publicly accessible. For example, in the case of DNS records, the control plane is separate from the data plane.

The data plane for your site is handled exclusively by Cloudflare. All proxied traffic is resolved to an IP address owned by Cloudflare and routed through Cloudflare's Anycast network to the nearest data center capable of processing the request. The request is processed by Cloudflare based on the zone's configuration. After all configured services (such as firewall rules, WAF rules, rate limits, global load balancing, and so on) are applied, Cloudflare replies to the request from its cache, or by requesting the necessary resources from the website's origin, which is controlled by the customer.

Non-proxied requests go directly from the client to the requested resource's origin. In this case, only DNS resolution is done by Cloudflare. The request data flows through the public internet.



Cloud Internet Services (CIS) Architecture overview

CIS workload isolation and deployment model

CIS is a public multitenant solution. Both control and data plane are shared between tenants, and accessed through public endpoints. Some data is required to be publicly accessible. In all other cases data is encrypted, at rest and in transit, using TLS.

Dependencies to other IBM Cloud services

Review the IBM Cloud services that IBM Cloud® Internet Services connects to or uses.

Critical dependencies

The following dependencies of IBM Cloud Internet Services are considered critical. Any loss of connectivity or service of one of these dependencies results in a functional impact to the customer on IBM Cloud Internet Services.

Service name	Description
IBM Cloud Resource Controller API and IBM Cloud catalog API	Used to load required information about your service instance and offering plan.
Global Search and Tagging (Ghost)	Used to look up information about other IBM Cloud services. For example, if you set up a job to push edge logs to your own IBM Cloud Object Storage bucket, the available instances and buckets are searched with Ghost.

IBM Cloud Kubernetes Service	Provides the infrastructure that IBM Cloud Internet Services is running on.
Business Support Services for IBM Cloud (BSS)	Used to access information about the IBM Cloud account, service subscription, service usage, and billing.
IBM Cloud Command Line (CLI)	Used to run commands from a command prompt. When IBM Cloud Internet Services runs commands, the service connects to the service API endpoint over the public service endpoint.
IBM Log Analysis	IBM Cloud Internet Services sends service logs to IBM Log Analysis. The service team uses these logs for analysis to identify issues and malicious activity.
IBM Cloud Activity Tracker	IBM Cloud Internet Services integrates with IBM Cloud Activity Tracker to forward auditable events to the IBM Cloud Activity Tracker service instance that is set up and owned by the user. For more information, see Auditing events for CIS . This service is also used by IBM Cloud Internet Services to store auditable events.
Identity and Access Management (IAM)	IBM Cloud Internet Services authenticates requests and determines authorization for all user actions based on platform and service access roles and policies in IAM. To learn more, see Managing access for CIS .
Object Storage (COS)	Used to store edge logs for a customer's data path traffic. You can also use this service to store operational logs of IBM Cloud Internet Services itself.

Critical dependencies

Other dependencies

Service name	Description
Certificate Manager	Used to store the TLS certificates for IBM Cloud Internet Services.
IBM Cloud Container Registry	Used to store the images that IBM Cloud Internet Services uses to run the service.
IBM Cloud Monitoring	IBM Cloud Internet Services sends service metrics to IBM Cloud Monitoring. The service team uses these metrics to identify capacity and performance issues of the service and monitor the operational health of the service.

Other dependencies

Dependencies to third-party services

Review the list of third-party services that IBM Cloud Kubernetes Service connects to over the public network.

Critical dependencies

Service name	Description
Cloudflare	Cloudflare is the third party provider for all data path services that are offered by IBM Cloud Internet Services, such as WAF, DDoS protection, and global load balancing.

Critical third party dependencies

Other dependencies

Service name	Description
PagerDuty	PagerDuty is used to notify on-call support of emergency and non-emergency issues that are related to operating IBM Cloud Internet Services or its support.
Other third party dependencies	

Securing your data in CIS

To ensure that you can securely manage your data when you use IBM Cloud® Internet Services, it is important to know exactly what data is stored and encrypted and how to delete any stored personal data.

How your data is stored and encrypted in CIS

CIS interacts with Cloudflare services using a channel that is fully encrypted end-to-end using Transport Layer Security (TLS) 1.2. CIS does not store any customer data. Configuration data about your specific CIS configuration is encrypted in transit and at rest. CIS configuration data is deleted on your request through the UI, CLI, or API.

Protecting your sensitive data in CIS

All data related to CIS configuration is not considered sensitive data. The configuration data is encrypted at rest. No customer-managed keys are managed by the CIS offering. Therefore, neither Key Protect nor Hyper Protect Crypto Services are used.

Working with customer-managed keys for CIS

No customer-managed keys are managed by the CIS offering.

Deleting your data in CIS

The CIS configuration data is deleted on request through the UI, CLI, or API.

Deleting CIS instances

The CIS data retention policy describes how long your data is stored after you delete the service. The data retention policy is included in the CIS service description, which you can find in [IBM Cloud Terms](#). When a CIS instance is deleted by the UI, CLI, or API, the instance data is retained for seven days from deletion.

**Note:** Before you delete an instance, all the domains in the instance must be removed.

Deleting the CIS instance removes all data.

Restoring deleted data for CIS

CIS can currently restore the deleted instance. After you delete an instance of CIS, you can restore the deleted service instance within the data retention period of seven days. After the seven-day period expires, the service instance is permanently deleted.

To view which service instances are available for restoration, use the `ibmcloud resource reclamations` command. To restore a deleted service instance, use the `ibmcloud resource reclamation-restore` command.

Using virtual private endpoints for VPC to privately connect to CIS

IBM Cloud® Virtual Private Endpoints (VPE) for VPC enables private connectivity from your VPC network to CIS by using the IP addresses of your choosing that are allocated from a subnet within your VPC.

VPEs are virtual IP interfaces that are bound to an endpoint gateway created on a per service, or service instance, basis (depending on the service operation model). VPEs are virtual IP interfaces that are bound to an endpoint gateway created on a per service, or service instance, basis (depending on the service operation model). The endpoint gateway is a virtualized

component that scales horizontally, is redundant, highly available, and spans all availability zones in your VPC. VPE gateways enable communication between virtual server instances in your VPC and cloud services. VPE for VPC gives you the experience of controlling all the private addressing within your cloud. For more information, see [About virtual private endpoint gateways](#).

Before you begin

Before you target a VPE for CIS, you must complete the following tasks.

- Ensure that a [Virtual Private Cloud is created](#).
- Make a plan for your [virtual private endpoints](#).
- Ensure that [correct access controls](#) are set for your VPE.
- Review VPE [limitations](#).

Setting up a VPE for CIS

When you create a VPE by using the CLI or API, use the following CRN information.

Location	Region	Cloud Resource Name (CRN)
Global	global	crn:v1:bluemix:public:internet-svcs:global:::

Region availability and Cloud Resource Names (CRNs) for connecting CIS over IBM Cloud private networks

Configuring an endpoint gateway

To configure a VPE gateway, follow these steps:

- List the available services, including IBM Cloud infrastructure services that are available by default to all VPC users.
- [Create an endpoint gateway](#) for CIS that you want to be privately available to the VPC.
- [Bind a reserved IP address](#) to the endpoint gateway.
- View the created VPE gateways associated with the CIS instance. For more information, see [Viewing details of an endpoint gateway](#).

Now your virtual server instances in the VPC can privately access your CIS instance through the VPE gateway.

Using an endpoint gateway for CIS

After you create a VPE for CIS, follow these steps:

Using an endpoint gateway from the CLI

To update to the latest version of the CLI and the CIS plug-in, follow these steps:

- Update the IBM Cloud CLI to the latest version:

```
$ ibmcloud update
```

- Update the CIS CLI plug-in:

```
$ ibmcloud plugin update cloud-internet-services
```

- Log in by using the private IBM Cloud endpoint at `private.cloud.ibm.com`. For more information about logging into the private cloud, see [Securing your connection when using the IBM Cloud CLI](#).

Using an endpoint gateway with the VPC API

After you create a VPE for CIS, use the service endpoint FQDN `api.private.cis.cloud.ibm.com` in the URL to access the service. For example:

```
$ curl https://api.private.cis.cloud.ibm.com/v1/internet-svcs -H "Authorization: Bearer $iam_token"
```

Using an endpoint gateway with the SDK

After you create a VPE for CIS, you must use the private endpoint FQDN when you set the service endpoint during construction of the CIS gateway service object.

```
$ api.private.cis.cloud.ibm.com
```

For examples of setting the service's FQDN for the specific SDK language, see [SDK API examples](#).

Using an endpoint gateway with Terraform

If you plan to access CIS with Terraform, make sure to set the `IBMCLLOUD_PRIVATE_CIS_API_ENDPOINT` environment variable to `api.private.cis.cloud.ibm.com`. For example:

```
$ export IBMCLLOUD_PRIVATE_CIS_API_ENDPOINT=api.private.cis.cloud.ibm.com
```

For more information, see [Getting started with Terraform on IBM Cloud](#).

Observability

Activity tracking events for CIS

IBM Cloud services, such as IBM Cloud Internet Services, generate activity tracking events.

Activity tracking events report on activities that change the state of a service in IBM Cloud. You can use the events to investigate abnormal activity and critical actions and to comply with regulatory audit requirements.

You can use IBM Cloud Activity Tracker Event Routing, a platform service, to route auditing events in your account to destinations of your choice by configuring targets and routes that define where activity tracking events are sent. For more information, see [About IBM Cloud Activity Tracker Event Routing](#).

You can use IBM Cloud Logs to visualize and alert on events that are generated in your account and routed by IBM Cloud Activity Tracker Event Routing to an IBM Cloud Logs instance.

CIS uses global control planes located in Dallas (DAL), Washington DC (WDC), and Frankfurt (FRA). Events are generated by the control plane that processes the request, regardless of where the resource itself is located.

Locations where activity tracking events are generated

By default, CIS Activity Tracker events are stored in the Frankfurt (`eu-de`) region. If you prefer not to store these events in that region, you can configure Internet Services Activity Tracker Event Routing to send them to any supported region.

Viewing activity tracking events for CIS

You can use IBM Cloud Logs to visualize and alert on events that are generated in your account and routed by IBM Cloud Activity Tracker Event Routing to an IBM Cloud Logs instance.

Launching IBM Cloud Logs from the Observability page

For information on launching the IBM Cloud Logs UI, see [Launching the UI in the IBM Cloud Logs documentation](#).

Events for DNS domains

Action	Description
<code>internet-svcs.zones.create</code>	Create a DNS domain.
<code>internet-svcs.zones.update</code>	Update a DNS domain.
<code>internet-svcs.zones.delete</code>	Delete a DNS domain.
<code>internet-svcs.zones-activation-check.update</code>	Run activation check for a DNS domain.
<code>internet-svcs.dnssec.update</code>	Enable or disable DNSSEC for a DNS domain.

Actions that generate DNS domain events

Events for DNS records

The following table lists the actions that are related to DNS records and generate an event:

Action	Description
<code>internet-svcs.dns-records.create</code>	Create a DNS record.
<code>internet-svcs.dns-records.update</code>	Update a DNS record.
<code>internet-svcs.dns-records.delete</code>	Delete a DNS record.

internet-svcs.dns-records-bulk.create	Import DNS records from zone file.
internet-svcs.dns-records-batch.create	Create DNS records in batch.
internet-svcs.dns-records-batch.update	Update DNS records in batch.
internet-svcs.dns-records-batch.delete	Delete DNS records in batch.

Actions that generate DNS record events

Events for load balancers

The following table lists the actions that are related to load balancers and generate an event:

Action	Description
internet-svcs.load-balancers.create	Create a global load balancer.
internet-svcs.load-balancers.update	Update a global load balancer.
internet-svcs.load-balancers.delete	Delete a global load balancer.
internet-svcs.load-balancer-monitors.create	Create a global load balancer health check.
internet-svcs.load-balancer-monitors.update	Update a global load balancer health check.
internet-svcs.load-balancer-monitors.delete	Delete a global load balancer health check.
internet-svcs.load-balancer-pools.create	Create a global load balancer pool.
internet-svcs.load-balancer-pools.update	Update a global load balancer pool.
internet-svcs.load-balancer-pools.delete	Delete a global load balancer pool.

Actions that generate load balancer events

Events for purging the cache

The following table lists the actions that are related to purging the cache and generate an event:

Action	Description
internet-svcs.purge-cache-all.update	Purge all cached assets of a domain from edge server.
internet-svcs.purge-cache-by-urls.update	Purge cached assets by URLs from edge server.
internet-svcs.purge-cache-by-cache-tags.update	Purge cached assets by cache tags from edge server.
internet-svcs.purge-cache-by-hosts.update	Purge cached assets by hostnames from edge server.
internet-svcs.purge-cache-by-prefixes.update	Purge cache by URL prefixes.

Actions that generate cache purge events

Events for page rules

The following table lists the actions that are related to page rules and generate an event:

Action	Description
internet-svcs.pagerules.create	Create a page rule.
internet-svcs.pagerules.update	Update a page rule.
internet-svcs.pagerules.delete	Delete a page rule.
Actions that generate page rule events	

Events for firewalls

The following table lists the actions that are related to firewalls and generate an event:

Action	Description
internet-svcs.waf-groups.update	Enable or disable a group of WAF rulesets.
internet-svcs.waf-rules.update	Enable or disable a WAF rule.
internet-svcs.ip-firewall-rules.create	Create an IP firewall rule at the domain level or instance level.
internet-svcs.ip-firewall-rules.update	Update an IP firewall rule at the domain level or instance level.
internet-svcs.ip-firewall-rules.delete	Delete an IP firewall rule at the domain level or instance level.
internet-svcs.filters.create	Create filters.
internet-svcs.filters.update	Update filters.
internet-svcs.filters.delete	Delete filters.
internet-svcs.filters-validate-expr.create	Validate a filter expression.
internet-svcs.firewall-rules.create	Create a filter-based firewall rule.
internet-svcs.firewall-rules.update	Update a filter-based firewall rule.
internet-svcs.firewall-rules.delete	Delete a filter-based firewall rule.
internet-svcs.ua-rules.create	Create a user agent blocking rule.
internet-svcs.ua-rules.update	Update a user agent blocking rule.
internet-svcs.ua-rules.delete	Delete a user agent blocking rule.
internet-svcs.domain-lockdown-rules.create	Create a domain lockdown rule.
internet-svcs.domain-lockdown-rules.update	Update a domain lockdown rule.
internet-svcs.domain-lockdown-rules.delete	Delete a domain lockdown rule.
Actions that generate firewall events	

Events for WAF overrides

The following table lists the actions that are related to WAF overrides and generate an event:

Action	Description
internet-svcs.waf-overrides.create	Create a WAF package override.
internet-svcs.waf-overrides.update	Update a WAF package override.
internet-svcs.waf-overrides.delete	Delete a WAF package override.
Actions that generate WAF override events	

Events for WAF rulesets

The following table lists the actions that are related to WAF rulesets and generate an event:

Action	Description
internet-svcs.zone-rulesets.create	Create a zone-level WAF ruleset.
internet-svcs.zone-rulesets.update	Update a zone-level WAF ruleset.
internet-svcs.zone-rulesets.delete	Delete a zone-level WAF ruleset.
internet-svcs.zone-rulesets-rules.create	Create a rule in a zone ruleset.
internet-svcs.zone-rulesets-rules.update	Update a rule in a zone ruleset.
internet-svcs.zone-rulesets-rules.delete	Delete a rule from a zone ruleset.
internet-svcs.zone-rulesets-phases.create	Create a zone ruleset phase entry.
internet-svcs.zone-rulesets-phases.update	Update a zone ruleset phase entry.
internet-svcs.zone-rulesets-phases.delete	Delete a zone ruleset phase entry.
internet-svcs.zone-rulesets-phases-entripoint.update	Update the zone ruleset phase entripoint.
Actions that generate WAF ruleset events	

Events for custom lists

The following table lists the actions that are related to custom lists and generate an event:

Action	Description
internet-svcs.custom-lists.create	Create a custom IP/ASN/hostname list.
internet-svcs.custom-lists.update	Update a custom list.
internet-svcs.custom-lists.delete	Delete a custom list.
internet-svcs.custom-lists-items.create	Add items to a custom list.
internet-svcs.custom-lists-items.update	Update items in a custom list.
internet-svcs.custom-lists-items.delete	Delete items from a custom list.
internet-svcs.custom-lists-operations.create	Perform a bulk operation on a custom list.
Actions that generate custom list events	

Events for managed lists

The following table lists the actions that are related to managed lists and generate an event:

Action	Description
internet-svcs.managed-lists.create	Create a managed IP list.
internet-svcs.managed-lists.update	Update a managed list.
internet-svcs.managed-lists.delete	Delete a managed list.
Actions that generate managed list events	

Events for rate limiting

The following table lists the actions that are related to rate limiting and generate an event:

Action	Description
internet-svcs.rate-limits.create	Create a rate-limiting rule.
internet-svcs.rate-limits.update	Update a rate-limiting rule.
internet-svcs.rate-limits.delete	Delete a rate-limiting rule.
Actions that generate a rate limiting events	

Events for routing

The following table lists the actions that are related to routing and generate an event:

Action	Description
internet-svcs.smart-routing.update	Enable or disable smart routing.
internet-svcs.tiered-caching.update	Enable or disable tiered caching.
Actions that generate routing events	

Events for certificate packs

The following table lists the actions that are related to certificate packs and generate an event:

Action	Description
internet-svcs.certificate-packs.create	Order a dedicated wildcard or custom certificate.
internet-svcs.certificate-packs.delete	Delete a dedicated wildcard or custom certificate.
Actions that generate certificate pack events	

Events for custom certificates

The following table lists the actions that are related to custom certificates and generate an event:

Action	Description
internet-svcs.custom-certificates.create	Upload a custom certificate.

internet-svcs.custom-certificates.update	Update a custom certificate.
internet-svcs.custom-certificates.delete	Delete a custom certificate.
internet-svcs.custom-certificates-prioritize.update	Re-order custom SSL certificate priority.

Actions that generate custom certificate events

Events for origin certificates

The following table lists the actions that are related to origin certificates and generate an event:

Action	Description
internet-svcs.origin-certificates.create	Create an origin certificate.
internet-svcs.origin-certificates.delete	Revoke an origin certificate.

Actions that generate origin certificate events

Events for authenticated origin pull

The following table lists the actions that are related to authenticated origin pull and generate an event:

Action	Description
internet-svcs.origin-tls-client-auth.create	Upload a zone-level client auth certificate.
internet-svcs.origin-tls-client-auth.delete	Delete a zone-level client auth certificate.
internet-svcs.origin-tls-client-auth-settings.update	Toggle authenticated origin Pull for a zone.
internet-svcs.origin-tls-client-auth-hostnames.update	Configure per-hostname Authenticated Origin Pull.
internet-svcs.origin-tls-client-auth-hostname-certificates.create	Upload a per-hostname client auth certificate.
internet-svcs.origin-tls-client-auth-hostname-certificates.delete	Delete a per-hostname client auth certificate.

Actions that generate authenticated origin pull events

Events for edge functions

The following table lists the actions that are related to edge functions and generate an event:

Action	Description
internet-svcs.edge-functions-scripts.create	Create an edge functions script.
internet-svcs.edge-functions-scripts.update	Update a new version of edge functions script.
internet-svcs.edge-functions-scripts.delete	Delete an edge functions script.
internet-svcs.edge-functions-routes.create	Create an edge functions route.
internet-svcs.edge-functions-routes.update	Update an edge functions route.
internet-svcs.edge-functions-routes.delete	Delete an edge functions route.

Actions that generate edge functions events

Events for range applications

The following table lists the actions that are related to range applications and generate an event:

Action	Description
internet-svcs.range-apps.create	Create a range application.
internet-svcs.range-apps.update	Update a range application.
internet-svcs.range-apps.delete	Delete a range application.
Actions that generate range events	

Events for logpush

The following table lists the actions that are related to logpush and generate an event:

Action	Description
internet-svcs.logpush-ownership.create	Initiate Logpush ownership challenge.
internet-svcs.logpush-ownership-validate.create	Validate Logpush ownership challenge.
internet-svcs.logpush-jobs.create	Create a Logpush job.
internet-svcs.logpush-jobs.update	Update a Logpush job.
internet-svcs.logpush-jobs.delete	Delete a Logpush job.
Actions that generate logpush events	

Events for instant logs

The following table lists the actions that are related to instant logs and generate an event:

Action	Description
internet-svcs.instant-logs-jobs.create	Create an instant logs job.
internet-svcs.instant-logs-jobs.update	Update an instant logs job.
internet-svcs.instant-logs-jobs.delete	Delete an instant logs job.
Actions that generate instant logs events	

Events for custom error pages

The following table lists the actions that are related to custom error pages and generate an event:

Action	Description
internet-svcs.custom-pages.create	Create a custom error page.
internet-svcs.custom-pages.update	Update a custom error page.
Actions that generate custom error page events	

Events for settings

The following table lists the actions that are related to configuring settings and generate an event:

Action	Description
internet-svcs.cache-level-setting.update	Change caching level.
internet-svcs.browser-cache-ttl-setting.update	Change browser cache TTL.
internet-svcs.development-mode-setting.update	Enable or disable development mode.
internet-svcs.security-level-setting.update	Change security level.
internet-svcs.ssl-setting.update	Change SSL setting.
internet-svcs.tls-1-2-only-setting.update	Enable or disable TLS 1.2 support.
internet-svcs.waf-setting.update	Enable or disable web application firewall.
internet-svcs.cname-flattening-setting.update	Change CNAME flattening setting.
internet-svcs.always-online-setting.update	Enable or disable serve stale content for the domain.
internet-svcs.sort-query-string-for-cache-setting.update	Enable or disable sorting query arguments when querying content in cache.
internet-svcs.tls-1-3-setting.update	Change TLS 1.3 setting.
internet-svcs.automatic-https-rewrites-setting.update	Enable or disable automatic HTTPS rewrites.
internet-svcs.opportunistic-encryption-setting.update	Enable or disable opportunistic encryption.
internet-svcs.browser-check-setting.update	Enable or disable browser integrity check.
internet-svcs.challenge-ttl-setting.update	Update challenge TTL.
internet-svcs.always-use-https-setting.update	Enable or disable Always Use HTTPS .
internet-svcs.true-client-ip-header-setting.update	Enable or disable True client IP header.
internet-svcs.image-size-optimization-setting.update	Enable or disable image size optimization.
internet-svcs.script-load-optimization-setting.update	Enable or disable script load optimization.
internet-svcs.image-load-optimization-setting.update	Enable or disable image load optimization.
internet-svcs.minify-setting.update	Enable or disable minification for HTML, CSS, or JavaScript files.
internet-svcs.min-tls-version-setting.update	Change minimum TLS version.
internet-svcs.ip-geolocation-setting.update	Enable or disable IP geolocation header.

internet-svcs.http2-setting.update	Enable or disable HTTP2 for the domain.
internet-svcs.max-upload-setting.update	Change the amount of data that visitors can upload to the website in a single request.
internet-svcs.origin-error-page-pass-thru-setting.update	Enable or disable the proxy of 502 and 504 error pages that are returned from origin server.
internet-svcs.bot-management.update	Change Bot Management settings.
internet-svcs.universal-ssl-setting.update	Toggle Universal SSL.
internet-svcs.logs-retention.update	Update log retention settings.

Actions that generate settings events

Events for alerting

The following table lists the actions that are related to alert notifications and generate an event:

Action	Description
internet-svcs.alerting-policies.create	Create an alert notification policy.
internet-svcs.alerting-policies.update	Update an alert notification policy.
internet-svcs.alerting-policies.delete	Delete an alert notification policy.
internet-svcs.alerting-webhooks.create	Create an alert webhook destination.
internet-svcs.alerting-webhooks.update	Update an alert webhook destination.
internet-svcs.alerting-webhooks.delete	Delete an alert webhook destination.

Actions that generate alerting events

Additional information

When you monitor IBM Cloud Activity Tracker events that are generated by the IBM Cloud Internet Services, and you identify an API request for which you need additional information, check the `requestData` field in the event.

Open a Support case and include the value of the field `requestId` that is available in `requestData`.

Analyzing CIS activity tracking events

For more information about calling auditing events with the API, see the Auditing section of each method in the [CIS API documentation](#).

IAM and Activity Tracker actions by API

List of IAM actions and Activity Tracker actions by API method.

Overview

Action	Method	IAM ACTION	AT ACTION
Get an overview	GET /v2/{crn}/zones/{domain_id}/overview	internet-svcs.zones.read	internet-svcs.overview.read

Overview

DNS Domains

Action	Method	IAM ACTION	AT ACTION
List the domains or get domain details	GET /v1/{crn}/zones	internet-svcs.zones.read	internet-svcs.zones.read
Create a domain	POST /v1/{crn}/zones	internet-svcs.zones.manage	internet-svcs.zones.create
Update a domain	PATCH /v1/{crn}/zones/{domain_id}	internet-svcs.reliability.update	internet-svcs.zones.update
Delete a domain	DELETE /v1/{crn}/zones/{domain_id}	internet-svcs.reliability.manage	internet-svcs.zones.delete
Run an activation check on a domain	PUT /v1/{crn}/zones/{domain_id}/activation_check	internet-svcs.reliability.update	internet-svcs.zones-activation-check.update
Get the DNSSEC configuration	GET /v1/{crn}/zones/{domain_id}/dnssec	internet-svcs.reliability.read	internet-svcs.dnssec.read
Enable or disable DNSSEC	PATCH /v1/{crn}/zones/{domain_id}/dnssec	internet-svcs.reliability.update	internet-svcs.dnssec.update

DNS domains

DNS Records

Action	Method	IAM ACTION	AT ACTION
Get the DNS records	GET /v1/{crn}/zones/{domain_id}/dns_records	internet-svcs.reliability.read	internet-svcs.dns-records.read
Create a DNS record	POST /v1/{crn}/zones/{domain_id}/dns_records	internet-svcs.reliability.manage	internet-svcs.dns-records.create
Update a DNS record	PUT /v1/{crn}/zones/{domain_id}/dns_records/{record_id}	internet-svcs.reliability.update	internet-svcs.dns-records.update
Delete a DNS record	DELETE /v1/{crn}/zones/{domain_id}/dns_records/{record_id}	internet-svcs.reliability.manage	internet-svcs.dns-records.delete
Import the DNS records from zone file	GET /v1/{crn}/zones/{domain_id}/dns_records_bulk	internet-svcs.reliability.read	internet-svcs.dns-records-bulk.read
Export the DNS records to a zone file	POST /v1/{crn}/zones/{domain_id}/dns_records_bulk	internet-svcs.reliability.manage	internet-svcs.dns-records-bulk.create

DNS records

GLB

Action	Method	IAM ACTION	AT ACTION
--------	--------	------------	-----------

Get the global load balancers	GET /v1/{crn}/zones/{domain_id}/load_balancers	internet-svcs.reliability.read	internet-svcs.load-balancers.read
Create a global load balancer	POST /v1/{crn}/zones/{domain_id}/load_balancers	internet-svcs.reliability.manage	internet-svcs.load-balancers.create
Update a global load balancer	PUT /v1/{crn}/zones/{domain_id}/load_balancers/{glb_id}	internet-svcs.reliability.update	internet-svcs.load-balancers.update
Delete a global load balancer	DELETE /v1/{crn}/zones/{domain_id}/load_balancers/{glb_id}	internet-svcs.reliability.manage	internet-svcs.load-balancers.delete
Get the health monitors	GET /v1/{crn}/load_balancers/monitors	internet-svcs.zones.read	internet-svcs.load-balancer-monitors.read
Create a health monitor	POST /v1/{crn}/load_balancers/monitors	internet-svcs.zones.manage	internet-svcs.load-balancer-monitors.create
Update a health monitor	PUT /v1/{crn}/load_balancers/monitors/{monitor_id}	internet-svcs.zones.update	internet-svcs.load-balancer-monitors.update
Delete a health monitor	DELETE /v1/{crn}/load_balancers/monitors/{monitor_id}	internet-svcs.zones.manage	internet-svcs.load-balancer-monitors.delete
Get the load balancer pools	GET /v1/{crn}/load_balancers/pools	internet-svcs.zones.read	internet-svcs.load-balancer-pools.read
Create a load balancer pool	POST /v1/{crn}/load_balancers/pools	internet-svcs.zones.manage	internet-svcs.load-balancer-pools.create
Update a load balancer pool	PUT /v1/{crn}/load_balancers/pools/{pool_id}	internet-svcs.zones.update	internet-svcs.load-balancer-pools.update
Delete a load balancer pool	DELETE /v1/{crn}/load_balancers/pools/{pool_id}	internet-svcs.zones.manage	internet-svcs.load-balancer-pools.delete
Get the load balancer events	GET /v1/{crn}/load_balancers/events	internet-svcs.zones.read	internet-svcs.load-balancer-events.read

GLB

Metrics

Action	Method	IAM ACTION	AT ACTION
Get the dashboard analytics	GET /v1/{crn}/zones/{domain_id}/analytics/dashboard	internet-svcs.reliability.read	internet-svcs.dashboard-analytics.read
Get the HTTP requests analytics	GET /v1/{crn}/zones/{domain_id}/analytics/http_requests	internet-svcs.reliability.read	internet-svcs.http-requests-analytics.read
Get the HTTP requests analytics by colocations	GET /v1/{crn}/zones/{domain_id}/analytics/colos	internet-svcs.reliability.read	internet-svcs.colos-analytics.read

Get the DNS analytics	GET <code>/v1/{crn}/zones/{domain_id}/dns_analytics/report</code>	internet-svcs.reliability.read	internet-svcs.dns-analytics.read
-----------------------	--	--------------------------------	----------------------------------

Metrics

WAF

Action	Method	IAM ACTION	AT ACTION
Deprecat ed Get the WAF packages	GET <code>/v1/{crn}/zones/{domain_id}/firewall/waf/packages</code>	internet-svcs.security.read	internet-svcs.waf-packages.read
Change the sensitivity and action mode of a WAF package	PATCH <code>/v1/{crn}/zones/{domain_id}/firewall/waf/packages/{package_id}</code>	internet-svcs.security.update	internet-svcs.waf-packages.update
Deprecat ed Get the WAF groups	GET <code>/v1/{crn}/zones/{domain_id}/firewall/waf/packages/{package_id}/groups</code>	internet-svcs.security.read	internet-svcs.waf-groups.read
Enable or disable a WAF group	PATCH <code>/v1/{crn}/zones/{domain_id}/firewall/waf/packages/{package_id}/groups/{group_id}</code>	internet-svcs.security.update	internet-svcs.waf-groups.update
Deprecat ed Get the WAF rules	GET <code>/v1/{crn}/zones/{domain_id}/firewall/waf/packages/{package_id}/rules</code>	internet-svcs.security.read	internet-svcs.waf-rules.read
Change the action mode of a WAF rule	PATCH <code>/v1/{crn}/zones/{domain_id}/firewall/waf/packages/{package_id}/rules/{rule_id}</code>	internet-svcs.security.update	internet-svcs.waf-rules.update

WAF

IP Firewall

Action	Method	IAM ACTION	AT ACTION
Get the IP firewall rules	GET <code>/v1/{crn}/zones/{domain_id}/firewall/access_rules/rules</code>	internet-svcs.security.read	internet-svcs.ip-firewall-rules.read
Create an IP firewall rule	POST <code>/v1/{crn}/zones/{domain_id}/firewall/access_rules/rules</code>	internet-svcs.security.manage	internet-svcs.ip-firewall-rules.create

Update an IP firewall rule	PATCH <code>/v1/{crn}/zones/{domain_id}/firewall/access_rules/rules/{rule_id}</code>	internet-svcs.security.update	internet-svcs.ip-firewall-rules.update
Delete an IP firewall rule	DELETE <code>/v1/{crn}/zones/{domain_id}/firewall/access_rules/rules/{rule_id}</code>	internet-svcs.security.manage	internet-svcs.ip-firewall-rules.delete
Get the user-agent blocking rules	GET <code>/v1/{crn}/zones/{domain_id}/firewall/ua_rules</code>	internet-svcs.security.read	internet-svcs.ua-rules.read
Create a user-agent blocking rule	POST <code>/v1/{crn}/zones/{domain_id}/firewall/ua_rules</code>	internet-svcs.security.manage	internet-svcs.ua-rules.create
Update a user-agent blocking rule	PUT <code>/v1/{crn}/zones/{domain_id}/firewall/ua_rules/{rule_id}</code>	internet-svcs.security.update	internet-svcs.ua-rules.update
Delete a user-agent blocking rule	DELETE <code>/v1/{crn}/zones/{domain_id}/firewall/ua_rules/{rule_id}</code>	internet-svcs.security.manage	internet-svcs.ua-rules.delete
Get the domain lockdown rules	GET <code>/v1/{crn}/zones/{domain_id}/firewall/lockdowns</code>	internet-svcs.security.read	internet-svcs.domain-lockdown-rules.read
Create a domain lockdown rule	POST <code>/v1/{crn}/zones/{domain_id}/firewall/lockdowns</code>	internet-svcs.security.manage	internet-svcs.domain-lockdown-rules.create
Update a domain lockdown rule	PUT <code>/v1/{crn}/zones/{domain_id}/firewall/lockdowns/{rule_id}</code>	internet-svcs.security.update	internet-svcs.domain-lockdown-rules.update
Delete a domain lockdown rule	DELETE <code>/v1/{crn}/zones/{domain_id}/firewall/lockdowns/{rule_id}</code>	internet-svcs.security.manage	internet-svcs.domain-lockdown-rules.delete

IP firewall

Firewall Rules

Action	Method	IAM ACTION	AT ACTION
Get the firewall rules	GET <code>/v1/{crn}/zones/{domain_id}/firewall/rules</code>	internet-svcs.security.read	internet-svcs.firewall-rules.read
Create a firewall rule	POST <code>/v1/{crn}/zones/{domain_id}/firewall/rules</code>	internet-svcs.security.manage	internet-svcs.firewall-rules.create
Update a firewall rule	PATCH <code>/v1/{crn}/zones/{domain_id}/firewall/rules/{rule_id}</code>	internet-svcs.security.update	internet-svcs.firewall-rules.update
Delete a firewall rule	DELETE <code>/v1/{crn}/zones/{domain_id}/firewall/rules/{rule_id}</code>	internet-svcs.security.manage	internet-svcs.firewall-rules.delete

Get the filters	GET /v1/{crn}/zones/{domain_id}/filters	internet-svcs.security.read	internet-svcs.filters.read
Create a filter	POST /v1/{crn}/zones/{domain_id}/filters	internet-svcs.security.manage	internet-svcs.filters.create
Update a filter	PATCH /v1/{crn}/zones/{domain_id}/filters/{filter_id}	internet-svcs.security.update	internet-svcs.filters.update
Delete a filter	DELETE /v1/{crn}/zones/{domain_id}/filters/{filter_id}	internet-svcs.security.manage	internet-svcs.filters.delete
Validate the expression of a filter.	POST /v1/{crn}/zones/{domain_id}/filters/validate-expr	internet-svcs.security.manage	internet-svcs.filters-validate-expr.create

Firewall rules

Security Events

Action	Method	IAM ACTION	AT ACTION
Get the security events	GET /v1/{crn}/zones/{domain_id}/security/events	internet-svcs.security.read	internet-svcs.security-events.read
Get the firewall events (Deprecated)	GET /v1/{crn}/zones/{domain_id}/analytics/firewall_events [DEPRECATED]	internet-svcs.security.read	internet-svcs.firewall-events-analytics.read

Security events

Rate Limiting

Action	Method	IAM ACTION	AT ACTION
Get the rate-limiting rules	GET /v1/{crn}/zones/{domain_id}/rate_limits	internet-svcs.security.read	internet-svcs.rate-limits.read
Create a rate-limiting rule	POST /v1/{crn}/zones/{domain_id}/rate_limits	internet-svcs.security.manage	internet-svcs.rate-limits.create
Update a rate-limiting rule	PUT /v1/{crn}/zones/{domain_id}/rate_limits/{ratelimit_id}	internet-svcs.security.update	internet-svcs.rate-limits.update
Delete a rate-limiting rule	DELETE /v1/{crn}/zones/{domain_id}/rate_limits/{ratelimit_id}	internet-svcs.security.manage	internet-svcs.rate-limits.delete
Get the rate limiting analytics	GET /v1/{crn}/zones/{domain_id}/rate_limit_analytics	internet-svcs.security.read	internet-svcs.rate-limit-analytics.read

Rate limiting

Caching

Action	Method	IAM ACTION	AT ACTION
Purge all cached assets of a domain from edge server	PUT /v1/{crn}/zones/{domain_id}/purge_cache/purge_all	internet-svcs.performance.update	internet-svcs.purge-cache-all.update

Purge the cached assets by URLs from edge server	PUT /v1/{crn}/zones/{domain_id}/purge_cache/purge_by_urls	internet-svcs.performance.update	internet-svcs.purge-cache-by-urls.update
Purge the cached assets by cache tags from edge server	PUT /v1/{crn}/zones/{domain_id}/purge_cache/purge_by_cache_tags	internet-svcs.performance.update	internet-svcs.purge-cache-by-cache-tags.update
Purge the cached assets by hostnames from edge server	PUT /v1/{crn}/zones/{domain_id}/purge_cache/purge_by_hosts	internet-svcs.performance.update	internet-svcs.purge-cache-by-hosts.update

Caching

Routing

Action	Method	IAM ACTION	AT ACTION
Get the smart routing configurations	GET /v1/{crn}/zones/{domain_id}/routing/smart_routing	internet-svcs.performance.read	internet-svcs.smart-routing.read
Enable or disable smart routing	PATCH /v1/{crn}/zones/{domain_id}/routing/smart_routing	internet-svcs.performance.update	internet-svcs.smart-routing.update
Get the tiered caching configurations	GET /v1/{crn}/zones/{domain_id}/routing/tiered_caching	internet-svcs.performance.read	internet-svcs.tiered-caching.read
Enable or disable tiered caching	PATCH /v1/{crn}/zones/{domain_id}/routing/tiered_caching	internet-svcs.performance.update	internet-svcs.tiered-caching.update

Routing

Page Rules

Action	Method	IAM ACTION	AT ACTION
Get the page rules	GET /v1/{crn}/zones/{domain_id}/pagerules	internet-svcs.performance.read	internet-svcs.pagerules.read
Create a page rule	POST /v1/{crn}/zones/{domain_id}/pagerules	internet-svcs.performance.manage	internet-svcs.pagerules.create
Update a page rule	PUT /v1/{crn}/zones/{domain_id}/pagerules/{rule_id}	internet-svcs.performance.update	internet-svcs.pagerules.update
Delete a page rule	DELETE /v1/{crn}/zones/{domain_id}/pagerules/{rule_id}	internet-svcs.performance.manage	internet-svcs.pagerules.delete

Page rules

TLS

Action	Method	IAM ACTION	AT ACTION
--------	--------	------------	-----------

Get the universal SSL setting	GET /v1/{crn}/zones/{domain_id}/ssl/universal/settings	internet-svcs.security.read	internet-svcs.universal-ssl-setting.read
Update a universal SSL setting	PATCH /v1/{crn}/zones/{domain_id}/ssl/universal/settings	internet-svcs.security.update	internet-svcs.universal-ssl-setting.update
Get the edge certificates ordered from CIS	GET /v1/{crn}/zones/{domain_id}/ssl/certificate_packs	internet-svcs.security.read	internet-svcs.certificate-packs.read
Order an edge certificate	POST /v1/{crn}/zones/{domain_id}/ssl/certificate_packs	internet-svcs.security.manage	internet-svcs.certificate-packs.create
Delete an edge certificate	DELETE /v1/{crn}/zones/{domain_id}/ssl/certificate_packs/{cert_id}	internet-svcs.security.manage	internet-svcs.certificate-packs.delete
Get the uploaded certificates	GET /v1/{crn}/zones/{domain_id}/custom_certificates	internet-svcs.security.read	internet-svcs.custom-certificates.read
Upload a certificate to CIS edge	POST /v1/{crn}/zones/{domain_id}/custom_certificates	internet-svcs.security.manage	internet-svcs.custom-certificates.create
Update the certificate that is uploaded to CIS edge	PATCH /v1/{crn}/zones/{domain_id}/custom_certificates/{cert_id}	internet-svcs.security.update	internet-svcs.custom-certificates.update
Delete a certificate that was uploaded to CIS edge	DELETE /v1/{crn}/zones/{domain_id}/custom_certificates/{cert_id}	internet-svcs.security.manage	internet-svcs.custom-certificates.delete
Get an origin certificate issued by CIS	GET /v1/{crn}/zones/{domain_id}/origin_certificates	internet-svcs.security.read	internet-svcs.origin-certificates.read
Create an origin certificate issued by CIS	POST /v1/{crn}/zones/{domain_id}/origin_certificates	internet-svcs.security.manage	internet-svcs.origin-certificates.create
Revoke an origin certificate issued by CIS	DELETE /v1/{crn}/zones/{domain_id}/origin_certificates/{cert_id}	internet-svcs.security.manage	internet-svcs.origin-certificates.delete

TLS

Edge Functions

Action	Method	IAM ACTION	AT ACTION
Get the edge function scripts	GET /v1/{crn}/workers/scripts	internet-svcs.performance.read	internet-svcs.edge-functions-scripts.read
Create an edge function script	POST /v1/{crn}/workers/scripts	internet-svcs.performance.manage	internet-svcs.edge-functions-scripts.create

Update an edge function script	PUT /v1/{crn}/workers/scripts/{script_name}	internet-svcs.performance.update	internet-svcs.edge-functions-scripts.update
Delete an edge function script	DELETE /v1/{crn}/workers/scripts/{script_name}	internet-svcs.performance.manage	internet-svcs.edge-functions-scripts.delete
Get the edge function routes	GET /v1/{crn}/zones/{domain_id}/workers/routes	internet-svcs.performance.read	internet-svcs.edge-functions-routes.read
Create an edge function route	POST /v1/{crn}/zones/{domain_id}/workers/routes	internet-svcs.performance.manage	internet-svcs.edge-functions-routes.create
Update an edge function route	PUT /v1/{crn}/zones/{domain_id}/workers/routes/{route_id}	internet-svcs.performance.update	internet-svcs.edge-functions-routes.update
Delete an edge function route	DELETE /v1/{crn}/zones/{domain_id}/workers/routes/{route_id}	internet-svcs.performance.manage	internet-svcs.edge-functions-routes.delete

Edge functions

Range

Action	Method	IAM ACTION	AT ACTION
Get the range apps	GET /v1/{crn}/zones/{domain_id}/range/apps	internet-svcs.security.read	internet-svcs.range-apps.read
Create a range app	POST /v1/{crn}/zones/{domain_id}/range/apps	internet-svcs.security.manage	internet-svcs.range-apps.create
Update a range app	PUT /v1/{crn}/zones/{domain_id}/range/apps/{app_id}	internet-svcs.security.update	internet-svcs.range-apps.update
Delete a range app	DELETE /v1/{crn}/zones/{domain_id}/range/apps/{app_id}	internet-svcs.security.manage	internet-svcs.range-apps.delete
Get the analytics of range apps	GET /v1/{crn}/zones/{domain_id}/range/analytics/events/summary GET /v1/{crn}/zones/{domain_id}/range/analytics/events/bytime	internet-svcs.security.read	internet-svcs.range-analytics.read

Range

Logpush

Action	Method	IAM ACTION	AT ACTION
--------	--------	------------	-----------

Get the Logpush fields	GET /v1/{crn}/zones/{domain_id}/logpush/datasets/{dataset}/fields	internet-svcs.zones.read	internet-svcs.logpush-fields.read
Get the Logpush jobs	GET /v1/{crn}/zones/{domain_id}/logpush/jobs	internet-svcs.zones.read	internet-svcs.logpush-jobs.read
Create a Logpush job	POST /v1/{crn}/zones/{domain_id}/logpush/jobs	internet-svcs.zones.manage	internet-svcs.logpush-jobs.create
Update a Logpush job	PUT /v1/{crn}/zones/{domain_id}/logpush/jobs/{job_id}	internet-svcs.zones.update	internet-svcs.logpush-jobs.update
Delete a Logpush job	DELETE /v1/{crn}/zones/{domain_id}/logpush/jobs/{job_id}	internet-svcs.zones.manage	internet-svcs.logpush-jobs.delete
Initiate the Logpush ownership challenge	POST /v1/{crn}/zones/{domain_id}/logpush/ownership	internet-svcs.zones.manage	internet-svcs.logpush-ownership.create
Validate the Logpush ownership challenge token	POST /v1/{crn}/zones/{domain_id}/logpush/ownership/validate	internet-svcs.zones.manage	internet-svcs.logpush-ownership-validate.create

Logpush

Custom Pages

Action	Method	IAM ACTION	AT ACTION
Get the custom error pages	GET /v1/{crn}/zones/{domain_id}/custom_pages	internet-svcs.zones.read	internet-svcs.custom-pages.read
Update the custom error page	PUT /v1/{crn}/zones/{domain_id}/custom_pages/{page_id}	internet-svcs.zones.update	internet-svcs.custom-pages.update

Custom pages

Mutual TLS

Action	Method	IAM ACTION	AT ACTION
Get the certificates uploaded for mTLS	GET /v1/{crn}/zones/{domain_id}/access/certificates	internet-svcs.security.read	internet-svcs.access-certificates.read
Upload a certificate for mTLS	POST /v1/{crn}/zones/{domain_id}/access/certificates	internet-svcs.security.manage	internet-svcs.access-certificates.create
Update a certificate for mTLS	PUT /v1/{crn}/zones/{domain_id}/access/certificates/{cert_id}	internet-svcs.security.update	internet-svcs.access-certificates.update
Delete a certificate for mTLS	DELETE /v1/{crn}/zones/{domain_id}/access/certificates/{cert_id}	internet-svcs.security.manage	internet-svcs.access-certificates.delete

Get the mTLS applications	GET /v1/{crn}/zones/{domain_id}/access/apps	internet-svcs.security.read	internet-svcs.access-apps.read
Create a mTLS application	POST /v1/{crn}/zones/{domain_id}/access/apps	internet-svcs.security.manage	internet-svcs.access-apps.create
Update a mTLS application	PUT /v1/{crn}/zones/{domain_id}/access/apps/{app_id}	internet-svcs.security.update	internet-svcs.access-apps.update
Delete a mTLS application	DELETE /v1/{crn}/zones/{domain_id}/access/apps/{app_id}	internet-svcs.security.manage	internet-svcs.access-apps.delete
Get the mTLS policies	GET /v1/{crn}/zones/{domain_id}/access/apps/{app_id}/policies	internet-svcs.security.read	internet-svcs.access-policies.read
Create a mTLS policy	POST /v1/{crn}/zones/{domain_id}/access/apps/{app_id}/policies	internet-svcs.security.manage	internet-svcs.access-policies.create
Update a mTLS policy	PUT /v1/{crn}/zones/{domain_id}/access/apps/{app_id}/policies/{policy_id}	internet-svcs.security.update	internet-svcs.access-policies.update
Delete a mTLS policy	DELETE /v1/{crn}/zones/{domain_id}/access/apps/{app_id}	internet-svcs.security.manage	internet-svcs.access-policies.delete

Mutual TLS

Origin TLS Client Authentication

Action	Method	IAM ACTION	AT ACTION
List the certificates that are used for origin TLS client authentication at the domain level.	GET /v1/{crn}/zones/{domain_id}/origin_tls_client_auth	internet-svcs.security.read	internet-svcs.origin-tls-client-auth.read
Create a certificate that is used for origin TLS client authentication at the domain level.	POST /v1/{crn}/zones/{domain_id}/origin_tls_client_auth/{cert_id}	internet-svcs.security.manage	internet-svcs.origin-tls-client-auth.create

Delete a certificate that is used for origin TLS client authentication that is used at the domain level.	DELETE /v1/{crn}/zones/{domain_id}/origin_tls_client_auth/{cert_id}	internet-svcs.security.manage	internet-svcs.origin-tls-client-auth.delete
Get the origin TLS client authentication settings that are used at the domain level.	` GET /v1/{crn}/zones/{domain_id}/origin_tls_client_auth/settings	internet-svcs.security.read	internet-svcs.origin-tls-client-auth-settings.read`
Update the origin TLS client authentication settings that are used at the domain level.	PUT /v1/{crn}/zones/{domain_id}/origin_tls_client_auth/settings	internet-svcs.security.update	internet-svcs.origin-tls-client-auth-settings.update
Get the origin TLS client authentication settings that are used for the hostname.	GET /v1/{crn}/zones/{domain_id}/origin_tls_client_auth/hostnames/{hostname}	internet-svcs.security.read	internet-svcs.origin-tls-client-auth-hostnames.read
Update the origin TLS client authentication settings for a hostname.	PUT /v1/{crn}/zones/{domain_id}/origin_tls_client_auth/hostnames	internet-svcs.security.update	internet-svcs.origin-tls-client-auth-hostnames.update
Get the origin TLS client authentication certificates used at the hostname level.	GET /v1/{crn}/zones/{domain_id}/origin_tls_client_auth/hostnames/certificates/{cert_id}	internet-svcs.security.read	internet-svcs.origin-tls-client-auth-hostname-certificates.read
Create an origin TLS client authentication certificate used at the hostname level.	POST /v1/{crn}/zones/{domain_id}/origin_tls_client_auth/hostnames/certificates	internet-svcs.security.manage	internet-svcs.origin-tls-client-auth-hostname-certificates.create
Delete an origin TLS client authentication certificate used at the hostname level.	DELETE /v1/{crn}/zones/{domain_id}/origin_tls_client_auth/hostnames/certificates/{cert_id}	internet-svcs.security.manage	internet-svcs.origin-tls-client-auth-hostname-certificates.delete

Domain settings

Action	Method	IAM ACTION	AT ACTION
Get the CNAME flattening settings.	GET /v1/{crn}/zones/{domain_id}/setting/cname_flattening	internet-svcs.reliability.read	internet-svcs.cname-flattening-setting.read
Update the CNAME flattening settings.	PUT /v1/{crn}/zones/{domain_id}/setting/cname_flattening	internet-svcs.reliability.update	internet-svcs.cname-flattening-setting.update
Get the cache level settings.	GET /v1/{crn}/zones/{domain_id}/setting/cache_level	internet-svcs.performance.read	internet-svcs.cache-level-setting.read
Update the cache level settings.	PUT /v1/{crn}/zones/{domain_id}/setting/cache_level	internet-svcs.performance.update	internet-svcs.cache-level-setting.update
Get the browser cache TTL settings.	GET /v1/{crn}/zones/{domain_id}/setting/browser_cache_ttl	internet-svcs.performance.read	internet-svcs.browser-cache-ttl-setting.read
Update the browser cache TTL settings.	PUT /v1/{crn}/zones/{domain_id}/setting/browser_cache_ttl	internet-svcs.performance.update	internet-svcs.browser-cache-ttl-setting.update
Get the always online (serve stale content) settings.	GET /v1/{crn}/zones/{domain_id}/setting/always_online	internet-svcs.performance.read	internet-svcs.always-online-setting.read
Update the always online (serve stale content) settings.	PUT /v1/{crn}/zones/{domain_id}/setting/always_online	internet-svcs.performance.update	internet-svcs.always-online-setting.update
Get the development mode settings.	GET /v1/{crn}/zones/{domain_id}/setting/development_mode	internet-svcs.performance.read	internet-svcs.development-mode-setting.read
Update the development mode settings.	PUT /v1/{crn}/zones/{domain_id}/setting/development_mode	internet-svcs.performance.update	internet-svcs.development-mode-setting.update
Get the sort query string for cache settings.	GET /v1/{crn}/zones/{domain_id}/setting/sort_query_string_for_cache	internet-svcs.performance.read	internet-svcs.sort-query-string-for-cache-setting.read

Update the sort query string for cache settings.	PUT /v1/{crn}/zones/{domain_id}/setting/sort_query_string_for_cache	internet-svcs.performance.update	internet-svcs.sort-query-string-for-cache-setting.update
Get the SSL settings.	GET /v1/{crn}/zones/{domain_id}/setting/ssl	internet-svcs.security.read	internet-svcs.ssl-setting.read
Update the SSL settings.	PUT /v1/{crn}/zones/{domain_id}/setting/ssl	internet-svcs.security.update	internet-svcs.ssl-setting.update
Get the security level settings.	GET /v1/{crn}/zones/{domain_id}/setting/security_level	internet-svcs.security.read	internet-svcs.security-level-setting.read
Update the security level settings.	PUT /v1/{crn}/zones/{domain_id}/setting/security_level	internet-svcs.security.update	internet-svcs.security-level-setting.update
Get the TLS 1.2 settings.	GET /v1/{crn}/zones/{domain_id}/setting/tls_1_2_only	internet-svcs.security.read	internet-svcs.tls-1-2-only-setting.read
Update the TLS 1.2 settings.	PUT /v1/{crn}/zones/{domain_id}/setting/tls_1_2_only	internet-svcs.security.update	internet-svcs.tls-1-2-only-setting.update
Get the TLS 1.3 only settings.	GET /v1/{crn}/zones/{domain_id}/setting/tls_1_3	internet-svcs.security.read	internet-svcs.tls-1-3-setting.read
Update the TLS 1.3 only settings.	PUT /v1/{crn}/zones/{domain_id}/setting/tls_1_3	internet-svcs.security.update	internet-svcs.tls-1-3-setting.update
Get the automatic HTTPS rewrites settings.	GET /v1/{crn}/zones/{domain_id}/setting/automatic_https_rewrites	internet-svcs.security.read	internet-svcs.automatic-https-rewrites-setting.read
Update the automatic HTTPS rewrites settings.	PUT /v1/{crn}/zones/{domain_id}/setting/automatic_https_rewrites	internet-svcs.security.update	internet-svcs.automatic-https-rewrites-setting.update
Get the WAF settings.	GET /v1/{crn}/zones/{domain_id}/setting/waf	internet-svcs.security.read	internet-svcs.waf-setting.read
Update the WAF settings.	PUT /v1/{crn}/zones/{domain_id}/setting/waf	internet-svcs.security.update	internet-svcs.waf-setting.update
Get the browser integrity check settings.	GET /v1/{crn}/zones/{domain_id}/setting/browser_check	internet-svcs.security.read	internet-svcs.browser-check-setting.read

Update the browser integrity check settings.	PUT /v1/{crn}/zones/{domain_id}/setting/browser_check	internet-svcs.security.update	internet-svcs.browser-check-setting.update
Get the opportunistic encryption settings.	GET /v1/{crn}/zones/{domain_id}/setting/opportunistic_encryption	internet-svcs.security.read	internet-svcs.opportunistic-encryption-setting.read
Update the opportunistic encryption settings.	PUT /v1/{crn}/zones/{domain_id}/setting/opportunistic_encryption	internet-svcs.security.update	internet-svcs.opportunistic-encryption-setting.update
Get the challenge TTL settings.	GET /v1/{crn}/zones/{domain_id}/setting/challenge_ttl	internet-svcs.security.read	internet-svcs.challenge-ttl-setting.read
Update the challenge TTL settings.	PUT /v1/{crn}/zones/{domain_id}/setting/challenge_ttl	internet-svcs.security.update	internet-svcs.challenge-ttl-setting.update
Get the always use HTTPS settings.	GET /v1/{crn}/zones/{domain_id}/setting/always_use_https	internet-svcs.security.read	internet-svcs.always-use-https-setting.read
Update the always use HTTPS settings.	PUT /v1/{crn}/zones/{domain_id}/setting/always_use_https	internet-svcs.security.update	internet-svcs.always-use-https-setting.update
Get the true client IP header settings.	GET /v1/{crn}/zones/{domain_id}/setting/true_client_ip_header	internet-svcs.zones.read	internet-svcs.true-client-ip-header-setting.read
Update the true client IP header settings.	PUT /v1/{crn}/zones/{domain_id}/setting/true_client_ip_header	internet-svcs.zones.update	internet-svcs.true-client-ip-header-setting.update
Get the image size optimization settings.	GET /v1/{crn}/zones/{domain_id}/setting/image_size_optimization	internet-svcs.performance.read	internet-svcs.image-size-optimization-setting.read
Update the image size optimization settings.	PUT /v1/{crn}/zones/{domain_id}/setting/image_size_optimization	internet-svcs.performance.update	internet-svcs.image-size-optimization-setting.update
Get the script load optimization settings.	GET /v1/{crn}/zones/{domain_id}/setting/script_load_optimization	internet-svcs.performance.read	internet-svcs.script-load-optimization-setting.read
Update the script load optimization settings.	PUT /v1/{crn}/zones/{domain_id}/setting/script_load_optimization	internet-svcs.performance.update	internet-svcs.script-load-optimization-setting.update

Get the image load optimization settings.	GET /v1/{crn}/zones/{domain_id}/setting/image_load_optimization	internet-svcs.performance.read	internet-svcs.image-load-optimization-setting.read
Update the image load optimization settings.	PUT /v1/{crn}/zones/{domain_id}/setting/image_load_optimization	internet-svcs.performance.update	internet-svcs.image-load-optimization-setting.update
Get the minification settings.	GET /v1/{crn}/zones/{domain_id}/setting/minify	internet-svcs.performance.read	internet-svcs.minify-setting.read
Update the minification settings.	PUT /v1/{crn}/zones/{domain_id}/setting/minify	internet-svcs.performance.update	internet-svcs.minify-setting.update
Get the minimum TLS version settings.	GET /v1/{crn}/zones/{domain_id}/setting/min_tls_version	internet-svcs.security.read	internet-svcs.min-tls-version-setting.read
Update the minimum TLS version settings.	PUT /v1/{crn}/zones/{domain_id}/setting/min_tls_version	internet-svcs.security.update	internet-svcs.min-tls-version-setting.update
Get the IP geolocation settings.	GET /v1/{crn}/zones/{domain_id}/setting/ip_geolocation	internet-svcs.zones.read	internet-svcs.ip-geolocation-setting.read
Update the IP geolocation settings.	PUT /v1/{crn}/zones/{domain_id}/setting/ip_geolocation	internet-svcs.zones.update	internet-svcs.ip-geolocation-setting.update
Get the server-side exclude settings.	GET /v1/{crn}/zones/{domain_id}/setting/server_side_exclude	internet-svcs.security.read	internet-svcs.server-side-exclude-setting.read
Update the server-side exclude settings.	PUT /v1/{crn}/zones/{domain_id}/setting/server_side_exclude	internet-svcs.security.update	internet-svcs.server-side-exclude-setting.update
Get the security header settings.	GET /v1/{crn}/zones/{domain_id}/setting/security_header	internet-svcs.security.read	internet-svcs.security-header-setting.read
Update the security header settings.	PUT /v1/{crn}/zones/{domain_id}/setting/security_header	internet-svcs.security.update	internet-svcs.security-header-setting.update
Get the mobile redirect settings.	GET /v1/{crn}/zones/{domain_id}/setting/mobile_redirect	internet-svcs.performance.read	internet-svcs.mobile-redirect-setting.read

Update the mobile redirect settings.	PUT /v1/{crn}/zones/{domain_id}/setting/mobile_redirect	internet-svcs.performance.update	internet-svcs.mobile-redirect-setting.update
Get the prefetch / preload settings.	GET /v1/{crn}/zones/{domain_id}/setting/prefetch_preload	internet-svcs.performance.read	internet-svcs.prefetch-preload-setting.read
Update the prefetch / preload settings.	PUT /v1/{crn}/zones/{domain_id}/setting/prefetch_preload	internet-svcs.performance.update	internet-svcs.prefetch-preload-setting.update
Get the HTTP2 settings.	GET /v1/{crn}/zones/{domain_id}/setting/http2	internet-svcs.security.read	internet-svcs.http2-setting.read
Update the HTTP2 settings.	PUT /v1/{crn}/zones/{domain_id}/setting/http2	internet-svcs.security.update	internet-svcs.http2-setting.update
Get the IPv6 settings.	GET /v1/{crn}/zones/{domain_id}/setting/ipv6	internet-svcs.zones.read	internet-svcs.ipv6-setting.read
Update the IPv6 settings.	PUT /v1/{crn}/zones/{domain_id}/setting/ipv6	internet-svcs.zones.update	internet-svcs.ipv6-setting.update
Get the websocket settings.	GET /v1/{crn}/zones/{domain_id}/setting/websocket	internet-svcs.zones.read	internet-svcs.websockets-setting.read
Update the websocket settings.	PUT /v1/{crn}/zones/{domain_id}/setting/websocket	internet-svcs.zones.update	internet-svcs.websockets-setting.update
Get the response buffering settings.	GET /v1/{crn}/zones/{domain_id}/setting/response_buffering	internet-svcs.performance.read	internet-svcs.response-buffering-setting.read
Update the response buffering settings.	PUT /v1/{crn}/zones/{domain_id}/setting/response_buffering	internet-svcs.performance.update	internet-svcs.response-buffering-setting.update
Get the hotlink protection settings.	GET /v1/{crn}/zones/{domain_id}/setting/hotlink_protection	internet-svcs.performance.read	internet-svcs.hotlink-protection-setting.read
Update the hotlink protection settings.	PUT /v1/{crn}/zones/{domain_id}/setting/hotlink_protection	internet-svcs.performance.update	internet-svcs.hotlink-protection-setting.update
Get the maximum upload size settings.	GET /v1/{crn}/zones/{domain_id}/setting/max_upload	internet-svcs.performance.read	internet-svcs.max-upload-setting.read

Update the maximum upload size settings.	PUT /v1/{crn}/zones/{domain_id}/setting/max_upload	internet-svcs.performance.update	internet-svcs.max-upload-setting.update
Get the TLS client authentication settings.	GET /v1/{crn}/zones/{domain_id}/setting/tls_client_auth	internet-svcs.security.read	internet-svcs.tls-client-auth-setting.read
Update the TLS client authentication settings.	PUT /v1/{crn}/zones/{domain_id}/setting/tls_client_auth	internet-svcs.security.update	internet-svcs.tls-client-auth-setting.update
Get the pseudo IPv4 settings.	GET /v1/{crn}/zones/{domain_id}/setting/pseudo_ipv4	internet-svcs.zones.read	internet-svcs.pseudo-ipv4-setting.read
Update the pseudo IPv4 settings.	PUT /v1/{crn}/zones/{domain_id}/setting/pseudo_ipv4	internet-svcs.zones.update	internet-svcs.pseudo-ipv4-setting.update
Get the origin error page passthrough settings.	GET /v1/{crn}/zones/{domain_id}/setting/origin_error_page_pass_thru	internet-svcs.zones.read	internet-svcs.origin-error-page-pass-thru-setting.read
Update the origin error page passthrough settings.	GET /v1/{crn}/zones/{domain_id}/setting/origin_error_page_pass_thru	internet-svcs.zones.update	internet-svcs.origin-error-page-pass-thru-setting.update
Get the brotli compression settings.	GET /v1/{crn}/zones/{domain_id}/setting/brotli	internet-svcs.performance.read	internet-svcs.brotli-setting.read
Update the brotli compression settings.	PUT /v1/{crn}/zones/{domain_id}/setting/brotli	internet-svcs.performance.update	internet-svcs.brotli-setting.update
Get the Email obfuscation settings.	GET /v1/{crn}/zones/{domain_id}/setting/email_obfuscation	internet-svcs.security.read	internet-svcs.email-obfuscation-setting.read
Update the Email obfuscation settings.	PUT /v1/{crn}/zones/{domain_id}/setting/email_obfuscation	internet-svcs.security.update	internet-svcs.email-obfuscation-setting.update
Get the ciphers settings.	GET /v1/{crn}/zones/{domain_id}/setting/ciphers	internet-svcs.security.read	internet-svcs.ciphers-setting.read
Update the ciphers settings.	PUT /v1/{crn}/zones/{domain_id}/setting/ciphers	internet-svcs.security.update	internet-svcs.ciphers-setting.update

Domain settings

Bot Management

Action	Method	IAM ACTION	AT ACTION
Get the Bot Management settings.	GET /v1/{crn}/zones/{domain_id}/bot_management	internet-svcs.reliability.read	internet-svcs.bot-management.read
Update the Bot Management settings.	PUT /v1/{crn}/zones/{domain_id}/bot_management	internet-svcs.reliability.update	internet-svcs.bot-management.update
Get the Bot Analytics Score Source.	GET /v1/{crn}/zones/{domain_id}/bot_analytics/score_source	internet-svcs.security.read	internet-svcs.bot-analytics.read
Get the Bot Analytics Timeseries.	GET /v1/{crn}/zones/{domain_id}/bot_analytics/timeseries	internet-svcs.security.read	internet-svcs.bot-analytics.read
Get the Bot Analytics Top Attributes.	GET /v1/{crn}/zones/{domain_id}/bot_analytics/top_ns	internet-svcs.security.read	internet-svcs.bot-analytics.read
Bot management			

Deleting a CIS instance

Removing your domain cancels all active subscriptions, which will not be refunded per our billing policy. If you add this domain back, you must re-purchase all subscriptions. Removing your domain does not change your domain registration.

Before you begin

Before you delete a CIS instance, make sure that you have satisfied the following requirements:

- If you experience website issues, temporarily pause CIS to evaluate your website's performance.
- If you have to re-add the domain in a different account, make sure to save the current settings. For example, you might import and export DNS records.
- Removing a domain prevents your domain from using DNS resolution. To avoid DNS errors, update your name servers at your domain registrar to use name servers not owned by CIS.
- Confirm that your name servers no longer point to CIS.
- At your registrar, make sure you do not have a DS DNS record. This record enables DNS Security Extensions (DNSSEC) and could prevent your DNS records from being changed.

Deleting a CIS instance in the console

To delete a CIS instance in the console, follow these steps:

- a. Recommended: Delete all the Logpush jobs for that domain. To do so, go to the **Accounts** page, then scroll to the Logpush list on the Logs tab. Click **Delete** from the Actions menu  on the right side of the Logpush name.



Note: You can't undo this action.

- b. To start the deletion process, go to the **Overview** page and click **Delete** from the domain's Actions menu.



Important: If you are moving your domain to a different provider, be sure to migrate your DNS records and other configuration information to the new provider before you activate the domain there. Activating the domain before you migrate from CIS can cause your domain to change to the [Moved state](#).

Deleting a CIS instance from the CLI

To delete a CIS instance from the CLI, follow these steps:

- a. Recommended: Delete all the Logpush jobs for that domain.



Note: You can't undo this action.

- b. Enter the following command:

```
$ ibmcloud cis instance-delete INSTANCE [-f, --force][-h, --help]
```

Where:

`INSTANCE`

Is the name or ID of a CIS service instance.

`-f, --force`

Force the operation without confirmation.

`-h, --help`

Get help on this command.



Important: If you are moving your domain to a different provider, be sure to migrate your DNS records and other

configuration information to the new provider before you activate the domain there. Activating the domain before you migrate from CIS can cause your domain to change to the [Moved state](#).

Setting up Terraform for CIS

Terraform on IBM Cloud® enables predictable and consistent creation of IBM Cloud services so that you can rapidly build complex, multi-tier cloud environments following Infrastructure as Code (IaC) principles. Similar to using the IBM Cloud CLI or API and SDKs, you can automate the creation, update, and deletion of your IBM Cloud Internet Services instances by using HashiCorp Configuration Language (HCL).



Tip: Looking for a managed Terraform on IBM Cloud® solution? Try out [IBM Cloud® Schematics](#). With Schematics, you can use the Terraform scripting language that you are familiar with, but you don't have to worry about setting up and maintaining the Terraform command line and the IBM Cloud® Provider plug-in. Schematics also provides pre-defined Terraform templates that you can install from the IBM Cloud® catalog.

Installing Terraform and configuring resources for IBM Cloud Internet Services

Before creating an authorization by using Terraform, complete the following prerequisites:

- Make sure that you have the [required access](#) to create and work with IBM Cloud Internet Services.
 - Install the Terraform CLI and configure the IBM Cloud Provider plug-in for Terraform. For more information, see [Getting started with Terraform on IBM Cloud](#). The plug-in abstracts the IBM Cloud APIs that are used to complete this task.
 - Create a Terraform configuration file named `main.tf`. Use this file to define the authorization between services by writing Terraform code in HashiCorp Configuration Language (HCL). For more information, see the [Terraform documentation](#).
- a. In your Terraform configuration file, find the Terraform code that you used to create the CIS instance.
- b. Create a CIS instance by using the `ibm_resource_instance` resource argument in your `main.tf` file. The CIS instance in the following example is named `test` and is created with the Standard pricing plan. The `resource_group_id` is `data.ibm_resource_group.group.id`, and the location property is set to `global`.

```
data "ibm_resource_group" "group" {
  name = "test"
}

resource "ibm_cis" "cis_instance" {
  name          = "test"
  plan          = "standard-next"
  resource_group_id = data.ibm_resource_group.group.id
  tags          = ["tag1", "tag2"]
  location      = "global"

  //User can increase timeouts
  timeouts {
    create = "15m"
    update = "15m"
    delete = "15m"
  }
}
```

If you don't specify the `resource_group_id`, the CIS instance is created in the default resource group. The `API_KEY` must have permissions for this group.



Important: If you migrate from one plan to another, the change is considered a modification of an existing resource. It isn't considered as a resource creation or deletion.

- c. After you finish building your configuration file, initialize the Terraform CLI. For more information, see [Initialize the Working Directory](#).

```
$ terraform init
```

- d. Provision the resources from the `main.tf` file. For more information, see [Terraform workflow for provisioning](#)

[infrastructure](#).

- a. Run `terraform plan` to generate a Terraform execution plan to preview the proposed actions.

```
$ terraform plan
```

- b. Run `terraform apply` to create the resources that are defined in the plan.

```
$ terraform apply
```

- e. From the [IBM Cloud resource list](#), select the CIS instance that you created and note the instance ID.
- f. Verify that the access policy is successfully assigned. For more information, see [Reviewing assigned access in the console](#).

What's next?

Now that you successfully created your first CIS service instance with Terraform on IBM Cloud, you can visit the CIS [Cloud Internet Services \(CIS\) Terraform registry](#) to perform additional tasks using Terraform.

Generating configurations with Terraform

Terraform can generate code for the resources you define in `import` blocks that do not already exist in your configuration.

Experimental Configuration generation is available in Terraform v1.5 as an [experimental feature](#).

To generate a configuration with Terraform, perform the following procedure:

- a. Add the `import` block:

```
import {
  to = ibm_cis_ruleset.test
  id = "dcdec3fe0cbe41edac08619503da8de5:601b728b86e630c744c81740f72570c3:crn:v1:staging:public:internet-svcs-
ci:global:a/01652b251c3ae2787110a995d8db0135:1a9174b6-0106-417a-844b-c8eb43a72f63:."
}
```

Where:

- `to`: Points to the address a resource will have in your state file. In this example, the resource module is followed by the resource name that is being added.
- `id`: Can be a literal string of your resource's import ID, or an expression that evaluates to a string. In this example, the domain ID and the CRN are separated by a colon. See the [CIS Terraform import example](#).

- b. Plan and generate the configuration:

```
$ terraform plan -generate-config-out=generated.tf
ibm_cis_ruleset.test: Preparing import...
[id=dcdec3fe0cbe41edac08619503da8de5:601b728b86e630c744c81740f72570c3:crn:v1:staging:public:internet-svcs-
ci:global:a/01652b251c3ae2787110a995d8db0135:1a9174b6-0106-417a-844b-c8eb43a72f63:]
ibm_cis_ruleset.test: Refreshing state...
[id=dcdec3fe0cbe41edac08619503da8de5:601b728b86e630c744c81740f72570c3:crn:v1:staging:public:internet-svcs-
ci:global:a/01652b251c3ae2787110a995d8db0135:1a9174b6-0106-417a-844b-c8eb43a72f63:]

Terraform will perform the following actions:

# ibm_cis_ruleset.test will be imported
# (config will be generated)
resource "ibm_cis_ruleset" "test" {
  cis_id    = "crn:v1:staging:public:internet-svcs-ci:global:a/01652b251c3ae2787110a995d8db0135:1a9174b6-0106-417a-844b-
c8eb43a72f63:."
  domain_id = "601b728b86e630c744c81740f72570c3"
  id        = "dcdec3fe0cbe41edac08619503da8de5"
  ruleset_id = "dcdec3fe0cbe41edac08619503da8de5"

  rulesets_obj {
    description = "Updating a zone ruleset"
    kind        = "zone"
    last_updated = "2024-04-29T06:04:06.58962Z"
    name        = "default"
    phase       = "http_request_firewall_managed"
    version     = "2"

    rules {
      categories      = []
      description     = "deploying a managed rule"
      enabled         = true
      expression      = "ip.src ne 1.1.1.2"
      id              = "7787447241554207a18404a1b020fe50"
      rule_action     = "execute"
      rule_last_updated_at = "2024-04-29T06:04:06.58962Z"
      rule_logging    = {}
      rule_ref        = "efb7b8c949ac4650a09736fc376e9aee"
      rule_version    = "1"

      action_parameters {
        id      = "efb7b8c949ac4650a09736fc376e9aee"
        rulesets = []
        version  = "latest"
      }
    }
  }
}
```

```
overrides {
  action = "block"
  enabled = true

  categories {
    action = "block"
    category = "wordpress"
    enabled = true
  }

  rules {
    action = "block"
    enabled = true
    ruleset_id = "5de7edfa648c4d6891dc3e7f84534ffa"
  }
}
}
```

Plan: 1 to import, 0 to add, 0 to change, 0 to destroy.

```
|
| Warning: Config generation is experimental
|
| Generating configuration during import is currently experimental, and the generated configuration format might change in future versions.
|
```

Terraform has generated configuration and written it to generated.tf. Please review the configuration and edit it as necessary before adding it to version control.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

- Use the command `terraform plan -generate-config-out=generated.tf` to generate the resource configurations. These configurations are added to the `generated.tf` file.

c. Apply the configuration:

```
$ terraform apply
ibm_cis_ruleset.test: Preparing import...
[id=dcdec3fe0cbe41edac08619503da8de5:601b728b86e630c744c81740f72570c3:crn:v1:staging:public:internet-svcs-ci:global:a/01652b251c3ae2787110a995d8db0135:1a9174b6-0106-417a-844b-c8eb43a72f63::]
ibm_cis_ruleset.test: Refreshing state...
[id=dcdec3fe0cbe41edac08619503da8de5:601b728b86e630c744c81740f72570c3:crn:v1:staging:public:internet-svcs-ci:global:a/01652b251c3ae2787110a995d8db0135:1a9174b6-0106-417a-844b-c8eb43a72f63::]

Terraform will perform the following actions:

# ibm_cis_ruleset.test will be imported
resource "ibm_cis_ruleset" "test" {
  cis_id = "crn:v1:staging:public:internet-svcs-ci:global:a/01652b251c3ae2787110a995d8db0135:1a9174b6-0106-417a-844b-c8eb43a72f63::"
  domain_id = "601b728b86e630c744c81740f72570c3"
  id = "dcdec3fe0cbe41edac08619503da8de5"
  ruleset_id = "dcdec3fe0cbe41edac08619503da8de5"

  rulesets_obj {
    description = "Updating a zone ruleset"
    kind = "zone"
    last_updated = "2024-04-29T06:04:06.58962Z"
    name = "default"
    phase = "http_request_firewall_managed"
    version = "2"

    rules {
```

```
categories      = []
description     = "deploying a managed rule"
enabled        = true
expression     = "ip.src ne 1.1.1.2"
id             = "7787447241554207a18404a1b020fe50"
rule_action    = "execute"
rule_last_updated_at = "2024-04-29T06:04:06.58962Z"
rule_logging   = {}
rule_ref       = "efb7b8c949ac4650a09736fc376e9aee"
rule_version   = "1"

action_parameters {
  id      = "efb7b8c949ac4650a09736fc376e9aee"
  rulesets = []
  version  = "latest"

  overrides {
    action = "block"
    enabled = true

    categories {
      action = "block"
      category = "wordpress"
      enabled = true
    }

    rules {
      action = "block"
      enabled = true
      ruleset_id = "5de7edfa648c4d6891dc3e7f84534ffa"
    }
  }
}
}
```

Plan: 1 to import, 0 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

ibm_cis_ruleset.test: Importing...
[id=dcdec3fe0cbe41edac08619503da8de5:601b728b86e630c744c81740f72570c3:crn:v1:staging:public:internet-svcs-ci:global:a/01652b251c3ae2787110a995d8db0135:1a9174b6-0106-417a-844b-c8eb43a72f63::]
ibm_cis_ruleset.test: Import complete
[id=dcdec3fe0cbe41edac08619503da8de5:601b728b86e630c744c81740f72570c3:crn:v1:staging:public:internet-svcs-ci:global:a/01652b251c3ae2787110a995d8db0135:1a9174b6-0106-417a-844b-c8eb43a72f63::]

Apply complete! Resources: 1 imported, 0 added, 0 changed, 0 destroyed.

d. Review the generated file, which should look like the following example:

```
# __generated__ by Terraform
# Review these resources and move them into your main configuration files.

# __generated__ by Terraform from
"dcdec3fe0cbe41edac08619503da8de5:601b728b86e630c744c81740f72570c3:crn:v1:staging:public:internet-svcs-ci:global:a/01652b251c3ae2787110a995d8db0135:1a9174b6-0106-417a-844b-c8eb43a72f63::"
resource "ibm_cis_ruleset" "test" {
  cis_id      = "crn:v1:staging:public:internet-svcs-ci:global:a/01652b251c3ae2787110a995d8db0135:1a9174b6-0106-417a-844b-c8eb43a72f63::"
  domain_id  = "601b728b86e630c744c81740f72570c3"
  ruleset_id = "dcdec3fe0cbe41edac08619503da8de5"
  rulesets_obj {
    description = "Updating a zone ruleset"
    kind       = "zone"
  }
}
```

```

last_updated = "2024-04-29T06:04:06.58962Z"
name      = "default"
phase     = "http_request_firewall_managed"
ruleset_id = null
version   = "2"
rules {
  categories      = []
  description     = "deploying a managed rule"
  enabled         = true
  expression      = "ip.src ne 1.1.1.2"
  id              = "7787447241554207a18404a1b020fe50"
  rule_action     = "execute"
  rule_last_updated_at = "2024-04-29T06:04:06.58962Z"
  rule_logging    = {}
  rule_ref        = "efb7b8c949ac4650a09736fc376e9aee"
  rule_version    = "1"
  action_parameters {
    id      = "efb7b8c949ac4650a09736fc376e9aee"
    ruleset = null
    rulesets = []
    version = "latest"
    overrides {
      action      = "block"
      enabled     = true
      sensitivity_level = null
      categories {
        action = "block"
        category = "wordpress"
        enabled = true
      }
      rules {
        action      = "block"
        enabled     = true
        ruleset_id  = "5de7edfa648c4d6891dc3e7f84534ffa"
        sensitivity_level = null
      }
    }
  }
}
}
}
}
}

```

CIS WAF rule change log

Stay informed with daily updates to CIS WAF managed rules, including newly identified threats, detection improvements, ruleset changes, and enhanced security protections.

This log records all newly released, updated, disabled, and emergency rule changes are they are published.

This topic includes:

- Newly released and updated managed WAF rules
- Rules deployed in logging-only (beta) mode
- Changes to default rule actions
- Disabled remediation rules
- Emergency rule releases outside the standard release cycle
- General WAF protection updates

WAF - WAF Release - Scheduled changes for 2026-09-29

Published on: Tue, 22 Sep 2026 00:00:00 GMT

Announcement Date	Release Date	Release Behavior	Legacy Rule ID	Rule ID	Description	Comments
2026-09-22	2026-09-29	Log	N/A	f2445b9131214302a11477a2cb14ded8	Broken Access Control - Directory Traversal	This is a new c
2026-09-22	2026-09-29	Log	N/A	5c1ba1fdb0a742d4beaaefd00364bd7e	HTTP Request Smuggling - Request Body Anomaly - Beta	This rule will b rule "HTTP/2 I Request Body a80f214f094'
2026-09-22	2026-09-29	Disabled	N/A	09055ff9f80046419a3c0be5d498a69a	Command Injection - Generic 8 - body - Beta	This rule will b rule "Commar body" (ID: 5b3ce84c099
2026-09-22	2026-09-29	Log	N/A	e6fd9291701d4fea96741ba1a056caff	Command Injection - Generic 8 - uri - Beta	This rule will b rule "Commar (ID: ff8df24181aa
2026-09-22	2026-09-29	Log	N/A	c825487afc274e34966052bb87ae8cfc	GitLab - Path Traversal- CVE:CVE-2026-85706	This is a new c

WAF - WAF Release - 2026-09-25 - Emergency

Published on: Thu, 25 Sep 2026 00:00:00 GMT

This update provides immediate defense against critical vulnerabilities affecting WordPress and JFrog Artifactory, including path traversal, local file inclusion (LFI), cross-site scripting (XSS), and authentication bypass exploits.

Key Findings

- CVE-2026-87902: A high-severity Path Traversal and Local File Inclusion (LFI) vulnerability affecting WordPress. Unauthenticated attackers can exploit this flaw to read arbitrary files on the host server, potentially exposing sensitive configuration data or system files.
- CVE-2026-42018 & CVE-2026-82329: Critical authentication bypass vulnerabilities affecting JFrog Artifactory. Successful exploitation allows unauthenticated attackers to bypass security controls and achieve unauthorized access to the Artifactory instance.

Impact

We strongly recommend that administrators apply the latest vendor patches for WordPress and JFrog Artifactory to fully secure origin servers.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
Cloudflare Managed Ruleset	64bceec36a74e445891d08a9870a43f96	N/A	Wordpress - Path Traversal, Local File Inclusion - CVE:CVE-2026-87902	N/A	Block	This is a new detection.
Cloudflare Managed Ruleset	755ce44008cd482a964cb96e909a4db4	N/A	Wordpress - XSS - Comment	N/A	Block	This is a new detection.
Cloudflare Managed Ruleset	76f7d07e2ff549789ab222cac797ef03	N/A	JFrog Artifactory - Authentication Bypass - CVE:CVE-2026-42018	N/A	Block	This is a new detection.
Cloudflare Managed Ruleset	c6dafaadb861478dbca81469a813ac74	N/A	JFrog Artifactory - Authentication Bypass - CVE:CVE-2026-82329	N/A	Block	This is a new detection.

WAF - WAF Release - 2026-09-22

Published on: Tue, 22 Sep 2026 00:00:00 GMT

Announcement Date	Release Date	Release Behavior	Legacy Rule ID	Rule ID	Description	Comments
2026-09-15	2026-09-22	Log	N/A	40b93de7a8f848709c4ec3e60f0313d6	SSRF - Block jar HTTP loopback payload	This is a new detection.
2026-09-15	2026-09-22	Log	N/A	ca05d6c847834f75a317c33b5f21b651	SSRF - Cloud,Link-Local non-standard IP notation	This is a new detection.

2026-09-15	2026-09-22	Log	N/A	48dfa3e5bef84063914edfe175cd912a	SSRF - Local non-standard IP notation	This is a new detection.
2026-09-15	2026-09-22	Log	N/A	cd1de1fd21c443508f9073f2a1ba83f	SSTI - Jinja Dangerous Globals Chain	This is a new detection.

WAF - WAF Release - 2026-09-15

Published on: Tue, 15 Sep 2026 00:00:00 GMT

This release introduces new threat detections to enhance protection against command injection attempts, Server-Side Request Forgery (SSRF) targeting cloud metadata, and information disclosure within version control history.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	b2170b7b1a2c4b8eba0b498eca453d31	N/A	SSRF - Cloud - 3	Log	Block	This is a new detection
CIS Managed Ruleset	02c818297e6d42aaa55e67f5e540f17f	N/A	Version Control - Information Disclosure - Beta	Log	Block	This rule is merged into "Version Control - Information Disclosure - Beta" (ID: 23548ee2b36547a1b)
CIS Managed Ruleset	93793848937f4f988f1dfdabba458b4b	N/A	Command Injection - Generic 10	Log	Block	This is a new detection

WAF - WAF Release - 2026-09-10 - Emergency

Published on: Thu, 10 Sep 2026 00:00:00 GMT

This update provides immediate defense against a high-severity, actively exploited zero-day vulnerability targeting Adobe Commerce and Magento Open Source storefronts.

Key Findings

\$ * Adobe Commerce and Magento RCE (CVE-2026-75650 / "StyleSmuggler"): Unauthenticated Remote Code Execution (RCE) vulnerability caused by improper neutralization of special elements in the platform's template engine. Unauthenticated attackers can inject arbitrary PHP payloads through style properties to execute system commands and deploy persistent malware.

Impact

This emergency rule provides immediate edge-level mitigation and virtual patching, origin applications must be urgently updated. We strongly recommend to apply the hotfix outlined in Adobe Security Bulletin [APSB26-146](#) and immediately rotate all potentially exposed encryption keys, integration tokens, and system credentials, as patching alone does not remediate an existing compromise.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
---------	---------	----------------	-------------	-----------------	------------	----------

CIS Managed Ruleset	f9a3026b0fdc4d63b7338346440f5c55	N/A	Adobe Commerce - Remote Code Execution - CVE:CVE-2026-75650	N/A	Block	This is a new detection.
---------------------	----------------------------------	-----	---	-----	-------	--------------------------

WAF - WAF Release - 2026-09-08

Published on: Tue, 08 Sep 2026 00:00:00 GMT

This release enhances detection logic for existing rules targeting Next.js remote code execution (RCE) vulnerabilities by consolidating active beta rules into baseline signatures.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	d5d9f863e50b416faf43934dc76ba662	N/A	Next.js - Image Optimizer Remote Code Execution via Crafted AVIF - Beta	Log	Block	This rule is merged into "Next.js - Image Optimizer Remote Code Execution via Crafted AVIF - Beta"
CIS Managed Ruleset	771ac3761dcd485cb0e91ea0208457cf	N/A	Next.js - Remote Code Execution - CVE:CVE-2026-75604 - Beta	Log	Block	This rule is merged into "Next.js - Remote Code Execution - CVE:CVE-2026-75604 - Beta"

WAF - Enforce positive security with Application Profiles

Published on: Mon, 07 Sep 2026 00:00:00 GMT

Application Profiles add a positive-security layer to CIS WAF. Instead of looking only for requests that resemble known attacks, Application Profiles learn what valid requests to your application look like and identify traffic that deviates from the expected structure.

The first available profile type, Schema Profiles, can learn path variables, query parameters, headers, cookies, JSON bodies, and form-encoded bodies. Profiles model field types and constraints such as numeric ranges, string lengths, and character classes. After a profile becomes available, an always-on detection classifies requests as conforming or non-conforming without blocking traffic.

Use **Profile Analysis** in [Security Analytics](#) to review conformance trends and sampled violation details before enforcing a profile. When you are ready to mitigate traffic, use a [Custom Rule](#) to scope enforcement by hostname, path, operation, or other security signals such as Attack Score.

Customers with API Security already have access to Schema Profiles through Schema Learning and Schema Validation. CIS is also opening a closed beta to invited Enterprise customers without API Security. Contact your CIS account team to express interest.

For more information, refer to [Application Profiles](#).

WAF - WAF Release - 2026-09-01

Published on: Tue, 01 Sep 2026 00:00:00 GMT

This release introduces a new threat detection to enhance protection against SQL injection (SQLi) attempts exploiting complex query syntax.

Key Findings

\$ * SQLi Protection: Improved coverage for SQL injection patterns involving WHERE comparisons combined with WITH clauses.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	d2d75b2f0614405f9fab0354bcfa0966	N/A	SQLi - WHERE Comparison With WITH Clause	Log	Block	This is a new detection.

WAF - WAF Release - 2026-08-26 - Emergency

Published on: Wed, 26 Aug 2026 00:00:00 GMT

This emergency release updates an existing Next.js remote code execution rule to identify CVE-2026-75604 and adds a new rule for remote code execution in the Next.js Image Optimizer via crafted AVIF images.

Key Findings

\$ * CVE-2026-75604 affects Windows-hosted Next.js applications using both the Pages Router and App Router without Cache Components and can lead to unauthenticated remote code execution.

* GHSA-2xp9-vwfh-vxw4 affects the Next.js Image Optimizer and can lead to unauthenticated remote code execution when it optimizes an attacker-controlled AVIF image.

Impact

Next.js recommends updating to version 16.3.3 or 15.5.24 to address these vulnerabilities.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	2b6b94ec864d47f99630ecf72ca6cce3	N/A	Next.js - Remote Code Execution - CVE:CVE-2026-75604	Block	N/A	Rule metadata description refined. Detection unchanged.
CIS Managed Ruleset	18b22b0bd423423c945b3a0180256efe	N/A	Next.js - Image Optimizer Remote Code Execution via Crafted AVIF	N/A	Block	This is a new detection.

WAF - WAF Release - 2026-08-25

Published on: Tue, 25 Aug 2026 00:00:00 GMT

This release moves four new detections from Log to Block, merges the XSS, HTML Injection - Script Tag - Beta rule into the original rule, and adds a Generic Rules - Remote Code Execution rule in Block mode.

Key Findings

\$ * Four new detections move from Log to Block: HTTP/2 Request Smuggling - Request Body Anomaly and XSS - JavaScript Event Handler Coercion across Headers, Body, and URI.

* The XSS, HTML Injection - Script Tag - Beta rule is merged into the original rule.

* A Generic Rules - Remote Code Execution detection is added in Block mode.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	a80f214f0947435dabb2ba2d1489d892	N/A	HTTP/2 Request Smuggling - Request Body Anomaly	Log	Block	This is a new detection.
CIS Managed Ruleset	58a184412d2b4113bca6379b20646260	N/A	XSS - JavaScript Event Handler Coercion - Headers	Log	Block	This is a new detection.
CIS Managed Ruleset	e79cb939d6aa41db984e6db3d706d517	N/A	XSS - JavaScript Event Handler Coercion - Body	Log	Block	This is a new detection.
CIS Managed Ruleset	7e3249c7a5d8469697478746660886c8	N/A	XSS - JavaScript Event Handler Coercion - URI	Log	Block	This is a new detection.
CIS Managed Ruleset	d34bc5db8cbc4e18a44ed115c293b926	N/A	XSS, HTML Injection - Script Tag - Beta	Log	Block	This rule is merged into the original rule "XSS, HTML Injection - Script Tag" (ID: ...7b58420b).
CIS Managed Ruleset	2b6b94ec864d47f99630ecf72ca6cce3	N/A	Generic Rules - Remote Code Execution	N/A	Block	This is a new detection.

WAF - Leaked credentials detection now scans Authorization headers - 2026-08-20

Published on: Thu, 20 Aug 2026 00:00:00 GMT

[Leaked credentials detection](#) now scans the `Authorization` request header for Basic Authentication credentials. Previously, the detection only inspected request bodies, query strings, and headers for well-known web applications or custom

detection locations, which meant credentials sent through HTTP Basic Authentication were not covered by default.

This new default scan location decodes the `Authorization: Basic <credentials>` header and compares the extracted username and password against CIS's database of leaked credentials, the same way as other default scan locations. Matches populate the existing [leaked credentials fields](#), such as `cf.waf.credential_check.password_leaked`, and trigger the [Exposed-Credential-Check managed transform header](#) if configured, so you can reuse existing [custom rules](#) and [rate limiting rules](#) without changes.

This change was applied automatically for zones with leaked credentials detection enabled. No configuration changes are required.

For more information, refer to [Leaked credentials detection](#).

WAF - WAF Release - 2026-08-17

Published on: Mon, 17 Aug 2026 00:00:00 GMT

This release updates WordPress remote code execution rule metadata in the CIS Managed Ruleset and CIS Free Ruleset to identify CVE-2026-65640.

Key Findings

\$ * CVE-2026-65640: A remote code execution vulnerability affecting WordPress core and plugin components. Remote, unauthenticated attackers can execute arbitrary system commands to gain unauthorized access or establish backdoors on host servers.

Impact

The WordPress changes update rule metadata only; detection behavior and actions remain unchanged.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	dcf635ab2e744e1a994443973590a4ad	N/A	Wordpress - Remote Code Execution - CVE:CVE-2026-65640	Block	N/A	Rule metadata description refined. Detection unchanged.
CIS Free Ruleset	6ad9f2049b094c608be0f8adcfe1a93c	N/A	Wordpress - Remote Code Execution - CVE:CVE-2026-65640	Block	N/A	Rule metadata description refined. Detection unchanged.

WAF - WAF Release - 2026-08-11

This release introduces new protection for a remote code execution vulnerability in vBulletin and improves two existing detections.

Key Findings

\$ * A new detection provides protection against vBulletin CVE-2026-61511.
* Two existing detections have been improved to strengthen coverage.

Impact

Successful exploitation of CVE-2026-61511 may lead to remote code execution on affected vBulletin systems, potentially resulting in unauthorized access, data exposure, service disruption, and broader compromise of the hosting environment. Administrators are strongly encouraged to apply vendor updates and recommended mitigations.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	1b0775f0f092483387cfb23f94f3006b	N/A	vBulletin - Remote Code Execution - CVE:CVE-2026-61511	Log	Block	This is a new detection.
CIS Managed Ruleset	784d3824b6cf419db6af0b64098b749e	N/A	Version Control - Information Disclosure - Beta	Log	Block	This rule is merged into the or rule "Version Control - Inform Disclosure" (ID: 23548ee2b36547a1be09bb2c055
CIS Managed Ruleset	a561c9138b46470ca6db96edd56225d8	N/A	vBulletin - Code Injection - Invalid image format - CVE:CVE-2019-17132 - Beta	Log	Block	This rule is merged into the or rule "vBulletin - Code Injectio Invalid image format - CVE:C\ 2019-17132" (ID: 5137834eb8634842852273a08fe5

WAF - WAF Release - 2026-08-07

This release updates WordPress XSS rule metadata in the CIS Managed Ruleset and CIS Free Ruleset to identify XSS2Shell (CVE-2026-64638). It also disables the Command Injection - Obfuscation rule.

Key Findings

\$ * CVE-2026-64638: A pre-authentication reflected cross-site scripting vulnerability affecting the WordPress login screen. Exploitation requires social engineering and explicit interaction by the target user. Under additional conditions, it may be escalated to remote code execution.

Impact

The WordPress changes update rule metadata only; detection behavior and actions remain unchanged.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	d3852d0891634686a46114069c6dff1c	N/A	Wordpress - XSS - CVE:CVE-2026-64638	Block	N/A	Rule metadata description refined. Detection unchanged.
CIS Free Ruleset	5bdf578fff504b8cbe3b7f699ab5ed95	N/A	Wordpress - XSS - CVE:CVE-2026-64638	Block	N/A	Rule metadata description refined. Detection unchanged.

CIS Managed Ruleset	95a84ab1645a49c685648c17761e7a4c	N/A	Command Injection - Obfuscation	Block	Disabled	Detection logic has been deprecated.
---------------------------	----------------------------------	-----	---------------------------------------	-------	----------	---

WAF - WAF Release - 2026-08-04

Published on: Tue, 04 Aug 2026 00:00:00 GMT

This release introduces new rules and updates Microsoft SharePoint RCE alongside enhanced SSRF cloud protection rule actions.

Key Findings

- \$ * CVE-2026-50522: An insecure deserialization vulnerability in Microsoft SharePoint Server. This may allow an unauthenticated attacker to execute arbitrary code using crafted requests.
- * CVE-2026-66066: An improper input processing vulnerability in Ruby on Rails Active Storage image variant transformations. This may allow an unauthenticated attacker to perform arbitrary file reads and achieve Remote Code Execution (RCE) using maliciously crafted payload requests.
- * Generic Cloud Protections: Added improved detection logic targeting Server-Side Request Forgery (SSRF) in cloud-hosted applications.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	91aee93c31944828bf86f068052b07cf	N/A	Microsoft SharePoint - Remote Code Execution - CVE:CVE-2026-50522	Log	Block	This is a new detection.
CIS Managed Ruleset	89d0243997d24c6ea1d610a23a5b40d6	N/A	Rails - Arbitrary File Read & RCE - CVE:CVE-2026-66066	Block	Block	This was labeled as File Upload - RCE.
CIS Managed Ruleset	ae40661b4ef24f9c8abd98338242627b	N/A	SSRF - Local	Disabled	■	This detection has been removed.
CIS Managed Ruleset	98bfd6bb46074d5b8d1c4b39743a63ec	N/A	SSRF - Local - 2 - Beta	Disabled	■	This detection has been removed.
CIS Managed Ruleset	54e1733b10da4a599e06c6fbc2e84e2d	N/A	SSRF - Cloud - Beta	Disabled	■	This detection has been removed.
CIS Managed Ruleset	ecd26d61a75e46f6a4449a06ab8af26f	N/A	SSRF - Cloud - 2 - Beta	Disabled	■	This detection has been removed.

CIS Managed Ruleset	281a1b7086b84db7a695220725ba9d7c	N/A	SSRF - Cloud	Disabled	Block	We are changing the action for this rule from Disabled to BLOCK
CIS Managed Ruleset	158177dec2504acdba1f2da201a076eb	N/A	SSRF - Local - Beta	Disabled	■	This detection has been removed.

WAF - WAF Release - 2026-07-29

Published on: Wed, 29 Jul 2026 00:00:00 GMT

This release introduces new rules and updates existing threat signatures to provide targeted protections for vulnerabilities in Nuxt Server Island components and Alibaba Fastjson deserialization routines, alongside enhanced protections for cloud metadata Server-Side Request Forgery (SSRF) and obfuscated command injection attempts.

Key Findings

- \$ * Nuxt Server Island - RCE(GHSA-9473-5f9j-94wq): An unauthenticated vulnerability in Nuxt Server Islands where remote attackers can supply arbitrary component names or props to endpoints. Manipulating these parameters allows unauthenticated component Remote Code Execution (RCE) on the server.
- * Alibaba Fastjson JSONType Remote Code Execution: A unauthenticated remote code execution vulnerability in Alibaba Fastjson (≤ 1.2.83) during JSON deserialization. Under default configurations, attackers can execute arbitrary system commands, bypassing traditional classpath and gadget-based defenses.
- * Generic Protections (SSRF & Command Injection): Added improved detection logic targeting Server-Side Request Forgery (SSRF) in cloud-hosted applications, alongside new rules targeting obfuscated command injection patterns across request parameters.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	54e1733b10da4a599e06c6fbc2e84e2d	N/A	SSRF - Cloud - Beta	Log	Block	This is an improved detection.
CIS Managed Ruleset	95a84ab1645a49c685648c17761e7a4c	N/A	Command Injection - Obfuscation	Log	Block	This is a new detection.
CIS Managed Ruleset	58df9693db4d454a8764fcda7347c892	N/A	Alibaba Fastjson JSONType Remote Code Execution - Body	Log	Block	This is a new detection.
CIS Managed Ruleset	6159ead63d284147943dc5a18ec012ea	N/A	Nuxt Server Island - RCE	N/A	Block	This is a new detection.This was labeled as Generic Rules - RCE.
CIS Managed Ruleset	dcf635ab2e744e1a994443973590a4ad	N/A	Generic Rules - RCE	N/A	Block	This is a new detection.

CIS Managed Ruleset	d3852d0891634686a46114069c6dff1c	N/A	Generic Rules - XSS	N/A	Block	This is a new detection.
CIS Managed Ruleset	89d0243997d24c6ea1d610a23a5b40d6	N/A	File Upload - RCE	N/A	Block	This is a new detection.
CIS Free Ruleset	6ad9f2049b094c608be0f8adcfe1a93c	N/A	Generic Rules - RCE	N/A	Block	This is a new detection.
CIS Free Ruleset	5bdf578fff504b8cbe3b7f699ab5ed95	N/A	Generic Rules - XSS	N/A	Block	This is a new detection.
CIS Free Ruleset	7ecac499d14a4750aa58c1e21b7f9c67	N/A	File Upload - RCE	N/A	Block	This is a new detection.

WAF - WAF Release - 2026-07-21

Published on: Tue, 21 Jul 2026 00:00:00 GMT

This release introduces new rules for vulnerabilities in Adobe ColdFusion, Next.js, WordPress alongside updates to existing rules thereby providing enhanced generic protections against Server-Side Request Forgery (SSRF), Local File Inclusion (LFI), and Cross-Site Scripting (XSS).

WAF and framework adapter mitigations for Next.js vulnerabilities

Multiple [security vulnerabilities ↗](#) were disclosed and patched by the Next.js team through July 2026 security release. These include denial of service, middleware and proxy bypass, server-side request forgery, information disclosure, and cache poisoning across a range of severities.

Several of the disclosed vulnerabilities are not possible to block at WAF layer,we strongly recommend updating your application and its dependencies immediately. Patched versions are available through v16.2.11 (Active LTS) and v15.5.21 (Maintenance LTS) to address these issues.

Advisory	CVE	Severity	Issue	WAF Coverage
Denial of Service in App Router using Server Actions	CVE-2026-64641	High	Crafted requests targeting Next.js applications using App Router with at least one Server Action can lead to excessive CPU usage. The CPU usage blocks processing of further requests in the same process, leading to Denial of Service.	WAF rule Next.js - DoS - CVE-2026-64641 (b013b67c357547b4b866234390dcdb0a) has been deployed to provide coverage.
Middleware / Proxy bypass in App Router applications using Turbopack and single locale	CVE-2026-64642	High	Next.js applications using App Router built with Turbopack and a single entry in config.i18n.locales are vulnerable to a middleware/proxy bypass. Accordingly, any authentication or security checks that a middleware/proxy may perform are bypassed.	This is a middleware bypass that unfortunately cannot be covered through CIS WAF signature engine.

Server-Side Request Forgery in rewrites via attacker-controlled destination hostname	CVE-2026-64645	High	A rewrites() or redirects() rule that builds its external destination hostname from request-controlled input can be pointed at an arbitrary hostname, regardless of the rule's hostname suffix. For rewrites, this behavior enables Server-Side Request Forgery (SSRF); for redirects, Open Redirect can be achieved.	Existing SSRF rules provide adequate coverage for this vulnerability, no tailored WAF rule was developed.
Server-Side Request Forgery in Server Actions on custom servers	CVE-2026-64649	High	When a Server Action forwards or redirects a request, an attacker can cause the server to send that outbound request to a malicious host (Server-Side Request Forgery). This requires the attacker's request to control Host-associated headers.	WAF rule Next.js - SSRF - CVE-2026-64649 (7fe6d6f3df774ae2a0011f20930091a3) has been deployed to provide coverage.
Denial of Service in the Image Optimization API using SVGs	CVE-2026-64644	Medium	When self-hosting Next.js with the default image loader, the Image Optimization API can optimize remotely hosted images if configured (not enabled by default). If those images contain malicious content, the images can cause CPU exhaustion in the /_next/image endpoint.	Malicious request is unfortunately indistinguishable from a legitimate image optimization request, so no WAF rule has been created to address this vulnerability.
Unbounded Server Action payload in Edge runtime	CVE-2026-64646	Medium	A crafted request can lead to memory consumption on Server Actions in the Edge runtime. Next.js applications which use App Router and have at least one Server Action are affected.	Unfortunately there is no one size fits all rule that can be deployed through WAF in lieu of custom bodySizeLimit configurations, so no WAF rule has been created to address this vulnerability.
Unauthenticated disclosure of internal Server Function endpoints	CVE-2026-64643	Medium	In Next.js applications using App Router, Server Actions (use server) or use cache endpoint IDs can be globally disclosed. An attacker can use this for reconnaissance and as part of a broader attack chain.	WAF rule Next.js - Information Disclosure - CVE-2026-64643 (6c4135d4d9d745e4866ad83672952826) has been deployed to provide coverage.
Cache confusion of response bodies for requests with bodies	CVE-2026-64648	Medium	A server-side fetch with a request body may return a cached response body from a different request to the same URL but different body. This only applies for fetch calls of the shape fetch(new Request(init), aDifferentInit)	This is an application logic bug that unfortunately cannot be covered through CIS WAF signature engine.
Cache confusion of response bodies for requests with bodies containing invalid UTF-8 byte sequences	CVE-2026-64647	Medium	A server-side fetch with a request body may return a cached response body from a different request to the same URL but different body. This only applies when receiving request bodies which contain invalid UTF-8 characters.	This is an application logic bug that unfortunately cannot be covered through CIS WAF signature engine.

Key Findings

\$ * CVE-2026-48276: A path traversal vulnerability in Adobe ColdFusion file upload mechanisms allows unauthenticated attackers to write or upload files to arbitrary locations outside designated directories on the origin server.

* CVE-2026-48282: A path traversal vulnerability in Adobe ColdFusion enables unauthenticated attackers to manipulate directory sequences and access restricted system files on the host filesystem.

* CVE-2026-60137: An unauthenticated SQL injection vulnerability affecting WordPress. Threat actors exploit unsanitized input parameters to execute arbitrary SQL queries, leading to unauthorized database access, record manipulation, or data exfiltration.

* CVE-2026-63030: A remote code execution vulnerability affecting WordPress core and plugin components. Remote, unauthenticated attackers can execute arbitrary system commands to gain unauthorized access or establish backdoors on host servers.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	7fbd9407bdb4a4eae2b3d91215e7d31	N/A	SSRF - Restricted Protocol	Log	Block	This is a new detection.
CIS Managed Ruleset	6ca512d240d848d6a0c7ef42a935ee5d	N/A	SSRF - Obfuscated Host	Log	Block	This is a new detection.
CIS Managed Ruleset	a3fb0870c38440d8a9a0eba81b0230ac	N/A	LFI - Path Traversal	Log	Block	This is a new detection.
CIS Managed Ruleset	452a04be3f73458c863d8dae61349c8b	N/A	Adobe ColdFusion - File Upload Path Traversal - CVE:CVE-2026-48276	Log	Block	This is a new detection.
CIS Managed Ruleset	a53a3fb491c64d74908081ee9cb61eac	N/A	Adobe ColdFusion - Path Traversal - CVE:CVE-2026-48282	Log	Block	This is a new detection.
CIS Managed Ruleset	d8b63828c2344d919b94d2594ac5e21f	N/A	XSS — JS Bracket Concat Obfuscation - Body	Log	Disabled	This is a new detection.
CIS Managed Ruleset	264a83a764be428ca41d516ff31f5559	N/A	XSS — JS Bracket Concat Obfuscation - Headers	Log	Disabled	This is a new detection.
CIS Managed Ruleset	4ba21a60837244029183b782987984fd	N/A	XSS — JS Bracket Concat Obfuscation - URI	Log	Block	This is a new detection.

CIS Managed Ruleset	1c060d3a371549219ee290d7ed933fcc	N/A	Wordpress - SQL Injection - CVE:CVE-2026-60137	N/A	Block	This was labeled as Generic Rules - SQLi.
CIS Managed Ruleset	7dfb2bd4708d4b88b9911dc0550664b6	N/A	Wordpress - Remote Code Execution - CVE:CVE-2026-63030	N/A	Block	This was labeled as Generic Rules - Unauthenticated RCE.
CIS Free Ruleset	db003b39b7774859a8d588ce33697a1a	N/A	Wordpress - SQL Injection - CVE:CVE-2026-60137	N/A	Block	This was labeled as Generic Rules - SQLi.
CIS Free Ruleset	ebd3f2df15c74ddcbf6220c9b5ec246a	N/A	Wordpress - Remote Code Execution - CVE:CVE-2026-63030	N/A	Block	This was labeled as Generic Rules - Unauthenticated RCE.
CIS Managed Ruleset	6c4135d4d9d745e4866ad83672952826	N/A	Next.js - Information Disclosure - CVE-2026-64643	N/A	Block	This was labeled as Generic Rules - Information Disclosure.
CIS Managed Ruleset	7fe6d6f3df774ae2a0011f20930091a3	N/A	Next.js - SSRF - CVE-2026-64649	N/A	Block	This was labeled as Generic Rules - Auth Bypass - 2.
CIS Managed Ruleset	c4ca56c0a6a348299d5a93e663167195	N/A	Next.js - Remote Code Execution - Cache Components	N/A	Block	This was labeled as Generic Rules - RCE.
CIS Managed Ruleset	b013b67c357547b4b866234390dcdb0a	N/A	Next.js - DoS - CVE-2026-64641	N/A	Block	This was labeled as Generic Rules - DoS.
CIS Managed Ruleset	aa21c9b8b97743bfb217748b2049a60c	N/A	Generic Rules - Command Execution - Body - Beta	Disabled	■	This detection has been removed.
CIS Managed Ruleset	e7ee67e824844754b513cdf3836855a4	N/A	Generic Rules - Command Execution - Header - Beta	Disabled	■	This detection has been removed.

CIS Managed Ruleset	5f2a6681a2b94442b23816286d060a0d	N/A	Generic Rules - Command Execution - URI - Beta	Disabled	■	This detection has been removed.
---------------------	----------------------------------	-----	--	----------	---	----------------------------------

WAF - WAF Release - 2026-07-17 - Emergency

Published on: Fri, 17 Jul 2026 00:00:00 GMT

This emergency release adds a new managed rule to block active exploitation of a critical remote code execution (RCE) and SQL injection (SQLi) vulnerability found in popular web frameworks.

Key Findings

\$ * Generic Frameworks - Unauthenticated RCE: Attackers can execute arbitrary system commands with web server privileges by sending malicious input containing invalid path sequences during request processing.

* Generic Frameworks - SQLi: Attackers can execute unauthorized database queries due to a failure to sanitize input values within request parameters.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	7dfb2bd4708d4b88b9911dc0550664b6	N/A	Generic Rules - Unauthenticated RCE	N/A	Block	This is a new detection.
CIS Managed Ruleset	1c060d3a371549219ee290d7ed933fcc	N/A	Generic Rules - SQLi	N/A	Block	This is a new detection.
CIS Free Ruleset	ebd3f2df15c74ddcbf6220c9b5ec246a	N/A	Generic Rules - Unauthenticated RCE	N/A	Block	This is a new detection.
CIS Free Ruleset	db003b39b7774859a8d588ce33697a1a	N/A	Generic Rules - SQLi	N/A	Block	This is a new detection.

WAF - WAF Release - 2026-07-14

Published on: Tue, 14 Jul 2026 00:00:00 GMT

This release introduces new rules targeting critical infrastructure vulnerabilities. These include an unauthenticated memory disclosure flaw in Citrix NetScaler ADC and Gateway (CVE-2026-8451) and a high-severity pre-authentication remote code execution (RCE) vulnerability in Progress Kemp LoadMaster (CVE-2026-8037).

Key Findings

\$ * CVE-2026-8451: An insufficient input validation vulnerability affects Citrix NetScaler ADC and NetScaler Gateway appliances configured as a SAML Identity Provider (IdP). Remote, unauthenticated attackers can exploit this flaw by sending malformed requests to trigger a memory overread, allowing them to leak chunks of sensitive data from adjacent appliance memory.

* CVE-2026-8037: A critical OS command injection vulnerability in Progress Kemp LoadMaster load balancers allows unauthenticated remote attackers to achieve remote code execution (RCE).

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
---------	---------	----------------	-------------	-----------------	------------	----------

CIS Managed Ruleset	78826e3223b94da493a2ade876973ac4	N/A	Citrix Netscaler ADC - Insufficient Input Validation - CVE:CVE-2026-8451	Log	Block	This is a new detection.
CIS Managed Ruleset	6b64d216620449fbb273d07910233f36	N/A	Progress Kemp LoadMaster - Remote Code Execution - CVE:CVE-2026-8037	Log	Block	This is a new detection.

WAF - WAF Release - 2026-07-01

Published on: Wed, 01 Jul 2026 00:00:00 GMT

This release adds targeted coverage for a path traversal flaw in Fortinet FortiSandbox (CVE-2026-39813) and transitions the Anomaly:Header:User-Agent - Fake Bing or MSN Bot rule action from Block to Disabled.

Key Findings

\$ * CVE-2026-39813: A path traversal vulnerability in Fortinet FortiSandbox allows remote, unauthenticated attackers to read arbitrary files from the underlying filesystem due to insufficient validation of user-supplied input paths.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	32075e19b1494117ac5915e8d84c92c9	N/A	Fortinet FortiSandbox - Path Traversal - CVE:CVE-2026-39813	Log	Block	This is a new detection.
CIS Managed Ruleset	ae20608d93b94e97988db1bbc12cf9c8	N/A	Anomaly:Header:User-Agent - Fake Bing or MSN Bot	Enabled	Disabled	We are changing the action for this rule from BLOCK to Disabled

WAF - WAF Release - 2026-06-23

Published on: Tue, 23 Jun 2026 00:00:00 GMT

This week's release introduces new managed protection to address a critical pre-authentication OS command injection vulnerability in Ivanti Sentry (CVE-2026-10520).

Key Findings

\$ * CVE-2026-10520: An OS command injection vulnerability in Ivanti Sentry allows remote, unauthenticated attackers to execute arbitrary system commands with root privileges. The flaw stems from improper sanitization of input strings parsed during internal configuration handling.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
---------	---------	----------------	-------------	-----------------	------------	----------

CIS Managed Ruleset	...242fdf83	N/A	Ivanti Sentry - Command Injection - CVE:CVE-2026-10520	Log	Block	This is a new detection.
---------------------	-------------	-----	--	-----	-------	--------------------------

WAF - Use Cloudforce One threat intelligence in WAF rules

Published on: Mon, 15 Jun 2026 00:00:00 GMT

You can now match incoming requests against Cloudforce One threat intelligence in your WAF rules. A new detection looks up the client IP address of each request against the threat intelligence database. If the IP was involved in threat activity in the past seven days, CIS populates `cf.intel.ip.*` fields that you can use in [custom rules](#) and [rate limiting rules](#).

The detection populates the following fields. Use the `any()` function with the `[*]` wildcard to match array values:

```
$ * `cf.intel.ip.datasets` — the dataset that flagged the IP address (`ddos` or `waf`).
* `cf.intel.ip.target_industries` — industries the IP address has targeted.
* `cf.intel.ip.attacker_names` — known threat actors associated with the IP address.
* `cf.intel.ip.attacker_countries` — source countries of the threat activity.
* `cf.intel.ip.target_countries` — countries the IP address has targeted.
```

For example, the following custom rule expression blocks requests from IP addresses associated with DDoS activity that have targeted France:

```
$ any(cf.intel.ip.target_countries[*] == "FR") and any(cf.intel.ip.datasets[*] == "ddos")
```

These fields work with the CIS API and Terraform. Matches are logged in [Security Analytics](#).

The threat intelligence detection is available to customers with an active [Cloudforce One](#) subscription. For more information, refer to [Threat intelligence](#).

WAF - WAF Release - 2026-06-15

Published on: Mon, 15 Jun 2026 00:00:00 GMT

This week's release introduces new managed protection to address a critical SQL injection vulnerability in Ghost CMS (CVE-2026-26980) and a new generic rule designed to identify and block sophisticated SQL Injection (SQLi) bypass attempts leveraging obfuscated boolean logic. These rules protect affected installations from unauthorized data exfiltration at the network edge.

Key Findings

```
$ * CVE-2026-26980: A blind SQL injection vulnerability in the Ghost CMS Content API (versions 3.24.0 to 6.19.0) allows unauthenticated remote attackers to inject malicious SQL commands via query parameters due to improper input validation.
```

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...b4c29bc6	N/A	Ghost CMS - SQLi - CVE:CVE-2026-26980	Log	Block	This is a new detection.
CIS Managed Ruleset	...b56f403f	N/A	SQLi - Obfuscated Boolean - URI	Log	Disabled	This is a new detection.

WAF - WAF Release - 2026-06-09

Published on: Tue, 09 Jun 2026 00:00:00 GMT

This release introduces new detections for a critical SQL injection vulnerability in Drupal installations utilizing PostgreSQL (CVE-2026-9082), alongside targeted protection for an unsafe deserialization flaw in the Mirasvit Cache Warmer extension (CVE-2026-45247). Additionally, this release includes coverage for a prototype pollution vector in Axios (CVE-2026-40175)

and a new generic rule designed to identify and block sophisticated SQL Injection (SQLi) bypass attempts leveraging obfuscated boolean logic.

Key Findings

\$ * CVE-2026-9082: A database abstraction vulnerability affects Drupal sites configured with a PostgreSQL backend. Remote, unauthenticated attackers can exploit this flaw via crafted inputs to inject malicious SQL commands and access or manipulate backend data.

* CVE-2026-45247: A PHP Object Injection vulnerability exists in the Mirasvit Cache Warmer extension for Magento and Adobe Commerce. This flaw stems from unsafe deserialization of untrusted user input, enabling unauthenticated attackers to execute arbitrary code on the hosting server.

* CVE-2026-40175: A prototype pollution vulnerability affects the Axios HTTP client library. Attackers can exploit this to inject malicious properties into the global JavaScript object prototype, potentially causing application crashes (Denial of Service) or executing unauthorized code depending on the application structure.

Impact

Successful exploitation of these vulnerabilities could allow unauthenticated attackers to execute arbitrary code, manipulate database contents, or induce application crashes, leading to severe operational disruption or complete server compromise. These newly deployed signatures intercept these advanced malicious payloads at the edge before they can interact with vulnerable software configurations.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...387cf935	N/A	Axios - Prototype Pollution - CVE:CVE-2026-40175	Log	Block	This is a new detection.
CIS Managed Ruleset	...417eb9e0	N/A	Drupal - PostgreSQL SQLi - CVE:CVE-2026-9082 - Body	Log	Block	This is a new detection.
CIS Managed Ruleset	...fd3857fd	N/A	Drupal - PostgreSQL SQLi - CVE:CVE-2026-9082 - URI	Log	Block	This is a new detection.
CIS Managed Ruleset	...aee241d2	N/A	SQLi - Obfuscated Boolean - Body	N/A	Disabled	This is a new detection.
CIS Managed Ruleset	...d8620070	N/A	SQLi - Obfuscated Boolean - Headers	N/A	Disabled	This is a new detection.
CIS Managed Ruleset	...e0be4d47	N/A	Mirasvit Cache Warmer - PHP Object Injection - CVE:CVE-2026-45247	N/A	Block	This is a new detection.

WAF - WAF Release - 2026-05-20

Published on: Wed, 20 May 2026 00:00:00 GMT

Key Findings

\$ * Existing rule enhancements have been deployed to improve detection resilience against broad classes of web attacks and strengthen behavioral coverage.

Continuous Rule Improvements

We are continuously refining our managed rules to provide more resilient protection and deeper insights into attack patterns. To ensure an optimal security posture, we recommend consistently monitoring the Security Events dashboard and adjusting

rule actions as these enhancements are deployed.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...9e9c068d	N/A	Sitecore - Cache Poisoning - CVE:CVE-2025-53693 Beta	N/A	Block	This rule is merged into the original rule "Sitecore - Cache Poisoning - CVE:CVE-2025-53693" (ID: ...7c5b669c).

WAF - WAF Release - 2026-05-15 - Emergency

Published on: Fri, 15 May 2026 00:00:00 GMT

This emergency release introduces two new rules to detect nginx heap buffer overflow and heap spray exploitation attempts targeting the rewrite module's `is_args` stale-state bug (CVE-2026-42945).

Key Findings

CVE-2026-42945: nginx Heap Buffer Overflow via Stale `is_args` in Rewrite Module

Successful exploitation allows remote attackers to trigger a heap buffer overflow in nginx's rewrite module by sending crafted URIs containing escapable characters. A length/copy pass mismatch in `ngx_http_script_copy_capture_code()` causes the copy pass to write escaped data into an undersized buffer, leading to heap corruption. This enables denial of service (worker process crash) and, with heap feng shui techniques, potential remote code execution.

We strongly recommend upgrading to nginx 1.30.1 (or later) immediately to address the underlying vulnerability. If you cannot upgrade immediately, avoid `rewrite` directives with `?` in the replacement string followed by `set` or `if` referencing capture groups.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...7e52be73	N/A	nginx - Remote Code Execution - Buffer Overread - CVE:CVE-2026-42945	N/A	Block	This is a new detection.
CIS Managed Ruleset	...9df0ee6c	N/A	nginx - Remote Code Execution - Heap Spray - CVE:CVE-2026-42945	N/A	Block	This is a new detection.

WAF - WAF Release - 2026-05-11

Published on: Mon, 11 May 2026 00:00:00 GMT

Key Findings

\$ * Existing rule enhancements have been deployed to improve detection resilience against broad classes of web attacks and strengthen behavioral coverage.

Continuous Rule Improvements

We are continuously refining our managed rules to provide more resilient protection and deeper insights into attack patterns. To ensure an optimal security posture, we recommend consistently monitoring the Security Events dashboard and adjusting rule actions as these enhancements are deployed.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
---------	---------	----------------	-------------	-----------------	------------	----------

CIS Managed Ruleset	...68b3c389	N/A	Remote Code Execution - Java Deserialization - Body - Beta	Block	Disabled	This is a new detection. This rule is merged into the original rule "Remote Code Execution - Java Deserialization" (ID: ...744305c4).
---------------------	-------------	-----	--	-------	----------	---

Workers, WAF - WAF and framework adapter mitigations for React and Next.js vulnerabilities

Published on: Thu, 07 May 2026 12:00:00 GMT

Multiple security vulnerabilities were disclosed by the React team and Vercel affecting React Server Components and Next.js. These include denial of service, middleware and proxy bypass, server-side request forgery, cross-site scripting, and cache poisoning issues across a range of severity levels.

We strongly recommend updating your application and its dependencies immediately. Patched versions are available for React (`react-server-dom-webpack` , `react-server-dom-parcel` , and `react-server-dom-turbopack` `19.0.6` , `19.1.7` , and `19.2.6`) and Next.js (`15.5.16` and `16.2.5`).

WAF protections

CIS WAF rules deployed in response to prior React Server Component CVEs ([CVE-2025-55184 ↗](#) and [CVE-2026-23864 ↗](#)) already provide coverage for the newly disclosed denial-of-service vulnerabilities. These rules are enabled by default with a Block action for all customers using the CIS Managed Ruleset, including Free plan customers using the Free Managed Ruleset.

Ruleset	Rule description	Rule ID	Default action
CIS Managed Ruleset	React - DoS - CVE-2025-55184 ↗	2694f1610c0b471393b21aef102ec699	Block
CIS Managed Ruleset	React - DoS - CVE-2026-23864 ↗	aaede80b4d414dc89c443cea61680354	Block

The existing rules detect the underlying attack patterns generically. As a result, they apply to the new [CVE-2026-23870 ↗](#) denial-of-service vulnerability in Server Components and the corresponding Next.js advisory [GHSA-8h8q-6873-q5fj ↗](#).

CIS is investigating whether WAF rules can be safely and effectively deployed for three of the high-severity advisories: [CVE-2026-23870 ↗](#) / [GHSA-8h8q-6873-q5fj ↗](#) , [GHSA-267c-6grr-h53f ↗](#) , and [GHSA-mg66-mrh9-m8jx ↗](#) . If it is possible to create a managed WAF rule that mitigates these CVEs and does not potentially break application behavior, CIS will add additional managed WAF rules. These rules will be announced through the [WAF changelog](#) . Because these vulnerabilities were shared with CIS with minimal advance notice, we are still investigating what WAF mitigations are possible.

Several of the disclosed vulnerabilities are not possible to block in WAF. We strongly recommend updating your applications so they are not purely reliant on WAF mitigations.

Customers on Pro, Business, or Enterprise plans should ensure that [Managed Rules are enabled](#) .

Next.js adapters

Vinext: [Vinext ↗](#) is a Vite plugin that reimplements the Next.js API surface. Vinext's latest release is not vulnerable to any of the disclosed CVEs. Vinext's architecture differs from stock Next.js in ways that sidestep the affected code paths. For example, it does not implement the PPR resume protocol, does not expose Pages Router data-route endpoints, and strips internal headers such as `x-nextjs-data` at request boundaries. As an extra layer of defense, we added a React `19.2.6` or later requirement when running `vinext init` ([PR #1118 ↗](#) , [PR #1112 ↗](#)) to prevent accidentally running a vulnerable version of React with Vinext.

OpenNext on CIS: OpenNext is an adapter that lets you deploy Next.js apps to the CIS Workers platform. OpenNext itself is not directly vulnerable to the React denial-of-service CVE, but users must update the Next.js version in their application. The OpenNext team has updated the adapter to further harden against these vectors and released a new version of the CIS adapter. Test fixtures and examples have been updated to use patched versions ([PR #1255 ↗](#)).

Summary of disclosed vulnerabilities

Advisory	Severity	Issue	WAF status
CVE-2026-23870 ↗ / GHSA-8h8q-6873-q5fj ↗	High	Denial of service in Server Components	WAF rules in place: 2694f1610c0b471393b21aef102ec699, aaede80b4d414dc89c443cea61680354
CIS is investigating additional managed WAF coverage			
GHSA-267c-6grr-h53f ↗	High	Middleware bypass via segment-prefetch routes	CIS is investigating if this can be safely and effectively mitigated by a managed WAF rule
GHSA-mg66-mrh9-m8jx ↗	High	Denial of service via connection exhaustion in Cache Components	CIS is investigating if this can be safely and effectively mitigated by a managed WAF rule
GHSA-492v-c6pp-mqqv ↗	High	Middleware bypass via dynamic route parameter injection	Not possible to safely enable a managed WAF rule without potentially breaking application behavior
GHSA-c4j6-fc7j-m34r ↗	High	SSRF via WebSocket upgrades	Not possible to safely enable a managed WAF rule without potentially breaking application behavior
GHSA-36gx-fr4f-26g5 ↗	High	Middleware bypass in Pages Router i18n	Custom WAF rule possible; global managed rule could potentially break application behavior
GHSA-ffhc-5mcf-pf4q ↗	Moderate	XSS via CSP nonces	Custom WAF rule possible; global managed rule could potentially break application behavior
GHSA-gx5p-jg67-6x7h ↗	Moderate	XSS in <code>beforeInteractive</code> scripts	Not possible to safely enable a managed WAF rule without potentially breaking application behavior
GHSA-h64f-5h5j-ijjh ↗	Moderate	Denial of service in Image Optimization API	Custom WAF rule possible; global managed rule could potentially break application behavior
GHSA-wfc6-r584-vfw7 ↗	Moderate	Cache poisoning in RSC responses	Custom WAF rule possible; global managed rule could potentially break application behavior
GHSA-vfv6-92ff-j949 ↗	Low	Cache poisoning via RSC cache-busting collisions	Not possible to safely enable a managed WAF rule without potentially breaking application behavior
GHSA-3g8h-86w9-wvmq ↗	Low	Middleware redirect cache poisoning	Custom WAF rule possible; global managed rule could potentially break application behavior

WAF - WAF Release - 2026-05-07 - Emergency

Published on: Thu, 07 May 2026 00:00:00 GMT

This emergency release introduces a new rule to detect Next.js App Router middleware and proxy bypass attempts via segment-prefetch routes (CVE-2026-44575).

Key Findings

CVE-2026-44575: Next.js Middleware / Proxy Bypass in App Router Applications via Segment-Prefetch Routes

Successful exploitation allows unauthenticated attackers to bypass middleware or proxy-based authorization checks in affected Next.js App Router applications. This leads to unauthorized access to protected content, potential exposure of sensitive application data, and compromise of application security boundaries.

We strongly recommend upgrading to Next.js 15.5.16 or 16.2.5 (or later) immediately to address the underlying vulnerability. If you cannot upgrade immediately, enforce authorization in the underlying route or page logic instead of relying solely on middleware.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...e77e4a53	N/A	Next.js - Middleware Bypass via Invalid RSC Header - CVE:CVE-2026-44575	N/A	Disabled	This is a new detection.

WAF - WAF Release - 2026-05-04

Published on: Mon, 04 May 2026 00:00:00 GMT

This week's release focuses on new detections to expand coverage across command injection, SQL injection, PHP object injection, remote code execution, and XSS attack vectors.

Key Findings

\$ * Existing rule enhancements have been deployed to improve detection resilience against broad classes of web attacks and strengthen behavioral coverage.

Continuous Rule Improvements

We are continuously refining our managed rules to provide more resilient protection and deeper insights into attack patterns. To ensure an optimal security posture, we recommend consistently monitoring the Security Events dashboard and adjusting rule actions as these enhancements are deployed.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...f0884a68	N/A	XSS, HTML Injection - Object Tag - Body (beta)	Log	Block	This is a new detection. This rule is merged into the original rule "XSS, HTML Injection - Object Tag" (ID: ...0c14e284).
CIS Managed Ruleset	...ff012303	N/A	XSS, HTML Injection - Object Tag - Headers	Log	Block	This is a new detection. The rule previously known as "XSS, HTML Injection - Object Tag - Headers (beta)" is now renamed to "XSS, HTML Injection - Object Tag - Headers".
CIS Managed Ruleset	...16f921d9	N/A	XSS, HTML Injection - Object Tag - URI	Log	Block	This is a new detection. The rule previously known as "XSS, HTML Injection - Object Tag - URI (beta)" is now renamed to "XSS, HTML Injection - Object Tag - URI".

CIS Managed Ruleset	...dc90d21a	N/A	Command Injection - Generic 9 - Body Vector - Beta	N/A	Disabled	This is a new detection. This rule is merged into the original rule "Command Injection - Generic 9 - Body Vector" (ID: ...0677175f)
CIS Managed Ruleset	...f8960375	N/A	Command Injection - Generic 9 - Header Vector - Beta	N/A	Disabled	This is a new detection. This rule is merged into the original rule "Command Injection - Generic 9 - Header Vector" (ID: ...1eb7a999)
CIS Managed Ruleset	...ef47a800	N/A	Command Injection - Generic 9 - URI Vector - Beta	N/A	Disabled	This is a new detection. This rule is merged into the original rule "Command Injection - Generic 9 - URI Vector" (ID: ...97321c6c)
CIS Managed Ruleset	...beebf804	N/A	Command Injection - Sleep - Body	N/A	Disabled	This is a new detection. The rule previously known as "Command Injection

\$ * Sleep" is now renamed to "Command Injection - Sleep - Body".

CIS Managed Ruleset| ...0d257566| N/A| Command Injection - Sleep - Headers| N/A| Disabled| This is a new detection. CIS Managed Ruleset| ...1856fe86| N/A| Command Injection - Sleep - URI| N/A| Disabled| This is a new detection. CIS Managed Ruleset| ...e6e43c37| N/A| Fortinet FortiSandbox - Command Injection - CVE:CVE-2026-39808| Log| Block| This is a new detection. CIS Managed Ruleset| ...88118795| N/A| Remote Code Execution - Common Bash Bypass - Headers| N/A| Disabled| This is a new detection. CIS Managed Ruleset| ...9299d53b| N/A| Remote Code Execution - Common Bash Bypass - URI| N/A| Disabled| This is a new detection. CIS Managed Ruleset| ...b0bf33f9| N/A| Remote Code Execution - Common Bash Bypass - Body - Beta| N/A| Disabled| This is a new detection. This rule is merged into the original rule "Remote Code Execution - Common Bash Bypass Body" (ID: ...efb7e5b9). The rule previously known as "Remote Code Execution - Common Bash Bypass Beta" is now renamed to "Remote Code Execution - Common Bash Bypass Body". CIS Managed Ruleset| ...33bfe8b9| N/A| PHP Object Injection - 2 - Body - Beta| N/A| Disabled| This is a new detection. This rule is merged into the original rule "PHP Object Injection - 2" (ID: ...161aafdc) CIS Managed Ruleset| ...29552387| N/A| PHP Object Injection - 2 - Headers| N/A| Disabled| This is a new detection. CIS Managed Ruleset| ...8104f4c5| N/A| PHP Object Injection - 2 - URI| N/A| Disabled| This is a new detection. CIS Managed Ruleset| ...6a46201e| N/A| SQLi - DROP - 2 - Beta| N/A| Disabled| This is a new detection. This rule is merged into the original rule "SQLi - DROP - 2" (ID: ...48ac2221) CIS Managed Ruleset| ...8b7f85ee| N/A| SQLi - DROP - 2 - Headers| N/A| Disabled| This is a new detection. CIS Managed Ruleset| ...1546b5f0| N/A| SQLi - DROP - 2 - URI| N/A| Disabled| This is a new detection. CIS Managed Ruleset| ...1e053dce| N/A| SmarterMail - Remote Code Execution - CVE:CVE-2026-24423| Log| Block| This is a new detection. CIS Managed Ruleset| ...d0023a36| N/A| SQLi - SELECT Expression - Body| Block| Disabled| Action changed CIS Managed Ruleset| ...26cc211f| N/A| SQLi - String Concatenation - URI| Block| Disabled| Action changed

WAF - WAF Release - 2026-04-30 - Emergency

Published on: Thu, 30 Apr 2026 00:00:00 GMT

This emergency release introduces a new rule to block a cPanel & WHM Authentication Bypass related to CVE-2026-41940.

Key Findings

\$ * CVE-2026-41940: A critical authentication bypass vulnerability in cPanel & WHM allows unauthenticated remote attackers to bypass authentication mechanisms and gain unauthorized administrative access to the web hosting control panel. This vulnerability affects the session validation logic, enabling attackers to craft malicious requests that circumvent normal authentication checks.

Impact

Successful exploitation allows unauthenticated attackers to gain administrative control over affected cPanel & WHM installations. This leads to complete server compromise, potential theft or manipulation of hosted data, and significant service disruption across managed environments.

We strongly recommend applying official vendor patches for cPanel & WHM immediately to address the underlying

vulnerability.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...eb2b9e2f	N/A	cPanel - Auth Bypass - CVE:CVE-2026-41940	N/A	Block	This is a new detection.

WAF - WAF Release - 2026-04-27

Published on: Mon, 27 Apr 2026 00:00:00 GMT

This week's release focuses on new improvements to enhance coverage.

Key Findings

\$ * Existing rule enhancements have been deployed to improve detection resilience against broad classes of web attacks and strengthen behavioral coverage.

Continuous Rule Improvements

We are continuously refining our managed rules to provide more resilient protection and deeper insights into attack patterns. To ensure an optimal security posture, we recommend consistently monitoring the Security Events dashboard and adjusting rule actions as these enhancements are deployed.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...80cec1dd	N/A	PostgreSQL - SQLi - COPY - Beta	Log	Block	This is a new detection. This rule is merged into the original rule "PostgreSQL - SQLi - COPY - Body (ID: ...e7265a4e)". The rule previously known as "PostgreSQL - SQLi - COPY" is now renamed to "PostgreSQL - SQLi - COPY - Body".
CIS Managed Ruleset	...2903de89	N/A	PostgreSQL - SQLi - COPY - Headers	Log	Block	This is a new detection.
CIS Managed Ruleset	...1036cfa6	N/A	PostgreSQL - SQLi - COPY - URI	Log	Block	This is a new detection.
CIS Managed Ruleset	...55ff389e	N/A	SQLi - AND/OR MAKE_SET/ELT - Beta	Log	Block	This is a new detection. This rule is merged into the original rule "SQLi - AND/OR MAKE_SET/ELT - Body" (ID: ...252d3934). The rule previously known as "SQLi - AND/OR MAKE_SET/ELT" is now renamed to "SQLi - AND/OR MAKE_SET/ELT - Body".
CIS Managed Ruleset	...346487f9	N/A	SQLi - AND/OR MAKE_SET/ELT - Headers	Log	Block	This is a new detection.
CIS Managed Ruleset	...1ac6ceca	N/A	SQLi - AND/OR MAKE_SET/ELT - URI	Log	Block	This is a new detection.

CIS Managed Ruleset	...dd471337	N/A	SQLi - Common Patterns - Beta	Log	Block	This is a new detection. This rule is merged into the original rule "SQLi - Common Patterns - Body" (ID: ...cb5d0b9b). The rule previously known as "SQLi - Common Patterns" is now renamed to "SQLi - Common Patterns - Body".
CIS Managed Ruleset	...975c07b7	N/A	SQLi - Common Patterns - Headers	Log	Block	This is a new detection.
CIS Managed Ruleset	...05b1b06b	N/A	SQLi - Common Patterns - URI	Log	Block	This is a new detection.
CIS Managed Ruleset	...dd0ba3c7	N/A	SQLi - Equation - Beta	Log	Block	This is a new detection. This rule is merged into the original rule "SQLi - Equation - Body" (ID: ...c2eb3e7f). The rule previously known as "SQLi - Equation" is now renamed to "SQLi - Equation - Body".
CIS Managed Ruleset	...3d1c2384	N/A	SQLi - Equation - Headers	Log	Block	This is a new detection.
CIS Managed Ruleset	...e1149ea6	N/A	SQLi - Equation - URI	Log	Block	This is a new detection.
CIS Managed Ruleset	...205adbb0	N/A	SQLi - AND/OR Digit Operator Digit - Beta	Log	Block	This is a new detection. This rule is merged into the original rule "SQLi - AND/OR Digit Operator Digit - Body" (ID: ...3893c564). The rule previously known as "SQLi - AND/OR Digit Operator Digit" is now renamed to "SQLi - AND/OR Digit Operator Digit - Body".
CIS Managed Ruleset	...ad2abbaa	N/A	SQLi - AND/OR Digit Operator Digit - Headers	Log	Block	This is a new detection.
CIS Managed Ruleset	...53acbc0d	N/A	SQLi - AND/OR Digit Operator Digit - URI	Log	Block	This is a new detection.
CIS Managed Ruleset	...2b45a97d	N/A	SQLi - Benchmark Function - Beta	Log	Block	This is a new detection. This rule is merged into the original rule "SQLi - Benchmark Function - Body" (ID: ...2ebc44ad). The rule previously known as "SQLi - Benchmark Function" is now renamed to "SQLi - Benchmark Function - Body".
CIS Managed Ruleset	...9889aadc	N/A	SQLi - Benchmark Function - Headers	Log	Block	This is a new detection.

CIS Managed Ruleset	...491b28e9	N/A	SQLi - Benchmark Function - URI	Log	Block	This is a new detection.
CIS Managed Ruleset	...2aa649de	N/A	SQLi - Comparison - Beta	Log	Block	This is a new detection. This rule is merged into the original rule "SQLi - Comparison - Body" (ID: ...e7907480). The rule previously known as "SQLi - Comparison" is now renamed to "SQLi - Comparison - Body".
CIS Managed Ruleset	...39e3e013	N/A	SQLi - Comparison - Headers	Log	Block	This is a new detection.
CIS Managed Ruleset	...f4bdb492	N/A	SQLi - Comparison - URI	Log	Block	This is a new detection.
CIS Managed Ruleset	...a1ff3b34	N/A	SQLi - String Concatenation - Body - Beta	Log	Block	This is a new detection. This rule is merged into the original rule "SQLi - String Concatenation - Headers" (ID: ...2116d2fe).The rule previously known as "SQLi - String Concatenation - Headers" is now renamed to "SQLi - String Concatenation - Body".
CIS Managed Ruleset	...0d0e6c3b	N/A	SQLi - String Concatenation - Headers	Log	Block	This is a new detection.(Former Id was ...846d1940)
CIS Managed Ruleset	...26cc211f	N/A	SQLi - String Concatenation - URI	Log	Block	This is a new detection. (Former Id was ...8fae8c84)
CIS Managed Ruleset	...eacc78ab	N/A	SQLi - SELECT Expression - Beta	Log	Block	This is a new detection. This rule is merged into the original rule "SQLi - SELECT Expression - Body" (ID: ...d0023a36). The rule previously known as "SQLi - SELECT Expression" is now renamed to "SQLi - SELECT Expression - Body".
CIS Managed Ruleset	...630bb223	N/A	SQLi - SELECT Expression - Headers	Log	Block	This is a new detection.
CIS Managed Ruleset	...dcd6efb5	N/A	SQLi - SELECT Expression - URI	Log	Block	This is a new detection.
CIS Managed Ruleset	...18c47cea	N/A	SQLi - ORD and ASCII - Beta	Log	Block	This is a new detection. This rule is merged into the original rule "SQLi - ORD and ASCII- Body" (ID: ...d0d207f9). The rule previously known as "SQLi - ORD and ASCII" is now renamed to "SQLi - ORD and ASCII- Body".

CIS Managed Ruleset	...bdb1618f	N/A	SQLi - ORD and ASCII - URI	Log	Block	This is a new detection.
CIS Managed Ruleset	...1d0906b6	N/A	SQLi - ORD and ASCII - Headers	Log	Block	This is a new detection.
CIS Managed Ruleset	...9fe4eff5	N/A	SQLi - Destructive Operations	Log	Block	This is a new detection.

WAF - WAF Release - 2026-04-21

Published on: Tue, 21 Apr 2026 00:00:00 GMT

This week's release introduces a new detection for a Remote Code Execution (RCE) vulnerability in Apache ActiveMQ (CVE-2026-34197) and an updated signature for Magento 2 - Unrestricted File Upload. Alongside these detections, we are continuing our work on rule refinements to provide deeper security insights for our customers.

Key Findings

\$ * Apache ActiveMQ (CVE-2026-34197): A vulnerability in Apache ActiveMQ allows an unauthenticated, remote attacker to execute arbitrary code. This flaw occurs during the processing of specially crafted network packets, leading to potential full system compromise.

* Magento 2 - Unrestricted File Upload - 2: This is a follow-up enhancement to our existing protections for Magento and Adobe Commerce.

Impact

Successful exploitation of these vulnerabilities could allow unauthenticated attackers to execute arbitrary code or gain full administrative control over affected servers. We strongly recommend applying official vendor patches for Apache ActiveMQ and Magento to address the underlying vulnerabilities.

Continuous Rule Improvements

We are continuously refining our managed rules to provide more resilient protection and deeper insights into attack patterns. To ensure an optimal security posture, we recommend consistently monitoring the Security Events dashboard and adjusting rule actions as these enhancements are deployed.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...ee159e2e	N/A	Command Injection - Generic 8 - uri	Log	Block	This is a new detection. Previous description was "Command Injection - Generic 8 - uri - Beta"
CIS Managed Ruleset	...a15308cf	N/A	Command Injection - Generic 8 - body - Beta	Disabled	Disabled	This is a new detection. This rule is merged into the original rule "Command Injection - Generic 8 - body" (ID: ...413592e2). The rule previously known as "Command Injection - Generic 8" is now renamed to "Command Injection - Generic 8 - body".

CIS Managed Ruleset	...958047ed	N/A	MySQL - SQLi - Executable Comment - Beta	Log	Block	This is a new detection. This rule is merged into the original rule "MySQL - SQLi - Executable Comment - Body" (ID: ...7bd2d8fa) The rule previously known as "MySQL - SQLi - Executable Comment" is now renamed to "MySQL - SQLi - Executable Comment - Body".
CIS Managed Ruleset	...582cc559	N/A	MySQL - SQLi - Executable Comment - Headers	Log	Block	This is a new detection.
CIS Managed Ruleset	...a16639d3	N/A	MySQL - SQLi - Executable Comment - URI	Log	Block	This is a new detection.
CIS Managed Ruleset	...44f24211	N/A	Magento 2 - Unrestricted file upload - 2	Log	Block	This is a new detection.
CIS Managed Ruleset	...bf170a17	N/A	Apache ActiveMQ - Remote Code Execution - CVE:CVE-2026-34197	Log	Block	This is a new detection.
CIS Managed Ruleset	...8c2ac1a7	N/A	SQLi - Sleep Function - Beta	Log	Block	This is a new detection. This rule is merged into the original rule "SQLi - Sleep Function" (ID: ...f77e8d54)
CIS Managed Ruleset	...4dacaeb8	N/A	SQLi - Sleep Function - Headers	Log	Block	This is a new detection.
CIS Managed Ruleset	...ed4c9ece	N/A	SQLi - Sleep Function - URI	Log	Block	This is a new detection.
CIS Managed Ruleset	...1dfa64df	N/A	SQLi - Probing - uri	Log	Block	This is a new detection.
CIS Managed Ruleset	...4c62e2e7	N/A	SQLi - Probing - header	Log	Block	This is a new detection.
CIS Managed Ruleset	...aab28ea1	N/A	SQLi - Probing - body	Disabled	Disabled	This is a new detection. This rule is merged into the original rule "SQLi - Probing" (ID: ...b4026c88)

CIS Managed Ruleset	...20999be0	N/A	SQLi - Probing 2	Disabled	Disabled	This rule had duplicate detection logic and has been deprecated.
CIS Managed Ruleset	...d7aa0008	N/A	SQLi - UNION in MSSQL - Body	Disabled	Disabled	This rule has been renamed to differentiate from "SQLi - UNION in MSSQL" (ID: ...ee5e35fd) and contains updated rule logic.
CIS Managed Ruleset	...a67d8561	N/A	SQLi - UNION - 3	Disabled	Disabled	This rule had duplicate detection logic and has been deprecated.
CIS Managed Ruleset	...0af34bba	N/A	XSS, HTML Injection - Embed Tag - URI	Disabled	Disabled	This is a new detection.
CIS Managed Ruleset	...70282f38	N/A	XSS, HTML Injection - Embed Tag - Headers	Log	Block	This is a new detection.
CIS Managed Ruleset	...23f4d718	N/A	XSS, HTML Injection - IFrame Tag - Src and Srcdoc Attributes - Headers	Log	Disabled	This is a new detection.
CIS Managed Ruleset	...6978def1	N/A	XSS, HTML Injection - Link Tag - Headers	Log	Disabled	This is a new detection.
CIS Managed Ruleset	...ebd81645	N/A	XSS, HTML Injection - Link Tag - URI	Disabled	Disabled	This is a new detection.

WAF - WAF Release - 2026-04-15

Published on: Wed, 15 Apr 2026 00:00:00 GMT

This week's release introduces a new detection for a critical Remote Code Execution (RCE) vulnerability in Mesop (CVE-2026-33057), alongside protections for high-impact vulnerabilities in Cisco Secure Firewall Management Center (CVE-2026-20079) and FortiClient EMS (CVE-2026-21643). Additionally, this release includes an update to our existing React Server DoS coverage to address recently identified resource exhaustion vectors (CVE-2026-23869).

Key Findings

- \$ * Cisco Secure FMC (CVE-2026-20079): A vulnerability in the web-based management interface of Cisco Secure Firewall Management Center (FMC) that allows an unauthenticated, remote attacker to execute arbitrary commands or bypass security filters.
- * FortiClient EMS (CVE-2026-21643): A critical vulnerability in the FortiClient EMS permitting unauthorized access or administrative configuration manipulation via crafted HTTP requests.
- * Mesop (CVE-2026-33057): A vulnerability in the Mesop Python-based UI framework where unauthenticated attackers can execute arbitrary code by sending specially crafted, Base64-encoded payloads in the request body.

Impact

Successful exploitation of these vulnerabilities could allow unauthenticated attackers to execute arbitrary code, gain

administrative control over network management infrastructure, or trigger server-side resource exhaustion. Administrators are strongly encouraged to apply official vendor updates.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...aef9415b	N/A	Cisco Secure FMC - RCE via upgradeReadinessCall - CVE:CVE-2026-20079	Log	Block	This is a new detection.
CIS Managed Ruleset	...ee7be621	N/A	FortiClient EMS - Pre-Auth SQL Injection - CVE:CVE-2026-21643	Log	Block	This is a new detection.
CIS Managed Ruleset	...c953a72b	N/A	Mesop - Remote Code Execution - Base64 Payload - CVE:CVE-2026-33057	Log	Block	This is a new detection.
CIS Managed Ruleset	...50c08f6f	N/A	React Server - DOS - CVE:CVE-2026-23864 - 1 - Beta	Log	Block	This rule has been merged into the original rule "React Server - DOS - CVE:CVE-2026-23864 - 1" (ID: ...61680354)
CIS Managed Ruleset	...ebd81645	N/A	XSS, HTML Injection - Link Tag - URI (beta)	N/A	Disabled	This is a new detection.
CIS Managed Ruleset	...0af34bba	N/A	XSS, HTML Injection - Embed Tag - URI (beta)	N/A	Disabled	This is a new detection.

WAF - Email obfuscation decode script is now non-render-blocking

Published on: Tue, 14 Apr 2026 00:00:00 GMT

The decode script injected by [Email Address Obfuscation](#) now loads with the `defer` attribute. This means the script no longer blocks page rendering. It downloads in parallel with HTML parsing and executes after the document is fully parsed, before the `DOMContentLoaded` event.

This improves page loading performance, contributing to better Core Web Vitals, for all zones with Email Address Obfuscation on. No action is required.

If you have custom JavaScript that depends on email addresses being decoded at a specific point during page load, note that the decode script now executes after HTML parsing completes rather than inline during parsing.

WAF - WAF Release - 2026-04-07

Published on: Tue, 07 Apr 2026 00:00:00 GMT

This week's release introduces new detections for a critical Remote Code Execution (RCE) vulnerability in MCP Server (CVE-2026-23744), alongside targeted protection for an authentication bypass vulnerability in SolarWinds products (CVE-2025-40552). Additionally, this release includes a new generic detection rule designed to identify and block Cross-Site Scripting (XSS) injection attempts leveraging "OnEvent" handlers within HTTP cookies.

Key Findings

\$ * MCP Server (CVE-2026-23744): A vulnerability in the Model Context Protocol (MCP) server implementation where malformed input payloads can trigger a memory corruption state, allowing for arbitrary code execution.

* SolarWinds (CVE-2025-40552): A critical flaw in the authentication module allows unauthenticated attackers to bypass security filters and gain unauthorized access to the management console due to improper identity token validation.

* XSS OnEvents Cookies: This generic rule identifies malicious event handlers (such as onload or onerror) embedded within HTTP cookie values.

Impact

Successful exploitation of the MCP Server and SolarWinds vulnerabilities could allow unauthenticated attackers to execute arbitrary code or gain administrative control, leading to a full system takeover. Additionally, the new generic XSS detection prevents attackers from leveraging browser event handlers in cookies to hijack user sessions or execute malicious scripts.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...0aa410af	N/A	Generic Rules - Command Execution - 5 - Body	Log	Disabled	This is a new detection.
CIS Managed Ruleset	...9131ec2f	N/A	Generic Rules - Command Execution - 5 - Header	Log	Disabled	This is a new detection.
CIS Managed Ruleset	...551eb9e5	N/A	Generic Rules - Command Execution - 5 - URI	Log	Block	This is a new detection.
CIS Managed Ruleset	...d46229eb	N/A	MCP Server - Remote Code Execution - CVE:CVE-2026-23744	Log	Block	This is a new detection.
CIS Managed Ruleset	...a864b9c2	N/A	XSS - OnEvents - Cookies	Log	Block	This is a new detection.
CIS Managed Ruleset	...9712a863	N/A	SQLi - Evasion - Body	Log	Disabled	This is a new detection.
CIS Managed Ruleset	...40732d48	N/A	SQLi - Evasion - Headers	Log	Disabled	This is a new detection.
CIS Managed Ruleset	...e68a99b5	N/A	SQLi - Evasion - URI	Log	Disabled	This is a new detection.
CIS Managed Ruleset	...3e8143d2	N/A	SQLi - LIKE 3 - Body	Log	Disabled	This is a new detection.
CIS Managed Ruleset	...70e7fb97	N/A	SQLi - LIKE 3 - URI	Log	Disabled	This is a new detection.
CIS Managed Ruleset	...4c538bd9	N/A	SQLi - UNION - 2 - Body	Log	Disabled	This is a new detection.

CIS Managed Ruleset	...61c439c9	N/A	SQLi - UNION - 2 - URI	Log	Disabled	This is a new detection.
CIS Managed Ruleset	...cf33ea10	N/A	SolarWinds - Auth Bypass - CVE:CVE-2025-40552	Log	Block	This is a new detection.

WAF - WAF Release - 2026-03-30

Published on: Mon, 30 Mar 2026 00:00:00 GMT

This week's release introduces new detections for a critical authentication bypass vulnerability in Fortinet products (CVE-2025-59718), alongside three new generic detection rules designed to identify and block HTTP Parameter Pollution attempts. Additionally, this release includes targeted protection for a high-impact unrestricted file upload vulnerability in Magento and Adobe Commerce.

Key Findings

\$ * CVE-2025-59718: An improper cryptographic signature verification vulnerability in Fortinet FortiOS, FortiProxy, and FortiSwitchManager. This may allow an unauthenticated attacker to bypass the FortiCloud SSO login authentication using a maliciously crafted SAML message, if that feature is enabled on the device.

* Magento 2 - Unrestricted File Upload: A critical flaw in Magento and Adobe Commerce allows unauthenticated attackers to bypass security checks and upload malicious files to the server, potentially leading to Remote Code Execution (RCE).

Impact

Successful exploitation of the Fortinet and Magento vulnerabilities could allow unauthenticated attackers to gain administrative control or deploy webshells, leading to complete server compromise and data theft.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...2f7f95e9	N/A	Generic Rules - Parameter Pollution - Body	Log	Disabled	This is a new detection.
CIS Managed Ruleset	...319731a4	N/A	Generic Rules - Parameter Pollution - Header - Form	Log	Disabled	
						This is a new detection.
CIS Managed Ruleset	...def262dd	N/A	Generic Rules - Parameter Pollution - URI	Log	Disabled	
						This is a new detection.
CIS Managed Ruleset	...70a36147	N/A	Magento 2 - Unrestricted file upload	Log	Block	This is a new detection.
CIS Managed Ruleset	...2ffcca9f	N/A	Fortinet FortiCloud SSO - Authentication Bypass - CVE:CVE-2025-59718	Log	Block	This is a new detection.

WAF - WAF Release - 2026-03-23

Published on: Mon, 23 Mar 2026 00:00:00 GMT

This week's release focuses on new improvements to enhance coverage.

Key Findings

\$ * Existing rule enhancements have been deployed to improve detection resilience against broad classes of web attacks and strengthen behavioral coverage.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...97321c6c	N/A	Command Injection - Generic 9 - URI Vector	Log	Disabled	This is a new detection.
CIS Managed Ruleset	...1eb7a999	N/A	Command Injection - Generic 9 - Header Vector	Log	Disabled	This is a new detection.
CIS Managed Ruleset	...0677175f	N/A	Command Injection - Generic 9 - Body Vector	Log	Disabled	This is a new detection.
CIS Managed Ruleset	...479da68f	N/A	PHP, vBulletin, jQuery File Upload - Code Injection, Dangerous File Upload - CVE:CVE-2018-9206, CVE:CVE-2019-17132 (beta)	Log	Block	This rule has been merged into the original rule "PHP, vBulletin, jQuery File Upload - Code Injection, Dangerous File Upload - CVE:CVE-2018-9206, CVE:CVE-2019-17132" (ID: ...824b817c)

WAF - WAF Release - 2026-03-12 - Emergency

Published on: Thu, 12 Mar 2026 00:00:00 GMT

This week's release introduces new detections for vulnerabilities in Ivanti Endpoint Manager Mobile (CVE-2026-1281 and CVE-2026-1340), alongside a new generic detection rule designed to identify and block Cross-Site Scripting (XSS) injection attempts within the **Content-Security-Policy** (CSP) HTTP request header.

Key Findings

\$ * CVE-2026-1281 & CVE-2026-1340: Ivanti Endpoint Manager Mobile processes HTTP requests through Apache RewriteMap directives that pass user-controlled input to Bash scripts (`/mi/bin/map-appstore-url` and `/mi/bin/map-aft-store-url`). Bash scripts do not sanitize user input and are vulnerable to shell arithmetic expansion thereby allowing attackers to achieve unauthenticated remote code execution.

* Generic XSS in CSP Header: This rule identifies malicious payloads embedded within the request's `Content-Security-Policy` header. It specifically targets scenarios where web frameworks or applications trust and extract values directly from the CSP header in the incoming request without sufficient validation. Attackers can provide crafted header values to inject scripts or malicious directives that are subsequently

processed by the server.

Impact

Successful exploitation of Ivanti EPMM vulnerability allows unauthenticated remote code execution and generic XSS in CSP header allows attackers to inject malicious scripts during page rendering. In environments using server-side caching, this poisoned XSS content can subsequently be cached and automatically served to all visitors.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...796ea2f6	N/A	Ivanti EPMM - Code Injection - CVE:CVE-2026-1281 CVE:CVE-2026-1340	Log	Block	This is a new detection.
CIS Managed Ruleset	...ee964a8c	N/A	Anomaly:Header:Content-Security-Policy	N/A	Block	This is a new detection.

WAF - WAF Release - 2026-03-02

Published on: Mon, 02 Mar 2026 00:00:00 GMT

This week's release introduces new detections for vulnerabilities in SmarterTools SmarterMail (CVE-2025-52691 and CVE-2026-23760), alongside improvements to an existing Command Injection (nslookup) detection to enhance coverage.

Key Findings

\$ * CVE-2025-52691: SmarterTools SmarterMail mail server is vulnerable to Arbitrary File Upload, allowing an unauthenticated attacker to upload files to any location on the mail server, potentially enabling remote code execution.
* CVE-2026-23760: SmarterTools SmarterMail versions prior to build 9511 contain an authentication bypass vulnerability in the password reset API permitting unauthenticated to reset system administrator accounts failing to verify existing password or reset token.

Impact

Successful exploitation of these SmarterMail vulnerabilities could lead to full system compromise or unauthorized administrative access to mail servers. Administrators are strongly encouraged to apply vendor patches without delay.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...966ec6b1	N/A	SmarterMail - Arbitrary File Upload - CVE-2025-52691	Log	Block	This is a new detection.
CIS Managed Ruleset	...ee964a8c	N/A	SmarterMail - Authentication Bypass - CVE-2026-23760	Log	Block	This is a new detection.
CIS Managed Ruleset	...75b64d99	N/A	Command Injection - Nslookup - Beta	Log	Block	This rule is merged into the original rule "Command Injection - Nslookup" (ID: ...b090ba9a)

WAF - WAF Release - 2026-02-16

Published on: Mon, 16 Feb 2026 00:00:00 GMT

This week’s release introduces new detections for CVE-2025-68645 and CVE-2025-31125.

Key Findings

\$ * CVE-2025-68645: A Local File Inclusion (LFI) vulnerability in the Webmail Classic UI of Zimbra Collaboration Suite (ZCS) 10.0 and 10.1 allows unauthenticated remote attackers to craft requests to the ``/h/rest`` endpoint, improperly influence internal dispatching, and include arbitrary files from the WebRoot directory.

* CVE-2025-31125: Vite, the JavaScript frontend tooling framework, exposes content of non-allowed files via ``?inline&import`` when its development server is network-exposed, enabling unauthorized attackers to read arbitrary files and potentially leak sensitive information.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...833761f7	N/A	Zimbra - Local File Inclusion - CVE:CVE-2025-68645	Log	Block	This is a new detection.
CIS Managed Ruleset	...950ed8c8	N/A	Vite - WASM Import Path Traversal - CVE:CVE-2025-31125	Log	Block	This is a new detection.

WAF - WAF Release - 2026-02-10

Published on: Tue, 10 Feb 2026 00:00:00 GMT

This week’s release changes the rule action from BLOCK to Disabled for Anomaly:Header:User-Agent - Fake Google Bot.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...6aa0bef8	N/A	Anomaly:Header:User-Agent - Fake Google Bot	Enabled	Disabled	We are changing the action for this rule from BLOCK to Disabled

WAF - WAF Release - 2026-02-02

Published on: Mon, 02 Feb 2026 00:00:00 GMT

This week’s release introduces new detections for CVE-2025-64459 and CVE-2025-24893.

Key Findings

\$ * CVE-2025-64459: Django versions prior to 5.1.14, 5.2.8, and 4.2.26 are vulnerable to SQL injection via crafted dictionaries passed to QuerySet methods and the ``Q()`` class.

* CVE-2025-24893: XWiki allows unauthenticated remote code execution through crafted requests to the SolrSearch endpoint, affecting the entire installation.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...30698ff3	N/A	XWiki - Remote Code Execution - CVE:CVE-2025-24893 2	Log	Block	This is a new detection.
CIS Managed Ruleset	...da8ba7e6	N/A	Django SQLI - CVE:CVE-2025-64459	Log	Block	This is a new detection.

CIS Managed Ruleset	...8d667511	N/A	NoSQL, MongoDB - SQLi - Comparison - 2	Block	Block	Rule metadata description refined. Detection unchanged.
---------------------	-------------	-----	--	-------	-------	---

WAF - WAF Release - 2026-01-26

Published on: Mon, 26 Jan 2026 00:00:00 GMT

This week’s release introduces new detections for denial-of-service attempts targeting React CVE-2026-23864 ([https://www.cve.org/CVERecord?id=CVE-2026-23864 ↗](https://www.cve.org/CVERecord?id=CVE-2026-23864)).

Key Findings

\$ * CVE-2026-23864 (<<https://www.cve.org/CVERecord?id=CVE-2026-23864>>[↗](<https://www.cve.org/CVERecord?id=CVE-2026-23864>)) affects `react-server-dom-parcel`, `react-server-dom-turbopack`, and `react-server-dom-webpack` packages.
* Attackers can send crafted HTTP requests to Server Function endpoints, causing server crashes, out-of-memory exceptions, or excessive CPU usage.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...61680354	N/A	React Server - DOS - CVE:CVE-2026-23864 - 1	N/A	Block	This is a new detection.
CIS Managed Ruleset	...dcdffcf8	N/A	React Server - DOS - CVE:CVE-2026-23864 - 2	N/A	Block	This is a new detection.
CIS Managed Ruleset	...349edbc6	N/A	React Server - DOS - CVE:CVE-2026-23864 - 3	N/A	Block	This is a new detection.

WAF - WAF Release - 2026-01-20

Published on: Tue, 20 Jan 2026 00:00:00 GMT

This week's release focuses on improvements to existing detections to enhance coverage.

Key Findings

\$ * Existing rule enhancements have been deployed to improve detection resilience against SQL injection.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...68d90c8f	N/A	SQLi - Comment - Beta	Log	Block	This rule is merged into the original rule "SQLi - Comment" (ID: ...6d8d8fe4)
CIS Managed Ruleset	...faa045cf	N/A	SQLi - Comparison - Beta	Log	Block	This rule is merged into the original rule "SQLi - Comparison" (ID: ...e7907480)

WAF - WAF Release - 2026-01-15

Published on: Thu, 15 Jan 2026 00:00:00 GMT

This week's release focuses on improvements to existing detections to enhance coverage.

Key Findings

\$ * Existing rule enhancements have been deployed to improve detection resilience against SQL Injection.						
Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...ad7dad3e	N/A	SQLi - String Function - Beta	Log	Block	This rule is merged into the original rule "SQLi - String Function" (ID: ...d32b798c)
CIS Managed Ruleset	...9e553ad3	N/A	SQLi - Sub Query - Beta	Log	Block	
This rule is merged into the original rule "SQLi - Sub Query" (ID: ...743e66b1)						

WAF - WAF Release - 2026-01-12

Published on: Mon, 12 Jan 2026 00:00:00 GMT

This week's release focuses on improvements to existing detections to enhance coverage.

Key Findings

\$ * Existing rule enhancements have been deployed to improve detection resilience against SQL Injection.						
Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...48a1841a	N/A	SQLi - AND/OR MAKE_SET/ELT - Beta	Log	Block	This rule is merged into the original rule "SQLi - AND/OR MAKE_SET/ELT" (ID: ...252d3934)
CIS Managed Ruleset	...9e553ad3	N/A	SQLi - Benchmark Function - Beta	Log	Block	
This rule is merged into the original rule "SQLi - Benchmark Function" (ID: ...2ebc44ad)						

WAF - WAF Release - 2025-12-18

Published on: Thu, 18 Dec 2025 00:00:00 GMT

This week's release focuses on improvements to existing detections to enhance coverage.

Key Findings

\$ * Existing rule enhancements have been deployed to improve detection resilience against broad classes of web attacks and strengthen behavioral coverage.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...be5ec20c	N/A	Atlassian Confluence - Code Injection - CVE:CVE-2021-26084 - Beta	Log	Block	This rule is merged into the original rule "Atlassian Confluence - Code Injection - CVE:CVE-2021-26084" (ID: ...69e0b97a)
CIS Managed Ruleset	...0d9206e3	N/A	PostgreSQL - SQLi - Copy - Beta	Log	Block	
			This rule is merged into the original rule "PostgreSQL - SQLi - COPY" (ID: ...e7265a4e)			
CIS Managed Ruleset	...0cd00ba7	N/A	Generic Rules - Command Execution - Body	Log	Disabled	
			This is a new detection.			
CIS Managed Ruleset	...cd679ad4	N/A	Generic Rules - Command Execution - Header	Log	Disabled	This is a new detection.
CIS Managed Ruleset	...fd181fb3	N/A	Generic Rules - Command Execution - URI	Log	Disabled	This is a new detection.
CIS Managed Ruleset	...7a95bc3a	N/A	SQLi - Tautology - URI - Beta	Log	Block	This rule is merged into the original rule "SQLi - Tautology - URI" (ID: ...b3de2e0a)

CIS Managed Ruleset	...432ac90d	N/A	SQLi - WaitFor Function - Beta	Log	Block	This rule is merged into the original rule "SQLi - WaitFor Function" (ID: ...d5faba59)
CIS Managed Ruleset	...596c741e	N/A	SQLi - AND/OR Digit Operator Digit 2 - Beta	Log	Block	This rule is merged into the original rule "SQLi - AND/OR Digit Operator Digit" (ID: ...88d80772)
CIS Managed Ruleset	...03b2f3fe	N/A	SQLi - Equation 2 - Beta	Log	Block	This rule is merged into the original rule "SQLi - Equation" (ID: ...a72a6b3a)

WAF - WAF Release - 2025-12-11 - Emergency

Published on: Thu, 11 Dec 2025 00:00:00 GMT

This emergency release introduces rules for CVE-2025-55183 and CVE-2025-55184, targeting server-side function exposure and resource-exhaustion patterns, respectively.

Key Findings

Added coverage for Leaking Server Functions (CVE-2025-55183) and React Function DoS detection (CVE-2025-55184).

Impact

These updates strengthen protection for server-function abuse techniques (CVE-2025-55183, CVE-2025-55184) that may expose internal logic or disrupt application availability.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...fefb4e9b	N/A	React - Leaking Server Functions - CVE:CVE-2025-55183	N/A	Block	This was labeled as Generic - Server Function Source Code Exposure.
CIS Free Ruleset	...251e86aa	N/A	React - Leaking Server Functions - CVE:CVE-2025-55183	N/A	Block	This was labeled as Generic - Server Function Source Code Exposure.
CIS Managed Ruleset	...102ec699	N/A	React - DoS - CVE:CVE-2025-55184	N/A	Disabled	This was labeled as Generic – Server Function Resource Exhaustion.

WAF - WAF Release - 2025-12-10 - Emergency

Published on: Wed, 10 Dec 2025 00:00:00 GMT

This additional week's emergency release introduces improvements to our existing rule for React – Remote Code Execution – CVE-2025-55182 - 2, along with two new generic detections covering server-side function exposure and resource-exhaustion patterns.

Key Findings

Enhanced detection logic for React – RCE – CVE-2025-55182, added Generic – Server Function Source Code Exposure, and added Generic – Server Function Resource Exhaustion.

Impact

These updates strengthen protection against React RCE exploitation attempts and broaden coverage for common server-function abuse techniques that may expose internal logic or disrupt application availability.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...15fce168	N/A	React - Remote Code Execution - CVE:CVE-2025-55182 - 2	N/A	Block	This is an improved detection.
CIS Free Ruleset	...74746aff	N/A	React - Remote Code Execution - CVE:CVE-2025-55182 - 2	N/A	Block	This is an improved detection.
CIS Managed Ruleset	...fefb4e9b	N/A	Generic - Server Function Source Code Exposure	N/A	Block	This is a new detection.
CIS Free Ruleset	...251e86aa	N/A	Generic - Server Function Source Code Exposure	N/A	Block	This is a new detection.
CIS Managed Ruleset	...102ec699	N/A	Generic - Server Function Resource Exhaustion	N/A	Disabled	This is a new detection.

WAF - Increased WAF payload limit for all plans

Published on: Fri, 05 Dec 2025 00:00:00 GMT

CIS WAF now inspects request-payload size of up to 1 MB across all plans to enhance our detection capabilities for React RCE (CVE-2025-55182).

Key Findings

React payloads commonly have a default maximum size of 1 MB. CIS WAF previously inspected up to 128 KB on Enterprise plans, with even lower limits on other plans.

Update: We later reinstated the maximum request-payload size the CIS WAF inspects. Refer to [Updating the WAF maximum payload values](#) for details.

WAF - Updating the WAF maximum payload values

Published on: Fri, 05 Dec 2025 00:00:00 GMT

We are reinstating the maximum request-payload size the CIS WAF inspects, with WAF on Enterprise zones inspecting up to 128 KB.

Key Findings

On [December 5, 2025](#), we initially attempted to increase the maximum WAF payload limit to 1 MB across all plans. However, an automatic rollout for all customers proved impractical because the increase led to a surge in false positives for existing managed rules.

This issue was particularly notable within the CIS Managed Ruleset and the CIS OWASP Core Ruleset, impacting customer traffic.

Impact

Customers on paid plans can increase the limit to 1 MB for any of their zones by contacting CIS Support. Free zones are already protected up to 1 MB and do not require any action.

WAF - WAF Release - 2025-12-03 - Emergency

Published on: Wed, 03 Dec 2025 00:00:00 GMT

The WAF rule deployed yesterday to block unsafe deserialization-based RCE has been updated. The rule description now reads “React – RCE – CVE-2025-55182”, explicitly mapping to the recently disclosed React Server Components vulnerability. Detection logic remains unchanged.

Key Findings

Rule description updated to reference React – RCE – CVE-2025-55182 while retaining existing unsafe-deserialization detection.

Impact

Improved classification and traceability with no change to coverage against remote code execution attempts.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...5fb92fba	N/A	React - RCE - CVE:CVE-2025-55182	N/A	Block	Rule metadata description changed. Detection unchanged.
CIS Free Ruleset	...99702280	N/A	React - RCE - CVE:CVE-2025-55182	N/A	Block	Rule metadata description changed. Detection unchanged.

WAF - WAF Release - 2025-12-02 - Emergency

Published on: Tue, 02 Dec 2025 00:00:00 GMT

This week's emergency release introduces a new rule to block a critical RCE vulnerability in widely-used web frameworks through unsafe deserialization patterns.

Key Findings

New WAF rule deployed for RCE Generic Framework to block malicious POST requests containing unsafe deserialization patterns. If successfully exploited, this vulnerability allows attackers with network access via HTTP to execute arbitrary code remotely.

Impact

\$ * Successful exploitation allows unauthenticated attackers to execute arbitrary code remotely through crafted serialization payloads, enabling complete system compromise, data exfiltration, and potential lateral movement within affected environments.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...5fb92fba	N/A	RCE Generic - Framework	N/A	Block	This is a new detection.
CIS Free Ruleset	...99702280	N/A	RCE Generic - Framework	N/A	Block	This is a new detection.

WAF - WAF Release - 2025-12-01

Published on: Mon, 01 Dec 2025 00:00:00 GMT

This week’s release introduces new detections for remote code execution attempts targeting Monsta FTP (CVE-2025-34299), alongside improvements to an existing XSS detection to enhance coverage.

Key Findings

\$ * CVE-2025-34299 is a critical remote code execution flaw in Monsta FTP, arising from improper handling of user-supplied parameters within the file-handling interface. Certain builds allow crafted requests to bypass sanitization and reach backend PHP functions that execute arbitrary commands. Attackers can send manipulated parameters through the web panel to trigger command execution within the application’s runtime environment.

Impact

If exploited, the vulnerability enables full remote command execution on the underlying server, allowing takeover of the hosting environment, unauthorized file access, and potential lateral movement. As the flaw can be triggered without authentication on exposed Monsta FTP instances, it represents a severe risk for publicly reachable deployments.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...a4fcc8a8	N/A	Monsta FTP - Remote Code Execution - CVE:CVE-2025-34299	Log	Block	This is a new detection
CIS Managed Ruleset	...b7492846	N/A	XSS - JS Context Escape - Beta	Log	Block	This rule is merged into the original rule "XSS - JS Context Escape" (ID: ...7a3769d3)

WAF - WAF Release - 2025-11-24

Published on: Mon, 24 Nov 2025 00:00:00 GMT

This week highlights enhancements to detection signatures improving coverage for vulnerabilities in FortiWeb, linked to CVE-2025-64446, alongside new detection logic expanding protection against PHP Wrapper Injection techniques.

Key Findings

This vulnerability enables an unauthenticated attacker to bypass access controls by abusing the `CGIINFO` header. The latest update strengthens detection logic to ensure a reliable identification of crafted requests attempting to exploit this flaw.

Impact

\$ * FortiWeb (CVE-2025-64446): Exploitation allows a remote unauthenticated adversary to circumvent authentication mechanisms by sending a manipulated ``CGIINFO`` header to FortiWeb’s backend CGI handler. Successful exploitation grants unintended access to restricted administrative functionality, potentially enabling configuration tampering or system-level actions.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...4e2e1a2e	N/A	FortiWeb - Authentication Bypass via CGIINFO Header - CVE:CVE-2025-64446	Log	Block	This is a new detection

CIS Managed Ruleset	...b6c44ed5	N/A	PHP Wrapper Injection - Body - Beta	Log	Disabled	This rule has been merged into the original rule "PHP Wrapper Injection - Body" (ID:...1a3e521e)
CIS Managed Ruleset	...900f4015	N/A	PHP Wrapper Injection - URI - Beta	Log	Disabled	This rule has been merged into the original rule "PHP Wrapper Injection - URI" (ID:...8f76bd74)

WAF - WAF Release - 2025-11-21

Published on: Fri, 21 Nov 2025 00:00:00 GMT

This week’s release introduces a critical detection for CVE-2025-61757, a vulnerability in the Oracle Identity Manager REST WebServices component.

Key Findings

This flaw allows unauthenticated attackers with network access over HTTP to fully compromise the Identity Manager, potentially leading to a complete takeover.

Impact

Oracle Identity Manager (CVE-2025-61757): Exploitation could allow an unauthenticated remote attacker to bypass security checks by sending specially crafted requests to the application's message processor. This enables the creation of arbitrary employee accounts, which can be leveraged to modify system configurations and achieve full system compromise.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...39fdb7e	N/A	Oracle Identity Manager - Pre-Auth RCE - CVE:CVE-2025-61757	N/A	Block	This is a new detection.

WAF - WAF Release - 2025-11-17

Published on: Mon, 17 Nov 2025 00:00:00 GMT

This week highlights enhancements to detection signatures improving coverage for vulnerabilities in DELMIA Apriso, linked to CVE-2025-6205.

Key Findings

This vulnerability allows unauthenticated attackers to gain privileged access to the application. The latest update provides enhanced detection logic for resilient protection against exploitation attempts.

Impact

\$ * DELMIA Apriso (CVE-2025-6205): Exploitation could allow an unauthenticated remote attacker to bypass security checks by sending specially crafted requests to the application's message processor. This enables the creation of arbitrary employee accounts, which can be leveraged to modify system configurations and achieve full system compromise.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
---------	---------	----------------	-------------	-----------------	------------	----------

CIS Managed Ruleset	...d256f4bc	N/A	DELMIA Apriso - Auth Bypass - CVE:CVE-2025-6205	Log	Block	This is a new detection.
CIS Managed Ruleset	...1a3e521e	N/A	PHP Wrapper Injection - Body	N/A	Disabled	Rule metadata description refined. Detection unchanged.
CIS Managed Ruleset	...8f76bd74	N/A	PHP Wrapper Injection - URI	N/A	Disabled	Rule metadata description refined. Detection unchanged.

WAF - WAF Release - 2025-11-10

Published on: Mon, 10 Nov 2025 00:00:00 GMT

This week’s release introduces new detections for Prototype Pollution across three common vectors: URI, Body, and Header/Form.

Key Findings

\$ * These attacks can affect both API and web applications by altering normal behavior or bypassing security controls.

Impact

Exploitation may allow attackers to change internal logic or cause unexpected behavior in applications using JavaScript or Node.js frameworks. Developers should sanitize input keys and avoid merging untrusted data structures.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...606285e6	N/A	Generic Rules - Prototype Pollution - URI	Log	Disabled	This is a new detection
CIS Managed Ruleset	...4f59ff26	N/A	Generic Rules - Prototype Pollution - Body	Log	Disabled	This is a new detection
CIS Managed Ruleset	...7efbeb39	N/A	Generic Rules - Prototype Pollution - Header - Form	Log	Disabled	This is a new detection

WAF - WAF Release - 2025-11-05 - Emergency

Published on: Wed, 05 Nov 2025 00:00:00 GMT

This week’s emergency release introduces a new detection signature that enhances coverage for a critical vulnerability in the React Native Metro Development Server, tracked as CVE-2025-11953.

Key Findings

The Metro Development Server exposes an HTTP endpoint that is vulnerable to OS command injection (CWE-78). An unauthenticated network attacker can send a crafted request to this endpoint and execute arbitrary commands on the host running Metro. The vulnerability affects Metro/cli-server-api builds used by React Native Community CLI in pre-patch development releases.

Impact

Successful exploitation of CVE-2025-11953 may result in remote command execution on developer workstations or CI/build agents, leading to credential and secret exposure, source tampering, and potential lateral movement into internal

networks. Administrators and developers are strongly advised to apply the vendor's patches and restrict Metro’s network exposure to reduce this risk.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...c8e30c5b	N/A	React Native Metro - Command Injection - CVE:CVE-2025-11953	N/A	Block	This is a New Detection

WAF - WAF Release - 2025-11-03

Published on: Mon, 03 Nov 2025 00:00:00 GMT

This week highlights enhancements to detection signatures improving coverage for vulnerabilities in Adobe Commerce and Magento Open Source, linked to CVE-2025-54236.

Key Findings

This vulnerability allows unauthenticated attackers to take over customer accounts through the Commerce REST API and, in certain configurations, may lead to remote code execution. The latest update provides enhanced detection logic for resilient protection against exploitation attempts.

Impact

\$ * Adobe Commerce (CVE-2025-54236): Exploitation may allow attackers to hijack sessions, execute arbitrary commands, steal data, and disrupt storefronts, resulting in confidentiality and integrity risks for merchants. Administrators are strongly encouraged to apply vendor patches without delay.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...cb6d5fe5	100774C	Adobe Commerce - Remote Code Execution - CVE:CVE-2025-54236	Log	Block	This is an improved detection.

WAF - WAF Release - 2025-10-30 - Emergency

Published on: Thu, 30 Oct 2025 00:00:00 GMT

This week’s release introduces a new detection signature that enhances coverage for a critical vulnerability in Oracle E-Business Suite, tracked as CVE-2025-61884.

Key Findings

The flaw is easily exploitable and allows an unauthenticated attacker with network access to compromise Oracle Configurator, which can grant access to sensitive resources and configuration data. The affected versions include 12.2.3 through 12.2.14.

Impact

Successful exploitation of CVE-2025-61884 may result in unauthorized access to critical business data or full exposure of information accessible through Oracle Configurator. Administrators are strongly advised to apply vendor's patches and recommended mitigations to reduce this exposure.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...8827402f	N/A	Oracle E-Business Suite - SSRF - CVE:CVE-2025-61884	N/A	Block	This is a New Detection

WAF - WAF Release - 2025-10-24 - Emergency

Published on: Fri, 24 Oct 2025 00:00:00 GMT

This week’s release introduces a new detection signature that enhances coverage for a critical vulnerability in Windows Server Update Services (WSUS), tracked as CVE-2025-59287.

Key Findings

The vulnerability allows unauthenticated attackers to potentially achieve remote code execution. The updated detection logic strengthens defenses by improving resilience against exploitation attempts targeting this flaw.

Impact

Successful exploitation of CVE-2025-59287 could enable attackers to hijack sessions, execute arbitrary commands, exfiltrate sensitive data, and disrupt storefront operations. These actions pose significant confidentiality and integrity risks to affected environments. Administrators should apply vendor patches immediately to mitigate exposure.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...bd72ba08	N/A	Windows Server - Deserialization - CVE:CVE-2025-59287	N/A	Block	This is a New Detection

WAF - WAF Release - 2025-10-23 - Emergency

Published on: Thu, 23 Oct 2025 00:00:00 GMT

This week highlights enhancements to detection signatures improving coverage for vulnerabilities in Adobe Commerce and Magento Open Source, linked to CVE-2025-54236.

Key Findings

This vulnerability allows unauthenticated attackers to take over customer accounts through the Commerce REST API and, in certain configurations, may lead to remote code execution. The latest update enhances detection logic to provide more resilient protection against exploitation attempts.

Impact

Adobe Commerce (CVE-2025-54236): Exploitation may allow attackers to hijack sessions, execute arbitrary commands, steal data, and disrupt storefronts, resulting in confidentiality and integrity risks for merchants. Administrators are strongly encouraged to apply vendor patches without delay.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...c6ef59a1	N/A	Adobe Commerce - Remote Code Execution - CVE:CVE-2025-54236	N/A	Block	This is a New Detection

WAF - WAF Release - 2025-10-20

Published on: Mon, 20 Oct 2025 00:00:00 GMT

This week’s update introduces an enhanced rule that expands detection coverage for a critical vulnerability in Oracle E-Business Suite. It also improves an existing rule to provide more reliable coverage in request processing.

Key Findings

New WAF rule deployed for Oracle E-Business Suite (CVE-2025-61882) to block unauthenticated attacker's network access via HTTP to compromise Oracle Concurrent Processing. If successfully exploited, this vulnerability may result in remote code execution.

Impact

\$ * Successful exploitation of CVE-2025-61882 allows unauthenticated attackers to execute arbitrary code remotely by chaining multiple weaknesses, enabling lateral movement into internal services, data exfiltration, and large-scale extortionware deployment within Oracle E-Business Suite environments.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...2b4101ab	100598A	Remote Code Execution - Common Bash Bypass - Beta	Log	Block	This rule is merged into the original rule "Remote Code Execution - Common Bash Bypass" (ID: ...50cec478)
CIS Managed Ruleset	...a1118614	100916A	Oracle E-Business Suite - Remote Code Execution - CVE:CVE-2025-61882 - 2	Log	Block	This is a New Detection
CIS Managed Ruleset	...c22b51d3	N/A	HTTP Truncated	N/A	Disabled	This is a New Detection

WAF - New detections released for WAF managed rulesets

Published on: Fri, 17 Oct 2025 00:00:00 GMT

This week we introduced several new detections across CIS Managed Rulesets, expanding coverage for high-impact vulnerability classes such as SSRF, SQLi, SSTI, Reverse Shell attempts, and Prototype Pollution. These rules aim to improve protection against attacker-controlled payloads that exploit misconfigurations or unvalidated input in web applications.

Key Findings

New detections added for multiple exploit categories:

SSRF (Server-Side Request Forgery) — new rules targeting both local and cloud metadata abuse patterns (Beta).

SQL Injection (SQLi) — rules for common patterns, sleep/time-based injections, and string/wait function exploitation across headers and URIs.

SSTI (Server-Side Template Injection) — arithmetic-based probe detections introduced across URI, header, and body fields.

Reverse Shell and XXE payloads — enhanced heuristics for command execution and XML external entity misuse.

Prototype Pollution — new Beta rule identifying common JSON payload structures used in object prototype poisoning.

PHP Wrapper Injection and HTTP Parameter Pollution detections — to catch path traversal and multi-parameter manipulation attempts.

Anomaly Header Checks — detecting CRLF injection attempts in header names.

Impact

These updates help detect multi-vector payloads that blend SSRF + RCE or SQLi + SSTI attacks, especially in cloud-hosted applications with exposed metadata endpoints or unsafe template rendering.

Prototype Pollution and HTTP parameter pollution rules address emerging JavaScript supply-chain exploitation patterns increasingly seen in real-world incidents.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
---------	---------	----------------	-------------	-----------------	------------	----------

CIS Managed Ruleset	...589f2a1d	N/A	Anomaly:Header - name - CR, LF	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...132fab7e	N/A	Generic Rules - Reverse Shell - Body	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...1a027008	N/A	Generic Rules - Reverse Shell - Header	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...958d0486	N/A	Generic Rules - Reverse Shell - URI	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...8e0cf7ad	N/A	Generic Rules - XXE - Body	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...bf8aab5e	N/A	Generic Rules - SQLi - Common Patterns - Header URI	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...2e466337	N/A	Generic Rules - SQLi - Sleep Function - Header URI	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...b686ab47	N/A	Generic Rules - SQLi - String Function - Header URI	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...b0633709	N/A	Generic Rules - SQLi - WaitFor Function - Header URI	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...01a076eb	N/A	SSRF - Local - Beta	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...743a63ec	N/A	SSRF - Local - 2 - Beta	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...c2e84e2d	N/A	SSRF - Cloud - Beta	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...ab8af26f	N/A	SSRF - Cloud - 2 - Beta	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...e6e8dc5b	N/A	SSTI - Arithmetic Probe - URI	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...2550d794	N/A	SSTI - Arithmetic Probe - Header	N/A	Disabled	This is a New Detection

CIS Managed Ruleset	...659d12a6	N/A	SSTI - Arithmetic Probe - Body	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...1a3e521e	N/A	PHP Wrapper Injection	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...8f76bd74	N/A	PHP Wrapper Injection	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...091e296d	N/A	HTTP parameter pollution	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...e34214ef	N/A	Prototype Pollution - Common Payloads - Beta	N/A	Disabled	This is a New Detection

WAF - WAF Release - 2025-10-13

Published on: Mon, 13 Oct 2025 00:00:00 GMT

This week’s highlights include a new JinJava rule targeting a sandbox-bypass flaw that could allow malicious template input to escape execution controls. The rule improves detection for unsafe template rendering paths.

Key Findings

New WAF rule deployed for JinJava (CVE-2025-59340) to block a sandbox bypass in the template engine that permits attacker-controlled type construction and arbitrary class instantiation; in vulnerable environments this can escalate to remote code execution and full server compromise.

Impact

\$ * CVE-2025-59340 — Exploitation enables attacker-supplied type descriptors / Jackson `ObjectMapper` abuse, allowing arbitrary class loading, file/URL access (LFI/SSRF primitives) and, with suitable gadget chains, potential remote code execution and system compromise.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...c04bab5f	100892	JinJava - SSTI - CVE:CVE-2025-59340	Log	Block	This is a New Detection

WAF - WAF Release - 2025-10-07 - Emergency

Published on: Tue, 07 Oct 2025 00:00:00 GMT

This week highlights multiple critical Cisco vulnerabilities (CVE-2025-20363, CVE-2025-20333, CVE-2025-20362). This flaw stems from improper input validation in HTTP(S) requests. An authenticated VPN user could send crafted requests to execute code as root, potentially compromising the device. The initial two rules were made available on September 28, with a third rule added today, October 7, for more robust protection.

\$ * Cisco (CVE-2025-20333, CVE-2025-20362, CVE-2025-20363): Multiple vulnerabilities that could allow attackers to exploit unsafe deserialization and input validation flaws. Successful exploitation may result in arbitrary code execution, privilege escalation, or command injection on affected systems.

Impact

Cisco (CVE-2025-20333, CVE-2025-20362, CVE-2025-20363): Exploitation enables attackers to escalate privileges or achieve remote code execution via command injection. Administrators are strongly advised to apply vendor updates immediately.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...3a4d1bd6	100788B	Cisco Secure Firewall Adaptive Security Appliance - Remote Code Execution - CVE:CVE-2025-20333, CVE:CVE-2025-20362, CVE:CVE-2025-20363	N/A	Block	This is a New Detection

WAF - WAF Release - 2025-10-06

Published on: Mon, 06 Oct 2025 00:00:00 GMT

This week’s highlights prioritise an emergency Oracle E-Business Suite RCE rule deployed to block active, high-impact exploitation. Also addressed are high-severity Chaos Mesh controller command-injection flaws that enable unauthenticated in-cluster RCE and potential cluster compromise, plus a form-data multipart boundary issue that permits HTTP Parameter Pollution (HPP). Two new generic SQLi detections were added to catch inline-comment obfuscation and information disclosure techniques.

Key Findings

- \$ * New emergency rule released for Oracle E-Business Suite (CVE-2025-61882) addressing an actively exploited remote code execution vulnerability in core business application modules. Immediate mitigation deployed to protect enterprise workloads.
- * Chaos Mesh (CVE-2025-59358,CVE-2025-59359,CVE-2025-59360,CVE-2025-59361): A GraphQL debug endpoint on the Chaos Controller Manager is exposed without authentication; several controller mutations (`cleanTcs`, `killProcesses`, `cleanIptables`) are vulnerable to OS command injection.
- * Form-Data (CVE-2025-7783): Attackers who can observe `Math.random()` outputs and control request fields in form-data may exploit this flaw to perform HTTP parameter pollution, leading to request tampering or data manipulation.
- * Two new generic SQLi detections added to enhance baseline coverage against inline-comment obfuscation and information disclosure attempts.

Impact

- \$ * CVE-2025-61882 — Oracle E-Business Suite remote code execution (emergency detection): attacker-controlled input can yield full system compromise, data exfiltration, and operational outage; immediate blocking enforced.
- * CVE-2025-59358 / CVE-2025-59359 / CVE-2025-59360 / CVE-2025-59361 — Unauthenticated command-injection in Chaos Mesh controllers allowing remote code execution, cluster compromise, and service disruption (high availability risk).
- * CVE-2025-7783 — Predictable multipart boundaries in form-data enabling HTTP Parameter Pollution; results include request tampering, parameter overwrite, and downstream data integrity loss.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...8650f52f	100882	Chaos Mesh - Missing Authentication - CVE:CVE-2025-59358	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...2b8c3680	100883	Chaos Mesh - Command Injection - CVE:CVE-2025-59359	Log	Block	This is a New Detection
CIS Managed Ruleset	...ef859a04	100884	Chaos Mesh - Command Injection - CVE:CVE-2025-59361	Log	Block	This is a New Detection
CIS Managed Ruleset	...961f26a7	100886	Form-Data - Parameter Pollution - CVE:CVE-2025-7783	Log	Block	This is a New Detection

CIS Managed Ruleset	...26a4074c	100888	Chaos Mesh - Command Injection - CVE:CVE-2025-59360	Log	Block	This is a New Detection
CIS Managed Ruleset	...31101b2f	100916	Oracle E-Business Suite - Remote Code Execution - CVE:CVE-2025-61882	N/A	Block	This is a New Detection
CIS Managed Ruleset	...29aa43c3	100917	Generic Rules - SQLi - Inline Comment Injection	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...73c10b6f	100918	Generic Rules - SQLi - Information Disclosure	N/A	Disabled	This is a New Detection

WAF - WAF Release - 2025-10-03

Published on: Fri, 03 Oct 2025 00:00:00 GMT

Managed Ruleset Updated

This update introduces 21 new detections in the CIS Managed Ruleset (all currently set to Disabled mode to preserve remediation logic and allow quick activation if needed). The rules cover a broad spectrum of threats - SQL injection techniques, command and code injection, information disclosure of common files, URL anomalies, and cross-site scripting.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...d61fac74	100902	Generic Rules - Command Execution - 2	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...514aeeb8	100908	Generic Rules - Command Execution - 3	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...8d46a6f4	100910	Generic Rules - Command Execution - 4	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...1bd0a329	100915	Generic Rules - Command Execution - 5	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...5e51450a	100899	Generic Rules - Content-Type Abuse	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...7996012f	100914	Generic Rules - Content-Type Injection	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...93209312	100911	Generic Rules - Cookie Header Injection	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...0f373b3f	100905	Generic Rules - NoSQL Injection	N/A	Disabled	This is a New Detection

CIS Managed Ruleset	...78a0ed04	100913	Generic Rules - NoSQL Injection - 2	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...5d649624	100907	Generic Rules - Parameter Pollution	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...fd1c674e	100906	Generic Rules - PHP Object Injection	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...34c88168	100904	Generic Rules - Prototype Pollution	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...3ab43f7e	100897	Generic Rules - Prototype Pollution 2	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...0d94ee22	100903	Generic Rules - Reverse Shell	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...d5add8e3	100909	Generic Rules - Reverse Shell - 2	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...565c78b0	100898	Generic Rules - SSJI NoSQL	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...12b837a0	100896	Generic Rules - SSRF	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...11c4fb00	100895	Generic Rules - Template Injection	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...d3ed0123	100895A	Generic Rules - Template Injection - 2	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...7501a1d9	100912	Generic Rules - XXE	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...dc55cdb6	100900	Relative Paths - Anomaly Headers	N/A	Disabled	This is a New Detection

WAF - WAF Release - 2025-09-29

Published on: Mon, 29 Sep 2025 00:00:00 GMT

This week highlights four important vendor- and component-specific issues: an authentication bypass in SimpleHelp (CVE-2024-57727), an information-disclosure flaw in Flowise Cloud (CVE-2025-58434), an SSRF in the WordPress plugin Ditty (CVE-2025-8085), and a directory-traversal bug in Vite (CVE-2025-30208). These are paired with improvements to our generic detection coverage (SQLi, SSRF) to raise the baseline and reduce noisy gaps.

Key Findings

\$ * SimpleHelp (CVE-2024-57727): Authentication bypass in SimpleHelp that can allow unauthorized access to management interfaces or sessions.

* Flowise Cloud (CVE-2025-58434): Information-disclosure vulnerability in Flowise Cloud that may expose sensitive configuration or user data to unauthenticated or low-privileged actors.

* WordPress:Plugin: Ditty (CVE-2025-8085): SSRF in the Ditty WordPress plugin enabling server-side requests that could reach internal services or cloud metadata endpoints.

* Vite (CVE-2025-30208): Directory-traversal vulnerability in Vite allowing access to filesystem paths outside the intended web root.

Impact

These vulnerabilities allow attackers to gain access, escalate privileges, or execute actions that were previously unavailable:

\$ * SimpleHelp (CVE-2024-57727): An authentication bypass that can let unauthenticated attackers access management interfaces or hijack sessions — enabling lateral movement, credential theft, or privilege escalation within affected environments.

* Flowise Cloud (CVE-2025-58434): Information-disclosure flaw that can expose sensitive configuration, tokens, or user data; leaked secrets may be chained into account takeover or privileged access to backend services.

* WordPress:Plugin: Ditty (CVE-2025-8085): SSRF that enables server-side requests to internal services or cloud metadata endpoints, potentially allowing attackers to retrieve credentials or reach otherwise inaccessible infrastructure, leading to privilege escalation or cloud resource compromise.

* Vite (CVE-2025-30208): Directory-traversal vulnerability that can expose filesystem contents outside the web root (configuration files, keys, source code), which attackers can use to escalate privileges or further compromise systems.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...8c2e30fb	100717	SimpleHelp - Auth Bypass - CVE:CVE-2024- 57727	Log	Block	This rule is merged to 100717 in legacy WAF and ...958094d3 in new WAF
CIS Managed Ruleset	...d58b886b	100775	Flowise Cloud - Information Disclosure - CVE:CVE-2025-58434	Log	Block	This is a New Detection
CIS Managed Ruleset	...9bce1ff4	100881	WordPress:Plugin:Ditty - SSRF - CVE:CVE-2025- 8085	Log	Block	This is a New Detection
CIS Managed Ruleset	...ddc329dd	100887	Vite - Directory Traversal - CVE:CVE-2025-30208	Log	Block	This is a New Detection

WAF - WAF Release - 2025-09-28 - Emergency

Published on: Sun, 28 Sep 2025 00:00:00 GMT

This week highlights multiple critical Cisco vulnerabilities (CVE-2025-20363, CVE-2025-20333, CVE-2025-20362). This flaw stems from improper input validation in HTTP(S) requests. An authenticated VPN user could send crafted requests to execute code as root, potentially compromising the device.

Key Findings

\$ * Cisco (CVE-2025-20333, CVE-2025-20362, CVE-2025-20363): Multiple vulnerabilities that could allow attackers to exploit unsafe deserialization and input validation flaws. Successful exploitation may result in arbitrary code execution, privilege escalation, or command injection on affected systems.

Impact

Cisco (CVE-2025-20333, CVE-2025-20362, CVE-2025-20363): Exploitation enables attackers to escalate privileges or achieve remote code execution via command injection.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...9ee0ab84	100788	Cisco Secure Firewall Adaptive Security Appliance - Remote Code Execution - CVE:CVE-2025-20333, CVE:CVE-2025-20362, CVE:CVE-2025-20363	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...d30f768e	100788A	Cisco Secure Firewall Adaptive Security Appliance - Remote Code Execution - CVE:CVE-2025-20333, CVE:CVE-2025-20362, CVE:CVE-2025-20363	N/A	Disabled	This is a New Detection

WAF - WAF Release - 2025-09-26

Published on: Fri, 26 Sep 2025 00:00:00 GMT

Managed Ruleset Updated

This update introduces 11 new detections in the CIS Managed Ruleset (all currently set to Disabled mode to preserve remediation logic and allow quick activation if needed). The rules cover a broad spectrum of threats - SQL injection techniques, command and code injection, information disclosure of common files, URL anomalies, and cross-site scripting.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...a67d8561	100859A	SQLi - UNION - 3	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...4de80468	100889	Command Injection - Generic 9	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...f2be3ddf	100890	Information Disclosure - Common Files - 2	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...80a252a8	100891	Anomaly:URL - Relative Paths	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...7e7d3865	100894	XSS - Inline Function	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...3792565c	100895	XSS - DOM	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...42978e38	100896	SQLi - MSSQL Length Enumeration	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...3ab43f7e	100897	Generic Rules - Code Injection - 3	N/A	Disabled	This is a New Detection

CIS Managed Ruleset	...c1686741	100898	SQLi - Evasion	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...20999be0	100899	SQLi - Probing 2	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...b4026c88	100900	SQLi - Probing	N/A	Disabled	This is a New Detection

WAF - WAF Release - 2025-09-24 - Emergency

Published on: Wed, 24 Sep 2025 00:00:00 GMT

This week highlights a critical vendor-specific vulnerability: a deserialization flaw in the License Servlet of Fortra’s GoAnywhere MFT. By forging a license response signature, an attacker can trigger deserialization of arbitrary objects, potentially leading to command injection.

Key Findings

\$ * GoAnywhere MFT (CVE-2025-10035): Deserialization vulnerability in the License Servlet that allows attackers with a forged license response signature to deserialize arbitrary objects, potentially resulting in command injection.

Impact

GoAnywhere MFT (CVE-2025-10035): Exploitation enables attackers to escalate privileges or achieve remote code execution via command injection.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...e08b39f3	100787	Fortra GoAnywhere - Auth Bypass - CVE:CVE-2025-10035	N/A	Block	This is a New Detection

WAF - WAF Release - 2025-09-22

Published on: Mon, 22 Sep 2025 00:00:00 GMT

This week emphasizes two critical vendor-specific vulnerabilities: a full elevation-of-privilege in Microsoft Azure Networking (CVE-2025-54914) and a server-side template injection (SSTI) leading to remote code execution (RCE) in Skyvern (CVE-2025-49619). These are complemented by enhancements in generic detections (SQLi, SSRF) to improve baseline coverage.

Key Findings

\$ * Azure (CVE-2025-54914): Vulnerability in Azure Networking allowing elevation of privileges.

* Skyvern (CVE-2025-49619): Skyvern ≤ 0.1.85 has a server-side template injection (SSTI) vulnerability in its Prompt field (workflow blocks) via Jinja2. Authenticated users with low privileges can get remote code execution (blind).

* Generic SQLi / SSRF improvements: Expanded rule coverage to detect obfuscated SQL injection patterns and SSRF across host, local, and cloud contexts.

Impact

These vulnerabilities allow attackers to escalate privileges or execute code under conditions where previously they could not:

\$ * Azure CVE-2025-54914 enables an attacker from the network with no credentials to gain high-level access within Azure Networking; could lead to full compromise of networking components.

* Skyvern CVE-2025-49619 allows authenticated users with minimal privilege to exploit SSTI for remote code execution, undermining

isolation of workflow components.

* The improvements for SQLi and SSRF reduce risk from common injection and request-based attacks.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...6a135cbf	100146	SSRF - Host - 2	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...57035abf	100146B	SSRF - Local - 2	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...bbe18d50	100146C	SSRF - Cloud - 2	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...956c1961	100714	Azure - Auth Bypass - CVE:CVE-2025-54914	Log	Block	This is a New Detection
CIS Managed Ruleset	...c5ced231	100758	Skyvern - Remote Code Execution - CVE:CVE-2025-49619	Log	Block	This is a New Detection
CIS Managed Ruleset	...84a619a1	100773	Next.js - SSRF	Log	Block	This is a New Detection
CIS Managed Ruleset	...983ff2dd	100774	Adobe Commerce - Remote Code Execution - CVE:CVE-2025-54236	Log	Block	This is a New Detection
CIS Managed Ruleset	...0380a1a6	100800_BETA	SQLi - Obfuscated Boolean - Beta	Log	Block	This rule has been merged into the original rule (ID: ...5563445f)

WAF - WAF Release - 2025-09-15

Published on: Mon, 15 Sep 2025 00:00:00 GMT

This week's update

This week's focus highlights newly disclosed vulnerabilities in DevOps tooling, data visualization platforms, and enterprise CMS solutions. These issues include sensitive information disclosure and remote code execution, putting organizations at risk of credential leakage, unauthorized access, and full system compromise.

Key Findings

- \$ * Argo CD (CVE-2025-55190): Exposure of sensitive information could allow attackers to access credential data stored in configurations, potentially leading to compromise of Kubernetes workloads and secrets.
- * DataEase (CVE-2025-57773): Insufficient input validation enables JNDI injection and insecure deserialization, resulting in remote code execution (RCE). Successful exploitation grants attackers control over the application server.
- * Sitecore (CVE-2025-53694): A sensitive information disclosure flaw allows unauthorized access to confidential information stored in Sitecore deployments, raising the risk of data breaches and privilege escalation.

Impact

These vulnerabilities expose organizations to serious risks, including credential theft, unauthorized access, and full system compromise. Argo CD's flaw may expose Kubernetes secrets, DataEase exploitation could give attackers remote execution capabilities, and Sitecore's disclosure issue increases the likelihood of sensitive data leakage and business impact.

Administrators are strongly advised to apply vendor patches immediately, rotate exposed credentials, and review access controls to mitigate these risks.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...2ee2085f	100646	Argo CD - Information Disclosure - CVE:CVE-2025-55190s	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...f5e20788	100874	DataEase - JNDI injection - CVE:CVE-2025-57773	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...005a12fd	100880	Sitecore - Information Disclosure - CVE:CVE-2025-53694	Log	Block	This is a New Detection

WAF - WAF Release - 2025-09-08

Published on: Mon, 08 Sep 2025 00:00:00 GMT

This week's update

This week’s focus highlights newly disclosed vulnerabilities in web frameworks, enterprise applications, and widely deployed CMS plugins. The vulnerabilities include SSRF, authentication bypass, arbitrary file upload, and remote code execution (RCE), exposing organizations to high-impact risks such as unauthorized access, system compromise, and potential data exposure. In addition, security rule enhancements have been deployed to cover general command injection and server-side injection attacks, further strengthening protections.

Key Findings

- \$ * Next.js (CVE-2025-57822): Improper handling of redirects in custom middleware can lead to server-side request forgery (SSRF) when user-supplied headers are forwarded. Attackers could exploit this to access internal services or cloud metadata endpoints. The issue has been resolved in versions 14.2.32 and 15.4.7. Developers using custom middleware should upgrade and verify proper redirect handling in `next()` calls.
- * ScriptCase (CVE-2025-47227, CVE-2025-47228): In the Production Environment extension in Netmake ScriptCase through 9.12.006 (23), two vulnerabilities allow attackers to reset admin accounts and execute system commands, potentially leading to full compromise of affected deployments.
- * Sar2HTML (CVE-2025-34030): In Sar2HTML version 3.2.2 and earlier, insufficient input sanitization of the plot parameter allows remote, unauthenticated attackers to execute arbitrary system commands. Exploitation could compromise the underlying server and its data.
- * Zhiyuan OA (CVE-2025-34040): An arbitrary file upload vulnerability exists in the Zhiyuan OA platform. Improper validation in the `wpsAssistServlet` interface allows unauthenticated attackers to upload crafted files via path traversal, which can be executed on the web server, leading to remote code execution.
- * WordPress:Plugin:InfiniteWP Client (CVE-2020-8772): A vulnerability in the InfiniteWP Client plugin allows attackers to perform restricted actions and gain administrative control of connected WordPress sites.

Impact

These vulnerabilities could allow attackers to gain unauthorized access, execute malicious code, or take full control of affected systems. The Next.js SSRF flaw may expose internal services or cloud metadata endpoints to attackers. Exploitations of ScriptCase and Sar2HTML could result in remote code execution, administrative takeover, and full server compromise. In Zhiyuan OA, the arbitrary file upload vulnerability allows attackers to execute malicious code on the web server, potentially exposing sensitive data and applications. The authentication bypass in WordPress InfiniteWP Client enables attackers to gain administrative access, risking data exposure and unauthorized control of connected sites.

Administrators are strongly advised to apply vendor patches immediately, remove unsupported software, and review authentication and access controls to mitigate these risks.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...963d7afc	100007D	Command Injection - Common Attack Commands Args	Log	Block	This rule has been merged into the original rule "Command Injection - Common Attack Commands" (ID: ...28345b9b) for New WAF customers only.
CIS Managed Ruleset	...8230a75b	100617	Next.js - SSRF - CVE:CVE-2025-57822	Log	Block	This is a New Detection
CIS Managed Ruleset	...a22dabf1	100659_BETA	Common Payloads for Server-Side Template Injection - Beta	Log	Block	This rule is merged into the original rule "Common Payloads for Server-Side Template Injection" (ID: ...a28a42c4)
CIS Managed Ruleset	...b416b7ca	100824B	CrushFTP - Remote Code Execution - CVE:CVE-2025-54309 - 3	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...5db1fa6b	100848	ScriptCase - Auth Bypass - CVE:CVE-2025-47227	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...2c62d330	100849	ScriptCase - Command Injection - CVE:CVE-2025-47228	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...ef971afd	100872	WordPress:Plugin:InfiniteWP Client - Missing Authorization - CVE:CVE-2020-8772	Log	Block	This is a New Detection
CIS Managed Ruleset	...bab19b0b	100873	Sar2HTML - Command Injection - CVE:CVE-2025-34030	Log	Block	This is a New Detection
CIS Managed Ruleset	...f24c0fbe	100875	Zhiyuan OA - Remote Code Execution - CVE:CVE-2025-34040	Log	Block	This is a New Detection

WAF - WAF Release - 2025-09-04 - Emergency

Published on: Thu, 04 Sep 2025 00:00:00 GMT

This week's update

This week, new critical vulnerabilities were disclosed in Sitecore’s Sitecore Experience Manager (XM), Sitecore Experience Platform (XP), specifically versions 9.0 through 9.3, and 10.0 through 10.4. These flaws are caused by unsafe data deserialization and code reflection, leaving affected systems at high risk of exploitation.

Key Findings

\$ * CVE-2025-53690: Remote Code Execution through Insecure Deserialization
* CVE-2025-53691: Remote Code Execution through Insecure Deserialization
* CVE-2025-53693: HTML Cache Poisoning through Unsafe Reflections

Impact

Exploitation could allow attackers to execute arbitrary code remotely on the affected system and conduct cache poisoning attacks, potentially leading to further compromise. Applying the latest vendor-released solution without delay is strongly recommended.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...0ee2c15e	100878	Sitecore - Remote Code Execution - CVE:CVE-2025-53691	N/A	Block	This is a new detection
CIS Managed Ruleset	...7c5b669c	100631	Sitecore - Cache Poisoning - CVE:CVE-2025-53693	N/A	Block	This is a new detection
CIS Managed Ruleset	...6c410240	100879	Sitecore - Remote Code Execution - CVE:CVE-2025-53690	N/A	Block	This is a new detection

WAF - WAF Release - 2025-09-01

Published on: Mon, 01 Sep 2025 00:00:00 GMT

This week's update

This week, a critical vulnerability was disclosed in Fortinet FortiWeb (versions 7.6.3 and below, versions 7.4.7 and below, versions 7.2.10 and below, and versions 7.0.10 and below), linked to improper parameter handling that could allow unauthorized access.

Key Findings

\$ * Fortinet FortiWeb (CVE-2025-52970): A vulnerability may allow an unauthenticated remote attacker with access to non-public information to log in as any existing user on the device via a specially crafted request.

Impact

Exploitation could allow an unauthenticated attacker to impersonate any existing user on the device, potentially enabling them to modify system settings or exfiltrate sensitive information, posing a serious security risk. Upgrading to the latest vendor-released version is strongly recommended.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...c49b7cf8	100586	Fortinet FortiWeb - Auth Bypass - CVE:CVE-2025-52970	Log	Disabled	This is a New Detection

CIS Managed Ruleset	...790c9dde	100136C	XSS - JavaScript - Headers and Body	N/A	N/A	Rule metadata description refined. Detection unchanged.
---------------------	-------------	---------	-------------------------------------	-----	-----	---

WAF - WAF Release - 2025-08-29 - Emergency

Published on: Fri, 29 Aug 2025 00:00:00 GMT

This week's update

This week, new critical vulnerabilities were disclosed in Next.js’s image optimization functionality, exposing a broad range of production environments to risks of data exposure and cache manipulation.

Key Findings

\$ * CVE-2025-55173: Arbitrary file download from the server via image optimization.
* CVE-2025-57752: Cache poisoning leading to unauthorized data disclosure.

Impact

Exploitation could expose sensitive files, leak user or backend data, and undermine application trust. Given Next.js’s wide use, immediate patching and cache hardening are strongly advised.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...9ff4bfe3	100613	Next.js - Dangerous File Download - CVE:CVE-2025-55173	N/A	Block	This is a new detection
CIS Managed Ruleset	...69b9ea7d	100616	Next.js - Information Disclosure - CVE:CVE-2025-57752	N/A	Block	This is a new detection

WAF - WAF Release - 2025-08-25

Published on: Mon, 25 Aug 2025 00:00:00 GMT

This week's update

This week, critical vulnerabilities were disclosed that impact widely used open-source infrastructure, creating high-risk scenarios for code execution and operational disruption.

Key Findings

\$ * Apache HTTP Server – Code Execution (CVE-2024-38474): A flaw in Apache HTTP Server allows attackers to achieve remote code execution, enabling full compromise of affected servers. This vulnerability threatens the confidentiality, integrity, and availability of critical web services.
* Laravel (CVE-2024-55661): A security flaw in Laravel introduces the potential for remote code execution under specific conditions. Exploitation could provide attackers with unauthorized access to application logic and sensitive backend data.

Impact

These vulnerabilities pose severe risks to enterprise environments and open-source ecosystems. Remote code execution enables attackers to gain deep system access, steal data, disrupt services, and establish persistent footholds for broader intrusions. Given the widespread deployment of Apache HTTP Server and Laravel in production systems, timely patching and mitigation are critical.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...28050359	100822_BETA	WordPress:Plugin:WPBookit - Remote Code Execution - CVE:CVE-2025-6058	N/A	Disabled	This was merged in to the original rule "WordPress:Plugin:WPBo - Remote Code Execution CVE:CVE-2025-6058" (ID ...194f7b2d)
CIS Managed Ruleset	...3bdcdbad	100831	Apache HTTP Server - Code Execution - CVE:CVE-2024-38474	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...02eaac5b	100846	Laravel - Remote Code Execution - CVE:CVE-2024-55661	Log	Disabled	This is a New Detection

WAF - WAF Release - 2025-08-22

Published on: Fri, 22 Aug 2025 00:00:00 GMT

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...5fcca5c8	100850	Command Injection - Generic 2	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...744305c4	100851	Remote Code Execution - Java Deserialization	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...2b083459	100852	Command Injection - Generic 3	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...efb7e5b9	100853	Remote Code Execution - Common Bash Bypass Beta	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...78513ad7	100854	XSS - Generic JavaScript	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...e9a5daac	100855	Command Injection - Generic 4	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...480f6093	100856	PHP Object Injection	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...d4ae0a33	100857	Generic - Parameter Fuzzing	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...1121ee45	100858	Code Injection - Generic 4	N/A	Disabled	This is a New Detection

CIS Managed Ruleset	...20de01e3	100859	SQLi - UNION - 2	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...c0177e21	100860	Command Injection - Generic 5	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...85f4d7b3	100861	Command Execution - Generic	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...3fa8ee7f	100862	GraphQL Injection - 2	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...c7a41d4b	100863	Command Injection - Generic 6	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...65e3c165	100864	Code Injection - Generic 2	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...161aafdc	100865	PHP Object Injection - 2	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...1cc3c3f8	100866	SQLi - LIKE 2	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...48ac2221	100867	SQLi - DROP - 2	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...1f4eec13	100868	Code Injection - Generic 3	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...2755f99e	100869	Command Injection - Generic 7	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...413592e2	100870	Command Injection - Generic 8	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...d2dd41b5	100871	SQLi - LIKE 3	N/A	Disabled	This is a New Detection

WAF - WAF Release - 2025-08-18

Published on: Mon, 18 Aug 2025 00:00:00 GMT

This week's update

This week, a series of critical vulnerabilities were discovered impacting core enterprise and open-source infrastructure. These flaws present a range of risks, providing attackers with distinct pathways for remote code execution, methods to breach internal network boundaries, and opportunities for critical data exposure and operational disruption.

Key Findings

\$ * SonicWall SMA (CVE-2025-32819, CVE-2025-32820, CVE-2025-32821): A remote authenticated attacker with SSLVPN user privileges can bypass path traversal protections. These vulnerabilities enable a attacker to bypass security checks to read, modify, or delete arbitrary files. An attacker with administrative privileges can escalate this further, using a command injection flaw to upload malicious files, which could ultimately force the appliance to reboot to its factory default settings.

* Ms-Swift Project (CVE-2025-50460): An unsafe deserialization vulnerability exists in the Ms-Swift project's handling of YAML configuration files. If an attacker can control the content of a configuration file passed to the application, they can embed a malicious payload that will execute arbitrary code and it can be executed during deserialization.

* Apache Druid (CVE-2023-25194): This vulnerability in Apache Druid allows an attacker to cause the server to connect to a malicious LDAP server. By sending a specially crafted LDAP response, the attacker can trigger an unrestricted deserialization of untrusted data. If specific "gadgets" (classes that can be abused) are present in the server's classpath, this can be escalated to achieve Remote Code Execution (RCE).

* Tenda AC8v4 (CVE-2025-51087, CVE-2025-51088): Vulnerabilities allow an authenticated attacker to trigger a stack-based buffer overflow. By sending malformed arguments in a request to specific endpoints, an attacker can crash the device or potentially achieve arbitrary code execution.

* Open WebUI (CVE-2024-7959): This vulnerability allows a user to change the OpenAI URL endpoint to an arbitrary internal network address without proper validation. This flaw can be exploited to access internal services or cloud metadata endpoints, potentially leading to remote command execution if the attacker can retrieve instance secrets or access sensitive internal APIs.

* BentoML (CVE-2025-54381): The vulnerability exists in the serialization/deserialization handlers for multipart form data and JSON requests, which automatically download files from user-provided URLs without proper validation of internal network addresses. This allows attackers to fetch from unintended internal services, including cloud metadata and localhost.

* Adobe Experience Manager Forms (CVE-2025-54254): An Improper Restriction of XML External Entity Reference ('XXE') vulnerability that could lead to arbitrary file system read in Adobe AEM (≤6.5.23).

Impact

These vulnerabilities affect core infrastructure, from network security appliances like SonicWall to data platforms such as Apache Druid and ML frameworks like BentoML. The code execution and deserialization flaws are particularly severe, offering deep system access that allows attackers to steal data, disrupt services, and establish a foothold for broader intrusions. Simultaneously, SSRF and XXE vulnerabilities undermine network boundaries, exposing sensitive internal data and creating pathways for lateral movement. Beyond data-centric threats, flaws in edge devices like the Tenda router introduce the tangible risk of operational disruption, highlighting a multi-faceted threat to the security and stability of key enterprise systems.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...418d9a3b	100574	SonicWall SMA - Remote Code Execution - CVE:CVE-2025-32819, CVE:CVE-2025-32820, CVE:CVE-2025-32821	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...1e6fefdb	100576	Ms-Swift Project - Remote Code Execution - CVE:CVE-2025-50460	Log	Block	This is a New Detection
CIS Managed Ruleset	...08ac45b3	100585	Apache Druid - Remote Code Execution - CVE:CVE-2023-25194	Log	Block	This is a New Detection
CIS Managed Ruleset	...e4855472	100834	Tenda AC8v4 - Auth Bypass - CVE:CVE-2025-51087, CVE:CVE-2025-51088	Log	Block	This is a New Detection
CIS Managed Ruleset	...022ab542	100835	Open WebUI - SSRF - CVE:CVE-2024-7959	Log	Block	This is a New Detection

CIS Managed Ruleset	...6339f132	100837	SQLi - OOB	Log	Block	This is a New Detection
CIS Managed Ruleset	...b83d2625	100841	BentoML - SSRF - CVE:CVE-2025-54381	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...739180d2	100841A	BentoML - SSRF - CVE:CVE-2025-54381 - 2	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...6ab910c2	100841B	BentoML - SSRF - CVE:CVE-2025-54381 - 3	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...2197ec51	100845	Adobe Experience Manager Forms - XSS - CVE:CVE-2025-54254	Log	Block	This is a New Detection
CIS Managed Ruleset	...96f34ce3	100845A	Adobe Experience Manager Forms - XSS - CVE:CVE-2025-54254 - 2	Log	Block	This is a New Detection

WAF - WAF Release - 2025-08-11

Published on: Mon, 11 Aug 2025 00:00:00 GMT

This week's update focuses on a wide range of enterprise software, from network infrastructure and security platforms to content management systems and development frameworks. Flaws include unsafe deserialization, OS command injection, SSRF, authentication bypass, and arbitrary file upload — many of which allow unauthenticated remote code execution. Notable risks include Cisco Identity Services Engine and Ivanti EPMM, where successful exploitation could grant attackers full administrative control of core network infrastructure and popular web services such as WordPress, SharePoint, and Ingress-Nginx, where security bypasses and arbitrary file uploads could lead to complete site or server compromise.

Key Findings

- \$ * Cisco Identity Services Engine (CVE-2025-20281): Insufficient input validation in a specific API of Cisco Identity Services Engine (ISE) and ISE-PIC allows an unauthenticated, remote attacker to execute arbitrary code with root privileges on an affected device.
- * Wazuh Server (CVE-2025-24016): An unsafe deserialization vulnerability in Wazuh Server (versions 4.4.0 to 4.9.0) allows for remote code execution and privilege escalation. By injecting unsanitized data, an attacker can trigger an exception to execute arbitrary code on the server.
- * CrushFTP (CVE-2025-54309): A flaw in AS2 validation within CrushFTP allows remote attackers to gain administrative access via HTTPS on systems not using the DMZ proxy feature. This flaw can lead to unauthorized file access and potential system compromise.
- * Kentico Xperience CMS (CVE-2025-2747, CVE-2025-2748): Vulnerabilities in Kentico Xperience CMS could enable cross-site scripting (XSS), allowing attackers to inject malicious scripts into web pages. Additionally, a flaw could allow unauthenticated attackers to bypass the Staging Sync Server's authentication, potentially leading to administrative control over the CMS.
- * Node.js (CVE-2025-27210): An incomplete fix for a previous vulnerability (CVE-2025-23084) in Node.js affects the `path.join()` API method on Windows systems. The vulnerability can be triggered using reserved Windows device names such as `CON`, `PRN`, or `AUX`.
- * WordPress:Plugin:Simple File List (CVE-2025-34085, CVE-2020-36847): This vulnerability in the Simple File List plugin for WordPress allows an unauthenticated remote attacker to upload arbitrary files to a vulnerable site. This can be exploited to achieve remote code execution on the server.

(Note: CVE-2025-34085 has been rejected as a duplicate.)

- \$ * GeoServer (CVE-2024-29198): A Server-Side Request Forgery (SSRF) vulnerability exists in GeoServer's Demo request endpoint, which can be exploited where the Proxy Base URL has not been configured.
- * Ivanti EPMM (CVE-2025-6771): An OS command injection vulnerability in Ivanti Endpoint Manager Mobile (EPMM) before versions 12.5.0.2, 12.4.0.3, and 12.3.0.3 allows a remote, authenticated attacker with high privileges to execute arbitrary code.

* Microsoft SharePoint (CVE-2024-38018): This is a remote code execution vulnerability affecting Microsoft SharePoint Server.

* Manager-IO (CVE-2025-54122): A critical unauthenticated full read Server-Side Request Forgery (SSRF) vulnerability is present in the proxy handler of both Manager Desktop and Server editions up to version 25.7.18.2519. This allows an unauthenticated attacker to bypass network isolation and access internal services.

* Ingress-Nginx (CVE-2025-1974): A vulnerability in the Ingress-Nginx controller for Kubernetes allows an attacker to bypass access control rules. An unauthenticated attacker with access to the pod network can achieve arbitrary code execution in the context of the ingress-nginx controller.

* PaperCut NG/MF (CVE-2023-2533): A Cross-Site Request Forgery (CSRF) vulnerability has been identified in PaperCut NG/MF. Under specific conditions, an attacker could exploit this to alter security settings or execute arbitrary code if they can deceive an administrator with an active login session into clicking a malicious link.

* SonicWall SMA (CVE-2025-40598): This vulnerability could allow an unauthenticated attacker to bypass security controls. This allows a remote, unauthenticated attacker to potentially execute arbitrary JavaScript code.

* WordPress (CVE-2025-5394): The "Alone – Charity Multipurpose Non-profit WordPress Theme" for WordPress is vulnerable to arbitrary file uploads. A missing capability check allows unauthenticated attackers to upload ZIP files containing webshells disguised as plugins, leading to remote code execution.

Impact

These vulnerabilities span a broad range of enterprise technologies, including network access control systems, monitoring platforms, web servers, CMS platforms, cloud services, and collaboration tools. Exploitation techniques range from remote code execution and command injection to authentication bypass, SQL injection, path traversal, and configuration weaknesses.

A critical flaw in perimeter devices like Ivanti EPMM or SonicWall SMA could allow an unauthenticated attacker to gain remote code execution, completely breaching the primary network defense. A separate vulnerability within Cisco's Identity Services Engine could then be exploited to bypass network segmentation, granting an attacker widespread internal access. Insecure deserialization issues in platforms like Wazuh Server and CrushFTP could then be used to run malicious payloads or steal sensitive files from administrative consoles. Weaknesses in web delivery controllers like Ingress-Nginx or popular content management systems such as WordPress, SharePoint, and Kentico Xperience create vectors to bypass security controls, exfiltrate confidential data, or fully compromise servers.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...51bc8df1	100538	GeoServer - SSRF - CVE:CVE-2024-29198	Log	Block	This is a New Detection
CIS Managed Ruleset	...c9e0b290	100548	Ivanti EPMM - Remote Code Execution - CVE:CVE-2025-6771	Log	Block	This is a New Detection
CIS Managed Ruleset	...ad93cda8	100550	Microsoft SharePoint - Remote Code Execution - CVE:CVE-2024-38018	Log	Block	This is a New Detection
CIS Managed Ruleset	...1dca5738	100562	Manager-IO - SSRF - CVE:CVE-2025-54122	Log	Block	This is a New Detection
CIS Managed Ruleset	...2e9137e1	100565	Cisco Identity Services Engine - Remote Code Execution - CVE:CVE-2025-20281	Log	Block	This is a New Detection
CIS Managed Ruleset	...5d8102e1	100567	Ingress-Nginx - Remote Code Execution - CVE:CVE-2025-1974	Log	Disabled	This is a New Detection

CIS Managed Ruleset	...99105c43	100569	PaperCut NG/MF - Remote Code Execution - CVE:CVE-2023-2533	Log	Block	This is a New Detection
CIS Managed Ruleset	...2c81dc88	100571	SonicWall SMA - XSS - CVE:CVE-2025-40598	Log	Block	This is a New Detection
CIS Managed Ruleset	...926c96d4	100573	WordPress - Dangerous File Upload - CVE:CVE-2025-5394	Log	Block	This is a New Detection
CIS Managed Ruleset	...40ec2fda	100806				
Wazuh Server - Remote Code Execution - CVE:CVE-2025-24016	Log	Block	This is a New Detection			
CIS Managed Ruleset	...2401fa3b	100824	CrushFTP - Remote Code Execution - CVE:CVE-2025-54309	Log	Block	
This is a New Detection						
CIS Managed Ruleset	...74920ace	100824A	CrushFTP - Remote Code Execution - CVE:CVE-2025-54309 - 2	Log	Block	
This is a New Detection						
CIS Managed Ruleset	...c7e63549	100825	AMI MegaRAC - Auth Bypass - CVE:CVE-2024-54085	Log	Block	This is a New Detection
CIS Managed Ruleset	...b79037e6	100826	Kentico Xperience CMS - Auth Bypass - CVE:CVE-2025-2747	Log	Block	This is a New Detection
CIS Managed Ruleset	...caf261aa	100827	Kentico Xperience CMS - XSS - CVE:CVE-2025-2748	Log	Block	This is a New Detection
CIS Managed Ruleset	...6f1c2d12	100828	Node.js - Directory Traversal - CVE:CVE-2025-27210	Log	Block	This is a New Detection
CIS Managed Ruleset	...0341fccc	100829	WordPress:Plugin:Simple File List - Remote Code Execution - CVE:CVE-2025-34085	Log	Block	This is a New Detection
CIS Managed Ruleset	...4cefeeda	100829A	WordPress:Plugin:Simple File List - Remote Code Execution - CVE:CVE-2025-34085 - 2	Log	Disabled	This is a New Detection

WAF - WAF Release - 2025-08-07 - Emergency

Published on: Thu, 07 Aug 2025 00:00:00 GMT

This week’s highlight focuses on two critical vulnerabilities affecting key infrastructure and enterprise content management platforms. Both flaws present significant remote code execution risks that can be exploited with minimal or no user interaction.

Key Findings

- \$ * Squid (≤6.3) — CVE-2025-54574: A heap buffer overflow occurs when processing Uniform Resource Names (URNs). This vulnerability may allow remote attackers to execute arbitrary code on the server. The issue has been resolved in version 6.4.
- * Adobe AEM (≤6.5.23) — CVE-2025-54253: Due to a misconfiguration, attackers can achieve remote code execution without requiring any user interaction, posing a severe threat to affected deployments.

Impact

Both vulnerabilities expose critical attack vectors that can lead to full server compromise. The Squid heap buffer overflow allows remote code execution by crafting malicious URNs, which can lead to server takeover or denial of service. Given Squid’s widespread use as a caching proxy, this flaw could be exploited to disrupt network traffic or gain footholds inside secure environments.

Adobe AEM’s remote code execution vulnerability enables attackers to run arbitrary code on the content management server without any user involvement. This puts sensitive content, application integrity, and the underlying infrastructure at extreme risk. Exploitation could lead to data theft, defacement, or persistent backdoor installation.

These findings reinforce the urgency of updating to the patched versions — Squid 6.4 and Adobe AEM 6.5.24 or later — and reviewing configurations to prevent exploitation.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...ef7e015b	100844	Adobe Experience Manager Forms - Remote Code Execution - CVE:CVE-2025-54253	N/A	Block	This is a New Detection
CIS Managed Ruleset	...baec057a	100840	Squid - Buffer Overflow - CVE:CVE-2025-54574	N/A	Block	This is a New Detection

WAF - WAF Release - 2025-08-04

Published on: Mon, 04 Aug 2025 00:00:00 GMT

This week's highlight focuses on a series of significant vulnerabilities identified across widely adopted web platforms, from enterprise-grade CMS to essential backend administration tools. The findings reveal multiple vectors for attack, including critical flaws that allow for full server compromise and others that enable targeted attacks against users.

Key Findings

- \$ * Sitecore (CVE-2025-34509, CVE-2025-34510, CVE-2025-34511): A hardcoded credential allows remote attackers to access administrative APIs. Once authenticated, they can exploit an additional vulnerability to upload arbitrary files, leading to remote code execution.
- * Grafana (CVE-2025-4123): A cross-site scripting (XSS) vulnerability allows an attacker to redirect users to a malicious website, which can then execute arbitrary JavaScript in the victim's browser.
- * LaRecipe (CVE-2025-53833): Through Server-Side Template Injection, attackers can execute arbitrary commands on the server, potentially access sensitive environment variables, and escalate access depending on server configuration.
- * CentOS WebPanel (CVE-2025-48703): A command injection vulnerability could allow a remote attacker to execute arbitrary commands on the server.
- * WordPress (CVE-2023-5561): This vulnerability allows unauthenticated attackers to determine the email addresses of users who have published public posts on an affected website.

* WordPress Plugin - WPBookit (CVE-2025-6058): A missing file type validation allows unauthenticated attackers to upload arbitrary files to the server, creating the potential for remote code execution.

* WordPress Theme - Motors (CVE-2025-4322): Due to improper identity validation, an unauthenticated attacker can change the passwords of arbitrary users, including administrators, to gain access to their accounts.

Impact

These vulnerabilities pose a multi-layered threat to widely adopted web technologies, ranging from enterprise-grade platforms like Sitecore to everyday solutions such as WordPress, and backend tools like CentOS WebPanel. The most severe risks originate in remote code execution (RCE) flaws found in Sitecore, CentOS WebPanel, LaRecipe, and the WPBookit plugin. These allow attackers to bypass security controls and gain deep access to the server, enabling them to steal sensitive data, deface websites, install persistent malware, or use the compromised server as a launchpad for further attacks.

The privilege escalation vulnerability is the Motors theme, which allows for a complete administrative account takeover on WordPress sites. This effectively hands control of the application to an attacker, who can then manipulate content, exfiltrate user data, and alter site functionality without needing to breach the server itself.

The Grafana cross-site scripting (XSS) flaw can be used to hijack authenticated user sessions or steal credentials, turning a trusted user's browser into an attack vector.

Meanwhile, the information disclosure flaw in WordPress core provides attackers with valid user emails, fueling targeted phishing campaigns that aim to secure the same account access achievable through the other exploits.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...52f30a13	100535A	Sitecore - Dangerous File Upload - CVE:CVE-2025-34510, CVE:CVE-2025-34511	Log	Block	This is a New Detection
CIS Managed Ruleset	...5045a97f	100535	Sitecore - Information Disclosure - CVE:CVE-2025-34509	Log	Block	This is a New Detection
CIS Managed Ruleset	...579cd3e0	100543	Grafana - Directory Traversal - CVE:CVE-2025-4123	Log	Block	This is a New Detection
CIS Managed Ruleset	...0cbd9abc	100545	WordPress - Information Disclosure - CVE:CVE-2023-5561	Log	Block	This is a New Detection
CIS Managed Ruleset	...8f634977	100820	CentOS WebPanel - Remote Code Execution - CVE:CVE-2025-48703	Log	Block	This is a New Detection
CIS Managed Ruleset	...82ae64c1	100821	LaRecipe - SSTI - CVE:CVE-2025-53833	Log	Block	This is a New Detection
CIS Managed Ruleset	...194f7b2d	100822	WordPress:Plugin:WPBookit - Remote Code Execution - CVE:CVE-2025-6058	Log	Block	This is a New Detection
CIS Managed Ruleset	...0bf1b661	100823	WordPress:Theme:Motors - Privilege Escalation - CVE:CVE-2025-4322	Log	Block	This is a New Detection

WAF - WAF Release - 2025-07-28

Published on: Mon, 28 Jul 2025 00:00:00 GMT

This week’s update spotlights several vulnerabilities across Apache Tomcat, MongoDB, and Fortinet FortiWeb. Several flaws related with a memory leak in Apache Tomcat can lead to a denial-of-service attack. Additionally, a code injection flaw in MongoDB's Mongoose library allows attackers to bypass security controls to access restricted data.

Key Findings

- \$ * Fortinet FortiWeb (CVE-2025-25257): An improper neutralization of special elements used in a SQL command vulnerability in Fortinet FortiWeb versions allows an unauthenticated attacker to execute unauthorized SQL code or commands.
- * Apache Tomcat (CVE-2025-31650): A improper Input Validation vulnerability in Apache Tomcat that could create memory leak when incorrect error handling for some invalid HTTP priority headers resulted in incomplete clean-up of the failed request.
- * MongoDB (CVE-2024-53900, CVE:CVE-2025-23061): Improper use of `\$where` in match and a nested `\$where` filter with a `populate()` match in Mongoose can lead to search injection.

Impact

These vulnerabilities target user-facing components, web application servers, and back-end databases. A SQL injection flaw in Fortinet FortiWeb can lead to data theft or system compromise. A separate issue in Apache Tomcat involves a memory leak from improper input validation, which could be exploited for a denial-of-service (DoS) attack. Finally, a vulnerability in MongoDB's Mongoose library allows attackers to bypass security filters and access unauthorized data through malicious search queries.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...3461ec9e	100804	BerriAI - SSRF - CVE:CVE-2024-6587	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...0cb13e1d	100812	Fortinet FortiWeb - Remote Code Execution - CVE:CVE-2025-25257	Log	Block	This is a New Detection
CIS Managed Ruleset	...67fae7f7	100813	Apache Tomcat - DoS - CVE:CVE-2025-31650	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...4b6a5bb1	100815	MongoDB - Remote Code Execution - CVE:CVE-2024-53900, CVE:CVE-2025-23061	Log	Block	This is a New Detection
CIS Managed Ruleset	...688f8e79	100816	MongoDB - Remote Code Execution - CVE:CVE-2024-53900, CVE:CVE-2025-23061	Log	Block	This is a New Detection

WAF - WAF Release - 2025-07-21 - Emergency

Published on: Mon, 21 Jul 2025 00:00:00 GMT

This week's update highlights several high-impact vulnerabilities affecting Microsoft SharePoint Server. These flaws, involving unsafe deserialization, allow unauthenticated remote code execution over the network, posing a critical threat to enterprise environments relying on SharePoint for collaboration and document management.

Key Findings

- \$ * Microsoft SharePoint Server (CVE-2025-53770): A critical vulnerability involving unsafe deserialization of untrusted data, enabling unauthenticated remote code execution over the network. This flaw allows attackers to execute arbitrary code on vulnerable SharePoint servers without user interaction.
- * Microsoft SharePoint Server (CVE-2025-53771): A closely related deserialization issue that can be exploited by unauthenticated attackers, potentially leading to full system compromise. The vulnerability highlights continued risks around insecure serialization logic in enterprise collaboration platforms.

Impact

Together, these vulnerabilities significantly weaken the security posture of on-premise Microsoft SharePoint Server deployments. By enabling remote code execution without authentication, they open the door for attackers to gain persistent access, deploy malware, and move laterally across enterprise environments.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...2168f6f0	100817	Microsoft SharePoint - Deserialization - CVE:CVE-2025-53770	N/A	Block	This is a New Detection
CIS Managed Ruleset	...8de656c4	100818	Microsoft SharePoint - Deserialization - CVE:CVE-2025-53771	N/A	Block	This is a New Detection

For more details, also refer to [our blog ↗](#).

WAF - WAF Release - 2025-07-21

Published on: Mon, 21 Jul 2025 00:00:00 GMT

This week's update spotlights several critical vulnerabilities across Citrix NetScaler Memory Disclosure, FTP servers and network application. Several flaws enable unauthenticated remote code execution or sensitive data exposure, posing a significant risk to enterprise security.

Key Findings

\$ * Wing FTP Server (CVE-2025-47812): A critical Remote Code Execution (RCE) vulnerability that enables unauthenticated attackers to execute arbitrary code with root/SYSTEM-level privileges by exploiting a Lua injection flaw.

* Infoblox NetMRI (CVE-2025-32813): A remote unauthenticated command injection flaw that allows an attacker to execute arbitrary commands, potentially leading to unauthorized access.

* Citrix Netscaler ADC (CVE-2025-5777, CVE-2023-4966): A sensitive information disclosure vulnerability, also known as "Citrix Bleed2", that allows the disclosure of memory and subsequent remote access session hijacking.

* Akamai CloudTest (CVE-2025-49493): An XML External Entity (XXE) injection that could lead to read local files on the system by manipulating XML input.

Impact

These vulnerabilities affect critical enterprise infrastructure, from file transfer services and network management appliances to application delivery controllers. The Wing FTP RCE and Infoblox command injection flaws offer direct paths to deep system compromise, while the Citrix "Bleed2" and Akamai XXE vulnerabilities undermine system integrity by enabling session hijacking and sensitive data theft.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...3461ec9e	100804	BerriAI - SSRF - CVE:CVE-2024-6587	Log	Log	This is a New Detection
CIS Managed Ruleset	...5199b58a	100805	Wing FTP Server - Remote Code Execution - CVE:CVE-2025-47812	Log	Block	This is a New Detection
CIS Managed Ruleset	...919a91a4	100807	Infoblox NetMRI - Command Injection - CVE:CVE-2025-32813	Log	Block	This is a New Detection
CIS Managed Ruleset	...7899130f	100808	Citrix Netscaler ADC - Buffer Error - CVE:CVE-2025-5777	Log	Disabled	This is a New Detection

CIS Managed Ruleset	...d1cf8e08	100809	Citrix Netscaler ADC - Information Disclosure - CVE:CVE-2023-4966	Log	Block	This is a New Detection
CIS Managed Ruleset	...6e70469f	100810	Akamai CloudTest - XXE - CVE:CVE-2025-49493	Log	Block	This is a New Detection

WAF - WAF Release - 2025-07-14

Published on: Mon, 14 Jul 2025 00:00:00 GMT

This week’s vulnerability analysis highlights emerging web application threats that exploit modern JavaScript behavior and SQL parsing ambiguities. Attackers continue to refine techniques such as attribute overloading and obfuscated logic manipulation to evade detection and compromise front-end and back-end systems.

Key Findings

\$ * XSS – Attribute Overloading: A novel cross-site scripting technique where attackers abuse custom or non-standard HTML attributes to smuggle payloads into the DOM. These payloads evade traditional sanitization logic, especially in frameworks that loosely validate attributes or trust unknown tokens.

* XSS – onToggle Event Abuse: Exploits the lesser-used onToggle event (triggered by elements like `<details>`) to execute arbitrary JavaScript when users interact with UI elements. This vector is often overlooked by static analyzers and can be embedded in seemingly benign components.

Impact

These vulnerabilities target both user-facing components and back-end databases, introducing potential vectors for credential theft, session hijacking, or full data exfiltration. The XSS variants bypass conventional filters through overlooked HTML behaviors, while the obfuscated SQLi enables attackers to stealthily probe back-end logic, making them especially difficult to detect and block.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...2aa3d845	100798	XSS - Attribute Overloading	Log	Block	This is a New Detection
CIS Managed Ruleset	...37548d06	100799	XSS - OnToggle	Log	Block	This is a New Detection

WAF - Increased IP List Limits for Enterprise Accounts

Published on: Mon, 07 Jul 2025 00:00:00 GMT

We have significantly increased the limits for [IP Lists](#) on Enterprise plans to provide greater flexibility and control:

\$ * **Total number of lists** : Increased from 10 to 1,000.

* **Total number of list items** : Increased from 10,000 to 500,000.

Limits for other list types and plans remain unchanged. For more details, refer to the [lists availability](#).

WAF - WAF Release - 2025-07-07

Published on: Mon, 07 Jul 2025 00:00:00 GMT

This week’s roundup uncovers critical vulnerabilities affecting enterprise VoIP systems, webmail platforms, and a popular JavaScript framework. The risks range from authentication bypass to remote code execution (RCE) and buffer handling flaws, each offering attackers a path to elevate access or fully compromise systems.

Key Findings

\$ * Next.js - Auth Bypass: A newly detected authentication bypass flaw in the Next.js framework allows attackers to access protected routes

or APIs without proper authorization, undermining application access controls.

* Fortinet FortiVoice (CVE-2025-32756): A buffer error vulnerability in FortiVoice systems that could lead to memory corruption and potential code execution or service disruption in enterprise telephony environments.

* Roundcube (CVE-2025-49113): A critical RCE flaw allowing unauthenticated attackers to execute arbitrary PHP code via crafted requests, leading to full compromise of mail servers and user inboxes.

Impact

These vulnerabilities affect core business infrastructure, from web interfaces to voice communications and email platforms. The Roundcube RCE and FortiVoice buffer flaw offer potential for deep system access, while the Next.js auth bypass undermines trust boundaries in modern web apps.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...7eb35ee6	100795	Next.js - Auth Bypass	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...c329aeb0	100796	Fortinet FortiVoice - Buffer Error - CVE:CVE-2025-32756	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...ab314023	100797	Roundcube - Remote Code Execution - CVE:CVE-2025-49113	Log	Disabled	This is a New Detection

WAF - WAF Release - 2025-06-16

Published on: Mon, 16 Jun 2025 00:00:00 GMT

This week’s roundup highlights multiple critical vulnerabilities across popular web frameworks, plugins, and enterprise platforms. The focus lies on remote code execution (RCE), server-side request forgery (SSRF), and insecure file upload vectors that enable full system compromise or data exfiltration.

Key Findings

\$ * Cisco IOS XE (CVE-2025-20188): Critical RCE vulnerability enabling unauthenticated attackers to execute arbitrary commands on network infrastructure devices, risking total router compromise.

* Axios (CVE-2024-39338): SSRF flaw impacting server-side request control, allowing attackers to manipulate internal service requests when misconfigured with unsanitized user input.

* vBulletin (CVE-2025-48827, CVE-2025-48828): Two high-impact RCE flaws enabling attackers to remotely execute PHP code, compromising forum installations and underlying web servers.

* Invision Community (CVE-2025-47916): A critical RCE vulnerability allowing authenticated attackers to run arbitrary code in community platforms, threatening data and lateral movement risk.

* CrushFTP (CVE-2025-32102, CVE-2025-32103): SSRF vulnerabilities in upload endpoint processing permit attackers to pivot internal network scans and abuse internal services.

* Roundcube (CVE-2025-49113): RCE via email processing enables attackers to execute code upon viewing a crafted email — particularly dangerous for webmail deployments.

* WooCommerce WordPress Plugin (CVE-2025-47577): Dangerous file upload vulnerability permits unauthenticated users to upload executable payloads, leading to full WordPress site takeover.

* Cross-Site Scripting (XSS) Detection Improvements: Enhanced detection patterns.

Impact

These vulnerabilities span core systems — from routers to e-commerce to email. RCE in Cisco IOS XE, Roundcube, and vBulletin poses full system compromise. SSRF in Axios and CrushFTP supports internal pivoting, while WooCommerce’s file upload bug opens doors to mass WordPress exploitation.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
---------	---------	----------------	-------------	-----------------	------------	----------

CIS Managed Ruleset	...35fe5d53	100783	Cisco IOS XE - Remote Code Execution - CVE:CVE-2025-20188	Log	Block	This is a New Detection
CIS Managed Ruleset	...8332af5d	100784	Axios - SSRF - CVE:CVE-2024-39338	Log	Block	This is a New Detection
CIS Managed Ruleset	...2e1648d2	100785	vBulletin - Remote Code Execution - CVE:CVE-2025-48827, CVE:CVE-2025-48828	Log	Block	This is a New Detection
CIS Managed Ruleset	...0edcf1ef	100786	Invision Community - Remote Code Execution - CVE:CVE-2025-47916	Log	Block	This is a New Detection
CIS Managed Ruleset	...d6f5eb48	100791	CrushFTP - SSRF - CVE:CVE-2025-32102, CVE:CVE-2025-32103	Log	Block	This is a New Detection
CIS Managed Ruleset	...30baa18a	100792	Roundcube - Remote Code Execution - CVE:CVE-2025-49113	Log	Block	This is a New Detection
CIS Managed Ruleset	...229ba236	100793	XSS - Ontoggle	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...fa338296	100794	WordPress WooCommerce Plugin - Dangerous File Upload - CVE:CVE-2025-47577	Log	Block	This is a New Detection

WAF - WAF Release - 2025-06-09

Published on: Mon, 09 Jun 2025 00:00:00 GMT

This week’s update spotlights four critical vulnerabilities across CMS platforms, VoIP systems, and enterprise applications. Several flaws enable remote code execution or privilege escalation, posing significant enterprise risks.

Key Findings

\$ * WordPress OttoKit Plugin (CVE-2025-27007): Privilege escalation flaw allows unauthenticated attackers to create or elevate user accounts, compromising WordPress administrative control.

* SAP NetWeaver (CVE-2025-42999): Remote Code Execution vulnerability enables attackers to execute arbitrary code on SAP NetWeaver systems, threatening core ERP and business operations.

* Fortinet FortiVoice (CVE-2025-32756): Buffer error vulnerability may lead to memory corruption and potential code execution, directly impacting enterprise VoIP infrastructure.

* Camaleon CMS (CVE-2024-46986): Remote Code Execution vulnerability allows attackers to gain full control over Camaleon CMS installations, exposing hosted content and underlying servers.

Impact

These vulnerabilities target widely deployed CMS, ERP, and VoIP systems. RCE flaws in SAP NetWeaver and Camaleon CMS allow full takeover of business-critical applications. Privilege escalation in OttoKit exposes WordPress environments to full administrative compromise. FortiVoice buffer handling issues risk destabilizing or fully compromising enterprise telephony systems.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...0debd86e	100769	WordPress OttoKit Plugin - Privilege Escalation - CVE:CVE-2025-27007	Log	Block	This is a New Detection

CIS Managed Ruleset	...5f57b448	100770	SAP NetWeaver - Remote Code Execution - CVE:CVE-2025-42999	Log	Block	This is a New Detection
CIS Managed Ruleset	...4df8857a	100779	Fortinet FortiVoice - Buffer Error - CVE:CVE-2025-32756	Log	Block	This is a New Detection
CIS Managed Ruleset	...3b840107	100780	Camaleon CMS - Remote Code Execution - CVE:CVE-2024-46986	Log	Block	This is a New Detection

WAF - WAF Release - 2025-06-02

Published on: Mon, 02 Jun 2025 00:00:00 GMT

This week’s roundup highlights five high-risk vulnerabilities affecting SD-WAN, load balancers, and AI platforms. Several flaws enable unauthenticated remote code execution or authentication bypass.

Key Findings

- \$ * Versa Concerto SD-WAN (CVE-2025-34026, CVE-2025-34027): Authentication bypass vulnerabilities allow attackers to gain unauthorized access to SD-WAN management interfaces, compromising network segmentation and control.
- * Kemp LoadMaster (CVE-2024-7591): Remote Code Execution vulnerability enables attackers to execute arbitrary commands, potentially leading to full device compromise within enterprise load balancing environments.
- * AnythingLLM (CVE-2024-0759): Server-Side Request Forgery (SSRF) flaw allows external attackers to force the LLM backend to make unauthorized internal network requests, potentially exposing sensitive internal resources.
- * Anyscale Ray (CVE-2023-48022): Remote Code Execution vulnerability affecting distributed AI workloads, allowing attackers to execute arbitrary code on Ray cluster nodes.
- * Server-Side Request Forgery (SSRF) - Generic & Obfuscated Payloads: Ongoing advancements in SSRF payload techniques observed, including obfuscation and expanded targeting of cloud metadata services and internal IP ranges.

Impact

These vulnerabilities expose critical infrastructure across networking, AI platforms, and SaaS integrations. Unauthenticated RCE and auth bypass flaws in Versa Concerto, Kemp LoadMaster, and Anyscale Ray allow full system compromise. AnythingLLM and SSRF payload variants expand attack surfaces into internal cloud resources, sensitive APIs, and metadata services, increasing risk of privilege escalation, data theft, and persistent access.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...39b52f02	100764	Versa Concerto SD-WAN - Auth Bypass - CVE:CVE-2025-34027	Log	Block	This is a New Detection
CIS Managed Ruleset	...a34edb97	100765	Versa Concerto SD-WAN - Auth Bypass - CVE:CVE-2025-34026	Log	Block	This is a New Detection
CIS Managed Ruleset	...0d99b2db	100766	Kemp LoadMaster - Remote Code Execution - CVE:CVE-2024-7591	Log	Block	This is a New Detection
CIS Managed Ruleset	...95aa3a4f	100767	AnythingLLM - SSRF - CVE:CVE-2024-0759	Log	Block	This is a New Detection
CIS Managed Ruleset	...840a0966	100768	Anyscale Ray - Remote Code Execution - CVE:CVE-2023-48022	Log	Block	This is a New Detection

CIS Managed Ruleset	...9d16ee18	100781	SSRF - Generic Payloads	N/A	Disabled	This is a New Detection
CIS Managed Ruleset	...5c963d9d	100782	SSRF - Obfuscated Payloads	N/A	Disabled	This is a New Detection

WAF - Updated attack score model

Published on: Wed, 28 May 2025 00:00:00 GMT

We have deployed an updated attack score model focused on enhancing the detection of multiple false positives (FPs).

As a result of this improvement, some changes in observed attack scores are expected.

WAF - WAF Release - 2025-05-27

Published on: Tue, 27 May 2025 00:00:00 GMT

This week’s roundup covers nine vulnerabilities, including six critical RCEs and one dangerous file upload. Affected platforms span cloud services, CI/CD pipelines, CMSs, and enterprise backup systems. Several are now addressed by updated WAF managed rulesets.

Key Findings

- \$ * Ingress-Nginx (CVE-2025-1098): Unauthenticated RCE via unsafe annotation handling. Impacts Kubernetes clusters.
- * GitHub Actions (CVE-2025-30066): RCE through malicious workflow inputs. Targets CI/CD pipelines.
- * Craft CMS (CVE-2025-32432): Template injection enables unauthenticated RCE. High risk to content-heavy sites.
- * F5 BIG-IP (CVE-2025-31644): RCE via TMUI exploit, allowing full system compromise.
- * AJ-Report (CVE-2024-15077): RCE through untrusted template execution. Affects reporting dashboards.
- * NAKIVO Backup (CVE-2024-48248): RCE via insecure script injection. High-value target for ransomware.
- * SAP NetWeaver (CVE-2025-31324): Dangerous file upload flaw enables remote shell deployment.
- * Ivanti EPMM (CVE-2025-4428, 4427): Auth bypass allows full access to mobile device management.
- * Vercel (CVE-2025-32421): Information leak via misconfigured APIs. Useful for attacker recon.

Impact

These vulnerabilities expose critical components across Kubernetes, CI/CD pipelines, and enterprise systems to severe threats including unauthenticated remote code execution, authentication bypass, and information leaks. High-impact flaws in Ingress-Nginx, Craft CMS, F5 BIG-IP, and NAKIVO Backup enable full system compromise, while SAP NetWeaver and AJ-Report allow remote shell deployment and template-based attacks. Ivanti EPMM’s auth bypass further risks unauthorized control over mobile device fleets.

GitHub Actions and Vercel introduce supply chain and reconnaissance risks, allowing malicious workflow inputs and data exposure that aid in targeted exploitation. Organizations should prioritize immediate patching, enhance monitoring, and deploy updated WAF and IDS signatures to defend against likely active exploitation.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...d127592a	100746	Vercel - Information Disclosure	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...95442495	100754	AJ-Report - Remote Code Execution - CVE:CVE-2024-15077	Log	Block	This is a New Detection
CIS Managed Ruleset	...dfee7ae4	100756	NAKIVO Backup - Remote Code Execution - CVE:CVE-2024-48248	Log	Block	This is a New Detection

CIS Managed Ruleset	...1c52f6d0	100757	Ingress-Nginx - Remote Code Execution - CVE:CVE-2025-1098	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...95442495	100759	SAP NetWeaver - Dangerous File Upload - CVE:CVE-2025-31324	Log	Block	This is a New Detection
CIS Managed Ruleset	...5366ccc1	100760	Craft CMS - Remote Code Execution - CVE:CVE-2025-32432	Log	Block	This is a New Detection
CIS Managed Ruleset	...eb40686b	100761	GitHub Action - Remote Code Execution - CVE:CVE-2025-30066	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...60fc041c	100762	Ivanti EPMM - Auth Bypass - CVE:CVE-2025-4428, CVE:CVE-2025-4427	Log	Block	This is a New Detection
CIS Managed Ruleset	...ebafdf6	100763	F5 Big IP - Remote Code Execution - CVE:CVE-2025-31644	Log	Disabled	This is a New Detection

WAF - WAF Release - 2025-05-19

Published on: Mon, 19 May 2025 00:00:00 GMT

This week's analysis covers four vulnerabilities, with three rated critical due to their Remote Code Execution (RCE) potential. One targets a high-traffic frontend platform, while another targets a popular content management system. These detections are now part of the CIS Managed Ruleset in *Block* mode.

Key Findings

\$ * Commvault Command Center (CVE-2025-34028) exposes an unauthenticated RCE via insecure command injection paths in the web UI. This is critical due to its use in enterprise backup environments.

* BentoML (CVE-2025-27520) reveals an exploitable vector where serialized payloads in model deployment APIs can lead to arbitrary command execution. This targets modern AI/ML infrastructure.

* Craft CMS (CVE-2024-56145) allows RCE through template injection in unauthenticated endpoints. It poses a significant risk for content-heavy websites with plugin extensions.

* Apache HTTP Server (CVE-2024-38475) discloses sensitive server config data due to misconfigured `mod\_proxy` behavior. While not RCE, this is useful for pre-attack recon.

Impact

These newly detected vulnerabilities introduce critical risk across modern web stacks, AI infrastructure, and content platforms: unauthenticated RCEs in Commvault, BentoML, and Craft CMS enable full system compromise with minimal attacker effort.

Apache HTTPD information leak can support targeted reconnaissance, increasing the success rate of follow-up exploits. Organizations using these platforms should prioritize patching and monitor for indicators of exploitation using updated WAF detection rules.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...75129820	100745	Apache HTTP Server - Information Disclosure - CVE:CVE-2024-38475	Log	Block	This is a New Detection
CIS Managed Ruleset	...26a517f1	100747	Commvault Command Center - Remote Code Execution - CVE:CVE-2025-34028	Log	Block	This is a New Detection

CIS Managed Ruleset	...d7619ccb	100749	BentoML - Remote Code Execution - CVE:CVE-2025-27520	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...f15bfda4	100753	Craft CMS - Remote Code Execution - CVE:CVE-2024-56145	Log	Block	This is a New Detection

WAF - Improved Payload Logging for WAF Managed Rules

Published on: Thu, 08 May 2025 00:00:00 GMT

We have upgraded WAF Payload Logging to enhance rule diagnostics and usability:

\$ * **Targeted logging** * : Logs now capture only the specific portions of requests that triggered WAF rules, rather than entire request segments.
* **Visual highlighting** * : Matched content is visually highlighted in the UI for faster identification.
* **Enhanced context** * : Logs now include surrounding context to make diagnostics more effective.

Payload Logging is available to all Enterprise customers. If you have not used Payload Logging before, check how you can [get started](#).

Note: The structure of the `encrypted_matched_data` field in Logpush has changed from `Map<Field, Value>` to `Map<Field, {Before: bytes, Content: Value, After: bytes}>`. If you rely on this field in your Logpush jobs, you should review and update your processing logic accordingly.

WAF - WAF Release - 2025-05-05

Published on: Mon, 05 May 2025 00:00:00 GMT

This week's analysis covers five CVEs with varying impact levels. Four are rated critical, while one is rated high severity. Remote Code Execution vulnerabilities dominate this set.

Key Findings

GFI KerioControl (CVE-2024-52875) contains an unauthenticated Remote Code Execution (RCE) vulnerability that targets firewall appliances. This vulnerability can let attackers gain root level system access, making this CVE particularly attractive for threat actors.

The SonicWall SMA vulnerabilities remain concerning due to their continued exploitation since 2021. These critical vulnerabilities in remote access solutions create dangerous entry points to networks.

Impact

Customers using the Managed Ruleset will receive rule coverage following this week's release. Below is a breakdown of the recommended prioritization based on current exploitation trends:

\$ * GFI KerioControl (CVE-2024-52875) - Highest priority; unauthenticated RCE
* SonicWall SMA (Multiple vulnerabilities) - Critical for network appliances
* XWiki (CVE-2025-24893) - High priority for development environments
* Langflow (CVE-2025-3248) - Important for AI workflow platforms
* MinIO (CVE-2025-31489) - Important for object storage implementations

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...d0b7a392	100724	GFI KerioControl - Remote Code Execution - CVE:CVE-2024-52875	Log	Block	This is a New Detection

CIS Managed Ruleset	...717a9e42	100748	XWiki - Remote Code Execution - CVE:CVE-2025-24893	Log	Block	This is a New Detection
CIS Managed Ruleset	...e9cf745d	100750	SonicWall SMA - Dangerous File Upload - CVE:CVE-2021-20040, CVE:CVE-2021-20041, CVE:CVE-2021-20042	Log	Block	This is a New Detection
CIS Managed Ruleset	...d29da333	100751	Langflow - Remote Code Execution - CVE:CVE-2025-3248	Log	Block	This is a New Detection
CIS Managed Ruleset	...caa7b208	100752	MinIO - Auth Bypass - CVE:CVE-2025-31489	Log	Block	This is a New Detection

WAF - WAF Release - 2025-04-26 - Emergency

Published on: Sat, 26 Apr 2025 00:00:00 GMT

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...19fcc883	100755	React.js - Router and Remix Vulnerability - CVE:CVE-2025-43864, CVE:CVE-2025-43865	Block	Block	This is a New Detection

WAF - WAF Release - 2025-04-22

Published on: Tue, 22 Apr 2025 00:00:00 GMT

Each of this week's rule releases covers a distinct CVE, with half of the rules targeting Remote Code Execution (RCE) attacks. Of the 6 CVEs covered, four were scored as critical, with the other two scored as high.

When deciding which exploits to tackle, CIS tunes into the attackers' areas of focus. CIS's network intelligence provides a unique lens into attacker activity – for instance, through the volume of blocked requests related with CVE exploits after updating WAF Managed Rules with new detections.

From this week's releases, one indicator that RCE is a "hot topic" attack type is the fact that the Oracle PeopleSoft RCE rule accounts for half of all of the new rule matches. This rule patches CVE-2023-22047, a high-severity vulnerability in the Oracle PeopleSoft suite that allows unauthenticated attackers to access PeopleSoft Enterprise PeopleTools data through remote code execution. This is particularly concerning because of the nature of the data managed by PeopleSoft – this can include payroll records or student profile information. This CVE, along with five others, are addressed with the latest detection update to WAF Managed Rules.

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...a5be3327	100738	GitLab - Auth Bypass - CVE:CVE-2023-7028	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...6c9531fa	100740	Splunk Enterprise - Remote Code Execution - CVE:CVE-2025-20229	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...f40bbc2b	100741	Oracle PeopleSoft - Remote Code Execution - CVE:CVE-2023-22047	Log	Disabled	This is a New Detection

CIS Managed Ruleset	...5462167c	100742	CrushFTP - Auth Bypass - CVE:CVE-2025-31161	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...caa7b208	100743	Ivanti - Buffer Error - CVE:CVE-2025-22457	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...d52139a8	100744	Oracle Access Manager - Remote Code Execution - CVE:CVE-2021-35587	Log	Disabled	This is a New Detection

WAF - WAF Release - 2025-04-14

Published on: Mon, 14 Apr 2025 00:00:00 GMT

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...d6b2d36c	100739A	Next.js - Auth Bypass - CVE:CVE-2025-29927 - 2	Log	Disabled	This is a New Detection

WAF - WAF Release - 2025-04-02

Published on: Wed, 02 Apr 2025 00:00:00 GMT

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...622f0483	100732	Sitecore - Code Injection - CVE:CVE-2025-27218	Log	Block	This is a New Detection
CIS Managed Ruleset	...0f101cca	100733	Angular-Base64-Upload - Remote Code Execution - CVE:CVE-2024-42640	Log	Block	This is a New Detection
CIS Managed Ruleset	...1bbcd247	100734	Apache Camel - Remote Code Execution - CVE:CVE-2025-29891	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...90aea1ca	100735	Progress Software WhatsUp Gold - Remote Code Execution - CVE:CVE-2024-4885	Log	Block	This is a New Detection
CIS Managed Ruleset	...d9d8c5f2	100737	Apache Tomcat - Remote Code Execution - CVE:CVE-2025-24813	Log	Block	This is a New Detection
CIS Managed Ruleset	...a28a42c4	100659	Common Payloads for Server-side Template Injection	N/A	Disabled	N/A
CIS Managed Ruleset	...daa4b037	100659	Common Payloads for Server-side Template Injection - Base64	N/A	Disabled	N/A

CIS Managed Ruleset	...48f6a9cf	100642	LDAP Injection	N/A	Disabled	N/A
CIS Managed Ruleset	...e0713e9f	100642	LDAP Injection Base64	N/A	Disabled	N/A
CIS Managed Ruleset	...1bc977d1	100005	DotNetNuke - File Inclusion - CVE:CVE-2018-9126, CVE:CVE-2011-1892, CVE:CVE-2022-31474	N/A	Disabled	N/A
CIS Managed Ruleset	...bb70a463	100527	Apache Struts - CVE:CVE-2021-31805	N/A	Block	N/A
CIS Managed Ruleset	...0c99546a	100702	Command Injection - CVE:CVE-2022-24108	N/A	Block	N/A
CIS Managed Ruleset	...9a5581d0	100622C	Ivanti - Command Injection - CVE:CVE-2023-46805, CVE:CVE-2024-21887, CVE:CVE-2024-22024	N/A	Block	N/A
CIS Managed Ruleset	...06d0b009	100536C	GraphQL Command Injection	N/A	Disabled	N/A
CIS Managed Ruleset	...1651d0c8	100536	GraphQL Injection	N/A	Disabled	N/A
CIS Managed Ruleset	...af00f61d	100536A	GraphQL Introspection	N/A	Disabled	N/A
CIS Managed Ruleset	...a41e5b67	100536B	GraphQL SSRF	N/A	Disabled	N/A
CIS Managed Ruleset	...433e5b3d	100559A	Prototype Pollution - Common Payloads	N/A	Disabled	N/A
CIS Managed Ruleset	...4816b26f	100559A	Prototype Pollution - Common Payloads - Base64	N/A	Disabled	N/A
CIS Managed Ruleset	...fcea5ed2	100734	Apache Camel - Remote Code Execution - CVE:CVE-2025-29891	N/A	Disabled	N/A

WAF - WAF Release - 2025-03-22 - Emergency

Published on: Sat, 22 Mar 2025 00:00:00 GMT

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
---------	---------	----------------	-------------	-----------------	------------	----------

CIS Managed Ruleset	...f472013e	100739	Next.js - Auth Bypass - CVE:CVE-2025-29927	N/A	Disabled	This is a New Detection
---------------------	-------------	--------	--	-----	----------	-------------------------

Workers, Pages, WAF - New Managed WAF rule for Next.js CVE-2025-29927.

Published on: Sat, 22 Mar 2025 00:00:00 GMT

Update: Mon Mar 24th, 11PM UTC : Next.js has made further changes to address a smaller vulnerability introduced in the patches made to its middleware handling. Users should upgrade to Next.js versions `15.2.4` , `14.2.26` , `13.5.10` or `12.3.6` . **If you are unable to immediately upgrade or are running an older version of Next.js, you can enable the WAF rule described in this changelog as a mitigation.**

Update: Mon Mar 24th, 8PM UTC : Next.js has now [backported the patch for this vulnerability ↗](#) to cover Next.js v12 and v13. Users on those versions will need to patch to `13.5.9` and `12.3.5` (respectively) to mitigate the vulnerability.

Update: Sat Mar 22nd, 4PM UTC : We have changed this WAF rule to opt-in only, as sites that use auth middleware with third-party auth vendors were observing failing requests.

We strongly recommend updating your version of Next.js (if eligible) to the patched versions, as your app will otherwise be vulnerable to an authentication bypass attack regardless of auth provider.

Enable the Managed Rule (strongly recommended)

This rule is opt-in only for sites on the Pro plan or above in the [WAF managed ruleset](#).

To enable the rule:

\$ 1. Head to Security > WAF > Managed rules in the CIS dashboard for the zone (website) you want to protect.

2. Click the three dots next to **CIS Managed Ruleset** and choose **Edit**

3. Scroll down and choose **Browse Rules**

4. Search for **CVE-2025-29927** (ruleId: ``34583778093748cc83ff7b38f472013e``)

5. Change the **Status** to **Enabled** and the **Action** to **Block**. You can optionally set the rule to Log, to validate potential impact before enabling it. Log will not block requests.

6. Click **Next**

7. Scroll down and choose **Save**

This will enable the WAF rule and block requests with the `x-middleware-subrequest` header regardless of Next.js version.

Create a WAF rule (manual)

For users on the Free plan, or who want to define a more specific rule, you can create a [Custom WAF rule](#) to block requests with the `x-middleware-subrequest` header regardless of Next.js version.

To create a custom rule:

\$ 1. Head to Security > WAF > Custom rules in the CIS dashboard for the zone (website) you want to protect.

2. Give the rule a name - e.g. ``next-js-CVE-2025-29927``

3. Set the matching parameters for the rule match any request where the ``x-middleware-subrequest`` header ``exists`` per the rule expression below.

```
(len(http.request.headers["x-middleware-subrequest"]) > 0)
```

4. Set the action to 'block'. If you want to observe the impact before blocking requests, set the action to 'log' (and edit the rule later).

5. **Deploy** the rule.

Next.js CVE-2025-29927

We've made a WAF (Web Application Firewall) rule available to all sites on CIS to protect against the [Next.js authentication bypass vulnerability ↗](#) (`CVE-2025-29927`) published on March 21st, 2025.

Note : This rule is not enabled by default as it blocked requests across sites for specific authentication middleware.

\$ * This managed rule protects sites using Next.js on Workers and Pages, as well as sites using CIS to protect Next.js applications hosted elsewhere.

* This rule has been made available (but not enabled by default) to all sites as part of our [WAF Managed Ruleset] (<https://developers.cloudflare.com/waf/managed-rules/reference/cloudflare-managed-ruleset/>) and blocks requests that attempt to bypass authentication in Next.js applications.

* The vulnerability affects almost all Next.js versions, and has been fully patched in Next.js `14.2.26` and `15.2.4`. Earlier, interim releases did not fully patch this vulnerability.

* **Users on older versions of Next.js (`11.1.4` to `13.5.6`) did not originally have a patch available**, but this the patch for this vulnerability and a subsequent additional patch have been backported to Next.js versions `12.3.6` and `13.5.10` as of Monday, March 24th. Users on Next.js v11 will need to deploy the stated workaround or enable the WAF rule.

The managed WAF rule mitigates this by blocking *external* user requests with the `x-middleware-subrequest` header regardless of Next.js version, but we recommend users using Next.js 14 and 15 upgrade to the patched versions of Next.js as an additional mitigation.

WAF - WAF Release - 2025-03-19 - Emergency

Published on: Wed, 19 Mar 2025 00:00:00 GMT

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...a2cafae7	100736	Generic HTTP Request Smuggling	N/A	Disabled	This is a New Detection

WAF - WAF Release - 2025-03-17

Published on: Mon, 17 Mar 2025 00:00:00 GMT

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...e59ec18a	100725	Fortinet FortiManager - Remote Code Execution - CVE:CVE-2023-42791, CVE:CVE-2024-23666	Log	Block	
CIS Managed Ruleset	...1dbf58df	100726	Ivanti - Remote Code Execution - CVE:CVE-2024-8190	Log	Block	
CIS Managed Ruleset	...0ad61fa7	100727	Cisco IOS XE - Remote Code Execution - CVE:CVE-2023-20198	Log	Disabled	Fixed action value in changelog; no rule changes.
CIS Managed Ruleset	...7ee56b66	100728	Sitecore - Remote Code Execution - CVE:CVE-2024-46938	Log	Block	
CIS Managed Ruleset	...a6752a38	100729	Microsoft SharePoint - Remote Code Execution - CVE:CVE-2023-33160	Log	Block	

CIS Managed Ruleset	...98d47b69	100730	Pentaho - Template Injection - CVE:CVE-2022-43769, CVE:CVE-2022-43939	Log	Block
CIS Managed Ruleset	...69fe1e0d	100700	Apache SSRF vulnerability CVE-2021-40438	N/A	Block

WAF - WAF Release - 2025-03-11 - Emergency

Published on: Tue, 11 Mar 2025 00:00:00 GMT

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...73febb31	100731	Apache Camel - Code Injection - CVE:CVE-2025-27636	N/A	Block	This is a New Detection

WAF - WAF Release - 2025-03-10

Published on: Mon, 10 Mar 2025 00:00:00 GMT

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...b2a51e3d	100722	Ivanti - Information Disclosure - CVE:CVE-2025-0282	Log	Block	This is a New Detection
CIS Managed Ruleset	...259073d5	100723	Cisco IOS XE - Information Disclosure - CVE:CVE-2023-20198	Log	Block	This is a New Detection

WAF - Updated leaked credentials database

Published on: Fri, 07 Mar 2025 00:00:00 GMT

Added new records to the leaked credentials database. The record sources are: Have I Been Pwned (HIBP) database, RockYou 2024 dataset, and another third-party database.

WAF - WAF Release - 2025-03-03

Published on: Mon, 03 Mar 2025 00:00:00 GMT

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...93e63099	100721	Ivanti - Remote Code Execution - CVE:CVE-2024-13159, CVE:CVE-2024-13160, CVE:CVE-2024-13161	Log	Block	This is a New Detection
CIS Managed Ruleset	...cac42ce2	100596	Citrix Content Collaboration ShareFile - Remote Code Execution - CVE:CVE-2023-24489	N/A	Block	

WAF - WAF Release - 2025-02-24

Published on: Mon, 24 Feb 2025 00:00:00 GMT

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...4916911e	100718A	SonicWall SSLVPN 2 - Auth Bypass - CVE:CVE-2024-53704	Log	Block	This is a New Detection
CIS Managed Ruleset	...c382fdec	100720	Palo Alto Networks - Auth Bypass - CVE:CVE-2025-0108	Log	Block	This is a New Detection

WAF - WAF Release - 2025-02-18

Published on: Tue, 18 Feb 2025 00:00:00 GMT

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...a2ffa4b8	100715	FortiOS - Auth Bypass - CVE:CVE-2024-55591	Log	Block	This is a New Detection
CIS Managed Ruleset	...5a883e12	100716	Ivanti - Auth Bypass - CVE:CVE-2021-44529	Log	Block	This is a New Detection
CIS Managed Ruleset	...958094d3	100717	SimpleHelp - Auth Bypass - CVE:CVE-2024-57727	Log	Block	This is a New Detection
CIS Managed Ruleset	...3b66df22	100718	SonicWall SSLVPN - Auth Bypass - CVE:CVE-2024-53704	Log	Block	This is a New Detection
CIS Managed Ruleset	...9184699f	100719	Yeti Platform - Auth Bypass - CVE:CVE-2024-46507	Log	Block	This is a New Detection

WAF - WAF Release - 2025-02-11

Published on: Tue, 11 Feb 2025 00:00:00 GMT

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...483b4c26	100708	Aviatrix Network - Remote Code Execution - CVE:CVE-2024-50603	Log	Block	This is a New Detection
CIS Managed Ruleset	...7e924ca3	100709	Next.js - Remote Code Execution - CVE:CVE-2024-46982	Log	Disabled	This is a New Detection
CIS Managed Ruleset	...83a7d8ff	100710	Progress Software WhatsUp Gold - Directory Traversal - CVE:CVE-2024-12105	Log	Block	This is a New Detection

CIS Managed Ruleset	...baa8eb34	100711	WordPress - Remote Code Execution - CVE:CVE-2024-56064	Log	Block	This is a New Detection
CIS Managed Ruleset	...87f5d34e	100712	WordPress - Remote Code Execution - CVE:CVE-2024-9047	Log	Block	This is a New Detection
CIS Managed Ruleset	...bf72cf8a	100713	FortiOS - Auth Bypass - CVE:CVE-2022-40684	Log	Block	This is a New Detection

WAF - Updated leaked credentials database

Published on: Tue, 04 Feb 2025 00:00:00 GMT

Added new records to the leaked credentials database from a third-party database.

WAF - WAF Release - 2025-01-21

Published on: Tue, 21 Jan 2025 00:00:00 GMT

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...b090ba9a	100303	Command Injection - Nslookup	Log	Block	This was released as ...b8d152f4
CIS Managed Ruleset	...49e6b538	100534	Web Shell Activity	Log	Block	This was released as ...82fe4e7f

WAF - WAF Release - 2025-01-13

Published on: Mon, 13 Jan 2025 00:00:00 GMT

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Managed Ruleset	...f49e5840	100704	Cleo Harmony - Auth Bypass - CVE:CVE-2024-55956, CVE:CVE-2024-55953	Log	Block	New Detection
CIS Managed Ruleset	...a6d43bc2	100705	Sentry - SSRF	Log	Block	New Detection
CIS Managed Ruleset	...ce6311bb	100706	Apache Struts - Remote Code Execution - CVE:CVE-2024-53677	Log	Block	New Detection
CIS Managed Ruleset	...2233da1f	100707	FortiWLM - Remote Code Execution - CVE:CVE-2023-48782, CVE:CVE-2023-34993, CVE:CVE-2023-34990	Log	Block	New Detection
CIS Managed Ruleset	...e31d972a	100007C_BETA	Command Injection - Common Attack Commands		Disabled	

WAF - WAF Release - 2025-01-06

Published on: Mon, 06 Jan 2025 00:00:00 GMT

Ruleset	Rule ID	Legacy Rule ID	Description	Previous Action	New Action	Comments
CIS Specials	...9da08beb	100678	Pandora FMS - Remote Code Execution - CVE:CVE-2024-11320	Log	Block	New Detection
CIS Specials	...ecdf3d02	100679	Palo Alto Networks - Remote Code Execution - CVE:CVE-2024-0012, CVE:CVE-2024-9474	Log	Block	New Detection
CIS Specials	...a40f2a35	100680	Ivanti - Command Injection - CVE:CVE-2024-37397	Log	Block	New Detection
CIS Specials	...58ae3c89	100681	Really Simple Security - Auth Bypass - CVE:CVE-2024-10924	Log	Block	New Detection
CIS Specials	...e37f2da6	100682	Magento - XXE - CVE:CVE-2024-34102	Log	Block	New Detection
CIS Specials	...5054c752	100683	CyberPanel - Remote Code Execution - CVE:CVE-2024-51567	Log	Block	New Detection
CIS Specials	...dfe93d7b	100684	Microsoft SharePoint - Remote Code Execution - CVE:CVE-2024-38094, CVE:CVE-2024-38024, CVE:CVE-2024-38023	Log	Block	New Detection
CIS Specials	...1454c856	100685	CyberPanel - Remote Code Execution - CVE:CVE-2024-51568	Log	Block	New Detection
CIS Specials	...e92362e5	100686	Seeyon - Remote Code Execution	Log	Block	New Detection
CIS Specials	...b9f1c9f8	100687	WordPress - Remote Code Execution - CVE:CVE-2024-10781, CVE:CVE-2024-10542	Log	Block	New Detection
CIS Specials	...0d7ca374	100688	ProjectSend - Remote Code Execution - CVE:CVE-2024-11680	Log	Block	New Detection
CIS Specials	...a5260b70	100689	Palo Alto GlobalProtect - Remote Code Execution - CVE:CVE-2024-5921	Log	Block	New Detection
CIS Specials	...d007118b	100690	Ivanti - Remote Code Execution - CVE:CVE-2024-37404	Log	Block	New Detection
CIS Specials	...c3e49f64	100691	Array Networks - Remote Code Execution - CVE:CVE-2023-28461	Log	Block	New Detection
CIS Specials	...fcc6f5bb	100692	CyberPanel - Remote Code Execution - CVE:CVE-2024-51378	Log	Block	New Detection
CIS Specials	...b615335e	100693	Symfony Profiler - Auth Bypass - CVE:CVE-2024-50340	Log	Block	New Detection

CIS Specials	...09d08c8a	100694	Citrix Virtual Apps - Remote Code Execution - CVE:CVE-2024-8069	Log	Block	New Detection
CIS Specials	...8aafb2f5	100695	MSMQ Service - Remote Code Execution - CVE:CVE-2023-21554	Log	Block	New Detection
CIS Specials	...11b7a8c7	100696	Nginxui - Remote Code Execution - CVE:CVE-2024-49368	Log	Block	New Detection
CIS Specials	...45954c7e	100697	Apache ShardingSphere - Remote Code Execution - CVE:CVE-2022-22733	Log	Block	New Detection
CIS Specials	...f5311209	100698	Mitel MiCollab - Auth Bypass - CVE:CVE-2024-41713	Log	Block	New Detection
CIS Specials	...b3e5e46e	100699	Apache Solr - Auth Bypass - CVE:CVE-2024-45216	Log	Block	New Detection

CIS API reference

CIS API change log

In this change log, you can learn about the latest changes, improvements, and updates for the [Exposed Credentials Check ruleset](#), this allows users to define exactly which fields in your own login requests to check.

3 May 2026

Support for RayID log lookup

Support has been added to the CIS API for retrieving logs associated with a specific RayID from your domain's traffic. For more information, see [Get logs by RayID](#).

28 April 2026

Batch DNS records API enhancements

The Batch DNS records API has been updated to improve consistency and expand record support. The operation ID is now `batch_dns_records`, and the request and response schemas have been renamed for better clarity. Support for DNS record types `PTR` and `DS` has been added, while `SPF` is no longer supported. For more information, see [Batch DNS Records API](#) to understand the updated schema and supported record types.

Custom list items API improvements

The `GET` custom list items endpoint now supports additional query parameters to enhance pagination and filtering. You can use `cursor` and `per_page` for pagination, and the `search` parameter to filter results based on specific criteria. For more information, see [Custom list items API](#).

Zone settings API enhancements

Support has been added to the CIS API for retrieving and updating zone-level settings, including the security level and email address obfuscation. For more information, see [Get email address obfuscation](#), [Update email address obfuscation](#), and [Set security-level setting](#).

03 December 2025

Custom and CIS-managed list support

You can now create and manage custom lists or use CIS-managed lists through the API. You can list all [managed](#) and [custom lists](#), [create](#) a custom list, [retrieve](#) or [update](#) an existing custom list, or [delete](#) one you no longer need. List items can also be managed with the API: you can [get](#), [create](#), [update](#), or [delete](#) individual items, update all items at once, [retrieve](#) a single list item, or [check the status](#) of a list operation.

For more information, see [managed and custom lists](#) and explore the available API methods to start integrating these capabilities into your workflow.

28 February 2025

Ruleset Engine migration

Two existing features are now part of the Ruleset Engine rules language, and have been updated to use the [Migrating to WAF custom rules](#).

3. Rate-limiting rules: The new version of these rules allows you to apply rate-limiting to more specific kinds of traffic. For more information, see [Migrating to new rate-limiting rules](#).

16 December 2024

CIS Enterprise-level plans can now configure the [Logpush jobs](#) to push request logs to IBM Cloud Logs with the API. For more information, see [Managed rules feature](#) has been added to the CIS API. See [Zone rulesets](#) for the available options.

30 May 2023

The [zone hold](#) feature has been added to the CIS API.

30 November 2022

The [Create rate limit](#) API updated the `period` parameter to change the time during which to count traffic.

- The minimum time in seconds changed from `1` to `10`
- The maximum time in seconds changed from `3600` to `86400`

CIS CLI reference

IBM Cloud® Internet Services has several families of commands that are available from the command line interface (CLI). Find the full set of commands for IBM Cloud Internet Services (CIS) within each set, such as `Create`, `Delete`, and `Update`.

Before you begin

1. Download the [IBM CLI](#).
2. Log in to IBM Cloud.

```
$ ibmcloud login -a
```

3. Install the CIS CLI plug-in.

```
$ ibmcloud plugin install cis
```

4. Set the context instance.

```
$ ibmcloud cis instance-set <instance-name>
```

To see a list of plug-ins and which versions are installed, run this command.

```
$ ibmcloud plugin list
```

The list returns whether the CLI has any updates available. Run the following command to update the CIS CLI plug-in.

```
$ ibmcloud plugin update cis
```

To learn about installing and configuring the IBM Cloud CLI, see [Getting started with the IBM Cloud CLI](#).

Access application

ibmcloud cis access-app-create

Enterprise Plans Only

Create an access application for a DNS domain.

```
$ ibmcloud cis access-app-create DNS_DOMAIN_ID --name NAME --domain DOMAIN [--session-duration SESSION_DURATION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Create an access application for domain `31984fea73a15b45779fa0df4ef62f9b`.

```
$ ibmcloud cis access-app-create DNS_DOMAIN_ID --name exampleCreate --domain example.com --session-duration 12h
```

ibmcloud cis access-apps

Enterprise Plans Only

List all access applications for a DNS domain.

```
$ ibmcloud cis access-apps DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

List all access applications for domains `31984fea73a15b45779fa0df4ef62f9b`.

```
$ ibmcloud cis access-apps DNS_DOMAIN_ID
```

ibmcloud cis access-app

Enterprise Plans Only

Show details of an access application.

```
$ ibmcloud cis access-app DNS_DOMAIN_ID ACCESS_APPLICATION_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

ACCESS_APPLICATION_ID

The ID of the access application. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Show details of access application `a5836c2a7ea72d2e225890caea70ae32`.

```
$ ibmcloud cis access-app DNS_DOMAIN_ID ACCESS_APPLICATION_ID
```

ibmcloud cis access-app-update

Enterprise Plans Only

Update an access application.

```
$ ibmcloud cis access-app-update DNS_DOMAIN_ID ACCESS_APPLICATION_ID --name NAME --domain DOMAIN [--session-duration SESSION_DURATION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`ACCESS_APPLICATION_ID`

The ID of the access application. Required.

`--name`

The name of the Application. Required.

`--domain`

The domain and path that Access blocks. Required.

`--session-duration`

Defines the amount of time that the tokens issued for this application are valid. Valid values are `30m`, `6h`, `12h`, `24h`, `168h`, and `730h`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update access application `a5836c2a7ea72d2e225890caea70ae32`.

```
$ ibmcloud cis access-app-update DNS_DOMAIN_ID ACCESS_APPLICATION_ID --name exampleUpdate --domain example.com --session-duration 24h
```

ibmcloud cis access-app-delete

Enterprise Plans Only

Delete an access application.

```
$ ibmcloud cis access-app-delete DNS_DOMAIN_ID ACCESS_APPLICATION_ID [-i, --instance INSTANCE]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

ACCESS_APPLICATION_ID

The ID of the access application. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete access application `a5836c2a7ea72d2e225890caea70ae32`.

```
$ ibmcloud cis access-app-delete DNS_DOMAIN_ID ACCESS_APPLICATION_ID
```

Access certificate

ibmcloud cis access-certificate-create

Enterprise Plans Only

Create an access certificate for a DNS domain.

```
$ ibmcloud cis access-certificate-create DNS_DOMAIN_ID --name NAME --ca-cert-file CERT_FILE [--associated-hostnames ASSOCIATED_HOSTNAMES] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--name

The name of the Certificate. Required.

--ca-cert-file

The Root CA file for your certificates. Required.

--associated-hostnames

The hostnames that are prompted for this certificate.

ACCESS_APPLICATION_ID

The ID of the access application.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Create an access certificate for domain `31984fea73a15b45779fa0df4ef62f9b`.

```
$ ibmcloud cis access-certificate-create DNS_DOMAIN_ID --name example --ca-cert-file CERT_FILE --associated-hostnames example.com
```

ibmcloud cis access-certificates

Enterprise Plans Only

List all access certificates for a DNS domain.

```
$ ibmcloud cis access-certificates DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

List all access certificates for domain `31984fea73a15b45779fa0df4ef62f9b`.

```
$ ibmcloud cis access-certificates DNS_DOMAIN_ID
```

ibmcloud cis access-certificate

Enterprise Plans Only

Show details of an access certificate.

```
$ ibmcloud cis access-certificate DNS_DOMAIN_ID ACCESS_CERTIFICATE_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

ACCESS_CERTIFICATE_ID

The ID of the access certificate. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show details of access certificate `a5836c2a7ea72d2e225890caea70ae32`.

```
$ ibmcloud cis access-certificate DNS_DOMAIN_ID ACCESS_CERTIFICATE_ID
```

ibmcloud cis access-certificate-update

Enterprise Plans Only

Update an access certificate.

```
$ ibmcloud cis access-certificate-update DNS_DOMAIN_ID ACCESS_CERTIFICATE_ID --name NAME --associated-hostnames ASSOCIATED_HOSTNAMES [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

ACCESS_CERTIFICATE_ID

The ID of the access certificate. Required.

`--name`

The name of the Certificate. Required.

`--associated-hostnames`

The hostnames that are prompted for this certificate. Required.



Note: The associated hostnames are reset if not specified by `associated-hostnames` .

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update the access certificate `a5836c2a7ea72d2e225890caea70ae32` .

```
$ ibmcloud cis access-certificate-update DNS_DOMAIN_ID ACCESS_CERTIFICATE_ID --name example --associated-hostnames example.com
```

ibmcloud cis access-certificate-delete

Enterprise Plans Only

Delete an access certificate.

```
$ ibmcloud cis access-certificate-delete DNS_DOMAIN_ID ACCESS_CERTIFICATE_ID [-i, --instance INSTANCE]
```



Note: Must clear the associated hostnames before you delete the certificate.

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`ACCESS_CERTIFICATE_ID`

The ID of the access certificate. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete the access certificate `a5836c2a7ea72d2e225890caea70ae32` .

```
$ ibmcloud cis access-certificate-delete DNS_DOMAIN_ID ACCESS_CERTIFICATE_ID
```

ibmcloud cis access-certificates-settings

Enterprise Plans Only

Get access certificates settings for a DNS domain.

```
$ ibmcloud cis access-certificates-settings DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Get access certificates settings for Domain `31984fea73a15b45779fa0df4ef62f9b`.

```
$ ibmcloud cis access-certificates-settings DNS_DOMAIN_ID
```

ibmcloud cis access-certificates-settings-update

Enterprise Plans Only

Update access certificates settings for a DNS domain.

```
$ ibmcloud cis access-certificates-settings-update DNS_DOMAIN_ID (-f, --feature FEATURE) (-v, --value VALUE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-f, --feature

Feature of certificates settings. Valid values:

client_certificate_forwarding

The client certificate payload and its SHA256 signature are forwarded to origin servers through `CF-Client-Cert-DER_BASE64` and `CF-Client-Cert-SHA256` headers.

-v, --value

The value set to the feature for certificates.

client_certificate_forwarding

Specify the hostname to forward the client certificate or not. For example, `-v host1=on,host2=on,host3=off`.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update access certificates settings for Domain `31984fea73a15b45779fa0df4ef62f9b`.

```
$ ibmcloud cis access-certificates-settings-update DNS_DOMAIN_ID -f client_certificate_forwarding -v mtls1.example.com=on,mtls2.example.com=off
```

Access policy

ibmcloud cis access-policy-create

Enterprise Plans Only

Create an access policy for an access application.

```
$ ibmcloud cis access-policy-create DNS_DOMAIN_ID ACCESS_APPLICATION_ID --name NAME --decision DECISION --include INCLUDE [-i, --instance INSTANCE] [-output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

ACCESS_APPLICATION_ID

The ID of the access application. Required.

--name

The name of the policy. Required.

--decision

Defines the action Access takes if the policy matches the user. Valid value is **non_identity**. Required.

--include

The included rule of the policy. Valid values are **certificate** and **common_name**. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

Create an access policy for access application **a5836c2a7ea72d2e225890caea70ae32**.

```
$ ibmcloud cis access-policy-create DNS_DOMAIN_ID ACCESS_APPLICATION_ID --name examplePolicy --decision non_identity --include certificate --include common_name=test
```

ibmcloud cis access-policies

Enterprise Plans Only

List all access policies for an access application.

```
$ ibmcloud cis access-policies DNS_DOMAIN_ID ACCESS_APPLICATION_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

ACCESS_APPLICATION_ID

The ID of the access application. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

List all access policies for access application **a5836c2a7ea72d2e225890caea70ae32**.

```
$ ibmcloud cis access-policies DNS_DOMAIN_ID ACCESS_APPLICATION_ID
```

ibmcloud cis access-policy

Enterprise Plans Only

Show details of an access policy.

```
$ ibmcloud cis access-policy DNS_DOMAIN_ID ACCESS_APPLICATION_ID ACCESS_POLICY_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

ACCESS_APPLICATION_ID

The ID of the access application. Required.

ACCESS_POLICY_ID

The ID of access policy. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show details of access policy `a5836c2a7ea72d2e225890caea70ae32`.

```
$ ibmcloud cis access-policy DNS_DOMAIN_ID ACCESS_APPLICATION_ID ACCESS_POLICY_ID
```

ibmcloud cis access-policy-delete

Enterprise Plans Only

Delete an access policy.

```
$ ibmcloud cis access-policy-delete DNS_DOMAIN_ID ACCESS_APPLICATION_ID ACCESS_POLICY_ID [-i, --instance INSTANCE]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

ACCESS_APPLICATION_ID

The ID of the access application. Required.

ACCESS_POLICY_ID

The ID of access policy. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete access policy `65fe21071877669cc69544642bc6c4c4`.

```
$ ibmcloud cis access-policy-delete DNS_DOMAIN_ID ACCESS_APPLICATION_ID ACCESS_POLICY_ID
```

Cache

Manipulate how the cache performs by using the following `cache` commands:

ibmcloud cis cache-purge

Clear the cached assets file by file or entirely for a DNS domain to guarantee that the served assets are updated.

```
$ ibmcloud cis cache-purge DNS_DOMAIN_ID (--all | --file file1 --file file2... | --tag tag1 --tag tag2...|--host host1 --host host...| --prefix prefix1 --prefix prefix2...) [-f, --force] [-i, --instance INSTANCE_NAME] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--all`

Purging all cached files. This option is mutually exclusive with `--file`.

`--file`

Granularly remove one or more files by specifying URLs. This option is mutually exclusive with `--all`.

`--tag`

Granularly remove one or more files by the associated Cache-Tag. This option is mutually exclusive with `--all`. Enterprise Plans Only

`--host`

Granularly remove one or more files by specifying the host. This option is mutually exclusive with `--all`. Enterprise Plans Only

`--prefix`

Granularly remove one or more files by a prefix. This option is mutually exclusive with `--all`. Enterprise Plans Only

`-f, --force`

Purging all cached files without prompting for confirmation.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Clear all cached assets file for domain `31984fea73a15b45779fa0df4ef62f9b`.

```
$ ibmcloud cis cache-purge DNS_DOMAIN_ID --all --force
```

ibmcloud cis cache-settings

Get caching settings for a DNS domain.

```
$ ibmcloud cis cache-settings DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get caching settings for domain `31984fea73a15b45779fa0df4ef62f9b` .

```
$ ibmcloud cis cache-settings DNS_DOMAIN_ID
```

ibmcloud cis cache-settings-update

Update cache settings for a give DNS domain.

```
$ ibmcloud cis cache-settings-update DNS_DOMAIN_ID [--caching-level LEVEL][--browser-expiration EXPIRATION] [--development-mode (on | off)] [--serve-stale-content (on | off)] [--query-string-sort (on | off)] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--caching-level

Specify under what URL conditions that you want to deliver cached assets to the user. Valid values are `no-query-string` , `query-string-independent` , and `query-string-dependent` .

- `no-query-string` : Delivers resources from cache only when no query string is present.
- `query-string-independent` : Delivers the same resource to everyone independent of the query string.
- `query-string-dependent` : Delivers a different resource each time the query string changes.

--browser-expiration

Specify how long you want the browser to store cached assets.

- Valid values are `respect-existing-header` , `30s` , `1M` , `5M` , `20M` , `30M` , `1h` , `2h` , `4h` , `8h` , `16h` , `1d` , `3d` , `8d` , `16d` , `1m` , `6m` , and `1y` .
- `30s` , `1M` , `5M` , and `20M` are only available for an Enterprise or Security plan instance.
- `30s` means `30 seconds` .
- `30M` means `30 minutes` .
- `1h` means `1 hour` .
- `1d` means `1 day` .
- `1m` means `1 month` .
- `1y` means `1 year` .

--development-mod

Bypass all edge caches and send traffic toward your origin servers.

--serve-stale-content

Continue serving cached content to users when origin servers are offline, even if the content is expired.

`--query-string-sort` : In the cache, CIS treats files with the same query strings as the same file, regardless of the order of the query strings.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update caching settings for domain `31984fea73a15b45779fa0df4ef62f9b` .

```
$ ibmcloud cis cache-settings-update DNS_DOMAIN_ID --caching-level no-query-string --browser-expiration 1h
```


Custom lists

Manipulate how the custom list performs by using the following `custom-lists` commands:

`ibmcloud cis custom-lists lists`

List the custom lists for your instance.

```
$ ibmcloud cis custom-lists lists [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Example

```
$ ibmcloud cis custom-lists lists
```

`ibmcloud cis custom-lists list`

Get a custom list for your instance.

```
$ ibmcloud cis custom-lists list LIST_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`LIST_ID`

The ID of the custom list.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Example

```
$ ibmcloud cis custom-lists list LIST_ID --output json
```

`ibmcloud cis custom-lists list-create`

Create a custom list for your instance.

```
$ ibmcloud cis custom-lists list-create (--kind KIND) (--name NAME) [--description DESCRIPTION] [-i, --instance INSTANCE]
```

You can also accept JSON input (from a file or directly as a string):

```
$ ibmcloud cis custom-lists list-create (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE]
```

Command options

`--kind`

Custom list kind. Valid values are `ip` , `asn` , and `hostname` .

`--name`

The list name.

`--description`

Description of the list.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

`--json`

The JSON file or JSON string that is used to describe a custom list.

The required fields in JSON data are:

`kind` : Custom list kind. Valid values are `ip` , `asn` , and `hostname` .

`name` : The list name.

The optional field is:

`"description"` : Description of the list.

Sample JSON data:

```
{
  "kind": "ip",
  "name": "string",
  "description": "string"
}
```

Examples

```
$ ibmcloud cis custom-lists list-create --kind ip --name iplistone --instance INSTANCE
```

```
$ ibmcloud cis custom-lists list-create --json @example.json --instance INSTANCE
```

ibmcloud cis custom-lists list-update

Update the description of a custom list.

```
$ ibmcloud cis custom-lists list-update LIST_ID (--description DESCRIPTION) [-i, --instance INSTANCE] [--output FORMAT]
```

```
$ ibmcloud cis custom-lists list-update LIST_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`LIST_ID`

The ID of the custom list.

`--description`

Briefly describe the list.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

json

The JSON file or JSON string that is used to describe a custom list.

The optional field is:

"description" : To briefly describe the list.

Sample JSON data:

```
{
  "description": "string"
}
```

Examples

```
$ ibmcloud cis custom-lists list-update LIST_ID --description "description example"
```

```
$ ibmcloud cis custom-lists list-update LIST_ID --json @example.json
```

ibmcloud cis custom-lists list-delete

Delete a custom list for your instance.

```
$ ibmcloud cis custom-lists list-delete LIST_ID [-f, --force] [-i, --instance INSTANCE]
```

Command options

LIST_ID

The ID of the custom list.

-f, --force

Attempt to delete a custom list without prompting for confirmation.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

Example

```
$ ibmcloud cis custom-lists list-delete LIST_ID
```

ibmcloud cis custom-lists items

```
$ ibmcloud cis custom-lists items LIST_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

LIST_ID

The ID of the custom list.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Example

```
$ ibmcloud cis custom-lists items LIST_ID
```

ibmcloud cis custom-lists item

View a specific item in a custom list.

```
$ ibmcloud cis custom-lists item LIST_ID ITEM_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

LIST_ID

The ID of the custom list.

ITEM_ID

The ID of the custom list item.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Example

```
$ ibmcloud cis custom-lists item LIST_ID ITEM_ID
```

ibmcloud cis custom-lists item-create

Create a new item in a custom list.

```
$ ibmcloud cis custom-lists item-create LIST_ID (--asn ASN | --ip IP | --hostname HOSTNAME) [--comment COMMENT] [-i, --instance INSTANCE]
```

```
$ ibmcloud cis custom-lists item-create LIST_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE]
```

Command options

LIST_ID

The ID of the custom list.

--asn

The ASN value.

--ip

The IPv4 address.

--hostname

The hostname.

--comment

To provide a brief comment on the item.

-i, --instance

Instance name or ID. If instance or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

--json

The JSON file or JSON string that is used to describe a custom list.

The required fields in JSON data are:

items : List of custom list items to create.

asn : The ASN.

hostname : The hostname.

ip : The IPv4 address.

comment : To provide a brief comment on the item.

Sample JSON data:

```
[
  {
    "asn": 19604,
    "comment": "My list of developer IPs.",
    "hostname": "cloud.ibm.com",
    "ip": "172.64.0.0/13"
  }
]
```

Command examples

```
$ ibmcloud cis custom-lists item-create LIST_ID --ip 192.0.0.3
```

```
$ ibmcloud cis custom-lists item-create LIST_ID --json @example.json
```

ibmcloud cis custom-lists item-update

Update all list items for your custom list.

```
$ ibmcloud cis custom-lists item-update LIST_ID (--json @JSON_FILE | JSON_STRING) [-f, --force] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

LIST_ID

The ID of the custom list.

--json

The JSON file or JSON string that is used to describe a custom list.

The required fields in JSON data are:

items : List of custom list items to create.

asn : The ASN value.

hostname : The hostname.

ip : The IPv4 address.

comment : To provide a brief comment on the item.

Sample JSON data:

```
[
  {
    "asn": 19604,
    "comment": "My list of developer IPs.",
    "hostname": "cloud.ibm.com",
    "ip": "172.64.0.0/13"
  }
]
```

-f, --force

Attempt to delete a custom list without prompting for confirmation.

`--output value`

The output format. Currently, `json` is the only supported value.

`-i, --instance`

Instance name or ID. If instance value or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

Example

```
$ ibmcloud cis custom-lists item-update LIST_ID --json @example.json -f
```

ibmcloud cis custom-lists item-delete

Delete an item from a custom list.

```
$ ibmcloud cis custom-lists item-delete LIST_ID (--item-id CUSTOM_LIST_ITEM_ID) [-f, --force] [-i, --instance INSTANCE]
```

```
$ ibmcloud cis custom-lists item-delete LIST_ID (--json @JSON_FILE | JSON_STRING) [-f, --force] [-i, --instance INSTANCE]
```

Command options

`LIST_ID`

The ID of the custom list.

`--item-id`

`CUSTOM_LIST_ITEM_ID` is the ID of the custom list item.

`-f, --force`

Attempt to delete a custom list without prompting for confirmation.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

`--json`

The JSON file or JSON string that is used to describe a custom list.

The required fields in JSON data are:

`items` : List of custom list items to delete by ID.

`id` : Unique ID of the custom list item.

Sample JSON data:

```
{
  "items": [
    {
      "id": "70c2009751b24ffc9ed1ab462ba957b4"
    }
  ]
}
```

Examples

```
$ ibmcloud cis custom-lists item-delete LIST_ID --item-id CUSTOM_LIST_ITEM_ID --force
```

```
$ ibmcloud cis custom-lists item-delete LIST_ID --json @example.json --force
```

ibmcloud cis custom-lists operation

Get the status for the custom list operation.

```
$ ibmcloud cis custom-lists operation OPERATION_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

OPERATION_ID

The ID of the custom list operation.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Example

```
$ ibmcloud cis custom-lists operation OPERATION_ID --output json
```

Custom page

Manipulate how the Custom Page performs by using the following `custom-page` commands:

ibmcloud cis custom-page-update

Update a specific custom page.

```
$ ibmcloud cis custom-page-update PAGE_ID PAGE_URL [-d, --domain DNS_DOMAIN_ID] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

PAGE_ID

The name of the Custom Page type. Valid values are `basic_challenge`, `country_challenge`, `ip_block`, `ratelimit_block`, `serve_stale_content`, `under_attack`, `waf_block`, `waf_challenge`, `1000_errors`, `500_errors`. Required.

PAGE_URL

A URL that is associated with the Custom Page. For example, `http://www.example.com/example.html`. Value `default` means to use the default page. Required.

-d, --domain

DNS Domain ID.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update `basic_challenge` page for domain `31984fea73a15b45779fa0df4ef62f9b`.

```
$ ibmcloud cis custom-page-update basic_challenge http://www.example.com/example.html -d DNS_DOMAIN_ID
```

ibmcloud cis custom-page

Retrieve a specific custom page.


```
$ ibmcloud cis custom-page PAGE_ID [-d, --domain DNS_DOMAIN_ID] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

PAGE_ID

The name of the Custom Page type. Valid values are `basic_challenge`, `country_challenge`, `ip_block`, `ratelimit_block`, `serve_stale_content`, `under_attack`, `waf_block`, `waf_challenge`, `1000_errors`, and `500_errors`. Required.

-d, --domain

DNS Domain ID.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Get `basic_challenge` page for domain `31984fea73a15b45779fa0df4ef62f9b`.

```
$ ibmcloud cis custom-page basic_challenge -d DNS_DOMAIN_ID
```

ibmcloud cis custom-pages

Retrieve a list of currently existing custom pages.

```
$ ibmcloud cis custom-pages [-d, --domain DNS_DOMAIN_ID] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

-d, --domain

DNS Domain ID.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

List existing custom pages for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis custom-pages -d DNS_DOMAIN_ID
```

DNS record

Manipulate how the DNS Record performs by using the following `dns-record` commands:

ibmcloud cis dns-record-create

Create a DNS record for a domain of a service instance.

```
$ ibmcloud cis dns-record-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis dns-record-create DNS_DOMAIN_ID --type TYPE --name NAME --content CONTENT [--ttl TTL] [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis dns-record-create DNS_DOMAIN_ID --json-str JSON_STR [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis dns-record-create DNS_DOMAIN_ID --json-file JSON_FILE [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--name

DNS record name.

--type

DNS record type.

--content

DNS record content.

--ttl

Time to live for DNS record. A value of 1 is `automatic`. The default value is `1`.

--proxied

Control whether traffic flows through the security and performance functions on CIS. CIS proxies traffic only for `A`, `AAAA`, and `CNAME` records. Valid values are `true` and `false`.

--json

The JSON file or JSON string that is used to describe a DNS Record. Supported DNS Record types are: `A`, `AAAA`, `CNAME`, `NS`, `TXT`, `MX`, `LOC`, `SRV`, `CAA`, `PTR`.

- For type `A`, `AAAA`, `CNAME`, `NS`, `TXT` :
 - The required fields in JSON data are `name`, `type`, `content`.
 - The optional fields are `ttl`, `proxied` :
 - `proxied` Control whether traffic flows through the security and performance functions on CIS. CIS proxies only traffic for `A`, `AAAA`, and `CNAME` records.

Sample JSON data:

```
{
  "name": "testA",
  "type": "A",
  "content": "127.0.0.1",
  "proxied": true
}

{
  "name": "testAAAA",
  "type": "AAAA",
  "content": "2001:0db8:0012:0001:3c5e:7354:0000:5db1",
  "proxied": false
}

{
  "name": "testCNAME",
  "type": "CNAME",
  "content": "example.com"
}

{
  "name": "testNS",
  "type": "NS",
  "content": "ns1.example.com"
}

{
  "name": "testTXT",
  "type": "TXT",
```



```
"content": "text information"
}
```

- For type **PTR** :
 - The required fields in JSON data are **name** , **type** , **content** .
 - The optional field is **ttl** .

Sample JSON data:

```
{
  "name": "1.2.3.4",
  "type": "PTR",
  "content": "abc.test.com"
}
```

- For type **MX** :
 - The required fields in JSON data are **name** , **type** , **content** .
 - The optional fields are **ttl** and **priority** .

Sample JSON data:

```
{
  "name": "testMX",
  "type": "MX",
  "content": "smtp.example.com",
  "priority": 10
}
```

- For type **LOC** :
 - The required fields in JSON data are **name** , **type** , **data** :
 - **data** :
 - **lat_degrees** : Degrees of latitude.
 - **lat_minutes** : Minutes of latitude
 - **lat_seconds** : Seconds of latitude.
 - **lat_direction** : Latitude direction.
 - **long_degrees** : Degrees of longitude.
 - **long_minutes** : Minutes of longitude.
 - **long_seconds** : Seconds of longitude.
 - **long_direction** : Longitude direction.
 - **altitude** : Altitude of location in meters.
 - **size** : Size of location in meters.
 - **precision_horz** : Horizontal precision of location.
 - **precision_vert** : Vertical precision of location.
 - The optional field is **ttl** .

Sample JSON data:

```
{
  "name": "testLOC",
  "type": "LOC",
  "data": {
    "lat_degrees": 45,
    "lat_minutes": 0,
    "lat_seconds": 0,
    "lat_direction": "N",
    "long_degrees": 45,
    "long_minutes": 0,
    "long_seconds": 0,
    "long_direction": "E",
    "altitude": 20,
    "size": 0,

```

```
"precision_horz": 0,
"precision_vert": 0
}
}
```

- For type `SRV` :
 - The required fields in JSON data are `type` , `data` :
 - `data` : - `service` : A service type, prefixed with an underscore. - `proto` : A valid protocol. - `priority` : Priority. - `weight` : The record weight. - `port` : The port of the service. - `target` : A valid hostname.
 - The optional field is `ttl` .

Sample JSON data:

```
{
  "type": "SRV",
  "data": {
    "service": "_ftp",
    "proto": "_tcp",
    "name": "testSRV",
    "priority": 1,
    "weight": 1,
    "port": 21,
    "target": "example.com"
  }
}
```

- For type `CAA` :
 - The required fields in JSON data are `name` , `type` , and `data` .
 - The optional field is `ttl` .

Sample JSON data:

```
{
  "name": "testCAA.yourdomain.com",
  "type": "CAA",
  "data": {
    "tag": "issue",
    "value": "letsencrypt.org"
  }
}
```

`-s, --json-str`

Deprecated. The JSON data used to describe a DNS Record.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a DNS record in the domain `DNS_DOMAIN_ID` .

```
ibmcloud cis dns-record-create DNS_DOMAIN_ID --json '{"name": "testCNAME", "type": "CNAME", "content": "example.com"}'
ibmcloud cis dns-record-create DNS_DOMAIN_ID --type A --name testA --content "127.0.0.1"
```

ibmcloud cis dns-record-update

Update a DNS record for a domain of a service instance.

```
$ ibmcloud cis dns-record-update DNS_DOMAIN_ID DNS_RECORD_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis dns-record-update DNS_DOMAIN_ID DNS_RECORD_ID [--type TYPE] [--name NAME] [--content CONTENT] [--proxied PROXIED] [--ttl TTL] [--output
FORMAT]
[Deprecated] ibmcloud cis dns-record-update DNS_DOMAIN_ID DNS_RECORD_ID --json-str JSON_STR [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis dns-record-update DNS_DOMAIN_ID DNS_RECORD_ID --json-file JSON_FILE [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

DNS_RECORD_ID

The ID of the DNS record. Required.

--name

DNS record name.

--type

DNS record type.

--content

DNS record content.

--ttl

Time to live for DNS record. A value of 1 is **automatic**. The default value is **1**.

--proxied

Control whether traffic flows through the security and performance functions on CIS. CIS proxies traffic only for **A**, **AAAA**, and **CNAME** records. Valid values are **true** and **false**.

--json

The JSON file or JSON string that is used to describe a DNS Record. Supported DNS Record types are: **A**, **AAAA**, **CNAME**, **NS**, **TXT**, **MX**, **LOC**, **SRV**, **CAA**, **PTR**.

- For type **A**, **AAAA**, **CNAME**, **NS**, **TXT** :
 - The required fields in JSON data are **name**, **type**, **content**.
 - The optional fields are **ttl** and **proxied** :
 - proxied** Control whether traffic flows through the security and performance functions on CIS. CIS proxies only traffic for **A**, **AAAA**, and **CNAME** records.

Sample JSON data:

```
{
  "name": "testA",
  "type": "A",
  "content": "127.0.0.1",
  "proxied": true
}

{
  "name": "testAAAA",
  "type": "AAAA",
  "content": "2001:0db8:0012:0001:3c5e:7354:0000:5db1",
  "proxied": false
}

{
  "name": "testCNAME",
  "type": "CNAME",
  "content": "example.com"
}
```

```
{
  "name": "testNS",
  "type": "NS",
  "content": "ns1.example.com"
}

{
  "name": "testTXT",
  "type": "TXT",
  "content": "text information"
}
```

- For type `PTR` :
 - The required fields in JSON data are `name` , `type` , `content` .
 - The optional field is `ttl` .

Sample JSON data:

```
{
  "name": "1.2.3.4",
  "type": "PTR",
  "content": "abc.test.com"
}
```

- For type `MX` :
 - The required fields in JSON data are `name` , `type` , `content` .
 - The optional fields are `ttl` and `priority` .

Sample JSON data:

```
{
  "name": "testMX",
  "type": "MX",
  "content": "smtp.example.com",
  "priority": 10
}
```

- For type `LOC` :
 - The required fields in JSON data are `name` , `type` , `data` :
 - `data` :
 - `lat_degrees` : Degrees of latitude.
 - `lat_minutes` : Minutes of latitude
 - `lat_seconds` : Seconds of latitude.
 - `lat_direction` : Latitude direction.
 - `long_degrees` : Degrees of longitude.
 - `long_minutes` : Minutes of longitude.
 - `long_seconds` : Seconds of longitude.
 - `long_direction` : Longitude direction.
 - `altitude` : Altitude of location in meters.
 - `size` : Size of location in meters.
 - `precision_horz` : Horizontal precision of location.
 - `precision_vert` : Vertical precision of location.
 - The optional field is `ttl` .

Sample JSON data:

```
{
  "name": "testLOC",
  "type": "LOC",
```

```
"data": {
  "lat_degrees": 45,
  "lat_minutes": 0,
  "lat_seconds": 0,
  "lat_direction": "N",
  "long_degrees": 45,
  "long_minutes": 0,
  "long_seconds": 0,
  "long_direction": "E",
  "altitude": 20,
  "size": 0,
  "precision_horz": 0,
  "precision_vert": 0
}
}
```

- For type `SRV` :
 - The required fields in JSON data are `type` , `data` :
 - `data` :
 - `service` : A service type, prefixed with an underscore.
 - `proto` : A valid protocol.
 - `priority` : Priority.
 - `weight` : The record weight.
 - `port` : The port of the service.
 - `target` : A valid hostname.
 - The optional field is `ttl` .

Sample JSON data:

```
{
  "type": "SRV",
  "data": {
    "service": "_ftp",
    "proto": "_tcp",
    "name": "testSRV",
    "priority": 1,
    "weight": 1,
    "port": 21,
    "target": "example.com"
  }
}
```

- For type `CAA` :
 - The required fields in JSON data are `name` , `type` , `data`
 - The optional field is `ttl` .

Sample JSON data:

```
{
  "name": "testCAA.yourdomain.com",
  "type": "CAA",
  "data": {
    "tag": "issue",
    "value": "letsencrypt.org"
  }
}
```

`-s, --json-str`

Deprecated. The JSON data used to describe a DNS Record.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update a DNS record in the domain `DNS_DOMAIN_ID`.

```
ibmcloud cis dns-record-update DNS_DOMAIN_ID DNS_RECORD_ID --json '{"name": "testCNAME", "type": "CNAME", "content": "example.com"}'
ibmcloud cis dns-record-update DNS_DOMAIN_ID DNS_RECORD_ID --type A --name testA --content "127.0.0.1"
```

ibmcloud cis dns-record

Get a DNS record details for a domain under a service instance.

```
$ ibmcloud cis dns-record DNS_DOMAIN_ID DNS_RECORD_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`DNS_RECORD_ID`

The ID of the DNS record. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get DNS record details in the domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis dns-record DNS_DOMAIN_ID DNS_RECORD_ID
```

ibmcloud cis dns-record-delete

Delete a DNS record for a domain of a service instance.

```
$ ibmcloud cis dns-record-delete DNS_DOMAIN_ID DNS_RECORD_ID [-i, --instance INSTANCE]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`DNS_RECORD_ID`

The ID of the DNS record. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete a DNS record in the domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis dns-record-delete DNS_DOMAIN_ID DNS_RECORD_ID
```

ibmcloud cis dns-records

List all DNS records for a domain of a service instance.

```
$ ibmcloud cis dns-records DNS_DOMAIN_ID [--type TYPE] [--name NAME] [--content CONTENT] [--page PAGE] [--per-page PER_PAGE] [--order ORDER] [--direction DIRECTION] [--match MATCH] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--type`

Type of DNS records to display.

`--name`

Value of name field to filter by.

`--content`

Value of content field to filter by.

`--page`

Page number of paginated results.

`--per_page`

Maximum number of DNS records per page.

`--order`

Field by which to order the list of DNS records. Valid values are `type`, `name`, `content`, `ttl`, and `proxied`.

`--direction`

Direction in which to order the results (ascending or descending order). Valid values are `asc` and `desc`.

`--match`

Whether to match all or at least one search parameter. Valid values are `any` and `all`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List all DNS records in domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis dns-records DNS_DOMAIN_ID
```

ibmcloud cis dns-records-import

Import your BIND config.

```
$ ibmcloud cis dns-records-import DNS_DOMAIN_ID --file FILE [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--file`

BIND config to import. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Import BIND config in the domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis dns-records-import DNS_DOMAIN_ID --file bind_config_file.txt
```

ibmcloud cis dns-records-export

Export BIND config.

```
$ ibmcloud cis dns-records-export DNS_DOMAIN_ID [--file FILE] [-i, --instance INSTANCE]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--file`

The BIND config file that saves exported DNS records.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Export BIND config for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis dns-records-export DNS_DOMAIN_ID --file bind_config_file.txt
```

Domain

Manipulate domains by using the following `domain` commands.

ibmcloud cis domain-add

Add a domain.

```
$ ibmcloud cis domain-add DNS_DOMAIN_NAME [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`type`

Specify the domain type setup. Valid values are `full` and `partial` (default `full`).

- `full` : A full zone implies that the DNS is hosted.
- `partial` : A partial zone implies a CNAME setup domain.

`jump-start`

Automatically attempt to fetch existing DNS records.

DNS_DOMAIN_NAME

The FQDN of DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Add a domain `DNS_DOMAIN_NAME`.

```
$ ibmcloud cis domain-add DNS_DOMAIN_NAME
```

ibmcloud cis domain-resume

Resume the domain.

```
$ ibmcloud cis domain-resume DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Resume the specified domain.

```
$ ibmcloud cis domain-resume DNS_DOMAIN_ID
```

ibmcloud cis domain-pause

Pause the domain.

```
$ ibmcloud cis domain-pause DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Pause the specified domain.

```
$ ibmcloud cis domain-pause DNS_DOMAIN_ID
```

ibmcloud cis domain

Display the domain details.

```
$ ibmcloud cis domain DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Display the specified domain details.

```
$ ibmcloud cis domain DNS_DOMAIN_ID
```

ibmcloud cis domain-remove

Remove a domain.

```
$ ibmcloud cis domain-remove DNS_DOMAIN_ID [-i, --instance INSTANCE]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Remove the specified domain.

```
$ ibmcloud cis domain-remove DNS_DOMAIN_ID
```

ibmcloud cis domains

List domains for a service instance.

```
$ ibmcloud cis domains [--instance INSTANCE_NAME] [--output FORMAT]
```

Command options

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List domains for the service instance.

```
$ ibmcloud cis domains
```

`ibmcloud cis domain-activation-check`

Check the activation on the domain.

```
$ ibmcloud cis domain-activation-check DNS_DOMAIN_ID [-i, --instance INSTANCE]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Perform activation check on the specified domain.

```
$ ibmcloud cis domain-activation-check DNS_DOMAIN_ID
```

Domain settings

Manipulate domain settings by using the following `domain-settings` commands:

`ibmcloud cis domain-settings`

Get details of a feature for the domain.

```
$ ibmcloud cis domain-settings DNS_DOMAIN_ID [-g, --group GROUP | -f, --feature FEATURE] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-g, --group`

Display features in a same group. Valid values for `group` are `all`, `domain`, `reliability`, `performance`, and `security`. This option is mutually exclusive with `-f, --feature`.

`-f, --feature`

Feature of domain settings to check. This option is mutually exclusive with `g, --group`. Valid values are as follows:

- `always_use_https` : Redirect all requests with scheme `http` to `https`. This setting applies to all HTTP requests to the domain.
- `automatic_https_rewrites` : Help fix mixed content by changing `http` to `https` for all resources or links on your website that can be served with HTTPS.
- `bot_management` : Detect and mitigate bot traffic on your domain.
- `brotli` : When the client that is requesting an asset supports the brotli compression algorithm, CIS serves a brotli compressed version of the asset.
- `browser_check` : Evaluate HTTP headers from your visitors' browser for threats. If a threat is found, then a block page is delivered.

- `challenge_ttl` : Specify how long a visitor with a bad IP reputation is allowed access to your website after they complete a challenge.
- `ciphers` : An allowlist of ciphers for TLS termination in the BoringSSL format. This command lists ciphers that are allowlisted by customers. If no ciphers are allowlisted, the list is empty and the default ciphers are used. See [Edge cipher suites](#) and [Origin cipher suites](#) for the list of default ciphers.
- `cname_flattening` : Follow a CNAME to where it points and return that IP address instead of the CNAME record. By default, flatten only the CNAME at the root of your domain.
- `domain_hold` : Domain holds prevent teams in your organization from adding domains that are already active in another account.
Enterprise Plans Only
- `email_obfuscation` : Encrypt the email addresses on your web page from bots while it's kept visible to humans.
- `opportunistic_onion` : Allow legitimate users of the Tor Browser to access your websites.
- `hotlink_protection` : Protect your images from off-site linking.
- `http2` : Accelerate your website with HTTP/2.
- `http3` : Accelerate your website with HTTP/3.
- `image_load_optimization` : *Deprecated*. Improve load time for pages that include images on mobile devices with slow network connections.
- `image_size_optimization` : Improve image load time by optimizing images hosted on your domain.
- `image_resizing` : Provide on-demand resizing, conversion, and optimization for images served through the CIS network.
- `ip_geolocation` : Include the country code of the visitor location with all requests to your website.
- `ipv6` : Enable IPv6 support and gateway.
- `max_upload` : The number of data visitors who can upload to your website in a single request.
- `min_tls_version` : Allow only HTTPS connections from visitors that support the selected TLS protocol version or newer.
- `minify` : Reduce the file size of source code on your website.
- `mobile_redirect` : Redirect visitors that are using mobile devices to a mobile-optimized website.
- `opportunistic_encryption` : Opportunistic Encryption allows browsers to benefit from the improved performance of HTTP/2 by letting them know that your site is available over an encrypted connection.
- `origin_error_page_pass_thru` : When the Origin Error Page is set to `On`, CIS proxies the 502 and 504 error pages directly from the origin.
Enterprise Plans Only
- `origin_max_http_version` : Configure the HTTP version to Origin.
- `origin_post_quantum_encryption` : Instructs CIS to use Post-Quantum (PQ) key agreement algorithms when it connects to your origin.
- `prefetch_preload` : CIS prefetches any URLs included in the prefetch HTTP header. **Enterprise Plans Only**
- `pseudo_ipv4` : Adds an IPv4 header to requests when a client is using IPv6, but the server only supports IPv4.
- `response_buffering` : Enable or disable buffering of responses from the origin server. **Enterprise Plans Only**
- `script_load_optimization` : Improve the paint time for pages that include JavaScript.
- `security_header` : Enforce a web security policy for your website.
- `security_level` : Choose the appropriate security profile for your website.
- `server_side_exclude` : Automatically hide specific content from suspicious visitors.
- `tls_client_auth` : TLS client certificate presented for authentication on origin pull. **Enterprise Plans Only**
- `true_client_ip_header` : CIS sends the user's IP address in the True-Client-IP header. **Enterprise Plans Only**
- `waf` : A Web Application Firewall (WAF) blocks requests that contain malicious content.
- `websockets` : Allow WebSockets connections to your origin server.
- `proxy_read_timeout` : Maximum time between two read operations from origin. **Enterprise Plans Only**
- `url_normalization` : Modify the URLs of incoming requests.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get `ciphers` settings for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis domain-settings DNS_DOMAIN_ID -f ciphers
```


ibmcloud cis domain-settings-update

Update a feature for the domain.

```
$ ibmcloud cis domain-settings-update DNS_DOMAIN_ID (-f, --feature FEATURE) (-v, --value VALUE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-f, --feature

Feature of domain settings to update. Required. Valid values:

- **always_use_https** : Redirect all requests with scheme **http** to **https**. This redirect applies to all http requests to the domain.
- **automatic_https_rewrites** : Help fix mixed content by changing **http** to **https** for all resources or links on your website that can be served with HTTPS.
- **bot_management** : Detect and mitigate bot traffic on your domain.
- **brotli** : When the client that is requesting an asset supports the brotli compression algorithm, CIS serves a brotli compressed version of the asset.
- **browser_check** : Evaluate HTTP headers from your visitors' browser for threats. If a threat is found, then a block page is delivered.
- **challenge_ttl** : Specify how long a visitor with a bad IP reputation is allowed access to your website after they complete a challenge.
- **ciphers** : An allowlist of ciphers for TLS termination. These ciphers must be in the BoringSSL format.
- **cname_flattening** : Follow a CNAME to where it points and return that IP address instead of the CNAME record. By default, only flatten the CNAME at the root of your domain.
- **domain_hold** : Domain holds prevent teams in your organization from adding domains that are already active in another account.
Enterprise Plans Only
- **email_obfuscation** : Encrypt the email addresses on your web page from bots while it's kept visible to humans.
- **opportunistic_onion** : Allow legitimate users of the Tor Browser to access your websites.
- **hotlink_protection** : Protect your images from off-site linking.
- **http2** : Accelerate your website with HTTP/2.
- **http3** : Accelerate your website with HTTP/3.
- **image_load_optimization** : *Deprecated*. Improve load time for pages that include images on mobile devices with slow network connections.
- **image_size_optimization** : Improve image load time by optimizing images hosted on your domain.
- **image_resizing** : Provide on-demand resizing, conversion, and optimization for images served through the CIS network.
- **ip_geolocation** : Include the country code of the visitor location with all requests to your website.
- **ipv6** : Enable IPv6 support and gateway.
- **max_upload** : The number of data visitors who can upload to your website in a single request.
- **min_tls_version** : Allow only HTTPS connections from visitors that support the selected TLS protocol version or newer.
- **minify** : Reduce the file size of source code on your website.
- **mobile_redirect** : Redirect visitors that are using mobile devices to a mobile-optimized website.
- **opportunistic_encryption** : Opportunistic Encryption allows browsers to benefit from the improved performance of HTTP/2 by letting them know that your site is available over an encrypted connection.
- **origin_error_page_pass_thru** : When Origin Error Page is set to **On**, CIS proxies the 502 and 504 error pages directly from the origin.
Enterprise Plans Only
- **origin_max_http_version** : Configure the HTTP version to Origin.
- **origin_post_quantum_encryption** : Instructs CIS to use Post-Quantum (PQ) key agreement algorithms when it connects to your origin.
- **prefetch_preload** : CIS prefetches any URLs included in the prefetch HTTP header. **Enterprise Plans Only**
- **pseudo_ipv4** : Adds an IPv4 header to requests when a client is using IPv6, but the server supports IPv4 only.
- **response_buffering** : Enable or disable buffering of responses from the origin server. **Enterprise Plans Only**
- **script_load_optimization** : Improve the paint time for pages that include JavaScript.
- **security_header** : Enforce a web security policy for your website.
- **security_level** : Choose the appropriate security profile for your website.
- **server_side_exclude** : Automatically hide specific content from suspicious visitors.

- `tls_client_auth` : TLS client certificate presented for authentication on origin pull. **Enterprise Plans Only**
- `true_client_ip_header` : CIS sends the user's IP address in the True-Client-IP header. **Enterprise Plans Only**
- `waf` : A Web Application Firewall (WAF) blocks requests that contain malicious content.
- `websockets` : Allow WebSockets connections to your origin server.
- `proxy_read_timeout` : Maximum time between two read operations from origin.
- `url_normalization` : Modify the URLs of incoming requests.

`-v, --value`

The value set to the feature for domain. Required.

- Valid values for `always_use_https` are `on` and `off`.
- Valid values for `automatic_https_rewrites` are `on` and `off`.
- Valid values for `bot_management` are "use_latest_model", "fight_mode", "session_score", "enable_js". For example, `-v fight_mode=true,session_score=true`
 - `use_latest_model` : Whether to enable the latest model version. Valid values for `use_latest_model` are `true` and `false`.
 - `fight_mode` : Whether to enable the fight mode. Valid values for `fight_mode` are `true` and `false`.
 - `session_score` : Whether to enable the session score. Valid values for `session_score` are `true` and `false`.
 - `enable_js` : Whether to enable JavaScript detections. Valid values for `enable_js` are `true` and `false`.
- Valid values for `browser_check` are `on` and `off`.
- Valid values for `challenge_ttl` are `300, 900, 1800, 2700, 3600, 7200, 10800, 14400, 28800, 57600, 86400, 604800, 2592000, 31536000`.
- Valid values for `cname_flattening` are `flatten_at_root`, `flatten_all`.
 - `flatten_at_root` : Flatten CNAME at the root domain. This value is the default value.
 - `flatten_all` : Flatten all CNAME records under your domain.
- Valid values for `domain_hold` are `hold`, `include_subdomains` and `hold_after`.
 - `hold` : Whether to enable the domain hold. Valid values for `hold` are `true` and `false`.
 - `include_subdomains` : Whether to enable the domain hold. Valid values for `include_subdomains` are `true` and `false`.
 - `hold_after` : If `hold_after` is provided, the hold is temporarily disabled, then automatically re-enabled by the system at the time specified.

For enable domain and subdomains hold: `-v hold=true,include_subdomains=true`. For disable domain hold: `-v hold=false,hold_after=2023-05-31T15:56:36+00:00`.

- Valid values for `hotlink_protection` are `on`, `off`.
- Valid values for `email_obfuscation` are `on`, `off`.
- Valid values for `opportunistic_onion` are `on`, `off`.
- Valid values for `http2` are `on`, `off`.
- Valid values for `http3` are `on`, `off`.
- *Deprecated*. Valid values for `image_load_optimization` are `on`, `off`.
- Valid values for `image_resizing` are `on`, `off`.
- Valid values for `image_size_optimization` are `off`, `lossless`, `lossy`.
 - `off` : Disable Image Size Optimization.
 - `lossless` : Reduce the size of image files without impacting visual quality.
 - `lossy` : The file size of JPEG images is reduced by using lossy compression, which might reduce visual quality.
- Valid values for `ip_geolocation` are `on`, `off`.
- Valid values for `ipv6` are `on`, `off`.
- Valid values(in MB) for `max_upload` are: `100, 125, 150, 175, 200 and 225, 250, 275, 300, 325, 350, 375, 400, 425, 450, 475, 500`.

Enterprise Plans Only

- Valid values for `min_tls_version` are `1.0`, `1.1`, `1.2`, `1.3`.
- Valid values for `minify` are `css`, `html`, `js`. For example, `-v css=on,html=off,js=on`
 - `css`: Automatically minify all CSS for your website. Valid values for `css` are `on`, `off`.
 - `html`: Automatically minify all HTML for your website. Valid values for `html` are `on`, `off`.
 - `js`: Automatically minify all JS for your website. Valid values for `js` are `on`, `off`.
- Valid values for `mobile_redirect` are `status`, `mobile_subdomain`, `strip_uri`. For example, `-v status=on,mobile_subdomain=m,strip_uri=true`
 - `status`: Whether the mobile redirection is enabled. Valid values for `status` are `on` and `off`.
 - `mobile_subdomain`: Which subdomain prefix you want to redirect visitors on mobile devices to (subdomain must exist).
 - `strip_uri`: Whether to drop the current page path and redirect to the mobile subdomain URL root. Valid values for `strip_uri` are `true`, `false`.
- Valid values for `opportunistic_encryption` are `on`, `off`.
- Valid values for `origin_error_page_pass_thru` are `1`, `2`.
- Valid values for `origin_max_http_version` are `supported`, `preferred`, `off`.
 - `supported`: Post-Quantum algorithms are advertised but only used when requested by the origin.
 - `preferred`: Preferred instructs CIS to opportunistically send a Post-Quantum (PQ) keyshare in the first message to the origin (for fastest connections when the origin supports and prefers PQ).
 - `off`: Post-Quantum algorithms are not advertised.
- Valid values for `origin_post_quantum_encryption` are `on`, `off`.
- Valid values for `brotli` are `on`, `off`.
- Valid values for `prefetch_preload` are `on`, `off`.
- Valid values for `pseudo_ipv4` are `off`, `add_header`, `overwrite_header`.
 - `off`: Disable Pseudo IPv4.
 - `add_header`: Add an additional Cf-Pseudo-IPv4 header only.
 - `overwrite_header`: Overwrite the existing Cf-Connecting-IP and X-Forwarded-For headers with a pseudo IPv4 address.
- Valid values for `response_buffering` are `on`, `off`.
- Valid values for `script_load_optimization` are `on`, `off`.
- Valid values for `security_header` are `enabled`, `max_age`, `include_subdomains`, `preload`, `nosniff`. For example, `-v enabled=true,max_age=100,include_subdomains=true,preload=true,nosniff=true`
 - `enabled`: Whether the security_header is enabled. Valid values for `enabled` are `true`, `false`.
 - `max_age`: Specify the duration(in seconds) security_header are cached in browsers.
 - `include_subdomains`: Every domain below the domain inherits the same security_header. Valid values for `include_subdomains` are `true`, `false`.
 - `preload`: Whether to permit browsers to preload security_header config. Valid values for `enabled` are `true`, `false`.
 - `nosniff`: Whether to send `X-Content-Type-Options: nosniff` header. Valid values for `nosniff` are `true`, `false`.
- Valid values for `server_level` are `off`, `essentially_off`, `low`, `medium`, `high`, `under_attack`.
- Valid values for `server_side_exclude` are `on`, `off`.
- Valid values for `tls_client_auth` are `on`, `off`.
- Valid values for `true_client_ip_header` are `on`, `off`.
- Valid values for `waf` are `on`, `off`.
- Valid values for `websockets` are `on`, `off`.
- Valid values for `proxy_read_timeout`, 1-6000, default: 100.
- Valid values for `ciphers` are `ECDHE-ECDSA-AES128-GCM-SHA256`, `ECDHE-ECDSA-CHACHA20-POLY1305`, `ECDHE-RSA-AES128-GCM-SHA256`, `ECDHE-RSA-CHACHA20-POLY1305`, `ECDHE-ECDSA-AES128-SHA256`, `ECDHE-ECDSA-AES128-SHA`, `ECDHE-RSA-AES128-SHA256`, `ECDHE-RSA-AES128-SHA`, `AES128-GCM-SHA256`, `AES128-SHA256`, `AES128-SHA`, `ECDHE-ECDSA-AES256-GCM-SHA384`,

`ECDHE-ECDSA-AES256-SHA384` , `ECDHE-RSA-AES256-GCM-SHA384` , `ECDHE-RSA-AES256-SHA384` , `ECDHE-RSA-AES256-SHA` , `AES256-GCM-SHA384` , `AES256-SHA256` , `AES256-SHA` , `DES-CBC3-SHA` , `default` . For example, `-v AES256-SHA256,AES256-SHA` , using `-v default` to reset configured cipher suites to the default value.

- Valid values for `url_normalization` are "type", "scope". For example, `-v type=cis,scope=both`
 - `type` : Selects the type of URL normalization that is performed by CIS. Valid values for `type` are `cis` , `rfc3986` .
 - `scope` : Configures the scope of the URL normalization. Valid values for `scope` are `both` , `incoming` .

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Enable `tls_client_auth` for domain `DNS_DOMAIN_ID` .

```
$ ibmcloud cis domain-settings-update DNS_DOMAIN_ID -f tls_client_auth -v on
```

Edge functions

Manipulate how Edge Functions perform by using the following `edge-functions` commands:

ibmcloud cis edge-functions-actions

List all Edge Functions actions of a service instance.

```
$ ibmcloud cis edge-functions-actions [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List all Edge Functions actions.

```
$ ibmcloud cis edge-functions-actions
```

ibmcloud cis edge-functions-action

Show an Edge Functions action of a service instance.

```
$ ibmcloud cis edge-functions-action [--name ACTION_NAME] [-i, --instance INSTANCE]
```

Command options

`--name`

Action name. Enterprise Plans Only

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Show details of Edge Functions action `ACTION_NAME`.

```
$ ibmcloud cis edge-functions-action --name ACTION_NAME
```

ibmcloud cis edge-functions-action-create

Create an Edge Functions action for a service instance.

```
$ ibmcloud cis edge-functions-action-create [--name ACTION_NAME] (--javascript-str JAVASCRIPT_STR | --javascript-file JAVASCRIPT_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

- `--name`
Action name. Enterprise Plans Only
- `--javascript-str`
JavaScript string. For example, `addEventListener('fetch', event => { event.respondWith(fetch(event.request))})`
- `--javascript-file`
JavaScript file.
- `-i, --instance`
Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.
- `--output`
The output format. Currently, `json` is the only supported value.

Examples

Create an Edge Functions action.

```
$ ibmcloud cis edge-functions-action-create --name ACTION_NAME --javascript-str "addEventListener('fetch', event => { event.respondWith(fetch(event.request)) })"
```

ibmcloud cis edge-functions-action-update

Update an Edge Functions action of a service instance.

```
$ ibmcloud cis edge-functions-action-update (--javascript-str JAVASCRIPT_STR | --javascript-file JAVASCRIPT_FILE) [--name ACTION_NAME] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

- `--name`
Action name. Enterprise Plans Only
- `--javascript-str`
JavaScript string. For example, `addEventListener('fetch', event => { event.respondWith(fetch(event.request))})`
- `--javascript-file`
JavaScript file.
- `-i, --instance`
Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.
- `--output`
The output format. Currently, `json` is the only supported value.

Examples

Update an Edge Functions action.

```
$ ibmcloud cis edge-functions-action-update --name ACTION_NAME --javascript-str "addEventListener('fetch', event => { event.respondWith(fetch(event.request)) })"
```

ibmcloud cis edge-functions-action-delete

Delete an Edge Functions action of a service instance.

```
$ ibmcloud cis edge-functions-action-delete [--name ACTION_NAME] [-i, --instance INSTANCE]
```

Command options

`--name`

Action name. Enterprise Plans Only

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete Edge Functions action `ACTION_NAME`.

```
$ ibmcloud cis edge-functions-action-delete --name ACTION_NAME
```

ibmcloud cis edge-functions-triggers

List all Edge Functions triggers for a domain of a service instance.

```
$ ibmcloud cis edge-functions-triggers DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List all Edge Functions triggers for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis edge-functions-triggers DNS_DOMAIN_ID
```

ibmcloud cis edge-functions-trigger

Show an Edge Functions trigger for a domain of a service instance.

```
$ ibmcloud cis edge-functions-trigger DNS_DOMAIN_ID TRIGGER_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

TRIGGER_ID

The ID of the trigger. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show details of Edge Functions trigger `TRIGGER_ID`.

```
$ ibmcloud cis edge-functions-trigger DNS_DOMAIN_ID TRIGGER_ID
```

ibmcloud cis edge-functions-trigger-create

Create an Edge Functions trigger for a domain of a service instance.

```
$ ibmcloud cis edge-functions-trigger-create DNS_DOMAIN_ID PATTERN_URL [--name ACTION_NAME] [--disable] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

PATTERN_URL

The request URL, which triggers the action. Required.

name

The action name to which the created trigger is attached. Enterprise Plans Only

disable

Disable an Edge Functions trigger.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Create an Edge Functions trigger for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis edge-functions-trigger-create DNS_DOMAIN_ID PATTERN_URL --name ACTION_NAME
```

ibmcloud cis edge-functions-trigger-update

Update an Edge Functions trigger for a domain of a service instance.

```
$ ibmcloud cis edge-functions-trigger-update DNS_DOMAIN_ID TRIGGER_ID PATTERN_URL [--name ACTION_NAME] [--disable] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

TRIGGER_ID

The ID of the trigger. Required.

PATTERN_URL

The request URL, which triggers the action. Required.

name

The action name, which the created trigger is attached to. Enterprise Plans Only

disable

Disable an Edge Functions trigger.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update Edge Functions trigger `TRIGGER_ID`.

```
$ ibmcloud cis edge-functions-trigger-update DNS_DOMAIN_ID TRIGGER_ID PATTERN_URL --name ACTION_NAME
```

ibmcloud cis edge-functions-trigger-delete

Delete an Edge Functions trigger for a domain of a service instance.

```
$ ibmcloud cis edge-functions-trigger-delete DNS_DOMAIN_ID TRIGGER_ID [-i, --instance INSTANCE]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

TRIGGER_ID

The ID of the trigger. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete Edge Functions trigger `TRIGGER_ID`.

```
$ ibmcloud cis edge-functions-trigger-delete DNS_DOMAIN_ID TRIGGER_ID
```

Firewall (Deprecated)



Deprecated: Firewall rules are deprecated. CIS moved existing firewall rules to [WAF custom rules](#). If you haven't migrated to WAF custom rules yet, you might have some invalid configuration that prevents the migration from happening. In this case, contact your account team to get help with the migration to WAF custom rules. For more information, see [Migrating to WAF custom rules](#).

Manipulate firewalls by using the following `firewall` commands.

ibmcloud cis firewall-create

Create a new firewall rule.

```
$ ibmcloud cis firewall-create (-t, --type Type) (--json @JSON_FILE | JSON_STRING) [-d, --domain DNS_DOMAIN_ID] [-i, --instance INSTANCE] [--output FORMAT]
```

[Deprecated] ibmcloud cis firewall-create (-t, --type Type) (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-d, --domain DNS_DOMAIN_ID] [-i, --instance INSTANCE] [--output FORMAT]

Command options

-t, --type

Type of firewall rules to create. Valid values: `access-rules`, `ua-rules`, `lockdowns`. Required.

- `access-rules`: Access Rules are a way to allow, challenge, or block requests to your website. You can apply access rules to one domain only or all domains in the same service instance.
- `ua-rules`: Perform access control when matching the exact UserAgent reported by the client. The access control mechanisms can be defined within a rule to help manage traffic from particular clients. This option enables you to customize the access to your site.
- `lockdowns`: Lock access to URLs in this domain to only permitted addresses or address ranges.

-d, --domain

DNS Domain ID. For `ua-rules` and `lockdowns` type rule, it is a required parameter.

--json

The JSON file or JSON string that is used to describe a firewall rule. Required.

- For `--type access-rules`: The JSON data that describes a firewall access rule as follows.
 - Required fields are `mode`, `configuration`.
 - `mode`: The type of action to perform. Valid values are `block`, `challenge`, `whitelist`, and `js_challenge`.
 - `configuration`: Target/Value pair to use for this rule.
 - `target`: The request property to target. Valid values are `ip`, `ip_range`, `asn`, and `country`.
 - `value`: The value for the selected target.
 - For ip, the value is a valid IP address.
 - For ip_range, the value specifies an ip range that is limited to `/16` and `/24`.
 - For asn, the value is an AS number.
 - For a country, the value is a country code for the country.
 - Option fields are `notes`.
 - `notes`: Some useful information about this rule to help identify the purpose of it.

Sample JSON data:

```
{
  "mode": "block",
  "notes": "This rule is added because of event X that occurred on date xyz",
  "configuration": {
    "target": "ip",
    "value": "127.0.0.1"
  }
}
```

- For `--type ua-rules`: The JSON data that describes a user-agent rule as follows.
 - Required fields are `mode`, `configuration`.
 - `mode`: The type of action to perform. Valid values are `block`, `challenge`, and `js_challenge`.
 - `configuration`: Target/Value pair to use for this rule.
 - `target`: The request property to target. Valid value is `ua`.
 - `value`: The exact UserAgent string to match with this rule.
 - Option fields are `paused`, `description`.
 - `paused`: Whether this rule is currently disabled.
 - `description`: Some useful information about this rule to help identify the purpose of it.

Sample JSON data:

```
{
  "mode": "block",
  "configuration": {
```



```
"target": "ua",
"value": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_12_5) AppleWebKit/603.2.4 (KHTML, like Gecko) Version/10.1.1 Safari/603.2.4"
}
}
```

- For `--type lockdowns` : The JSON data that describes a lockdown rule is as follows.
 - Required fields are `urls` , `configurations` .
 - `urls` : URLs to be included in this rule definition.
 - Wildcards are permitted.
 - The URL pattern entered here is escaped before use.
 - This field limits the URL to simple wildcard patterns.
 - `configurations` : List of IP addresses or CIDR ranges to use for this rule.
 - This field can include any number of ip or ip_range configurations that can access the provided URLs.
 - `target` : The request property to target. Valid values are `ip` , and `ip_range` .
 - `value` : IP addresses or CIDR. If target is `ip` , then value must be an IP address, otherwise CIDR.
 - Option fields are `paused` , `description` .
 - `paused` : Whether this rule is currently disabled.
 - `description` : Some useful information about this rule to help identify the purpose of it.

Sample JSON data:

```
{
  "urls": [
    "api.mysite.com/some/endpoint*"
  ],
  "configurations": [
    {
      "target": "ip",
      "value": "127.0.0.1"
    },
    {
      "target": "ip_range",
      "value": " 2.2.2.0/24"
    }
  ]
}
```

`-s, --json-str`

Deprecated. The JSON data describing a firewall rule.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create firewall rules.

```
ibmcloud cis firewall-create --type access-rules --json '{"mode": "block", "notes": "This rule is added because of event X that occurred on date xyz", "configuration": {"target": "ip", "value": "127.0.0.1"}}' --instance INSTANCE
ibmcloud cis firewall-create --type ua-rules --domain DNS_DOMAIN_ID --json '{"mode": "block", "configuration": {"target": "ua", "value": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_12_5) AppleWebKit/603.2.4 (KHTML, like Gecko) Version/10.1.1 Safari/603.2.4"}}' --instance INSTANCE
ibmcloud cis firewall-create --type lockdowns --domain DNS_DOMAIN_ID --json '{"urls": ["api.mysite.com/some/endpoint*"], "configurations": [{"target": "ip", "value": "127.0.0.1"}, {"target": "ip_range", "value": "2.2.2.0/24"}]}' --instance INSTANCE
```

ibmcloud cis firewall-update

Update a firewall rule.

```
$ ibmcloud cis firewall-update FIREWALL_RULE_ID (-t, --type Type) (--json @JSON_FILE | JSON_STRING) [-d, --domain DNS_DOMAIN_ID] [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis firewall-update FIREWALL_RULE_ID (-t, --type Type) (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-d, --domain DNS_DOMAIN_ID] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

- **FIREWALL_RULE_ID**: The ID of firewall rule. Required.

-t, --type

The type of firewall rule to create. Valid values are `access-rules`, `ua-rules`, and `lockdowns`. Required.

- `access-rules`: Access Rules are a way to allow, challenge, or block requests to your website. You can apply access rules to one domain only or all domains in the same service instance.
- `ua-rules`: Perform access control when matching the exact UserAgent reported by the client. The access control mechanisms can be defined within a rule to help manage traffic from particular clients. This enables you to customize the access to your site.
- `lockdowns`: Lock access to URLs in this domain to only permitted addresses or address ranges.

-d, --domain

DNS Domain ID. For `ua-rules` and `lockdowns` type rule, it is a required parameter.

--json

The JSON file or JSON string that is used to describe a firewall rule. Required.

- For `--type access-rules`: The JSON data that describes a firewall access rule is as follows.
 - Option fields are `mode`, `notes`.
 - `mode`: The type of action to perform. Valid values are `block`, `challenge`, `whitelist`, and `js_challenge`.
 - `notes`: Some useful information about this rule to help identify the purpose of it.

Sample JSON data:

```
{
  "mode": "challenge",
  "notes": "This rule is added because of event X that occurred on date xyz",
}
```

- For `--type ua-rules`: The JSON data that describes a user-agent rule is as follows.
 - Required fields are `mode`, `configuration`.
 - `mode`: The type of action to perform. Valid values are `block`, `challenge`, and `js_challenge`.
 - `configuration`: Target/Value pair to use for this rule.
 - `target`: The request property to target. Valid value is `ua`.
 - `value`: The exact UserAgent string to match with this rule.
 - Option fields are `paused`, `description`.
 - `paused`: Whether this rule is currently disabled.
 - `description`: Some useful information about this rule to help identify the purpose of it.

Sample JSON data:

```
{
  "mode": "block",
  "configuration": {
    "target": "ua",
    "value": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_12_5) AppleWebKit/603.2.4 (KHTML, like Gecko) Version/10.1.1 Safari/603.2.4"
  }
}
```

- For `--type lockdowns`: The JSON data that describes a lockdown rule is as follows.

- Required fields are `urls` , `configurations` .
 - `urls` : URLs to be included in this rule definition.
 - Wildcards are permitted.
 - The URL pattern entered here is escaped before use.
 - This field limits the URL to simple wildcard patterns.
 - `configurations` : List of IP addresses or CIDR ranges to use for this rule.
 - This field can include any number of ip or ip_range configurations that can access the provided URLs.
 - `target` : The request property to target. Valid values are `ip` and `ip_range` .
 - `value` : IP addresses or CIDR. If the target is `ip` , then the value must be an IP addresses, otherwise CIDR.
- Option fields are `paused` , `description` .
 - `paused` : Whether this rule is currently disabled.
 - `description` : Some useful information about this rule to help identify the purpose of it.

Sample JSON data:

```
{
  "urls": [
    "api.mysite.com/some/endpoint*"
  ],
  "configurations": [
    {
      "target": "ip",
      "value": "127.0.0.1"
    },
    {
      "target": "ip_range",
      "value": " 2.2.2.0/24"
    }
  ]
}
```

`-s, --json-str`

Deprecated. The JSON data that describes a firewall rule.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update firewall rules.

```
ibmcloud cis firewall-update FIREWALL_RULE_ID -t access-rules --json '{"mode": "challenge", "notes": "This rule is added because of event X that occurred on date xyz"}'
```

```
ibmcloud cis firewall-update FIREWALL_RULE_ID -t ua-rules -d DNS_DOMAIN_ID --json '{"mode": "block", "configuration": {"target": "ua", "value": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_12_5) AppleWebKit/603.2.4 (KHTML, like Gecko) Version/10.1.1 Safari/603.2.4"}'}
```

```
ibmcloud cis firewall-update FIREWALL_RULE_ID -t lockdowns -d DNS_DOMAIN_ID --json '{"urls": ["api.mysite.com/some/endpoint*"], "configurations": [{"target": "ip", "value": "127.0.0.1"}, {"target": "ip_range", "value": "2.2.2.0/24"}]}'
```

ibmcloud cis firewalls

List firewall rules.

```
$ ibmcloud cis firewalls (-t, --type Type) [-d, --domain DNS_DOMAIN_ID] [--page PAGE] [--per-page PER_PAGE ] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`-t, --type`

The type of firewall rule to create. Valid values are `access-rules`, `ua-rules`, and `lockdowns`. Required.

- `access-rules` : Access Rules are a way to allow, challenge, or block requests to your website. You can apply access rules to one domain only or all domains in the same service instance.
- `ua-rules` : Perform access control when matching the exact UserAgent reported by the client. The access control mechanisms can be defined within a rule to help manage traffic from particular clients. This action enables you to customize the access to your site.
- `lockdowns` : Lock access to URLs in this domain to only permitted addresses or address ranges.

`-d, --domain`

DNS Domain ID. For `ua-rules` and `lockdowns` type rule, it is a required parameter.

`--page`

Page number of paginated results. The default value is `0`.

`--per-page`

Maximum number of access rules per page. The minimum value is `5`. The default value is `20`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List firewall rules.

```
ibmcloud cis firewalls --type access-rules --instance INSTANCE
ibmcloud cis firewalls --type ua-rules --domain DNS_DOMAIN_ID --instance INSTANCE
ibmcloud cis firewalls --type lockdowns --domain DNS_DOMAIN_ID --instance INSTANCE
```

ibmcloud cis firewall

Get details of a firewall rule.

```
$ ibmcloud cis firewall FIREWALL_RULE_ID (-t, --type Type) [-d, --domain DNS_DOMAIN_ID] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`FIREWALL_RULE_ID`

The ID of firewall rule. Required.

`-t, --type`

Type of firewall rule to create. Valid values are `access-rules`, `ua-rules`, and `lockdowns`.

- `access-rules` : Access Rules are a way to allow, challenge, or block requests to your website. You can apply access rules to one domain only or all domains in the same service instance.
- `ua-rules` : Perform access control when matching the exact UserAgent reported by the client. The access control mechanisms can be defined within a rule to help manage traffic from particular clients. This enables you to customize the access to your site.
- `lockdowns` : Lock access to URLs in this domain to only permitted addresses or address ranges.

`-d, --domain`

DNS Domain ID. For `ua-rules` and `lockdowns` type rule, it is a required parameter.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get firewall rule details.

```
ibmcloud cis firewall FIREWALL_RULE_ID -t access-rules
ibmcloud cis firewall FIREWALL_RULE_ID -t access-rules -d DNS_DOMAIN_ID
ibmcloud cis firewall FIREWALL_RULE_ID -t ua-rules -d DNS_DOMAIN_ID
ibmcloud cis firewall FIREWALL_RULE_ID -t lockdowns -d DNS_DOMAIN_ID
```

ibmcloud cis firewall-delete

Delete a firewall rule by ID.

```
$ ibmcloud cis firewall-delete FIREWALL_RULE_ID (-t, --type Type) [-d, --domain DNS_DOMAIN_ID] [-i, --instance INSTANCE]
```

Command options

FIREWALL_RULE_ID

The ID of firewall rule. Required.

-t, --type

Type of firewall rule to create. Valid values are `access-rules`, `ua-rules`, and `lockdowns`. Required.

- `access-rules` : Access Rules are a way to allow, challenge, or block requests to your website. You can apply access rules to one domain only or all domains in the same service instance.
- `ua-rules` : Perform access control when matching the exact UserAgent reported by the client. The access control mechanisms can be defined within a rule to help manage traffic from particular clients. This field enables you to customize the access to your site.
- `lockdowns` : Lock access to URLs in this domain to only permitted addresses or address ranges.

-d, --domain

DNS Domain ID. For `ua-rules` and `lockdowns` type rule, it is a required parameter.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete a firewall rule.

```
$ ibmcloud cis firewall-delete FIREWALL_RULE_ID -t access-rules
ibmcloud cis firewall-delete FIREWALL_RULE_ID -t access-rules -d DNS_DOMAIN_ID
ibmcloud cis firewall-delete FIREWALL_RULE_ID -t ua-rules -d DNS_DOMAIN_ID
ibmcloud cis firewall-delete FIREWALL_RULE_ID -t lockdowns -d DNS_DOMAIN_ID
```

Firewall rules

Manipulate how firewall rules perform by using the following `firewall-rules` commands:

ibmcloud cis firewall-rules

Retrieve a list of currently existing firewall-rules for a DNS domain.

```
$ ibmcloud cis firewall-rules DNS_DOMAIN_ID [--page PAGE] [--per-page PER_PAGE] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--page

Page number of paginated results. The default value is `1`.

`--per-page`

Number of firewall rules per page. The minimum value is `5` and the maximum value is `100`. The default value is `25`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List existing firewall-rules in the domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis firewall-rules DNS_DOMAIN_ID
```

ibmcloud cis firewall-rule

Retrieve a specific firewall-rule for a DNS domain.

```
$ ibmcloud cis firewall-rule DNS_DOMAIN_ID FIREWALL_RULE_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`FIREWALL_RULE_ID`

The ID of firewall-rule. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get the details of firewall-rule `FIREWALL_RULE_ID`.

```
$ ibmcloud cis firewall-rule DNS_DOMAIN_ID FIREWALL_RULE_ID
```

ibmcloud cis firewall-rule-create

Create a firewall-rule for a DNS domain.

```
$ ibmcloud cis firewall-rule-create DNS_DOMAIN_ID --expression EXPRESSION --action ACTION [--priority PRIORITY] [--paused on|off] [--products PRODUCTS][--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis firewall-rule-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis firewall-rule-create DNS_DOMAIN_ID (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--expression`

A filter expression. For example, `ip.src eq 93.184.216.0`.

`--action`

The rule action to perform. Valid values are `log` , `allow` , `challenge` , `js_challenge` , `block` , and `bypass` .

`--priority`

The rule's priority. Valid values range from `0` to `2147483647` . The value `0` means to set to the default value.

`--description`

To briefly describe the rule.

`--paused`

Indicates whether the rule is active or not. Valid values are `on` and `off` . The default value is `off` .

`--products`

The list of security products to be bypassed. Valid values are `zoneLockdown` , `uaBlock` , `bic` , `hot` , `securityLevel` , `rateLimit` , and `waf` .

`--json`

The JSON file or JSON strin that isused to describe a firewall-rule.

- The required fields in JSON data are `expression` and `action` .
 - `expression` : A filter expression. For example, `ip.src eq 93.184.216.0`
 - `action` : The rule action to perform. Valid values are `log` , `allow` , `challenge` , `js_challenge` , `block` , and `bypass` .
- The optional fields are `description` , `priority` , `paused` , `products` .
 - `description` : To briefly describe the rule.
 - `priority` : The rule's priority. Valid values range from `0` to `2147483647` . The value `0` means to set to the default value.
 - `paused` : Indicates whether the rule is active or not. Valid values are `on` and `off` . The default value is `off` .
 - `products` : The list of security products to be bypassed. Valid values are `zoneLockdown` , `uaBlock` , `bic` , `hot` , `securityLevel` , `rateLimit` , and `waf` For example, `--products zoneLockdown, rateLimit`

Sample JSON data:

```
{
  "expression": "ip.src eq 93.184.216.1 and http.request.uri.path ~ \"^.*wp-login.php$\"",
  "action": "allow",
  "priority": 100,
  "paused": false,
  "description": "do not challenge login from office"
}
```

`-s, --json-str`

Deprecated. The JSON data that describes a firewall-rule.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a firewall-rule in the domain `DNS_DOMAIN_ID` .

```
ibmcloud cis firewall-rule-create DNS_DOMAIN_ID --expression "ip.src eq 93.184.216.1 and http.request.uri.path ~ \"^.*wp-login.php$\"" --action allow --priority 200 --
paused off --description "do not challenge login from office"
```

```
ibmcloud cis firewall-rule-create DNS_DOMAIN_ID --json '{"expression": "ip.src eq 93.184.216.1 and http.request.uri.path ~ \"^.*wp-login.php$\"", "action": "allow",
```



```
"priority": 100, "paused": false, "description": "do not challenge login from office"}
```

ibmcloud cis firewall-rule-update

Update a specific firewall-rule for a DNS domain.

```
$ ibmcloud cis firewall-rule-update DNS_DOMAIN_ID FIREWALL_RULE_ID [--expression EXPRESSION] [--action ACTION] [--priority PRIORITY] [--paused on|off] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis firewall-rule-update DNS_DOMAIN_ID FIREWALL_RULE_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis firewall-rule-update DNS_DOMAIN_ID FIREWALL_RULE_ID (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-i, --instance INSTANCE] [-output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

FIREWALL_RULE_ID

The ID of firewall-rule. Required.

--expression

A filter expression. For example, `ip.src eq 93.184.216.0`.

--action

The rule action to perform. Valid values are `log`, `allow`, `challenge`, `js_challenge`, `block`, and `bypass`.

--priority

The rule's priority. Valid values range from `0` to `2147483647`. The value `0` means to set to the default value.

--description

To briefly describe the rule.

--paused

Indicates whether the rule is active or not. Valid values are `on` and `off`. The default value is `off`.

--products

The list of security products to be bypassed. Valid values are `zoneLockdown`, `uaBlock`, `bic`, `hot`, `securityLevel`, `rateLimit`, and `waf`.

--json

The JSON file or JSON string that is used to describe a firewall-rule.

- The required fields in JSON data are `expression`, and `action`.
 - `expression` : A filter expression. For example, `ip.src eq 93.184.216.0`
 - `action` : The rule action to perform. Valid values are `log`, `allow`, `challenge`, `js_challenge`, `block`, and `bypass`.
- The optional fields are `description`, `priority`, `paused`, `products`.
 - `description` : To briefly describe the rule.
 - `priority` : The rule's priority. Valid values range from `0` to `2147483647`. The value `0` means to set to the default value.
 - `paused` : Indicates whether the rule is active or not. Valid values are `on` and `off`. The default value is `off`.
 - `products` : The list of security products to be bypassed. Valid values are `zoneLockdown`, `uaBlock`, `bic`, `hot`, `securityLevel`, `rateLimit`, and `waf` For example, `--products zoneLockdown, rateLimit`



Note: Fields `description`, `priority`, and `paused`, which aren't explicitly set in JSON data are overwritten by the default value.

Sample JSON data:

```
{
  "expression": "ip.src eq 93.184.216.1 and http.request.uri.path ~ \"^.*wp-login.php$\"",

```

```
"action": "allow",
"priority": 100,
"paused": false,
"description": "do not challenge login from office"
}
```

`-s, --json-str`

Deprecated. The JSON data that describes a firewall-rule.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update firewall-rule `FIREWALL_RULE_ID`.

```
$ ibmcloud cis firewall-rule-update DNS_DOMAIN_ID FIREWALL_RULE_ID --expression "ip.src eq 93.184.216.1 and http.request.uri.path ~ \"^.*wp-login.php$\" --
action allow --priority 200 --paused off --description \"do not challenge login from office\"

ibmcloud cis firewall-rule-update DNS_DOMAIN_ID FIREWALL_RULE_ID --json '{"expression": "ip.src eq 93.184.216.1 and http.request.uri.path ~ \"^.*wp-login.php$\"",
"action": "allow", "priority": 100, "paused": false, "description": "do not challenge login from office"}'
```

ibmcloud cis firewall-rule-delete

Delete a specific firewall-rule for a DNS domain.

```
$ ibmcloud cis firewall-rule-delete DNS_DOMAIN_ID FIREWALL_RULE_ID [-i, --instance INSTANCE]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`FIREWALL_RULE_ID`

The ID of firewall-rule. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete firewall-rule `FIREWALL_RULE_ID`.

```
$ ibmcloud cis firewall-rule-delete DNS_DOMAIN_ID FIREWALL_RULE_ID
```

ibmcloud cis firewall-rule-validate

Validate a firewall-rule expression.

```
$ ibmcloud cis firewall-rule-validate DNS_DOMAIN_ID EXPRESSION [-i, --instance INSTANCE]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

EXPRESSION

The filter expression. For example, `ip.src eq 93.184.216.0` . Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Validate firewall-rule expression.

```
$ ibmcloud cis firewall-rule-validate DNS_DOMAIN_ID "ip.src eq 93.184.216.0"
```

Global load balancer

Manipulate global load balancers by using the following `glb` commands.

ibmcloud cis glb-create

Create a global load balancer under a DNS domain.

```
$ ibmcloud cis glb-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis glb-create DNS_DOMAIN_ID (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--json

The JSON file or JSON string that is used to describe a global load balancer. Required.

- The required fields in JSON data are `name` , `fallback_pool` and `default_pools` :
 - `name` : The DNS hostname to associate with your load balancer.
 - `fallback_pool` : The pool ID to use when all other pools are detected as unhealthy.
 - `default_pools` : A list of pool IDs ordered by their failover priority.
- The optional fields are `description` , `ttl` , `region_pools` , `proxied` , `enabled` , `session_affinity` , `session_affinity_ttl` , `steering_policy` :
 - `description` : The description of your load balancer.
 - `ttl` : Time to live (TTL) of the DNS entry for the IP address returned by this load balancer.
 - `region_pools` : A mapping of region and country codes to a list of pool IDs (ordered by their failover priority) for the region.
 - `proxied` : Control whether traffic should flow through the security and performance functions on CIS.
 - `enabled` : Whether to enable (the default) this load balancer.
 - `session_affinity` : Ensures that a user's requests are consistently directed to the same backend server during a session. Valid values are `cookie` and `none` .
 - `session_affinity_ttl` : Time, in seconds, until this load balancers session affinity cookie expires after it is created. Valid value is between `[1800, 604800]` . The default value is `82800` .
 - `steering_policy` : Valid values for `steering_policy` are `off` , `geo` , `random` , `dynamic_latency` .
 - `off` : Use `default_pools` .
 - `geo` : Use `region_pools/pop_pools` .
 - `random` : Select a pool randomly.
 - `dynamic_latency` : Use round-trip time to select the closest pool in `default_pools` (requires pool health checks).

Sample JSON data:

```
{
  "name": "www.example.com",
  "fallback_pool": "17b5962d775c646f3f9725cbc7a53df4",
  "default_pools": [
    "17b5962d775c646f3f9725cbc7a53df4",
    "9290f38c5d07c2e2f4df57b1f61d4196"
  ],
  "description": "Example global load balancer.",
  "ttl": 60,
  "region_pools": {
    "WNAM": [
      "de90f38ced07c2e2f4df50b1f61d4194",
      "9290f38c5d07c2e2f4df57b1f61d4196"
    ],
    "ENAM": [
      "00920f38ce07c2e2f4df50b1f61d4194"
    ]
  }
}
```

`-s, --json-str`

Deprecated. The JSON data describing a global load balancer.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a global load balancer in the domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis glb-create DNS_DOMAIN_ID --json '{"description":"Example global load balancer.", "name":"www.example.com", "ttl":60, "fallback_pool":"17b5962d775c646f3f9725cbc7a53df4", "default_pools": ["17b5962d775c646f3f9725cbc7a53df4", "9290f38c5d07c2e2f4df57b1f61d4196"], "region_pools":{"WNAM": ["de90f38ced07c2e2f4df50b1f61d4194", "9290f38c5d07c2e2f4df57b1f61d4196"], "ENAM":["00920f38ce07c2e2f4df50b1f61d4194"]}}'
```

ibmcloud cis glb-update

Update a global load balancer under a DNS domain.

```
$ ibmcloud cis glb-update DNS_DOMAIN_ID GLB_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis glb-update DNS_DOMAIN_ID GLB_ID (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`GLB_ID`

The ID of the global load balancer. Required.

`--json`

The JSON file or JSON string that is used to describe a global load balancer. Required.

- The required fields in JSON data are `name`, `fallback_pool`, and `default_pools` :
 - `name` : The DNS hostname to associate with your load balancer.
 - `fallback_pool` : The pool ID to use when all other pools are detected as unhealthy.
 - `default_pools` : A list of pool IDs ordered by their failover priority.

- The optional fields are `description` , `ttl` , `region_pools` , `proxied` , `enabled` , `session_affinity` , `session_affinity_ttl` , `steering_policy` :
 - `description` : The description of your Load Balancer.
 - `ttl` : Time to live (TTL) of the DNS entry for the IP address returned by this load balancer.
 - `region_pools` : A mapping of region and country codes to a list of pool IDs (ordered by their failover priority) for the region.
 - `proxied` : Control whether traffic must flow through the security and performance functions on CIS.
 - `enabled` : Whether to enable (the default) this load balancer.
 - `session_affinity` : Ensures that a user's requests are consistently directed to the same backend server during a session. Valid values are `cookie` and `none` .
 - `session_affinity_ttl` : Time, in seconds, until this load balancers session affinity cookie expires it is created. Valid value is between `[1800, 604800]` . The default value is `82800` .
 - `steering_policy` : Valid values for `steering_policy` are `off` , `geo` , `random` , `dynamic_latency` .
 - `off` : Use `default_pools` .
 - `geo` : Use `region_pools/pop_pools` .
 - `random` : Select a pool randomly.
 - `dynamic_latency` : Use round-trip time to select the closest pool in `default_pools` (requires pool health checks).

Sample JSON data:

```
{
  "name": "www.example.com",
  "fallback_pool": "17b5962d775c646f3f9725cbc7a53df4",
  "default_pools": [
    "17b5962d775c646f3f9725cbc7a53df4",
    "9290f38c5d07c2e2f4df57b1f61d4196"
  ],
  "description": "Example global load balancer.",
  "ttl": 60,
  "region_pools": {
    "WNAM": [
      "de90f38ced07c2e2f4df50b1f61d4194",
      "9290f38c5d07c2e2f4df57b1f61d4196"
    ],
    "ENAM": [
      "00920f38ce07c2e2f4df50b1f61d4194"
    ]
  }
}
```

`-s, --json-str`

Deprecated. The JSON data describing a global load balancer.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update the global load balancer `GLB_ID` in the domain `DNS_DOMAIN_ID` .

```
$ ibmcloud cis glb-update DNS_DOMAIN_ID GLB_ID --json '{"description":"Example global load balancer.", "name":"www.example.com", "ttl":60, "fallback_pool":"17b5962d775c646f3f9725cbc7a53df4", "default_pools": ["17b5962d775c646f3f9725cbc7a53df4", "9290f38c5d07c2e2f4df57b1f61d4196"], "region_pools":{"WNAM": ["de90f38ced07c2e2f4df50b1f61d4194", "9290f38c5d07c2e2f4df57b1f61d4196"], "ENAM":["00920f38ce07c2e2f4df50b1f61d4194"]}}'
```


ibmcloud cis glb

Show a global load balancer under a DNS domain.

```
$ ibmcloud cis glb DNS_DOMAIN_ID GLB_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

GLB_ID

The ID of the global load balancer. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show global load balancer `GLB_ID` in domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis glb DNS_DOMAIN_ID GLB_ID
```

ibmcloud cis glb-delete

Delete a global load balancer under a DNS domain.

```
$ ibmcloud cis glb-delete DNS_DOMAIN_ID GLB_ID [-i, --instance INSTANCE]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

GLB_ID

The ID of the global load balancer. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete global load balancer `GLB_ID` in domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis glb-delete DNS_DOMAIN_ID GLB_ID
```

ibmcloud cis glbs

List all load balancers for the domain.

```
$ ibmcloud cis glbs DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List load balancers for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis glbs DNS_DOMAIN_ID
```

ibmcloud cis glb-pools

List all GLB pools for a service instance.

```
$ ibmcloud cis glb-pools [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

Specify the output format. Only JSON is supported.

Examples

List all GLB pools.

```
$ ibmcloud cis glb-pools
```

ibmcloud cis glb-pool-create

Create a GLB pool for a service instance.

```
$ ibmcloud cis glb-pool-create (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis glb-pool-create (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`--json`

The JSON file or JSON string that is used to describe a GLB pool. Required.

- The required fields in JSON data are `name`, `origins` and `check_regions` :
 - `name` : A short name (tag) for the pool.
 - `origins` : A list of origins within this pool.
 - `check_regions` : A list of geographic region code.
- The optional fields are `description`, `minimum_origins`, `enabled`, `monitor`.

Sample JSON data:

```
{
  "name": "us-pool",
  "description": "application server pool in US",
  "origins": [
```



```
{
  "name": "us-app-dal01",
  "address": "1.1.1.1",
  "enabled": true,
  "header": {
    "host": ["test.com"]
  }
},
{
  "name": "us-app-dal02",
  "address": "2.2.2.2",
  "enabled": true,
  "header": {
    "host": ["example.com"]
  }
}
],
"minimum_origins": 1,
"check_regions": [ "WNAM" ],
"monitor": "f1aba936b94213e5b8dca0c0dbf1f9cc",
"enabled": true
}
```

`-s, --json-str`

Deprecated. The JSON data used to describe a GLB pool.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a GLB pool.

```
$ ibmcloud cis glb-pool-create --json '{"description":"application server pool in US", "name":"us-pool", "enabled":true, "check_regions":["WNAM"], "minimum_origins":1,"monitor":"f1aba936b94213e5b8dca0c0dbf1f9cc", "origins":[{"name":"us-app-dal01","address":"1.1.1.1","enabled":true,"header":{"host":["test.com"]}}, {"name":"us-app-dal02","address":"2.2.2.2","enabled":true,"header":{"host":["example.com"]}}]}'
```

ibmcloud cis glb-pool

Show the details of a GLB pool.

```
$ ibmcloud cis glb-pool GLB_POOL_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`GLB_POOL_ID`

The ID of the global load balancer pool. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Show the details of the GLB pool `GLB_POOL_ID` .

```
$ ibmcloud cis glb-pool GLB_POOL_ID
```

ibmcloud cis glb-pool-delete

Delete a GLB pool.

```
$ ibmcloud cis glb-pool-delete GLB_POOL_ID [-i, --instance INSTANCE]
```

Command options

`GLB_POOL_ID`

The ID of the global load balancer pool. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete GLB pool `GLB_POOL_ID` .

```
$ ibmcloud cis glb-pool-delete GLB_POOL_ID
```

ibmcloud cis glb-pool-update

Update a GLB pool.

```
$ ibmcloud cis glb-pool-update GLB_POOL_ID [--enable-origin ORIGIN_NAME --enable-origin ORIGIN_NAME ...] [--disable-origin ORIGIN_NAME --disable-origin ORIGIN_NAME ...] [--add-origin ORIGIN_PARAMETER --add-origin ORIGIN_PARAMETER ...] [--remove-origin ORIGIN_NAME --remove-origin ORIGIN_NAME ...] [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis glb-pool-update GLB_POOL_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis glb-pool-update GLB_POOL_ID (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`GLB_POOL_ID`

The ID of the global load balancer pool. Required.

`--json`

The JSON file or JSON string that is used to describe a GLB pool.

- The required fields in JSON data are `name` , `origins` and `check_regions` :
 - `name` : A short name (tag) for the pool.
 - `origins` : A list of origins within this pool.
 - `check_regions` : A list of geographic region code.
- The optional fields are `description` , `minimum_origins` , `enabled` , `monitor` .

Sample JSON data:

```
{
  "name": "us-pool",
  "description": "application server pool in US",
  "origins": [
    {
      "name": "us-app-dal01",
      "address": "1.1.1.1",
      "enabled": true,
      "header": {
        "host": ["example.com"]
      }
    }
  ]
}
```

```
    }
  },
  {
    "name": "us-app-dal02",
    "address": "2.2.2.2",
    "enabled": true
  }
],
"minimum_origins": 1,
"check_regions": [ "WNAM" ],
"monitor": "f1aba936b94213e5b8dca0c0dbf1f9cc",
"enabled": true
}
```

--enable-origin

Enable the origin within the Pool. The value can be ORIGIN_NAME or ORIGIN_ADDRESS.

--disable-origin

Disable the origin within the Pool. The value can be ORIGIN_NAME or ORIGIN_ADDRESS.

--add-origin

Add an origin into the Pool. ORIGIN_NAME and ORIGIN_ADDRESS are required. For example, --add-origin name=us-app-dal01,address=1.1.1.1,enabled=true,weight=0.5,host=example.com

--remove-origin

Remove an origin from the Pool. The value can be ORIGIN_NAME or ORIGIN_ADDRESS.

-s, --json-str

Deprecated. The JSON data used to describe a GLB pool.

-j, --json-file

Deprecated. A file contains input JSON data.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update GLB pool `GLB_POOL_ID`.

```
$ ibmcloud cis glb-pool-update GLB_POOL_ID --json '{"description":"application server pool in US", "name":"us-pool", "enabled":true, "check_regions":["WNAM"], "minimum_origins":1,"monitor":"f1aba936b94213e5b8dca0c0dbf1f9cc", "origins":[{"name":"us-app-dal01","address":"1.1.1.1","enabled":true,"header":{"host":["example.com"]}}, {"name":"us-app-dal02","address":"2.2.2.2","enabled":true}]}'
```

ibmcloud cis glb-monitors

List GLB monitors for a service instance.

```
$ ibmcloud cis glb-monitors [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

List all GLB monitors.

```
$ ibmcloud cis glb-monitors
```

ibmcloud cis glb-monitor-create

Create a GLB monitor for a service instance.

```
$ ibmcloud cis glb-monitor-create (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis glb-monitor-create (-s, --json-str JSON_STR | -, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

--json

The JSON file or JSON string that is used to describe a GLB monitor. Required.

- The required fields in JSON data are `type` .
 - `type` : The protocol to use for the healthcheck. Valid values are `HTTP` , `HTTPS` , and `TCP` .
- The optional fields are `description` , `timeout` , `retries` , `interval` .
 - `description` : Description.
 - `timeout` : The timeout (in seconds) before marking the health check as failed.
 - `retries` : The number of retries to attempt in case of a timeout before marking the origin as unhealthy.
 - `interval` : The interval between each health check.
- For `TCP` type health check. Extra required fields are `port` .
 - `port` : The TCP port to use for the health check.
- For `HTTP/HTTPS` type health check. Extra option fields are `port` , `expected_body` , `expected_codes` , `method` , `path` , `header` , `follow_redirects` , `allow_insecure` .
 - `port` : The TCP port to use for the health check.
 - `expected_body` : A case-insensitive substring to look for in the response body.
 - `expected_codes` : The expected HTTP response code or code range of the health check.
 - `method` : The HTTP method to use for the health check.
 - `path` : The endpoint path to health check against.
 - `header` : The HTTP request headers to send in the health check.
 - `follow_redirects` : Follow redirects if returned by the origin.
 - `allow_insecure` : Do not validate the certificate when monitor use HTTPS.
 - `probe_zone` : Assign this monitor to emulate the specified zone while probing.

Sample JSON data:

For HTTP/HTTPS:

```
{
  "description": "Health monitor of web service",
  "type": "https",
  "method": "GET",
  "path": "/health",
  "header": {
    "Host": [
      "example.com"
    ],
    "X-App-ID": [
      "abc123"
    ]
  },
  "timeout": 5,
  "retries": 2,
  "interval": 90,
```

```
"follow_redirects": true,
"allow_insecure": false,
"expected_codes": "2xx",
"expected_body": "alive",
"probe_zone": "example.com"
}
```

For TCP:

```
{
  "description": "Health monitor of TCP",
  "type": "tcp",
  "port": 80,
  "timeout": 5,
  "retries": 2,
  "interval": 90
}
```

`-s, --json-str`

Deprecated. The JSON data used to describe a GLB monitor.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a GLB monitor.

```
$ ibmcloud cis glb-monitor-create --json '{"type":"https", "description":"Health monitor of web service", "method":"GET", "path":"/health", "header":{"Host":["example.com"],"X-App-ID":["abc123"]}', "port":8080, "timeout":5, "retries":2, "interval":90, "expected_body":"alive", "expected_codes":"2xx", "follow_redirects":true, "allow_insecure":true}'
```

ibmcloud cis glb-monitor

Show the details of a GLB monitor.

```
$ ibmcloud cis glb-monitor GLB_MON_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`GLB_MON_ID`

The ID of the global load balancer monitor. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Show the details of the GLB monitor `GLB_MON_ID`.

```
$ ibmcloud cis glb-monitor GLB_MON_ID
```


ibmcloud cis glb-monitor-delete

Delete the GLB monitor for a service instance.

```
$ ibmcloud cis glb-monitor-delete GLB_MON_ID [-i, --instance INSTANCE]
```

Command options

`GLB_MON_ID`

The ID of global load balancer monitor. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete the GLB monitor `GLB_MON_ID`.

```
$ ibmcloud cis glb-monitor-delete GLB_MON_ID
```

ibmcloud cis glb-monitor-update

Update the GLB monitor for a service instance.

```
$ ibmcloud cis glb-monitor-update GLB_MON_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis glb-monitor-update GLB_MON_ID (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`GLB_MON_ID`

The ID of the global load balancer monitor. Required.

`--json`

The JSON file or JSON string that is used to describe a GLB monitor. Required.

- The required fields in JSON data are `type`.
 - `type` : The protocol to use for the healthcheck. Valid values are `HTTP`, `HTTPS`, and `TCP`.
- The optional fields are `description`, `timeout`, `retries`, `interval`.
 - `description` : Description.
 - `timeout` : The timeout (in seconds) before marking the health check as failed.
 - `retries` : The number of retries to attempt in case of a timeout before marking the origin as unhealthy.
 - `interval` : The interval between each health check.
- For `TCP` type health check. Extra required fields are `port`.
 - `port` : The TCP port to use for the health check.
- For `HTTP/HTTPS` type health check. Extra option fields are `port`, `expected_body`, `expected_codes`, `method`, `path`, `header`, `follow_redirects`, `allow_insecure`.
 - `port` : The TCP port to use for the health check.
 - `expected_body` : A case-insensitive substring to look for in the response body.
 - `expected_codes` : The expected HTTP response code or code range of the health check.
 - `method` : The HTTP method to use for the health check.
 - `path` : The endpoint path to health check against.
 - `header` : The HTTP request headers to send in the health check.
 - `follow_redirects` : Follow redirects if returned by the origin.
 - `allow_insecure` : Do not validate the certificate when monitor use HTTPS.
 - `probe_zone` : Assign this monitor to emulate the specified zone while probing.

Sample JSON data:

For HTTP/HTTPS:

```
{
  "description": "Health monitor of web service",
  "type": "https",
  "method": "GET",
  "path": "/health",
  "header": {
    "Host": [
      "example.com"
    ],
    "X-App-ID": [
      "abc123"
    ]
  },
  "timeout": 5,
  "retries": 2,
  "interval": 90,
  "follow_redirects": true,
  "allow_insecure": false,
  "expected_codes": "2xx",
  "expected_body": "alive",
  "probe_zone": "example.com"
}
```

For TCP:

```
{
  "description": "Health monitor of TCP",
  "type": "tcp",
  "port": 80,
  "timeout": 5,
  "retries": 2,
  "interval": 90
}
```

`-s, --json-str`

Deprecated. The JSON data used to describe a GLB monitor.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update GLB monitor `GLB_MON_ID`.

```
$ ibmcloud cis glb-monitor-update GLB_MON_ID --json '{"type":"https", "description":"Health monitor of web service", "method":"GET", "path":"/health", "header":{"Host":["example.com"],"X-App-ID":["abc123"]}, "port":8080, "timeout":5, "retries":2, "interval":90, "expected_body":"alive", "expected_codes":"2xx", "follow_redirects":true, "allow_insecure":true}'
```

ibmcloud cis glb-events

List status changes from origins that are connected to a GLB monitor.

```
$ ibmcloud cis glb-events [-s, --since START_DATE] [-u, --until END_DATE] [--origin-name ORIGIN_NAME] [--pool-name POOL_NAME]
```



```
[--origin-healthy (true | false)] [--pool-healthy (true | false)]
[-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`-s, --since`

Start date requesting data period in the ISO8601 format. For example `2018-11-26` .

`-u, --until`

End date requesting data period in the ISO8601 format. For example `2018-11-28` .

`--origin-name`

The name for the origin to filter for.

`--pool-name`

The name for the pool to filter for.

`--origin-healthy`

If true, filter events where the origin status is healthy, if false, filter events where the origin status is unhealthy. The default value is `true` and valid values are `true` and `false` .

`--pool-healthy`

If true, filter events where the pool status is healthy, if false, filter events where the pool status is unhealthy. The default value is `true` and valid values are `true` and `false` .

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get glb events.

```
$ ibmcloud cis glb-events --since "2020-05-20" --until "2020-05-22" --origin-name "dal09" --origin-healthy true --instance INSTANCE
```

Instant Logs

Create and get logs of serverless functions instantly by using the following `instant-log` commands.

instant-log-create

Creates an instant logs job for a domain. The command returns a `Destination` , which is valid for 60 minutes.

```
$ cis instant-log-create DNS_DOMAIN_ID [--fields FIELD1,FIELD2,FIELD3|all] [--filter FILTER] [--sample SAMPLE] [-i, --instance INSTANCE] [--output FORMAT] [-h, -help HELP]
```



Note: You can have only one active Instant Logs session per domain and the maximum session time is 60 minutes.

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--fields`

Define the field set in return.

- This field must be specified as a comma-separated list without any whitespaces, and all fields must exist.
- The order in which fields are specified doesn't matter, and the order of fields in the response is not specified.

- The fields are expected to be case-sensitive.

`--filter`

Filters to drill down into specific events. Filters consist of three parts: key, operator, and value. For information about supported operators, see [Using fields, functions, and expressions](#).

`--sample`

The sample rate of the records set by the client: `sample`: 1 is 100% of records.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

`-h, --help`

Get help on this command.

Examples

Create an instant log for domain `DNS_DOMAIN_ID`:

```
$ ibmcloud cis instant-log-create DNS_DOMAIN_ID --fields all --sample 1
```

The following are three examples of filters:

- Filter when client IP country is not Canada:

```
"filter": "{\"where\":{\"and\":{\"key\":\"ClientCountry\",\"operator\":\"neq\",\"value\":\"ca\"}}}"
```

- Filter when the status code returned from CIS is either 200 or 201:

```
"filter": "{\"where\":{\"and\":{\"key\":\"EdgeResponseStatus\",\"operator\":\"in\",\"value\":\"200,201\"}}}"
```

- Filter when the request path contains "/static" and the request hostname is "example.com":

```
"filter": "{\"where\":{\"and\":{\"key\":\"ClientRequestPath\",\"operator\":\"contains\",\"value\":\"/static\"}, {\"where\":{\"and\":{\"key\":\"ClientRequestHost\",\"operator\":\"eq\",\"value\":\"example.com\"}}}}}"
```

instant-log-get

Get the instant logs job for a domain.

```
$ cis instant-log-get DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT] [-h, --help HELP]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

`-h, --help`

Help on this command.

Example

Get the instant logs job for domain `DNS_DOMAIN_ID`:

```
ibmcloud cis instant-log-get DNS_DOMAIN_ID
```

Leaked credential checks

Enterprise Plans Only

Manage leaked credential checks and custom detections for a DNS domain by using the following `leaked-credential-checks` commands.

`ibmcloud cis leaked-credential-checks status-get`

Get the leaked credential checks status for a DNS domain.

```
$ ibmcloud cis leaked-credential-checks status-get DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Example

```
$ ibmcloud cis leaked-credential-checks status-get DNS_DOMAIN_ID
```

`ibmcloud cis leaked-credential-checks status-update`

Enable or disable leaked credential checks for a DNS domain.

```
$ ibmcloud cis leaked-credential-checks status-update DNS_DOMAIN_ID (--enabled true|false) [-i, --instance INSTANCE] [--output FORMAT]
```

You can also provide JSON input from a file or directly as a string.

```
$ ibmcloud cis leaked-credential-checks status-update DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--enabled`

Indicates whether leaked credential checks are enabled for the DNS domain. Valid values are `true` and `false`.

`--json`

The JSON file or JSON string that describes the leaked credential checks configuration. The required field is `enabled`.

```
{
  "enabled": true
}
```

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

```
$ ibmcloud cis leaked-credential-checks status-update DNS_DOMAIN_ID --enabled true
ibmcloud cis leaked-credential-checks status-update DNS_DOMAIN_ID --json @status.json --output json
```

ibmcloud cis leaked-credential-checks detection-list

List custom leaked credential detections for a DNS domain.

```
$ ibmcloud cis leaked-credential-checks detection-list DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Example

```
$ ibmcloud cis leaked-credential-checks detection-list DNS_DOMAIN_ID --output json
```

ibmcloud cis leaked-credential-checks detection-get

Get a custom leaked credential detection.

```
$ ibmcloud cis leaked-credential-checks detection-get DNS_DOMAIN_ID DETECTION_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

DETECTION_ID

The ID of the custom leaked credential detection. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Example

```
$ ibmcloud cis leaked-credential-checks detection-get DNS_DOMAIN_ID DETECTION_ID
```

ibmcloud cis leaked-credential-checks detection-create

Create a custom leaked credential detection.

```
$ ibmcloud cis leaked-credential-checks detection-create DNS_DOMAIN_ID (--username EXPRESSION) (--password EXPRESSION) [-i, --instance INSTANCE] [--output FORMAT]
```

You can also provide JSON input from a file or directly as a string.

```
$ ibmcloud cis leaked-credential-checks detection-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--username

The ruleset expression that locates the username in a request. Required when **--json** is not used.

--password

The ruleset expression that locates the password in a request. Required when **--json** is not used.

--json

The JSON file or JSON string that describes the custom detection. The required fields are **username** and **password** .

```
{
  "username": "lookup_json_string(http.request.body.raw, \"username\")",
  "password": "lookup_json_string(http.request.body.raw, \"password\")"
}
```

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

```
$ ibmcloud cis leaked-credential-checks detection-create DNS_DOMAIN_ID --username 'lookup_json_string(http.request.body.raw, "username")' --password 'lookup_json_string(http.request.body.raw, "password")'
ibmcloud cis leaked-credential-checks detection-create DNS_DOMAIN_ID --json @detection.json --output json
```

ibmcloud cis leaked-credential-checks detection-update

Update a custom leaked credential detection. Specify at least one of **--username** or **--password** , or provide JSON input.

```
$ ibmcloud cis leaked-credential-checks detection-update DNS_DOMAIN_ID DETECTION_ID [--username EXPRESSION] [--password EXPRESSION] [-i, --instance INSTANCE] [--output FORMAT]
```

You can also provide JSON input from a file or directly as a string.

```
$ ibmcloud cis leaked-credential-checks detection-update DNS_DOMAIN_ID DETECTION_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

DETECTION_ID

The ID of the custom leaked credential detection. Required.

--username

The ruleset expression that locates the username in a request.

--password

The ruleset expression that locates the password in a request.

`--json`

The JSON file or JSON string that describes the custom detection. The optional fields are `username` and `password` ; specify at least one.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

```
$ ibmcloud cis leaked-credential-checks detection-update DNS_DOMAIN_ID DETECTION_ID --username 'lookup_json_string(http.request.body.raw, "email")'
ibmcloud cis leaked-credential-checks detection-update DNS_DOMAIN_ID DETECTION_ID --json @detection.json --output json
```

ibmcloud cis leaked-credential-checks detection-delete

Delete a custom leaked credential detection.

```
$ ibmcloud cis leaked-credential-checks detection-delete DNS_DOMAIN_ID DETECTION_ID [-f, --force] [-i, --instance INSTANCE]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`DETECTION_ID`

The ID of the custom leaked credential detection. Required.

`-f, --force`

Delete the detection without prompting for confirmation.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Example

```
$ ibmcloud cis leaked-credential-checks detection-delete DNS_DOMAIN_ID DETECTION_ID --force
```

Logpull

ibmcloud cis log-lookup-rayid

Enterprise Plans Only

Retrieve the log event that is associated with a specific Ray ID. Log retention must be enabled for the DNS domain before you use this command.

**Note:** If the Ray ID includes a data center suffix, such as `-XXX` , remove the suffix before you run the command.

```
$ ibmcloud cis log-lookup-rayid DNS_DOMAIN_ID RAY_ID [--fields FIELDS] [--timestamps FORMAT] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`RAY_ID`

The Ray ID of the request. Required.

`--fields`

A comma-separated list of case-sensitive log fields to return.

`--timestamps`

The format for timestamp fields. Valid values are `unix`, `unixnano`, and `rfc3339`. The default value is `unix`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

```
$ ibmcloud cis log-lookup-rayid DNS_DOMAIN_ID RAY_ID
ibmcloud cis log-lookup-rayid DNS_DOMAIN_ID RAY_ID --fields RayID,ClientIP,EdgeResponseStatus --timestamps rfc3339 --output json
```

ibmcloud cis logpull

Manipulate Logpull services by using the following `logpull` commands.

```
$ ibmcloud cis logpull DNS_DOMAIN_ID --available-fields [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--available-fields`

List of all available fields.

`--ray-id`

Lookup logs by specific Ray ID.

`--fields`

Define the field set in return. This field must be specified as a comma-separated list without any spaces, and all fields must exist. The order in which fields are specified doesn't matter, and the order of fields in the response is not specified. Note that fields are expected to be case sensitive.

`--start`

The (inclusive) beginning of the requested time frame. This can be a unix timestamp (in seconds or nanoseconds), or an absolute timestamp that conforms to RFC 3339. Currently, it cannot exceed a time in the past greater than 7 days. Default is 65 minutes earlier.

`--end`

The (exclusive) end of the requested time frame. This value can be a unix timestamp (in seconds or nanoseconds), or an absolute timestamp that conforms to RFC 3339. The `end` must be at least 5 minutes earlier than now and must be later than `start`. The difference between `start` and `end` must be not greater than 1h. The default is 5 minutes earlier.

`--count`

Number of logs to retrieve. The default value is `-1`.

`--sample`

Percentage of sampling. When a sample is provided, a sample of matching records is returned. If `sample=0.1` then 10% of records are returned. The sampling is random: repeated calls not only return different records, but likely also vary slightly in the number of returned records. When count is also specified, count is applied to the number of returned records, not the sampled records. So, with `sample=0.05` and `count=7`, when there is a total of 100 records available, approximately 5 records are returned. When there are 1000 records, 7 records are returned. When there are 10,000 records, 7 records are returned. The default value is `1`.

`--timestamps`

Set the format in which response timestamps are returned. Valid values are `unix`, `unixnano` and `rfc3339`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

```
ibmcloud cis logpull DNS_DOMAIN_ID --available-fields
ibmcloud cis logpull DNS_DOMAIN_ID --ray-id 59348abde87afe50 --all-fields --timestamps rfc3339 --output json
ibmcloud cis logpull DNS_DOMAIN_ID --start 2020-05-18T12:14:58Z --end 2020-05-18T13:14:58Z --fields ClientIP,EdgeServerIP,ClientRequestHost --count 10 --sample 1 --timestamps rfc3339 --output json
```

Logpush

Enterprise Plans Only

ibmcloud cis logpush-job-create

Enterprise Plans Only

Create a new logpush job for a domain.

```
$ ibmcloud cis logpush-job-create DNS_DOMAIN_ID --destination DESTINATION_URL --name NAME [--enable true|false] [--fields FIELDS | all] [--timestamps format][--dataset DATASET] [--frequency FREQUENCY] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--destination`

Specify a destination where data is pushed. For more information about supported destinations, see [Managing Logpush jobs](#).

- Syntax for Cloud Object Storage Path: `cos://<BUCKET_OBJECT_PATH>?region=<REGION>&instance-id=<IBM_CLOUD_OBJECT_STORAGE_INSTANCE_ID>&ibm_api_key=<API_KEY>` Example: `'cos://cis-test-bucket/logs?region=us&instance-id=f75e6d90-4212-4026-851c-d572071146cd&ibm_api_key=XXX'` To separate logs in to daily subfolders, use the special string `{DATE}` in the bucket path. It is substituted with the date in `YYYYMMDD` format, for example '20190423'. Subfolders are created as appropriate, for example: `'cos://cis-test-bucket/logs/{DATE}?region=us&instance-id=f75e6d90-4212-4026-851c-d572071146cd&ibm_api_key=XXX'`

`--name`

Job name. Required.

`--enable`

Enable the job. The job is disabled by default.

`--fields`

Define the list of log fields to be included in log files. Multiple fields can be separated by commas and use command `[ ibmcloud cis logpush-available-fields DNS_DOMAIN_ID ]` to get the comprehensive list of available log fields, or use `all` to include all available fields in log files. Note that fields are expected to be case sensitive.

`--timestamps`

Set the format in which response timestamps are returned. Valid values are `unix`, `unixnano` and `rfc3339`.

`--dataset`

The category of logs you want to receive. This value cannot be changed after the job is created. Valid values are `http_requests`, `range_events`, `firewall_events`, `dns_logs`. The default value is `http_requests`.

`--frequency`

The frequency at which CIS sends batches of logs to your destination. Setting the frequency to high sends your logs in larger quantities of smaller files. Setting the frequency to low sends logs in smaller quantities of larger files. Valid values are `high`, `low`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a logpush job for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis logpush-job-create DNS_DOMAIN_ID --destination cos://cis-test-bucket/logs?region=us&instance-id=f75e6d90-4212-4026-851c-d572071146cd&ibm_api_key=XXX --name logpushcreate --enable true --fields all --timestamps rfc3339 --dataset http_requests --frequency low --output json
```

ibmcloud cis logpush-job-update

Enterprise Plans Only

Update a logpush job for a domain.

```
$ ibmcloud cis logpush-job-update DNS_DOMAIN_ID [--destination DESTINATION_URL] [--enable true|false] [--fields FIELDS | all] [--timestamps format] [--dataset DATASET] [--jobid JOB_ID] [--frequency FREQUENCY] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--destination`

Specify a destination where data is pushed. For more information about supported destinations, see [Managing Logpush jobs](#). Syntax for Cloud Object Storage Path: `cos://<BUCKET_OBJECT_PATH>?region=<REGION>&instance-id=<IBM_CLOUD_OBJECT_STORAGE_INSTANCE_ID>ibm_api_key=<API_KEY>` Example: `'cos://cis-test-bucket/logs?region=us&instance-id=f75e6d90-4212-4026-851c-d572071146cdibm_api_key=XXX'` To separate logs into daily subfolders, use the special string `{DATE}` in the bucket path. It is to be substituted with the date in `YYYYMMDD` format, for example `'20190423'`. Subfolders are created as appropriate, for example: `'cos://cis-test-bucket/logs/{DATE}?region=us&instance-id=f75e6d90-4212-4026-851c-d572071146cdibm_api_key=XXX'`

`--enable`

Enable the job. The job is disabled by default.

`--fields`

Define the list of log fields to be included in log files. Multiple fields can be separated by commas and use command `ibmcloud cis logpush-available-fields DNS_DOMAIN_ID` to get the comprehensive list of available log fields, or use `all` to include all available fields in log files. Note that fields are expected to be case sensitive.

`--timestamps`

Set the format in which response timestamps are returned. Valid values are `unix`, `unixnano`, `rfc3339`.

`--dataset`

The category of logs you want to receive. This value cannot be changed after the job is created. Valid values are `http_requests`, `range_events`, `firewall_events`, `dns_logs`. The default value is `http_requests`.

`--jobid`

JOB_ID is the ID of the logpush job.

`--frequency`

The frequency at which CIS sends batches of logs to your destination. Setting the frequency to high sends your logs in larger quantities of smaller files. Setting the frequency to low sends logs in smaller quantities of larger files. Valid values are `high` and `low`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update `range_events` logpush job for domain `DNS_DOMAIN_ID` .

```
$ ibmcloud cis logpush-job-update DNS_DOMAIN_ID --destination cos://cis-test-bucket/logs?region=us&instance-id=f75e6d90-4212-4026-851c-d572071146cd&ibm_api_key=XXX --enable true --fields all --timestamps rfc3339 --dataset range_events --frequency high --output json
```

ibmcloud cis logpush-jobs

Enterprise Plans Only

Get all logpush jobs for a domain.

```
$ ibmcloud cis logpush-jobs DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

-output

The output format. Currently, `json` is the only supported value.

Examples

Get all logpush jobs for domain `DNS_DOMAIN_ID` .

```
$ ibmcloud cis logpush-jobs DNS_DOMAIN_ID --output json
```

ibmcloud cis logpush-job

Enterprise Plans Only

Get the details of a logpush job for a domain.

```
$ ibmcloud cis logpush-job DNS_DOMAIN_ID [--dataset DATASET] [--jobid JOB_ID] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--dataset

The category of logs you want to receive. This value cannot be changed after the job is created. Valid values are `http_requests` , `range_events` , `firewall_events` , `dns_logs` . The default value is `http_requests` .

--jobid

JOB_ID is the ID of the logpush job.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Get details of `http_requests` logpush job.

```
$ ibmcloud cis logpush-job DNS_DOMAIN_ID --dataset http_requests --output json
```

ibmcloud cis logpush-job-delete

Enterprise Plans Only

Delete a logpush job for a domain.

```
$ ibmcloud cis logpush-job-delete DNS_DOMAIN_ID [--dataset DATASET] [--jobid JOB_ID] [-f, --force] [-i, --instance INSTANCE]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--dataset

The category of logs you want to receive. This value cannot be changed after the job is created. Valid values are `http_requests` , `range_events` , `firewall_events` , `dns_logs` . The default value is `http_requests` .

--jobid

JOB_ID is the ID of the logpush job.

-f, --force

Delete logpush job without prompting for confirmation.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete `http_requests` logpush job for domain `DNS_DOMAIN_ID` .

```
$ ibmcloud cis logpush-job-delete DNS_DOMAIN_ID --dataset http_requests --force
```

ibmcloud cis logpush-available-fields

Enterprise Plans Only

Get all available fields for a data set.

```
$ ibmcloud cis logpush-available-fields DNS_DOMAIN_ID [--dataset DATASET] [-i, --instance INSTANCE]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--dataset

The category of logs you want to receive. This value cannot be changed after the job is created. Valid values are `http_requests` , `range_events` , `firewall_events` , `dns_logs` . The default value is `http_requests` .

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Get all available fields for `http_requests` logs.

```
$ ibmcloud cis logpush-available-fields DNS_DOMAIN_ID --dataset http_requests
```

Log retention

Enterprise Plans Only

ibmcloud cis log-retention

Get a log retention setting for the domain.

```
$ ibmcloud cis log-retention DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Get a log retention setting for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis log-retention DNS_DOMAIN_ID --output json
```

ibmcloud cis log-retention-update

Update the log retention setting for the domain.

```
$ ibmcloud cis log-retention-update DNS_DOMAIN_ID (--flag on|off) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--flag

Whether to turn log retention on or off. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Enable log retention for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis log-retention-update DNS_DOMAIN_ID --flag on --output json
```

Managed lists

Manipulate managed lists by using the following `managed-lists` commands.

cis managed-lists

List managed lists in an instance.


```
$ cis managed-lists [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Example

```
$ ibmcloud cis managed-lists --output json --instance INSTANCE
```

Metrics

Manipulate metrics by using the following `metrics` commands.

ibmcloud cis firewall-event-analytics

Retrieve a full log of firewall events.

```
$ ibmcloud cis firewall-event-analytics DNS_DOMAIN_ID [--dataset DATA_SET] [--filter FILTER] [--order FILTER_ORDER] [--limit LIMIT_NUMBER] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--dataset`

Requested dataset. The default value is `firewallEventsAdaptiveGroups`.

Use the following table to identify which datasets are included in your plan and the range of historical data you can query.

Dataset	Trial / Standard / Standard-Next	Enterprise / Security / GLB
firewallEventsAdaptiveGroups	30 days	30 days
firewallEventsAdaptive	30 days	30 days

Datasets included in your plan

`--filter`

Filter events. The default value is the last 6 hours of data.

The following operators are supported for all filter options:

Operator	Comparison
gt	greater than
lt	less than
geq	greater or equal to
leq	less or equal to

neq	not equal
in	in

Operators supported for filter options

- firewallEventsAdaptiveGroups

 filter options.
 - datetime
 - datetimeFifteenMinutes
 - datetimeHour
 - datetimeFiveMinutes
 - datetimeMinute
 - matchIndex
 - sampleInterval
- The following filter options support

like

 and

notlike

 operators.
 - action
 - clientASNDescription
 - clientAsn
 - clientCountryName
 - clientIP
 - clientRefererHost
 - clientRefererPath
 - clientRefererQuery
 - clientRefererScheme
 - clientRequestHTTPHost
 - clientRequestHTTPMethodName
 - clientRequestHTTPProtocol
 - clientRequestPath
 - clientRequestQuery
 - clientRequestScheme
 - edgeColoName
 - edgeResponseStatus
 - kind
 - originResponseStatus
 - originatorRayName
 - rayName
 - ref
 - ruleId
 - source
 - userAgent

--order

Output order. (The default value is

datetime_ASC

)

The following list is usable order options for corresponding dataset and all of order options support ASC and DESC action. Combine these filter options and action with

_

.

For example,

datetime_ASC

 orders by datetime ascending.

- firewallEventsAdaptiveGroups

 order options.
 - datetime
 - datetimeFifteenMinutes
 - datetimeHour
 - datetimeFiveMinutes
 - datetimeMinute

- `action`
- `avg_sampleInterval`
- `clientASNDescription`
- `clientAsn`
- `clientCountryName`
- `clientIPClass`
- `clientIP`
- `clientRefererHost`
- `clientRefererPath`
- `clientRefererQuery`
- `clientRefererScheme`
- `clientRequestHTTPHost`
- `clientRequestHTTPMethodName`
- `clientRequestHTTPProtocol`
- `clientRequestPath`
- `clientRequestQuery`
- `clientRequestScheme`
- `count`
- `edgeColoName`
- `edgeResponseStatus`
- `kind`
- `matchIndex`
- `originResponseStatus`
- `originatorRayName`
- `rayName`
- `ref`
- `ruleId`
- `sampleInterval`
- `source`
- `userAgent`
- `visibility`

`--limit`

The number of events to return. (minimum: `1`, maximum: `10000`, default: `10000`)

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get firewall event analytics for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis firewall-event-analytics DNS_DOMAIN_ID --order datetime_ASC --filter "datetime_geq:2020-06-28T00:00:00Z" --filter "datetime_leq:2020-06-29T00:00:00Z" --output json
```

ibmcloud cis http-request-analytics

Retrieve a full log of http request events.

```
$ ibmcloud cis http-request-analytics DNS_DOMAIN_ID [--dataset DATA_SET] [--filter FILTER] [--order FILTER_ORDER] [--limit LIMIT_NUMBER] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--dataset

Requested dataset. The default value is `httpRequests1dGroups` . Use the following table to identify which datasets are included in your plan and the range of historical data you can query.

Dataset	Trial / Standard / Standard-next	Enterprise / Security / GLB
httpRequests1dGroups	365 days	365 days
httpRequests1hGroups	30 days	90 days
httpRequests1mGroups	3 days	7 days

Identify datasets included in your plan

--filter

Filter events. The default value is the last 7 days data. The following operators are supported for all filter options:

Operator	Comparison
gt	greater than
lt	less than
geq	greater or equal to
leq	less or equal to
neq	not equal
in	in

Operators supported for filter options

- `httpRequests1dGroups` and `httpRequests1hGroups` filter options.
 - `date`
- `httpRequests1mGroups` filter options.
 - `datetime`
 - `datetimeFifteenMinutes`
 - `datetimeHour`
 - `datetimeDay`

--order

Output order. (The default value is `datetime_ASC`)

The following list is usable order options for corresponding dataset and all of order options support ASC and DESC action. Combine these order options and action with `_` .

For example, `date_ASC` orders by date ascending.

- Common order options for every http dataset.
 - `orderByParams`
 - `date`
 - `sum_bytes`

- `sum_cachedBytes`
- `sum_cachedRequests`
- `sum_requests`
- `httpRequests1dGroups` order options.
 - `avg_bytes`
 - `sum_encryptedBytes`
 - `sum_encryptedRequests`
 - `sum_pageViews`
 - `sum_threats`
 - `uniq_uniques`
- `httpRequests1hGroups` order options.
 - `avg_bytes`
 - `sum_encryptedBytes`
 - `sum_encryptedRequests`
 - `sum_pageViews`
 - `sum_threats`
 - `uniq_uniques`
 - `datetime`
- `httpRequests1mGroups` order options.
 - `avg_bytes`
 - `sum_encryptedBytes`
 - `sum_encryptedRequests`
 - `sum_pageViews`
 - `sum_threats`
 - `uniq_uniques`
 - `datetime`
 - `datetimeFifteenMinutes`
 - `datetimeHour`
 - `datetimeFifteenMinutes`
 - `datetimeHour`
 - `datetimeDay`

`--limit`

The number of events to return. (minimum: `1`, maximum: `10000`, default: `10000`)

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get http request analytics for domain `DNS_DOMAIN_ID` .

```
$ ibmcloud cis http-request-analytics DNS_DOMAIN_ID --order date_ASC --dataset httpRequests1dGroups --limit 500 --filter "date_geq:2020-06-28" --filter "date_leq:2020-06-29" --output json
```

ibmcloud cis web-analytics (Deprecated)

Web analytics are deprecated on 2 November 2020. Use [ibmcloud cis http-request-analytics](#) instead. Get analytics of the DNS domain.

```
$ ibmcloud cis web-analytics DNS_DOMAIN_ID [--recent DURATION] [-t, --table requests | bandwidth | uniques | threats | status_code] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--recent`

The beginning of the requested time frame. Valid values are `6h` (6 hours ago), `12h`, `1d` (1 day ago), `1w` (1 week ago), `1m` (1 month ago), `2m`, `3m`. The default value is `1w`.

`-t, --table`

Output table. Valid values are `requests`, `bandwidth`, `uniques`, `threats` and, `status_code`. If this field is not set, it outputs all the tables.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get web analytics for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis web-analytics DNS_DOMAIN_ID --recent 1d -t requests
```

ibmcloud cis dns-analytics

Get DNS analytics of the domain.

```
$ ibmcloud cis dns-analytics DNS_DOMAIN_ID DIMENSION [-s, --since TIME] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`DIMENSION`

The queried dimension. Valid values are `queries-by-response-code`, `queries-by-type`, `queries-by-name`. Required.

`-s, --since`

Since time to now. Valid values are `6h` (6 hours ago), `12h`, `1d` (1 day ago), `1w` (1 week ago).

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get DNS analytics for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis dns-analytics DNS_DOMAIN_ID queries-by-response-code -s 6h --output json
```

ibmcloud cis ratelimit-analytics

Get rate limit analytics for a DNS domain.

```
$ ibmcloud cis ratelimit-analytics DNS_DOMAIN_ID [--recent DURATION] [--time-delta SECONDS] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--recent

The beginning of the requested time frame. Valid values are `6h` (6 hours ago), `12h`, `1d` (1 day ago), `1w` (1 week ago), `1m` (1 month ago), `2m`, `3m`. The default value is `1w`.

--time-delta

The time interval (seconds) of each analytic's record. Valid values are `60`, `3600`, `86400`, `2592000`. The default value is `3600`.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Get rate limit analytics for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis ratelimit-analytics DNS_DOMAIN_ID --recent 6h --time-delta 3600 --output json
```

MTLS enable

ibmcloud cis access-enable

Enterprise Plans Only

Enable Mutual TLS for a service instance.

```
$ ibmcloud cis access-enable [-i, --instance INSTANCE]
```

Command options

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Enable Mutual TLS.

```
$ ibmcloud cis access-enable
```

Origin certificates

ibmcloud cis origin-certificates

List all origin certificates for a DNS domain.

```
$ ibmcloud cis origin-certificates DNS_DOMAIN_ID [--instance INSTANCE_NAME] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List all origin certificates for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis origin-certificates DNS_DOMAIN_ID --output json
```

ibmcloud cis origin-certificate-create

Create a CIS-signed certificate.

```
$ ibmcloud cis origin-certificate-create DNS_DOMAIN_ID [--request-type REQUEST_TYPE] [--hostnames HOST_NAME1] [--hostnames HOST_NAME2] [--requested-
validity DAYS] [--csr CSR] [--instance INSTANCE_NAME] [--output FORMAT]
ibmcloud cis origin-certificate-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis origin-certificate-create DNS_DOMAIN_ID --json-str JSON_STR [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis origin-certificate-create DNS_DOMAIN_ID --json-file JSON_FILE [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--request-type REQUEST_TYPE`

Signature type that you want on the certificate. Valid values are `origin-rsa` and `origin-ecc`.

`--hostnames HOSTNAME`

hostname or wildcard name that is bound to the certificate.

`--requested-validity DAYS`

The number of days for which the certificate must be valid. The default value is `5475`.

`--csr CSR`

The Certificate Signing Request (CSR). If this field is not set, CIS generates one.

`--json value*`

The JSON file or JSON string that is used to describe an origin certificate.

- The required fields in JSON data are `request_type`, `hostnames`.
 - `request_type` : Signature type that you want on the certificate. Valid values are `origin-rsa`, `origin-ecc`.
 - `hostnames` : An array of hostnames or wildcard names that are bound to the certificate.
- The optional fields are `requested_validity`, `csr`.
 - `requested_validity` : The number of days for which the certificate must be valid. Valid values are `0`, `7`, `30`, `90`, `365`, `730`, `1095`, `5475`.
 - `csr` : The Certificate Signing Request (CSR). If this field is not set, CIS generates one.

Sample JSON data:

```
{
  "request_type": "origin-rsa",
  "hostnames": [
    "*.example.com",
    "example.com",
  ],
  "requested_validity": 5475,
  "csr": "your_csr"
```

```
}

```

`-s, --json-str`

Deprecated. The JSON data that describes an origin certificate.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a CIS-signed certificate for domain `DNS_DOMAIN_ID`.

```
ibmcloud cis origin-certificate-create DNS_DOMAIN_ID --request-type origin-rsa --hostnames "*.example.com" --hostnames "example.com" --requested-validity 5475 --csr your_csr --output json

ibmcloud cis origin-certificate-create DNS_DOMAIN_ID --json '{"hostnames":["example.com","*.example.com"], "requested_validity":5475,"request_type": "origin-rsa","csr":"-----BEGIN CERTIFICATE REQUEST-----
\nMIICxzCCAa8CAQAwSDELMakGA1UEBhMCVVMx FjAUBgNVBAgTDVNhbiBGcmFuY2lz\nY28xCzAJBgNVBACtAkNBMRQwEgYDVQQDEwtleGFtcGxLM5ldDCCA
SlwDQYJKoZI\nhvcNAQEBAADggEPADCCAQoCggEBALxejtu4b+jPdFeFi6OUsyE8TYJQBm3WfCvL\nHu5EvijMO/4Z2TImwASbwUF7Ir8OLgH+mGIQZe qyNvGoSOM
EaZVXcYfpR1hIVak8\n4GGVr+04IGfOCqaBokaBFIwzclGMZbzKmLGwIQioNxGfqFm6RGYGA3be2Je2iseBc\nN8GV1wYmvYE0RR+yWweJCTJ157exyRzu7sVxaEW9F8
7zBQLyOnwXc64rfIXsIRqilng7F7w5laQYOI8yvmk/jEPCAh a7fkiUfE pj4N12+oPRiMvleJF98chxjD4MH39c5\nnuOsIU LhrWunfh7GB1jwWNA9y44H0snrf+xvoy2TcHmxvma9
Eln8CAwEAAaA6MDgG\nCSqGSib3DQEJDjErMCkwJwYDVR0RBCAwHolLZXhhbXBsZS5uZXSCD3d3dy5leGFt\ncGxLM5ldDANBgkqhkiG9w0BAQsFAAOCAQEAcBa
X6dOnI8ncARrI9ZSF2AJX+8mx\nnpTHY2+Y2C0VvrVDGMtbBRH8R9yMbqWtlxeeNGf//LeMkSKSFa4kbpd x226lfui8\nanauRDBTJGx2R1ccUxmLZXx4my0W5ilMxunu+kez
+BDlu7bTT2io0uXMRHue4i6quH\nnyc5ibxvbJMjR7dqbc anVE10/34oprzXQsJ/VmSuZNXtjbtSKDImcpw6To/eeAJ+J\nnhXykcUihvHyG4A1m2R6qpANBjnA0pHexfwM/Sgfv
pbvUg0T1ubmer8BgTwCKIWslndcWYTthM51JlqRBfNqy4QcBnX+GY05yltEEswQI55wdiS3CJT TA67sdbcQ==\n-----END CERTIFICATE REQUEST-----"}' --output json

```

ibmcloud cis origin-certificate

Get details of an origin certificate.

```
$ ibmcloud cis origin-certificate DNS_DOMAIN_ID CERT_ID [--instance INSTANCE_NAME] [--output FORMAT]

```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`CERT_ID`

The ID of the Origin Certificate. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get details of origin certificate `CERT_ID`.

```
$ ibmcloud cis origin-certificate DNS_DOMAIN_ID CERT_ID --output json

```

ibmcloud cis origin-certificate-delete

Delete an origin certificate.


```
$ ibmcloud cis origin-certificate-delete DNS_DOMAIN_ID CERT_ID [--instance INSTANCE_NAME]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

CERT_ID

The ID of Origin Certificate. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete origin certificate `CERT_ID`.

```
$ ibmcloud cis origin-certificate-delete DNS_DOMAIN_ID CERT_ID
```

Overview

View the overview information for a domain.

ibmcloud cis overview

Show the overview information for a domain.

```
$ ibmcloud cis overview DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show the overview information for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis overview DNS_DOMAIN_ID
```

Page rules

Manipulate page rules by using the following `pagerule` commands.

ibmcloud cis page-rule-create

Create a page rule of the DNS domain.

```
$ ibmcloud cis page-rule-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis page-rule-create DNS_DOMAIN_ID (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--json

The JSON file or JSON string that is used to describe a page rule. Required.

- The required fields in JSON data are targets, actions :
 - **targets** : The target URL pattern to evaluate on a request.
 - **actions** : An array of actions to perform if the targets of this rule match the request. Available actions are:
 - **disable_security**
 - **always_use_https**
 - **ssl**
 - **browser_cache_ttl**
 - **security_level**
 - **cache_level**
 - **edge_cache_ttl**
 - **bypass_cache_on_cookie**
 - **browser_check**
 - **server_side_exclude**
 - **email_obfuscation**
 - **automatic_https_rewrites**
 - **opportunistic_encryption**
 - **ip_geolocation**
 - **explicit_cache_control**
 - **cache_deception_armor**
 - **waf**
 - **forwarding_url**
 - *Deprecated.* **image_load_optimization** .
 - **image_size_optimization**
 - **script_load_optimization**
 - **host_header_override**
 - **resolve_override**
 - Some actions are limited to Enterprise plans:
 - **cache_on_cookie**
 - **disable_apps**
 - **disable_performance**
 - **minify**
 - **origin_error_page_pass_thru**
 - **response_buffering**
 - **true_client_ip_header**
 - **sort_query_string_for_cache**
 - **respect_strong_etag**
- The optional fields are **priority** and **status** :
 - **priority** : A number that indicates the preference for a page rule over another. The default value is **1**.
 - **status** : Status of the page rule. The valid values are **active** and **disabled** (default).

Sample JSON data:

```
{
  "targets": [
    {
      "target": "url",
      "constraint": {
        "operator": "matches",
```

```
      "value": "*example.com/images/*"
    }
  },
],
"actions": [
  {
    "id": "ssl",
    "value": "flexible"
  },
  {
    "id": "browser_cache_ttl",
    "value": 14400
  },
  {
    "id": "security_level",
    "value": "medium"
  },
  {
    "id": "cache_level",
    "value": "basic"
  },
  {
    "id": "edge_cache_ttl",
    "value": 7200
  },
  {
    "id": "bypass_cache_on_cookie",
    "value": "wp-.*|wordpress.*|comment_.*"
  }
]
}
```

`-s, --json-str`

Deprecated. The JSON data describing a page rule.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a page rule for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis page-rule-create DNS_DOMAIN_ID --json '{"targets":[{"target":"url", "constraint":{"operator": "matches", "value":"*example.com/images/*"}}, {"actions":[{"id":"always_online", "value":"on"}], "priority":1, "status": "active"}' --output json
```

ibmcloud cis page-rule-update

Update the page rule of the DNS domain.

```
$ ibmcloud cis page-rule-update DNS_DOMAIN_ID PAGE_RULE_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis page-rule-update DNS_DOMAIN_ID PAGE_RULE_ID (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

PAGE_RULE_ID

The ID of page rule. Required.

--json VALUE

The JSON file or JSON string that is used to describe a page rule. Required.

- The required fields in JSON data are **targets** and **actions** :
 - **targets** : The target URL pattern to evaluate on a request.
 - **actions** : An array of actions to perform if the targets of this rule match the request. Available actions are:
 - **disable_security**
 - **always_use_https**
 - **ssl**
 - **browser_cache_ttl**
 - **security_level**
 - **cache_level**
 - **edge_cache_ttl**
 - **bypass_cache_on_cookie**
 - **browser_check**
 - **server_side_exclude**
 - **email_obfuscation**
 - **automatic_https_rewrites**
 - **opportunistic_encryption**
 - **ip_geolocation**
 - **explicit_cache_control**
 - **cache_deception_armor**
 - **waf**
 - **forwarding_url**
 - *Deprecated.* **image_load_optimization**
 - **image_size_optimization**
 - **script_load_optimization**
 - **host_header_override**
 - **resolve_override**
 - Some actions are limited to Enterprise plans:
 - **cache_on_cookie**
 - **disable_apps**
 - **disable_performance**
 - **minify**
 - **origin_error_page_pass_thru**
 - **response_buffering**
 - **true_client_ip_header**
 - **sort_query_string_for_cache**
 - **respect_strong_etag**
- The optional fields are **priority** and **status** :
 - **priority** : A number that indicates the preference for a page rule over another. The default value is **1**.
 - **status** : Status of the page rule. Valid values are **active** and **disabled**. The default value is **disabled**.

Sample JSON data:

```
{
  "targets": [
    {
      "target": "url",
      "constraint": {
        "operator": "matches",
```

```
      "value": "*example.com/images/*"
    }
  },
],
"actions": [
  {
    "id": "ssl",
    "value": "flexible"
  },
  {
    "id": "browser_cache_ttl",
    "value": 14400
  },
  {
    "id": "security_level",
    "value": "medium"
  },
  {
    "id": "cache_level",
    "value": "basic"
  },
  {
    "id": "edge_cache_ttl",
    "value": 7200
  },
  {
    "id": "bypass_cache_on_cookie",
    "value": "wp-.*|wordpress.*|comment_.*"
  }
]
}
```

`-s, --json-str`

Deprecated. The JSON data describing a page rule.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update page rule `PAGE_RULE_ID`.

```
$ ibmcloud cis page-rule-update DNS_DOMAIN_ID PAGE_RULE_ID --json '{"targets":[{"target":"url", "constraint":{"operator":"matches", "value":"*example.com/images/*"}}, {"actions":[{"id":"always_online", "value":"on"}], "priority":1, "status":"active"}' --output json
```

ibmcloud cis page-rule-delete

Delete a page rule of the DNS domain.

```
$ ibmcloud cis page-rule-delete DNS_DOMAIN_ID PAGE_RULE_ID [-i, --instance INSTANCE]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`PAGE_RULE_ID`

The ID of page rule. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete page rule `PAGE_RULE_ID`.

```
$ ibmcloud cis page-rule-delete DNS_DOMAIN_ID PAGE_RULE_ID
```

ibmcloud cis page-rules

List page rules of the DNS domain.

```
$ ibmcloud cis page-rules DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List all page rules in domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis page-rules DNS_DOMAIN_ID --output json
```

ibmcloud cis page-rule

Get details of a page rule.

```
$ ibmcloud cis page-rule DNS_DOMAIN_ID PAGE_RULE_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`PAGE_RULE_ID`

The ID of page rule. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get details of page rule `PAGE_RULE_ID`.

```
$ ibmcloud cis page-rule DNS_DOMAIN_ID PAGE_RULE_ID --output json
```

Range app

Manipulate how the Range App performs by using the following `range-app` commands:

ibmcloud cis range-app-create

Enterprise Plans Only

Create a new range application.

```
$ ibmcloud cis range-app-create DNS_DOMAIN_ID --name NAME --edge-port EDGE_PORT --origin-direct ORIGIN_DIRECT [--origin-direct ORIGIN_DIRECT] [--proxy-protocol on|off] [--ip-firewall on|off] [--edge-connectivity all|ipv4|ipv6] [--edge-tls off|flexible|full|strict] [--traffic-type direct/http/https] [-i, --instance INSTANCE] [--output FORMAT]
```

```
ibmcloud cis range-app-create DNS_DOMAIN_ID --name NAME --edge-port EDGE_PORT --origin-lb-name ORIGIN_LB_NAME --origin-lb-port ORIGIN_LB_PORT [--proxy-protocol on|off] [--ip-firewall on|off] [--edge-connectivity all|ipv4|ipv6] [--edge-tls off|flexible|full|strict] [--traffic-type direct/http/https] [-i, --instance INSTANCE] [--output FORMAT]
```

```
ibmcloud cis range-app-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

[Deprecated] `ibmcloud cis range-app-create DNS_DOMAIN_ID -s JSON_STR [-i, --instance INSTANCE] [--output FORMAT]`

[Deprecated] `ibmcloud cis range-app-create DNS_DOMAIN_ID -j JSON_FILE [-i, --instance INSTANCE] [--output FORMAT]`

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--name`

The name of the DNS record for the range application.

`--edge-port`

Port configuration at CIS's edge. The default value is `22`.

`--origin-direct`

Destination addresses to the origin.

`--origin-lb-name`

The Load Balancer name associated with the range application.

`--origin-lb-port`

The Load Balancer port associated with the range application. The default value is `22`.

`--protocol`

Protocol type. Valid values are `tcp` and `udp`. UDP protocol support is in early access, request custom UDP from CIS dashboard before creating range UDP app. The default value is `tcp`.

`--proxy-protocol`

Enable Proxy Protocol to the origin. Valid values are `on`, `off`, `v1`, `v2`, `simple`. The default value is `off`. *Deprecated*. The value `on` is equivalent to `v1`.

`--ip-firewall`

Control whether the IP Firewall for this application is enabled. Valid values are `on` and `off`. The default value is `off`.

`--edge-connectivity`

The IP versions supported for inbound connections on a range of anycast IPs. Valid values are `all`, `ipv4`, `ipv6`. The default value is `all`.

`--edge-tls`

The type of TLS termination associated with the application. Valid values are `off`, `flexible`, `full`, `strict`. The default value is `off`.

`--traffic-type`

Determines how data travels from the edge to your origin. Valid values are `direct`, `http`, `https`. The default value is `direct`.

`--json`

The JSON file or JSON string that is used to describe a range application.

- The required fields in JSON data are `protocol` and `dns`.
 - `protocol` : Port configuration at CIS's edge.
 - `dns` : The name and type of the DNS record for the range application.
 - `name` : The name of the DNS record for the range application.
 - `type` : The type of the DNS record associated with the application. Valid value is `CNAME`.
- The optional fields are `origin_direct`, `origin_dns`, `origin_port`, `proxy_protocol`, `ip_firewall`, `edge_ips`, `tls`, and `traffic_type`.
 - `origin_direct` : A list of destination addresses to the origin.
 - `origin_dns` : Method and parameters that are used to discover the origin server address via DNS.
 - `name` : DNS record name.
 - `origin_port` : The destination port at the origin.
 - `proxy_protocol` : Enable the Proxy Protocol to the origin. Valid values are `on`, `off`, `v1`, `v2`, `simple`. The default value is `off`. *Deprecated*. The value `on` is equivalent to `v1`.
 - `ip_firewall` : Control whether the IP Firewall for this application is enabled. Valid values are `on` and `off`.
 - `edge_ips` : The anycast edge IP configuration for the hostname of this application.
 - `type` : The type of edge IP configuration specified. Dynamically allocated edge IPs use range anycast IPs in accordance with the connectivity you specify. Valid value is `dynamic`.
 - `connectivity` : The IP versions supported for inbound connections on a range of anycast IPs. Valid values: `all`, `ipv4`, `ipv6`.
 - `tls` : The type of TLS termination associated with the application. Valid values are `off`, `flexible`, `full`, and `strict`.
 - `traffic_type` : Determines how data travels from the edge to your origin. When set to `direct`, range sends traffic directly to your origin, and the application's type is derived from the protocol. When set to `http` or `https`, range applies CIS's HTTP/HTTPS features as it sends traffic to your origin, and the application type matches this property exactly. Valid values are `direct`, `http`, and `https`. The default value is `direct`.

Sample JSON data:

```
{
  "protocol": "tcp/22",
  "dns": {
    "type": "CNAME",
    "name": "ssh.example.com"
  },
  "origin_direct": [
    "tcp://1.2.3.4:22",
    "tcp://1.2.3.4:23",
    "tcp://1.2.3.4:24"
  ],
  "proxy_protocol": false,
  "ip_firewall": false,
  "edge_ips": {
    "type": "dynamic",
    "connectivity": "all"
  },
  "tls": "full",
  "traffic_type": "direct"
}

{
  "protocol": "tcp/22",
  "dns": {
    "type": "CNAME",
    "name": "glb.example.com"
  },
  "origin_dns": {
    "name": "name-to-glb.example.com"
  },
  "origin_port": 22,
  "proxy_protocol": false,
  "ip_firewall": false,
  "edge_ips": {
    "type": "dynamic",
    "connectivity": "all"
  }
}
```

```
  },
  "tls": "full",
  "traffic_type": "direct"
}
```

`-s, --json-str`

Deprecated. The JSON data describing a range application.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a range app for the domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis range-app-create DNS_DOMAIN_ID --json '{"protocol":"tcp/22", "dns":{"type":"CNAME","name":"ssh.example.com"}, "origin_direct":["tcp://1.2.3.4:22"], "proxy_protocol":"off", "ip_firewall":true, "tls":"full", "edge_ips":{"type":"dynamic", "connectivity":"all"}, "traffic_type":"direct"}'
```

ibmcloud cis range-app-update

Enterprise Plans Only

Update a previously existing application's configuration.

```
$ ibmcloud cis range-app-update DNS_DOMAIN_ID APP_ID --origin-direct ORIGIN_DIRECT [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis range-app-update DNS_DOMAIN_ID APP_ID [--add-origin-direct ORIGIN_DIRECT] [--remove-origin-direct ORIGIN_DIRECT] [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis range-app-update DNS_DOMAIN_ID APP_ID [--origin-lb-name ORIGIN_LB_NAME] [--origin-lb-port ORIGIN_LB_PORT] [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis range-app-update DNS_DOMAIN_ID APP_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis range-app-update DNS_DOMAIN_ID APP_ID -s JSON_STR [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis range-app-update DNS_DOMAIN_ID APP_ID -j JSON_FILE [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`APP_ID`

The ID of the range application. Required.

`--name`

The name of the DNS record for the range application.

`--add-origin-direct`

Add new destination addresses to the origin.

`--remove-origin-direct`

Remove destination addresses from origin.

`--origin-direct`

Destination addresses to the origin.

`--origin-lb-name`

The Load Balancer name associated with the range application.

`--origin-lb-port`

The Load Balancer port associated with the range application. The default value is `22`.

`--proxy-protocol`

Enable the Proxy Protocol to the origin. Valid values are `on`, `off`, `v1`, `v2`, `simple`. The default value is `off`. *Deprecated*. The value `on` is equivalent to `v1`.

`--ip-firewall`

Control whether the IP Firewall for this application is enabled. Valid values are `on` and `off`. The default value is `off`.

`--edge-connectivity`

The IP versions supported for inbound connections on a range of anycast IPs. Valid values are `all`, `ipv4`, `ipv6`. The default value is `all`.

`--edge-tls`

The type of TLS termination associated with the application. Valid values are `off`, `flexible`, `full`, `strict`. The default value is `off`.

`--traffic-type`

Determines how data travels from the edge to your origin. Valid values are `direct`, `http`, `https`. The default value is `direct`.

`--json`

The JSON file or JSON string that is used to describe a range application.

- The required fields in JSON data are `protocol` and `dns`.
 - `protocol` : Port configuration at CIS's edge.
 - `dns` : The name and type of the DNS record for the range application.
 - `name` : The name of the DNS record for the range application.
 - `type` : The type of the DNS record associated with the application. Valid value is `CNAME`.
- The optional fields are `origin_direct`, `origin_dns`, `origin_port`, `proxy_protocol`, `ip_firewall`, `edge_ips`, `tls`, and `traffic_type`.
 - `origin_direct` : A list of destination addresses to the origin.
 - `origin_dns` : Method and parameters that are used to discover the origin server address via DNS.
 - `name` : DNS record name.
 - `origin_port` : The destination port at the origin.
 - `proxy_protocol` : Enable Proxy Protocol to the origin. Valid values are `on`, `off`, `v1`, `v2`, `simple`. The default value is `off`. *Deprecated*. The value `on` is equivalent to `v1`.
 - `ip_firewall` : Control whether the IP Firewall for this application is enabled. Valid values are `on` and `off`.
 - `edge_ips` : The anycast edge IP configuration for the hostname of this application.
 - `type` : The type of edge IP configuration specified. Dynamically allocated edge IPs use range anycast IPs in accordance with the connectivity you specify. Valid value is `dynamic`.
 - `connectivity` : The IP versions supported for inbound connections on range anycast IPs. Valid values are `all`, `ipv4`, `ipv6`.
 - `tls` : The type of TLS termination associated with the application. Valid values are `off`, `flexible`, `full`, `strict`.
 - `traffic_type` : Determines how data travels from the edge to your origin. When set to `direct`, range sends traffic directly to your origin, and the application's type is derived from the protocol. When set to `http` or `https`, range applies CIS's HTTP/HTTPS features as it sends traffic to your origin, and the application type matches this property exactly. Valid values are `direct`, `http`, and `https`. The default value is `direct`.

Sample JSON data:

```
{
  "protocol": "tcp/22",
  "dns": {
    "type": "CNAME",
    "name": "ssh.example.com"
  },
  "origin_direct": [
    "tcp://1.2.3.4:22",
    "tcp://1.2.3.4:23",
    "tcp://1.2.3.4:24"
  ],
  "proxy_protocol": false,
  "ip_firewall": false,
  "edge_ips": {
    "type": "dynamic",
    "connectivity": "all"
  }
}
```

```
    },
    "tls": "full",
    "traffic_type": "direct"
  }

  {
    "protocol": "tcp/22",
    "dns": {
      "type": "CNAME",
      "name": "glb.example.com"
    },
    "origin_dns": {
      "name": "name-to-glb.example.com"
    },
    "origin_port": 22,
    "proxy_protocol": false,
    "ip_firewall": false,
    "edge_ips": {
      "type": "dynamic",
      "connectivity": "all"
    },
    "tls": "full",
    "traffic_type": "direct"
  }
}
```

`-s, --json-str`

Deprecated. The JSON data describing a range application.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update range app `APP_ID`.

```
$ ibmcloud cis range-app-update DNS_DOMAIN_ID APP_ID --json '{"protocol":"tcp/22", "dns":{"type":"CNAME","name":"ssh.example.com"}, "origin_direct":
["tcp://1.2.3.4:22"], "proxy_protocol":"off", "ip_firewall":true, "tls":"full", "edge_ips":{"type":"dynamic", "connectivity":"all"}, "traffic_type":"direct"}
```

ibmcloud cis range-app-delete

Enterprise Plans Only

Delete a previously existing application.

```
$ ibmcloud cis range-app-delete DNS_DOMAIN_ID APP_ID [--instance INSTANCE]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`APP_ID`

The ID of range application. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete range application `APP_ID`.

```
$ ibmcloud cis range-app-delete DNS_DOMAIN_ID APP_ID
```

ibmcloud cis range-app

Enterprise Plans Only

Get the application configuration of a specific application.

```
$ ibmcloud cis range-app DNS_DOMAIN_ID APP_ID [--instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`APP_ID`

The ID of range application. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Show details of range application `APP_ID`.

```
$ ibmcloud cis range-app DNS_DOMAIN_ID APP_ID
```

ibmcloud cis range-apps

Enterprise Plans Only

Retrieve a list of currently existing range applications for a DNS domain.

```
$ ibmcloud cis range-apps DNS_DOMAIN_ID [--instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List all range applications in the domain.

```
$ ibmcloud cis range-apps DNS_DOMAIN_ID
```


ibmcloud cis range-analytics

Enterprise Plans Only

Get analytics data for range applications.

```
$ ibmcloud cis range-analytics DNS_DOMAIN_ID [--metrics METRICS] [--dimensions DIMENSION] [--filters FILTERS] [--sort SORT] [--since SINCE] [--until UNTIL]
ibmcloud cis range-analytics DNS_DOMAIN_ID --bytime [--time_delta DELTA] [--metrics METRICS] [--dimensions DIMENSION] [--filters FILTERS] [--sort SORT] [--since SINCE] [--until UNTIL]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--metrics

One or more metrics to compute. To get all metrics, set metrics to `count,bytesIngress,bytesEgress,durationAvg,durationMedian,duration90th,duration99th`.

--dimension

Can be used to break down the data by attributes. To get all dimensions, set dimensions to `event,appID,coloName,ipVersion`. `--filters`

Used to filter rows by one or more dimensions. Filters can be combined by using OR and AND Boolean logic. AND takes precedence over OR in all the expressions. The OR operator is defined by using a comma (,) or OR keyword that is surrounded by whitespace. The AND operator is defined by using a semicolon (;) or AND keyword that is surrounded by whitespace. Comparison options are: `==`, `!=`, `>`, `<`, `>=`, `<=`. An example value for filters is: `event==connect AND coloName!=SFO`.

--sort

The sort order for the result set. Sort fields must be included in metrics or dimensions. An example value for sort is: `+count,-bytesIngress`.

--since

Start of time interval to query, defaults to until - 6 hours. This value must be an absolute timestamp that conforms to RFC 3339.

--until

End of time interval to query, defaults to current time. This value must be an absolute timestamp that conforms to RFC 3339.

--bytime

Analytics data for range applications grouped by time interval.

--time-delta

Used to select time series resolution. Valid values are `year`, `quarter`, `month`, `week`, `day`, `hour`, `dekaminute`, `minute`. Only valid when `--bytime` is given.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Get analytics data for range applications in the domain.

```
$ ibmcloud cis range-analytics DNS_DOMAIN_ID --metrics "count,bytesIngress" --dimensions "event,appID" --since "2020-05-22T02:20:00Z" --until "2020-05-23T02:20:00Z"
```

Rate limiting

Manipulate rate limits by using the following `ratelimit` commands.

ibmcloud cis ratelimit-rule-create

Enterprise Plans Only

Create a new rate limiting rule for a DNS domain.

```
$ ibmcloud cis ratelimit-rule-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis ratelimit-rule-create DNS_DOMAIN_ID --url URL [--description DESCRIPTION] [--threshold NUM] [--period SECONDS] [...]
[Deprecated] ibmcloud cis ratelimit-rule-create DNS_DOMAIN_ID --json-str JSON_STR [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis ratelimit-rule-create DNS_DOMAIN_ID --json-file JSON_FILE [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--json

The JSON file or JSON string that is used to describe a rate limiting rule.

- The required fields in JSON data are **match**, **threshold**, **period** and **action** :
 - **match** : Determines which traffic the rate limiting rule counts toward the threshold.
 - **request** : Matches HTTP requests.
 - **methods** : HTTP Methods, can be a subset **[POST,PUT]** or all **[_ALL_]**. This field is not required to create a rate limit rule. Valid values are **GET**, **POST**, **PUT**, **DELETE**, **PATCH**, **HEAD**, **_ALL_**.
 - **schemes** : HTTP Schemes, can be one **[HTTPS]**, both **[HTTP, HTTPS]** or all **[_ALL_]**. This field is not required.
 - **url** : The URL pattern to match composed of the host and path, for instance, **example.org/path**. Wildcards are expanded to match applicable traffic, query strings are not matched. Use ***** for all traffic to your zone. The max length is **1024**.
 - **response** : Matches HTTP responses before they are returned to the client. If this is defined, then the entire counting of traffic occurs at this stage.
 - **status** : HTTP Status codes, can be one **[403]**, many **[401,403]** or indicate all by not providing this value. This field is not required. The min value is **100** and the max value is **999**.
 - **headers** : Array of response headers to match. If a response does not meet the header criteria, then the request is not counted towards the rate limiting rule. The header matching criteria includes the following properties.
 - **name** : The name of the response header to match.
 - **op** : The operator when matching, eq means equals, ne means not equals. Valid values are **eq** and **ne**.
 - **value** : The value of the header, which is exactly matched.
 - **threshold** : The threshold that triggers the rate limit mitigations, which are combined with a period. For example, the threshold per period. The min value is **2** and the max value is **1000000**.
 - **period** : The time, in seconds, to count matching traffic. If the count exceeds the threshold within this period, the action is performed. The min value is **10** and the max value is **86400**.
 - **action** : The action performed when the threshold of matched traffic within the period defined is exceeded.
 - **mode** : The type of action performed. Valid values are: **simulate**, **ban**, **challenge**, **js_challenge**.
 - **timeout** : The time in seconds, as an integer to perform the mitigation action. The timeout can be the same or greater than the period. This field is valid only when the mode is **simulate** or **ban**. The min value is **10** and the max value is **86400**.
 - **response** : Custom content-type and body to return. This overrides the custom error for the zone. This field is not required. Omission results in the default HTML error page. This field is valid only when mode is **simulate** or **ban**.
 - **content_type** : The content-type of the body, which must be one of the following: **text/plain**, **text/xml**, **application/json**.
 - **body** : The body to return. The content here must conform to the **content_type**. The max length is **10240**.
- The optional fields are **id**, **disabled**, **description**, **correlate** and **bypass** :
 - **id** : Identifier of the rate limiting rule.
 - **disabled** : Whether this rate limiting rule is currently disabled.
 - **description** : A note that you can use to describe the reason for a rate limiting rule.
 - **correlate** : Whether to enable NAT-based rate limiting.
 - **by** : Valid value is **nat**.
 - **bypass** : Criteria that allow the rate limit to be bypassed. For example, to express that you shouldn't apply a rate limit to a set of URLs.
 - **name** : Valid value is **url**.
 - **value** : The url to bypass.

Sample JSON data:

```
{
  "id": "92f17202ed8bd63d69a66b86a49a8f6b",
  "disabled": false,
  "description": "Prevent multiple login failures to mitigate brute force attacks",
  "bypass": [
    {
      "name": "url",
      "value": "api.example.com/*"
    }
  ],
  "threshold": 60,
  "period": 900,
  "correlate": {
    "by": "nat"
  },
  "action": [
    {
      "mode": "simulate",
      "timeout": 86400,
      "response": {
        "content_type": "text/plain",
        "body": "<error>This request has been rate-limited.</error>"
      }
    }
  ],
  "match": {
    "request": {
      "methods": [
        "GET"
      ],
      "schemes": [
        "HTTP",
        "HTTPS"
      ],
      "url": "*.example.org/path*"
    },
    "response": {
      "status": [
        403, 401
      ],
      "headers": [
        {
          "name": "Cf-Cache-Status",
          "op": "eq",
          "value": "HIT"
        }
      ]
    }
  }
}
```

`-s, --json-st`

Deprecated. The JSON data describing a rate limiting rule.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a rate limiting rule for domain.


```
$ ibmcloud cis ratelimit-rule-create DNS_DOMAIN_ID --json '{"id":"372e67954025e0ba6aaa6d586b9e0b59","disabled":false,"description":"Prevent multiple login failures to mitigate brute force attacks","match":{"request":{"methods":["GET","POST"],"schemes":["HTTP","HTTPS"],"url":"*.example.org/path*"},"response":{"status": [403, 401],"headers":[{"name":"Cf-Cache-Status","op":"ne","value":"HIT"}]}},'bypass":[{"name":"url","value":"api.example.com/*"}],"threshold":60,"period":900,"action":{"mode":"challenge","timeout":86400,"response":{"content_type":"text/xml","body":"<error>This request has been rate-limited.</error>"}}}'
```

ibmcloud cis ratelimit-rule-update

Update a rate limiting rule of a DNS domain.

```
$ ibmcloud cis ratelimit-rule-update DNS_DOMAIN_ID RATELIMIT_RULE_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis ratelimit-rule-update DNS_DOMAIN_ID RATELIMIT_RULE_ID [--url URL] [--description DESCRIPTION] [--threshold NUM] [--period SECONDS] [...]
[Deprecated] ibmcloud cis ratelimit-rule-update DNS_DOMAIN_ID RATELIMIT_RULE_ID --json-str JSON_STR [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis ratelimit-rule-update DNS_DOMAIN_ID RATELIMIT_RULE_ID --json-file JSON_FILE [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

RATELIMIT_RULE_ID

The ID of rate limiting rule. Required.

--json

The JSON file or JSON strinthat is used to describe a rate limiting rule.

- The required fields in JSON data are match , threshold , period and action :
 - match : Determines which traffic the rate limiting rule counts towards the threshold.
 - request : Matches HTTP requests.
 - methods : HTTP Methods, can be a subset [POST,PUT] or all [ALL] . This field is not required to create a rate limit rule. Valid values are GET , POST , PUT , DELETE , PATCH , HEAD , ALL .
 - schemes : HTTP Schemes, can be one [HTTPS] , both [HTTP,HTTPS] or all [_ALL_] . This field is not required.
 - url : The URL pattern to match consisted of the host and path, for instance, example.org/path . Wildcards are expanded to match applicable traffic, query strings are not matched. Use * for all traffic to your zone. The max length is 1024 .
 - response : Matches HTTP responses before they are returned to the client. If this field is defined, then the entire counting of traffic occurs at this stage.
 - status : HTTP Status codes, can be one [403] , many [401,403] or indicate all by not providing this value. This field is not required. The min value is 100 and the max value is 999 .
 - headers : Array of response headers to match. If a response does not meet the header criteria, then the request is not counted towards the rate limiting rule. An array of header matching criteria includes the following properties.
 - name : The name of the response header to match.
 - op : The operator when matching, eq means equals, ne means not equals. Valid values are eq and ne .
 - value : The value of the header, which is exactly matched.
 - threshold : The threshold that triggers the rate limit mitigations, which are combined with period. For example, the threshold per period. The min value is 2 and the max value is 1000000 .
 - period : The time, in seconds, to count matching traffic. If the count exceeds the threshold within this period the action is performed. The min value is 1 and the max value is 3600 .
 - action : The action is performed when the threshold of matched traffic within the defined period is exceeded.
 - mode : The type of action performed. Valid values are simulate , ban , challenge , js_challenge .
 - timeout : The time, in seconds, as an integer to perform the mitigation action. Timeout is the same or greater than the period. This field is valid only when mode is simulate or ban . The min value is 10 and the max value is 86400 .
 - response : Custom content-type and body to return. This overrides the custom error for the zone. This field is not required. Omission results in the default HTML error page. This field is valid only when mode is simulate or ban .
 - content_type : The content-type of the body, which must be one of the following: text/plain , text/xml , application/json .
 - body : The body to return. The content here must conform to the content_type . The max length is 10240 .
 - The optional fields are disabled , description , correlate and bypass :
 - disabled : Whether this rate limiting rule is currently disabled.
 - description : A note that you can use to describe the reason for a rate limiting rule.
 - correlate : Whether to enable NAT-based rate limiting.

- `by` : valid value is `nat` .
- `bypass` : Criteria that allow the rate limit to be bypassed. For example, to express that you shouldn't apply a rate limit to a set of URLs.
 - `name` : Valid value is `url` .
 - `value` : The url to bypass.

Sample JSON data:

```
{
  "disabled": false,
  "description": "Prevent multiple login failures to mitigate brute force attacks",
  "bypass": [
    {
      "name": "url",
      "value": "api.example.com/*"
    }
  ],
  "threshold": 60,
  "period": 900,
  "correlate": {
    "by": "nat"
  },
  "action": [
    {
      "mode": "simulate",
      "timeout": 86400,
      "response": {
        "content_type": "text/plain",
        "body": "<error>This request has been rate-limited.</error>"
      }
    }
  ],
  "match": {
    "request": {
      "methods": [
        "GET"
      ],
      "schemes": [
        "HTTP",
        "HTTPS"
      ],
      "url": "/*.example.org/path*"
    },
    "response": {
      "status": [
        403, 401
      ],
      "headers": [
        {
          "name": "Cf-Cache-Status",
          "op": "eq",
          "value": "HIT"
        }
      ]
    }
  }
}
```

`-s, --json-str`

Deprecated. The JSON data describing a rate limiting rule.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update rate limiting rule for domain.

```
$ ibmcloud cis ratelimit-rule-update DNS_DOMAIN_ID RATELIMIT_RULE_ID --json '{"disabled":false,"description":"Prevent multiple login failures to mitigate brute force attacks","match":{"request":{"methods":["GET","POST"],"schemes":["HTTP","HTTPS"],"url":"*.example.org/path*"},"response":{"status": [403, 401],"headers":{"name":"Cf-Cache-Status","op":"ne","value":"HIT"}}},"bypass":{"name":"url","value":"api.example.com/*"},"threshold":60,"period":900,"action":{"mode":"challenge","timeout":86400,"response":{"content_type":"text/xml","body":"<error>This request has been rate-limited.</error>"}}}'
```

ibmcloud cis ratelimit-rules

List rate limiting rules of a DNS domain.

```
$ ibmcloud cis ratelimit-rules DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List rate limiting rules in domain.

```
$ ibmcloud cis ratelimit-rules DNS_DOMAIN_ID
```

ibmcloud cis ratelimit-rule

Get details of a rate limiting rule by ID.

```
$ ibmcloud cis ratelimit-rule DNS_DOMAIN_ID RATELIMIT_RULE_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`RATELIMIT_RULE_ID`

The ID of rate limit rule. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get the details of rate limiting rule.

```
$ ibmcloud cis ratelimit-rule DNS_DOMAIN_ID RATELIMIT_RULE_ID
```

ibmcloud cis ratelimit-rule-delete

Delete a rate limiting rule by ID.

```
$ ibmcloud cis ratelimit-rule-delete DNS_DOMAIN_ID RATELIMIT_RULE_ID [--instance INSTANCE]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

RATELIMIT_RULE_ID

The ID of rate limit rule. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete rate limiting rule.

```
$ ibmcloud cis ratelimit-rule-delete DNS_DOMAIN_ID RATELIMIT_RULE_ID
```

Resource instance

Manipulate CIS Service instances by using the following `instance` commands.

ibmcloud cis instances

List all CIS service instances.

```
$ ibmcloud cis instances [--output FORMAT]
```

Command options

--output

The output format. Currently, `json` is the only supported value.

Examples

List all CIS instance in current account.

```
$ ibmcloud cis instances
```

ibmcloud cis instance-set

Set the context service instance to operate.

```
$ ibmcloud cis instance-set [INSTANCE_NAME] [--unset]
```

Command options

INSTANCE_NAME

The name of the CIS service instance. If it is presented, set the context instance to operate, if not, show the current context instance.

`--unset`

Unset context instance.

Examples

Set the context service instance to `cis-demo`

```
$ ibmcloud cis instance-set cis-demo
```

`ibmcloud cis instance-create`

Create a CIS service instance.

```
$ ibmcloud cis instance-create INSTANCE_NAME PLAN [--output FORMAT]
```

Command options

`INSTANCE_NAME`

The name of CIS service instance. Required.

`PLAN`

The name or ID of a service plan. Required.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a standard-next plan CIS instance `cis-demo`

```
$ ibmcloud cis instance-create cis-demo standard-next
```

`ibmcloud cis instance-delete`

Delete a CIS service instance.

```
$ ibmcloud cis instance-delete INSTANCE [-f, --force]
```

Command options

`INSTANCE`

The name or ID of a CIS service instance. Required.

`-f, --force`

Delete instance without prompting for confirmation.

Examples

Delete CIS instance `cis-demo`

```
$ ibmcloud cis instance-delete cis-demo --force
```

`ibmcloud cis instance-update`

Update a CIS service instance.

```
$ ibmcloud cis instance-update INSTANCE [--name NAME] [--plan PLAN] [--output FORMAT]
```


Command options

INSTANCE

The name or ID of a CIS service instance. Required.

NAME

The name of CIS service instance.

PLAN

The name or ID of a service plan.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update cis instance `cis-demo` to enterprise-usage plan.

```
$ ibmcloud cis instance-update cis-demo --plan enterprise-usage
```

ibmcloud cis instance

Show details of a CIS service instance.

```
$ ibmcloud cis instance INSTANCE [--output FORMAT]
```

Command options

INSTANCE

The name or ID of a CIS service instance. Required.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show details of cis instance `cis-demo`.

```
$ ibmcloud cis instance cis-demo
```

ibmcloud cis plans

List all CIS service plans.

```
$ ibmcloud cis plans [--refresh] [--output FORMAT]
```

Command options

--refresh

Force refresh from catalog.

--output

The output format. Currently, `json` is the only supported value.

Examples

List CIS services plans.

```
$ ibmcloud cis plans --refresh
```

Routing

Manipulate routing by using the following `routing` commands.

ibmcloud cis routing

Enterprise Plans Only

Get details of Routing settings.

```
$ ibmcloud cis routing DNS_DOMAIN_ID (--smart-routing | --tiered-caching) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--smart-routing`

Uses real-time network intelligence to route traffic across paths from the origin to a CIS data center.

`--tiered-caching`

Uses regional Tier 1 CIS data centers to accelerate content delivery.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get the details of routing settings for domain.

```
$ ibmcloud cis routing DNS_DOMAIN_ID --smart-routing
```

ibmcloud cis routing-update

Enterprise Plans Only

Update Routing setting.

```
$ ibmcloud cis routing-update DNS_DOMAIN_ID (--smart-routing (on|off) | --tiered-caching (on|off)) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--smart-routing`

Uses real-time network intelligence to route traffic across paths from the origin to a CIS data center. Valid values: `on`, `off`.

`--tiered-caching`

Uses regional Tier 1 CIS data centers to accelerate content delivery. Valid values are `on` and `off`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update routing settings for domain `372e67954025e0ba6aaa6d586b9e0b60`.

```
$ ibmcloud cis routing-update 31984fea73a15b45779fa0df4ef62f9b --smart-routing on --instance "cis-demo"
```

ibmcloud cis routing-analytics

Enterprise Plans Only

Get analytics of smart-routing latency.

```
$ ibmcloud cis routing-analytics DNS_DOMAIN_ID [--colos] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--colos

Analytics of smart-routing latency colos.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Get analytics of smart-routing latency for domain `372e67954025e0ba6aaa6d586b9e0b60`.

```
$ ibmcloud cis routing-analytics 31984fea73a15b45779fa0df4ef62f9b --instance "cis-demo"
```

Security events (Deprecated)

Security events are deprecated on 26 October 2020. Use [ibmcloud cis firewall-event-analytics](#) instead. Manage how the Security Events performs by using the following `security-events` command:

ibmcloud cis security-events

The `security-events` command is replacing the `firewall-events` command. It can pull up to 30 days of security events which might be triggered from a wider variety of sources (other than firewall) such as rate-limiting, L7 DDoS, and browser-integrity-check. With the new `security-events` command, you are able to list only firewall events by specifying the `--source` options.

Retrieve a full log of security events include Firewall Rules, Rate Limiting, Security Level, Access Rules, WAF, User Agent Blocking, Zone Lockdown, and Advanced DDoS Protection.

```
$ ibmcloud cis security-events DNS_DOMAIN_ID [--ip-class IP_CLASS] [--method METHOD] [--scheme SCHEME] [--ip IP_ADDR] [--host HOSTNAME] [--protocol PROTOCOL] [--uri URI] [--ua USER_AGENT] [--colo COLO] [--ray-id RAY_ID] [--kind KIND] [--action ACTION] [--cursor CURSOR] [--country COUNTRY] [--since START_DATE] [--until END_DATE] [--source SOURCE] [--limit LIMIT] [--rule_id RULE_ID] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--ip-class

IP class is a map of client IP to visitor classification. Valid values are `unknown`, `clean`, `badHost`, `searchEngine`, `whitelist`, `greylist`, `monitoringService`, `securityScanner`, `noRecord`, `scan`, `backupService`, `mobilePlatform`, `tor`.

`--method`

The HTTP method of the request. Valid values are `GET`, `POST`, `DELETE`, `PUT`, `HEAD`, `PURGE`, `OPTIONS`, `PROPFIND`, `MKCOL`, `PATCH`, `ACL`, `BCOPY`, `BDELETE`, `BMOVE`, `BPROPFIND`, `BPROPPATCH`, `CHECKIN`, `CHECKOUT`, `CONNECT`, `COPY`, `LABEL`, `LOCK`, `MERGE`, `MKACTIVITY`, `MKWORKSPACE`, `MOVE`, `NOTIFY`, `ORDERPATCH`, `POLL`, `PROPPATCH`, `REPORT`, `SEARCH`, `SUBSCRIBE`, `TRACE`, `UNCHECKOUT`, `UNLOCK`, `UNSUBSCRIBE`, `UPDATE`, `VERSION-CONTROL`, `BASELINE-CONTROL`, `X-MS-ENUMATTS`, `RPC_OUT_DATA`, `RPC_IN_DATA`, `JSON`, `COOK`, `TRACK`.

`--scheme`

The scheme of the URI. Valid values are `unknown`, `http` and `https`.

`--ip`

The IPv4 or IPv6 address from which the request originated.

`--host`

The hostname the request attempted to access.

`--protocol`

The protocol of the request. Valid values are `UNK`, `HTTP/1.0`, `HTTP/1.1`, `HTTP/1.2`, `HTTP/2` and `SPDY/3.1`.

`--uri`

The URI requested from the hostname.

`--ua`

The client user agent that initiated the request.

`--colo`

The 3-letter airport code of the Cloudflare data center that handled the request. For example, `SJC`.

`--ray-id`

Ray ID of the request.

`--action`

What type of action was taken. Valid values are `unknown`, `allow`, `drop`, `challenge`, `jschallenge`, `simulate`, `connectionClose` and `log`.

`--cursor`

Cursor position and direction for requesting the next set of records when the number of results are limited by the `limit` parameter. A valid value for the cursor can be obtained from the `cursors` object in the `result_info` structure.

`--country`

The 2-digit country code in which the request originated. For example, `US`.

`--since`

Start date and time of requesting data period in the ISO8601 format. Can't go back more than a year. For example, `2016-11-11T12:00:00Z`.

`--until`

End date and time of requesting data period in the ISO8601 format. For example, `2016-11-11T12:00:00Z`.

`--source`

Source of the event. Valid values are `unknown`, `asn`, `country`, `ip`, `ipRange`, `securityLevel`, `zoneLockdown`, `waf`, `uaBlock`, `rateLimit`, `firewallRules`, `bic`, `hot`, and `I7ddos`.

`--limit`

The number of events to return. The `cursor` attribute can be used to iterate over the next batch of events, if there are more events in the queried time range. Note that the `scanned_range` parameter in the `result_info` structure gives an indication of when events were considered in the current resultset if a limit was applied. Valid values are from 10 to 1000. Default value: 50.

`--rule-id`

The ID of the rule that triggered the event, which must be considered in the context of source.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get security events for domain `31984fea73a15b45779fa0df4ef62f9b` .

```
$ ibmcloud cis security-events 31984fea73a15b45779fa0df4ef62f9b --action challenge --colo SJC --country US --host "www.example.com" --ip-class clean
--method POST --ray-id 187d944c61940c77 --cursor
"6yDGxLKVeeHZZmORS_8XeSuhz9SjlJRaSa2lnsF01tQOHrfTGAP3R5X1Kv5iVUuMbNKhWNAXHOI6ePB0TUL8nw" -i "cis-demo"
```

TLS

Manipulate TLS by using the following `tls` commands.

ibmcloud cis tls-settings

Get TLS settings for a domain.

```
$ ibmcloud cis tls-settings DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get TLS settings for domain `372e67954025e0ba6aaa6d586b9e0b60` .

```
$ ibmcloud cis tls-settings 31984fea73a15b45779fa0df4ef62f9b --instance "cis-demo"
```

ibmcloud cis tls-settings-update

Update TLS settings for a DNS domain.

```
$ ibmcloud cis tls-settings-update DNS_DOMAIN_ID [--mode MODE] [--universal (true|false)] [--tls-1-2-only (on|off)] [--tls-1-3 (on|off)] [-i, --instance INSTANCE][--output
FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--mode`

Specify whether visitors can browse your website over a secure connection, and when they do, how CIS connects to your origin server. Valid values are `off` , `client-to-edge` , `end-to-end-flexible` , `end-to-end-ca-signed` , `https-only-origin-pull` . See the following documentation link for detailed [TLS mode description](#).

`--universal`

Specify whether universal ssl is enabled for your domain. Valid values are `true` and `false` .

`--tls-1-2-only`

Specify whether Crypto TLS 1.2 feature is enable for your domain. Enabling this feature prevents use of previous versions. Valid values are `on` and `off` .

`--tls-1-3`

Specify whether Crypto TLS 1.3 feature is enabled for your domain. Valid values are `on` , `off` .

`--min-tls-version`

Only accept HTTPS requests that use at least the TLS protocol version specified. Valid values are `1.0`, `1.1`, `1.2`, `1.3`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update TLS settings for the domain `372e67954025e0ba6aaa6d586b9e0b60`.

```
$ ibmcloud cis tls-settings-update 31984fea73a15b45779fa0df4ef62f9b --mode end-to-end-ca-signed --tls-1-2-only on --instance "cis-demo"
```

ibmcloud cis certificates

List all certificates for a DNS domain, including shared, dedicated, and custom certificates.

```
$ ibmcloud cis certificates DNS_DOMAIN_ID [--keyless] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--keyless`

List all keyless certificates.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List all certificates for domain `31984fea73a15b45779fa0df4ef62f9b`.

```
$ ibmcloud cis certificates 31984fea73a15b45779fa0df4ef62f9b --instance "cis-demo"
```

ibmcloud cis certificate

Get the details of a shared, dedicated, or custom certificate.

```
$ ibmcloud cis certificate DNS_DOMAIN_ID (--cert-id CERT_ID | --universal) [--keyless] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--cert-id`

ID of the dedicated or custom certificate.

`--universal`

Show universal certificate details.

`--keyless`

Show keyless certificate details.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Show details for a certificate.

```
ibmcloud cis certificate 31984fea73a15b45779fa0df4ef62f9b --universal --instance "cis-demo"
ibmcloud cis certificate 31984fea73a15b45779fa0df4ef62f9b --cert-id 5a7805061c76ada191ed06f989cc3dac --instance "cis-demo"
ibmcloud cis certificate 31984fea73a15b45779fa0df4ef62f9b --cert-id 5a7805061c76ada191ed06f989cc3dac --keyless --instance "cis-demo"
```

ibmcloud cis certificate-order

Order a certificate pack with an optional list of hostnames for a DNS domain.

```
$ ibmcloud cis certificate-order DNS_DOMAIN_ID [--hostnames host1 --hostnames host2 ...] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--hostnames`

Valid host names for the certificate packs. Add up to 50 custom hostnames - Can affect the price.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Order a certificate pack for domain `31984fea73a15b45779fa0df4ef62f9b`.

```
$ ibmcloud cis certificate-order 31984fea73a15b45779fa0df4ef62f9b --hostnames www.example.com --hostnames api.example.com --instance "cis-demo"
```

ibmcloud cis certificate-upload

Upload a custom certificate for a DNS domain.

```
$ ibmcloud cis certificate-upload DNS_DOMAIN_ID [--keyless] (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis certificate-upload DNS_DOMAIN_ID [--keyless] (-s, --json-str JSON_STR | -, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--keyless`

Upload a keyless certificate.

`--json`

The JSON file or JSON string that is used to describe a custom certificate. Required.

- The required fields in JSON data are `certificate`, `private_key`, `host` and `port` :
 - `certificate` : SSL certificate or certificate and one or more intermediates for the domain

- `certificate` : SSL certificate or certificate and one or more intermediates for the domain.
- `private_key` : Private key for the domain.
- `host` : The keyless SSL host name.
- `port` : The keyless SSL port used to communicate between CIS and the client's Keyless SSL server.
- The optional fields are `bundle_method` and `name` :
 - `bundle_method` : Bundle method, default value is `compatible`, valid values are `compatible`, `modern` and `user-defined`.
 - `name` : The keyless SSL name.

Sample JSON data:

```
{
  "certificate": "xxx",
  "private_key": "xxx",
  "bundle_method": "compatible"
}

For keyless ssl
{
  "host": "www.example.com",
  "port": 8000,
  "certificate": "xxx",
  "bundle_method": "user-defined",
  "name": "test"
}
```

`-s, --json-str`

Deprecated. The JSON data used to upload a custom certificate.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Upload a custom certificate for domain `31984fea73a15b45779fa0df4ef62f9b`.

```
$ ibmcloud cis certificate-upload 31984fea73a15b45779fa0df4ef62f9b --json '{"certificate": "-----BEGIN CERTIFICATE-----
\nMIIDtTCCAp2gAwIBAgIJAMHAWfXZ5/PWMA0GCSqGSIb3DQEBCwUAMEUxCzAJBgNV\nBAYTAkFVMRMwEQYDVQQIEwpTb21lLVN0YXRIMSEwHwYDVQQKEzhJbnRlcm5ldCBX\
-----END CERTIFICATE-----\n", "private_key": "-----BEGIN RSA PRIVATE KEY-----
\nMIIIEowIBAAKCAQEAwQHoetcl9+5ikGzV6cMzWtWPJHqXT3wpbEkRU9Yz7lgvddmG\ndtcGbg/1CGZu0jJGkMoppoUo4c3dts3iwqRYmBikUP77wwY2QGmDZw2FvkJCJIKn\nabIRuGvE
-----END RSA PRIVATE KEY-----\n", "bundle_method": "compatible"}' -i "cis-demo"
```

ibmcloud cis certificate-update

Update a custom certificate for a DNS domain.

```
$ ibmcloud cis certificate-update DNS_DOMAIN_ID CERT_ID [--keyless] (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis certificate-update DNS_DOMAIN_ID CERT_ID [--keyless] [-s, --json-str JSON_STR | -j, --json-file JSON_FILE] [-i, --instance INSTANCE] [--
output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

CERT_ID

The ID of the custom certificate. Required.

--keyless

Update a keyless certificate.

--json

The JSON file or JSON string that is used to describe a custom certificate. Required.

- The required fields in JSON data are `certificate`, `private_key`, `host` and `port` :
 - `certificate` : SSL certificate or certificate and one or more intermediates for the domain.
 - `private_key` : Private key for the domain.
 - `host` : The keyless SSL host name.
 - `port` : The keyless SSL port used to communicate between CIS and the client's Keyless SSL server.
- The optional fields are `bundle_method` :
 - `bundle_method` : Bundle method, default value is `compatible`, valid values are `compatible`, `modern` and `user-defined`.
 - `name` : The keyless SSL name.

Sample JSON data:

```
{
  "certificate": "xxx",
  "private_key": "xxx",
  "bundle_method": "compatible"
}

For keyless ssl
{
  "host": "www.example.com",
  "port": 8000,
  "certificate": "xxx",
  "bundle_method": "user-defined",
  "name": "test"
}
```

-s, --json-str

Deprecated. The JSON data used to update a custom certificate.

-j, --json-file

Deprecated. A file contains input JSON data.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update the custom certificate `5a7805061c76ada191ed06f989cc3dac`.

```
$ ibmcloud cis certificate-update 31984fea73a15b45779fa0df4ef62f9b 5a7805061c76ada191ed06f989cc3dac --json '{"certificate": "-----BEGIN CERTIFICATE-----
\nMIIDTCCAp2gAwIBAgIJAMHAWfXZ5/PWMA0GCSqGSIb3DQEBCwUAMEUxCzAJBgNV\nBAYTAkFVMRMwEQYDVQQIEwpTb21lLVN0YXRIMSEwHwYDVQQKEzhJbnRlcm5ldCBX\
-----END CERTIFICATE-----\n", "private_key": "-----BEGIN RSA PRIVATE KEY-----
\nMIIIEowIBAAKCAQEAwQHoetcl9+5ikGzV6cMzWtWPJHqXT3wpbEkRU9Yz7lgvddmG\ndtcGbg/1CGZu0jJGkMoppoUo4c3dts3iwqRYmBikUP77wwY2QGmDZw2FvkJCJlKn\nabIRuGvE
-----END RSA PRIVATE KEY-----\n", "bundle_method": "compatible"}' -i "cis-demo"
```

ibmcloud cis certificate-priority-change

Change custom certificates' priority for a DNS domain.

```
$ ibmcloud cis certificate-priority-change DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis certificate-priority-change DNS_DOMAIN_ID (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output
FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--json

The JSON file or JSON string that is used to describe the custom certificates' priority. Required.

- The required fields in JSON data are **certificates** :
 - **certificates** : An array of objects with the following fields.
 - **id** : Custom certificate identifier.
 - **priority** : The order or priority in which the certificate is used in a request. Higher numbers are tried first.

Sample JSON data:

```
{
"certificates":[
  {
    "id":"5a7805061c76ada191ed06f989cc3dac",
    "priority":2
  },
  {
    "id":"da534493b38266b17fea74f3312be21c",
    "priority":1
  }
]
}
```

-s, --json-str

Deprecated. The JSON data used to change the custom certificates' priority.

-j, --json-file

Deprecated. A file contains input JSON data.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

Change custom certificates' priority for domain.

```
$ ibmcloud cis certificate-priority-change DNS_DOMAIN_ID --json '{"certificates": [{"id":"5a7805061c76ada191ed06f989cc3dac", "priority":2}, {"id":"9a7806061c88ada191ed06f989cc3dac", "priority":1}]}'
```

ibmcloud cis certificate-delete

Delete a dedicated or custom certificate.

```
$ ibmcloud cis certificate-delete DNS_DOMAIN_ID CERT_ID [--keyless][-i, --instance INSTANCE]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

CERT_ID

The ID of the dedicated or custom certificate. Required.

--keyless

Delete a keyless certificate.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete the custom certificate.

```
$ ibmcloud cis certificate-delete DNS_DOMAIN_ID CERT_ID
```

Web application firewall (WAF)

Manage Web Application Firewalls by using the following `waf` commands.

ibmcloud cis waf-setting

Show WAF setting.

```
$ ibmcloud cis waf-setting DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show WAF settings for domain.

```
$ ibmcloud cis waf-setting DNS_DOMAIN_ID
```

ibmcloud cis waf-setting-update

Update the WAF setting.

```
$ ibmcloud cis waf-setting-update DNS_DOMAIN_ID WAF_MODE [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

WAF_MODE

The mode of WAF setting. Valid values are `waf-enable` and `waf-disable`. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Enable WAF for domain.

```
$ ibmcloud cis waf-setting-update DNS_DOMAIN_ID waf-enable
```

ibmcloud cis waf-packages

List all WAF packages.

```
$ ibmcloud cis waf-packages DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List all WAF packages for domain.

```
$ ibmcloud cis waf-packages DNS_DOMAIN_ID
```

ibmcloud cis waf-package

Get detail of a WAF package.

```
$ ibmcloud cis waf-package DNS_DOMAIN_ID WAF_PACKAGE_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`WAF_PACKAGE_ID`

The ID of the WAF package. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get detail of a WAF package.

```
$ ibmcloud cis waf-package DNS_DOMAIN_ID WAF_PACKAGE_ID
```

ibmcloud cis waf-package-set

Update OWASP Package setting.

```
$ ibmcloud cis waf-package-set DNS_DOMAIN_ID OWASP_PACKAGE_ID [--sensitivity SENSITIVITY] [--action-mode ACTION_MODE] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

OWASP_PACKAGE_ID

The ID of the WAF package. Required.

--sensitivity

The sensitivity of the firewall package. Valid values are **high**, **medium**, **low**, and **off**.

--action-mode

The default action that is taken for rules under the firewall package. Valid values are **simulate** and **block**, **challenge**.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

Update the OWASP setting for package.

```
$ ibmcloud cis waf-package-set DNS_DOMAIN_ID OWASP_PACKAGE_ID --sensitivity medium --action-mode simulate
```

ibmcloud cis waf-groups

List the WAF groups in a WAF package.

```
$ ibmcloud cis waf-groups DNS_DOMAIN_ID WAF_PACKAGE_ID [--page PAGE] [--per-page NUM] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

WAF_PACKAGE_ID

The ID of the WAF package. Required.

--page

The page number of paginated results. The default value is **1**.

--per-page

The number of groups per page. The min value is **5** and the max value is **1000**. The default value is **50**.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

List the WAF groups in a WAF package.

```
$ ibmcloud cis waf-groups DNS_DOMAIN_ID WAF_PACKAGE_ID --page 1 --per-page 100
```

ibmcloud cis waf-group

Get detail of a WAF group.

```
$ ibmcloud cis waf-group DNS_DOMAIN_ID WAF_PACKAGE_ID WAF_GROUP_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

WAF_PACKAGE_ID

The ID of the WAF package. Required.

WAF_GROUP_ID

The ID of the WAF group.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Get details of WAF group in WAF package.

```
$ ibmcloud cis waf-group DNS_DOMAIN_ID WAF_PACKAGE_ID WAF_GROUP_ID
```

ibmcloud cis waf-group-mode-set

Set the mode of a WAF group.

```
$ ibmcloud cis waf-group-mode-set DNS_DOMAIN_ID WAF_PACKAGE_ID WAF_GROUP_ID WAF_GROUP_MODE [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

WAF_PACKAGE_ID

The ID of the WAF package. Required.

WAF_GROUP_ID

The ID of the WAF group. Required.

WAF_GROUP_MODE

The mode of WAF group. Valid values are `on` and `off`. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Enable the WAF group in the WAF package.

```
$ ibmcloud cis waf-group-mode-set DNS_DOMAIN_ID WAF_PACKAGE_ID WAF_GROUP_ID on
```

ibmcloud cis waf-rules

List all WAF rules of a WAF package.

```
$ ibmcloud cis waf-rules DNS_DOMAIN_ID WAF_PACKAGE_ID [--page PAGE] [--per-page NUM] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

WAF_PACKAGE_ID

The ID of the WAF package. Required.

--page

Page number of the paginated results. The default value is **1**.

--per-page

Number of rules per page. The default value is **50**.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

List all WAF rules in the WAF package.

```
$ ibmcloud cis waf-rules DNS_DOMAIN_ID WAF_PACKAGE_ID --page 1 --per-page 100
```

ibmcloud cis waf-rule

Get detail of a WAF rule.

```
$ ibmcloud cis waf-rule DNS_DOMAIN_ID WAF_PACKAGE_ID WAF_RULE_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

WAF_PACKAGE_ID

The ID of WAF package. Required.

WAF_RULE_ID

The ID of WAF rule. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

Get details of WAF rule in WAF package.

```
$ ibmcloud cis waf-rule DNS_DOMAIN_ID WAF_PACKAGE_ID WAF_RULE_ID
```

ibmcloud cis waf-rule-mode-set

Set the mode of a WAF rule.

```
$ ibmcloud cis waf-rule-mode-set DNS_DOMAIN_ID WAF_PACKAGE_ID WAF_RULE_ID WAF_RULE_MODE [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

WAF_PACKAGE_ID

The ID of the WAF package. Required.

WAF_RULE_ID

The ID of the WAF rule. Required.

WAF_RULE_MODE

The mode of WAF rule. Valid values are `on` , `off` , `default` , `disable` , `simulate` , `block` , and `challenge` . Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Disable WAF rule in WAF package.

```
$ ibmcloud cis waf-rule-mode-set DNS_DOMAIN_ID WAF_PACKAGE_ID WAF_RULE_ID disable
```

ibmcloud cis cis waf-override-create

Create a URL-based Web Application Firewall (WAF) rule.

```
$ ibmcloud cis waf-override-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--json

The JSON file or JSON string that is used to describe an override WAF rule. Required.

- The required fields in JSON data are `urls` and `rules` .
 - `urls` : URLs to be included in this rule definition. Wildcards are permitted.
 - `rules` : Change the action that is assigned to a WAF rule. The keys of this object are WAF rule IDs and the values must be a valid WAF action. Unless disabling the rule, ensure that you also enable the rule group that this WAF rule belongs to. The max length is `1024` .
- The optional fields are `paused` , `description` , `priority` , `groups` , and `rewrite_action` .
 - `paused` : Whether this package is currently paused. Valid values are `true` and `false` .
 - `description` : A note that you can use to describe the purpose of this rule.
 - `priority` : Relative priority of this configuration when multiple configurations match a single URL. Higher priority configurations

might overwrite values set by lower priority configurations. The min value is `-1000000000` and the max value is `1000000000` .

- `groups` Enable or disable WAF rule groups. The keys of this object are WAF rule group IDs and the values must be a valid WAF action (usually `default` or `disable`).
- `rewrite_action` : When a WAF rule matches, substitute its configured action for a different action that is specified by this object.

Sample JSON data:

```
{
  "description": "Enable IBM Magento ruleset for www.example.com",
  "urls": [
    "www.example.com/*"
  ],
  "priority": 1,
  "groups": {
    "ea8687e59929c1fd05ba97574ad43f77": "default"
  },
  "rules": {
    "100015": "disable"
  },
  "rewrite_action": {
    "default": "block",
    "challenge": "block",
    "simulate": "disable"
  }
}
```

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a WAF override rule.

```
$ ibmcloud cis waf-override-create DNS_DOMAIN_ID --json '{"description":"Enable IBM Magento ruleset for www.example.com","urls":["www.example.com/*"],"priority":1,"groups":{"ea8687e59929c1fd05ba97574ad43f77":"default"},"rules":{"100015":"disable"},"rewrite_action":{"default":"block","challenge":"block","simulate":"disable"}}'
```

ibmcloud cis cis waf-override-update

Update a URL-based Web Application Firewall (WAF) rules.

```
$ ibmcloud cis waf-override-update DNS_DOMAIN_ID OVERRIDE_WAF_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`OVERRIDE_WAF_ID`

The ID of the override WAF rule. Required.

`--json`

The JSON file or JSON string that is used to describe a override WAF rule. Required.

- The required fields in JSON data are `urls` and `rules` .
 - `urls` : URLs to be included in this rule definition. Wildcards are permitted.
 - `rules` : Change the action that is assigned to a WAF rule. The keys of this object are WAF rule IDs and the values must be a valid

- WAF action. Unless you disable the rule, ensure that you also enable the rule group that this WAF rule belongs to. The max length is `1024` .
- The optional fields are `paused` , `description` , `priority` , `groups` , and `rewrite_action` .
 - `paused` : Whether this package is currently paused. Valid values are `true` and `false` .
 - `description` : A note that you can use to describe the purpose of this rule.
 - `priority` : Relative priority of this configuration when multiple configurations match a single URL. Higher priority configurations might overwrite values set by lower priority configurations. The min value is `-1000000000` and the max value is `1000000000` .
 - `groups` Enable or disable WAF rule groups. The keys of this object are WAF rule group IDs and the values must be a valid WAF action (usually `default` or `disable`).
 - `rewrite_action` : When a WAF rule matches, substitute its configured action for a different action that is specified by this object.

Sample JSON data:

```
{
  "description": "Enable IBM Magento ruleset for www.example.com",
  "urls": [
    "www.example.com/*"
  ],
  "priority": 1,
  "groups": {
    "ea8687e59929c1fd05ba97574ad43f77": "default"
  },
  "rules": {
    "100015": "disable"
  },
  "rewrite_action": {
    "default": "block",
    "challenge": "block",
    "simulate": "disable"
  }
}
```

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update a WAF override rule.

```
$ ibmcloud cis waf-override-update DNS_DOMAIN_ID OVERRIDE_WAF_ID --json '{"description":"Enable IBM Magento ruleset for www.example.com","urls":["www.example.com/*"],"priority":1,"groups":{"ea8687e59929c1fd05ba97574ad43f77":"default"},"rules":{"100015":"disable"},"rewrite_action":{"default":"block","challenge":"block","simulate":"disable"}}'
```

ibmcloud cis waf-overrides

List all URL-based Web Application Firewall (WAF) rules.

```
$ ibmcloud cis waf-overrides DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--page`

Page number of paginated results. The default value is `1` .

`--per-page`

Number of rules per page. The default value is `50`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List WAF override rules.

```
$ ibmcloud cis waf-overrides DNS_DOMAIN_ID
```

ibmcloud cis cis waf-override

Get a URL-based Web Application Firewall (WAF) rule.

```
$ ibmcloud cis waf-override DNS_DOMAIN_ID OVERRIDE_WAF_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`OVERRIDE_WAF_ID`

The ID of override WAF rule. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get a WAF override rule.

```
$ ibmcloud cis waf-override DNS_DOMAIN_ID OVERRIDE_WAF_ID
```

ibmcloud cis cis waf-override-delete

Delete a URL-based Web Application Firewall (WAF) rule.

```
$ ibmcloud cis waf-override-delete DNS_DOMAIN_ID OVERRIDE_WAF_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`OVERRIDE_WAF_ID`

The ID of override WAF rule. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`-f, --force`

Attempt to delete the URL-based WAF rule without prompting for confirmation.

Examples

Delete a WAF override rule.

```
$ ibmcloud cis waf-override-delete DNS_DOMAIN_ID OVERRIDE_WAF_ID
```

Authenticated Origin Pull

Manage Authenticated Origin Pull by using the following `authenticated-origin-pull` commands.

`ibmcloud cis authenticated-origin-pull-settings`

Get authenticated origin pull settings for a domain.

```
$ ibmcloud cis authenticated-origin-pull-settings DNS_DOMAIN_ID [--level zone|hostname] [--hostname HOSTNAME] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`----level`

Specify the authenticated origin pull certificate or settings per zone or hostname level. Valid values are `zone` and `hostname`. The default value is `zone`.

`--hostname`

The authenticated origin pull settings on a hostname. (hostname level only)

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List authenticated origin pull settings on zone level for domain.

```
$ ibmcloud cis authenticated-origin-pull-settings DNS_DOMAIN_ID
```

`ibmcloud cis authenticated-origin-pull-setting-update`

Update authenticated origin pull settings for a domain.

```
$ ibmcloud cis authenticated-origin-pull-settings-update DNS_DOMAIN_ID [--level zone|hostname] [--hostname HOSTNAME] [--cert_id CERT_ID] [--enabled on|off] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`----level`

Specify the authenticated origin pull certificate or settings per zone or hostname level. Valid values are `zone` and `hostname`. The default value is `zone`.

`--hostname`

The authenticated origin pull settings on a hostname. (hostname level only)

----cert_id

The certificate id, which the hostname is bundled to. (hostname level only)

----enabled

Enable authenticated origin pull. Valid values are `on` and `off`.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update authenticated origin pull setting on zone level for domain.

```
$ ibmcloud cis authenticated-origin-pull-settings-update DNS_DOMAIN_ID --enabled on
```

ibmcloud cis authenticated-origin-pull-certificates

List zone-level authenticated origin pull certificates for a domain.

```
$ ibmcloud cis authenticated-origin-pull-certificates DNS_DOMAIN_ID [--level zone|hostname][-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

----level

Specify the authenticated origin pull certificate or settings per zone or hostname level. Valid values are `zone` and `hostname`. The default value is `zone`.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show authenticated origin pull certificates on zone level for domain.

```
$ ibmcloud cis authenticated-origin-pull-certificates DNS_DOMAIN_ID
```

ibmcloud cis authenticated-origin-pull-certificate

Get an authenticated origin pull certificate for a domain.

```
$ ibmcloud cis authenticated-origin-pull-certificate DNS_DOMAIN_ID CERT_ID [--level zone|hostname] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

CERT_ID

The ID of the certificate. Required.

----level

Specify the authenticated origin pull certificate or settings per zone or hostname level. Valid values are `zone` and `hostname`. The default value is `zone`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get an authenticated origin pull certificate on zone level for domain.

```
$ ibmcloud cis authenticated-origin-pull-certificate DNS_DOMAIN_ID CERT_ID
```

ibmcloud cis authenticated-origin-pull-certificate-upload

Upload an authenticated origin pull certificate for a domain.

```
$ ibmcloud cis authenticated-origin-pull-certificate-upload DNS_DOMAIN_ID [--level zone|hostname] (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [-output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--level`

Specify the authenticated origin pull certificate or settings per zone or hostname level. Valid values are `zone` and `hostname`. The default value is `zone`.

`--json`

The JSON file or JSON string that is used to describe a custom certificate.

- The required fields in JSON data are `certificate` and `private_key` :
 - `certificate` : SSL certificate or certificate and one or more intermediates for the domain.
 - `private_key` : Private key for the domain.

Sample JSON data:

```
{
  "certificate": "-----BEGIN CERTIFICATE-----\n...-----END CERTIFICATE-----\n",
  "private_key": "-----BEGIN PRIVATE KEY-----\n...-----END PRIVATE KEY-----\n"
}
```

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Upload an authenticated origin pull certificate on zone level for domain.

```
$ ibmcloud cis authenticated-origin-pull-certificate-upload DNS_DOMAIN_ID --json '{"certificate": "-----BEGIN CERTIFICATE-----\nMIIDtTCCAp2gAwIBAgIJAMHAWfXZ5/PWMA0GCSqGSIb3DQEBCwUAMEUxCzAJBgNV\nBAYTAkFVMRMwEQYDVQQIEwpTb211LVN0YXRIMSEwHwYDVQQKEjhbnRlcm5ldCBX\n\\
```

```
-----END CERTIFICATE-----\n", "private_key": "-----BEGIN RSA PRIVATE KEY-----\nMIIIEowIBAAKCAQEAwQHoetcI9+5ikGzV6cMzWtWPJHqXT3wpbEkRU9Yz7IgvddmG\ndtcGbg/1CGZu0jJGkMoppoUo4c3dts3iwqRYmBikUP77wwY2QGmDZw2FvkJCJlKn\nabiRuGvE-----END RSA PRIVATE KEY-----\n"}'
```

ibmcloud cis authenticated-origin-pull-certificate-delete

Delete the authenticated origin pull certificate for a domain.

```
$ ibmcloud cis authenticated-origin-pull-certificate-delete DNS_DOMAIN_ID CERT_ID [--level zone|hostname] [-i, --instance INSTANCE] [--output FORMAT] [-f, --force]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

CERT_ID

The ID of certificate. Required.

--level

Specify the authenticated origin pull certificate or settings per zone or hostname level. Valid values are **zone** and **hostname**. The default value is **zone**.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

Delete authenticated origin pull certificate on zone level for domain.

```
$ ibmcloud cis authenticated-origin-pull-certificate-delete DNS_DOMAIN_ID CERT_ID
```

Alert policy

Manage alert policies.

ibmcloud cis alert-policy list (List)

List all alert policies.

```
$ ibmcloud cis alert-policy list [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

List all policies for instance.

```
$ ibmcloud cis alert-policy list
```

ibmcloud cis alert-policy get (Show)

Show the details of a policy.

```
$ ibmcloud cis alert-policy get POLICY_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`POLICY_ID`

The ID of alert policy. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Show the details of the alert policy.

```
$ ibmcloud cis alert-policy get POLICY_ID
```

ibmcloud cis alert-policy ddos-attack-l7-alert-create

Create an alert policy for DDoS attack l7.

```
$ ibmcloud cis alert-policy ddos-attack-l7-alert-create --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

`--webhooks`

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

`--enabled`

Whether the alert policy is enabled.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a DDoS attack alert policy.

```
$ ibmcloud cis alert-policy ddos-attack-l7-alert-create --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --enabled true --instance INSTANCE
```

ibmcloud cis alert-policy ddos-attack-l3-l4-alert-create

Create an alert policy for DDoS attack L3/L4.

```
$ ibmcloud cis alert-policy ddos-attack-l3-l4-alert-create --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

--name

The name of the alert policy.

--description

The description for the alert policy.

--emails

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

--webhooks

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

--enabled

Whether the alert policy is enabled.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Create a DDoS L3/L4 attack alert policy.

```
$ ibmcloud cis alert-policy ddos-attack-l3-l4-alert-create --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --enabled true --instance INSTANCE
```

ibmcloud cis alert-policy failing-logpush-job-alert-create

Create an alert policy when logpush job did not complete at least one successful push in the last 24 hours.

```
$ ibmcloud cis alert-policy failing-logpush-job-alert-create --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

--name

The name of the alert policy.

--description

The description for the alert policy.

--emails

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

--webhooks

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

--enabled

Whether the alert policy is enabled.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Create a failing logpush job disabled alert policy.

```
$ ibmcloud cis alert-policy failing-logpush-job-alert-create --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --enabled true --instance INSTANCE
```

ibmcloud cis alert-policy pool-toggle-alert-create (Pool toggle alert)

Create an alert policy for pool toggle alert.

```
$ ibmcloud cis alert-policy pool-toggle-alert-create --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) --pools POOLS --trigger-condition (enabled | disabled | either) [--include-future-pools (true | false)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

```
--name
```

The name of the alert policy.

```
--description
```

The description for the alert policy.

```
--emails
```

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

```
--webhooks
```

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

```
--enabled
```

Whether the alert policy is enabled.

```
--pools
```

The IDs of the origin pool, if set to all, the all pool IDs are used.

```
--trigger-condition
```

The condition of the pool toggle status.

```
--include-future-pools
```

Whether to include the future pools.

```
-i, --instance
```

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

```
--output
```

The output format. Currently, `json` is the only supported value.

Examples

Create a pool toggle alert policy.

```
$ ibmcloud cis alert-policy pool-toggle-alert-create --name test1 --emails test1@cn.ibm.com --enabled true --pools all --trigger-condition enabled --include-future-pools true --instance INSTANCE
```

ibmcloud cis alert-policy firewall-events-alert-create

Create an alert policy about spikes in firewall events. Firewall events alerts use a [z-score] (https://en.m.wikipedia.org/wiki/Standard_score): external} calculation over the last six hours and five-minute buckets of events. An alert is triggered whenever the z-score is greater than the threshold of 3.5. You will not receive duplicate alerts within the same two-hour time frame.

```
$ ibmcloud cis alert-policy firewall-events-alert-create --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) --domains DOMAINS [--
```

```
services SERVICES] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

`--webhooks`

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

`--enabled`

Whether the alert policy is enabled.

`--domains`

The domain IDs that for the alert policy. For example, `--domains domainID1,domainID2`

`--services`

Specify which services the alert should monitor. Valid values are `country-access-rules`, `waf`, `firewall-rules`, `ratelimit`, `securitylevel`, `ip-access-rules`, `browser-integrity-check`, `ua-rules`, `lockdowns`, `iprange-access-rules`, `asn-access-rules`, `Managed-firewall` [Enterprise Plans Only](#)

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a firewall-events alert.

```
$ ibmcloud cis alert-policy firewall-events-alert-create --name test1 --emails test1@cn.ibm.com --enabled true --domains d2633e61-1b61-2512-1321-b61a17c3db7e --services waf,ratelimit --instance INSTANCE
```

ibmcloud cis alert-policy certificate-alert-create

Create an alert policy for certificate events.

```
$ ibmcloud cis alert-policy certificate-alert-create --type (universal | dedicated | mtls ) --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`--type`

The type of the certificate.

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

`--webhooks`

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

`--enabled`

Whether the alert policy is enabled.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a certificate alert.

```
$ ibmcloud cis alert-policy certificate-alert-create --type universal --name test1 --emails test1@cn.ibm.com --enabled true --instance INSTANCE
```

ibmcloud cis alert-policy glb-healthcheck-alert-create

Create an alert policy for changes in health status for global load balancer, pools, and origins.

```
$ ibmcloud cis alert-policy glb-healthcheck-alert-create --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) --pools POOLS [--include-future-pools (true | false)] [--health-status-trigger (healthy | unhealthy | either)] [--event-source-trigger (pool | origin | either)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

`--webhooks`

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

`--enabled`

Whether the alert policy is enabled.

`--pools`

The IDs of origin pool. If set to `all`, all pool IDs are used.

`--include-future-pools`

Whether to include the future pools. (The default value is `false`)

`--health-status-trigger`

The trigger condition to fire the notification. Valid values are `healthy`, `unhealthy`, and `either`. (The default value is `either`)

`--event-source-trigger`

The event source of trigger to fire the notification. Valid values are `pool`, `origin`, and `either`. (The default value is `either`)

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a glb healthcheck alert policy.

```
$ ibmcloud cis alert-policy glb-healthcheck-alert-create --name test1 --emails test1@cn.ibm.com --enabled true --pools all --include-future-pools true --instance INSTANCE
```

ibmcloud cis alert-policy web-analytics-alert-create

Create an alert policy for web metrics report.

```
$ ibmcloud cis alert-policy web-analytics-alert-create --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

`--webhooks`

The webhook ID for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

`--enabled`

Whether the alert policy is enabled.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a web metrics report alert policy.

```
$ ibmcloud cis alert-policy web-analytics-alert-create --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --enabled true --instance INSTANCE
```

ibmcloud cis alert-policy maintenance-event-alert-create

Create an alert policy for maintenance event.

```
$ ibmcloud cis alert-policy maintenance-event-alert-create --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) --event-type TYPE [--airport-code AIRPORT_CODE] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

`--webhooks`

The webhook ID for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

`--enabled`

Whether the alert policy is enabled.

`--event-type`

The type of the maintenance event. Valid values are `scheduled` , `changed` , and `canceled` .

`--airport-code`

Comma-separated three-letter IATA Codes.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a maintenance event alert policy.

```
$ ibmcloud cis alert-policy maintenance-event-alert-create --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --event-type scheduled,changed,canceled --airport-code IAD,AUS --enabled true
```

ibmcloud cis alert-policy ddos-attack-l7-alert-update

Update an alert policy for DDoS attack l7.

```
$ ibmcloud cis alert-policy ddos-attack-l7-alert-update POLICY_ID [--name NAME] [--emails EMAILS] [--webhooks WEBHOOKS] [--enabled (true | false)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`POLICY_ID`

The ID of alert policy. Required.

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

`--webhooks`

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

`--enabled`

Whether the alert policy is enabled.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update a DDoS attack alert policy.

```
$ ibmcloud cis alert-policy ddos-attack-l7-alert-update POLICY_ID --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --enabled true
```

ibmcloud cis alert-policy ddos-attack-l3-l4-alert-update

Update an alert policy for DDoS attack L3/L4.

```
$ ibmcloud cis alert-policy ddos-attack-l3-l4-alert-update POLICY_ID [--name NAME] [--emails EMAILS] [--webhooks WEBHOOKS] [--enabled (true | false)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

- POLICY_ID**
The ID of alert policy. Required.
- name**
The name of the alert policy.
- description**
The description for the alert policy.
- emails**
The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`
- webhooks**
The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`
- enabled**
Whether the alert policy is enabled.
- i, --instance**
Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.
- output**
The output format. Currently, `json` is the only supported value.

Examples

Update a DDoS attack alert policy.

```
$ ibmcloud cis alert-policy ddos-attack-l3-l4-alert-update POLICY_ID --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --enabled true
```

ibmcloud cis alert-policy failing-logpush-job-alert-update

Update an alert policy when logpush job did not complete at least one successful push in the last 24 hours.

```
$ ibmcloud cis alert-policy failing-logpush-job-alert-update POLICY_ID [--name NAME] [--emails EMAILS] [--webhooks WEBHOOKS] [--enabled (true | false)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

- POLICY_ID**
The ID of alert policy. Required.
- name**
The name of the alert policy.
- description**
The description for the alert policy.
- emails**
The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`
- webhooks**
The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`
- enabled**

Whether the alert policy is enabled.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update a failing logpush job alert policy.

```
$ ibmcloud cis alert-policy failing-logpush-job-alert-update POLICY_ID --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e -
-enabled true
```

ibmcloud cis alert-policy pool-toggle-alert-update

Update an alert policy for pool toggle alert.

```
$ ibmcloud cis alert-policy pool-toggle-alert-update POLICY_ID --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) --pools POOLS --
trigger-condition (enabled | disabled | either) [--include-future-pools (true | false)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`POLICY_ID`

The ID of alert policy. Required.

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

`--webhooks`

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

`--enabled`

Whether the alert policy is enabled.

`--pools`

The IDs of the origin pool. If set to `all`, all pool IDs are used.

`--trigger-condition`

The condition of the pool toggle status.

`--include-future-pools`

Whether to include the future pools.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update a pool toggle alert policy.

```
$ ibmcloud cis alert-policy pool-toggle-alert-update POLICY_ID --name test1 --emails test1@cn.ibm.com --enabled true --pools all --trigger-condition enabled --include-
future-pools true
```


ibmcloud cis alert-policy firewall-events-alert-update

Update an alert policy about spikes in firewall events.

```
$ ibmcloud cis alert-policy firewall-events-alert-update POLICY_ID [--name NAME] [--emails EMAILS] [--webhooks WEBHOOKS] [--enabled (true | false)] [--domains DOMAINS] [--services SERVICES] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

POLICY_ID

The ID of alert policy. Required.

--name

The name of the alert policy.

--description

The description for the alert policy.

--emails

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

--webhooks

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

--enabled

Whether the alert policy is enabled.

--domains

The domain IDs that for the alert policy. For example, `--domains domainID1,domainID2`

--services

Specify which services the alert must monitor. Valid values are `country-access-rules`, `waf`, `firewall-rules`, `ratelimit`, `securitylevel`, `ip-access-rules`, `browser-integrity-check`, `ua-rules`, `lockdowns`, `iprange-access-rules`, `asn-access-rules`, `Managed-firewall` [Enterprise Plans Only](#)



Note: The 'SERVICES' is only used for advanced waf alert. If the alert policy you wanted to update is created without services that are specified, create a new one with sevices specified instead of updating.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update a firewall-events-alert policy.

```
$ ibmcloud cis alert-policy firewall-events-alert-update POLICY_ID --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --enabled true --domains d2633e61-1b61-2512-1321-b61a17c3db7e
```

ibmcloud cis alert-policy certificate-alert-update

Update an alert policy for certificate events.

```
$ ibmcloud cis alert-policy certificate-alert-update POLICY_ID [--name NAME] [--emails EMAILS] [--webhooks WEBHOOKS] [--enabled (true | false)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`POLICY_ID`

The ID of alert policy. Required.

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

`--webhooks`

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

`--enabled`

Whether the alert policy is enabled.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update a certificate alert policy.

```
$ ibmcloud cis alert-policy certificate-alert-update POLICY_ID --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --enabled true
```

ibmcloud cis alert-policy glb-healthcheck-alert-update

Update an alert policy for changes in health status for global load balancer, pools, and origins.

```
$ ibmcloud cis alert-policy glb-healthcheck-alert-update POLICY_ID [--name NAME] [--emails EMAILS] [--webhooks WEBHOOKS] [--enabled (true | false)] [--pools POOLS] [--include-future-pools (true | false)] [--health-status-trigger (healthy | unhealthy | either)] [--event-source-trigger (pool | origin | either)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`POLICY_ID`

The ID of alert policy. Required.

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

`--webhooks`

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

`--enabled`

Whether the alert policy is enabled.

`--pools`

The IDs of origin pool. If set to `all`, all pool IDs are used.

`--include-future-pools`

Whether to include the future pools. (The default value is `false`)

`--health-status-trigger`

The trigger condition to send the notification. Valid values are `healthy` , `unhealthy` , and `either` . (The default value is `either`)

`--event-source-trigger`

The event source of trigger to send the notification. Valid values are `pool` , `origin` , and `either` . (The default value is `either`)

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update a certificate alert policy.

```
$ ibmcloud cis alert-policy glb-healthcheck-alert-update POLICY_ID --name test1 --emails test1@cn.ibm.com --enabled true --pools all --include-future-pools true
```

ibmcloud cis alert-policy web-analytics-alert-update

Update an alert policy for web metric report.

```
$ ibmcloud cis alert-policy web-analytics-alert-update POLICY_ID [--name NAME] [--emails EMAILS] [--webhooks WEBHOOKS] [--enabled (true | false)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`POLICY_ID`

The ID of alert policy. Required.

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

`--webhooks`

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

`--enabled`

Whether the alert policy is enabled.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update a web metric report alert policy.

```
$ ibmcloud cis alert-policy web-analytics-alert-update POLICY_ID --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --enabled true
```

ibmcloud cis alert-policy maintenance-event-alert-update

Update an alert policy for maintenance event.

```
$ ibmcloud cis alert-policy maintenance-event-alert-update POLICY_ID [--name NAME] [--emails EMAILS] [--webhooks WEBHOOKS] [--enabled (true | false)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

POLICY_ID

The ID of alert policy. Required.

--name

The name of the alert policy.

--description

The description for the alert policy.

--emails

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

--webhooks

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

--enabled

Whether the alert policy is enabled.

--event-type

The type of the maintenance event. Valid values are `scheduled` , `changed` , and `canceled` .

--airport-code

Comma-separated three-letter IATA Codes.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update a maintenance event alert policy.

```
$ ibmcloud cis alert-policy maintenance-event-alert-update POLICY_ID --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --enabled true --event-type scheduled,changed,canceled --airport-code IAD,AUS
```

ibmcloud cis alert-policy delete

Delete an alert policy.

```
$ cis alert-policy delete POLICY_ID [-i, --instance INSTANCE] [-f, --force]
```

Command options

POLICY_ID

The ID of alert policy. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

-f, --force

Attempt to delete the policy without prompting for confirmation.

Examples

Delete an alert policy.

```
$ ibmcloud cis alert-policy delete POLICY_ID -f
```

ibmcloud cis alert-policy test

Send a test alert for an alert policy.

```
$ cis alert-policy test POLICY_ID [-i, --instance INSTANCE] [-f, --force]
```

Command options

POLICY_ID

The ID of alert policy. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

-f, --force

Attempt to send a test alert without prompting for confirmation.

Examples

Send a test notification for an alert policy.

```
$ ibmcloud cis alert-policy test POLICY_ID -f
```

Alert Webhook

ibmcloud cis alert-webhooks

List all alert webhooks.

```
$ ibmcloud cis alert-webhooks [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

List all webhooks for instance.

```
$ ibmcloud cis alert-webhooks
```

ibmcloud cis alert-webhook

Show the details of a webhook.

```
$ ibmcloud cis alert-webhook WEBHOOK_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

WEBHOOK_ID

The ID of the alert webhook. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show the details of the alert webhook.

```
$ ibmcloud cis alert-webhook WEBHOOK_ID
```

ibmcloud cis alert-webhook-create

Create an alert webhook for an instance.

```
$ ibmcloud cis alert-webhook-create --name NAME --url URL [--secret SECRET] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

--name

The name of the webhook. Required.

--url

The POST endpoint to call when dispatching an alert. Required.

--secret

The secret that is passed in the webhook auth header when dispatching a webhook alert.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Create an alert webhook.

```
$ ibmcloud cis alert-webhook-create --name testwebhook --url https://hooks.slack.com/services/Ds3fdBFbV/1234568 --secret 007 --instance INSTANCE
```

ibmcloud cis alert-webhook-update

Update an alert webhook.

```
$ cis alert-webhook-update WEBHOOK_ID [--name NAME] [--url URL] [--secret SECRET] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

WEBHOOK_ID

The ID of the alert webhook. Required.

--name

The name of the webhook.

--url

The POST endpoint to call when dispatching an alert.

`--secret`

The secret that is passed in the webhook auth header when dispatching a webhook alert.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update an alert webhook.

```
$ ibmcloud cis alert-webhook-update WEBHOOK_ID --name testwebhook --url https://hooks.slack.com/services/Ds3fdBFbV/1234568
```

ibmcloud cis alert-webhook-delete

Delete an alert webhook.

```
$ ibmcloud cis alert-webhook-delete WEBHOOK_ID [-i, --instance INSTANCE] [-f, --force]
```

Command options

`WEBHOOK_ID`

The ID of the alert webhook. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`-f, --force`

Attempt to delete the webhook without prompting for confirmation.

Examples

Delete an alert webhook.

```
$ ibmcloud cis alert-webhook-delete WEBHOOK_ID -f
```

Advanced Rate Limiting Rules

Manage the advanced rate limiting rules by using the following `advanced-rate-limiting` commands.

ibmcloud cis advanced-rate-limiting rules

List all advanced rate limiting rules.

```
$ ibmcloud cis advanced-rate-limiting rules DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List all advanced rate limiting rules for domain.

```
$ ibmcloud cis advanced-rate-limiting rules DNS_DOMAIN_ID
```

ibmcloud cis advanced-rate-limiting rule

Get details of an advanced rate limiting rule.

```
$ ibmcloud cis advanced-rate-limiting rule DNS_DOMAIN_ID RULE_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain.

RULE_ID

RULE_ID is the id of the advanced rate limiting rule.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show a rule for domain.

```
$ ibmcloud cis advanced-rate-limiting rule DNS_DOMAIN_ID RULE_ID
```

ibmcloud cis advanced-rate-limiting rule-create

Create an advanced rate limiting rule.

```
$ ibmcloud cis advanced-rate-limiting rule-create DNS_DOMAIN_ID --name NAME --match EXPRESSION --action ACTION --same-characteristics CHARACTERSTICS -
-requests REQUEST_PER_PERIOD --period PERIOD [--timeout TIMEOUT] [--enabled true|false] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain.

--name

The rule name.

--match

Specifies the conditions that must be matched for the rule to run. For match value, see [Using fields, functions, and expressions](#).

--action

Action to perform when the rate that is specified in the rule is reached. Valid values are `block`, `challenge`, `js_challenge`, `managed_challenge`, and `log`.

--same-characteristics

A set of parameters defining how CIS tracks the request rate for the rule. Use one or more of the characteristics: `ip`, `ip_nat`, `host`, `path`, `country`, `asnum`. For complex characteristics, use JSON file or JSON string instead.

`--requests`

The number of requests over the period of time that triggers the rule. Valid values range from `1-1000000`

`--period`

The period of time to consider (in seconds) when evaluating the request rate. Valid values are `10`, `60`, `120`, `300`, `600`, and `3600`.

`--timeout`

The rate limiting rule applies the rule action to further requests for the period of time. Valid values are `0`, `10`, `60`, `120`, `300`, `600`, `3600`, and `86400`.

`--enabled`

Indicates whether the rule is active or not. Valid values for "enabled" are `true` and `false`. (The default value is `false`)

`--json`

The JSON file or JSON string that is used to describe an advanced rate limiting rule.

- The required fields in JSON data are `expression`, `ratelimit` and `action`.
 - `expression` : Defines the criteria for the advanced rate limiting rule to match a request.
 - `ratelimit` : Define the rate-limit parameters.
 - `characteristics` : A set of parameters defining how CIS tracks the request rate for the rule.
 - `requests_per_period` : The number of requests over the period of time that triggers the rule.
 - `period` : The period of time to consider (in seconds) when evaluating the request rate. Valid values are `10`, `60`, `120`, `300`, `600`, and `3600`.
 - `requests_to_origin` : Apply the rate limiting to cached assets or not.
 - `mitigation_timeout` : The rate limiting rule applies the rule action to further requests for the period of time. Valid values are `0`, `10`, `60`, `120`, `300`, `600`, `3600`, and `86400`.
 - `counting_expression` : Defines the criteria that are used for determining the request rate.
 - `action` : Action to perform when the rate that is specified in the rule is reached. Valid values are `block`, `challenge`, `js_challenge`, `managed_challenge`, and `log`.
- The optional fields are `description`, `action_parameters`, and `enabled`.
 - `description` : The descriptive name of your rule.
 - `action_parameters` : Define the action parameters.
 - `response` : Define a custom response for block action.
 - `status_code` : Defines the HTTP status code that is returned to the visitor when blocking the request due to rate limiting. Only available when the rule action is Block. Valid values range from `400-499`. The default value is `429`.
 - `content_type` : Defines the content type of a custom response when blocking a request due to rate limiting. Only available when the rule action is Block.
 - `content` : Defines the body of the returned HTTP response when the request is blocked due to rate limiting. Only available when the rule action is Block.
 - `enabled` : Whether enable this rule or not.

\$ Sample JSON data:

```
{
  "description": "description",
  "expression": "(http.request.method eq \"POST\")",
  "ratelimit": {
    "characteristics": [
      "cf.unique_visitor_id",
      "cf.colo.id"
    ],
    "requests_to_origin": false,
    "counting_expression": "(ip.src.continent in {\"AN\"})",
    "requests_per_period": 10,
    "period": 10,
    "mitigation_timeout": 120
  },
  "action": "block",
```

```
"action_parameters": {
  "response": {
    "status_code": 429,
    "content_type": "text/xml",
    "content": "reject"
  }
},
"enabled": false
}
```

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create an advanced rate limiting rule for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis advanced-rate-limiting rule-create DNS_DOMAIN_ID --name rule-name --match "(http.request.method eq \"POST\")" --action log --same-characteristics ip,ip_nat --requests 100 --period 10
```

ibmcloud cis advanced-rate-limiting rule-update

Update an advanced rate limiting rule.

```
$ ibmcloud cis advanced-rate-limiting rule-update DNS_DOMAIN_ID RULE_ID --name NAME --match EXPRESSION --action ACTION --same-characteristics CHARACTERSTICS --requests REQUEST_PER_PERIOD --period PERIOD [--timeout TIMEOUT] [--enabled true|false] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain.

`RULE_ID`

RULE_ID is the id of the advanced rate limiting rule.

`--name`

The rule name.

`--match`

Specifies the conditions that must be matched for the rule to run. For match value, see [Using fields, functions, and expressions](#).

`--action`

Action to perform when the rate that is specified in the rule is reached. Valid values are `block`, `challenge`, `js_challenge`, `managed_challenge`, and `log`.

`--same-characteristics`

A set of parameters defining how CIS tracks the request rate for the rule. Use one or more of the characteristics: `ip`, `ip_nat`, `host`, `path`, `country`, `asnum`. For complex characteristics, use JSON file or JSON string instead.

`--requests`

The number of requests over the period of time that triggers the rule. Valid values range from `1-1000000`

`--period`

The period of time to consider (in seconds) when evaluating the request rate. Valid values are `10`, `60`, `120`, `300`, `600`, and `3600`.

`--timeout`

The rate limiting rule applies the rule action to further requests for the period of time. Valid values are `0`, `10`, `60`, `120`, `300`, `600`, `3600` and `86400`.

`--enabled`

Indicates whether the rule is active or not. Valid values for "enabled" are `true` and `false`. (The default value is `false`)

`--json`

The JSON file or JSON string that is used to describe an advanced rate limiting rule.

- The required fields in JSON data are `expression`, `ratelimit` and `action`.
 - `expression`: Defines the criteria for the advanced rate limiting rule to match a request.
 - `ratelimit`: Define the ratelimit parameters.
 - `characteristics`: A set of parameters defining how CIS tracks the request rate for the rule.
 - `requests_per_period`: The number of requests over the period of time that triggers the rule.
 - `period`: The period of time to consider (in seconds) when evaluating the request rate. Valid values are `10`, `60`, `120`, `300`, `600`, and `3600`.
 - `requests_to_origin`: Apply the rate limiting to cached assets or not.
 - `mitigation_timeout`: The rate limiting rule applies the rule action to further requests for the period of time. Valid values are `0`, `10`, `60`, `120`, `300`, `600`, `3600`, and `86400`.
 - `counting_expression`: Defines the criteria that are used for determining the request rate.
 - `action`: Action to perform when the rate that is specified in the rule is reached. Valid values are `block`, `challenge`, `js_challenge`, `managed_challenge`, and `log`.
- The optional fields are `description`, `action_parameters`, and `enabled`.
 - `description`: The descriptive name of your rule.
 - `action_parameters`: Define the action parameters.
 - `response`: Define a custom response for block action.
 - `status_code`: Defines the HTTP status code that is returned to the visitor when blocking the request due to rate limiting. Only available when the rule action is Block. Valid values range from `400-499`. The default value is `429`.
 - `content_type`: Defines the content type of a custom response when blocking a request due to rate limiting. Only available when the rule action is Block.
 - `content`: Defines the body of the returned HTTP response when the request is blocked due to rate limiting. Only available when the rule action is Block.
 - `enabled`: Whether enable this rule or not.

\$ Sample JSON data:

```
{
  "description": "description",
  "expression": "(http.request.method eq \"POST\")",
  "ratelimit": {
    "characteristics": [
      "cf.unique_visitor_id",
      "cf.colo.id"
    ],
    "requests_to_origin": false,
    "counting_expression": "(ip.src.continent in {\"AN\"})",
    "requests_per_period": 10,
    "period": 10,
    "mitigation_timeout": 120
  },
  "action": "block",
  "action_parameters": {
    "response": {
      "status_code": 429,
      "content_type": "text/xml",
      "content": "reject"
    }
  },
  "enabled": false
}
```



```
}

```

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update an advanced rate limiting rule `c2e184081120413c86c3ab7e14069605` for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis advanced-rate-limiting rule-update DNS_DOMAIN_ID RULE_ID --name rule-name --match "(http.request.method eq \"POST\")" --action log --same-characteristics ip,ip_nat --requests 100 --period 10
```

ibmcloud cis advanced-rate-limiting rule-delete

Delete an advanced rate limiting rule by id.

```
$ ibmcloud cis advanced-rate-limiting rule-delete DNS_DOMAIN_ID RULE_ID [-f, --force] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain.

`RULE_ID`

RULE_ID is the id of the advanced rate limiting rule.

`-f, --force`

Attempt to delete an advanced rate limiting rule without prompting for confirmation.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Delete a rule `c2e184081120413c86c3ab7e14069605` for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis advanced-rate-limiting rule-delete DNS_DOMAIN_ID RULE_ID
```

WAF managed rules

Manage the WAF-managed rulesets and rules by using the following `managed-waf` commands. Migrate to a new WAF by API or GUI first before you use managed WAF commands. Keep in mind that the previous version of WAF commands stops working after you migrate.

ibmcloud cis managed-waf rulesets

List all managed WAF rulesets.

```
$ ibmcloud cis managed-waf rulesets DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List all managed WAF rulesets for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis managed-waf rulesets DNS_DOMAIN_ID
```

ibmcloud cis managed-waf ruleset

Get details of a managed WAF ruleset.

```
$ ibmcloud cis managed-waf ruleset DNS_DOMAIN_ID RULESET_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of DNS domain.

`RULESET_ID`

The ID of the ruleset.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Show a ruleset `c2e184081120413c86c3ab7e14069605` for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis managed-waf ruleset DNS_DOMAIN_ID RULESET_ID
```

ibmcloud cis managed-waf deployment

Get details of a deployed managed WAF rule.

```
$ ibmcloud cis managed-waf deployment DNS_DOMAIN_ID RULE_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain.

`RULE_ID`

The ID of the rule.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Show a deployment rule `a2121e23-9e68-1218-a356-b78e23a8ec8a` for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis managed-waf deployment DNS_DOMAIN_ID RULE_ID
```

ibmcloud cis managed-waf deployments

List all deployed managed WAF rules.

```
$ ibmcloud cis managed-waf deployments DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of DNS domain.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List all deployment rules for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis managed-waf deployments DNS_DOMAIN_ID
```

ibmcloud cis managed-waf deployment-add-exception

Create an exception rule to skip execution of specified managed WAF rules.

```
$ ibmcloud cis managed-waf deployment-add-exception DNS_DOMAIN_ID --match EXPRESSION [--skip-rules RULES] [--enabled true|false] [--logging true|false] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis managed-waf deployment-add-exception DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of DNS domain.

`--match`

Specifies the conditions that must be matched for the rule to run. For match value, see [Using fields, functions, and expressions](#).

`--skip-rules`

Skip all remaining rules, WAF managed rulesets, or rules of WAF managed rulesets. For example, `--skip-rules RULESETID-1:RULEID-a,RULEID-b;RULESETID-2:RULEID-x,RULEID-y`. Set `current` to skip all remaining rules. The default value is `current`.

`--enabled`

Indicates whether the rule is active or not. The default value is `true`.

`--logging`

Log requests matching the skip rule. The default value is `true`.

`--description`

A brief description of the rule.

`--json`

The JSON file or JSON string that is used to describe a managed WAF.

- The required fields in JSON data are `expression`, `action` and `action_parameters`.

`expression` : The rule expression. `action` : The rule action to perform. Valid value is `skip`. `action_parameters` : The rule action parameters. `ruleset` : Skip all remaining rules or one or more WAF-managed rulesets. `rules` : Skip one or more rules of WAF-managed rulesets.

- The optional fields are `description`, `enabled`, and `logging`. `description` : Briefly describes the rule. `enabled` : Indicates whether the rule is active or not. `logging` : Log requests matching the skip rule. - `enabled` : When disabled, matched requests don't appear in firewall events.

Sample JSON data: `json { "action": "skip", "expression": "(http.cookie eq \"example.com/contact?page=1234\")", "description": "rule name", "enabled": true, "logging": { "enabled": true }, "action_parameters": { "rules": { "efb7b8c949ac4650a09736fc376e9aee": ["5de7edfa648c4d6891dc3e7f84534ffa", "e3a567afc347477d9702d9047e97d760"], "c2e184081120413c86c3ab7e14069605": ["ef21b0a932ae422790f9249d213b85e6"] } } }`

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create exception rule for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis managed-waf deployment-add-exception DNS_DOMAIN_ID --match "(http.cookie eq \"example.com/contact?page=1234\")" --skip-rules 'efb7b8c949ac4650a09736fc376e9aee:5de7edfa648c4d6891dc3e7f84534ffa' --enabled false --logging true
```

ibmcloud cis managed-waf deployment-update-exception

Update an exception rule in the deployed managed WAF rules.

```
$ ibmcloud cis managed-waf deployment-update-exception DNS_DOMAIN_ID RULE_ID [--match MATCH] [--skip-rules RULES] [--enabled true|false] [--logging true|false] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis managed-waf deployment-update-exception DNS_DOMAIN_ID RULE_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of DNS domain.

`RULE_ID`

The ID of rule.

`--match`

Specifies the conditions that must be matched for the rule to run. For match value, see [Using fields, functions, and expressions](#).

`--skip-rules`

Skip all remaining rules, WAF managed rulesets, or rules of WAF managed rulesets. For example, `--skip-rules RULESETID-1:RULEID-a,RULEID-b;RULESETID-2:RULEID-x,RULEID-y`. Set `current` to skip all remaining rules. The default value is `current`.

`--enabled`

Indicates whether the rule is active or not. The default value is `true`.

`--logging`

Log requests matching the skip rule. The default value is `true`.

`--description`

To briefly describe the rule.

--json

The JSON file or JSON string that is used to describe a managed WAF.

- The required fields in JSON data are `expression`, `action`, and `action_parameters`.

`expression`: The rule expression. `action`: The rule action to perform. Valid value is `skip`. `action_parameters`: The rule action parameters.
`ruleset`: Skip all remaining rules or one or more WAF-managed rulesets. `rules`: Skip one or more rules of WAF-managed rulesets.
- The optional fields are `description`, `enabled`, and `logging`. `description`: Briefly describes the rule. `enabled`: Indicates whether the rule is active or not. `logging`: Log requests matching the skip rule. - `enabled`: When disabled, matched requests don't appear in firewall events.

Sample JSON data:

```
$ {
  "action": "skip",
  "expression": "(http.cookie eq \"example.com/contact?page=1234\")",
  "description": "rule name",
  "enabled": true,
  "logging": {
    "enabled": true
  },
  "action_parameters": {
    "rules": {
      "efb7b8c949ac4650a09736fc376e9aee": [
        "5de7edfa648c4d6891dc3e7f84534ffa",
        "e3a567afc347477d9702d9047e97d760"
      ],
      "c2e184081120413c86c3ab7e14069605": [
        "ef21b0a932ae422790f9249d213b85e6"
      ]
    }
  }
}
```

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update an exception rule `e7ead74deb2b4c30a91c793f502f5e14` for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis managed-waf deployment-update-exception DNS_DOMAIN_ID RULE_ID --match "(http.cookie eq \"example.com/contact?page=1234\")" --skip-rules 'efb7b8c949ac4650a09736fc376e9aee:5de7edfa648c4d6891dc3e7f84534ffa' --enabled false --logging true
```

ibmcloud cis managed-waf deployment-add-ruleset

Add a managed ruleset to the deployed managed WAF rules.

```
$ ibmcloud cis managed-waf deployment-add-ruleset DNS_DOMAIN_ID RULESET_ID [--match EXPRESSION] [--enabled true|false] [--override-action ACTION] [--override-status STATUS] [--paranoia-level LEVEL] [--override-rules RULE] [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis managed-waf deployment-add-ruleset DNS_DOMAIN_ID RULESET_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of DNS domain.

RULESET_ID

The ID of managed ruleset.

--match

Specifies the conditions that must be matched for the rule to run. For match value, see [Using fields, functions, and expressions](#).

--enabled

Indicates whether the rule is active or not. The default value is `true`.

--override-action

The ruleset action of the overrides. Valid values are `managed_challenge`, `block`, `js_challenge`, `log`, and `challenge`.

--override-status

The ruleset status of the overrides. Valid values are `true` and `false`.

--paranoia-level

OWASP paranoia level, higher paranoia levels activate more aggressive rules. Valid values are `PL1`, `PL2`, `PL3`, `PL4` and it's only available for `CIS OWASP Core Ruleset`.

--override-rules

The rules options of the overrides. For example `--override-rules rule=RULE_ID,action=ACTION,enabled=STATUS`. For OWASP Core Ruleset, you can also override the Score Threshold. For example, `--override-rules rule=6179ae15870a4bb7b2d480d4843b323c,score-threshold=25`.

--json

The JSON file or JSON string that is used to describe a managed WAF rule.

- The required fields in JSON data are `expression`, `action`, and `action_parameters`.

`expression`: The rule expression. `action`: The rule action to perform. Valid value is `skip`. `action_parameters`: The rule action parameters.
`id`: The ruleset ID of the overrides. `overrides`: The rules options of the overrides. `action`: The ruleset action of the overrides. Valid values are `managed_challenge`, `block`, and `js_challenge`, "log", "challenge". `enabled`: The ruleset status of the overrides. Valid values are `true` and `false`. `rules`: The rules options of the overrides. `id`: The rule ID of the overrides. `action`: The rule action of the overrides. Valid values are `managed_challenge`, `block`, `js_challenge`, `log`, and `challenge`. `enabled`: The rule status of the overrides. `score_threshold`: OWASP Anomaly Score Threshold. Set the score threshold, which triggers the Firewall. `categories`: Define OWASP Paranoia Level and only valid for `CIS OWASP core ruleset` `category`: OWASP paranoia level, higher paranoia levels activate more aggressive rules. `enabled`: Whether this OWASP Paranoia Level is enabled.
- The optional fields are `description` and `enabled`. `description`: Briefly describes the rule. `enabled`: Indicates whether the rule is active or not.

Sample JSON data:

```
$ {
  "action": "execute",
  "description": "CIS Managed Ruleset",
  "enabled": true,
  "expression": "(http.cookie eq \"example.com/contact?page=1234\")",
  "action_parameters": {
    "id": "efb7b8c949ac4650a09736fc376e9aee",
    "overrides": {
      "action": "block",
      "enabled": false,
      "rules": [
        {
          "id": "5de7edfa648c4d6891dc3e7f84534ffa",
          "action": "managed_challenge"
        },
        {
          "id": "e3a567afc347477d9702d9047e97d760",
          "action": "log",
          "enabled": true
        }
      ]
    }
  }
}
```

```
    ]
  }
}
```

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Deploy a managed ruleset for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis managed-waf deployment-add-ruleset DNS_DOMAIN_ID RULESET_ID --match true --enabled true --override-action block --override-status true --
override-rules rule=5de7edfa648c4d6891dc3e7f84534ffa,action=managed_challenge --override-rules
rule=e3a567afc347477d9702d9047e97d760,action=action,enabled=true
```

ibmcloud cis managed-waf deployment-update-ruleset

Update a managed ruleset in the deployed managed WAF rules.

```
$ ibmcloud cis managed-waf deployment-update-ruleset DNS_DOMAIN_ID RULE_ID [--match EXPRESSION] [--enabled true|false] [--override-action ACTION] [--
override-status STATUS] [--paranoia-level LEVEL] [--override-rules RULE] [--reset-all] [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis managed-waf deployment-update-ruleset DNS_DOMAIN_ID RULE_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output
FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of DNS domain.

`RULE_ID`

The ID of deployed managed rule.

`--match`

Specifies the conditions that must be matched for the rule to run. For match value, see [Using fields, functions, and expressions](#).

`--enabled`

Indicates whether the rule is active or not. The default value is `true`.

`--override-action`

The ruleset action of the overrides. Valid values are `managed_challenge`, `block`, `js_challenge`, `log`, and `challenge`.

`--override-status`

The ruleset status of the overrides. Valid values are `true` and `false`.

`--paranoia-level`

OWASP paranoia level, higher paranoia levels activate more aggressive rules. Valid values are `PL1`, `PL2`, `PL3`, `PL4` and it's only available for `CIS OWASP Core Ruleset`.

`--override-rules`

The rules options of the overrides. For example, `--override-rules rule=RULE_ID,action=ACTION,enabled=STATUS`. For OWASP Core Ruleset, you can also override the Score Threshold. For example, `--override-rules rule=6179ae15870a4bb7b2d480d4843b323c,score-threshold=25`.

`--reset-all`

Reset all the overrides rules to the default settings.

`--json`

The JSON file or JSON string that is used to describe a managed waf rule.

- The required fields in JSON data are `expression` , `action` , and `action_parameters` .

`expression` : The rule expression. `action` : The rule action to perform. Valid value is `skip` . `action_parameters` : The rule action parameters.
`id` : The ruleset id of the overrides. `overrides` : The rules options of the overrides. `action` : The ruleset action of the overrides. Valid values are `managed_challenge` , `block` , and `js_challenge` , `log` , `challenge` . `enabled` : The ruleset status of the overrides. Valid values are `true` and `false` . `rules` : The rules options of the overrides. `id` : The rule ID of the overrides. `action` : The rule action of the overrides. Valid values are `managed_challenge` , `block` , `js_challenge` , `log` , and `challenge` . `enabled` : The rule status of the overrides. `score_threshold` : OWASP Anomaly Score Threshold. Set the score threshold, which triggers the Firewall. `categories` : Define OWASP Paranoia Level and only valid for `CIS OWASP core ruleset` `category` : OWASP paranoia level, higher paranoia levels activate more aggressive rules. `enabled` : Whether this OWASP Paranoia Level enabled.
- The optional fields are `description` and `enabled` . `description` : Briefly describes the rule. `enabled` : Indicates whether the rule is active or not.

Sample JSON data:

```
$ {
  "action": "execute",
  "description": "CIS Managed Ruleset",
  "enabled": true,
  "expression": "(http.cookie eq \"example.com/contact?page=1234\")",
  "action_parameters": {
    "id": "efb7b8c949ac4650a09736fc376e9aee",
    "overrides": {
      "action": "block",
      "enabled": false,
      "rules": [
        {
          "id": "5de7edfa648c4d6891dc3e7f84534ffa",
          "action": "managed_challenge"
        },
        {
          "id": "e3a567afc347477d9702d9047e97d760",
          "action": "log",
          "enabled": true
        }
      ]
    }
  }
}
```

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update a managed ruleset rule `1a18a1ea7fc043c68761bc69adcbb11c` for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo` .

```
$ ibmcloud cis managed-waf deployment-update-ruleset DNS_DOMAIN_ID RULE_ID --match true --enabled true --override-action block --override-status true --override-rules rule=5de7edfa648c4d6891dc3e7f84534ffa,action=managed_challenge --override-rules rule=e3a567afc347477d9702d9047e97d760,action=action,enabled=true
```

WAF custom rules

Manage the WAF custom rules by using the following `custom-waf` commands. Firewall rules are now managed by WAF custom rules.

ibmcloud cis custom-waf rules

List all custom rules.


```
$ ibmcloud cis custom-waf rules DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

List all custom rules for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis custom-waf rules DNS_DOMAIN_ID
```

ibmcloud cis custom-waf rule

Get details of a custom rule.

```
$ ibmcloud cis custom-waf rule DNS_DOMAIN_ID RULE_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of DNS domain.

RULE_ID

The ID of the rule.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show a custom rule `b94632a4cd5a49ed830544d91417a98c` for domain `9343630b9bd5c6e6899834d77f9e50ff` under instance `cis-demo`.

```
$ ibmcloud cis custom-waf rule DNS_DOMAIN_ID RULE_ID
```

ibmcloud cis custom-waf rule-create

Create a custom rule.

```
$ ibmcloud cis custom-waf rule-create DNS_DOMAIN_ID --match EXPRESSION --action ACTION [--description DESCRIPTION] [--enabled true|false] [-i, --instance INSTANCE] [--output FORMAT]
```

```
ibmcloud cis custom-waf rule-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain.

`--match`

Specifies the conditions that must be matched for the rule to run. For match value, see [Using fields, functions, and expressions](#).

`--action`

The rule action to perform. Valid values are `block`, `challenge`, `js_challenge`, `managed_challenge`, `log`, and `skip`. For 'block' and 'skip' actions, use JSON file or JSON string instead.

`--enabled`

Indicates whether the rule is active or not. The default value is `false`.

`--description`

A brief description of the rule.

`--json`

The JSON file or JSON string that is used to describe a custom rule.

- The required fields in JSON data are `expression` and `action`.
 - `expression` : Specifies the conditions that must be matched for the rule to run.
 - `action` : The rule action to perform. Valid values are `block`, `challenge`, `js_challenge`, `managed_challenge`, `log`, and `skip`.
- The optional fields are `description`, `enabled`, `logging`, and `action_parameters`.
 - `action_parameters` : The rule action parameters.
 - `ruleset` : Skip all remaining rules or one or more WAF managed rulesets. Valid value is `current`.
 - `phases` : Skips WAF components for matching requests. Valid values are `http_ratelimit`, `http_request_firewall_managed`, and `http_request_sbfm`.
 - `products` : Skips specific security products for matching requests. Valid values are `waf`, `rateLimit`, `securityLevel`, `hot`, `bic`, `uaBlock`, and `zoneLockdown`.
 - `response` : Define a custom response for `block` action.
 - `status_code` : Choose an HTTP status code for the response in the range `400-499`.
 - `content_type` : The content type of a custom response. Valid response types are : `text/html`, `text/plain`, `application/json`, `text/xml`.
 - `content` : The response body.
 - `description` : Briefly describes the rule.
 - `enabled` : Indicates whether the rule is active or not. When this field is disabled, matched requests don't appear in firewall requests.
 - `logging` : Log requests matching the skip rule. This field is only available for 'skip' action.

Sample JSON data:

```
$ {
  "description": "test-custom-rule",
  "expression": "(http.cookie contains \"test\")",
  "action": "skip",
  "logging": {
    "enabled": true
  },
  "action_parameters": {
    "ruleset": "current",
    "phases": [
      "http_ratelimit",
      "http_request_firewall_managed",
      "http_request_sbfm"
    ],
    "products": [
      "waf",
      "rateLimit",
      "securityLevel",
      "hot",
      "bic",
      "uaBlock",
      "zoneLockdown"
    ]
  }
}
```

```
    ]
  },
  "enabled": true
}
```

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a custom rule for domain `9343630b9bd5c6e6899834d77f9e50ff` under instance `cis-demo`.

```
$ ibmcloud cis custom-waf rule-create DNS_DOMAIN_ID --action challenge --description "rule 1" --enabled true --match "(http.host eq \"www.example.com\")"
```

ibmcloud cis custom-waf rule-update

Update a custom rule.

```
$ ibmcloud cis custom-waf rule-update DNS_DOMAIN_ID [--match EXPRESSION] [--action ACTION] [--description DESCRIPTION] [--enabled true|false] [-i, --instance INSTANCE] [--output FORMAT]

ibmcloud cis custom-waf rule-update DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain.

`RULE_ID`

The ID of the rule.

`--match`

Specifies the conditions that must be matched for the rule to run. For match value, see [Using fields, functions, and expressions](#).

`--action`

The rule action to perform. Valid values are `block`, `challenge`, `js_challenge`, `managed_challenge`, `log`, and `skip`. For 'block' and 'skip' actions, use JSON file or JSON string instead.

`--enabled`

Indicates whether the rule is active or not. The default value is `false`.

`--description`

A brief description of the rule.

`--json`

The JSON file or JSON string that is used to describe a custom rule.

- The required fields in JSON data are `expression` and `action`.
 - `expression` : Specifies the conditions that must be matched for the rule to run.
 - `action` : The rule action to perform. Valid values are `block`, `challenge`, `js_challenge`, `managed_challenge`, `log`, and `skip`.
- The optional fields are `description`, `enabled`, `logging`, and `action_parameters`.
 - `action_parameters` : The rule action parameters.
 - `ruleset` : Skip all remaining rules or one or more WAF managed rulesets. Valid value is `current`
 - `phases` : Skips WAF components for matching requests. Valid values are `http_ratelimit`, `http_request_firewall_managed`, and

- `http_request_sbfm` .
- `products` : Skips specific security products for matching requests. Valid values are `waf` , `rateLimit` , `securityLevel` , `hot` , `bic` , `uaBlock` , and `zoneLockdown` .
- `response` : Define a custom response for 'block' action.
 - `status_code` : Choose an HTTP status code for the response in the range `400-499` .
 - `content_type` : The content type of a custom response. Valid response types are `text/html` , `text/plain` , `application/json` , and `text/xml` .
 - `content` : The response body.
- `description` : Briefly describes the rule.
- `enabled` : Indicates whether the rule is active or not. When this field is disabled, matched requests don't appear in firewall requests.
- `logging` : Log requests matching the skip rule. This field is only available for 'skip' action.

Sample JSON data:

```
$ {
  "description": "test-custom-rule",
  "expression": "(http.cookie contains \"test\")",
  "action": "block",
  "action_parameters": {
    "response": {
      "status_code": 429,
      "content_type": "text/xml",
      "content": "reject"
    }
  },
  "enabled": true
}
```

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update a custom rule `b94632a4cd5a49ed830544d91417a98c` for the domain `9343630b9bd5c6e6899834d77f9e50ff` under instance `cis-demo` .

```
$ ibmcloud cis custom-waf rule-update DNS_DOMAIN_ID RULE_ID --enabled false --description "rule 1" --match "(http.host eq \"www.example.com\")"
```

ibmcloud cis custom-waf rule-order-update

Change the execution order of the custom rule.

```
$ ibmcloud cis custom-waf rule-order-update DNS_DOMAIN_ID RULE_ID [--before RULE_ID] [--after RULE_ID] [--index INDEX] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain.

`RULE_ID`

The ID of the custom rule.

`--before`

Places the rule before rule `<RULE_ID>` .

`--after`

Places the rule after rule `<RULE_ID>`.

`--index`

Places the rule in the exact position that is specified by the integer number.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Put custom rule `4eae81b170f644f795da017001383de7` before rule `2ed2dd160cb745feb415414544d97c70` for domain `9343630b9bd5c6e6899834d77f9e50ff` under instance `cis-demo`.

```
$ ibmcloud cis custom-waf rule-order-update DNS_DOMAIN_ID RULE_ID --before RULE_ID
```

ibmcloud cis custom-waf rule-delete

Delete a custom rule by ID.

```
$ ibmcloud cis custom-waf rule-delete DNS_DOMAIN_ID RULE_ID [-f, --force] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain.

`RULE_ID`

The ID of the custom rule.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`-f, --force`

Attempt to delete a custom rule without prompting for confirmation.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Delete custom rule `e9fad806880c4c42bd7ebeec8dcba4e6` for domain `9343630b9bd5c6e6899834d77f9e50ff` under instance `cis-demo`.

```
$ ibmcloud cis custom-waf rule-delete DNS_DOMAIN_ID RULE_ID
```

Private endpoint support

To ensure that you have enhanced control and security over your data when you use the CIS CLI, you have the option of using private routes to CIS endpoints. Private routes are not accessible or reachable over the internet. By using CIS private endpoints, you can protect your data from threats from the public network and logically extend your private network.

Regional support is provided for a limited number of CLI commands. The following regions support private endpoints:

- us-south
- us-east

Logging in to the CLI with a private endpoint

Use the following command to log in to a private endpoint from the CLI:


```
$ ibmcloud login -a private.cloud.ibm.com
```

Targeting a supported region

A region must be targeted when a private endpoint is set. Use the following command to target a supported region:

```
$ ibmcloud target -r [region]
```

Using CIS CLI with private endpoints

All the commands support private endpoints, for example:

```
$ ibmcloud cis domains --instance cis-demo
```

CIS CLI reference

IBM Cloud® Internet Services has several families of commands that are available from the command line interface (CLI). Find the full set of commands for IBM Cloud Internet Services (CIS) within each set, such as `Create` , `Delete` , and `Update` .

Before you begin

1. Download the [IBM CLI](#).
2. Log in to IBM Cloud.

```
$ ibmcloud login -a
```

3. Install the CIS CLI plug-in.

```
$ ibmcloud plugin install cis
```

4. Set the context instance.

```
$ ibmcloud cis instance-set <instance-name>
```

To see a list of plug-ins and which versions are installed, run this command.

```
$ ibmcloud plugin list
```

The list returns whether the CLI has any updates available. Run the following command to update the CIS CLI plug-in.

```
$ ibmcloud plugin update cis
```

To learn about installing and configuring the IBM Cloud CLI, see [Getting started with the IBM Cloud CLI](#).

Access application

ibmcloud cis access-app-create

Enterprise Plans Only

Create an access application for a DNS domain.

```
$ ibmcloud cis access-app-create DNS_DOMAIN_ID --name NAME --domain DOMAIN [--session-duration SESSION_DURATION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create an access application for domain `31984fea73a15b45779fa0df4ef62f9b`.

```
$ ibmcloud cis access-app-create DNS_DOMAIN_ID --name exampleCreate --domain example.com --session-duration 12h
```

ibmcloud cis access-apps

Enterprise Plans Only

List all access applications for a DNS domain.

```
$ ibmcloud cis access-apps DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List all access applications for domains `31984fea73a15b45779fa0df4ef62f9b`.

```
$ ibmcloud cis access-apps DNS_DOMAIN_ID
```

ibmcloud cis access-app

Enterprise Plans Only

Show details of an access application.

```
$ ibmcloud cis access-app DNS_DOMAIN_ID ACCESS_APPLICATION_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`ACCESS_APPLICATION_ID`

The ID of the access application. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Show details of access application `a5836c2a7ea72d2e225890caea70ae32` .

```
$ ibmcloud cis access-app DNS_DOMAIN_ID ACCESS_APPLICATION_ID
```

ibmcloud cis access-app-update

Enterprise Plans Only

Update an access application.

```
$ ibmcloud cis access-app-update DNS_DOMAIN_ID ACCESS_APPLICATION_ID --name NAME --domain DOMAIN [--session-duration SESSION_DURATION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

ACCESS_APPLICATION_ID

The ID of the access application. Required.

--name

The name of the Application. Required.

--domain

The domain and path that Access blocks. Required.

--session-duration

Defines the amount of time that the tokens issued for this application are valid. Valid values are `30m` , `6h` , `12h` , `24h` , `168h` , and `730h` .

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update access application `a5836c2a7ea72d2e225890caea70ae32` .

```
$ ibmcloud cis access-app-update DNS_DOMAIN_ID ACCESS_APPLICATION_ID --name exampleUpdate --domain example.com --session-duration 24h
```

ibmcloud cis access-app-delete

Enterprise Plans Only

Delete an access application.

```
$ ibmcloud cis access-app-delete DNS_DOMAIN_ID ACCESS_APPLICATION_ID [-i, --instance INSTANCE]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

ACCESS_APPLICATION_ID

The ID of the access application. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete access application `a5836c2a7ea72d2e225890caea70ae32` .

```
$ ibmcloud cis access-app-delete DNS_DOMAIN_ID ACCESS_APPLICATION_ID
```

Access certificate

ibmcloud cis access-certificate-create

Enterprise Plans Only

Create an access certificate for a DNS domain.

```
$ ibmcloud cis access-certificate-create DNS_DOMAIN_ID --name NAME --ca-cert-file CERT_FILE [--associated-hostnames ASSOCIATED_HOSTNAMES] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--name

The name of the Certificate. Required.

--ca-cert-file

The Root CA file for your certificates. Required.

--associated-hostnames

The hostnames that are prompted for this certificate.

ACCESS_APPLICATION_ID

The ID of the access application.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Create an access certificate for domain `31984fea73a15b45779fa0df4ef62f9b` .

```
$ ibmcloud cis access-certificate-create DNS_DOMAIN_ID --name example --ca-cert-file CERT_FILE --associated-hostnames example.com
```

ibmcloud cis access-certificates

Enterprise Plans Only

List all access certificates for a DNS domain.

```
$ ibmcloud cis access-certificates DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

List all access certificates for domain `31984fea73a15b45779fa0df4ef62f9b`.

```
$ ibmcloud cis access-certificates DNS_DOMAIN_ID
```

ibmcloud cis access-certificate

Enterprise Plans Only

Show details of an access certificate.

```
$ ibmcloud cis access-certificate DNS_DOMAIN_ID ACCESS_CERTIFICATE_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

ACCESS_CERTIFICATE_ID

The ID of the access certificate. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show details of access certificate `a5836c2a7ea72d2e225890caea70ae32`.

```
$ ibmcloud cis access-certificate DNS_DOMAIN_ID ACCESS_CERTIFICATE_ID
```

ibmcloud cis access-certificate-update

Enterprise Plans Only

Update an access certificate.

```
$ ibmcloud cis access-certificate-update DNS_DOMAIN_ID ACCESS_CERTIFICATE_ID --name NAME --associated-hostnames ASSOCIATED_HOSTNAMES [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

ACCESS_CERTIFICATE_ID

The ID of the access certificate. Required.

--name

The name of the Certificate. Required.

--associated-hostnames

The hostnames that are prompted for this certificate. Required.



Note: The associated hostnames are reset if not specified by `associated-hostnames` .

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update the access certificate `a5836c2a7ea72d2e225890caea70ae32` .

```
$ ibmcloud cis access-certificate-update DNS_DOMAIN_ID ACCESS_CERTIFICATE_ID --name example --associated-hostnames example.com
```

ibmcloud cis access-certificate-delete

Enterprise Plans Only

Delete an access certificate.

```
$ ibmcloud cis access-certificate-delete DNS_DOMAIN_ID ACCESS_CERTIFICATE_ID [-i, --instance INSTANCE]
```



Note: Must clear the associated hostnames before you delete the certificate.

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`ACCESS_CERTIFICATE_ID`

The ID of the access certificate. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete the access certificate `a5836c2a7ea72d2e225890caea70ae32` .

```
$ ibmcloud cis access-certificate-delete DNS_DOMAIN_ID ACCESS_CERTIFICATE_ID
```

ibmcloud cis access-certificates-settings

Enterprise Plans Only

Get access certificates settings for a DNS domain.

```
$ ibmcloud cis access-certificates-settings DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Get access certificates settings for Domain `31984fea73a15b45779fa0df4ef62f9b` .

```
$ ibmcloud cis access-certificates-settings DNS_DOMAIN_ID
```

ibmcloud cis access-certificates-settings-update

Enterprise Plans Only

Update access certificates settings for a DNS domain.

```
$ ibmcloud cis access-certificates-settings-update DNS_DOMAIN_ID (-f, --feature FEATURE) (-v, --value VALUE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-f, --feature

Feature of certificates settings. Valid values:

client_certificate_forwarding

The client certificate payload and its SHA256 signature are forwarded to origin servers through `CF-Client-Cert-DER_BASE64` and `CF-Client-Cert-SHA256` headers.

-v, --value

The value set to the feature for certificates.

client_certificate_forwarding

Specify the hostname to forward the client certificate or not. For example, `-v host1=on,host2=on,host3=off` .

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update access certificates settings for Domain `31984fea73a15b45779fa0df4ef62f9b` .

```
$ ibmcloud cis access-certificates-settings-update DNS_DOMAIN_ID -f client_certificate_forwarding -v mtls1.example.com=on,mtls2.example.com=off
```

Access policy

ibmcloud cis access-policy-create

Enterprise Plans Only

Create an access policy for an access application.

```
$ ibmcloud cis access-policy-create DNS_DOMAIN_ID ACCESS_APPLICATION_ID --name NAME --decision DECISION --include INCLUDE [-i, --instance INSTANCE] [-output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

ACCESS_APPLICATION_ID

The ID of the access application. Required.

--name

The name of the policy. Required.

--decision

Defines the action Access takes if the policy matches the user. Valid value is `non_identity`. Required.

--include

The included rule of the policy. Valid values are `certificate` and `common_name`. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Create an access policy for access application `a5836c2a7ea72d2e225890caea70ae32`.

```
$ ibmcloud cis access-policy-create DNS_DOMAIN_ID ACCESS_APPLICATION_ID --name examplePolicy --decision non_identity --include certificate --include common_name=test
```

ibmcloud cis access-policies

Enterprise Plans Only

List all access policies for an access application.

```
$ ibmcloud cis access-policies DNS_DOMAIN_ID ACCESS_APPLICATION_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

ACCESS_APPLICATION_ID

The ID of the access application. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

List all access policies for access application `a5836c2a7ea72d2e225890caea70ae32`.

```
$ ibmcloud cis access-policies DNS_DOMAIN_ID ACCESS_APPLICATION_ID
```

ibmcloud cis access-policy

Enterprise Plans Only

Show details of an access policy.


```
$ ibmcloud cis access-policy DNS_DOMAIN_ID ACCESS_APPLICATION_ID ACCESS_POLICY_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

ACCESS_APPLICATION_ID

The ID of the access application. Required.

ACCESS_POLICY_ID

The ID of access policy. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show details of access policy `a5836c2a7ea72d2e225890caea70ae32`.

```
$ ibmcloud cis access-policy DNS_DOMAIN_ID ACCESS_APPLICATION_ID ACCESS_POLICY_ID
```

ibmcloud cis access-policy-delete

Enterprise Plans Only

Delete an access policy.

```
$ ibmcloud cis access-policy-delete DNS_DOMAIN_ID ACCESS_APPLICATION_ID ACCESS_POLICY_ID [-i, --instance INSTANCE]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

ACCESS_APPLICATION_ID

The ID of the access application. Required.

ACCESS_POLICY_ID

The ID of access policy. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete access policy `65fe21071877669cc69544642bc6c4c4`.

```
$ ibmcloud cis access-policy-delete DNS_DOMAIN_ID ACCESS_APPLICATION_ID ACCESS_POLICY_ID
```

Cache

Manipulate how the cache performs by using the following `cache` commands:

ibmcloud cis cache-purge

Clear the cached assets file by file or entirely for a DNS domain to guarantee that the served assets are updated.

```
$ ibmcloud cis cache-purge DNS_DOMAIN_ID (--all | --file file1 --file file2... | --tag tag1 --tag tag2...|--host host1 --host host...| --prefix prefix1 --prefix prefix2...) [-f, --force] [-i, --instance INSTANCE_NAME] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--all

Purging all cached files. This option is mutually exclusive with *--file*.

--file

Granularly remove one or more files by specifying URLs. This option is mutually exclusive with *--all*.

--tag

Granularly remove one or more files by the associated Cache-Tag. This option is mutually exclusive with *--all*. **Enterprise Plans Only**

--host

Granularly remove one or more files by specifying the host. This option is mutually exclusive with *--all*. **Enterprise Plans Only**

--prefix

Granularly remove one or more files by a prefix. This option is mutually exclusive with *--all*. **Enterprise Plans Only**

-f, --force

Purging all cached files without prompting for confirmation.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

Clear all cached assets file for domain **31984fea73a15b45779fa0df4ef62f9b** .

```
$ ibmcloud cis cache-purge DNS_DOMAIN_ID --all --force
```

ibmcloud cis cache-settings

Get caching settings for a DNS domain.

```
$ ibmcloud cis cache-settings DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

Get caching settings for domain **31984fea73a15b45779fa0df4ef62f9b** .

```
$ ibmcloud cis cache-settings DNS_DOMAIN_ID
```

ibmcloud cis cache-settings-update

Update cache settings for a give DNS domain.

```
$ ibmcloud cis cache-settings-update DNS_DOMAIN_ID [--caching-level LEVEL][--browser-expiration EXPIRATION] [--development-mode (on | off)] [--serve-stale-content (on | off)] [--query-string-sort (on | off)] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--caching-level

Specify under what URL conditions that you want to deliver cached assets to the user. Valid values are `no-query-string`, `query-string-independent`, and `query-string-dependent`.

- `no-query-string` : Delivers resources from cache only when no query string is present.
- `query-string-independent` : Delivers the same resource to everyone independent of the query string.
- `query-string-dependent` : Delivers a different resource each time the query string changes.

--browser-expiration

Specify how long you want the browser to store cached assets.

- Valid values are `respect-existing-header`, `30s`, `1M`, `5M`, `20M`, `30M`, `1h`, `2h`, `4h`, `8h`, `16h`, `1d`, `3d`, `8d`, `16d`, `1m`, `6m`, and `1y`.
- `30s`, `1M`, `5M`, and `20M` are only available for an Enterprise or Security plan instance.
- `30s` means `30 seconds`.
- `30M` means `30 minutes`.
- `1h` means `1 hour`.
- `1d` means `1 day`.
- `1m` means `1 month`.
- `1y` means `1 year`.

--development-mod

Bypass all edge caches and send traffic toward your origin servers.

--serve-stale-content

Continue serving cached content to users when origin servers are offline, even if the content is expired.

`--query-string-sort` : In the cache, CIS treats files with the same query strings as the same file, regardless of the order of the query strings.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update caching settings for domain `31984fea73a15b45779fa0df4ef62f9b`.

```
$ ibmcloud cis cache-settings-update DNS_DOMAIN_ID --caching-level no-query-string --browser-expiration 1h
```

Custom lists

Manipulate how the custom list performs by using the following `custom-lists` commands:

ibmcloud cis custom-lists lists

List the custom lists for your instance.

```
$ ibmcloud cis custom-lists lists [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Example

```
$ ibmcloud cis custom-lists lists
```

ibmcloud cis custom-lists list

Get a custom list for your instance.

```
$ ibmcloud cis custom-lists list LIST_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`LIST_ID`

The ID of the custom list.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Example

```
$ ibmcloud cis custom-lists list LIST_ID --output json
```

ibmcloud cis custom-lists list-create

Create a custom list for your instance.

```
$ ibmcloud cis custom-lists list-create (--kind KIND) (--name NAME) [--description DESCRIPTION] [-i, --instance INSTANCE]
```

You can also accept JSON input (from a file or directly as a string):

```
$ ibmcloud cis custom-lists list-create (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE]
```

Command options

`--kind`

Custom list kind. Valid values are `ip`, `asn`, and `hostname`.

`--name`

The list name.

`--description`

Description of the list.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

`--json`

The JSON file or JSON string that is used to describe a custom list.

The required fields in JSON data are:

`kind` : Custom list kind. Valid values are `ip`, `asn`, and `hostname` .

`name` : The list name.

The optional field is:

`"description"` : Description of the list.

Sample JSON data:

```
{
  "kind": "ip",
  "name": "string",
  "description": "string"
}
```

Examples

```
$ ibmcloud cis custom-lists list-create --kind ip --name iplistone --instance INSTANCE
```

```
$ ibmcloud cis custom-lists list-create --json @example.json --instance INSTANCE
```

ibmcloud cis custom-lists list-update

Update the description of a custom list.

```
$ ibmcloud cis custom-lists list-update LIST_ID (--description DESCRIPTION) [-i, --instance INSTANCE] [--output FORMAT]
```

```
$ ibmcloud cis custom-lists list-update LIST_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`LIST_ID`

The ID of the custom list.

`--description`

Briefly describe the list.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

`json`

The JSON file or JSON string that is used to describe a custom list.

The optional field is:

"description" : To briefly describe the list.

Sample JSON data:

```
{
  "description": "string"
}
```

Examples

```
$ ibmcloud cis custom-lists list-update LIST_ID --description "description example"
```

```
$ ibmcloud cis custom-lists list-update LIST_ID --json @example.json
```

ibmcloud cis custom-lists list-delete

Delete a custom list for your instance.

```
$ ibmcloud cis custom-lists list-delete LIST_ID [-f, --force] [-i, --instance INSTANCE]
```

Command options

LIST_ID

The ID of the custom list.

-f, --force

Attempt to delete a custom list without prompting for confirmation.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

Example

```
$ ibmcloud cis custom-lists list-delete LIST_ID
```

ibmcloud cis custom-lists items

```
$ ibmcloud cis custom-lists items LIST_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

LIST_ID

The ID of the custom list.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Example

```
$ ibmcloud cis custom-lists items LIST_ID
```

ibmcloud cis custom-lists item

View a specific item in a custom list.

```
$ ibmcloud cis custom-lists item LIST_ID ITEM_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

LIST_ID

The ID of the custom list.

ITEM_ID

The ID of the custom list item.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Example

```
$ ibmcloud cis custom-lists item LIST_ID ITEM_ID
```

ibmcloud cis custom-lists item-create

Create a new item in a custom list.

```
$ ibmcloud cis custom-lists item-create LIST_ID (--asn ASN | --ip IP | --hostname HOSTNAME) [--comment COMMENT] [-i, --instance INSTANCE]
```

```
$ ibmcloud cis custom-lists item-create LIST_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE]
```

Command options

LIST_ID

The ID of the custom list.

--asn

The ASN value.

--ip

The IPv4 address.

--hostname

The hostname.

--comment

To provide a brief comment on the item.

-i, --instance

Instance name or ID. If instance or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

--json

The JSON file or JSON string that is used to describe a custom list.

The required fields in JSON data are:

`items` : List of custom list items to create.

`asn` : The ASN.

`hostname` : The hostname.

`ip` : The IPv4 address.

`comment` : To provide a brief comment on the item.

Sample JSON data:

```
[
  {
    "asn": 19604,
    "comment": "My list of developer IPs.",
    "hostname": "cloud.ibm.com",
    "ip": "172.64.0.0/13"
  }
]
```

Command examples

```
$ ibmcloud cis custom-lists item-create LIST_ID --ip 192.0.0.3
```

```
$ ibmcloud cis custom-lists item-create LIST_ID --json @example.json
```

ibmcloud cis custom-lists item-update

Update all list items for your custom list.

```
$ ibmcloud cis custom-lists item-update LIST_ID (--json @JSON_FILE | JSON_STRING) [-f, --force] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`LIST_ID`

The ID of the custom list.

`--json`

The JSON file or JSON string that is used to describe a custom list.

The required fields in JSON data are:

`items` : List of custom list items to create.

`asn` : The ASN value.

`hostname` : The hostname.

`ip` : The IPv4 address.

`comment` : To provide a brief comment on the item.

Sample JSON data:

```
[
  {
    "asn": 19604,
    "comment": "My list of developer IPs.",
    "hostname": "cloud.ibm.com",
    "ip": "172.64.0.0/13"
  }
]
```

`-f, --force`

Attempt to delete a custom list without prompting for confirmation.

`--output value`

The output format. Currently, `json` is the only supported value.

`-i, --instance`

Instance name or ID. If instance value or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

Example

```
$ ibmcloud cis custom-lists item-update LIST_ID --json @example.json -f
```

ibmcloud cis custom-lists item-delete

Delete an item from a custom list.

```
$ ibmcloud cis custom-lists item-delete LIST_ID (--item-id CUSTOM_LIST_ITEM_ID) [-f, --force] [-i, --instance INSTANCE]
```

```
$ ibmcloud cis custom-lists item-delete LIST_ID (--json @JSON_FILE | JSON_STRING) [-f, --force] [-i, --instance INSTANCE]
```

Command options

`LIST_ID`

The ID of the custom list.

`--item-id`

`CUSTOM_LIST_ITEM_ID` is the ID of the custom list item.

`-f, --force`

Attempt to delete a custom list without prompting for confirmation.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

`--json`

The JSON file or JSON string that is used to describe a custom list.

The required fields in JSON data are:

`items` : List of custom list items to delete by ID.

`id` : Unique ID of the custom list item.

Sample JSON data:

```
{
  "items": [
    {
      "id": "70c2009751b24ffc9ed1ab462ba957b4"
    }
  ]
}
```

Examples

```
$ ibmcloud cis custom-lists item-delete LIST_ID --item-id CUSTOM_LIST_ITEM_ID --force
```

```
$ ibmcloud cis custom-lists item-delete LIST_ID --json @example.json --force
```

ibmcloud cis custom-lists operation

Get the status for the custom list operation.

```
$ ibmcloud cis custom-lists operation OPERATION_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

OPERATION_ID

The ID of the custom list operation.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Example

```
$ ibmcloud cis custom-lists operation OPERATION_ID --output json
```

Custom page

Manipulate how the Custom Page performs by using the following `custom-page` commands:

ibmcloud cis custom-page-update

Update a specific custom page.

```
$ ibmcloud cis custom-page-update PAGE_ID PAGE_URL [-d, --domain DNS_DOMAIN_ID] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

PAGE_ID

The name of the Custom Page type. Valid values are `basic_challenge`, `country_challenge`, `ip_block`, `ratelimit_block`, `serve_stale_content`, `under_attack`, `waf_block`, `waf_challenge`, `1000_errors`, `500_errors`. Required.

PAGE_URL

A URL that is associated with the Custom Page. For example, `http://www.example.com/example.html`. Value `default` means to use the default page. Required.

-d, --domain

DNS Domain ID.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update `basic_challenge` page for domain `31984fea73a15b45779fa0df4ef62f9b`.

```
$ ibmcloud cis custom-page-update basic_challenge http://www.example.com/example.html -d DNS_DOMAIN_ID
```

ibmcloud cis custom-page

Retrieve a specific custom page.

```
$ ibmcloud cis custom-page PAGE_ID [-d, --domain DNS_DOMAIN_ID] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

PAGE_ID

The name of the Custom Page type. Valid values are `basic_challenge` , `country_challenge` , `ip_block` , `ratelimit_block` , `serve_stale_content` , `under_attack` , `waf_block` , `waf_challenge` , `1000_errors` , and `500_errors` . Required.

-d, --domain

DNS Domain ID.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Get `basic_challenge` page for domain `31984fea73a15b45779fa0df4ef62f9b` .

```
$ ibmcloud cis custom-page basic_challenge -d DNS_DOMAIN_ID
```

ibmcloud cis custom-pages

Retrieve a list of currently existing custom pages.

```
$ ibmcloud cis custom-pages [-d, --domain DNS_DOMAIN_ID] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

-d, --domain

DNS Domain ID.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

List existing custom pages for domain `DNS_DOMAIN_ID` .

```
$ ibmcloud cis custom-pages -d DNS_DOMAIN_ID
```

DNS record

Manipulate how the DNS Record performs by using the following `dns-record` commands:

ibmcloud cis dns-record-create

Create a DNS record for a domain of a service instance.

```
$ ibmcloud cis dns-record-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis dns-record-create DNS_DOMAIN_ID --type TYPE --name NAME --content CONTENT [--ttl TTL] [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis dns-record-create DNS_DOMAIN_ID --json-str JSON_STR [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis dns-record-create DNS_DOMAIN_ID --json-file JSON_FILE [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--name

DNS record name.

--type

DNS record type.

--content

DNS record content.

--ttl

Time to live for DNS record. A value of 1 is `automatic`. The default value is `1`.

--proxied

Control whether traffic flows through the security and performance functions on CIS. CIS proxies traffic only for `A`, `AAAA`, and `CNAME` records. Valid values are `true` and `false`.

--json

The JSON file or JSON string that is used to describe a DNS Record. Supported DNS Record types are: `A`, `AAAA`, `CNAME`, `NS`, `TXT`, `MX`, `LOC`, `SRV`, `CAA`, `PTR`.

- For type `A`, `AAAA`, `CNAME`, `NS`, `TXT` :
 - The required fields in JSON data are `name`, `type`, `content`.
 - The optional fields are `ttl`, `proxied` :
 - `proxied` Control whether traffic flows through the security and performance functions on CIS. CIS proxies only traffic for `A`, `AAAA`, and `CNAME` records.

Sample JSON data:

```
{
  "name": "testA",
  "type": "A",
  "content": "127.0.0.1",
  "proxied": true
}

{
  "name": "testAAAA",
  "type": "AAAA",
  "content": "2001:0db8:0012:0001:3c5e:7354:0000:5db1",
  "proxied": false
}

{
  "name": "testCNAME",
  "type": "CNAME",
  "content": "example.com"
}

{
  "name": "testNS",
  "type": "NS",
  "content": "ns1.example.com"
}

{
  "name": "testTXT",
  "type": "TXT",
  "content": "text information"
}
```

- For type `PTR` :
 - The required fields in JSON data are `name` , `type` , `content` .
 - The optional field is `ttl` .

Sample JSON data:

```
{
  "name": "1.2.3.4",
  "type": "PTR",
  "content": "abc.test.com"
}
```

- For type `MX` :
 - The required fields in JSON data are `name` , `type` , `content` .
 - The optional fields are `ttl` and `priority` .

Sample JSON data:

```
{
  "name": "testMX",
  "type": "MX",
  "content": "smtp.example.com",
  "priority": 10
}
```

- For type `LOC` :
 - The required fields in JSON data are `name` , `type` , `data` :
 - `data` :
 - `lat_degrees` : Degrees of latitude.
 - `lat_minutes` : Minutes of latitude
 - `lat_seconds` : Seconds of latitude.
 - `lat_direction` : Latitude direction.
 - `long_degrees` : Degrees of longitude.
 - `long_minutes` : Minutes of longitude.
 - `long_seconds` : Seconds of longitude.
 - `long_direction` : Longitude direction.
 - `altitude` : Altitude of location in meters.
 - `size` : Size of location in meters.
 - `precision_horz` : Horizontal precision of location.
 - `precision_vert` : Vertical precision of location.
 - The optional field is `ttl` .

Sample JSON data:

```
{
  "name": "testLOC",
  "type": "LOC",
  "data": {
    "lat_degrees": 45,
    "lat_minutes": 0,
    "lat_seconds": 0,
    "lat_direction": "N",
    "long_degrees": 45,
    "long_minutes": 0,
    "long_seconds": 0,
    "long_direction": "E",
    "altitude": 20,
    "size": 0,
    "precision_horz": 0,
    "precision_vert": 0
  }
}
```

```
}
}
```

- For type `SRV` :
 - The required fields in JSON data are `type` , `data` :
 - `data` : - `service` : A service type, prefixed with an underscore. - `proto` : A valid protocol. - `priority` : Priority. - `weight` : The record weight. - `port` : The port of the service. - `target` : A valid hostname.
 - The optional field is `ttl` .

Sample JSON data:

```
{
  "type": "SRV",
  "data": {
    "service": "_ftp",
    "proto": "_tcp",
    "name": "testSRV",
    "priority": 1,
    "weight": 1,
    "port": 21,
    "target": "example.com"
  }
}
```

- For type `CAA` :
 - The required fields in JSON data are `name` , `type` , and `data` .
 - The optional field is `ttl` .

Sample JSON data:

```
{
  "name": "testCAA.yourdomain.com",
  "type": "CAA",
  "data": {
    "tag": "issue",
    "value": "letsencrypt.org"
  }
}
```

`-s, --json-str`

Deprecated. The JSON data used to describe a DNS Record.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a DNS record in the domain `DNS_DOMAIN_ID` .

```
ibmcloud cis dns-record-create DNS_DOMAIN_ID --json '{"name": "testCNAME", "type": "CNAME", "content": "example.com"}'
ibmcloud cis dns-record-create DNS_DOMAIN_ID --type A --name testA --content "127.0.0.1"
```

ibmcloud cis dns-record-update

Update a DNS record for a domain of a service instance.

```
$ ibmcloud cis dns-record-update DNS_DOMAIN_ID DNS_RECORD_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

```
ibmcloud cis dns-record-update DNS_DOMAIN_ID DNS_RECORD_ID [--type TYPE] [--name NAME] [--content CONTENT] [--proxied PROXIED] [--ttl TTL] [--output FORMAT]
[Deprecated] ibmcloud cis dns-record-update DNS_DOMAIN_ID DNS_RECORD_ID --json-str JSON_STR [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis dns-record-update DNS_DOMAIN_ID DNS_RECORD_ID --json-file JSON_FILE [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

DNS_RECORD_ID

The ID of the DNS record. Required.

--name

DNS record name.

--type

DNS record type.

--content

DNS record content.

--ttl

Time to live for DNS record. A value of 1 is **automatic**. The default value is **1**.

--proxied

Control whether traffic flows through the security and performance functions on CIS. CIS proxies traffic only for **A**, **AAAA**, and **CNAME** records. Valid values are **true** and **false**.

--json

The JSON file or JSON string that is used to describe a DNS Record. Supported DNS Record types are: **A**, **AAAA**, **CNAME**, **NS**, **TXT**, **MX**, **LOC**, **SRV**, **CAA**, **PTR**.

- For type **A**, **AAAA**, **CNAME**, **NS**, **TXT** :
 - The required fields in JSON data are **name**, **type**, **content**.
 - The optional fields are **ttl** and **proxied** :
 - **proxied** Control whether traffic flows through the security and performance functions on CIS. CIS proxies only traffic for **A**, **AAAA**, and **CNAME** records.

Sample JSON data:

```
{
  "name": "testA",
  "type": "A",
  "content": "127.0.0.1",
  "proxied": true
}

{
  "name": "testAAAA",
  "type": "AAAA",
  "content": "2001:0db8:0012:0001:3c5e:7354:0000:5db1",
  "proxied": false
}

{
  "name": "testCNAME",
  "type": "CNAME",
  "content": "example.com"
}
```

```
{
  "name": "testNS",
  "type": "NS",
  "content": "ns1.example.com"
}

{
  "name": "testTXT",
  "type": "TXT",
  "content": "text information"
}
```

- For type **PTR** :
 - The required fields in JSON data are **name** , **type** , **content** .
 - The optional field is **ttl** .

Sample JSON data:

```
{
  "name": "1.2.3.4",
  "type": "PTR",
  "content": "abc.test.com"
}
```

- For type **MX** :
 - The required fields in JSON data are **name** , **type** , **content** .
 - The optional fields are **ttl** and **priority** .

Sample JSON data:

```
{
  "name": "testMX",
  "type": "MX",
  "content": "smtp.example.com",
  "priority": 10
}
```

- For type **LOC** :
 - The required fields in JSON data are **name** , **type** , **data** :
 - **data** :
 - **lat_degrees** : Degrees of latitude.
 - **lat_minutes** : Minutes of latitude
 - **lat_seconds** : Seconds of latitude.
 - **lat_direction** : Latitude direction.
 - **long_degrees** : Degrees of longitude.
 - **long_minutes** : Minutes of longitude.
 - **long_seconds** : Seconds of longitude.
 - **long_direction** : Longitude direction.
 - **altitude** : Altitude of location in meters.
 - **size** : Size of location in meters.
 - **precision_horz** : Horizontal precision of location.
 - **precision_vert** : Vertical precision of location.
 - The optional field is **ttl** .

Sample JSON data:

```
{
  "name": "testLOC",
  "type": "LOC",
  "data": {
```



```

    "lat_degrees": 45,
    "lat_minutes": 0,
    "lat_seconds": 0,
    "lat_direction": "N",
    "long_degrees": 45,
    "long_minutes": 0,
    "long_seconds": 0,
    "long_direction": "E",
    "altitude": 20,
    "size": 0,
    "precision_horz": 0,
    "precision_vert": 0
  }
}

```

- For type `SRV` :
 - The required fields in JSON data are `type` , `data` :
 - `data` :
 - `service` : A service type, prefixed with an underscore.
 - `proto` : A valid protocol.
 - `priority` : Priority.
 - `weight` : The record weight.
 - `port` : The port of the service.
 - `target` : A valid hostname.
 - The optional field is `ttl` .

Sample JSON data:

```

{
  "type": "SRV",
  "data": {
    "service": "_ftp",
    "proto": "_tcp",
    "name": "testSRV",
    "priority": 1,
    "weight": 1,
    "port": 21,
    "target": "example.com"
  }
}

```

- For type `CAA` :
 - The required fields in JSON data are `name` , `type` , `data`
 - The optional field is `ttl` .

Sample JSON data:

```

{
  "name": "testCAA.yourdomain.com",
  "type": "CAA",
  "data": {
    "tag": "issue",
    "value": "letsencrypt.org"
  }
}

```

`-s, --json-str`

Deprecated. The JSON data used to describe a DNS Record.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update a DNS record in the domain `DNS_DOMAIN_ID`.

```
ibmcloud cis dns-record-update DNS_DOMAIN_ID DNS_RECORD_ID --json '{"name": "testCNAME", "type": "CNAME", "content": "example.com"}'
ibmcloud cis dns-record-update DNS_DOMAIN_ID DNS_RECORD_ID --type A --name testA --content "127.0.0.1"
```

ibmcloud cis dns-record

Get a DNS record details for a domain under a service instance.

```
$ ibmcloud cis dns-record DNS_DOMAIN_ID DNS_RECORD_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`DNS_RECORD_ID`

The ID of the DNS record. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get DNS record details in the domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis dns-record DNS_DOMAIN_ID DNS_RECORD_ID
```

ibmcloud cis dns-record-delete

Delete a DNS record for a domain of a service instance.

```
$ ibmcloud cis dns-record-delete DNS_DOMAIN_ID DNS_RECORD_ID [-i, --instance INSTANCE]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`DNS_RECORD_ID`

The ID of the DNS record. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete a DNS record in the domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis dns-record-delete DNS_DOMAIN_ID DNS_RECORD_ID
```

ibmcloud cis dns-records

List all DNS records for a domain of a service instance.

```
$ ibmcloud cis dns-records DNS_DOMAIN_ID [--type TYPE] [--name NAME] [--content CONTENT] [--page PAGE] [--per-page PER_PAGE] [--order ORDER] [--direction DIRECTION] [--match MATCH] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--type

Type of DNS records to display.

--name

Value of name field to filter by.

--content

Value of content field to filter by.

--page

Page number of paginated results.

--per_page

Maximum number of DNS records per page.

--order

Field by which to order the list of DNS records. Valid values are **type**, **name**, **content**, **ttl**, and **proxied**.

--direction

Direction in which to order the results (ascending or descending order). Valid values are **asc** and **desc**.

--match

Whether to match all or at least one search parameter. Valid values are **any** and **all**.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

List all DNS records in domain **DNS_DOMAIN_ID**.

```
$ ibmcloud cis dns-records DNS_DOMAIN_ID
```

ibmcloud cis dns-records-import

Import your BIND config.

```
$ ibmcloud cis dns-records-import DNS_DOMAIN_ID --file FILE [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--file

BIND config to import. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Import BIND config in the domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis dns-records-import DNS_DOMAIN_ID --file bind_config_file.txt
```

ibmcloud cis dns-records-export

Export BIND config.

```
$ ibmcloud cis dns-records-export DNS_DOMAIN_ID [--file FILE] [-i, --instance INSTANCE]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--file`

The BIND config file that saves exported DNS records.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Export BIND config for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis dns-records-export DNS_DOMAIN_ID --file bind_config_file.txt
```

Domain

Manipulate domains by using the following `domain` commands.

ibmcloud cis domain-add

Add a domain.

```
$ ibmcloud cis domain-add DNS_DOMAIN_NAME [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`type`

Specify the domain type setup. Valid values are `full` and `partial` (default `full`).

- `full` : A full zone implies that the DNS is hosted.
- `partial` : A partial zone implies a CNAME setup domain.

`jump-start`

Automatically attempt to fetch existing DNS records.

`DNS_DOMAIN_NAME`

The FQDN of DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Add a domain `DNS_DOMAIN_NAME`.

```
$ ibmcloud cis domain-add DNS_DOMAIN_NAME
```

ibmcloud cis domain-resume

Resume the domain.

```
$ ibmcloud cis domain-resume DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Resume the specified domain.

```
$ ibmcloud cis domain-resume DNS_DOMAIN_ID
```

ibmcloud cis domain-pause

Pause the domain.

```
$ ibmcloud cis domain-pause DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Pause the specified domain.

```
$ ibmcloud cis domain-pause DNS_DOMAIN_ID
```

ibmcloud cis domain

Display the domain details.

```
$ ibmcloud cis domain DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Display the specified domain details.

```
$ ibmcloud cis domain DNS_DOMAIN_ID
```

ibmcloud cis domain-remove

Remove a domain.

```
$ ibmcloud cis domain-remove DNS_DOMAIN_ID [-i, --instance INSTANCE]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Remove the specified domain.

```
$ ibmcloud cis domain-remove DNS_DOMAIN_ID
```

ibmcloud cis domains

List domains for a service instance.

```
$ ibmcloud cis domains [--instance INSTANCE_NAME] [--output FORMAT]
```

Command options

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

List domains for the service instance.

```
$ ibmcloud cis domains
```

ibmcloud cis domain-activation-check

Check the activation on the domain.

```
$ ibmcloud cis domain-activation-check DNS_DOMAIN_ID [-i, --instance INSTANCE]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Perform activation check on the specified domain.

```
$ ibmcloud cis domain-activation-check DNS_DOMAIN_ID
```

Domain settings

Manipulate domain settings by using the following `domain-settings` commands:

ibmcloud cis domain-settings

Get details of a feature for the domain.

```
$ ibmcloud cis domain-settings DNS_DOMAIN_ID [-g, --group GROUP | -f, --feature FEATURE] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-g, --group

Display features in a same group. Valid values for `group` are `all`, `domain`, `reliability`, `performance`, and `security`. This option is mutually exclusive with `-f, --feature`.

-f, --feature

Feature of domain settings to check. This option is mutually exclusive with `g, --group`. Valid values are as follows:

- `always_use_https` : Redirect all requests with scheme `http` to `https`. This setting applies to all HTTP requests to the domain.
- `automatic_https_rewrites` : Help fix mixed content by changing `http` to `https` for all resources or links on your website that can be served with HTTPS.
- `bot_management` : Detect and mitigate bot traffic on your domain.
- `brotli` : When the client that is requesting an asset supports the brotli compression algorithm, CIS serves a brotli compressed version of the asset.
- `browser_check` : Evaluate HTTP headers from your visitors' browser for threats. If a threat is found, then a block page is delivered.
- `challenge_ttl` : Specify how long a visitor with a bad IP reputation is allowed access to your website after they complete a challenge.
- `ciphers` : An allowlist of ciphers for TLS termination in the BoringSSL format. This command lists ciphers that are allowlisted by

customers. If no ciphers are allowlisted, the list is empty and the default ciphers are used. See [Edge cipher suites](#) and [Origin cipher suites](#) for the list of default ciphers.

- `cname_flattening` : Follow a CNAME to where it points and return that IP address instead of the CNAME record. By default, flatten only the CNAME at the root of your domain.
- `domain_hold` : Domain holds prevent teams in your organization from adding domains that are already active in another account.
Enterprise Plans Only
- `email_obfuscation` : Encrypt the email addresses on your web page from bots while it's kept visible to humans.
- `opportunistic_onion` : Allow legitimate users of the Tor Browser to access your websites.
- `hotlink_protection` : Protect your images from off-site linking.
- `http2` : Accelerate your website with HTTP/2.
- `http3` : Accelerate your website with HTTP/3.
- `image_load_optimization` : *Deprecated*. Improve load time for pages that include images on mobile devices with slow network connections.
- `image_size_optimization` : Improve image load time by optimizing images hosted on your domain.
- `image_resizing` : Provide on-demand resizing, conversion, and optimization for images served through the CIS network.
- `ip_geolocation` : Include the country code of the visitor location with all requests to your website.
- `ipv6` : Enable IPv6 support and gateway.
- `max_upload` : The number of data visitors who can upload to your website in a single request.
- `min_tls_version` : Allow only HTTPS connections from visitors that support the selected TLS protocol version or newer.
- `minify` : Reduce the file size of source code on your website.
- `mobile_redirect` : Redirect visitors that are using mobile devices to a mobile-optimized website.
- `opportunistic_encryption` : Opportunistic Encryption allows browsers to benefit from the improved performance of HTTP/2 by letting them know that your site is available over an encrypted connection.
- `origin_error_page_pass_thru` : When the Origin Error Page is set to `On`, CIS proxies the 502 and 504 error pages directly from the origin.
Enterprise Plans Only
- `origin_max_http_version` : Configure the HTTP version to Origin.
- `origin_post_quantum_encryption` : Instructs CIS to use Post-Quantum (PQ) key agreement algorithms when it connects to your origin.
- `prefetch_preload` : CIS prefetches any URLs included in the prefetch HTTP header. **Enterprise Plans Only**
- `pseudo_ipv4` : Adds an IPv4 header to requests when a client is using IPv6, but the server only supports IPv4.
- `response_buffering` : Enable or disable buffering of responses from the origin server. **Enterprise Plans Only**
- `script_load_optimization` : Improve the paint time for pages that include JavaScript.
- `security_header` : Enforce a web security policy for your website.
- `security_level` : Choose the appropriate security profile for your website.
- `server_side_exclude` : Automatically hide specific content from suspicious visitors.
- `tls_client_auth` : TLS client certificate presented for authentication on origin pull. **Enterprise Plans Only**
- `true_client_ip_header` : CIS sends the user's IP address in the True-Client-IP header. **Enterprise Plans Only**
- `waf` : A Web Application Firewall (WAF) blocks requests that contain malicious content.
- `websockets` : Allow WebSockets connections to your origin server.
- `proxy_read_timeout` : Maximum time between two read operations from origin. **Enterprise Plans Only**
- `url_normalization` : Modify the URLs of incoming requests.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get `ciphers` settings for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis domain-settings DNS_DOMAIN_ID -f ciphers
```

`ibmcloud cis domain-settings-update`

Update a feature for the domain.

```
$ ibmcloud cis domain-settings-update DNS_DOMAIN_ID (-f, --feature FEATURE) (-v, --value VALUE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-f, --feature

Feature of domain settings to update. Required. Valid values:

- **always_use_https** : Redirect all requests with scheme **http** to **https**. This redirect applies to all http requests to the domain.
- **automatic_https_rewrites** : Help fix mixed content by changing **http** to **https** for all resources or links on your website that can be served with HTTPS.
- **bot_management** : Detect and mitigate bot traffic on your domain.
- **brotli** : When the client that is requesting an asset supports the brotli compression algorithm, CIS serves a brotli compressed version of the asset.
- **browser_check** : Evaluate HTTP headers from your visitors' browser for threats. If a threat is found, then a block page is delivered.
- **challenge_ttl** : Specify how long a visitor with a bad IP reputation is allowed access to your website after they complete a challenge.
- **ciphers** : An allowlist of ciphers for TLS termination. These ciphers must be in the BoringSSL format.
- **cname_flattening** : Follow a CNAME to where it points and return that IP address instead of the CNAME record. By default, only flatten the CNAME at the root of your domain.
- **domain_hold** : Domain holds prevent teams in your organization from adding domains that are already active in another account.
Enterprise Plans Only
- **email_obfuscation** : Encrypt the email addresses on your web page from bots while it's kept visible to humans.
- **opportunistic_onion** : Allow legitimate users of the Tor Browser to access your websites.
- **hotlink_protection** : Protect your images from off-site linking.
- **http2** : Accelerate your website with HTTP/2.
- **http3** : Accelerate your website with HTTP/3.
- **image_load_optimization** : *Deprecated*. Improve load time for pages that include images on mobile devices with slow network connections.
- **image_size_optimization** : Improve image load time by optimizing images hosted on your domain.
- **image_resizing** : Provide on-demand resizing, conversion, and optimization for images served through the CIS network.
- **ip_geolocation** : Include the country code of the visitor location with all requests to your website.
- **ipv6** : Enable IPv6 support and gateway.
- **max_upload** : The number of data visitors who can upload to your website in a single request.
- **min_tls_version** : Allow only HTTPS connections from visitors that support the selected TLS protocol version or newer.
- **minify** : Reduce the file size of source code on your website.
- **mobile_redirect** : Redirect visitors that are using mobile devices to a mobile-optimized website.
- **opportunistic_encryption** : Opportunistic Encryption allows browsers to benefit from the improved performance of HTTP/2 by letting them know that your site is available over an encrypted connection.
- **origin_error_page_pass_thru** : When Origin Error Page is set to **On**, CIS proxies the 502 and 504 error pages directly from the origin.
Enterprise Plans Only
- **origin_max_http_version** : Configure the HTTP version to Origin.
- **origin_post_quantum_encryption** : Instructs CIS to use Post-Quantum (PQ) key agreement algorithms when it connects to your origin.
- **prefetch_preload** : CIS prefetches any URLs included in the prefetch HTTP header. **Enterprise Plans Only**
- **pseudo_ipv4** : Adds an IPv4 header to requests when a client is using IPv6, but the server supports IPv4 only.
- **response_buffering** : Enable or disable buffering of responses from the origin server. **Enterprise Plans Only**
- **script_load_optimization** : Improve the paint time for pages that include JavaScript.
- **security_header** : Enforce a web security policy for your website.
- **security_level** : Choose the appropriate security profile for your website.
- **server_side_exclude** : Automatically hide specific content from suspicious visitors.
- **tls_client_auth** : TLS client certificate presented for authentication on origin pull. **Enterprise Plans Only**

- `true_client_ip_header` : CIS sends the user's IP address in the True-Client-IP header. **Enterprise Plans Only**
- `waf` : A Web Application Firewall (WAF) blocks requests that contain malicious content.
- `websockets` : Allow WebSockets connections to your origin server.
- `proxy_read_timeout` : Maximum time between two read operations from origin.
- `url_normalization` : Modify the URLs of incoming requests.

`-v, --value`

The value set to the feature for domain. Required.

- Valid values for `always_use_https` are `on` and `off` .
- Valid values for `automatic_https_rewrites` are `on` and `off` .
- Valid values for `bot_management` are "use_latest_model", "fight_mode", "session_score", "enable_js". For example, `-v fight_mode=true,session_score=true`
 - `use_latest_model` : Whether to enable the latest model version. Valid values for `use_latest_model` are `true` and `false` .
 - `fight_mode` : Whether to enable the fight mode. Valid values for `fight_mode` are `true` and `false` .
 - `session_score` : Whether to enable the session score. Valid values for `session_score` are `true` and `false` .
 - `enable_js` : Whether to enable JavaScript detections. Valid values for `enable_js` are `true` and `false` .
- Valid values for `browser_check` are `on` and `off` .
- Valid values for `challenge_ttl` are `300, 900, 1800, 2700, 3600, 7200, 10800, 14400, 28800, 57600, 86400, 604800, 2592000, 31536000` .
- Valid values for `cname_flattening` are `flatten_at_root` , `flatten_all` .
 - `flatten_at_root` : Flatten CNAME at the root domain. This value is the default value.
 - `flatten_all` : Flatten all CNAME records under your domain.
- Valid values for `domain_hold` are `hold` , `include_subdomains` and `hold_after` .
 - `hold` : Whether to enable the domain hold. Valid values for `hold` are `true` and `false` .
 - `include_subdomains` : Whether to enable the domain hold. Valid values for `include_subdomains` are `true` and `false` .
 - `hold_after` : If `hold_after` is provided, the hold is temporarily disabled, then automatically re-enabled by the system at the time specified.

For enable domain and subdomains hold: `-v hold=true,include_subdomains=true` . For disable domain hold: `-v hold=false,hold_after=2023-05-31T15:56:36+00:00` .

- Valid values for `hotlink_protection` are `on` , `off` .
- Valid values for `email_obfuscation` are `on` , `off` .
- Valid values for `opportunistic_onion` are `on` , `off` .
- Valid values for `http2` are `on` , `off` .
- Valid values for `http3` are `on` , `off` .
- *Deprecated*. Valid values for `image_load_optimization` are `on` , `off` .
- Valid values for `image_resizing` are `on` , `off` .
- Valid values for `image_size_optimization` are `off` , `lossless` , `lossy` .
 - `off` : Disable Image Size Optimization.
 - `lossless` : Reduce the size of image files without impacting visual quality.
 - `lossy` : The file size of JPEG images is reduced by using lossy compression, which might reduce visual quality.
- Valid values for `ip_geolocation` are `on` , `off` .
- Valid values for `ipv6` are `on` , `off` .
- Valid values(in MB) for `max_upload` are: `100, 125, 150, 175, 200 and 225, 250, 275, 300, 325, 350, 375, 400, 425, 450, 475, 500` .
Enterprise Plans Only
- Valid values for `min_tls_version` are `1.0` , `1.1` , `1.2` , `1.3` .

- Valid values for `minify` are `css`, `html`, `js`. For example, `-v css=on,html=off,js=on`
 - `css`: Automatically minify all CSS for your website. Valid values for `css` are `on`, `off`.
 - `html`: Automatically minify all HTML for your website. Valid values for `html` are `on`, `off`.
 - `js`: Automatically minify all JS for your website. Valid values for `js` are `on`, `off`.
- Valid values for `mobile_redirect` are `status`, `mobile_subdomain`, `strip_uri`. For example, `-v status=on,mobile_subdomain=m,strip_uri=true`
 - `status`: Whether the mobile redirection is enabled. Valid values for `status` are `on` and `off`.
 - `mobile_subdomain`: Which subdomain prefix you want to redirect visitors on mobile devices to (subdomain must exist).
 - `strip_uri`: Whether to drop the current page path and redirect to the mobile subdomain URL root. Valid values for `strip_uri` are `true`, `false`.
- Valid values for `opportunistic_encryption` are `on`, `off`.
- Valid values for `origin_error_page_pass_thru` are `1`, `2`.
- Valid values for `origin_max_http_version` are `supported`, `preferred`, `off`.
 - `supported`: Post-Quantum algorithms are advertised but only used when requested by the origin.
 - `preferred`: Preferred instructs CIS to opportunistically send a Post-Quantum (PQ) keyshare in the first message to the origin (for fastest connections when the origin supports and prefers PQ).
 - `off`: Post-Quantum algorithms are not advertised.
- Valid values for `origin_post_quantum_encryption` are `on`, `off`.
- Valid values for `brrotli` are `on`, `off`.
- Valid values for `prefetch_preload` are `on`, `off`.
- Valid values for `pseudo_ipv4` are `off`, `add_header`, `overwrite_header`.
 - `off`: Disable Pseudo IPv4.
 - `add_header`: Add an additional Cf-Pseudo-IPv4 header only.
 - `overwrite_header`: Overwrite the existing Cf-Connecting-IP and X-Forwarded-For headers with a pseudo IPv4 address.
- Valid values for `response_buffering` are `on`, `off`.
- Valid values for `script_load_optimization` are `on`, `off`.
- Valid values for `security_header` are `enabled`, `max_age`, `include_subdomains`, `preload`, `nosniff`. For example, `-v enabled=true,max_age=100,include_subdomains=true,preload=true,nosniff=true`
 - `enabled`: Whether the security_header is enabled. Valid values for `enabled` are `true`, `false`.
 - `max_age`: Specify the duration(in seconds) security_header are cached in browsers.
 - `include_subdomains`: Every domain below the domain inherits the same security_header. Valid values for `include_subdomains` are `true`, `false`.
 - `preload`: Whether to permit browsers to preload security_header config. Valid values for `enabled` are `true`, `false`.
 - `nosniff`: Whether to send `X-Content-Type-Options: nosniff` header. Valid values for `nosniff` are `true`, `false`.
- Valid values for `server_level` are `off`, `essentially_off`, `low`, `medium`, `high`, `under_attack`.
- Valid values for `server_side_exclude` are `on`, `off`.
- Valid values for `tls_client_auth` are `on`, `off`.
- Valid values for `true_client_ip_header` are `on`, `off`.
- Valid values for `waf` are `on`, `off`.
- Valid values for `websockets` are `on`, `off`.
- Valid values for `proxy_read_timeout`, 1-6000, default: 100.
- Valid values for `ciphers` are `ECDHE-ECDSA-AES128-GCM-SHA256`, `ECDHE-ECDSA-CHACHA20-POLY1305`, `ECDHE-RSA-AES128-GCM-SHA256`, `ECDHE-RSA-CHACHA20-POLY1305`, `ECDHE-ECDSA-AES128-SHA256`, `ECDHE-ECDSA-AES128-SHA`, `ECDHE-RSA-AES128-SHA256`, `ECDHE-RSA-AES128-SHA`, `AES128-GCM-SHA256`, `AES128-SHA256`, `AES128-SHA`, `ECDHE-ECDSA-AES256-GCM-SHA384`, `ECDHE-ECDSA-AES256-SHA384`, `ECDHE-RSA-AES256-GCM-SHA384`, `ECDHE-RSA-AES256-SHA384`, `ECDHE-RSA-AES256-SHA`, `AES256-`

GCM-SHA384 , AES256-SHA256 , AES256-SHA , DES-CBC3-SHA , default . For example, -v AES256-SHA256,AES256-SHA , using -v default to reset configured cipher suites to the default value.

- Valid values for url_normalization are "type", "scope". For example, -v type=cis,scope=both
 - type : Selects the type of URL normalization that is performed by CIS. Valid values for type are cis , rfc3986 .
 - scope : Configures the scope of the URL normalization. Valid values for scope are both , incoming .

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by ibmcloud cis instance-set INSTANCE is used.

--output

The output format. Currently, json is the only supported value.

Examples

Enable tls_client_auth for domain DNS_DOMAIN_ID .

```
$ ibmcloud cis domain-settings-update DNS_DOMAIN_ID -f tls_client_auth -v on
```

Edge functions

Manipulate how Edge Functions perform by using the following edge-functions commands:

ibmcloud cis edge-functions-actions

List all Edge Functions actions of a service instance.

```
$ ibmcloud cis edge-functions-actions [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by ibmcloud cis instance-set INSTANCE is used.

--output

The output format. Currently, json is the only supported value.

Examples

List all Edge Functions actions.

```
$ ibmcloud cis edge-functions-actions
```

ibmcloud cis edge-functions-action

Show an Edge Functions action of a service instance.

```
$ ibmcloud cis edge-functions-action [--name ACTION_NAME] [-i, --instance INSTANCE]
```

Command options

--name

Action name. Enterprise Plans Only

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by ibmcloud cis instance-set INSTANCE is used.

Examples

Show details of Edge Functions action `ACTION_NAME` .

```
$ ibmcloud cis edge-functions-action --name ACTION_NAME
```

ibmcloud cis edge-functions-action-create

Create an Edge Functions action for a service instance.

```
$ ibmcloud cis edge-functions-action-create [--name ACTION_NAME] (--javascript-str JAVASCRIPT_STR | --javascript-file JAVASCRIPT_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

- `--name`
Action name. Enterprise Plans Only
- `--javascript-str`
JavaScript string. For example, `addEventListener('fetch', event => { event.respondWith(fetch(event.request))})`
- `--javascript-file`
JavaScript file.
- `-i, --instance`
Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.
- `--output`
The output format. Currently, `json` is the only supported value.

Examples

Create an Edge Functions action.

```
$ ibmcloud cis edge-functions-action-create --name ACTION_NAME --javascript-str "addEventListener('fetch', event => { event.respondWith(fetch(event.request)) })"
```

ibmcloud cis edge-functions-action-update

Update an Edge Functions action of a service instance.

```
$ ibmcloud cis edge-functions-action-update (--javascript-str JAVASCRIPT_STR | --javascript-file JAVASCRIPT_FILE) [--name ACTION_NAME] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

- `--name`
Action name. Enterprise Plans Only
- `--javascript-str`
JavaScript string. For example, `addEventListener('fetch', event => { event.respondWith(fetch(event.request))})`
- `--javascript-file`
JavaScript file.
- `-i, --instance`
Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.
- `--output`
The output format. Currently, `json` is the only supported value.

Examples

Update an Edge Functions action.

```
$ ibmcloud cis edge-functions-action-update --name ACTION_NAME --javascript-str "addEventListener('fetch', event => { event.respondWith(fetch(event.request)) })"
```

ibmcloud cis edge-functions-action-delete

Delete an Edge Functions action of a service instance.

```
$ ibmcloud cis edge-functions-action-delete [--name ACTION_NAME] [-i, --instance INSTANCE]
```

Command options

`--name`

Action name. Enterprise Plans Only

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete Edge Functions action `ACTION_NAME`.

```
$ ibmcloud cis edge-functions-action-delete --name ACTION_NAME
```

ibmcloud cis edge-functions-triggers

List all Edge Functions triggers for a domain of a service instance.

```
$ ibmcloud cis edge-functions-triggers DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List all Edge Functions triggers for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis edge-functions-triggers DNS_DOMAIN_ID
```

ibmcloud cis edge-functions-trigger

Show an Edge Functions trigger for a domain of a service instance.

```
$ ibmcloud cis edge-functions-trigger DNS_DOMAIN_ID TRIGGER_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

TRIGGER_ID

The ID of the trigger. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show details of Edge Functions trigger `TRIGGER_ID`.

```
$ ibmcloud cis edge-functions-trigger DNS_DOMAIN_ID TRIGGER_ID
```

ibmcloud cis edge-functions-trigger-create

Create an Edge Functions trigger for a domain of a service instance.

```
$ ibmcloud cis edge-functions-trigger-create DNS_DOMAIN_ID PATTERN_URL [--name ACTION_NAME] [--disable] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

PATTERN_URL

The request URL, which triggers the action. Required.

name

The action name to which the created trigger is attached. Enterprise Plans Only

disable

Disable an Edge Functions trigger.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Create an Edge Functions trigger for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis edge-functions-trigger-create DNS_DOMAIN_ID PATTERN_URL --name ACTION_NAME
```

ibmcloud cis edge-functions-trigger-update

Update an Edge Functions trigger for a domain of a service instance.

```
$ ibmcloud cis edge-functions-trigger-update DNS_DOMAIN_ID TRIGGER_ID PATTERN_URL [--name ACTION_NAME] [--disable] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

TRIGGER_ID

The ID of the trigger. Required.

PATTERN_URL

The request URL, which triggers the action. Required.

name

The action name, which the created trigger is attached to. Enterprise Plans Only

disable

Disable an Edge Functions trigger.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update Edge Functions trigger `TRIGGER_ID`.

```
$ ibmcloud cis edge-functions-trigger-update DNS_DOMAIN_ID TRIGGER_ID PATTERN_URL --name ACTION_NAME
```

ibmcloud cis edge-functions-trigger-delete

Delete an Edge Functions trigger for a domain of a service instance.

```
$ ibmcloud cis edge-functions-trigger-delete DNS_DOMAIN_ID TRIGGER_ID [-i, --instance INSTANCE]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

TRIGGER_ID

The ID of the trigger. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete Edge Functions trigger `TRIGGER_ID`.

```
$ ibmcloud cis edge-functions-trigger-delete DNS_DOMAIN_ID TRIGGER_ID
```

Firewall (Deprecated)



Deprecated: Firewall rules are deprecated. CIS moved existing firewall rules to [WAF custom rules](#). If you haven't migrated to WAF custom rules yet, you might have some invalid configuration that prevents the migration from happening. In this case, contact your account team to get help with the migration to WAF custom rules. For more information, see [Migrating to WAF custom rules](#).

Manipulate firewalls by using the following `firewall` commands.

ibmcloud cis firewall-create

Create a new firewall rule.

```
$ ibmcloud cis firewall-create (-t, --type Type) (--json @JSON_FILE | JSON_STRING) [-d, --domain DNS_DOMAIN_ID] [-i, --instance INSTANCE] [--output FORMAT]
```


[Deprecated] ibmcloud cis firewall-create (-t, --type Type) (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-d, --domain DNS_DOMAIN_ID] [-i, --instance INSTANCE] [--output FORMAT]

Command options

-t, --type

Type of firewall rules to create. Valid values: `access-rules`, `ua-rules`, `lockdowns`. Required.

- `access-rules`: Access Rules are a way to allow, challenge, or block requests to your website. You can apply access rules to one domain only or all domains in the same service instance.
- `ua-rules`: Perform access control when matching the exact UserAgent reported by the client. The access control mechanisms can be defined within a rule to help manage traffic from particular clients. This option enables you to customize the access to your site.
- `lockdowns`: Lock access to URLs in this domain to only permitted addresses or address ranges.

-d, --domain

DNS Domain ID. For `ua-rules` and `lockdowns` type rule, it is a required parameter.

--json

The JSON file or JSON string that is used to describe a firewall rule. Required.

- For `--type access-rules`: The JSON data that describes a firewall access rule as follows.
 - Required fields are `mode`, `configuration`.
 - `mode`: The type of action to perform. Valid values are `block`, `challenge`, `whitelist`, and `js_challenge`.
 - `configuration`: Target/Value pair to use for this rule.
 - `target`: The request property to target. Valid values are `ip`, `ip_range`, `asn`, and `country`.
 - `value`: The value for the selected target.
 - For ip, the value is a valid IP address.
 - For ip_range, the value specifies an ip range that is limited to `/16` and `/24`.
 - For asn, the value is an AS number.
 - For a country, the value is a country code for the country.
 - Option fields are `notes`.
 - `notes`: Some useful information about this rule to help identify the purpose of it.

Sample JSON data:

```
{
  "mode": "block",
  "notes": "This rule is added because of event X that occurred on date xyz",
  "configuration": {
    "target": "ip",
    "value": "127.0.0.1"
  }
}
```

- For `--type ua-rules`: The JSON data that describes a user-agent rule as follows.
 - Required fields are `mode`, `configuration`.
 - `mode`: The type of action to perform. Valid values are `block`, `challenge`, and `js_challenge`.
 - `configuration`: Target/Value pair to use for this rule.
 - `target`: The request property to target. Valid value is `ua`.
 - `value`: The exact UserAgent string to match with this rule.
 - Option fields are `paused`, `description`.
 - `paused`: Whether this rule is currently disabled.
 - `description`: Some useful information about this rule to help identify the purpose of it.

Sample JSON data:

```
{
  "mode": "block",
  "configuration": {
```

```
"target": "ua",
"value": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_12_5) AppleWebKit/603.2.4 (KHTML, like Gecko) Version/10.1.1 Safari/603.2.4"
}
}
```

- For `--type lockdowns` : The JSON data that describes a lockdown rule is as follows.
 - Required fields are `urls` , `configurations` .
 - `urls` : URLs to be included in this rule definition.
 - Wildcards are permitted.
 - The URL pattern entered here is escaped before use.
 - This field limits the URL to simple wildcard patterns.
 - `configurations` : List of IP addresses or CIDR ranges to use for this rule.
 - This field can include any number of ip or ip_range configurations that can access the provided URLs.
 - `target` : The request property to target. Valid values are `ip` , and `ip_range` .
 - `value` : IP addresses or CIDR. If target is `ip` , then value must be an IP address, otherwise CIDR.
 - Option fields are `paused` , `description` .
 - `paused` : Whether this rule is currently disabled.
 - `description` : Some useful information about this rule to help identify the purpose of it.

Sample JSON data:

```
{
  "urls": [
    "api.mysite.com/some/endpoint*"
  ],
  "configurations": [
    {
      "target": "ip",
      "value": "127.0.0.1"
    },
    {
      "target": "ip_range",
      "value": " 2.2.2.0/24"
    }
  ]
}
```

`-s, --json-str`

Deprecated. The JSON data describing a firewall rule.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create firewall rules.

```
ibmcloud cis firewall-create --type access-rules --json '{"mode": "block", "notes": "This rule is added because of event X that occurred on date xyz", "configuration": {"target": "ip", "value": "127.0.0.1"}}' --instance INSTANCE
ibmcloud cis firewall-create --type ua-rules --domain DNS_DOMAIN_ID --json '{"mode": "block", "configuration": {"target": "ua", "value": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_12_5) AppleWebKit/603.2.4 (KHTML, like Gecko) Version/10.1.1 Safari/603.2.4"}}' --instance INSTANCE
ibmcloud cis firewall-create --type lockdowns --domain DNS_DOMAIN_ID --json '{"urls": ["api.mysite.com/some/endpoint*"], "configurations": [{"target": "ip", "value": "127.0.0.1"}, {"target": "ip_range", "value": "2.2.2.0/24"}]}' --instance INSTANCE
```

ibmcloud cis firewall-update

Update a firewall rule.

```
$ ibmcloud cis firewall-update FIREWALL_RULE_ID (-t, --type Type) (--json @JSON_FILE | JSON_STRING) [-d, --domain DNS_DOMAIN_ID] [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis firewall-update FIREWALL_RULE_ID (-t, --type Type) (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-d, --domain DNS_DOMAIN_ID] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

- **FIREWALL_RULE_ID**: The ID of firewall rule. Required.

-t, --type

The type of firewall rule to create. Valid values are `access-rules`, `ua-rules`, and `lockdowns`. Required.

- `access-rules` : Access Rules are a way to allow, challenge, or block requests to your website. You can apply access rules to one domain only or all domains in the same service instance.
- `ua-rules` : Perform access control when matching the exact UserAgent reported by the client. The access control mechanisms can be defined within a rule to help manage traffic from particular clients. This enables you to customize the access to your site.
- `lockdowns` : Lock access to URLs in this domain to only permitted addresses or address ranges.

-d, --domain

DNS Domain ID. For `ua-rules` and `lockdowns` type rule, it is a required parameter.

--json

The JSON file or JSON string that is used to describe a firewall rule. Required.

- For `--type access-rules` : The JSON data that describes a firewall access rule is as follows.
 - Option fields are `mode`, `notes`.
 - `mode` : The type of action to perform. Valid values are `block`, `challenge`, `whitelist`, and `js_challenge`.
 - `notes` : Some useful information about this rule to help identify the purpose of it.

Sample JSON data:

```
{
  "mode": "challenge",
  "notes": "This rule is added because of event X that occurred on date xyz",
}
```

- For `--type ua-rules` : The JSON data that describes a user-agent rule is as follows.
 - Required fields are `mode`, `configuration`.
 - `mode` : The type of action to perform. Valid values are `block`, `challenge`, and `js_challenge`.
 - `configuration` : Target/Value pair to use for this rule.
 - `target` : The request property to target. Valid value is `ua`.
 - `value` : The exact UserAgent string to match with this rule.
 - Option fields are `paused`, `description`.
 - `paused` : Whether this rule is currently disabled.
 - `description` : Some useful information about this rule to help identify the purpose of it.

Sample JSON data:

```
{
  "mode": "block",
  "configuration": {
    "target": "ua",
    "value": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_12_5) AppleWebKit/603.2.4 (KHTML, like Gecko) Version/10.1.1 Safari/603.2.4"
  }
}
```

- For `--type lockdowns` : The JSON data that describes a lockdown rule is as follows.

- Required fields are `urls` , `configurations` .
 - `urls` : URLs to be included in this rule definition.
 - Wildcards are permitted.
 - The URL pattern entered here is escaped before use.
 - This field limits the URL to simple wildcard patterns.
 - `configurations` : List of IP addresses or CIDR ranges to use for this rule.
 - This field can include any number of ip or ip_range configurations that can access the provided URLs.
 - `target` : The request property to target. Valid values are `ip` and `ip_range` .
 - `value` : IP addresses or CIDR. If the target is `ip` , then the value must be an IP addresses, otherwise CIDR.
- Option fields are `paused` , `description` .
 - `paused` : Whether this rule is currently disabled.
 - `description` : Some useful information about this rule to help identify the purpose of it.

Sample JSON data:

```
{
  "urls": [
    "api.mysite.com/some/endpoint*"
  ],
  "configurations": [
    {
      "target": "ip",
      "value": "127.0.0.1"
    },
    {
      "target": "ip_range",
      "value": " 2.2.2.0/24"
    }
  ]
}
```

`-s, --json-str`

Deprecated. The JSON data that describes a firewall rule.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update firewall rules.

```
ibmcloud cis firewall-update FIREWALL_RULE_ID -t access-rules --json '{"mode": "challenge", "notes": "This rule is added because of event X that occurred on date xyz"}'
ibmcloud cis firewall-update FIREWALL_RULE_ID -t ua-rules -d DNS_DOMAIN_ID --json '{"mode": "block", "configuration": {"target": "ua", "value": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_12_5) AppleWebKit/603.2.4 (KHTML, like Gecko) Version/10.1.1 Safari/603.2.4"}'
ibmcloud cis firewall-update FIREWALL_RULE_ID -t lockdowns -d DNS_DOMAIN_ID --json '{"urls": ["api.mysite.com/some/endpoint*"], "configurations": [{"target": "ip", "value": "127.0.0.1"}, {"target": "ip_range", "value": "2.2.2.0/24"}]}'
```

ibmcloud cis firewalls

List firewall rules.

```
$ ibmcloud cis firewalls (-t, --type Type) [-d, --domain DNS_DOMAIN_ID] [--page PAGE] [--per-page PER_PAGE ] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`-t, --type`

The type of firewall rule to create. Valid values are `access-rules`, `ua-rules`, and `lockdowns`. Required.

- `access-rules` : Access Rules are a way to allow, challenge, or block requests to your website. You can apply access rules to one domain only or all domains in the same service instance.
- `ua-rules` : Perform access control when matching the exact UserAgent reported by the client. The access control mechanisms can be defined within a rule to help manage traffic from particular clients. This action enables you to customize the access to your site.
- `lockdowns` : Lock access to URLs in this domain to only permitted addresses or address ranges.

`-d, --domain`

DNS Domain ID. For `ua-rules` and `lockdowns` type rule, it is a required parameter.

`--page`

Page number of paginated results. The default value is `0`.

`--per-page`

Maximum number of access rules per page. The minimum value is `5`. The default value is `20`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List firewall rules.

```
ibmcloud cis firewalls --type access-rules --instance INSTANCE
ibmcloud cis firewalls --type ua-rules --domain DNS_DOMAIN_ID --instance INSTANCE
ibmcloud cis firewalls --type lockdowns --domain DNS_DOMAIN_ID --instance INSTANCE
```

ibmcloud cis firewall

Get details of a firewall rule.

```
$ ibmcloud cis firewall FIREWALL_RULE_ID (-t, --type Type) [-d, --domain DNS_DOMAIN_ID] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`FIREWALL_RULE_ID`

The ID of firewall rule. Required.

`-t, --type`

Type of firewall rule to create. Valid values are `access-rules`, `ua-rules`, and `lockdowns`.

- `access-rules` : Access Rules are a way to allow, challenge, or block requests to your website. You can apply access rules to one domain only or all domains in the same service instance.
- `ua-rules` : Perform access control when matching the exact UserAgent reported by the client. The access control mechanisms can be defined within a rule to help manage traffic from particular clients. This enables you to customize the access to your site.
- `lockdowns` : Lock access to URLs in this domain to only permitted addresses or address ranges.

`-d, --domain`

DNS Domain ID. For `ua-rules` and `lockdowns` type rule, it is a required parameter.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get firewall rule details.

```
ibmcloud cis firewall FIREWALL_RULE_ID -t access-rules
ibmcloud cis firewall FIREWALL_RULE_ID -t access-rules -d DNS_DOMAIN_ID
ibmcloud cis firewall FIREWALL_RULE_ID -t ua-rules -d DNS_DOMAIN_ID
ibmcloud cis firewall FIREWALL_RULE_ID -t lockdowns -d DNS_DOMAIN_ID
```

ibmcloud cis firewall-delete

Delete a firewall rule by ID.

```
$ ibmcloud cis firewall-delete FIREWALL_RULE_ID (-t, --type Type) [-d, --domain DNS_DOMAIN_ID] [-i, --instance INSTANCE]
```

Command options

FIREWALL_RULE_ID

The ID of firewall rule. Required.

-t, --type

Type of firewall rule to create. Valid values are `access-rules`, `ua-rules`, and `lockdowns`. Required.

- `access-rules` : Access Rules are a way to allow, challenge, or block requests to your website. You can apply access rules to one domain only or all domains in the same service instance.
- `ua-rules` : Perform access control when matching the exact UserAgent reported by the client. The access control mechanisms can be defined within a rule to help manage traffic from particular clients. This field enables you to customize the access to your site.
- `lockdowns` : Lock access to URLs in this domain to only permitted addresses or address ranges.

-d, --domain

DNS Domain ID. For `ua-rules` and `lockdowns` type rule, it is a required parameter.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete a firewall rule.

```
$ ibmcloud cis firewall-delete FIREWALL_RULE_ID -t access-rules
ibmcloud cis firewall-delete FIREWALL_RULE_ID -t access-rules -d DNS_DOMAIN_ID
ibmcloud cis firewall-delete FIREWALL_RULE_ID -t ua-rules -d DNS_DOMAIN_ID
ibmcloud cis firewall-delete FIREWALL_RULE_ID -t lockdowns -d DNS_DOMAIN_ID
```

Firewall rules

Manipulate how firewall rules perform by using the following `firewall-rules` commands:

ibmcloud cis firewall-rules

Retrieve a list of currently existing firewall-rules for a DNS domain.

```
$ ibmcloud cis firewall-rules DNS_DOMAIN_ID [--page PAGE] [--per-page PER_PAGE] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--page

Page number of paginated results. The default value is `1`.

`--per-page`

Number of firewall rules per page. The minimum value is `5` and the maximum value is `100`. The default value is `25`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List existing firewall-rules in the domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis firewall-rules DNS_DOMAIN_ID
```

ibmcloud cis firewall-rule

Retrieve a specific firewall-rule for a DNS domain.

```
$ ibmcloud cis firewall-rule DNS_DOMAIN_ID FIREWALL_RULE_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`FIREWALL_RULE_ID`

The ID of firewall-rule. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get the details of firewall-rule `FIREWALL_RULE_ID`.

```
$ ibmcloud cis firewall-rule DNS_DOMAIN_ID FIREWALL_RULE_ID
```

ibmcloud cis firewall-rule-create

Create a firewall-rule for a DNS domain.

```
$ ibmcloud cis firewall-rule-create DNS_DOMAIN_ID --expression EXPRESSION --action ACTION [--priority PRIORITY] [--paused on|off] [--products PRODUCTS][--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis firewall-rule-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis firewall-rule-create DNS_DOMAIN_ID (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--expression`

A filter expression. For example, `ip.src eq 93.184.216.0`.

`--action`

The rule action to perform. Valid values are `log` , `allow` , `challenge` , `js_challenge` , `block` , and `bypass` .

`--priority`

The rule's priority. Valid values range from `0` to `2147483647` . The value `0` means to set to the default value.

`--description`

To briefly describe the rule.

`--paused`

Indicates whether the rule is active or not. Valid values are `on` and `off` . The default value is `off` .

`--products`

The list of security products to be bypassed. Valid values are `zoneLockdown` , `uaBlock` , `bic` , `hot` , `securityLevel` , `rateLimit` , and `waf` .

`--json`

The JSON file or JSON strin that isused to describe a firewall-rule.

- The required fields in JSON data are `expression` and `action` .
 - `expression` : A filter expression. For example, `ip.src eq 93.184.216.0`
 - `action` : The rule action to perform. Valid values are `log` , `allow` , `challenge` , `js_challenge` , `block` , and `bypass` .
- The optional fields are `description` , `priority` , `paused` , `products` .
 - `description` : To briefly describe the rule.
 - `priority` : The rule's priority. Valid values range from `0` to `2147483647` . The value `0` means to set to the default value.
 - `paused` : Indicates whether the rule is active or not. Valid values are `on` and `off` . The default value is `off` .
 - `products` : The list of security products to be bypassed. Valid values are `zoneLockdown` , `uaBlock` , `bic` , `hot` , `securityLevel` , `rateLimit` , and `waf` . For example, `--products zoneLockdown, rateLimit`

Sample JSON data:

```
{
  "expression": "ip.src eq 93.184.216.1 and http.request.uri.path ~ \"^.*wp-login.php$\"",
  "action": "allow",
  "priority": 100,
  "paused": false,
  "description": "do not challenge login from office"
}
```

`-s, --json-str`

Deprecated. The JSON data that describes a firewall-rule.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a firewall-rule in the domain `DNS_DOMAIN_ID` .

```
ibmcloud cis firewall-rule-create DNS_DOMAIN_ID --expression "ip.src eq 93.184.216.1 and http.request.uri.path ~ \"^.*wp-login.php$\"" --action allow --priority 200 --
paused off --description "do not challenge login from office"
```

```
ibmcloud cis firewall-rule-create DNS_DOMAIN_ID --json '{"expression": "ip.src eq 93.184.216.1 and http.request.uri.path ~ \"^.*wp-login.php$\"", "action": "allow",
```



```
"priority": 100, "paused": false, "description": "do not challenge login from office"}
```

ibmcloud cis firewall-rule-update

Update a specific firewall-rule for a DNS domain.

```
$ ibmcloud cis firewall-rule-update DNS_DOMAIN_ID FIREWALL_RULE_ID [--expression EXPRESSION] [--action ACTION] [--priority PRIORITY] [--paused on|off] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis firewall-rule-update DNS_DOMAIN_ID FIREWALL_RULE_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis firewall-rule-update DNS_DOMAIN_ID FIREWALL_RULE_ID (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-i, --instance INSTANCE] [-output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

FIREWALL_RULE_ID

The ID of firewall-rule. Required.

--expression

A filter expression. For example, `ip.src eq 93.184.216.0`.

--action

The rule action to perform. Valid values are `log`, `allow`, `challenge`, `js_challenge`, `block`, and `bypass`.

--priority

The rule's priority. Valid values range from `0` to `2147483647`. The value `0` means to set to the default value.

--description

To briefly describe the rule.

--paused

Indicates whether the rule is active or not. Valid values are `on` and `off`. The default value is `off`.

--products

The list of security products to be bypassed. Valid values are `zoneLockdown`, `uaBlock`, `bic`, `hot`, `securityLevel`, `rateLimit`, and `waf`.

--json

The JSON file or JSON string that is used to describe a firewall-rule.

- The required fields in JSON data are `expression`, and `action`.
 - `expression` : A filter expression. For example, `ip.src eq 93.184.216.0`
 - `action` : The rule action to perform. Valid values are `log`, `allow`, `challenge`, `js_challenge`, `block`, and `bypass`.
- The optional fields are `description`, `priority`, `paused`, `products`.
 - `description` : To briefly describe the rule.
 - `priority` : The rule's priority. Valid values range from `0` to `2147483647`. The value `0` means to set to the default value.
 - `paused` : Indicates whether the rule is active or not. Valid values are `on` and `off`. The default value is `off`.
 - `products` : The list of security products to be bypassed. Valid values are `zoneLockdown`, `uaBlock`, `bic`, `hot`, `securityLevel`, `rateLimit`, and `waf` For example, `--products zoneLockdown, rateLimit`



Note: Fields `description`, `priority`, and `paused`, which aren't explicitly set in JSON data are overwritten by the default value.

Sample JSON data:

```
{
  "expression": "ip.src eq 93.184.216.1 and http.request.uri.path ~ \"^.*wp-login.php$\"",

```

```
"action": "allow",
"priority": 100,
"paused": false,
"description": "do not challenge login from office"
}
```

`-s, --json-str`

Deprecated. The JSON data that describes a firewall-rule.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update firewall-rule `FIREWALL_RULE_ID`.

```
$ ibmcloud cis firewall-rule-update DNS_DOMAIN_ID FIREWALL_RULE_ID --expression "ip.src eq 93.184.216.1 and http.request.uri.path ~ \"^.*wp-login.php$\" --
action allow --priority 200 --paused off --description \"do not challenge login from office\"

ibmcloud cis firewall-rule-update DNS_DOMAIN_ID FIREWALL_RULE_ID --json '{"expression": "ip.src eq 93.184.216.1 and http.request.uri.path ~ \"^.*wp-login.php$\"",
"action": "allow", "priority": 100, "paused": false, "description": "do not challenge login from office"}'
```

ibmcloud cis firewall-rule-delete

Delete a specific firewall-rule for a DNS domain.

```
$ ibmcloud cis firewall-rule-delete DNS_DOMAIN_ID FIREWALL_RULE_ID [-i, --instance INSTANCE]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`FIREWALL_RULE_ID`

The ID of firewall-rule. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete firewall-rule `FIREWALL_RULE_ID`.

```
$ ibmcloud cis firewall-rule-delete DNS_DOMAIN_ID FIREWALL_RULE_ID
```

ibmcloud cis firewall-rule-validate

Validate a firewall-rule expression.

```
$ ibmcloud cis firewall-rule-validate DNS_DOMAIN_ID EXPRESSION [-i, --instance INSTANCE]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

EXPRESSION

The filter expression. For example, `ip.src eq 93.184.216.0` . Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Validate firewall-rule expression.

```
$ ibmcloud cis firewall-rule-validate DNS_DOMAIN_ID "ip.src eq 93.184.216.0"
```

Global load balancer

Manipulate global load balancers by using the following `glb` commands.

ibmcloud cis glb-create

Create a global load balancer under a DNS domain.

```
$ ibmcloud cis glb-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis glb-create DNS_DOMAIN_ID (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--json

The JSON file or JSON string that is used to describe a global load balancer. Required.

- The required fields in JSON data are `name` , `fallback_pool` and `default_pools` :
 - `name` : The DNS hostname to associate with your load balancer.
 - `fallback_pool` : The pool ID to use when all other pools are detected as unhealthy.
 - `default_pools` : A list of pool IDs ordered by their failover priority.
- The optional fields are `description` , `ttl` , `region_pools` , `proxied` , `enabled` , `session_affinity` , `session_affinity_ttl` , `steering_policy` :
 - `description` : The description of your load balancer.
 - `ttl` : Time to live (TTL) of the DNS entry for the IP address returned by this load balancer.
 - `region_pools` : A mapping of region and country codes to a list of pool IDs (ordered by their failover priority) for the region.
 - `proxied` : Control whether traffic should flow through the security and performance functions on CIS.
 - `enabled` : Whether to enable (the default) this load balancer.
 - `session_affinity` : Ensures that a user's requests are consistently directed to the same backend server during a session. Valid values are `cookie` and `none` .
 - `session_affinity_ttl` : Time, in seconds, until this load balancers session affinity cookie expires after it is created. Valid value is between `[1800, 604800]` . The default value is `82800` .
 - `steering_policy` : Valid values for `steering_policy` are `off` , `geo` , `random` , `dynamic_latency` .
 - `off` : Use `default_pools` .
 - `geo` : Use `region_pools/pop_pools` .
 - `random` : Select a pool randomly.
 - `dynamic_latency` : Use round-trip time to select the closest pool in `default_pools` (requires pool health checks).

Sample JSON data:


```
{
  "name": "www.example.com",
  "fallback_pool": "17b5962d775c646f3f9725cbc7a53df4",
  "default_pools": [
    "17b5962d775c646f3f9725cbc7a53df4",
    "9290f38c5d07c2e2f4df57b1f61d4196"
  ],
  "description": "Example global load balancer.",
  "ttl": 60,
  "region_pools": {
    "WNAM": [
      "de90f38ced07c2e2f4df50b1f61d4194",
      "9290f38c5d07c2e2f4df57b1f61d4196"
    ],
    "ENAM": [
      "00920f38ce07c2e2f4df50b1f61d4194"
    ]
  }
}
```

`-s, --json-str`

Deprecated. The JSON data describing a global load balancer.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a global load balancer in the domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis glb-create DNS_DOMAIN_ID --json '{"description":"Example global load balancer.", "name":"www.example.com", "ttl":60, "fallback_pool":"17b5962d775c646f3f9725cbc7a53df4", "default_pools": ["17b5962d775c646f3f9725cbc7a53df4", "9290f38c5d07c2e2f4df57b1f61d4196"], "region_pools":{"WNAM": ["de90f38ced07c2e2f4df50b1f61d4194", "9290f38c5d07c2e2f4df57b1f61d4196"], "ENAM":["00920f38ce07c2e2f4df50b1f61d4194"]}}'
```

ibmcloud cis glb-update

Update a global load balancer under a DNS domain.

```
$ ibmcloud cis glb-update DNS_DOMAIN_ID GLB_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis glb-update DNS_DOMAIN_ID GLB_ID (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`GLB_ID`

The ID of the global load balancer. Required.

`--json`

The JSON file or JSON string that is used to describe a global load balancer. Required.

- The required fields in JSON data are `name`, `fallback_pool`, and `default_pools` :
 - `name` : The DNS hostname to associate with your load balancer.
 - `fallback_pool` : The pool ID to use when all other pools are detected as unhealthy.
 - `default_pools` : A list of pool IDs ordered by their failover priority.

- The optional fields are `description` , `ttl` , `region_pools` , `proxied` , `enabled` , `session_affinity` , `session_affinity_ttl` , `steering_policy` :
 - `description` : The description of your Load Balancer.
 - `ttl` : Time to live (TTL) of the DNS entry for the IP address returned by this load balancer.
 - `region_pools` : A mapping of region and country codes to a list of pool IDs (ordered by their failover priority) for the region.
 - `proxied` : Control whether traffic must flow through the security and performance functions on CIS.
 - `enabled` : Whether to enable (the default) this load balancer.
 - `session_affinity` : Ensures that a user's requests are consistently directed to the same backend server during a session. Valid values are `cookie` and `none` .
 - `session_affinity_ttl` : Time, in seconds, until this load balancers session affinity cookie expires it is created. Valid value is between `[1800, 604800]` . The default value is `82800` .
 - `steering_policy` : Valid values for `steering_policy` are `off` , `geo` , `random` , `dynamic_latency` .
 - `off` : Use `default_pools` .
 - `geo` : Use `region_pools/pop_pools` .
 - `random` : Select a pool randomly.
 - `dynamic_latency` : Use round-trip time to select the closest pool in `default_pools` (requires pool health checks).

Sample JSON data:

```
{
  "name": "www.example.com",
  "fallback_pool": "17b5962d775c646f3f9725cbc7a53df4",
  "default_pools": [
    "17b5962d775c646f3f9725cbc7a53df4",
    "9290f38c5d07c2e2f4df57b1f61d4196"
  ],
  "description": "Example global load balancer.",
  "ttl": 60,
  "region_pools": {
    "WNAM": [
      "de90f38ced07c2e2f4df50b1f61d4194",
      "9290f38c5d07c2e2f4df57b1f61d4196"
    ],
    "ENAM": [
      "00920f38ce07c2e2f4df50b1f61d4194"
    ]
  }
}
```

`-s, --json-str`

Deprecated. The JSON data describing a global load balancer.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update the global load balancer `GLB_ID` in the domain `DNS_DOMAIN_ID` .

```
$ ibmcloud cis glb-update DNS_DOMAIN_ID GLB_ID --json '{"description":"Example global load balancer.", "name":"www.example.com", "ttl":60, "fallback_pool":"17b5962d775c646f3f9725cbc7a53df4", "default_pools": ["17b5962d775c646f3f9725cbc7a53df4", "9290f38c5d07c2e2f4df57b1f61d4196"], "region_pools":{"WNAM": ["de90f38ced07c2e2f4df50b1f61d4194", "9290f38c5d07c2e2f4df57b1f61d4196"], "ENAM":["00920f38ce07c2e2f4df50b1f61d4194"]}}'
```

ibmcloud cis glb

Show a global load balancer under a DNS domain.

```
$ ibmcloud cis glb DNS_DOMAIN_ID GLB_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

GLB_ID

The ID of the global load balancer. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

Show global load balancer **GLB_ID** in domain **DNS_DOMAIN_ID** .

```
$ ibmcloud cis glb DNS_DOMAIN_ID GLB_ID
```

ibmcloud cis glb-delete

Delete a global load balancer under a DNS domain.

```
$ ibmcloud cis glb-delete DNS_DOMAIN_ID GLB_ID [-i, --instance INSTANCE]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

GLB_ID

The ID of the global load balancer. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

Examples

Delete global load balancer **GLB_ID** in domain **DNS_DOMAIN_ID** .

```
$ ibmcloud cis glb-delete DNS_DOMAIN_ID GLB_ID
```

ibmcloud cis glbs

List all load balancers for the domain.

```
$ ibmcloud cis glbs DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List load balancers for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis glbs DNS_DOMAIN_ID
```

ibmcloud cis glb-pools

List all GLB pools for a service instance.

```
$ ibmcloud cis glb-pools [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

Specify the output format. Only JSON is supported.

Examples

List all GLB pools.

```
$ ibmcloud cis glb-pools
```

ibmcloud cis glb-pool-create

Create a GLB pool for a service instance.

```
$ ibmcloud cis glb-pool-create (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis glb-pool-create (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`--json`

The JSON file or JSON string that is used to describe a GLB pool. Required.

- The required fields in JSON data are `name`, `origins` and `check_regions` :
 - `name` : A short name (tag) for the pool.
 - `origins` : A list of origins within this pool.
 - `check_regions` : A list of geographic region code.
- The optional fields are `description`, `minimum_origins`, `enabled`, `monitor`.

Sample JSON data:

```
{
  "name": "us-pool",
  "description": "application server pool in US",
  "origins": [
```

```
{
  "name": "us-app-dal01",
  "address": "1.1.1.1",
  "enabled": true,
  "header": {
    "host": ["test.com"]
  }
},
{
  "name": "us-app-dal02",
  "address": "2.2.2.2",
  "enabled": true,
  "header": {
    "host": ["example.com"]
  }
}
],
"minimum_origins": 1,
"check_regions": [ "WNAM" ],
"monitor": "f1aba936b94213e5b8dca0c0dbf1f9cc",
"enabled": true
}
```

`-s, --json-str`

Deprecated. The JSON data used to describe a GLB pool.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a GLB pool.

```
$ ibmcloud cis glb-pool-create --json '{"description":"application server pool in US", "name":"us-pool", "enabled":true, "check_regions":["WNAM"],
"minimum_origins":1,"monitor":"f1aba936b94213e5b8dca0c0dbf1f9cc", "origins":[{"name":"us-app-dal01","address":"1.1.1.1","enabled":true,"header":{"host":
["test.com"]}}, {"name":"us-app-dal02","address":"2.2.2.2","enabled":true,"header":{"host":["example.com"]}}]}'
```

ibmcloud cis glb-pool

Show the details of a GLB pool.

```
$ ibmcloud cis glb-pool GLB_POOL_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`GLB_POOL_ID`

The ID of the global load balancer pool. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Show the details of the GLB pool `GLB_POOL_ID` .

```
$ ibmcloud cis glb-pool GLB_POOL_ID
```

ibmcloud cis glb-pool-delete

Delete a GLB pool.

```
$ ibmcloud cis glb-pool-delete GLB_POOL_ID [-i, --instance INSTANCE]
```

Command options

`GLB_POOL_ID`

The ID of the global load balancer pool. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete GLB pool `GLB_POOL_ID` .

```
$ ibmcloud cis glb-pool-delete GLB_POOL_ID
```

ibmcloud cis glb-pool-update

Update a GLB pool.

```
$ ibmcloud cis glb-pool-update GLB_POOL_ID [--enable-origin ORIGIN_NAME --enable-origin ORIGIN_NAME ...] [--disable-origin ORIGIN_NAME --disable-origin ORIGIN_NAME ...] [--add-origin ORIGIN_PARAMETER --add-origin ORIGIN_PARAMETER ...] [--remove-origin ORIGIN_NAME --remove-origin ORIGIN_NAME ...] [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis glb-pool-update GLB_POOL_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis glb-pool-update GLB_POOL_ID (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`GLB_POOL_ID`

The ID of the global load balancer pool. Required.

`--json`

The JSON file or JSON string that is used to describe a GLB pool.

- The required fields in JSON data are `name` , `origins` and `check_regions` :
 - `name` : A short name (tag) for the pool.
 - `origins` : A list of origins within this pool.
 - `check_regions` : A list of geographic region code.
- The optional fields are `description` , `minimum_origins` , `enabled` , `monitor` .

Sample JSON data:

```
{
  "name": "us-pool",
  "description": "application server pool in US",
  "origins": [
    {
      "name": "us-app-dal01",
      "address": "1.1.1.1",
      "enabled": true,
      "header": {
        "host": ["example.com"]
      }
    }
  ]
}
```

```
    }
  },
  {
    "name": "us-app-dal02",
    "address": "2.2.2.2",
    "enabled": true
  }
],
"minimum_origins": 1,
"check_regions": [ "WNAM" ],
"monitor": "f1aba936b94213e5b8dca0c0dbf1f9cc",
"enabled": true
}
```

--enable-origin

Enable the origin within the Pool. The value can be ORIGIN_NAME or ORIGIN_ADDRESS.

--disable-origin

Disable the origin within the Pool. The value can be ORIGIN_NAME or ORIGIN_ADDRESS.

--add-origin

Add an origin into the Pool. ORIGIN_NAME and ORIGIN_ADDRESS are required. For example, --add-origin name=us-app-dal01,address=1.1.1.1,enabled=true,weight=0.5,host=example.com

--remove-origin

Remove an origin from the Pool. The value can be ORIGIN_NAME or ORIGIN_ADDRESS.

-s, --json-str

Deprecated. The JSON data used to describe a GLB pool.

-j, --json-file

Deprecated. A file contains input JSON data.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update GLB pool `GLB_POOL_ID`.

```
$ ibmcloud cis glb-pool-update GLB_POOL_ID --json '{"description":"application server pool in US", "name":"us-pool", "enabled":true, "check_regions":["WNAM"], "minimum_origins":1,"monitor":"f1aba936b94213e5b8dca0c0dbf1f9cc", "origins":[{"name":"us-app-dal01","address":"1.1.1.1","enabled":true,"header":{"host":["example.com"]}}, {"name":"us-app-dal02","address":"2.2.2.2","enabled":true}]}'
```

ibmcloud cis glb-monitors

List GLB monitors for a service instance.

```
$ ibmcloud cis glb-monitors [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

List all GLB monitors.

```
$ ibmcloud cis glb-monitors
```

ibmcloud cis glb-monitor-create

Create a GLB monitor for a service instance.

```
$ ibmcloud cis glb-monitor-create (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis glb-monitor-create (-s, --json-str JSON_STR | -, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

--json

The JSON file or JSON string that is used to describe a GLB monitor. Required.

- The required fields in JSON data are `type` .
 - `type` : The protocol to use for the healthcheck. Valid values are `HTTP` , `HTTPS` , and `TCP` .
- The optional fields are `description` , `timeout` , `retries` , `interval` .
 - `description` : Description.
 - `timeout` : The timeout (in seconds) before marking the health check as failed.
 - `retries` : The number of retries to attempt in case of a timeout before marking the origin as unhealthy.
 - `interval` : The interval between each health check.
- For `TCP` type health check. Extra required fields are `port` .
 - `port` : The TCP port to use for the health check.
- For `HTTP/HTTPS` type health check. Extra option fields are `port` , `expected_body` , `expected_codes` , `method` , `path` , `header` , `follow_redirects` , `allow_insecure` .
 - `port` : The TCP port to use for the health check.
 - `expected_body` : A case-insensitive substring to look for in the response body.
 - `expected_codes` : The expected HTTP response code or code range of the health check.
 - `method` : The HTTP method to use for the health check.
 - `path` : The endpoint path to health check against.
 - `header` : The HTTP request headers to send in the health check.
 - `follow_redirects` : Follow redirects if returned by the origin.
 - `allow_insecure` : Do not validate the certificate when monitor use HTTPS.
 - `probe_zone` : Assign this monitor to emulate the specified zone while probing.

Sample JSON data:

For HTTP/HTTPS:

```
{
  "description": "Health monitor of web service",
  "type": "https",
  "method": "GET",
  "path": "/health",
  "header": {
    "Host": [
      "example.com"
    ],
    "X-App-ID": [
      "abc123"
    ]
  },
  "timeout": 5,
  "retries": 2,
  "interval": 90,
```



```
"follow_redirects": true,
"allow_insecure": false,
"expected_codes": "2xx",
"expected_body": "alive",
"probe_zone": "example.com"
}
```

For TCP:

```
{
  "description": "Health monitor of TCP",
  "type": "tcp",
  "port": 80,
  "timeout": 5,
  "retries": 2,
  "interval": 90
}
```

`-s, --json-str`

Deprecated. The JSON data used to describe a GLB monitor.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a GLB monitor.

```
$ ibmcloud cis glb-monitor-create --json '{"type":"https", "description":"Health monitor of web service", "method":"GET", "path":"/health", "header":{"Host":["example.com"],"X-App-ID":["abc123"]}', "port":8080, "timeout":5, "retries":2, "interval":90, "expected_body":"alive", "expected_codes":"2xx", "follow_redirects":true, "allow_insecure":true}'
```

ibmcloud cis glb-monitor

Show the details of a GLB monitor.

```
$ ibmcloud cis glb-monitor GLB_MON_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`GLB_MON_ID`

The ID of the global load balancer monitor. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Show the details of the GLB monitor `GLB_MON_ID`.

```
$ ibmcloud cis glb-monitor GLB_MON_ID
```

ibmcloud cis glb-monitor-delete

Delete the GLB monitor for a service instance.

```
$ ibmcloud cis glb-monitor-delete GLB_MON_ID [-i, --instance INSTANCE]
```

Command options

GLB_MON_ID

The ID of global load balancer monitor. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete the GLB monitor `GLB_MON_ID`.

```
$ ibmcloud cis glb-monitor-delete GLB_MON_ID
```

ibmcloud cis glb-monitor-update

Update the GLB monitor for a service instance.

```
$ ibmcloud cis glb-monitor-update GLB_MON_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis glb-monitor-update GLB_MON_ID (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

GLB_MON_ID

The ID of the global load balancer monitor. Required.

--json

The JSON file or JSON string that is used to describe a GLB monitor. Required.

- The required fields in JSON data are `type`.
 - `type` : The protocol to use for the healthcheck. Valid values are `HTTP`, `HTTPS`, and `TCP`.
- The optional fields are `description`, `timeout`, `retries`, `interval`.
 - `description` : Description.
 - `timeout` : The timeout (in seconds) before marking the health check as failed.
 - `retries` : The number of retries to attempt in case of a timeout before marking the origin as unhealthy.
 - `interval` : The interval between each health check.
- For `TCP` type health check. Extra required fields are `port`.
 - `port` : The TCP port to use for the health check.
- For `HTTP/HTTPS` type health check. Extra option fields are `port`, `expected_body`, `expected_codes`, `method`, `path`, `header`, `follow_redirects`, `allow_insecure`.
 - `port` : The TCP port to use for the health check.
 - `expected_body` : A case-insensitive substring to look for in the response body.
 - `expected_codes` : The expected HTTP response code or code range of the health check.
 - `method` : The HTTP method to use for the health check.
 - `path` : The endpoint path to health check against.
 - `header` : The HTTP request headers to send in the health check.
 - `follow_redirects` : Follow redirects if returned by the origin.
 - `allow_insecure` : Do not validate the certificate when monitor use HTTPS.
 - `probe_zone` : Assign this monitor to emulate the specified zone while probing.

Sample JSON data:

For HTTP/HTTPS:

```
{
  "description": "Health monitor of web service",
  "type": "https",
  "method": "GET",
  "path": "/health",
  "header": {
    "Host": [
      "example.com"
    ],
    "X-App-ID": [
      "abc123"
    ]
  },
  "timeout": 5,
  "retries": 2,
  "interval": 90,
  "follow_redirects": true,
  "allow_insecure": false,
  "expected_codes": "2xx",
  "expected_body": "alive",
  "probe_zone": "example.com"
}
```

For TCP:

```
{
  "description": "Health monitor of TCP",
  "type": "tcp",
  "port": 80,
  "timeout": 5,
  "retries": 2,
  "interval": 90
}
```

`-s, --json-str`

Deprecated. The JSON data used to describe a GLB monitor.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update GLB monitor `GLB_MON_ID`.

```
$ ibmcloud cis glb-monitor-update GLB_MON_ID --json '{"type":"https", "description":"Health monitor of web service", "method":"GET", "path":"/health", "header":{"Host":["example.com"],"X-App-ID":["abc123"]}, "port":8080, "timeout":5, "retries":2, "interval":90, "expected_body":"alive", "expected_codes":"2xx", "follow_redirects":true, "allow_insecure":true}'
```

ibmcloud cis glb-events

List status changes from origins that are connected to a GLB monitor.

```
$ ibmcloud cis glb-events [-s, --since START_DATE] [-u, --until END_DATE] [--origin-name ORIGIN_NAME] [--pool-name POOL_NAME]
```

```
[--origin-healthy (true | false)] [--pool-healthy (true | false)]
[-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`-s, --since`

Start date requesting data period in the ISO8601 format. For example `2018-11-26` .

`-u, --until`

End date requesting data period in the ISO8601 format. For example `2018-11-28` .

`--origin-name`

The name for the origin to filter for.

`--pool-name`

The name for the pool to filter for.

`--origin-healthy`

If true, filter events where the origin status is healthy, if false, filter events where the origin status is unhealthy. The default value is `true` and valid values are `true` and `false` .

`--pool-healthy`

If true, filter events where the pool status is healthy, if false, filter events where the pool status is unhealthy. The default value is `true` and valid values are `true` and `false` .

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get glb events.

```
$ ibmcloud cis glb-events --since "2020-05-20" --until "2020-05-22" --origin-name "dal09" --origin-healthy true --instance INSTANCE
```

Instant Logs

Create and get logs of serverless functions instantly by using the following `instant-log` commands.

instant-log-create

Creates an instant logs job for a domain. The command returns a `Destination` , which is valid for 60 minutes.

```
$ cis instant-log-create DNS_DOMAIN_ID [--fields FIELD1,FIELD2,FIELD3|all] [--filter FILTER] [--sample SAMPLE] [-i, --instance INSTANCE] [--output FORMAT] [-h, -help HELP]
```



Note: You can have only one active Instant Logs session per domain and the maximum session time is 60 minutes.

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--fields`

Define the field set in return.

- This field must be specified as a comma-separated list without any whitespaces, and all fields must exist.
- The order in which fields are specified doesn't matter, and the order of fields in the response is not specified.

- The fields are expected to be case-sensitive.

`--filter`

Filters to drill down into specific events. Filters consist of three parts: key, operator, and value. For information about supported operators, see [Using fields, functions, and expressions](#).

`--sample`

The sample rate of the records set by the client: `sample`: 1 is 100% of records.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

`-h, --help`

Get help on this command.

Examples

Create an instant log for domain `DNS_DOMAIN_ID`:

```
$ ibmcloud cis instant-log-create DNS_DOMAIN_ID --fields all --sample 1
```

The following are three examples of filters:

- Filter when client IP country is not Canada:

```
"filter": "{\n  \"where\": {\n    \"and\": [\n      {\n        \"key\": \"ClientCountry\",\n        \"operator\": \"neq\",\n        \"value\": \"ca\"\n      }\n    ]\n  }\n}"
```

- Filter when the status code returned from CIS is either 200 or 201:

```
"filter": "{\n  \"where\": {\n    \"and\": [\n      {\n        \"key\": \"EdgeResponseStatus\",\n        \"operator\": \"in\",\n        \"value\": \"200,201\"\n      }\n    ]\n  }\n}"
```

- Filter when the request path contains "/static" and the request hostname is "example.com":

```
"filter": "{\n  \"where\": {\n    \"and\": [\n      {\n        \"key\": \"ClientRequestPath\",\n        \"operator\": \"contains\",\n        \"value\": \"/static\"\n      },\n      {\n        \"key\": \"ClientRequestHost\",\n        \"operator\": \"eq\",\n        \"value\": \"example.com\"\n      }\n    ]\n  }\n}"
```

instant-log-get

Get the instant logs job for a domain.

```
$ cis instant-log-get DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT] [-h, --help HELP]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

`-h, --help`

Help on this command.

Example

Get the instant logs job for domain `DNS_DOMAIN_ID`:


```
ibmcloud cis instant-log-get DNS_DOMAIN_ID
```

Leaked credential checks

Enterprise Plans Only

Manage leaked credential checks and custom detections for a DNS domain by using the following `leaked-credential-checks` commands.

`ibmcloud cis leaked-credential-checks status-get`

Get the leaked credential checks status for a DNS domain.

```
$ ibmcloud cis leaked-credential-checks status-get DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Example

```
$ ibmcloud cis leaked-credential-checks status-get DNS_DOMAIN_ID
```

`ibmcloud cis leaked-credential-checks status-update`

Enable or disable leaked credential checks for a DNS domain.

```
$ ibmcloud cis leaked-credential-checks status-update DNS_DOMAIN_ID (--enabled true|false) [-i, --instance INSTANCE] [--output FORMAT]
```

You can also provide JSON input from a file or directly as a string.

```
$ ibmcloud cis leaked-credential-checks status-update DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--enabled`

Indicates whether leaked credential checks are enabled for the DNS domain. Valid values are `true` and `false`.

`--json`

The JSON file or JSON string that describes the leaked credential checks configuration. The required field is `enabled`.

```
{
  "enabled": true
}
```

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

```
$ ibmcloud cis leaked-credential-checks status-update DNS_DOMAIN_ID --enabled true
ibmcloud cis leaked-credential-checks status-update DNS_DOMAIN_ID --json @status.json --output json
```

ibmcloud cis leaked-credential-checks detection-list

List custom leaked credential detections for a DNS domain.

```
$ ibmcloud cis leaked-credential-checks detection-list DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Example

```
$ ibmcloud cis leaked-credential-checks detection-list DNS_DOMAIN_ID --output json
```

ibmcloud cis leaked-credential-checks detection-get

Get a custom leaked credential detection.

```
$ ibmcloud cis leaked-credential-checks detection-get DNS_DOMAIN_ID DETECTION_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

DETECTION_ID

The ID of the custom leaked credential detection. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Example

```
$ ibmcloud cis leaked-credential-checks detection-get DNS_DOMAIN_ID DETECTION_ID
```

ibmcloud cis leaked-credential-checks detection-create

Create a custom leaked credential detection.

```
$ ibmcloud cis leaked-credential-checks detection-create DNS_DOMAIN_ID (--username EXPRESSION) (--password EXPRESSION) [-i, --instance INSTANCE] [--output FORMAT]
```

You can also provide JSON input from a file or directly as a string.

```
$ ibmcloud cis leaked-credential-checks detection-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--username

The ruleset expression that locates the username in a request. Required when **--json** is not used.

--password

The ruleset expression that locates the password in a request. Required when **--json** is not used.

--json

The JSON file or JSON string that describes the custom detection. The required fields are **username** and **password** .

```
{
  "username": "lookup_json_string(http.request.body.raw, \"username\")",
  "password": "lookup_json_string(http.request.body.raw, \"password\")"
}
```

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

```
$ ibmcloud cis leaked-credential-checks detection-create DNS_DOMAIN_ID --username 'lookup_json_string(http.request.body.raw, "username")' --password 'lookup_json_string(http.request.body.raw, "password")'
ibmcloud cis leaked-credential-checks detection-create DNS_DOMAIN_ID --json @detection.json --output json
```

ibmcloud cis leaked-credential-checks detection-update

Update a custom leaked credential detection. Specify at least one of **--username** or **--password** , or provide JSON input.

```
$ ibmcloud cis leaked-credential-checks detection-update DNS_DOMAIN_ID DETECTION_ID [--username EXPRESSION] [--password EXPRESSION] [-i, --instance INSTANCE] [--output FORMAT]
```

You can also provide JSON input from a file or directly as a string.

```
$ ibmcloud cis leaked-credential-checks detection-update DNS_DOMAIN_ID DETECTION_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

DETECTION_ID

The ID of the custom leaked credential detection. Required.

--username

The ruleset expression that locates the username in a request.

--password

The ruleset expression that locates the password in a request.

`--json`

The JSON file or JSON string that describes the custom detection. The optional fields are `username` and `password` ; specify at least one.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

```
$ ibmcloud cis leaked-credential-checks detection-update DNS_DOMAIN_ID DETECTION_ID --username 'lookup_json_string(http.request.body.raw, "email")'
ibmcloud cis leaked-credential-checks detection-update DNS_DOMAIN_ID DETECTION_ID --json @detection.json --output json
```

ibmcloud cis leaked-credential-checks detection-delete

Delete a custom leaked credential detection.

```
$ ibmcloud cis leaked-credential-checks detection-delete DNS_DOMAIN_ID DETECTION_ID [-f, --force] [-i, --instance INSTANCE]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`DETECTION_ID`

The ID of the custom leaked credential detection. Required.

`-f, --force`

Delete the detection without prompting for confirmation.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Example

```
$ ibmcloud cis leaked-credential-checks detection-delete DNS_DOMAIN_ID DETECTION_ID --force
```

Logpull

ibmcloud cis log-lookup-rayid

Enterprise Plans Only

Retrieve the log event that is associated with a specific Ray ID. Log retention must be enabled for the DNS domain before you use this command.

**Note:** If the Ray ID includes a data center suffix, such as `-XXX` , remove the suffix before you run the command.

```
$ ibmcloud cis log-lookup-rayid DNS_DOMAIN_ID RAY_ID [--fields FIELDS] [--timestamps FORMAT] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`RAY_ID`

The Ray ID of the request. Required.

`--fields`

A comma-separated list of case-sensitive log fields to return.

`--timestamps`

The format for timestamp fields. Valid values are `unix`, `unixnano`, and `rfc3339`. The default value is `unix`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

```
$ ibmcloud cis log-lookup-rayid DNS_DOMAIN_ID RAY_ID
ibmcloud cis log-lookup-rayid DNS_DOMAIN_ID RAY_ID --fields RayID,ClientIP,EdgeResponseStatus --timestamps rfc3339 --output json
```

ibmcloud cis logpull

Manipulate Logpull services by using the following `logpull` commands.

```
$ ibmcloud cis logpull DNS_DOMAIN_ID --available-fields [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--available-fields`

List of all available fields.

`--ray-id`

Lookup logs by specific Ray ID.

`--fields`

Define the field set in return. This field must be specified as a comma-separated list without any spaces, and all fields must exist. The order in which fields are specified doesn't matter, and the order of fields in the response is not specified. Note that fields are expected to be case sensitive.

`--start`

The (inclusive) beginning of the requested time frame. This can be a unix timestamp (in seconds or nanoseconds), or an absolute timestamp that conforms to RFC 3339. Currently, it cannot exceed a time in the past greater than 7 days. Default is 65 minutes earlier.

`--end`

The (exclusive) end of the requested time frame. This value can be a unix timestamp (in seconds or nanoseconds), or an absolute timestamp that conforms to RFC 3339. The `end` must be at least 5 minutes earlier than now and must be later than `start`. The difference between `start` and `end` must be not greater than 1h. The default is 5 minutes earlier.

`--count`

Number of logs to retrieve. The default value is `-1`.

`--sample`

Percentage of sampling. When a sample is provided, a sample of matching records is returned. If `sample=0.1` then 10% of records are returned. The sampling is random: repeated calls not only return different records, but likely also vary slightly in the number of returned records. When count is also specified, count is applied to the number of returned records, not the sampled records. So, with `sample=0.05` and `count=7`, when there is a total of 100 records available, approximately 5 records are returned. When there are 1000 records, 7 records are returned. When there are 10,000 records, 7 records are returned. The default value is `1`.

`--timestamps`

Set the format in which response timestamps are returned. Valid values are `unix`, `unixnano` and `rfc3339`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

```
ibmcloud cis logpull DNS_DOMAIN_ID --available-fields
ibmcloud cis logpull DNS_DOMAIN_ID --ray-id 59348abde87afe50 --all-fields --timestamps rfc3339 --output json
ibmcloud cis logpull DNS_DOMAIN_ID --start 2020-05-18T12:14:58Z --end 2020-05-18T13:14:58Z --fields ClientIP,EdgeServerIP,ClientRequestHost --count 10 --sample 1 --timestamps rfc3339 --output json
```

Logpush

Enterprise Plans Only

ibmcloud cis logpush-job-create

Enterprise Plans Only

Create a new logpush job for a domain.

```
$ ibmcloud cis logpush-job-create DNS_DOMAIN_ID --destination DESTINATION_URL --name NAME [--enable true|false] [--fields FIELDS | all] [--timestamps format][--dataset DATASET] [--frequency FREQUENCY] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--destination`

Specify a destination where data is pushed. For more information about supported destinations, see [Managing Logpush jobs](#).

- Syntax for Cloud Object Storage Path: `cos://<BUCKET_OBJECT_PATH>?region=<REGION>&instance-id=<IBM_CLOUD_OBJECT_STORAGE_INSTANCE_ID>&ibm_api_key=<API_KEY>` Example: `'cos://cis-test-bucket/logs?region=us&instance-id=f75e6d90-4212-4026-851c-d572071146cd&ibm_api_key=XXX'` To separate logs in to daily subfolders, use the special string `{DATE}` in the bucket path. It is substituted with the date in `YYYYMMDD` format, for example '20190423'. Subfolders are created as appropriate, for example: `'cos://cis-test-bucket/logs/{DATE}?region=us&instance-id=f75e6d90-4212-4026-851c-d572071146cd&ibm_api_key=XXX'`

`--name`

Job name. Required.

`--enable`

Enable the job. The job is disabled by default.

`--fields`

Define the list of log fields to be included in log files. Multiple fields can be separated by commas and use command `[ ibmcloud cis logpush-available-fields DNS_DOMAIN_ID ]` to get the comprehensive list of available log fields, or use `all` to include all available fields in log files. Note that fields are expected to be case sensitive.

`--timestamps`

Set the format in which response timestamps are returned. Valid values are `unix`, `unixnano` and `rfc3339`.

`--dataset`

The category of logs you want to receive. This value cannot be changed after the job is created. Valid values are `http_requests`, `range_events`, `firewall_events`, `dns_logs`. The default value is `http_requests`.

`--frequency`

The frequency at which CIS sends batches of logs to your destination. Setting the frequency to high sends your logs in larger quantities of smaller files. Setting the frequency to low sends logs in smaller quantities of larger files. Valid values are `high`, `low`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a logpush job for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis logpush-job-create DNS_DOMAIN_ID --destination cos://cis-test-bucket/logs?region=us&instance-id=f75e6d90-4212-4026-851c-d572071146cd&ibm_api_key=XXX --name logpushcreate --enable true --fields all --timestamps rfc3339 --dataset http_requests --frequency low --output json
```

ibmcloud cis logpush-job-update

Enterprise Plans Only

Update a logpush job for a domain.

```
$ ibmcloud cis logpush-job-update DNS_DOMAIN_ID [--destination DESTINATION_URL] [--enable true|false] [--fields FIELDS | all] [--timestamps format] [--dataset DATASET] [--jobid JOB_ID] [--frequency FREQUENCY] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--destination`

Specify a destination where data is pushed. For more information about supported destinations, see [Managing Logpush jobs](#). Syntax for Cloud Object Storage Path: `cos://<BUCKET_OBJECT_PATH>?region=<REGION>&instance-id=<IBM_CLOUD_OBJECT_STORAGE_INSTANCE_ID>ibm_api_key=<API_KEY>` Example: `'cos://cis-test-bucket/logs?region=us&instance-id=f75e6d90-4212-4026-851c-d572071146cdibm_api_key=XXX'` To separate logs into daily subfolders, use the special string `{DATE}` in the bucket path. It is to be substituted with the date in `YYYYMMDD` format, for example `'20190423'`. Subfolders are created as appropriate, for example: `'cos://cis-test-bucket/logs/{DATE}?region=us&instance-id=f75e6d90-4212-4026-851c-d572071146cdibm_api_key=XXX'`

`--enable`

Enable the job. The job is disabled by default.

`--fields`

Define the list of log fields to be included in log files. Multiple fields can be separated by commas and use command `ibmcloud cis logpush-available-fields DNS_DOMAIN_ID` to get the comprehensive list of available log fields, or use `all` to include all available fields in log files. Note that fields are expected to be case sensitive.

`--timestamps`

Set the format in which response timestamps are returned. Valid values are `unix`, `unixnano`, `rfc3339`.

`--dataset`

The category of logs you want to receive. This value cannot be changed after the job is created. Valid values are `http_requests`, `range_events`, `firewall_events`, `dns_logs`. The default value is `http_requests`.

`--jobid`

JOB_ID is the ID of the logpush job.

`--frequency`

The frequency at which CIS sends batches of logs to your destination. Setting the frequency to high sends your logs in larger quantities of smaller files. Setting the frequency to low sends logs in smaller quantities of larger files. Valid values are `high` and `low`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update `range_events` logpush job for domain `DNS_DOMAIN_ID` .

```
$ ibmcloud cis logpush-job-update DNS_DOMAIN_ID --destination cos://cis-test-bucket/logs?region=us&instance-id=f75e6d90-4212-4026-851c-d572071146cd&ibm_api_key=XXX --enable true --fields all --timestamps rfc3339 --dataset range_events --frequency high --output json
```

ibmcloud cis logpush-jobs

Enterprise Plans Only

Get all logpush jobs for a domain.

```
$ ibmcloud cis logpush-jobs DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

-output

The output format. Currently, `json` is the only supported value.

Examples

Get all logpush jobs for domain `DNS_DOMAIN_ID` .

```
$ ibmcloud cis logpush-jobs DNS_DOMAIN_ID --output json
```

ibmcloud cis logpush-job

Enterprise Plans Only

Get the details of a logpush job for a domain.

```
$ ibmcloud cis logpush-job DNS_DOMAIN_ID [--dataset DATASET] [--jobid JOB_ID] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--dataset

The category of logs you want to receive. This value cannot be changed after the job is created. Valid values are `http_requests` , `range_events` , `firewall_events` , `dns_logs` . The default value is `http_requests` .

--jobid

JOB_ID is the ID of the logpush job.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Get details of `http_requests` logpush job.


```
$ ibmcloud cis logpush-job DNS_DOMAIN_ID --dataset http_requests --output json
```

ibmcloud cis logpush-job-delete

Enterprise Plans Only

Delete a logpush job for a domain.

```
$ ibmcloud cis logpush-job-delete DNS_DOMAIN_ID [--dataset DATASET] [--jobid JOB_ID] [-f, --force] [-i, --instance INSTANCE]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--dataset

The category of logs you want to receive. This value cannot be changed after the job is created. Valid values are `http_requests`, `range_events`, `firewall_events`, `dns_logs`. The default value is `http_requests`.

--jobid

JOB_ID is the ID of the logpush job.

-f, --force

Delete logpush job without prompting for confirmation.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete `http_requests` logpush job for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis logpush-job-delete DNS_DOMAIN_ID --dataset http_requests --force
```

ibmcloud cis logpush-available-fields

Enterprise Plans Only

Get all available fields for a data set.

```
$ ibmcloud cis logpush-available-fields DNS_DOMAIN_ID [--dataset DATASET] [-i, --instance INSTANCE]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--dataset

The category of logs you want to receive. This value cannot be changed after the job is created. Valid values are `http_requests`, `range_events`, `firewall_events`, `dns_logs`. The default value is `http_requests`.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Get all available fields for `http_requests` logs.

```
$ ibmcloud cis logpush-available-fields DNS_DOMAIN_ID --dataset http_requests
```

Log retention

Enterprise Plans Only

ibmcloud cis log-retention

Get a log retention setting for the domain.

```
$ ibmcloud cis log-retention DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Get a log retention setting for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis log-retention DNS_DOMAIN_ID --output json
```

ibmcloud cis log-retention-update

Update the log retention setting for the domain.

```
$ ibmcloud cis log-retention-update DNS_DOMAIN_ID (--flag on|off) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--flag

Whether to turn log retention on or off. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Enable log retention for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis log-retention-update DNS_DOMAIN_ID --flag on --output json
```

Managed lists

Manipulate managed lists by using the following `managed-lists` commands.

cis managed-lists

List managed lists in an instance.

```
$ cis managed-lists [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Example

```
$ ibmcloud cis managed-lists --output json --instance INSTANCE
```

Metrics

Manipulate metrics by using the following `metrics` commands.

ibmcloud cis firewall-event-analytics

Retrieve a full log of firewall events.

```
$ ibmcloud cis firewall-event-analytics DNS_DOMAIN_ID [--dataset DATA_SET] [--filter FILTER] [--order FILTER_ORDER] [--limit LIMIT_NUMBER] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--dataset`

Requested dataset. The default value is `firewallEventsAdaptiveGroups`.

Use the following table to identify which datasets are included in your plan and the range of historical data you can query.

Dataset	Trial / Standard / Standard-Next	Enterprise / Security / GLB
firewallEventsAdaptiveGroups	30 days	30 days
firewallEventsAdaptive	30 days	30 days

Datasets included in your plan

`--filter`

Filter events. The default value is the last 6 hours of data.

The following operators are supported for all filter options:

Operator	Comparison
gt	greater than
lt	less than
geq	greater or equal to
leq	less or equal to

neq	not equal
in	in

Operators supported for filter options

- firewallEventsAdaptiveGroups

 filter options.
 - datetime
 - datetimeFifteenMinutes
 - datetimeHour
 - datetimeFiveMinutes
 - datetimeMinute
 - matchIndex
 - sampleInterval
- The following filter options support

like

 and

notlike

 operators.
 - action
 - clientASNDescription
 - clientAsn
 - clientCountryName
 - clientIP
 - clientRefererHost
 - clientRefererPath
 - clientRefererQuery
 - clientRefererScheme
 - clientRequestHTTPHost
 - clientRequestHTTPMethodName
 - clientRequestHTTPProtocol
 - clientRequestPath
 - clientRequestQuery
 - clientRequestScheme
 - edgeColoName
 - edgeResponseStatus
 - kind
 - originResponseStatus
 - originatorRayName
 - rayName
 - ref
 - ruleId
 - source
 - userAgent

--order

Output order. (The default value is

datetime_ASC

)

The following list is usable order options for corresponding dataset and all of order options support ASC and DESC action. Combine these filter options and action with

_

.

For example,

datetime_ASC

 orders by datetime ascending.

- firewallEventsAdaptiveGroups

 order options.
 - datetime
 - datetimeFifteenMinutes
 - datetimeHour
 - datetimeFiveMinutes
 - datetimeMinute

- action
- avg_sampleInterval
- clientASNDescription
- clientAsn
- clientCountryName
- clientIPClass
- clientIP
- clientRefererHost
- clientRefererPath
- clientRefererQuery
- clientRefererScheme
- clientRequestHTTPHost
- clientRequestHTTPMethodName
- clientRequestHTTPProtocol
- clientRequestPath
- clientRequestQuery
- clientRequestScheme
- count
- edgeColoName
- edgeResponseStatus
- kind
- matchIndex
- originResponseStatus
- originatorRayName
- rayName
- ref
- ruleId
- sampleInterval
- source
- userAgent
- visibility

`--limit`

The number of events to return. (minimum: `1`, maximum: `10000`, default: `10000`)

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get firewall event analytics for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis firewall-event-analytics DNS_DOMAIN_ID --order datetime_ASC --filter "datetime_geq:2020-06-28T00:00:00Z" --filter "datetime_leq:2020-06-29T00:00:00Z" --output json
```

ibmcloud cis http-request-analytics

Retrieve a full log of http request events.

```
$ ibmcloud cis http-request-analytics DNS_DOMAIN_ID [--dataset DATA_SET] [--filter FILTER] [--order FILTER_ORDER] [--limit LIMIT_NUMBER] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--dataset

Requested dataset. The default value is `httpRequests1dGroups` . Use the following table to identify which datasets are included in your plan and the range of historical data you can query.

Dataset	Trial / Standard / Standard-next	Enterprise / Security / GLB
httpRequests1dGroups	365 days	365 days
httpRequests1hGroups	30 days	90 days
httpRequests1mGroups	3 days	7 days

Identify datasets included in your plan

--filter

Filter events. The default value is the last 7 days data. The following operators are supported for all filter options:

Operator	Comparison
gt	greater than
lt	less than
geq	greater or equal to
leq	less or equal to
neq	not equal
in	in

Operators supported for filter options

- `httpRequests1dGroups` and `httpRequests1hGroups` filter options.
 - `date`
- `httpRequests1mGroups` filter options.
 - `datetime`
 - `datetimeFifteenMinutes`
 - `datetimeHour`
 - `datetimeDay`

--order

Output order. (The default value is `datetime_ASC`)

The following list is usable order options for corresponding dataset and all of order options support ASC and DESC action. Combine these order options and action with `_` .

For example, `date_ASC` orders by date ascending.

- Common order options for every http dataset.
 - `orderByParams`
 - `date`
 - `sum_bytes`

- `sum_cachedBytes`
- `sum_cachedRequests`
- `sum_requests`
- `httpRequests1dGroups` order options.
 - `avg_bytes`
 - `sum_encryptedBytes`
 - `sum_encryptedRequests`
 - `sum_pageViews`
 - `sum_threats`
 - `uniq_uniques`
- `httpRequests1hGroups` order options.
 - `avg_bytes`
 - `sum_encryptedBytes`
 - `sum_encryptedRequests`
 - `sum_pageViews`
 - `sum_threats`
 - `uniq_uniques`
 - `datetime`
- `httpRequests1mGroups` order options.
 - `avg_bytes`
 - `sum_encryptedBytes`
 - `sum_encryptedRequests`
 - `sum_pageViews`
 - `sum_threats`
 - `uniq_uniques`
 - `datetime`
 - `datetimeFifteenMinutes`
 - `datetimeHour`
 - `datetimeFifteenMinutes`
 - `datetimeHour`
 - `datetimeDay`

`--limit`

The number of events to return. (minimum: `1`, maximum: `10000`, default: `10000`)

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get http request analytics for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis http-request-analytics DNS_DOMAIN_ID --order date_ASC --dataset httpRequests1dGroups --limit 500 --filter "date_geq:2020-06-28" --filter "date_leq:2020-06-29" --output json
```

ibmcloud cis web-analytics (Deprecated)

Web analytics are deprecated on 2 November 2020. Use [ibmcloud cis http-request-analytics](#) instead. Get analytics of the DNS domain.

```
$ ibmcloud cis web-analytics DNS_DOMAIN_ID [--recent DURATION] [-t, --table requests | bandwidth | uniques | threats | status_code] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--recent`

The beginning of the requested time frame. Valid values are `6h` (6 hours ago), `12h`, `1d` (1 day ago), `1w` (1 week ago), `1m` (1 month ago), `2m`, `3m`. The default value is `1w`.

`-t, --table`

Output table. Valid values are `requests`, `bandwidth`, `uniques`, `threats` and, `status_code`. If this field is not set, it outputs all the tables.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get web analytics for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis web-analytics DNS_DOMAIN_ID --recent 1d -t requests
```

ibmcloud cis dns-analytics

Get DNS analytics of the domain.

```
$ ibmcloud cis dns-analytics DNS_DOMAIN_ID DIMENSION [-s, --since TIME] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`DIMENSION`

The queried dimension. Valid values are `queries-by-response-code`, `queries-by-type`, `queries-by-name`. Required.

`-s, --since`

Since time to now. Valid values are `6h` (6 hours ago), `12h`, `1d` (1 day ago), `1w` (1 week ago).

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get DNS analytics for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis dns-analytics DNS_DOMAIN_ID queries-by-response-code -s 6h --output json
```

ibmcloud cis ratelimit-analytics

Get rate limit analytics for a DNS domain.

```
$ ibmcloud cis ratelimit-analytics DNS_DOMAIN_ID [--recent DURATION] [--time-delta SECONDS] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--recent

The beginning of the requested time frame. Valid values are `6h` (6 hours ago), `12h`, `1d` (1 day ago), `1w` (1 week ago), `1m` (1 month ago), `2m`, `3m`. The default value is `1w`.

--time-delta

The time interval (seconds) of each analytic's record. Valid values are `60`, `3600`, `86400`, `2592000`. The default value is `3600`.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Get rate limit analytics for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis ratelimit-analytics DNS_DOMAIN_ID --recent 6h --time-delta 3600 --output json
```

MTLS enable

ibmcloud cis access-enable

Enterprise Plans Only

Enable Mutual TLS for a service instance.

```
$ ibmcloud cis access-enable [-i, --instance INSTANCE]
```

Command options

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Enable Mutual TLS.

```
$ ibmcloud cis access-enable
```

Origin certificates

ibmcloud cis origin-certificates

List all origin certificates for a DNS domain.

```
$ ibmcloud cis origin-certificates DNS_DOMAIN_ID [--instance INSTANCE_NAME] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List all origin certificates for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis origin-certificates DNS_DOMAIN_ID --output json
```

ibmcloud cis origin-certificate-create

Create a CIS-signed certificate.

```
$ ibmcloud cis origin-certificate-create DNS_DOMAIN_ID [--request-type REQUEST_TYPE] [--hostnames HOST_NAME1] [--hostnames HOST_NAME2] [--requested-
validity DAYS] [--csr CSR] [--instance INSTANCE_NAME] [--output FORMAT]
ibmcloud cis origin-certificate-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis origin-certificate-create DNS_DOMAIN_ID --json-str JSON_STR [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis origin-certificate-create DNS_DOMAIN_ID --json-file JSON_FILE [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--request-type REQUEST_TYPE`

Signature type that you want on the certificate. Valid values are `origin-rsa` and `origin-ecc`.

`--hostnames HOSTNAME`

hostname or wildcard name that is bound to the certificate.

`--requested-validity DAYS`

The number of days for which the certificate must be valid. The default value is `5475`.

`--csr CSR`

The Certificate Signing Request (CSR). If this field is not set, CIS generates one.

`--json value*`

The JSON file or JSON string that is used to describe an origin certificate.

- The required fields in JSON data are `request_type`, `hostnames`.
 - `request_type` : Signature type that you want on the certificate. Valid values are `origin-rsa`, `origin-ecc`.
 - `hostnames` : An array of hostnames or wildcard names that are bound to the certificate.
- The optional fields are `requested_validity`, `csr`.
 - `requested_validity` : The number of days for which the certificate must be valid. Valid values are `0`, `7`, `30`, `90`, `365`, `730`, `1095`, `5475`.
 - `csr` : The Certificate Signing Request (CSR). If this field is not set, CIS generates one.

Sample JSON data:

```
{
  "request_type": "origin-rsa",
  "hostnames": [
    "*.example.com",
    "example.com",
  ],
  "requested_validity": 5475,
  "csr": "your_csr"
```

```
}

```

`-s, --json-str`

Deprecated. The JSON data that describes an origin certificate.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a CIS-signed certificate for domain `DNS_DOMAIN_ID`.

```
ibmcloud cis origin-certificate-create DNS_DOMAIN_ID --request-type origin-rsa --hostnames "*.example.com" --hostnames "example.com" --requested-validity 5475 --csr your_csr --output json

ibmcloud cis origin-certificate-create DNS_DOMAIN_ID --json '{"hostnames":["example.com","*.example.com"], "requested_validity":5475,"request_type": "origin-rsa","csr":"-----BEGIN CERTIFICATE REQUEST-----
\nMIICxzCCAa8CAQAwSDELMakGA1UEBhMCVVMx FjAUBgNVBAgTDVNhbiBGcmFuY2lz\nY28xCzAJBgNVBACtAkNBMRQwEgYDVQQDEwtleGFtcGxLM5ldDCCA
SlwDQYJKoZI\nhvcNAQEBAQADggEPADCCAQoCggEBALxejtu4b+jPdFeFi6OUsyE8TYJQBm3WfCvL\nHu5EvijMO/4Z2TImwASbwUF7Ir8OLgH+mGIQZe qyNvGoSOM
EaZVXcYfpR1hI\nVak8\n4GGVr+04IGfOCqaBokaBF\nIwzclGMZbzKmLGwIQioNxGfqFm6RGYGA3be2Je2iseBc\nN8GV1wYmvYE0RR+yWweJCTJ157exyRzu7sVxaEW9F8
7zBQLyOnwXc64rf\nXsIRq\nlng7F7w5laQYOl8yvmk/jEPCAh a7fkiUfEpj4N12+oPRiMvleJF98chxjD4MH39c5\n\nuOslULhrWunfh7GB1jwWNA9y44H0snrf+xvoy2TcHmxvma9
Eln8CAwEAAaA6MDgG\nnCSqGS\nIb3DQEJDjErMCKwJwYDVR0RBCAwHolLZXhhbXBsZS5uZXSCD3d3dy5leGFt\nncGxLM5ldDANBgkqhkiG9w0BAQsFAAOCAQEAcBa
X6dOnl8ncARrI9ZSF2AJX+8mx\nnpTHY2+Y2C0VvrVDGMtbBRH8R9yMbqWtlxeeNGf//LeMkSKSFa4kbpd x226lfui8\n\nauRDBTJGx2R1ccUxmLZXx4my0W5ilMxunu+kez
+BDlu7bTT2io0uXMRHue4i6quH\nnyc5ibxvbJMjR7dqbc anVE10/34oprzXQsJ/VmSuZNXtjbtSKD\nImcpw6To/eeAJ+J\n\nhXykcUihvHyG4A1m2R6qpANBjnA0pHexfwM/Sgfv
pbvUg0T1ubmer8BgTwCKIWslndcWYTthM51JlqRBfNqy4QcBnX+GY05yltEEswQI55wdiS3CjTTA67sdbcQ==\n-----END CERTIFICATE REQUEST-----"}' --output json

```

ibmcloud cis origin-certificate

Get details of an origin certificate.

```
$ ibmcloud cis origin-certificate DNS_DOMAIN_ID CERT_ID [--instance INSTANCE_NAME] [--output FORMAT]

```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`CERT_ID`

The ID of the Origin Certificate. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get details of origin certificate `CERT_ID`.

```
$ ibmcloud cis origin-certificate DNS_DOMAIN_ID CERT_ID --output json

```

ibmcloud cis origin-certificate-delete

Delete an origin certificate.


```
$ ibmcloud cis origin-certificate-delete DNS_DOMAIN_ID CERT_ID [--instance INSTANCE_NAME]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

CERT_ID

The ID of Origin Certificate. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete origin certificate `CERT_ID`.

```
$ ibmcloud cis origin-certificate-delete DNS_DOMAIN_ID CERT_ID
```

Overview

View the overview information for a domain.

ibmcloud cis overview

Show the overview information for a domain.

```
$ ibmcloud cis overview DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show the overview information for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis overview DNS_DOMAIN_ID
```

Page rules

Manipulate page rules by using the following `pagerule` commands.

ibmcloud cis page-rule-create

Create a page rule of the DNS domain.

```
$ ibmcloud cis page-rule-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis page-rule-create DNS_DOMAIN_ID (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--json

The JSON file or JSON string that is used to describe a page rule. Required.

- The required fields in JSON data are targets, actions :
 - **targets** : The target URL pattern to evaluate on a request.
 - **actions** : An array of actions to perform if the targets of this rule match the request. Available actions are:
 - **disable_security**
 - **always_use_https**
 - **ssl**
 - **browser_cache_ttl**
 - **security_level**
 - **cache_level**
 - **edge_cache_ttl**
 - **bypass_cache_on_cookie**
 - **browser_check**
 - **server_side_exclude**
 - **email_obfuscation**
 - **automatic_https_rewrites**
 - **opportunistic_encryption**
 - **ip_geolocation**
 - **explicit_cache_control**
 - **cache_deception_armor**
 - **waf**
 - **forwarding_url**
 - *Deprecated.* **image_load_optimization** .
 - **image_size_optimization**
 - **script_load_optimization**
 - **host_header_override**
 - **resolve_override**
 - Some actions are limited to Enterprise plans:
 - **cache_on_cookie**
 - **disable_apps**
 - **disable_performance**
 - **minify**
 - **origin_error_page_pass_thru**
 - **response_buffering**
 - **true_client_ip_header**
 - **sort_query_string_for_cache**
 - **respect_strong_etag**
- The optional fields are **priority** and **status** :
 - **priority** : A number that indicates the preference for a page rule over another. The default value is **1** .
 - **status** : Status of the page rule. The valid values are **active** and **disabled** (default).

Sample JSON data:

```
{
  "targets": [
    {
      "target": "url",
      "constraint": {
        "operator": "matches",
```

```
      "value": "*example.com/images/*"
    }
  },
],
"actions": [
  {
    "id": "ssl",
    "value": "flexible"
  },
  {
    "id": "browser_cache_ttl",
    "value": 14400
  },
  {
    "id": "security_level",
    "value": "medium"
  },
  {
    "id": "cache_level",
    "value": "basic"
  },
  {
    "id": "edge_cache_ttl",
    "value": 7200
  },
  {
    "id": "bypass_cache_on_cookie",
    "value": "wp-.*|wordpress.*|comment_.*"
  }
]
}
```

`-s, --json-str`

Deprecated. The JSON data describing a page rule.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a page rule for domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis page-rule-create DNS_DOMAIN_ID --json '{"targets":[{"target":"url", "constraint":{"operator": "matches", "value":"*example.com/images/*"}}, {"actions":[{"id":"always_online", "value":"on"}], "priority":1, "status": "active"}' --output json
```

ibmcloud cis page-rule-update

Update the page rule of the DNS domain.

```
$ ibmcloud cis page-rule-update DNS_DOMAIN_ID PAGE_RULE_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis page-rule-update DNS_DOMAIN_ID PAGE_RULE_ID (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

PAGE_RULE_ID

The ID of page rule. Required.

--json VALUE

The JSON file or JSON string that is used to describe a page rule. Required.

- The required fields in JSON data are **targets** and **actions** :
 - **targets** : The target URL pattern to evaluate on a request.
 - **actions** : An array of actions to perform if the targets of this rule match the request. Available actions are:
 - **disable_security**
 - **always_use_https**
 - **ssl**
 - **browser_cache_ttl**
 - **security_level**
 - **cache_level**
 - **edge_cache_ttl**
 - **bypass_cache_on_cookie**
 - **browser_check**
 - **server_side_exclude**
 - **email_obfuscation**
 - **automatic_https_rewrites**
 - **opportunistic_encryption**
 - **ip_geolocation**
 - **explicit_cache_control**
 - **cache_deception_armor**
 - **waf**
 - **forwarding_url**
 - *Deprecated.* **image_load_optimization**
 - **image_size_optimization**
 - **script_load_optimization**
 - **host_header_override**
 - **resolve_override**
 - Some actions are limited to Enterprise plans:
 - **cache_on_cookie**
 - **disable_apps**
 - **disable_performance**
 - **minify**
 - **origin_error_page_pass_thru**
 - **response_buffering**
 - **true_client_ip_header**
 - **sort_query_string_for_cache**
 - **respect_strong_etag**
- The optional fields are **priority** and **status** :
 - **priority** : A number that indicates the preference for a page rule over another. The default value is **1**.
 - **status** : Status of the page rule. Valid values are **active** and **disabled**. The default value is **disabled**.

Sample JSON data:

```
{
  "targets": [
    {
      "target": "url",
      "constraint": {
        "operator": "matches",
```

```
      "value": "*example.com/images/*"
    }
  },
],
"actions": [
  {
    "id": "ssl",
    "value": "flexible"
  },
  {
    "id": "browser_cache_ttl",
    "value": 14400
  },
  {
    "id": "security_level",
    "value": "medium"
  },
  {
    "id": "cache_level",
    "value": "basic"
  },
  {
    "id": "edge_cache_ttl",
    "value": 7200
  },
  {
    "id": "bypass_cache_on_cookie",
    "value": "wp-.*|wordpress.*|comment_.*"
  }
]
}
```

`-s, --json-str`

Deprecated. The JSON data describing a page rule.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update page rule `PAGE_RULE_ID`.

```
$ ibmcloud cis page-rule-update DNS_DOMAIN_ID PAGE_RULE_ID --json '{"targets":[{"target":"url", "constraint":{"operator":"matches", "value":"*example.com/images/*"}}, {"actions":[{"id":"always_online", "value":"on"}], "priority":1, "status":"active"}' --output json
```

ibmcloud cis page-rule-delete

Delete a page rule of the DNS domain.

```
$ ibmcloud cis page-rule-delete DNS_DOMAIN_ID PAGE_RULE_ID [-i, --instance INSTANCE]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`PAGE_RULE_ID`

The ID of page rule. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete page rule `PAGE_RULE_ID`.

```
$ ibmcloud cis page-rule-delete DNS_DOMAIN_ID PAGE_RULE_ID
```

ibmcloud cis page-rules

List page rules of the DNS domain.

```
$ ibmcloud cis page-rules DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List all page rules in domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis page-rules DNS_DOMAIN_ID --output json
```

ibmcloud cis page-rule

Get details of a page rule.

```
$ ibmcloud cis page-rule DNS_DOMAIN_ID PAGE_RULE_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`PAGE_RULE_ID`

The ID of page rule. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get details of page rule `PAGE_RULE_ID`.

```
$ ibmcloud cis page-rule DNS_DOMAIN_ID PAGE_RULE_ID --output json
```

Range app

Manipulate how the Range App performs by using the following `range-app` commands:

ibmcloud cis range-app-create

Enterprise Plans Only

Create a new range application.

```
$ ibmcloud cis range-app-create DNS_DOMAIN_ID --name NAME --edge-port EDGE_PORT --origin-direct ORIGIN_DIRECT [--origin-direct ORIGIN_DIRECT] [--proxy-protocol on|off] [--ip-firewall on|off] [--edge-connectivity all|ipv4|ipv6] [--edge-tls off|flexible|full|strict] [--traffic-type direct/http/https] [-i, --instance INSTANCE] [--output FORMAT]

ibmcloud cis range-app-create DNS_DOMAIN_ID --name NAME --edge-port EDGE_PORT --origin-lb-name ORIGIN_LB_NAME --origin-lb-port ORIGIN_LB_PORT [--proxy-protocol on|off] [--ip-firewall on|off] [--edge-connectivity all|ipv4|ipv6] [--edge-tls off|flexible|full|strict] [--traffic-type direct/http/https] [-i, --instance INSTANCE] [--output FORMAT]

ibmcloud cis range-app-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]

[Deprecated] ibmcloud cis range-app-create DNS_DOMAIN_ID -s JSON_STR [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis range-app-create DNS_DOMAIN_ID -j JSON_FILE [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--name`

The name of the DNS record for the range application.

`--edge-port`

Port configuration at CIS's edge. The default value is `22`.

`--origin-direct`

Destination addresses to the origin.

`--origin-lb-name`

The Load Balancer name associated with the range application.

`--origin-lb-port`

The Load Balancer port associated with the range application. The default value is `22`.

`--protocol`

Protocol type. Valid values are `tcp` and `udp`. UDP protocol support is in early access, request custom UDP from CIS dashboard before creating range UDP app. The default value is `tcp`.

`--proxy-protocol`

Enable Proxy Protocol to the origin. Valid values are `on`, `off`, `v1`, `v2`, `simple`. The default value is `off`. *Deprecated*. The value `on` is equivalent to `v1`.

`--ip-firewall`

Control whether the IP Firewall for this application is enabled. Valid values are `on` and `off`. The default value is `off`.

`--edge-connectivity`

The IP versions supported for inbound connections on a range of anycast IPs. Valid values are `all`, `ipv4`, `ipv6`. The default value is `all`.

`--edge-tls`

The type of TLS termination associated with the application. Valid values are `off`, `flexible`, `full`, `strict`. The default value is `off`.

`--traffic-type`

Determines how data travels from the edge to your origin. Valid values are `direct`, `http`, `https`. The default value is `direct`.

`--json`

The JSON file or JSON string that is used to describe a range application.

- The required fields in JSON data are `protocol` and `dns`.
 - `protocol` : Port configuration at CIS's edge.
 - `dns` : The name and type of the DNS record for the range application.
 - `name` : The name of the DNS record for the range application.
 - `type` : The type of the DNS record associated with the application. Valid value is `CNAME`.
- The optional fields are `origin_direct`, `origin_dns`, `origin_port`, `proxy_protocol`, `ip_firewall`, `edge_ips`, `tls`, and `traffic_type`.
 - `origin_direct` : A list of destination addresses to the origin.
 - `origin_dns` : Method and parameters that are used to discover the origin server address via DNS.
 - `name` : DNS record name.
 - `origin_port` : The destination port at the origin.
 - `proxy_protocol` : Enable the Proxy Protocol to the origin. Valid values are `on`, `off`, `v1`, `v2`, `simple`. The default value is `off`. *Deprecated*. The value `on` is equivalent to `v1`.
 - `ip_firewall` : Control whether the IP Firewall for this application is enabled. Valid values are `on` and `off`.
 - `edge_ips` : The anycast edge IP configuration for the hostname of this application.
 - `type` : The type of edge IP configuration specified. Dynamically allocated edge IPs use range anycast IPs in accordance with the connectivity you specify. Valid value is `dynamic`.
 - `connectivity` : The IP versions supported for inbound connections on a range of anycast IPs. Valid values: `all`, `ipv4`, `ipv6`.
 - `tls` : The type of TLS termination associated with the application. Valid values are `off`, `flexible`, `full`, and `strict`.
 - `traffic_type` : Determines how data travels from the edge to your origin. When set to `direct`, range sends traffic directly to your origin, and the application's type is derived from the protocol. When set to `http` or `https`, range applies CIS's HTTP/HTTPS features as it sends traffic to your origin, and the application type matches this property exactly. Valid values are `direct`, `http`, and `https`. The default value is `direct`.

Sample JSON data:

```
{
  "protocol": "tcp/22",
  "dns": {
    "type": "CNAME",
    "name": "ssh.example.com"
  },
  "origin_direct": [
    "tcp://1.2.3.4:22",
    "tcp://1.2.3.4:23",
    "tcp://1.2.3.4:24"
  ],
  "proxy_protocol": false,
  "ip_firewall": false,
  "edge_ips": {
    "type": "dynamic",
    "connectivity": "all"
  },
  "tls": "full",
  "traffic_type": "direct"
}

{
  "protocol": "tcp/22",
  "dns": {
    "type": "CNAME",
    "name": "glb.example.com"
  },
  "origin_dns": {
    "name": "name-to-glb.example.com"
  },
  "origin_port": 22,
  "proxy_protocol": false,
  "ip_firewall": false,
  "edge_ips": {
    "type": "dynamic",
    "connectivity": "all"
  }
}
```



```
  },
  "tls": "full",
  "traffic_type": "direct"
}
```

`-s, --json-str`

Deprecated. The JSON data describing a range application.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a range app for the domain `DNS_DOMAIN_ID`.

```
$ ibmcloud cis range-app-create DNS_DOMAIN_ID --json '{"protocol":"tcp/22", "dns":{"type":"CNAME","name":"ssh.example.com"}, "origin_direct":["tcp://1.2.3.4:22"], "proxy_protocol":"off", "ip_firewall":true, "tls":"full", "edge_ips":{"type":"dynamic", "connectivity":"all"}, "traffic_type":"direct"}'
```

ibmcloud cis range-app-update

Enterprise Plans Only

Update a previously existing application's configuration.

```
$ ibmcloud cis range-app-update DNS_DOMAIN_ID APP_ID --origin-direct ORIGIN_DIRECT [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis range-app-update DNS_DOMAIN_ID APP_ID [--add-origin-direct ORIGIN_DIRECT] [--remove-origin-direct ORIGIN_DIRECT] [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis range-app-update DNS_DOMAIN_ID APP_ID [--origin-lb-name ORIGIN_LB_NAME] [--origin-lb-port ORIGIN_LB_PORT] [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis range-app-update DNS_DOMAIN_ID APP_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis range-app-update DNS_DOMAIN_ID APP_ID -s JSON_STR [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis range-app-update DNS_DOMAIN_ID APP_ID -j JSON_FILE [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`APP_ID`

The ID of the range application. Required.

`--name`

The name of the DNS record for the range application.

`--add-origin-direct`

Add new destination addresses to the origin.

`--remove-origin-direct`

Remove destination addresses from origin.

`--origin-direct`

Destination addresses to the origin.

`--origin-lb-name`

The Load Balancer name associated with the range application.

`--origin-lb-port`

The Load Balancer port associated with the range application. The default value is `22`.

`--proxy-protocol`

Enable the Proxy Protocol to the origin. Valid values are `on`, `off`, `v1`, `v2`, `simple`. The default value is `off`. *Deprecated*. The value `on` is equivalent to `v1`.

`--ip-firewall`

Control whether the IP Firewall for this application is enabled. Valid values are `on` and `off`. The default value is `off`.

`--edge-connectivity`

The IP versions supported for inbound connections on a range of anycast IPs. Valid values are `all`, `ipv4`, `ipv6`. The default value is `all`.

`--edge-tls`

The type of TLS termination associated with the application. Valid values are `off`, `flexible`, `full`, `strict`. The default value is `off`.

`--traffic-type`

Determines how data travels from the edge to your origin. Valid values are `direct`, `http`, `https`. The default value is `direct`.

`--json`

The JSON file or JSON string that is used to describe a range application.

- The required fields in JSON data are `protocol` and `dns`.
 - `protocol` : Port configuration at CIS's edge.
 - `dns` : The name and type of the DNS record for the range application.
 - `name` : The name of the DNS record for the range application.
 - `type` : The type of the DNS record associated with the application. Valid value is `CNAME`.
- The optional fields are `origin_direct`, `origin_dns`, `origin_port`, `proxy_protocol`, `ip_firewall`, `edge_ips`, `tls`, and `traffic_type`.
 - `origin_direct` : A list of destination addresses to the origin.
 - `origin_dns` : Method and parameters that are used to discover the origin server address via DNS.
 - `name` : DNS record name.
 - `origin_port` : The destination port at the origin.
 - `proxy_protocol` : Enable Proxy Protocol to the origin. Valid values are `on`, `off`, `v1`, `v2`, `simple`. The default value is `off`. *Deprecated*. The value `on` is equivalent to `v1`.
 - `ip_firewall` : Control whether the IP Firewall for this application is enabled. Valid values are `on` and `off`.
 - `edge_ips` : The anycast edge IP configuration for the hostname of this application.
 - `type` : The type of edge IP configuration specified. Dynamically allocated edge IPs use range anycast IPs in accordance with the connectivity you specify. Valid value is `dynamic`.
 - `connectivity` : The IP versions supported for inbound connections on range anycast IPs. Valid values are `all`, `ipv4`, `ipv6`.
 - `tls` : The type of TLS termination associated with the application. Valid values are `off`, `flexible`, `full`, `strict`.
 - `traffic_type` : Determines how data travels from the edge to your origin. When set to `direct`, range sends traffic directly to your origin, and the application's type is derived from the protocol. When set to `http` or `https`, range applies CIS's HTTP/HTTPS features as it sends traffic to your origin, and the application type matches this property exactly. Valid values are `direct`, `http`, and `https`. The default value is `direct`.

Sample JSON data:

```
{
  "protocol": "tcp/22",
  "dns": {
    "type": "CNAME",
    "name": "ssh.example.com"
  },
  "origin_direct": [
    "tcp://1.2.3.4:22",
    "tcp://1.2.3.4:23",
    "tcp://1.2.3.4:24"
  ],
  "proxy_protocol": false,
  "ip_firewall": false,
  "edge_ips": {
    "type": "dynamic",
    "connectivity": "all"
  }
}
```

```
    },
    "tls": "full",
    "traffic_type": "direct"
  }

  {
    "protocol": "tcp/22",
    "dns": {
      "type": "CNAME",
      "name": "glb.example.com"
    },
    "origin_dns": {
      "name": "name-to-glb.example.com"
    },
    "origin_port": 22,
    "proxy_protocol": false,
    "ip_firewall": false,
    "edge_ips": {
      "type": "dynamic",
      "connectivity": "all"
    },
    "tls": "full",
    "traffic_type": "direct"
  }
}
```

`-s, --json-str`

Deprecated. The JSON data describing a range application.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update range app `APP_ID`.

```
$ ibmcloud cis range-app-update DNS_DOMAIN_ID APP_ID --json '{"protocol":"tcp/22", "dns":{"type":"CNAME","name":"ssh.example.com"}, "origin_direct":
["tcp://1.2.3.4:22"], "proxy_protocol":"off", "ip_firewall":true, "tls":"full", "edge_ips":{"type":"dynamic", "connectivity":"all"}, "traffic_type":"direct"}'
```

ibmcloud cis range-app-delete

Enterprise Plans Only

Delete a previously existing application.

```
$ ibmcloud cis range-app-delete DNS_DOMAIN_ID APP_ID [--instance INSTANCE]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`APP_ID`

The ID of range application. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete range application `APP_ID`.

```
$ ibmcloud cis range-app-delete DNS_DOMAIN_ID APP_ID
```

ibmcloud cis range-app

Enterprise Plans Only

Get the application configuration of a specific application.

```
$ ibmcloud cis range-app DNS_DOMAIN_ID APP_ID [--instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`APP_ID`

The ID of range application. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Show details of range application `APP_ID`.

```
$ ibmcloud cis range-app DNS_DOMAIN_ID APP_ID
```

ibmcloud cis range-apps

Enterprise Plans Only

Retrieve a list of currently existing range applications for a DNS domain.

```
$ ibmcloud cis range-apps DNS_DOMAIN_ID [--instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List all range applications in the domain.

```
$ ibmcloud cis range-apps DNS_DOMAIN_ID
```

ibmcloud cis range-analytics

Enterprise Plans Only

Get analytics data for range applications.

```
$ ibmcloud cis range-analytics DNS_DOMAIN_ID [--metrics METRICS] [--dimensions DIMENSION] [--filters FILTERS] [--sort SORT] [--since SINCE] [--until UNTIL]
ibmcloud cis range-analytics DNS_DOMAIN_ID --bytime [--time_delta DELTA] [--metrics METRICS] [--dimensions DIMENSION] [--filters FILTERS] [--sort SORT] [--since SINCE] [--until UNTIL]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--metrics

One or more metrics to compute. To get all metrics, set metrics to `count,bytesIngress,bytesEgress,durationAvg,durationMedian,duration90th,duration99th`.

--dimension

Can be used to break down the data by attributes. To get all dimensions, set dimensions to `event,appID,coloName,ipVersion`. `--filters`

Used to filter rows by one or more dimensions. Filters can be combined by using OR and AND Boolean logic. AND takes precedence over OR in all the expressions. The OR operator is defined by using a comma (,) or OR keyword that is surrounded by whitespace. The AND operator is defined by using a semicolon (;) or AND keyword that is surrounded by whitespace. Comparison options are: `==`, `!=`, `>`, `<`, `>=`, `<=`. An example value for filters is: `event==connect AND coloName!=SFO`.

--sort

The sort order for the result set. Sort fields must be included in metrics or dimensions. An example value for sort is: `+count,-bytesIngress`.

--since

Start of time interval to query, defaults to until - 6 hours. This value must be an absolute timestamp that conforms to RFC 3339.

--until

End of time interval to query, defaults to current time. This value must be an absolute timestamp that conforms to RFC 3339.

--bytime

Analytics data for range applications grouped by time interval.

--time-delta

Used to select time series resolution. Valid values are `year`, `quarter`, `month`, `week`, `day`, `hour`, `dekaminute`, `minute`. Only valid when `--bytime` is given.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Get analytics data for range applications in the domain.

```
$ ibmcloud cis range-analytics DNS_DOMAIN_ID --metrics "count,bytesIngress" --dimensions "event,appID" --since "2020-05-22T02:20:00Z" --until "2020-05-23T02:20:00Z"
```

Rate limiting

Manipulate rate limits by using the following `ratelimit` commands.

ibmcloud cis ratelimit-rule-create

Enterprise Plans Only

Create a new rate limiting rule for a DNS domain.


```
$ ibmcloud cis ratelimit-rule-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis ratelimit-rule-create DNS_DOMAIN_ID --url URL [--description DESCRIPTION] [--threshold NUM] [--period SECONDS] [...]
[Deprecated] ibmcloud cis ratelimit-rule-create DNS_DOMAIN_ID --json-str JSON_STR [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis ratelimit-rule-create DNS_DOMAIN_ID --json-file JSON_FILE [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--json

The JSON file or JSON string that is used to describe a rate limiting rule.

- The required fields in JSON data are **match**, **threshold**, **period** and **action** :
 - **match** : Determines which traffic the rate limiting rule counts toward the threshold.
 - **request** : Matches HTTP requests.
 - **methods** : HTTP Methods, can be a subset **[POST,PUT]** or all **[_ALL_]**. This field is not required to create a rate limit rule. Valid values are **GET**, **POST**, **PUT**, **DELETE**, **PATCH**, **HEAD**, **_ALL_**.
 - **schemes** : HTTP Schemes, can be one **[HTTPS]**, both **[HTTP, HTTPS]** or all **[_ALL_]**. This field is not required.
 - **url** : The URL pattern to match composed of the host and path, for instance, **example.org/path**. Wildcards are expanded to match applicable traffic, query strings are not matched. Use ***** for all traffic to your zone. The max length is **1024**.
 - **response** : Matches HTTP responses before they are returned to the client. If this is defined, then the entire counting of traffic occurs at this stage.
 - **status** : HTTP Status codes, can be one **[403]**, many **[401,403]** or indicate all by not providing this value. This field is not required. The min value is **100** and the max value is **999**.
 - **headers** : Array of response headers to match. If a response does not meet the header criteria, then the request is not counted towards the rate limiting rule. The header matching criteria includes the following properties.
 - **name** : The name of the response header to match.
 - **op** : The operator when matching, eq means equals, ne means not equals. Valid values are **eq** and **ne**.
 - **value** : The value of the header, which is exactly matched.
 - **threshold** : The threshold that triggers the rate limit mitigations, which are combined with a period. For example, the threshold per period. The min value is **2** and the max value is **1000000**.
 - **period** : The time, in seconds, to count matching traffic. If the count exceeds the threshold within this period, the action is performed. The min value is **10** and the max value is **86400**.
 - **action** : The action performed when the threshold of matched traffic within the period defined is exceeded.
 - **mode** : The type of action performed. Valid values are: **simulate**, **ban**, **challenge**, **js_challenge**.
 - **timeout** : The time in seconds, as an integer to perform the mitigation action. The timeout can be the same or greater than the period. This field is valid only when the mode is **simulate** or **ban**. The min value is **10** and the max value is **86400**.
 - **response** : Custom content-type and body to return. This overrides the custom error for the zone. This field is not required. Omission results in the default HTML error page. This field is valid only when mode is **simulate** or **ban**.
 - **content_type** : The content-type of the body, which must be one of the following: **text/plain**, **text/xml**, **application/json**.
 - **body** : The body to return. The content here must conform to the **content_type**. The max length is **10240**.
- The optional fields are **id**, **disabled**, **description**, **correlate** and **bypass** :
 - **id** : Identifier of the rate limiting rule.
 - **disabled** : Whether this rate limiting rule is currently disabled.
 - **description** : A note that you can use to describe the reason for a rate limiting rule.
 - **correlate** : Whether to enable NAT-based rate limiting.
 - **by** : Valid value is **nat**.
 - **bypass** : Criteria that allow the rate limit to be bypassed. For example, to express that you shouldn't apply a rate limit to a set of URLs.
 - **name** : Valid value is **url**.
 - **value** : The url to bypass.

Sample JSON data:

```
{
  "id": "92f17202ed8bd63d69a66b86a49a8f6b",
  "disabled": false,
  "description": "Prevent multiple login failures to mitigate brute force attacks",
  "bypass": [
    {
      "name": "url",
      "value": "api.example.com/*"
    }
  ],
  "threshold": 60,
  "period": 900,
  "correlate": {
    "by": "nat"
  },
  "action": [
    {
      "mode": "simulate",
      "timeout": 86400,
      "response": {
        "content_type": "text/plain",
        "body": "<error>This request has been rate-limited.</error>"
      }
    }
  ],
  "match": {
    "request": {
      "methods": [
        "GET"
      ],
      "schemes": [
        "HTTP",
        "HTTPS"
      ],
      "url": "*.example.org/path*"
    },
    "response": {
      "status": [
        403, 401
      ],
      "headers": [
        {
          "name": "Cf-Cache-Status",
          "op": "eq",
          "value": "HIT"
        }
      ]
    }
  }
}
```

`-s, --json-st`

Deprecated. The JSON data describing a rate limiting rule.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a rate limiting rule for domain.


```
$ ibmcloud cis ratelimit-rule-create DNS_DOMAIN_ID --json '{"id":"372e67954025e0ba6aaa6d586b9e0b59","disabled":false,"description":"Prevent multiple login failures to mitigate brute force attacks","match":{"request":{"methods":["GET","POST"],"schemes":["HTTP","HTTPS"],"url":"*.example.org/path*"},"response":{"status": [403,401],"headers":{"name":"Cf-Cache-Status","op":"ne","value":"HIT"}}},"bypass":[{"name":"url","value":"api.example.com/*"}],"threshold":60,"period":900,"action":{"mode":"challenge","timeout":86400,"response":{"content_type":"text/xml","body":"<error>This request has been rate-limited.</error>"}}}'
```

ibmcloud cis ratelimit-rule-update

Update a rate limiting rule of a DNS domain.

```
$ ibmcloud cis ratelimit-rule-update DNS_DOMAIN_ID RATELIMIT_RULE_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis ratelimit-rule-update DNS_DOMAIN_ID RATELIMIT_RULE_ID [--url URL] [--description DESCRIPTION] [--threshold NUM] [--period SECONDS] [...]
[Deprecated] ibmcloud cis ratelimit-rule-update DNS_DOMAIN_ID RATELIMIT_RULE_ID --json-str JSON_STR [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis ratelimit-rule-update DNS_DOMAIN_ID RATELIMIT_RULE_ID --json-file JSON_FILE [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

RATELIMIT_RULE_ID

The ID of rate limiting rule. Required.

--json

The JSON file or JSON strinthat is used to describe a rate limiting rule.

- The required fields in JSON data are match , threshold , period and action :
 - match : Determines which traffic the rate limiting rule counts towards the threshold.
 - request : Matches HTTP requests.
 - methods : HTTP Methods, can be a subset [POST,PUT] or all [ALL] . This field is not required to create a rate limit rule. Valid values are GET , POST , PUT , DELETE , PATCH , HEAD , ALL .
 - schemes : HTTP Schemes, can be one [HTTPS] , both [HTTP,HTTPS] or all [_ALL_] . This field is not required.
 - url : The URL pattern to match consisted of the host and path, for instance, example.org/path . Wildcards are expanded to match applicable traffic, query strings are not matched. Use * for all traffic to your zone. The max length is 1024 .
 - response : Matches HTTP responses before they are returned to the client. If this field is defined, then the entire counting of traffic occurs at this stage.
 - status : HTTP Status codes, can be one [403] , many [401,403] or indicate all by not providing this value. This field is not required. The min value is 100 and the max value is 999 .
 - headers : Array of response headers to match. If a response does not meet the header criteria, then the request is not counted towards the rate limiting rule. An array of header matching criteria includes the following properties.
 - name : The name of the response header to match.
 - op : The operator when matching, eq means equals, ne means not equals. Valid values are eq and ne .
 - value : The value of the header, which is exactly matched.
 - threshold : The threshold that triggers the rate limit mitigations, which are combined with period. For example, the threshold per period. The min value is 2 and the max value is 1000000 .
 - period : The time, in seconds, to count matching traffic. If the count exceeds the threshold within this period the action is performed. The min value is 1 and the max value is 3600 .
 - action : The action is performed when the threshold of matched traffic within the defined period is exceeded.
 - mode : The type of action performed. Valid values are simulate , ban , challenge , js_challenge .
 - timeout : The time, in seconds, as an integer to perform the mitigation action. Timeout is the same or greater than the period. This field is valid only when mode is simulate or ban . The min value is 10 and the max value is 86400 .
 - response : Custom content-type and body to return. This overrides the custom error for the zone. This field is not required. Omission results in the default HTML error page. This field is valid only when mode is simulate or ban .
 - content_type : The content-type of the body, which must be one of the following: text/plain , text/xml , application/json .
 - body : The body to return. The content here must conform to the content_type . The max length is 10240 .
 - The optional fields are disabled , description , correlate and bypass :
 - disabled : Whether this rate limiting rule is currently disabled.
 - description : A note that you can use to describe the reason for a rate limiting rule.
 - correlate : Whether to enable NAT-based rate limiting.

- `by` : valid value is `nat` .
- `bypass` : Criteria that allow the rate limit to be bypassed. For example, to express that you shouldn't apply a rate limit to a set of URLs.
 - `name` : Valid value is `url` .
 - `value` : The url to bypass.

Sample JSON data:

```
{
  "disabled": false,
  "description": "Prevent multiple login failures to mitigate brute force attacks",
  "bypass": [
    {
      "name": "url",
      "value": "api.example.com/*"
    }
  ],
  "threshold": 60,
  "period": 900,
  "correlate": {
    "by": "nat"
  },
  "action": [
    {
      "mode": "simulate",
      "timeout": 86400,
      "response": {
        "content_type": "text/plain",
        "body": "<error>This request has been rate-limited.</error>"
      }
    }
  ],
  "match": {
    "request": {
      "methods": [
        "GET"
      ],
      "schemes": [
        "HTTP",
        "HTTPS"
      ],
      "url": "/*.example.org/path*"
    },
    "response": {
      "status": [
        403, 401
      ],
      "headers": [
        {
          "name": "Cf-Cache-Status",
          "op": "eq",
          "value": "HIT"
        }
      ]
    }
  }
}
```

`-s, --json-str`

Deprecated. The JSON data describing a rate limiting rule.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update rate limiting rule for domain.

```
$ ibmcloud cis ratelimit-rule-update DNS_DOMAIN_ID RATELIMIT_RULE_ID --json '{"disabled":false,"description":"Prevent multiple login failures to mitigate brute force attacks","match":{"request":{"methods":["GET","POST"],"schemes":["HTTP","HTTPS"],"url":"*.example.org/path*"},"response":{"status": [403, 401],"headers":{"name":"Cf-Cache-Status","op":"ne","value":"HIT"}}},"bypass":{"name":"url","value":"api.example.com/*"},"threshold":60,"period":900,"action":{"mode":"challenge","timeout":86400,"response":{"content_type":"text/xml","body":"<error>This request has been rate-limited.</error>"}}}'
```

ibmcloud cis ratelimit-rules

List rate limiting rules of a DNS domain.

```
$ ibmcloud cis ratelimit-rules DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List rate limiting rules in domain.

```
$ ibmcloud cis ratelimit-rules DNS_DOMAIN_ID
```

ibmcloud cis ratelimit-rule

Get details of a rate limiting rule by ID.

```
$ ibmcloud cis ratelimit-rule DNS_DOMAIN_ID RATELIMIT_RULE_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`RATELIMIT_RULE_ID`

The ID of rate limit rule. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get the details of rate limiting rule.

```
$ ibmcloud cis ratelimit-rule DNS_DOMAIN_ID RATELIMIT_RULE_ID
```

ibmcloud cis ratelimit-rule-delete

Delete a rate limiting rule by ID.

```
$ ibmcloud cis ratelimit-rule-delete DNS_DOMAIN_ID RATELIMIT_RULE_ID [--instance INSTANCE]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

RATELIMIT_RULE_ID

The ID of rate limit rule. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete rate limiting rule.

```
$ ibmcloud cis ratelimit-rule-delete DNS_DOMAIN_ID RATELIMIT_RULE_ID
```

Resource instance

Manipulate CIS Service instances by using the following `instance` commands.

ibmcloud cis instances

List all CIS service instances.

```
$ ibmcloud cis instances [--output FORMAT]
```

Command options

--output

The output format. Currently, `json` is the only supported value.

Examples

List all CIS instance in current account.

```
$ ibmcloud cis instances
```

ibmcloud cis instance-set

Set the context service instance to operate.

```
$ ibmcloud cis instance-set [INSTANCE_NAME] [--unset]
```

Command options

INSTANCE_NAME

The name of the CIS service instance. If it is presented, set the context instance to operate, if not, show the current context instance.

`--unset`

Unset context instance.

Examples

Set the context service instance to `cis-demo`

```
$ ibmcloud cis instance-set cis-demo
```

`ibmcloud cis instance-create`

Create a CIS service instance.

```
$ ibmcloud cis instance-create INSTANCE_NAME PLAN [--output FORMAT]
```

Command options

`INSTANCE_NAME`

The name of CIS service instance. Required.

`PLAN`

The name or ID of a service plan. Required.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a standard-next plan CIS instance `cis-demo`

```
$ ibmcloud cis instance-create cis-demo standard-next
```

`ibmcloud cis instance-delete`

Delete a CIS service instance.

```
$ ibmcloud cis instance-delete INSTANCE [-f, --force]
```

Command options

`INSTANCE`

The name or ID of a CIS service instance. Required.

`-f, --force`

Delete instance without prompting for confirmation.

Examples

Delete CIS instance `cis-demo`

```
$ ibmcloud cis instance-delete cis-demo --force
```

`ibmcloud cis instance-update`

Update a CIS service instance.

```
$ ibmcloud cis instance-update INSTANCE [--name NAME] [--plan PLAN] [--output FORMAT]
```

Command options

INSTANCE

The name or ID of a CIS service instance. Required.

NAME

The name of CIS service instance.

PLAN

The name or ID of a service plan.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update cis instance `cis-demo` to enterprise-usage plan.

```
$ ibmcloud cis instance-update cis-demo --plan enterprise-usage
```

ibmcloud cis instance

Show details of a CIS service instance.

```
$ ibmcloud cis instance INSTANCE [--output FORMAT]
```

Command options

INSTANCE

The name or ID of a CIS service instance. Required.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show details of cis instance `cis-demo`.

```
$ ibmcloud cis instance cis-demo
```

ibmcloud cis plans

List all CIS service plans.

```
$ ibmcloud cis plans [--refresh] [--output FORMAT]
```

Command options

--refresh

Force refresh from catalog.

--output

The output format. Currently, `json` is the only supported value.

Examples

List CIS services plans.


```
$ ibmcloud cis plans --refresh
```

Routing

Manipulate routing by using the following `routing` commands.

ibmcloud cis routing

Enterprise Plans Only

Get details of Routing settings.

```
$ ibmcloud cis routing DNS_DOMAIN_ID (--smart-routing | --tiered-caching) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--smart-routing`

Uses real-time network intelligence to route traffic across paths from the origin to a CIS data center.

`--tiered-caching`

Uses regional Tier 1 CIS data centers to accelerate content delivery.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get the details of routing settings for domain.

```
$ ibmcloud cis routing DNS_DOMAIN_ID --smart-routing
```

ibmcloud cis routing-update

Enterprise Plans Only

Update Routing setting.

```
$ ibmcloud cis routing-update DNS_DOMAIN_ID (--smart-routing (on|off) | --tiered-caching (on|off)) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--smart-routing`

Uses real-time network intelligence to route traffic across paths from the origin to a CIS data center. Valid values: `on`, `off`.

`--tiered-caching`

Uses regional Tier 1 CIS data centers to accelerate content delivery. Valid values are `on` and `off`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update routing settings for domain `372e67954025e0ba6aaa6d586b9e0b60`.

```
$ ibmcloud cis routing-update 31984fea73a15b45779fa0df4ef62f9b --smart-routing on --instance "cis-demo"
```

ibmcloud cis routing-analytics

Enterprise Plans Only

Get analytics of smart-routing latency.

```
$ ibmcloud cis routing-analytics DNS_DOMAIN_ID [--colos] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--colos

Analytics of smart-routing latency colos.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Get analytics of smart-routing latency for domain `372e67954025e0ba6aaa6d586b9e0b60`.

```
$ ibmcloud cis routing-analytics 31984fea73a15b45779fa0df4ef62f9b --instance "cis-demo"
```

Security events (Deprecated)

Security events are deprecated on 26 October 2020. Use [ibmcloud cis firewall-event-analytics](#) instead. Manage how the Security Events performs by using the following `security-events` command:

ibmcloud cis security-events

The `security-events` command is replacing the `firewall-events` command. It can pull up to 30 days of security events which might be triggered from a wider variety of sources (other than firewall) such as rate-limiting, L7 DDoS, and browser-integrity-check. With the new `security-events` command, you are able to list only firewall events by specifying the `--source` options.

Retrieve a full log of security events include Firewall Rules, Rate Limiting, Security Level, Access Rules, WAF, User Agent Blocking, Zone Lockdown, and Advanced DDoS Protection.

```
$ ibmcloud cis security-events DNS_DOMAIN_ID [--ip-class IP_CLASS] [--method METHOD] [--scheme SCHEME] [--ip IP_ADDR] [--host HOSTNAME] [--protocol PROTOCOL] [--uri URI] [--ua USER_AGENT] [--colo COLO] [--ray-id RAY_ID] [--kind KIND] [--action ACTION] [--cursor CURSOR] [--country COUNTRY] [--since START_DATE] [--until END_DATE] [--source SOURCE] [--limit LIMIT] [--rule_id RULE_ID] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--ip-class

IP class is a map of client IP to visitor classification. Valid values are `unknown`, `clean`, `badHost`, `searchEngine`, `whitelist`, `greylist`, `monitoringService`, `securityScanner`, `noRecord`, `scan`, `backupService`, `mobilePlatform`, `tor`.

`--method`

The HTTP method of the request. Valid values are `GET`, `POST`, `DELETE`, `PUT`, `HEAD`, `PURGE`, `OPTIONS`, `PROPFIND`, `MKCOL`, `PATCH`, `ACL`, `BCOPY`, `BDELETE`, `BMOVE`, `BPROPFIND`, `BPROPPATCH`, `CHECKIN`, `CHECKOUT`, `CONNECT`, `COPY`, `LABEL`, `LOCK`, `MERGE`, `MKACTIVITY`, `MKWORKSPACE`, `MOVE`, `NOTIFY`, `ORDERPATCH`, `POLL`, `PROPPATCH`, `REPORT`, `SEARCH`, `SUBSCRIBE`, `TRACE`, `UNCHECKOUT`, `UNLOCK`, `UNSUBSCRIBE`, `UPDATE`, `VERSION-CONTROL`, `BASELINE-CONTROL`, `X-MS-ENUMATTS`, `RPC_OUT_DATA`, `RPC_IN_DATA`, `JSON`, `COOK`, `TRACK`.

`--scheme`

The scheme of the URI. Valid values are `unknown`, `http` and `https`.

`--ip`

The IPv4 or IPv6 address from which the request originated.

`--host`

The hostname the request attempted to access.

`--protocol`

The protocol of the request. Valid values are `UNK`, `HTTP/1.0`, `HTTP/1.1`, `HTTP/1.2`, `HTTP/2` and `SPDY/3.1`.

`--uri`

The URI requested from the hostname.

`--ua`

The client user agent that initiated the request.

`--colo`

The 3-letter airport code of the Cloudflare data center that handled the request. For example, `SJC`.

`--ray-id`

Ray ID of the request.

`--action`

What type of action was taken. Valid values are `unknown`, `allow`, `drop`, `challenge`, `jschallenge`, `simulate`, `connectionClose` and `log`.

`--cursor`

Cursor position and direction for requesting the next set of records when the number of results are limited by the `limit` parameter. A valid value for the cursor can be obtained from the `cursors` object in the `result_info` structure.

`--country`

The 2-digit country code in which the request originated. For example, `US`.

`--since`

Start date and time of requesting data period in the ISO8601 format. Can't go back more than a year. For example, `2016-11-11T12:00:00Z`.

`--until`

End date and time of requesting data period in the ISO8601 format. For example, `2016-11-11T12:00:00Z`.

`--source`

Source of the event. Valid values are `unknown`, `asn`, `country`, `ip`, `ipRange`, `securityLevel`, `zoneLockdown`, `waf`, `uaBlock`, `rateLimit`, `firewallRules`, `bic`, `hot`, and `I7ddos`.

`--limit`

The number of events to return. The `cursor` attribute can be used to iterate over the next batch of events, if there are more events in the queried time range. Note that the `scanned_range` parameter in the `result_info` structure gives an indication of when events were considered in the current resultset if a limit was applied. Valid values are from 10 to 1000. Default value: 50.

`--rule-id`

The ID of the rule that triggered the event, which must be considered in the context of source.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get security events for domain `31984fea73a15b45779fa0df4ef62f9b` .

```
$ ibmcloud cis security-events 31984fea73a15b45779fa0df4ef62f9b --action challenge --colo SJC --country US --host "www.example.com" --ip-class clean
--method POST --ray-id 187d944c61940c77 --cursor
"6yDGxLKVeeHZZmORS_8XeSuhz9SjlJRaSa2lnsF01tQOHrfTGAP3R5X1Kv5iVUuMbNKhWNAXHOI6ePB0TUL8nw" -i "cis-demo"
```

TLS

Manipulate TLS by using the following `tls` commands.

ibmcloud cis tls-settings

Get TLS settings for a domain.

```
$ ibmcloud cis tls-settings DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get TLS settings for domain `372e67954025e0ba6aaa6d586b9e0b60` .

```
$ ibmcloud cis tls-settings 31984fea73a15b45779fa0df4ef62f9b --instance "cis-demo"
```

ibmcloud cis tls-settings-update

Update TLS settings for a DNS domain.

```
$ ibmcloud cis tls-settings-update DNS_DOMAIN_ID [--mode MODE] [--universal (true|false)] [--tls-1-2-only (on|off)] [--tls-1-3 (on|off)] [-i, --instance INSTANCE][--output
FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--mode`

Specify whether visitors can browse your website over a secure connection, and when they do, how CIS connects to your origin server. Valid values are `off` , `client-to-edge` , `end-to-end-flexible` , `end-to-end-ca-signed` , `https-only-origin-pull` . See the following documentation link for detailed [TLS mode description](#).

`--universal`

Specify whether universal ssl is enabled for your domain. Valid values are `true` and `false` .

`--tls-1-2-only`

Specify whether Crypto TLS 1.2 feature is enable for your domain. Enabling this feature prevents use of previous versions. Valid values are `on` and `off` .

`--tls-1-3`

Specify whether Crypto TLS 1.3 feature is enabled for your domain. Valid values are `on` , `off` .

`--min-tls-version`

Only accept HTTPS requests that use at least the TLS protocol version specified. Valid values are `1.0`, `1.1`, `1.2`, `1.3`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update TLS settings for the domain `372e67954025e0ba6aaa6d586b9e0b60`.

```
$ ibmcloud cis tls-settings-update 31984fea73a15b45779fa0df4ef62f9b --mode end-to-end-ca-signed --tls-1-2-only on --instance "cis-demo"
```

ibmcloud cis certificates

List all certificates for a DNS domain, including shared, dedicated, and custom certificates.

```
$ ibmcloud cis certificates DNS_DOMAIN_ID [--keyless] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--keyless`

List all keyless certificates.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List all certificates for domain `31984fea73a15b45779fa0df4ef62f9b`.

```
$ ibmcloud cis certificates 31984fea73a15b45779fa0df4ef62f9b --instance "cis-demo"
```

ibmcloud cis certificate

Get the details of a shared, dedicated, or custom certificate.

```
$ ibmcloud cis certificate DNS_DOMAIN_ID (--cert-id CERT_ID | --universal) [--keyless] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--cert-id`

ID of the dedicated or custom certificate.

`--universal`

Show universal certificate details.

`--keyless`

Show keyless certificate details.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Show details for a certificate.

```
ibmcloud cis certificate 31984fea73a15b45779fa0df4ef62f9b --universal --instance "cis-demo"
ibmcloud cis certificate 31984fea73a15b45779fa0df4ef62f9b --cert-id 5a7805061c76ada191ed06f989cc3dac --instance "cis-demo"
ibmcloud cis certificate 31984fea73a15b45779fa0df4ef62f9b --cert-id 5a7805061c76ada191ed06f989cc3dac --keyless --instance "cis-demo"
```

ibmcloud cis certificate-order

Order a certificate pack with an optional list of hostnames for a DNS domain.

```
$ ibmcloud cis certificate-order DNS_DOMAIN_ID [--hostnames host1 --hostnames host2 ...] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--hostnames`

Valid host names for the certificate packs. Add up to 50 custom hostnames - Can affect the price.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Order a certificate pack for domain `31984fea73a15b45779fa0df4ef62f9b`.

```
$ ibmcloud cis certificate-order 31984fea73a15b45779fa0df4ef62f9b --hostnames www.example.com --hostnames api.example.com --instance "cis-demo"
```

ibmcloud cis certificate-upload

Upload a custom certificate for a DNS domain.

```
$ ibmcloud cis certificate-upload DNS_DOMAIN_ID [--keyless] (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis certificate-upload DNS_DOMAIN_ID [--keyless] (-s, --json-str JSON_STR | -, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--keyless`

Upload a keyless certificate.

`--json`

The JSON file or JSON string that is used to describe a custom certificate. Required.

- The required fields in JSON data are `certificate`, `private_key`, `host` and `port` :
 - `certificate` : SSL certificate or certificate and one or more intermediates for the domain

- `certificate` : SSL certificate or certificate and one or more intermediates for the domain.
- `private_key` : Private key for the domain.
- `host` : The keyless SSL host name.
- `port` : The keyless SSL port used to communicate between CIS and the client's Keyless SSL server.
- The optional fields are `bundle_method` and `name` :
 - `bundle_method` : Bundle method, default value is `compatible`, valid values are `compatible`, `modern` and `user-defined`.
 - `name` : The keyless SSL name.

Sample JSON data:

```
{
  "certificate": "xxx",
  "private_key": "xxx",
  "bundle_method": "compatible"
}
```

For keyless ssl

```
{
  "host": "www.example.com",
  "port": 8000,
  "certificate": "xxx",
  "bundle_method": "user-defined",
  "name": "test"
}
```

`-s, --json-str`

Deprecated. The JSON data used to upload a custom certificate.

`-j, --json-file`

Deprecated. A file contains input JSON data.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Upload a custom certificate for domain `31984fea73a15b45779fa0df4ef62f9b`.

```
$ ibmcloud cis certificate-upload 31984fea73a15b45779fa0df4ef62f9b --json '{"certificate": "-----BEGIN CERTIFICATE-----
\nMIIDtTCCAp2gAwIBAgIJAMHAWfXZ5/PWMA0GCSqGSIb3DQEBCwUAMEUxCzAJBgNV\nBAYTAkFVMRMwEQYDVQQIEwpTb21lLVN0YXRIMSEwHwYDVQQKEzhJbnRlcm5ldCBX\
-----END CERTIFICATE-----\n", "private_key": "-----BEGIN RSA PRIVATE KEY-----
\nMIIIEowIBAAKCAQEAwQHoeetcl9+5ikGzV6cMzWtWPJHqXT3wpbEkRU9Yz7lgvddmG\ndtcGbg/1CGZu0jJGkMoppoUo4c3dts3iwqRYmBikUP77wwY2QGmDZw2FvkJCJIKn\nabIRuGvE
-----END RSA PRIVATE KEY-----\n", "bundle_method": "compatible"}' -i "cis-demo"
```

ibmcloud cis certificate-update

Update a custom certificate for a DNS domain.

```
$ ibmcloud cis certificate-update DNS_DOMAIN_ID CERT_ID [--keyless] (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis certificate-update DNS_DOMAIN_ID CERT_ID [--keyless] [-s, --json-str JSON_STR | -j, --json-file JSON_FILE] [-i, --instance INSTANCE] [--
output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

CERT_ID

The ID of the custom certificate. Required.

--keyless

Update a keyless certificate.

--json

The JSON file or JSON string that is used to describe a custom certificate. Required.

- The required fields in JSON data are `certificate`, `private_key`, `host` and `port` :
 - `certificate` : SSL certificate or certificate and one or more intermediates for the domain.
 - `private_key` : Private key for the domain.
 - `host` : The keyless SSL host name.
 - `port` : The keyless SSL port used to communicate between CIS and the client's Keyless SSL server.
- The optional fields are `bundle_method` :
 - `bundle_method` : Bundle method, default value is `compatible`, valid values are `compatible`, `modern` and `user-defined`.
 - `name` : The keyless SSL name.

Sample JSON data:

```
{
  "certificate": "xxx",
  "private_key": "xxx",
  "bundle_method": "compatible"
}

For keyless ssl
{
  "host": "www.example.com",
  "port": 8000,
  "certificate": "xxx",
  "bundle_method": "user-defined",
  "name": "test"
}
```

-s, --json-str

Deprecated. The JSON data used to update a custom certificate.

-j, --json-file

Deprecated. A file contains input JSON data.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update the custom certificate `5a7805061c76ada191ed06f989cc3dac` .

```
$ ibmcloud cis certificate-update 31984fea73a15b45779fa0df4ef62f9b 5a7805061c76ada191ed06f989cc3dac --json '{"certificate": "-----BEGIN CERTIFICATE-----
\nMIIDTCCAp2gAwIBAgIJAMHAWfXZ5/PWMA0GCSqGSIb3DQEBCwUAMEUxCzAJBgNV\nBAYTAkFVMRMwEQYDVQQIEwpTb21lLVN0YXRIMSEwHwYDVQQKEzhJbnRlcm5ldCBX\
-----END CERTIFICATE-----\n", "private_key": "-----BEGIN RSA PRIVATE KEY-----
\nMIIIEowIBAAKCAQEAwQHoetcl9+5ikGzV6cMzWtWPJHqXT3wpbEkRU9Yz7lgvddmG\ndtcGbg/1CGZu0jJGkMoppoUo4c3dts3iwqRYmBikUP77wwY2QGmDZw2FvkJCJlKn\nabIRuGvE
-----END RSA PRIVATE KEY-----\n", "bundle_method": "compatible"}' -i "cis-demo"
```

ibmcloud cis certificate-priority-change

Change custom certificates' priority for a DNS domain.


```
$ ibmcloud cis certificate-priority-change DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
[Deprecated] ibmcloud cis certificate-priority-change DNS_DOMAIN_ID (-s, --json-str JSON_STR | -j, --json-file JSON_FILE) [-i, --instance INSTANCE] [--output
FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--json

The JSON file or JSON string that is used to describe the custom certificates' priority. Required.

- The required fields in JSON data are **certificates** :
 - **certificates** : An array of objects with the following fields.
 - **id** : Custom certificate identifier.
 - **priority** : The order or priority in which the certificate is used in a request. Higher numbers are tried first.

Sample JSON data:

```
{
"certificates":[
  {
    "id":"5a7805061c76ada191ed06f989cc3dac",
    "priority":2
  },
  {
    "id":"da534493b38266b17fea74f3312be21c",
    "priority":1
  }
]
}
```

-s, --json-str

Deprecated. The JSON data used to change the custom certificates' priority.

-j, --json-file

Deprecated. A file contains input JSON data.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

Change custom certificates' priority for domain.

```
$ ibmcloud cis certificate-priority-change DNS_DOMAIN_ID --json '{"certificates": [{"id":"5a7805061c76ada191ed06f989cc3dac", "priority":2}, {"id":"9a7806061c88ada191ed06f989cc3dac", "priority":1}]}'
```

ibmcloud cis certificate-delete

Delete a dedicated or custom certificate.

```
$ ibmcloud cis certificate-delete DNS_DOMAIN_ID CERT_ID [--keyless][-i, --instance INSTANCE]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

CERT_ID

The ID of the dedicated or custom certificate. Required.

--keyless

Delete a keyless certificate.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

Examples

Delete the custom certificate.

```
$ ibmcloud cis certificate-delete DNS_DOMAIN_ID CERT_ID
```

Web application firewall (WAF)

Manage Web Application Firewalls by using the following `waf` commands.

ibmcloud cis waf-setting

Show WAF setting.

```
$ ibmcloud cis waf-setting DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show WAF settings for domain.

```
$ ibmcloud cis waf-setting DNS_DOMAIN_ID
```

ibmcloud cis waf-setting-update

Update the WAF setting.

```
$ ibmcloud cis waf-setting-update DNS_DOMAIN_ID WAF_MODE [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_ID

The ID of the DNS domain. Required.

WAF_MODE

The mode of WAF setting. Valid values are `waf-enable` and `waf-disable`. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Enable WAF for domain.

```
$ ibmcloud cis waf-setting-update DNS_DOMAIN_ID waf-enable
```

ibmcloud cis waf-packages

List all WAF packages.

```
$ ibmcloud cis waf-packages DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List all WAF packages for domain.

```
$ ibmcloud cis waf-packages DNS_DOMAIN_ID
```

ibmcloud cis waf-package

Get detail of a WAF package.

```
$ ibmcloud cis waf-package DNS_DOMAIN_ID WAF_PACKAGE_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`WAF_PACKAGE_ID`

The ID of the WAF package. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get detail of a WAF package.


```
$ ibmcloud cis waf-package DNS_DOMAIN_ID WAF_PACKAGE_ID
```

ibmcloud cis waf-package-set

Update OWASP Package setting.

```
$ ibmcloud cis waf-package-set DNS_DOMAIN_ID OWASP_PACKAGE_ID [--sensitivity SENSITIVITY] [--action-mode ACTION_MODE] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

OWASP_PACKAGE_ID

The ID of the WAF package. Required.

--sensitivity

The sensitivity of the firewall package. Valid values are **high**, **medium**, **low**, and **off**.

--action-mode

The default action that is taken for rules under the firewall package. Valid values are **simulate** and **block**, **challenge**.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

Update the OWASP setting for package.

```
$ ibmcloud cis waf-package-set DNS_DOMAIN_ID OWASP_PACKAGE_ID --sensitivity medium --action-mode simulate
```

ibmcloud cis waf-groups

List the WAF groups in a WAF package.

```
$ ibmcloud cis waf-groups DNS_DOMAIN_ID WAF_PACKAGE_ID [--page PAGE] [--per-page NUM] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

WAF_PACKAGE_ID

The ID of the WAF package. Required.

--page

The page number of paginated results. The default value is **1**.

--per-page

The number of groups per page. The min value is **5** and the max value is **1000**. The default value is **50**.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

List the WAF groups in a WAF package.

```
$ ibmcloud cis waf-groups DNS_DOMAIN_ID WAF_PACKAGE_ID --page 1 --per-page 100
```

ibmcloud cis waf-group

Get detail of a WAF group.

```
$ ibmcloud cis waf-group DNS_DOMAIN_ID WAF_PACKAGE_ID WAF_GROUP_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

WAF_PACKAGE_ID

The ID of the WAF package. Required.

WAF_GROUP_ID

The ID of the WAF group.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Get details of WAF group in WAF package.

```
$ ibmcloud cis waf-group DNS_DOMAIN_ID WAF_PACKAGE_ID WAF_GROUP_ID
```

ibmcloud cis waf-group-mode-set

Set the mode of a WAF group.

```
$ ibmcloud cis waf-group-mode-set DNS_DOMAIN_ID WAF_PACKAGE_ID WAF_GROUP_ID WAF_GROUP_MODE [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

WAF_PACKAGE_ID

The ID of the WAF package. Required.

WAF_GROUP_ID

The ID of the WAF group. Required.

WAF_GROUP_MODE

The mode of WAF group. Valid values are `on` and `off`. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Enable the WAF group in the WAF package.

```
$ ibmcloud cis waf-group-mode-set DNS_DOMAIN_ID WAF_PACKAGE_ID WAF_GROUP_ID on
```

ibmcloud cis waf-rules

List all WAF rules of a WAF package.

```
$ ibmcloud cis waf-rules DNS_DOMAIN_ID WAF_PACKAGE_ID [--page PAGE] [--per-page NUM] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

WAF_PACKAGE_ID

The ID of the WAF package. Required.

--page

Page number of the paginated results. The default value is **1**.

--per-page

Number of rules per page. The default value is **50**.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

List all WAF rules in the WAF package.

```
$ ibmcloud cis waf-rules DNS_DOMAIN_ID WAF_PACKAGE_ID --page 1 --per-page 100
```

ibmcloud cis waf-rule

Get detail of a WAF rule.

```
$ ibmcloud cis waf-rule DNS_DOMAIN_ID WAF_PACKAGE_ID WAF_RULE_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

WAF_PACKAGE_ID

The ID of WAF package. Required.

WAF_RULE_ID

The ID of WAF rule. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

Get details of WAF rule in WAF package.

```
$ ibmcloud cis waf-rule DNS_DOMAIN_ID WAF_PACKAGE_ID WAF_RULE_ID
```

ibmcloud cis waf-rule-mode-set

Set the mode of a WAF rule.

```
$ ibmcloud cis waf-rule-mode-set DNS_DOMAIN_ID WAF_PACKAGE_ID WAF_RULE_ID WAF_RULE_MODE [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

WAF_PACKAGE_ID

The ID of the WAF package. Required.

WAF_RULE_ID

The ID of the WAF rule. Required.

WAF_RULE_MODE

The mode of WAF rule. Valid values are `on` , `off` , `default` , `disable` , `simulate` , `block` , and `challenge` . Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Disable WAF rule in WAF package.

```
$ ibmcloud cis waf-rule-mode-set DNS_DOMAIN_ID WAF_PACKAGE_ID WAF_RULE_ID disable
```

ibmcloud cis cis waf-override-create

Create a URL-based Web Application Firewall (WAF) rule.

```
$ ibmcloud cis waf-override-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

--json

The JSON file or JSON string that is used to describe an override WAF rule. Required.

- The required fields in JSON data are `urls` and `rules` .
 - `urls` : URLs to be included in this rule definition. Wildcards are permitted.
 - `rules` : Change the action that is assigned to a WAF rule. The keys of this object are WAF rule IDs and the values must be a valid WAF action. Unless disabling the rule, ensure that you also enable the rule group that this WAF rule belongs to. The max length is `1024` .
- The optional fields are `paused` , `description` , `priority` , `groups` , and `rewrite_action` .
 - `paused` : Whether this package is currently paused. Valid values are `true` and `false` .
 - `description` : A note that you can use to describe the purpose of this rule.
 - `priority` : Relative priority of this configuration when multiple configurations match a single URL. Higher priority configurations

might overwrite values set by lower priority configurations. The min value is `-1000000000` and the max value is `1000000000` .

- `groups` Enable or disable WAF rule groups. The keys of this object are WAF rule group IDs and the values must be a valid WAF action (usually `default` or `disable`).
- `rewrite_action` : When a WAF rule matches, substitute its configured action for a different action that is specified by this object.

Sample JSON data:

```
{
  "description": "Enable IBM Magento ruleset for www.example.com",
  "urls": [
    "www.example.com/*"
  ],
  "priority": 1,
  "groups": {
    "ea8687e59929c1fd05ba97574ad43f77": "default"
  },
  "rules": {
    "100015": "disable"
  },
  "rewrite_action": {
    "default": "block",
    "challenge": "block",
    "simulate": "disable"
  }
}
```

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a WAF override rule.

```
$ ibmcloud cis waf-override-create DNS_DOMAIN_ID --json '{"description":"Enable IBM Magento ruleset for www.example.com","urls":["www.example.com/*"],"priority":1,"groups":{"ea8687e59929c1fd05ba97574ad43f77":"default"},"rules":{"100015":"disable"},"rewrite_action":{"default":"block","challenge":"block","simulate":"disable"}}'
```

ibmcloud cis cis waf-override-update

Update a URL-based Web Application Firewall (WAF) rules.

```
$ ibmcloud cis waf-override-update DNS_DOMAIN_ID OVERRIDE_WAF_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`OVERRIDE_WAF_ID`

The ID of the override WAF rule. Required.

`--json`

The JSON file or JSON string that is used to describe a override WAF rule. Required.

- The required fields in JSON data are `urls` and `rules` .
 - `urls` : URLs to be included in this rule definition. Wildcards are permitted.
 - `rules` : Change the action that is assigned to a WAF rule. The keys of this object are WAF rule IDs and the values must be a valid

- WAF action. Unless you disable the rule, ensure that you also enable the rule group that this WAF rule belongs to. The max length is 1024 .
- The optional fields are `paused` , `description` , `priority` , `groups` , and `rewrite_action` .
 - `paused` : Whether this package is currently paused. Valid values are `true` and `false` .
 - `description` : A note that you can use to describe the purpose of this rule.
 - `priority` : Relative priority of this configuration when multiple configurations match a single URL. Higher priority configurations might overwrite values set by lower priority configurations. The min value is -1000000000 and the max value is 1000000000 .
 - `groups` Enable or disable WAF rule groups. The keys of this object are WAF rule group IDs and the values must be a valid WAF action (usually `default` or `disable`).
 - `rewrite_action` : When a WAF rule matches, substitute its configured action for a different action that is specified by this object.

Sample JSON data:

```
{
  "description": "Enable IBM Magento ruleset for www.example.com",
  "urls": [
    "www.example.com/*"
  ],
  "priority": 1,
  "groups": {
    "ea8687e59929c1fd05ba97574ad43f77": "default"
  },
  "rules": {
    "100015": "disable"
  },
  "rewrite_action": {
    "default": "block",
    "challenge": "block",
    "simulate": "disable"
  }
}
```

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update a WAF override rule.

```
$ ibmcloud cis waf-override-update DNS_DOMAIN_ID OVERRIDE_WAF_ID --json '{"description":"Enable IBM Magento ruleset for www.example.com","urls":["www.example.com/*"],"priority":1,"groups":{"ea8687e59929c1fd05ba97574ad43f77":"default"},"rules":{"100015":"disable"},"rewrite_action":{"default":"block","challenge":"block","simulate":"disable"}}'
```

ibmcloud cis waf-overrides

List all URL-based Web Application Firewall (WAF) rules.

```
$ ibmcloud cis waf-overrides DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--page`

Page number of paginated results. The default value is 1 .

`--per-page`

Number of rules per page. The default value is `50`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List WAF override rules.

```
$ ibmcloud cis waf-overrides DNS_DOMAIN_ID
```

ibmcloud cis cis waf-override

Get a URL-based Web Application Firewall (WAF) rule.

```
$ ibmcloud cis waf-override DNS_DOMAIN_ID OVERRIDE_WAF_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`OVERRIDE_WAF_ID`

The ID of override WAF rule. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get a WAF override rule.

```
$ ibmcloud cis waf-override DNS_DOMAIN_ID OVERRIDE_WAF_ID
```

ibmcloud cis cis waf-override-delete

Delete a URL-based Web Application Firewall (WAF) rule.

```
$ ibmcloud cis waf-override-delete DNS_DOMAIN_ID OVERRIDE_WAF_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`OVERRIDE_WAF_ID`

The ID of override WAF rule. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`-f, --force`

Attempt to delete the URL-based WAF rule without prompting for confirmation.

Examples

Delete a WAF override rule.

```
$ ibmcloud cis waf-override-delete DNS_DOMAIN_ID OVERRIDE_WAF_ID
```

Authenticated Origin Pull

Manage Authenticated Origin Pull by using the following `authenticated-origin-pull` commands.

`ibmcloud cis authenticated-origin-pull-settings`

Get authenticated origin pull settings for a domain.

```
$ ibmcloud cis authenticated-origin-pull-settings DNS_DOMAIN_ID [--level zone|hostname] [--hostname HOSTNAME] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`----level`

Specify the authenticated origin pull certificate or settings per zone or hostname level. Valid values are `zone` and `hostname`. The default value is `zone`.

`--hostname`

The authenticated origin pull settings on a hostname. (hostname level only)

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List authenticated origin pull settings on zone level for domain.

```
$ ibmcloud cis authenticated-origin-pull-settings DNS_DOMAIN_ID
```

`ibmcloud cis authenticated-origin-pull-setting-update`

Update authenticated origin pull settings for a domain.

```
$ ibmcloud cis authenticated-origin-pull-settings-update DNS_DOMAIN_ID [--level zone|hostname] [--hostname HOSTNAME] [--cert_id CERT_ID] [--enabled on|off] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`----level`

Specify the authenticated origin pull certificate or settings per zone or hostname level. Valid values are `zone` and `hostname`. The default value is `zone`.

`--hostname`

The authenticated origin pull settings on a hostname. (hostname level only)

----cert_id

The certificate id, which the hostname is bundled to. (hostname level only)

----enabled

Enable authenticated origin pull. Valid values are `on` and `off`.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update authenticated origin pull setting on zone level for domain.

```
$ ibmcloud cis authenticated-origin-pull-settings-update DNS_DOMAIN_ID --enabled on
```

ibmcloud cis authenticated-origin-pull-certificates

List zone-level authenticated origin pull certificates for a domain.

```
$ ibmcloud cis authenticated-origin-pull-certificates DNS_DOMAIN_ID [--level zone|hostname][-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

----level

Specify the authenticated origin pull certificate or settings per zone or hostname level. Valid values are `zone` and `hostname`. The default value is `zone`.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show authenticated origin pull certificates on zone level for domain.

```
$ ibmcloud cis authenticated-origin-pull-certificates DNS_DOMAIN_ID
```

ibmcloud cis authenticated-origin-pull-certificate

Get an authenticated origin pull certificate for a domain.

```
$ ibmcloud cis authenticated-origin-pull-certificate DNS_DOMAIN_ID CERT_ID [--level zone|hostname] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

CERT_ID

The ID of the certificate. Required.

----level

Specify the authenticated origin pull certificate or settings per zone or hostname level. Valid values are `zone` and `hostname`. The default value is `zone`.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Get an authenticated origin pull certificate on zone level for domain.

```
$ ibmcloud cis authenticated-origin-pull-certificate DNS_DOMAIN_ID CERT_ID
```

ibmcloud cis authenticated-origin-pull-certificate-upload

Upload an authenticated origin pull certificate for a domain.

```
$ ibmcloud cis authenticated-origin-pull-certificate-upload DNS_DOMAIN_ID [--level zone|hostname] (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [-output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain. Required.

`--level`

Specify the authenticated origin pull certificate or settings per zone or hostname level. Valid values are `zone` and `hostname`. The default value is `zone`.

`--json`

The JSON file or JSON string that is used to describe a custom certificate.

- The required fields in JSON data are `certificate` and `private_key` :
 - `certificate` : SSL certificate or certificate and one or more intermediates for the domain.
 - `private_key` : Private key for the domain.

Sample JSON data:

```
{
  "certificate": "-----BEGIN CERTIFICATE-----\n...-----END CERTIFICATE-----\n",
  "private_key": "-----BEGIN PRIVATE KEY-----\n...-----END PRIVATE KEY-----\n"
}
```

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Upload an authenticated origin pull certificate on zone level for domain.

```
$ ibmcloud cis authenticated-origin-pull-certificate-upload DNS_DOMAIN_ID --json '{"certificate": "-----BEGIN CERTIFICATE-----\nMIIDtTCCAp2gAwIBAgIJAMHAWfXZ5/PWMA0GCSqGSIb3DQEBCwUAMEUxCzAJBgNV\nBAYTAkFVMRMwEQYDVQQIEwpTb211LVN0YXRIMSEwHwYDVQQKEjhbnRlcm5ldCBX\n\
```

```
-----END CERTIFICATE-----\n", "private_key": "-----BEGIN RSA PRIVATE KEY-----\nMIIIEowIBAAKCAQEAwQHoetcI9+5ikGzV6cMzWtWPJHqXT3wpbEkRU9Yz7lgvddmG\ndtcGbg/1CGZu0jJGkMoppoUo4c3dts3iwqRYmBikUP77wwY2QGmDZw2FvkJCJlKn\nabiRuGvE\n-----END RSA PRIVATE KEY-----\n"}'
```

ibmcloud cis authenticated-origin-pull-certificate-delete

Delete the authenticated origin pull certificate for a domain.

```
$ ibmcloud cis authenticated-origin-pull-certificate-delete DNS_DOMAIN_ID CERT_ID [--level zone|hostname] [-i, --instance INSTANCE] [--output FORMAT] [-f, --force]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain. Required.

CERT_ID

The ID of certificate. Required.

--level

Specify the authenticated origin pull certificate or settings per zone or hostname level. Valid values are **zone** and **hostname**. The default value is **zone**.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

Delete authenticated origin pull certificate on zone level for domain.

```
$ ibmcloud cis authenticated-origin-pull-certificate-delete DNS_DOMAIN_ID CERT_ID
```

Alert policy

Manage alert policies.

ibmcloud cis alert-policy list (List)

List all alert policies.

```
$ ibmcloud cis alert-policy list [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

List all policies for instance.

```
$ ibmcloud cis alert-policy list
```

ibmcloud cis alert-policy get (Show)

Show the details of a policy.

```
$ ibmcloud cis alert-policy get POLICY_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

POLICY_ID

The ID of alert policy. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show the details of the alert policy.

```
$ ibmcloud cis alert-policy get POLICY_ID
```

ibmcloud cis alert-policy ddos-attack-l7-alert-create

Create an alert policy for DDoS attack l7.

```
$ ibmcloud cis alert-policy ddos-attack-l7-alert-create --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

--name

The name of the alert policy.

--description

The description for the alert policy.

--emails

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

--webhooks

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

--enabled

Whether the alert policy is enabled.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Create a DDoS attack alert policy.

```
$ ibmcloud cis alert-policy ddos-attack-l7-alert-create --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --enabled true --instance INSTANCE
```

ibmcloud cis alert-policy ddos-attack-l3-l4-alert-create

Create an alert policy for DDoS attack L3/L4.

```
$ ibmcloud cis alert-policy ddos-attack-l3-l4-alert-create --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

--name

The name of the alert policy.

--description

The description for the alert policy.

--emails

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

--webhooks

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

--enabled

Whether the alert policy is enabled.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Create a DDoS L3/L4 attack alert policy.

```
$ ibmcloud cis alert-policy ddos-attack-l3-l4-alert-create --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --enabled true --instance INSTANCE
```

ibmcloud cis alert-policy failing-logpush-job-alert-create

Create an alert policy when logpush job did not complete at least one successful push in the last 24 hours.

```
$ ibmcloud cis alert-policy failing-logpush-job-alert-create --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

--name

The name of the alert policy.

--description

The description for the alert policy.

--emails

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

--webhooks

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

--enabled

Whether the alert policy is enabled.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Create a failing logpush job disabled alert policy.

```
$ ibmcloud cis alert-policy failing-logpush-job-alert-create --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --enabled true --instance INSTANCE
```

ibmcloud cis alert-policy pool-toggle-alert-create (Pool toggle alert)

Create an alert policy for pool toggle alert.

```
$ ibmcloud cis alert-policy pool-toggle-alert-create --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) --pools POOLS --trigger-condition (enabled | disabled | either) [--include-future-pools (true | false)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

`--webhooks`

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

`--enabled`

Whether the alert policy is enabled.

`--pools`

The IDs of the origin pool, if set to all, the all pool IDs are used.

`--trigger-condition`

The condition of the pool toggle status.

`--include-future-pools`

Whether to include the future pools.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a pool toggle alert policy.

```
$ ibmcloud cis alert-policy pool-toggle-alert-create --name test1 --emails test1@cn.ibm.com --enabled true --pools all --trigger-condition enabled --include-future-pools true --instance INSTANCE
```

ibmcloud cis alert-policy firewall-events-alert-create

Create an alert policy about spikes in firewall events. Firewall events alerts use a [z-score] (https://en.m.wikipedia.org/wiki/Standard_score): external} calculation over the last six hours and five-minute buckets of events. An alert is triggered whenever the z-score is greater than the threshold of 3.5. You will not receive duplicate alerts within the same two-hour time frame.

```
$ ibmcloud cis alert-policy firewall-events-alert-create --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) --domains DOMAINS [--
```

```
services SERVICES] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

`--webhooks`

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

`--enabled`

Whether the alert policy is enabled.

`--domains`

The domain IDs that for the alert policy. For example, `--domains domainID1,domainID2`

`--services`

Specify which services the alert should monitor. Valid values are `country-access-rules`, `waf`, `firewall-rules`, `ratelimit`, `securitylevel`, `ip-access-rules`, `browser-integrity-check`, `ua-rules`, `lockdowns`, `iprange-access-rules`, `asn-access-rules`, `Managed-firewall` [Enterprise Plans Only](#)

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a firewall-events alert.

```
$ ibmcloud cis alert-policy firewall-events-alert-create --name test1 --emails test1@cn.ibm.com --enabled true --domains d2633e61-1b61-2512-1321-b61a17c3db7e --services waf,ratelimit --instance INSTANCE
```

ibmcloud cis alert-policy certificate-alert-create

Create an alert policy for certificate events.

```
$ ibmcloud cis alert-policy certificate-alert-create --type (universal | dedicated | mtls ) --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`--type`

The type of the certificate.

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

`--webhooks`

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

`--enabled`

Whether the alert policy is enabled.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a certificate alert.

```
$ ibmcloud cis alert-policy certificate-alert-create --type universal --name test1 --emails test1@cn.ibm.com --enabled true --instance INSTANCE
```

ibmcloud cis alert-policy glb-healthcheck-alert-create

Create an alert policy for changes in health status for global load balancer, pools, and origins.

```
$ ibmcloud cis alert-policy glb-healthcheck-alert-create --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) --pools POOLS [--include-future-pools (true | false)] [--health-status-trigger (healthy | unhealthy | either)] [--event-source-trigger (pool | origin | either)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

`--webhooks`

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

`--enabled`

Whether the alert policy is enabled.

`--pools`

The IDs of origin pool. If set to `all`, all pool IDs are used.

`--include-future-pools`

Whether to include the future pools. (The default value is `false`)

`--health-status-trigger`

The trigger condition to fire the notification. Valid values are `healthy`, `unhealthy`, and `either`. (The default value is `either`)

`--event-source-trigger`

The event source of trigger to fire the notification. Valid values are `pool`, `origin`, and `either`. (The default value is `either`)

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a glb healthcheck alert policy.


```
$ ibmcloud cis alert-policy glb-healthcheck-alert-create --name test1 --emails test1@cn.ibm.com --enabled true --pools all --include-future-pools true --instance INSTANCE
```

ibmcloud cis alert-policy web-analytics-alert-create

Create an alert policy for web metrics report.

```
$ ibmcloud cis alert-policy web-analytics-alert-create --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

`--webhooks`

The webhook ID for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

`--enabled`

Whether the alert policy is enabled.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a web metrics report alert policy.

```
$ ibmcloud cis alert-policy web-analytics-alert-create --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --enabled true --instance INSTANCE
```

ibmcloud cis alert-policy maintenance-event-alert-create

Create an alert policy for maintenance event.

```
$ ibmcloud cis alert-policy maintenance-event-alert-create --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) --event-type TYPE [--airport-code AIRPORT_CODE] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

`--webhooks`

The webhook ID for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

`--enabled`

Whether the alert policy is enabled.

`--event-type`

The type of the maintenance event. Valid values are `scheduled` , `changed` , and `canceled` .

`--airport-code`

Comma-separated three-letter IATA Codes.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a maintenance event alert policy.

```
$ ibmcloud cis alert-policy maintenance-event-alert-create --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --event-type scheduled,changed,canceled --airport-code IAD,AUS --enabled true
```

ibmcloud cis alert-policy ddos-attack-l7-alert-update

Update an alert policy for DDoS attack l7.

```
$ ibmcloud cis alert-policy ddos-attack-l7-alert-update POLICY_ID [--name NAME] [--emails EMAILS] [--webhooks WEBHOOKS] [--enabled (true | false)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`POLICY_ID`

The ID of alert policy. Required.

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

`--webhooks`

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

`--enabled`

Whether the alert policy is enabled.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update a DDoS attack alert policy.

```
$ ibmcloud cis alert-policy ddos-attack-l7-alert-update POLICY_ID --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --enabled true
```

ibmcloud cis alert-policy ddos-attack-l3-l4-alert-update

Update an alert policy for DDoS attack L3/L4.

```
$ ibmcloud cis alert-policy ddos-attack-l3-l4-alert-update POLICY_ID [--name NAME] [--emails EMAILS] [--webhooks WEBHOOKS] [--enabled (true | false)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

POLICY_ID

The ID of alert policy. Required.

--name

The name of the alert policy.

--description

The description for the alert policy.

--emails

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

--webhooks

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

--enabled

Whether the alert policy is enabled.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update a DDoS attack alert policy.

```
$ ibmcloud cis alert-policy ddos-attack-l3-l4-alert-update POLICY_ID --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --enabled true
```

ibmcloud cis alert-policy failing-logpush-job-alert-update

Update an alert policy when logpush job did not complete at least one successful push in the last 24 hours.

```
$ ibmcloud cis alert-policy failing-logpush-job-alert-update POLICY_ID [--name NAME] [--emails EMAILS] [--webhooks WEBHOOKS] [--enabled (true | false)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

POLICY_ID

The ID of alert policy. Required.

--name

The name of the alert policy.

--description

The description for the alert policy.

--emails

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

--webhooks

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

--enabled

Whether the alert policy is enabled.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update a failing logpush job alert policy.

```
$ ibmcloud cis alert-policy failing-logpush-job-alert-update POLICY_ID --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e -
-enabled true
```

ibmcloud cis alert-policy pool-toggle-alert-update

Update an alert policy for pool toggle alert.

```
$ ibmcloud cis alert-policy pool-toggle-alert-update POLICY_ID --name NAME (--emails EMAILS | --webhooks WEBHOOKS) --enabled (true | false) --pools POOLS --
trigger-condition (enabled | disabled | either) [--include-future-pools (true | false)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`POLICY_ID`

The ID of alert policy. Required.

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

`--webhooks`

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

`--enabled`

Whether the alert policy is enabled.

`--pools`

The IDs of the origin pool. If set to `all`, all pool IDs are used.

`--trigger-condition`

The condition of the pool toggle status.

`--include-future-pools`

Whether to include the future pools.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update a pool toggle alert policy.

```
$ ibmcloud cis alert-policy pool-toggle-alert-update POLICY_ID --name test1 --emails test1@cn.ibm.com --enabled true --pools all --trigger-condition enabled --include-
future-pools true
```

ibmcloud cis alert-policy firewall-events-alert-update

Update an alert policy about spikes in firewall events.

```
$ ibmcloud cis alert-policy firewall-events-alert-update POLICY_ID [--name NAME] [--emails EMAILS] [--webhooks WEBHOOKS] [--enabled (true | false)] [--domains DOMAINS] [--services SERVICES] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

POLICY_ID

The ID of alert policy. Required.

--name

The name of the alert policy.

--description

The description for the alert policy.

--emails

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

--webhooks

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

--enabled

Whether the alert policy is enabled.

--domains

The domain IDs that for the alert policy. For example, `--domains domainID1,domainID2`

--services

Specify which services the alert must monitor. Valid values are `country-access-rules`, `waf`, `firewall-rules`, `ratelimit`, `securitylevel`, `ip-access-rules`, `browser-integrity-check`, `ua-rules`, `lockdowns`, `iprange-access-rules`, `asn-access-rules`, `Managed-firewall` [Enterprise Plans Only](#)



Note: The 'SERVICES' is only used for advanced waf alert. If the alert policy you wanted to update is created without services that are specified, create a new one with sevices specified instead of updating.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update a firewall-events-alert policy.

```
$ ibmcloud cis alert-policy firewall-events-alert-update POLICY_ID --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --enabled true --domains d2633e61-1b61-2512-1321-b61a17c3db7e
```

ibmcloud cis alert-policy certificate-alert-update

Update an alert policy for certificate events.

```
$ ibmcloud cis alert-policy certificate-alert-update POLICY_ID [--name NAME] [--emails EMAILS] [--webhooks WEBHOOKS] [--enabled (true | false)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```


Command options

POLICY_ID

The ID of alert policy. Required.

--name

The name of the alert policy.

--description

The description for the alert policy.

--emails

The email addresses for dispatching an alert notification. For example, **--emails test1@cn.ibm.com,test2@cn.ibm.com**

--webhooks

The webhook ID that for dispatching an alert notification. For example, **--webhook webhookID1,webhookID2**

--enabled

Whether the alert policy is enabled.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by **ibmcloud cis instance-set INSTANCE** is used.

--output

The output format. Currently, **json** is the only supported value.

Examples

Update a certificate alert policy.

```
$ ibmcloud cis alert-policy certificate-alert-update POLICY_ID --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --enabled true
```

ibmcloud cis alert-policy glb-healthcheck-alert-update

Update an alert policy for changes in health status for global load balancer, pools, and origins.

```
$ ibmcloud cis alert-policy glb-healthcheck-alert-update POLICY_ID [--name NAME] [--emails EMAILS] [--webhooks WEBHOOKS] [--enabled (true | false)] [--pools POOLS] [--include-future-pools (true | false)] [--health-status-trigger (healthy | unhealthy | either)] [--event-source-trigger (pool | origin | either)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

POLICY_ID

The ID of alert policy. Required.

--name

The name of the alert policy.

--description

The description for the alert policy.

--emails

The email addresses for dispatching an alert notification. For example, **--emails test1@cn.ibm.com,test2@cn.ibm.com**

--webhooks

The webhook ID that for dispatching an alert notification. For example, **--webhook webhookID1,webhookID2**

--enabled

Whether the alert policy is enabled.

--pools

The IDs of origin pool. If set to **all**, all pool IDs are used.

`--include-future-pools`

Whether to include the future pools. (The default value is `false`)

`--health-status-trigger`

The trigger condition to send the notification. Valid values are `healthy` , `unhealthy` , and `either` . (The default value is `either`)

`--event-source-trigger`

The event source of trigger to send the notification. Valid values are `pool` , `origin` , and `either` . (The default value is `either`)

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update a certificate alert policy.

```
$ ibmcloud cis alert-policy glb-healthcheck-alert-update POLICY_ID --name test1 --emails test1@cn.ibm.com --enabled true --pools all --include-future-pools true
```

ibmcloud cis alert-policy web-analytics-alert-update

Update an alert policy for web metric report.

```
$ ibmcloud cis alert-policy web-analytics-alert-update POLICY_ID [--name NAME] [--emails EMAILS] [--webhooks WEBHOOKS] [--enabled (true | false)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`POLICY_ID`

The ID of alert policy. Required.

`--name`

The name of the alert policy.

`--description`

The description for the alert policy.

`--emails`

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

`--webhooks`

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

`--enabled`

Whether the alert policy is enabled.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update a web metric report alert policy.

```
$ ibmcloud cis alert-policy web-analytics-alert-update POLICY_ID --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --enabled true
```

ibmcloud cis alert-policy maintenance-event-alert-update

Update an alert policy for maintenance event.

```
$ ibmcloud cis alert-policy maintenance-event-alert-update POLICY_ID [--name NAME] [--emails EMAILS] [--webhooks WEBHOOKS] [--enabled (true | false)] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

POLICY_ID

The ID of alert policy. Required.

--name

The name of the alert policy.

--description

The description for the alert policy.

--emails

The email addresses for dispatching an alert notification. For example, `--emails test1@cn.ibm.com,test2@cn.ibm.com`

--webhooks

The webhook ID that for dispatching an alert notification. For example, `--webhook webhookID1,webhookID2`

--enabled

Whether the alert policy is enabled.

--event-type

The type of the maintenance event. Valid values are `scheduled` , `changed` , and `canceled` .

--airport-code

Comma-separated three-letter IATA Codes.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update a maintenance event alert policy.

```
$ ibmcloud cis alert-policy maintenance-event-alert-update POLICY_ID --name test1 --emails test1@cn.ibm.com --webhooks b2633e68-9a64-4519-b361-a64a67c8db8e --enabled true --event-type scheduled,changed,canceled --airport-code IAD,AUS
```

ibmcloud cis alert-policy delete

Delete an alert policy.

```
$ cis alert-policy delete POLICY_ID [-i, --instance INSTANCE] [-f, --force]
```

Command options

POLICY_ID

The ID of alert policy. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

-f, --force

Attempt to delete the policy without prompting for confirmation.

Examples

Delete an alert policy.

```
$ ibmcloud cis alert-policy delete POLICY_ID -f
```

ibmcloud cis alert-policy test

Send a test alert for an alert policy.

```
$ cis alert-policy test POLICY_ID [-i, --instance INSTANCE] [-f, --force]
```

Command options

POLICY_ID

The ID of alert policy. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

-f, --force

Attempt to send a test alert without prompting for confirmation.

Examples

Send a test notification for an alert policy.

```
$ ibmcloud cis alert-policy test POLICY_ID -f
```

Alert Webhook

ibmcloud cis alert-webhooks

List all alert webhooks.

```
$ ibmcloud cis alert-webhooks [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

List all webhooks for instance.

```
$ ibmcloud cis alert-webhooks
```

ibmcloud cis alert-webhook

Show the details of a webhook.

```
$ ibmcloud cis alert-webhook WEBHOOK_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

WEBHOOK_ID

The ID of the alert webhook. Required.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show the details of the alert webhook.

```
$ ibmcloud cis alert-webhook WEBHOOK_ID
```

ibmcloud cis alert-webhook-create

Create an alert webhook for an instance.

```
$ ibmcloud cis alert-webhook-create --name NAME --url URL [--secret SECRET] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

--name

The name of the webhook. Required.

--url

The POST endpoint to call when dispatching an alert. Required.

--secret

The secret that is passed in the webhook auth header when dispatching a webhook alert.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Create an alert webhook.

```
$ ibmcloud cis alert-webhook-create --name testwebhook --url https://hooks.slack.com/services/Ds3fdBFbV/1234568 --secret 007 --instance INSTANCE
```

ibmcloud cis alert-webhook-update

Update an alert webhook.

```
$ cis alert-webhook-update WEBHOOK_ID [--name NAME] [--url URL] [--secret SECRET] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

WEBHOOK_ID

The ID of the alert webhook. Required.

--name

The name of the webhook.

--url

The POST endpoint to call when dispatching an alert.

`--secret`

The secret that is passed in the webhook auth header when dispatching a webhook alert.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update an alert webhook.

```
$ ibmcloud cis alert-webhook-update WEBHOOK_ID --name testwebhook --url https://hooks.slack.com/services/Ds3fdBFbV/1234568
```

ibmcloud cis alert-webhook-delete

Delete an alert webhook.

```
$ ibmcloud cis alert-webhook-delete WEBHOOK_ID [-i, --instance INSTANCE] [-f, --force]
```

Command options

`WEBHOOK_ID`

The ID of the alert webhook. Required.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`-f, --force`

Attempt to delete the webhook without prompting for confirmation.

Examples

Delete an alert webhook.

```
$ ibmcloud cis alert-webhook-delete WEBHOOK_ID -f
```

Advanced Rate Limiting Rules

Manage the advanced rate limiting rules by using the following `advanced-rate-limiting` commands.

ibmcloud cis advanced-rate-limiting rules

List all advanced rate limiting rules.

```
$ ibmcloud cis advanced-rate-limiting rules DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List all advanced rate limiting rules for domain.

```
$ ibmcloud cis advanced-rate-limiting rules DNS_DOMAIN_ID
```

ibmcloud cis advanced-rate-limiting rule

Get details of an advanced rate limiting rule.

```
$ ibmcloud cis advanced-rate-limiting rule DNS_DOMAIN_ID RULE_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain.

RULE_ID

RULE_ID is the id of the advanced rate limiting rule.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show a rule for domain.

```
$ ibmcloud cis advanced-rate-limiting rule DNS_DOMAIN_ID RULE_ID
```

ibmcloud cis advanced-rate-limiting rule-create

Create an advanced rate limiting rule.

```
$ ibmcloud cis advanced-rate-limiting rule-create DNS_DOMAIN_ID --name NAME --match EXPRESSION --action ACTION --same-characteristics CHARACTERSTICS -
-requests REQUEST_PER_PERIOD --period PERIOD [--timeout TIMEOUT] [--enabled true|false] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain.

--name

The rule name.

--match

Specifies the conditions that must be matched for the rule to run. For match value, see [Using fields, functions, and expressions](#).

--action

Action to perform when the rate that is specified in the rule is reached. Valid values are `block`, `challenge`, `js_challenge`, `managed_challenge`, and `log`.

--same-characteristics

A set of parameters defining how CIS tracks the request rate for the rule. Use one or more of the characteristics: `ip`, `ip_nat`, `host`, `path`, `country`, `asnum`. For complex characteristics, use JSON file or JSON string instead.

`--requests`

The number of requests over the period of time that triggers the rule. Valid values range from `1-1000000`

`--period`

The period of time to consider (in seconds) when evaluating the request rate. Valid values are `10`, `60`, `120`, `300`, `600`, and `3600`.

`--timeout`

The rate limiting rule applies the rule action to further requests for the period of time. Valid values are `0`, `10`, `60`, `120`, `300`, `600`, `3600`, and `86400`.

`--enabled`

Indicates whether the rule is active or not. Valid values for "enabled" are `true` and `false`. (The default value is `false`)

`--json`

The JSON file or JSON string that is used to describe an advanced rate limiting rule.

- The required fields in JSON data are `expression`, `ratelimit` and `action`.
 - `expression` : Defines the criteria for the advanced rate limiting rule to match a request.
 - `ratelimit` : Define the rate-limit parameters.
 - `characteristics` : A set of parameters defining how CIS tracks the request rate for the rule.
 - `requests_per_period` : The number of requests over the period of time that triggers the rule.
 - `period` : The period of time to consider (in seconds) when evaluating the request rate. Valid values are `10`, `60`, `120`, `300`, `600`, and `3600`.
 - `requests_to_origin` : Apply the rate limiting to cached assets or not.
 - `mitigation_timeout` : The rate limiting rule applies the rule action to further requests for the period of time. Valid values are `0`, `10`, `60`, `120`, `300`, `600`, `3600`, and `86400`.
 - `counting_expression` : Defines the criteria that are used for determining the request rate.
 - `action` : Action to perform when the rate that is specified in the rule is reached. Valid values are `block`, `challenge`, `js_challenge`, `managed_challenge`, and `log`.
- The optional fields are `description`, `action_parameters`, and `enabled`.
 - `description` : The descriptive name of your rule.
 - `action_parameters` : Define the action parameters.
 - `response` : Define a custom response for block action.
 - `status_code` : Defines the HTTP status code that is returned to the visitor when blocking the request due to rate limiting. Only available when the rule action is Block. Valid values range from `400-499`. The default value is `429`.
 - `content_type` : Defines the content type of a custom response when blocking a request due to rate limiting. Only available when the rule action is Block.
 - `content` : Defines the body of the returned HTTP response when the request is blocked due to rate limiting. Only available when the rule action is Block.
 - `enabled` : Whether enable this rule or not.

\$ Sample JSON data:

```
{
  "description": "description",
  "expression": "(http.request.method eq \"POST\")",
  "ratelimit": {
    "characteristics": [
      "cf.unique_visitor_id",
      "cf.colo.id"
    ],
    "requests_to_origin": false,
    "counting_expression": "(ip.src.continent in {\"AN\"})",
    "requests_per_period": 10,
    "period": 10,
    "mitigation_timeout": 120
  },
  "action": "block",
```



```
"action_parameters": {
  "response": {
    "status_code": 429,
    "content_type": "text/xml",
    "content": "reject"
  }
},
"enabled": false
}
```

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create an advanced rate limiting rule for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis advanced-rate-limiting rule-create DNS_DOMAIN_ID --name rule-name --match "(http.request.method eq \"POST\")" --action log --same-characteristics ip,ip_nat --requests 100 --period 10
```

ibmcloud cis advanced-rate-limiting rule-update

Update an advanced rate limiting rule.

```
$ ibmcloud cis advanced-rate-limiting rule-update DNS_DOMAIN_ID RULE_ID --name NAME --match EXPRESSION --action ACTION --same-characteristics CHARACTERSTICS --requests REQUEST_PER_PERIOD --period PERIOD [--timeout TIMEOUT] [--enabled true|false] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain.

`RULE_ID`

RULE_ID is the id of the advanced rate limiting rule.

`--name`

The rule name.

`--match`

Specifies the conditions that must be matched for the rule to run. For match value, see [Using fields, functions, and expressions](#).

`--action`

Action to perform when the rate that is specified in the rule is reached. Valid values are `block`, `challenge`, `js_challenge`, `managed_challenge`, and `log`.

`--same-characteristics`

A set of parameters defining how CIS tracks the request rate for the rule. Use one or more of the characteristics: `ip`, `ip_nat`, `host`, `path`, `country`, `asnum`. For complex characteristics, use JSON file or JSON string instead.

`--requests`

The number of requests over the period of time that triggers the rule. Valid values range from `1-1000000`

`--period`

The period of time to consider (in seconds) when evaluating the request rate. Valid values are `10`, `60`, `120`, `300`, `600`, and `3600`.

`--timeout`

The rate limiting rule applies the rule action to further requests for the period of time. Valid values are `0`, `10`, `60`, `120`, `300`, `600`, `3600` and `86400`.

`--enabled`

Indicates whether the rule is active or not. Valid values for "enabled" are `true` and `false`. (The default value is `false`)

`--json`

The JSON file or JSON string that is used to describe an advanced rate limiting rule.

- The required fields in JSON data are `expression`, `ratelimit` and `action`.
 - `expression`: Defines the criteria for the advanced rate limiting rule to match a request.
 - `ratelimit`: Define the ratelimit parameters.
 - `characteristics`: A set of parameters defining how CIS tracks the request rate for the rule.
 - `requests_per_period`: The number of requests over the period of time that triggers the rule.
 - `period`: The period of time to consider (in seconds) when evaluating the request rate. Valid values are `10`, `60`, `120`, `300`, `600`, and `3600`.
 - `requests_to_origin`: Apply the rate limiting to cached assets or not.
 - `mitigation_timeout`: The rate limiting rule applies the rule action to further requests for the period of time. Valid values are `0`, `10`, `60`, `120`, `300`, `600`, `3600`, and `86400`.
 - `counting_expression`: Defines the criteria that are used for determining the request rate.
 - `action`: Action to perform when the rate that is specified in the rule is reached. Valid values are `block`, `challenge`, `js_challenge`, `managed_challenge`, and `log`.
- The optional fields are `description`, `action_parameters`, and `enabled`.
 - `description`: The descriptive name of your rule.
 - `action_parameters`: Define the action parameters.
 - `response`: Define a custom response for block action.
 - `status_code`: Defines the HTTP status code that is returned to the visitor when blocking the request due to rate limiting. Only available when the rule action is Block. Valid values range from `400-499`. The default value is `429`.
 - `content_type`: Defines the content type of a custom response when blocking a request due to rate limiting. Only available when the rule action is Block.
 - `content`: Defines the body of the returned HTTP response when the request is blocked due to rate limiting. Only available when the rule action is Block.
 - `enabled`: Whether enable this rule or not.

\$ Sample JSON data:

```
{
  "description": "description",
  "expression": "(http.request.method eq \"POST\")",
  "ratelimit": {
    "characteristics": [
      "cf.unique_visitor_id",
      "cf.colo.id"
    ],
    "requests_to_origin": false,
    "counting_expression": "(ip.src.continent in {\"AN\"})",
    "requests_per_period": 10,
    "period": 10,
    "mitigation_timeout": 120
  },
  "action": "block",
  "action_parameters": {
    "response": {
      "status_code": 429,
      "content_type": "text/xml",
      "content": "reject"
    }
  },
  "enabled": false
}
```

```
}

```

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update an advanced rate limiting rule `c2e184081120413c86c3ab7e14069605` for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis advanced-rate-limiting rule-update DNS_DOMAIN_ID RULE_ID --name rule-name --match "(http.request.method eq \"POST\")" --action log --same-characteristics ip,ip_nat --requests 100 --period 10
```

ibmcloud cis advanced-rate-limiting rule-delete

Delete an advanced rate limiting rule by id.

```
$ ibmcloud cis advanced-rate-limiting rule-delete DNS_DOMAIN_ID RULE_ID [-f, --force] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain.

`RULE_ID`

RULE_ID is the id of the advanced rate limiting rule.

`-f, --force`

Attempt to delete an advanced rate limiting rule without prompting for confirmation.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Delete a rule `c2e184081120413c86c3ab7e14069605` for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis advanced-rate-limiting rule-delete DNS_DOMAIN_ID RULE_ID
```

WAF managed rules

Manage the WAF-managed rulesets and rules by using the following `managed-waf` commands. Migrate to a new WAF by API or GUI first before you use managed WAF commands. Keep in mind that the previous version of WAF commands stops working after you migrate.

ibmcloud cis managed-waf rulesets

List all managed WAF rulesets.

```
$ ibmcloud cis managed-waf rulesets DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List all managed WAF rulesets for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis managed-waf rulesets DNS_DOMAIN_ID
```

ibmcloud cis managed-waf ruleset

Get details of a managed WAF ruleset.

```
$ ibmcloud cis managed-waf ruleset DNS_DOMAIN_ID RULESET_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of DNS domain.

`RULESET_ID`

The ID of the ruleset.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Show a ruleset `c2e184081120413c86c3ab7e14069605` for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis managed-waf ruleset DNS_DOMAIN_ID RULESET_ID
```

ibmcloud cis managed-waf deployment

Get details of a deployed managed WAF rule.

```
$ ibmcloud cis managed-waf deployment DNS_DOMAIN_ID RULE_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain.

`RULE_ID`

The ID of the rule.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Show a deployment rule `a2121e23-9e68-1218-a356-b78e23a8ec8a` for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis managed-waf deployment DNS_DOMAIN_ID RULE_ID
```

ibmcloud cis managed-waf deployments

List all deployed managed WAF rules.

```
$ ibmcloud cis managed-waf deployments DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of DNS domain.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

List all deployment rules for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis managed-waf deployments DNS_DOMAIN_ID
```

ibmcloud cis managed-waf deployment-add-exception

Create an exception rule to skip execution of specified managed WAF rules.

```
$ ibmcloud cis managed-waf deployment-add-exception DNS_DOMAIN_ID --match EXPRESSION [--skip-rules RULES] [--enabled true|false] [--logging true|false] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis managed-waf deployment-add-exception DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of DNS domain.

`--match`

Specifies the conditions that must be matched for the rule to run. For match value, see [Using fields, functions, and expressions](#).

`--skip-rules`

Skip all remaining rules, WAF managed rulesets, or rules of WAF managed rulesets. For example, `--skip-rules RULESETID-1:RULEID-a,RULEID-b;RULESETID-2:RULEID-x,RULEID-y`. Set `current` to skip all remaining rules. The default value is `current`.

`--enabled`

Indicates whether the rule is active or not. The default value is `true`.

`--logging`

Log requests matching the skip rule. The default value is `true`.

`--description`

A brief description of the rule.

`--json`

The JSON file or JSON string that is used to describe a managed WAF.

- The required fields in JSON data are `expression`, `action` and `action_parameters` .

`expression` : The rule expression. `action` : The rule action to perform. Valid value is `skip`. `action_parameters` : The rule action parameters.
`ruleset` : Skip all remaining rules or one or more WAF-managed rulesets. `rules` : Skip one or more rules of WAF-managed rulesets.
- The optional fields are `description`, `enabled`, and `logging`. `description` : Briefly describes the rule. `enabled` : Indicates whether the rule is active or not. `logging` : Log requests matching the skip rule. - `enabled` : When disabled, matched requests don't appear in firewall events.

Sample JSON data: `json { "action": "skip", "expression": "(http.cookie eq \"example.com/contact?page=1234\")", "description": "rule name", "enabled": true, "logging": { "enabled": true }, "action_parameters": { "rules": { "efb7b8c949ac4650a09736fc376e9aee": ["5de7edfa648c4d6891dc3e7f84534ffa", "e3a567afc347477d9702d9047e97d760"], "c2e184081120413c86c3ab7e14069605": ["ef21b0a932ae422790f9249d213b85e6"] } } }`

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create exception rule for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis managed-waf deployment-add-exception DNS_DOMAIN_ID --match "(http.cookie eq \"example.com/contact?page=1234\")" --skip-rules 'efb7b8c949ac4650a09736fc376e9aee:5de7edfa648c4d6891dc3e7f84534ffa' --enabled false --logging true
```

ibmcloud cis managed-waf deployment-update-exception

Update an exception rule in the deployed managed WAF rules.

```
$ ibmcloud cis managed-waf deployment-update-exception DNS_DOMAIN_ID RULE_ID [--match MATCH] [--skip-rules RULES] [--enabled true|false] [--logging true|false] [--description DESCRIPTION] [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis managed-waf deployment-update-exception DNS_DOMAIN_ID RULE_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of DNS domain.

`RULE_ID`

The ID of rule.

`--match`

Specifies the conditions that must be matched for the rule to run. For match value, see [Using fields, functions, and expressions](#).

`--skip-rules`

Skip all remaining rules, WAF managed rulesets, or rules of WAF managed rulesets. For example, `--skip-rules RULESETID-1:RULEID-a,RULEID-b;RULESETID-2:RULEID-x,RULEID-y`. Set `current` to skip all remaining rules. The default value is `current`.

`--enabled`

Indicates whether the rule is active or not. The default value is `true`.

`--logging`

Log requests matching the skip rule. The default value is `true`.

`--description`

To briefly describe the rule.

--json

The JSON file or JSON string that is used to describe a managed WAF.

- The required fields in JSON data are `expression`, `action`, and `action_parameters`.

`expression`: The rule expression. `action`: The rule action to perform. Valid value is `skip`. `action_parameters`: The rule action parameters.
`ruleset`: Skip all remaining rules or one or more WAF-managed rulesets. `rules`: Skip one or more rules of WAF-managed rulesets.
- The optional fields are `description`, `enabled`, and `logging`. `description`: Briefly describes the rule. `enabled`: Indicates whether the rule is active or not. `logging`: Log requests matching the skip rule. - `enabled`: When disabled, matched requests don't appear in firewall events.

Sample JSON data:

```
$ {
  "action": "skip",
  "expression": "(http.cookie eq \"example.com/contact?page=1234\")",
  "description": "rule name",
  "enabled": true,
  "logging": {
    "enabled": true
  },
  "action_parameters": {
    "rules": {
      "efb7b8c949ac4650a09736fc376e9aee": [
        "5de7edfa648c4d6891dc3e7f84534ffa",
        "e3a567afc347477d9702d9047e97d760"
      ],
      "c2e184081120413c86c3ab7e14069605": [
        "ef21b0a932ae422790f9249d213b85e6"
      ]
    }
  }
}
```

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Update an exception rule `e7ead74deb2b4c30a91c793f502f5e14` for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis managed-waf deployment-update-exception DNS_DOMAIN_ID RULE_ID --match "(http.cookie eq \"example.com/contact?page=1234\")" --skip-rules 'efb7b8c949ac4650a09736fc376e9aee:5de7edfa648c4d6891dc3e7f84534ffa' --enabled false --logging true
```

ibmcloud cis managed-waf deployment-add-ruleset

Add a managed ruleset to the deployed managed WAF rules.

```
$ ibmcloud cis managed-waf deployment-add-ruleset DNS_DOMAIN_ID RULESET_ID [--match EXPRESSION] [--enabled true|false] [--override-action ACTION] [--override-status STATUS] [--paranoia-level LEVEL] [--override-rules RULE] [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis managed-waf deployment-add-ruleset DNS_DOMAIN_ID RULESET_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of DNS domain.

RULESET_ID

The ID of managed ruleset.

--match

Specifies the conditions that must be matched for the rule to run. For match value, see [Using fields, functions, and expressions](#).

--enabled

Indicates whether the rule is active or not. The default value is `true`.

--override-action

The ruleset action of the overrides. Valid values are `managed_challenge`, `block`, `js_challenge`, `log`, and `challenge`.

--override-status

The ruleset status of the overrides. Valid values are `true` and `false`.

--paranoia-level

OWASP paranoia level, higher paranoia levels activate more aggressive rules. Valid values are `PL1`, `PL2`, `PL3`, `PL4` and it's only available for `CIS OWASP Core Ruleset`.

--override-rules

The rules options of the overrides. For example `--override-rules rule=RULE_ID,action=ACTION,enabled=STATUS`. For OWASP Core Ruleset, you can also override the Score Threshold. For example, `--override-rules rule=6179ae15870a4bb7b2d480d4843b323c,score-threshold=25`.

--json

The JSON file or JSON string that is used to describe a managed WAF rule.

- The required fields in JSON data are `expression`, `action`, and `action_parameters`.

`expression`: The rule expression. `action`: The rule action to perform. Valid value is `skip`. `action_parameters`: The rule action parameters.
`id`: The ruleset ID of the overrides. `overrides`: The rules options of the overrides. `action`: The ruleset action of the overrides. Valid values are `managed_challenge`, `block`, and `js_challenge`, "log", "challenge". `enabled`: The ruleset status of the overrides. Valid values are `true` and `false`. `rules`: The rules options of the overrides. `id`: The rule ID of the overrides. `action`: The rule action of the overrides. Valid values are `managed_challenge`, `block`, `js_challenge`, `log`, and `challenge`. `enabled`: The rule status of the overrides. `score_threshold`: OWASP Anomaly Score Threshold. Set the score threshold, which triggers the Firewall. `categories`: Define OWASP Paranoia Level and only valid for `CIS OWASP core ruleset` `category`: OWASP paranoia level, higher paranoia levels activate more aggressive rules. `enabled`: Whether this OWASP Paranoia Level is enabled.
- The optional fields are `description` and `enabled`. `description`: Briefly describes the rule. `enabled`: Indicates whether the rule is active or not.

Sample JSON data:

```
$ {
  "action": "execute",
  "description": "CIS Managed Ruleset",
  "enabled": true,
  "expression": "(http.cookie eq \"example.com/contact?page=1234\")",
  "action_parameters": {
    "id": "efb7b8c949ac4650a09736fc376e9aee",
    "overrides": {
      "action": "block",
      "enabled": false,
      "rules": [
        {
          "id": "5de7edfa648c4d6891dc3e7f84534ffa",
          "action": "managed_challenge"
        },
        {
          "id": "e3a567afc347477d9702d9047e97d760",
          "action": "log",
          "enabled": true
        }
      ]
    }
  }
}
```



```
    ]
  }
}
```

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Deploy a managed ruleset for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis managed-waf deployment-add-ruleset DNS_DOMAIN_ID RULESET_ID --match true --enabled true --override-action block --override-status true --
override-rules rule=5de7edfa648c4d6891dc3e7f84534ffa,action=managed_challenge --override-rules
rule=e3a567afc347477d9702d9047e97d760,action=action,enabled=true
```

ibmcloud cis managed-waf deployment-update-ruleset

Update a managed ruleset in the deployed managed WAF rules.

```
$ ibmcloud cis managed-waf deployment-update-ruleset DNS_DOMAIN_ID RULE_ID [--match EXPRESSION] [--enabled true|false] [--override-action ACTION] [--
override-status STATUS] [--paranoia-level LEVEL] [--override-rules RULE] [--reset-all] [-i, --instance INSTANCE] [--output FORMAT]
ibmcloud cis managed-waf deployment-update-ruleset DNS_DOMAIN_ID RULE_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output
FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of DNS domain.

`RULE_ID`

The ID of deployed managed rule.

`--match`

Specifies the conditions that must be matched for the rule to run. For match value, see [Using fields, functions, and expressions](#).

`--enabled`

Indicates whether the rule is active or not. The default value is `true`.

`--override-action`

The ruleset action of the overrides. Valid values are `managed_challenge`, `block`, `js_challenge`, `log`, and `challenge`.

`--override-status`

The ruleset status of the overrides. Valid values are `true` and `false`.

`--paranoia-level`

OWASP paranoia level, higher paranoia levels activate more aggressive rules. Valid values are `PL1`, `PL2`, `PL3`, `PL4` and it's only available for `CIS OWASP Core Ruleset`.

`--override-rules`

The rules options of the overrides. For example, `--override-rules rule=RULE_ID,action=ACTION,enabled=STATUS`. For OWASP Core Ruleset, you can also override the Score Threshold. For example, `--override-rules rule=6179ae15870a4bb7b2d480d4843b323c,score-threshold=25`.

`--reset-all`

Reset all the overrides rules to the default settings.

`--json`

The JSON file or JSON string that is used to describe a managed waf rule.

- The required fields in JSON data are `expression`, `action`, and `action_parameters`.

`expression`: The rule expression. `action`: The rule action to perform. Valid value is `skip`. `action_parameters`: The rule action parameters.
`id`: The ruleset id of the overrides. `overrides`: The rules options of the overrides. `action`: The ruleset action of the overrides. Valid values are `managed_challenge`, `block`, and `js_challenge`, `log`, `challenge`. `enabled`: The ruleset status of the overrides. Valid values are `true` and `false`. `rules`: The rules options of the overrides. `id`: The rule ID of the overrides. `action`: The rule action of the overrides. Valid values are `managed_challenge`, `block`, `js_challenge`, `log`, and `challenge`. `enabled`: The rule status of the overrides. `score_threshold`: OWASP Anomaly Score Threshold. Set the score threshold, which triggers the Firewall. `categories`: Define OWASP Paranoia Level and only valid for `CIS OWASP core ruleset` `category`: OWASP paranoia level, higher paranoia levels activate more aggressive rules. `enabled`: Whether this OWASP Paranoia Level enabled.
- The optional fields are `description` and `enabled`. `description`: Briefly describes the rule. `enabled`: Indicates whether the rule is active or not.

Sample JSON data:

```
$ {
  "action": "execute",
  "description": "CIS Managed Ruleset",
  "enabled": true,
  "expression": "(http.cookie eq \"example.com/contact?page=1234\")",
  "action_parameters": {
    "id": "efb7b8c949ac4650a09736fc376e9aee",
    "overrides": {
      "action": "block",
      "enabled": false,
      "rules": [
        {
          "id": "5de7edfa648c4d6891dc3e7f84534ffa",
          "action": "managed_challenge"
        },
        {
          "id": "e3a567afc347477d9702d9047e97d760",
          "action": "log",
          "enabled": true
        }
      ]
    }
  }
}
```

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update a managed ruleset rule `1a18a1ea7fc043c68761bc69adcbb11c` for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis managed-waf deployment-update-ruleset DNS_DOMAIN_ID RULE_ID --match true --enabled true --override-action block --override-status true --override-rules rule=5de7edfa648c4d6891dc3e7f84534ffa,action=managed_challenge --override-rules rule=e3a567afc347477d9702d9047e97d760,action=action,enabled=true
```

WAF custom rules

Manage the WAF custom rules by using the following `custom-waf` commands. Firewall rules are now managed by WAF custom rules.

ibmcloud cis custom-waf rules

List all custom rules.

```
$ ibmcloud cis custom-waf rules DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

List all custom rules for domain `31984fea73a15b45779fa0df4ef62f9b` under instance `cis-demo`.

```
$ ibmcloud cis custom-waf rules DNS_DOMAIN_ID
```

ibmcloud cis custom-waf rule

Get details of a custom rule.

```
$ ibmcloud cis custom-waf rule DNS_DOMAIN_ID RULE_ID [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of DNS domain.

RULE_ID

The ID of the rule.

-i, --instance

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

--output

The output format. Currently, `json` is the only supported value.

Examples

Show a custom rule `b94632a4cd5a49ed830544d91417a98c` for domain `9343630b9bd5c6e6899834d77f9e50ff` under instance `cis-demo`.

```
$ ibmcloud cis custom-waf rule DNS_DOMAIN_ID RULE_ID
```

ibmcloud cis custom-waf rule-create

Create a custom rule.

```
$ ibmcloud cis custom-waf rule-create DNS_DOMAIN_ID --match EXPRESSION --action ACTION [--description DESCRIPTION] [--enabled true|false] [-i, --instance INSTANCE] [--output FORMAT]
```

```
ibmcloud cis custom-waf rule-create DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

DNS_DOMAIN_ID

The ID of the DNS domain.

`--match`

Specifies the conditions that must be matched for the rule to run. For match value, see [Using fields, functions, and expressions](#).

`--action`

The rule action to perform. Valid values are `block`, `challenge`, `js_challenge`, `managed_challenge`, `log`, and `skip`. For 'block' and 'skip' actions, use JSON file or JSON string instead.

`--enabled`

Indicates whether the rule is active or not. The default value is `false`.

`--description`

A brief description of the rule.

`--json`

The JSON file or JSON string that is used to describe a custom rule.

- The required fields in JSON data are `expression` and `action`.
 - `expression` : Specifies the conditions that must be matched for the rule to run.
 - `action` : The rule action to perform. Valid values are `block`, `challenge`, `js_challenge`, `managed_challenge`, `log`, and `skip`.
- The optional fields are `description`, `enabled`, `logging`, and `action_parameters`.
 - `action_parameters` : The rule action parameters.
 - `ruleset` : Skip all remaining rules or one or more WAF managed rulesets. Valid value is `current`.
 - `phases` : Skips WAF components for matching requests. Valid values are `http_ratelimit`, `http_request_firewall_managed`, and `http_request_sbfm`.
 - `products` : Skips specific security products for matching requests. Valid values are `waf`, `rateLimit`, `securityLevel`, `hot`, `bic`, `uaBlock`, and `zoneLockdown`.
 - `response` : Define a custom response for `block` action.
 - `status_code` : Choose an HTTP status code for the response in the range `400-499`.
 - `content_type` : The content type of a custom response. Valid response types are : `text/html`, `text/plain`, `application/json`, `text/xml`.
 - `content` : The response body.
 - `description` : Briefly describes the rule.
 - `enabled` : Indicates whether the rule is active or not. When this field is disabled, matched requests don't appear in firewall requests.
 - `logging` : Log requests matching the skip rule. This field is only available for 'skip' action.

Sample JSON data:

```
$ {
  "description": "test-custom-rule",
  "expression": "(http.cookie contains \"test\")",
  "action": "skip",
  "logging": {
    "enabled": true
  },
  "action_parameters": {
    "ruleset": "current",
    "phases": [
      "http_ratelimit",
      "http_request_firewall_managed",
      "http_request_sbfm"
    ],
    "products": [
      "waf",
      "rateLimit",
      "securityLevel",
      "hot",
      "bic",
      "uaBlock",
      "zoneLockdown"
    ]
  }
}
```

```
    ]
  },
  "enabled": true
}
```

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Create a custom rule for domain `9343630b9bd5c6e6899834d77f9e50ff` under instance `cis-demo`.

```
$ ibmcloud cis custom-waf rule-create DNS_DOMAIN_ID --action challenge --description "rule 1" --enabled true --match "(http.host eq \"www.example.com\")"
```

ibmcloud cis custom-waf rule-update

Update a custom rule.

```
$ ibmcloud cis custom-waf rule-update DNS_DOMAIN_ID [--match EXPRESSION] [--action ACTION] [--description DESCRIPTION] [--enabled true|false] [-i, --instance INSTANCE] [--output FORMAT]

ibmcloud cis custom-waf rule-update DNS_DOMAIN_ID (--json @JSON_FILE | JSON_STRING) [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain.

`RULE_ID`

The ID of the rule.

`--match`

Specifies the conditions that must be matched for the rule to run. For match value, see [Using fields, functions, and expressions](#).

`--action`

The rule action to perform. Valid values are `block`, `challenge`, `js_challenge`, `managed_challenge`, `log`, and `skip`. For 'block' and 'skip' actions, use JSON file or JSON string instead.

`--enabled`

Indicates whether the rule is active or not. The default value is `false`.

`--description`

A brief description of the rule.

`--json`

The JSON file or JSON string that is used to describe a custom rule.

- The required fields in JSON data are `expression` and `action`.
 - `expression` : Specifies the conditions that must be matched for the rule to run.
 - `action` : The rule action to perform. Valid values are `block`, `challenge`, `js_challenge`, `managed_challenge`, `log`, and `skip`.
- The optional fields are `description`, `enabled`, `logging`, and `action_parameters`.
 - `action_parameters` : The rule action parameters.
 - `ruleset` : Skip all remaining rules or one or more WAF managed rulesets. Valid value is `current`
 - `phases` : Skips WAF components for matching requests. Valid values are `http_ratelimit`, `http_request_firewall_managed`, and

- `http_request_sbfm` .
- `products` : Skips specific security products for matching requests. Valid values are `waf` , `rateLimit` , `securityLevel` , `hot` , `bic` , `uaBlock` , and `zoneLockdown` .
- `response` : Define a custom response for 'block' action.
 - `status_code` : Choose an HTTP status code for the response in the range `400-499` .
 - `content_type` : The content type of a custom response. Valid response types are `text/html` , `text/plain` , `application/json` , and `text/xml` .
 - `content` : The response body.
- `description` : Briefly describes the rule.
- `enabled` : Indicates whether the rule is active or not. When this field is disabled, matched requests don't appear in firewall requests.
- `logging` : Log requests matching the skip rule. This field is only available for 'skip' action.

Sample JSON data:

```
$ {
  "description": "test-custom-rule",
  "expression": "(http.cookie contains \"test\")",
  "action": "block",
  "action_parameters": {
    "response": {
      "status_code": 429,
      "content_type": "text/xml",
      "content": "reject"
    }
  },
  "enabled": true
}
```

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Update a custom rule `b94632a4cd5a49ed830544d91417a98c` for the domain `9343630b9bd5c6e6899834d77f9e50ff` under instance `cis-demo` .

```
$ ibmcloud cis custom-waf rule-update DNS_DOMAIN_ID RULE_ID --enabled false --description "rule 1" --match "(http.host eq \"www.example.com\")"
```

ibmcloud cis custom-waf rule-order-update

Change the execution order of the custom rule.

```
$ ibmcloud cis custom-waf rule-order-update DNS_DOMAIN_ID RULE_ID [--before RULE_ID] [--after RULE_ID] [--index INDEX] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain.

`RULE_ID`

The ID of the custom rule.

`--before`

Places the rule before rule `<RULE_ID>` .

`--after`

Places the rule after rule `<RULE_ID>` .

`--index`

Places the rule in the exact position that is specified by the integer number.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Put custom rule `4eae81b170f644f795da017001383de7` before rule `2ed2dd160cb745feb415414544d97c70` for domain `9343630b9bd5c6e6899834d77f9e50ff` under instance `cis-demo` .

```
$ ibmcloud cis custom-waf rule-order-update DNS_DOMAIN_ID RULE_ID --before RULE_ID
```

ibmcloud cis custom-waf rule-delete

Delete a custom rule by ID.

```
$ ibmcloud cis custom-waf rule-delete DNS_DOMAIN_ID RULE_ID [-f, --force] [-i, --instance INSTANCE] [--output FORMAT]
```

Command options

`DNS_DOMAIN_ID`

The ID of the DNS domain.

`RULE_ID`

The ID of the custom rule.

`-i, --instance`

Instance name or ID. If instance name or ID is not set, the context instance that is specified by `ibmcloud cis instance-set INSTANCE` is used.

`-f, --force`

Attempt to delete a custom rule without prompting for confirmation.

`--output`

The output format. Currently, `json` is the only supported value.

Examples

Delete custom rule `e9fad806880c4c42bd7ebeec8dcba4e6` for domain `9343630b9bd5c6e6899834d77f9e50ff` under instance `cis-demo` .

```
$ ibmcloud cis custom-waf rule-delete DNS_DOMAIN_ID RULE_ID
```

Private endpoint support

To ensure that you have enhanced control and security over your data when you use the CIS CLI, you have the option of using private routes to CIS endpoints. Private routes are not accessible or reachable over the internet. By using CIS private endpoints, you can protect your data from threats from the public network and logically extend your private network.

Regional support is provided for a limited number of CLI commands. The following regions support private endpoints:

- us-south
- us-east

Logging in to the CLI with a private endpoint

Use the following command to log in to a private endpoint from the CLI:

```
$ ibmcloud login -a private.cloud.ibm.com
```

Targeting a supported region

A region must be targeted when a private endpoint is set. Use the following command to target a supported region:

```
$ ibmcloud target -r [region]
```

Using CIS CLI with private endpoints

All the commands support private endpoints, for example:

```
$ ibmcloud cis domains --instance cis-demo
```

CIS CLI change log

In this change log, you can learn about the latest changes, improvements, and updates for the IBM Cloud Internet Services CLI.

Version 1.21.0

Version 1.21.0 of the CLI was released on 19 August 2026.

Added the `leaked-credential-checks` commands to manage leaked credential detection for a zone, including enabling/disabling the feature and managing custom detection rules.

Fixed the `log-lookup-rayid` command to correctly display log fields and resolve the status message showing raw template placeholders instead of values.

Version 1.20.0

Version 1.20.0 of the CLI was released on 17 July 2026.

Added the `log-lookup-rayid` command to retrieve log events for a given `RayID`.

Added the `ai_bots_protection` field support to bot management domain setting.

Added the auto-pagination and `--search` filter support for custom list items.

Upgraded the Go and Python security dependencies.

Version 1.19.2

Version 1.19.2 of the CLI was released on 23 May 2026.

Upgraded the Go version to v1.26.2.

Version 1.19.1

Version 1.19.1 of the CLI was released on 9 March 2026.

Upgraded the Go version to v1.26.0.

Version 1.19.0

Version 1.19.0 of the CLI was released on 17 November 2025.

Added the deprecation notice to legacy WAF commands.

Added the [Total TLS commands](#).

Removed the notification-email parameter from `glb-create` and `glb-update` commands.

Version 1.18.1

Version 1.18.1 of the CLI was released on 11 July 2025.

Added [custom lists](#).

Added [managed lists](#).

Version 1.16.4

Version 1.16.4 of the CLI was released on 7 April 2025.

Added [WAF managed rules](#).

Added [WAF custom rules](#).

Version 1.16.3

Version 1.16.3 of the CLI was released on 19 July 2024.

Fix CVE certifi.

Add `replace_insecure_js` domain setting.

Fix managed rules and advanced rate-limiting rule bugs.

CLI GVT translation.

Update the dependency `requests` to v2.32.3.

Version 1.16.2

Version 1.16.2 of the CLI was released on 13 June 2024.

Support Advanced rate-limiting rules.

Update the description of listing dns record per page.

Version 1.16.1

Version 1.16.1 of the CLI was released on 11 May 2024.

Fix WAF overrides list and domain settings issue.

Display WAF migration notice.

Version 1.16.0

Version 1.16.0 of the CLI was released on 6 May 2024.

Support managed WAF rules.

Version 1.15.13

Version 1.15.13 of the CLI was released on 12 April 2024.

Add Origin max HTTP version support.

Add origin post quantum encryption support.

Version 1.15.12

Version 1.15.12 of the CLI was released on 22 March 2024.

Enhance the custom certificates.

Go modules upgrade.

Version 1.15.11

Version 1.15.11 of the CLI was released on 28 February 2024.

Fix listing certificate issue.

Version 1.15.10

Version 1.15.10 of the CLI was released on 1 February 2024.

Support HTTP origin alert policy.

Version 1.15.8

Version 1.15.8 of the CLI was released on 11 January 2023.

fix bug of deleting filter and firewall rules.

gvt translation.

Update go to v20.

Version 1.15.7

Version 1.15.7 of the CLI was released on 8 November 2023.

Support ddos **L3/L4** alerting policy.

Support mtls certificate expiration alerting policy.

Failing Logpush job disabled alerting policy.

Fix bot metrics issue.

Version 1.15.6

Version 1.15.6 of the CLI was released on 16 October 2023.

Support instant logs.

Version 1.15.5

Version 1.15.5 of the CLI was released on 7 September 2023.

Support alert policy test.

Support maintenance event alert policy.

Bug fix: certificate deletion issue.

Version 1.15.4

Version 1.15.4 of the CLI was released on 29 August 2023.

Support `dns_logs` dataset for logpush job.

Update security fields for logpull.

Support Web analytic report alert policy.

Version 1.15.3

Version 1.15.3 of the CLI was released on 23 July 2023.

Support `linux/arm64` binary.

Version 1.15.2

Version 1.15.2 of the CLI was released on 6 June 2023.

Support Zone hold.

Support Image Resizing.

Fix bugs of alerts.

Version 1.15.0

Version 1.15.0 of the CLI was released on 24 April 2023.

Enterprise 2.0 plan support.

Bot management support.

FVT/GVT.

Version 1.14.11

Version 1.14.11 of the CLI was released on 16 March 2023.

- Unhide standard plan.

Version 1.14.10

Version 1.14.10 of the CLI was released on 2 March 2023.

- Add URL normalization for domain setting.

- Add certificate authority options.

- Add universal CA options.

- Add binary M1 and s390x support.

- Fix origin certificate list issue.

- Upgrade go dependency package versions.

- Unhide the standard plan for 45 days.

Version 1.14.9

Version 1.14.9 of the CLI was released on 27 October 2022.

- Add authenticated origin pull list.

- Enhance certificates order.

- Add `probe_zone` option for the GLB monitor.

Version 1.14.8

Version 1.14.8 of the CLI was released on 26 August 2022.

- Add new alert policies.

- Enable Domain CNAME setup.

- Extend rate limit threshold range.

Version 1.14.5

Version 1.14.5 of the CLI was released on 9 August 2022.

- Fix edge functions trigger.

- Fix cross account resource access.

- Add `ppc64le` binary release.

- Page rule action remove always online.

CIS use cases

Accessing your Cloudant database through CIS

Follow these steps to access your Cloudant database through IBM Cloud® Internet Services (CIS).

Before you begin

These instructions assume that you already added a domain to CIS as outlined in the [Getting started](#) page.

Step 1: Add your CIS domain to the Cross-Origin Resource Sharing (CORS)

- Navigate to your Cloudant database and open the **Account > CORS** page.
- Add your CIS domain to the origin domains input field. For example, `https://cloudant.test.foo.com`.

Step 2. Configure CIS to point to your Cloudant database

- Navigate to the CIS dashboard and create a load balancer or DNS record that points to your Cloudant database hostname. For example, `https://cloudant.test.foo.com` -> `111-222-333-444-555-test.cloudant.com`.
- Enable `proxy` for the DNS record or load balancer.

Step 3. Create a page rule to set the Host Header Override

- In the CIS dashboard, navigate to **Performance > Page rules**.
- Create a page rule for the URL that you want, for example, `https://cloudant.test.foo.com/*`.
- Select the Rule Behavior setting `Host Header Override`.
- Set as the Cloudant database hostname, for example, `111-222-333-444-555-test.cloudant.com`.

To learn more about Cloudant, see the [Cloudant documentation](#).

Edge functions use cases

The following use cases are provided solely as examples, and are not intended for exact duplication in your environment.



Important: Exercise caution when you test any of the following code because it might cause a disruption in service.

A/B testing

You can create a CIS Edge function to control A/B tests.

```
addEventListener('fetch', event => {
  event.respondWith(fetchAndApply(event.request))
})

async function fetchAndApply(request) {
  const name = 'experiment-0'
  let group      // 'control' or 'test', set below
  let isNew = false // is the group newly-assigned?

  // Determine which group this request is in.
  const cookie = request.headers.get('Cookie')
  if (cookie && cookie.includes(`${name}=control`)) {
    group = 'control'
  } else if (cookie && cookie.includes(`${name}=test`)) {
    group = 'test'
  } else {
    // 50/50 Split
    group = Math.random() < 0.5 ? 'control' : 'test'
    isNew = true
  }

  // We'll prefix the request path with the experiment name. This way,
  // the origin server merely has to have two copies of the site under
  // top-level directories named "control" and "test".
  let url = new URL(request.url)
```

```

// Note that `url.pathname` always begins with a `/`, so we don't
// need to explicitly add one after `${group}`.
url.pathname = `${group}${url.pathname}`

const modifiedRequest = new Request(url, {
  method: request.method,
  headers: request.headers
})

const response = await fetch(modifiedRequest)

if (isNew) {
  // The experiment was newly-assigned, so add a Set-Cookie header
  // to the response.
  const newHeaders = new Headers(response.headers)
  newHeaders.append('Set-Cookie', `${name}=${group}; path=/`)
  return new Response(response.body, {
    status: response.status,
    statusText: response.statusText,
    headers: newHeaders
  })
} else {
  // Return response unmodified.
  return response
}
}

```

Adding a response header

To modify the response headers, first make a copy of the response so that you can make it mutable. Then, you can use the Headers interface to add, change, or remove headers.

```

addEventListener('fetch', event => {
  event.respondWith(handleRequest(event.request))
})

/**
 * Set the `x-my-header` header
 * @param {Request} request
 */
async function handleRequest(request) {
  let response = await fetch(request);

  // Make the headers mutable by re-constructing the Response.
  response = new Response(response.body, response);
  response.headers.set('x-my-header', 'custom value');
  return response;
}

```

Aggregating multiple requests

This example makes multiple requests to different API endpoints, aggregates the responses, and sends it back as a single response.

```

addEventListener('fetch', event => {
  event.respondWith(fetchAndApply(event.request))
})

/**
 * Make multiple requests,
 * aggregate the responses and
 * send it back as a single response
 */
async function fetchAndApply(request) {
  const init = {
    method: 'GET',
    headers: {'Authorization': 'XXXXXX'}
  }
  const [btcResp, ethResp, ltcResp] = await Promise.all([

```

```

    fetch('https://api.coinbase.com/v2/prices/BTC-USD/spot', init),
    fetch('https://api.coinbase.com/v2/prices/ETH-USD/spot', init),
    fetch('https://api.coinbase.com/v2/prices/LTC-USD/spot', init)
  ])

  const btc = await btcResp.json()
  const eth = await ethResp.json()
  const ltc = await ltcResp.json()

  let combined = {}
  combined['btc'] = btc['data'].amount
  combined['ltc'] = ltc['data'].amount
  combined['eth'] = eth['data'].amount

  const responseInit = {
    headers: {'Content-Type': 'application/json'}
  }
  return new Response(JSON.stringify(combined), responseInit)
}

```

Conditional routing

The easiest way to deliver different content based on which device is being used is to rewrite the URL of the request based on the condition you care about. See the following examples.

Device type

```

addEventListener('fetch', event => {
  event.respondWith(fetchAndApply(event.request))
})

async function fetchAndApply(request) {
  let uaSuffix = ""

  const ua = request.headers.get('user-agent')
  if (ua.match(/iphone/i) || ua.match(/ipod/i)) {
    uaSuffix = '/mobile'
  } else if (ua.match(/ipad/i)) {
    uaSuffix = '/tablet'
  }

  return fetch(request.url + uaSuffix, request)
}

```

Custom headers

```

addEventListener('fetch', event => {
  event.respondWith(fetchAndApply(event.request))
})

async function fetchAndApply(request) {
  let suffix = ""
  //Assuming that the client is sending a custom header
  const cryptoCurrency = request.headers.get('X-Crypto-Currency')
  if (cryptoCurrency === 'BTC') {
    suffix = '/btc'
  } else if (cryptoCurrency === 'XRP') {
    suffix = '/xrp'
  } else if (cryptoCurrency === 'ETH') {
    suffix = '/eth'
  }

  return fetch(request.url + suffix, request)
}

```

Hot-link protection

You can use CIS Edge functions to protect your hot-links on your web properties.

```

addEventListener('fetch', event => {
  event.respondWith(fetchAndApply(event.request))
})

/**
 * If the browser is requesting an image and
 * the referer does not match your host
 * we redirect the request to your page
 */
async function fetchAndApply(request) {
  // Fetch the response.
  let response = await fetch(request)

  // If it's an image, engage hotlink protection based on the
  // Referer header.
  let referer = request.headers.get('Referer')
  let contentType = response.headers.get('Content-Type') || ""
  if (referer && contentType.startsWith('image/')) {
    // It's an image and there's a Referer. Verify that the
    // hostnames match.
    if (new URL(referer).hostname !==
        new URL(request.url).hostname) {
      // Hosts don't match. This is a hotlink. Redirect the
      // user to our homepage.
      return new Response("", {
        status: 302,
        headers: {
          'Location': '/'
        }
      })
    }
  }

  // Everything is fine, return the response normally.
  return response
}

```

Originless responses

You can return responses directly from the edge. No need to send a request to your origin.

Ignore POST and PUT HTTP requests

Ignore POST and PUT HTTP requests. This snippet allows all other requests to pass through to the origin.

```

addEventListener('fetch', event => {
  event.respondWith(fetchAndApply(event.request))
})

async function fetchAndApply(request) {
  if (request.method === 'POST' || request.method === 'PUT') {
    return new Response('Sorry, this page is not available.',
      { status: 403, statusText: 'Forbidden' })
  }

  return fetch(request)
}

```

Deny a spider or crawler

Protect your origin from unwanted spiders or crawlers. In this case, if the user-agent is “annoying-robot”, the Edge function returns the response instead of sending the request to the origin.

```

addEventListener('fetch', event => {
  event.respondWith(fetchAndApply(event.request))
})

async function fetchAndApply(request) {
  if (request.headers.get('user-agent').includes('annoying_robot')) {

```



```
    return new Response('Sorry, this page is not available.',
      { status: 403, statusText: 'Forbidden' })
  }

  return fetch(request)
}
```

Prevent a specific IP from connecting

Blocklist IP addresses. This snippet of code prevents a specific IP (in this case `225.0.0.1`) from connecting to the origin.

```
addEventListener('fetch', event => {
  event.respondWith(fetchAndApply(event.request))
})

async function fetchAndApply(request) {
  if (request.headers.get('cf-connecting-ip') === '225.0.0.1') {
    return new Response('Sorry, this page is not available.',
      { status: 403, statusText: 'Forbidden' })
  }

  return fetch(request)
}
```

Post requests

Reading content from an HTTP POST request:

```
addEventListener('fetch', event => {
  event.respondWith(fetchAndApply(event.request))
})

/**
 * Making a curl request that looks like
 * curl -X POST --data 'key=world' example.com
 * or
 * curl -X POST --form 'key=world' example.com
 */
async function fetchAndApply(request) {
  try {
    const postData = await request.formData();
    return new Response(`hello ${postData.get('key')}`)
  } catch (err) {
    return new Response('could not unbundle post data')
  }
}
```

Creating an HTTP POST request from an Edge function:

```
addEventListener('fetch', event => {
  event.respondWith(fetchAndApply(event.request))
})

/**
 * Create a POST request with body 'key=world'
 * Here, we are assuming that example.com acknowledges the POST request with body key=world
 */
async function fetchAndApply(request) {
  let content = 'key=world'
  let headers = {
    'Content-Type': 'application/x-www-form-urlencoded'
  }
  const init = {
    method: 'POST',
    headers: headers,
    body: content
  }
  const response = await fetch('https://example.com', init)
  console.log('Got response', response)
```

```
return response
}
```

Setting a cookie

You can set cookies by using CIS Edge functions.

```
addEventListener('fetch', event => {
  event.respondWith(fetchAndApply(event.request))
})

async function fetchAndApply(request) {
  let response = await fetch(request)

  const randomStuff = `randomcookie=${Math.random()}; Expires=Wed, 21 Oct 2018 07:28:00 GMT; Path='/`;

  // Make the headers mutable by re-constructing the Response.
  response = new Response(response.body, response)
  response.headers.set('Set-Cookie', randomStuff)

  return response
}
```

Signed requests

A common URL authentication method known as request signing can be implemented in an Edge function with the help of the Web Crypto API.

In the example presented here, CIS authenticates the path of a URL along with an accompanying expiration timestamp by using a Hash-based Message Authentication Code (HMAC) with an SHA-256 digest algorithm. To successfully fetch an authenticated resource, the user agent needs to provide the correct path, expiration timestamp, and HMAC by using query parameters. If any of these three parameters are tampered with, the request fails.

The authenticity of the expiration timestamp is covered by the HMAC, which means that you can rely on the user-provided timestamp to be correct if the HMAC is correct, and when the URL expires. You can also determine whether a URL in their possession is expired.

Verifying signed requests

This example verifies the HMAC for any request URL where the path name starts with `/verify/`.

For debugging convenience, this Edge function returns a 403 message if the URL or HMAC is invalid, or if the URL is expired. You might want to return 404 in an actual implementation.

```
addEventListener('fetch', event => {
  event.respondWith(verifyAndFetch(event.request))
})

async function verifyAndFetch(request) {
  const url = new URL(request.url)

  // If the path doesn't begin with our protected prefix, just pass the request
  // through.
  if (!url.pathname.startsWith("/verify/")) {
    return fetch(request)
  }

  // Make sure we have the minimum necessary query parameters.
  if (!url.searchParams.has("mac") || !url.searchParams.has("expiry")) {
    return new Response("Missing query parameter", { status: 403 })
  }

  // We'll need some super-secret data to use as a symmetric key.
  const encoder = new TextEncoder()
  const secretKeyData = encoder.encode("my secret symmetric key")
  const key = await crypto.subtle.importKey(
    "raw", secretKeyData,
    { name: "HMAC", hash: "SHA-256" },
    false, [ "verify" ]
  )
```

```

// Extract the query parameters we need and run the HMAC algorithm on the
// parts of the request we're authenticating: the path and the expiration
// timestamp.
const expiry = Number(url.searchParams.get("expiry"))
const dataToAuthenticate = url.pathname + expiry

// The received MAC is Base64-encoded, so we have to go to some trouble to
// get it into a buffer type that crypto.subtle.verify() can read.
const receivedMacBase64 = url.searchParams.get("mac")
const receivedMac = byteStringToUint8Array(atob(receivedMacBase64))

// Use crypto.subtle.verify() to guard against timing attacks. Since HMACs use
// symmetric keys, we could implement this by calling crypto.subtle.sign() and
// then doing a string comparison -- this is insecure, as string comparisons
// bail out on the first mismatch, which leaks information to potential
// attackers.
const verified = await crypto.subtle.verify(
  "HMAC", key,
  receivedMac,
  encoder.encode(dataToAuthenticate)
)

if (!verified) {
  const body = "Invalid MAC"
  return new Response(body, { status: 403 })
}

if (Date.now() > expiry) {
  const body = `URL expired at ${new Date(expiry)}`
  return new Response(body, { status: 403 })
}

// We've verified the MAC and expiration time; we're good to pass the request
// through.
return fetch(request)
}

// Convert a ByteString (a string whose code units are all in the range
// [0, 255]), to a Uint8Array. If you pass in a string with code units larger
// than 255, their values overflow!
function byteStringToUint8Array(byteString) {
  const ui = new Uint8Array(byteString.length)
  for (let i = 0; i < byteString.length; ++i) {
    ui[i] = byteString.charCodeAt(i)
  }
  return ui
}

```

Generating signed requests

Typically, signed requests are delivered to you in some out-of-band way, such as an email, or you generated one by yourself if you have the symmetric key. You can also generate the signed requests in an Edge function.

For any request URL that begins with `/generate/`, CIS replaces `/generate/` with `/verify/`, signs the resulting path with its timestamp, and returns the full, signed URL in the response body.

```

addEventListener('fetch', event => {
  const url = new URL(event.request.url)
  const prefix = "/generate/"
  if (url.pathname.startsWith(prefix)) {
    // Replace the "/generate/" path prefix with "/verify/", which we
    // use in the first example to recognize authenticated paths.
    url.pathname = `/verify/${url.pathname.slice(prefix.length)}`
    event.respondWith(generateSignedUrl(url))
  } else {
    event.respondWith(fetch(event.request))
  }
})

async function generateSignedUrl(url) {

```



```
// We'll need some super-secret data to use as a symmetric key.
const encoder = new TextEncoder()
const secretKeyData = encoder.encode("my secret symmetric key")
const key = await crypto.subtle.importKey(
  "raw", secretKeyData,
  { name: "HMAC", hash: "SHA-256" },
  false, [ "sign" ]
)

// Signed requests expire after one minute. Note that you could choose
// expiration durations dynamically, depending on, e.g. the path or a query
// parameter.
const expirationMs = 60000
const expiry = Date.now() + expirationMs
const dataToAuthenticate = url.pathname + expiry

const mac = await crypto.subtle.sign(
  "HMAC", key,
  encoder.encode(dataToAuthenticate)
)

// `mac` is an ArrayBuffer, so we need to jump through a couple hoops to get
// it into a ByteString, then a Base64-encoded string.
const base64Mac = btoa(String.fromCharCode(...new Uint8Array(mac)))

url.searchParams.set("mac", base64Mac)
url.searchParams.set("expiry", expiry)

return new Response(url)
}
```

Streaming responses

An Edge function script doesn't need to prepare its entire response body before it delivers a Response to `event.respondWith()`. By using a TransformStream, you can stream a response body after you send the response's front matter (for example, HTTP status line and headers). This streamlining helps CIS to minimize the visitor's time-to-first-byte and the amount of buffering that must be done in the Edge function script.

Minimizing buffering is especially important if you must process or transform response bodies that are larger than the Edge function's memory limit. In these cases, streaming is the only feasible implementation strategy.



Note: The CIS Edge Function service already streams by default wherever possible. These APIs are only needed if you want to modify the response body in some way, while you maintain the streaming behavior. If your Edge function script passes subrequest responses back to the client verbatim, without reading their bodies, then the body handling is already optimal.

Streaming pass-through

Get started with the following minimal pass-through example.

```
addEventListener("fetch", event => {
  event.respondWith(fetchAndStream(event.request))
})

async function fetchAndStream(request) {
  // Fetch from origin server.
  let response = await fetch(request)

  // Create an identity TransformStream (a.k.a. a pipe).
  // The readable side becomes our new response body.
  let { readable, writable } = new TransformStream()

  // Start pumping the body. NOTE: No await!
  streamBody(response.body, writable)

  // ... and deliver our Response while that's running.
  return new Response(readable, response)
}

async function streamBody(readable, writable) {
```

```

let reader = readable.getReader()
let writer = writable.getWriter()

while (true) {
  const { done, value } = await reader.read()
  if (done) break
  // Optionally transform value's bytes here.
  await writer.write(value)
}

await writer.close()
}

```

Some important details to note:

- Although `streamBody()` is an asynchronous function, you do not want to call `await` on it so that it does not block the progress of calling `fetchAndStream()` function. The function continues to run asynchronously for the period in which it has an outstanding `reader.read()` or `writer.write()` operation.
- Backpressure: `await` the read operation before you call the write operation. Likewise, `await` the write operation before you call the next read operation. Following this pattern propagates backpressure to the origin.
- Completion: call `writer.close()` at the end, which signals to the Edge function runtime that you're done writing this response body. After being called, `streamBody()` terminates — if this behavior is undesirable, pass its returned promise to `FetchEvent.waitUntil()`. If your script never calls `writer.close()`, the body appears truncated to the runtime, though it might continue to function as intended.

Aggregate and stream multiple requests

This use case is similar to the aggregating multiple requests recipe, but this time you start writing the response as soon as you verify that every subrequest succeeded — no need to wait for the response bodies.

```

addEventListener('fetch', event => {
  event.respondWith(fetchAndApply(event.request))
})

/**
 * Make multiple requests,
 * aggregate the responses and
 * stream it back as a single response.
 */
async function fetchAndApply(request) {
  const requestInit = {
    headers: { "Authorization": "XXXXXXX" }
  }
  const fetches = [
    "https://api.coinbase.com/v2/prices/BTC-USD/spot",
    "https://api.coinbase.com/v2/prices/ETH-USD/spot",
    "https://api.coinbase.com/v2/prices/LTC-USD/spot"
  ].map(url => fetch(url, requestInit))

  // Wait for each fetch() to complete.
  let responses = await Promise.all(fetches)

  // Make sure every subrequest succeeded.
  if (!responses.every(r => r.ok)) {
    return new Response(null, { status: 502 })
  }

  // Create a pipe and stream the response bodies out
  // as a JSON array.
  let { readable, writable } = new TransformStream()
  streamJsonBodies(responses.map(r => r.body), writable)

  return new Response(readable)
}

async function streamJsonBodies(bodies, writable) {
  // We're presuming these bodies are JSON, so we
  // concatenate them into a JSON array. Since we're
  // streaming, we can't use JSON.stringify(), but must
  // instead manually write an initial '[' before the

```

```

// bodies, interpolate ',' between them, and write a
// terminal ']' after them.

let writer = writable.getWriter()
let encoder = new TextEncoder()

await writer.write(encoder.encode("[\n"))

for (let i = 0; i < bodies.length; ++i) {
  if (i > 0) {
    await writer.write(encoder.encode(",\n"))
  }
  writer.releaseLock()
  await bodies[i].pipeTo(writable, { preventClose: true })
  writer = writable.getWriter()
}

await writer.write(encoder.encode("]"))

await writer.close()
}

```



Note: The runtime expects to receive TypedArrays on the readable side of the TransformStream. Therefore, you never pass a string to `writer.write()`, only Uint 8Arrays. If you need to write a string, use a TextEncoder.

Custom load balancer with Edge functions

Load balancing helps you maintain the scalability and reliability of the websites you host. You can use Edge functions to create custom load balancers that are designed to address your specific needs.

```

const US_HOSTS = [
  "0.us.example.com",
  "1.us.example.com",
  "2.us.example.com"
];

const IN_HOSTS = [
  "0.in.example.com",
  "1.in.example.com",
  "2.in.example.com"
];

var COUNTRIES_MAP = {
  IN: IN_HOSTS,
  PK: IN_HOSTS,
  BD: IN_HOSTS,
  SL: IN_HOSTS,
  NL: IN_HOSTS
}

addEventListener('fetch', event => {
  var url = new URL(event.request.url);

  var countryCode = event.request.headers.get('CF-IPCountry');
  var hostnames = US_HOSTS;
  if (COUNTRIES_MAP[countryCode]) {
    hostnames = COUNTRIES_MAP[countryCode];
  }
  // Randomly pick the next host
  var primary = hostnames[getRandomInt(hostnames.length)];

  var primaryUrl = new URL(event.request.url);
  primaryUrl.hostname = hostnames[primary];

  // Fallback if there is no response within timeout
  var timeoutId = setTimeout(function() {
    var backup;
    do {
      // Naive solution to pick a backup host

```

```

        backup = getRandomInt(hostnames.length);
    } while(backup === primary);

    var backupUrl = new URL(event.request.url);
    backupUrl.hostname = hostnames[backup];

    event.respondWith(fetch(backupUrl));
}, 2000 /* 2 seconds */);

fetch(primaryUrl)
    .then(function(response) {
        clearTimeout(timeoutId);
        event.respondWith(response);
    });
});

function getRandomInt(max) {
    return Math.floor(Math.random() * max);
}

```

Caching using fetch

Determine how to cache a resource by setting TTLs, custom cache keys, and cache headers in a fetch request.

```

async function handleRequest(request) {
    const url = new URL(request.url)

    // Only use the path for the cache key, removing query strings
    // and always store using HTTPS, for example, https://www.example.com/file-uri-here
    const someCustomKey = `https://${url.hostname}${url.pathname}`

    let response = await fetch(request, {
        cf: {
            // Always cache this fetch regardless of content type
            // for a max of 5 seconds before revalidating the resource
            cacheTtl: 5,
            cacheEverything: true,
            //Enterprise only feature, see Cache API for other plans
            cacheKey: someCustomKey,
        },
    })
    // Reconstruct the Response object to make its headers mutable.
    response = new Response(response.body, response)

    //Set cache control headers to cache on browser for 25 minutes
    response.headers.set("Cache-Control", "max-age=1500")
    return response
}

addEventListener("fetch", event => {
    return event.respondWith(handleRequest(event.request))
})

```

Caching HTML resources

```

// Force CIS to cache an asset
fetch(event.request, { cf: { cacheEverything: true } })

```

Setting the cache level to Cache Everything overrides the default "cacheability" of the asset. For TTL, CIS still relies on headers set by the origin.

Custom cache keys



Note: This feature is available only to enterprise customers.

A request's cache key is what determines whether two requests are "the same" for caching purposes. If a request has the same cache key as some previous request, then we can serve the same cached response for both.

```
// Set cache key for this request to "some-string".
fetch(event.request, { cf: { cacheKey: "some-string" } })
```

CIS computes the cache key for a request based on the request's URL, but you might want different URLs to be treated as if they were the same for caching purposes. For example, if your website content is hosted from both Amazon S3 and Google Cloud Storage (you have the same content in both places), and you then use an edge function to randomly balance between the two. However, you don't want to cache two copies of your content. You can use custom cache keys to cache based on the original request URL rather than the subrequest URL.

```
addEventListener("fetch", (event) => {
  let url = new URL(event.request.url)
  if (Math.random() < 0.5) {
    url.hostname = "example.s3.amazonaws.com"
  }
  else {
    url.hostname = "example.storage.googleapis.com"
  }

  let request = new Request(url, event.request)
  event.respondWith(
    fetch(request, {
      cf: { cacheKey: event.request.url },
    })
  )
})
```

Remember, edge functions that operate on behalf of different zones can't affect each other's cache. You can override cache keys only when you make requests within your own zone (in the previous example `event.request.url` was the key stored), or requests to hosts that are not on CIS. When you make a request to another CIS zone (for example, a zone that belongs to a different CIS customer), that zone fully controls how its own content is cached within CIS; you can't override it.

Override based on origin response code



Note: This feature is available only to Enterprise customers.

```
// Force response to be cached for 86400 seconds for 200 status
// codes, 1 second for 404, and do not cache 500 errors.
fetch(request, {
  cf: { cacheTtlByStatus: { "200-299": 86400, 404: 1, "500-599": 0 } },
})
```

This option is a version of the `cacheTtl` feature which chooses a TTL based on the response's status code and does not automatically set `cacheEverything: true`. If the response to this request has a status code that matches, CIS caches for the instructed time, and override cache directives sent by the origin.

TTL interpretation

The following TTL values are interpreted by CIS.

- Positive values: Indicate in seconds how long CIS should cache the asset for.
- `0`: The asset is cached but expires immediately (revalidate from origin every time).
- `-1` or any negative value: Instructs CIS not to cache at all.

Cache API

Cache by using the CIS Cache API. This example can also cache POST requests.

```
const someOtherHostname = "my.herokuapp.com"

async function handleRequest(event) {
  const request = event.request
  const cacheUrl = new URL(request.url)

  // Hostname for a different zone
  cacheUrl.hostname = someOtherHostname
```



```

const cacheKey = new Request(cacheUrl.toString(), request)
const cache = caches.default

// Get this request from this zone's cache
let response = await cache.match(cacheKey)

if (!response) {
  //If not in cache, get it from origin
  response = await fetch(request)

  // Must use Response constructor to inherit all of response's fields
  response = new Response(response.body, response)

  // Cache API respects Cache-Control headers. Setting max-age to 10
  // will limit the response to be in cache for 10 seconds max
  response.headers.append("Cache-Control", "max-age=10")

  // Store the fetched response as cacheKey
  // Use waitUntil so computational expensive tasks don't delay the response
  event.waitUntil(cache.put(cacheKey, response.clone()))
}
return response
}

async function sha256(message) {
  // encode as UTF-8
  const msgBuffer = new TextEncoder().encode(message)

  // hash the message
  const hashBuffer = await crypto.subtle.digest("SHA-256", msgBuffer)

  // convert ArrayBuffer to Array
  const hashArray = Array.from(new Uint8Array(hashBuffer))

  // convert bytes to hex string
  const hashHex = hashArray.map(b => ("00" + b.toString(16)).slice(-2)).join("")
  return hashHex
}

async function handlePostRequest(event) {
  const request = event.request
  const body = await request.clone().text()
  const hash = await sha256(body)
  const cacheUrl = new URL(request.url)

  // Store the URL in cache by prepending the body's hash
  cacheUrl.pathname = "/posts" + cacheUrl.pathname + hash

  // Convert to a GET to be able to cache
  const cacheKey = new Request(cacheUrl.toString(), {
    headers: request.headers,
    method: "GET",
  })

  const cache = caches.default

  //Find the cache key in the cache
  let response = await cache.match(cacheKey)

  // Otherwise, fetch response to POST request from origin
  if (!response) {
    response = await fetch(request)
    event.waitUntil(cache.put(cacheKey, response.clone()))
  }
  return response
}

addEventListener("fetch", event => {
  try {
    const request = event.request
    if (request.method.toUpperCase() === "POST")

```

```
        return event.respondWith(handlePostRequest(event))
    }
    return event.respondWith(handleRequest(event))
} catch (e) {
    return event.respondWith(new Response("Error thrown " + e.message))
}
})
```

Service dependencies

Service dependency map for IBM Cloud Internet Services

If a service depends on other IBM Cloud services, there can be impacts if any of the dependent services are having issues. The dependency severity indicates the impact to the service when the dependency is down.

Critical

When the the dependency is down, the service is down.

Significant

When the dependency is down, the service features are impacted.

Medium

When the dependency is down, the service might be impacted and a workaround is possible.

Minimal

When the dependency is down, the main service features are not impacted.

CIS Control Plane

The following dependencies apply to the following deployment locations: Dallas (us-south), Frankfurt (eu-de), Washington DC (us-east).

Dependencies	Dependency impacts	Customer provided	Control or data plane	Location of dependency
IBM Cloud® Internet Services	Availability, Disaster recovery, Operations, Security compliance	No	control-plane	Same region
IBM Cloud Global Resource Catalog	Availability, Instance control	No	control-plane	Global
IBM Cloud Service Endpoints	Availability, Disaster recovery, Instance control	No	control-plane	Same data center
IBM Cloud Kubernetes Service and Red Hat OpenShift on IBM Cloud and ROVS	Availability, Change management, Disaster recovery, Instance control, Operations, Security compliance	No	control-plane	Same region
IBM Cloud® Identity and Access Management	Access management, Availability, Disaster recovery, Instance control, Security compliance	No	control-plane	Global
IBM® Key Protect for IBM Cloud®	Access management, Availability, Disaster recovery, Instance control, Security compliance	No	control-plane	Same region
IBM Cloud® Virtual Servers	Availability, Change management, Disaster recovery, Instance control	No	control-plane	Same data center
IBM Cloud Business Support Services	Availability	No	control-plane	Global

IBM Cloud Internet Services - CIS Control Plane service dependency information - Critical dependencies

Dependencies	Dependency impacts	Customer provided	Control or data plane	Location of dependency

IBM Cloud Console	Availability, Instance control, Operations	No	control-plane	Global
IBM Cloud® Secrets Manager	Availability, Change management, configuration-management, Disaster recovery, Instance control, Operations, Security compliance	No	control-plane	Same region
IBM Cloud Virtual Private Endpoint for VPC	Availability, Disaster recovery, Instance control	No	control-plane	Same zone
IBM Cloud Log Routing	Availability, Operations	No	control-plane	Same region

IBM Cloud Internet Services - CIS Control Plane service dependency information - Significant dependencies				
Dependencies	Dependency impacts	Customer provided	Control or data plane	Location of dependency
IBM Cloud Global Protect VPN	Security compliance	No	control-plane	Same data center
IBM Cloud Global Search and Tagging	Security compliance	No	control-plane	Global
IBM Cloud® Monitoring	Operations, Security compliance	No	control-plane	Same region
IBM Cloud® Activity Tracker Event Routing	Operations, Security compliance	No	control-plane	Same region
IBM Cloud® Object Storage	Disaster recovery, Security compliance	No	control-plane	Same data center
Synthetics	Operations	No	control-plane	Same region

IBM Cloud Internet Services - CIS Control Plane service dependency information - Minimal dependencies				
---	--	--	--	--

The CIS control plane is managed by IBM, while the data plane is managed by Cloudflare.

The following dependencies apply to the following deployment locations: Dallas (us-south).

Dependencies	Dependency impacts	Customer provided	Control or data plane	Location of dependency
IBM Cloud® Internet Services	Availability, Disaster recovery, Operations, Security compliance	No	data-plane	Same region

IBM Cloud Internet Services - The CIS control plane is managed by IBM, while the data plane is managed by Cloudflare. service dependency information - Critical dependencies				
--	--	--	--	--

Understanding service dependency data

If you have any questions about the service dependency data as you review the service dependency information in the tables, you can refer to the following FAQ:

What is the expected impact to the functions described?

Each severity tab in the table indicates the impact that your provisioned service might encounter if the dependency were to go offline. This means that the dependency high availability and disaster recovery influences the severity of the impact and therefore is used for general guidance to help you understand potential issues that might arise if the dependency was impacted by an incident.



Note: Services that are regional are not impacted by a severe outage of a single zone because of the failover that is built in to default to another zone. For these occurrences, there might be a slight performance impact, if any, while the system fails over to the other location. This also applies to global services where the impact is lowered even more as it can fail over to other regions if necessary. This reduces the

frequency at which these items might have the impact that is shown.

What services does my service depend on?

The **Dependencies** column lists the services. These are the major service to service dependencies including major internal dependencies that might not be visible externally.

What function does the dependency impact?

Functions include access management, availability, change management, configuration management, customer responsibility, disaster recovery, instance control, operations, security compliance, or none. If the dependency goes offline, these functions might be impacted. Definitions for each available value are as follows:

Access management

Authentication, authorization, and governance of the customer users access to the service and service instances.

Availability

Availability of the service and service instances.

Change management

Deployment, upgrade, patch, and so on, of the service and service instances.

Configuration management

Deployment, upgrade, patch, and so on, of the service and service instances.

Customer responsibility

Functions provided by customers to support specific service and service instances function. For example, IBM Key Protect for IBM Cloud instances provided by customer to support service BYOK encryption.

Disaster recovery

Backup, recovery, restart of the service and service instances in the case of a disruption.

Instance control

Creation, deletion, start, stop actions on the lifecycle of the service instances.

Operations

Monitoring, troubleshooting, and so on, of the service and service instances.

Security compliance

Vulnerability management and other security and compliance management of the service and service instances.

None

No function impacted.

What does customer provided mean for dependencies?

The **Customer provided** column reports if there is any dependency that has been provided by the customer to enable specific functionality. One such example is to properly configure and set up using BYOK into a service, the customer would provision a service like IBM Key Protect for IBM Cloud. For more information about how to enable the features and which services you need to provision, see the documentation for the service.

Where do dependency services need to be deployed regarding my service?

In the **Location of dependency** column, you can view if the dependency is located in the same region or deployed to a specific data center. You can use this data with the data in the **Control or data plane** column for a quick reference to identify if your data leaves the region or not in a standard setup.

To find where your service can be deployed, see [Service and infrastructure availability by location](#).

 **Note:** The table shows a standard cloud deployment. If a special deployment is used like FedRAMP or other region-bound deployment models, the data might differ from the details available in the table. Refer to the specific deployment that you are using for that information.

Where are the separate control plane and data plane located, if applicable?

Sometimes, the dependency might have a separate control plane and data plane. In these cases, there are separate lines that show the location in relation to the deployed customer instance of the service where these will be provisioned. The lines might have different impacts and different functions. See the **Control or data plane** column to understand what possible impact this type of outage might have.

Same region means that the dependent services are in the same region as the provisioned instance. Other values might show **data center** or region names if the service must be used from a specific region, a specific zone, or set of zones. If a service is tied to a specific region or site, and the region goes offline, the service might go offline as well. Review the high availability and disaster recovery documentation of the dependency to determine whether any steps are needed to mitigate these types of risks.

Additional resources

For more information about the policies that are related to the services, you can refer to the following resources:

- [Service Level Agreement](#)
- [Shared responsibilities for using IBM Cloud products](#)
- [Service and infrastructure availability by location](#)

High availability and disaster recovery

Understanding high availability for CIS

High availability (HA) is a core discipline in an IT infrastructure to keep your apps up and running, even after a partial or full site failure. The main purpose of high availability is to eliminate potential points of failures in an IT infrastructure.

CIS is a globally available GA service, with public API endpoints accessible worldwide through a global load balancer across three multi-zone regions: Dallas (`us-south`), Washington, DC (`us-east`), and London (`eu-gb`). In the event of an outage in one region, the global load balancer automatically routes API traffic to the nearest available region. Data is replicated across these regions for optimized latency and high availability.

See [How IBM Cloud ensures high availability and disaster recovery](#) to learn more about the high availability and disaster recovery standards in IBM Cloud. You can also find information about [Service Level Agreements](#).

Responsibilities

To find out more about responsibility ownership for using CIS between IBM and the customer, see [Understanding your responsibilities when using CIS](#).

What level of availability do I need?

You can achieve high availability on different levels in your IT infrastructure and within different components of your cluster. The level of availability that is appropriate for you depends on several factors, such as your business requirements, the service level agreements (SLAs) that you have with your customers, and the resources that you want to expend.

What level of availability does IBM Cloud offer?

The level of availability that you set up for your cluster impacts your coverage under the IBM Cloud high availability service level agreement terms.

Service level objectives (SLOs) describe the design points that the IBM Cloud services are engineered to meet. CIS is designed to achieve the following availability target.

Availability Target	Target Value
Availability %	99.999%
SLO for CIS	

The SLO is not a warranty and IBM will not issue credits for failure to meet an objective. Refer to the SLAs for commitments and credits that are issued for failure to meet any committed SLAs. For a summary of all SLOs, see [IBM Cloud service level objectives](#).

Locations

For more information about service availability within regions and data centers, see [Service and infrastructure availability by location](#).

Understanding business continuity and disaster recovery for CIS

Disaster recovery involves a set of policies, tools, and procedures for returning a system, an application, or an entire data center to full operation after a catastrophic interruption. It includes procedures for copying and storing an installed system's essential data in a secure location, and for recovering that data to restore normalcy of operation.

Responsibilities

To find out more about responsibility ownership for using IBM Cloud® products between IBM and customer see [Understanding your responsibilities when using CIS](#).

Disaster recovery strategy

IBM Cloud has *business continuity* plans in place to provide for the recovery of services within hours if a disaster occurs. You are responsible for your data backup and associated recovery of your content.

CIS provides mechanisms to protect your data and restore service functions. Business continuity plans are in place to achieve targeted *recovery point objective* (RPO) and *recovery time objective* (RTO) for the service. The following table outlines the targets for CIS.

Disaster recovery objective	Target value
RPO	Failover happens almost instantly (within seconds) if internal, as it switches to another cluster. However, if Cloudflare experiences downtime or issues, data will be unavailable until they are back online, as CIS is a stateless service.
RTO	Recovery time for CIS is approximately 1 hour per cluster, including verification and other processes.

Table 1. RPO and RTO for CIS

Locations

For more information about service availability within regions and data centers, see [Service and infrastructure availability by location](#).

Understanding high availability and disaster recovery for CIS

High Availability (HA) is the ability for a service to remain operational and accessible in the presence of unexpected failures. *Disaster recovery* (DR) is the process of recovering the service instance to a working state.

Cloud Internet Services (CIS) is a global service designed to remain operational during regional outages. CIS is designed to meet the [Service Level Objectives \(SLO\)](#) with all plans.

CIS is a globally available GA service, with public API endpoints accessible worldwide through a global load balancer across three multi-zone regions: Dallas (`us-south`), Washington, DC (`us-east`), and London (`eu-gb`). In the event of an outage in one region, the global load balancer automatically routes API traffic to the nearest available region. Data is replicated across these regions for optimized latency and HA.

See [How IBM Cloud ensures high availability and disaster recovery](#) to learn more about the HA and DR standards in IBM Cloud. You can also find information about [Service Level Agreements](#).

High availability architecture

CIS is architected to support HA across multiple layers of your IT infrastructure. You can design for HA at different levels, depending on business requirements, Service Level Agreements (SLAs), and available resources.

CIS separates its architecture into two core components: the control plane (API and configuration layer) and the data plane (traffic processing and enforcement layer). This separation is key to ensuring service continuity during component-specific issues.

- The CIS API control plane allows you to configure and manage services such as DNS records, firewall rules, and load-balancing policies. It runs on IBM Cloud and may experience occasional outages or maintenance windows. While rare, control plane availability might not always match that of the data plane. If unavailable, you may not be able to make changes, but existing configurations continue to be enforced.
- The CIS data plane, powered by [Cloudflare's global Anycast network](#), is responsible for serving live traffic, including DNS resolution, DDoS protection, content delivery, and firewall enforcement. It is designed for extremely high uptime and resilience. The data plane continues to operate normally even when the control plane is down, preserving performance and security.

This architecture ensures that control plane disruptions do not impact end-user traffic, providing a resilient foundation for highly available application delivery.

CIS is designed to meet the following Service Level Objectives (SLOs):

Availability Target	Target Value
Availability %	99.999%
Global Anycast Network Uptime	100%
DNS Resolution Time	< 50 ms
DDoS Mitigation Response Time	< 3 seconds
Configuration Propagation Time	< 60 seconds

SLO for CIS

These targets reflect the robustness of the data plane, which delivers the actual services to your users. You get the most value from CIS through the performance and reliability of this layer, which is engineered for global scale and HA.

**Note:** The SLO is not a warranty and IBM will not issue credits for failure to meet an objective. Refer to the SLAs for commitments and credits that are issued for failure to meet any committed SLAs. For a summary of all SLOs, see [IBM Cloud service level objectives](#).

High availability features

HA in CIS is achieved through a globally distributed, resilient infrastructure and integrated service-level mechanisms. These features are designed to minimize downtime and ensure continuous service delivery even in the face of failures or network disruptions.

CIS supports the following HA features:

Feature	Description	Consideration
Cloudflare's Global Anycast Network	Routes traffic through the nearest data center to reduce latency and reroute in case of failure.	Ensures consistent availability and low latency across regions.
Redundant DNS infrastructure	Uses distributed DNS servers for HA and rapid failover.	Helps maintain DNS resolution even during regional disruptions.
Always-On DDoS protection	Automatically detects and mitigates large-scale DDoS attacks in real-time.	Does not require manual intervention; protection is active 24/7.
HA features for CIS		

As a customer, you can create and configure the following other HA options:

Feature	Description	Consideration
Load balancing with health checks	Distributes traffic across multiple, healthy origins to avoid single points of failure.	Health checks must be correctly configured to detect outages accurately.
Rate-limiting rules (not on by default)	You can define rules to limit request rates per IP or endpoint to prevent overload.	Enterprise Plan only
Web Application Firewall (WAF)	Filters, monitors, and blocks malicious HTTP/S traffic based on predefined and custom rules.	Proper rule tuning is essential to avoid false positives or missed threats.
Custom rules	Allow you to define logic (for example, IP blocking, header inspection) to discard or redirect malicious traffic.	Misconfigured rules could block valid users or degrade service performance.
Range applications	Enables Layer 4 DDoS protection for TCP applications (for example, SSH, FTP) by proxying traffic through Cloudflare.	Enterprise Plan only
Bot management for bot mitigation	Identifies and mitigates automated, potentially harmful traffic to protect application uptime.	Enterprise Premier Plan only
Customer HA features for CIS		

Disaster recovery architecture

DR for CIS builds on the same architectural separation of the control plane and data plane, enabling resilient operations even during service disruptions. Outages in one plane do not necessarily affect the other, which is central to DR planning.

- The control plane, used for service configuration and management, runs on IBM Cloud and integrates with Cloudflare infrastructure. In a control plane outage (such as a regional IBM Cloud issue or Cloudflare management disruption), you might not be able to make changes or access analytics. However, existing configurations remain in effect, and your application traffic is not impacted. Deferred changes can be applied once services are restored.
- The data plane continues to enforce traffic policies and serve users during a control plane outage. In the rare event of a regional data plane disruption, Cloudflare’s Anycast network automatically reroutes traffic to healthy nodes, minimizing downtime and maintaining service continuity.

CIS provides mechanisms to protect your data and restore service functions. Business continuity plans are in place to achieve targeted *Recovery Point Objective* (RPO) and *Recovery Time Objective* (RTO) for the service. The following table outlines the targets IBM Cloud for the recovery of the CIS service.

DR objective	Target value
RPO	Failover happens almost instantly (within seconds) if internal, as it switches to another web application. If Cloudflare experiences downtime or issues, data will be unavailable until they are back online, as CIS is a stateless service.
RTO	Recovery time for the control plane is approximately 1 hour per web application, including verification and other processes.

RPO and RTO for CIS

IBM Cloud has *business continuity* plans in place to provide for the recovery of services within hours if a disaster occurs. You are responsible for your data backup and associated recovery of your content.

To support DR readiness, you can monitor live service status through the [Cloudflare System Status](#) page and configure alerts to detect incidents early. Best practices include minimizing the need for urgent config changes, using version-controlled configurations, and maintaining redundancy in your origin infrastructure.

This resilient architecture ensures most disruptions are isolated, and your applications remain available even during partial service outages.

Disaster recovery features

DR for CIS is supported through a combination of platform capabilities and customer-managed configurations. The service is designed to maintain resilience and minimize downtime through stateless architecture and global infrastructure.

CIS supports the following DR features:

Feature	Description	Consideration
Stateless architecture	CIS does not persist your data, enabling quick re-deployment in disaster events.	Store configuration backups externally if needed.
Geo-distributed infrastructure	CIS leverages a global Anycast network to reduce regional DR risks.	Limits impact of localized failures and ensures traffic rerouting.

DR features for CIS

As a customer, you can create and support the following other DR options:

Feature	Description	Consideration
Configuration export & backup	Manually export DNS records, WAF rules, and other CIS settings using the CIS API	Provides a backup in case configurations are lost or corrupted. Useful for offline review or reconfiguration.
Infrastructure as Code (IaC)	Use Terraform or similar tools to define and manage CIS configurations.	Enables consistent, repeatable deployments and rapid restoration of services in new or recovered environments.
Logpush & Security Event review	Use Logpush to export logs and review security events.	Helps you investigate incidents, analyze trends, and identify root causes during or after a disruption.
Traffic metrics & monitoring	Continuously monitor traffic volume, error rates, and performance metrics.	Spikes or anomalies in traffic can indicate issues such as DDoS attacks, misconfigurations, or origin failures. Early detection supports faster response.

Customer DR features for CIS

Planning for disaster recovery

The steps outlined in your HA and DR plans should be regularly reviewed and tested. As you build and refine your strategy, consider the following common failure scenarios and how CIS responds at both the control path (management/API layer) and data path (traffic processing layer).

CIS is built on a globally distributed, stateless architecture that minimizes the risk of persistent service failures. While the platform includes HA and DR capabilities by design, you are responsible for applying best practices, managing configuration state, and preparing for application-layer recovery as needed.

Failure	Resolution
Control plane outage	You might be unable to modify configurations or retrieve updated metrics/logs, but existing data path policies (such as DNS routing and WAF enforcement) remain active. IBM Cloud opens a Customer-Initiated Event (CIE) to restore control path functionality, and you are notified. Recovery time varies; see the Cloudflare System Status page for updates.
Data path outage	In rare cases, a localized outage in edge network might impact traffic. CIS will route requests to healthy data centers automatically. If needed, you can temporarily disable the CIS proxy (grey-cloud the record) to route traffic directly to their origin.
Regional failure	CIS will continue to serve traffic globally, but if your origin servers are not deployed across multiple regions, end-user traffic can still experience downtime. You should deploy origin infrastructure redundantly and test failover scenarios to ensure full-stack availability.
Configuration loss or misconfiguration	CIS is stateless, so customer-defined configurations are not retained by default. Use Infrastructure as Code (for example, Terraform) or export configurations via the CIS API (GET calls). Recovery requires manually recreating resources (POST) using saved data. API changes (PATCH/PUT) can be tracked using Activity Tracker to help identify and revert unintended changes.
DDoS attack or malicious traffic	CIS includes always-on DDoS protection, bot management, and rate limiting. These features operate at the data path and do not require control path interaction. Protections are enforced automatically, but you should monitor traffic metrics and tune thresholds to reflect their application profile.

DR scenarios for CIS

Your responsibilities for high availability and disaster recovery

It is your responsibility to continuously test your plan for HA and DR when using CIS.

**Note:** Interruptions in network connectivity and short periods of service unavailability may occur. You must ensure that your application includes [client availability retry logic](#) to help maintain overall application availability.

Use the following checklists for key CIS features to help you create, verify, and practice your HA and DR plans:

- Configuration backups
 - ☐ Regularly export and back up DNS, WAF, and security settings
 - ☐ Validate restoration of configuration backups in a test environment
- Origin infrastructure resilience
 - ☐ Deploy origin servers across multiple zones and regions on your web application
 - ☐ Test failover and recovery of origin endpoints during simulated outages
 - ☐ Configure the global load balancer with active health checks and multiple origin pools to enable automatic failover to healthy endpoints
- Monitoring and alerts
 - ☐ Set up notifications for CIS service health and performance metrics
 - ☐ Review Logpush security data to detect anomalies or compare against baseline metrics
 - ☐ Regularly review logs and dashboard data to detect anomalies

For more information about responsibility ownership between you and IBM Cloud for CIS, see [Understanding your responsibilities when using CIS](#).

Recovery time objective (RTO) and recovery point objective (RPO)

Feature	RTO and RPO
Stateless architecture	RTO = Near-zero (no service state to recover); RPO = N/A (no persistent data stored in CIS)

DNS and WAF configuration backups	RTO = minutes (manual or IaC redeployment); RPO = based on frequency of customer configuration backups. Use CIS API to restore a previous configuration from a saved backup. The RTO depends on the size of the restore, while the RPO depends on when the last configuration backup was created.
-----------------------------------	---

RTO/RPO features for CIS

How IBM supports disaster recovery planning

A zone failure refers to the failure of one data center or availability zone within a region. CIS mitigates the impact in these ways:

- CIS leverages a globally distributed Anycast network that automatically reroutes traffic away from failed zones to healthy data centers, minimizing downtime.
- The stateless architecture of CIS enables rapid failover without data loss or service interruption when a zone becomes unavailable.
- IBM continuously monitors the health of CIS web applications and network endpoints to detect and respond to zone-level outages in real time.
- If IBM can’t restore the service instance, you must restore the service as described in [Disaster recovery architecture](#).

How IBM recovers from zone failures

CIS is designed with HA and resilience in both the control plane and the data plane, but it's important to understand how each behaves in the event of failure.

Control plane resiliency

The CIS control plane, which handles configuration and API requests, operates across multiple geographically distributed regions. If a regional control plane becomes unavailable, traffic is automatically redirected to the next closest available region. This failover is automated and requires no action from you, ensuring continued access to your CIS configuration even if a specific control plane region is temporarily unreachable.

Data plane availability

The CIS data plane runs on Cloudflare’s global Anycast network and is independent of IBM Cloud infrastructure. The data plane is responsible for serving live traffic, including DNS resolution, DDoS protection, WAF enforcement, and content caching. Cloudflare's network is engineered for extremely HA, providing 100% uptime for its data plane services under their SLA.

Because the data plane is globally distributed and operates outside of IBM Cloud, there is no concept of CIS-specific zone failures. If your traffic is impacted, it is typically due to issues with origin infrastructure, DNS configuration, or the way customer-managed zones are set up.

How IBM recovers from regional failures

CIS is architected for HA and resilience across regions. The CIS control plane operates in multiple geographic regions and includes automatic failover. If a control plane region becomes unavailable, requests are automatically redirected to another available region. No customer action is required.

For your own applications and infrastructure, CIS can support regional failover capabilities, but only if you configure them. To maintain service availability during regional disruptions, you must:

- Deploy your origin servers and backend resources across multiple regions.
- Use CIS features, such as global load balancing or DNS-based failover, to route traffic to healthy regions.
- Ensure health checks are in place to detect regional issues and trigger routing changes.

In short, CIS provides the global infrastructure and tools to enable regional failover, but it is up to you to architect your backend services and configure CIS accordingly for full DR coverage.

Change management

Change management includes tasks such as upgrades, configuration changes, and deletion.



Tip: Grant users and processes the Identity and Access Management (IAM) roles and actions with the least privilege that is required for their work. For more information, see [How can I prevent accidental deletion of services?](#).

Best practices for managing change in CIS also include:

- Plan and document all configuration changes by maintaining a detailed change log for your CIS settings, including DNS records, WAF rules, and traffic policies.
- Use audit logs and IBM Cloud Activity Tracker Event Routing to review prior configurations and track changes. Understanding what has changed allows for better testing, validation, and rollback if necessary.
- Create backups of critical CIS configurations before performing upgrades or major changes to ensure rapid recovery if needed.
- Schedule upgrades or high-impact configuration changes during low-traffic periods, and communicate planned changes to relevant stakeholders to minimize disruption.
- Backup the CIS configuration as the new baseline after performing a successful upgrade or implementing changes.
- Monitor CIS service health and performance metrics to detect any issues early and verify that changes behave as expected.
- Export and manually back up your CIS configuration before upgrading to a new service version or applying significant changes.

How IBM maintains services

All upgrades follow IBM service best practices, including recovery plans and rollback processes. Regular maintenance might cause short interruptions, mitigated by [client availability retry logic](#). Changes are rolled out sequentially, region by region, and zone by zone within a region. IBM reverts updates at the first sign of a defect.

IBM provides advance notice for all planned maintenance activities. If a change is expected to affect your workloads, IBM communicates this through official notifications. To stay updated on maintenance, service announcements, and other updates, see the [Monitoring notifications and status](#).

Understanding data portability for CIS

Data portability involves a set of tools and procedures that enable customers to export the digital artifacts that are needed to implement similar workload and data processing on different service providers or on-premises software. It includes procedures for copying and storing the service customer content, including the related configuration that is used by the service to store and process the data, on the customer's own location.

Responsibilities

IBM Cloud services provide interfaces and instructions to guide the customer to copy and store the service customer content, including the related configuration, on their own selected location.

The customer is responsible for the use of the exported data and configuration for data portability to other infrastructures, which includes:

- The planning and execution for setting up alternative infrastructure on different cloud providers or on-premises software that provide similar capabilities to the IBM services.
- The planning and execution for the porting of the required application code on the alternative infrastructure, including the adaptation of customer's application code, deployment automation, and so on.
- The conversion of the exported data and configuration to the format that's required by the alternative infrastructure and adapted applications.

For more information about your responsibilities for CIS, see [Understanding your responsibilities when using CIS](#).

Data export procedures

CIS provides the mechanisms to export your content that's uploaded, stored, and processed when you use the service.

You can use the CLI and API to export settings and configurations that are used to process the customer's content. For example, you can export global load balancers, DNS records, advanced settings in zones, and page and firewall rules.

You can't export custom certificates, advanced certificates, or universal certificates that Cloudflare manages for you.

Exporting CIS data with the CLI and API

The following table provides mechanisms to export the settings and configurations that are used to process configuration content through the means of the CIS [CLI](#) and [ibmcloud cis access-apps](#)

[ibmcloud cis access-certificates](#)

[ibmcloud cis access-policies](#)

[ibmcloud cis alert-webhooks](#)

[ibmcloud cis authenticated-origin-pull-certificates](#)

[ibmcloud cis certificates](#)

[ibmcloud cis dns-records](#)

[ibmcloud cis domains](#)

[ibmcloud cis edge-functions-actions](#)

[ibmcloud cis edge-functions-triggers](#)

[ibmcloud cis firewalls](#)

[ibmcloud cis glb-events](#)

[ibmcloud cis glb-monitors](#)

[ibmcloud cis glb-pools](#)

[ibmcloud cis glbs](#)

[ibmcloud cis instances](#)

[ibmcloud cis origin-certificates](#)

[ibmcloud cis page-rules](#)

[ibmcloud cis ratelimit-rules](#)

[ibmcloud cis waf-groups](#)

[ibmcloud cis cis waf-overrides](#)

[ibmcloud cis waf-packages](#)

[ibmcloud cis waf-rules](#)

[ibmcloud cis alert-policy list](#)

[ibmcloud cis advanced-rate-limiting rules](#)

[ibmcloud cis managed-waf rulesets](#)

[ibmcloud cis managed-waf deployments](#) [ibmcloud cis glbs](#) command:

```
$ ibmcloud cis glbs DNS_DOMAIN_ID [-i, --instance INSTANCE] [--output FORMAT]
```

- Example API request using the [list-all-load-balancers](#) API:

```
curl -X GET https://api.cis.cloud.ibm.com/v1/crn/zones/:zone_id/load_balancers \
-H 'content-type: application/json' \
-H 'x-auth-user-token: Bearer xxxxxx'
```

CIS doesn't support the export of other data formats and other schema of the exported data, configuration, and application, including the following:

- Exporting [zone files](#)

Data ownership

All exported data is classified as customer content. Apply the full customer ownership and licensing rights, as stated in the [IBM Cloud Service Agreement](#).

Understanding your responsibilities when using CIS

Learn about the management responsibilities and terms and conditions that you have when you use IBM Cloud® Internet Services. For a high-level view of the service types in IBM Cloud® and the breakdown of responsibilities between the customer and IBM for each type, see [Shared responsibilities for IBM Cloud offerings](#).

Review the following sections for the specific responsibilities for you and for IBM when you use IBM Cloud Internet Services. For the overall terms of use, see [IBM Cloud® Terms and Notices](#).

Incident and operations management

Task	IBM Responsibilities	Your responsibilities
Availability	IBM Cloud Internet Services provides high availability through multi-zone region deployment and automatic failover between different regions in case of a region-wide failure.	
Monitoring and ongoing operation of production environments	IBM Cloud Internet Services provides continuous around the clock operational monitoring and coverage by on-call personnel with minimal response times.	
Deployments and cluster management	New features, updates, and bug fixes are continuously delivered as needed in a manner transparent to the customer. Maintenance with client impact is scheduled in advance with notifications posted to the IBM Cloud® status page.	Set preferences to receive email notifications. Monitor the IBM Cloud® status page for general announcements.
Incident management	Unplanned incidents with customer impact are communicated using the CIE process.	Impacted customers can obtain a report about the incident upon request.
Logging	Logs system-level security events and patch activities.	Enable and manage logging for the customer's domains using Logpush jobs .

Responsibilities for incident and operations

Change management

Task	IBM Responsibilities	Your responsibilities
Updates, fixes, and new features	IBM Cloud Internet Services provides regular updates and bug fixes, as well as new features following a continuous delivery model in a manner transparent to the customer.	
Updates to edge capabilities	Updates are made regularly as part of continuous delivery. IBM sends notifications for customer impacting changes.	Set preferences to receive email notifications. Monitor the IBM Cloud® status page for general announcements.

Responsibilities for change management

Identity and access management

Task	IBM Responsibilities	Your responsibilities
Service and platform permissions	IBM Cloud Internet Services provides administrators a way to control access to work with an instance, domain, or subcategory.	Grant, revoke, and manage access to service instances, domains, and subcategories by using IAM, the IBM Cloud Internet Services Access page, or the equivalent CLI.
Security monitoring	IBM Cloud Internet Services performs regular code scans and other measures to ensure ongoing security of the service.	

Logging	Logs access control actions within the CIS platform and IAM system.	Monitor access changes to CIS instances using IBM Cloud Activity Tracker Routing or other audit solutions.
Responsibilities for identity and access management		

Security and regulation compliance

Task	IBM Responsibilities	Your Responsibilities
Maintenance of controls	IBM Cloud Internet Services maintains controls pertaining to industry standards for which it is certified.	
Security and vulnerability updates	IBM Cloud Internet Services applies security and vulnerability patches on a regular and timely schedule that is transparent to the user.	
Security of instance and domain configuration		Customer is responsible for setting up and maintaining the security and compliance of their domain within IBM Cloud® Internet Services.
Logging	Logs patch management and platform security operations to maintain compliance.	Customer is responsible for consuming and monitoring logs for their domains from configured Logpush job.
Responsibilities for security and regulation compliance		

Disaster recovery

The data for a domain in IBM Cloud® Internet Services with its properties, settings, and configuration is stored and maintained in data centers across the network of our partner, Cloudflare. IBM does not operate any data centers or maintain any customer data in the data path of the application.

Task	IBM Responsibilities	Your responsibilities
Backup of domain configuration data		The customer is responsible for maintaining backup copies and records pertaining to the configuration of the customer's domain. This includes DNS records, firewall rules, and any custom configuration of the customer's domain.
Recovery and verification of domain configuration	IBM is responsible for restoring the domains and configurations it manages, and for restoring full data path functionality between client and origin.	After disaster recovery, it is the customer's responsibility to verify their domain is functional, and their configuration is as expected. The customer is also responsible for the health of origins that are serving their site content.
Recovery of control path capabilities	IBM is responsible for recovering control path capabilities and making sure API, CLI, and/or UI are available to customers to manage their domains. IBM works with our partner, Cloudflare, to restore capabilities end to end.	
Responsibilities for disaster recovery		

Adhering to IBM Cloud sanctions on embargoed countries

Compliance with IBM Cloud sanctions involves the following restrictions that are related to specific embargoed countries: **Iran, Cuba, North Korea,** and **Syria**. Adhering to these sanctions is crucial for maintaining compliance with international laws and regulations.

Understanding sanctions and embargoes

Sanctions are restrictive measures that are imposed by governments or international organizations to influence the behavior of a country or group. Embargoes are a specific type of sanction that prohibits trade and other commercial activities with certain countries.

IBM Cloud is committed to complying with all applicable laws and regulations, including those laws related to sanctions and embargoes. Users must ensure that their use of IBM Cloud services doesn't involve any activities with the embargoed countries. For more information, see [Sanctions and Embargoes](#) in the IBM Cloud Notices page.

Adding custom rules from the CLI

CIS custom rules help you to define conditions that allow or block traffic. To block the traffic from US-embargoed countries, create the country-blocking custom rule and place it first in the custom rules order. Use the following commands to create the rule and add it as the first custom rule.

```
$ ibmcloud cis custom-waf rule-create DNS_DOMAIN_ID -i INSTANCE --match "(ip.src.country in {\\"CU\\" \\"IR\\" \\"SY\\" \\"KP\\"})" --action block --description "Embargo" --enabled true
ibmcloud cis custom-waf rule-order-update $DNS_DOMAIN_ID -i INSTANCE RULE_ID --index 1
```

Where:

DNS_DOMAIN_ID

The ID of the DNS domain.

INSTANCE

The name or ID of the instance.

RULE_ID

The ID of the custom rule for blocking the embargoed countries.

Adding custom rules with the API

To add a custom rule with the API, follow these steps:

1. First, create a filter for each embargoed country by sending the following request:

```
curl --request POST \
--url https://api.cis.cloud.ibm.com/v1/<CRN>/zones/<ZONE ID>/filters \
--header 'Accept: application/json' \
--header 'Authorization: Bearer <TOKEN>' \
--header 'Content-Type: application/json' \
--data '[
{
  "expression": "(ip.src.country in {\\"IR\\" \\"CU\\" \\"SY\\" \\"KP\\"})",
  "description": "Block Traffic from US Embargoed Countries",
  "paused": false
}
]
```

2. Create a firewall rule to block traffic from these countries

```
curl --request POST \
--url https://api.cis.cloud.ibm.com/v1/<CRN>/zones/<ZONE ID>/firewall/rules \
--header 'Accept: application/json' \
--header 'Authorization: Bearer <TOKEN>' \
--header 'Content-Type: application/json' \
--data '[
{
```

```
"filter": {  
  "id": "<Filter 1 ID>"  
},  
"action": "block",  
"paused": false  
}]'
```


Comparing CIS plans

IBM Cloud® Internet Services offers several plans to choose from: no-cost Trial, Standard Next, and Enterprise Tier variations.

**Note:** The no-cost Trial plan is the same as the Standard Next plan, except that it expires after 30 days. Only one trial is available per account. After the Trial plan expires, it is automatically subject to reclamation.

The no-cost Trial and Standard plans are not Financial Services regulatory compliant. All other plans are fully compliant.

The following table compares each offering to help you choose the best plan for your needs. Enterprise Tier plans are available with different pricing models.

**Tip:** The numbers in these tables are quotas or resource limits for the associated feature. Features are pay-as-you-go, except for Trial and Standard Next plans.

	Standard Next	Enterprise Essentials	Enterprise Advanced	Enterprise Premier	Enterprise Usage
Domain Overage charges apply	1 included More available for monthly cost per domain	1 included More available for monthly cost per domain	2 included More available for monthly cost per domain	2 included More available for monthly cost per domain	Up to 1000, but recommend no more than 20
Included protected traffic (does not include traffic that is related to an attack) Overage charges apply	0.5 TB	5 TB	5 TB	5 TB	Not applicable (usage based)
Included requests and queries Overage charges apply	5 M requests 1 M queries	150 M requests and queries combined	150 M requests and queries combined	150 M requests and queries combined	Not applicable (usage based)
DNS	250 records	3500 records	3500 records	3500 records	3500 records

Global load balancers	3 pools 3 origin servers 5 Health checks 60s health checks - Geo Routing - Health checks from single region 60s minimum TTL for nonproxied global load balancers	20 pools 20 origin servers (more origins available for charge) 100 health checks 5s health checks - Smart Routing - Health checks from multiple regions 10s minimum TTL for nonproxied global load balancers	20 pools 20 origin servers (more origins available for charge) 100 health checks 5s health checks - Smart Routing - Health checks from multiple regions 10s minimum TTL for nonproxied global load balancers	20 pools 20 origin servers (more origins available for charge) 100 health checks 5s health checks - Smart Routing - Health checks from multiple regions 10s minimum TTL for nonproxied global load balancers	- Up to 100 pools 100 origin servers - Up to 100 health checks 5s health checks - Smart Routing - Health checks from multiple regions 10s minimum TTL for nonproxied global load balancers
WAF	OWASP and CIS ruleset	OWASP and CIS ruleset	OWASP and CIS ruleset	OWASP and CIS ruleset	OWASP and CIS ruleset
Custom certificate [1]	1 uploaded custom certificate per domain	25 included custom certificates More certificates can be added at a monthly rate	25 included custom certificates More certificates can be added at a monthly rate	25 included custom certificates More certificates can be added at a monthly rate	1 uploaded custom certificate per domain
Page rules	50 page rules per domain	100 page rules per domain - More settings for fine-grained control	100 page rules per domain - More settings for fine-grained control	100 page rules per domain - More settings for fine-grained control	100 page rules per domain - More settings for fine-grained control
IP firewall	All IP firewall rule actions - Block is not available to Country filter 10 Domain lockdown rules 250 User agent rules	All IP firewall rule actions 200 Domain lockdown rules 1000 User agent rules	All IP firewall rule actions 200 Domain lockdown rules 1000 User agent rules	All IP firewall rule actions 200 Domain lockdown rules 1000 User agent rules	All IP firewall rule actions 200 Domain lockdown rules 1000 User agent rules
Caching	Basic caching options	Browser cache minimum TTL 30 seconds	Browser cache minimum TTL 30 seconds	Browser cache minimum TTL 30 seconds	Advanced caching options

Range	No	No	Yes 10 unique FQDNs - Overage fees incurred past 5 TB of traffic	Yes 10 unique FQDNs - Overage fees incurred past 5 TB of traffic	Yes 10 unique FQDNs
Edge functions	1 action (must be named after domain) 1 M Edge function requests	Unlimited actions	Unlimited actions	Unlimited actions	Unlimited actions
Custom rules	100 active rules - Does not support Log action - Does not support matches operator	1000 active rules - Supports all actions - Supports all operators	1000 active rules - Supports all actions - Supports all operators	1000 active rules - Supports all actions - Supports all operators - Supports bot management	1000 active rules - Supports all actions - Supports all operators
Lists	Managed Lists Not available Custom Lists 10 lists 10000 items (across all lists) - List types: IP	Managed Lists - Open Proxies 1000 lists 500000 items (across all lists) - List types: IP, ASN, Hostname	Managed Lists - Open Proxies - Anonymizers - VPNs - Malware - Botnets, Command and Control Servers Custom Lists 1000 lists 500000 items (across all lists) - List types: IP, ASN, Hostname	Managed Lists - Open Proxies - Anonymizers - VPNs - Malware - Botnets, Command and Control Servers Custom Lists 1000 lists 500000 items (across all lists) - List types: IP, ASN, Hostname	Managed Lists - Open Proxies 1000 lists 500000 items (across all lists) - List types: IP, ASN, Hostname
GraphQL	No	Yes	Yes	Yes	Yes
Smart routing	No	Yes	Yes	Yes	Yes
Logging	No	Yes	Yes	Yes	Yes
Alerts	No	Yes	Yes	Yes	Yes
Rate limiting	No	No	Advanced	Advanced	Default
Bot management	No	No	No	Yes	No

Additional details

- Traffic is protected when it is traffic through proxied CIS components, metered as traffic from the edge to the origin
- HTTP request made on proxied resources (for example, DNS records and load balancers) count toward your protected traffic
- When traffic is not proxied, it will only consume DNS queries during hostname resolution

Deprecated plans

The following plans are scheduled for deprecation or deprecated.

- The Standard plan reached the end of marketing on 30 April 2023. The end of support is not yet determined.
- Enterprise Package, Enterprise GLB, and Enterprise Security plans will reach the end of marketing on 31 August 2023. The end of support is not yet determined.

For more information about changing to a new plan if you are currently on a deprecated plan, see [Transitioning to updated plans](#).

1. One legacy custom certificate without Server Name Indication (non-SNI) is supported across all plans. [↩](#)

Transitioning to updated plans

Although the transition to updated IBM Cloud® Internet Services plans began in 2023, some users are still on the original plans. This topic remains available to assist users who have not yet migrated.

**Note:** Migrating from one plan to another is considered a modification of an existing resource, not a deletion or creation of a resource.

Transitioning from a Standard plan to a Standard Next plan

As of 30 April 2023, the original CIS Standard plan is no longer available for new instances. Any instance created on or after this date automatically uses the Standard Next plan.

The Standard Next plan follows a more metered pricing model, which reduces the base cost compared to the original Standard plan. The following table outlines the key differences between the two plans.

Feature	Standard	Standard Next	Overage
Domain	1 included More available for monthly cost per domain	1 included More available for monthly cost per domain	Each additional domain incurs a fee
Protected traffic (TB)	5	0.5	Traffic over the limit (per TB) incurs a fee
Range traffic	No	No	Not applicable
DNS queries (million per domain)	10	1	Fees assessed per additional million queries
HTTP requests (million)	10	5	Fees assessed per additional million requests
Edge function requests (million)	1	1	Fees assessed per additional million queries
DDoS protection	Yes	Yes	Not applicable
WAF (million)	Not applicable	Yes	Part of HTTP requests
IP Firewall	Yes	Yes	Not applicable
DNS records per domain	3500	250	Upgrade plan to get more
GLB origin servers	6	3	Not applicable
Smart routing	No	No	Not applicable
Rate limiting	No	No	Not applicable
Logging	No	No	Not applicable
Page rules per domain	50	50	Not applicable
Firewall rules	100	100	Not applicable

Comparison of CIS Standard and Standard Next plans

Migrating to the Standard Next plan

To migrate from the Standard plan to the Standard Next plan:

- 1. Navigate to your existing CIS instance.
- 2. From the **Plan** page, select the Standard Next plan.
- 3. Accept the terms and conditions.
- 4. Click **Create** and follow any prompts.

Transitioning to Enterprise Tier plans

As of 31 August 2023, the original CIS Enterprise Package, Enterprise GLB, and Enterprise Security plans are no longer available for new instances. All new instances created on or after this date use one of the new Enterprise Tier plans.

The Enterprise Tier plans adopt a more flexible, metered pricing model, lowering base costs compared to legacy Enterprise plans. The following table summarizes key features of the new plans.

Features	Enterprise Essential	Enterprise Advanced	Enterprise Premier
Included domains	1 included More available for monthly cost per domain	2 included More available for monthly cost per domain	2 included More available for monthly cost per domain
Included DNS records	3500	3500	3500
Protected traffic included (DDoS, CDN, DNS, WAF, GLB)	5 TB	5 TB	5 TB
Requests included (CDN, WAF, GLB) in millions	150	150	150
Custom uploaded certificates	25	25	25
GLB: Origins, pools, GLB included	20	20	20
Range (Layer 3/4)	No	Yes	Yes
Logging	Yes	Yes	Yes
Rate limiting	No	Yes	Yes
Advanced WAF	No	Yes	Yes
Bot management	No	No	Yes

Comparison of CIS Enterprise Tier plans

Migrating to Enterprise Tier plans

To migrate from your current Enterprise package, Enterprise GLB, or Enterprise Security plans:

- 1. Navigate to your existing CIS instance.
- 2. From the **Plan** page, select the Enterprise Tier plan that you want.
- 3. Accept the terms and conditions.
- 4. Click **Create** and follow any prompts.

All plans comparison

For a detailed comparison of current plans, see [Comparing CIS plans](#).

CIS allowlisted IP addresses

The following API lists all IP addresses used by the CIS proxy. The CIS proxy uses only addresses from this list, for both client-to-proxy and proxy-to-origin communication.

<https://api.cis.cloud.ibm.com/v1/ips>

Polling this API one time a week is sufficient to get the information you need to update your allowlists.



Note: The IP addresses the CIS proxy uses for communication with the origins are not necessarily the same as the ones used for client-to-proxy communication, although all addresses are derived from the same list.

Network port support for the CIS proxy

Network ports control how the CIS proxy handles traffic for your domain and which services can be accessed.

Supported ports for proxied traffic

By default, the CIS proxy only handles traffic on specific HTTP and HTTPS ports.

Supported HTTP ports

CIS supports the following HTTP ports:

- 80
- 8080
- 8880
- 2052
- 2082
- 2086
- 2095

Supported HTTPS ports

CIS supports the following HTTPS ports:

- 443
- 2053
- 2083
- 2087
- 2096
- 8443

Traffic on unsupported ports

The CIS proxy does not handle traffic on ports outside the supported lists. If your application uses a different port (for example, SSH on port 22), you must configure access by [creating a Range application](#) for the hostname. Range applications support traffic on all ports; however, full TCP and UDP port support is available only on the Enterprise plan.

Block traffic on other ports

To block traffic on ports other than 80 and 443, use WAF rules. To restrict traffic on nonstandard ports, choose one of the following options:

- If you are using the previous version of WAF managed rules, enable `rule ID 100015 (Anomaly: Port - Non-standard port (not 80 or 443))`.
- If you are using the current WAF, enable `rule ID 664ed6fe (Anomaly: Port - Non-standard port (not 80 or 443))`. This rule is part of the CIS Managed Ruleset and is disabled by default.

The WAF [CIS Managed Ruleset](#) includes a rule that blocks the traffic at the application layer (layer 7), preventing HTTP/HTTPS requests over nonstandard ports from reaching the origin server.

Due to the anycast network, some ports might appear open because they are used by shared services. This behavior is expected and does not mean that your application is publicly accessible.

Control traffic by port using custom rules

CIS allows you to control how traffic is handled on specific ports by using web application firewall (WAF) custom rules.

CIS evaluates incoming requests at the edge and exposes request attributes as fields that can be used in rule expressions. The `cf.edge.server_port` field represents the destination port on which the request is received at the CIS edge.

By using this field in custom rule expressions, you can define policies to allow, block, or challenge traffic based on the port.

Restrict traffic by port

To restrict traffic on specific ports, create a custom rule that matches the `cf.edge.server_port` field. For example, to allow traffic only on standard HTTP and HTTPS ports (`80` and `443`), create a rule that blocks all other ports:

```
cf.edge.server_port ne 80 and cf.edge.server_port ne 443
```

Match a specific port

To apply rules to a specific port, match the `cf.edge.server_port` field with the required value:

```
cf.edge.server_port eq 8443
```

Custom rules are evaluated at the application layer (layer 7) and can be used to enforce port based access controls before requests reach the origin server.

For more information about available fields and expression syntax, see [Custom rules fields and expressions](#).

Helpful tools for managing your CIS deployment

Some public-domain Unix system administration tools can help you manage your IBM Cloud® Internet Services (CIS) deployment.

Sysadmin tools

- `whois` (domain identification tool)
- `dig` (DNS tool)
- `cURL` (HTTP and HTTPS tool)
- `netcat` (IP and port tool)
- `traceroute` (network tool)

Commercial tools for external and remote testing

- GTMetrix (HTTP)
- Web page test (HTTP)
- WhatsMyDNS (DNS tool)
- G Suite Toolbox (DNS and HTTP)

Tools for looking at logs and history

HTTP Archive files (HAR files)

Using `whois`

`whois` is a UNIX system command line tool that you can use to look up registrar information for a given domain name or IP address. For example, the domain's given authoritative servers or the owner of a particular IP address.

Examples:

```
whois example.com
```

```
whois 8.8.8.8
```

Using `dig`

`dig` is a Unix command line tool that can perform DNS queries and check DNS records for a specific domain. It is similar to `nslookup`.

The schema of this command is `dig <record_type> <domainname> <options>`

For example:

- `dig example.com`
- `dig my.example.com`
- `dig example.com +trace`
- `dig NS example.com`
- `dig example.com @ns.example.com`

Using `cURL`

`cURL` is a Unix command line tool that lets you transmit data by using the URL syntax. It's commonly used to make HTTP requests or compare server responses.

The schema for this command is: `curl -option1 -option2 http://example.com/url`

For example:

- `curl -svo /dev/null http://www.example.com`
- `curl -svo /dev/null -A "USER_AGENT_STRING" http://www.example.com`
- `curl -svo /dev/null -H "host: www.example.com" http://ORIGIN_IP`
- `curl -svo /dev/null -H https://www.example.com --resolve www.example.com:443:ORIGIN_IP`

Using `mtr` and `traceroute`

`mtr` and `tracert` are Unix command line tools that let you measure performance or latency along a specific network path to a specified host or destination server.

For example:

- `mtr -rwc 20 example.com -T -4`
- `mtr -rwc 20 8.8.8.8 -T -6`
- `tracert example.com -T -4`
- `tracert 8.8.8.8 -T -6`

Option	Definition
-c	Sets the number of pings sent
-T	Forces a TCP traceroute (normally ICMP)
-4	Forces the use of IPv4
-6	Forces the use of IPv6

Command options

Generating an HAR file

An HAR file is a recording of HTTP requests from a web browser. Browsers, such as Chrome, have a Developer Tools section that can help you get set up to make HAR files. For more information on how to create a HAR file, see [How do I generate a HAR file?](#) in the troubleshooting section.

Looking at logs and history by using My Traceroute

MTR (My Traceroute) is a network diagnostic tool that combines the functions of `tracert` and `ping` into a single program. Where `tracert` shows the path that packets take to reach a destination, and `ping` measures round-trip time to a single host, MTR does both simultaneously and continuously. MTR probes each hop on the network path repeatedly, collecting statistics such as packet loss percentage, average latency, and jitter for every intermediate router between your source and destination.

MTR is valuable for diagnosing intermittent network issues that a one-time `tracert` might miss. By running continuously over time, MTR reveals unstable hops, asymmetric packet loss, and routing anomalies that are otherwise difficult to detect.

MTR output fields

The following table describes the columns in MTR output.

Field	Description
Host	The hostname or IP address of the hop.
Loss%	The percentage of packets lost at this hop.
Snt	The total number of packets sent to this hop.
Last	The round-trip time (in milliseconds) of the most recent packet.
Avg	The average round-trip time across all packets sent.
Best	The best (lowest) round-trip time recorded.
Wrst	The worst (highest) round-trip time recorded.
StDev	The standard deviation of round-trip times, indicating jitter.

MTR output fields

**Important:** Packet loss that is shown at an intermediate hop does not always indicate a problem at that hop. Many routers deprioritize or rate-limit ICMP traffic that is used by MTR while still forwarding data traffic normally. Investigate a hop as a potential problem only if loss persists at that hop and all subsequent hops.

Generating an MTR report on Linux

MTR is available on most Linux distributions and can be installed from the default package manager.

Prerequisites

- Root or sudo access on the Linux host.
- MTR installed. If MTR is not installed, install it using the package manager for your distribution:
 - RHEL or CentOS or Fedora:

```
$ sudo yum install mtr
```

- Debian or Ubuntu:

```
$ sudo apt-get install mtr
```

Running MTR and traceroute

You can use either MTR or traceroute to identify the network path between your system and a destination. While traceroute performs a one-time trace of the route, MTR continuously probes each network hop and collects latency and packet loss statistics. MTR is recommended when you are troubleshooting intermittent connectivity or performance issues because it provides more comprehensive diagnostic information.

The following examples show how to run MTR and traceroute using TCP probes over IPv4 and IPv6.

- `mtr -rwc 20 example.com -T -4`
- `mtr -rwc 20 8.8.8.8 -T -6`
- `traceroute example.com -T -4`
- `traceroute 8.8.8.8 -T -6`

Option	Definition
-c	Sets the number of pings sent
-T	Forces a TCP traceroute (normally ICMP)
-4	Forces the use of IPv4
-6	Forces the use of IPv6

Command options

Running MTR in report mode

Use report mode (`--report`) to collect a fixed number of samples and print a summary. This is the recommended format for sharing MTR results with support teams.

```
$ mtr --report --report-cycles 60 <destination>
```

Replace `<destination>` with the hostname or IP address that you want to trace. For example, to diagnose connectivity to a CIS-proxied domain:

```
$ mtr --report --report-cycles 60 example.com
```

The `--report-cycles 60` flag sends 60 packets to each hop before you print the report. Increasing the cycle count provides more statistically reliable results for intermittent issues.

Saving MTR output to a file

To save the report to a file for sharing with support:

```
$ mtr --report --report-cycles 60 example.com > mtr-report.txt
```

Generating an MTR report on Windows (WinMTR)

WinMTR is the Windows equivalent of the MTR tool. It provides a graphical user interface (GUI) and can also export results as text or HTML files.

Prerequisites

- Administrator rights are recommended for accurate results.
- WinMTR downloaded from the [WinMTR project page](#).

Running WinMTR

1. Extract the WinMTR compressed file archive and run `WinMTR.exe`. No installation is required.
2. In the **Host** field, enter the hostname or IP address of your destination. For example, enter `example.com` to trace the path to a CIS-proxied domain.
3. Click **Start** to begin collecting data. Allow WinMTR to run for at least **5 minutes** (or longer for intermittent issues) to accumulate enough samples for meaningful statistics.
4. Click **Stop** when you collect sufficient data.

Exporting WinMTR results

WinMTR can export results in two formats:

- **Export TEXT**: Saves a plain-text summary that is suitable for pasting into a support ticket.
- **Export HTML**: Saves an HTML file with the full hop-by-hop table.

To export your results, click **Export TEXT** or **Export HTML** in the WinMTR toolbar, then choose a save location.

WinMTR output fields

WinMTR displays the same diagnostic statistics as the Linux MTR tool. For field descriptions, see [MTR output fields](#).



Tip: When you open a support case with IBM Cloud CIS, always include both a forward MTR (from your host to the CIS endpoint) and a reverse MTR (from a host near the CIS endpoint back to your IP, if possible). Asymmetric routing means that the outbound and return paths can differ, and both reports help support teams pinpoint the problem.

Interpreting MTR results with CIS

When you experience connectivity issues with your CIS-proxied domain, use the following guidelines to interpret your MTR output:

- **Loss only at one intermediate hop, not at subsequent hops** : The router at that hop rate-limits ICMP. This is not a network problem.
- **Loss starting at a specific hop and continuing through all subsequent hops** : The problem is at or before that hop. Share the MTR report with IBM Cloud Support.
- **High latency at the first hop** : Check your local network and default gateway.
- **High latency at a CIS anycast IP hop** : Open an IBM Cloud Support case and include your MTR report. You can identify CIS anycast IPs by looking for Cloudflare or IBM Cloud network ASN entries in the hop list.

HIPAA advisory



Use of caching (CDN) for regulated data (for example, PHI, ITAR) is **prohibited**. All regulated data flows that use IBM Cloud® Internet Services must be set to **not cache**.

There are multiple ways to disable content caching by the CIS CDN.

- The origin server can set **no-cache** in the **Cache-Control** header for the regulated content
- Use the CIS page rules to disable caching for any content on a specified path, even if the origin does not send a **no-cache Cache-Control** header.

FAQ for IBM Cloud Internet Services

Have a question about IBM Cloud® Internet Services? Review these frequently asked questions, which provide answers to provisioning concerns, application access, and other common inquiries.

If you have additional questions you'd like to see addressed here, open an issue by using the **Open doc issue** or **Edit topic** links at the end of this page.

Plans, trials, and billing

What is included in the Free Trial plan?

The Free Trial plan allows one zone per account. You should create only one instance and verify the zone name before adding it. The zone name must be verified before it is accepted. If you delete a zone during the Free Trial period, you cannot add the same zone or a different zone again under the Free Trial plan.

How many Free Trial instances are allowed per account?

Only one Free Trial instance is allowed per account for the lifetime of the account. If you create a Free Trial instance, whether you delete it or allow it to expire, you cannot create another Free Trial instance. However, you can create instances under paid plans at any time.

Can Standard Next be downgraded to the Free Trial plan?

No. Downgrading from Standard Next plan to a Free Trial plan is not supported. You can't downgrade any plan to the Free Trial plan.

What happens when a Free Trial plan expires?

To avoid data loss, upgrade the instance to a paid plan before the expiration date. After expiration, you can either upgrade the plan or delete the instance. If the instance is not upgraded or deleted within 45 days of creation, CIS automatically deletes:

- The configuration domain
- Global load balancers
- Origin pools
- Health checks

What happened to the Enterprise Package plan?

As of 11 August 2023, the Enterprise Package plan was discontinued. The functionality of this plan was split across various tiers and is now available in Enterprise Essential, Enterprise Advanced, and Enterprise Premier plans. For more information, see [Transition updated plans](#).

Is DDoS attack traffic billed?

No. CIS provides unlimited and unmetered DDoS protection. Traffic identified as part of a DDoS attack is excluded from billing. There are no limits on the size, duration, or number of identified attacks.

How does CIS protect against unexpected billing charges?

CIS does not meter or bill for traffic that is blocked by DDoS mitigation, firewall, or rate limiting. Only traffic that passes through CIS and reaches the origin is counted for usage or billing.

CIS also helps keep egress bandwidth charges from your origin under control by only passing along good requests that the origin needs to respond to. All CIS plans offer unlimited and unmetered mitigation of DDoS attacks. You are never charged for attack traffic, and there's no penalty or chargeback for traffic spikes caused by attacks.

Account and access management

Why does a user receive authentication errors after being granted access?

Authentication errors usually occur because the user was not assigned the required service access roles. CIS uses two types of roles:

- Platform access: Allow users to create and manage service instances.
- Service access: Allow users to perform service-specific operations within an instance.

Both role types must be assigned as appropriate for the user’s responsibilities. To update roles in the console, go to **Manage > Security > Identity and Access**.

How do I find the service instance ID?

To find your service instance ID, copy the CRN on the Overview page. For example:

```
$ crn:v1:test:public:internet-svcs:global:a/2c38d9a9913332006a27665dab3d26e8:836f33a5-d3e1-4bc6-876a-982a8668b1bb::
```

The last part of the CRN is your service instance: `836f33a5-d3e1-4bc6-876a-982a8668b1bb`.

Alternatively, you can click the row containing the CIS instance on the resource list main page and copy the GUID for the service instance ID.

Domain onboarding and DNS setup

Why is my domain in Pending state, and how do I activate it?

A domain remains in **Pending** status until the required name server (NS) records are correctly configured.

When you add a domain (or subdomain) to CIS, you are provided with two CIS name servers. You must configure both name servers in one of the following locations:

- At your domain registrar (when adding a domain)
- At your existing DNS provider (when adding a subdomain)

CIS periodically checks public DNS for the required NS records. After the name server change is detected (which can take up to 24 hours), the domain status changes to **Active**. You can manually trigger a check by selecting **Recheck name servers** on the Overview page.

How do I identify my domain registrar?

You can look up your domain registrar using the ICANN WHOIS tool: <https://lookup.icann.org/>



Note: To add your domain to CIS, you must have administrator privilege to edit the domain's configuration at the registrar to update or add the name servers for your domain. If you don't know who the registrar is for the domain you're trying to add to CIS, it is unlikely you have the administrator privilege. Work with the owner of the domain in your organization to make the necessary changes.

Can I delegate a subdomain to CIS while keeping my current DNS provider?

Yes. You can delegate a subdomain to CIS without changing the authoritative DNS provider for the parent domain.

When you add the subdomain to CIS, you receive two CIS name servers. Create NS records for the subdomain at your existing DNS provider that point to those CIS name servers. After the NS records are publicly visible and verified, CIS activates the subdomain. If you do not manage the parent domain, work with the domain owner to add the required NS records.

Can I onboard a domain to CIS without changing the authoritative DNS provider?

Yes. CIS supports a [CNAME \(partial\)](#) setup. This configuration allows you to proxy specific hostnames through the CIS network while keeping your existing authoritative DNS provider. In a partial setup:

- You create CNAME records at your authoritative DNS provider.
- Those CNAME records point to CIS.
- Only the specified hostnames are proxied.



Note: DNS resolution behavior differs from a full name server setup.

What is the default DNS TTL value?

For A and CNAME records, the default automatic TTL is 300 seconds.

Can I configure a CNAME at the root of my domain?

Yes. CIS supports "CNAME Flattening," which allows you to configure a CNAME at the root (apex) of your domain. Instead of returning the CNAME record itself, CIS resolves the CNAME target and returns the corresponding A or AAAA records. This allows the root domain to behave like a CNAME

without violating DNS standards.

What is a proxied record, and when should I use one?

A proxied DNS record routes traffic through CIS before it reaches your origin server. Only proxied records receive CIS benefits, including IP masking, where a CIS IP is substituted for your origin IP to protect it:

```
$ $ whois 104.28.22.57 | grep OrgName
OrgName:      IBM
```

If you would rather bypass CIS on a domain (we still resolve DNS), then non-proxying the record is a possible solution.

How do I resolve DNS validation error 1004?

For page rules to work, DNS needs to resolve for your zone. As a result, you must have a proxied DNS record for your zone.

Can I use CIS with private (RFC1918) IP addresses?

Yes, but with limitations. If you configure a non-proxyed DNS record that points to a private (RFC1918) IP address:

- CIS performs DNS resolution only.
- Advanced features, such as CDN, WAF, and DDoS protection are not applied.

CIS does not provide connectivity to private networks. Network access to the private IP must be handled by your infrastructure.

SSL/TLS and certificates

Why do I see a browser privacy warning?

The TLS certificates issued by IBM Cloud CIS cover the root domain (`example.com`) and one level of subdomain (`*.example.com`). If you’re trying to reach a second-level subdomain (`**.example.com`), a privacy warning appears in your browser, because these host names are not added to the SAN.

Allow up to 15 minutes for one of our partner Certificate Authorities (CAs) to issue a new certificate. A privacy warning appears in your browser if your new certificate has not yet been issued.

How do I resolve error 526: Invalid SSL certificate?

Error 526 indicates that the origin server is presenting an invalid or untrusted SSL/TLS certificate.

When the CIS proxy is enabled and the SSL mode is **End-to-end CA Signed** (default for new domains), the origin must present a valid certificate that is signed by a trusted certificate authority.

To resolve the error:

- Ensure the origin certificate is valid (not expired or self-signed).
- Verify that the certificate matches the hostname.
- Confirm the certificate chain is complete.
- Install a valid CA-signed certificate on the origin server.

If necessary, you can change the SSL mode to a less strict setting. However, this is not recommended for production environments because it reduces security.

How does CIS mitigate SSL/TLS negotiation and handshake attacks?

CIS mitigates SSL/TLS negotiation and handshake-based attacks by terminating TLS sessions at the edge network before traffic reaches the origin server. It enforces secure TLS settings and cipher suites to protect against known vulnerabilities such as BEAST, POODLE, and CRIME. CIS forwards traffic to the origin only after a successful TLS handshake, preventing TLS exhaustion attacks. Automated DDoS systems then analyze traffic patterns, cipher behavior, and request metadata to detect and block additional SSL/TLS based attacks.

DDoS protection

What is a distributed denial-of-service (DDoS) attack?

A distributed denial-of-service (DDoS) attack attempts to make an online service unavailable by overwhelming it with traffic from multiple sources.

Attackers use compromised systems to generate large volumes of traffic or malformed requests that:

- Exhaust server resources
- Disrupt network connectivity
- Prevent legitimate users from accessing the service

DDoS attacks can target applications (Layer 7), protocols (Layer 3/4), or network infrastructure.

What are LOIC and HOIC attack tools?

The Low Orbit Ion Cannon (LOIC) and High Orbit Ion Cannon (HOIC) are tools associated with DoS attacks and often referenced in discussions of Layer 7 DDoS mitigation.

LOIC generates large volumes of TCP, UDP, or HTTP requests to overwhelm a target. It is known for:

- Simple interface and low barrier to use
- High volumes of repetitive traffic
- Overwhelming smaller or poorly protected systems

HOIC is an evolution of LOIC, generating higher-volume, more flexible HTTP traffic, targeting multiple sites simultaneously, and supporting “boosters” to extend attack scope. This makes HOIC attacks harder for signature-based defenses to detect.

Both tools exploit network behaviors to cause denial of service. Use outside controlled testing environments is illegal and unethical. Organizations should rely on defenses such as rate limiting and managed DDoS protection.

What should I do if I am under a DDoS attack?

If you suspect an active DDoS attack:

1. Enable “Defense mode” from the **Overview** page.
2. Set your DNS records for maximum security.
3. Do not rate-limit or throttle requests from CIS, IBM needs the bandwidth to assist you with your situation.
4. Block specific countries and visitors, if necessary.

These actions allow CIS to inspect, absorb, and mitigate attack traffic at the edge network.

How does CIS protect against low-and-slow DDoS attacks?

CIS protects against low-and-slow attacks by acting as an HTTP reverse proxy in front of the origin. The proxy buffers and validates requests at the edge, waiting for a complete HTTP request before forwarding it. Slow, incomplete, or malformed requests are absorbed or dropped, and never reach the origin.

CIS also enforces timeouts and applies WAF and firewall checks without requiring a traffic threshold, preventing attacks, such as Slowloris and RUDY from exhausting server resources.

Can I exclude specific user agents from HTTP DDoS mitigation?

Yes, you can create a custom rule [override](#) and use the expression fields to match against HTTP requests with the `User-Agent` header. There are a variety of [fields](#) that you can use.

You can then adjust the [sensitivity level](#) or [mitigation action](#).

Use this capability carefully to avoid weakening protection for legitimate traffic patterns.

How does CIS handle traffic scrubbing?

CIS uses its 388 Tbps global edge network to mitigate high-volume DDoS attacks without scrubbing centers. Attacks are analyzed and blocked at the edge, near the source. Only protected traffic (clean requests and responses) is billed. Malicious traffic is excluded.

Load balancing and health checks

What is the default health check timeout value?

The default health check timeout for the Free Trial and Standard plans is 60 seconds.

Can health checks be configured for non-HTTP/HTTPS traffic?

No, health checks only support HTTP/HTTPS.

Can global load balancers be configured for non-HTTP/HTTPS protocols?

No. Global load balancers only support HTTP/HTTPS.

What happens if all origins in a pool are disabled?

Yes. If all origins in a pool are disabled, traffic is routed to the next-priority pool or fallback pool.

Which network does CIS use for global traffic and health checks?

CIS runs its data plane on Cloudflare’s global Anycast network, which spans hundreds of cities worldwide. This network ensures fast, reliable traffic routing and DDoS mitigation.

Health check requests originate from this distributed network, so the available regions for health checks are based on the [Cloudflare Global Anycast Network](#).

Performance and optimization

Does CIS apply content compression (gzip or Brotli)?

Yes. CIS applies gzip and Brotli compression to some content types and can compress items based on the browser’s User-Agent to improve page load times.

If your origin already uses gzip, CIS honors those settings when the web server includes them in headers.

CIS only supports gzip for origin content and delivers content as gzip, Brotli, or uncompressed. Its reverse proxy can convert between compressed and uncompressed formats independently of caching.

The `Accept-Encoding` header from the client is removed and not respected.

What is the payload limit for WAF?

CIS Web Application Firewall (WAF) now inspects request payloads up to 1 MB for all plans. This allows the WAF to detect more complex threats that may appear in larger request bodies. For more information, see [Request body inspection limit](#).

What is the API rate limit for CIS?

The global rate limit for the CIS API is 1200 requests per five minutes per user, across all interfaces (UI, CLI, Terraform, API).

Networking and infrastructure

What port range is used for edge-to-origin traffic?

When the proxy is enabled, traffic is routed through the Cloudflare network to the origin server. This traffic can originate from any port within the range of `1024-65535`. When configuring Network Access Control Lists (NACLs) for your environment, ensure that both ingress and egress traffic from this port range is allowed.

How does CIS manage clock synchronization (NTP)?

To meet ISO 27001 requirements, all relevant systems must synchronize with a unified time source. IBM CIS ensures clock synchronization across its infrastructure by using Network Time Protocol (NTP) servers. CIS uses the following internal NTP servers:

- `time.adn.networklayer.com`
- `time.service.networklayer.com`

Does CIS support outbound (egress) traffic filtering?

No. CIS secures inbound traffic only. It doesn't inspect, log, or filter outbound traffic from cloud resources, such as virtual server instances, containers, or VPC resources.



Note: CIS acts as a reverse proxy to protect traffic coming into your applications. CIS doesn't function as a forward proxy or egress filter.

For outbound traffic control, consider:

- [VPC Security Groups](#): Control outbound ports/IPs at the instance level.
- [VPC Network ACLs \(NACLs\)](#): Subnet-level inbound/outbound rules.
- Firewall appliances: Deploy third-party firewalls within your VPC.
- DNS filtering: Use DNS-based services to restrict domains.

What is the CF-Connecting-IP header?

The `CF-Connecting-IP` header provides the original client IP address to the origin web server. CIS adds this header at the edge, and it is included only in requests that are forwarded from the CIS edge to the origin server.

Troubleshooting and error codes

How do I resolve error 522 (connection timed out)?

A 522 error occurs when CIS cannot connect to your origin server. After ~15 seconds of failed connection, the request times out and the 522 page is displayed.

This usually happens if a firewall or security software blocks CIS IPs. Because CIS is a reverse proxy, connections appear to come from CIS IP ranges, which must be allowlisted. See the [CIS allowlisted IP addresses](#) page.

Also check that your server and network are healthy and not overloaded.

If issues persist, contact IBM CIS support with a recent Ray ID and confirm that:

- All CIS IP ranges are allowlisted
- Your server/network is online and healthy

How do I resolve a 502 error when saving an Edge Functions action?

Contact [IBM Support](#) and provide the script that you were attempting to save.

How do I resolve a Kubernetes Ingress hostname validation error?

The hostname in a Kubernetes ingress must consist of lower case alphanumeric characters, `-` or `.`, and must start and end with an alphanumeric character. Using `_` in the load balancer name, though permitted, can cause an ingress error in Kubernetes clusters. Avoid the use of `-` in load balancer names to avoid issues with Kubernetes clusters.

Known issues and limitations

The following information describes some limitations when working with IBM Cloud® Internet Services (CIS), as well as some suggested courses of action to improve your experience.

General

- Use Chrome for best compatibility.
- The free trial plan is limited to one instance per account. After you create a resource instance and add a domain to it, you are not allowed to add new resource instances for CIS. This restriction is enforced even if you delete a trial domain and then attempt to add a domain again to the same resource instance. You'll encounter an error if you attempt to do so.
- For this service, subdomain delegation is supported only by using NS records from another provider. CNAME delegation is not supported.
- A, AAAA, and CNAME wildcard records ("*") can't be proxied.
- IP rules created with two letter country codes can only be made with the `Challenge` action. If you want to block visitors from a country, upgrade to the Enterprise plan or place rules on your server to fully block.

Certificates

- Universal CA: CIS can change the CA of Universal certificates without prior notice, and will not notify you of these changes. If you prefer to select your own issuing certificate authority, order an advanced certificate.
- Certificate pinning: Certificate pinning is not supported natively in CIS. If you want to use certificate pinning, you must use custom certificates.
- Full setup: Universal SSL certificates only support SSL for the root or first-level subdomains such as `example.com` and `www.example.com`. To enable SSL support on second, third, and fourth-level subdomains such as `dev.www.example.com` or `app3.dev.www.example.com` use either an advanced certificate or a custom certificate.
- CNAME (partial) setup: On a CNAME setup zone, each subdomain has its own Universal SSL certificate and does not require extra features or purchases.
- Range: Universal SSL is not compatible with an CIS Range application. If you are trying to use Range, use either an advanced certificate or a custom certificate.
- When you delete a dedicated certificate, it might reappear in the list for a short time before the deletion is complete.
- To modify your custom dedicated certificate's hostnames after ordering, you must order a new certificate and then delete the old one.

DNS

- Exporting DNS records includes Cloudflare CNAME records that should be hidden. These records begin with `_` and usually have a second record with the same name but the `_` is removed.

```
$ Ex.  
_cf.generate.yourdomain.com 0 IN CNAME address.alias.com  
cf.generate.yourdomain.com 0 IN CNAME address2.alias.com
```

These records must be removed from the zone file to properly import.

- Exporting CAA DNS records does not work correctly. The `<tag>` and `<value>` are HEX encoded.

 **Note:** CAA `<flags>` `<tag>` `<value>`

Ex.
Original CAA record
caa.yourdomain.com. 1 IN CAA 0 issue "letsencrypt.org"

Exported CAA record
caa.yourdomain.com. 1 IN CAA 0 6973737565 "6c657473656e63727970742e6f7267"

These records must be converted from HEX to string or removed and added manually before importing.

Edge functions

Before changing your CIS instance from an Enterprise plan to a Standard plan, you must remove all edge function actions and triggers first. Any edge function actions and triggers brought from an Enterprise plan to a Standard plan are not editable, which might cause damage to the normal datapath behavior on your domain. After your plan downgrade is complete, you can recreate the edge functions actions and triggers in the new plan.

Global load balancer

- Cloud Internet Services allows you to use the character `_` in load balancer hostnames. However, Kubernetes clusters can't use `_`.
- The Standard plan permits a maximum of 5 load balancers, pools, and health checks. Each pool can have a total of 6 origins, but only 6 unique origins are permitted throughout each CIS instance.
- Health check events for deleted pools and origins can't be filtered, but they still appear in the table.
- If you filter Health check events by `Pool Health`, `Degraded` pools are included because they technically are healthy, but might contain 1 or more critical origins.
- When adding the request header name for a health check, use `Host`, capitalized. Using a lower-case `host` for a health check fails.

Page rules

- Updating page rules settings using the CIS plugin for IBM Cloud CLI might result in an error if the page rule ID is not included in the JSON string or JSON file for the update. To work around this, submit the update using a complete JSON configuration file for the page rule, including the ID.
- Removing page rule settings using the CIS UI might not remove the setting, even though the UI reports a successful update. To work around this problem, use the CIS plugin for IBM Cloud CLI to remove the setting and include the page rule ID in the JSON update document:

```
$ $> ibmcloud cis page-rule-update <domain-id> <rule-id> -j <file>
```

The JSON file should include the complete page rule configuration, including ID, with the necessary updates.

Protocols

The following limitations apply to supported protocols:

- HTTPS: Currently, HTTPS applications do not support HTTP/3.
- UDP: Currently, CIS doesn't support UDP packet fragmentation. Fragmented UDP packets are dropped at the CIS edge.
- Minecraft: Only Minecraft Java Edition is supported; Minecraft Bedrock Edition isn't supported.

Troubleshooting

Troubleshooting your CIS network connection

Use the following methods to gather information that can help you to troubleshoot your network connection.

Determine if your data is passing through your IBM Cloud Internet Services connection

IBM Cloud® Internet Services (CIS) uses HTTP headers, which it can read, add, or modify. The header lets us trace how a request was routed, using a CF-Ray number. The CF-Ray number can be found by a `curl` command or with a Google Chrome plugin in called "Claire".

To know whether data has passed through CIS, locate the `Ray ID` which is present on every packet.

Unix command line tools

- curl for HTTP: `$ curl -vso /dev/null http://example.com`
- dig for DNS: `$ dig www.example.com`
- traceroute for network: `$ traceroute example.com`

For example, the terminal command `curl -svo /dev/null YOUR_URL_HERE. -L` results in the following:

```
CF-RAY: 1ca349b6c1300da3-SJC
```

Perform a traceroute?

To see whether a route goes through your CIS pathway, you can perform a `dig` in a Terminal window for Mac or Linux or use `nslookup` in the Windows command prompt for Windows.

If the packet has a CF-Ray value, then it has traveled through CIS.

The `traceroute` command shows the entire path that an IP request has taken.

The Support team makes use of these commands to assist you.

Adding CF-Ray headers

The CF-RAY header is added to help trace a request to a website through the network. Use it when working with Support to help troubleshoot any related issues with connectivity. You can reveal this "Ray ID" in your logs by making some edits to configuration files in Apache and nginx.

Apache

```
$ LogFormat "%h %l %u %t \"%r\" %>s %b \"%{Referer}i\" \"%{User-agent}i\" %{CF-Ray}i" cf_custom
```

```
CustomLog log/access_log cf_custom
```

NGINX

```
$ log_format cf_custom '$remote_addr - $remote_user [$time_local] '
    '$request' $status $body_bytes_sent '
    '$http_referer' '$http_user_agent' '
    '$http_cf_ray';
```

```
access_log /var/log/nginx/access.log cf_custom;
```

Troubleshooting 400 class errors

The 4xx class error code responses occur when an issue is at the client end, and is potentially a network issue.

- 4xx codes can be used as a response to any request method.
- The origin server might include an explanation in the response, which should be displayed by the User-Agent, with the exception of a HEAD request.
- Custom rules can return any response code in the range 400-499 in your HTML page, if the site owner has created a rule with Block action and configured a custom response code.

Error 400: Bad request

The client did not send a correct request to the server. This is a client error, such as a malformed request syntax, invalid request, message framing, or deceptive request routing. For example, if the request contains a special character that is not correctly URL encoded (or percent-encoded), the **HTTP 400** error is returned.

If you're getting an HTTP error when using the CIS API, make sure that you are using the correct syntax, the correct parameters, and correct body for your API call.

Error 401: Unauthorized

The request was sent without the proper authentication credentials.

Error 403: Forbidden

If you're seeing a 403 error without CIS branding, this is always returned directly from the origin web server, not CIS, and is generally related to permission rules on your server. The most common reasons for this error are:

- Permission rules you have set on the origin web server (in the Apache .htaccess for example)
- Mod_security rules
- IP deny rules. Ensure that CIS's IP ranges aren't being blocked.

CIS will serve 403 responses if the request violated either a default WAF managed rule or a WAF managed rule enabled for that particular zone.

Resolution If you're seeing a 403 response that contains CIS branding in the response body, this is the HTTP response code returned along with security features:

- WAF custom or managed rules with the challenge or block action
- Security level that is set to Medium by default
- Most 1xxx CIS error codes
- The Browser Integrity Check

Error 404: Not found

The origin server was unable or unwilling to find the resource requested. This usually means the host server couldn't find the resource. To serve a more permanent version of this error, use a **410** error code.

These errors typically occur when someone mistypes a URL on your site when there is a broken link from another page, when a page that previously existed is moved or removed, or there is an error when a search engine indexes your site. For a typical site, these errors account for approximately 3% of the total page views, but they're often untracked by traditional analytics platforms.

Website owners usually implement a custom page to be served when this error is generated.

Resolution CIS does not generate **404** errors for customer websites. CIS only proxies the request from the origin server. When you see a **404** for your site, contact your hosting provider for help.

Error 405: Method not allowed

The origin server is aware of the requested resource, but the request method is not supported.

Resolution The origin server must also provide an Allow header with a list of supported targets for that resource.

Error 406: Not acceptable

The server can't produce a response that matches the list of acceptable values defined in the request's content negotiation headers, and the server is unwilling to supply a default representation.

Resolution Instead of generating this error, you can serve the less-preferred method to the User-Agent.

Error 407: Authentication required

The client did not send the required authentication with the request.

Resolution Retry the request with the necessary authentication.

Error 408: Request timeout

The origin server did not receive the complete request in a reasonable time. This error implies that the server does not want to wait and continue the connection.

Resolution This error is not common because servers typically use the "close" connection option.

Error 409: Conflict

The request did not complete because of a conflict with the current state of the resource. This error typically happens on a `PUT` request where multiple clients are attempting to edit the same resource.

Resolution The server should generate a payload that includes enough information for the client to recognize the source of the conflict. Clients can and should try the request again.

CIS generates and serves a `409` response for an `Error 1001: DNS Resolution Error`.

Error 410: Gone

The resource requested is permanently missing at the origin.

Resolution The server is suggesting the links reference the resource should be removed. The server is not qualified to use this status code instead of a `404` response, or required to have this response for any specific period of time.

Error 411: Length required

The client did not define the Content-Length of the request body in the headers, and this parameter is required to obtain the resource.

Resolution The client can resend the request after adding the header field.

Error 412: Precondition failed

The server denies the request because the resource failed to meet the conditions specified by the client.

For an example of version control, a client is modifying an existing resource and sets the `If-Unmodified-Since` header to match the date that the client downloaded the resource and began edits. If the resource was edited (likely by another client) after this date and before the upload of the edits, this response is generated because the date of the last edit comes after the date set in `If-Unmodified-Since` by the client.

Resolution CIS will serve this response.

Error 413: Payload too large

Refusal from the server to process the request because the payload sent from the client is larger than the server accepts. The server can close the connection.

If this refusal happens only temporarily, then the server should send a `Retry-After` header to specify when the client should try the request again.

The upload limit for the CIS depends on your plan. If you exceed this limit, your API call receives a `413 Request Entity Too Large` error.

	Standard	Enterprise
Availability	Yes	Yes
Max upload size	200 MB	500 MB
Upload limit by plan		

Resolution If you require a larger upload, break up requests into smaller chunks, change your DNS record to DNS-only, or upgrade your plan.

Error 414: URI too long

Refusal from the server that the URI was too long to be processed. For example, if a client is attempting a `GET` request with an unusually long URI after a POST, this can be interpreted as a security risk and a `414` error is generated.

Resolution CIS will generate this response for a URI longer than 32KB.

Error 415: Unsupported media type

Refusal from the server to process the format of the current payload. One way to identify and fix this issue is to look at the `Content-Type` or `Content-Encoding` headers sent in the client's request.

Error 416: Range not satisfiable

The `416` error response code indicates that a server can't serve the requested ranges. For example:

- `HTTP/1.1 416 Range Not Satisfiable`
- `Content-Range: bytes */12777`

Resolution The most common reason for a `416` error is that the file doesn't include such ranges. Browsers usually request the entire file again or abort the operation.

Error 417: Expectation failed

The server failed to meet the requirements specified in the `Expect` header of the client's request.

Error 429: Too many requests

The client sent too many requests in the specified amount of time, according to the server (often known as “rate-limiting”). The server might respond with information allowing the requester to retry the request after a specific period of time.

The default global rate limit for the CIS API is 100 requests per minute per user, and applies cumulatively regardless of whether the request is made through the dashboard, API key, or API token. If you exceed this limit, all API calls for the next five minutes are blocked, receiving an `HTTP 429` response.

Some specific API calls have their own limits and are documented separately, such as Cache Purge APIs, GraphQL APIs, and rulesets APIs.

CIS generates and sends this status code when a request is being rate limited. If visitors to your site are receiving these error codes, you are able to see this in the Rate Limiting Analytics.

Error 451: Unavailable for legal reason

The server is unable to deliver the resource due to legal actions.

Typically search engines and ISPs are affected by this response code, not the origin server. The response should include an explanation in the response body with details of the legal demand.

Error 499: Client close request

Error `499` is an nginx-specific response code to indicate when the connection has been closed by the client while the server is still processing its request, which makes the server unable to send a status code back.

This error is shown in CIS logs and status code analytics for Enterprise customers.



Note: Because the CIS partner Cloudflare is built on nginx, there is a 499 HTTP code in Cloudflare logs and analytics for connections, which are terminated before CIS has finished processing the request. It is expected behavior to see these in your logs when clients close connections.

To provide more context, a TCP connection must be established between CIS and the website's origin server before any higher protocol starts the “conversation”. To establish a connection, TCP uses a three-way handshake:

1. SYN: CIS sends three SYN packets to the origin server.
2. SYN+ACK: In response, the origin server replies with an SYN+ACK.
3. ACK: Finally, CIS sends an ACK back to the origin server.

At this point, both CIS and the origin server have received an acknowledgment of the connection, and communication is established. However, if the origin server does not send a SYN+ACK back to CIS within 15 seconds, CIS retries one more time.

Depending on the timeout value on the client side, you might see three different scenarios with their own status code generated.

- If the client has a shorter timeout (less than 30 seconds), they give up the connection, and CIS logs the `499` error.
- If the client has a higher timeout (more than 30 seconds), after the TCP connection is established, the HTTP transaction continues. In this case, CIS returns a normal status code of `HTTP 200`.
- If the client has a higher timeout and CIS was not able to establish the TCP handshake with the origin server, CIS returns `HTTP 522`.

Troubleshooting 500 class errors

Diagnose and resolve 500 class errors for CIS proxied sites using the following sections.

Before you begin

To assist in troubleshooting errors in the 500 class range, check your origin logs or contact site administrator to gather the following information:

1. The specific 500 class error code and message, and any Ray ID (`cf-ray`) that might be associated
2. The time and time zone in which the error occurred
3. The URL that resulted in the HTTP 500 class error



Tip: Check the logs between CIS and the origin web server, including all load balancers, caches, proxies and firewalls.

Error 500: Internal server error

Error 500 generally indicates an issue with your origin web server. *Error establishing database connection* is a common HTTP 500 error message generated by your origin web server. Check your origin logs for more details to resolve.

Error 500 resolution

If the 500 error contains “cloudflare” or “cloudflare-nginx” in the HTML response body, provide support with the following information:

- Your domain ID and instance CRN
- The time and timezone of the 500 error occurrence
- The output of `www.example.com/cdn-cgi/trace` from the browser where the 500 error was observed (replace `www.example.com` with your actual domain and host name)



Note: If you observe blank or white pages when visiting your website, confirm whether the issue occurs when temporarily pausing CIS and check your origin logs for more details.

Error 502 or 504: bad gateway or gateway timeout

An HTTP 502 or 504 error occurs when CIS is unable to establish contact with your origin web server.

There are two possible causes:

- 502/504 from your origin web server (most common)
- 502/504 from Cloudflare

Error 502 resolution

If the 502/504 error is from your origin web server, CIS returns a HTTP 502 or 504 error when your origin web server responds with a standard HTTP 502 bad gateway or 504 gateway timeout error. Check the origin logs to troubleshoot these common causes at your origin web server:

- Ensure the origin server responds to requests for the hostname and domain within the visitor’s URL that generated the 502 or 504 error
- Investigate excessive server loads, crashes, or network failures
- Identify applications or services that timed out or were blocked

If the 502/504 error comes from Cloudflare, provide these required details to Support:

- Time and timezone the issue occurred
- URL that resulted in the HTTP 502 or 504 response (for example: `https://www.example.com/images/icons/image1.png`)
- Output from browsing to `www.example.com/cdn-cgi/trace` (replace `www.example.com` with the domain and host name that caused the HTTP 502 or 504 error)

Error 503: service temporarily unavailable

HTTP error 503 occurs when your origin web server is overloaded.

Error 503 resolution

There are two possible causes discernible by error message:

- If the error doesn't contain "cloudflare" or "cloudflare-nginx" in the HTML response body, check your origin logs for additional information.
- If the error does contain "cloudflare" or "cloudflare-nginx" in the HTML response body, a connectivity issue occurred in a Cloudflare data center. Provide support with the following information:
 - Your domain ID and instance CRN
 - The time and timezone of the 503 error occurrence
 - The output of `www.example.com/cdn-cgi/trace` from the browser where the 503 error was observed (replace `www.example.com` with your actual domain and host name)

Error 520: web server returns an unknown error

Error 520 occurs when the origin server returns an empty, unknown, or unexpected response to CIS.

Error 520 resolution

A quick workaround while further investigating 520 errors is to either disabling the proxy of the DNS record or temporarily pause CIS.

Check your origin web server error logs for crashes and look for these common causes:

- Origin web server application crashes
- CIS IPs not allowed at your origin
- Headers exceeding 16 KB (typically due to too many cookies)
- An empty response from the origin web server that lacks an HTTP status code or response body
- Missing response headers or origin web server not returning proper HTTP error responses

520 errors are prevalent with certain PHP applications that crash the origin web server.

If HTTP/2 is enabled at your origin web server, make sure that HTTP/2 is correctly configured. A 520 error is returned if the origin web server accepts the HTTP/2 connection but then doesn't respect or support the protocol. The maximum origin HTTP version can be set under the Advanced tab of the Reliability page in the console.

If 520 errors continue to occur, provide the following information to Support:

- Full URL(s) of the resource requested when the error occurred
- Cloudflare `cf-ray` from the 520 error message
- Output from `http://www.example.com/cdn-cgi/trace` (replace `www.example.com` with your hostname and domain where the 520 error occurred)
- Two HAR files:
 - one with CIS enabled on your website
 - one with CIS temporarily disabled

Error 521: web server is down

Error 521 occurs when the origin web server refuses connections from the cloud edge. Security solutions at your origin might block legitimate connections from certain CIS IP addresses.

The two most common causes of 521 errors are:

- Offline origin web server application
- Blocked CIS requests

Error 521 resolution

Check your origin web server error logs to eliminate these common causes:

- Ensure your origin web server is responsive
- Review origin web server error logs to identify web server application crashes or outages.
- Confirm CIS IP addresses are not blocked or rate limited
- Allow all CIS IP ranges in your origin web server's firewall or other security software
- Confirm that — if you have your TLS mode set to **End-to-end flexible**, **End-to-end CA Signed** or **HTTPS only origin pull** — you have installed a CIS Origin Certificate

Error 522: connection timed out

Error 522 occurs when the connection to the origin web server times out. Two different timeouts cause HTTP error 522 depending on when they

occur between CIS and the origin web server:

- Before a connection is established, the origin web server does not return a SYN+ACK to CIS within 15 seconds of CIS sending a SYN.
- After a connection is established, the origin web server doesn't acknowledge (ACK) CIS's resource request within 90 seconds.

Error 522 resolution

Check your origin web server error logs for the following common causes at your origin web server:

- (Most common cause) CIS IP addresses are rate limited or blocked in `.htaccess`, `iptables`, or `firewalls`. Confirm that your origin configuration allows CIS IP addresses
- An overloaded or offline origin web server drops incoming requests
- Keepalives are disabled at the origin web server
- The origin IP address in CIS does not match the IP address currently provisioned to your origin web server
- Packets were dropped at your origin web server

If none of these leads to a resolution, attempt to gather the following information before contacting support:

- An MTR or traceroute from your origin web server to a CIS IP address that most commonly connected to your origin web server before the issue occurred Identify a connecting CIS IP recorded in the origin web server logs
- Any relevant details from the origin logs

Error 523: origin is unreachable

Error 523 occurs when CIS can't contact your origin web server. This occurs when a network device between CIS and the origin web server doesn't have a route to the origin's IP address.

Error 523 resolution

Check your origin web server error logs to eliminate these common causes at your origin web server:

- Confirm the correct origin IP address is listed for A or AAAA records
- Troubleshoot internet routing issues between your origin and CIS, or with the origin itself

If none of the previous tips leads to a resolution, request an MTR or traceroute from your origin web server to a CIS IP address that most commonly connected to your origin web server *before* the issue occurred. Identify a connecting CIS IP from the logs of the origin web server.

Error 524: a timeout occurred

Error 524 indicates that the connection to the origin web server was established successfully, but the origin did not provide an HTTP response before the connection timed out. The default timeout is 100 seconds.



Tip: Enterprise plans can increase the 524 timeout up to 600 seconds.

Error 524 resolution

Check your origin web server error logs to eliminate these common causes at your origin web server:

- A long-running process on the origin web server
- An overloaded origin web server

Logging request response time at your origin web server helps identify the cause of resource slowness.

If you regularly run HTTP requests that take over 100 seconds to complete (for example large data exports), move those processes behind a subdomain that is not proxied.

Error 525: SSL handshake failed

525 errors are often caused by a configuration issue on the origin web server. Error 525 occurs when these two conditions are true:

1. The SSL handshake fails between CIS and the origin web server
2. Your TLS setting is **End-to-end flexible**, **End-to-end CA Signed** or **HTTPS only origin pull**

Error 525 resolution

Check your origin web server error logs to eliminate these common causes at your origin web server:

- No valid SSL certificate installed
- Port 443 (or other custom secure port) is not open
- No SNI support
- The cipher suites accepted by Cloudflare do not match the cipher suites supported by the origin web server

If 525 errors occur intermittently, review the origin web server error logs to determine the cause. Configure Apache to log `mod_ssl` errors. You might be able to get SSL errors in your NginX standard error log, but might need an increased log level.

Error 526: invalid SSL certificate

Error 526 occurs when the following two conditions are true:

1. CIS can't validate the TLS certificate at your origin web server
2. Your TLS setting is **End-to-end CA Signed** or **HTTPS only origin pull**

Error 526 resolution

Set your TLS to **End-to-end flexible** instead of **End-to-end CA Signed** or **HTTPS only origin pull** to see if that fixes the error.

Review the origin web server's TLS certificates and verify that:

1. The certificate is not expired
2. The certificate is not revoked
3. The certificate is signed by a Certificate Authority (not self-signed)
4. The requested or target domain name and hostname are in the certificate's Common Name or Subject Alternative Name
5. Your origin web server accepts connections over port 443
6. Temporarily pause CIS and go to `https://www.sslshopper.com/ssl-checker.html#hostname=www.example.com` (replace `www.example.com` with your hostname and domain) to verify no issues exists with the origin TLS certificate

Error 530

Error 530 is accompanied by an error code in the 1XXX range. See [Troubleshooting 1XXX errors](#) for more information.

Troubleshooting 1000 class errors

1000 class errors will appear in the HTML body of the response. HTTP 409, 530, 403, and 429 errors are the HTTP error codes returned in the HTTP status header of a response.

Error 1000: DNS points to prohibited IP

CIS halted the request for one of the following reasons:

- An A record within your CIS DNS points to a CIS IP address.
- Your CIS DNS A or CNAME record references another reverse proxy (such as an nginx web server that uses the proxy_pass function) that then proxies the request to CIS a second time.
- The request X-Forwarded-For header is longer than 100 characters.
- The request includes two X-Forwarded-For headers.

Resolution

- If an A record within your CIS DNS points to a CIS IP address, update the IP address to your origin web server IP address.
- There is a reverse-proxy at your origin that sends the request back through the CIS proxy. Instead of using a reverse-proxy, contact your hosting provider or site administrator to configure an HTTP redirect at your origin.

Error 1001: DNS resolution error

Common causes for 1001 errors are:

- A web request was sent to a CIS IP address for a non-existent CIS domain.
- An external domain that is not on using CIS has a CNAME record to a domain active on CIS
- The target of the DNS CNAME record does not resolve.

- A CNAME record in your CIS DNS requires resolution via a DNS provider that is currently offline.

Resolution

A non-CIS domain can't CNAME to a CIS domain unless the non-CIS domain is added to a CIS account.

Attempting to directly access DNS records used for CIS CNAME setups also causes error 1001.

Error 1002: DNS points to prohibited IP

Common causes for 1002 errors when a DNS points to a prohibited IP address are:

- A DNS record in your CIS DNS points to one of CIS's IP addresses.
- An incorrect target is specified for a CNAME record in your CIS DNS.
- Your domain is not on CIS but has a CNAME that refers to a CIS domain.

Resolution

Update your CIS A or CNAME record to point to your origin IP address instead of a CIS IP address:

1. Contact your hosting provider to confirm your origin IP address or CNAME record target.
2. Log in to your CIS account.
3. Select the domain that generates error 1002.
4. Select the DNS app.
5. Click Value for the A record to update.
6. Update the A record.

To ensure your origin web server doesn't proxy its own requests through CIS, configure your origin webserver to resolve your CIS domain to:

- The internal NAT-ed IP address, or
- The public IP address of the origin web server.

Error 1002: Restricted

The most common cause of 1002: Restricted errors is when the CIS domain resolves to a local or disallowed IP address or an IP address not associated with the domain.

Resolution

If you own the website:

1. Confirm your origin web server IP addresses with your hosting provider
2. Log in to your CIS account
3. Update the A records in the CIS DNS to the IP address confirmed by your hosting provider

Error 1003 Access denied: Direct IP access not allowed

The most common cause of 1003 errors is when a client or browser directly accesses a CIS IP address.

Resolution

Browse to the website domain name in your URL instead of the CIS IP address.

Error 1004: Host not configured to serve web traffic

Common causes of 1004 errors are:

- CIS staff disabled proxying for the domain due to abuse or terms of service violations.
- DNS changes have not yet propagated or the site owner's DNS A records point to CIS IP addresses.

Resolution

If the issue persists beyond 5 minutes, contact CIS support.

Errors 1006, 1007, 1008 or 1106 Access denied: Your IP address has been banned

Common causes for errors 1006, 1007, and 1008 errors are:

- A CIS customer blocked traffic from your client or browser.
- Error 1006 also occurs in the CIS Edge Functions under the Preview tab when a customer uses Zone Lockdown or any other CIS security feature to block the Google Cloud Platform IPs that the Preview tab relies upon.

Resolution

Request the website owner to investigate their CIS security settings or allow your client IP address. Because the website owner blocked your request, CIS support can't override a customer's security settings.

Errors 1009 Access denied: Country or region banned

A common cause of 1009 errors is when the owner of the website (for example, `example.com`) has banned the country or region your IP address in from accessing the website.

Resolution

Ensure your IP address is allowed under the IP Access Rules security feature.

Error 1010: The owner of this website has banned your access based on your browser's signature

A common cause of 1010 errors is when a website owner blocks your request based on your client's web browser.

Resolution

Notify the website owner of the blocking. If you can't determine how to contact the website owner, lookup contact information for the domain via the Whois database. Site owners disable Browser Integrity Check via the **Security Advanced** tab.

Because the website owner performed the blocking, CIS support can't override a customer's security settings.

Error 1011: Access denied (Hotlinking denied)

A common cause of 1011 errors is when a request is made for a resource that uses CIS hotlink protection.

Resolution

Notify the website owner of the blocking. If you can't determine how to contact the website owner, lookup contact information for the domain via the Whois database.

Because the website owner performed the blocking, CIS support can't override a customer's security settings.

Error 1012: Access denied

A common cause of 1012 errors is when a website owner forbids access based on malicious activity detected from the visitor's computer or network (`ip_address`). The most likely cause is a virus or malware infection on the visitor's computer.

Resolution

Update your antivirus software and run a full system scan. CIS can't override the security settings the site owner has set for the domain. To request website access, contact the site owner to allow your IP address. If you can't determine how to contact the website owner, look up contact information for the domain by using the Whois database.

Because the website owner performed the blocking, CIS support can't override a customer's security settings.

Error 1013: HTTP hostname and TLS SNI hostname mismatch

A common cause of 1013 errors is when the hostname sent by the client or browser via Server Name Indication (SNI) does not match the request host header.

Error 1013 is commonly caused by the following:

- your local browser setting the incorrect SNI host header
- a network proxying SSL traffic caused a mismatch between SNI and the Host header of the request.

Resolution

Test for an SNI mismatch via an online tool such as SSL Shopper.

Provide CIS Support a HAR file captured while duplicating the error.

Error 1014: CNAME cross-user banned

By default, CIS prohibits a DNS CNAME record between domains in different CIS accounts. CNAME records are permitted within a domain (`www.example.com` CNAME to `api.example.com`) and across zones within the same user account (`www.example.com` CNAME to `www.example.net`).

Resolution

To allow CNAME record resolution to a domain in a different CIS account, the domain owner of the CNAME target must contact CIS Support and specify the domains allowed to CNAME to their target domain.

Error 1015: You are being rate limited

A common cause of 1015 errors is when the site owner implemented Rate Limiting that affects your visitor traffic.

Unable to purge is another 1015 error code relating to CIS cache purge. Retry the cache purge and contact CIS support if errors persist.

Resolution

- If you are a site visitor, contact the site owner to request exclusion of your IP from rate limiting.
- If you are the site owner, review CIS Rate Limiting thresholds and adjust your Rate Limiting configuration.
- If your Rate Limiting blocks requests in a short time period (for instance, 1 second) try increasing the time period to 10 seconds.

For guidance on when you expect a new Edge function to exceed rate limits, see [Working with Edge functions](#).

Error 1016: Origin DNS error

Error 1016 occurs when CIS can't resolve the origin web server's IP address.

Common causes for Error 1016 are:

- A missing DNS A record that mentions origin IP address.
- A CNAME record in the CIS DNS points to an unresolvable external domain.
- The origin host names (CNAMEs) in your CIS Load Balancer default, region, and fallback pools are unresolvable. Use a fallback pool configured with an origin IP as a backup in case all other pools are unavailable.
- When creating a Range app with a CNAME origin, you need first to create a CNAME on the CIS DNS side that points to the origin.

Resolution

To resolve an error 1016:

- Verify your CIS DNS settings include an A record that points to a valid IP address that resolves via a DNS lookup tool.
- For a CNAME record pointing to a different domain, ensure that the target domain resolves via a DNS lookup tool.

Error 1018: Could not find host

Common causes of an 1018 error are:

- The CIS domain was recently activated and there is a delay propagating the domain's settings to the CIS edge network.
- The CIS domain was created via a CIS partner (for example, a hosting provider) and the provider's DNS failed.

Error 1018 is returned via an HTTP 409 response code.

Resolution

Contact Support with the following details:

- Your domain name
- A screenshot of the 1018 error including the RayID mentioned in the error message
- The time and timezone the 1018 error occurred

Error 1019: Compute server error

A common cause of error 1019 is when a CIS Edge function script recursively references itself.

Resolution

Ensure your CIS Edge function does not access a URL that calls the same Edge function script.

Error 1020: Access denied

A common cause of an error 1020 is when a client or browser is blocked by a CIS customer's firewall rules.

Resolution

If you are not the website owner, provide the website owner with a screenshot of the 1020 error message you received.

If you are the website owner:

1. Retrieve a screenshot of the 1020 error from your customer
2. Search the Firewall Events Log within the Overview tab of your Firewall app for the RayID or client IP Address from the visitor's 1020 error message.
3. Convert the UTC timestamp of the 1020 error to your local timezone when searching in the Firewall Events Log.
4. Assess the cause of the block and either update the Firewall Rule or allow the visitor's IP address in IP Access Rules.

Error 1023: Could not find host

Common causes of an error 1023 are:

- If the owner just signed up for CIS it can take a few minutes for the website's information to be distributed to our global network.
- Something is wrong with the site's configuration. Usually, this happens when accounts have been signed up with a partner organization (for example, a hosting provider) and the provider's DNS fails.

Error 1023 is returned via a HTTP 409 response code.

Resolution

Contact Support with the following details:

- Your domain name
- A screenshot of the 1023 error including the RayID mentioned in the error message
- The time and timezone the 1023 error occurred

Error 1025: Please check back later

A common cause of error 1025 is when a request is not serviced because the domain has reached plan limits for Edge functions.

Resolution

Contact Support to adjust the limits for Edge functions.

Error 1035: Invalid request rewrite (invalid URI path)

Common causes of error 1035 are when the value or expression of your rewritten URI path is not valid, and when the destination of the URL rewrite is a path under `/cdn-cgi/`.

Resolution

Make sure that the rewritten URI path is not empty and that it starts with a `/` (slash) character.

For example, the following URI path rewrite expression is not valid:

```
concat(lower(ip.src.country), http.request.uri.path)
```

To fix the expression, add a `/` prefix:

```
concat("/", lower(ip.src.country), http.request.uri.path)
```

Error 1036: Invalid request rewrite (maximum length exceeded)

A common cause of error 1036 is when the value or expression of your rewritten URI path or query string is too long.

Resolution

Use a shorter value or expression for the new URI path/query string value.

Error 1037: Invalid rewrite rule (failed to evaluate expression)

A common cause of error 1037 is when the expression of the rewrite rule could not be evaluated. There are several causes for this error, but it can mean that one expression element contained an undefined value when it was evaluated.

For example, you get a 1037 error when using the following URL rewrite dynamic expression and the X-Source header is not included in the request:

```
http.request.headers["x-source"][0]
```

Resolution

Make sure that all the elements of your rewrite expression are defined. For example, if you are referring to a header value, ensure the header is set.

Error 1040: Invalid request rewrite (header modification not allowed)

A common cause of error 1040 is when you are trying to modify an HTTP header that HTTP Request Header Modification Rules can't change.

Resolution

Make sure you are not trying to modify one of the reserved HTTP request headers.

Error 1041: Invalid request rewrite (invalid header value)

A common cause of error 1041 is when the added/modified header value is too long or it contains characters that are not allowed.

Resolution

- Use a shorter value or expression to define the header value.
- Remove the characters that are not allowed. See Format of HTTP request header names and values in Developer Docs for more information on the allowed characters.

Error 1101: Rendering error

A common cause of error 1101 is when an CIS Edge function throws a runtime JavaScript exception.

Resolution

Provide appropriate issues details to Support.

Error 1102: Rendering error

A common cause of error 1102 is when an Edge function exceeds a CPU time limit. CPU time is the time spent executing code (for example, loops, parsing JSON, etc). Time spent on network requests (fetching, responding) does not count towards CPU time.

Resolution

Contact the developer of your Edge function code to optimize code for a reduction in CPU usage in the active Edge function scripts.

Error 1200: Cache connection limit

Error 1200 occurs when there are too many requests queued on the edge that are awaiting process by your origin web server. This limit protects CIS's systems.

Resolution

Tune your origin webserver to accept incoming connections faster. Adjust your caching settings to improve cache-hit rates so that fewer requests reach your origin web server. Reach out to your hosting provider or web administrator for assistance.

How do I troubleshoot false positives and false negatives in WAF?

By default, the Web Application Firewall (WAF) is fully managed through the dashboard and is compatible with most websites and web applications.

However, you might encounter false positives and false negatives.

Why it’s happening

False positives happen when legitimate requests are detected and filtered as malicious. False negatives occur when malicious requests are not filtered.

Troubleshooting WAF false positives

What’s happening

The definition of suspicious content is subjective for each website. For example, PHP code posted to your website is suspicious unless your website teaches how to code and requires PHP code submissions from visitors. Therefore, such a website must disable related WAF rules that interfere with normal operation.

To test for false positives, set the WAF to Simulate mode, to record the response to possible attacks without challenging or blocking. Also, use the Firewall Analytics Activity log to determine which WAF rules caused false positives.

How to fix it

If you encounter a false positive, there are several potential resolutions:

- Add the client’s IP addresses to the IP Access Rules allowlist: If the browser or client visits from the same IP addresses, allowing is recommended.
- Disable the corresponding WAF rules. Doing so stops blocking or challenging false positives, but reduces overall site security. A request blocked by WAF Rule ID 981176 refers to OWASP rules. Decrease OWASP sensitivity to resolve the issue.
- Bypass the WAF with a Firewall Rule: Create a Firewall Rule with the bypass action to deactivate the WAF for a specific combination of parameters. For example, bypass the WAF for a specific URL and a specific IP address or user agent.
- Disable WAF for traffic to a URL (not recommended). Disabling WAF using page rules lowers security on the particular URL endpoint.



Note: If you contact IBM Support to verify whether a WAF rule triggers as expected, provide a HAR file captured while sending the specific request of concern.

If one specific rule causes false positives, set rule’s Mode to Disable rather than turning Off the entire rule group. For false positives with the administrator content on your website, create a page rule to disable security for the admin section of your site resources, for example, `yoursite.com/admin`.

Troubleshooting WAF false negatives

How to fix it

To identify false negatives, review the HTTP logs on your origin web server.

To reduce false negatives, use the following checklist:

- Is Web Application Firewall **On** in the Firewall app under Managed Rules?
- Is Web Application Firewall **Off** using page rules?
- Not all WAF rules are enabled by default, so review individual WAF rule default actions. For example, CIS allows requests with empty user agents by default.

To block requests with an empty user agent, change the WAF rule Mode to **Block**.

- Are DNS records that serve HTTP traffic proxied through CIS?
- Does a firewall rule bypass CIS managed rules?
- Does an allowed country, ASN, IP range, or IP in IP access rules or firewall rules match the attack traffic?
- Is the malicious traffic directed at your origin IP addresses to bypass CIS protection? Block all traffic except from CIS's IP addresses at your origin web server.

What do I do if I’m under a DDoS attack?

Why it’s happening

Your site is under DDoS attack.

How to fix it

To help resolve a DDoS attack, take the following steps:

1. Turn on "Defense Mode" from your dashboard.
2. Set your DNS records for maximum security.
3. Do not rate-limit or throttle requests from CIS.

During "Defense Mode", each new visitor is met with a "Captcha" security challenge, which they must pass before being given a cookie for unchallenged access. That way, botnet traffic is blocked until the "Defense Mode" is turned off. Visitors that do not meet the security challenge are added to the (bad) IP Reputation database.

What do I do if my universal certificate won't progress from pending state?

You might encounter issues where a Universal Certificate remains in a Pending Validation state, leading to SSL errors.

What's happening

You encounter SSL errors, such as:

- Chrome: `ERR_SSL_VERSION_OR_CIPHER_MISMATCH`
- Firefox: `SSL_ERROR_NO_CYPHER_OVERLAP`

How to fix it

To resolve this issue, follow these steps:

1. Check the Universal Certificate status:
 - a. Log in to IBM Cloud and view your CIS instance.
 - b. Navigate to **Security > TLS > Edge certificates**.
 - c. Locate the certificate with Type **Universal** and check its status.
 - If the status is **Active**, the certificate is working correctly.
 - If the status is **Pending Validation**, wait for the certificate to be issued.



Note: Activation can take some time.

2. Validate Certificate Authority Authorization (CAA) records.



Note: If the certificate remains in **Pending Validation** status, ensure that the CAA records allow CIS to issue the certificate.

If the issue persists after CAA validation, [contact IBM Support](#) for further troubleshooting.

What do I do if I use the Cloudflare Origin root certificate, and it's expiring?

What's happening

Your origin root certificate has expired, or will soon expire.

Why it's happening

Some origin web servers (such as IIS and cPanel) validate origin root CA certificates and require you to upload one. If you are using an expired version of the Cloudflare origin root CA, you must take the following actions to avoid site disruptions.

How to fix it

Select one of the following root certificates to download and install.

- [RSA version certificate](#)
- [ECC version certificate](#)

Why is my website offline?

Here is what you might see: `IBM CIS can't connect to the origin server (error 521, 522, 523)` .

What's happening

Website offline - no cached version

1. The server is online, but it is blocking the CIS request.
2. The origin server is offline and CISS does not have a backup website image
 - Verify that the CIS IP addresses are allowlisted.
 - Make sure that CIS IPs are not being rate-limited.
 - Review the list of [IPs to allowlist](#).

Why do I see a privacy warning?

The certificates issued by CIS cover the root domain (`example.com`) and one level of subdomain (`*.example.com`).

What's happening

A privacy warning appears in your browser if your new certificate has not yet been issued.

Why it's happening

If you're trying to reach a second-level subdomain (`**.example.com`) a privacy warning appears in your browser, because these host names are not added to the SAN.

How to fix it

Allow up to 15 minutes for one of our partner Certificates Authorities (CAs) to issue a new certificate.

Why am I not seeing any network traffic?

What's happening

You don't see any network traffic.

Why it's happening

There might be a redirect that is routing the traffic to the root domain.

How to fix it

If you're not seeing traffic, and you're using a CNAME, make sure that there are no redirects in place that are routing the traffic to the root domain.

Remember that some DNS propagations can take up to 48 hours to complete.

Why is my rule not working?

What's happening

You have a rule that should execute, but it isn't working.

Why it's happening

The order of execution can sometimes disrupt the rules you have in place.

How to fix it

Check to confirm that the rule you are expecting to execute is not getting dropped because another rule is executing before it.

Refer to [Traffic sequencing](#) to see the execution order. This list helps you identify where your rule lands within the execution sequence, so you can make any necessary adjustments.

How do I generate a HAR file?

When diagnosing issues in IBM Cloud Internet Services, it's often necessary to capture a detailed record of your browser's interaction with a web page. This record, called an HTTP Archive (HAR) file, logs requests and responses between your browser and the server. HAR files help IBM support engineers pinpoint performance issues, failed requests, and configuration problems.



Important: HAR files can contain sensitive data such as cookies, authentication tokens, passwords, or payment details. Before sharing the file with IBM support, carefully review its contents in a text editor and remove any confidential information.

Why it's happening

You're experiencing unexpected behavior, errors, or latency when using CIS, and IBM support has requested a HAR file to investigate the issue further.

How to fix it

Use the following sections to generate HAR files in Firefox, Chrome, Safari, or Edge browsers.

Generating a HAR file in Firefox

To generate a HAR file:

1. In Firefox, go to the page within CIS where you are experiencing trouble.
2. Select the Firefox menu (three horizontal parallel lines) at the upper-right of your browser window, then select **Developer > Network**.
3. The Developer Network Tools open as a docked panel at the side or bottom of the browser window. Click the **Network** tab.
4. The recording starts automatically after you start performing actions in the browser.
5. Refresh the CIS page that you are on, or go through the steps to reproduce the problem you've been experiencing while Firefox is recording activity.
6. After you have reproduced the issue, right-click anywhere under the **File** column in the docked panel and click **Save All As HAR**.
7. Save the HAR file in a location you choose, and send the file to [IBM support](#).

Generating a HAR file in Chrome

To generate a HAR file:

1. In Chrome, go to the page within CIS where you are experiencing trouble.
2. Select the Chrome menu (three vertical dots) at the top-right of your browser window, then select **Tools > Developer Tools**.
3. The Developer Tools open as a docked panel at the side or bottom of the browser window. Click the **Network** tab.
4. Select the option **Preserve log**.
5. The recording starts automatically, and a red **Record network log** icon appears at the top left of the Network tab. If not, click the black **Stop recording network log** icon to start recording activity in your browser.
6. Refresh the CIS page you are on or go through the steps to reproduce the problem you've been experiencing while Chrome is recording activity.
7. After you have successfully reproduced the issue while recording, right-click any of the items within the Network tab and select **Save all as HAR with Content** to save a copy of the activity that you recorded.
8. Save the HAR file in a location you choose, and send the file to [IBM support](#).

Generating a HAR file in Safari

To generate a HAR file:

1. In Safari, first check to see if you have a **Develop** menu at the top of the page. If not, go to **Safari > Preferences > Advanced** and select **Show Develop Menu in menu bar**.
2. In Safari, go to the page within CIS where you are experiencing trouble.
3. Navigate to **Develop > Show Web Inspector**.



Note: This menu option is not available until you browse somewhere beyond Safari's starting page. The Web Inspector opens as a docked panel at the bottom of Safari.

4. Click the **Network** tab.

5. Refresh the CIS page that you are on or go through the steps to reproduce the problem you've been experiencing while Safari is recording activity.
6. After you have reproduced the issue, Control + click the resource that you want to capture the HAR file for, and click **"Export HAR"**.
7. Save the HAR file in a location you choose, and send the file to [IBM support](#).

Generating a HAR file in Microsoft Edge

To generate a HAR file:

1. In Microsoft Edge, go to the page within CIS where you are experiencing trouble.
2. Click the Edge menu (three vertical dots) at the top-right of your browser window, then select **More Tools > Developer Tools**.
3. Select the Network tab, then click **Record** in the upper left corner of the tab and verify that it is recording. If it is not yet recording, click **Record** to start recording.
4. Check the box next to **Preserve Log**.
5. Select **Clear** to remove any existing logs from the **Network** tab.



Note: Do not close the Developer Tools panel. This ends the recording and the HAR file will be lost.

6. Refresh the CIS page you are on or go through the steps to reproduce the problem you've been experiencing while Edge is recording activity.
7. After you have reproduced the issue, right-click anywhere on the grid of network requests and choose **Save as HAR with Content**.
8. Save the HAR file in a location you choose, and send the file to [IBM support](#).

Why is my domain still in Moved status?

What's happening

You have a domain that should be **Active**, but is still in the **Moved** status.

Why it's happening

CIS becomes the authoritative DNS provider for your domain when your domain points to CIS name servers. During the setup process CIS detects the name servers and activates the domain.

- If the name servers no longer point to CIS, the system automatically mark your domain as **Moved** after 7 days. An email to inform you of this status change is sent to the email address on file. A domain in **Moved** status remains in the system for an additional 7 days before it is permanently deleted.
- Conflicting NS records can cause a domain to change to **Moved** status.

How to fix it

To resolve the condition where your domain remains in **Moved** status longer than expected, take the following troubleshooting steps.

- Check your DNS registrar and ensure that the NS records match with the NS records given by CIS.
 - Find the NS records in the **CIS name servers** section of the **DNS** tab in the **Reliability** page. For more information, see [Step 6. Configure your name servers with the registrar or existing DNS provider](#).
- Run the activation check API again. You can use the [open a Support](#) case with details about which steps you took.

Why can't I delete my CIS instance?

When you want to delete an instance of CIS, sometimes the delete operation doesn't work.

What's happening

You attempted to delete your CIS instance, but it failed.

Why it's happening

Instances that have domains attached can't be deleted.

If any global load balancers are attached, they can also cause the delete operation to fail.

Before deleting an instance, all of the domains in the instance must be removed.

Additionally, make sure that all global load balancers have been removed.

How can I help IBM Support trace my CLI API request?

When investigating CLI-related issues, effective traceability depends on receiving complete and properly formatted request data. The new `X-Request-ID` header in the CIS CLI improves transparency and traceability across systems, making it easier for IBM Support to pinpoint and resolve issues efficiently.

What’s happening

IBM Support might encounter difficulty tracing specific CLI-initiated API calls if the necessary request identifiers are missing from the logs you share.

Why it’s happening

Although the CLI automatically includes the `X-Request-ID` header in every API request, this information might be inadvertently excluded when you collect and share trace logs. Without it, support teams may be unable to track the request across back-end systems.

How to fix it

To ensure that IBM Support can trace and analyze the request properly, follow these steps when capturing and sharing CLI logs:

1. Enable trace logging from the CLI before reproducing the issue.
2. Capture and share the full API request and response logs, including the `X-Request-ID` and, if available, the `X-Correlation-Id` from the response.
3. Look for details similar to the following:

```
REQUEST: [2025-07-17T19:47:24+05:30]
POST /v1/crn:v1:staging:public:internet-svcs-ci:global:a/01652b251c3ae2787110a995d8db0135:1a9174b6-0106-417a-844b-c8eb43a72f63::/rules/lists
HTTP/1.1
Host: api-int.cis.dev.cloud.ibm.com
Accept: application/json
Authorization: [PRIVATE DATA HIDDEN]
Content-Type: application/json
X-Auth-User-Token: [PRIVATE DATA HIDDEN]

{"name": "ac", "kind": "asn"}

RESPONSE: [2025-07-17T19:47:25+05:30] Elapsed: 1189ms
HTTP/2.0 200 OK
Cache-Control: no-cache, no-store, must-revalidate
Cf-Cache-Status: DYNAMIC
Cf-Ray: 960a5cd87e565520-DEL
Content-Security-Policy: frame-ancestors 'none'; default-src 'self'; form-action 'self'
Content-Type: application/json
Date: Thu, 17 Jul 2025 14:17:25 GMT
Server: cloudflare
Strict-Transport-Security: max-age=31536000; includeSubdomains
X-Content-Type-Options: nosniff
X-Correlation-Id: f9e06c3c-8276-4436-9ad3-0bf63c27291c
X-Envoy-Upstream-Service-Time: 842
```

4. Attach this information to your support case to assist IBM Support in isolating and debugging the issue quickly.



Note: Make sure to redact sensitive data, such as Authorization tokens, before sharing logs externally.

Why am I seeing ERR_TOO_MANY_REDIRECTS in CIS?

The `ERR_TOO_MANY_REDIRECTS` error occurs when a browser is unable to load a website because it becomes stuck in a continuous redirect cycle. Instead of reaching the intended page, the request is repeatedly sent between URLs. In CIS, this issue most commonly appears after

adding a [new domain](#) or modifying security, SSL/TLS, or redirect configurations.

When you try to access a website:

- The browser sends a request to a URL (for example, `http://example.com`).
- The server or CIS responds with a redirect to another URL.
- That redirected URL triggers another redirect.
- The cycle continues indefinitely.

Because the browser never reaches a final destination page, it eventually stops and displays:

- `ERR_TOO_MANY_REDIRECTS`
- The page isn't redirecting properly

`ERR_TOO_MANY_REDIRECTS` does not indicate a connectivity issue. It indicates a configuration conflict where redirect logic never resolves to a final destination.

What's happening

A redirect loop happens when multiple components attempt to enforce conflicting rules.

In CIS, this typically occurs because of one of the following configuration conflicts:

- SSL/TLS encryption mode misconfigurations
- Edge certificate settings conflict with origin behavior
- Conflicting redirect rules

Why it's happening

To determine which scenario applies, consider what changed before the issue started:

- Did the issue begin after modifying the SSL/TLS encryption mode?
- Did the issue begin after enabling Always Use HTTPS or HSTS?
- Did the issue begin after creating or editing redirect rules?

How to fix it

The resolution depends on which configuration conflict is causing the redirect loop.

Review the following scenarios and apply the solution that matches your configuration.

Scenario 1: SSL/TLS encryption mode misconfigurations

Your domain's SSL/TLS encryption mode determines how CIS connects to your origin server and validates how SSL certificates presented by your origin. This configuration causes redirect loops when the value you set in CIS conflicts with the settings at your origin web server.

Flexible mode + origin forces HTTP → HTTPS

If the domain encryption mode is set to End-to-End flexible, CIS sends unencrypted requests to the origin server over HTTP.

1. CIS forwards the request to the origin using HTTP.
2. The origin server redirects the HTTP request to HTTPS.
3. CIS again connects to the origin over HTTP (because Flexible mode enforces HTTP to origin).
4. The origin again redirects to HTTPS.

This process repeats, creating a redirect loop.

CA Signed mode + origin forces HTTPS → HTTP

When the domain encryption mode is set to End-to-end CA signed, CIS sends encrypted requests to your origin server over HTTPS.

1. CIS forwards the request to the origin using HTTPS.
2. The origin server redirects HTTPS requests to HTTP.
3. CIS continues sending HTTPS requests to the origin.

4. The origin continues redirecting to HTTP.

This creates an HTTPS-to-HTTP redirect loop.

What's happening

Redirect loops occur when the SSL/TLS encryption mode configured in CIS conflicts with redirect behavior on the origin server:

- In Flexible mode, CIS communicates with the origin over HTTP while the origin enforces HTTP-to-HTTPS redirection.
- In CA Signed mode, CIS communicates with the origin over HTTPS while the origin redirects HTTPS traffic back to HTTP.

Why it's happening

Because both layers attempt to control protocol behavior differently, the request never reaches a final destination.

Choose one of the following options:

- Remove the HTTP-to-HTTPS or HTTPS-to-HTTP redirect from the origin server.
- Change the SSL/TLS encryption mode to Full (or higher) if appropriate. This option requires a valid SSL certificate installed on the origin server.

Ensure that the encryption mode and the origin server configuration are compatible to prevent redirect loops.

How to fix it

If you are unsure whether your origin server is issuing redirects, contact your hosting provider or site administrator for assistance.

Scenario 2: Edge certificate settings conflict with origin behavior

Edge certificate settings in CIS determine how HTTPS is enforced between end users and CIS. When these settings do not align with your origin server configuration or domain encryption mode, they can unintentionally create redirect loops.

Redirect loops occur when multiple layers attempt to enforce different protocol behaviors (HTTP versus HTTPS), preventing the request from reaching a final destination.

The most common edge settings involved in this issue are **Always Use HTTPS** and **HTTP Strict Transport Security (HSTS)**.

Always Use HTTPS conflicts with origin redirects

If you have **Always Use HTTPS** enabled for your domain, CIS redirects all `http` requests to `https` for all subdomains and hosts in your application.

What's happening

When **Always Use HTTPS** is enabled:

1. A user sends a request using `HTTP`.
2. CIS immediately redirects the request to `HTTPS`.
3. CIS forwards the `HTTPS` request to the origin server.
4. If the origin redirects `HTTPS` traffic back to `HTTP`, the request cycles again.
5. The browser eventually stops the repeated redirects and displays a redirect error.

A redirect loop occurs when:

- CIS upgrades HTTP traffic to HTTPS.
- The origin server downgrades HTTPS traffic back to HTTP.

Why it's happening

Because both layers enforce opposite protocol behavior, the request continuously alternates between HTTP and HTTPS without reaching a final page.

How to fix it

Use one of the following options:

- Remove the **HTTPS-to-HTTP** redirect from the origin server.
- Disable **Always Use HTTPS**.

HSTS forces HTTPS while origin downgrades to HTTP

HTTP Strict Transport Security (HSTS) is a security policy that instructs supported browsers to always access your domain using **HTTPS**. When enabled, CIS sends an **HSTS** response header that tells browsers to automatically convert future **HTTP** requests to **HTTPS**.

What's happening

When HSTS is enabled:

1. The browser receives the HSTS header from CIS.
2. The browser automatically converts future HTTP requests to HTTPS.
3. If the origin server redirects HTTPS traffic to HTTP, the browser upgrades it back to HTTPS.
4. This repeated upgrade/downgrade behavior creates a redirect loop.
5. The browser eventually stops loading the page and displays an error.

Why it's happening

This typically occurs when:

- The origin server redirects HTTPS requests to HTTP, or
- The domain encryption mode is set to **Off**.

HSTS enforces HTTPS at the client side, while the origin attempts to downgrade the connection. This protocol mismatch causes continuous redirection.

How to fix it

To resolve this issue:

- Remove any **HTTPS-to-HTTP** redirects from the origin server and ensure the domain encryption mode is set to Flexible or higher.
- Alternatively, disable HSTS.

Scenario 3: Conflicting redirect rules

Redirect loops can also occur when multiple redirect configurations conflict with each other. This may involve redirect rules, page rules, load balancer rules, or application-level redirects that unintentionally overlap.

When more than one rule attempts to control the same request path, traffic can be redirected in a circular pattern.

What's happening

A redirect loop occurs when:

- One rule redirects a request from URL A to URL B.
- Another rule redirects URL B back to URL A (or to another URL that eventually points back to A).
- The browser repeatedly follows these redirects.
- No final destination page is reached. As a result, the browser stops the process and displays a redirect error.

Why it's happening

This typically happens because:

- Multiple redirect rules target overlapping URL patterns.
- Redirect rules and Page Rules enforce different behaviors.
- Both CIS and the origin server define redirects for the same traffic.
- Wildcard or broad match conditions unintentionally apply to unintended URLs.

Without careful rule order and validation, these configurations can create circular redirect paths.

How to fix it

Review your various redirect rules and Page Rules to make sure no rules are in conflict with each other. Ensure that no rules redirect traffic in a circular pattern.

Why am I seeing Too Many Requests in Terraform plan?

When you use Terraform to manage CIS, you might encounter the error **Too Many Requests (429)** during `terraform plan` or `terraform apply`. These errors occur when API request rates exceed the supported limits, which can happen in environments with multiple resources, repeated operations, or when requests are issued in quick succession. Because Terraform can perform concurrent operations and retry failed requests, the total number of API calls can increase rapidly, leading to rate limiting and potentially causing deployments to fail or become unreliable.

What's happening

During Terraform operations, CIS API requests can fail with this error: **Too Many Requests (429)**

This error can occur repeatedly, typically when you manage CIS resources, such as:

- `ibm_cis_firewall_rules`
- `ibm_cis_mtls_app`

The issue can persist even when Terraform parallelism is reduced and might block deployments.

Why it's happening

This issue is caused by exceeding CIS API rate limits.

- CIS enforces a default API rate limit (for example, approximately 1,200 requests per 5 minutes per user).
- Terraform runs operations in parallel by default, which can generate a high number of API requests in a short time.
- Even with reduced parallelism, the following factors can still trigger rate limiting:
 - A large number of CIS resources in a single deployment
 - Repeated retries of failed API calls by Terraform
 - Use of a single technical user for all API operations

Because CIS does not currently show configurable throttling or backoff controls for Terraform, request bursts can exceed the allowed limits.

How to fix it

Use the following resolutions to reduce the number of API requests and minimize the likelihood of encountering rate limiting during Terraform operations with CIS.

1. Reduce Terraform parallelism.

Terraform runs up to 10 operations concurrently by default. Lowering this value reduces the number of simultaneous API calls. Run Terraform with reduced parallelism: `terraform apply -parallelism=2` or `terraform apply -parallelism=1`

Lowering parallelism reduces the number of simultaneous API requests and can help prevent rate limiting.

2. Check CIS API rate limits. When you use Terraform with CIS APIs, API requests are subject to rate limits (for example, approximately 1,200 requests per 5 minutes per user). If Terraform attempts to create or modify many resources in a short period, these limits can be exceeded.

Review your deployment size and frequency to ensure that the total number of API requests stays within supported limits.

3. Check for repeated API retries. Terraform might retry failed API requests, which can increase the total number of requests and contribute to rate limiting.

You can enable debug logging to investigate request behavior: `TF_LOG=DEBUG terraform apply`

Review the logs for repeated API calls, such as:

- `GET`
- `POST`
- `PATCH`
- `PUT`

Frequent or repeated requests can indicate retry activity that is contributing to the issue.

4. Contact support if the issue persists. If the issue continues after applying these steps, contact IBM Support.

Provide the following information to assist with troubleshooting:

- Approximate number of resources being created or updated
- Sample Terraform configuration (if available)
- Estimated number of API requests being sent
- Relevant Terraform debug logs

Why am I seeing Origin address is not globally routable in CIS

The `Origin address is not globally routable` error occurs when the configured origin is not reachable from the CIS global network.

What's happening

CIS validates origin addresses when you add them to a load balancer pool. CIS routes traffic and runs health checks from its public global network. Because of this, the origin must use a publicly routable IP address or hostname.

If the origin uses a private IP address, internal hostname, or private-only endpoint, CIS cannot reach the origin and the validation fails.

Why it's happening

This issue can occur in the following situations:

- The origin uses a private IP address.
- The origin is behind a private VPC load balancer.
- The application is accessible only within a Kubernetes or Red Hat OpenShift cluster.
- The origin does not have a public IP address or public DNS record.
- Firewall or network rules block traffic from the CIS network.

CIS health checks originate from the public CIS network. Private or nonroutable addresses cannot be reached from this network.

How to fix it

To resolve the issue, complete the following actions:

1. Configure the origin with a publicly routable IP address or hostname.
2. Ensure that the origin is reachable from the internet.
3. Allow required traffic from CIS IP ranges.
4. Verify that firewalls, security groups, and network ACLs allow CIS health checks.
5. For VPC or Kubernetes workloads, access the application through a public ingress endpoint or public load balancer.

Getting help and support for CIS

If you experience an issue or have questions when using CIS, you can use the following resources before you open a support case.

- Ask a question in the [AI assistant](#) from the console or the IBM Cloud CLI.
- Review [FAQs](#) in the product documentation.
- Review [Troubleshooting](#) to diagnose and resolve common issues.
- Check the status of the IBM Cloud platform and resources by going to the [Status page](#).

If you still can't resolve the problem, you can open a support case. For more information, see [Creating support cases](#). And, if you're looking to provide feedback, see [Submitting feedback](#).

Providing support case details for CIS

To ensure that the support team has all of the details for investigating your issue to provide a timely resolution, you must provide information about your issue. Review the following tips about the type of information to include in your support case for issues with CIS.

Provide the following details:

1. Provide your CRN:
 - In the console, navigate to your CIS instance and go to the **Overview** page.
 - In the CLI, run `ibmcloud resource service-instances --long`
2. Provide your Domain name or ID.
3. Depending on the issue the following information might also be helpful:
 - Account ID: Log into <https://cloud.ibm.com> and go to **View Profile > Billing**
 - Instance ID: Run the CLI command `ibmcloud resource service-instances --long`
 - Geo location

© Copyright IBM Corporation 2026

IBM Corporation
New Orchard Road
Armonk, NY 10504

Produced in the United States of America
2026-10-01

IBM, the IBM logo, and ibm.com are trademarks or registered trademarks of International Business Machines Corp., registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the web at <https://www.ibm.com/legal/copytrade>.

This document is current as of the initial date of publication and may be changed by IBM at any time. Not all offerings are available in every country in which IBM operates.

THE INFORMATION IN THIS DOCUMENT IS PROVIDED "AS IS" WITHOUT ANY WARRANTY, EXPRESS OR IMPLIED, INCLUDING WITHOUT ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND ANY WARRANTY OR CONDITION OF NON-INFRINGEMENT.

IBM products are warranted according to the terms and conditions of the agreements under which they are provided.

